国 外 计 算 机 科 学 教 材 系 列

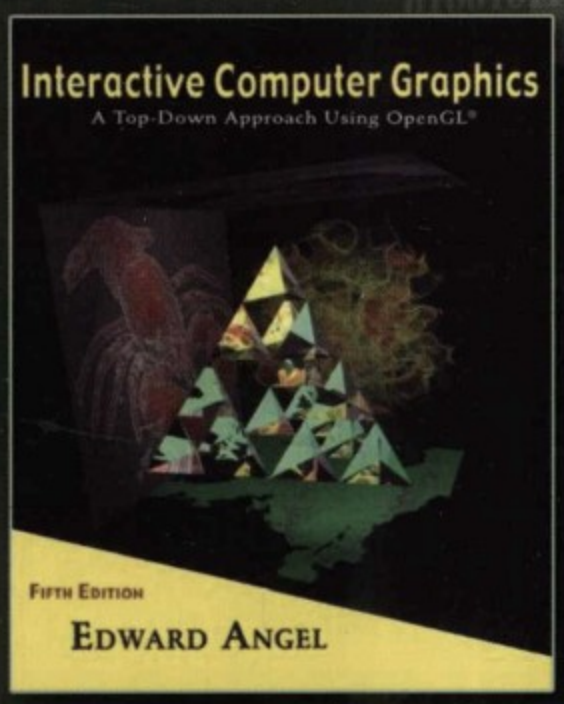
PEARSON

交互式计算机图形学

——基于OpenGL的自顶向下方法（第五版）

Interactive Computer Graphics

A Top-Down Approach Using OpenGL, Fifth Edition



[美] Edward Angel 著

张荣华 姜小磊 宋 雨 等译

電子工業出版社
PUBLISHING HOUSE OF ELECTRONICS INDUSTRY

http://www.phei.com.cn

权威经典

交互式计算机图形学
——基于OpenGL的自顶向下方法（第五版）

Interactive Computer Graphics
A Top-Down Approach Using OpenGL, Fifth Edition

在广泛结合OpenGL并注重图形应用编程的基础上，本书介绍了计算机图形学的核心概念。书中代码采用C和C++语言，并使用了自顶向下和面向编程的方法，使读者能够迅速地创建自己的三维图形。在结构安排上，本书在读者学会了编写交互式图形程序之后再介绍底层的算法，如线段的绘制以及多边形填充等算法。

本书覆盖了计算机图形学基础课程中的所有主题，包括光与材质的相互作用、明暗绘制、建模、曲线和曲面、反走样、光栅化、纹理映射和图像合成等内容。

主要特点

- 新！覆盖最新的纹理技术，如多通道纹理映射和凹凸映射
- 新！扩充了建模和过程建模方法的内容，包括过程噪声
- 新！扩充了场景图和实时绘制的内容
- 新！扩充了可编程着色器的内容
- 重新修订了演示程序

Edward Angel

美国新墨西哥大学计算机科学系、电气与计算机工程系以及媒体艺术系荣誉退休教授，他曾经是该大学首位兼任教学工作的研究生。Edward获得了美国南加州大学哲学博士学位和加州理工学院工程理学学士学位。他还是新墨西哥大学艺术、研究、技术和科学实验室（http://artslab.unm.edu）的主任。

PEARSON
www.PearsonEd.com

ISBN 978-7-121-08869-8
9 787121 088698 >

定价：69.00 元

策划编辑：许菊芳
责任编辑：周宏敏
责任美编：喻 晓

欢迎登录 免费 获取本书教学资源
华信 www.huaxin.edu.cn www.hxedu.com.cn

PEARSON Education

本书贴有激光防伪标志，凡没有防伪标志者，属盗版图书。

国外计算机科学教材系列

交互式计算机图形学

——基于OpenGL的自顶向下方法

（第五版）

Interactive Computer Graphics

A Top-Down Approach Using OpenGL

Fifth Edition

［美］ Edward Angel 著

张荣华 姜小磊 宋 雨 等译

電子工業出版社

Publishing House of Electronics Industry

北京 · BEIJING

内容简介

在广泛结合OpenGL并注重图形应用编程的基础上，本书向读者介绍了计算机图形学的核心概念。书中代码采用C和C++语言，并使用了自顶向下和面向编程的方法，使读者能够迅速地创建自己的三维图形。在结构安排上，本书在读者学会了编写交互式图形程序之后再介绍底层的算法，如线段的绘制以及多边形填充等算法。

本书覆盖了计算机图形学基础课程中的所有主题，包括光与材质的相互作用、明暗绘制、建模、曲线和曲面、反走样、光栅化、纹理映射和图像合成等内容。

本书可作为计算机及相关专业本科生和研究生的计算机图形学教材，也适合作为相关程序员、工程技术人员及科研人员的参考书。

Authorized translation from the English language edition, entitled Interactive Computer Graphics: A Top-Down Approach Using OpenGL, Fifth Edition, 978-0-321-53586-3 by Edward Angel, published by Pearson Education, Inc, publishing as Addison Wesley Higher Education, Copyright © 2009 by Pearson Education, Inc.

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording or by any information storage retrieval system, without permission from Pearson Education, Inc.

CHINESE SIMPLIFIED language edition published by PEARSON EDUCATION ASIA LTD., and PUBLISHING HOUSE OF ELECTRONICS INDUSTRY Copyright © 2009.

本书中文简体字版专有出版权由 Pearson Education（培生教育出版集团）授予电子工业出版社。未经出版者预先书面许可，不得以任何方式复制或抄袭本书的任何部分。

本书贴有 Pearson Education（培生教育出版集团）激光防伪标签，无标签者不得销售。

版权贸易合同登记号　图字：01-2008-2494

图书在版编目（CIP）数据

交互式计算机图形学：基于OpenGL的自顶向下方法：第5版/（美）安吉尔（Angel, E.）著；张荣华等译.
北京：电子工业出版社，2009.7
（国外计算机科学教材系列）
书名原文：Interactive Computer Graphics: A Top-Down Approach Using OpenGL, Fifth Edition
ISBN 978-7-121-08869-8

Ⅰ.交…　Ⅱ.①安…　②张…　Ⅲ.计算机图形学－高等学校－教材　Ⅳ.TP391.41

中国版本图书馆CIP数据核字（2009）第077027号

策划编辑：许菊芳
责任编辑：周宏敏
印　　刷：北京东光印刷厂
装　　订：三河市鑫金马印装有限公司
出版发行：电子工业出版社
　　北京市海淀区万寿路173信箱　邮编：100036
开　　本：787 × 1092　1/16　　印张：38.75　　字数：1121千字　　彩插：8
印　　次：2009年7月第1次印刷
定　　价：69.00元

凡所购买电子工业出版社的图书有缺损问题，请向购买书店调换；若书店售缺，请与本社发行部联系。联系及邮购电话：（010）88254888。

质量投诉请发邮件至zlts@phei.com.cn，盗版侵权举报请发邮件至dbqq@phei.com.cn。

服务热线：（010）88258888。

出 版 说 明

21世纪初的5至10年是我国国民经济和社会发展的重要时期，也是信息产业快速发展的关键时期。在我国加入WTO后的今天，培养一支适应国际化竞争的一流IT人才队伍是我国高等教育的重要任务之一。信息科学和技术方面人才的优劣与多寡，是我国面对国际竞争时成败的关键因素。

当前，正值我国高等教育特别是信息科学领域的教育调整、变革的重大时期，为使我国教育体制与国际化接轨，有条件的高等院校正在为某些信息学科和技术课程使用国外优秀教材和优秀原版教材，以使我国在计算机教学上尽快赶上国际先进水平。

电子工业出版社秉承多年来引进国外优秀图书的经验，翻译出版了“国外计算机科学教材系列”丛书，这套教材覆盖学科范围广、领域宽、层次多，既有本科专业课程教材，也有研究生课程教材，以适应不同院系、不同专业、不同层次的师生对教材的需求，广大师生可自由选择和自由组合使用。这些教材涉及的学科方向包括网络与通信、操作系统、计算机组织与结构、算法与数据结构、数据库与信息处理、编程语言、图形图像与多媒体、软件工程等。同时，我们也适当引进了一些优秀英文原版教材，本着翻译版本和英文原版并重的原则，对重点图书既提供英文原版又提供相应的翻译版本。

在图书选题上，我们大都选择国外著名出版公司出版的高校教材，如Pearson Education培生教育出版集团、麦格劳－希尔教育出版集团、麻省理工学院出版社、剑桥大学出版社等。撰写教材的许多作者都是蜚声世界的教授、学者，如道格拉斯·科默（Douglas E. Comer）、威廉·斯托林斯（William Stallings）、哈维·戴特尔（Harvey M. Deitel）、尤利斯·布莱克（Uyless Black）等。

为确保教材的选题质量和翻译质量，我们约请了清华大学、北京大学、北京航空航天大学、复旦大学、上海交通大学、南京大学、浙江大学、哈尔滨工业大学、华中科技大学、西安交通大学、国防科学技术大学、解放军理工大学等著名高校的教授和骨干教师参与了本系列教材的选题、翻译和审校工作。他们中既有讲授同类教材的骨干教师、博士，也有积累了几十年教学经验的老教授和博士生导师。

在该系列教材的选题、翻译和编辑加工过程中，为提高教材质量，我们做了大量细致的工作，包括对所选教材进行全面论证；选择编辑时力求达到专业对口；对排版、印制质量进行严格把关。对于英文教材中出现的错误，我们通过与作者联络和网上下载勘误表等方式，逐一进行了修订。

此外，我们还将与国外著名出版公司合作，提供一些教材的教学支持资料，希望能为授课老师提供帮助。今后，我们将继续加强与各高校教师的密切联系，为广大师生引进更多的国外优秀教材和参考书，为我国计算机科学教学体系与国际教学体系的接轨做出努力。

电子工业出版社

教材出版委员会

主　任　杨芙清　北京大学教授
中国科学院院士
北京大学信息与工程学部主任
北京大学软件工程研究所所长

委　员　王　珊　中国人民大学信息学院院长、教授

胡道元　清华大学计算机科学与技术系教授
国际信息处理联合会通信系统中国代表

钟玉琢　清华大学计算机科学与技术系教授、博士生导师
清华大学深圳研究生院信息学部主任

谢希仁　中国人民解放军理工大学教授
全军网络技术研究中心主任、博士生导师

尤晋元　上海交通大学计算机科学与工程系教授
上海分布计算技术中心主任

施伯乐　上海国际数据库研究中心主任、复旦大学教授
中国计算机学会常务理事、上海市计算机学会理事长

邹　鹏　国防科学技术大学计算机学院教授、博士生导师
教育部计算机基础课程教学指导委员会副主任委员

张昆藏　青岛大学信息工程学院教授

译 者 序

实施和推进“双语教学”是我国高等教育适应国际化的趋势，为了培养富有创新精神和国际视野的复合型高素质人才，译者所在学校的计算机系从2005年开始对《计算机图形学》课程开展了双语教学工作。译者多年从事计算机图形学的教学与研究，深知选择一本“基础性与先进性、理论性与应用性、科学性与通俗性”相结合的优秀教材的重要性。为了保证双语教学的顺利实施，经过认真筛选，我们最终选择并沿用了教育部高等教育司推荐的国外高质量的外文原版影印教材 *Interactive Computer Graphics —A Top-Down Approach with OpenGL*，本书一版再版，先后由高等教育出版社、清华大学出版社及电子工业出版社出版。

本书原版是国外诸多大学采用的一本经典计算机图形学教材。为了满足国内高校对高质量计算机图形学教材的需求，电子工业出版社希望将本书的最新版本（第五版）翻译成中文出版，译者甚感荣幸承接了本书第五版的翻译工作。本书的整个翻译过程持续了半年多，为了确保译文的质量，译者和校对人员花了大量时间对译文进行认真的校对和统稿。在翻译的过程中，对于原文可能有误的地方，译文中均以“译者注”的形式做了统一标注。

本书的主要特色如下：

- 内容丰富而系统，基础性内容和拓展性主题相结合。本书涉及了现代计算机图形学课程几乎所有的主要内容，包括基于OpenGL的交互式图形编程、基于图形绘制流水线的各种几何变换操作和基本算法、光与材质的相互作用、可编程着色器、层级建模、过程建模、曲线和曲面、反走样、纹理映射、图像合成和高级绘制等内容。
- 内容编排和组织独具特色，综合了知识性和实用性。本书采用了“自顶向下”和面向图形应用编程的方法，使学生既能基于OpenGL迅速地编写自己的三维图形程序，又能系统地掌握计算机图形学的基本原理、方法和高级技术。
- 本书作者是新墨西哥州大学计算机科学系、电气与计算机工程系，以及媒体艺术系的荣誉退休教授，作者结合自己在计算机图形学领域多年的科研和教学工作，在书中提供了大量的经典示例（包括附录A中提供的19个完整的示例程序）、插图和习题，并在每章后面提供了非常难得的建议阅读资料，以便读者能全面且深入地理解计算机图形学的基本理论和方法。
- 本书既不乏经典理论，又侧重近几年计算机图形学发展的最新成果。为了及时反映计算机图形学目前的发展现状，本书对第四版中某些章节提及的某些反映图形硬件性能的数据做了修改，并调整和增删了某些内容：
  - ◇ 把第四版的“建模”一章划分为“层级建模”和“过程建模”两个独立的章，扩充并完善了层级建模和过程建模的相关内容。例如，在层级建模中增加了开放场景图（OSG）和实时绘制（如遮挡剔除和LOD）等内容，而在过程建模中增加了一个简单而完整的粒子系统示例程序，并增加了基于语言的建模和过程噪声等内容。
  - ◇ 扩充并调整了可编程着色器的内容，从而使该章的层次更加清晰，读者也更容易理解OpenGL着色语言（GLSL）及其执行过程。扩充并调整后的内容反映了最新可编程图形硬件GPU的发展现状，例如在第9章的小结与注释中提到了最新可编程GPU内部的一个称为“几何着色器”（geometry shader）的可编程处理单元。

◇ 重新修订了书中有关的示例程序以及附录A中的完整示例程序。例如，把第四版第3章（输入与交互）中的“一个简单的画图程序”修改为“一个简单的CAD程序”；在附录A中增加了一个完整的粒子模拟程序。

翻译国外的著作是一个再创作过程，本书的翻译过程得到了很多人的帮助和参与，没有他们为此付出的辛劳，就不会在如此短的时间内顺利地完成本书的翻译，在此衷心感谢为本书翻译付出努力的每一个人！除了本人之外，姜小磊、宋雨、李继荣也参与了本书的翻译工作，另外，刘书刚、宋亚奇、李莉、张长明、崔克彬、王蓝婧、阎蕾、王艳、郭军、祁在山、刘晓峰和潘德锋等老师参与了本书的校对工作。

由于译者自身知识的局限性和精力有限，译文难免有错漏之处，谨向原书作者和读者表示歉意，并欢迎读者批评指正。

华北电力大学计算机系
张荣华
2008年12月　于保定

前　言

本书是计算机图形学入门教材，重点介绍应用程序的编写。在1997年出版本书第一版之前我曾写过一本图形学的教材，这两本书相隔了7年，在此期间，计算机图形学有了巨大的发展，发展速度之快超出大多数人的预期，也包括我自己。从那以后到现在的11年里（本书出到了第四版），我们看到计算机图形学发生了更多的变化，长篇计算机动画电影在商业和艺术两方面都获得了成功，在电影中使用计算机效果已经成为标准，实景拍摄和计算机合成效果常常很难区分。现在从硬件上已难以区分计算机和游戏电玩（game box），可编程图形处理器在商品系统中提供了相当的灵活性，而在几年前即使是最昂贵的工作站也不可能达到这种层次。

现在，不但图形系统的功能大为增强，而且从高端到低端工作站的价格也大幅下降。最近几年，图形系统每秒钟能够绘制超过1亿个具有光照效果和经过纹理映射的三维多边形，价格已经从超过10万美元下降到不足1000美元。用于个人计算机的商用图形板的出现具有特别重要的意义，这些图形板支持复杂的三维应用程序，最低价格大约为100美元。在软件方面，OpenGL仍旧是编写应用程序和为多平台应用开发高级产品的标准程序员接口，OpenGL支持从大型科学计算可视化到手机游戏的广阔应用领域。

自顶向下的方法

图形学的新进展和本书前4版的成功使我一直坚持采用一种自顶向下的、面向编程的方法来介绍计算机图形学的基本知识。虽然现在许多院校的计算机科学和工程系在图形学方面开设了多门课程，但是多数学生只选修其中的一门课程。学生们在已经学习了程序设计、数据结构、算法、软件工程和基础数学以后，给他们安排一门计算机图形学课程，将允许教师以内容充实并且有趣的方式按上述已学课程来组织课程内容，我想学生们会尽可能地编写出三维应用程序。底层的算法，例如绘制线段或者填充多边形，可以在学生们编写了图形程序之后再考虑。

计算机教育的先驱John Kemeny曾经把自顶向下的编程方法与我们熟悉的汽车来类比：你没必要知道汽车罩下面是什么，但是如果知道内部的工作原理，你就会去驾驶汽车而不是坐在汽车后座上。同样的类比适用于讲授计算机图形学的方法。第一种方法（算法的方法）是把汽车工作原理的方方面面都交代清楚：发动机、传动装置、燃烧过程等。第二种方法（概览的方法）是雇一名司机，自己坐在后面观光。第三种方法（本书采用的编程方法）是教会你如何驾驶并把车开到想要去的地方，正如一句过时的汽车租赁广告词所说的那样："让我们把你放到驾驶员的位子上去。"

使用OpenGL和C语言编程

25年前我开始讲授计算机图形学，当时以面向编程的方式讲授这门课程并编写出教材的最大障碍是缺乏一个广泛接受的图形库或者应用程序接口（API）。遇到的困难包括：高成本、可用性受限、缺乏通用性以及高复杂性。开发OpenGL解决了我们中很多人在其他API（例如GKS和PHIGS）中遇到的大多数困难以及在使用自己编写的软件作为替代时所面临的大多数困难，今天所有的平台都支持OpenGL，它已和Microsoft Windows以及Apple Macintosh操作系统集成在一起，实际上所

有的图形卡都有驱动程序。在大多数Linux分布式系统中也都包含一个称为Mesa的OpenGL API。现在也有一些存在于嵌入式系统中的OpenGL版本，这使得OpenGL也可应用于手提式设备。

一门图形学的课程不仅仅是讲授如何使用一种特定的API，而是一种好的API会使得讲授图形学中的一些关键内容变得更容易，这些内容包括三维图形学、光照和明暗处理、客户/服务器图形系统、建模以及实现算法。我认为，OpenGL广泛的功能以及良好定义的结构可以为讲授图形学理论和实践这两方面都提供更好的基础，而且OpenGL同样也可以为讲授包括纹理映射、图像组合以及可编程着色器在内的一些高级概念提供更好的基础。

大约15年以前我开始在授课时使用OpenGL，结果令我非常惊喜。在那个学期的期中之前，所有的学生都能编写出中等复杂程度的三维图形程序了，而编写这样的程序需要理解三维显示和事件驱动输入。在我以前讲授图形学的时候，从没有收到过这么好的教学效果，那次课程使我开始重新编写以前的书。

本书是关于计算机图形学的教科书，并不是OpenGL使用手册。因此，我没有把OpenGL API的方方面面都一一介绍，而是仅仅解释了对于掌握本书内容来说必要的那些部分。我是以一种允许其他API用户毫无困难地阅读本书的方式来介绍OpenGL的，对于想要了解OpenGL更多细节的学生们，我新近的书 *OpenGL: A Primer, Third Edition*（Addison-Wesley 2008出版）是一本有价值的补充读物。

这个版本使用的编程语言是C和C++，在大多数情况下使用C语言。这样做出于两个原因：第一，OpenGL不是面向对象的，所以使用C++或者Java并不能为介绍OpenGL增加益处，除非我想在OpenGL和用户之间插入一个面向对象的几何图形库。尽管面向对象有诱人的特性，但我并没有这么做，因为这样不但会偏离图形学的主题，而且会使编程能力很强但并不熟悉面向对象语言的学生觉得本书难以接受。第二，我的经验表明，面向对象的方法为用户隐藏了一些内部机制（这正是面向对象方法所要做的），但本书是计算机图形学的导论，我想让读者清楚地知道最底层所发生的事情。虽然以计算机图形学为背景来介绍面向对象编程是非常精彩的，但是在我看来，面向对象的方法并不是向计算机科学和工程专业的学生讲授图形学最有效的方法。不过，场景图是面向对象的，使用C++来介绍它是有好处的，这是例外。我的本科学生在低年级课程中学过了Java和C++，他们能够读懂本书中的C和C++代码。

适用的读者

本书适用的读者包括：计算机科学和工程专业的高年级本科生和一年级研究生，以及具有良好程序设计能力的其他专业学生。本书对于许多专业人士也是有价值的。我已经为专业人员讲授了100多次短期课程，我从这些非传统意义的学生身上获得的经验极大地影响了本书的内容。

在阅读本书之前，读者应该熟练掌握C或者C++语言、理解链表和树等基本数据结构并具有线性代数和三角学的初步知识。我发现计算机科学专业学生的数学背景相差很大，不管本科生还是研究生都是如此。因此，本书包含了很多计算机基础图形学所需的线性代数和几何学知识，附录B和附录C中对这些内容进行了总结。

本书的内容组织

本书内容组织如下：第1章概览整个领域并通过光学设备介绍图像生成，这样我们从一开始就引入了三维图形学的概念。第2章介绍OpenGL编程，虽然编写的第一个例子程序是二维的（每一章都有一个或者多个完整的编程例子），但是这个程序是嵌入在三维环境下编写的，并且可以扩展到三维。第3章讨论现代客户/服务器环境下的交互式图形学并开发事件驱动图形程序。第4章和

第5章讨论三维图形学概念，第4章讨论如何定义和处理三维对象，而第5章讨论如何观察它们。第6章介绍光线和材质之间的相互作用以及明暗处理。在讲授前6章时应该按照先后顺序进行，在一个总共15周的学期里，大约10周内就可以讲完这些内容。

后面的7章可以按照任何顺序阅读。所有这7章都或多或少是开放式的，读者可以概览，也可以深究其中的一些课题。第7章讨论光栅化，对光栅化的每个基本步骤都给出了一种或两种主要算法，这些基本步骤包括裁剪、直线段生成和多边形填充。第8章介绍图形硬件和OpenGL现在所支持的许多新功能，所有这些技术都需要与各种缓存打交道，这一章还简单讨论了计算机图形学中的走样问题。第9章介绍OpenGL着色语言（GLSL）中的可编程着色程序，它现在是OpenGL的一部分。我们可以利用可编程着色程序来改进技术方法，如凹凸映射现在能以实时方式实现。第8章和第9章讨论了用于所有交互式图形系统的标准观察流水线。

第10章包含若干松散内容，涉及的课题从构造封装各部分之间联系的模型到Internet图形学高级方法。这一章包含一个简单场景图API，并介绍了开放场景图（Open Scene Graph）。第11章介绍了若干过程方法，包括粒子系统、分形和过程噪声。第12章讨论了曲线和曲面，包括细分曲面。第13章考察了其他一些绘制方法，进一步讨论了光线跟踪和辐射度方法，还介绍了基于图像的绘制和并行绘制。

附录A主要是本书第一部分的程序，它们也可以从网上得到（参看辅助材料），附录B和附录C包含对背景数学知识的回顾，附录D包括本书用到的OpenGL函数一览。

相对第四版的变化

读者对于本书前4版的绝大多数评价都是肯定的，特别是对于采用OpenGL和自顶向下的方法更是如此。但是，总有某些改进之处，而且总有新的进展要加进来。可编程图形处理器的出现是一个主要的进展，我相信这对于如何讲授计算机图形学会有很大影响。第四版中介绍了可编程着色器和OpenGL着色语言。除了必须对以前版本中的内容进行重新组织以外，对于原先在多数系统上实现不了且只能以敷衍的方式介绍的技术，由于现在这些技术已经很平常了，所以适合在计算机图形学的初次课中来讲解它们。重新组织内容的另一个好处是，在现在的版本中，各章的内容同以前的版本相比与OpenGL绘制流水线更加一致。

在重写的第9章中，介绍了用OpenGL着色语言编写的可编程着色器，与以前的版本相比，重写后的这一章包含了更多的例子。利用可编程着色器，许多技术现在能够实时实现。因此，我对前3版中的一些材料进行了重新组织。这样，第三版把凹凸映射和纹理映射放到一起介绍，而在第四版则把凹凸映射和片元着色器放到一起介绍。虽然一些教师在开始时会觉得这个改变有些奇怪，但我相信当他们讲授顶点和片元着色器时就会看到这样做的好处。

第四版中去掉了第三版中科学计算可视化和过程方法这两章。关于科学计算可视化的一些内容，例如移动正方形方法被分散到其他章节里。然而，鉴于过程图形学的重要性，第五版中把它作为一章重新介绍，内容包括粒子系统、分形和过程噪声。

前几版的读者会注意到新版本对一些示例程序进行了修改。特别是把第3章中的绘图程序替换成一个操纵多边形的程序，我认为这个程序可以更清楚地演示交互和设计。第12章给出了一个基本的粒子系统的代码。第9章的可编程着色器的例子要比第四版更好。

读者对第1章到第6章的核心内容反响很好，所以这6章的改动较少，主要是探索改进教授方法。我相信这些修改对于一学期的导论课程的影响不大，但后面几章中的新内容可以为教师在两学期的课程中使用本书提供更多便利。

辅助材料

在作者的网站 www.cs.unm.edu/~angel 和 Addison-Wesley 出版公司的网站 www.aw.com/cssupport 上可以找到本书的辅助材料。所有读者都可以获得下面的辅助材料：

- 关于 OpenGL 的资源信息
- 在大多数流行的系统上利用 OpenGL 进行编程的操作指南
- 源程序代码
- 部分习题的答案
- PowerPoint 讲稿

此外还有一些附加的辅助材料，包括所有非编程习题的答案，但仅提供给在课堂上采用本书作为教材的教师。

致谢

在过去的几年中，我有幸在新墨西哥州大学遇到了许多优秀的学生，是他们最早使我对 OpenGL 产生了兴趣，从他们那里我学到了很多东西。这些学生包括：Hue Bumgarner-Kirby (Walker)、Ye Cong、Pat Crossno、Tommie Daniel、Chris Davis、Lisa Desjarlais、Kim Edlund、Lee Ann Fisk、Maria Gallegos，Brian Jones、Christopher Jordan、Max Hazelrigg、Sheryl Hurley、Thomas Keller、Ge Li、Pat McCormick、Al McPherson、Ken Moreland、Martin Muller、David Munich、Jim Pinkerton、Jim Prewett、Dave Rogers、Hal Smyer、Takeshi Hakamata、Dave Vick、Brian Wylie 和 Jin Xiong，许多彩图中的例子都出自这些学生。

本书第一版是我在休假期间写的，我在 5 个不同的国家完成了本书。这个经历很好地说明了便携式计算机的灵活性和 Internet 的无处不在。然而，如果没有一些人以及一些学校给我提供的帮助，我是完不成本书的，因为他们给我提供了便利。我非常感激的人包括：委内瑞拉安第斯大学（Universidad de los Andes）的 Jonas Montilva 和 Chris Birkbeck、厄瓜多尔天球赤道理工大学（Universidad Tecnologica Equinoccial）的 Rodrigo Gallegos 和 Aristides Novoa、台湾国立清华大学（National Tsing Hua University）的 Long Wen Chang、香港中文大学（Chinese University of Hong Kong）的 Kin Hong Wong 和 Pheng Ann Heng、国际超导技术中心（ISTEC）和新墨西哥州大学（The University of New Mexico）的 Ramiro Jordan 使我到许多地方的参观访问成为可能。无论我何时有问题，新墨西哥州大学的 John Brayer 和 Jason Stewart、Addison-Wesley 出版社的 Helen Goldstein 都设法帮我解决。我的网站上描述了写作第一版的经历。

第五版是我在新墨西哥州写的，许多使用过本书前 4 版的教师和学生向我提出了建议，我从他们的反馈中获益极大，许多对以前版本的修改都是对他们的要求和建议的回应。

NVIDIA 公司的 David Kirk 和 Mark Kilgard 慷慨地为很多测试算法提供了图形卡。其他一些人对我的帮助也很重要，我要感谢 Ben Bederson、Gonzalo Cartagenova、Tom Caudell、Kathi Collins、Kathleen Danielson、Roger Ehrich、Robert Geist、Chuck Hansen、Mark Henne、Bernard Moret、Dick Nordhaus、Helena Saona、Dave Shreiner、Vicki Shreiner、Gwen Sylvan 和 Mason Woo。承蒙 OpenGL 协会 Mark Kilgard、Brian Paul 和 Nate Robins 的极大帮助，他们编写了 OpenGL 软件，能使 OpenGL 代码在各种平台上都可以开发。

在新墨西哥州大学，艺术、研究、技术和科学实验室（ARTS Lab）对我的许多项目提供了支持，计算机科学系、美术学院的艺术技术中心、国家科学基金、美国 Sandia 国家实验室和 Los Alamos

国家实验室资助了许多我的学生和研究项目，本书的部分内容就来自这些研究项目中的一部分。David Beining和北极星天文馆为Full Dome项目[Sheryl Hurley, Christopher Jordan, Laurel Ladwig, Hue (Bumgarner-Kirby) Walker] 提供了极大的支持，该项目提供了本书彩图中的一些图像。Hue Walker为最近两个版本设计了精美的封面。

评阅人在本书应该包括哪些内容和应该介绍到什么程度这两方面提出了各种各样的看法，他们是：Gur Saran Adhar（威明顿北加州大学）、Mario Agrular（杰克逊维尔州立大学）、Michael Anderson（哈特福大学）、C. S. Bauer（中佛罗里达大学）、Marty Barrett（东田纳西州立大学）、Eric Brown和Robert P. Burton（杨伯翰大学）、Sam Buss（加利福尼亚大学圣地亚哥分校）、Kai H. Chang（奥本大学）、Ron DiNapoli（康奈尔大学）、Eric Alan Durant（密尔瓦基工程学校）、David S. Ebert（普度大学）、Chenyi Hu（阿肯色中央大学）、Mark Kilgard（NVIDIA公司）、Lisa B. Lancor（南康涅狄格州立大学）、Chung Lee（加州波莫纳理工大学）、John L. Lowther（密歇根理工大学）、R. Marshall（波士顿大学和桥水大学）、Bruce A. Maxwell（史威特摩尔学院）、James R. Miller（堪萨斯大学）、Rodrigo Obando（哥伦布州立大学）、Han-Wei Shen（俄亥俄州立大学）、Oliver Staadt（加州大学戴维斯分校）、Stephen L. Stepoway（南卫理工会大学）、Bill Toll（泰勒大学）、Michael Wainer（南伊利诺伊大学卡本戴尔分校）、George Wolberg（纽约市立大学）、Yang Wang（南卫理工会州立大学）、Steve Warren（堪萨斯州立大学）、Mike Way（佛罗里达南方学院）和Ying Zhu（乔治亚州立大学），虽然最后的定稿可能没有反映出他们的意见（因为有些意见彼此相左），但每位评阅人的意见都迫使我反思书稿的每一页内容。

我要感谢Addison-Wesley出版社的整个出版团队。在本书的前4版和*The OpenGL Primer*的出版过程中，我与Peter Gordon、Maite Suarez-Rivas和Matt Goldstein三位编辑合作得非常愉快，我在Addison-Wesley出版社所有书的出版过程中都得到了Katherine Harutunian的帮助。在出版前4版时，Paul Anagnostopoulos帮助我解决使用Windfall Software工具时的排版问题，帮助之大溢于言表。我要特别感谢编辑Lyn Dupré，因为我并不生来就会写作，如果读者能够看到第一版的原稿，就会理解Lyn Dupré为润色原稿所做的奇迹。

我的妻子Rose Mary Molnar制作了我的第一本图形学教材中的插图，其中许多插图构成了现在这本书的插图的基础。也许只有经历过图书出版过程并撰写过许多文稿的人以及为出版工作做出过贡献的人才能充分理解写书过程的艰辛。我真诚地把本书献给我的妻子，尽管这仍不足以表达我对她的谢意。

目　录

第 1 章　图形系统和模型 1

1.1　计算机图形学的应用 1
1.2　图形系统 3
1.3　图像：物理的和合成的 7
1.4　成像系统 11
1.5　虚拟照相机模型 14
1.6　程序员接口 16
1.7　图形绘制系统的体系结构 22
1.8　可编程流水线 25
1.9　性能特征 25
小结和注释 26
建议阅读资料 27
习题 27

第 2 章　图形学编程 29

2.1　Sierpinski 镂垫 29
2.2　编写二维图形应用程序 30
2.3　OpenGL API 34
2.4　图元和属性 37
2.5　颜色 44
2.6　观察 49
2.7　控制函数 52
2.8　Sierpinski 镂垫绘制程序 56
2.9　多边形和递归 57
2.10　三维 Sierpinski 镂垫 59
2.11　绘制隐函数曲线 63
小结和注释 69
建议阅读资料 70
习题 71

第 3 章　输入和交互 74

3.1　交互 74
3.2　输入设备 75
3.3　客户和服务器 80
3.4　显示列表 81
3.5　显示列表和建模 87

3.6 事件驱动输入编程 88
3.7 菜单 93
3.8 拾取 94
3.9 一个简单的CAD程序 100
3.10 建立交互式模型 105
3.11 交互式动画程序 108
3.12 交互式程序的设计 111
3.13 逻辑运算 112
小结和注释 117
建议阅读资料 117
习题 118

第4章 几何对象和变换 121
4.1 标量、点和向量 121
4.2 三维图元 129
4.3 坐标系和标架 130
4.4 OpenGL中的标架 140
4.5 建模一个彩色立方体 143
4.6 仿射变换 149
4.7 平移、旋转和缩放 151
4.8 变换的齐次坐标表示 154
4.9 变换的级联 158
4.10 OpenGL变换矩阵 164
4.11 三维应用程序的接口 167
4.12 四元数 171
小结和注释 174
建议阅读资料 174
习题 175

第5章 观察 177
5.1 经典观察和计算机观察 177
5.2 计算机观察 181
5.3 定位照相机 182
5.4 简单的投影 191
5.5 OpenGL中的投影 194
5.6 隐藏面消除 196
5.7 交互式网格显示 198
5.8 平行投影变换矩阵 202
5.9 透视投影变换矩阵 206
5.10 投影和阴影 210
小结和注释 212

建议阅读资料 213
习题 213

第 6 章 光照和明暗绘制 215
6.1 光线和材质 215
6.2 光源 218
6.3 Phong 光照模型 221
6.4 向量的计算 227
6.5 多边形明暗绘制 231
6.6 通过递归细分逼近球面 234
6.7 OpenGL 中的光源 237
6.8 OpenGL 中的材质属性 239
6.9 球面模型的明暗绘制 240
6.10 全局照明 242
小结和注释 243
建议阅读资料 244
习题 244

第 7 章 从顶点到片元 246
7.1 图形绘制流水线的基本实现策略 246
7.2 图形绘制系统的 4 个主要任务 248
7.3 裁剪 251
7.4 线段的裁剪 251
7.5 多边形的裁剪 255
7.6 其他图元的裁剪 257
7.7 三维裁剪 258
7.8 光栅化 260
7.9 Bresenham 算法 263
7.10 多边形光栅化算法 264
7.11 隐藏面消除 267
7.12 反走样 276
7.13 显示方面的问题 278
小结和注释 283
建议阅读资料 284
习题 284

第 8 章 离散技术 287
8.1 缓存 287
8.2 数字图像 288
8.3 缓存的写操作 290
8.4 OpenGL 中的位操作与像素操作 293
8.5 举例 298
8.6 映射方法 300

8.7 纹理映射 301
8.8 OpenGL 中的纹理映射 306
8.9 纹理生成 314
8.10 环境贴图 315
8.11 合成技术 319
8.12 多通道绘制与累积缓存 324
8.13 采样与走样 328
小结和注释 333
建议阅读资料 334
习题 334

第 9 章 可编程着色器 336
9.1 可编程流水线 336
9.2 着色语言 337
9.3 OpenGL 扩展 338
9.4 OpenGL 着色语言 339
9.5 OpenGL 着色语言 342
9.6 着色器与 OpenGL 程序的连接 346
9.7 移动顶点 350
9.8 基于顶点程序的光照计算 354
9.9 片元着色器 357
9.10 基于顶点与基于片元的光照计算 358
9.11 sampler 变量类型 359
9.12 立方体贴图 361
9.13 凹凸映射 366
小结和注释 372
建议阅读资料 372
习题 373

第 10 章 层级建模方法 374
10.1 图符和实例 374
10.2 层级模型 375
10.3 机器人手臂 377
10.4 树与树的遍历 379
10.5 使用树形数据结构 382
10.6 动画 385
10.7 图形对象 386
10.8 场景图 391
10.9 一个简单的场景图 API 393
10.10 开放场景图（OSG） 406
10.11 图形和因特网 408
10.12 其他树形结构 410

小结和注释 414
建议阅读资料 415
习题 415

第 11 章　过程建模方法 417
11.1　基于算法的模型 417
11.2　基于物理的模型和粒子系统 418
11.3　牛顿粒子 419
11.4　求解粒子系统方程 423
11.5　约束条件 425
11.6　一个简单的子系统 428
11.7　基于语言的模型 431
11.8　递归方法和分形 434
11.9　过程噪声 439
小结和注释 442
建议阅读资料 442
习题 442

第 12 章　曲线和曲面 444
12.1　曲线和曲面的表示形式 444
12.2　设计准则 448
12.3　三次参数多项式曲线 450
12.4　插值 450
12.5　Hermite 曲线和曲面 454
12.6　Bezier 曲线和曲面 457
12.7　三次 B 样条 460
12.8　普通 B 样条 464
12.9　曲线和曲面的绘制 467
12.10　Utah 茶壶模型 472
12.11　代数曲面 473
12.12　OpenGL 中的曲线和曲面 477
小结和注释 481
建议阅读资料 482
习题 482

第 13 章　高级绘制 484
13.1　超越流水线绘制结构 484
13.2　光线跟踪 485
13.3　构建一个简单的光线跟踪器 487
13.4　绘制方程 491
13.5　辐射度方法 493
13.6　RenderMan 498

13.7 并行绘制 499
13.8 基于图像的绘制 504
小结和注释 506
建议阅读资料 507
习题 507

附录 A 示例程序 509

附录 B 空间 567

附录 C 矩阵 574

附录 D OpenGL 函数简介 581

参考文献 593

第1章　图形系统和模型

也许，21世纪最显著的特征是：计算机和通信技术成为了人们生活中的支配力量。当这些技术改变我们日常生活方式的时候，社会活动的许多方面，比如电影制作、出版业、银行业和教育业，都在持续地经历着翻天覆地的变化。计算机、网络和复杂的人类视觉系统通过计算机图形学结合起来，使得我们能够以崭新的方式来显示信息、观察虚拟世界以及与人和机器进行通信。

计算机图形学（computer graphics）考虑的是用计算机生成图片或者图像的方方面面。大约在50年前，阴极射线管（CRT）可以显示几条线，这个领域由此起步，不过在当时并没有受到重视；而现在，我们可以用计算机生成和真实物体的照片不可区分的图像。仿真飞机能够生成虚拟环境的实时图形显示，由此可以训练飞行员，这已经不稀奇了。完全由计算机制作的长篇电影不仅获得了好评，在经济上也取得了成功。大型多人游戏可以同时容纳上万名玩家。

这一章将首先简短地讨论计算机图形学的应用，然后简单介绍图形系统和图像的生成。自始至终，本书的方法是强调计算机图形学与手画或者拍照这些人们更熟悉的成像过程之间的联系。后面会讲到，这种联系可以帮助人们设计应用程序、图形库和图形系统的体系结构。

本书将介绍一个特定的图形软件系统——OpenGL。在开发图形应用程序方面，OpenGL已经成为一个被广泛接受的标准。很幸运，OpenGL易于学习，而且它也具备其他流行的图形系统的大多数特征。本书采用自顶向下的方法。我们希望读者能够尽快开始编写具有图形输出的应用程序。在读者开始编写简单的程序之后，会讨论底层的图形库和硬件是如何实现的。本章将为读者开始编写程序提供足够的概要介绍。

1.1　计算机图形学的应用

用户的需求和软/硬件技术的进展推动了计算机图形学的发展。虽然计算机图形学的应用多种多样，但是可以把它们分成4个主要领域：

1. 信息的显示
2. 设计
3. 仿真和动画
4. 用户界面

虽然许多应用涉及两个或者更多的领域，但是各领域的发展是独立的，计算机图形学的发展则建立在这些领域的基础之上。

1.1.1　信息的显示

古典的图形技术是作为一种在人与人之间传递信息的媒介而出现的。尽管口说和书写也服务于类似的目的，但就作为数据处理器和模式识别器来讲，它们无法和人类视觉系统相比拟。4000多年以前，古巴比伦人在石块上绘制建筑物的平面图。2000多年以前，古希腊人就已经能以图形的方式表达他们的建筑构想，尽管相关的数学直到文艺复兴时期才发展出来。今天，建筑师、机械设计师和绘图人员使用计算机图形系统来生成同样类型的信息。

几个世纪以来，制图学家绘制地图以显示天文和地理信息。这样的地图对于航海家是至关重要的，因为他们期望探究地球的尽头。今天，地图对诸如地理信息系统这样的领域同样重要。现在，地图可以在因特网上实时地显现和处理。

在过去的150年里，统计学工作者研究了生成图表的技术，这些图表可以帮助观察者理解数据集中的信息。现在，我们有计算机绘图包，它可以提供各种各样的绘图技术和颜色工具，能处理多个大型数据集。尽管如此，仍然需要由人来识别出视觉模式，这种模式最终使人们能够解释包含在数据中的信息。为了发现潜在的安全威胁，必须理解生物信息学中的许多复杂现象，所以信息可视化这个领域越来越重要。

医学成像提出了有趣且重要的数据分析问题。像计算机断层扫描（CT）、磁共振成像（MRI）、超声波和正电子发射断层扫描（PET）这样的现代成像技术生成三维数据，但这些数据必须经过算法的处理才能提供有用的信息。彩图24展示了一幅人体头部的图像，其中皮肤显示成透明的，内部结构显示为不透明的。尽管数据是由医学成像系统采集的，但展示其内部信息的图像是由计算机图形学生成的。

现在，超级计算机使得许多领域的研究人员能够求解原先难以解决的问题。科学可视化这个领域提供了图形工具，这些工具帮助研究人员解释他们生成的海量数据。在诸如流体力学、分子生物学和数学这样的领域中，计算机图形学把数据转化成可被显示的几何实体，由此生成的图像使人们对许多复杂过程有了新的理解。例如，彩图23展示了地幔的流动特性，其采用的绘制系统通过一个数学模型来生成所用到的数据。信息可视化这个领域利用计算机图形学来辅助人们发现数据集中的联系，这里在数据和数据如何被可视化之间没有物理上的关联。本书的其余部分会介绍各种各样的可视化技术。

1.1.2　设计

像工程和建筑这样的行业需要考虑设计。从一组规范开始，工程师和建筑师期望找到满足这些规范的有成本效益且符合美学的解。设计是一个迭代的过程。现实世界中的问题很少存在唯一的最优解。设计问题要么是超定的，即不存在满足所有规范的解；要么是欠定的，即存在多个满足设计规范的解。于是，设计人员是以迭代的方式工作的，即先提出一种可能的设计方案，然后对其进行试验，再把得到的结果作为寻找其他方案的基础。

人与CRT屏幕上的图像进行交互，这一范型的威力在40多年前被Ivan Sutherland认识到。今天，交互式图形工具在计算机辅助设计（CAD）中的使用已经遍及许多领域，比如建筑、机械工程、超大规模集成电路（VLSI）的设计和动画人物的创作。在所有这些应用中，使用图形学的方式各有不同。例如，在VLSI设计中，图形学一般通过像菜单和图标这样的工具在用户和设计包之间提供一个交互式接口。此外，在用户生成了一个可能的设计之后，其他的工具对这个设计进行分析并把分析结果用图形的方式显示出来。彩图13和彩图14展示了同一建筑设计的两个视图。这两幅图像是由同一个CAD系统生成的，它们演示了在设计过程的不同阶段能够生成同一对象的不同图像是重要的。

1.1.3　仿真和动画

当图形系统发展到能够实时生成复杂图像时，工程师和研究人员就开始把它用做仿真器。最重要的应用之一是训练飞行员。图形飞行仿真器已被证明可以增加安全性并减少训练费用。特殊VLSI芯片的使用导致了同飞行仿真器一样复杂的一代街机游戏的出现。

家用计算机使用的游戏软件和教育软件一样使人印象深刻。彩图20展示了一个机器人和相应的图形仿真。这个仿真器可以用来设计机器人，规划它的路径，仿真它在复杂环境中的行为。

飞行仿真器的成功使得计算机图形学又被应用到电视、运动图像和广告业中的动画制作。现在整个动画电影都可以用计算机制作，代价低于传统的手工动画技术。计算机图形学和手工动画结合在一起可以实现许多技术和艺术效果，这些效果如果只采用其中一种方法是无法获得的。计算机动画看起来有些独特，不过我们也可以用计算机生成具有真实感的图像。在电视、电影和杂志中看到的图像往往是如此逼真以至于无法把计算机生成或是修改的图像和照片相区别。第6章和第9章将讨论在生成计算机动画时使用的许多光照效果。彩图 17 展示了由计算机生成的视频的一个场景。艺术家和工程师在制作该场景时所使用的图形软件已经商品化了。这个彩图绘制的是工厂里的焊接机器人，从中可以看到计算机图形学生成逼真图像的能力。图中的火花是利用过程方法生成的特效，在第 11 章会讨论这些技术。彩图 35 展示了计算机图形学生成效果的另一个例子，其中的逼真效果无法用其他方法获得。彩图 27 和彩图 28 也是真实感绘制的例子。

虚拟现实（VR）为计算机图形学的应用开启了许多新的视野。头盔式显示器可以使人的左眼和右眼看到的图像不同，从而可以获得立体视觉的效果。此外计算机还可以跟踪人体的位置和姿势，可能还包括头和手指的姿态。还可能有其他的交互设备，包括力传感手套和声音定位设备等。这样，人就可以作为由计算机生成场景的一部分，人在这个虚拟场景中的活动仅仅受限于计算机生成图像的能力。例如，可以用这种方式训练外科实习医生做手术或者训练宇航员在失重环境中工作。彩图 26 展示了 VR 仿真的一帧图像，其中有一个供医疗人员进行远程训练的虚拟病人。

为了支持交互式视频游戏，图形绘制系统不但需要使用标准的商用计算机，还需要配备专用的图形硬件。在很大程度上，游戏推动了图形硬件的发展。从商业角度来看，来自视频游戏的税收已经超过了来自商业电影的税收。个人计算机中图形卡上的图形处理单元以及像 Xbox 和 PlayStation 这样的电玩都是针对游戏开发的，它们正在取代仿真中所使用的昂贵的专用硬件。彩图 20 展示了一个机器人战士，这是一款交互游戏的一帧图像，其中使用了层级建模方法（见第 10 章）和过程建模方法来生成烟和火（见第 11 章），还使用了噪声纹理来生成地形（见第 11 章），而驱动该游戏的引擎则使用了第 10 章介绍的场景图。

1.1.4　用户界面

我们与计算机的交互方式受可视化对象的影响，包括窗口、菜单、图标和定位设备，比如鼠标等。从用户的角度来看，像 X Window System、Microsoft Windows 和 Macintosh OS X 这样的视窗系统只是在细节上不同。最近，许许多多的人成为了因特网用户。他们借助像 Firefox 和 Internet Explorer这样的图形网络浏览器来访问网络，这些浏览器也使用了同样的界面工具。现在人们已经对这些界面风格习以为常了，经常忘记是在和计算机图形学打交道。

尽管人们对于在大多数工作站[①]上使用的图形用户界面已经很熟悉了，然而计算机图形学的进展使得其他的界面形式成为可能。彩图 18 展示了一个高级建模软件包所使用的界面，其中演示了这样的软件包所提供的多种工具和用户在建模几何对象时可以利用的交互设备。

1.2　图形系统

一个计算机图形系统首先是一个计算机系统，因此，它必然包含一个通用计算机系统的所有部件。从较高层面来看，图形系统可以用图 1.1 来表示。从该图可以看到，图形系统包含 5 个主要元素：

① 虽然个人计算机和工作站是从多少有些不同的途径发展而来，但是现在两者之间并没有本质的差别，所以我们把个人计算机和工作站当做同义词。

1. 输入设备
2. 处理器
3. 存储器
4. 帧缓存
5. 输出设备

这个模型足够一般，包括工作站、个人计算机、交互式游戏系统和复杂的图像生成系统。虽然图 1.1 中所有的部件（可能帧缓存是个例外）都出现在标准计算机中，但是其每个部件服务于计算机图形学的方式可以体现出图形系统的特征。

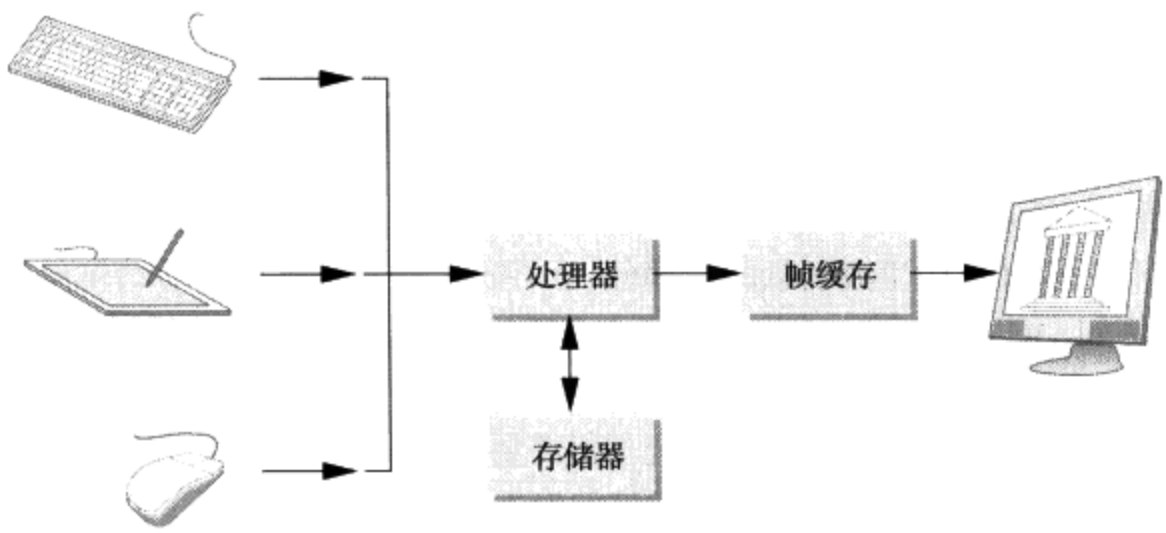


图 1.1　一个图形系统

1.2.1　像素和帧缓存

目前，几乎所有的图形系统都是基于光栅的。在这样的图形系统中，一幅图片是作为图片元素的阵列来生成的，图片元素也称像素（pixel），像素阵列也称光栅（raster）。从图 1.2 可以看到，每一个像素对应于图像中的一个位置，或者一块小的区域。这些像素都保存在一个称为帧缓存（frame buffer）的储存区域中。帧缓存可以看做是图形系统的核心元素。帧缓存中像素的数目称为分辨率（resolution），它决定了从图像中可以分辨出多少细节。帧缓存的深度（depth）或者精度（precision）是表示每个像素所用的比特数，它决定了诸如给定系统中可以表示多少种颜色之类的性质。例如，深度为 1 比特的帧缓存只允许有两种颜色，而深度为 8 比特的帧缓存可以表示 256（2^8）种颜色。在全彩色（full-color）系统中，每个像素有 24 比特（或者更多）。这样的系统可以表示足够多的颜色，从而能够逼真地显示绝大多数图像。它们也被称为真彩色（true-color）系统或者 RGB 颜色（RGB-color）系统，因为每个像素对应的三组比特位被分别分配给三种原色，即大多数显示器所使用的红、绿、蓝三原色。

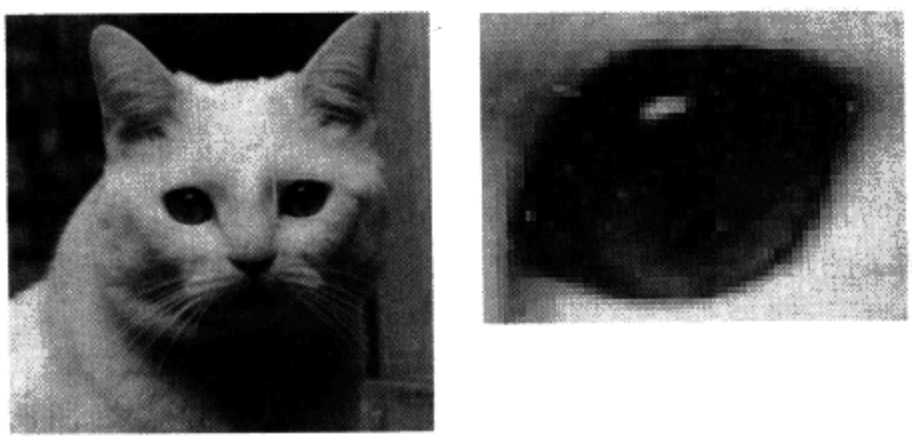


图 1.2　像素：(a) 一幅雪猫的图像；(b) 眼睛区域的细节，显示出了单个的像素

高动态范围的应用需要使用超过 24 比特的定点数来表示 RGB 颜色。近来有些帧缓存把 RGB 颜色值存储为标准 IEEE 格式的浮点数。因此，术语真彩色应该解释为具有足够深度的帧缓存，这样可以用 RGB 值，而不是用一个有限颜色集的索引来表示颜色。第 2 章还会讨论这一点。

帧缓存一般使用特殊类型的存储器芯片来实现，利用这种芯片可以快速重新显示帧缓存中的内容。在基于软件的图形系统中，帧缓存是系统存储器的一部分。属于这种情况的系统有：用于高分辨率绘制的系统，以及生成复杂的、不能实时生成的视觉效果的系统。

在一个非常简单的系统中，帧缓存只存储屏幕上显示的像素。在大多数系统中，帧缓存存储的信息比这要多很多，比如为了生成三维数据的图像需要有深度信息。在这样的系统中，帧缓存包括许多缓存，其中有一个或多个存储要显示的像素颜色，称为颜色缓存（color buffer）。眼下可以把帧缓存和颜色缓存当做同义词使用而不会产生混淆。

在一个简单的系统中，可能只有一个处理器，即系统的中央处理单元（central processing unit，CPU），常规的处理和图形处理都必须由这个处理器来完成。处理器要完成的图形处理功能主要是获取由应用程序生成的图元（比如线、圆、多边形）的属性，并为帧缓存中的像素赋值，以最佳地表示这些图元。例如，一个三角形由它的三个顶点所确定，但为了通过三条边来显示这个三角形的轮廓，图形系统必须生成一组像素，这些像素在观察者看来构成了线段。从几何实体到帧缓存中像素的颜色和位置的转换称为光栅化（rasterization）或者扫描转换（scan conversion）。在早期的图形系统中，帧缓存是标准存储器的一部分，可以被 CPU 直接访问。今天，几乎所有图形系统都已经使用专用的图形处理单元（graphics processing unit，GPU），它是专门用来完成图形处理功能的。GPU 可以在系统的主板上或者图形卡中。帧缓存可以被 GPU 访问，而且可能就在 GPU 中。

1.2.2　输出设备

很多年以来，显示器（监视器）主要是阴极射线管（cathoderay tube，CRT）。虽然各种各样的平板技术现在更为流行，但 CRT 的基本功能和这些较新的显示设备是相同的。图 1.3 是 CRT 的简化示意图。CRT 因电子轰击涂在管子上的磷光物质而发光。电子束的方向由两对偏转板控制。计算机的输出由数模转换器变成 x 偏转板和 y 偏转板上的电压。当强度足够大的电子束轰击到磷光物质上时 CRT 的屏幕就会发光。

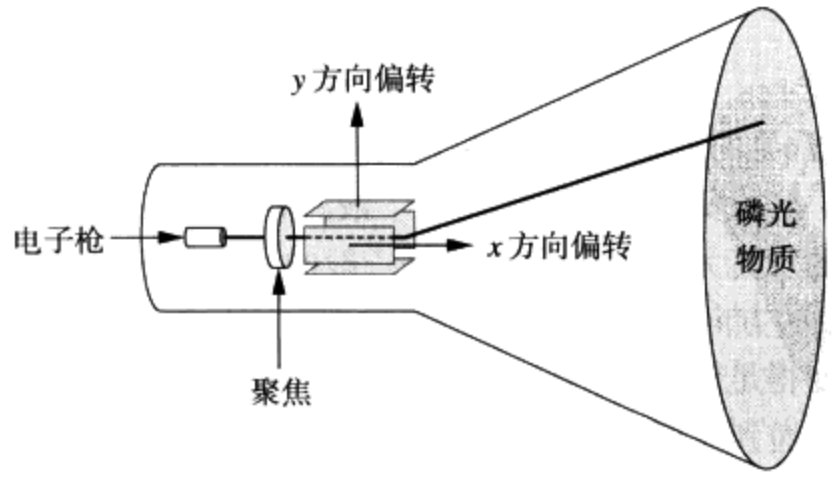


图 1.3　阴极射线管（CRT）

如果控制电子束偏转的电压以恒定的速率改变，那么电子束会扫过一条直线轨迹，观察者是可以看到这条直线轨迹的。这样的设备称为随机扫描（random-scan）CRT、画线（calligraphic）CRT 或者向量（vector）CRT，因为电子束可以直接从任意一个位置移动到另一个任意位置。如果关闭电子枪，那么电子束可以移动到新的位置而不会改变可见的显示输出。这些就是早期图形系统的基础，今天的光栅技术是在此之后发展起来的。

在磷光物质被电子束激发以后，典型CRT的发光只能持续很短的时间，一般是几毫秒。为了让人们看到稳定的、不闪烁的图像，电子束必须以足够高的速率重复扫描相同的路径，这就是刷新（refresh），刷新的速率称为刷新率（refresh rate）。在早期的系统中，刷新率由电源的频率决定，在美国是60 Hz（赫兹），在世界上其他大多数地方是50 Hz。现代的显示器不再受限于这些较低的频率，刷新率大约可达85 Hz。

在光栅系统中，有两种基本方式把帧缓存中的像素显示为屏幕上的点。在逐行扫描（progressive）或者非隔行扫描（noninterlaced）显示器中，像素按照刷新率一行一行地显示，或者说一条扫描线一条扫描线地显示。在隔行扫描（interlaced）显示器中，奇数行和偶数行被交替刷新。商用电视使用的是隔行扫描显示器。在刷新率是60 Hz的隔行扫描显示器中，每秒钟屏幕只被完整刷新30次，但是人们会被自己的视觉系统所欺骗，感觉刷新率是60 Hz而不是30 Hz。不过距离屏幕较近的观察者能够观察到隔行扫描和逐行扫描的差异。虽然非隔行扫描显示器要以两倍于隔行扫描显示器的速度处理像素，但这类显示器的应用还是越来越广泛。

彩色CRT中有三种不同的有色磷光物质（红、绿、蓝），这些磷光物质被布置成许多小的颜色组。一种通常的做法是把这三种磷光物质布置到称为三色组（triad）的一个三角形的颜色组中，每个三色组有三种磷光物质，其中每种磷光物质对应一种原色。大多数彩色CRT有三个电子枪，这是和三种磷光物质相对应的。在图1.4所示的荫罩式CRT中，用一个带有许多小孔的金属屏来确保每个电子枪不会激发其他颜色的磷光物质，这个带孔的金属屏称为荫罩（shadow mask）。

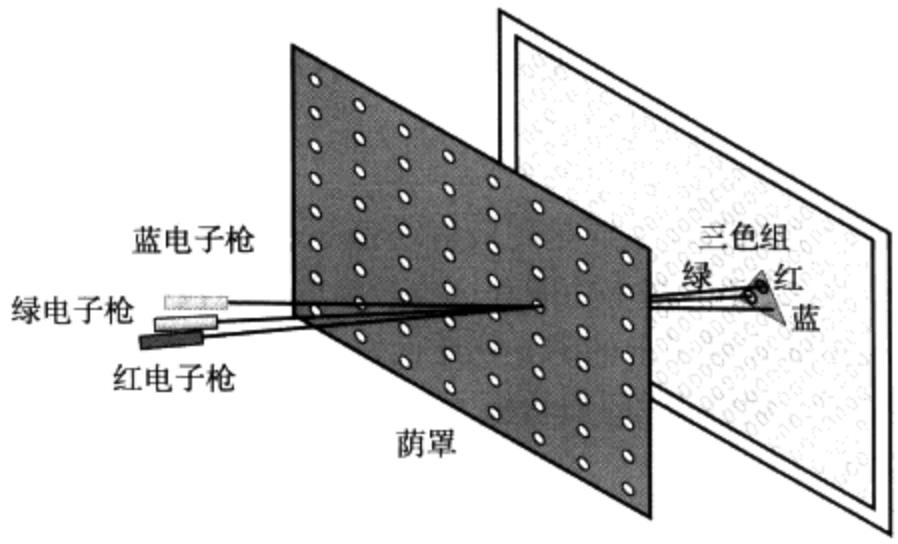


图1.4　荫罩式CRT

虽然CRT仍旧是最常见的显示设备，但它将很快被平板技术取代。平板显示器一定是光栅显示器。许多可用的技术，包括发光二极管（LED）、液晶显示器（LCD）、等离子显示器，都使用二维栅格来寻址每个单独的发光元件。图1.5所示是一般的平板显示器。靠外的两块板包含平行的电线栅，这两块板上电线栅的方向是互相垂直的。通过把电信号加到每个电线栅合适的电线上，这两条电线交点处的电场会变得足够强大，从而可以控制中间板上对应的元件。LED显示器的中间板上有发光二极管，可以由传递到栅格上的电信号控制开启和关闭。在LCD显示器中，利用电场来改变中间板上的液晶分子的排列方向，这样可以允许或者阻挡光线通过。在等离子显示器中，包含电线栅的玻璃板之间有气体，通过栅格上的电压来激发玻璃板之间的气体，获得能量的气体就变成了发光的等离子态。

大多数投影系统也是光栅设备，这些系统使用的技术多种多样，包括CRT和DLP（数字光处理投影机）。从用户的角度来看，它们的作用和具有相似分辨率和精度的标准监视器相同。硬拷贝设备，比如打印机和绘图仪，也是基于光栅的，只是它们不需要刷新。

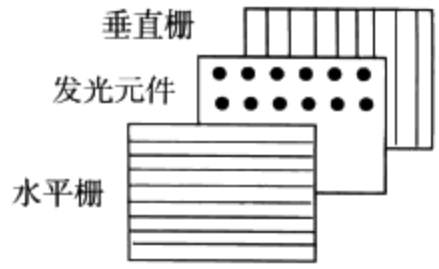


图 1.5　一般的平板显示器

直到最近，大多数显示器的宽高比（aspect ratio）还都是4：3，这个宽高比是和商用电视相对应的。早期的VGA分辨率是640 × 480像素，这和标准NTSC制式的扫描线数目一致[①]。后来计算机显示器的分辨率又发展到流行的1024 × 768（XGA）和1280 × 1024（SXGA）。较新的高清电视（HDTV）标准的宽高比介于早期电视和电影的宽高比之间，为16：9。HDTV监视器有逐行扫描模式（1080p, 780p）和隔行扫描模式（1080i, 780i），扫描线条数是780或1080。因此，现在最流行的计算机显示器的分辨率是与HDTV的宽高比相一致的1920 × 1080和1280 × 720，以及与XGA和SXGA显示器的垂直分辨率相一致的1280 × 768和1920 × 1024。对高端应用，现在有适合于商用数字电影的“4K”（4096 × 2160）数字投影仪。

1.2.3　输入设备

大多数图形系统都有键盘，并且至少还有一个其他类型的输入设备。最常见的输入设备是鼠标、操纵杆和数据板。它们为系统提供位置信息，并且都装有一个或多个按钮来向处理器发送信号。这些设备经常被称做定位设备（pointing device），它们允许用户在屏幕上指示一个特定位置。本书将在第3章介绍这些设备。

游戏控制台没有键盘，但包含的输入设备比标准工作站要多。一个典型的控制台可能会有多个按钮、一个操纵杆和多个刻度盘。像Nintendo Wii这样的设备是无线的，并且可以检测到三个方向的加速度。

游戏、CAD和虚拟现实都需要输入设备能够提供多于二维的数据。有许多设备可以获得真实世界中物体的三维位置，包括激光测距仪和声传感器。像数据手套这样的设备装有很多传感器并配备计算机视觉系统，可以提供更高维的数据。

1.3　图像：物理的和合成的

讲授计算机图形学的传统方法把重点放在如何在帧缓存中构造简单二维几何实体（比如点、直线、多边形）的光栅图像。大多数教科书接下来会讨论如何在计算机中定义二维或三维数学对象并利用二维光栅化的图元来绘制它们。

这种方法对于绘制简单对象的简单图像是很好的。可是，在现代图形系统中，人们希望充分利用软/硬件的性能来绘制由计算机生成的三维对象的逼真图像，这个任务涉及图像生成的许多方面，比如光照、明暗处理和材质的属性。因为大多数现代计算机图形系统都直接支持这样的功能，所以在这里更愿意为介绍生成这些逼真图像的方法做准备，而不是以后再去扩展一个功能有限的模型。

计算机生成的图像是合成的，或者说是人造的，因为被绘制的对象也许在物理上并不存在。我们要讨论的用计算机来生成图像的方法和传统的成像方法是相似的，照相机和人类视觉系统都是使用传统成像方法的例子。因此，在介绍如何编写生成图像的程序之前，先讨论在光学系统中图

① 在美国以外，PAL制式和SECAM制式的扫描线数目更多但帧速率更低。

像是如何生成的。我们先构造一个图像生成过程的模型，然后就可以用它来理解和开发计算机成像系统了。

本章将尽可能少地使用数学。目的是建立一个生成图像的模式并给出实现这个模式的计算机体系结构。细节问题将放到后面的章节里讲解，届时会导出相关的方程。

1.3.1　对象和观察者

我们生活在一个三维的世界里。像测量物体的尺寸和物体间的距离这样的想法都属于简单的思想，但把这些思想在概念上系统化之后就发展出数学的许多分支，包括几何和三角。人们经常想用图片或者图像来表达对空间关系的理解，比如地图、绘画、照片等，而且为了看到物体间的空间关系还设计出许多像照相机、显微镜和望远镜这样的物理设备。因此，在图像生成的物理方面和数学方面之间总是存在着一个基本联系，我们可以利用这个联系来学习计算机生成图像的方法。

在任何成像过程中都必须有两个基本的实体：对象和观察者，不管成像是数学的还是物理的。对象在空间中的存在是不依赖于任何图像生成过程和任何观察者的。计算机图形学考虑的是人造的对象，人们通过点、线和多边形这样的几何图元来构造对象。在大多数图形系统中，利用一组空间中的位置，或者说顶点（vertex），就能够定义或者近似描述大多数对象了。例如，两个顶点可以确定一条直线；一个有序的顶点列表可以确定一个多边形；球面可以由在球心的一个顶点和在球面上任意位置处的另一个顶点来确定。CAD系统的主要功能之一就是要提供一个界面，使用户可以方便地构造现实世界的虚拟模型。第2章会讲到如何用OpenGL构造简单的对象。第11章将介绍一种定义对象的方法，这种方法可以把对象之间的关系考虑进来。

每个成像系统都必须提供一种从对象生成图像的方法。为了生成一幅图像，必须得有某人或某物在观察对象。观察对象的可以是人、照相机或者数字化设备，是这个观察者（viewer）生成了对象的图像。人类视觉系统是在眼睛的后部成像而照相机是在胶片平面上成像。图像和对象容易被混淆。人们通常从个人的视角看一个对象，而没有注意对于位于其他位置的观察者来说，他们看到的该对象会和其他人所看到的不同。图1.6(a)所示的是两个观察者在观看同一个建筑物，该图是另一个观察者A所看到的图像。因为A离得足够远，所以建筑物和两个观察者B和C都能被A看到。从A的视角看来，B和C跟建筑物一样，都是对象。图1.6(b)和图1.6(c)分别给出了B和C所看到的图像。所有这三幅图像都包含同一个建筑物，但这个建筑物所成的像在这三幅图像中是各不相同的。

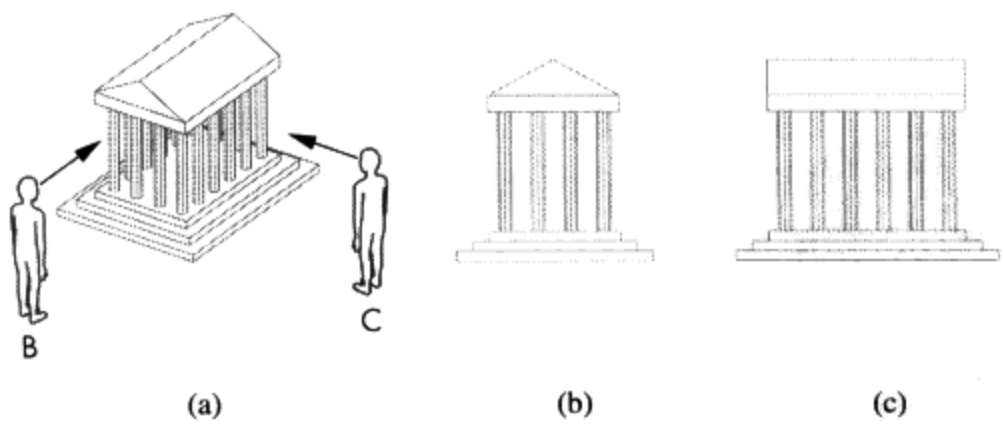


图1.6　由三个不同的观察者所看到的图像：(a) 观察者A的视图；(b) 观察者B的视图；(c) 观察者C的视图

图1.7所示的是一个对建筑物拍照的照相机系统。这里可以看出对象和观察者都处在一个三维世界里。可是，由观察者所定义的图像（在胶片平面上所成的像）是二维的。对对象的指定是与对观察者的指定组合在一起来获得一幅二维图像的，这个过程是图像生成的核心，后面会详细讨论。

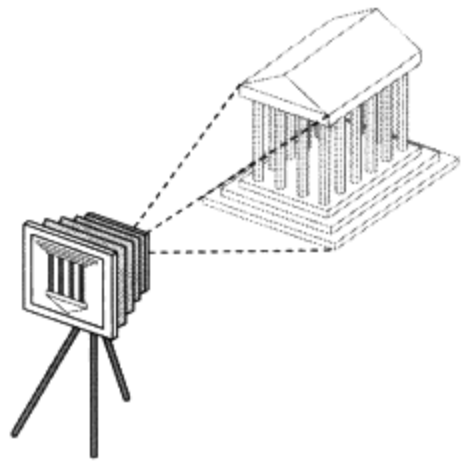


图 1.7　照相机系统

1.3.2　光和图像

前面对图像生成的描述远远不够完备。比如，我们还没有提到光。如果没有光源，对象看起来是黑的，图像里不会有任何可见的内容。我们没有提及颜色是如何出现在图像中的，也没有提及对象表面的材质属性会对图像有什么影响。

这里的分析方法更注重物理过程，让我们从图 1.8 所示的一个简单的物理成像系统开始讨论。我们仍旧看到了一个实际的对象和一个观察者（照相机），不过现在场景中有一个光源了。来自光源的光线照射到对象的各个表面，反射的光线中有一部分通过镜头进入到照相机中。光线和对象表面相互作用的细节决定了有多少光线进入照相机。

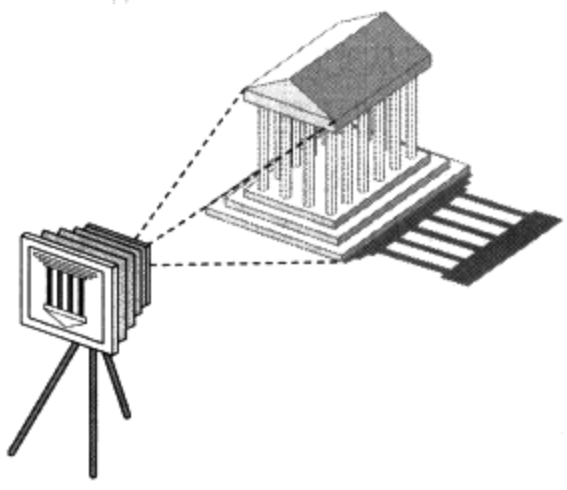


图 1.8　对象、光源以及照相机系统

光是电磁辐射的一种形式。电磁能以波的形式传播就是电磁波，电磁波可以由其波长或者频率来表征[①]。电磁波谱（见图 1.9）包括无线电波、红外线，还有能引起视觉系统响应的部分。波长为 350 ~ 780 nm（纳米）的可见光谱（visible spectrum）称为可见光。给定光源的颜色由其能量依波长的分布决定。可见光谱的中间部分，即波长大约为 520 nm 的光为绿色；波长大约为 450 nm 的光为蓝色；波长大约为 650 nm 的光为红色。正如在彩虹中所看到的那样，波长在红和绿之间的光为黄色；波长比蓝色更短的光为紫色。

光源所发出光线的频率可以是一组离散值，也可以是连续值。比如，激光的频率是单一的，而白炽灯则在一个频率范围内辐射能量。幸运的是，在计算机图形学中，除了需要认识到不同的频率表现为不同的颜色以外，很少需要考虑光的物理性质，例如它的波动本性。

① 频率（f）和波长（λ）之间的关系是 $f\lambda = c$，其中 c 是光速。

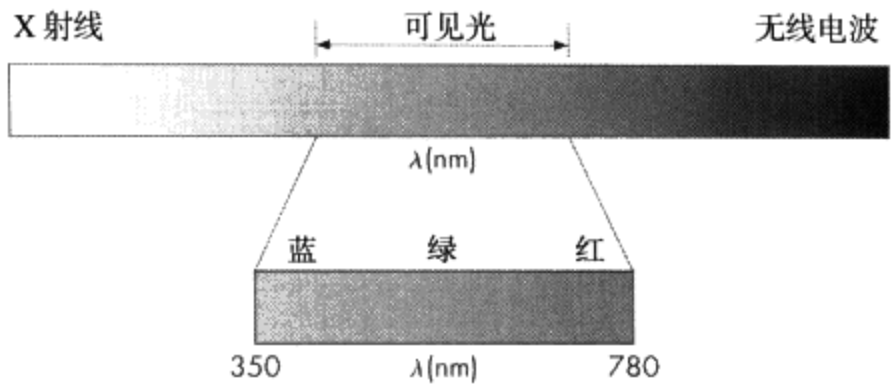


图 1.9　电磁波谱

可以采取更传统的方法来处理光，但前提是光线足够强并且光的波动本性不显著。几何光学（geometric optics）把光源建模为具有固定强度的光能的辐射器。几何光学认为，光线从光源出发沿直线到达与其相互作用的对象。一个理想的点光源（point source）从一个点向所有方向均匀地发射光线，发射出的光线可以是单一频率的，也可以包含多个频率。像灯泡这样更复杂的光源可以表征为从一块区域上辐射能量，并且向某个方向发射的光线比向其他方向发射的光线更强。一个特定的光源由其所发出光线的能量依频率和方向的分布来表征。现在只考虑点光源。更复杂的光源常常可以由一些精心布置的点光源来近似。第 6 章将讨论光源的建模。

1.3.3　图像生成模型

给定了场景中的对象、对象反射光的性质和场景中光源的性质，有许多方法可以生成该场景的图像。这一节将介绍两种物理方法。虽然这两种方法并不能够用来实现最终目的——实时生成图像，但是它们可以使读者理解如何构造一个有用的成像系统。

为了构造一个成像模型，可以先跟踪从光源发出的光线。考虑图 1.10 所示的只有一个点光源的场景。把观察者也画在图中是因为我们对到达观察者眼睛的光线感兴趣。观察者也可以是照相机，如图 1.11 所示。一条射线（ray）是从一个点出发，向着一个特定方向无限延伸的半无穷直线。因为光沿直线传播，所以可以把光线看做是从点光源出发向所有方向发出的许多条射线。这些射线当中的一部分到达照相机的胶片平面，就形成了图像。例如，如果从照相机可以看到光源，那么一部分光线就可以从光源出发，直接通过镜头到达胶片平面。然而，大多数光线一直照射到无穷远，既没有直接进入照相机，也没有到达任何对象。这些光线对图像没有贡献，尽管它们可能会被其他观察者看到。余下的光线照射到对象，这些光线可以与对象的表面以各种方式相互作用。例如，如果表面是镜子，那么反射光线可能会进入照相机的镜头（这要视表面的方向而定），从而对所成的图像有贡献。不是理想镜面的表面会把光线向各个方向散射。如果表面是透明的，那么来自光源的光线可以穿过它，然后可能和其他对象相互作用，进入照相机，或者没有到达其他对象，一直跑到无穷远[①]。图 1.11 给出了一些可能的情况。

光线跟踪（ray tracing）和光子映射（photon mapping）是基于上面这些思想的图像生成技术，它们可以作为利用计算机生成图像的基础。可以利用光线跟踪的思想来模拟任意复杂的物理效果，只要愿意进行所要求的计算。虽然跟踪光线可以提供对物理世界的一个很好的近似，但它并不适合于实时计算。

其他图像生成的物理方法以能量守恒为基础，其中在计算机图形学中最重要的是辐射度方法（radiosity）。这种方法最适合于把入射光线均匀地向所有方向散射的表面。但即使对这种情况，辐射度方法所需要的计算量仍然太大，因而不能实时实现。

① 这个图像生成的模型在 500 年前被 Leonardo da Vinci 使用过。

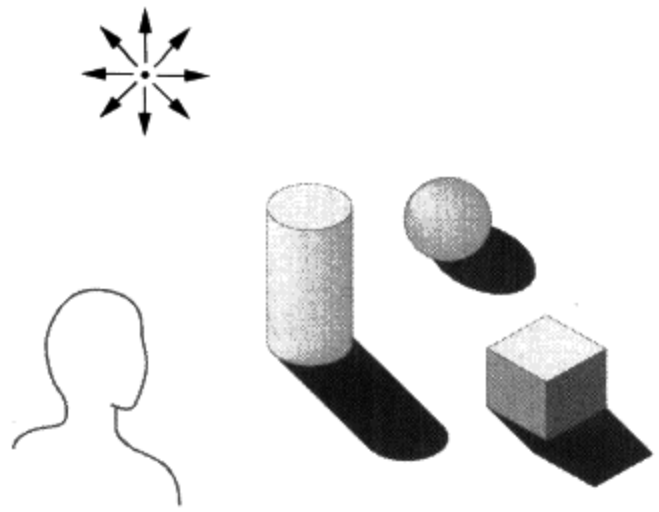


图 1.10　只有一个点光源的场景

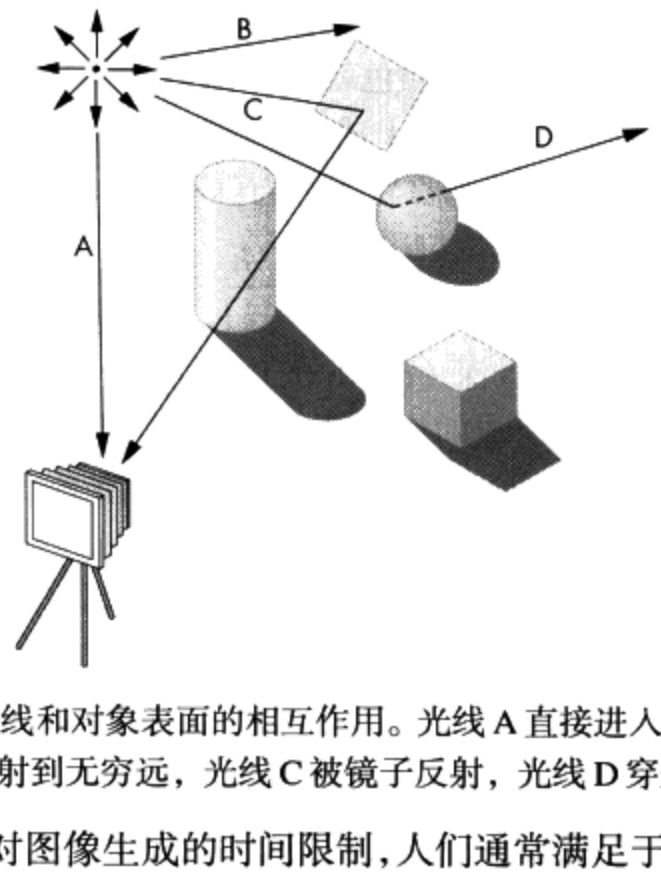


图 1.11　光线和对象表面的相互作用。光线 A 直接进入照相机，光线 B 射到无穷远，光线 C 被镜子反射，光线 D 穿过一个透明球

由于实时计算机图形学对图像生成的时间限制，人们通常满足于看起来合理的图像，而不是物理上正确的图像。当然对于不需要实时生成的图像，情况不是这样，比如那些为长篇电影而生成的图像，每一帧可以耗费计算机几小时的时间。随着当前硬件速度的不断提高，实时图形系统可以生成越来越接近于物理上正确的图像。因此，在第 13 章中还要回到这些方法上来。

1.4　成像系统

现在介绍两种物理成像系统：针孔照相机和人类视觉系统。针孔照相机是成像系统的一个简单例子，它可以帮助读者理解照相机和其他光学成像仪器的工作原理。通过模仿针孔照相机，可以得到一个图像生成的模型。人类视觉系统极其复杂，但是它和其他光学成像设备遵循相同的物理原理。本书介绍它不仅是作为成像系统的一个例子，也是因为理解它的性质会帮助读者更好地利用计算机图形系统所提供的功能。

1.4.1　针孔照相机

如图1.12所示的针孔照相机为图像生成提供了一个例子，可以借助简单的几何模型来理解它。针孔照相机（pinhole camera）是一个盒子，盒子一面的中心有一个小孔，胶片放在盒子内和小孔

相对的一面。小孔在初始时被遮挡，在照相时暴露一小段时间以使胶片曝光。假定把针孔照相机沿着z轴摆放，小孔位于原点。假定小孔非常小，以至于从一点出发只能有一条光线进入小孔。胶片平面与针孔的距离为d。图1.13所示是针孔照相机的侧视图，由该图可以计算出点(x, y, z)在胶片平面$z = -d$上所成像的位置。图中的两个三角形相似，由此可得像点的y坐标y_p为：

$$y_p = -\frac{y}{z/d}$$

同样，利用俯视图，可得：

$$x_p = -\frac{x}{z/d}$$

点$(x_p, y_p, -d)$称为点(x, y, z)的投影（projection）。注意，在(x, y, z)和$(x_p, y_p, -d)$的连线上的所有点都投影到$(x_p, y_p, -d)$，所以不能从成像平面上的一点按相反的方向找出生成该投影点的初始点。在这个理想化的模型中，胶片平面上点$(x_p, y_p, -d)$的颜色就是点(x, y, z)的颜色。照相机的视域（field of view）或者视角（angle of view）是能够在胶片平面上成完整像的最大物体所张的角度。根据图1.14可以计算出视域[①]。如果h是照相机的高度，那么视角θ为：

$$\theta = 2\tan^{-1}\frac{h}{2d}$$

理想的针孔照相机具有无穷大的景深（depth of field）：视域范围内的所有点不论与照相机的远近如何，都会被清晰成像，一个点所成的像还是一个点。针孔照相机有两个缺点：第一，因为孔如此之小，一个点光源只能有一条光线通过小孔，所以几乎没有光线进入照相机；第二，照相机的视角不能调节。

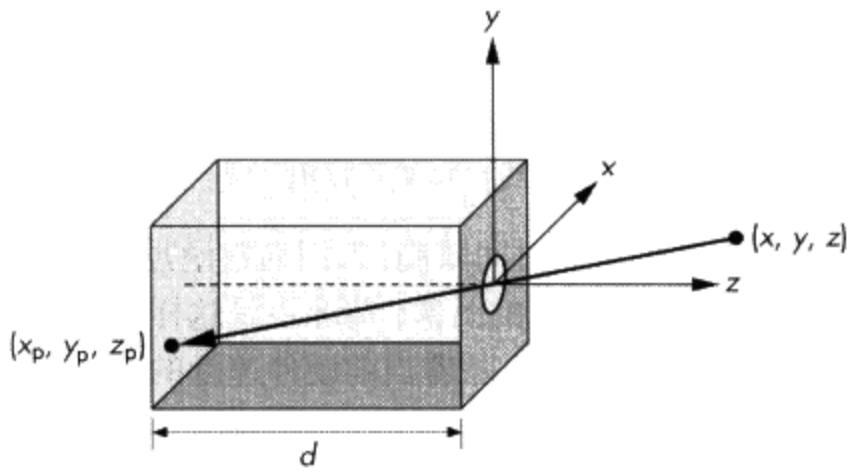


图1.12　针孔照相机

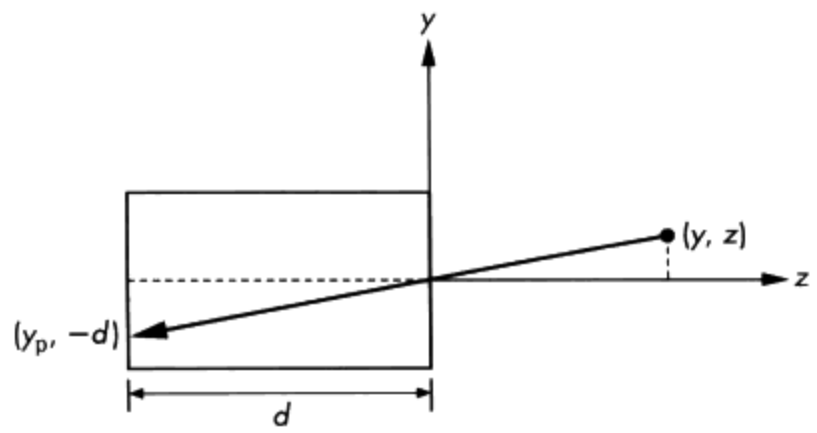


图1.13　针孔照相机的侧视图

① 如果是在三维空间而不是二维平面考虑问题，那么应该用胶片的对角线长度替换h。

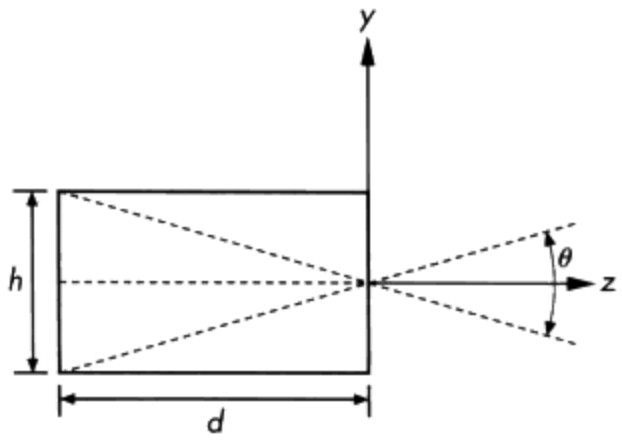


图 1.14　视角

针孔照相机的这两个缺点自然引出了更复杂的照相机和其他具有透镜的成像系统。通过用透镜替换小孔就可以克服上述两个缺点。首先，透镜收集的光线比小孔所能通过的光线要多。透镜的孔径越大，所能收集的光线就越多。其次，通过选择具有合适焦距的透镜（焦距和针孔照相机中的 d 等价）可以获得任何想要的视角（直到 180°）。然而，实际的透镜并不具有无限大的景深：不是所有透镜前的物体都可以清晰成像的。

本章主要考虑针孔照相机，并把从照相机的前面到胶片平面的距离 d 当做它的焦距。和针孔照相机类似，在计算机图形学生成的图像里，所有的物体都被清晰成像。

1.4.2　人类视觉系统

视觉系统极其复杂，像照相机或者显微镜这样的物理成像系统所具有的一切部件在人类视觉系统中都能找到相对应的部分。人类视觉系统的主要组成部分示于图 1.15。光线通过角膜和晶状体进入眼睛。角膜是一种透明结构，对眼睛有保护作用。通过虹膜的收缩和舒张可以控制进入眼睛光线的强弱。晶状体把像成在眼睛后部一个称为视网膜（retina）的二维结构上。分布在视网膜上的视杆细胞和视锥细胞（因其放大后的外观而得名）是光感受器，它们对 350 ~ 780 nm 范围内的电磁波敏感。

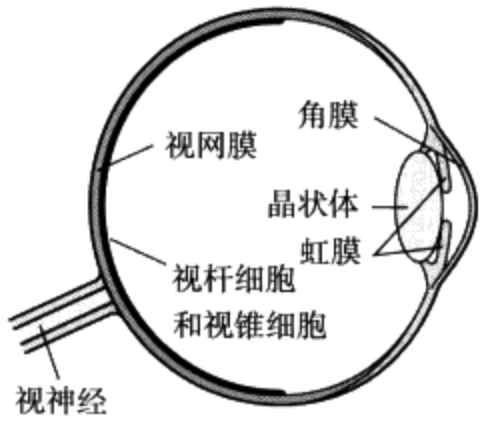


图 1.15　人类视觉系统

视杆细胞是低亮度光线的感受器，对颜色不敏感，人们在夜间看东西时就是视杆细胞在起作用。视锥细胞负责颜色视觉。视杆细胞和视锥细胞的大小，以及晶状体和角膜的光学性质，决定了视觉系统的分辨率（resolution）或者视觉灵敏度（visual acuity）。分辨率是对人们所能看到物体的尺寸的度量。说得技术性更强一些，分辨率是这样两个点之间的距离，如果这两个点相距再近一些的话就不能分辨出是两个点了。

人眼中的光感受器对不同波长的光线能量的响应是不同的。视锥细胞有三种，视杆细胞只有一种。强度是对光线能量的物理度量，而亮度（brightness）度量人们所感受到的从物体辐射出的光线

有多强。人类视觉系统对单一频率红光和单一频率绿光的响应不相同。即使红光和绿光的能量相同，在人们看来它们也并不具有相同的亮度，因为视锥细胞对红光和绿光的敏感程度不同。人们对绿光最敏感，对红光和蓝光最不敏感。

亮度度量的是对光线强度总的感受。人类能够区分颜色是由于三种视锥细胞具有不同的光谱敏感度。正因为具有三种视锥细胞，所以可以使用三种标准原色来近似人类能够感知的所有颜色，而不需要单独处理所有可见光的波长。因此，大多数图像生成系统，包括电影和视频，只使用三种基色或者原色（primary color）。本书将在第2章深入讨论颜色。

在人类视觉系统中，对光线的初级处理所利用的原理和大多数光学系统相同。然而，人类视觉系统的后端处理比照相机或者望远镜要复杂得多。视神经与视杆细胞和视锥细胞的连接方式非常复杂，具有一个复杂的信号处理器的许多特征。最后的处理是在大脑中一个称为视皮层的部分进行的，这里执行像对象识别这类的高级功能。我们只考虑视网膜上所成的图像，它由视杆细胞和视锥细胞感知并传递到大脑，而不讨论任何高级处理。

1.5　虚拟照相机模型

从前面介绍的光学成像系统的模型可以自然地引出现代三维计算机图形学的概念基础。我们认为，利用计算机生成图像和在光学系统中成像是相似的，这一模式称为虚拟照相机模型（synthetic-camera model）。考虑图1.16所示的成像系统，图中仍旧包含对象和一个观察者，这里的观察者是一个折叠暗箱照相机[①]。图像是在照相机后部的胶片平面上生成的。为了能够模仿这个过程以生成人造图像，需要明确一些基本原理。

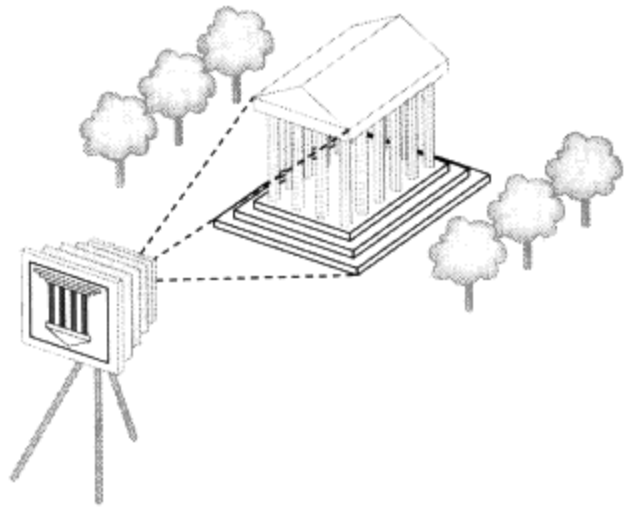


图1.16　成像系统

第一，对象的确定不依赖于观察者。因此，在一个图形库中，可以期望有不同的函数来分别指定对象和观察者。

第二，可以像对针孔照相机所做的那样，利用简单的几何方法计算图像。考虑如图1.17所示的包含观察者和简单对象的侧视图。图1.17(a)所示的视图和针孔照相机的情形类似。注意物体的像相对于物体是倒立的。在实际的照相机中，可以通过翻转胶片来获得正立的像，但对于虚拟照相机，可以使用一个简单的技巧而无须翻转。在透镜前画另一个平面（见图1.17(b)），三维的情景示于图1.18。为了找出对象上的一点在虚拟成像平面上所成的像，从该点到透镜的中心连一条线，这

① 在折叠暗箱照相机中，前面和后面之间的侧面是可变动的。照相机的前面装有透镜，后面是胶片平面。这样，可以移动照相机的胶片平面而不影响前面的透镜，于是在图像生成过程中增加了灵活性。在第5章中会利用这种灵活性。

条线称为投影线（projector），透镜的中心称为投影中心（center of projection，COP）。注意，所有的投影线都经过投影中心。在这个虚拟照相机中，移到透镜前面来的虚拟成像平面称为投影平面（projection plane）。对象上一点的像位于投影线和投影平面的交点。第 5 章将详细讨论这个过程并导出相关的数学公式。

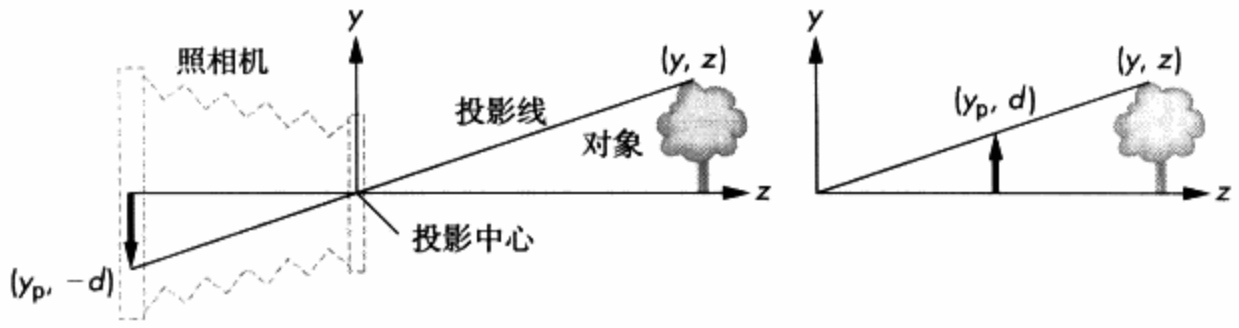


图 1.17　图像生成的两个等价视图：(a) 在照相机的后部成像；(b) 把成像平面移到照相机前面

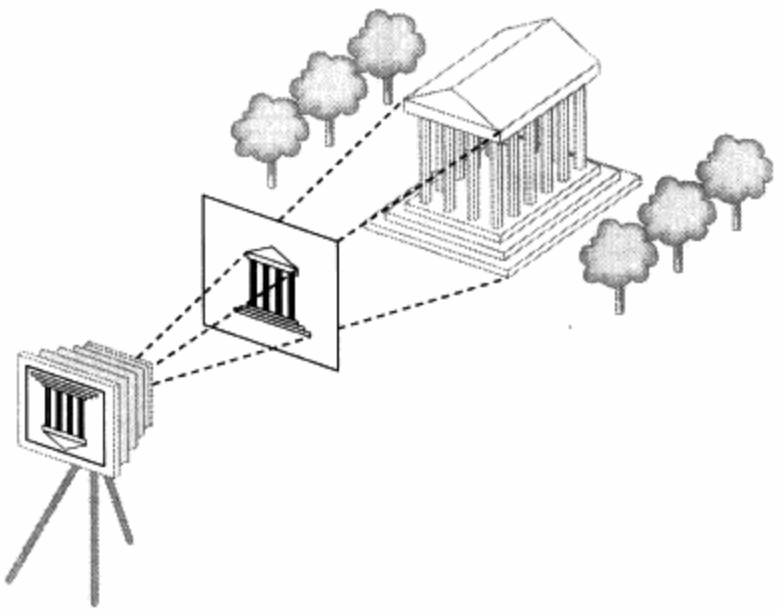


图 1.18　利用虚拟照相机成像

还必须考虑到图像的大小是受限的。前面已经讲到，不是所有物体都可以在针孔照相机的胶片平面上完整地成像，视角就是用来描述这个限制的。如图 1.19 所示，在虚拟照相机中，可以通过在投影平面内设置一个裁剪矩形（clipping rectangle）或者裁剪窗口（clipping window）来把这个限制移到透镜的前面。图中的矩形起窗口的作用，位于投影中心的观察者透过这个窗口观察场景。给定投影中心的位置、投影平面的位置和方向以及裁剪窗口的大小，就能够确定哪些对象会在图像中出现。

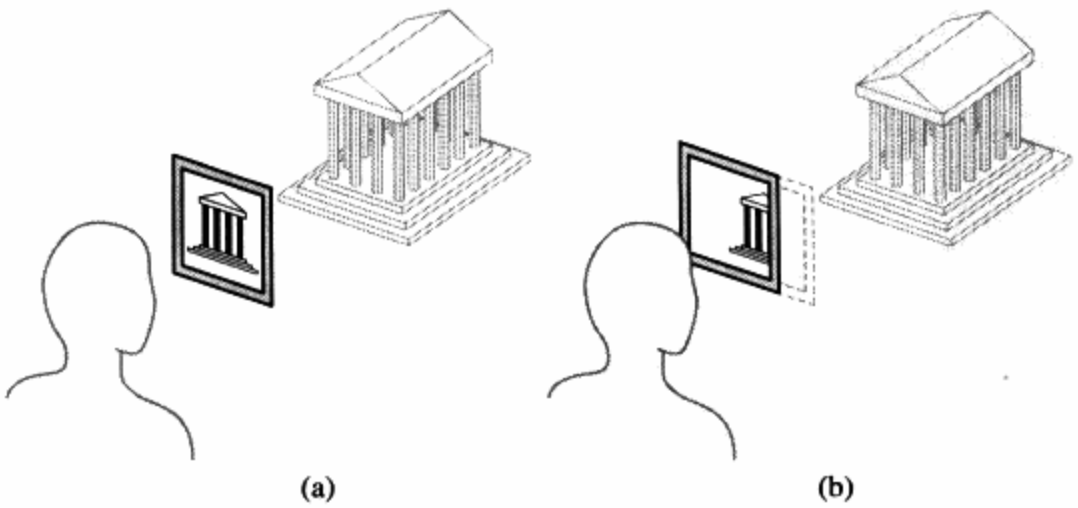


图 1.19　裁剪：(a) 裁剪窗口位于初始位置；(b) 裁剪窗口被移动之后

1.6　程序员接口

用户和图形系统交互的方式有很多种。利用功能完备的软件包，比如在CAD中所使用的软件包，用户在生成图像时可以使用像鼠标和键盘这样的输入设备和显示器进行交互。在图1.20所示的画图程序这样的典型应用中，用户可以看到代表可能的操作的菜单和图标。通过点击这些菜单和图标，软件在用户的引导下就可以生成图像，用户是不需要编程的。

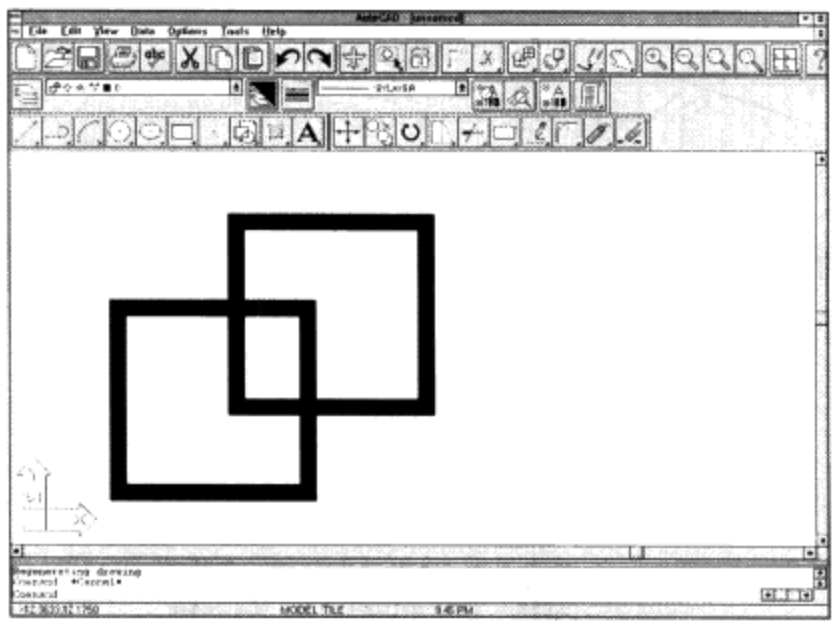


图 1.20　一个画图程序的界面

当然，得有人为这些应用程序编写代码，而且即使商用软件已经很复杂并且功能相当强大，很多人也仍然需要编写自己的图形应用程序（甚至是享受这样做的乐趣）。

应用程序和图形系统之间的接口可以通过图形库中的一组函数来指定，这个接口的规范称为应用程序编程人员接口（application programmer's interface，API）。从应用程序编程人员的角度来看，图形系统的模型如图1.21所示。应用程序编程人员只能看到API，因此图形系统的软/硬件实现细节都被屏蔽掉了。软件驱动程序（driver）负责解释API的输出并把这些数据转换为能被特定硬件识别的形式。从应用程序编写者的角度来看，API提供的功能应该同程序员用来确定图像的概念模型相匹配[①]。

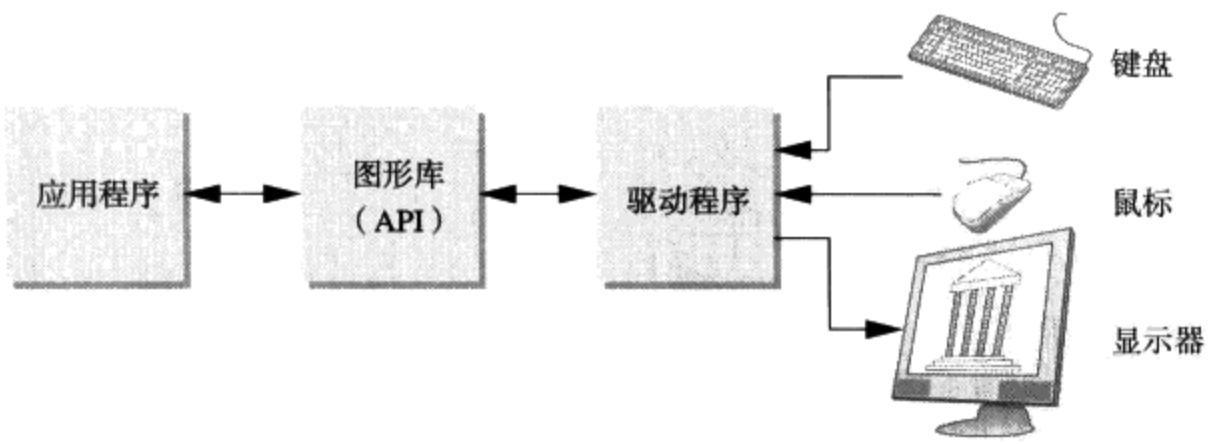


图 1.21　图形系统的应用程序编程人员模型

① 在API和驱动程序之间可能还有一个或者多个层，比如虚拟机或者硬件抽象层。不过由于应用程序编程人员只能看到API，所以不需要了解这些细节。

1.6.1　笔式绘图仪模型

历史上早期的大多数图形系统都是二维系统，它们所使用的概念模型现在被称为笔式绘图仪模型，这个名称与那个时候可用的输出设备有关。如图 1.22 所示，笔式绘图仪（pen plotter）通过安装在桥形台架上的绘图笔的移动来生成图像，这里的桥形台架是一种可以在纸张的范围内沿两个互相垂直方向移动绘图笔的结构。绘图仪可以根据需要提起和放下绘图笔以绘制想要的图形。笔式绘图仪现在仍在使用，它们很适合于绘制较大的图形，比如设计蓝图。许多API，比如LOGO和PostScript，都起源于笔式绘图仪模型。尽管存在其他差异，这些API都把图像生成的过程看做是类似于用笔在纸上绘图。用户考虑的是一定尺寸的纸面，在这个二维的纸面上来回移动画笔，从而把图画在纸上。

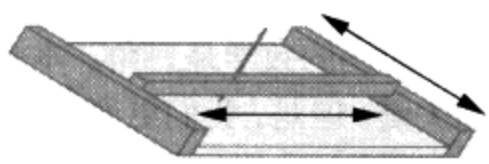


图 1.22　笔式绘图仪

可以用下面的绘图函数来描述这类图形系统：

```
moveto(x,y)
lineto(x,y)
```

函数 moveto 把画笔移动到纸面的位置(x, y)，但不留下画痕。函数 lineto 把画笔移动到(x, y)，并在画笔的旧位置和新位置之间画一条线。只需要再添加一些初始化和终止操作，还有改变画笔颜色或者线条宽度的功能，就得到了一个简单却完备的图形系统。下面是这个图形系统中的一个简单程序的片断：

```
moveto(0, 0);
lineto(1, 0);
lineto(1, 1);
lineto(0, 1);
lineto(0,0);
```

这个代码段生成的输出示于图 1.23(a)。如果再增加下面的代码：

```
moveto(0, 1);
lineto(0.5, 1.866);
lineto(1.5, 1.866);
lineto(1.5, 0.866);
lineto(1, 0);
moveto(1, 1);
lineto(1,5,1.866);
```

就得到了图 1.23(b)所示的立方体的斜投影。

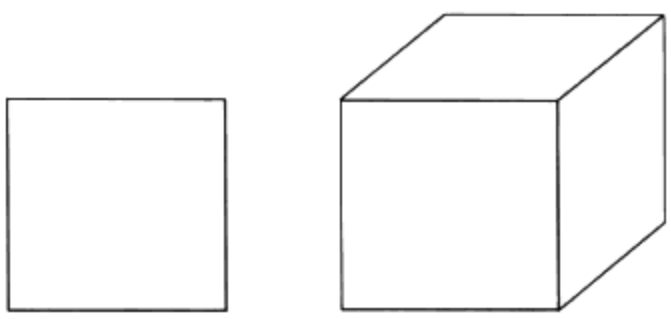


图 1.23　笔式绘图仪程序的输出：(a) 正方形；(b) 立方体的投影

对于某些应用，比如印刷行业的版面设计，这样的系统工作得很好。例如，页面描述语言PostScript在这些想法的基础上进行了复杂的扩展，现已成为排版系统和打印机的标准。

还有一种基于光栅的二维模型，它是把像素直接写到帧缓存中去，不过仍然具有局限性。这样的系统可以建立在下面单独一个函数的基础上：

```
write_pixel(x, y, color)
```

这里，x，y是帧缓存中像素的位置，color是在那个位置的像素的颜色。这样的系统很适合编写光栅化和处理数字图像的算法。

不过人们还是对三维世界更感兴趣。笔式绘图仪模型不能很好地扩展到三维图形系统。比如，要想利用笔式绘图仪模型在二维纸面上生成三维物体的图像，不管用手画还是通过计算机，都得考虑三维物体上的点要表示在二维纸面上的什么位置。像在1.5节讲到的那样，这些二维平面上的点是三维空间中点的投影。确定投影需要用到的数学是三角。本书将在第5章讨论投影的数学推导。理解投影是理解三维计算机图形学的关键。但是，我们更愿意通过一个API来允许用户直接在他们的问题领域中考虑问题，这样用户就不需要在应用程序中进行三角计算了，至于投影过程的细节则留给计算机来完成。这种方法不需要用户学会如何在绘图板上绘制各种投影或者绘制物体的透视草图，因而是很有好处的。更重要的是，API的实现中通过软/硬件来实现投影，这比用户在自己的程序中实现投影的效率要高得多。

1.6.2　三维API

虚拟照相机模型是所有流行API的基础，这些API包括OpenGL、Direct3D和Open Scene Graph。如果遵从虚拟照相机模型，那么在API中就需要有相应的函数来确定下列4个要素：

- 对象
- 观察者[①]
- 光源
- 材质属性

对象通常由一组顶点定义。对于简单的几何对象，比如线段、长方形和多边形，在一个顶点（vertex）列表，或者说一组空间位置与这些对象之间存在简单的关系。对于更复杂的对象，从一组顶点可能有许多方式来确定对象。例如，一个圆可以通过圆周上的三个点来确定，也可以通过圆心和圆周上的一个点来确定。

大多数API为用户提供的图元对象集是相似的。这些图元通常是那些可以在硬件上快速显示的对象。一般的图元集包括点、线段、多边形，有时候也包括文本。OpenGL程序通过顶点列表来定义图元。下面的OpenGL代码段通过5次函数调用来指定图1.24所示的三角形：

```
glBegin(GL_POLYGON);
    glVertex3f(0.0, 0.0, 0.0); /* 顶点 A */
    glVertex3f(0.0, 1.0, 0.0); /* 顶点 B */
    glVertex3f(0.0, 0.0, 1.0); /* 顶点 C */
glEnd( );
```

函数glBegin指定由顶点所定义的图元类型。接下来的每一个glVertex3f函数指定空间中一个位置的(x, y, z)坐标。函数glEnd结束顶点列表。注意，可以通过添加更多的顶点来定义任意多边形。如果改变类型参数GL_POLYGON，则可以用相同的顶点列表定义不同的几何图元。例如，类型

① 译文中有时也称为视点或虚拟照相机。——译者注

GL_LINE_STRIP 把这个顶点列表定义成两条相连的线段，而类型 GL_POINTS 则把相同的顶点列表定义成三个点。

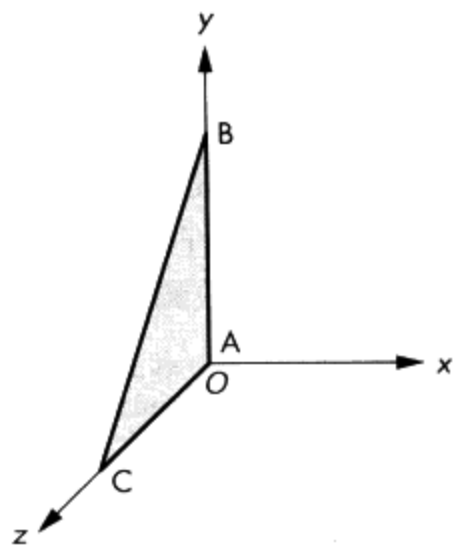


图 1.24　一个三角形

有一些API提供读像素和写像素的函数，从而允许用户直接访问帧缓存。还有一些API把曲线和曲面作为图元提供给用户，不过这些图元一般是在应用程序中通过一系列更简单的图元来逼近的。OpenGL 允许用户访问帧缓存，并把曲线和曲面作为图元提供给用户。

可以通过许多方式来指定观察者或者照相机。现有的各种API为用户选择照相机所提供的灵活性各有不同，为用户提供的指定照相机的方法也有多有少。通过观察图 1.25，可以发现为了把照相机确定下来，必须指定下列 4 个要素：

1. **位置**。照相机的位置通常由透镜的中心，即投影中心（COP）给出。
2. **方向**。在确定了照相机的位置后，就可以以投影中心为原点设置一个照相机坐标系[①]，接下来可以分别绕着这个坐标系的三个坐标轴旋转照相机。
3. **焦距**。透镜的焦距决定了在胶片平面上成像的尺寸，或者等价地说，决定了场景中能够被照相机成像的那部分有多大。
4. **胶片平面**。照相机的后部有一定的高度和宽度。在折叠暗箱照相机和一些API中，照相机后部胶片平面的方向与透镜的方向无关，可以独立调节。

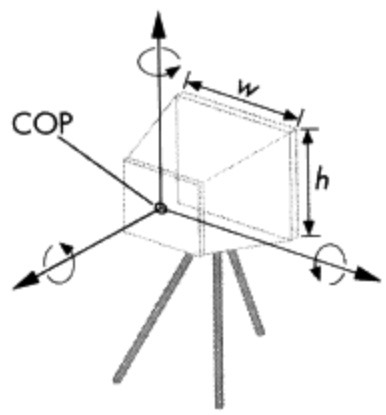


图 1.25　照相机的确定

上面这 4 个要素可以通过许多方法给予指定。一种方法是借助一系列坐标系变换来确定照相机的位置和方向。对象的位置起先在某个坐标系中通过对象的顶点坐标来指定，这些变换把对象的位置变换到另一个以COP为中心的坐标系中。不论是对于某个特定图形绘制系统的API实现，还

① 译文中有时也称为观察坐标系或眼坐标系。——译者注

是对于利用照相机获取各种可能的视图，这种方法都是有用的。从第5章开始，会广泛地使用这种方法。

然而，如果需要调整的参数太多，那么得到一幅想要的图像就变得困难了。造成这个问题的部分原因在于虚拟照相机模型本身。一些经典的观察技术（比如在建筑设计中使用的观察技术）强调的是观察者和对象之间的联系，而不是虚拟照相机模型所强调的二者各自的独立性。按照这种观点，图1.26所示的立方体的透视图之所以是两点透视，是因为观察者与立方体的各个面之间存在一种特定的位置关系（见习题1.7）。OpenGL API不但允许人们完全自由地设置坐标系的变换，而且还提供了附加的有用函数。作为例子，考虑下面的函数调用：

```
gluLookAt(cop_x, cop_y, cop_z, at_x, at_y, at_z, up_x, up_y, up_z);
glPerspective(field_of_view, aspect_ratio, near, far);
```

第一个函数调用使照相机从投影中心指向所期望的位置（点at），并且指定照相机的观察正向为up。第二个函数为透视投影选择了透镜（视域），还确定了场景中的哪一部分要被成像（宽高比与near和far这两个距离）。然而，基于虚拟照相机模型的API没有提供能够直接按要求确定照相机和观察者之间关系的函数。

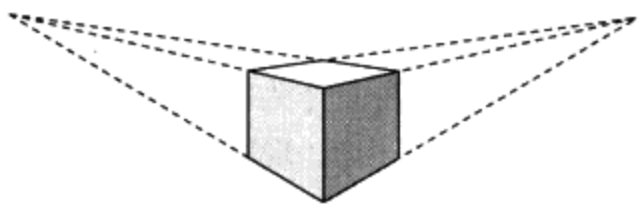


图 1.26　立方体的两点透视图

光源可由其位置、强度、颜色和方向性来定义。在API中有一组函数可以为每个光源指定这些参数。材质属性是对象的属性，这些属性在定义每一个对象时由一系列函数来指定。API允许用户设置的光源和材质属性都取决于它所支持的光和材质的相互作用模型，本书将在第6章讨论这些模型。

1.6.3　基于 OpenGL API 绘制的系列图像

本书从第2章开始详细讨论OpenGL API，对OpenGL的使用将贯穿全书。OpenGL程序所定义的图像是由图像生成模型的软/硬件实现自动生成的。

这里展示一系列图像，让读者看看利用OpenGL API可以得到什么样的效果。这一系列图像是用越来越复杂的技术绘制同样的对象得到的图像。它们的顺序不仅大致上和后面讨论各个相关主题的顺序一致，而且也反映了图形系统在过去30年里的发展历程。

彩图5所示的是艺术家创作的一个类似太阳的模型，彩图6是只利用线段绘制出的这个模型的图像。虽然这个模型包括许多部分，程序员也已经使用了复杂的数据结构来表示每个部分以及各个部分之间的联系，但是在绘制的图像中只显示出了各个部分的轮廓。这类图像称为线框（wireframe）图，因为只能看到表面的边线。如果对象由直线段构成的框架组成，在边线之间没有实体部分，那么就得到了线框图。在光栅图形系统出现之前，利用计算机只能够得到线框图。

彩图7所示的是对同样的对象利用多边形平面片绘制的图像。有一些表面是看不到的，因为在观察者和这些表面之间存在实体表面，这些看不到的表面已经通过隐藏面消除（hidden-surface-removal，HSR）技术去掉了。对于大多数光栅系统来说，用单一颜色填充多边形内部所需的时间和显示线框图所需的时间差不多。尽管对象是三维的，它的每个表面却被显示为单一颜色，这样就不能体现出对象的三维形状。早期的光栅系统可以生成这样的图像。

第2章和第3章将讨论如何生成包含简单几何对象的图像，这些几何对象包括点、线段和多边形。第4章和第5章将介绍如何对对象进行三维变换，以及如何获得想要的三维视图，并去掉隐藏的表面。

彩图8利用具有明暗渐变的多边形来近似该模型，它显示出了模型的三维效果，而且模型的表面是光滑的。第6章将介绍OpenGL支持的明暗模型。最近的工作站提供了对这些明暗模型的硬件支持，在这些系统上生成明暗图和生成线框图花费的时间差不多。

彩图9所示的更复杂的线框模型是利用NURBS曲面构造的。第12章将介绍NURBS曲面。这样的曲面为应用程序编程人员在设计过程中提供了很大的灵活性，不过最终还是要借助线段和多边形来绘制。

我们在彩图10和彩图11中为该模型增加了纹理。纹理是第8章要讨论的效果之一。近来的图形处理器都在硬件上支持纹理映射，所以绘制基于纹理映射的图像几乎不需要额外的时间开销。在彩图10中使用了一种称为凹凸映射的技术，虽然绘制的仍是和其他例子中一样的平面多边形，但利用这种技术获得了粗糙表面的效果。彩图11给出了把环境贴图技术应用到对象表面的效果，这使表面看上去像镜面。这些技术将在第8章和第9章中详细讨论。

彩图12所示是利用环境贴图技术绘制的对象的一小块区域。左边的图像显示出了锯齿状的称为走样误差的假象，这是由于帧缓存的离散本性造成的。右边的图像是利用了平滑或者反走样方法绘制的效果，第7章和第8章将介绍这些方法。

这些图像不仅展示了利用当今的硬件技术和API能够达到的效果，而且像下面几章里将要看到的那样，生成这些图像也并不复杂。此外，就像这些图像所显示出来的递进的绘制效果，生成它们的程序也相应地逐渐增加新的部分。

1.6.4　建模 – 渲染模式

在许多场合，尤其是CAD应用和电影中一些复杂图像的生成，可以把场景的建模和图像的绘制［或者称为场景的渲染（rendering）］分离开，这样就可以把图像生成看做是如图1.27所示的两步过程。虽然要完成的任务和前面讨论的一样，但是方框图的表示形式启发我们可以用不同的软件和硬件实现建模器和渲染器。例如，考虑生成动画序列中的单独一帧图像。首先需要设计并摆放对象，这一步是高度交互的，并不需要处理对象图像的细节。因此，我们更希望在一个具有高性能图形硬件的交互式工作站上执行这一步。一旦完成了场景的设计，就要渲染场景了。为了生成高质量的图像，需要增加光源、材质属性，以及许多其他的细节效果。这一步需要大量的计算，所以更希望利用高性能计算集群（high performance cluster 或 render farm）来完成。建模步骤和渲染步骤所需要的最优硬件是不同的，而且所使用的软件也可能是不同的。

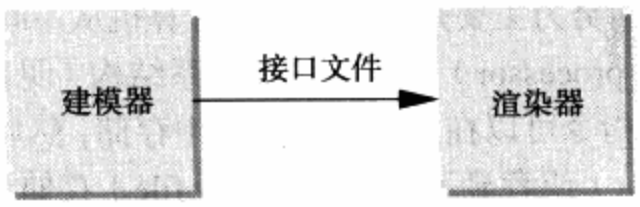


图 1.27　建模 – 渲染流水线

建模器和渲染器之间的接口可以简单到就是一个文件，这个文件由建模器生成，描述要渲染的对象并且包含只对渲染器来说是重要的附加信息，例如光源、观察者的位置和材质属性。Pixar公

司的RenderMan接口就采用了这种方法，定义了一种文件格式，允许建模器以文本形式把模型信息传递给渲染器。这种方法的另一个优点是可以允许用户针对特殊的应用开发其他建模器，这些建模器可以使用和原来一样的渲染器。同样，不同的渲染器也可以把相同的接口文件作为输入，甚至可以完全抛开建模器，使用一个标准的文本编辑器来生成接口文件，这至少在原则上是可能的。不过，除了最简单的场景以外，用户不能直接编辑渲染器所需信息的列表，他们使用的是交互式的建模软件。为了和建模器交互，至少需要有对象的简单图像，大多数建模器利用虚拟照相机模型来实时地生成这些图像。

建模－渲染模式已经成为网络游戏和在网络环境下生成图像所采用的主流方法。场景的模型，包括几何对象、光照、照相机和材质属性都被放到一个称为场景图（scene graph）的数据结构中，场景图再被传递给渲染器或者游戏引擎。本书会在第10章讨论场景图。建模－渲染模式也是动画制作的标准方法。用户借助像Maya和Lightwave这样的交互式软件利用线框或者不加光照的多边形来构造动画角色。在最后的渲染阶段，每帧花费的时间可能要以小时计。

1.7　图形绘制系统的体系结构

API的上层是应用程序，下层则是组合起来实现API功能的硬件和软件。研究人员已经采取了许多方法来开发支持图形API的体系结构。

早期的图形系统采用具有标准von Neumann体系结构的通用计算机。这类计算机的特征是具有单一的处理单元，每次只处理单独一条指令。图1.28是这类早期图形系统的一个简单模型。在这类系统中使用的显示器是画线CRT，包含必要的电路来显示连接两点的线段。主计算机的任务是运行应用程序并计算图像中线段的端点坐标（点坐标的单位遵从显示器的物理单位）。为了避免出现闪烁，线段的端点坐标必须以足够高的速率传递到显示器。在计算机图形学的早期，计算机的运算速度还很慢，就连刷新只包含几百条线段的简单图像这样的任务，对于那时昂贵的计算机来说都是个沉重的负担。

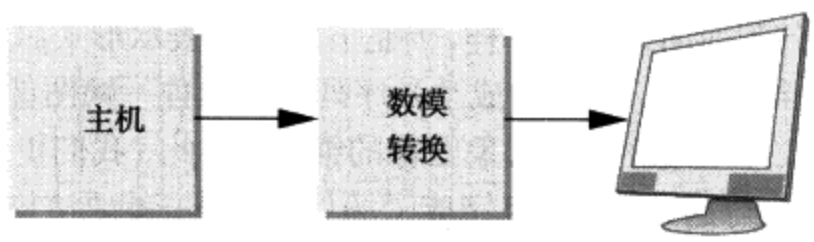


图1.28　早期的图形系统

1.7.1　显示处理器

最初，制造专用图形系统的努力主要是为了把通用计算机从不间断刷新显示器的任务中解放出来。这些显示处理器（display processor）具有传统的体系结构（见图1.29），但增加了在显示器上显示图元的指令。生成图像的指令可以在主计算机中集中存储，然后发送到显示处理器，这些指令可以作为显示列表（display list）或者显示文件（display file）存储到显示处理器自己的存储器中，这是显示处理器的主要优势。显示处理器会以足够的速率来重复地执行显示列表中的程序，从而避免闪烁，这样主计算机就可以执行其他任务了。这种体系结构和将在第3章讨论的客户/服务器体系结构有密切的联系。

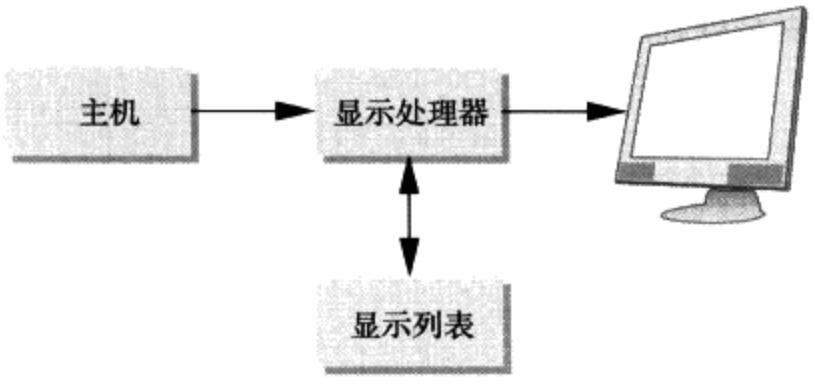


图 1.29　显示处理器体系结构

1.7.2　流水线体系结构

图形体系结构的主要进展可以和工作站的进展并驾齐驱。专用VLSI芯片对于二者来说都是关键的支撑技术。此外，廉价的固态存储器使光栅显示器得到普及。对计算机图形学的应用来说，定制 VLSI 电路最重要的应用是构建流水线（pipeline）体系结构。

流水线类似于汽车工厂里的装配线。当汽车底盘在装配线上传送的时候，要执行一系列的操作，每个操作都有专门的工具和工人，直到最后装配过程完成。在任何一个时刻，装配线上都有许多汽车在组装。从汽车底盘放到装配线上开始到汽车组装完毕之间有显著的延迟（delay 或 latency）。然而，在给定时间里生产出的汽车数量，即吞吐量（throughput），比由单独一组工人完成所有操作要高得多。

图 1.30 以一个简单的算术运算为例，解释了流水线的概念。在流水线中有一个加法器和一个乘法器。如果用这种配置来计算 $a + (b \times c)$，那么需要一次乘法和一次加法，这和只用一个处理器来执行这两个运算所需的计算量相同。然而考虑下面这种情况：同样的计算需要对不同的 a、b、c 执行许多次。现在，乘法器可以把它的运算结果传递给加法器，然后在加法器执行第一组数据的第二步计算的同时进行下一次乘法。这样，尽管对任何一组数据而言所需的计算时间仍然相同，可当一次处理两组数据时，总的计算时间被显著缩短。这里数据通过系统的速率（即系统的吞吐量）被加倍了。注意，当流水线中的模块数目增加以后，系统的延迟也增加了，所以在评估流水线的性能时必须对延迟和吞吐量进行权衡。

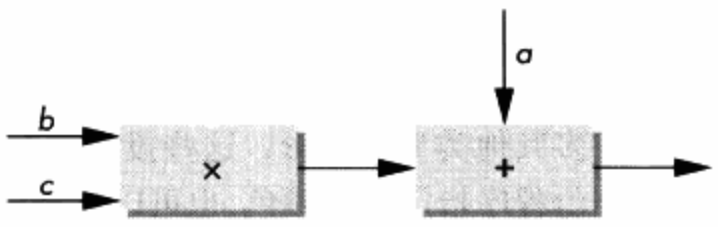


图 1.30　算术流水线

我们可以对更复杂的算术运算构造流水线，这样吞吐量会增加得更多。当然，除非要对许多数据集进行相同的操作，否则没必要构造流水线。不过在计算机图形学中，这正是我们要做的，因为图形学需要以相同的方式处理数量巨大的顶点。

1.7.3　图形绘制流水线

场景中包含一组对象，每个对象由一组图元组成，每个图元又包含一组顶点。可以认为是图元的类型和顶点集定义了场景的几何属性（geometry）。在一个复杂的场景里，可能会使用数千个甚至数百万个顶点来定义对象。为了生成存储在帧缓存中的图像，必须以相似的方式来处理所有这些

顶点。如果把生成图像看做是处理对象的几何属性，则可以用图1.31所示的框图来表示成像过程的4个主要步骤：

1. 顶点处理
2. 裁剪和图元组装
3. 光栅化
4. 片元处理

我们将在后面几章讨论这些步骤的细节，这里只是扼要介绍每个步骤，然后说明可以把这些步骤组织成流水线。

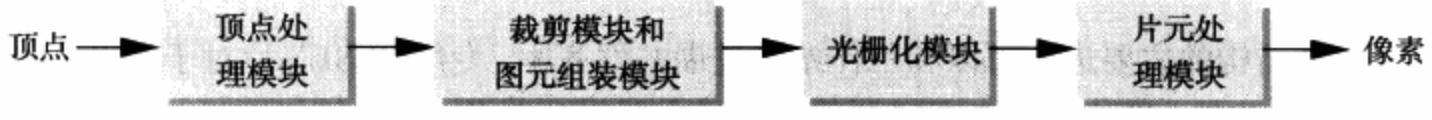


图 1.31　几何绘制流水线

1.7.4　顶点处理

在绘制流水线的第一个模块中，对各个顶点的处理是彼此独立的。这个模块的两个主要功能是执行坐标变换和计算顶点的颜色值。

在成像过程中，有许多步骤可以看做是对象在不同坐标系下的表示之间的变换。比如，在虚拟照相机成像模式里，观察的一个主要部分是把对象从其被定义的坐标系下的表示转换到照相机坐标系（或观察坐标系）下的表示。最后把图像传送到输出设备时也会涉及变换。对象的内部表示，不管是在观察坐标系下的还是在图形软件使用的其他坐标系下的，最终必须转换成在显示器坐标系下的表示。坐标系的每一次变换都可以用一个矩阵来表示。坐标系的多次变换可以表示为矩阵的相乘或者级联（concatenation），于是多个矩阵通过相乘就合并为一个矩阵。我们将在第4章详细讨论这些运算。因为用一个矩阵乘以另一个矩阵得到第三个矩阵，一个变换序列很明显是非常适合于用流水线体系结构实现的。此外，由于计算机图形学中使用的矩阵总是小尺寸的（4 × 4），因此还可以在流水线的变换模块里进行并行计算。

在经过多次变换之后，最后还要进行一个投影变换。第5章会讲到这一步可以用4 × 4的矩阵实现，这样投影变换也可以被纳入到流水线中来。一般而言，当对象在流水线中被处理时，希望尽可能久地保留三维信息，因此，绘制流水线中的投影变换多少要比1.5节所讲的投影更一般。除了保持三维信息以外，还可以实现许多其他类型的投影，这些投影将在第5章介绍。

对顶点颜色的指派可以简单到由程序指定一种颜色，也可以复杂到利用基于物理的真实感光照模型来计算，这样的光照模型考虑了对象的表面属性和场景中的特定光源。第6章将讨论光照模型。

1.7.5　裁剪和图元组装

标准图形绘制流水线中第二个基本模块的任务是裁剪和组装图元。因为没有成像系统能够一次对整个场景成像，所以我们必须进行裁剪。人类视网膜的尺寸是有限的，大约对应于90°的视域。照相机的胶片也是大小有限的，其视域可以通过选择不同的透镜来调节。

虚拟照相机的视域也是有限的，这是通过裁剪体（clipping volume）来体现的。图1.18中在透镜前面的锥体就是裁剪体的一个例子，在这个裁剪体里面的对象的投影可以在图像里出现，而在裁剪体外面的对象则不被成像，我们说它们被裁剪掉了。跨越裁剪体边界的对象在图像中是部分可见的。第7章将讨论高效的裁剪算法。

裁剪必须针对逐个图元进行，而不能针对逐个顶点进行。这样，在绘制流水线的这个阶段，在裁剪之前，必须把顶点组装成像线段和多边形这样的图元。所以，这个阶段的输出是一些其投影可以被成像的图元。

1.7.6　光栅化

由裁剪模块得到的图元仍然是用顶点表示的，为了生成帧缓存中的像素还必须做进一步处理。例如，如果三个顶点确定了一个由单色填充的三角形，光栅化模块就必须确定在帧缓存中有哪些像素位于这个三角形的内部。第 8 章将讨论线段和多边形的光栅化（扫描转换）过程。光栅化模块对每个图元输出一组片元（fragment）。可以把片元看做是携带自己信息的潜在像素，片元所携带的信息包括它的颜色和位置。我们用片元来更新帧缓存中的对应像素。片元还可以携带深度信息，这样在绘制流水线后面的阶段就可以确定某个片元是否位于其他片元的后面，这些片元都对应于同一个像素。

1.7.7　片元处理

绘制流水线的最后一个模块利用光栅化模块生成的片元来更新帧缓存中的像素。如果应用程序生成的是三维数据，那么一些片元可能是不可见的，因为它们所定义的表面在其他表面的后面。如彩图 10 和彩图 11 所示，片元的颜色可以通过纹理映射或者凹凸映射来改变。从帧缓存读取片元所对应的像素的颜色，再与片元的颜色融合，就可以生成半透明效果。第 8 章和第 9 章将讨论这些效果。

1.8　可编程流水线

图形绘制系统体系结构的发展经历了多次循环反复，专用硬件相对于标准 CPU 的重要性起伏不定。然而，流水线体系结构的重要性保持不变。其他的方法，包括光线跟踪、辐射度方法和光子映射都不能获得实时效果。因此，市场上占主流的图形卡都把绘制流水线内置到图形处理单元中。这些商用图形卡都实现了前面所讲的图形绘制流水线，但增加了更多的可选功能，后面的几章里将讨论许多这些可选功能。

很多年以来，这些流水线体系结构都具有固定的功能。尽管应用程序可以设置许多参数，但绘制流水线中提供的基本操作是固定的。例如，确定了光源和材质后，只能按照一种光照模型进行明暗计算。

近来，流水线体系结构有一个较大的进展。顶点处理模块和片元处理模块对于应用程序来说都是可编程的。这一进展最激动人心的方面之一是许多技术都可以实时实现了，这在以前是做不到的，因为在功能固定的绘制流水线中没有实现这些技术。

在可编程流水线中，顶点处理程序可以在顶点经过流水线处理时修改每个顶点的位置或颜色，这样就可以实现许多光线–材质模型或者创建新型的投影。片元处理程序也允许用户以许多新的方式来使用纹理。通过纹理映射，许多以前只能离线实现的算法现在是可编程的了，在彩图 10 中演示的凹凸映射就是这样一个例子。第 9 章将讨论这些新技术。

1.9　性能特征

在流水线体系结构中有两类本质上不同的处理。在流水线的前端是几何处理，通过各种变换、顶点明暗绘制、裁剪和图元组装来处理顶点。这样的处理很适合流水线实现，而且一般涉及浮点运算。由 Silicon Graphics, Inc.（SGI）开发的几何引擎把许多这类运算用 VLSI 技术实现在一块专用

芯片上，曾经成为一系列高速图形工作站的基础。后来，浮点运算加速芯片把4 × 4矩阵乘法单元做到芯片里，从而使矩阵乘法被简化为一条指令。现在，图形工作站和商用图形卡使用的图形处理单元可以在芯片级执行大多数图形运算。绝大部分高性能的图形系统都采用了流水线体系结构。

从光栅化开始，包括许多后面将要讨论的特性，绘制流水线中的处理涉及许多对帧缓存中二进制位（比特）的直接操作。这样，流水线中的后端处理和前端处理在根本上是不同的，为了有效地实现后端处理，需要使用能够快速批量传送二进制位的体系结构。系统的整体性能由绘制流水线能够以多快的速度处理几何实体以及每秒钟可以修改帧缓存中的像素数来表征。因此，最快的图形工作站的特征是在前端有一个或多个几何流水线，后端有并行的位处理器。直到大约10年前，前端处理和后端处理的差异还很明显，并且每种处理都有不同类型的专用部件和电路板。现在，商用图形卡使用的GPU把整个流水线做到一个芯片上。最新的芯片利用浮点运算实现整个流水线并且具有浮点帧缓存。这些GPU功能异常强大，还可应用于图形学之外。

流水线体系结构在图形学领域占主导地位，特别是对于需要实时性的场合。前面的介绍已经解释了为什么要利用流水线体系结构来实现系统中的硬件。商用图形卡把绘制流水线内置到它们的GPU中。价格不到100美元的图形卡每秒可以绘制数以百万计的进行过明暗计算和纹理映射的多边形。然而，也有同样充分的理由把流水线作为软件实现API的基础。虚拟照相机成像模式的威力在于它对这两种情况都是适用的。

不过，在对图像的真实感要求高的场合，其他渲染器会做得更好，代价是需要更多的计算时间。Pixar公司的RenderMan接口就是其离线渲染器的接口。基于物理的技术，比如光线跟踪和辐射度方法，能够以极高的逼真度生成真实感图像，但做不到实时。

小结和注释

本书以自顶向下的方式讲解计算机图形学，本章为这种自顶向下的方法打下了基础。其中交代了总的概貌，使读者可以在第2章里开始着手编写图形应用程序，而不会感到无所依托。

我们强调了计算机图形学是生成图像的一种方法，应该把这种方法和经典的图像生成方法，尤其是像照相机这样的光学系统联系起来。此外还介绍了针孔照相机和人类视觉系统，它们都是成像系统的例子。

本章对许多图像生成的典型例子进行了描述，这些例子对计算机图形学都有一定的适用性。虚拟照相机模型对计算机图形学有两个重要影响。首先，它强调了对象和观察者是彼此独立的，由此可以引出一种很好的组织图形库中函数的方式。其次，它引出了流水线体系结构的概念，流水线中的不同阶段对几何实体进行不同的操作再把处理后的对象传递到下一个阶段。

本章也对通过光线跟踪来生成图像的思想进行了介绍。这个范例对于理解光线和物质的相互作用非常有用，而光线和物质的相互作用对于基于物理的图像生成是必不可少的。由于光线跟踪和其他基于物理的策略不能实时渲染场景，因此对它们的进一步讨论将推迟到第13章。

建模－渲染模式现在变得越来越重要。一个标准的图形工作站每秒可以生成上百万条线段或上百万个多边形，分辨率可以达到1280 × 1024。这样的工作站能以同样的速率利用一个简单的明暗模型对多边形着色并且只显示可见的表面。不过，真实感图像可能需要高至4000 × 6000的分辨率来和电影所要求的分辨率相匹配，还可能要求不能实时实现的光照和材质效果。虽然硬件和软件的性能在不断提高，但建模和渲染的目标差异非常之大，所以可以预见建模系统和渲染系统的区别还会存在下去。

下一步是探讨图形编程的应用层面。本书使用的 API 是 OpenGL。OpenGL 功能强大，被大多数平台支持，并且具有一个独特的体系结构可以使读者理解计算机图形学是如何工作的，从应用程序到最后呈现在显示器上的图像。

建议阅读资料

优秀的图形学教科书有很多。Newman 和 Sproull[New73]的书首先采用了基于虚拟照相机模型的现代观点。Foley 等人的书有多个版本[Fol90，Fol94]，它成为标准参考书已经有 10 多年了。其他的优秀教材包括 Hearn 和 Baker[Hea04]、Hill 和 Kelly[Hil07]以及 Shirley 等[Shi05]。有趣的是几乎所有这些教科书的最新版本都使用了 OpenGL。

一些很好的参考文献包括: Computer Graphics，这是SIGGRAPH(the Association for Computing Machinery's Special Interest Group on Graphics ）的季刊，还有 IEEE Computer Graphics and Applications 以及 Visual Computer。在每年一次的 SIGGRAPH 会议录上可找到最新的技术。这个会议录以前是作为 Computer Graphics 夏季的那一期，现在是作为 ACM Transactions on Computer Graphics 的一期，并以 DVD 光盘的形式出版。对于初学者来说，特别感兴趣的是 SIGGRAPH 上的最新动画展示和会议上用于培训的课程讲稿，现在这些都以 DVD 光盘的形式出版。

Sutherland 的博士论文 Sketchpad: A Man-Machine Graphical Communication System[Sut63]对交互式计算机图形学的发展具有很大的影响。Sutherland第一个意识到了人与CRT上显示的图像进行交互这种新模式的巨大威力。Sutherland当年与CRT上显示的图像进行交互的录像带拷贝现在还可以找到。

Tufte 的书[Tuf83，Tuf90，Tuf97]指出了良好的视觉设计的重要性，还包含了许多有关图形学发展的历史信息。Carlbom 和 Paciorek 的文章[Car78]很好地讨论了用于建筑设计等领域中的经典观察和借助计算机来观察这二者之间的一些关系。

许多书籍都介绍了人类视觉系统。Pratt的书[Pra78]对光栅显示器给出了一个很好的简短讨论。其他的书籍有: Glassner[Gla95]、Wyszecki和Stiles[Wys82]、Gonzales和Wood[Gon07]和Hall[Hal89]。

习题

1.1　图像生成的流水线方法并不对应于物理系统的成像过程，这样一种非物理的方法主要有哪些优点和缺点?

1.2　在计算机图形学中，像球面这样的对象一般是利用由多边形面片构造的更简单的对象（多面体）来近似的。请利用经线和纬线定义一组简单的多边形来近似球心在原点的球面。只利用四边形或者三角形也能近似球面吗?

1.3　近似球面的另一种方法是从 4 个三角形构成的正四面体开始的。假定正四面体的中心在原点并且有一个顶点在 y 轴，求出其各个顶点的坐标。对正四面体的各个面不断细分，由此导出一种算法来获得越来越接近球面的多面体。

1.4　考虑相对矩形裁剪窗口来裁剪线段。证明只需要线段的两个端点就可以确定线段属于下列三种情况中的某一种：（1）完全没有被裁剪；（2）部分可见；（3）完全被裁剪掉。

1.5　对一条线段，证明相对矩形裁剪窗口的上边界进行裁剪与相对矩形窗口的其他边界进行裁剪是独立的。利用这个结果证明可以通过 4 个简单的裁剪器组成的流水线来实现相对矩形窗口裁剪线段。

1.6　把习题1.4和习题1.5扩展到利用三维的长方体来裁剪线段。

1.7　图1.32是立方体的透视投影视图。左边的视图称为一点透视，因为沿着一个方向的平行线（例如图中所示的立方体顶面上的两条直线）会聚到图像中的灭点。相比之下，右边的视图称为两点透视。在观察者或者一个简单的照相机与立方体之间存在的某种特定关系决定了左图是一点透视而右图是两点透视，请指出这种特定关系。

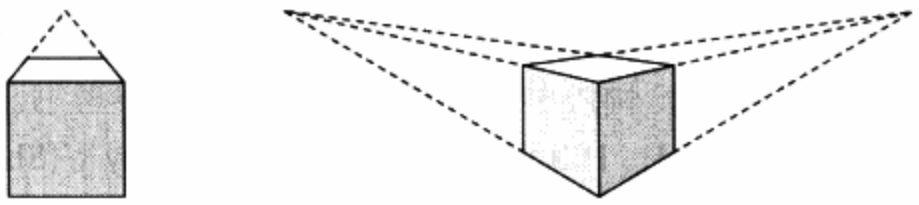


图1.32　立方体的透视投影

1.8　为了以足够高的速度刷新显示器以避免闪烁，帧缓存的速度必须足够快。一个典型的工作站显示器的分辨率可以是1280 × 1024。如果每秒刷新72次，那么帧缓存的速度必须有多快？这指的是从帧缓存中读取一个像素可以用多长时间。如果是刷新频率为60 Hz，分辨率为480 × 640的隔行扫描显示器呢？

1.9　制作电影的35 mm胶片所具有的分辨率大约是2000 × 3000。这样的分辨率对于制作与电影画质相当的电视动画意味着什么？

1.10　考虑设计一个二维图形API，这个API针对的是某个特定应用，比如VLSI设计。列举出在这个系统中应包含的所有图元和属性。

1.11　有可能设计这样一种彩色CRT，这种CRT只有一个电子枪，不需要荫罩。为了以想要的方式轰击磷光材料，电子枪需要在适当的时间打开和关闭。为什么这样的CRT比荫罩式CRT更难设计？

1.12　在一个典型的荫罩式CRT中，如果想要获得平滑的图像，每个像素的宽度必须大约是一个三色组宽度的三倍。假定某个监视器的分辨率是1280 × 1024，直径为50 cm（厘米），深度为25 cm。请估计在荫罩上两个孔之间的间隔。

1.13　作为帮助读者理解图形绘制系统的性能是如何飞速提高的一个有趣练习，请浏览一些GPU制造商（比如NVIDIA和ATI）的网站，看看其产品的技术指标。老的图形卡和GPU的技术指标往往也可以看到。请读者通过对比体会：几何处理的性能提高得有多快？像素处理呢？渲染单个三角形的代价是如何降低的？

第2章　图形学编程

本书讲解计算机图形学的方法是面向编程的，因此旨在使读者尽可能快地开始编写图形程序。为此，我们介绍一个最小化的应用程序编程接口（API）。读者利用这个API可以编写出许多有趣的二维和三维图形程序，还可以熟悉一些图形学中的基本概念。

这里把二维图形看做是三维图形的特殊情形。有了这样的看法，就可以开始编写程序了，尽管这一章很少涉及三维概念。本章编写的二维图形程序不需要修改就可以在三维图形系统里执行。

我们把绘制Sierpinski镂垫作为在这一章学习图形学编程的例子，这是一个简单的问题，但读者可以从中学到不少东西。从这个例子可以看到，使用少数几个图形函数就能生成一幅有趣的、复杂得出人意料的图像。本章将使用的API是OpenGL，但是所讨论的基础概念足够广泛，可以包含绝大多数现代图形系统。在这一章里介绍的功能足以让读者编写出基本的二维和三维图形程序，但这些程序还不能和用户交互。

2.1　Sierpinski镂垫

本章把绘制Sierpinski镂垫作为编写图形学程序的示例。Sierpinski镂垫是一个有趣的图形，它有很长的历史，在分形几何这样的领域中是重要的研究对象。可以按递归和随机的方式来定义Sierpinski镂垫，但当迭代的次数趋于无限时，它的性质是完全确定的，并不随机。我们从Sierpinski镂垫的二维版本开始，不过在2.11节中会看到，Sierpinski镂垫的三维版本与二维版本几乎相同。

假定从空间中的三个点开始。只要这些点不是共线的，它们就定义了一个三角形，而且也定义了一个平面。假定这个平面是$z=0$，并且这些点在某个方便的坐标系[①]下的坐标是$(x_1, y_1, 0)$、$(x_2, y_2, 0)$和$(x_3, y_3, 0)$。Sierpinski镂垫的构造过程如下：

1. 在三角形内随机选择一个初始点(x, y, z)。
2. 随机选择三个顶点之一。
3. 找出(x, y, z)和随机选择的顶点这二者连线的中点。
4. 在显示器上把这个中点所对应的位置用某种标记（比如小圆圈）显示出来。
5. 用这个中点替换(x, y, z)。
6. 转到步骤2。

这样，每当计算出一个新的点，就把它显示在输出设备上。这个过程示于图2.1，其中$\mathbf{p}_0$是初始位置，$\mathbf{p}_1$和$\mathbf{p}_2$是由算法生成的前两个位置。

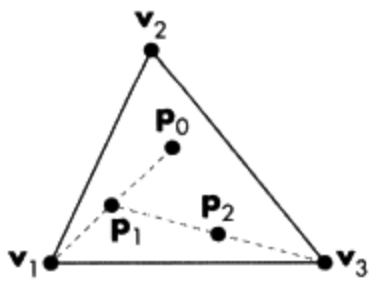


图2.1　Sierpinski镂垫的生成

① 在第4章，将把坐标系的概念扩展到更一般的标架。

在编写程序之前，读者可以尝试在纸上画画算法会生成什么样的图形，可能读者会对得到的结果感到惊奇。

实现上述算法程序的一种可能形式如下：

```
main( )
{
    initialize_the_system();

    for(some_number_of_points)
    {
        pt = generate_a_point();
        display_the_point(pt);
    }

    cleanup();
}
```

虽然最终的OpenGL程序在组织上会稍有不同，但它几乎就是这么简单。下面分阶段编写程序。首先把注意力集中到核心部分——生成和显示点。我们必须回答两个问题：怎样表示空间中的点？应该使用二维表示、三维表示还是其他表示？

2.2　编写二维图形应用程序

对于像Sierpinski镂垫这样的二维图形，虽然可以使用基于笔式绘图仪模型的API，但以后会看到这样的方法具有局限性，因此转而选择从三维世界开始，把现在用来绘制Sierpinski镂垫的二维系统看做三维系统的特殊情形。在数学上，把二维平面或者简单的二维曲面当做三维空间的一个子空间。因此，关于更大的三维空间的陈述，不论是实际的还是抽象的，都适用于更简单的二维空间。

平面 $z = 0$ 上的点，既可以表示为三维空间中的点 $\mathbf{p} = (x, y, 0)$，也可以表示为二维平面上的点 $\mathbf{p} = (x, y)$。和大多数三维图形系统一样，OpenGL允许使用这两种形式中的任何一种，不过无论用户使用哪种表示形式，系统内部的表示都是一样的。我们可以用许多方式来表示点，但是最简单的方法是把一个三维空间中的点表示为一个三元组 $\mathbf{p} = (x, y, z)$或者一个列矩阵：

$$\mathbf{p} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

列矩阵的分量给出了该点的位置。现在暂时不考虑点 $\mathbf{p}$ 的表示是在哪个坐标系下的。

在OpenGL中，术语顶点和点的含义并不完全相同。一个顶点（vertex）是空间中的一个位置，在计算机图形学中使用的空间包括二维的、三维的和四维的。我们利用顶点来定义图形系统可识别的基本几何图元。最简单的几何图元是空间中的一个点，这只需要由一个顶点就可以确定。两个顶点定义了第二种图元对象——一条线段。三个顶点可以定义一个三角形或者一个圆，4个顶点可以定义一个四边形，等等。

OpenGL中的许多函数都有多种形式，因此用户可以选择对他的问题而言是最合适的形式。顶点函数的一般形式为：

```
glVertex*()
```

这里，* 表示形如nt或者ntv这样的两个或者三个字符，其中n代表维数（2、3或4）；t表示数据类型，比如整型（i）、浮点型（f）或双精度型（d）；如果还有v，则表示变量由指向一个数组的指

针来给出，而不是通过参数列表给出。我们会使用最适合讨论的形式，读者可以在 *OpenGL Reference Manual*[Ope04]中找到其他函数形式的细节。无论用户使用哪种形式，系统内部的表示都是一样的。同样，用来绘制 Sierpinski 镂垫的平面既可以看做是二维空间，也可以看做是三维空间中对应于 $z=0$ 的子空间。在第 4 章会看到系统内部的表示是四维的，不过现在不需要关心这一点。

在 OpenGL 中，经常使用基本的 OpenGL 数据类型，比如 GLfloat 和 GLint，而不是 C 语言中的数据类型，比如 float 和 int。OpenGL 的头文件里一般以清晰的方式定义了这些数据类型，例如：

```
#define GLfloat float
```

使用 OpenGL 的数据类型可以增加实现的灵活性，例如，可能想把浮点型改为双精度型而不改变已有的应用程序。

现在再回到顶点函数，如果用户希望使用整数二维坐标，下面的形式是合适的：

```
glVertex2i(GLint xi, GLint yi)
```

而下面的形式利用浮点数确定了三维空间中的一个位置：

```
glVertex3f(GLfloat x, GLfloat y, GLfloat z)
```

最后，如果利用数组来存储三维顶点的信息：

```
GLfloat vertex[3]
```

那么可以使用

```
glVertex3fv(vertex)
```

顶点可以用来定义许多类型的几何图元，不同类型的几何图元需要不同数目的顶点来定义。可以把顶点函数放到函数 glBegin 和 glEnd 之间来组合任意多的顶点。glBegin 的参数确定了想要用顶点来定义的图元类型。因此，一条线段可以定义如下：

```
glBegin(GL_LINES);
  glVertex3f(x1,y1,z1);
  glVertex3f(x2,y2,z2);
glEnd();
```

也可以用下面的代码定义一对点：

```
glBegin(GL_POINTS);
  glVertex3f(x1,y1,z1);
  glVertex3f(x2,y2,z2);
glEnd();
```

现在可以考虑绘制 Sierpinski 镂垫的核心代码了。假定生成的所有点都位于一个大小是 50×50 的正方形里，原点位于正方形的左下角。选择这个范围来绘图比较方便，而且要改变也容易。

首先，必须确定在程序中如何表示几何数据。可以采用三个变量 x、y、z 这种最基本的表示。在这个程序中，因为要在平面 $z=0$ 内绘图，所以可以使用

```
glVertex3f(x,y,0);
```

或者

```
glVertex2f(x,y);
```

还可以定义一个新的数据类型：

```
typedef GLfloat point2[2];
```

然后利用像下面这样的代码来指定点[①]：

```
point2 p;
glVertex2fv(p);
```

通常，像这样定义数组来存放点并使用glVertex3fv或者glVertex2fv是最方便的。

我们创建一个函数display来绘制Sierpinski镂垫，每次调用这个函数会生成5000个点。在函数display中定义了一个顶点数组vertices[3]来表示三角形：

```
void display()
{
   GLfloat vertices[3][3] = {{0.0, 0.0, 0.0}, {25.0, 50.0, 0.0},
                             {50.0, 0.0, 0.0}};
            /* 平面z=0中的任意一个三角形 */
   GLfloat p[3] = {7.5 ,5.0, 0.0};
           /*或者设置为三角形内其他任何一点 */
   int j, k;
   int rand(); /* 标准的随机数生成器 */

   glBegin(GL_POINTS);
   for(k=0;k<5000;k++)
   {
     /* 从0、1、2中随机挑选一个顶点*/

     j=rand()%3;

     /* 计算新的位置*/
     p[0] = (p[0] + vertices[j][0])/2;
     p[1] = (p[1] + vertices[j][1])/2;
     /* 显示新的点 */

        glVertex3fv(p);
   }
   glEnd();
   glFlush();
}
```

注意，因为所有的点都在平面$z=0$中，也可以使用二维表示，这只需要把display函数中的三行代码改为：

```
GLfloat
vertices[3][2] = {{0.0, 0.0}, {25.0, 50.0}, {50.0, 0.0}};

GLfloat p[2] = {7.5 ,5.0};

glVertex2fv(p);
```

函数rand是标准的随机数生成器，每次调用它都会生成一个新的随机整数。我们利用求模运算把这个随机整数化为三个整数0、1、2之一。对于迭代次数比较少的情况，随机数生成器的特性并不重要，任何其他的随机数生成器都应该不会比rand差。

调用glFlush确保了点会尽可能快地绘制到屏幕上。如果省去它，程序也能正确执行，但在系统忙碌时或者在网络环境中可能会有延迟。现在还没有一个完整的程序，图2.2是期望看到的输出。

---

① 因为OpenGL不是面向对象的，所以不能定义一个点对象以及相应的一组重载运算符（见习题2.21）。

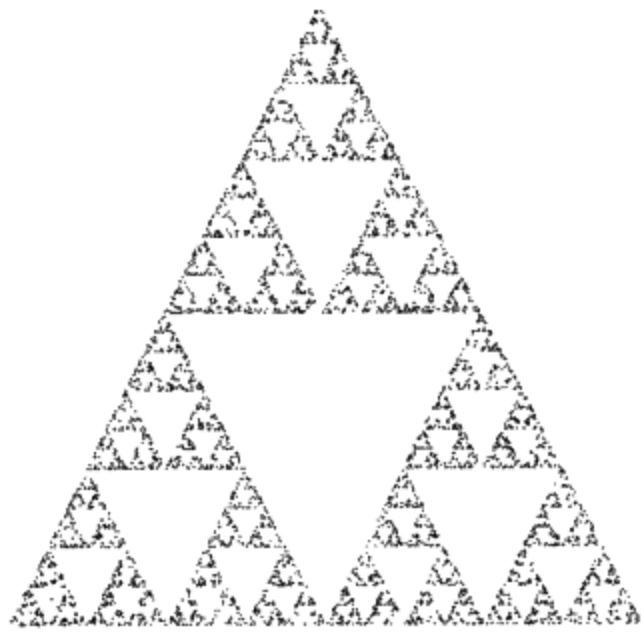


图 2.2　用 5000 个随机点生成的 Sierpinski 镂垫

请读者注意，因为不共线的三个点定义了一个平面，如果从三个点(x_1, y_1, z_1)、(x_2, y_2, z_2)和(x_3, y_3, z_3)开始，在这三个点定义的平面内选择初始点，并把下面的代码添加到程序里：

```
p[2]=(p[2]+vertices[j][2])/2.0;
```

那么生成的 Sierpinski 镂垫就将位于一开始那三个点定义的平面内。

现在已经编写出了程序的核心部分，但是还得考虑下面一些问题：

1. 用什么颜色来绘制？
2. 图像出现在屏幕的什么位置？
3. 图像会有多大？
4. 如何在显示器上创建一块区域，也就是一个窗口来显示图像？
5. 无限大的绘图表面有多大部分会显示在屏幕上？
6. 图像会在屏幕上持续显示多长时间？

对这些问题的回答很重要，尽管一开始它们可能看起来不在考虑之内。后面会看到，为了回答这些问题以及控制所绘图形的显示外观而编写的基本代码不会因应用程序的不同而有实质的改变。因此，现在所付出的努力以后会得到回报的。

2.2.1　坐标系

到了这个时候，读者可能会对如何解释用来确定顶点的x、y、z的数值感到困惑。度量它们的单位是什么？是英寸、米还是微米？原点在哪里？对每个这样的问题，简单的回答是由用户来决定。

最初的图形系统要求用户直接按照显示设备的单位来确定所有的信息，比如顶点的位置。如果对于高级应用程序也是如此，为了确定屏幕上点的位置就得给出这个点离显示器的转角处有多少像素或者厘米。这种方法明显有不少问题，其中一个严重的问题是：用计算机屏幕上的距离来描述自然度量单位是光年的现象（比如显示天文数据）或者微米的现象（集成电路的设计）很荒谬。图形软件系统的主要进展之一就是用户能够使用任何想用的坐标系。与设备无关的图形学（device-independent graphics）的出现使得应用程序员不需要再关心输入和输出设备的细节。用户的坐标系称为世界坐标系（world coordinate system），或者称为应用（application）、模型（model）或对象坐标系（object coordinate system）。我们可以使用适合于特定应用的任何数字，当然计算机中的浮点运算会对实际上能表示的数值稍有限制。

用显示器上的单位度量出的数值起初称为物理设备坐标（physical-device coordinate）或者称为设备坐标（device coordinate）。对于光栅设备，比如大多数CRT显示器，使用的术语是窗口坐标（window coordinate）或者屏幕坐标（screen coordinate）[①]。屏幕坐标总是用某种整型数来表示，因为帧缓存中任何像素的中心都必须位于固定的网格点上，或者等价地说，因为离散性是像素的固有属性，所以用整数来确定它们的位置。

在图像生成的某个阶段，世界坐标必须映射成屏幕坐标，如图2.3所示。这个映射作为绘制过程的一部分自动执行，它是由图形系统而不是用户来完成的。在下面几节将要看到，定义这个映射只需要用户指定几个参数，比如用户想要观察场景中的多大范围以及用显示器上的多大区域来显示图像。

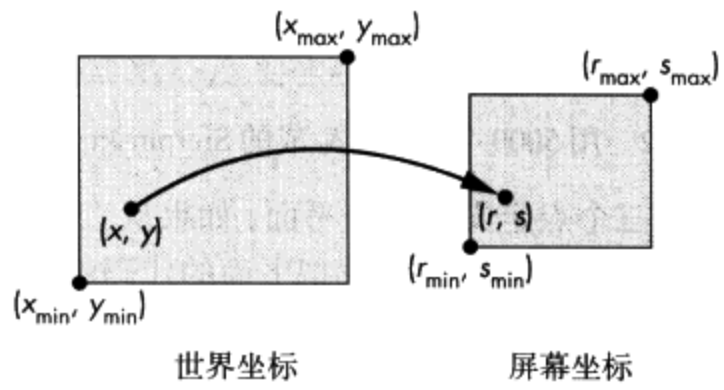


图2.3　从世界坐标到屏幕坐标的映射

2.3　OpenGL API

至此已经有了一个简单图形程序的核心部分。现在，我们想控制对象在屏幕上显示的方式，还想控制程序的执行流程，而且要和窗口系统交互。在完成程序之前，先对OpenGL API进行更详细的介绍。因为不管顶点被指定为二维实体还是三维实体，它们的内部表示都是一样的，所以这里讨论的一切也都适用于三维的情况。当然，三维系统支持的功能更多，可现在才刚刚开始。本章集中讨论如何定义要显示的图元，到第3章再讨论交互。

OpenGL的结构与包括Java3D和DirectX在内的大多数现代API相似。这样，读者学习OpenGL所付出的努力也将在学习其他软件系统时得到回报。尽管OpenGL易于学习，但和其他API相比，它的功能更强大。OpenGL不但支持从第2～6章编写的简单的二维和三维程序，而且还支持将要在第8～13章学习的高级渲染技术。

我们主要的目的是学习计算机图形学，使用API是为了帮助我们达到这个目的。因此，这里没有介绍所有的OpenGL函数，而且还省略了很多细节。不过样例程序是完备的。读者在这一章末尾的建议阅读资料中可以找到关于OpenGL和其他API的更多详细信息。

2.3.1　图形函数

图形软件包的基本模型是一个黑盒子（black box），工程师用这个术语来表示一个不知其内部工作原理的系统，它的性质只能通过输入和输出来描述。我们可以把图形系统看做是一个黑盒子，它的输入是来自应用程序的函数调用，来自像鼠标和键盘这样的输入设备的数据，以及其他可能的输入，比如来自操作系统的消息；它的输出主要是发送到输出设备的图形。就目前而言，可以采取

① OpenGL把光栅坐标和屏幕坐标看做是不同的，但这个区别只有到了第7章讨论OpenGL绘制流水线实现时才显得重要。

简化的观点：输入是函数调用，输出是显示在监视器上的图元，如图2.4所示。尽管使用的OpenGL只是一个特定的 API，但所有的图形 API 都具有与之相似的结构。

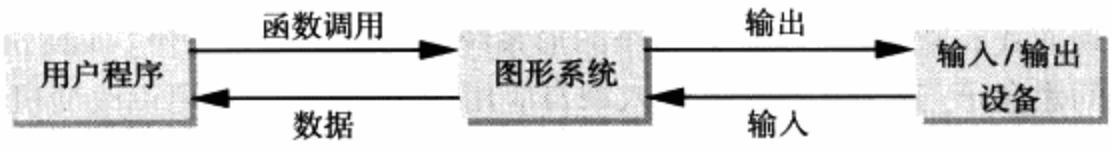


图 2.4　图形系统的黑盒子模型

API可以由其库中提供的函数来描述。一个好的API可能包含几百个函数，所以把这些函数分成以下 7 大类是有益的：

1. 图元函数
2. 属性函数
3. 观察函数
4. 变换函数
5. 输入函数
6. 控制函数
7. 查询函数

图元函数（primitive function）定义了系统可以显示的低级对象或者最基本的实体。视 API 的不同，图元可以包括点、线段、多边形、像素、文本和各种类型的曲线与曲面。

如果说图元就是API能显示的基本对象，那么属性就是API控制图元在显示器上显示的方式。从选择一条线段的颜色，到挑选多边形内部的填充模式，再到选择图形标题文本的字体，这些都是属性函数（attribute function）允许执行的操作。

如果要生成一幅图像，就必须对虚拟照相机进行设置。如在第1章已经看到的那样，应该指定照相机在场景中的位置和方向，还得指定作用相当于透镜的其他参数。这个过程不但把视图固定下来，而且允许用户把太近或者太远的对象裁剪掉。通过调用观察函数（viewing function）可以指定各种视图，尽管不同的 API 在选择视图的灵活性方面有所不同。

一个好的API的特征之一是它为用户提供了一组变换函数（transformation function），这样用户就可以对对象进行诸如旋转、平移和缩放之类的变换。在第 5 章讨论观察和第 10 章讨论建模时都要大量使用矩阵变换。在 OpenGL 中，通过几个变换函数来实现观察和建模的功能。

为了开发交互式应用程序，API必须提供一组输入函数（input function）。现代图形系统的特征是输入形式多种多样，通过调用输入函数可以对这些输入数据进行处理。我们需要API提供的输入函数以接收来自键盘、鼠标和数据板等设备的输入。第3章将介绍一些函数，它们能够在不同的输入模式下响应各种不同的输入设备。

在任何实际的应用中，还必须考虑到在一个多处理器和多窗口环境中存在的复杂性，这样的环境通常是有多个其他用户的网络环境。控制函数（control function）使用户能够与窗口系统通信、初始化程序以及处理在程序执行期间发生的任何错误。

如果要编写与设备无关的程序，用户会期望由API的具体实现来考虑不同设备间的差异，比如支持的颜色数目或者显示器的尺寸。然而，在一些应用中，需要知道API的一个特定实现的某些性质。例如，如果事先知道显示器只能支持两种而不是上百万种颜色，也许会采取其他更简单的方式来编写程序。更普遍的情形是：在应用程序中可能经常使用API中的其他信息，包括照相机的参数或者帧缓存中的值。一个好的 API 通过一组查询函数（query function）提供这些信息。

2.3.2　图形绘制流水线和状态机

如果把对图形API的一些看法组合在一起，可以获得另一种观点，这种观点与OpenGL实际上采用的组织和实现方式非常相近。可以把整个图形系统看做是一个状态机（state machine），即一个包含有穷状态机的黑盒子。这个状态机的输入来自应用程序。这些输入可以改变机器的状态或者使机器产生可见的输出。从API的角度来看，图形函数有两类：一类函数定义图元（图元要在状态机内部的绘制流水线上进行处理）；另一类函数改变机器内部的状态或者返回状态信息。在OpenGL中，像glVertex这样的函数属于第一类，几乎所有其他函数都属于第二类。

从这种看待OpenGL的观点可以得出一个重要结论：OpenGL中的参数大多数是持续性的，它们的值保持不变，除非通过调用改变状态的函数来显式地改变它们。例如，一旦设置了一种颜色，这种颜色就一直作为当前颜色，除非再次调用了改变颜色的函数。从这种观点可以得出的另一个结论是：虽然有些属性可以和对象在概念上结合起来，比如一条红色的线或者一个蓝色的圆，但是属性实际上是状态的一部分，于是要想把线绘制成红色的，只有当前的颜色状态是红色才行。尽管在应用程序中把属性和图元捆绑到一起通常是无害的，而且这往往还是更可取的，但是如果忽视了需要改变的状态或者失去了当前状态的信息，就可能导致不良的后果。

2.3.3　OpenGL接口

大多数应用程序将通过三个库中的函数来直接使用OpenGL。第一个是GL库（在Windows中称为OpenGL库），它是基本的图形库，其中的函数名以gl开头。第二个库是OpenGL工具库（OpenGL Utility Library，GLU），这个库只调用GL库中的函数，但包含了生成常用对象和简化观察的函数。所有GLU中的函数都可以从核心库GL来创建，不过应用程序员更愿意不重新编写这些代码。所有的OpenGL实现都包含GLU库，其中的函数名以glu开头。

为了和窗口系统交互以及把来自外部设备的输入传递给应用程序，至少还需要一个函数库。对每个主要的窗口系统都有一个专用的函数库作为窗口系统和OpenGL之间的“黏合剂”。对于X Window System，这个库是GLX；对于Windows，这个库是wgl；对于Macintosh，这个库是agl。与其对不同的窗口系统使用不同的库，还不如使用OpenGL实用工具库（OpenGL Utility Toolkit Library），这是一个很容易获得的函数库，它提供了任何现代窗口系统都需要的最基本的功能。本章将介绍GLUT中的几个函数，到第3章详细讨论输入和交互时再介绍其中更多的函数。

图2.5所示是在X Window System环境下OpenGL函数库的组织。对这个窗口系统，GLUT会使用GLX和X库。不过，应用程序只能够使用GLUT函数，因此和GLUT库一起重新编译后就可以移植到其他窗口系统。

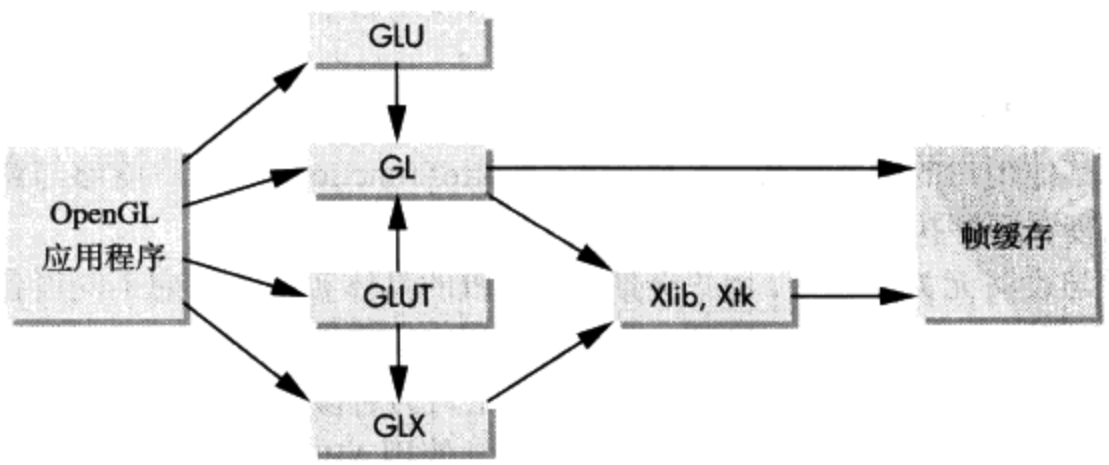


图2.5　在X Window System环境下OpenGL函数库的组织

为了增加代码的可读性和避免使用令人感到困惑的幻数(magic number), OpenGL大量地使用宏。像 GL_FILL 和 GL_POINTS 这样的字符串在头文件 (.h) 中有定义。对于 OpenGL 的大多数实现，只需要在应用程序中添加下面两行 include 代码之一：

```
#include <GL/glut.h>
```

或者

```
#include <GLUT/glut.h>
```

就可以把 glut.h、gl.h 和 glu.h 这三个头文件包含到程序中。

2.4　图元和属性

在图形学界，对 API 中应该支持哪些图元这个问题的讨论一直在继续。这个讨论很早就有，现在也没有得出完全统一的意见。最低限要求者的观点是API应该包含少数所有硬件都能支持的图元。此外，这些图元应该是互不相关的，各种图元提供的功能互不重叠。有代表性的最小系统支持线段、多边形和某种形式的文本 (字符串)，所有这些图元都可以由硬件高效生成。另一个极端是系统要支持各种图元，比如圆、曲线、曲面和实体。这一方的理由是用户需要更复杂的图元来构建复杂的应用程序。可是，很少有硬件系统能够支持全部用户的需要，而且利用这样的系统开发出来的程序很难移植，因为很少有 API 的实现能够支持整个图元集。

OpenGL 的选择介于这两个极端之间。基本库只有少量图元。GLU 库包含更丰富的图元，是从基本库推导出来的。

OpenGL 支持两类图元：几何图元 (geometric primitive) 和图像 (image)[或者光栅图元 (raster primitive)]。几何图元由用户的问题域来确定，包括点、线段、多边形、曲线和曲面。这些图元要经过几何流水线的处理，如图 2.6 所示，其中有一系列几何操作来确定某个图元是否可见，如果可见又将显示在屏幕的何处，以及如何把图元光栅化为帧缓存中的像素。因为几何图元存在于二维或者三维空间中，所以可以对它们进行诸如旋转和平移的几何操作。此外，通过这些几何操作，还可以把它们作为构造其他几何对象的模块。光栅图元的一个例子是像素阵列，它不具有几何属性，不能像对几何图元那样对光栅图元进行几何操作。光栅图元需要经过另一条流水线的处理，这条流水线与几何流水线并行，终点也是帧缓存。第 8 章将再讨论光栅图元。

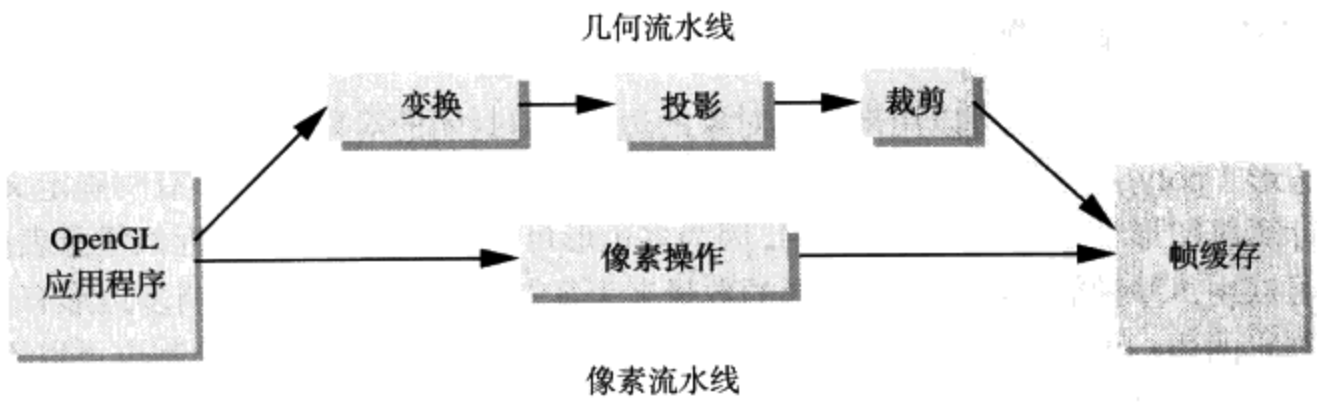


图 2.6　简化的 OpenGL 绘制流水线

基本的OpenGL几何图元是由顶点集合来确定的。因此，程序员通过下列形式的代码来定义其对象：

```
glBegin(type);
    glVertex*(...);
    .
    .
    .
    glVertex*(...);
glEnd();
```

参数type指定了OpenGL把顶点组装起来定义几何对象的方式。其他代码和OpenGL函数调用可以出现在glBegin和glEnd之间。例如，在glBegin和glEnd之间或者两次glVertex函数调用之间可以改变属性，也可以计算下一个顶点。是否包含内部是不同类型的基本几何图元之间在概念上的一个主要差异。所有的基本几何图元都是由顶点集合定义的。

直线上两个顶点之间的有限部分称为线段（line segment），它和无穷延伸的直线不同。线段在几何和计算机图形学中都非常重要。用户可以用线段来近似曲线，也可以用一系列线段把数据点连接起来构成图形。用户还可以用线段构成封闭对象的边界，比如包含内部的多边形。因此，顶点和线段都可以用来表示对象。

如图2.7所示，OpenGL提供了多种类型的点和线段图元。这几类图元和相应的type参数如下：

点（GL_POINTS）：每个顶点被显示的大小至少是一个像素。

线段（GL_LINES）：这种图元把相继的顶点配对后解释成线段的两个端点。注意，相继的线段一般不相连，因为顶点是两两配对来处理的。

折线（GL_LINE_STRIP，GL_LINE_LOOP）：要想让相继的顶点（以及线段）相连，可以使用折线（polyline）图元。许多曲线可以通过合适的折线来近似。如果希望折线闭合，可以把最后一个顶点设置成与第一个顶点重合，也可以使用GL_LINE_LOOP类型，它会在最后一个顶点和第一个顶点之间画一条线段，于是得到一条闭合的环路。

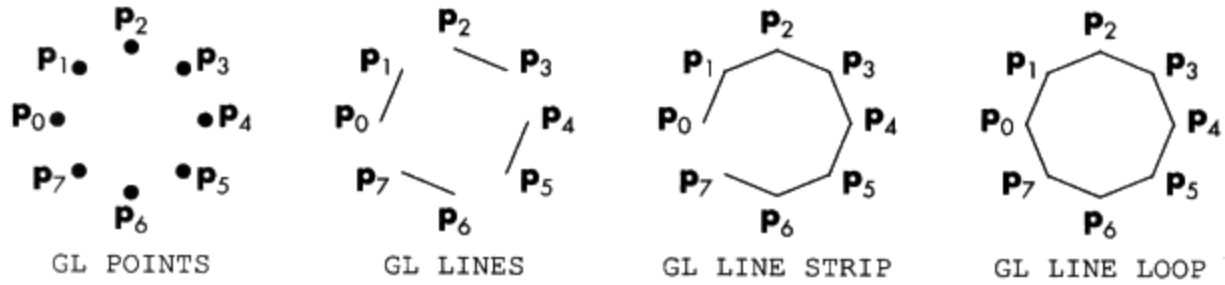


图2.7　点和线段的类型

2.4.1　多边形的基本概念

线段和折线这两个图元可以用来建模对象的边界，但封闭的对象也可能包含内部（见图2.8）。通常用多边形（polygon）这个术语来表示边界可以用封闭折线描述而且内部有明确定义的对象[①]。多边形在计算机图形学中起到了特殊作用，因为多边形可以快速显示并用来近似任意表面。图形系统的性能可以用每秒钟能够绘制的多边形的数量来表征[②]。如图2.9所示，绘制多边形有多种方式：可以只绘制它的边；可以用单色或者某种模式填充它的内部；它的边可以绘制也可以不绘制。尽管用有序顶点列表很容易就可以定义多边形的外部边界，但是如果多边形的内部没有明确定义，那么这个顶点列表可能根本不被绘制或者被绘制的方式不对。有三条性质可以确保多边形被正确显示：多边形必须是简单的、凸的和平面的。

① 有时用填充区域这个术语代替多边形。

② 多边形绘制速度的度量涉及顶点的数目和内部像素的数目。

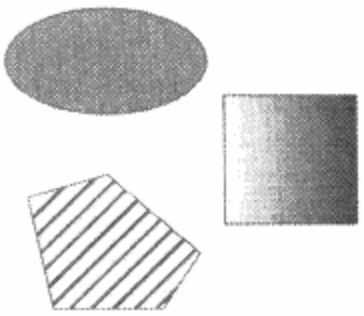


图 2.8　填充的对象

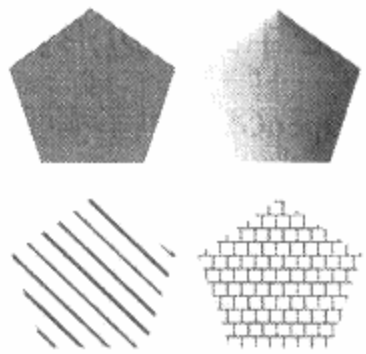


图 2.9　显示多边形的几种方式

在二维空间，只要多边形的任何两条边都不相交，就称它是一个简单（simple）多边形。从图2.10可以看到，简单二维多边形的内部是可以明确定义的。由顶点的位置就可以确定一个多边形是不是简单的，但是检验的代价太高了（见习题2.12），所以大部分图形系统要求任何必要的检验都由应用程序来完成。可以提出这样的问题：图形系统会如何显示一个非简单的多边形？是不是有一种办法可以定义一个非简单多边形的内部？第 8 章将进一步讨论这些问题。

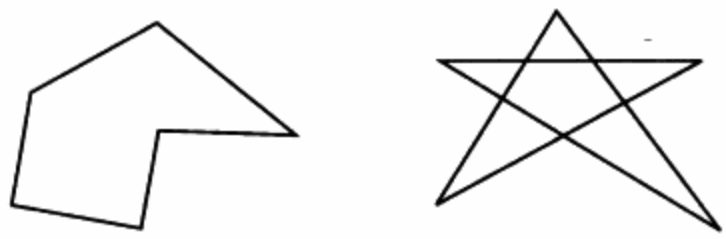


图 2.10　多边形：(a) 简单多边形；(b) 非简单多边形

从实现一个填充多边形内部的实际算法的角度来看，仅有简单性还不够。有些API只有对凸多边形才能保证一致的填充效果，否则不同的实现会有不同的填充结果。如果连接对象内部或边界上任意两点的线段上所有的点都在对象内部，那么这个对象就是凸的（convex）。如图 2.11 所示，$\mathbf{p}_1$和$\mathbf{p}_2$是多边形内部任意两点，连接它们的线段完全位于多边形的内部。尽管到现在为止一直考虑二维对象，但这个定义适用于任何类型和维数的对象。凸的对象包括三角形、四面体、矩形、圆、球和平行六面体（见图2.12）。有许多检验凸性的方法（见习题2.19），不过和简单性的检验类似，检验凸性的代价也很高，所以通常留给应用程序来完成。

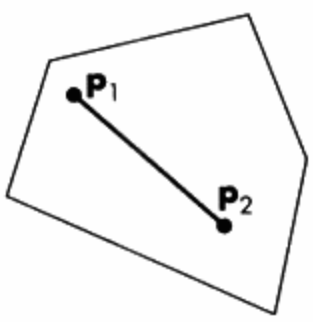


图 2.11　凸性

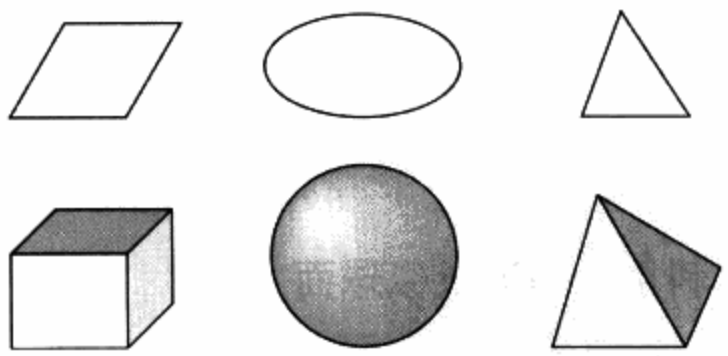


图 2.12　凸的对象

在三维空间，处理多边形会遇到更多的困难，因为与二维对象不同，定义多边形的全部顶点未必位于同一平面内。有一个被大多数图形系统利用的性质也要用到，即任何不共线的三点确定了一个三角形以及这个三角形所在的平面。于是，如果总是使用三角形，那么就可以确保这些对象被正确地绘制。在许多时候，几乎是被迫使用三角形，因为一些典型的绘制算法只有对平面凸多边形才能保证绘制的结果是正确的。此外，硬件和图形绘制系统往往支持三角形类型，与利用三个顶点定义的多边形类型相比，三角形类型的绘制速度更快。

2.4.2　OpenGL中的多边形类型

现在来看看OpenGL提供了哪些类型的多边形图元。对包含内部的对象，可以指定下列几种类型（见图2.13）：

多边形（GL_POLYGON）：这个对象的边界和封闭折线相同，相继的顶点之间都有线段，最后一个顶点和第一个顶点之间也有线段。多边形的内部按照相关属性的状态来填充。注意，数学上的多边形具有由边界分隔开的一个内部和一个外部，但边界本身没有宽度。因此，多数图形系统允许用户用一种颜色或者一种模式填充多边形，也允许沿着边界绘制线段，但不允许同时绘制内部和边界。在OpenGL中，可以用函数glPolygonMode告诉绘制器只生成边界或者顶点，而不做填充（默认情况下要做填充）。如果想要既填充内部又显示边界，那么就得绘制两次，在两次绘制时采用不同的模式，或者画一个填充的多边形和一个具有相同顶点的封闭折线。

三角形和四边形（GL_TRIANGLES，GL_QUADS）：这些对象是多边形的特殊情形。相继的三个顶点或者四个顶点组合在一起被分别解释为三角形和四边形。使用这些类型来绘制要比使用GL_POLYGON类型效率更高。

条带和扇（GL_TRIANGLE_STRIP，GL_QUAD_STRIP，GL_TRIANGLE_FAN）：这些对象把具有公共顶点和边界的三角形或者四边形组合在一起。在三角形带中，每个顶点和前两个顶点组合起来定义一个三角形（见图2.14）。在四边形带中，每两个顶点和前两个顶点组合起来定义一个四边形。三角形扇的第一个顶点是固定的公共顶点，这个顶点和接下来的两个顶点确定了第一个三角形，从第四个顶点开始，每个顶点和前一个顶点还有第一个顶点都确定了一个三角形。

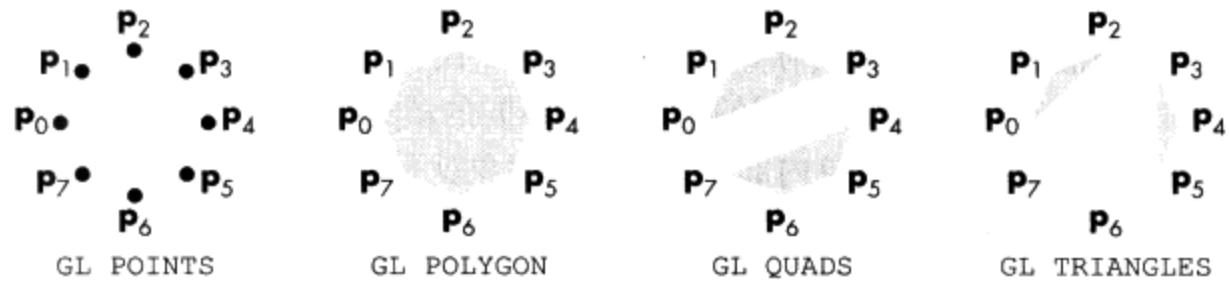


图2.13　多边形类型

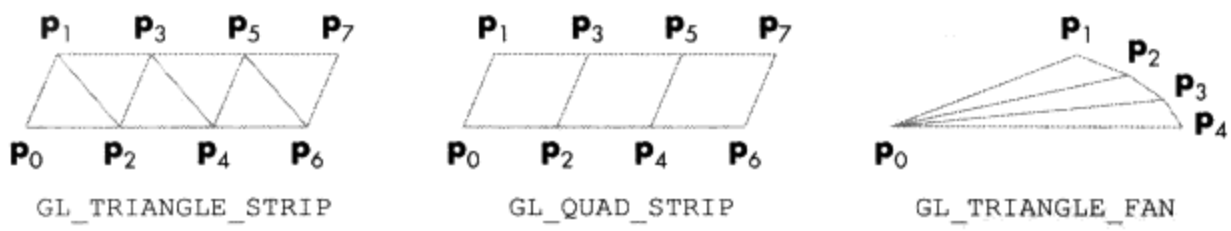


图2.14　三角形带、四边形带和三角形扇

2.4.3　利用多边形近似球面

利用三角形扇和三角形（四边形）带可以方便地近似许多曲面。例如，一种近似球面的方法是使用由一组经线和纬线定义的多边形，如图2.15所示。利用四边形带或者三角形带来生成这些多边形的效率很高。考虑一个单位球面，可以用下面三个方程来描述它：

$$x(\theta, \phi) = \sin\theta \cos\phi$$

$$y(\theta, \phi) = \cos\theta \cos\phi$$

$$z(\theta, \phi) = \sin\phi$$

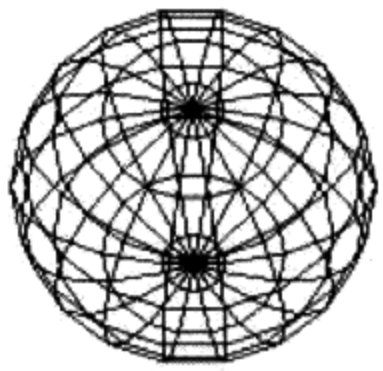


图 2.15　利用四边形近似球面

如果固定 θ，变动 ϕ，就得到了经线圈。同样，如果固定 ϕ，变动 θ，可以得到纬线圈。令 θ 和 ϕ 以固定的步长增加，计算出多个球面上的点，再把这些点作为四边形的顶点，就可以得到图 2.15 中的那些四边形。回忆对于标准的三角函数需要把度（°）化为弧度（rad）。下面的代码可以生成近似球面的四边形带，其中 θ 和 ϕ 的步长都是 20°：

```
for(phi=-80.0; phi<=80.0; phi+=20.0)
{
    phir=c*phi;
    phir20=c*(phi+20);
    glBegin(GL_QUAD_STRIP);
    for(theta=-180.0; theta<=180.0;theta+=20.0)
    {
        thetar=c*theta;
        x=sin(thetar)*cos(phir);
        y=cos(thetar)*cos(phir);
        z=sin(phir);
        glVertex3d(x,y,z);
        x=sin(thetar)*cos(phir20);
        y=cos(thetar)*cos(phir20);
        z=sin(phir20);
        glVertex3d(x,y,z);
    }
    glEnd();
}
```

不过，在球面的两极处会遇到问题：因为所有的经线在两极交会，所以不能再使用四边形带了。但可以对每个极点使用一个三角形扇，代码如下：

```
glBegin(GL_TRIANGLE_FAN);
   glVertex3d(0.0, 0.0 , 1.0);
   c=M_PI/180.0;
   c80=c*80.0;
   z=sin(c80);
   for(thet=-180.0; theta<=180.0;theta+=20.0)
   {
       thetar=c*theta;
       x=sin(thetar)*cos(c80);
       y=cos(thetar)*cos(c80);
       glVertex3d(x,y,z);
   }
glEnd();

glBegin(GL_TRIANGLE_FAN);
   glVertex3d(0.0, 0.0, -1.0);
```

```
    z=-sin(c80);
    for(theta=-180.0;theta<=180.0;theta+=20.0)
    {
        thetar=c*theta;
        x=sin(thetar)*cos(c80);
        y=cos(thetar)*cos(c80);
        glVertex3d(x,y,z);
    }
glEnd();
```

2.4.4　文本

对于像数据的分析和显示这样的应用，输出的图形需要有文字注释，比如曲线图上的标注。虽然在非图形应用程序中文本是标准输出，但是在计算机图形学中使用文本还是有很多问题的。在非图形应用程序中，通常满足于以同一种方式显示一个简单的字符集。然而，在计算机图形学中，经常希望以多种方式显示文本，为此需要设置字样的风格、字符的大小、颜色和其他参数。人们也希望有许多字体可供选择。字体（font）是有特定风格的一族字样，比如Times、Computer Modern或者Helvetica。

文本有两种形式：笔划文本和光栅文本。笔划文本（stroke text）（见图2.16）是像其他几何对象那样构造出来的。用顶点来定义勾勒每个字符轮廓的线段或曲线。如果字符是由封闭边界定义的，可以对其进行填充。笔划文本的优点是：对任何其他对象能够定义的细节，也可以对它定义。因为对笔划文本的定义方式和其他图形对象一样，所以可以像任何其他几何图元那样对其进行标准的变换和观察操作。通过变换，可以放大或者旋转一个笔划字符，同时保持它的细节和外观。因此，只需要对字符定义一次，就可以通过变换来生成大小和方向都符合要求的字符。

Computer
Graphics

图2.16　笔划文本（PostScript字体）

然而，定义一个包含128个或者256个字符的笔划字体会比较复杂，而且需要相当多的时空开销。标准PostScript字体是通过多项式曲线来定义的，它们具有笔划文本的所有优缺点。各种各样的PostScript字体可以应用到不同分辨率的场合。为了缓解笔划字符绘制速度慢的问题，开发人员常常把相当多的处理任务让打印机来完成。这个策略和要在第3章讨论的客户/服务器概念有关。

光栅文本（raster text）（见图2.17）更简单，绘制也更快。在光栅文本中，字符被定义成由0和1组成的矩形阵列，这样的比特阵列称为位块（bit block）。每个位块中0和1的模式定义了字符。光栅字符可以通过位块传递（bit-block-transfer，bitblt）操作快速读取到帧缓存中，整个过程只需要调用一个函数。第8章将讨论bitblt操作。OpenGL允许应用程序使用直接访问帧缓存的函数。

用户可以通过复制（replicate）像素来扩大光栅字符的尺寸，但是这样放大的字符会出现块状外观（见图2.18）。其他类似于旋转这样的变换对于光栅字符没有意义，因为这样的变换会把定义字符的比特移动到不对应于帧缓存中像素的位置。

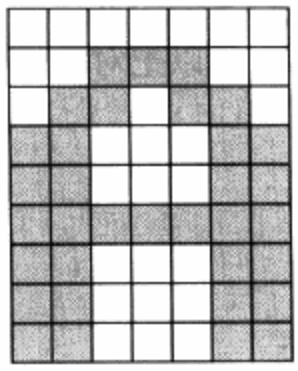
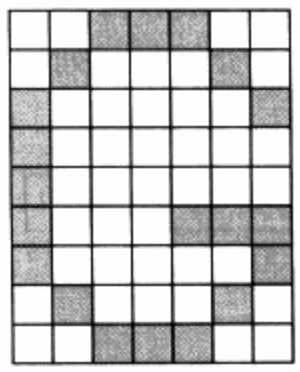


图 2.17　光栅文本



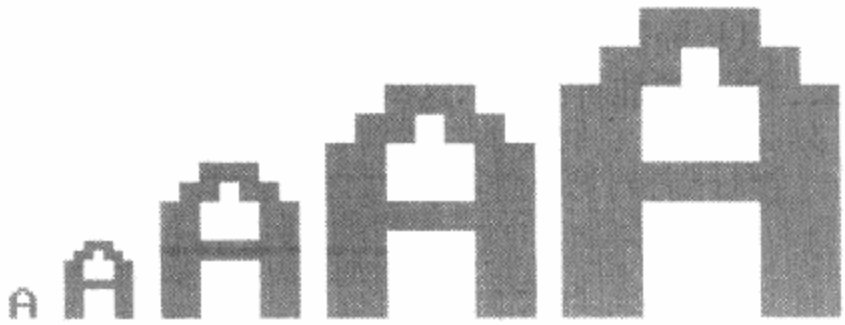


图 2.18　通过复制像素来放大光栅字符

因为笔划字符和光栅字符可以由其他图元来创建，所以 OpenGL 没有提供文本图元。不过 GLUT库提供了一些预先定义好的位图字符集和笔划字符集，这些字符集是由软件定义的，可以移植。例如，可以利用下面的函数显示一个 8 × 13 的位图字符：

```
glutBitmapCharacter(GLUT_BITMAP_8_BY_13, c)
```

其中，c是想要显示的那个字符的ASCII码。这个字符被显示在当前的光栅位置（raster position）（这个位置是当前状态的一部分），当前的光栅位置是用像素来度量的，可以通过函数glRasterPos*的多种形式来修改。第3章还会继续讨论文本，届时会讲到通过显示列表可以对笔划文本和光栅文本提供非常有效的支持。

2.4.5　曲线和曲面

前面已经讨论的所有基本图元都是由顶点来定义的。除了点类型是个例外，其他图元都由线段组成或者利用线段来定义区域的边界，区域的内部可以用单一颜色或按照某种模式填充。可以采用两种方法来创建更多的对象。

首先，可以使用已有的图元来近似曲线和曲面。例如，我们已经使用了三角形和四边形来近似球面。同样，可以利用正n边形来近似圆。更一般地讲，可以通过凸多边形网格或者细分（tessellation）处理来近似曲面，这个过程可以在绘制阶段进行，也可以在用户的程序中完成。

第 11 章将采用的另一种方法是从数学定义出发，编写图形函数来生成这些对象。像二次曲面以及参数多项式曲线和曲面这样的对象在数学上很好理解，可以利用一组顶点来确定它们。例如，可以用球心和球面上的一点来定义球面，或者用 4 个点定义一个三次多项式曲线。

多数图形系统对这两种方法都提供了一定的支持。在 OpenGL 中，GLU 库和 GLUT 库提供了对常见曲面的一些近似。也可以编写函数来定义自己的对象，还可以利用OpenGL的高级特性生成参数多项式曲线和曲面。

2.4.6　属性

在现代图形系统中，图元的类型和图元如何绘制是有区别的。一条红色的实线和一条绿色的虚线是同一几何类型的图元，但它们被绘制的方式不同。属性（attribute）决定了一个几何图元将如何绘制。很明显，颜色是属性，线段的宽度和多边形的填充模式也是属性。图2.19所示是线段和多边形的一些属性。

属性可以在建模和绘制流水线的任何阶段与图元结合或者绑定（bound）。现在考虑立即模式。在立即模式（immediate mode）下，图元不在系统中存储，而是在定义之后尽可能快地经过系统的处理以进行绘制。属性的当前值是图形系统状态的一部分。在绘制图元时要使用该类图元的当前属

性，在绘制完成后立即显示图元。系统内部不存储图元，只有最后在帧缓存中的结果会出现在屏幕上，一旦从屏幕擦除，图元的信息就完全丢失了。第3章将介绍显示列表，利用它可以把对象保存起来，这样就能再次显示这些对象。

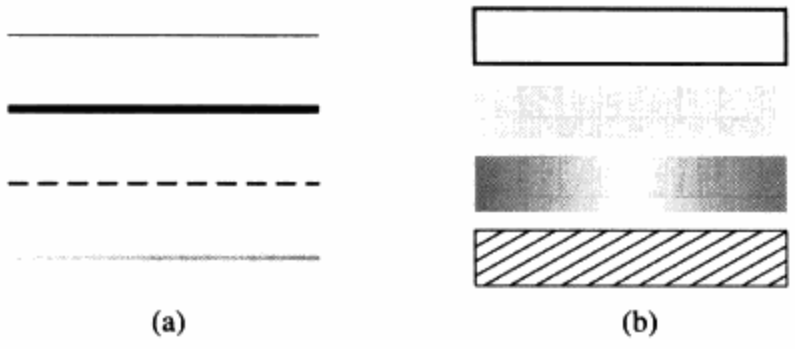


图 2.19　线段和多边形的属性：(a) 线段的属性；(b) 多边形的属性

OpenGL强调的是立即模式和流水线结构，这对于交互式应用是合适的，不过这也是OpenGL与面向对象系统之间的根本差异，在面向对象系统中可以创建对象并在需要时才调用这些对象。第10章将讨论场景图，它对于像Java3D这样的系统是很重要的，届时会看到它们提供了计算机图形学编程的另一种高级方法。

每个几何图元都有一组属性。例如，一个点有一个颜色属性和一个尺寸属性。线段的属性有颜色、宽度和模式（实线、虚线）。像多边形这样的填充图元具有更多的属性，因为必须用多个参数来指定如何进行填充。我们可以用单一颜色或者一种模式来填充，可以决定不填充多边形而仅显示其边界。如果对多边形进行填充，也可以用和内部不同的颜色来显示边界。

在把笔划文本作为图元来支持的系统中，文本图元具有多种属性。文本的一些属性如图2.20所示，这些属性包括文本串的方向，文本串中的字符路径，字符的高度和宽度，字体和式样（粗体、斜体、带下画线）。

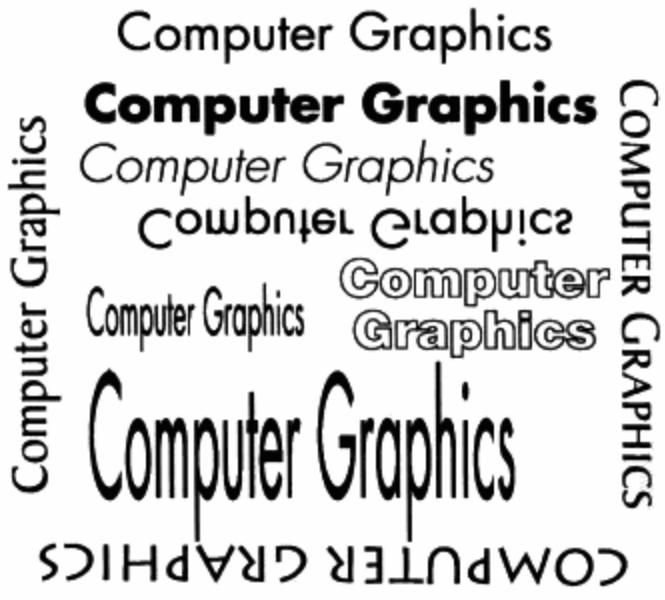


图 2.20　笔划文本的属性

2.5　颜色

颜色是人类感知、也是计算机图形学中最有趣的方面之一。我们可以利用第1章介绍的人类视觉系统的模型得出一个简单而有用的颜色模型。要想让计算机图形学完全体现人类视觉系统所能感知到的全部内容，需要对人体解剖学、生理学和心理物理学有更深入的理解。第7章将对颜色进行更详细的讨论。

一种可见的颜色可以由一个波长的函数 $C(\lambda)$来表征，这里 λ 的取值范围大致是 350 ~ 780 nm（纳米），$C(\lambda)$的一个例子如图 2.21 所示。$C(\lambda)$在可见光谱范围内一个给定波长 λ 处的值给出了颜色中那个波长分量的强度。

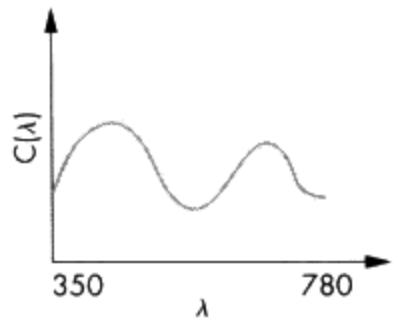


图 2.21　基于波长的颜色分布

尽管这样的表征对于在物理上能测量的颜色性质来说是精确的，但它并没有考虑人们是如何感知颜色的。第 1 章提到了人类视觉系统有三类视锥细胞负责颜色视觉。因此，对一种给定的颜色，大脑接收到的不是完整的分布 $C(\lambda)$，而是三类视锥细胞对颜色的响应值，这三个响应值称为三刺激值（tristimulus values）。因此，颜色被归结为三个数，由此引出了三色理论的基本原则（basic tenet of three-color theory）：如果两种颜色的三刺激值相同，那么人眼就无法把它们区分开。

从这个理论可以得出一个推论，那就是在原则上，显示器只需要三种原色就可以获得人类观察者能看到的颜色。我们通过调节每种原色的强度来生成一种颜色，第 1 章里已讲过在 CRT 中就是这样做的。CRT 是加色模型（additive color model）的一个例子，该模型是把原色相叠加来生成颜色的。加色模型的其他例子有投影仪和胶卷幻灯片。在这样的系统中，三种原色通常是红、绿和蓝。利用加色模型，显示器初始为黑色，原色光在上面叠加得到想要的颜色。

在像商业打印和绘图这样的应用中，减色模型（subtractive color model）更合适。这个模型从白色的表面（比如一张纸）开始，有色的颜料从照射到表面的光线中去掉一些颜色分量。假定白光照射表面，如果入射光中对应于红色的那部分波长分量被表面反射，除此以外的所有分量都被表面吸收，那么表面将是红颜色的。在减色系统中，所使用的原色通常是补色（complementary color）：青色、品红和黄色（CMY，如图 2.22 所示）。这里不讨论减色模型，读者只需要知道 RGB 加色模型和 CMY 减色模型是对偶的（见习题 2.8）即可。

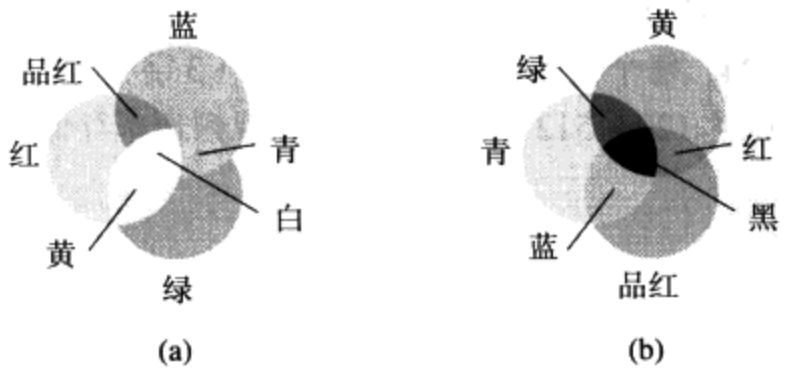


图 2.22　颜色生成：(a) 加色模型；(b) 减色模型

我们可以把一种颜色看做是颜色立方体（color solid）中的一点，如图 2.23 和彩图 25 所示。我们在一个对应于三种原色的坐标系里画这个颜色立方体。沿着一个坐标轴方向的距离代表了颜色中相应原色的分量。如果把每种原色的最大值规范化为 1，就可以把所能生成的任何颜色都表示为单位立方体中的一点。颜色立方体的各个顶点所对应的颜色为：黑（三种原色的分量都为零）；红、绿和蓝（一种原色的分量达到最大）；两种原色的组合，有青（绿和蓝两种原色的分量达到最大）、品

红（红和蓝两种原色的分量达到最大）和黄（红和绿两种原色的分量达到最大）；还有白（三种原色的分量都达到最大）。立方体的主对角线连接原点（黑）和对应于白色的顶点。所有主对角线上的颜色的三刺激值都相等，这些颜色是亮度不同的灰色。

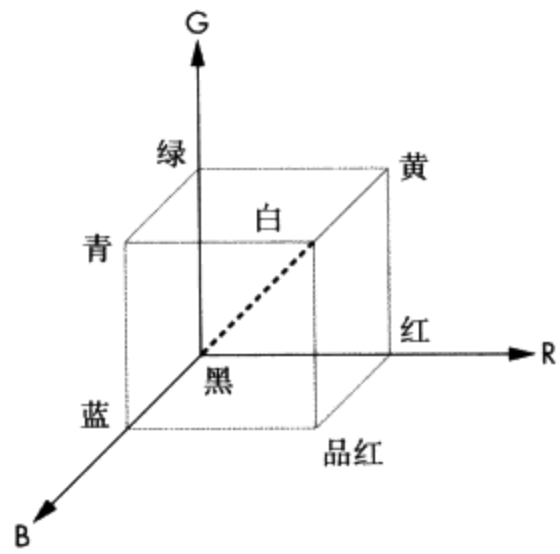


图 2.23　颜色立方体

有许多内容没有在这里详细展开，第7章再来讨论。这些内容主要是各种原色集之间的差异以及由于实际设备的物理限制而引起的一些问题。特别是一个设备可以生成的颜色集，称为它的色域（color gamut）。一个设备的色域不会与其他设备相同，并且也不同于人眼可感知的色域。此外，同样的一组三刺激值在不同设备上产生的可见颜色不会相同。

2.5.1　RGB 颜色

现在来看看在图形系统中从程序员的角度，即通过API如何处理颜色。有两种不同的方法。这里要强调的是RGB 颜色模式（RGB-color mode），因为理解它对以后讨论明暗绘制是至关重要的。在历史上，索引颜色模式（indexed-color mode）（见 2.5.2 节）更容易被硬件支持，因为这种模式对内存的要求更低，而且过去的显示器能显示的颜色数目也很有限。但 RGB 颜色模式已经成为现代图形系统的标准。

在使用加色模型的 RGB 三原色系统中，红、绿、蓝图像在概念上有各自的缓存。每个像素都分别有红、绿和蓝色分量，每种分量对应于存储器中的某个位置（见图 2.24）。在一个典型系统中，像素阵列可能为 1280 × 1024，每个像素可能包括 24 比特（3 字节）——红绿蓝各占 1 字节。现在商用图形卡的存储器容量可达 120 ~ 512 MB，所以按照视频速率存储和显示帧缓存中的内容已经不成问题。

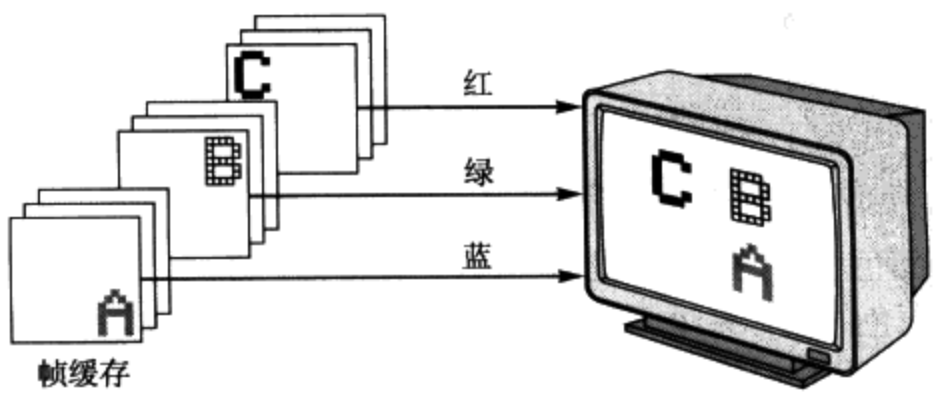


图 2.24　RGB 颜色

作为程序员，我们希望能够确定任何可以在帧缓存中存储的颜色。对上面提到的每个像素24比特的例子，总共有2^{24}种可能的颜色，有时称其为16M颜色，其中M代表1024^2。其他的系统可能每种颜色会多到12比特或者少到4比特。因为API应该独立于任何特定的硬件，所以希望以不依赖于帧缓存中位深度的方式来指定颜色，并且驱动程序和硬件应该对指定的颜色用尽可能接近的可用颜色来匹配。一种自然的技术是使用颜色立方体并用0.0 ~ 1.0之间的数来确定颜色分量，这里1.0代表对应原色的最大值［或者称为饱和（saturated）值］，0.0代表那种原色分量完全没有。在OpenGL中，我们按照下面的方式来使用颜色立方体。为了以红色来绘图，我们调用下面的函数：

```
glColor3f(1.0, 0.0, 0.0);
```

该函数会把当前的绘制颜色设置为红色。因为颜色是状态的一部分，在颜色被再次修改以前一直用红色绘图。函数名中的3f可以按照函数glVertex中出现过的类似情形来理解，它表达的信息是：在使用三色（RGB）模型并且每个颜色分量的值用C语言中的float型数给出。如果使用整型数或者字节型数来指定一种颜色，那么该数据类型的最大值对应某种原色分量最强，该数据类型的最小值对应于某种原色分量完全没有。

以后也会对四色系统（RGBA）感兴趣。第四个颜色分量（A，或者α）和RGB值一样也存储在帧缓存中，可以利用颜色函数的四维形式对它进行设置。第7章会讲到α分量的各种应用，比如生成雾的效果或者图像融合。在这里指定α值是作为OpenGL程序初始化的一部分。如果开启融合功能（见第8章），那么α值在OpenGL中的意义是不透明度（opacity）或者透明度（transparency）。透明度和不透明度在概念上是互补的。光线完全不能通过一个不透明的物体，但能完全通过一个透明物体。不透明度的值可以从完全透明（$A = 0.0$）到完全不透明（$A = 1.0$）。

OpenGL程序在初始化时要完成的任务之一是清空屏幕上的一块区域，这是将要显示输出的绘图窗口。无论在何时，只要想绘制一帧新的图像，也必须清空这个窗口。通过使用四维（RGBA）颜色系统，我们可以实现这样的效果：按照在清屏时设置的α值，绘图窗口中的内容与其他可能在它下面的窗口中的内容进行叠加，最后显示的是多个窗口中的内容相混合的结果。下面的函数定义的RGB清屏颜色是白色：

```
glClearColor(1.0, 1.0, 1.0, 1.0);
```

因为前三个分量是1.0，而且这种颜色是不透明的，因为α分量是1.0。接下来就可以利用函数glClear把屏幕上的绘图窗口清空成不透明的白色。注意，融合功能在默认情况下是关闭的。因此，在glClearColor中即使不把α值设置成1.0，默认的绘图窗口也仍然是不透明的。

2.5.2　索引颜色

早期图形系统帧缓存的位深度很有限。例如，我们可能用过这样的帧缓存，空间分辨率是1280 × 1024，但每个像素的位深度只有8。可以把每个像素的8比特分成更小的三组，对每组比特赋值，分别表示红、绿和蓝原色。虽然这样的技术对一些应用是足够的，但通常不能为用户指派颜色提供足够的灵活性。索引颜色为这个问题提供了一个解答，它可以允许应用程序显示范围很广的颜色，只要应用程序需要的颜色数目不超过像素的位深度可以索引的范围。直到今天人们还在使用这个技术。

图形系统利用索引颜色绘制图形和画家使用油画颜料来作画很相似。虽然装在小软管里的颜料数目有限，但画家通过把它们混合可以得到几乎无穷多种颜色。我们说画家潜在地拥有一个巨大的

颜色调色板（palette）。不过，在任何时候，也许由于画笔的数目有限，画家只能使用少数几种颜色。以这种方式，画家可以绘制一幅图像，虽然它包含的颜色种类不多，但却可以表达画家的想法，因为画家可以从一个大的调色板中选出这些颜色。

回到计算机中的模型上来，我们有理由认为：如果对每个具体的应用都能从一个很大的范围（调色板）中选择若干种颜色，那么即使选出的颜色种类不是很多，多数情况下也能生成高质量的图像。

我们可以按照这样的方式来选择颜色：把位深度有限的像素解释为颜色表的索引而不是颜色值。假定帧缓存中每个像素有 k 比特。每个像素的值或者索引是 $0 \sim 2^k - 1$ 之间的一个整数。假定显示颜色的精度是 m 比特，也就是说，可以有 2^m 种不同强度的红色、2^m 种不同强度的绿色和 2^m 种不同强度的蓝色。于是，我们可以在显示器上显示 2^{3m} 种颜色，然而帧缓存只能指定这些颜色中的 2^k 种。我们通过一个用户定义的大小为 $2^k \times 3m$ 的查色表（color-lookup table）（见图2.25）来解决指定颜色的问题。用户在程序中把想要使用的颜色填入查色表的 2^k 个条目（行），每种颜色的红、绿、蓝三个分量都是 m 比特。一旦用户把这个表构造好，就可以通过索引来指定一种颜色，颜色的索引指向查色表中一个合适的条目（见图2.26）。对 $k = m = 8$ 这样一种通常的配置，用户可以选择16M颜色中的256种。查色表中的256个条目构成了用户的调色板。

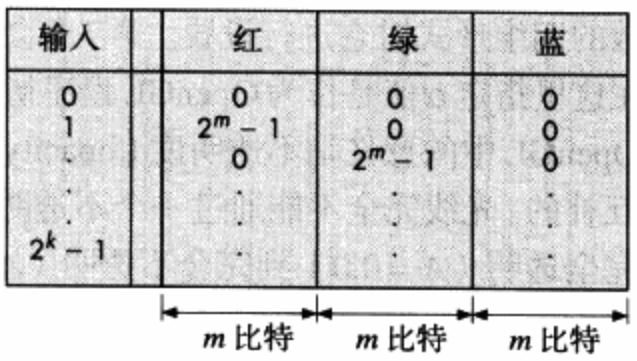


图2.25　查色表

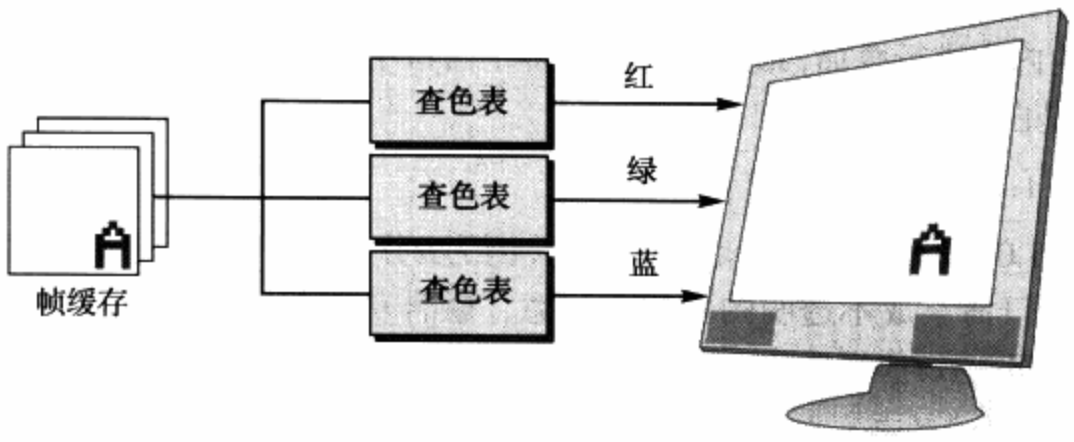


图2.26　索引颜色

如果使用索引颜色模式，下面这个函数可以指定当前颜色：

```
glIndexi(element);
```

该函数从查色表中选择了一个特定颜色。设置和修改查色表中的条目涉及与窗口系统的交互，第3章再来讨论如何与窗口系统打交道。如果窗口系统和底层硬件只支持有限种类的颜色，那么使用索引颜色会遇到一个困难：窗口系统可能只有一个查色表，所有的窗口都得使用这个查色表，于是为了使屏幕上的每个窗口都有各自的查色表，窗口系统需要经常更改查色表中的条目。GLUT允许用户通过下面的函数对每个窗口设置查色表中的条目：

```
glutSetColor(int color, GLfloat red, GLfloat green, GLfloat blue)
```

在历史上，索引颜色模式曾经是重要的，因为它对帧缓存要求的存储量更少，需要的其他硬件也更少。但是，现在这些设备已不再昂贵，而且索引颜色模式也带来了一些问题。当用户处理带明暗效果的动态图像时，索引颜色模式所能提供的颜色种类往往是不够的。此外，与窗口系统的交互也要比使用RGB颜色更复杂。因此，在绝大多数情况下，我们假定使用RGB颜色。

2.5.3　设置颜色属性

前面那个简单的示例程序使用的是RGB颜色。我们要设置三种属性。第一个是清屏的颜色，用下面的函数调用来设置：

```
glClearColor(1.0, 1.0, 1.0, 1.0);
```

可以通过下面的函数调用来把颜色状态变量设置为红色，因此点将用红色显示：

```
glColor3f(1.0, 0.0, 0.0);
```

通过下面的函数调用可以把点的大小设置成2像素宽：

```
glPointSize(2.0);
```

注意，像点大小和线宽度这样的属性是以像素为单位设置的。这样，如果两个显示器的像素大小不一样（由于其屏幕尺寸和分辨率不同），那么绘制的图像可能稍有不同。一些图形API为了确保相同的用户程序在所有系统上都生成相同的显示，按照与设备无关的方式来指定所有属性。遗憾的是，确保两个图形系统生成同样的显示是比较困难的。OpenGL兼顾理想的显示效果和现实约束，选择了更为实用的折中方案。

2.6　观察

现在可以把各种图形信息放到场景中去，并且可以描述我们希望这些对象以何种方式出现，但还没有一种方法来确切地指定这些对象中的哪些应该出现在屏幕上。正如在照片上所记录的内容依赖于把照相机对准何方以及使用什么样的透镜一样，在程序中必须做出相似的观察决策。

第1章介绍虚拟照相机模型时有一个基本概念，即在场景中指定对象与指定照相机完全独立。一旦把场景和照相机都确定好了，就可以生成图像。照相机通过曝光胶片生成图像，而计算机系统通过绘制流水线中一系列操作的执行来生成图像。应用程序只需要关心对象和照相机的参数设置，就像摄影师只关心得到的照片，而不关心快门如何工作以及胶片与光线之间发生的光化学作用的细节。

计算机生成图像有一些默认的观察条件，这类似于一个基本的透镜固定的照相机中有几挡设置。然而，为了使用位置固定、透镜无法调节的照相机来拍照，我们不得不改变场景。只有把大象放到离照相机足够远才可以对大象拍照，同样，只有把蚂蚁放到离照相机足够近才可以对蚂蚁拍照。我们更希望可以改变透镜，这种灵活性使得获取包含许多对象的图像变得更容易。在使用图形系统时情况也是如此。

2.6.1　正投影

正投影是最简单的，并且也是OpenGL中的默认设置。第5章会详细讨论这种投影和其他种类的投影，这里介绍正投影的目的是使读者可以开始编写三维程序。如果虚拟照相机模型中的照相机具有焦距无限大的远摄透镜，并且让照相机无限远离对象，就得到了数学上的正投影。如图2.27所示，可以通过固定图像平面并使照相机远离图像平面来近似实现这个效果。在极限情形下，所有的投影线彼此平行，而投影中心也被投影方向（direction of projection）所代替。

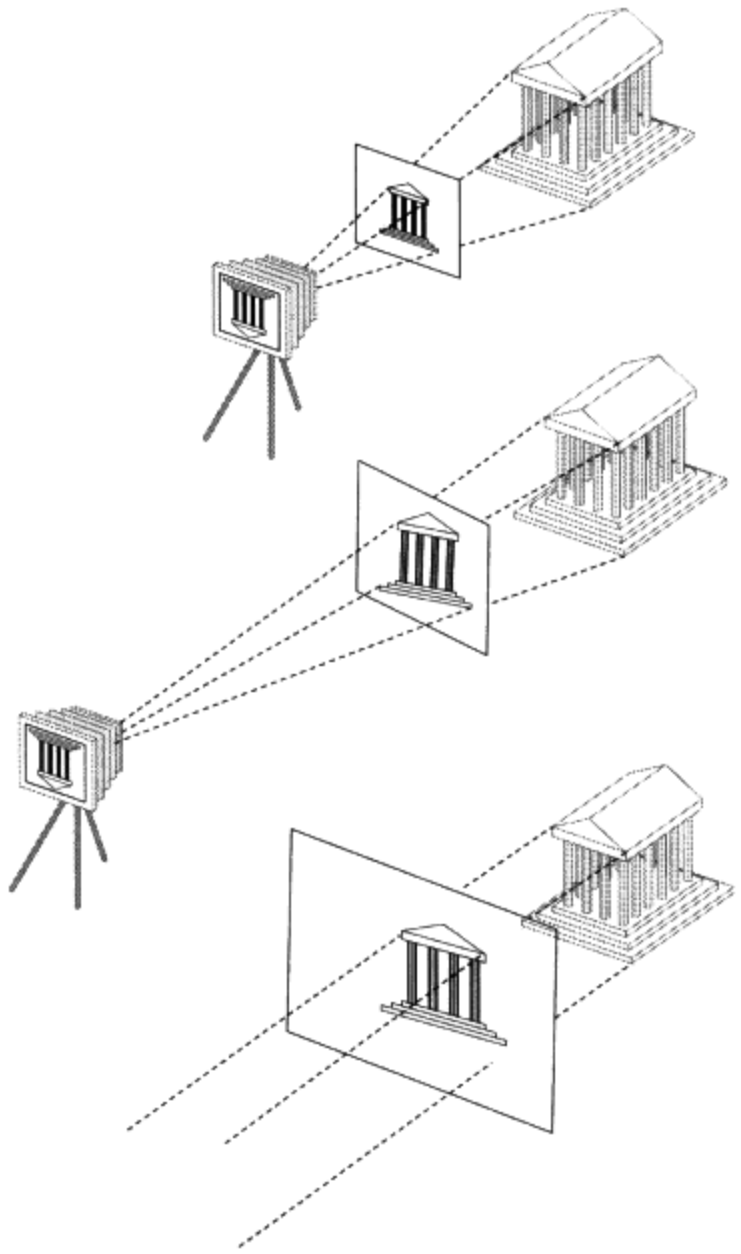


图 2.27 通过使照相机远离图像平面来构造正投影图

读者无须对照相机在无穷远处这一点感到不安。考虑如图 2.28 所示的投影线和投影平面，这里投影线平行于z轴正方向，投影平面是$z=0$。注意，不但投影线垂直或者说正交于投影平面，我们还可以沿z轴滑动投影平面，而不改变投影线和投影平面的交点。

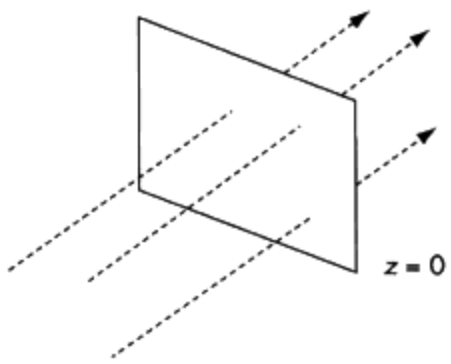


图 2.28 正投影的投影线和投影平面$z=0$

对于正投影，可以认为在投影平面内有一个特殊的正投影照相机，这对于其他类型的投影是不可能的。或许更精确地说：在投影平面内有一个参考点，以这个点为基准可以指定一个视见体和一个投影的方向。在 OpenGL 中，参考点在原点，照相机对准z轴的负向，如图 2.29 所示。正投影把点(x, y, z)投影到点$(x, y, 0)$，如图 2.30 所示。注意，如果场景是二维的，并且所有的顶点都在平面$z=0$内，那么一个点和对它进行投影之后得到的点相同。然而，我们仍然可以利用三维图形系统的机制来生成图像。在 OpenGL 中，可以通过下面的函数确定一个视见体为长方体的正投影。

```
void glOrtho(GLdouble left, GLdouble right, GLdouble bottom,
   GLdouble top, GLdouble near, GLdouble far)
```

注意，所有的参数都是从照相机出发测量的距离。正投影只“看到”了位于视见体内的对象。和真实的照相机不同，正投影可以对位于照相机后面的对象成像。这样，只要平面 $z = 0$ 位于近裁剪面和远裁剪面之间，它就与视见体相交。

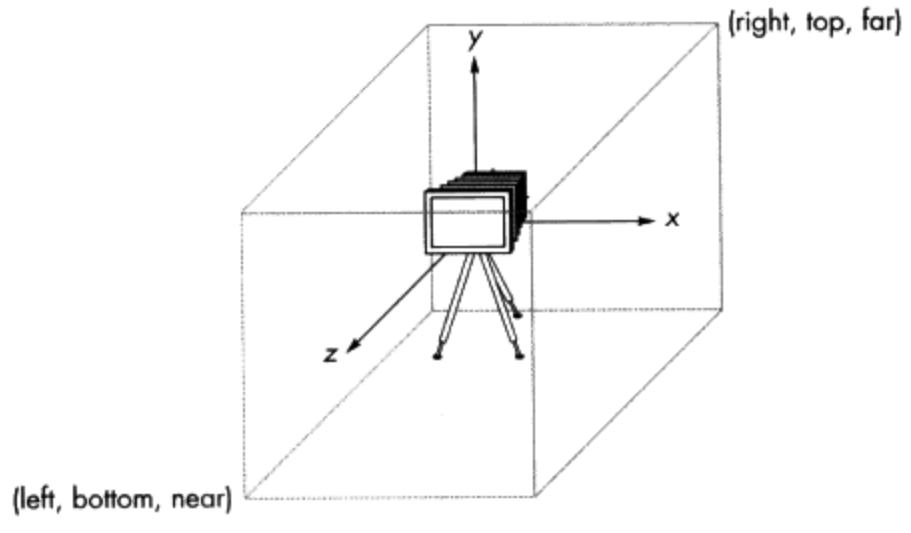


图 2.29　默认的照相机和一个正投影视见体

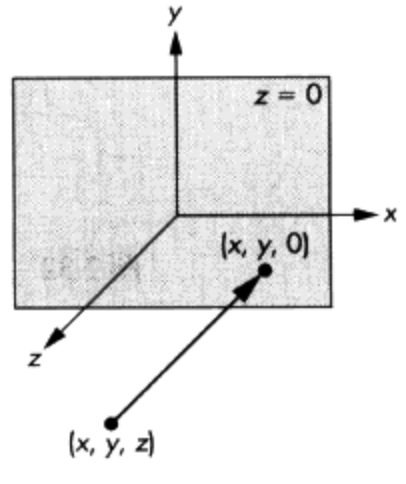


图 2.30　正投影

OpenGL 的默认视见体是一个中心位于原点的 2 × 2 × 2 立方体。平面 $z = 0$ 位于这个视见体内的部分是一个正方形，它的左下角在(–1.0, –1.0)，右上角在(1.0, 1.0)。

2.6.2　二维观察

按照观点来说，二维图形是三维图形的特殊情形。观察矩形是在三维视见体（viewing volume）内的平面 $z = 0$ 中，如图 2.31 所示。如果读者觉得在二维应用程序中使用三维视见体显得有些怪异，可以调用下面这个 GLU 库中的函数：

```
void gluOrtho2D(GLdouble left, GLdouble right,
   GLdouble bottom, GLdouble top)
```

它和读者的二维应用程序更一致。这个函数等价于在函数 glOrtho 中把参数 near 和 far 分别设置为 –1.0 和 1.0。第 4 章和第 5 章中将讨论如何移动照相机以及如何生成更复杂的投影图。

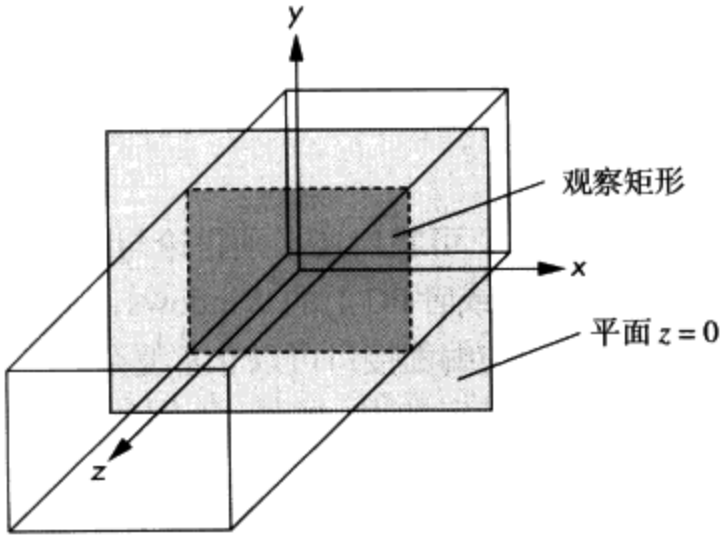


图 2.31　视见体

我们也可以用另一种更直接的方式考虑二维观察：从二维场景中选取一块矩形区域并把它的内容传递给显示器，如图 2.32 所示。场景中被成像的区域称为观察矩形（viewing rectangle），或者裁剪矩形（clipping rectangle）。位于这个矩形内的对象会出现在图像中，在矩形之外的对象被裁剪

掉（clipped out）并且不会被显示。跨越裁剪窗口边界的对象在图像中是部分可见的。在屏幕上显示图像的窗口与观察矩形是不同的，屏幕上窗口的尺寸和它在屏幕上的位置需要单独指定，我们将在2.7节再来讨论。

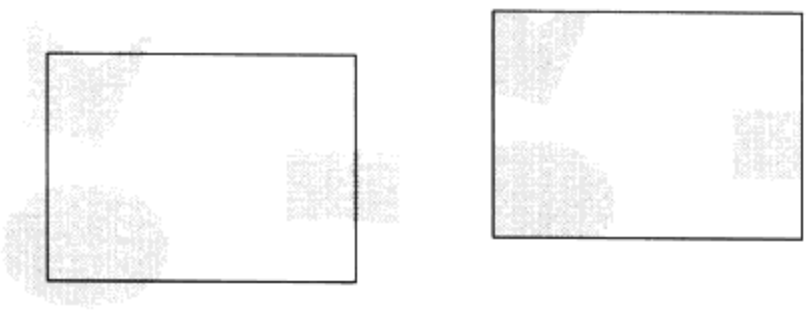


图2.32　二维观察：(a) 裁剪之前的图像；(b) 裁剪之后得到的图像

2.6.3　矩阵模式

为了对图元进行处理以获得想要的图像，采用流水线结构的图形系统需要把许多变换矩阵相乘，或者称为级联（concatenate）。像大多数其他的OpenGL变量一样，这些矩阵的值是系统状态的一部分，除非被修改，否则原来的值一直有效。两个最重要的矩阵是模－视（mode-view）变换矩阵和投影（projection）变换矩阵。在任何时候，系统状态都包括这两个矩阵的值，它们在初始时都是单位矩阵。第4章会介绍一组对这些矩阵进行操作的OpenGL函数。对矩阵的操作一般是应用一系列变换来修改初始的单位矩阵。只有单独一组函数可以应用到任何类型的矩阵上。我们通过设置矩阵模式（matrix mode）来选择要对哪个矩阵进行操作，矩阵模式这个变量也是系统状态的一部分，它可以被设置成几种矩阵模式中的一种。默认的矩阵模式是对模－视变换矩阵进行操作，因此为了修改投影变换矩阵，需要首先切换模式。下面的代码通常用来设置二维观察矩形：

```
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
gluOrtho2D(0.0, 50.0, 0.0, 50.0);
glMatrixMode(GL_MODELVIEW);
```

前三行代码定义的观察矩形大小为50.0 × 50.0，左下角位于二维坐标系的原点。最后一行代码又把矩阵模式切换回模－视模式。在复杂的程序中，为了明确任意时刻的矩阵模式，一个好的办法是在每次改变矩阵模式后都返回到一个约定的矩阵模式（这里是模－视模式）。

2.7　控制函数

现在，第一个示例程序几乎已经完成，可我们还必须讨论与窗口和操作系统的最低限度的交互。考虑Linux平台上的X Window System或者PC上的Windows，如果看看这些具体环境的细节，就会发现图形系统与窗口和操作系统之间的编程接口可以非常复杂。要充分利用这些环境对应用程序员提供的各种支持需要关于这些系统的特定知识。此外，不同的环境在细节上各有不同，对这些差异的讨论也不会增强人们对计算机图形学的理解。

与其详细讨论这些问题，不如从图形应用程序的角度来看看必须用到的最低限度的操作有哪些。前面介绍了OpenGL实用工具库（GLUT），这个函数库提供了图形系统与窗口和操作系统之间的一个简单接口。底层窗口系统或者操作系统特有的细节是具体实现应该关心的内容，而不是API的一部分。在实际编程时，只需把其他的函数库添加到标准库搜索路径中。在本章以及第3章，GLUT

将帮助读者理解现代交互式图形系统中API与操作系统、窗口系统之间的交互，这些交互刻画了现代交互式图形系统的特征。我们利用GLUT编写的应用程序在多种窗口系统下都能运行。

2.7.1　和窗口系统的交互

在有关图形学和工作站的文献中，术语窗口可以表示许多不同的意思。我们用窗口（window）或者屏幕窗口（screen window）来表示显示器上的一块矩形区域。我们只考虑光栅显示器。一个窗口有它的高度和宽度，并且由于窗口中显示的是帧缓存中的内容，窗口内的位置是用窗口坐标（window coordinate）或者屏幕坐标（screen coordinate）[①]来指定的，单位是像素。

在现代图形窗口环境中，可以在监视器上显示许多窗口。各个窗口的目的不同，从编辑文件到监视系统。术语窗口系统是指由诸如X Window System和Microsoft Windows这样的操作系统提供的多窗口环境。用于显示图形输出的窗口是窗口系统要管理的窗口之一。因此，窗口系统把图形窗口看做是一类特殊的窗口——可以显示或者绘制图形的窗口。在图形窗口中指定的位置是相对于窗口的某一角而言的，我们必须小心认定哪一个角是原点。在科学和工程中，左下角是原点，其窗口坐标为(0, 0)。然而，所有的光栅系统都和商业电视系统显示屏幕的方式一样：按照从上到下、从左到右的顺序进行扫描。从这个角度看，左上角应该是原点。OpenGL中的命令假定原点在左下角，可是从窗口系统返回的信息（比如鼠标的位置）往往假定原点在左上角，于是需要把窗口中的位置从一个坐标系转换到另一个坐标系。

给定了显示屏的分辨率，比如1280 × 1024，我们可以使用任意大小的窗口。帧缓存的分辨率必须和显示屏的相同。如果使用的窗口大小为300 × 400，那么可以在概念上把它和一个300 × 400的帧缓存相对应，尽管它只使用了实际帧缓存的一部分。

在能够打开一个窗口之前，窗口系统和OpenGL之间必须要进行交互。在GLUT中，这样的交互通过下面的函数调用来初始化：

```
glutInit(int *argcv, char **argv)
```

这个函数的两个参数允许用户传递命令行参数，就像在C语言的标准main函数中那样，而且这两个函数的参数通常是相同的。现在可以用下面的GLUT函数打开一个OpenGL窗口：

```
glutCreateWindow(char *title)
```

其中的字符串title是出现在窗口顶部的标题。

上面创建了一个窗口，它的大小、在屏幕上的位置以及像RGB颜色模式这样的特性都是默认的。我们也可以在窗口创建之前使用GLUT库中的函数来指定这些参数。例如，下面的代码在显示屏的左上角指定了一个640 × 480的窗口：

```
glutInitDisplayMode(GLUT_RGB | GLUT_DEPTH | GLUT_DOUBLE);
glutInitWindowSize(640, 480);
glutInitWindowPosition(0,0);
```

我们指定的是RGB颜色而不是索引颜色（GLUT_ INDEX），为隐藏面消除使用了深度缓存，还使用了双缓存而不是单缓存（GLUT_SINGLE）。默认的设置是RGB颜色、不进行隐藏面消除、单缓存，这些都是我们现在需要的。因此，我们不需要显式地指定这些选项，但对它们进行显式的指定可以使代码更清晰。注意，函数glutInitDisplayMode的参数是通过逻辑或结合起来的。

---

① 在OpenGL中，窗口坐标系是三维的而屏幕坐标系是二维的。这两个坐标系都以像素为单位，但窗口坐标保留了深度信息。

2.7.2　宽高比和视口

一个矩形的宽高比（aspect ratio）是这个矩形的宽度与高度之比。因为对象、观察和工作站窗口这三者的指定是彼此独立的，所以由 glOrtho 设置的观察矩形和由 glutInitWindowSize 设置的窗口可以具有不同的宽高比，这可能会导致不想要的结果。如图 2.33 所示，如果观察矩形和显示窗口的宽高比不同，屏幕上的对象就会变形。对象产生变形是由于在默认的操作模式下整个裁剪矩形被映射到显示窗口所致。为了把裁剪窗口中的全部内容映射到整个显示窗口，只能把前者的内容变形以使宽高比适应后者。如果保证裁剪矩形和显示窗口的宽高比相同，就可以避免出现这样的变形。

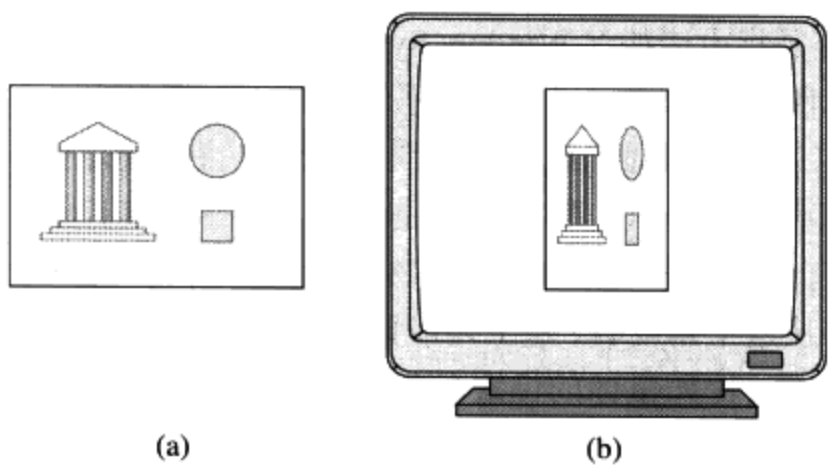


图 2.33　宽高比不匹配：(a) 观察矩形；(b) 显示窗口

另一种更灵活的方法是使用视口的概念。视口（viewport）是显示窗口中的一块矩形区域。默认情况下，视口是整个显示窗口，但可以通过下面的函数把它设置成任何更小的尺寸。

```
void glViewport(GLint x, GLint y, GLsizei w, GLsizei h)
```

这里，(x, y)是视口的左下角（相对于窗口的左下角测量），w 和 h 分别是视口的宽度和高度。这些参数都是整型数，允许用户以像素为单位指定位置和距离。图元显示在视口中，如图 2.34 所示。对一个给定的窗口，可以调节视口的高度和宽度来匹配裁剪矩形的宽高比，这样出现在图像中的对象就不会变形了。

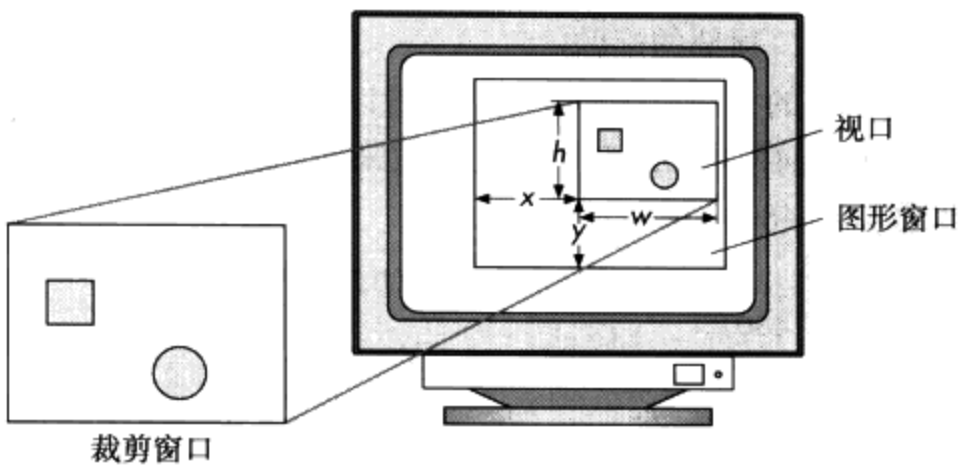


图 2.34　裁剪窗口到视口的映射

视口是状态的一部分。如果在绘制多个对象的中间改变视口或者用改变的视口重新绘制相同的对象，得到的效果是窗口的不同部分出现不同图像的多个视口。第 3 章将讲到视口进一步的应用，届时会考虑如何交互地改变窗口的形状和大小。

2.7.3　main、display 和 myinit 函数

原则上，把简单的初始化代码和2.1节中编写的代码组合起来，应该能够得到一个完整的生成Sierpinski镂垫的OpenGL程序。遗憾的是，在现代图形系统中，事情还不是这么简单。还要考虑两个问题：一个是所有的图形系统都要面临的问题；另一个与底层窗口系统的交互有更多的关系。

在立即模式下，图元一旦被指定，就尽可能快地被绘制到帧缓存中，系统使用当前的状态来确定图元的绘制方式。然后，程序继续执行下一条语句。在一个交互式程序中，我们会继续生成更多的图元。然而，像Sierpinski镂垫绘制程序这样的应用程序只是绘制一些图元然后就结束了。当应用程序结束后，在有机会看到输出以前，窗口可能会从显示器上消失。对于这个简单的程序，可以通过调用一个标准函数，比如sleep(enough_time)来插入一个延迟。然而，除了特别简单的程序，对任何应用程序都需要一种更复杂的机制。第3章将讨论事件处理，利用事件处理可以使程序具有更多的交互功能。目前可以使用下面的GLUT函数：

```
void glutMainLoop( );
```

这个函数的执行会使程序进入一个事件处理循环。如果没有事件需要处理，程序会处于等待状态，而图形仍然显示在屏幕上，直到通过某种外部方式来终止程序的执行，比如按下一个特殊键或者像Ctrl+C这样的组合键。

图形是通过一个称为显示回调（display callback）的函数发送到屏幕上的。这个函数通过下面的GLUT函数指定并且注册（register）到窗口系统：

```
void glutDisplayFunc(void (*func)(void))
```

只要窗口系统确定OpenGL窗口需要重新绘制，上面指定的func函数就要被调用，窗口第一次被打开时就是如此。因此，如果把所有指定图元的语句都放到这个函数中（对于我们这个非交互式程序可以这样做），func将被执行一次，Sierpinski镂垫就绘制出来了。虽然看起来对于显示回调函数的使用仅仅是为了组织程序的方便，但是这个显示回调函数却是GLUT所要求的。在其他情形下也会发生显示回调，例如，当窗口从屏幕的一个位置移动到另一个位置时，还有当OpenGL窗口前面的窗口被销毁使得整个OpenGL窗口都变成可见时。

下面的主程序可以用于任何非交互式的应用程序中：

```
#include <GL/glut.h>

void main(int argc, char **argv)
{

   glutInit(&argc,argv);
   glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);
   glutInitWindowSize(500,500);
   glutInitWindowPosition(0,0);
   glutCreateWindow("simple  OpenGL example");
   glutDisplayFunc(display);
   myinit();
   glutMainLoop();
}
```

这里用一个初始化函数myinit()[①]来设置OpenGL的观察和属性状态变量，对这些参数我们更愿意在显示函数之外一次性地设置好。程序在函数定义之前要求把GLUT库的头文件（glut.h）包含进来。在大多数编译器中，预编译指令

① 为了避免混淆，我们使用与 *OpenGL Programming Guide*[Ope07]和GLUT文档[Kil94a]中相同的函数名。

```
#include <GL/glut.h>
```

会把GLUT库的头文件、OpenGL库的头文件（gl.h）和OpenGL工具库的头文件（glu.h）都添加进来。例如，GL_LINES和GL_RGB等符号常量的宏定义就在这些文件中。

2.7.4　程序结构

我们以后将编写的每一个程序都和这个Sierpinski镂垫绘制程序的结构相同。我们总是使用GLUT工具包。main函数将调用GLUT函数来设置绘图窗口并确保窗口环境支持所要求的显示属性，main函数也将指定所需的回调和回调函数。每一个程序都必须有一个display回调函数，为了使程序具有交互功能还必须有其他的回调函数。myinit函数将设置用户的选项，通常这需要调用GL库和GLU库中的OpenGL函数。虽然这些选项也可以在main函数中设置，但把GLUT函数和OpenGL函数分开会更清楚。对于大多数程序，图形输出将在display回调函数中生成。

2.8　Sierpinski镂垫绘制程序

现在可以编写myinit和display函数了，再加上前面的main函数就可以完成绘制Sierpinski镂垫的程序了。我们在白色背景上绘制红色的点。我们设置一个二维坐标系，要绘制的点落在50.0 × 50.0的正方形中，正方形的左下角位于原点。这里使用点的二维表示。

下面是这两个函数的代码：

```
void myinit()
{
/* 设置属性 */
   glClearColor(1.0, 1.0, 1.0, 1.0); /* 白色的背景 */
   glColor3f(1.0, 0.0, 0.0); /* 用红色来绘制 */

/* 设置观察参数 */

   glMatrixMode(GL_PROJECTION);
   glLoadIdentity();
   gluOrtho2D(0.0, 50.0, 0.0, 50.0);
   glMatrixMode(GL_MODELVIEW);
}

void display()
{
   GLfloat vertices[3][2]={{0.0,0.0},{25.0,50.0},{50.0,0.0}};
        /* 三角形 */
   int i, j, k;
   int rand(); /*标准的随机数生成器 */
   GLfloat p[2] ={7.5 , 5.0}; /*三角形内的
                                任意一点*/

   glClear(GL_COLOR_BUFFER_BIT);  /* 清空窗口 */

  /* 计算并输出5000个点 */

   glBegin(GL_POINTS);

   for( k=0; k<5000; k++)
   {
```

```
        j=rand()%3; /* 随机挑选一个顶点 */

    /* 计算顶点与上一个点之间连线的中点*/

        p[0] = (p[0]+vertices[j][0])/2.0;
        p[1] = (p[1]+vertices[j][1])/2.0;
    /* 绘制点 */

        glVertex2fv(p);
      }
      glEnd();
      glFlush();
}
```

显示函数中定义了三个任意的顶点，并且在由这三个顶点定义的三角形内定义了一个任意的初始点。请读者注意，在 glOrtho 中选取的参数使得裁剪窗口刚刚可以包含三角形，而且显示器上的窗口和裁剪窗口这两者的大小并没有联系，尽管通过把它们都设置成正方形可以使绘制的Sierpinski镂垫没有变形。程序中有一个对 OpenGL 函数 glFlush 的调用，它迫使系统尽可能快地在显示器上绘制那些点。读者可以在附录A中找到这个程序的完整清单，在那里还有将在后面章节中编写的其他示例程序。

2.9　多边形和递归

Sierpinski 镂垫程序绘制的图形（见图 2.2）显示出了相当精细的结构。如果让程序迭代更多的次数，那么图像中大多数的随机性都会消失。检查这个结构发现：不管生成多少个点，在三角形的中心区域是没有点的。如果把初始的三角形各边的中点连接起来，就把它分成了 4 个三角形，中间的那个三角形不包含绘制的点（见图 2.35）。

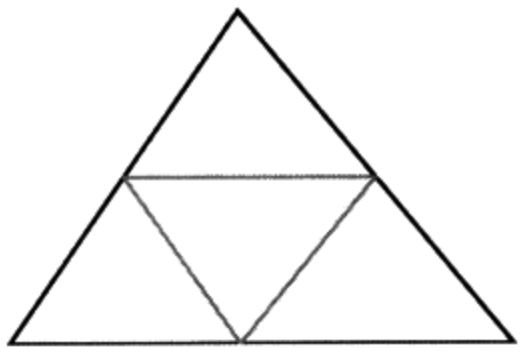


图 2.35　连接各边的中点把三角形分成 4 部分

看看其他三个三角形，发现每一个都是如此。也就是说，可以通过连接各边的中点把每个三角形分成 4 个三角形，每个中间的三角形也都不会包含绘制的点。

由此可以得出生成 Sierpinski 镂垫的第二种方法，这种方法使用多边形而不是点，而且也不需要使用随机数生成器。使用多边形的优点之一是可以在显示器上填充实体区域。策略是从一个三角形开始，通过平分各边把它分割成 4 个更小的三角形，然后去掉中间的小三角形不再考虑。重复进行这个步骤，直到要去掉的三角形尺寸小到大约一个像素，这使得我们可以绘制余下的三角形。

可以用一个递归程序实现上面描述的这个过程。从一个简单的函数开始，这个函数由给定的三个任意顶点绘制出一个三角形：

```
void triangle(GLfloat *a, GLfloat *b, GLfloat *c)
{
      glVertex2fv(a);
      glVertex2fv(b);
      glVertex2fv(c);
}
```

假定初始三角形的顶点由下列数组给出：

```
GLfloat v[3][2];
```

然后各边的中点由数组m[3][2]给出，它可以用下面的代码来计算：

```
for(j=0; j<2; j++) m[0][j]=(v[0][j]+v[1][j])/2.0;
for(j=0; j<2; j++) m[1][j]=(v[0][j]+v[2][j])/2.0;
for(j=0; j<2; j++) m[2][j]=(v[1][j]+v[2][j])/2.0;
```

有了这6个位置，即可调用triangle函数绘制三个三角形：(v[0], m[0], m[1]),(v[2], m[1], m[2]), (v[1], m[2], m[0])。我们并不是想简单地绘制这些三角形，而是想对它们进行细分。因此，我们需要递归调用函数自身，为此定义下面的递归函数：

```
divide_triangle(float *a, float *b, float *c, int k)
```

这个函数只有在k为零时才会绘制三角形，否则它将细分由a、b和c确定的三角形并把k的值减1。该函数的代码如下：

```
void divide_triangle(GLfloat *a, GLfloat *b, GLfloat *c, int k)
{
    GLfloat ab[2], ac[2], bc[2];
    int j;
    if(k>0)
    {
  /* 计算各边的中点 */

        for(j=0; j<2; j++) ab[j]=(a[j]+b[j])/2;
        for(j=0; j<2; j++) ac[j]=(a[j]+c[j])/2;
        for(j=0; j<2; j++) bc[j]=(b[j]+c[j])/2;

  /* 继续细分小三角形，除了中间的那个小三角形 */

        divide_triangle(a, ab, ac, k-1);
        divide_triangle(c, ac, bc, k-1);
        divide_triangle(b, bc, ab, k-1);
    }
     else triangle(a,b,c); /* 在递归结束时绘制三角形 */

}
```

至此，编写显示函数就很容易了。显示函数要调用一次divide_triangle，并利用由主程序指定的一个全局变量n的值来确定细分次数，下面是该函数的代码：

```
void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glBegin(GL_TRIANGLES);
         divide_triangle(v[0], v[1], v[2], n);
    glEnd();
    glFlush();
}
```

注意，这里把单独一对 glBegin()和 glEnd()放到了显示回调函数中，尽管把 glBegin()和 glEnd()放到 triangle 函数里会更直观。之所以能这么做是因为我们使用的是GL_TRIANGLES类型，这种类型和 GL_POLYGON类型不一样，每有三个顶点就可以生成一个新的三角形图元。通过这种方式可以使程序的效率提高很多。

除了需要读入n的值以外，这个程序的其余部分和前面的Sierpinski镂垫绘制程序都一样。5次细分之后的输出如图 2.36 所示。读者可以在附录 A 中找到完整的程序。

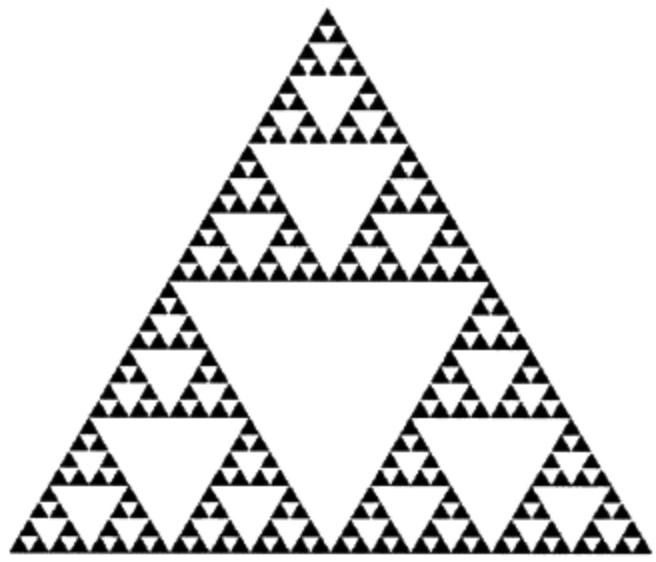


图 2.36　5 次细分后的三角形

2.10　三维 Sierpinski 镂垫

前面已经说明了二维图形是三维图形的特殊情形，但还没有看到一个完整的三维程序。下面把前面的二维Sierpinski程序转换成一个生成三维Sierpinski镂垫的程序，也就是说，要绘制的镂垫不再只是限制在一个平面里。我们可以仿效对二维镂垫所使用的两种方法中的任何一种。这两种方法的三维扩展都以相似的方式开始，即用一个四面体代替初始的三角形（见图 2.37）。

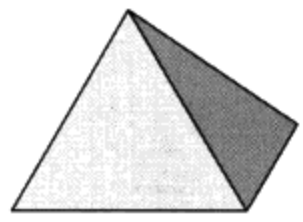


图 2.37　四面体

2.10.1　使用三维点

因为每个四面体都是凸的，所以一个顶点和四面体内的任何一点之间连线的中点也在四面体内。因此，可以仿效和前面一样的步骤，不过这一次需要用4个初始顶点来定义四面体，而不是用三个顶点定义三角形。注意，可以随机地选择四面体的4个顶点而不影响结果的特征，只要任何三个顶点都不共线即可。

需要修改的主要是函数 display。我们定义并初始化一个数组来存放顶点，代码如下：

```
                    /* 任意一个四面体的顶点 */
GLfloat vertices[4][3]={{0.0,0.0,0.0},{25.0,50.0,10.0},
                        {50.0,25.0,25.0},{25.0,10.0,25.0}};

                  /* 把四面体内任意一点作为初始位置 */
GLfloat p[3] ={25.0,10.0,25.0};
```

现在用函数glVertex3fv来定义点。在绘制三维镂垫时会遇到一个二维镂垫不会有的问题：因为所绘制的点不再只限于一个平面内，所以从显示出来的二维图像要看出三维结构可能比较困难。为了使绘制出的图像能体现出一些三维信息，我们添加了一个颜色函数，它使每个点的颜色由其位置来决定。下面是显示函数：

```
void display()
{
/* 计算并绘制一个新的位置 */

  int rand();
  int i;
  j=rand()%4; /* 随机挑选一个顶点 */

/* 计算顶点和上一个点之间连线的中点 */
  p[0] = (p[0]+vertices[j][0])/2.0;
  p[1] = (p[1]+vertices[j][1])/2.0;
  p[2] = (p[2]+vertices[j][2])/2.0;

/* 绘制点 */

  glBegin(GL_POINTS);
   glColor3f(p[0]/250.0,p[1]/250.0,p[2]/250.0);
   glVertex3fv( p );
  glEnd();

  glFlush();
}
```

现在我们是在三维空间中讨论问题了，所以用下面的函数定义一个三维裁剪体（在main.c中）：

```
glOrtho(-50.0 , 50.0, -50.0, 50.0 ,-50.0, 50.0);
```

如图2.38和彩图1所示，如果生成的点足够多，那么得到的图形看起来就像是在初始的四面体中不断地去掉越来越小的四面体之后剩下的部分。

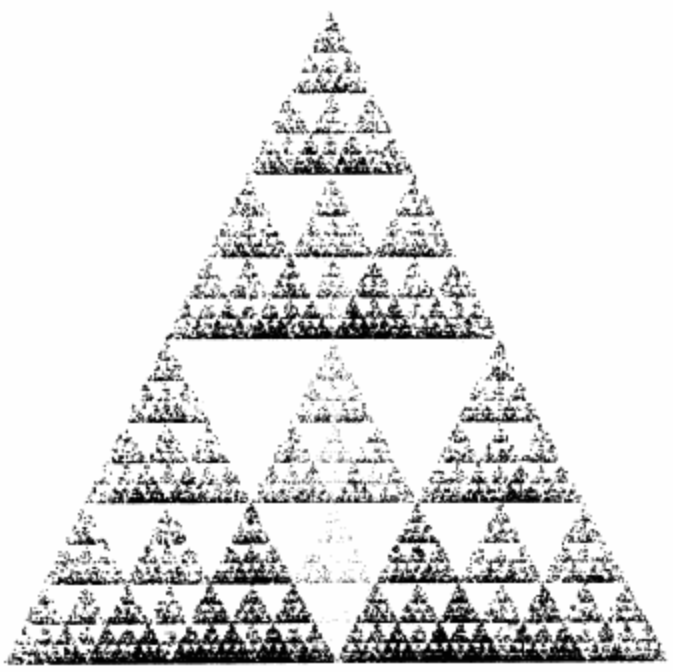


图2.38　三维Sierpinski镂垫

2.10.2　在三维空间中使用多边形

通过使用多边形和把四面体细分成更小的四面体可以得出一种更有趣的方法：假定从一个四面体开始，找出其 6 个棱的中点并连接起来，如图 2.39 所示。这样，在原来四面体的 4 个顶点处各有一个小四面体，原来四面体中剩下的部分应当去掉。

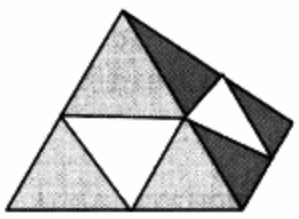


图 2.39　细分后的四面体

仿效生成二维Sierpinski镂垫的第二种方法，我们将对保留的那4个小四面体进行迭代细分。因为四面体的 4 个面是由其 4 个顶点决定的 4 个三角形，在细分结束时，可以通过绘制 4 个三角形来绘制每一个剩下的四面体。

大部分代码几乎和二维的情形相同。绘制三角形的函数现在使用三维点而不是二维点：

```
void triangle(GLfloat *a, GLfloat *b, GLfloat *c)
{
       glVertex3fv(a);
       glVertex3fv(b);
       glVertex3fv(c);
}
```

我们用下面的函数来绘制四面体，四面体的每个面被绘制成不同的颜色：

```
void tetra(GLfloat *a, GLfloat *b, GLfloat *c, GLfloat *d)
{
       glColor3fv(colors[0]);
       triangle(a, b, c);
       glColor3fv(colors[1]);
       triangle(a, c, d);
       glColor3fv(colors[2]);
       triangle(a, d, b);
       glColor3fv(colors[3]);
       triangle(b, d, c);
}
```

细分四面体的方式和细分三角形相似。

下面的函数和 divide_triangle 完成相似的任务：

```
void divide_tetra(GLfloat *a, GLfloat *b, GLfloat *c, GLfloat *d, int m)
{

    GLfloat mid[6][3];
    int j;
    if(m>0)
    {
        /* 计算6个中点 */

        for(j=0; j<3; j++) mid[0][j]=(a[j]+b[j])/2;
        for(j=0; j<3; j++) mid[1][j]=(a[j]+c[j])/2;
        for(j=0; j<3; j++) mid[2][j]=(a[j]+d[j])/2;
        for(j=0; j<3; j++) mid[3][j]=(b[j]+c[j])/2;
```

```
        for(j=0; j<3; j++) mid[4][j]=(c[j]+d[j])/2;
        for(j=0; j<3; j++) mid[5][j]=(b[j]+d[j])/2;

        /* 通过细分生成4个四面体 */

        divide_tetra(a, mid[0], mid[1], mid[2], m-1);
        divide_tetra(mid[0], b, mid[3], mid[5], m-1);
        divide_tetra(mid[1], mid[3], c, mid[4], m-1);
        divide_tetra(mid[2], mid[4], d, mid[5], m-1);

    }
    else(tetra(a,b,c,d)); /* 在递归结束时绘制四面体 */
}
```

现在可以在显示回调函数中从4个顶点开始进行n次细分，代码如下：

```
{
    glClear(GL_COLOR_BUFFER_BIT);
    glBegin(GL_TRIANGLES);
    divide_tetra(v[0], v[1], v[2], v[3], n);
    glEnd();
    glFlush();
}
```

在得到一个有用的三维程序之前，我们只剩下最后一个问题还需要考虑。

2.10.3　隐藏面消除

如果执行前面已经编写出的代码，得到的结果会使人迷惑。程序按照三角形被指定的顺序来绘制它们，这个顺序由程序中的递归过程来决定，而不是由三角形之间的几何关系来决定。每个三角形都用单色绘制（填充）而且覆盖了那些已经绘制在显示器上的三角形。

假设用小的实心四面体来构造三维Sierpinski镂垫，那么把此时这些三角形的显示方式和上面绘制三角形的顺序进行对比，就会发现只有在所有其他三角形前面的三角形才会被观察者看到。图2.40所示是这个隐藏面（hidden-surface）问题的一个简单情形。从观察者的位置，可以清楚地看到四边形A，但是三角形B被挡住，而三角形C只是部分可见。无须深入任何具体算法的细节，读者就能理解：给定了观察者和三角形的位置，就应该能合理地绘制三角形以得到正确的图像。对对象进行排序以正确绘制它们的算法称为可见表面算法（visible-surface algorithm），也可以称为隐藏面消除算法（hidden-surface-removal algorithm），这要看我们从什么角度来说。第4章和第7章将详细讨论这些算法。

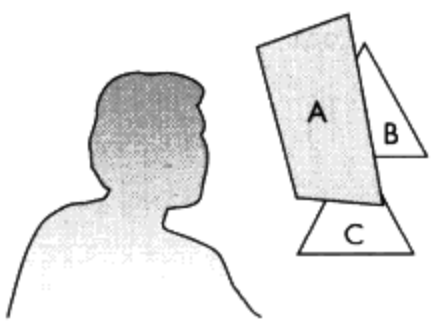


图2.40　隐藏面问题

对于这个例子，可以使用一种特定的隐藏面消除算法，这个算法称为z-缓存（z-buffer）算法，OpenGL也提供了对它的支持。开启或者关闭这个算法很容易。在主程序里必须申请辅助存储器：一个z（深度）缓存，为此把初始化显示模式的代码修改如下：

```
glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB | GLUT_DEPTH);
```

在递归结束时绘制四面体注意z缓存是构成帧缓存的众多缓存之一。我们通过下面的函数调用来启用这个算法：

```
glEnable(GL_DEPTH_TEST)
```

这个函数调用可以放在 main.c 中，也可以放在像 myinit.c 这样的初始化函数中。因为算法要在深度缓存中存储信息，所以必须在需要重绘窗口时清空这个缓存。为此，我们把显示函数中的清屏操作修改为：

```
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
```

显示回调函数如下：

```
void display()
{
  glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
  glBeginGL_TRIANGLES;
       divide_tetra(v[0], v[1], v[2], v[3], n);
  glEnd();
  glFlush();
}
```

迭代 4 次的结果如图 2.41 和彩图 2 所示。读者在附录 A 中可以找到完整的程序。

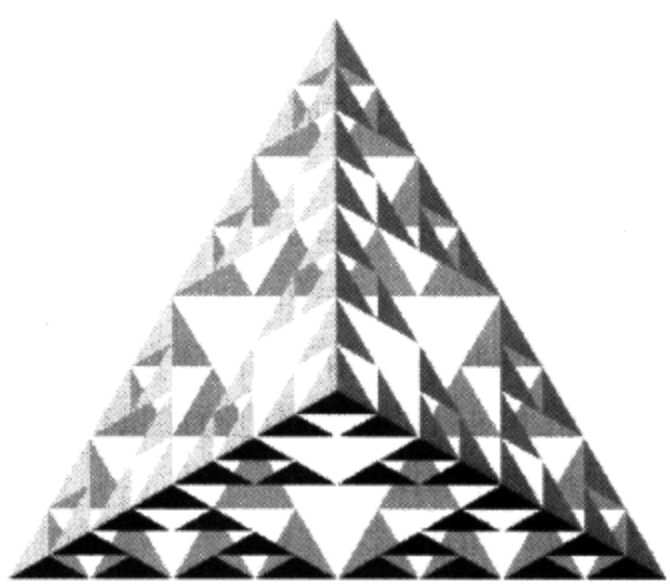


图 2.41　迭代 4 次后生成的三维 Sierpinski 镂垫

2.11　绘制隐函数曲线

在进入下一章学习交互式图形学之前[①]，让我们考虑一个不平凡的例子，这个例子演示了数据可视化，从中可以看到二维图形应用程序可以相当复杂。这里考虑的问题是如何显示由隐函数（implicit function）所定义的数据，隐函数的形式为：

$$g(x, y) = 0$$

其中，函数 g 的解析式已知，比如中心在原点的单位圆的方程：

$$g(x, y) = x^2 + y^2 - 1 = 0$$

显示如下形式的显函数（explicit function）：

① 在初次阅读时可以跳过这一节。

$$y = h(x)$$

再直截了当不过了，因为我们可以计算一组x值所对应的y值，然后再编写一个简单的程序把连接这些点(x, y)的线段显示出来。然而，对于隐函数，既不能对给定的x求出相应的y，也不能一般地断言这样的一个y值是否存在。例如，视a和b的值而定，隐函数方程

$$g(x, y) = (x^2 + y^2 + a^2)^2 - 4a^2x^2 - b^4$$

可以生成零条、一条或者两条闭曲线，该曲线称为Cassini卵形线。

这个问题的更一般形式是显示下面的函数：

$$z = f(x, y)$$

其中，f是某个表面上给定位置处的高度，比如地球表面的高度。我们经常想得到和一组固定的z值对应的等高线（contour curve）。这样，对于$z = c$，就要求解隐函数方程：

$$g(x, y) = f(x, y) - c = 0$$

这样的曲线我们经常见到，它与徒步旅行者和地质学者使用的地形图类似。

现在回到一开始的问题上来，我们不去试图求解一个隐函数方程，而是先计算g的值（采样）。一些采样值会大于零，而另一些会小于零。问题是从这些采样值中找出函数g的一个近似。我们利用一种称为步进方格（marching squares）的技术来求解这个问题，而且还将导出用于绘制等高线的方法。

2.11.1　步进方格算法

假定在矩形阵列（点阵或网格）上沿x和y方向的等间隔点对函数$f(x, y)$采样，采样值的集合为$\{f_{ij} = f(x_i, y_j)\}$，其中，

$$x_i = x_0 + i\Delta x, \quad i = 0, 1, \cdots, N-1$$
$$y_j = y_0 + j\Delta y, \quad j = 0, 1, \cdots, M-1$$

这里，Δx和Δy分别是在x和y方向采样的间隔。相等的间隔只是为了简化推导，并不是必需的。与此等价的是：可以通过在规则的网格上对某个物理量进行测量得到$N \times M$个采样值，这些采样值也可以直接来自激光测距仪或者卫星这样的设备。

假定希望对一个特定的z值找到下面隐函数曲线的一个近似：

$$z = f(x, y)$$

这里用c来表示这个特定的z值，可以把它看做高度值。对一个给定的c，可能没有等高线，可能有一条等高线，也可能有多条等高线。如果考虑的是采样数据，那么只能求出近似的等高线。构造近似等高线的策略是生成一条分段线性曲线（piecewise linear curve），它由连接起来的线段构成。考虑如图2.42所示的矩形单元（cell），这个矩形单元由4个网格点(x_i, y_j)、(x_{i+1}, y_j)、(x_{i+1}, y_{j+1})、(x_i, y_{j+1})确定。算法逐个单元地找出构成分段线性曲线的那些线段，在考察一个单元时只利用单元四角处的z值来确定等高线是否经过该单元。

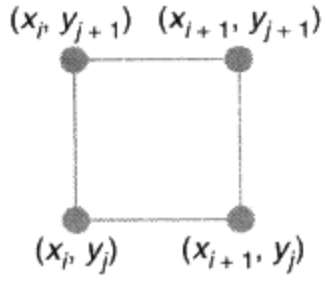


图2.42　矩形单元

通常，单元四角处的采样值不等于高度值。然而，等高线仍有可能穿过这个单元。考虑一种简单的情形，4 个角中只有一个角的采样值大于 c，其他角的采样值都小于 c：

$$f_{i,j} > c$$
$$f_{i+1,j} < c$$
$$f_{i+1,j+1} < c$$
$$f_{i,j+1} < c$$

我们把这种情形示于图2.43，其中图2.43(a)显示出了4个顶点的值相对 c 的大小关系，图2.43(b)把值小于 c 的顶点用黑色表示，把值大于 c 的顶点用白色表示。通过观察，可以发现如果生成这些数据的函数 f 具有相当好的性质，那么等高线必然穿过一个顶点是黑色、另一个顶点是白色的那两条边。换句话说，如果函数 $f(x, y) - c$ 在一个顶点处大于零，在相邻顶点处小于零，那么它在中间某处必然等于零，这种情形如图 2.44(a)所示。

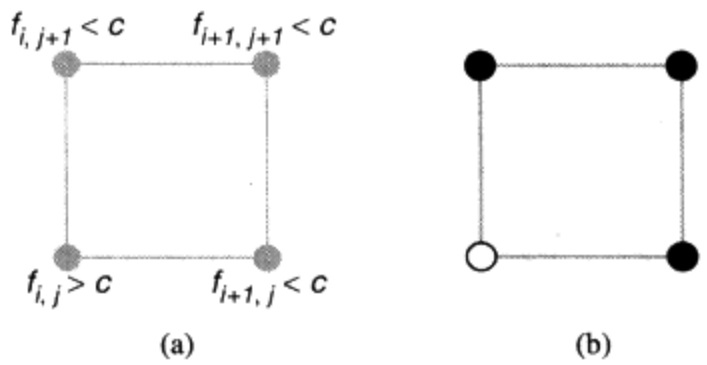


图 2.43　标记单元的顶点：(a) 对顶点阈值化；(b) 对顶点染色

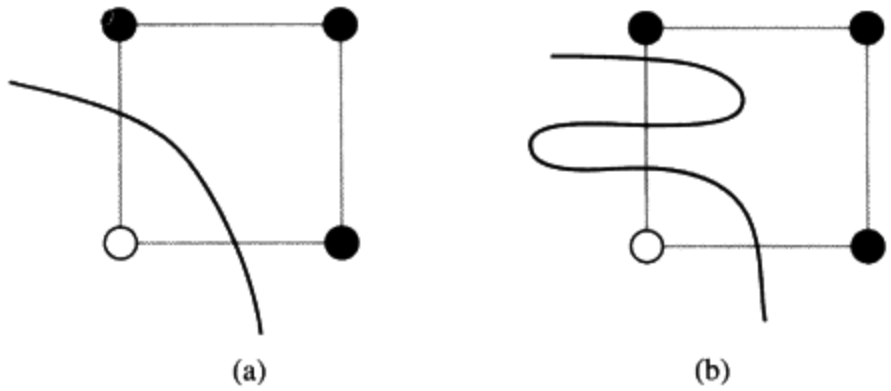


图 2.44　等高线穿过单元的边界：(a) 穿过一次；(b) 穿过多次

图 2.44(a)所示只是等高线穿过单元边界的一种可能的方式。如图 2.44(b)所示，等高线也可能穿过单元边界三次，还可能穿过任意奇数次。这些解释中的每一个都与采样数据相符。我们将总是使用这样的解释：对于光滑函数，等高线更有可能只穿过边界一次，而不是穿过多次。我们从多种解释中做出的这个选择是 Occam 剃刀原理（Principle of Occam's Razor）的一个例子，这个原理是说：如果对一个现象有多种可能的解释都和数据相符，那么选择其中最简单的解释。

继续考虑示例单元，如果能够估计出等高线在何处与两条边界相交，那么就能用一条线段把这两个交点连接起来。我们甚至可以立刻就把这条线段画出来，因为对其他单元的计算与对这个单元所做的无关。但是我们把交点放在哪里呢？有两个简单的策略。可以简单地认为交点在黑顶点和白顶点的正中间。可是，如果 f_{ij} 只比 c 大一点而 $f_{i+1,j}$ 比 c 小很多，那么我们会期望等高线和单元边界的交点到 (x_i, y_j) 的距离要比到 (x_{i+1}, y_j) 的距离更小。更复杂的策略使用插值。假定在单元的两个顶点中，一个顶点的值比 c 大，另一个顶点的值比 c 小。例如：

$$f(x_i, y_j) = a, \quad a > c$$
$$f(x_{i+1}, y_j) = b, \quad b < c$$

如果两个顶点在 x 方向的间隔为 Δx，那么可以利用一条线段来插值出交点，如图 2.45 所示。

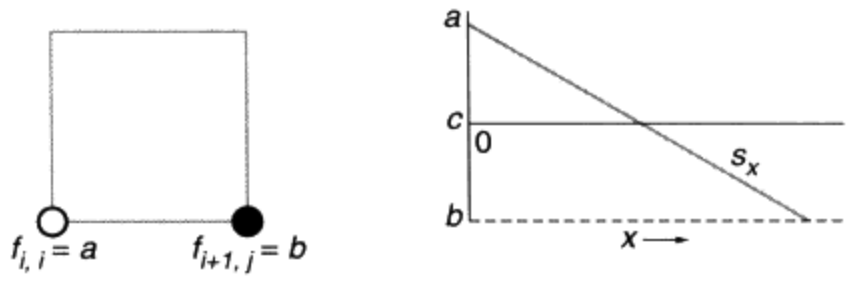


图 2.45　对单元边界与等高线交点的插值

这条线段和 x 轴相交于

$$x = x_i + \frac{(a-c)\Delta x}{a-b}$$

在这个单元里用一条线段来近似等高线，我们把上面插值出的点作为这条线段的一个端点。对等高线和单元的另一条边界的交点也按照类似的方法来计算。

到目前为止，讨论只限于一种特殊的单元，这种单元的一个顶点被染成白色，其他顶点被染成黑色。如果只使用黑色和白色，那么总共有 16（即 2^4）种对单元顶点进行染色的方式。所有这些情形在等高线问题中都可能出现。图2.46所示的是所有这些情形以及在每种情形下与数据相符合的近似线段，这里的近似线段是对数据的一种简单解释。按从左到右、从上到下的顺序把这些情形编号为 0 ~ 15。如果研究这些情形，就会发现有两种对称性：一种是旋转对称性。有些情形能够通过旋转互相转换，比如第 1 种和第 2 种情形，这些情形都只有一条近似线段，这条近似线段把一个顶点和其他顶点分隔开。对第 0 种和第 15 种情形，可以通过把顶点的颜色黑白颠倒来从一种情形得到另一种情形，这是另一种对称性。一旦考虑了对称性，真正不同的情形就只有 4 种了。这 4 种情形如图 2.47 所示。这样，我们只需要能够对 4 种情形绘制近似线段的代码，而其他的情形都可以变换成这 4 种情形之一。

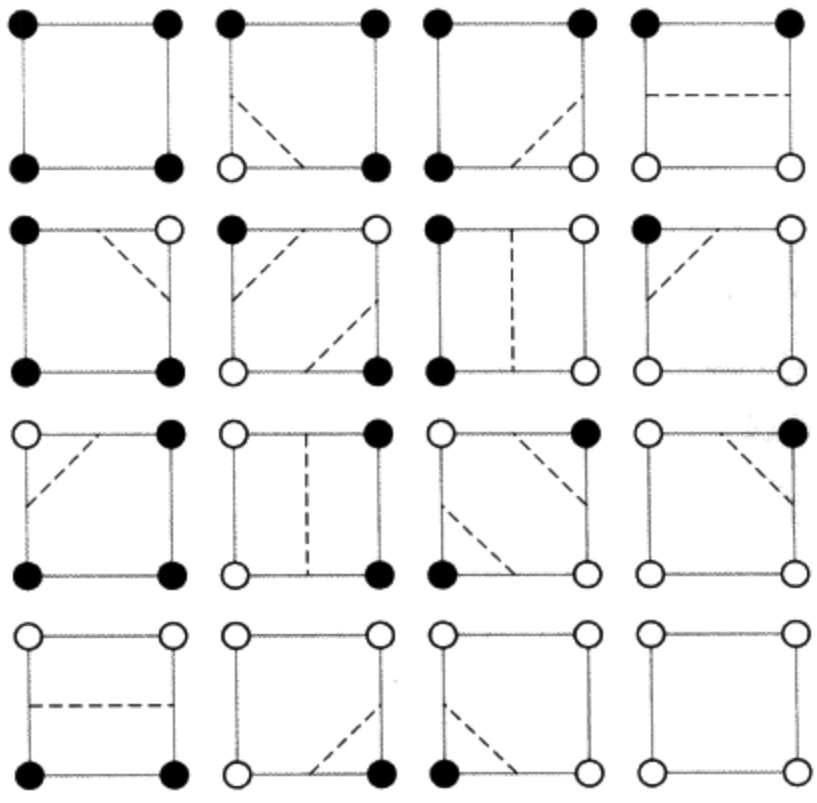


图 2.46　顶点标记和等高线的 16 种情形

第1种情形无须考虑，因为最简单的解释是等高线不经过这个单元，因此不用画近似线段；第2种情形是刚讨论过的，此时有一条近似线段；第3种情形也是简单的：我们把近似线段从一条边界画到和这条边界相对的另一条边界。

最后一种情形更困难也更有趣，因为它有歧义。如图2.48所示，有两个同样简单的解释，我们必须决定使用哪一个。如果没有其他的信息，那么没有理由更倾向于某一个，这时可以从两个解释里随机挑选一个。另一种办法是总是固定使用某一个解释。但是如图2.49所示，选择不同的解释会得出不同的结果。还有一种可能的办法是在单元中心生成一个新的数据点，把单元细分成4个更小的单元，如图2.50所示。这个新数据点的值可以由函数的解析式求出来，也可以通过对原来单元的4个角的值取平均。这4个小的单元可望不再是歧义的情形。如果有任何一个小单元有歧义，则可以进一步细分。

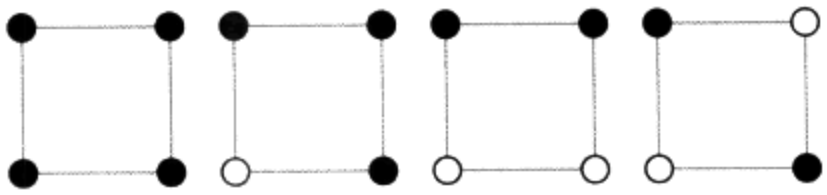


图2.47　顶点标记的4种不同情形

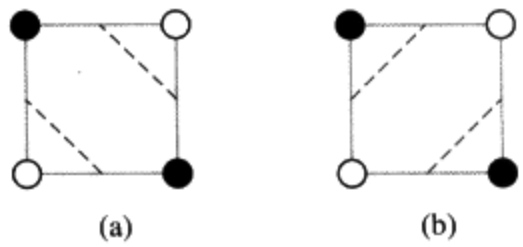


图2.48　解释的歧义性

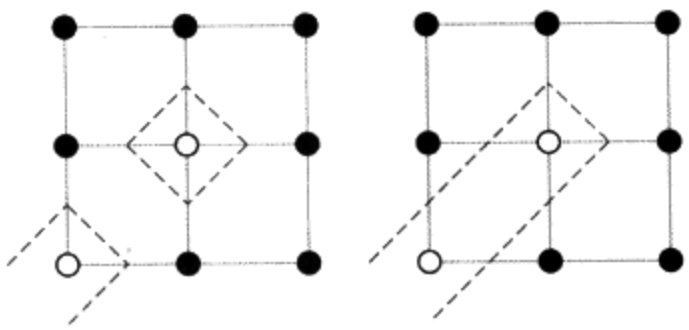


图2.49　标记相同、等高线却不同的一个例子

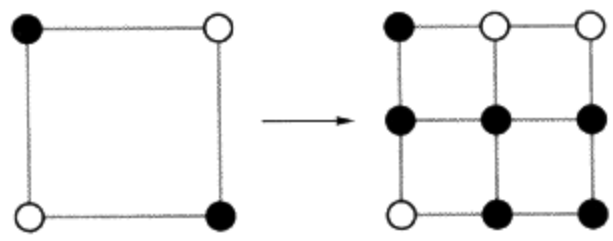


图2.50　单元的细分

生成等高线的代码相当简单，只需把相同的算法应用到所有单元即可。假定单元的4个顶点的值分别为a、b、c和d。下面的代码计算一个单元的类型编号：

```
int cell(double a, double b, double c , double d)
{
/* THRESHOLD = 高度值 */
    int n=0;
    if(a>THRESHOLD) n+=1;
    if(b>THRESHOLD) n+=8;
    if(c>THRESHOLD) n+=4;
    if(d>THRESHOLD) n+=2;
    return n;
}
```

接下来把单元归入4种规范情形之一，代码如下：

```
switch(n)
{
  case 1: case 2: case 4: case 7: case 8:
                  case 11: case 13: case 14:
   /* 等高线把一个顶点和其他顶点分隔开 */
   draw_one(num, i,j,a,b,c,d);
   break;
  case 3: case 6: case 9: case 12: /* 等高线的两侧各有两个顶点 */
```

```
    draw_adjacent(num,i,j,a,b,c,d);
    break;
  case 5: case 10: /* 有歧义的情形 */
    draw_opposite(num, i,j,a,b,c,d);
    break;
  case 0: case 15: break; /* 单元内没有等高线 */
}
```

最后，我们对三种需要画出近似线段的情形编写函数。例如，函数draw_one处理从一条边界到相邻边界画近似线段的8种情形。如果单元边界的长度在x方向和y方向分别为s_x和s_y，并且左下角位于(ox, oy)，如图2.51所示，那么部分代码如下：

```
void draw_one(int num, int i, int j, double a, double b,
                       double c, double d)
{

/* num = 单元类型的编号 */
/* i, j = 单元在阵列中的坐标 */
/* a, b, c, d 是单元4个顶点的值 */

/* s_x, s_y = 单元边界的长度 */
/* 这里是计算单元左下角坐标的代码 */

  switch(num) /* 根据单元的类型进行不同的处理 */
  {
   case 1: case 14:
       x1=ox;
       y1=oy+s_y;
       x2=ox+s_x;
       y2=oy;
       break;

/* 这里是处理其他情形的代码 */

  }
/* 画出线段 */

       glVertex2d(x1, y1);
       glVertex2d(x2, y2);
}
```

读者可以在附录A中找到完整的代码。图2.52所示是Cassini卵形线，它由下面的方程定义：

$$f(x, y) = (x^2 + y^2 + a^2)^2 - 4a^2x^2 - b^4$$

其中，$a = 0.49$，$b = 0.5$。图2.52给出了同一个高度值的两条等高线。我们在生成图2.52(a)时总是把单元边界的中点作为等高线和单元边界的交点，而在生成图2.52(b)时利用了插值来求等高线和单元边界的交点。我们在一个50 × 50的网格上对函数f采样。生成图2.53所用的地形数据来自Hawaii的Hononolu，该图以等间隔的高度值显示了多条等高线。没有等高线的区域是海洋，而左边部分对应于Diamond Head火山口。从本书的网站（www.cs.unm.edu/~angel）可以下载这个地形数据，它是由一些点的海拔高度组成的阵列。

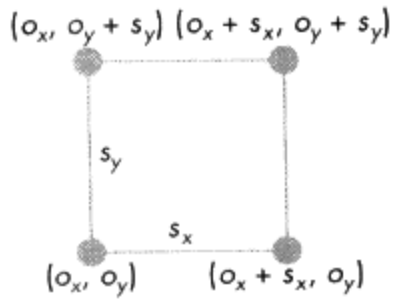


图 2.51　绘制穿过单元相邻两条边界的近似线段

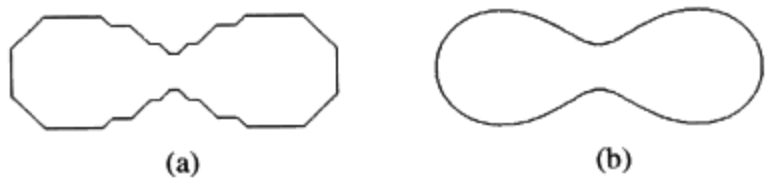


图 2.52　Cassini 卵形线：(a) 使用中点；(b) 使用插值

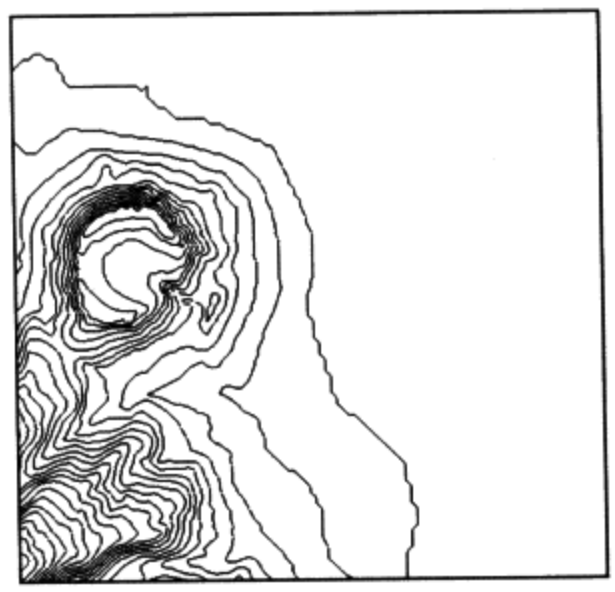


图 2.53　从 Hawaii Hononolu 地形数据绘制出的等高线

构造等高线还有其他一些方法，其中一种是从一个单元开始，已知这个单元包含一段等高线，然后顺着这段等高线扩展到相邻单元，如此不断扩展直到绘制出这条等高线。然而，步进方格算法的优点是可以独立地处理所有单元（在数据中行进），而且可以直截了当地扩展到三维体数据。

小结和注释

这一章对 OpenGL API 做了介绍，在此过程中用到了许多在第 1 章学到的基本概念。虽然我们编写的第一个应用程序是二维的，但可把二维图形看做是三维图形的特殊情形，然后只需要用最少的努力就可以把这个例子扩展到三维。

第一个例子是 Sierpinski 镂垫绘制程序，这是一个非平凡的例子。在这一章末尾的习题中可以找到对 Sierpinski 镂垫的一些扩展和有关的数学问题。在建议阅读资料部分列出的一些教材中可以找到许多有趣的曲线和曲面，利用简单的程序就可以生成它们。

从图形 API 和图形模型的发展历史，可以看到从三维开始学习图形学的重要性。第 1 章介绍的笔式绘图仪模型过去被使用了许多年，许多像 PostScript 这样重要的 API 也以它为基础。定义图形 API 国际标准的工作从 20 世纪 70 年代开始，直到 1984 年 GKS 被国际标准化组织（ISO）采纳。然而，GKS 的基础是笔式绘图仪模型，它作为一个二维 API 在 CAD 领域的应用很有限。尽管这个标准又由GKS-3D扩展到三维，基本模型造成的限制导致了它在许多方面的缺陷。像PHIGS和PHIGS+这样的 API 来自 CAD 领域，它们在本质上是三维的，并且基于虚拟照相机模型。

OpenGL是从GL API衍生出来的。GL API的基本假定是用流水线结构来实现虚拟照相机模型，它是为 Silicon Graphics 公司（SGI）的工作站开发的，这种工作站起初采用专用 VLSI 芯片来实现绘制流水线。因此，GL 是特别为高速实时绘制而设计的，尽管 GL 和 PHIGS 有许多相同之处。很多应用程序员意识到了利用 GL 编程的优点，并希望把这些优点扩展到其他平台，OpenGL 的出现

正是为了满足这一要求。因为OpenGL从GL中去掉了输入和窗口函数，功能只集中于绘制，所以这种新的API可以移植，同时还保留了使得GL功能如此强大的那些特征。

虽然大多数使用OpenGL的程序员更愿意使用C语言编程，但高级接口也受到了相当多的关注。用C++代替C只需要少量的代码改动，但这没有为OpenGL提供一个真正的面向对象接口。面向对象程序员对OpenGL和一些高级API都很感兴趣。虽然OpenGL没有官方的Java绑定，但已经有很多这方面的努力。OpenGL的Java绑定不是一个简单的问题，因为应用程序编程人员希望同时利用Java的面向对象特性，各种Java工具包，还有非面向对象的OpenGL。在Internet上可以找到一些Java绑定，最近Sun Microsystems公司还发布了它们的Java绑定。许多Java程序员使用JOGL绑定。

第10章将介绍场景图，场景图提供了图形硬件的更高级的面向对象接口。大多数场景图API都把OpenGL作为底层绘制API。

在游戏领域，Windows的主导地位使得游戏开发者只为单独一种平台编写代码成为可能。DirectX只在Windows平台上运行并且对这类系统做了速度优化。尽管多数DirectX代码看起来类似OpenGL代码，但DirectX程序员可以使用商用图形卡提供的特性，这些特性是依赖于设备的。因此，用DirectX编写的应用程序不具备OpenGL应用程序的可移植性和稳定性。由于这个原因，我们看到的情况是DirectX占据了游戏领域，而科学和工程领域的应用程序通常用OpenGL编写。对OpenGL程序员来说，如果想使用某种硬件特有的功能，OpenGL提供了对其访问的扩展机制，但这是以牺牲可移植性为代价的。OpenGL着色语言GLSL和Cg（见第9章）都支持可编程流水线，这使得OpenGL和DirectX对高端应用的性能差异正在变小。

例子和程序已经说明了如何以一种简单的方式描述和显示几何对象。从第1章介绍的建模-绘制模式的角度来看，我们专注于建模。然而，模型完全没有结构可言，只是用顶点和属性的列表来表示对象。第10章将会介绍构造层级模型，它可以表示对象之间的关系。然而，现在读者应该能够编写出有趣的程序。请完成这一章末尾的习题并把一些二维问题扩展到三维。

建议阅读资料

Sierpinski镂垫为分形几何的奥秘提供了一个很好的介绍。许多教科书[Bar93，Hil01，Man82，Pru90]都讨论了分形几何。

PostScript和LOGO都使用笔式绘图仪API。LOGO提供了海龟图形API，这种API学起来挺简单，而且它能够描述将在第11章讨论的几种二维数学曲线（见习题2.4）。

GKS[ANSI85]、GKS-3D[ISO88]、PHIGS[ANSI88]和PHIGS+[PHI89]既是美国标准，也是国际标准。从美国国家标准学会（ANSI）和ISO可以得到它们的形式描述。许多教科书使用这些API[Ang90，End84，Fol94，Hea04，Hop83，Hop91]。

X Window System[Sch88]已经成为UNIX工作站上的标准并且影响了其他平台上窗口系统的开发。[Ups89]介绍了RenderMan接口。

OpenGL的两本标准参考书是*OpenGL Programming Guide*[Ope05]和*OpenGL Reference Manual*[Ope04]。另外还有一本OpenGL的详细说明书[Seg92]。*OpenGL Programming Guide*从第二版开始直到目前的第四版[①]，都使用由Mark Kilgard开发的GLUT库[Kil94b]。*OpenGL Programming Guide*提供了大量的OpenGL代码示例。GLUT以前是为X Window System开发的[Kil96]，但现在也有Windows和Macintosh的版本。关于OpenGL的许多信息和示例程序都可以在Internet上找到。附录A的开头列出了一些有代表性的网址。

① 截至译者翻译本书，*OpenGL Programming Guide*的最新版本为第五版。——译者注

OpenGL: A Primer[Ang08]是本教材的配套书，其中包含了这里用到的 OpenGL 函数的细节和更多的示例程序。Windows用户可以在[Wri07]和[Fos97]里找到更多的例子，Mac OS X用户可以在[Koe08]中找到更多的细节。

DirectX API 的图形部分起初被称做 Direct3D，它的最新版本是 10.0。

步进方格算法是步进立方体算法的特例[Lor87]，步进立方体算法是针对体数据可视化提出的。

习题

2.1　对通过三角形细分生成Sierpinski镂垫的程序稍微做改动就可以得到在计算机动画中使用的分形山脉。具体的修改是：在找出三角形每条边的中点之后，对其进行扰动，然后再细分三角形。请绘制这些三角形，但不进行填充。以后读者可以在三维中做这个习题并增加明暗效果。按这个方法进行几次细分之后，就可以生成足够的细节使得那些三角形看起来像山脉。

2.2　Sierpinski 镂垫和习题 2.1 生成的分形山脉展示出了许多在分形几何中研究的几何复杂性[Man82]。假定利用只有长度而没有宽度的数学上的线来构造 Sierpinski 镂垫。在每次细分之后都把中心三角形去掉，令细分的次数趋于无穷，请问初始三角形的面积还剩下百分之多少？考虑每次去掉中心三角形之后剩下的三角形的总周长。令细分的次数趋于无穷，请问所有剩下的三角形的总周长会有什么变化？

2.3　在最低层的处理中，我们直接对帧缓存中的比特进行操作。OpenGL 提供了面向像素的命令，可以允许用户直接访问帧缓存。读者可以以 OpenGL 中的函数 glPoint 为基础建立一个简单的虚拟帧缓存函数库，然后用一些简单的光栅算法（比如画线或者圆）来测试这个函数库。编写一个函数库，使用户能够在内存中开辟出一个帧缓存，并对其进行操作。核心的函数应该是 WritePixel 和 ReadPixel。这个函数库应该允许用户设置并显示自己的帧缓存，还要允许用户在程序中对像素进行读 / 写。

2.4　海龟图形系统（Turtle graphics）是另一种定位系统。它的基本概念是海龟，海龟可以在屏幕上移动并且我们可以把一支画笔固定在海龟壳底。海龟的位置由一个三元组(x, y, θ)来描述，这个三元组给出了海龟的中心和方向。海龟图形系统的一个典型 API 包含下面的函数：

```
init(x,y,theta); /* 初始化海龟的位置和方向 */
forward(distance);
right(angle);
left(angle);
pen(up_down);
```

请利用 OpenGL 实现一个海龟图形函数库。

2.5　请利用习题2.4的海龟图形函数库来生成Sierpinski镂垫和习题2.1及习题2.2的分形山脉。

2.6　空间填充曲线引起数学家的兴趣已经有几个世纪了。在极限状态，这些曲线的长度无限大，但它们限制在一个有限的长方形里并且永不自相交。许多这样的曲线可以通过递归生成。考虑图 2.54 所示的“规则”，它把一条线段用 4 条更短的线段替换。请编写一个程序，从一个三角形开始，把那条替换规则递归地应用于所有的线段。最后生成的对象称为 Koch 雪花。[Hil07]和[Bar93]中有更多空间填充曲线的例子。

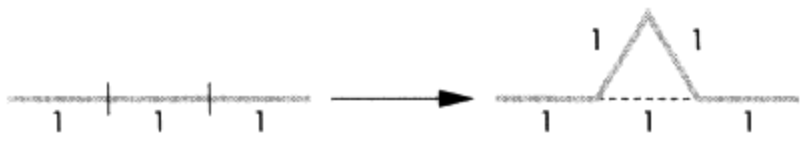


图2.54　Koch雪花的生成

2.7　从单元组成的矩形阵列开始可以生成一个简单的迷宫，这里的每个单元是包含4条边的小正方形。去掉单元的边（除了单元阵列的外边界）直到所有的单元都相连，然后通过从外边界去掉两条单元的边来生成一个入口和一个出口。图2.55所示是一个简单的例子。请利用OpenGL编写一个程序，它的输入是两个整数N和M，输出是一个$N \times M$迷宫图。

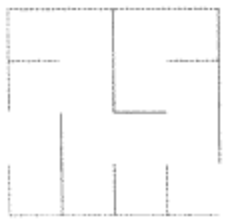


图2.55　迷宫

2.8　为了允许用户使用减色模型，应如何修改OpenGL中的RGB颜色模型?

2.9　我们看到图形系统中的一个基本操作是把裁剪矩形中的点(x, y)映射到屏幕窗口中的视口内的点(x_s, y_s)。假定这两个矩形是由下面的OpenGL函数定义的：

```
glViewport(u, v, w, h);
gluOrtho2D(x_min, x_max, y_min, y_max);
```

找出把(x, y)映射成(x_s, y_s)的数学公式。

2.10　许多图形学API使用相对定位，这种API包含像下面这样的函数：

```
move_rel(x,y);
line_rel(x,y);
```

这些函数可以用来绘制线段和多边形。函数move_rel把一个内部位置（或者称为光标）移动到一个新的位置。函数line_rel移动光标并且在光标的旧位置和新位置之间画出一条线段。相对定位和OpenGL中使用的绝对定位相比有什么优点和缺点？如何在OpenGL中实现相对定位?

2.11　有时需要判断一个区域内所有的点是否在一个多边形的内部。如果逐个点进行测试，那么效率是非常低的。请描述可以避免进行逐点测试的一般策略。

2.12　请设计一个算法来确定一个二维多边形是否是简单多边形。

2.13　图2.56所示是称为网格的一组多边形，这些多边形有公共的边和顶点。设计出一种或者多种简单的数据结构来表示网格。一种好的数据结构应该包含共享顶点和共享边的信息。基于OpenGL寻找一个有效的方法来绘制利用特定数据结构表示的网格。提示：先考虑包含顶点位置的数组或者链表。

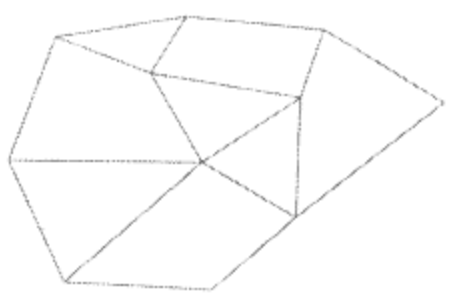


图2.56　多边形网格

2.14　在2.4节中，我们看到OpenGL利用顶点列表来定义多边形。为什么利用边来定义多边形可能会更好呢？提示：考虑如何有效地表示一个网格。

2.15　在OpenGL中，可以把颜色和顶点关联到一起。如果一条线段的两个端点具有不同的颜色，OpenGL会在绘制这条线段时对这两个颜色进行插值。OpenGL也是这样绘制多边形的。请利用这个性质显示Maxwell三角形：顶点分别为红色、绿色、蓝色的等边三角形。请问在Maxwell三角形和颜色立方体之间有什么关系？

2.16　在计算机图形学中，可以通过在模型中加入简单的物理机制来模拟许多真实效果。请模拟一个二维弹跳球，这需要考虑到重力以及与表面的弹性碰撞。读者可以把球建模成一个封闭的多边形，这个多边形的边数足够多以至看起来是光滑的。

2.17　习题2.16的一个有趣但却困难的扩展是模拟台球游戏。这里需要考虑多个球，而且球与球桌边之间会碰撞，球与球之间也会碰撞。提示：先考虑两个球，要考虑如何检测可能发生的碰撞。

2.18　广告上说某种具有CRT显示器的图形系统可以显示64种颜色中的任何一种。请问由此可以得出关于帧缓存和显示器质量的哪些信息？

2.19　请设计一个算法来判断一个二维多边形是否是凸多边形。

2.20　OpenGL的初学者可能会有一个疑惑：像glVertex这样的基本函数到底有多少种形式？在像C++这样的语言中，可以只用一个函数名，由编译器根据参数的类型来选择正确的函数。请利用C++编写一个介于OpenGL和用户程序之间的函数库，这个函数库的目的是最小化应用程序员需要使用的OpenGL函数的数目。

2.21　另外一种绘制三维Sierpinski镂垫的方法是在每次迭代时只对初始四面体的面进行细分。请按这种方法编写一个程序。用这种方法绘制的结果与2.10节中绘制的结果有什么不同？

2.22　在每次细分四面体时，我们只保留了4个顶点处的小四面体，所以每次细分之后都把体积减小为原来的f倍。请找出f。对一个四面体进行一次细分，得到的4个小四面体的面积之和与原来四面体面积的比是多少？

2.23　如果把步进方格算法扩展到三维空间中的表面，我们就得到了步进立方体算法。考察一个表面与立方体相交的每一种可能的方式，并且对立方体的顶点进行相应的染色。请问用黑色和白色对立方体的顶点染色，共有多少种方法？当考虑了对称性之后还有多少种不同的染色方法？

第3章　输入和交互

现在介绍交互式图形程序的开发。交互式计算机图形学使许多应用成为可能，从建筑的交互式设计，到通过图形界面控制的大型系统，到虚拟现实系统，再到计算机游戏。

讨论包括三个主要的部分。首先介绍可用于交互的各种设备。我们从两个不同的角度考虑输入设备：1）基于实际属性对物理设备进行描述；2）从应用程序的角度考察这些设备的工作方式。之后讨论客户/服务器网络结构和客户/服务器图形系统。我们利用这些思想来为图形程序开发基于事件驱动的输入。最后开发一个多边形建模程序，这个程序演示了交互式图形编程的许多重要特征。我们还将介绍图形编程的一些其他特征，虽然它们不是直接与交互相关的，但可以使程序的执行效率更高，从而更好地支持交互。

3.1　交互

计算机技术最重要的进展之一就是使用户能同计算机显示系统进行交互。Ivan Sutherland开发的Sketchpad系统最能代表交互式计算机图形学时代的开始。他引入的这个基本范型表面上看来十分简单，用户在显示器上看到一幅图像，然后借助像鼠标这样的交互式设备对这个图像做出反应。图像根据用户的输入做出改变，用户对这个变化再做出反应，如此反复进行。无论是利用现代窗口系统提供的工具编写程序，还是在交互式博物馆展览中使用的人机交互界面，我们都在使用这个范型。

从Sutherland的工作到现在已经有40多年了，硬件技术和软件技术都有了很大进展，但他所引入的观点和思想依然支配着交互式计算机图形学。从如何把人机交互界面概念化到如何利用图形数据结构来高效地实现这种人机交互界面，都可以看到这种支配作用的影响。

本章采用的方法稍微不同于本书的其他部分。尽管包括OpenGL在内的多数现代API主要考虑的是渲染，但是交互也是大部分应用程序的重要组成部分。然而，OpenGL并不直接支持交互，不考虑交互的主要原因是开发OpenGL的设计人员希望通过允许系统在各种环境下工作来增强其可移植性。因此，窗口和输入函数并没有包含在API之中。虽然这个决定使得渲染器成为可移植的，但也使对不涉及特定窗口系统的交互的讨论变得更加困难。此外，因为任何一个应用程序与窗口环境都至少有一些基本的接口，如果想编写完整而有价值的程序，就不能完全避免这些问题。如果从API中去掉交互，那么应用程序编程人员面对他的特殊环境就必须考虑其中一些晦涩难懂的细节。

我们可以通过使用GLUT工具包来避免这些潜在的困难，就像在第2章所做的那样。这个工具包提供了几乎所有系统都要求的最基本功能，比如打开窗口、使用键盘和鼠标，还有创建弹出式菜单等。我们选择使用GLUT工具包，尽管它不能提供任何一种特定窗口系统的全部特性，并且生成的代码既没有充分利用特定窗口系统的全部功能，也无法证明它与在某个特定环境下编写的代码具有相同的效率。然而，我们可以把和窗口系统进行交互的原理通过GLUT用最简单的方式演示出来，这也是为标准窗口系统编写代码所依据的原理。

和第2章一样，我们使用窗口系统（window system）这个术语来包括像X Window System、Microsoft Windows和Macintosh操作系统这样的系统所提供的所有环境，我们开发的图形程序将在

其中一个环境中的窗口里绘制图形。在有关窗口系统的文献中，这个术语可能会混淆像X Window和绘制图形所用的OpenGL窗口之间的区别。不过，只要读者把OpenGL窗口看做是其系统中一个特殊类型的窗口，这个窗口可以显示OpenGL程序的输出，那么一般就不会出现问题了。我们通过使用GLUT工具包可以避免窗口系统、窗口管理器和图形系统之间进行交互所固有的复杂性。和第2章一样，GLUT会使示例程序不依赖于任何特定的窗口系统。

我们先描述几种交互设备以及与之进行交互的各种方式。之后把这些设备放到客户/服务器网络环境中，并介绍一个包含最小交互功能的API。最后编写一些示例程序。

3.2　输入设备

我们可以用两种不同的方式看待输入设备。其中比较直观的方式是把它们看做像键盘或者鼠标这样的物理设备，并考虑它们的工作方式。当然，我们需要知道输入设备的某些物理性质，因此如果要对输入获得一个完整的理解，这样的考虑是必需的。然而从应用程序编程人员的角度来看，编写应用程序时并不需要知道某个特定物理设备的内部细节，而是更愿意把输入设备当做逻辑（logical）设备，从应用程序的角度来看，这些逻辑设备的性质是由其所实现的功能确定的。一个逻辑设备（logical device）的特性是由它与应用程序的高级接口而不是其物理性质来表征的。

考虑下面的C代码片断：

```
int x;
scanf(&x);
printf("%d", x);
```

该程序读取一个整数然后将其打印输出。尽管一般在一个工作站上执行该程序，从键盘输入数据，并在显示器上输出数据，但是也可以利用重定向这样的机制来接收其他程序的输出作为输入或者把输出写入到磁盘文件中。尽管用到了键盘和显示器，但是调用scanf和printf函数并不需要了解物理设备的物理性质，如键盘码或显示器分辨率等。scanf和printf是逻辑函数，从C程序的角度来看，它们是由其处理输入或输出字符串的方式来定义的。更一般地讲，C程序中数据的输入和输出是通过像printf、scanf、getchar和putchar这样的函数（这些函数的参数使用标准的C数据类型）和C++中的输入流（cin）和输出流（cout）来完成的。

当利用printf或者cout输出字符串时，接收输出的物理设备可能是一台打印机、一个终端或者是一个磁盘文件，该输出甚至可能是另一个程序的输入。应用程序编程人员几乎不用关心目标设备所需的具体存储格式。

在计算机图形学中，逻辑设备的使用稍微复杂些，这是因为输入所采用的格式要比非图形应用程序中通常只涉及的比特串或者字符串更加多样化。例如，可以使用鼠标（一种物理设备）来选择CRT屏幕上的一个位置或者确定想要选择的菜单项，这两个使用输入设备的例子需要用到不同的逻辑输入函数。在第一个例子中，一对x和y坐标值（在某个坐标系中）返回给用户程序；在第二个例子中，应用程序可能会接收到对应于某个菜单项的整数标识符。把物理设备和逻辑设备分开不仅允许用户以多种截然不同的逻辑方式使用相同的物理设备，而且如果鼠标被数据板或跟踪球等物理设备取代，仍然可以使用相同的程序。

3.2.1　物理输入设备

从物理的角度来看，每种输入设备都有其特殊的性质，这使它比其他设备更适合于某种任务。我们采用大多数有关工作站的文献中的观点，即把物理设备分成两种主要类型：定位设备和键盘设

备。定位设备（pointing device）允许用户在显示器上指示一个位置，而且为了允许用户发送信号或者中断计算机，这种设备几乎总是有一个或者多个按钮。键盘设备（keyboard device）几乎总是对应于物理键盘，但可以推广到包括任何返回字符码的设备。例如，平板电脑利用识别软件来辨认用户借助触控笔书写的字符，最终生成和标准键盘一样的字符码。

我们在示例中将使用美国标准信息交换码（American Standard Code for Information Interchange，ASCII），它对每一个字符指定一个无符号字节。之所以使用ASCII是因为它是一种非常流行的编码方式。不过要注意，其他的编码方式，特别是Internet应用程序所使用的编码方式使用多个字节表示一个字符，这样允许支持更丰富的字符集。

鼠标（见图3.1）和跟踪球（见图3.2）的使用方法类似，而且其构造通常也非常类似，一个典型的机械鼠标在翻过来之后看起来就像一个跟踪球。在这两种设备中都有一对由球的运动所驱动的编码器，它们测量球在两个正交方向上的运动，并把球的运动转换成信号传送给计算机。

图3.1　鼠标

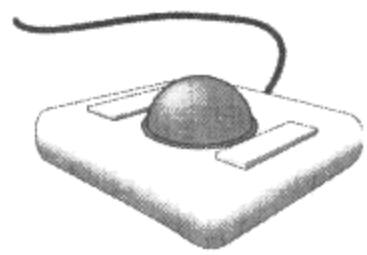


图3.2　跟踪球

这些设备还有许多变种形式，有的使用光检测器而不是机械检测器来测量运动。小型跟踪球在便携式电脑上很流行，因为可以把它们直接做到键盘上。在键盘上还可以使用各种压力敏感设备，其功能与鼠标和跟踪球相似但是不移动，这种设备的编码器用来测量一个小旋钮经受的压力，这个小旋钮经常位于键盘中部的两个按键之间。

我们可以把鼠标或者跟踪球的输出看做是由设备提供的两个独立的数值，这两个数值可以看做是坐标位置并由用户程序或者在图形系统内部转换为屏幕坐标或世界坐标中的一个二维坐标位置。如果以这样方式设置该设备，则可以利用它在显示器上自动放置一个标记（光标），不过人们很少以这种直接的方式使用这些设备。

鼠标或者跟踪球中编码器的输出不一定要解释成位置。设备驱动器或者用户程序也可以把来自编码器的信息解释成两个独立的速度（见习题3.4），计算机然后可以对这些值进行积分来得出一个二维位置。这样当鼠标在表面上移动时，对速度进行积分可以得到x和y的值，它们在经过转换后可以用来指示光标在屏幕上的位置，如图3.3所示。通过把球滚动的距离解释成速度，可以把这些设备当做灵敏度可变的输入设备来使用。如果鼠标相对静止位置的偏离较小，会使光标的位置缓慢地改变；如果鼠标相对静止位置的偏离较大，会使光标的位置快速地改变。对这两种设备，如果球不旋转，那么积分值没有变化，并且跟踪鼠标位置的光标也不会移动。在这种模式下，这些设备是相对定位（relative positioning）设备，因为球通过改变位置来使用户程序获得位置信息，应用程序并不使用球（或者鼠标）的绝对位置。

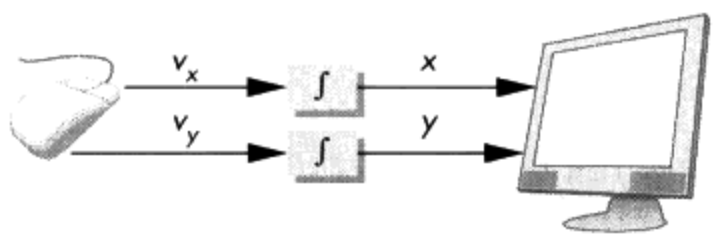


图3.3　光标的定位

由鼠标或者跟踪球提供的相对定位功能并不总是令人满意的，尤其是这样的设备不适合于线图跟踪这样的操作。例如，当用户试图用鼠标在屏幕上沿一条曲线移动时，如果释放并移动鼠标，则会失去所跟踪曲线上的绝对位置。数据板（data tablet）提供了绝对定位功能。一个典型的数据板（见图3.4）在其表面下方嵌入了行列交错的电线网。触控笔内的传感器和数据板内的电线都能够传递电信号，通过这两个信号之间的电磁相互作用可以确定触控笔的位置。对触摸敏感的透明触摸屏可以放在 CRT 的表面上，它具有许多和数据板一样的性质。小的矩形状的压力敏感触摸板可以嵌入到大部分便携式电脑的键盘中，这些触摸板可以配置成相对或绝对定位设备。一些触摸板能够检测到两个手指同时触摸不同的位置，这样的信息可以用来实现更复杂的功能。

光笔（lightpen）在计算机图形学中有很长的历史，Sutherland 早期开发的 Sketchpad 系统就使用这种设备。光笔中有一个像光电池一样的感光设备（见图 3.5）。如果把光笔放置在 CRT 屏幕上的某个位置处，当电子束从该位置的另一面轰击磷光物质所发出的光强度超过光笔中设定的某个阈值时，光笔就会给计算机发送一个信号。因为每一次利用帧缓存对 CRT 进行刷新的时间都很准确，所以可以利用这个信号出现的时间来确定光笔在 CRT 屏幕上的位置（见习题 3.19）。光笔最初是用在随机扫描设备上的，所以光笔向计算机发送中断信号的时间可以很容易和显示列表中的一段代码相匹配，这使得光笔非常适合于选取由应用程序定义的对象。对于光栅扫描设备，显示器上的位置可以通过扫描开始的时间和扫描每条扫描线所需的时间来确定。这样，我们就有了一个直接定位的设备。光笔不如鼠标、数据板和跟踪球用得那么普遍，它的一个缺陷是难以获得屏幕上黑暗区域中的位置。不过，平板电脑模仿了原始光笔的使用方式，用户可以使用一个触控笔在平板（显示器）表面上任意移动。

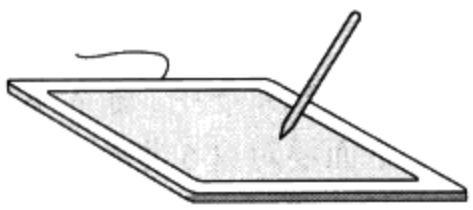


图 3.4　数据板

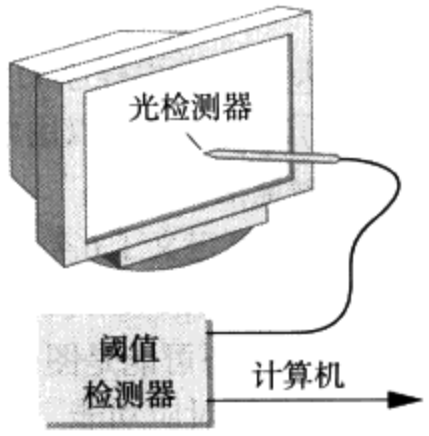


图 3.5　光笔

还有一种特别值得介绍的设备——操纵杆（joystick）（见图 3.6）。操纵杆沿两个正交方向的运动被编码并解释成两个速度，这两个速度再经过积分得到屏幕上的位置。积分意味着如果操纵杆处于静止位置，光标位置就不会改变；操纵杆离静止位置越远，光标在屏幕上的位置变化得就越快。因此，操纵杆是一种灵敏度可变的设备。操纵杆的另一个优点是它可以和弹簧及阻尼器这样的机械元件一起组装，这样可以在用户推动操纵杆时让用户有阻力的感觉，这种其他设备提供不了的机械感觉使得操纵杆很适合飞行模拟器和游戏控制器这样的应用。

对于三维图形应用，人们更愿意使用三维输入设备。虽然有许多这样的三维设备，但现在还没有一种三维输入设备能像二维输入设备那样获得人们的广泛接受。空间球（spaceball）看起来和操纵杆类似，只是在手杆的末端有一个球（见图 3.7），不过这个手杆是固定的。在球里面的压力传感器可以测量用户施加的力。空间球不但能测量三个笔直方向上的力（上－下，前－后，左－右），还可以测量三个独立的扭曲力。这种设备可以测量 6 个独立的值，所以具有 6 个自由度（degree of freedom）。作为空间球的一个应用示例，可以利用它来确定照相机的位置和方向。

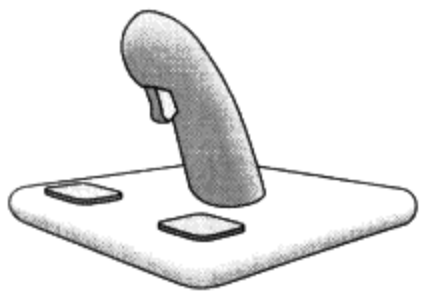


图3.6　操纵杆

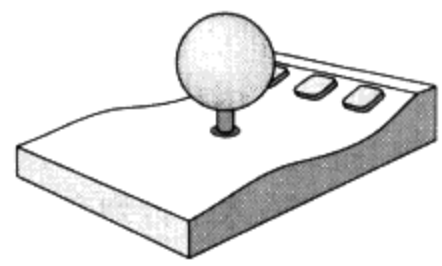


图3.7　空间球

其他的三维设备，比如激光扫描仪，可以直接用来测量对象的三维空间位置。用于虚拟现实的各种跟踪系统可以感知用户的姿势。我们前面描述的几种设备可以提供2～6个自由度，而在虚拟现实和机器人领域经常需要更多的自由度。像数据手套这样的设备可以感知人体各个部位的运动，这样就提供了更多的输入信号。最近，像Nintendo's Wii这样的输入设备除了能够进行无线数据传输之外，还能利用陀螺仪传感设备感知对象的位置和方向。

目前许多现有的设备都能够充分利用计算机的巨大计算性能，这是设计新型设备的有效途径。例如，运动捕捉（motion capture，mocap）系统把由标准数字照相机组成的阵列布置在环境中，并把许多小球形点放在人体的关键部位，比如胳膊和腿的关节，然后利用照相机阵列来捕捉从小球形点反射的光线。在一个典型系统中，会有8台照相机监视整个环境并以很高的帧率捕捉小球形点的位置，这样就获取了大量的数据。系统中的计算机通常就是一台普通的PC，它可以通过对小球形点的二维图片进行处理来确定在每一帧每一个小球形点在三维空间中的位置，从而得到最终的捕捉数据。

3.2.2　逻辑设备

现在可以回过头来从应用程序的内部讨论输入设备，也就是从逻辑设备的角度来讨论输入设备。一个输入设备的逻辑行为可以用两个主要特征来描述：1）该设备返回给用户程序的测量值；2）该设备返回这些测量值的时间。

一些早期的图形API定义了6类逻辑输入设备。因为在现代窗口系统中，输入不可能与物理设备的性质完全无关，所以OpenGL没有采用这种方法。然而，下面要对这6类输入设备进行简要的描述，因为这6类输入设备可能是图形应用开发人员会遇到的输入形式。我们还将介绍OpenGL如何提供每一类输入设备的对应功能。

1. **字符串设备（string）**：字符串设备是向用户程序提供ASCII码串的逻辑设备。通常用物理键盘来实现这种逻辑设备。在这种情形下，该术语与多数窗口系统和OpenGL中所用的术语是一致的，它们通常不区分逻辑字符串设备和物理键盘。
2. **定位设备（locator）**：定位器设备向用户程序提供对象位置的世界坐标。它通常由一种像鼠标或者跟踪球这样的指向设备来实现。在OpenGL中，通常以这种方式使用定位设备，尽管必须在自己的程序中完成从屏幕坐标到世界坐标的转换。
3. **拾取设备（pick）**：拾取设备把显示器上一个对象的标识符返回给用户程序。它通常由和定位器一样的物理设备来实现，但它有独立的面向用户程序的软件接口。在OpenGL中，可以用一个称为选择（selection）的过程来实现拾取（见3.8节）。
4. **选择设备（choice）**：选择设备允许用户从多个离散的选项中选择一个。在OpenGL中，可以使用由窗口系统提供的各种构件。构件（widget）是一种图形交互设备，由窗口系统或者某个工具包来提供。典型的构建包含菜单、滚动条和图形按钮。大多数构件是作为特殊类型

的窗口来实现的。例如，一个具有 n 个选项的菜单可以作为选择设备，它同样允许用户在 n 个选项中选择一个。构件集是定义图形用户接口（graphical user interface，GUI）的关键元素。

5. **定值设备**（valuator）：定值设备向用户程序提供模拟输入。在一些图形系统中，可以使用框或刻度盘来提供定值设备的输入。各种工具包中的构件通常通过滑动条和单选钮这样的图形设备来提供这种设备。
6. **笔划设备**（stroke）：笔划设备返回一个位置数组。尽管可以把笔划设备看成是相当于定位设备的多次使用，但笔划设备往往是这样来实现的：某个行动使得数据开始向指定的数组传送，第二个行动使得传送结束。这里的第一个行动可以是按下鼠标的某个按键，第二个行动可以是释放那个按键。

3.2.3　输入模式

输入设备向应用程序提供输入的方式可以从两个方面来描述：测量过程和设备触发器。一个设备的测量（measure）是这个设备返回给用户程序的数据。设备的触发器（trigger）是用户向这个设备提供的一个物理输入，用户通过它能够给计算机传送触发信号。例如，键盘的测量数据应该包含单个字符或者一串字符，触发器可以是 Return 或者 Enter 键。对于定位设备，测量数据包含定位设备的位置，相应的触发器可以是物理设备上的一个按钮。测量过程可以由操作系统或者图形 API 中的某个函数来初始化。

应用程序可以通过三种不同的模式获得一个设备的测量数据，每种模式都是由测量过程和触发器之间的关系来定义的。测量过程一旦开始，输入设备就会获取测量数据并把它存储在一个缓存中，即使应用程序可能还不能访问该缓存中的内容。例如，不管应用程序是否需要鼠标输入，窗口系统总是连续不断地跟踪鼠标的位置并显示对应的光标。

在请求模式（request mode）下，除非设备被触发，否则设备的测量数据不会返回给程序。请求模式在非图形应用程序中是标准的输入模式。例如，如果一个典型的C程序需要字符输入，则使用像 scanf 这样的函数。当程序需要输入时，它在执行到 scanf 语句时暂停并且等待，直到用户在终端上输入字符。可以通过退格来更正所输入的字符，并且可以花费任意长的时间来输入字符。数据存储在一个键盘缓存中，只有当按下一个特别的键，比如 Enter 键（触发器）时，这个缓存中的内容才会返回给程序。对于像定位设备这样的逻辑设备，可以把定位设备移到所需的位置上，然后用它的按钮触发它，这个触发会让应用程序获得定位设备的位置。在请求模式下测量数据和触发器之间的关系如图 3.8 所示。

在采样模式（sample mode）下，输入是即时的。只要在用户程序中遇到了函数调用，就把测量数据返回给程序，因此不需要触发器（见图 3.9）。在采样模式中，用户必须在函数调用之前就把定位设备放置好或者用键盘把数据输入完毕，因为缓存中的测量数据会立即返回给应用程序。

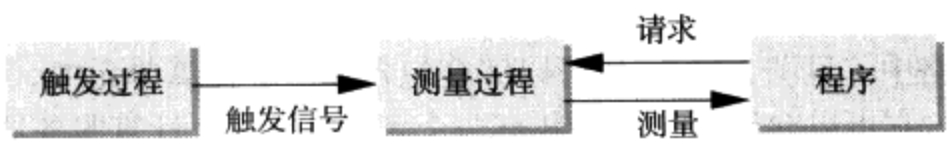


图 3.8　请求模式

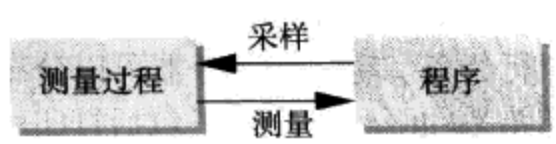


图 3.9　采样模式

在支持请求模式和采样模式的API中，这两种模式的输入有一个特征是用户必须识别出提供输入的那个设备。因此，除了指定的这个输入设备以外，我们忽略其他任何输入设备提供的任何其他信息。请求模式和采样模式对于通过程序来引导用户的情形是有用的，但对于用户控制程序流程的

情形就不是很有用了。例如，飞行模拟器或者计算机游戏可能有多个输入设备，比如操纵杆、刻度盘、按钮和开关等，用户在任何时刻都可以使用其中的大多数设备。要编写控制飞行模拟器的程序，只利用采样模式和请求模式输入几乎是不可能的，因为我们不知道在仿真过程中的某个时刻飞行员会使用什么设备。更一般地讲，在现代计算环境中可能出现的人机交互多种多样，要处理这些交互方式，光有采样输入模式和请求输入模式是不够的。

第三种模式是事件模式（event mode），该模式可以处理其他的交互方式。我们分三步来介绍事件输入模式。首先，我们说明如何把事件模式描述成测量-触发模式的另一种模式。然后，讨论客户和服务器的基本概念，因为在客户/服务器环境中事件模式是人们更愿意采用的交互模式。最后，借助GLUT介绍OpenGL的事件模式接口，并利用这个接口编写演示程序。

假定环境中有多个输入设备，每个输入设备都有它自己的触发器并且运行一个测量过程。每当一个设备被触发，就产生了一个事件（event）。设备的测量数据（包括这个设备的标识符）被放入一个事件队列（event queue）中。把事件放入事件队列的过程与应用程序如何对这些事件做出响应完全独立。应用程序处理事件的一种方式如图3.10所示。应用程序可以检查事件队列中排在前面的事件，或者如果事件队列是空的，应用程序可以等待直到有事件发生。如果在队列中有事件，程序可以检查第一个事件的类型，然后决定做什么。例如，如果第一个事件来自键盘，但应用程序对键盘输入不感兴趣，那么可以把这个事件丢弃并继续检查下一个事件。

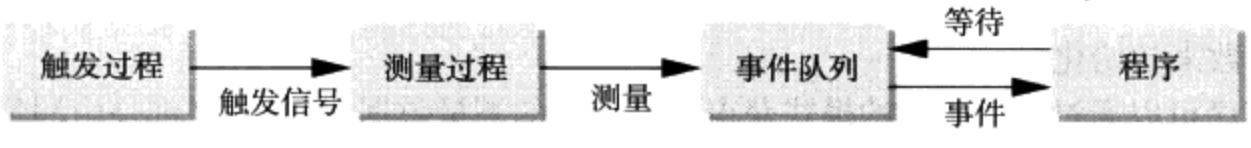


图3.10　事件模式

另一种方法是对每一种特定类型的事件关联一个称为回调（callback）的函数。从窗口系统的角度来看，操作系统定期检查事件队列中的事件，并且针对事件的类型执行相应的回调函数。我们将采用这种方法，因为它是一些主要窗口系统当前所采用的方法，并且在客户/服务器环境中已经证明了这种方法的有效性。

3.3　客户和服务器

到目前为止，我们对于输入的讨论并没有考虑在计算环境中可能发生的其他活动。我们把图形系统看做是一个庞大的盒子，除了由我们精心控制的输入设备和显示器与外界有联系之外，它与外部世界只有有限的联系。网络和多用户计算已经极大地改变了这种状况，而且这种变化之大已经到了这种程度：即使所用的是一个单用户的孤立系统，它的软件也可能会配置成一个简单的客户/服务器网络。

要想让计算机图形学对于各种实际应用都很有用，必须使它在分布式计算和网络环境中能够很好工作。在这种环境中，系统的构造模块是称为服务器（server）的实体，它们可以为客户（client）执行任务。客户和服务器可以分布在网络中（见图3.11），也可以完全包含在一个计算单元中。我们熟悉的服务器的例子包括打印服务器，它可以允许多个用户共享一个高速打印机；计算服务器，例如用户程序可以访问的远程高性能计算机；文件服务器，它允许用户不论登录到哪台机器都能共享文件和程序；还有终端服务器，它们处理拨号访问。用户和使用这些服务的用户程序是客户或者客户程序。服务器也可存在于单一操作系统内部更细粒度级别的模块中。例如，操作系统可能会提供一个可以由多个用户程序使用的时钟服务。

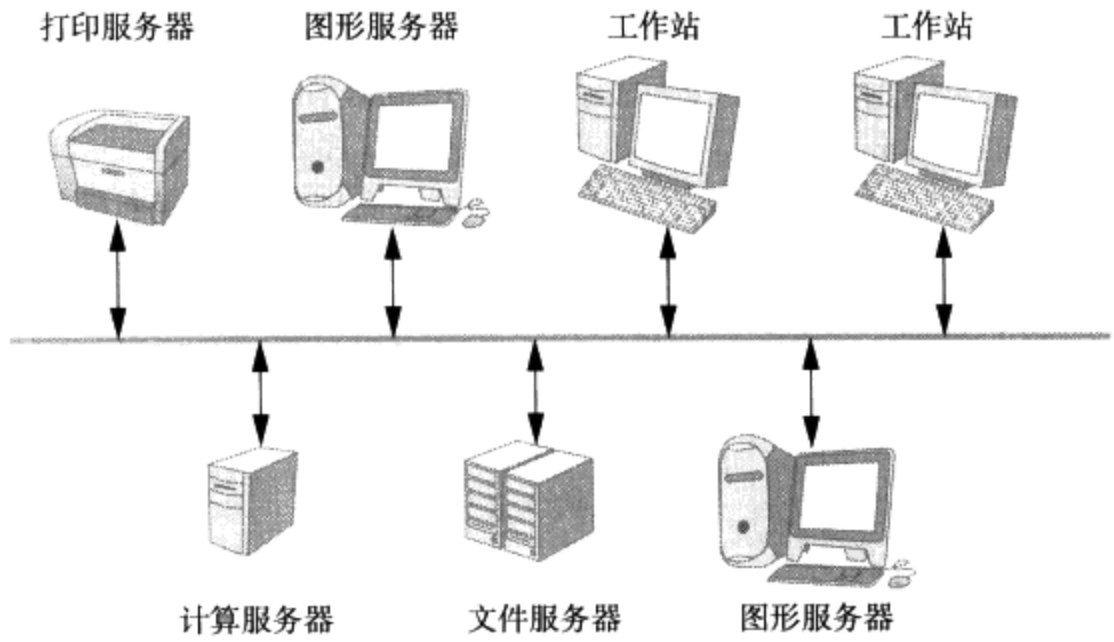


图 3.11　网络

要给连接到网络的工作站起个名字并不是显而易见的事情，它既可以是服务器也可以是客户，也许更合适的说法是：一个工作站可以并发执行客户程序和服务器程序。

这里使用的模型是由X Window操作系统流行开来的。我们使用了这个系统的大部分术语，现在多数窗口系统也使用这些术语，并且非常适合于在图形应用程序中使用。

一个工作站在配备了光栅显示器、键盘和像鼠标这样的定位设备之后就成了一个图形服务器（graphics server）。这个服务器可以在它的显示器上提供输出服务，还可以通过键盘和定位设备提供输入服务，这些服务对于网络中任何位置的客户都是可用的。

OpenGL应用程序都是利用图形服务器的客户。在一个孤立的系统中，这种区别可能不太明显，因为我们是在单独一台机器上编写、编译和运行软件。然而，我们也能利用网络上其他的图形服务器来运行相同的应用程序。

3.4　显示列表

我们可以通过显示列表来说明如何利用网络中的客户和服务器提高交互式图形系统的性能。显示列表起源于计算机图形学的早期。就像第1章讲到的那样，把一台通用计算机（或者主机）连接到一台显示器上（见图3.12）就构成了图形系统最初的体系结构的基础。为了避免让用户察觉到屏幕显示有明显的闪烁，计算机会以足够的帧速率发送必要的信息来刷新显示器①。在那个时候（大约是1960年），计算机的速度很慢而且价格昂贵，因此即使是显示一个简单的图形，所需要的刷新功能也使得成本非常高，除了少数几个应用以外一般都无法使人承受。

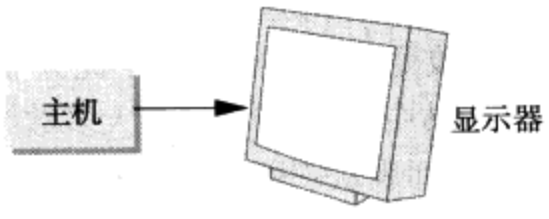


图 3.12　简单的图形体系结构

对这个问题的解决办法是建造一台专用计算机，这台专用计算机称为显示处理器（display processor），它的组成如图3.13所示。显示处理器有一个有限的指令集，其中的大多数指令是为了

① 对CRT显示器，刷新频率由CRT中的磷光物质决定，但通常为60～85 Hz。如果采用隔行扫描，刷新率减半。

在显示器上绘制图元。主机对用户程序进行编译后生成一个指令列表，然后把这个指令列表发送到显示处理器，显示处理器再把指令列表作为显示文件（display file）或者显示列表（display list）存储在显示存储器（display memory）中。对一个简单的非交互式应用程序，一旦显示列表被发送到显示处理器，主机就可以处理其他的任务，而显示处理器则会以足够的速率反复执行显示列表，从而避免出现闪烁现象。显示处理器不但解决了由于主机负担过重造成的瓶颈，还引入了专用绘制硬件并利用了其所带来的优点。

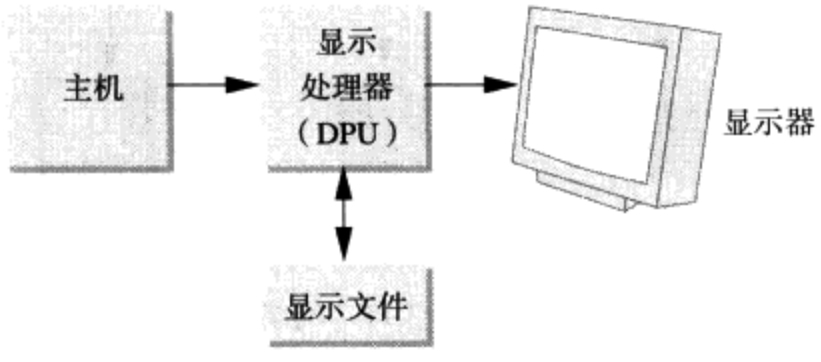


图 3.13　显示处理器体系结构

如今，这种老式的显示处理器已经变成了图形服务器，而主机上的应用程序则变成了客户。主要的瓶颈不再是显示器的刷新率（尽管这仍是一个重要的问题），而是在客户和服务器之间通过的数据流量。此外，使用专用硬件现在已经成为高端系统的特征。

可以用两种方式中的一种把图形实体发送到显示器。可以把对象的完整描述发送到图形服务器。对于典型的几何图元，除了观察信息之外，必须发送的对象描述包括顶点、属性和图元类型。立即模式（immediate mode）是基本操作模式，在这种模式下，只要程序执行到定义图元的语句，该图元就被发送到服务器进行显示，与它有关的信息不会在系统中保留[①]。要想在清屏之后重新显示图元或者在一次交互之后在新的位置绘制图元，程序必须重新确定图元，然后还要再次让这些信息经过显示处理过程。对于交互性很强的应用程序中的复杂对象，这个过程可能会使大量的数据从客户流向服务器。

显示列表还提供了另一种方法，这种方法（第二种方法）称为保留模式（retained mode）。我们只需定义对象一次，然后就把它的描述放到显示列表里。显示列表存放在服务器中，从客户向服务器发送一个简单的函数调用就可以重新显示它。这种方法的明显优点是减少了网络中的数据流量，此外这个模型还允许一些命令只执行一次，把结果存储在图形服务器的显示列表中，这样就减少了执行这些命令的开销。因此，在许多情形下，最优配置包括一个执行客户程序的性能优良的数值处理计算机和一个当做服务器的专用图形计算机（这虽然是一个陈旧的想法，但它在现代图形系统中仍然非常有效）。当然，显示列表也有一些缺点。显示列表需要占用服务器的内存，而且创建显示列表也需要额外的时间开销。尽管显示列表所带来的执行效率的提高常常可以抵消这些开销，但是如果显示列表中的数据经常需要改变，那么它的效率优势可能就不存在了。

在使用商用图形卡的现代工作站中，由GPU执行大多数绘制命令，所以使用显示列表也许并没有性能优势。然而，可以利用显示列表从它所定义的组件集来构造模型，这为构造模型提供了一种简捷的方法。

3.4.1　显示列表的定义和执行

显示列表和一般的文件有许多共同之处。必须有一种机制允许用户定义（创建）和操纵显示列表（把信息放入显示列表中）。对于哪些内容可以放到显示列表的定义必须足够灵活，以便留给用

① 图元被显示出来以后，它的图像（image）还保留在帧缓存中，但对象和图像不是一回事。

户相当多的自由。OpenGL提供了几个函数来操纵显示列表，它对显示列表中内容只有很少的限制。下面举几个简单的例子来说明这些函数的用法。

显示列表的定义和几何图元的定义类似，开头有一个glNewList，结尾有一个glEndList，中间是列表的内容。每个显示列表必须有一个唯一的标识符，这个标识符是一个整数，在C程序中通常用预编译宏指令#define把这个整数定义成一个适合于列表中的对象的名字。例如，下面的代码定义了一个红色的盒子。此处的代码和第2章的代码相似，但这一回信息被放到了显示列表中。

```
#define BOX 1 /* 或者是另一个没有使用过的整数 */

glNewList(BOX,  GL_COMPILE);
    glBegin(GL_POLYGON);
         glColor3f(1.0, 0.0, 0.0);
         glVertex2f(-1.0, -1.0);
         glVertex2f( 1.0, -1.0);
         glVertex2f( 1.0,  1.0);
         glVertex2f(-1.0,  1.0);
    glEnd();
glEndList();
```

其中的标志GL_COMPILE告诉系统把这个列表发送到服务器但不要显示它的内容。如果希望显示列表在构造的同时就立刻显示其内容，可以使用标志GL_COMPILE_AND_EXECUTE。

每当希望在服务器上绘制这个盒子时，可以像下面这样调用一个函数：

```
glCallList(BOX);
```

如同它对其他OpenGL函数的影响一样，当前的状态决定了显示列表中的图元要经历哪些变换。因此，如果在两次显示列表的执行之间改变了模-视变换矩阵或投影变换矩阵，那么该盒子会显示在不同的地方或者不会显示出来，就像下面的代码段所演示的：

```
glMatrixMode(GL_PROJECTION);
for(i= 1 ; i<5; i++)
{
     glLoadIdentity();
     gluOrtho2D(-2.0*i   , 2.0*i , -2.0*i , 2.0*i );
     glCallList(BOX);
}
```

每次执行glCallList时，这个盒子就被重新绘制，但是裁剪矩形一次比一次大。

接下来的几章将介绍各种变换矩阵，它们将使用户能够利用显示列表来建模。因为我们可以在一个显示列表内改变系统的状态，所以必须要小心避免这种改变所产生的不良（通常是意想不到的）效果。例如，前面所举的盒子显示列表示例就修改了绘制颜色。每当执行这个显示列表时，就把绘制颜色设置成了红色，程序中接下来定义的图元也将用红色绘制，除非这个颜色又被设置成了其他的值。

解决该问题最简单的保护措施就是使用OpenGL提供的矩阵栈和属性栈。栈（stack）是一种后进先出的数据结构。要保存当前的属性和矩阵的值，可以把它们分别放入或压入（push）栈的顶端；以后如果要恢复这些属性和矩阵的值，就可以把它们从栈中移走或者弹出（pop）。一个标准而且可靠的处理过程是：当进入一个显示列表时，总是把属性和矩阵压入它们各自的栈中，当退出这个显示列表时，再把它们从栈中弹出。因此，在一个显示列表的开头，通常会看到下面的函数调用：

```
glPushAttrib(GL_ALL_ATTRIB_BITS);
glPushMatrix();
```

而在一个显示列表的结尾，通常会看到下面的函数调用：

```
glPopAttrib();
glPopMatrix();
```

第10章将会广泛使用矩阵和属性栈来构建并显示层级模型。这里需要注意的是：尽管栈这种数据结构非常有助于建模，但每次把矩阵和属性压入或弹出栈时都会有性能开销。

还有几个附加的函数，它们可以使显示列表的操纵变得更容易。就像下一个例子将要说明的那样，用户经常希望使用多个显示列表。如果使用函数glGenLists(number)，就可以更容易地创建多个标识符连续的显示列表，这个函数的返回值是连续number个没有使用过的标识符中的第一个（或者称为基标识符）。函数glCallLists可以使用户用一个函数调用来执行多个显示列表。文本生成是一个很好的例子，它说明了如何有效利用显示列表的可选项，3.4.2节包含了许多这方面的OpenGL细节内容，也许读者在第一次阅读本章时可以跳过它。这一节介绍了为解决在文本生成中出现的棘手问题，OpenGL API提供给应用程序编程人员的灵活性。我们在第10章讨论图形学和Internet时会再次谈到其中的许多问题。

3.4.2　文本和显示列表

第2章介绍了笔划文本和光栅文本。不管选择使用哪种类型的文本，都要求描述一种字符集的代码量要合理。例如，假定使用光栅字体，其中每个字符是使用8 × 13的位图来存储，存储每个这样的字符需要13字节。如果想用最直接的方法显示一个字符串，那么可以在每一次想要显示它时向服务器发送它的每个字符。在这种方式下，每个字符至少需要传送13字节。如果只利用线段定义笔划字体，那么不同的字符可能需要不同数目的线段。如果用填充的多边形来定义字符，那么如图3.14所示，我们看到“I”的定义相当简单，但为了得到足够光滑的“O”就需要许多条线段。平均起来，为了表示一种笔划字体，每个字符需要的字节数超过13。对于显示大量文本的应用程序，如果每当需要显示某个字符时就把它发送给显示器，那么图形系统会承受很重的负荷。

(a)　　(b)

图3.14　笔划字符：(a) 填充的字符串；(b) 放大的轮廓

一种更有效的策略是对每个字符使用一个显示列表，把这种字体一次定义完毕，然后利用这些显示列表把字体存储在服务器上。这个解决办法和在标准的字母数字显示终端上显示位图字体所采用的办法相似。这些模式存储在终端里的只读存储器（read-only memory，ROM）中，对每个字符进行选择和显示时只基于一个字节——它的ASCII码。这里的差异只在于数量和质量。我们可以定义许多种字体，只要显示存储器能够装得下，而且我们可以像对待其他图形对象那样按照想要的方式对笔划字体进行平移、缩放和旋转。

我们可以用一种笔划字体和显示列表来定义并显示一个字符串（每个字符一个字节），这种方法为如何在OpenGL中使用显示列表提供了一个简单却重要的例子。这里所需的步骤基本上也可用于光栅字体。我们可以定义标准的96个可打印的ASCII码字符，也可以定义256个字符的扩展ACSII码字符集。

首先，定义一个函数OurFont(char c)，它将绘制任何可能出现在字符串中的ASCII字符c。这个函数的一种可能的形式如下：

```
void OurFont(char c)
    {
          switch(c)
          {
               case 'a':
                    ...
               break;
               case 'A':
                    ...
               break;
                    ...
          }
    }
```

在每一个case语句中，都必须注意字符之间的间隔，字符串里的每一个字符必须显示在前一个字符的右边。我们可以使用第4章将要介绍的平移函数来获得想要的字符间隔或者移动顶点的位置。假定正在定义字母“O”并且希望它正好可以绘制到一个单位正方形内。OurFont中对应的代码段可以编写如下：

```
case 'O':
    glTranslatef(0.5, 0.5, 0.0); /* 移动到中心 */
    glBegin{GL_QUAD_STRIP);
    for (i=0; i<=12; i++)  /* 12个顶点 */
    {
          angle = 3.14159 /6.0 * i; /* 30°的弧度值 */
          glVertex2f(0.4*cos(angle)+0.5; 0.4*sin(angle)+0.5);
          glVertex2f(0.5*cos(angle)+0.5, 0.5*sin(angle)+0.5);
}
glEnd();
break;
```

这段代码用了12个四边形来近似圆，每个四边形都将按照当前的状态进行填充。尽管本书要到第4章才讨论平移变换的全部功能，这里先解释一下代码中的平移函数的用法。因为处理的是二维字符，所以每个字符都定义在平面$z=0$内，并且可以用任何坐标系来定义字符。假定每一个字符都正好可以放到一个矩形框里[①]。通常的策略是从字符串的第一个字符的左下角开始，一次绘制一个字符，在绘制每个字符时都在该字符的矩形框的右下角结束，这个右下角同时也是下一个矩形框的左下角。

第一个平移移动到字符“O”所在的单位正方形框的中心，然后利用以该点为圆心的两个同心圆定义顶点（见图3.15）。我们可以把平移函数的作用想象成这样：它把位于该平移函数之后的所有绘制命令的原点都平移了。在定义了这12个四边形之后，我们移动到了矩形框的右下角。这两个平移变换的叠加使得我们位于绘制下一个字符的正确位置。注意，在这个例子中，我们不想把矩阵压入和弹出栈。其他字符可以用类似的方式来定义。

尽管代码看起来并不是很优雅，但是它对效率的影响并不大，因为只生成字符一次，然后就把它们作为一个编译好的显示列表发送到图形服务器。

① 不同字符的大小可能也不同，因此正好可以容纳下它们的矩形框的尺寸也不尽相同。如果定义的是固定宽度的单型字体（例如本书中显示代码所用的字体），那么所有字符的矩形框都具有相同尺寸。

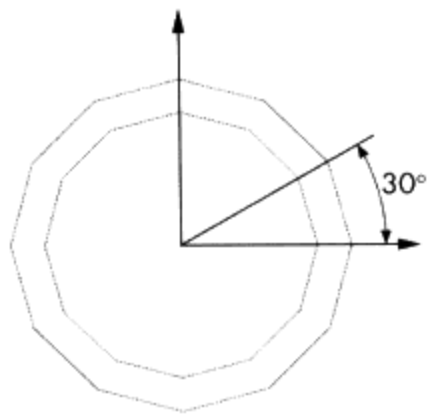


图3.15　字母“O”的绘制

假定我们想生成256个字符。借助OurFont函数，可以编写出下面的代码：

```
base = glGenLists(256); /* 返回256个连续可用的
                           标识符中的第一个索引 */
for(i=0; i<256; i++)
{
        glNewList(base + i, GL_COMPILE);
        OurFont(i);
        glEndList();
}
```

当希望使用这些显示列表来绘制单个字符时，为了确定显示列表的标识符，无须每次都把基标识符（base）和偏移量相加，可以像下面这样对基标识符进行设置：

```
glListBase(base);
```

最后，字符串的绘制通过下面的函数调用在服务器上完成：

```
char *text_string;

glCallLists( (GLint) strlen(text_string), GL_BYTE, text_string);
```

函数中利用了标准C函数库中的函数strlen来求出输入字符串text_string的长度。函数glCallLists的第一个参数是要执行的显示列表的数目。第三个参数是一个数组指针，这个数组的类型由第二个参数确定。第k个显示列表的标识符是基标识符（由glListBase设置）与字符数组中第k个字符的值之和。

3.4.3　GLUT中的字体

一般来说，人们更愿意使用已有的字体，而不是定义自己的字体。GLUT提供了几种光栅和笔划字体[①]。这些字体不需要显示列表，不过，在这一章最后一个例子里，我们创建的显示列表会包含一种GLUT字体。可以通过下面的函数调用来显示单型（monotype）字体或者称为等间隔字体的一个字符：

```
glutStrokeCharacter(GLUT_STROKE_MONO_ROMAN, int character)
```

GLUT_STROKE_ROMAN表示等间隔字体。读者在使用这些字体时应该小心，它们的大小（最大是大约120个单位）可能与用户程序其他部分的单位无关，因此，必须对它们进行缩放。为了控制字符的位置，通常在调用字符函数之前应用一个平移操作。此外，每次调用函数glutStrokeCharacter时都包括一个到字符框右下角的平移，从而为绘制下一个字符做好准备。缩放和平移都会影响

① 也可以利用窗口系统提供的字体，但这是以牺牲代码的可移植性为代价的。

OpenGL的状态，所以为了避免程序中后面定义的对象受到这些变换的影响，应该在必要的时候使用glPushMatrix和glPopMatrix。在第4章讨论变换之后读者就会知道如何有效地使用笔划字体了。

我们也可以用类似的方式生成光栅和位图字符。例如，下面的函数显示一个8 × 13字符：

```
glutBitmapCharacter(GLUT_BITMAP_8_BY_13, int character)
```

位图字符的定位比笔划字符的定位要简单得多，因为位图字符可以直接读入帧缓存，也无须进行几何变换，但显示笔划字符常常需要进行几何变换。光栅位置（raster position）也是OpenGL的状态之一，这个位置定义了下一个光栅图元将在哪里出现，它可由函数glRasterPos*()来设置。典型的做法是：用户程序先把光栅移动到想要的位置，然后再调用glutBitmapCharacter显示字符串中的第一个字符。光栅位置的改变不会对以后几何图元的绘制产生影响。如果字符有不同的宽度，可以使用函数glutBitmapWidth(font, char)来返回一个字符的宽度。然而，函数glutBitmapCharacter会自动把光栅位置前移，所以一般不需要操纵光栅位置，除非希望在屏幕的其他位置定义一串字符。这一章的后面有一个画图程序的例子，届时会在显示列表中使用位图字符。

3.5　显示列表和建模

如果看一看交互式计算机图形学的广泛应用，就会发现其中很大一部分涉及某种交互式建模技术。例如，借助CAD应用程序，用户可以交互式地设计建筑、电路模型或计算机动画。从计算机图形学的角度来看，我们必须为这样的应用程序设计用户接口，这将是下面几节要介绍的内容。但是和大多数计算机应用程序一样，这些应用程序的基础是用到各种数据结构的概念模型和实现模型，这种数据结构应该为交互提供有效的支持。

因为在显示列表中可以调用其他显示列表，所以它们是建造层级模型的有力工具。层级模型能够表示出一个模型中各个部分之间的关系。考虑一个简单的人脸建模系统，它可以生成如图3.16所示的卡通图像。每张人脸都有两个一样的眼睛和两个一样的耳朵，还有外轮廓、一个鼻子和一个嘴巴。我们可以利用显示列表来确定各个组成部分，下面的代码给出了一个例子：

```
#define EYE 1 /* 或者某个其他的整数 */

glNewList(EYE);
/* 绘制眼睛的代码 */
glEndList();
```

然后可以利用变换操作（见第5章）把每个组成部分放到合适的位置，下面的代码以此方式来绘制人脸：

```
#define FACE 2 /* 或者某个其他的整数*/

glNewList(FACE);
   /* 绘制轮廓 */
   glTranslatef(....) /* 右眼的位置 */
   glCallList(EYE);
   glTranslatef(....) /* 左眼的位置 */
   glCallList(EYE);
   glTranslatef(....) /* 鼻子的位置 */
   glCallList(NOSE);

/* 与上面类似的代码，用于绘制耳朵和嘴 */

glEndList();
```

这种方法有一些重要的优点。第一，如果同样的组件有多个实例，只需多次调用相同的显示列表；第二，如果想创建不同的人脸，可以只修改一个或者多个组件的显示列表而不用修改人脸的显示列表，因为人脸的结构没有变。

图 3.16　简单的卡通人脸

第 10 章讨论层级建模的时候还会再次讨论显示列表。因为多数 OpenGL 代码可以封装到 glNewList 和 glEndList 之间，所以读者应该很容易就能把多数代码转换成显示列表的形式。

3.6　事件驱动输入编程

本节通过一些简单的例子来讨论事件驱动输入，这些例子利用了3.2节介绍的回调机制。我们检查窗口系统能够识别出的各种事件，并且为应用程序需要处理的事件编写回调函数，以此来控制应用程序如何对这些事件做出响应。

3.6.1　使用定位设备

我们先来修改第 2 章的 Sierpinski 镂垫程序中的 main 函数。在 main 函数的最初版本中，我们使用GLUT库中的函数在屏幕上创建一个窗口，然后通过执行函数glutMainLoop进入事件循环。在那一章里，我们进入事件循环后什么也没做，甚至不能终止程序，除非通过与具体系统有关的外部机制，比如在键盘上按下Ctrl+C。我们将在第一个例子中弥补这个缺陷，即通过利用一个定位设备来终止程序。终止任务是这样完成的：当按下鼠标的某个按键时，程序会执行标准C语言的终止函数 exit。

我们只讨论GLUT能够识别的事件。像X Window操作系统或者Microsoft Windows这样的标准窗口系统可以识别出更多的事件，而且不同的系统能够识别的事件也不尽相同。不过，GLUT库能识别的事件不是很多，但这些事件是大多数窗口系统所共有的，并且对开发基本的交互式图形程序来说也够用了。因为有人已经在一些主要的窗口系统上实现了GLUT，所以应用程序在重新编译后就可以在不同的系统上运行。

我们一般把定位设备假定成鼠标，不过定位设备也可以是跟踪球或者数据板，有两类事件与定位设备相关联。如果鼠标在移动时有某个按键被按下，那么就发生了移动事件（move event）。如果鼠标在移动时没有按键被按下，这个事件称为被动移动事件（passive move event）。移动事件发生后，应用程序就可以获取鼠标的位置，也就是测量到的数据。当鼠标的一个按键被按下或者释放时，就发生了鼠标事件（mouse event）。一个处于按下状态的按键不会产生鼠标事件，直到它被释放。返回的信息包括生成这个事件的鼠标，在这个事件之后按键的状态（释放或者按下），以及在窗口坐标系（原点位于该窗口的左上角）中跟踪这个鼠标的光标的位置。我们通常在 main 函数中使用下面的 GLUT 函数来注册鼠标回调函数：

```
glutMouseFunc(myMouse);
```

鼠标回调函数必须为下面的形式并且由应用程序编程人员来编写：

```
void myMouse(int button, int state, int x, int y)
```

在回调函数里，我们定义在这个指定的事件发生时需要采取的行动。与鼠标按键和状态的多种可能的组合相对应，在鼠标回调函数中可能会定义多个行动。在这个简单的例子中，我们想要通过按下鼠标左键来终止程序。实现这一任务的回调函数只需要一行代码，如下所示：

```
void myMouse(int button, int state, int x, int y)
{
     if(button == GLUT_LEFT_BUTTON && state == GLUT_DOWN)
               exit(0);
}
```

如果发生了任何其他的鼠标事件（例如按下其他的某个按键），那么程序不会有任何响应，因为在这个回调函数里没有定义对应于这些事件的处理代码。

从下一个例子中可以看出第2章介绍的这种程序结构的优点。我们来编写一个程序，当鼠标左键按下时它会在屏幕上光标出现的位置绘制一个小的正方形。我们将使用鼠标中键来终止程序[①]。

首先来看主程序，它和前一个例子中的主程序基本上一样[②]：

```
int main(int argc, char **argv)
{
  glutInit(&argc,argv);
  glutInitWindowSize(ww, wh); /* 窗口的初始大小由全局变量定义 */
  glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
  glutCreateWindow("square");
  myInit();
  glutReshapeFunc(myReshape);
  glutMouseFunc(myMouse);
  glutDisplayFunc(myDisplay);
  glutMainLoop();
}
```

每当改变窗口的大小（比如由于用户执行某个交互操作）时，窗口调整事件（reshape event）就会发生，下一节将讨论该事件。在这个例子中不需要用显示回调函数来绘制图形，因为只有发生鼠标事件才会生成图元。不过由于GLUT要求所有的程序都要有显示回调函数，所以必须在程序中包含它，尽管在这里它的函数体很简单：

```
void myDisplay()
{
   glClear(GL_COLOR_BUFFER_BIT);
}
```

鼠标回调函数仍然是myMouse：

```
void myMouse(int btn, int state, int x, int y)
{
  if(btn==GLUT_LEFT_BUTTON && state==GLUT_DOWN) drawSquare(x,y);
  if(btn==GLUT_RIGHT_BUTTON && state==GLUT_DOWN) exit();
}
```

因为函数drawSquare只负责绘制图元，所以必须在其他地方对需要的属性进行设置，例如在初始化函数myInit中。

---

① 这里假定系统有一个三键鼠标。对于单键或者两键鼠标，可以把可用的鼠标键和键盘上的功能键（如Ctrl、Alt、Shift）组合起来实现同样的效果。

② 对回调函数的命名规范与 *OpenGL Programming Guide*[Ope07]类似。

我们需要三个全局变量。窗口的大小可能会在程序执行时动态地改变，reshape回调函数和绘图函数drawSquare都应该能够获取它的当前尺寸。如果绘制的正方形的大小需要改变，那么就会发现用全局变量来定义正方形的大小是有益的。初始化程序把裁剪窗口设置成和在main函数中创建的窗口一样大，并且把视口对应于整个窗口。清空窗口所用的颜色为黑色。注意，这里对窗口和视口的设置可以省略，因为我们仅仅是把它们设置为默认值。然而，把下面的代码和3.6.2节中的reshape回调函数进行比较有助于说明问题：

```
/* 全局变量 */

GLsizei wh = 500, ww = 500; /* 窗口的初始宽度和高度 */
GLfloat size = 3.0;    /* 正方形边长的一半 */

void myInit()
{
   /* 设置初始观察条件 */

      glViewport(0,0,ww,wh);
      glMatrixMode(GL_PROJECTION);
      glLoadIdentity();
      gluOrtho2D(0.0, (GLdouble) ww , 0.0, (GLdouble) wh);
      glMatrixMode(GL_MODELVIEW);
      glClearColor (0.0, 0.0, 0.0, 1.0);
      glColor3f(1.0, 0.0, 0.0); /* 绘制红色的正方形 */
}
```

在编写正方形绘制函数时必须考虑到一点：从鼠标事件返回的鼠标位置位于窗口坐标系中，该坐标系的原点在窗口的左上角。因此，必须利用窗口的当前高度（全局变量wh）减去返回的y值，代码如下所示：

```
void drawSquare(int x, int y)
{
      y=wh-y;

      glBegin(GL_POLYGON);
          glVertex2f(x+size, y+size);
          glVertex2f(x-size, y+size);
          glVertex2f(x-size, y-size);
          glVertex2f(x+size, y-size);
      glEnd();
      glFlush();
}
```

注意，drawSquare函数可以在它每次被调用时设置一种新的颜色，这样可以增加趣味性。在插入必要的include语句之后，只要窗口的尺寸保持不变，我们所编写的这个程序就可以正常运行。

3.6.2　窗口事件

大多数窗口系统允许用户交互式地调整窗口的大小，这个过程通常是用鼠标把窗口的一角拖曳到新的位置。这个事件是窗口事件（window event）的一个例子。如果发生了这样的事件，用户程序可以决定如何处理该事件[①]。如果窗口的大小改变了，则必须考虑三个问题：

① 和大多数其他的回调函数不同，有一个默认的reshape回调函数把视口简单地设置成和新窗口一样的大小，这可能不是用户想要的。

1. 在旧窗口中的所有对象都要重新绘制在新窗口中吗?
2. 如果新窗口的宽高比和旧窗口的宽高比不同，那么该做什么?
3. 如果新窗口的尺寸和旧窗口的尺寸不同，那么要改变图元的大小或者属性吗?

这些问题中的任何一个都不只有唯一的答案。如果是在显示真实场景的图像，那么reshape回调函数也许应该确保绘制的对象不会发生变形。但这个选择可能意味着改变大小之后的窗口有一部分没有利用起来或者场景中的某部分不能显示在窗口中。如果想在新窗口中重新绘制旧窗口中的对象，那么需要一种机制来存储和再次调用它们。为了再次绘制窗口中的对象，我们常常把所有的绘制代码封装到单独一个函数中，比如第2章中的myDisplay函数，它被注册为显示回调函数。但是在这个例子中，也许这么做不是最好的选择，因为我们要绘制的图形是在程序执行时通过与用户的交互来确定的。

在绘制正方形的这个例子中，不管窗口的形状和大小如何，我们都要确保绘制出来的正方形大小相同。每次调整窗口大小时都要清屏，并且把整个新窗口作为绘图区域。reshape事件返回新窗口的高度和宽度，我们按照这些返回值用gluOrtho2D函数创建一个新的OpenGL裁剪窗口，并且把新视口的宽高比设置成和裁剪窗口的一样。然后把绘图窗口清屏成黑色。这样我们编写的reshape回调函数如下：

```
void myReshape(GLsizei w, GLsizei h)
{

      /* 调整裁剪窗口 */
      glMatrixMode(GL_PROJECTION);
      glLoadIdentity();
      gluOrtho2D(0.0, (GLdouble)w, 0.0, (GLdouble)h);
      glMatrixMode(GL_MODELVIEW);
      glLoadIdentity();

      /* 调整视口并清屏 */
      glViewport(0,0,w,h);

      /* 把新窗口的大小保存到全局变量中 */

      ww=w;
      wh=h;
}
```

附录A给出了这个绘制正方形的完整程序。

这里还有其他可能的情形。为了和窗口大小的增大或减小相匹配，可以改变正方形的大小。我们没有考虑其他的事件，比如窗口移动到别的地方但窗口的大小并没有改变，当用户把窗口拖曳到一个新的位置时就会发生这个事件。如果窗口被另一个窗口挡在下面，后来又显示出来(或者成为最上面的当前窗口)，那么我们也没有确定当这个事件发生时程序要做什么。这些事件都有相应的回调函数，可以为它们编写像myReshape那样的简单函数或者依靠GLUT的默认行为。我们可以对程序做出的另一个简单修改是：只要鼠标按键处于被按下的状态就可以绘制新的正方形。相应的回调函数是鼠标移动回调函数，可以通过下面的函数来对它进行定义：

```
glutMotionFunc(drawSquare);
```

每当系统检测到鼠标的移动时，程序就绘制一个新的正方形，这相当于使用一个具有正方形笔尖的画笔来画图。

注意，reshape回调函数会调用显示回调函数。这个例子中的显示回调函数对窗口执行了清屏操作，所以屏幕上的正方形都消失了。如果想保留这些正方形并且把它们重新绘制在大小改变之后的新窗口中，那么必须有一种机制来保存它们的描述。3.10节将再次回到这个问题上来。

3.6.3　键盘事件

我们也可以把键盘用做输入设备。当鼠标位于窗口内并且键盘有某个键被按下或释放时，就发生了键盘事件。可以使用GLUT库中的函数glutKeyboardFunc来注册按下一个键所产生的事件的回调函数，而使用glutKeyboardUpFunc来注册释放一个键所产生的事件的回调函数。

当发生键盘事件时，系统返回用来生成该事件的键盘按键的ASCII码和鼠标的位置。所有的键盘回调函数都使用下面的单个回调函数注册：

```
glutKeyboardFunc(myKey);
```

例如，如果希望使用键盘只是为了退出程序，那么可以使用下面的回调函数：

```
void myKey(unsigned char key, int x, int y)
{
    if(key=='q' || key == 'Q') exit( );
}
```

GLUT中有一个函数glutGetModifiers，通过它用户可以使用功能键（如Ctrl键和Alt键等）定义程序的行为。如果使用的是单键或者双键鼠标，那么这些特殊键可能是非常重要的，因为借助它们可以实现和这一章假定使用的三键鼠标相同的功能。在这一章末尾的建议阅读资料中，读者可以找到关于这些函数的更多信息。

3.6.4　显示和空闲回调函数

在剩下的回调函数中还有两个值得注意。我们在第2章已经看到了显示回调函数，在GLUT中可以通过下面的函数调用来注册该显示回调函数：

```
glutDisplayFunc(myDisplay);
```

当GLUT确定应该重新绘制显示窗口时就会调用显示回调函数。窗口首次打开就属于这种情形，还有一种可能是发生了窗口大小调整事件。因为当窗口第一次打开时会发生一个显示事件，所以把生成大多数非交互式输出的代码放到显示回调函数中是合适的。

显示回调函数也可以用在其他场合（比如动画），在这种情形中程序里定义的各种值都可能改变。我们还可以用GLUT打开多个窗口。当前的窗口也属于GLUT的状态。通过改变当前窗口，可以把不同的对象绘制到不同的窗口中。我们还可以把窗口图标化（iconify），即用一个小的符号或者图片代替窗口。因此，在交互式程序和动画程序中会多次重复执行显示回调函数。我们并不直接调用它，而是使用下面的GLUT函数：

```
glutPostRedisplay();
```

这个函数对GLUT内部主循环中的一个标志进行设置，以此来指示需要重新绘制，所以调用这个函数而不直接调用显示回调函数可以避免额外的或者不必要的屏幕绘制。每当执行到这个主循环的结尾时，GLUT就利用那个标志来确定是否应该执行显示回调函数。因此，使用glutPostRedisplay可以确保程序在处理事件的每次循环中只进行一次绘制。

当没有其他事件发生时，GLUT会调用空闲回调（idle callback）函数，其默认返回值是空函数指针。空闲回调函数经常用来在没有任何其他事件发生时利用某个显示函数持续地生成图元。

我们可以在程序执行期间更改大多数回调函数，只需要重新注册一个新的回调函数即可。我们还可以通过把对应的回调函数设置成 NULL 来禁用某个回调函数。

3.6.5　窗口管理

GLUT还支持多窗口和一个给定窗口的多个子窗口。我们可以利用下面的函数调用打开第二个顶层窗口（其窗口标题为“second window”）：

```
id=glutCreateWindow("second window");
```

我们可以把这个窗口设置为当前窗口，从而使它成为绘制对象所用的窗口。为此，把 glutCreateWindow 返回的整数值作为参数来调用下面的函数：

```
glutSetWindow(id);
```

通过在调用 glutCreateWindow 之前调用 glutInitDisplayMode，还可以设置该窗口的显示模式，使得它的属性不同于其他窗口。另外，因为所使用的回调函数针对的是当前窗口，所以每一个窗口都有其自己的回调函数集。

3.7　菜单

我们可以利用各种图元和鼠标回调函数来构造各种图形输入设备。例如，为了构造如图 3.17 所示的滑动条，可以利用填充矩形绘制设备，用文本表示任何文本标签，用鼠标获取位置。然而，大多数这样的代码编写起来很麻烦，尤其是当我们试图创建外观优美且易于使用的图形设备（构件）时。大多数窗口系统提供了包含一组构件的工具包，但是因为我们不想把讨论局限于任何特定的窗口系统，所以不打算讨论这些构件的实现细节。幸运的是，GLUT 提供了一个附加的特性：弹出式菜单（pop-up menu），我们可以利用鼠标来使用它，这样就能够创建更复杂的交互式应用程序了。

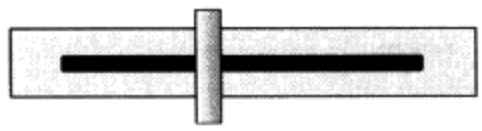


图 3.17　滑动条

使用菜单包括几个简单的步骤。我们必须定义对应于菜单中每个选项的功能，还必须把菜单和鼠标的某个特定按键关联起来。最后，我们必须为每个菜单注册一个回调函数。我们用一个包含三个选项的弹出式菜单来演示简单的菜单。第一个选项允许我们退出程序，第二个和第三个选项用来改变 drawSquare 函数所绘制的正方形的大小。我们把菜单回调函数命名为 demo_menu，把那些创建菜单并把菜单和鼠标右键关联起来的函数调用放到 main 函数里，这些函数调用的代码如下：

```
glutCreateMenu(demo_menu);
glutAddMenuEntry("quit",1);
glutAddMenuEntry("increase square size", 2);
glutAddMenuEntry("decrease square size", 3);
glutAttachMenu(GLUT_RIGHT_BUTTON);
```

函数 glutCreateMenu 注册菜单回调函数 demo_menu。

上面用来定义菜单选项的函数的第二个参数是当该选项被选中时传递给回调函数的标识符。因此，可以像下面这样定义菜单回调函数：

```
void demo_menu(int id)
{
     switch(id)
     {
          case 1: exit(0);
          break;
          case 2: size = 2 * size;
          break;
          case 3: if(size > 1) size = size/2;
          break;
     }
     glutPostRedisplay( );
}
```

调用函数glutPostRedisplay使得GLUT调用显示回调函数重新绘制窗口，这样重新绘制屏幕后菜单就不再出现了。

GLUT也支持如图3.18所示的级联菜单。例如，假定希望创建的主菜单只有两个选项。第一个选项仍旧用于终止程序，但现在第二个选项用于弹出一个子菜单。这个子菜单包含两个选项，用来改变正方形绘制程序中所绘正方形的大小。下面这些对菜单进行设置的代码（在main函数中）是容易理解的：

```
sub_menu = glutCreateMenu(size_menu);
glutAddMenuEntry("Increase square size", 2);
glutAddMenuEntry("Decrease square size", 3);
glutCreateMenu(top_menu);
glutAddMenuEntry("Quit",1);
glutAddSubMenu("Resize", sub_menu);
glutAttachMenu(GLUT_RIGHT_BUTTON);
```

作为一个简单的练习，请读者自己编写size_menu和top_menu的菜单回调函数（见习题3.5）。

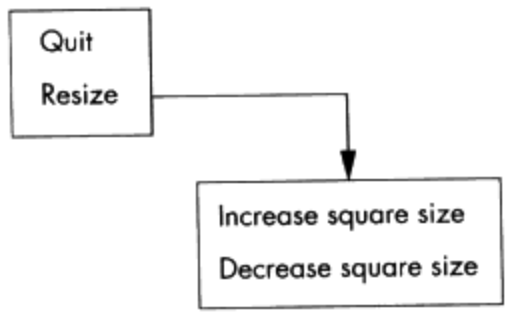


图3.18　级联菜单的结构

3.8　拾取

拾取（picking）是一种逻辑输入操作，它允许用户识别屏幕上的一个对象。尽管拾取使用了定位设备，但用户想返回给应用程序的信息并不是一个位置。在现代图形绘制系统中，实现一个拾取设备比实现一个定位器要困难得多。

但情况也并非总是如此，早期的显示处理器通过光笔很容易就可以实现拾取。屏幕的每一次刷新都从一个精确的时间开始，当刷新信号经过光笔的传感器时，光笔就会生成一个中断。通过比较中断的时间和刷新开始的时间，处理器就可以识别出显示列表中一个确切的位置，从而能够确定正在显示的是哪个对象。

在现代系统中难以实现拾取的一个原因是图形绘制流水线在本质上是基于前向绘制的[①]。图元由应用程序定义，然后依次经过几何操作，光栅化和片元处理，最后到达帧缓存。尽管这个绘制过程的大部分处理步骤在数学意义上是可逆的，但硬件是不可逆的。因此，不能通过一个直接的计算从显示器上的一个位置求出它所对应的图元。此外，唯一性也是潜在的问题（见习题3.11和习题3.12）。

至少有三个办法可以解决这个难题。在一个称为选择（selection）的过程里，通过对裁剪区域和视口的调整使得我们能够跟踪在一个小裁剪区域中有哪些图元被绘制在光标附近的区域中，这些图元保存在一个命中列表（hit list）中，以后用户程序可以检查其中的图元。OpenGL支持这种方法，我们在考察两种更简单但具有一般性欠缺的策略之后再来讨论它。

一种简单的方法是对感兴趣的对象使用与坐标轴对齐的（axis-aligned）包围盒（bounding box）或包围区域（bounding extent）。一个对象的包围区域是指与坐标轴对齐并且包含这个对象的最小矩形。对于二维应用程序，确定与对象坐标系或者世界坐标系中的矩形相对应的屏幕坐标系中的矩形要相对容易些。对于三维应用程序，包围盒是长方体，我们在第4章再来讨论这种情况。如果能够在应用程序中采用一种简单的数据结构把对象及其包围盒关联起来，那么就可以实现近似的拾取操作。

另一种简单的方法要用到后端缓存并且要进行一次附加的绘制过程。当我们使用双缓存模式（见3.11.2节）时，使用两个颜色缓存：一个前端缓存和一个后端缓存。因为后端缓存的内容不会显示在屏幕上，所以可以把它用于绘制以外的其他目的。假定把对象绘制到后端缓存中，并且每个对象的颜色都不同。应用程序编程人员可以随意确定一个对象的内容，采用的方法是每当在程序中定义一个新的对象时，只需简单地修改对象的颜色即可[②]。

我们可以分4步来实现拾取操作，并通过用户在应用程序中定义的拾取函数来对其进行初始化。第一步，用拾取颜色把对象绘制到后端缓存里。第二步，使用鼠标回调函数来获取鼠标的位置。第三步，使用函数glReadPixels( )来获取帧缓存中与鼠标位置相对应的位置的颜色。第四步，搜索一个颜色查找表，从中确定哪个对象对应于这个颜色。我们必须在这个过程之后对后端缓存进行一次常规的绘制。关于函数glReadPixels( )的使用还有一些细微之处需要考虑，我们将在第8章考虑这些细节问题。

3.8.1　拾取和选择模式

在OpenGL绘制流水线中实现拾取的困难之处在于：不能直接从鼠标显示在屏幕上的位置按相反的方向去确定屏幕上离该位置最近的那个已绘制图元。OpenGL使用一种称为选择模式（selection mode）的绘制模式来实现拾取。这个过程有些复杂，并且每次执行拾取操作时都要进行一次额外的绘制。选择模式的基本思想是场景中的对象可以绘制到颜色缓存里，但不一定非要绘制到颜色缓存里。当我们绘制对象时，OpenGL可以跟踪并确定哪些对象被绘制到了任何选定的区域。为此，OpenGL需要确定对象是否位于一个指定的裁剪体中，这个裁剪体不一定和应用程序中定义的裁剪体相同。使用选择模式进行拾取需要用到许多步骤和函数，我们将考察每个步骤，然后在一个简单的程序中把所有步骤都放到一起。我们可以使用函数glRenderMode来选择某种模式，OpenGL提供了三种模式：绘制到颜色缓存的常规绘制模式（GL_RENDER）、选择模式（GL_SELECT）和反馈模式（GL_FEEDBACK）。反馈模式可以用于获取已经绘制的图元列表，这里并不讨论这种模式。glRenderMode的返回值可以用来确定位于裁剪体中的图元的数目。

① 即从几何数据到生成像素这个方向，而后向绘制方向则与此相反。——译者注

② 除后端缓存之外，大部分最新的图形卡都提供了可用于拾取操作的额外缓存。

当进入选择模式并绘制一个场景时，裁剪体中的每一个图元都产生一个称为命中（hit）的消息，这个消息存放在一个称为名称栈（name stack）的缓存中。我们用函数glSelectBuffer来确定一个用于存储选择数据的数组。OpenGL提供了4个用于操作名称栈的函数，分别用来初始化名称栈，把信息压入和弹出名称栈，以及操纵名称栈的栈顶元素。我们把生成的信息存储在一个称为命中列表（hit list）的结构中，并在绘制之后通过检查该命中列表来获得拾取操作所需的信息。下面的函数glSelectBuffer指定了一个用于存储选择数据的大小为n的数组缓存：

```
void glSelectBuffer(GLsizei n, GLunint *buff)
```

函数glInitNames用来初始化名称栈：

```
void glInitNames( )
```

函数glPushName用来把名称压入名称栈：

```
void glPushName(GLuint name)
```

函数glPopName用来把栈顶的名称从名称栈中弹出：

```
void glPopName( )
```

最后，函数glLoadName用某个名称替换名称栈的栈顶元素：

```
void glLoadName(GLuint name)
```

通常情况下，要识别的每个对象都是由一组图元组成的，我们把同一个整数名称赋给组成某个对象的一组图元。在绘制某个对象之前，需要把它的名称压入名称栈。我们不能把某个名称压入空栈里，因此当对名称栈进行初始化时，通常把一个没用的名称压入名称栈。例如：

```
glInitNames();
glPushName(0);
```

我们一般利用鼠标回调函数来进入选择模式，并在鼠标回调函数结束之前退出选择模式。当回到常规的绘制模式时，函数glRenderMode返回已处理的命中数量。然后检查命中列表。为了在希望的区域（通常是鼠标位置附近的区域）命中对象，还需要在鼠标回调函数中对裁剪体进行设置。有两种设置裁剪体的方法。我们可以通过函数glOrtho2D简单地定义一个视见体（或使用其他的观察函数）。首先需要使用函数glPushMatrix来存储当前的裁剪体，此时无论鼠标位于何处，任何位于这个新裁剪体内部的图元都会产生一个命中消息。这种方法适合于选择操作，但是对于拾取操作，只需要识别那些绘制在光标附近的对象。

假定希望所有绘制到一个用户定义的小矩形区域内的对象都以光标为中心。矩阵的大小是拾取操作敏感性的一个度量单位。该矩形是视口中一个小区域。如果已知视口、光标的位置、矩形的大小以及裁剪窗口，就可以找到一个新的裁剪窗口，所有位于新裁剪窗口内的对象都被绘制到整个视口中。从数学的角度看，这是一个涉及投影矩阵求逆运算的比例缩放问题。我们可以利用OpenGL的GLU库中的函数gluPickMatrix来执行这种计算，当使用选择模式时，必须在调用函数glOrtho2D之前应用函数gluPickMatrix。我们按下面的形式调用函数gluPickMatrix：

```
gluPickMatrix( x,y,w,h,*vp);
```

该函数为拾取操作创建了一个投影矩阵，它把绘制限制在一个以(x, y)为中心，大小为w × h的区域内，而(x, y)是在视口vp内的窗口坐标系中的一个坐标点。图3.19说明了二维应用程序的拾取处理过程。在图3.19(a)中，可以看到标准的裁剪窗口和裁剪后在屏幕视口中显示的图像。我们还可以看到一个其周围用小矩形包围的光标，这意味着如果某个图元正好绘制到这个小矩形区域内，那么

就认为该对象被命中。当调用函数 gluPickMatrix 处理后，图 3.19(b)显示了修改后的裁剪窗口和屏幕上显示的图像。注意，在世界坐标系中的裁剪窗口已经改变了大小，因此只有位于拾取矩形区域内的那些对象才出现在新的裁剪窗口内部，并且经变换后占据了整个视口。正是因为这个原因，所以我们并不希望该图像出现在屏幕上。

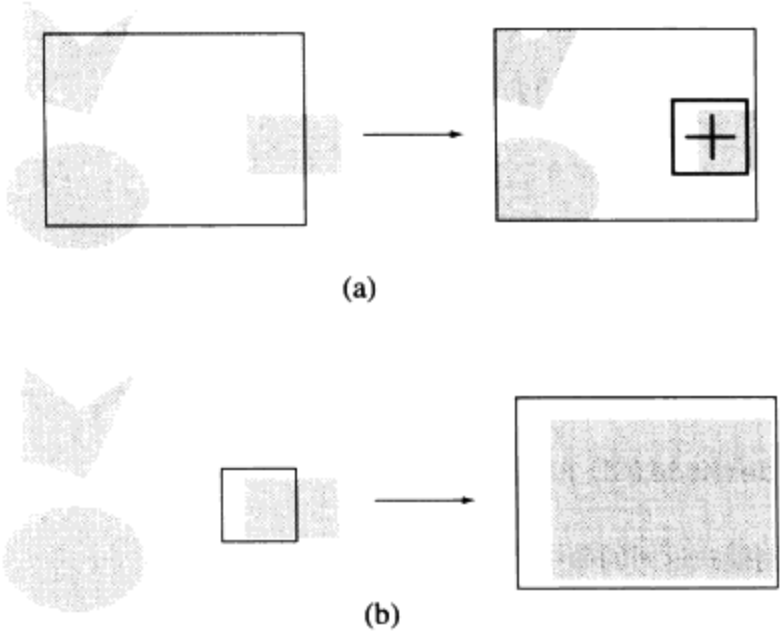


图 3.19　二维拾取操作的例子：(a) 标准的裁剪窗口和显示的图像；(b) 应用拾取矩阵后的裁剪窗口和显示的图像

假定在初始化函数或在 reshape 回调函数中对常规的绘制过程已经设置了相应的观察条件，那么鼠标和显示回调函数的代码形式如下所示：

```
#define N 2 /* 光标周围大小为 N × N 的拾取区域 */

 void mouse(int button, int state, int x, int y )
{
    GLuint nameBuffer[SIZE]; /* SIZE 常量定义在其他的地方 */
    GLint hits;
    GLint viewport[4];
    If( button == GLUT_LEFT_BUTTON && state == GLUT_DOWN)
    {
        /* 初始化名称栈 */

        glInitNames( );
        glPushName( 0 );
        glSelectBuffer( SIZE, nameBuffer);

        /*  为选择模式设置观察条件 */

        glGetIntegerv(GL_VIEWPORT, viewport);
        glMatrixMode(GL_PROJECTION);

        /* 保存初始的观察矩阵*/

        glPushMatrix();
        glLoadIdentity( );

        /* 光标周围大小为 N × N 的拾取区域  */
        /* 必须把鼠标位置的 y 坐标逆向变换到世界坐标系中 */
```

```
        gluPickMatrix( (GLdouble) x, (GLdouble)
            ( viewport[3] - y), N, N, viewport);

        /* 定义裁剪窗口，它与reshape回调函数中定义的裁剪窗口相同 */

        gluOrtho2D (xmin, xmax, ymin, ymax);

        draw_objects(GL_SELECT);
        glMatrixMode(GL_PROJECTION);

        /* 恢复观察矩阵 */

        glPopMatrix();
        glFlush();

        /* 返回到常规的绘制模式 */

        hits = glRenderMode(GL_RENDER);

        /* 处理选择绘制模式返回的命中消息 */

        processHits(hits, nameBuff);
        /* 常规的绘制过程 */

        glutPostRedisplay();
    }
}
```

```
void display( )
{
    glClear( GL_COLOR_BUFFER_BIT);
    draw_objects(GL_RENDER);
    glFlush();
}
```

注意，必须在鼠标回调函数中直接调用用于绘制对象的函数draw_objects，而不是两次使用函数glutPostRedisplay，这是因为在每个事件循环中，GLUT函数只能执行一次显示回调函数。下面这个简单的函数绘制两个部分重叠的矩形：

```
void drawObjects(GLenum mode)
{
    if(mode == GL_SELECT) glLoadName(1);
    glColor3f(1.0, 0.0, 0.0);
    glRectf(-0.5, -0.5, 1.0, 1.0);

    if(mode == GL_SELECT) glLoadName(2);
    glColor3f(0.0, 0.0, 1.0);
    glRectf(-1.0, -1.0, 0.5, 0.5);
}
```

我们在上面的代码中用到了函数glRectf (xmin, ymin, xmax, ymax)，使用这样的单个函数可以定义一个与坐标轴对齐的矩形。

注意，对于上述情形，我们只需改变名称栈中的栈顶元素。如果使用第10章将要介绍的层级对象，那么对于某个特定的命中对象，可以使用函数glPushName把多个名称压入栈中。对于一个

包含多个部件的层级对象，在光标附近的所有部件都要把它们的名称压入同一个栈中。最后一个需要编写的函数就是用于检查名称栈的函数。每当单击鼠标左键时，我们将通过该函数把存储在栈中命中的图元数量以及拾取的对象的名称打印出来。每当命中对象时，在命中缓存中都会存储一条相应的记录。因此，在鼠标光标附近绘制的每个对象都会在命中缓存中产生一条记录。如果由于没有图元在光标的附近绘制，从而导致在命中列表中没有记录任何对象的名称，那么命中记录为零。否则，我们发现在命中缓存中以整数类型储存了三类信息。第一类信息是当有对象命中时名称栈中的名称数量。对于前面举的这个例子，名称的数量只能取1。紧跟其后的第二类信息是两个整数，它们分别表示当前命中图元所有顶点的最小深度值和最大深度值。因为我们处理的是二维图形，所以这两个整数值对我们没有使用价值。对于三维应用程序，可以利用这两个整数值来确定离视点最近的拾取对象。紧跟这三个整数值之后的第三类信息是名称栈的入口地址。对于前面举的这个例子，我们将得到红色或蓝色矩形的标识符（1或者2）。下面的函数会把这个信息打印出来：

```
 void processHits (GLint hits, GLuint buffer[])
{
   unsigned int i, j;
   GLuint names, *ptr;

   printf ("hits = %d\n", hits);
   ptr = (GLuint *) buffer;

    /* 对命中的次数进行循环 */

   for (i = 0; i < hits; i++)
   {
      names = *ptr;

      /* 忽略名称的数量和深度信息 */

      ptr += 3;

      /* 检查命中记录中的每个名称 */

     for (j = 0; j < names; j++)
       {
        if(*ptr==1) printf ("red rectangle\n");
        else printf ("blue rectangle\n");

       /* 循环到下一个命中记录*/

        ptr++;
      }
   }
}
```

实现拾取操作还需要解决很多其他的困难。例如，如果某个对象定义为层级结构（就像我们将在第10章讨论的层级对象），那么它只是对象集中的一个成员。当使用定位设备拾取某个对象时，该对象所在的对象集中所有对象的成员列表都要返回给用户程序。

通常情况下，图形API并不明确说明必须把光标放在离要识别的对象多近的地方才能拾取该对象。之所以不考虑光标与拾取对象之间的实际靠近距离，一个原因是为了允许使用多种不同的方法实现拾取操作。例如，如果采用包围盒的方法，那么在拾取对象时也许就不必把光标指向对象内部

的任意点了。不必精确地考虑光标与拾取对象之间的靠近距离的另一个原因是：用户本身不可能精确地使用定位设备。尽管拾取对象时指向的显示设备可能具有很高的分辨率，但是我们发现要用定位设备可靠地定位屏幕上某个指定像素的位置是非常困难的。

3.9　一个简单的CAD程序

有很多有关计算机辅助设计（CAD）的应用实例，如交互式绘图、机械零部件的设计和计算机游戏中的人物建模等。虽然CAD的应用非常广泛，但是在图形和交互方面存在很多共性的地方，包括：

- 使用多窗口和视口来显示各种信息。
- 建立、删除和保存用户定义的对象的能力。
- 使用菜单、键盘和鼠标的多通道交互操作。

OpenGL的主要功能是绘制几何对象。我们已经利用GLUT库在图形应用程序中增加了一些基本的交互方法。无论是OpenGL还是GLUT都没有提供太多有关对象交互方面的概念，这主要是由于OpenGL在默认情况下使用了立即绘制模式。正如3.4节讲到的，显示列表提供了某种特定的方法来定义对象。然而，虽然我们可以建立和删除显示列表，但是不能对它进行编辑，从而限制了它在建模和CAD应用中的有效性。

有一种方法可以解决上述方法的局限性，即在OpenGL绘制层的上方增加一个更抽象的软件层，它为应用程序开发人员提供了更高级的API。这样的API要么是针对某类特定应用的专用API（如游戏开发或几何建模），要么是能够提供高级功能的通用API。第10章将要讨论的场景图就是这样一种方法，它提供了对层级数据结构的支持。因为基于这种方法提供的API位于应用层和底层API（如OpenGL）之间，所以这样的API工具包也称为中间件（middleware）。

在许多应用程序的开发中，为了避免这种附加的软件层所带来的系统开销，程序员更愿意直接使用OpenGL编写代码。就我们而言，虽然在亲自编写这个简单的CAD应用程序的过程中仍然要强调许多被上层API所隐藏的问题，但是这对于理解交互式计算机图形学是非常重要的。

下面将开发一个基于多边形建模的应用程序。虽然这个程序有点简单，但是它说明了CAD应用程序的许多特征。实际上，这个程序支持以下三个特征：

- 建立多边形
- 删除多边形
- 选择和移动多边形

如果要实现这些功能需求，就需要采用一个数据结构来支持这些操作并存储多边形，从而可以重新显示多边形。我们在这个例子中将使用简单的数组结构，尽管使用链表数据结构能更好地满足动态实现要求（见习题3.26）。

我们首先定义两个全局常量，它们分别用来存储多边形的最大数目以及每个多边形顶点的最大数目，如下所示：

```
#define MAX_POLYGONS 10
#define MAX_VERTICES 10
```

紧接着，我们定义一个数据类型用来存储单个多边形的数据。这个数据类型显然要包含多边形每个顶点的坐标位置。为了使这个例子尽可能地清晰，我们处理具有整数坐标值的顶点，因此鼠标回调

函数的返回值就是存储在多边形数据结构中的数据（鼠标位置的y坐标逆向变换到世界坐标系中之后）。多边形的数据结构至少要包含如下数据成员：

```
typedef struct polygon
{
   int nvertices;
   int x[MAX_VERTICES];
   int y[MAX_VERTICES];
} polygon;
```

我们使用下面的代码为多边形分配存储空间：

```
polygon polygons[MAX_POLYGONS];
```

如果要显示前N个多边形，那么可以使用如下形式的代码：

```
glClear(GL_COLOR_BUFFER_BIT);
for(i=0; i<N; i++)
{
  glBegin(GL_POLYGON);
     for(j=0; j<N; j++) glVertex2i(polygons[i].x[j], polygons[i].y[j]);
  glEnd();
}
```

我们需要实现的功能不仅仅是显示前N个多边形，还需要实现增加多边形和删除已定义的多边形的功能。我们可能希望给每个多边形赋一种颜色，还希望以交互方式在显示器屏幕上四处移动某个多边形。为了完成这些任务，需要利用菜单来指定某个希望执行的任务，还需要从已定义的多边形中选择某个多边形的功能，以及为增加的新多边形提供输入数据的手段。我们再次定义多边形的数据结构，这次使用了更复杂的数据结构：

```
typedef struct
polygon
{
      int color;
      bool used;
      int xmin, xmax, ymin, ymax;
      float xc, yc;
      int nvertices;
      int x[MAX_VERTICES];
      int y[MAX_VERTICES];
} polygon;
```

我们在多边形数据结构中增加了颜色索引成员。如果布尔变量used的值为true，则表示已经定义了该多边形。如果要删除一个多边形，只要把该多边形对应的变量used置为false，这也是它的初始值。注意，如果使用链表作为存储多边形的基本结构，那么通过插入和删除链表节点就可以更直截了当地实现增加和删除多边形的功能。我们还增加了一个表示多边形中心位置的成员，它是在建立多边形时通过计算得到的，这个成员有助于我们实现多边形的移动。这里使用了包围盒技术来实现多边形的拾取操作，尽管可以使用更具通用性的基于选择模式的方法。包围盒信息包含在变量xmin、xmax、ymin和ymax中，我们在建立多边形的时候设置这些变量。

为了显示存储在polygons数组中的所有多边形，在显示某个多边形之前，需要首先检测是否定义了该多边形，如果该多边形已经被定义，那么就可以利用该多边形的颜色索引值把颜色赋给该多边形，然后像前面一样绘制该多边形，代码如下：

```
 for(i=0; i<MAX_POLYGONS; i++)
{
   if(polygons[i].used)
   {
      glColor3fv(colors[polygons[i].color]);
      glBegin(GL_POLYGON);
      for(j=0; j<polygons[i].nvertices; j++)
         glVertex2i(polygons[i].x[j], polygons[i].y[j]);
      glEnd();
   }
}
```

现在可以设计菜单了，我们利用菜单就能够以交互方式执行相应的操作。我们使用主菜单选择要执行的任务，并使用子菜单选择多边形的绘制颜色，代码如下：

```
c_menu = glutCreateMenu(color_menu);
glutAddMenuEntry("Black",0);
glutAddMenuEntry("Red",1);
glutAddMenuEntry("Green",2);
glutAddMenuEntry("Blue",3);
glutAddMenuEntry("Cyan",4);
glutAddMenuEntry("Magenta",5);
glutAddMenuEntry("Yellow",6);
glutAddMenuEntry("White",7);

glutCreateMenu(main_menu);
glutAddMenuEntry("new polygon", 1);
glutAddMenuEntry("end polygon", 2);
glutAddMenuEntry("delete polygon", 3);
glutAddMenuEntry("move polygon", 4);
glutAddMenuEntry("quit",5);
glutAddSubMenu("Colors", c_menu);
glutAttachMenu(GLUT_MIDDLE_BUTTON);
```

为了实现移动或删除已定义的多边形的功能，程序必须支持某种拾取行为。因为用户可能不会以我们想象的方式来按鼠标按键。例如，一个已经处于创建状态中的多边形在完成之前，用户就开始建立另一个新的多边形，所以我们需要在程序中增加一些自定义的状态变量，按下面的方式对它们进行初始化：

```
bool picking = FALSE;
bool moving = FALSE;
int in_polygon = -1;
int present_color = 0;
```

present_color指定了默认的颜色索引值。我们可以分别通过picking和moving这两个变量得知当前的处理过程是否正在拾取或移动某个已定义的多边形。

交互的核心部分集中于main_menu回调函数，该函数使用switch-case语句来响应菜单项，代码如下：

```
void main_menu(int index)
{
   switch(index)
   {
      case(1):
      {
```

```
        /* 用来建立新多形性的代码*/
           break;
        }
        /* 其他的case语句*/
```

为了建立一个新的多边形(case 1),我们要在多边形列表中寻找第一个没有被使用的索引编号(即polygons[i].used = = FALSE)。如果在超过多边形的最大数目之前找到这样一个索引编号,那么就可以对该多边形进行初始化并设置相应的状态:

```
case(1):
{
        moving = FALSE;
        for(i=0; i<MAX_POLYGONS; i++)
           if(polygons[i].used == FALSE) break;
        if(i == MAX_POLYGONS)
        {
           printf("exceeded maximum number of polygons\n");
           exit(0);
        }
        polygons[i].color = present_color;
        polygons[i].used = TRUE;
        polygons[i].nvertices = 0;
        in_polygon = i;
        picking = FALSE;
        break;
}
```

一般来说,此时用户会通过按鼠标左键来输入新建多边形的顶点。鼠标回调函数中处理这种情况的代码看上去具有如下的形式:

```
if(btn==GLUT_LEFT_BUTTON && state==GLUT_DOWN &&!picking&&!moving)
{
    if(in_polygon>=0)
    {
         if(polygons[in_polygon].nvertices == MAX_VERTICES)
         {
             printf("exceeds maximum number vertices\n");
             exit(0);
         }
         i = polygons[in_polygon].nvertices;
         polygons[in_polygon].x[i] = x;
         polygons[in_polygon].y[i] = y;
         polygons[in_polygon].nvertices++;
     }
}
```

然后用户就可以通过菜单选择“end polygon”操作来完成多边形的定义。执行“end polygon”可以让程序为多边形计算包围盒及其中心位置,mainmenu回调函数中处理这种情况的代码如下:

```
case(2):
{
   moving = FALSE;
   if(in_polygon>=0)
   {
      polygons[in_polygon].xmax = polygons[in_polygon].xmin = polygons[in_polygon].x[0];
```

```
        polygons[in_polygon].ymax = polygons[in_polygon].ymin = polygons[in_polygon].y[0];
        polygons[in_polygon].xc = polygons[in_polygon].x[0];
        polygons[in_polygon].yc = polygons[in_polygon].y[0];
        for(i=1;i<polygons[in_polygon].nvertices;i++)
        {
           if(polygons[in_polygon].x[i]<polygons[in_polygon].xmin)
              polygons[in_polygon].xmin = polygons[in_polygon].x[i];
           else if(polygons[in_polygon].x[i]>polygons[in_polygon].xmax)
              polygons[in_polygon].xmax = polygons[in_polygon].x[i];
           if(polygons[in_polygon].y[i]<polygons[in_polygon].ymin)
              polygons[in_polygon].ymin = polygons[in_polygon].y[i];
           else if(polygons[in_polygon].y[i]>polygons[in_polygon].ymax)
              polygons[in_polygon].ymax = polygons[in_polygon].y[i];
           polygons[in_polygon].xc += polygons[in_polygon].x[i];
           polygons[in_polygon].yc += polygons[in_polygon].y[i];
        }
           polygons[in_polygon].xc = polygons[in_polygon]
                                    .xc/polygons[in_polygon].nvertices;
           polygons[in_polygon].yc = polygons[in_polygon]
                                    .yc/polygons[in_polygon].nvertices;
     }
     in_polygon = -1;
     glutPostRedisplay();
     break;
  }
```

我们考虑的其他交互行为需要用到拾取操作，通过拾取操作可以得知鼠标位于哪个多边形（如果存在这么一个多边形）上。我们使用简单的包围盒技术来编写拾取函数，该函数或者返回多边形的标识符，或者当鼠标不在任何多边形包围盒的内部时返回 -1，函数的代码如下：

```
int pick_polygon(int x, int y)
{
   int i;
   for(i=0; i<MAX_POLYGONS; i++)
   {
      if(polygons[i].used)
         if((x>=polygons[i].xmin)&&(x<=polygons[i].xmax)
                                  &&(y>=polygons[i].ymin)
            &&(y<polygons[i].ymax))
         {
            in_polygon = i;
            moving = TRUE;
            return(i);
         }
      return(-1);
   }
}
```

在mainmenu回调函数中按下面的形式设置相应的状态标志后，就可以分别在鼠标回调函数和鼠标移动回调函数中处理删除多边形和移动多边形的操作：

```
case(3):
{
     picking = TRUE;
     moving = FALSE;
     break;
}
```

```
case(4):
{
    moving = TRUE;
    break;
}
```

鼠标回调函数中删除多边形的代码调用了拾取函数，代码如下：

```
if(btn==GLUT_LEFT_BUTTON && state==GLUT_DOWN &&picking&&!moving)
{
    picking = FALSE;
    moving = FALSE;
    j = pick_polygon(x,y);
    if(j >= 0)
    {
        polygons[j].used = FALSE;
        in_polygon = -1;
        glutPostRedisplay();
    }
}
```

鼠标移动回调函数首先检测当前的状态，然后使所选择的多边形的中心随鼠标一起移动。注意，需要同时移动所选择的多边形的每个顶点并更新表示多边形中心位置的变量，代码如下：

```
void myMotion(int x, int y)
{
    float dx, dy;
    int i,j;
    if(moving)
    {
        y = wh - y;
        j = pick_polygon(x, y);
        if(j<0) return;
        dx = x - polygons[j].xc;
        dy = y - polygons[j].yc;
        for(i = 0; i< polygons[j].nvertices; i++)
        {
            polygons[j].x[i] += dx;
            polygons[j].y[i] += dy;
        }
        polygons[j].xc += dx;
        polygons[j].yc += dy;
        if(dx>0) polygons[j].xmax += dx;
          else polygons[j].xmin += dx;
        if(dy>0) polygons[j].ymax += dy;
          else polygons[j].ymin += dy;
        glutPostRedisplay();
     }
}
```

完整的程序参见附录A。

3.10　建立交互式模型

对于以交互式方式建立的对象，前面的多边形程序说明了如何保存、操作和删除这些对象。现在讨论一种能扩展程序现有功能的方法，从而使之包含显示列表和实例的思想。

假定希望使用一组预定义好的积木式部件（building block）来建立一个复杂的图像。这些积木式部件可能是一些简单的几何形状。对于室内设计等应用，积木式部件可能由椅子、桌子及其他的家用器具等组成。对于电路设计，可能包括电阻、电容和电源等对象。某个基本对象可能多次出现，每出现一次我们称之为一个实例（instance）。第4章将介绍实例的几何变换操作，但是在本节利用实例把基本对象直接放到场景中就足够了。

为了建立复杂的交互式模型，我们首先从一些简单的积木式部件（基本对象）开始：一个等边三角形、一个正方形、一条水平线段和一条垂直线段。目前暂时假定每个基本对象都以基本的尺寸定义其大小，并且以坐标原点为中心。我们还假定每个基本对象都是通过下面这种形式的OpenGL代码定义的：

```
void square(float x0, float y0))
{
  glBegin(GL_QUAD);
    glVertex2f(x0-1.0, y0-1.0);
    glVertex2f(x0+1.0, y0-1.0);
    glVertex2f(x0+1.0, y0+1.0);
    glVertex2f(x0-1.0, y0+1.0);
  glEnd();
}
```

上述代码绘制了一个以（x0，y0）为中心的正方形。就像前面介绍的多边形程序一样，可以通过菜单和拾取交互操作来实现程序的交互功能，用户只需选择所需的基本对象，然后通过单击鼠标确定该对象在屏幕上的位置，即对象的中心位置（x0，y0）。

首先，为了存储用于绘制一个简单对象所需的信息，需要建立一个更具通用性的基本结构体：

```
typedef struct object
{
   int type;
   float x, y;
   float color[3];
} object;
```

该结构体用来存储对象的类型、对象的中心位置和对象的颜色。我们把type等于0的对象类型作为保留类型，它表示某个对象不存在（即没有定义该对象）或即便定义了该对象却不能绘制该对象。这样就可以利用一个称为实例表（instance table）的数组来存储一个包含100个对象的对象集：

```
object table[100];
```

并且定义一个指向实例表最后一个对象的索引：

```
int last_object;
```

把对象添加到实例表中非常简单。假定用户要添加一个正方形，它的类型为SQUARE：

```
#define SQUARE 1;
```

一旦用户利用菜单选择对象的类型及其颜色，然后再使用鼠标确定了对象在屏幕上的位置，那么把这些信息存入实例表的代码有点类似于如下的形式：

```
table[last_object].type = SQUARE;
table[last_object].x = x0;
table[last_object].y = y0;
```

```
table[last_object].color[0] = red;
table[last_object].color[1] =green;
table[last_object].color[2] = blue;
last_object++;
```

现在，每当我们要显示所有的对象时，只要执行如下形式的代码即可：

```
for(i=0; i<last_object; i++)
{
  switch(table[i].type)
  {
     case 0: break;
     case 1:
       {
       glColor3fv(table[i].color);
       triangle(table[i].x, table[i].y);
       break;
       }
       /* 其他的对象类型 */
  }
}
```

到目前为止，一切都很简单，但是我们下面考虑一个比较复杂的功能，即为了从实例表中删除对象，如何以交互的方式编辑实例表。我们可以使用本章介绍的一种拾取方法来允许用户识别某个所要删除的对象。也许实现拾取操作最简单的方法就是利用3.9节的多边形程序使用的包围盒技术。为了实现这个功能，首先在对象的结构体类型中增加两个位置成员，它们定义了包围盒的左下角和右上角的位置：

```
/* 在结构体类型中定义包围盒 */
float bb[2][2];
```

因此，当用户指定一个正方形时，我们需要增加下面的代码：

```
bb[0][0] = x0-1,0;
bb[0][1] = y0-1,0;
bb[1][1] = x0+1,0;
bb[1][1] = y0+1,0;
```

现在假定用户通过选择菜单希望删除某个对象并且用鼠标定位该对象。为此，程序现在要从实例表中搜索出第一个包含该对象的包围盒，并将实例表中该对象的类型置为0。因此，显示过程在下次遍历实例表时，这个对象不会显示出来。

虽然这种方法在程序中定义的对象数量不超过预定义数组的大小时才是可行的，但是情况不一定总是这样。设计一个更好的数据结构有助于解决这个问题。本章后面的一个简单习题（见习题3.26）要求使用对象的链表结构来表示实例表并实现3.9节的多边形程序。每增加一个对象就是在链表中添加一个节点，而删除一个对象则需要搜索链表，然后从链表中删除相应的节点。

为了使对象的结构体类型能够包含其他选项，可以对其进行扩展。例如，我们可以在结构体中增加一个表示对象大小的参数，这样用户就可以对每个对象进行缩放变换操作了。除了颜色成员之外，为了表示对象的其他属性，还可以在结构体中增加其他成员变量。

我们还可以使用显示列表（见习题3.27）来存储多个对象，从而提高绘制的效率。例如，我们可以增加一个菜单用来创建显示列表和删除显示列表。当建立的对象显示在屏幕上时，把指向这些对象的指针添加到显示列表中。然而，由于只能删除OpenGL的显示列表而不能对它进行编辑，所

以如果创建了一个包含多个对象的显示列表，那么利用前面介绍的唯一的一个删除操作只能删除整个显示列表。

3.11　交互式动画程序

到目前为止，我们编写的程序只能显示静态的对象，即某个图元一旦显示在屏幕上，在执行清屏操作之前它的图像是不会改变的。假定我们希望生成一幅图像，在该图像中有一个或多个对象正在变化或移动，因此这幅图像一定也随着变化。例如，我们可能希望让一个动画角色走过屏幕，或者希望视点随时间而变化。下面通过一个简单的例子来深入理解处理动画的方法，该例子只包含一个旋转的对象。

3.11.1　旋转的正方形

考虑一个二维坐标点，其表示形式如下：

$$x = \cos\theta$$
$$y = \sin\theta$$

无论θ取什么值，该点总是位于一个单位圆上。位于单位圆上另外三个点是（$-\sin\theta, \cos\theta$）、（$-\cos\theta, -\sin\theta$）和（$\sin\theta, -\cos\theta$）。这4个点等距离地分布在圆周上，如图3.20所示。因此，如果把这4个点连接起来形成一个多边形，那么就得到一个以圆点为中心、边长为$\sqrt{2}$的正方形。

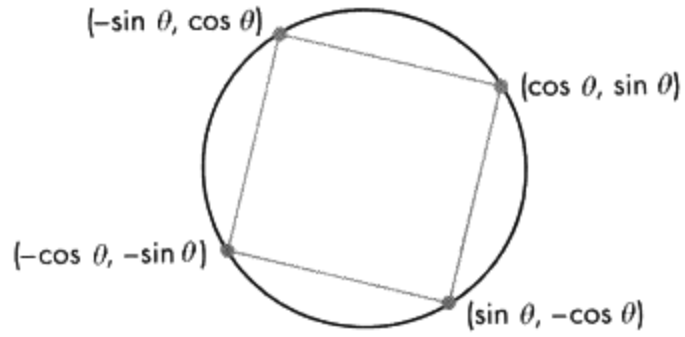


图3.20　基于圆周上的4个点构建的正方形

假定θ（即代码中的thetar）是一个全局变量，我们可以使用下面的显示函数绘制这个正方形：

```
void display()
{
  glClear(GL_COLOR_BUFFER_BIT);
  glBegin(GL_POLYGON);
    thetar = theta/(3.14159/180.0); /* 把角度转换为弧度 */
    glVertex2f(cos(thetar), sin(thetar));
    glVertex2f(-sin(thetar), cos(thetar));
    glVertex2f(-cos(thetar), -sin(thetar));
    glVertex2f(sin(thetar), -cos(thetar));
  glEnd();
}
```

该显示函数适用于任意的θ值，但是假定我们在程序运行的时候改变了θ的值，则可以使正方形绕着圆点旋转。我们只要通过调用glutPostRedisplay函数就可以重复执行该显示函数，从而显示这个旋转的正方形。

现在假定我们希望每当程序处于空闲状态时，θ就会增加一个固定的角度。我们可以使用空闲回调函数实现这个功能。因此，需要在主程序中注册这个空闲回调函数：

```
glutIdleFunc(idle);
```

此外，还需要在程序中定义空闲回调函数，代码如下：

```
void idle()
{
  theta+=2; /* 或某个其他的增量 */
  if(theta >= 360.0 theta-=360.0;
  glutPostRedisplay();
}
```

上面这个程序虽然可以正常工作，但是我们还希望对它做进一步的改进，即希望能够交互式地控制正方形是否旋转。完成这个任务的方法是利用鼠标回调函数来控制空闲回调函数。我们用下面的代码注册鼠标回调函数：

```
glutMouseFunc(mouse);
```

下面是鼠标回调函数的代码：

```
void mouse(int button, int state, int x, int y)
{
  if(button==GLUT_LEFT_BUTTON&&state==GLUT_DOWN)
      glutIdleFunc(idle);
  if(button==GLUT_MIDDLE_BUTTON&&state=GLUT_DOWN)
      glutIdleFunc(NULL);
}
```

因此，单击鼠标左键使正方形开始旋转，而单击鼠标中键则使正方形停止旋转。我们把这个完整的程序命名为single_double.c，附录A给出了该程序的源代码。如果运行该程序，读者可能发现显示的结果看起来并不是一个旋转的正方形，而是随着时间变化看到正方形的不同部位。究竟出现了什么问题呢？下一节将分析这个问题。

3.11.2　双缓存

当重新显示帧缓存中的内容时，需要采用足够快的刷新速度才能不会察觉到清屏和图像的重绘过程。大部分显示器自身的刷新频率必须设置在每秒60～85次，否则会察觉到屏幕的闪烁。只要不改变帧缓存中的内容，并且以60～85 Hz的频率刷新屏幕显示的内容，就不会察觉到这个刷新过程[①]。然而，如果在刷新的过程中改变了帧缓存中的内容，那么在图像显示的过程中就会看到不希望生成的非自然信号。

如果正要显示的图像很复杂并且无法在单个刷新周期内完成绘制，就会出现上述问题。在这种情况下，我们在连续的刷新过程中看到的只是对象的不同部位。如果某个对象处于运动状态，那么它在屏幕上显示的图像会变形。出现这个问题的另一个例子就是对屏幕的某个区域重复地执行清屏和重绘操作，如前面讨论的正方形旋转程序。尽管正方形旋转程序中的正方形非常简单并且可以在一个刷新周期内绘制出来，但是在这个程序中，在新的正方形绘制到帧缓存中与使用硬件重新显示帧缓存中的内容之间并不存在同步衔接。因此，能否正确地绘制该正方形取决于绘制帧缓存的时机是否恰当，如果绘制的时机不合适，则有可能在绘制该正方形时，帧缓存中只有它的一部分内容。

---

① 如果使用隔行扫描显示器，即帧缓存中的奇数行和偶数行以交替的方式显示，那么实际的刷新频率只是设置好的刷新频率的一半。在这样的显示器上，在显示器刷新周期交替时，可能会察觉到绘制的对象在上下方向会有轻微的移动现象。

双缓存（double buffering）机制为这些问题提供了一种解决方法。假定有两个颜色缓存可供我们使用，习惯上称之为前端缓存和后端缓存。前端缓存（front buffer）是指用于显示图像的那个缓存，而后端缓存（back buffer）是指用于存储用户绘制对象的内容的那个缓存。我们可以在应用程序中随意交换前端缓存和后端缓存。只有在程序中明确地调用了交换缓存的函数才会交换前端缓存和后端缓存。为此，可以使用GLUT库中的缓存交换函数：

```
glutSwapBuffers();
```

为了设置双缓存，应该在glutInitDisplayMode函数中使用选项GLUT_DOUBLE而不是GLUT_SINGLE。因此，如果在显示回调函数中绘制对象，那么其绘制结果用来更新后端缓存的内容。当整个对象绘制完成之后，再执行glutSwapBuffers函数显示绘制结果，因此即便使用了多个刷新周期把对象绘制到帧缓存的后端缓存中，也能确保得到平滑的显示效果。在显示回调函数中，首先要调用glClear函数来清空后端缓存，然后绘制对象，最后调用glutSwapBuffers函数交换缓存。

双缓存是实现计算机动画的一个标准技术，在计算机动画中，图元、属性和观察条件可能不断地发生变化。对于旋转正方形这个例子，为了从单缓存模式变换到双缓存模式，我们必须对原来的代码做两个小的改动。首先，需要在main主函数中通过glutInitDisplayMode函数选择双缓存显示模式：

```
glutInitDisplayMode(GLUT_RGB | GLUT_DOUBLE);
```

在显示回调函数中，当完成了正方形的绘制之后增加下面一行代码：

```
glutSwapBuffers();
```

双缓存技术并不能解决在绘制动画时遇到的所有问题。如果要显示的图像非常复杂，那么仍然可能需要把该图像分成好几帧画面依次绘制到帧缓存的后端缓存中。双缓存技术并不能使这个过程的处理速度更快，它只能确保对象显示的完整性。然而，如果使用双缓存机制，那么对于这种情况通常可以得到视觉上可接受的显示效果，即便显示帧率只有10～20帧/秒。

在交互式图形程序中还存在其他一些仅使用双缓存无法解决的问题。在某些交互式程序中，可以在显示回调函数之外定义图元，因此可能需要把对象绘制到不止一个缓存中。

前端缓存和后端缓存属于颜色缓存，用于存储可以显示在屏幕上的颜色信息。一般来说，当使用双缓存时，我们希望OpenGL把对象绘制到后端缓存，但是可以通过函数glDrawBuffer()来控制把对象绘制到哪个颜色缓存。默认情况下，当启用了双缓存时，这相当于：

```
glDrawBuffer(GL_BACK);
```

如果使用

```
glDrawBuffer(GL_FRONT_AND_BACK);
```

那么将把图元既绘制到前端缓存又绘制到后端缓存。第8章将介绍其他的缓存，把对象绘制到这些缓存而不是后端缓存可以生成一些新的视觉效果。

3.11.3　使用定时器

在结束讨论如何绘制平滑显示效果这个话题之前，我们希望考虑一个与之相反的问题，即需要绘制的图元数量远比图形绘制系统在一个刷新周期内能绘制的图元数量少时需要考虑的问题。现代GPU每秒能够绘制数以千万的图元，这个数量远高于许多应用程序需要绘制的图元数量。考虑这样一种情形，如果在一个运行速度非常快的GPU上执行一个像旋转立方体这样的简单程序，那么会

发生什么事情呢？不管是否使用双缓存，系统都会以每秒几千次的速度绘制该立方体，因此在屏幕上看到的图像会变得模糊不清。这样的显示效果不但不是我们所需要的，我们反而需要控制立方体旋转的速度。

至少有三种方法可以解决这个问题。我们可以使用库函数或操作系统提供的各种定时机制来延迟应用程序的运行，还可以利用许多商用显卡自带的方法，该方法允许用户设置某个选项，从而把前端缓存与后端缓存的交换频率锁定为刷新频率。

第三种方法是使用GLUT库提供的定时器函数：

```
glutTimerFunc(int delay, void (*timer_func)(int), int value)
```

执行该函数会在事件循环中启动一个定时器，并将该事件循环延迟delay毫秒（ms）。当定时器指定的时间到达后，就会执行time_func指向的回调函数。我们可以使用参数value把数据传送给定时器回调函数。为了利用定时器回调函数，常规的做法是通过调用glutTimer函数初始化一个新的定时器。因为GLUT库只允许使用一个定时器，并利用事件循环机制来允许程序以某个固定的速率执行显示回调函数。因此，假定我们编写了一个简单的程序，如旋转立方体，并且可以忽略图元的绘制时间，如果希望该程序的绘制速度达到n帧/秒，那么可以在main主函数中注册定时器回到函数并设置相应的参数：

```
int n = 60; /* 希望的绘制帧率 */
glutTimerFunc(100, myTimer, n);
```

然后在程序中定义回调函数，代码如下：

```
void myTimer(int v)
{
  glutPostRedisplay);
  glutTimerFunc(1000/n, myTimer, v);
}
```

3.12　交互式程序的设计

虽然给一个性能良好的交互式程序应满足的特性下定义非常困难，但是认定和鉴赏一个性能良好的交互式程序却比较简单。一个性能良好的交互式程序包含如下一些特性：

1. 平滑的显示效果，刷新过程既不会产生闪烁现象也不会产生非自然信号（失真）。
2. 能够使用各种交互设备控制屏幕上图像的显示。
3. 能够使用各种方法输入信息和显示信息。
4. 界面友好，易于使用和学习。
5. 对用户的操作具有反馈能力。
6. 对用户的误操作具有容忍性。
7. 其设计既考虑了人的视觉特性也考虑了人的运动神经特性。

我们不应该低估这些特性的重要性以及设计一个好的交互式程序的难度。人机交互（Human Computer Interaction，HCI）是一个非常活跃的研究领域，说实话，我们无法将该领域丰富的内容精简到几页的篇幅。本书讨论的是计算机图形学，而在这个领域中，我们主要关心的是图形绘制。然而，有些内容是计算机图形学和HCI所共有的，为了改善交互式程序的性能，下面介绍这方面的内容。

3.12.1　工具包、构件和帧缓存

在前面介绍的CAD程序中使用了一些交互式工具，如由GLUT函数实现的弹出式菜单以及在程序中自己构建的图形按钮。除此之外，还有许多交互式工具，如滑动条、刻度盘、屏幕的热区、声音和图标。尽管没有理由反对亲自编写程序来实现这些交互式工具，但是通常情况下，这些交互式工具是由各种工具集提供的。一般来说，这些工具集使用回调函数来实现与应用程序的交互，它们是用户开发程序的一个简单扩充。

然而，把几何对象绘制到颜色缓存这种简单的绘制模式不足以支持许多这样的交互操作。有两个例子说明了几何对象绘制的局限性，并说明了为什么有时候需要直接工作在帧缓存里。首先，考虑弹出式菜单的例子。当调用菜单的回调函数时，菜单就会出现屏幕上，而不管它覆盖的屏幕内容是什么。当选择好了菜单后，菜单就会消失并把屏幕恢复到菜单显示之前的状态。

第二个例子是橡皮条操作（rubberbanding），这是一种用于显示线段（和其他图元）的技术，它们在显示的过程中可以交互地控制其变化过程。在画图程序中，我们通过连续单击鼠标的位置确定希望绘制的线段的两个端点[①]。我们没有一种机制用来保存线段两个端点位置的某个特定关系，并且不能以交互式的方式控制屏幕上的线段来帮助我们放置这两个端点。假定用鼠标确定线段第一个端点的位置之后，当我们移动鼠标时，从第一个位置到鼠标当前位置的线段会自动绘制出来（并且在每次刷新时都会重新绘制当前的线段），如图3.21所示。我们把这个过程称为橡皮条操作，这是因为在屏幕上看到的线段看上去就像一根具有弹性的橡皮条，它的一端固定在鼠标确定的第一个位置，另一端可以拉伸到光标所在的任意位置。注意，在每条新的线段显示出来之前，必须擦除之前的那条线段。通常情况下，当按下鼠标按键时开始执行橡皮条操作，直到释放鼠标按键才结束该操作，释放鼠标按键时绘制出最终的线段。

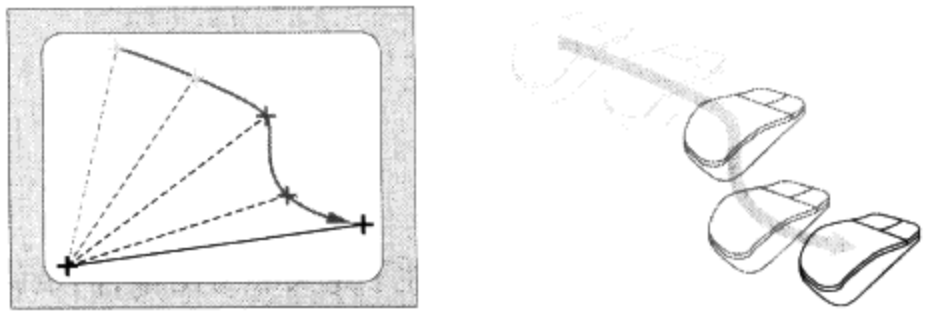


图3.21　橡皮条技术

利用目前已介绍的方法还无法实现这个操作序列。实现这些操作的一种方法是：把屏幕上被菜单（或线段）覆盖的那部分图像先保存起来，然后在不需要菜单（或线段）时再把它们复制到原来的位置。遗憾的是，这种处理方法要用到屏幕上显示的像素阵列，也就是帧缓存中的内容。有一种可能的解决方法不会涉及应用程序中定义的图元，而只涉及它们被扫描转换后的图像。因此，用于实现上述有关菜单（或线段）绘制的一系列操作要用到帧缓存中的内容，也就是需要使用位块移动操作（bit-block-transfer, bitblt）对位块进行处理。第8章介绍位和像素等内容之后就可以采取这种方法实现上述的操作了。还有另外一种得到大部分OpenGL图形绘制系统支持的方法，这种方法能以简单的方式实现上述许多功能。

3.13　逻辑运算

当程序处理一个可见的图元时，OpenGL把它绘制成一组带有颜色的像素并写入当前绘制缓存。在默认的运算模式下，这些像素只是简单地替换帧缓存中对应位置的已有像素。因此，如果开始时

① 这里的画图程序是本书第四版中的一个程序，而本书第五版将画图程序改成了3.9节的CAD程序。——译者注

把颜色缓存清空并初始化为黑色，然后绘制一个像素大小为 10 × 10 的蓝色矩形，那么图形绘制系统会把这 100 个蓝色像素写入颜色缓存，并替换已有的 100 个黑色像素。我们把这种模式称为复制模式（copy mode）或替换模式（replacement mode），在这种模式下，矩形覆盖的初始像素的颜色在被替换成蓝色之前究竟是什么颜色是无关紧要的，这是因为这种模式使用蓝色像素替换对应位置任意颜色的像素。虽然帧缓存的这种写入模式看起来很简单，但是这不是唯一的写入方式。

考虑如图 3.22 所示的模型，在这个模型中只考虑把单个像素写入颜色缓存。我们把希望写入的像素称为源像素（source pixel），而把绘制缓存中受源像素影响的像素称为目标像素（destination pixel）。在复制模式中，源像素直接替换目标像素。但是设想一下，假定图形绘制系统能够首先读取目标像素，然后使用它的值与源像素的值进行组合运算，从而确定最终写入颜色缓存的值。

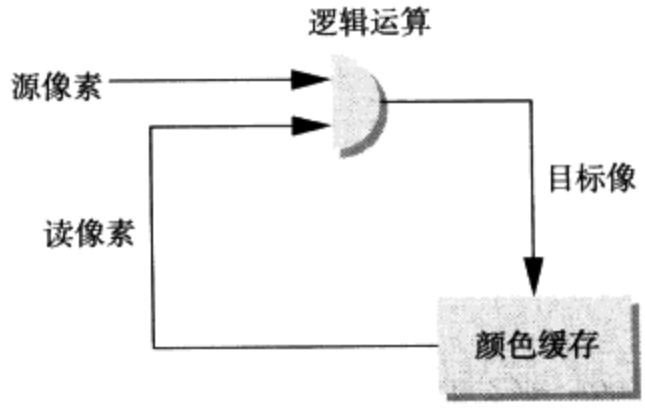


图 3.22　像素写入模型

虽然可以使用无限多种方式实现源像素与目标像素的组合函数，但是大部分硬件只支持有限的组合方式。本章只考虑像素之间的位运算，第 8 章介绍图像的融合时再讨论其他组合运算。两个位之间一共有 16 种可能的组合函数，每种组合函数定义了一种写入模式（writing mode）。第 8 章将详细讨论这些写入模式，本章只关注其中的两个写入模式。

第一个模式是替换模式，在这种模式中，源像素中的每个二进制位都要替换目标像素相应的位。第二个模式是异或模式（exclusive OR mode）或 XOR 模式，在这种模式中，每个像素对应的位之间进行异或逻辑运算。如果 s 和 d 分别是源像素和目标像素对应的位，那么可以把新的目标位表示为 d'：

$$d' = d \oplus s$$

其中，$\oplus$ 表示异或运算。异或运算最有趣的特性就是：如果两次应用异或运算，那么其结果又回到初始状态。也就是

$$d = (d \oplus s) \oplus s$$

因此，如果使用异或模式绘制了某个对象，那么只要再次绘制该对象即可删除它。

OpenGL 支持所有的 16 种逻辑运算模式。复制模式（GL_COPY）是默认的模式。如果要改变逻辑运算模式，那么首先必须开启逻辑运算功能：

```
glEnable(GL_COLOR_LOGIC_OP);
```

然后用下面的代码变换到所需的异或运算模式：

```
glLogicOp(GL_XOR);
```

3.13.1　绘制可擦除的直线

如果使用XOR绘制模式，那么可以利用许多不同的方法绘制可擦除的直线。下面介绍一种简单的方法。假定有一个大小为500像素 × 500像素的OpenGL绘图窗口，并且裁剪窗口是一个单位正方形，坐标原点位于正方形的左下角。我们使用鼠标确定线段的第一个端点，并把这个端点在对象坐标系中的位置存储起来：

```
xm = x/500.;
ym = (500-y)/500.;
```

然后就可以确定第二个端点并使用XOR模式绘制该线段：

```
xmm = x/500.;
ymm = (500-y)/500.;
glLogicOp(GL_XOR);
glBegin(GL_LINES);
    glVertex2f(xm, ym);
    glVertex2f(xmm, ymm);
glLogicOp(GL_COPY);
glEnd();
glFlush();
```

如果希望在常规绘制模式下绘制对象，那么需要切换到原来的复制模式。如果使用鼠标再输入另一个端点，那么先使用XOR模式绘制与上面相同的一条线段，然后再使用线段的第一个端点与鼠标输入的位置绘制第二条线段，代码如下：

```
glLogicOp(GL_XOR);
glBegin(GL_LINES);
    glVertex2f(xm, ym);
    glVertex2f(xmm, ymm);
glEnd();
glFlush();
xmm = x/500.0;
ymm = (500-y)/500.0;
glBegin(GL_LINES);
    glVertex2f(xm, ym);
    glVertex2f(xmm, ymm);
glEnd();
glLogicOp(GL_COPY);
glFlush();
```

为了实现这个操作序列，可以利用鼠标右键输入线段的端点位置，利用鼠标左键确认接受当前的线段并把最终的这条线段绘制出来，代码如下：

```
glLogicOp(GL_COPY);
glBegin(GL_LINES);
    glVertex2f(xm, ym);
    glVertex2f(xmm, ymm);
glEnd();
glFlush();
glLogicOp(GL_XOR);
```

就像画图程序一样[①]，我们也在程序中增加一个计数器来跟踪顶点，并存储后面要再次用到的顶点。

① 这里的画图程序是本书第四版中的一个程序，而本书第五版将画图程序改成了3.9节的CAD程序。——译者注

这个例子实现的还不完全是橡皮条线段，橡皮条线段在绘制时会随鼠标移动而连续地绘制。在鼠标移动回调函数中使用上述相同的操作可以绘制橡皮条线段。因此，当用户第一次按下鼠标的某个按键时，通过鼠标回调函数存储线段的第一个端点位置。在鼠标按键被按下的状态下移动鼠标时，绘制一条从第一个端点到由鼠标移动回调函数确定的第二个端点的线段，在绘制这条线段之前，需要重新绘制它的前一条线段，从而把它擦除掉。当释放鼠标按键时，在鼠标回调函数中以复制模式绘制最终的线段。可以采用类似的方法绘制矩形。当第一次按下鼠标左键时，我们通过鼠标回调函数得到矩形的第一个角点。然后，只要一直按住鼠标左键并移动鼠标，就可以利用鼠标移动回调函数获取矩形第二个角点的连续值。在第一次使用XOR模式绘制单个矩形之后，每当得到一个新的角点，必须首先通过重绘操作擦除当前的矩形，然后再利用第一个角点和新的角点来绘制一个新的矩形。最后，当释放鼠标左键时，再次执行鼠标回调函数，然后对矩形执行最后一次擦除和绘制操作，并切换到替换（或复制）模式。

注意，必须把计数器和矩形的角点定义为全局变量：

```
float xm, ym, xmm, ymm; /* 矩形的两个角点坐标 */
int first = 0; /* 顶点计数器 */
```

用下面的代码注册回调函数：

```
glutMouseFunc(mouse);
glutMotionFunc(move);
```

下面分别是鼠标回调函数和鼠标移动回调函数的代码：

```
void mouse(int btn, int state, int x, int y)
{
    if(btn==GLUT_LEFT_BUTTON && state == GLUT_DOWN)
    {
        xm = x/500.;
        ym = (500-y)/500.;
        glColor3f(0.0, 0.0, 1.0);
        glLogicOp(GL_XOR);
        first = 0;
    }
    if(btn==GLUT_LEFT_BUTTON && state == GLUT_UP)
    {
        glRectf(xm, ym, xmm, ymm);
        glFlush();
        glColor3f(0.0, 1.0, 0.0);
        glLogicOp(GL_COPY);
        xmm = x/500.0;
        ymm = (500-y)/500.0;
        glLogicOp(GL_COPY);
        glRectf(xm, ym, xmm, ymm);
        glFlush();
    }
}

void move(int x, int y)
{
    if( first == 1)
    {
        glRectf(xm, ym, xmm, ymm);
```

```
            glFlush();
        }
        xmm = x/500.0;
        ymm = (500-y)/500.0;
        glRectf(xm, ym, xmm, ymm);
        glFlush();
        first = 1;
    }
```

3.13.2　异或运算和颜色

如果运行前面的示例程序，读者可能会注意到一些奇怪的颜色组合，在已着色的像素上绘制临时线段时会出现这种情形。在这个示例程序中，我们在绘制橡皮条线段时使用蓝色作为临时颜色，而使用绿色作为最终的颜色。假定图形绘制系统使用一个字节存储颜色分量，因此蓝色分量可以表示为24位的RGB值（00000000，00000000，11111111）。另外还假定通过清屏操作把屏幕颜色设置为白色，或（11111111，11111111，11111111）。因此，当使用XOR模式绘制蓝色线段时，我们发现该线段的颜色可以表示为（11111111，11111111，00000000），其颜色是黄色而不是蓝色，这是因为蓝色（线段）和白色（背景）这两种颜色对应的二进制位之间执行了异或运算。如果该线段与一个红色对象相交，那么对象内部相交部分的颜色为品红色。之所以会产生这种出人意料的视觉效果，是因为使用了XOR写入模式，这就是为什么在绘制最终的线段时要使用复制模式的原因。

我们还经常使用XOR模式在屏幕上绘制光标。光标可以是小矩形、小十字形或其他形状。例如，我们经常在CAD程序中使用十字光标，它是互相垂直且延长到显示器边缘刻度尺的两条线段。如果使用XOR模式绘制这些对象，那么可以在屏幕上四处移动它们而不会使屏幕上任何其他对象变形。然而，如果在屏幕上显示了一些已绘制的对象，那么当光标越过它们时，这些位于光标下方的对象的颜色会改变。

3.13.3　光标和叠加位平面

由于绘制橡皮条线段和光标需要不断地更新显示的图像，这会使图形绘制系统承受极大的负荷。尽管使用XOR绘制模式似乎能够简化处理过程，但是XOR运算要求系统在计算新的目标像素颜色值之前读取当前目标像素的颜色值。相比于简单地把源像素复制到帧缓存中，XOR运算的执行速度比较慢。

另一种可取的方法是为这类交互操作提供硬件支持，采用的方法是增加所谓的叠加位平面（overlay planes），即在颜色缓存中增加一些额外的位平面。因此，如图3.23所示，在典型的颜色缓存中，每种颜色分量（红、绿、蓝）分别包含8个位平面和1个叠加位平面。

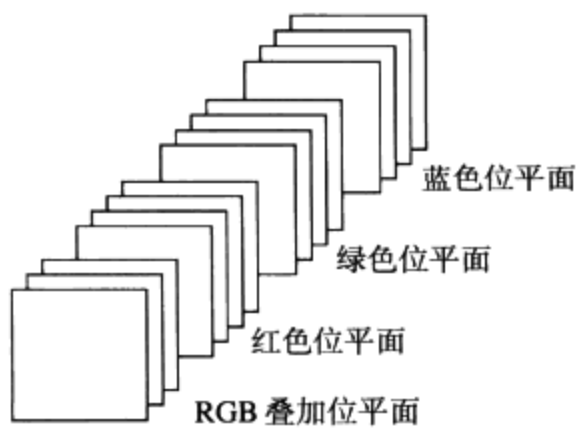


图3.23　具有叠加位平面的颜色缓存

叠加位平面中的内容与颜色缓存中的内容无关。此外，当显示颜色缓存和叠加位平面中的内容时，显示结果表面看上去是把叠加缓存中的内容复制到了颜色缓存中引起的。换句话说，如果红色叠加位平面中的某位设置为1，而绿色和蓝色叠加位平面中与之对应的位设置为0，那么屏幕窗口对应位置的像素点显示为红色，而不管RGB颜色缓存中与之对应的24位颜色值是什么。

叠加位平面属于硬件特性，并没有得到所有图形绘制系统的支持。如果硬件支持叠加位平面，那么GLUT库就会提供对它的支持。

小结和注释

本章初步探讨了许多与交互式计算机图形学有关的话题。这些交互式特性使计算机图形学这个领域变得有趣且激动人心。尽管我们使用的OpenGL图形API独立于任何操作系统或窗口系统，但是我们意识到任何程序都必须与计算机系统的其他部分进行最低限度的交互操作。我们利用GLUT库处理简单的交互功能，GLUT库是一个简单的实用工具集，它包含的API为我们提供了一些必要的且独立于特定操作系统或窗口系统的附加功能。

我们深受客户/服务器观点的影响。它不但允许我们在网络环境中开发应用程序，而且使我们设计的程序不但具有可移植性，还能充分利用图形硬件的某些特性。这些概念对于在第10章讨论面向对象的图形绘制以及图形在因特网上传送的问题非常重要。

从应用程序编程人员的观点来看，大部分图形绘制系统都具有相同的交互特性。我们把图形绘制系统的图形部分看做是服务器，它由光栅显示器、键盘和定位设备组成。几乎在所有的计算机系统中，我们必须工作在多任务的窗口环境中。许多处理过程很可能与图形程序并行执行。然而，窗口系统允许我们为某个特定的窗口编写程序，这个特定的窗口看上去就像是一个单用户系统的显示设备。

在这种多任务的窗口环境中创建一个程序的开销并不大。每个应用程序都包含一组几乎相同的函数调用。程序员可以在应用程序中使用逻辑设备，这使得他们不用考虑特定硬件的细节信息。

在上面描述的环境中，基于事件模式的输入是一种标准的模式。尽管还存在其他模式，如请求模式是用于键盘输入的标准方法，但是基于事件模式的输入为设计交互式程序提供了更大的灵活性。

最新一代图形处理器的高速性能不仅使我们能够执行甚至几年前还无法想象的交互式应用程序，而且还激发我们重新考虑这样一个反复思考的问题，即现在一直使用的技术在目前是否还是最好的。例如，尽管一些硬件新特性（如逻辑操作和叠加位平面）的出现使得许多交互式技术得以实现，但是现在通常可以使用快速的GPU以足够快的速度绘制整个画面，因此并非必须使用这些硬件新特性。

交互式计算机图形学在无数的应用领域中是一个强有力的工具。此时，读者应该有能力编写非常复杂的交互式程序。也许对读者最有帮助的练习就是立即编写一个交互式程序。本章后面的习题为读者提供了一些启发。

建议阅读资料

交互式界面中的窗口－图标－菜单－指向设备（window-icons-menus-pointing，WIMP）现在看来已是司空见惯的东西，而与之有关的许多基本概念（参阅[Sch97]）是由施乐帕洛阿尔托研究中心（Xerox Palo Alto Research Center，PARC）在20世纪70年代提出的。鼠标也是由该研究中心首次提出的[Eng68]。现在为人们所熟知的界面，如Macintosh操作系统、X Window操作系统和微软的Windows操作系统的界面设计都是以该研究中心的研究成果为基础的。

Foley和他的同事在他们的合著[Fol94]中全面地介绍了用户界面的开发技术，并重点介绍了图形绘制方面的内容。Schneiderman[Sch97]和Nielson[Nie94]在他们的著作中介绍了HCI（人机交互）方面的技术。

X Window操作系统[Sch88]最早是由麻省理工学院开发的，目前已是UNIX工作站用户群事实上的标准。最近，用于PC的Linux版的开发使得X Window操作系统也可以在PC上运行了。

本章讨论的输入和交互模式来自于GKS[ANSI85]和PHIGS[ANSI88]图形标准。这些图形标准既可用于画线显示器（也称为随机扫描显示器或矢量显示器），也可用于光栅扫描显示器，因此，它们并没有充分利用光栅显示系统独有的特性。

尽管本章只利用了GLUT实用工具集[Kil94b]实现交互操作，但是也可以利用X Window工具集[Kil94a，OSF89]来直接实现与X Window操作系统的交互操作，也可以利用相关的工具集直接实现与微软Windows操作系统的交互操作。使用脚本语言（如tcl/tk[Ost94]）开发基于OpenGL的用户界面也是我们感兴趣的话题。*OpenGL Primer*[Ang08]一书介绍了更多有关GLUT的细节内容。还可以获得其他一些用于OpenGL的交互式界面开发工具集，读者可以访问OpenGL官方网站www.opengl.org，其中有些工具集，如OpenGL实用工具集（GLUI）等，建立在GLUT之上。

文献[Sut63]介绍了Sutherland的画板程序，这是交互式计算机图形学的由来，也是本章结束时建议读者阅读的材料。

习题

3.1　试解释在定义笔划字体时所遇到的问题。建立一个简单的数据结构，并基于该数据结构只使用线段定义一组笔划字符。

3.2　重新编写第2章介绍的Sierpinski镂垫程序，要求该程序具有交互功能，即单击鼠标左键开始在屏幕上绘制点；单击鼠标右键停止绘制新的点；单击鼠标中键结束程序。要求程序包含reshape回调函数。

3.3　构建一个滑动条来允许用户定义3.9节介绍的CAD程序中的颜色。所设计的界面可以实现这样的功能，即用户使用某种颜色之前应该能够查看这种颜色。

3.4　可以利用鼠标构建一个虚拟的跟踪球，采用的方法是把鼠标垫映射到某个球面上。这种设备的一个主要用途就是允许用户通过鼠标“旋转”该球体来测量速度。构建一个这样的虚拟设备并编写一个图形应用程序来说明它的用法。

3.5　修改3.6节的正方形绘制程序，要求在程序中加入菜单操作，其功能与3.7节介绍的菜单相似。

3.6　利用自己设计的时钟在3.9节介绍的CAD程序中增加一个程序执行时间指示器。

3.7　要熟悉交互式图形编程，一个很好的方法就是编写简单的游戏。试编写一个西洋跳棋游戏，可以把棋盘上的每个格子看做是用户可拾取的对象。游戏开始时，用户可以同时作为游戏的双方来下棋。

3.8　编写一个简单的单人纸牌游戏。首先，只利用基本的图元设计一副简单的纸牌。程序的主要功能是实现对矩形对象的拾取操作。

3.9　模拟一个击球游戏会引发许多有趣的问题。类似于习题2.18，必须计算球的运动轨道并检测对象之间的碰撞。与交互有关的操作主要包括利用虚拟球杆初始化球的运动状态，确保平滑的显示效果以及创建双人游戏。

3.10　在交互式程序中，可以不使用按钮或菜单来选择选项，而是根据鼠标在窗口中的位置来选择相应的选项。试把这种机制应用到3.9节的CAD程序中。

3.11 我们能够准确地定义这样一个映射关系，即把位于对象坐标系或世界坐标系中的某个点映射到屏幕坐标系中的某个位置，但却不能按相反的方向定义一个逆向映射关系，这是因为正向映射关系反映的是从三维到二维的变换。然而，假定我们编写的是一个二维应用程序，那么这个二维映射关系可逆吗？如果利用二维映射关系把定位设备确定的屏幕位置映射到与之对应的位于对象坐标系或世界坐标系中的位置，会出现什么问题？

3.12 如何把习题3.11的结论应用到拾取操作？

3.13 在一个典型的应用程序中，必须由程序员来决定是否使用显示列表。至少考虑两个应用程序。对于每个应用程序，如果赞同使用显示列表，那么至少列举两点理由；如果反对使用显示列表，也至少列举两点理由。利用GLUT提供字体，测试所列举的理由在实际中是否正确。

3.14 编写一个交互式程序，用来引导一个虚拟的老鼠穿越习题2.7生成的迷宫。用户可以通过鼠标左键和右键来控制老鼠转动的方向，并通过鼠标中键引导老鼠向前运动。

3.15 把绘制橡皮条线段和橡皮条矩形的功能加入CAD程序中。

3.16 一些用于玩具和游戏的廉价游戏杆一般没有配置编码器，它们只包含一对三位置转换开关。这样的设备是如何工作的？

3.17 如图3.24所示的坐标系用来描述飞机的方位。游戏杆的前后运动控制飞机沿机身前后方向轴上下转动，称为俯仰（pitch）。游戏杆的左右运动控制绕该方向轴旋转，称为滚转（roll）。编写一个程序，要求利用鼠标控制飞机的俯仰和滚转，从而显示飞行员操纵飞机时看到的场景。读者可以在一个二维空间中做这个习题，具体要求是假定场景中有一组远离飞机的对象，用鼠标来模拟飞机并通过控制鼠标来显示这些对象的二维观察图像。

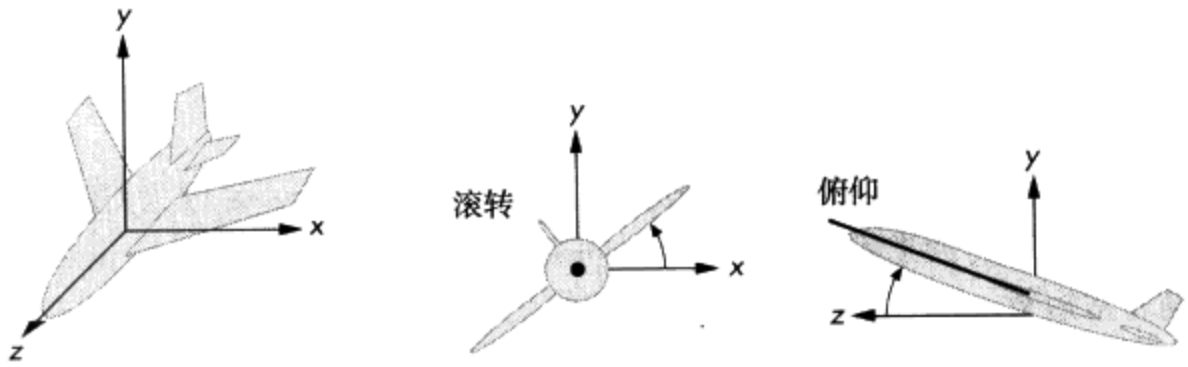


图3.24　飞机的坐标系

3.18 考虑如图3.25所示的一张桌子，桌面上有两个相互连接的手臂，并在手臂的末端放置一个传感设备。假定两个手臂的长度固定不变，并通过一个简单的（一个自由度）转轴连接起来。试确定关节角度 θ、ϕ 和传感设备位置之间的关系。

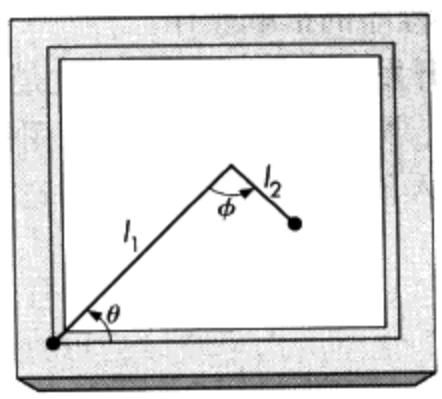


图3.25　二维传感手臂

3.19 假定某个CRT显示器具有大小为40 cm × 40 cm的正方形屏幕，并以60 Hz的刷新频率进行逐行刷新。绘制每条扫描线所用时间的10%用于把CRT的电子束从屏幕右边缘返回到屏幕左边缘（即水平回扫时间），并在每帧画面刷新完成时，绘制一帧画面所用时间的10%用于把电子束从屏幕右下角放回到屏幕左上角（即垂直回扫时间）。假定显示器的分辨率为1024 × 1024，试求光笔检测到电子束的时间与光笔位置之间的关系。试分别使用厘米和屏幕坐标表示光笔在屏幕上的位置。

3.20 电路布图设计程序是另一种形式的画图程序。考虑使用布尔操作AND（与）、OR（或）和NOT（否）来设计逻辑电路。其中的每个布尔操作分别对应于如图3.26所示的集成电路（门电路）。编写一个逻辑电路设计程序，用户可以通过菜单选择门电路并确定它在屏幕上的位置。试考虑将一个门电路输出连接到其他门电路输入的方法。

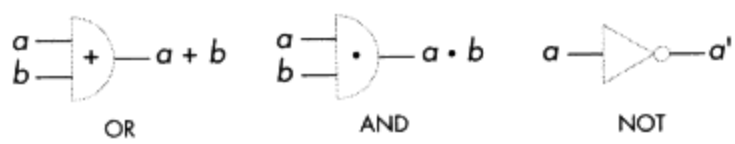


图3.26　逻辑电路符号

3.21 对习题3.20的程序进行扩展，扩展后的程序允许用户指定输入信号序列。要求程序能够显示在电路特定位置处的输出值。

3.22 对习题3.20的程序进行扩展，扩展后的程序允许用户输入逻辑表达式。要求程序能够根据输入的表达式生成逻辑电路图。

3.23 采用习题3.20中的方法绘制程序流程图的符号，或者绘制数据结构课程里学过的图形符号。

3.24 绘图软件包提供了各种显示数据的方法。试编写一个用于绘制曲线的交互式绘图程序。要求应用程序允许用户选择图形的绘制模式（数据的多线图显示模式、柱状图显示模式或饼图显示模式）、颜色和线型。

3.25 之所以要求把CRT显示器的刷新频率设置为50 ~ 85 Hz，是因为CRT显示器的刷新频率是由它使用的荧光粉的持续发光时间决定的，荧光粉被电子束轰击后持续发光的时间间隔非常短暂。目前也有持续发光时间较长的荧光粉，为什么大部分工作站显示器并没有利用这种持续发光时间较长的荧光粉呢？这种持续发光时间较长的荧光粉适用于何种应用程序？

3.26 修改3.9节的多边形程序，要求使用链表而不是数组来存储对象。修改后的程序允许用户以交互方式增加和删除对象。

3.27 重新编写多边形程序，要求使用显示列表存储对象。

3.28 使用选择模式实现多边形程序的拾取操作。

3.29 另一个可以使用OpenGL开发的CAD应用程序就是画图程序。用户可以显示各种可以绘制的对象，如直线、矩形、圆和三角形等，并利用拾取操作来选择希望绘制的对象。然后，用户可以利用鼠标输入顶点数据并从菜单中选择颜色等属性。试编写具有上述功能的画图程序。

第4章　几何对象和变换

现在可以把注意力放到三维图形上了。这一章的很多内容涉及下列问题：如何表示基本的几何类型？如何在各种表示之间进行转换？几何对象有哪些性质与具体的表示无关？

讨论从计算机图形学的数学基础开始。这样就不会把几何实体本身、几何实体在一种特别参考系下的表示和几何实体的数学抽象这三者混为一谈。

我们用仿射空间和Euclid空间的概念来为后面的叙述建立必要的数学基础。一个目的是建立一种与坐标系无关的方法来处理几何问题。当我们关心如何表示几何对象时，这种方法的优点就会体现出来了。与在某个特定的坐标系或者框架下表示对象相比，这种与坐标无关的方法更具鲁棒性。这种方法还引出了齐次坐标。借助齐次坐标，不仅可以对这种方法做出解释，还可以导出有效的实现技术。

我们使用抽象数据类型的术语来强化对象和它的表示之间的区别。从本章的讨论过程可以看到，如果想对一些基本的几何对象进行操作，那么自然要用到一些数学知识。这里要讨论的大部分内容可看做是向量空间、几何和线性代数的应用。附录B和附录C分别对向量空间和矩阵代数进行了简要总结。

与第2章和第3章类似，我们编写一个简单的应用程序来演示基本原理并从中观察API如何在内部实现我们所讲到的概念。这一章的例子主要针对如何表示一个立方体并对其进行变换。我们也考虑如何交互式地确定变换并把变换平滑地应用到几何对象上。

4.1　标量、点和向量

在计算机图形学中，我们与线段、多边形和多面体这样的几何对象打交道。这些对象存在于三维世界中，它们的性质可以用长度和角度这样的概念来描述。和二维的情况类似，我们可以用为数不多的几种简单实体来定义大多数几何对象。这些基本的几何对象以及对象之间的关系可以用三种基本类型来描述，它们是标量、点和向量。

虽然我们将从几何的角度来考虑这三种类型，但也可以像附录B中那样用一组公理来形式地定义它们。虽然最终使用的是每种类型的几何实例，但需要小心区分每种实体的抽象定义及其特殊例子或者具体实现，这样可以避免许多易犯的错误。我们将在三维空间中讨论问题，但所有的结果在n维空间里也都成立。

4.1.1　几何对象

最基本的几何对象是点。在三维几何空间中，点（point）是空间中的一个位置。点所具有的唯一一个性质是该点的位置。数学上的点既没有大小，也没有形状。

我们可以用点来定义几何对象，但仅有点还不够，我们需要用实数来确定数量，比如两点之间的距离。实数（有时也会用到复数）是标量（scalar）的例子。标量是服从一组规则的对象，这些规则是从普通的算术运算中抽象出来的。因此，可以对标量定义加法和乘法，并且这些运算满足交换律和结合律等规则。每个标量都有乘法运算和加法运算的逆元。逆元隐含地定义了减法和除法。

为了能够处理方向，还需要另一种类型——向量（vector）。物理学家和数学家用向量这个术语来表示任何既有方向又有大小的量。速度和力等物理量是向量。然而，向量在空间中没有固定的位置。

在计算机图形学中，我们经常把两个点用一条有向线段连接起来，如图4.1所示。一条有向线段既有大小（它的长度），又有方向（箭头所指的方向），因而是向量。图4.2所示的有向线段都具有相同的大小和方向，因为向量没有固定的位置，所以它们是相同的向量。我们常常把向量和有向线段这两个术语当做同义词来使用。

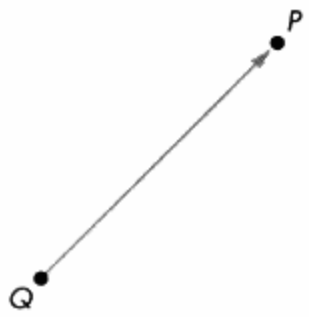


图4.1　连接两点的有向线段

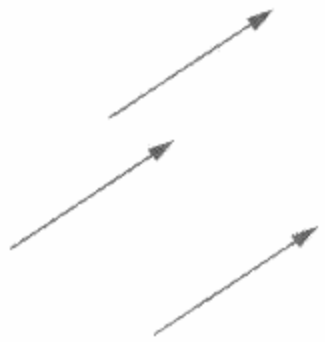


图4.2　相同的向量

我们可以用实数来改变向量的长度。在图4.3(a)中，有向线段A和有向线段B的方向相同，但B的长度是A的两倍，因此可以记为$B = 2A$。我们还可以根据三角形法则（head-to-tail rule）把两个有向线段合并成一个有向线段，如图4.3(b)所示。这里，我们把向量A的终点和向量C的起点放在同一个位置，从A的起点到C的终点的有向线段就是得出的新向量D。这个新向量D称为A与C的和，记为$D = A + C$。因为向量没有固定的位置，在借助图形进行向量加法时，如果必要可以移动向量。注意，至此已经描述了两种基本运算：向量的加法和向量的数乘。

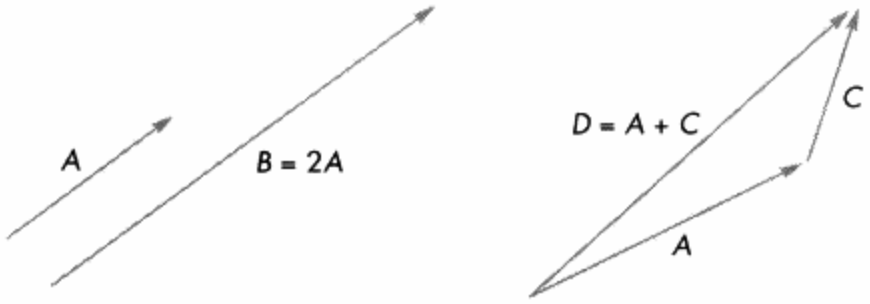


图4.3　(a) 平行的有向线段；(b) 有向线段的加法

考虑如图4.4所示的两条有向线段A和E，它们的长度相同但方向相反，所以根据三角形法则求出的和向量的长度为零。这个和向量是一个称为零向量（zero vector）的特殊向量，我们把它记为0。因为零向量的长度为零，所以它的方向没有定义。我们把E称为A的逆（inverse），记为$E = -A$。有了逆向量的概念，就可以对像$A + 2B - 3C$这样的标量－向量表达式赋予明确的意义。

虽然用一个标量去乘一个向量可以改变向量的长度，但是没有什么明显的方法可以对两个点定义一种合理的运算得到另一个点。同样，也没有什么好的方法可以对一个点和一个标量定义一种合理的运算得到另一个点。然而，对一个点和一条有向线段（向量）可以定义一种运算。如图4.5所示，我们把这个点作为有向线段的起点，这样有向线段的终点就给出了运算的结果。这个运算称为点－向量加法（point-vector addition），它的结果是一个新的点，记为$P = Q + v$。点－向量运算把点Q移动到新位置P，移动的长度和方向由向量v决定。

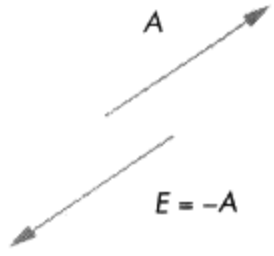


图4.4　逆向量

图4.5　点－向量加法

从稍微不同的角度来看，任何两个点都定义了一条有向线段（向量），这条有向线段从第一个点指向第二个点。我们把这个运算称为点－点减法（point-point subtraction），记为 $v = P - Q$。因为可以用标量来乘向量，所以可以对一些包含标量、向量和点的表达式赋予明确的意义，例如像 $P + 3v$ 和 $2P - Q + 3v$ 这样的表达式是有意义的，但是像 $P + 3Q - v$ 这样的表达式没有意义。

4.1.2　与坐标无关的几何

不管使用什么参考系或者坐标系，点都存在于空间中。因此，定义点或者向量并不需要坐标系。这个事实似乎不符合经验，但它对我们理解几何以及如何构建图形系统都是很关键的。考虑如图4.6所示的一个二维的例子，其中有一个坐标系（由一个原点和两个坐标轴定义）和一个简单的几何对象（正方形）。我们可以用坐标(1, 1)来表示这个正方形的左下角。正方形的相邻两边是互相垂直的，并且点(3, 1)和点(1, 1)之间的距离两倍于单位长度。这是两个值得注意的事实。如图4.7所示，假定去掉坐标轴，那么就不能指定点的坐标了。点的坐标是相对于一个原点和两个坐标轴的方向而言的，而原点位置和坐标轴的方向都是可以任意指定的，所以更重要的是那些保持不变的基本几何关系。例如，不管使用什么样的坐标系，正方形仍旧是正方形，互相垂直的直线仍旧互相垂直，点与点之间的距离仍旧保持不变。

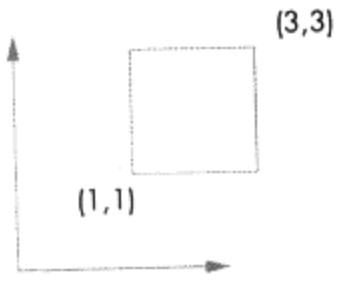


图4.6　对象和坐标系

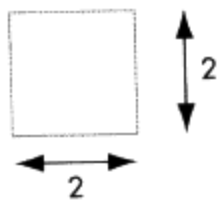


图4.7　只有对象，没有坐标系

没有了坐标系，要想指定某个点，就得用像“在那边的那个点”或者“在红色点的右边的那个蓝色的点”之类的说法，这的确不方便。要想方便地确定点的位置还是要使用坐标系和标架（frame）（见4.3节），但是眼下我们想沿着与坐标无关这个思路继续深入探讨下去，看看能得出哪些有价值的结果。

4.1.3　数学的观点：向量空间和仿射空间

我们把标量、点和向量看做是某些数学集合中的元素，然后来看看各种各样的抽象空间，这些抽象空间是用来表示和处理这三类对象的。数学家已经针对许多应用问题研究了各种抽象空间，这些应用问题涉及的范围很广，从微分方程的求解到数学函数的逼近。我们感兴趣的几种空间（向量空间、仿射空间和Euclid空间）的形式定义可以在附录B中找到。假定这些空间中的元素都是几何类型的。

我们先讨论由标量组成的集合。标量集中的任何两个标量都可以经过加法和乘法这两种运算得到另一个标量。如果这两种运算满足封闭性、结合律、交换律和对逆元素的要求（见附录B），那么这些标量就构成了一个标量场（scalar field）。我们熟悉的标量有实数、复数和有理函数等。

线性空间（向量空间）也许是最重要的数学空间。向量空间中包含两种不同的实体——向量和标量。除了两个标量之间的运算，向量空间中还定义了标量－向量乘法（scalar-vector multiplication）和向量－向量加法（vector-vector addition），前者由一个标量和一个向量得出另一个向量，后者由两个向量得出另一个向量。向量空间的例子有实数 n 元组和前面提到的有向线段。

向量空间中不一定要有度量。Euclid空间（Euclidean space）是向量空间的扩展，它增加了对大小或者距离的度量，我们可以用度量来定义线段的长度等概念。

仿射空间（affine space）也是向量空间的扩展，除了标量和向量，它还包括另外一种对象－点。尽管在仿射空间中对两个点以及一个点和一个标量没有定义运算，但对一个向量和一个点定义了一种运算——向量－点加法，它的结果是一个点。也可以说有一种称为点－点减法的运算，这种运算由两个点得到一个向量。4.1.1节中的点和有向线段就是仿射空间的一个例子。

在这些抽象空间中，对象的定义不依赖于任何特定的表示，对象只是集合中的元素。向量空间的主要概念之一是通过一组基向量来表示向量。这种表示（见4.3节）把抽象的对象和它们的实现联系在一起。在不同的表示之间可以进行转换，从而引出了几何变换。

4.1.4　计算机科学的观点

把标量、点和向量看做是集合中的元素，并且可以按照某些公理在元素之间进行运算，这是数学家更愿意采用的观点，而计算机科学家更愿意把它们看做是抽象数据类型（abstract data type，ADT）。ADT是在数据上定义的一些运算，这些运算的定义不依赖于数据在计算机内部的表示方式和运算的具体实现方式。数据抽象是现代计算机科学的基本概念。例如，把一个元素添加到一个列表中是一种运算，对这种运算的定义不依赖于列表的存储方式。再如，对多项式乘法的定义也不依赖于在一台特定的计算机中如何表示实数。对这个概念熟悉的读者应该可以很清楚地区分对象（对象之间的运算）和对象在一个特定系统中的表示（运算的实现）。从计算的角度来看，不管一个特定的系统在其内部如何表示对象，都应该能够通过下面的代码来定义几何对象：

```
vector u,v;
point p,q;
scalar a,b;
```

在C++等面向对象语言中，可以利用像类和算符重载这样的语言特性，所以我们能够对几何数据类型使用下面这样的代码：

```
q = p+a*v;
```

当然，必须定义执行这些运算的函数，而为了编写这些函数的代码，我们必须弄清楚运算的数学定义。首先要定义对象，然后看哪个抽象数学空间包含我们需要的运算。

4.1.5　几何ADT

看待标量、点和向量的这三种观点为我们提供了处理几何实体的数学框架和计算框架。总之，在计算机图形学中，标量是实数，在标量之间可以进行通常的加法和乘法；几何点是空间中的位置；向量是有向线段。这些对象及其运算合在一起构成了一个仿射空间。我们也可以在程序中创建与之相对应的ADT。

下一步是要说明如何由这三种对象构造各种几何对象以及如何实现这些对象之间的几何运算。我们将使用下面的符号：希腊字母 $\alpha, \beta, \gamma, \cdots$ 表示标量；大写字母 $P, Q, R, \cdots$ 表示点；小写字母 $u, v, w, \cdots$ 表示向量。我们还没有引入任何坐标系，所以向量和点的符号指的是抽象的对象，而不是这些对象在某个特定坐标系下的表示。在4.3节中用粗体字母表示后一种情况。向量 v 的长度（magnitude）是一个实数，记做 $|v|$。向量－标量乘法（见附录B）且有下面的性质：

$$|\alpha v| = |\alpha||v|$$

如果 α 为正，则 αv 的方向和 v 的方向相同；如果 α 为负，则两者的方向相反。

有两种等价的运算可以把点和向量联系起来。第一种运算是对两个点 P 和 Q 做减法，运算的结果是一个向量 v，记为：

$$v = P - Q$$

作为这种运算的一个推论，对任意点 Q 和向量 v，存在唯一一个点 P 满足上面的关系。这个结果可以表述为：给定一点 Q 和向量 v，存在一个点 P 使得下式成立：

$$P = Q + v$$

这样，通过点 – 向量加法运算得到了 P。图 4.8 给出了这种运算的图形解释。三角形法则使向量 – 向量加法显得非常直观。如图 4.9(a)所示，连接 u 的起点和 v 的终点的有向线段就是 $u + v$。如图 4.9(b)所示，我们还可以利用三角形法则证明，对任意三点 P、Q 和 R，都有：

$$(P - Q) + (Q - R) = P - R$$

图 4.8　点 – 点减法

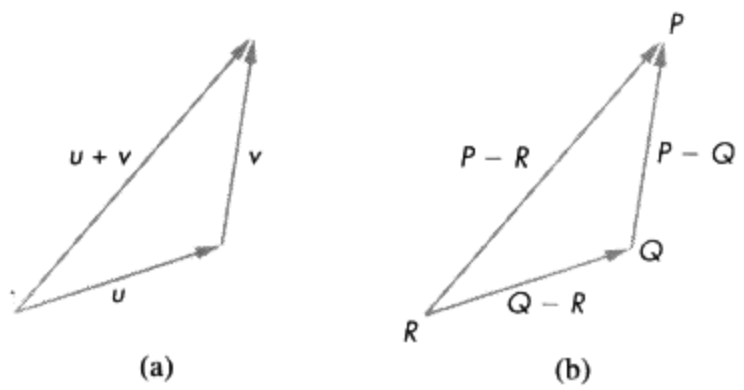


图 4.9　三角形法则：(a) 用于向量；(b) 用于点

4.1.6　直线

从点和向量之和（或者两个点之差）可以引出仿射空间中直线的概念。考虑所有形如

$$P(\alpha) = P_0 + \alpha d$$

的点，这里 P_0 是任意一个点，d 是任意一个向量，α 是一个可以在某个范围内变化的标量。如果给定了在仿射空间中对点、向量和标量进行运算的规则，那么对于 α 的任何值，求出的函数值 $P(\alpha)$ 都对应一个点。对于几何向量（有向线段），这些点位于一条直线上，如图 4.10 所示。$P(\alpha) = P_0 + \alpha d$ 称为直线的参数形式（parametric form），因为我们通过改变参数 α 来生成直线上的点。当 $\alpha = 0$ 时，直线经过点 P_0；当 α 增大时，生成的点都位于直线上向量 d 所指的那一侧。如果只允许 α 取非负值，就得到了从 P_0 出发、沿着 d 的方向延伸的一条射线（ray）。这样，直线在两个方向上都是无限长；线段是在直线上两个点之间的部分，长度有限；射线在一个方向上无限延伸。

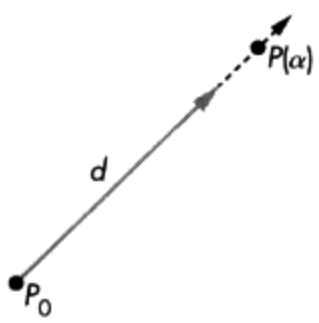


图 4.10　仿射空间中的直线

4.1.7　仿射加法

尽管在仿射空间中对两个向量可以做加法，可以用一个标量去乘一个向量，并且点和向量也可以相加，但对两个点没有定义加法，也不能用一个标量去乘一个点。不过，有一种称为仿射加法（affine addition）的运算，它和上面提到的后两种没有被定义的运算有某些相似。对任意的点P、向量v和正的标量α，下面的公式可以表示点Q沿向量v方向所在直线上的所有点，如图4.11所示：

$$P = Q + \alpha v$$

我们总可以找到一个点R，使得

$$v = R - Q$$

于是

$$P = Q + \alpha(R - Q) = \alpha R + (1 - \alpha)Q$$

这种运算看起来像是两个点的加法，我们还可以把它写成另一种等价的形式：

$$P = \alpha_1 R + \alpha_2 Q$$

其中

$$\alpha_1 + \alpha_2 = 1$$

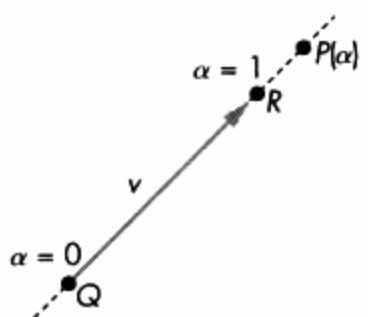


图4.11　仿射加法

4.1.8　凸性

如果对于一个对象中的任意两个点，连接它们的线段上所有的点仍然位于这个对象中，那么这个对象就是凸的（convex）。在第2章里讲到了凸性对于多边形的重要性。借助仿射和的概念可以更深刻地理解凸性。如图4.12所示，当$0 \leqslant \alpha \leqslant 1$时，仿射和定义了连接$R$和$Q$的线段，显然这条线段是凸的。可以把仿射和的概念扩展到由n个点$P_1, P_2, \cdots, P_n$定义的对象。考虑形如下式的点：

$$P = \alpha_1 P_1 + \alpha_2 P_2 + \cdots + \alpha_n P_n$$

由数学归纳法可以证明（见习题4.28）：当且仅当下式成立时上面的和有定义：

$$\alpha_1 + \alpha_2 + \cdots + \alpha_n = 1$$

如果满足下面的约束，那么由n点的仿射和所定义的点集称为这n个点的凸包（convex hull）（见图4.13）。

$$\alpha_i \geqslant 0, \quad i = 1, 2, \cdots, n$$

容易验证连接$\{P_1, P_2, \cdots, P_n\}$中任何两点的线段都在这$n$点的凸包中。从几何上看，如果用一张绷紧的曲面包围给定的一组点，就得到了这些点的凸包。凸包是包含这些点的最小凸对象。凸性的概念在设计曲线和曲面时非常有用，第12章还会用到它。

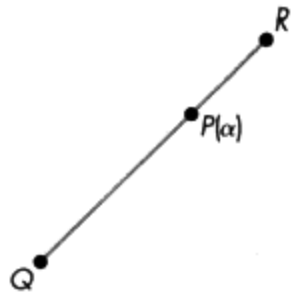


图 4.12　连接两个点的线段

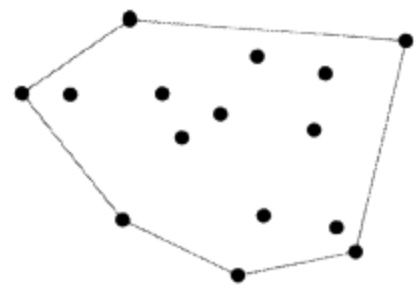


图 4.13　凸包

4.1.9　点积和叉积

与两个向量的位置关系有关的许多几何概念都是用这两个向量的点积（内积）（dot product 或 inner product）和叉积（外积）（cross product 或 outer product）来表述的。u 和 v 的内积记做 $u \cdot v$（见附录 B）。如果 $u \cdot v = 0$，就说 u 和 v 是正交的。在 Euclid 空间中，向量的长度是有定义的。一个向量的长度的平方可由点积给出：

$$|u|^2 = u \cdot u$$

下式给出了两个向量夹角的余弦：

$$\cos\theta = \frac{u \cdot v}{|u||v|}$$

此外，$|u|\cos\theta = u \cdot v/|v|$ 是 u 在 v 上的正交投影的长度，如图 4.14 所示。这样，借助点积可以表述一个几何上的结果：从向量 u 的终点作出正交于 v 的向量可以得出从一个点（u 的终点）到线段 v 的最短距离。我们还可以看到，向量 u 可以分解成 u 在 v 上的正交投影和一个与 v 正交的向量之和。

在向量空间中，如果一组向量中的任何一个向量都不能由其他向量通过标量 – 向量乘法和向量 – 向量加法表示，则称这组向量是线性无关的（linearly independent）。向量空间的维数（dimension）是线性无关向量组中向量数目的最大值。在三维空间中，给定了任何三个线性无关的向量，则可以通过点积构造出三个互相正交的向量。附录 B 对这个过程做了简要的介绍。我们还可以由不平行的两个向量 u 和 v 求出第三个向量 n，使得 n 与 u 和 v 都正交（见图 4.15）。这个向量是通过叉积得到的：

$$n = u \times v$$

注意，在三维空间里，对于任何不平行的两个向量，可以利用叉积导出三个两两正交的向量。开始时仍然是两个向量 u 和 v，我们先像上面那样求出 n，然后利用下式求出 w：

$$w = u \times n$$

这样得出的 u、n、w 两两正交。

在附录C中利用向量在坐标系下的表示导出了叉积的直接计算公式。利用 u 和 v 的叉积的长度可以求出它们夹角 θ 的正弦：

$$|\sin\theta| = \frac{|u \times v|}{|u||v|}$$

注意，向量 u、v 和 n 构成了一个右手坐标系（right-handed coordinate system），即右手拇指指向 u 的方向，食指指向 v 的方向，中指指向 n 的方向。

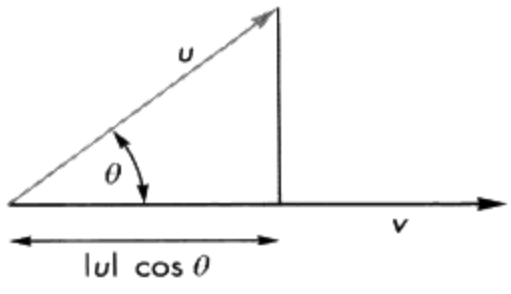


图 4.14　点积和投影

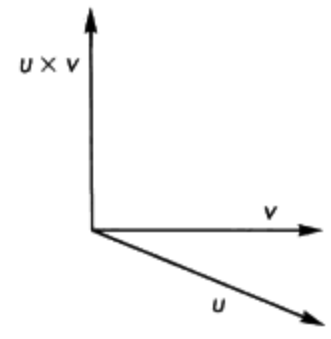


图 4.15　叉积

4.1.10　平面

在仿射空间中，直接推广直线的参数形式就可以得到平面的定义。由简单的几何知识可知：不在一条直线上的三点唯一确定了一个平面。假定 P、Q 和 R 是仿射空间中这样的三个点，连接 P 和 Q 的线段是形如下式的点的集合：

$$S(\alpha) = \alpha P + (1-\alpha)Q, \qquad 0 \leqslant \alpha \leqslant 1$$

假定在这条线段上随便取一点 S，并把 S 和 R 连接起来，如图 4.16 所示。可以用第二个参数 β 把线段 SR 上的点表示为：

$$T(\beta) = \beta S + (1-\beta)R, \qquad 0 \leqslant \beta \leqslant 1$$

每一个这样的点需要由两个参数 α 和 β 来共同确定，并且这些点都在由 P、Q 和 R 确定的平面上。把前两个等式组合起来，就得到了平面方程的一种形式：

$$T(\alpha, \beta) = \beta[\alpha P + (1-\alpha)Q] + (1-\beta)R$$

这个方程可以重新整理成下面的形式：

$$T(\alpha, \beta) = P + \beta(1-\alpha)(Q-P) + (1-\beta)(R-P)$$

我们注意到 $Q-P$ 和 $R-P$ 是任意两个不平行的向量，所以我们已经证明了可以借助一个点 P_0 和不平行的两个向量 u 与 v 把平面表示为：

$$T(\alpha, \beta) = P_0 + \alpha u + \beta v$$

我们还可以看到对于 $0 \leqslant \alpha, \beta \leqslant 1$，所有的点 $T(\alpha, \beta)$都位于三角形 PQR 中。如果点 P 在这个平面内，那么

$$P - P_0 = \alpha u + \beta v$$

如图 4.17 所示，可以找到一个与 u 和 v 都正交的向量 n。如果使用叉积

$$n = u \times v$$

那么平面的方程变为：

$$n \cdot (P - P_0) = 0$$

向量 n 称为平面的法向量，它和平面垂直或者说正交。前面对直线导出的点函数 $P(\alpha)$和这里对平面导出的点函数 $T(\alpha, \beta)$称为直线和平面的参数形式（parametric form），因为对参数 α 和 β 的每一个取值它们给出了空间中的一个点。

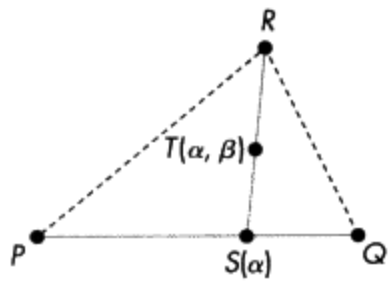


图 4.16　确定平面上的点

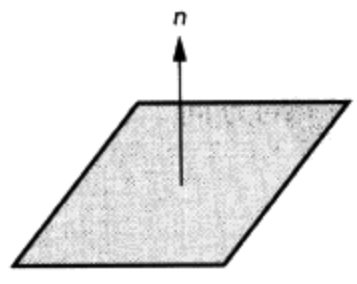


图 4.17　平面的法向量

4.2　三维图元

在三维空间里，我们遇到的几何对象要比在二维空间里多得多。第2章主要讨论位于一个平面内的图形，那时考虑的对象是像线段这样的简单曲线和像简单多边形这样的内部明确定义的平面对象。三维空间里仍然有这些对象，但不再把它们限制在同一个平面内。这样，平面曲线成为空间中的曲线（见图4.18），而包含内部区域的平面对象可以成为空间中的曲面（见图4.19）。此外，我们还会遇到实体对象，比如平行六面体和椭球体（见图4.20）。

图4.18　三维空间中的曲线

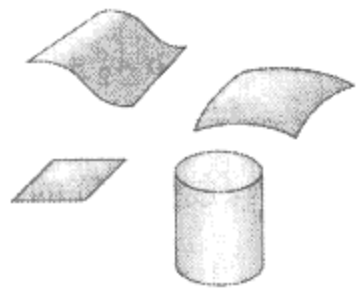


图4.19　三维空间中的曲面

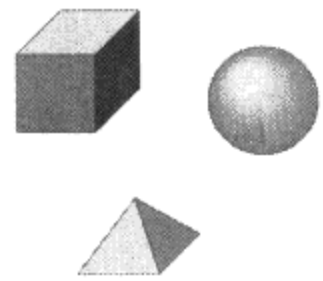


图4.20　实体对象

为了处理这些三维对象，要对二维图形系统进行扩展。这时会遇到两个问题：第一，这些对象的数学定义可能会很复杂；第二，我们只对能够在图形系统中有效实现的对象感兴趣。现有的图形系统不能支持所有的三维对象，除非通过近似的方法。

现有的图形硬件和软件非常适合处理某些三维对象，这些对象有三个特征：

1. 可以用这些三维对象的表面来描述它们，并认为它们是中空的。
2. 这些对象可以由三维空间中的一组顶点来确定。
3. 这些对象的表面要么由凸多边形组成，要么可以通过凸多边形来近似。

要理解上面这三个特征，只需考虑一下大多数现代图形系统最适合进行什么样的处理：它们最适于绘制三角形或者其他多边形。商用图形卡每秒钟可以绘制超过1亿个小的平面多边形[①]。

第一个条件意味着只需要二维图元就可以建模三维图元，因为表面是二维而不是三维实体。第二个条件在第1章和第2章中已经用过，这里把它扩展到三维图元。如果一个对象是用顶点定义的，就可以用流水线结构来高速处理这些顶点，而且可以在光栅化阶段用硬件生成对象的图像。最后一个条件在讨论平面内的多边形时已经遇到过，这里被推广到三维图元。大多数图形系统已经对处理点和多边形进行了优化。三维空间中的多边形可以通过一个有序的顶点列表来定义。然而，如果顶点的数目超过三个，那么这些顶点不一定位于同一个平面内。对于顶点不在同一平面内的多边形，没有简单的办法可以定义它的内部。因此，大多数图形系统要求用户指定简单的平面多边形，否则不能保证多边形光栅化后的结果是正确的。因为三角形总是位于同一个平面内，所以要么建模系统被设计成总是生成三角形，要么由图形系统把任意多边形分割成三角形或者说用三角形来细化（tessellate）多边形。如果用同样的方法来处理曲面，比如球面，我们就会意识到应该用许多小的平面多边形来近似球面。因此，尽管建模系统支持弯曲的对象，我们仍旧假定这类对象是用多边形近似实现的。

除了用多边形近似表面以外，还有其他的方法，其中主要的有构造实体几何（constructive solid geometry，CSG）。在这样的系统中，我们通过对一组体积元素进行并、交等操作来构造对象。

① 图形系统的性能通常由给定时间里能够绘制的三角形的数目来度量，这些三角形可以作为三角形带来绘制。此外，对三角形的绘制包括光照和明暗处理以及纹理映射，这些特性都已在现代图形卡的硬件里实现了。

第10章将讨论CSG模型。尽管这是一种很好的建模方法，但和基于表面的多边形模型比起来，CSG模型不容易绘制。虽然这种情况将来可能会改变，但这里只详细讨论表面建模方法。

我们要处理的所有图元都可以通过一组顶点来确定。当讨论从抽象对象转到实际对象时，必须考虑如何在图形系统中表示空间里的点。

4.3　坐标系和标架

到目前为止，向量和点都被当做抽象的对象，并没有考虑它们在某个坐标系下的表示。在一个三维向量空间中，可以用任意三个线性无关的向量 v_1、v_2、v_3 把任意一个向量 w 表示为（见附录B）：

$$w = \alpha_1 v_1 + \alpha_2 v_2 + \alpha_3 v_3$$

标量 α_1、α_2、α_3 是 w 关于基（basis）v_1、v_2、v_3 的分量（component），这些关系如图4.21所示。可以把 w 关于这个基的表示（representation）写成列矩阵的形式：

$$\mathbf{a} = \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \alpha_3 \end{bmatrix}$$

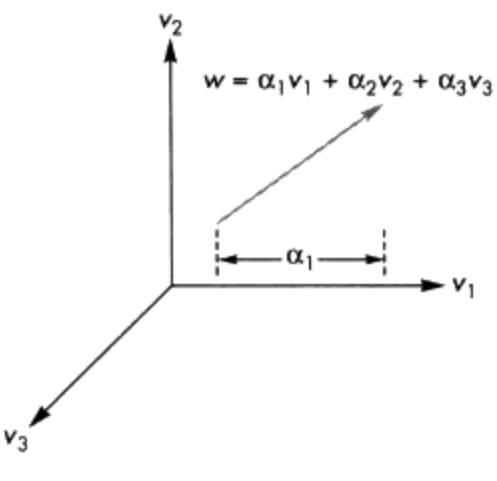


图4.21　用三个基向量来表示向量

这里为了和原来的抽象向量 w 相区别，我们用粗体字母代表 w 在某个特定基下的表示。还可以把这个关系写成：

$$w = \mathbf{a}^T \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} = \mathbf{a}^T \mathbf{v}$$

其中，

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix}$$

通常我们认为基向量 v_1、v_2、v_3 定义了一个坐标系（coordinate system）。然而，除了向量，我们还要处理点和标量，所以还要有一种更一般的方法。图4.22表示出了问题的一个方面。图4.22(a)中的三个向量构成了一个坐标系。通常在表示坐标系时把基向量的起点画在同一处，图4.22(a)也是如此。这三个基向量可以构成一个基，即可以表示三维空间中的任何向量。不过向量虽然有大小和方向，可是没有位置属性。于是，图4.22(b)与图4.22(a)是等价的，因为我们只是移动了基向量的位

置，并没有改变它们的大小和方向。虽然在数学上图4.22(b)与图4.22(a)表达了同样的意思，但多数读者可能会对图4.22(b)感到困惑。我们还没有解决如何表示点的问题。点和向量是不同的，它们是有固定位置的实体。

因为仿射空间中包含点，所以一旦在这样的空间里选定了一个特定的参考点（原点），就可以以一种明确的方式来表示所有的点。如图4.22(a)所示，习惯上把原点作为坐标轴的起点，这在仿射空间里是有意义的，因为我们既要表示点又要表示向量。不过，这种表示除了要确定基向量以外，还要确定参考点。原点和基向量决定了一个标架（frame）。关于标架的一个不太严谨的说法是：它把向量坐标系的原点固定在了某个点 P_0 处。在一个给定的标架下，每个向量可以唯一地表示为：

$$w = \alpha_1 v_1 + \alpha_2 v_2 + \alpha_3 v_3 = \mathbf{a}^T \mathbf{v}$$

这和向量空间中的表示一样；此外，每个点可以唯一地表示为：

$$P = P_0 + \beta_1 v_1 + \beta_2 v_2 + \beta_3 v_3 = P_0 + \mathbf{b}^T \mathbf{v}$$

这样，在一个标架下，表示一个向量需要三个标量，而表示一个点需要三个标量和原点的位置。4.3.4节将会讲到，通过放弃更熟悉的坐标系的概念转而使用标架的概念，可以避免由于向量具有大小和方向而没有固定位置所带来的困难。此外，我们能够借助矩阵来表示点和向量，并且点和向量这两种几何类型的矩阵表示是有区别的。

因为点和向量是两种不同类型的几何对象，所以不能在它们之间画等号，因此应该用怀疑的眼光来看待图4.23中采用的表示方法，因为该图把一个点和从原点到这个点的有向线段等同起来。图4.23的一种正确解释是：可以通过把一个固定的参考点（原点）当做起点和空间中的一个点当做终点来定义向量。注意，向量和点一样，其存在与参考系无关，不过我们最终还是要考虑点和向量在某个特定的参考系下的表示。

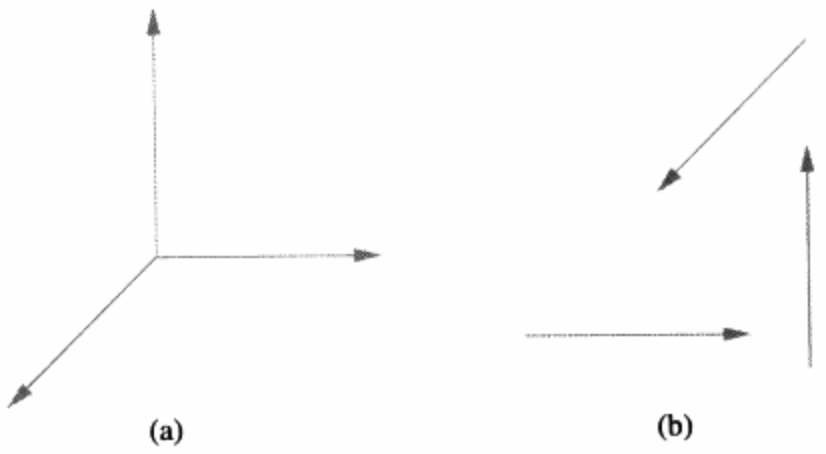


图 4.22　坐标系：(a) 起点重合的基向量；(b) 平移之后的基向量

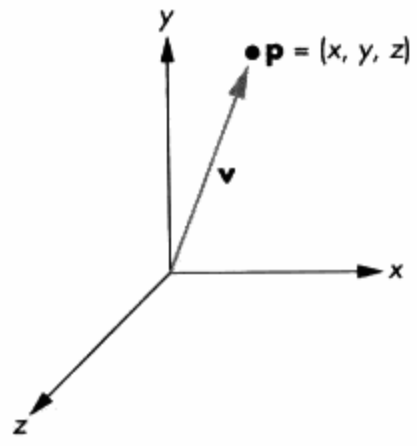


图 4.23　一种不妥的向量表示方法

4.3.1　向量的表示和 N 元组

假定向量 e_1、e_2 和 e_3 构成了一个基。任一向量 v 在这个基下的表示可由一个向量 a 的分量(α_1, α_2, α_3)给出，其中 α_1，α_2，α_3 满足：

$$v = \alpha_1 e_1 + \alpha_2 e_2 + \alpha_3 e_3$$

我们把这些基向量[①]自身的表示记为 $\mathbf{e}_1$、$\mathbf{e}_2$、$\mathbf{e}_3$，它们是：

① 许多教科书把这些向量称为单位基向量 $\mathbf{i}$、$\mathbf{j}$、$\mathbf{k}$，并把其他向量在这个基下的表示写成 $v = \alpha_1\mathbf{i} + \alpha_2\mathbf{j} + \alpha_3\mathbf{k}$。

$$\mathbf{e}_1 = (1, 0, 0)^T$$
$$\mathbf{e}_2 = (0, 1, 0)^T$$
$$\mathbf{e}_3 = (0, 0, 1)^T$$

换句话说，三元组(1, 0, 0)是第一个基向量的表示。因此，无须考虑抽象的向量，我们可以用三元组来思考问题，并且可以把任何向量 v 的表示写成列矩阵 $\mathbf{a}$ 或者三元组$(\alpha_1, \alpha_2, \alpha_3)$的形式，这里，

$$\mathbf{a} = \alpha_1\mathbf{e}_1 + \alpha_2\mathbf{e}_2 + \alpha_3\mathbf{e}_3$$

$\mathbf{e}_1$、$\mathbf{e}_2$、$\mathbf{e}_3$是我们熟悉的Euclid空间$\mathbf{R}^3$中的向量。向量空间$\mathbf{R}^3$和原来的那个由几何向量构成的向量空间是等价的，或者说是同构的（isomorphic）。从实际应用的角度来看，处理三元组（更一般的是 n 元组）几乎总是要比处理其他的表示更容易。

4.3.2　坐标系的变换

我们经常需要在基向量改变时求出向量在新坐标系下的表示。例如，在OpenGL中，我们利用对于模型来说是自然的坐标系或者标架来定义几何对象，这个标架称为对象标架（object frame）或者建模标架（model frame）。然后这些对象标架中的模型又被放到世界标架（world frame）中。有时我们需要知道这些对象在照相机中看来是什么样子的，此时就要从世界标架变换到照相机标架（camera frame）或者眼标架（eye frame）。从对象标架到眼标架的转换是由模–视变换矩阵来完成的。

我们先来考虑如何求出向量在新坐标系下的表示。假定$\{v_1, v_2, v_3\}$和$\{u_1, u_2, u_3\}$是两个基，第二个基中的每个基向量都可以用第一个基来表示（反之亦然）。因此，存在9个标量$\{\gamma_{ij}\}$，使得

$$u_1 = \gamma_{11}v_1 + \gamma_{12}v_2 + \gamma_{13}v_3$$
$$u_2 = \gamma_{21}v_1 + \gamma_{22}v_2 + \gamma_{23}v_3$$
$$u_3 = \gamma_{31}v_1 + \gamma_{32}v_2 + \gamma_{33}v_3$$

这些标量定义了一个矩阵：

$$\mathbf{M} = \begin{bmatrix} \gamma_{11} & \gamma_{12} & \gamma_{13} \\ \gamma_{21} & \gamma_{22} & \gamma_{23} \\ \gamma_{31} & \gamma_{32} & \gamma_{33} \end{bmatrix}$$

这两个基之间的关系可以表示为：

$$\begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix} = \mathbf{M} \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix}$$

或者

$$\mathbf{u} = \mathbf{M}\mathbf{v}$$

通过矩阵$\mathbf{M}$可以从向量在一个基下的表示得到它在另一个基下的表示。$\mathbf{M}$的逆是从$\{v_1, v_2, v_3\}$变换到$\{u_1, u_2, u_3\}$的矩阵表示。设向量 w 关于$\{v_1, v_2, v_3\}$的表示为$\{\alpha_1, \alpha_2, \alpha_3\}$，即

$$w = \alpha_1v_1 + \alpha_2v_2 + \alpha_3v_3$$

等价表示为：

$$w = \mathbf{a}^T\mathbf{v}$$

其中，

$$\mathbf{a} = \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \alpha_3 \end{bmatrix}$$

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix}$$

假定 **b** 是 w 关于$\{u_1, u_2, u_3\}$的表示，即

$$w = \beta_1 u_1 + \beta_2 u_2 + \beta_3 u_3$$

或者

$$w = \mathbf{b}^T \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix} = \mathbf{b}^T \mathbf{u}$$

其中，

$$\mathbf{b} = \begin{bmatrix} \beta_1 \\ \beta_2 \\ \beta_3 \end{bmatrix}$$

利用第二个基的基向量在第一个基下的表示，可得：

$$w = \mathbf{b}^T \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix} = \mathbf{b}^T \mathbf{M} \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} = \mathbf{a}^T \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix}$$

因此，

$$\mathbf{a} = \mathbf{M}^T \mathbf{b}$$

矩阵

$$\mathbf{T} = (\mathbf{M}^T)^{-1}$$

通过下面简单的矩阵等式把 **a** 变成 **b**：

$$\mathbf{b} = \mathbf{T}\mathbf{a}$$

由此看来，在进行坐标系变换时使用向量的三元组表示或者说 $\mathbf{R}^3$ 中的元素是非常方便的，这样就无须再考虑原来的向量，特别是有向线段了。这是一个重要的结果，因为它使我们从抽象的向量转到由标量组成的列矩阵（向量的表示）。需要记住的是：当把实数组成的列矩阵当做“向量”时不能丢掉基，否则就会得到错误坐标系下的结果了。

在上面的讨论中变化的是基，原点是不变的。我们可以利用这些变换来表示对一个基进行旋转和缩放得到另一个基的变换，如图 4.24 所示。然而，像图 4.25 那样对原点做一个简单的平移是不能用这种方式表示的。在讨论一个简单的例子之后，我们将介绍齐次坐标，借助它可以进行标架变换，同时仍能使用矩阵来表示标架变换。

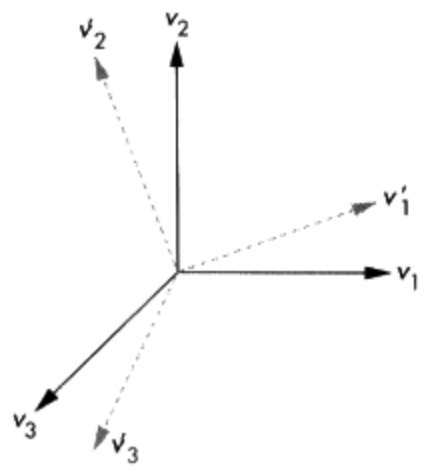


图 4.24　基的旋转和缩放

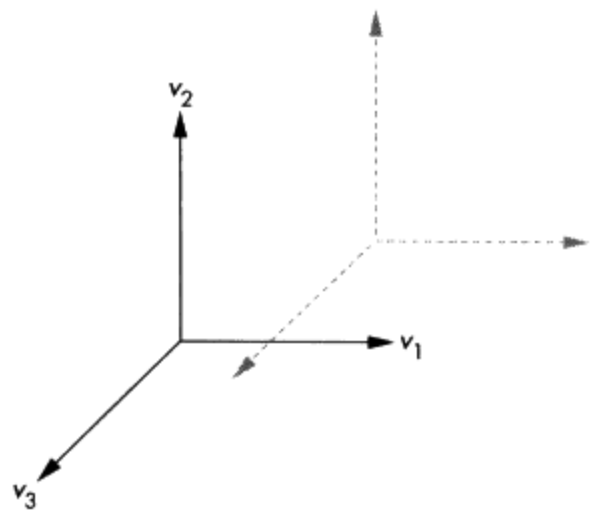


图 4.25　基的平移

4.3.3　表示变换的例子

假定向量 w 在某个基下的表示为：

$$\mathbf{a} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

我们把这三个基向量记做 v_1、v_2、v_3，于是有：

$$w = v_1 + 2v_2 + 3v_3$$

现在假定从 v_1、v_2、v_3 构造出一个新基：

$$u_1 = v_1$$

$$u_2 = v_1 + v_2$$

$$u_3 = v_1 + v_2 + v_3$$

矩阵 $\mathbf{M}$ 为：

$$\mathbf{M} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

把基$\{v_1, v_2, v_3\}$下的表示变换成基$\{u_1, u_2, u_3\}$下的表示的矩阵为：

$$\mathbf{T} = (\mathbf{M}^T)^{-1}$$

$$= \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}^{-1}$$

$$= \begin{bmatrix} 1 & -1 & 0 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{bmatrix}$$

在新坐标系下，w 的表示为：

$$\mathbf{b} = \mathbf{Ta} = \begin{bmatrix} -1 \\ -1 \\ 3 \end{bmatrix}$$

即

$$w = -u_1 - u_2 + 3u_3$$

如果在$\mathbf{R}^3$中考虑基的变换，那么基和向量就不那么抽象了，它们都是实数三元组，可以把v_1、v_2、v_3想象成$\mathbf{R}^3$中的单位基：

$$\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \qquad \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \qquad \mathbf{e}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

现在我们在$\mathbf{R}^3$这个具体的向量空间里考虑上面得到的结果。假定$\{v_1, v_2, v_3\}$是默认的(x, y, z)坐标系的三个基向量，它们两两正交。假定有三个向量，它们的表示分别为(1, 0, 0)、(1, 1, 0)和(1, 1, 1)。因此，第一个向量的方向沿着x轴，第二个向量平行于平面$z = 0$，第三个向量的方向关于原来坐标系的三个基向量的方向对称。这三个新向量虽然不是正交的，但却是线性无关的，因此它们定义了一个新的坐标系$x'y'z'$。原来的坐标系xyz的三个基向量在坐标系$x'y'z'$下的表示是矩阵$\mathbf{T}$的三个列向量。

4.3.4　齐次坐标

前面已经指出，向量和点可能会被搞混（见图 4.23），在点和向量的三维表示中这个问题仍旧存在。考虑由点P_0和向量v_1、v_2、v_3所定义的标架。通常我们会把位于(x, y, z)的点P用下面的列矩阵来表示：

$$\mathbf{p} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

其中，x、y和z满足：

$$P = P_0 + xv_1 + yv_2 + zv_3$$

如果以这种方式来表示点，那么点的表示和向量的表示在形式上是一样的，因为向量w可以表示为：

$$w = xv_1 + yv_2 + zv_3$$

齐次坐标（homogeneous coordinate）可以克服这个困难，它对三维空间中点和向量的表示都是四维的。在由(v_1, v_2, v_3, P_0)确定的标架中，任一点P可以唯一地表示为：

$$P = \alpha_1 v_1 + \alpha_2 v_2 + \alpha_3 v_3 + P_0$$

如果把点与标量 0 和 1 的“乘法”定义为：

$$0 \cdot P = \mathbf{0}$$

$$1 \cdot P = P$$

那么可以用矩阵乘法把点P形式地表示为：

$$P = \begin{bmatrix} \alpha_1 & \alpha_2 & \alpha_3 & 1 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ P_0 \end{bmatrix}$$

严格地说，这个表达式不是点积，因为矩阵中的元素不是同一类型的。然而，可以把它当做内积来计算：对应元素相乘然后再把乘积相加。等式右边的四维行矩阵就是点P在由v_1、v_2、v_3和P_0所确

定的标架下的齐次坐标表示（homogeneous-coordinate representation）。等价地讲，也可以说P的表示是下面的列矩阵：

$$\mathbf{p}=\begin{bmatrix}\alpha_1\\ \alpha_2\\ \alpha_3\\ 1\end{bmatrix}$$

在同一个标架下，任一向量w可以写成下面的形式：

$$\begin{aligned}w&=\delta_1 v_1+\delta_2 v_2+\delta_3 v_3\\ &=[\,\delta_1\quad\delta_2\quad\delta_3\quad 0\,]^T\begin{bmatrix}v_1\\ v_2\\ v_3\\ P_0\end{bmatrix}\end{aligned}$$

因此，可以用一个列矩阵来表示w：

$$\mathbf{w}=\begin{bmatrix}\delta_1\\ \delta_2\\ \delta_3\\ 0\end{bmatrix}$$

齐次坐标的几何意义可以从许多角度来解释，这里只强调一点：矩阵代数中的运算一般都可以应用到点和向量的齐次坐标表示上。作为一个例子，我们来考虑标架的变换。前面已经讲到，如果使用三维表示，则不能像对坐标系变换那样用矩阵来表示标架变换。如果(v_1, v_2, v_3, P_0)和(u_1, u_2, u_3, Q_0)是两个标架，那么可以把第二个标架的基向量和参考点用第一个标架来表示：

$$\begin{aligned}u_1&=\gamma_{11}v_1+\gamma_{12}v_2+\gamma_{13}v_3\\ u_2&=\gamma_{21}v_1+\gamma_{22}v_2+\gamma_{23}v_3\\ u_3&=\gamma_{31}v_1+\gamma_{32}v_2+\gamma_{33}v_3\\ Q_0&=\gamma_{41}v_1+\gamma_{42}v_2+\gamma_{43}v_3+P_0\end{aligned}$$

可以把这些方程写成矩阵的形式：

$$\begin{bmatrix}u_1\\ u_2\\ u_3\\ Q_0\end{bmatrix}=\mathbf{M}\begin{bmatrix}v_1\\ v_2\\ v_3\\ P_0\end{bmatrix}$$

其中，$\mathbf{M}$是一个4×4矩阵：

$$\mathbf{M}=\begin{bmatrix}\gamma_{11}&\gamma_{12}&\gamma_{13}&0\\ \gamma_{21}&\gamma_{22}&\gamma_{23}&0\\ \gamma_{31}&\gamma_{32}&\gamma_{33}&0\\ \gamma_{41}&\gamma_{42}&\gamma_{43}&1\end{bmatrix}$$

矩阵$\mathbf{M}$称为标架变换的矩阵表示（matrix representation）。

我们可以用$\mathbf{M}$直接计算出新标架下的表示。假定$\mathbf{a}$和$\mathbf{b}$是同一个点或者向量分别在这两个标架下的齐次坐标表示，则有：

$$\mathbf{b}^T \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ Q_0 \end{bmatrix} = \mathbf{b}^T \mathbf{M} \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ P_0 \end{bmatrix} = \mathbf{a}^T \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ P_0 \end{bmatrix}$$

因此，

$$\mathbf{a} = \mathbf{M}^T \mathbf{b}$$

多数情况下是同点和向量的表示打交道，此时我们对 $\mathbf{M}^T$ 更感兴趣，它的形式如下：

$$\mathbf{M}^T = \begin{bmatrix} \alpha_{11} & \alpha_{12} & \alpha_{13} & \alpha_{14} \\ \alpha_{21} & \alpha_{22} & \alpha_{23} & \alpha_{24} \\ \alpha_{31} & \alpha_{32} & \alpha_{33} & \alpha_{34} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

所以确定 $\mathbf{M}^T$ 需要 12 个系数。

齐次坐标还有其他的优点，后面的章节会经常利用这些优点。也许最重要的优点是所有的仿射变换（保持直线）都可以借助齐次坐标表示成矩阵相乘的形式。尽管在使用齐次坐标表示时，为了求解三维问题需要四维表示，但是齐次坐标涉及的算术运算更少。因为所有的仿射变换都可以用一致的形式来表示，所以如果连续进行多次变换（级联），那么使用四维表示要比三维表示方便得多。此外，现代硬件直接实现了齐次坐标的运算，而且利用并行机制来提高计算速度。

4.3.5　标架变换的例子

我们再来重新考虑 4.3.3 节的例子。假定仍然从基$\{v_1, v_2, v_3\}$变换到基$\{u_1, u_2, u_3\}$，并且向量 u_1、u_2、u_3 关于基$\{v_1, v_2, v_3\}$的表示不变：

$$u_1 = v_1$$

$$u_2 = v_1 + v_2$$

$$u_3 = v_1 + v_2 + v_3$$

因为参考点没有改变，所以增加下面的等式：

$$Q_0 = P_0$$

因此，我们感兴趣的矩阵是

$$\mathbf{M} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

和 $\mathbf{M}^T$，还有$(\mathbf{M}^T)^{-1}$。

假定除了改变基向量，我们还想把参考点移到另一个点，这个点在原来标架下的表示为(1, 2, 3, 1)。位移向量 $v = v_1 + 2v_2 + 3v_3$ 把 P_0 移动到 Q_0。齐次坐标表示的第 4 个分量把 Q_0 这个实体标识为点，因此用下面的等式来替换上面例子中的第 4 个等式：

$$Q_0 = P_0 + v_1 + 2v_2 + 3v_3$$

与此相应，矩阵 $\mathbf{M}^T$ 变为：

$$\mathbf{M}^T = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 2 \\ 0 & 0 & 1 & 3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

它的逆是：

$$\mathbf{T} = (\mathbf{M}^T)^{-1} = \begin{bmatrix} 1 & -1 & 0 & 1 \\ 0 & 1 & -1 & 1 \\ 0 & 0 & 1 & -3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

利用这对矩阵可以在两个标架下的表示之间来回转换。设某个点在原来标架下的表示为：

$$\mathbf{p} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 1 \end{bmatrix}$$

T把这个点的表示变换为：

$$\mathbf{p}' = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

这是新标架的原点的表示。设某个向量在原来标架下的表示为：

$$\mathbf{a} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 0 \end{bmatrix}$$

T把这个向量变换为：

$$\mathbf{b} = \begin{bmatrix} -1 \\ -1 \\ 3 \\ 0 \end{bmatrix}$$

这个结果和4.3.3节是一致的。从这两个具体的变换例子可以看到区分点和向量的重要性。

4.3.6　对表示进行处理

我们通常是和表示而不是和抽象的点打交道。因此，当指定一个点时，例如通过glVertex3f(x, y, z)，我们实际上是给出了这个点在某个标架下的表示，这个标架通常是在应用程序中所使用的对象标架。在表示之间的变换可由一个矩阵来确定：

$$\mathbf{a} = \mathbf{C}\mathbf{b}$$

其中，**a**和**b**是一个点或者向量的齐次坐标表示。4.3.4节已经讲到：矩阵**C**必定是齐次形式的，即它是矩阵**M**的转置并且可以写成下面的形式：

$$\mathbf{C}=\mathbf{M}^T=\begin{bmatrix}\alpha_{11} & \alpha_{12} & \alpha_{13} & \alpha_{14}\\ \alpha_{21} & \alpha_{22} & \alpha_{23} & \alpha_{24}\\ \alpha_{31} & \alpha_{32} & \alpha_{33} & \alpha_{34}\\ 0 & 0 & 0 & 1\end{bmatrix}$$

当处理表示时，如何找出 C 呢？其实这很容易。假定我们处于某个标架下，然后指定了另一个标架，指定的方式是给出新标架的原点和基向量在原来标架下的表示。因此，如果通过三个向量 u、v、n 以及新标架的原点 p 在原来标架下的表示来指定新标架，那么这 4 个实体的齐次坐标表示都是四元组或者说都是 $\mathbf{R}^4$ 中的元素。

下面考虑逆矩阵的问题。矩阵

$$\mathbf{T}=\mathbf{C}^{-1}$$

把新标架(u, v, n, p)下的表示变换成原标架下的表示。因此，

$$\mathbf{T}\begin{bmatrix}1\\0\\0\\0\end{bmatrix}=\mathbf{u}=\begin{bmatrix}u_1\\u_2\\u_3\\0\end{bmatrix}$$

与此类似有：

$$\mathbf{T}\begin{bmatrix}0\\1\\0\\0\end{bmatrix}=\mathbf{v}=\begin{bmatrix}v_1\\v_2\\v_3\\0\end{bmatrix},$$

$$\mathbf{T}\begin{bmatrix}0\\0\\1\\0\end{bmatrix}=\mathbf{n}=\begin{bmatrix}n_1\\n_2\\n_3\\0\end{bmatrix},$$

$$\mathbf{T}\begin{bmatrix}0\\0\\0\\1\end{bmatrix}=\mathbf{p}=\begin{bmatrix}p_1\\p_2\\p_3\\1\end{bmatrix}$$

把这些等式合并到一起，得到：

$$\mathbf{TI}=\mathbf{T}=[\,u\quad v\quad n\quad p\,]=\begin{bmatrix}u_1 & v_1 & n_1 & p_1\\ u_2 & v_2 & n_2 & p_2\\ u_3 & v_3 & n_3 & p_3\\ 0 & 0 & 0 & 1\end{bmatrix}$$

于是，

$$\mathbf{C}=[\,u\quad v\quad n\quad p\,]^{-1}=\begin{bmatrix}u_1 & v_1 & n_1 & p_1\\ u_2 & v_2 & n_2 & p_2\\ u_3 & v_3 & n_3 & p_3\\ 0 & 0 & 0 & 1\end{bmatrix}^{-1}$$

这样，利用新标架在原标架下的表示可以得到一个矩阵，该矩阵的逆可以把原标架下的表示变换成新标架下的表示。当然，我们必须求出这个逆，但是计算这样一个4 × 4矩阵的逆应该不是问题。

4.4 OpenGL中的标架

我们已经知道，OpenGL基于绘制流水线模型，而且绘制流水线的第一个步骤是对顶点进行一系列几何操作。这些几何操作可以用一系列标架变换来表征，这些标架变换要应用到由用户程序所定义的对象上。在一个典型的应用程序中，绘制流水线会涉及6个标架，但程序员一般不会直接看到全部6个标架。同一个顶点在不同的标架下会有不同的表示。按照它们在绘制流水线中出现的先后顺序，这6个坐标系依次是：

1. 对象坐标系或者建模坐标系
2. 世界坐标系
3. 眼坐标系或者照相机坐标系
4. 裁剪坐标系
5. 规范化的设备坐标系
6. 窗口坐标系或者屏幕坐标系

让我们来考虑一下当用户在程序中通过函数glVertex3f(x, y, z)指定一个顶点时发生了什么。这个顶点可以直接在应用程序中指定，也可以像第3章讨论的那样，通过某个基本对象的实例化来间接指定。在大多数应用程序里，我们往往在建模标架（model frame）或者称为对象标架（object frame）下指定对象，因为对象标架是对象自身的标架，在这个标架下可以方便地描述对象的大小、方向和位置。例如，在对象标架下，立方体的面和标架的轴平行，它的中心位于标架的原点，并且边长为1个或者2个单位。在相应的函数调用中，所用的坐标值都是相对于对象坐标系给出的。

一个应用程序可能包含成百上千个单独的对象，我们必须把它们放到一个公共的场景里。应用程序通常对每个对象实施一系列的变换，这些变换可以改变它们的大小、方向和位置，从而使它们处于一个适合于该特定应用程序的标架中。例如，在建筑设计应用程序中，可以通过实例化一个正方形来定义建筑物的一个窗户，为此需要通过缩放使其具有正确的比例和大小，这里使用的单位可能是英尺（ft）或者米（m）。应用程序中坐标系的原点可以是这个建筑物底层的中心。应用程序使用的这个标架称为世界标架（world frame），而在这个标架下的坐标就是世界坐标（world coordinate）。注意，如果在调用函数glVertex之前没有利用预先定义好的对象来建模，并且也没有应用任何变换，那么对象坐标系和世界坐标系是相同的。

对于应用程序来说，对象标架和世界标架都是很自然的标架，然而生成的图像依赖于照相机或者观察者所能看到的内容。几乎所有的图形系统都使用一个标架，该标架的原点位于照相机透镜的中心[①]，并且坐标轴平行于照相机的侧面。这就是照相机标架（camera frame），也可称为眼标架（eye frame）。因为每次标架变换都对应一个仿射变换，所以从模型坐标到世界坐标以及从世界坐标到眼坐标都可以用4 × 4矩阵来表示。OpenGL把这些变换合并为模–视变换，对应的矩阵是模–视变换矩阵。通常，使用模–视变换矩阵而不是两个单独的变换矩阵不会给应用程序员带来任何不便。第9章将讨论可编程绘制流水线，在那里会遇到必须分开处理这两个变换的情形。

① 对于透视投影，透镜的中心是投影中心（COP）；而对于正交投影（默认的投影方式），投影方向和照相机的侧面平行。

OpenGL 还使用了另外三种表示，但是目前还用不到，不过为了完整起见，这里对它们做一个简要的介绍。把对象变换到眼坐标系中以后，OpenGL 就要检查它们是否位于视见体内。如果一个对象位于视见体外，那么就在光栅化之前把它从场景里裁剪掉。为了更有效地进行裁剪，OpenGL 先执行投影变换，这个变换把所有可能是可见的对象变换到一个立方体中，该立方体的中心是裁剪坐标系（clip coordinate）的原点。第 5 章将介绍投影变换。在投影变换之后，顶点的表示仍然是齐次坐标。顶点的齐次坐标表示再经过透视除法（perspective division），即用 w 分量去除其他分量，就得到了在规范化的设备坐标系（normalized device coordinate）下的三维表示。最后一步变换根据视口提供的信息，把规范化的设备坐标系下的表示变换为窗口坐标系（window coordinate）下的三维表示。窗口坐标是用显示器上的像素来度量的，但仍然保留了深度信息。如果去掉深度坐标，就得到了二维的屏幕坐标（screen coordinate）。

从应用程序员的角度看，OpenGL 从两个标架开始：眼标架和对象标架。模 – 视变换矩阵是对象标架在眼标架下的表示。因此，模 – 视变换矩阵把点和向量在对象标架下的齐次坐标表示变换成眼标架下的表示。因为模 – 视矩阵是系统状态的一部分，所以总是有一个当前的照相机标架和一个当前的对象标架。OpenGL 提供了矩阵栈，这样就能把模 – 视变换矩阵保存起来，也相当于把标架保存起来。

初始的模 – 视矩阵是单位阵，所以此时的对象标架和眼标架是相同的。因此，如果不改变模 – 视变换矩阵，那么我们使用的标架是眼标架。正如第 2 章讲到的那样，照相机位于眼标架的原点，如图 4.26(a)所示。照相机坐标系的三个基向量的方向分别对应于：1）照相机的观察正向，这是 y 轴的正方向；2）照相机正对的方向，这是 z 轴的负方向；3）另一个正交的方向 x 轴，x 轴的正方向应使得 x 轴、y 轴、z 轴构成一个右手坐标系。如果相对于照相机标架定义了新的标架，那么利用相应的齐次坐标变换，就可以在新标架下建模。4.5 节将介绍如何定义这些变换，然后在 5.3 节通过它们确定照相机相对于对象的位置和方向。因为标架变换是由模 – 视变换矩阵表示的，而模 – 视变换矩阵可以保存，所以我们能够保存标架，而且只要改变当前的模 – 视变换矩阵，就可以在多个标架之间转换。

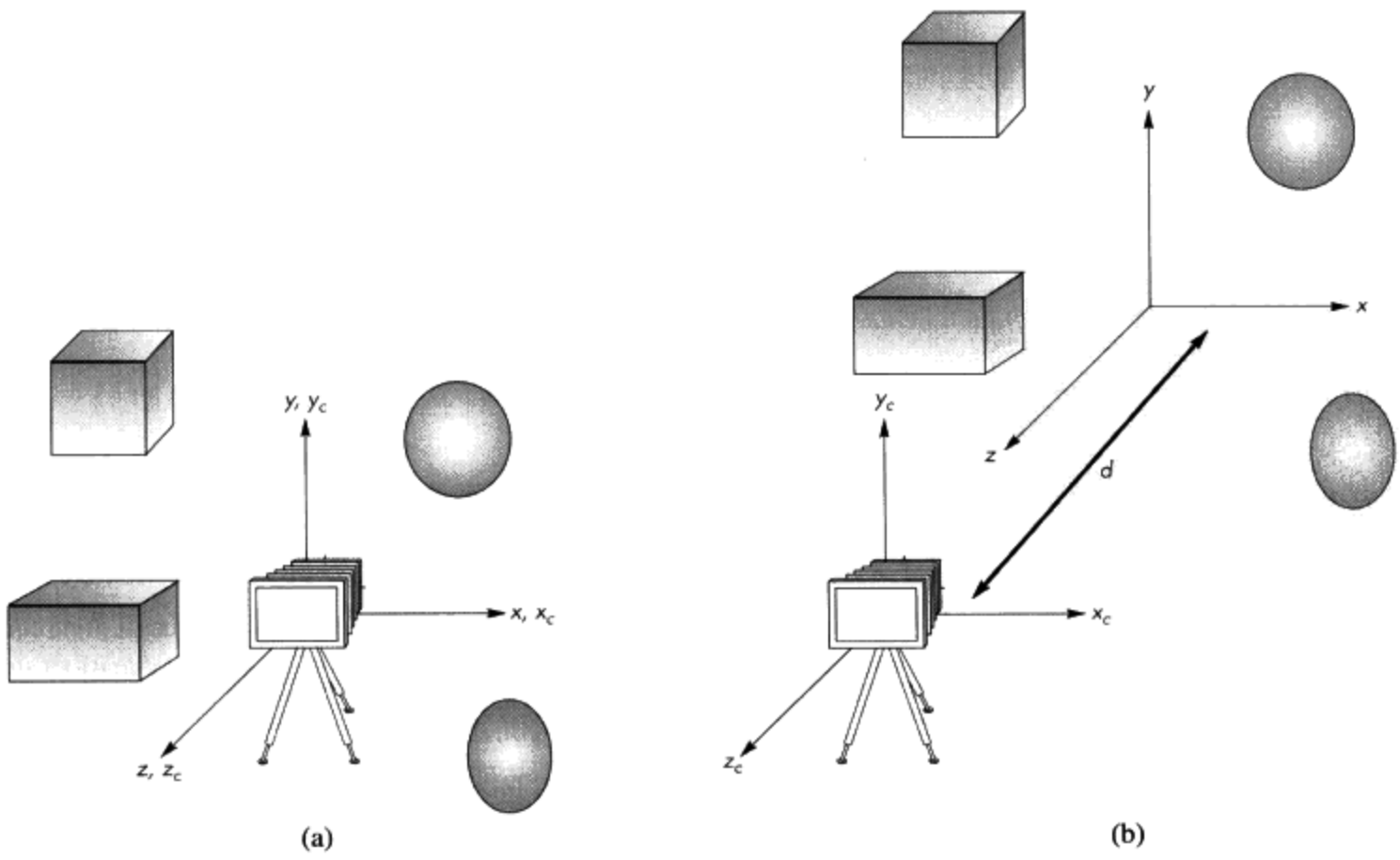


图 4.26　照相机标架和对象标架：(a) 默认的位置；(b) 对模 – 视变换矩阵进行设置之后

初次接触这么多个标架，读者可能搞不清哪些标架是固定的，哪些标架是可变的。因为模–视变换矩阵同时也确定了照相机相对于对象标架的位置和方向，所以把哪个标架看成是固定的通常可视其是否方便而定。大多数情况下把照相机看做是固定的，其他标架则相对于照相机移动，但是读者可能更愿意采用不同的观点。

在详细讨论变换以及在OpenGL中如何使用它们之前，我们先来看两个例子。在默认情况下，照相机标架和对象标架是重合的，并且照相机正对的方向是z轴的负方向，如图4.26(a)所示。在许多应用程序中，把对象指定在原点附近是很自然的，例如正方形的中心位于原点，或者一组对象的“质心”位于原点。我们通常把照相机设置成只能对它前面的对象成像，这个观察条件也是很自然的。因此，为了使生成的图像包含全部对象，必须让照相机远离对象，或者让对象远离照相机。等价地讲，我们可以相对于对象标架移动照相机标架。如果把照相机标架看成是固定的，并且在照相机标架下通过模–视变换矩阵来指定对象标架，那么下面的模–视变换矩阵

$$\mathbf{A}=\begin{bmatrix}1&0&0&0\\0&1&0&0\\0&0&1&-d\\0&0&0&1\end{bmatrix}$$

把对象标架下的点(x, y, z)变换成照相机标架下的点$(x, y, z-d)$。这样，世界标架可以相对于照相机标架移动，只要选择一个足够大的正数作为d，就可以把对象“移动”到照相机的前面，如图4.26(b)所示。注意，用户在建模时使用的是世界坐标系，并且仍然像以前那样摆放对象。对象标架和眼标架之间的关系包含在模–视变换矩阵中。为了把对象放到照相机的前面，也可以通过修改顶点来改变对象的位置，但是这一般不如设置模–视变换矩阵方便。

下面再来看一个直接和表示相关的例子。当通过顶点定义对象时，我们使用的是应用程序标架（世界标架）。在调用glVertex3f(x, y, z)确定顶点时使用的就是这个标架。因此，我们没有直接使用世界标架，而是隐含地使用了点（还有向量）在世界标架下的表示。请读者考虑图4.27所示的情形。

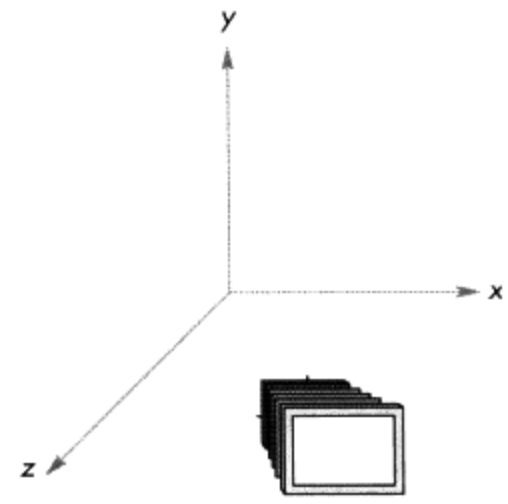


图4.27　照相机位于(1, 0, 1)，方向对准原点

图4.27给出了一个照相机相对于对象标架的位置和方向。如果在世界标架下使用齐次坐标，那么这个照相机的中心位于点$\mathbf{p}=(1, 0, 1, 1)^T$，并且它指向世界标架的原点。因此，在世界标架下表示为$\mathbf{n}=(-1, 0, -1, 0)^T$的向量垂直于照相机的背面并且指向原点。照相机的观察正向和世界坐标系y轴的正方向一样，都是$\mathbf{v}=(0, 1, 0, 0)^T$。我们可以利用叉积确定出第三个正交的方向$\mathbf{u}=(1, 0, -1, 0)^T$。有了这三个正交的方向和一个原点，就可以按照4.3.6节所讲的步骤为照相机建立一个标架。按照4.3.6节中使用的符号，$\mathbf{M}^T$中包含了照相机标架在世界标架下的齐次坐标表示。我们用下面的$(\mathbf{M}^T)^{-1}$就可以把点和向量在世界标架下的表示变换成在照相机标架下的表示。

$$(\mathbf{M}^T)^{-1} = \begin{bmatrix} 1 & 0 & -1 & 1 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & -1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}^{-1} = \begin{bmatrix} \frac{1}{2} & 0 & -\frac{1}{2} & 0 \\ 0 & 1 & 0 & 0 \\ -\frac{1}{2} & 0 & -\frac{1}{2} & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

注意，原来标架的原点现在位于照相机标架的n轴上，并且在n轴的正方向上与照相机标架的原点相距一个单位。换句话说，该点在照相机标架下的表示为(0, 0, 1, 1)。

在OpenGL中，可以通过向glLoadMatrix传递一个包含16个元素的数组来设置模－视变换矩阵。对于像上面这样的情形，我们是把一个标架的三个基向量和一个原点用另一个标架来表示，所以直接就可以确定出所需要的系数。然而，情况并不总是如此。对于大多数几何问题，通常是通过一系列像旋转、平移和缩放这样的几何变换实现从一个标架到另一个标架的变换。后面几节将采用这种方法，但下面还是先来看看构建几何对象的一些简单方法。

4.5　建模一个彩色立方体

我们已经学习了构建三维图形应用程序所需要的基本概念和实践知识。这一节将利用所学知识编写一个程序，这个程序可以绘制出旋转着的立方体。动画中的一帧图像如图4.28所示。不过，在让立方体旋转起来之前，我们先来考虑如何有效地对它进行建模。尽管可以像二维对象那样，用一组顶点来表示三维对象，但读者将会看到可以通过合适的数据结构来表示几何对象的顶点、边和面之间的关系。这一节的末尾将介绍顶点数组，它是OpenGL为了支持那些表示几何对象的数据结构而提供的一种工具。

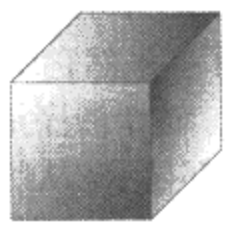


图4.28　立方体动画中的一帧图像

在建立了立方体的模型之后，可以通过仿射变换来生成动画。4.6节将介绍这些变换并用它们来改变OpenGL的模－视变换矩阵。第5章还要把这些变换作为观察过程的一部分。前面介绍的流水线模型可以很好地完成绘制任务。绘制流水线中的一些变换会应用到顶点上，所有这些变换都使用齐次坐标表示。在绘制流水线的末端是光栅化，目前可以假定光栅化这一步骤是自动完成的，只要已经正确执行了在此之前的步骤。

4.5.1　建模立方体的面

立方体是非常简单的三维对象。不过，建模它的方法不止一种。CSG系统会把立方体单独当做一种图元。在另一个极端，硬件把立方体看做是一个由8个顶点定义的对象。我们采用了基于表面的模型，这意味着把立方体看做是6个相交的平面或者6个多边形。这些多边形定义了立方体的面，它们称为面片（facet）。一种精心设计的数据结构应该既有利于立方体的高层表示，又便于底层硬件的高效实现。

假定立方体的顶点存储在一个数组中。例如，可以通过下面的代码来定义这个数组：

```
GLfloat vertices[8][3] =
   {{-1.0,-1.0,-1.0},{1.0,-1.0,-1.0},
   {1.0,1.0,-1.0}, {-1.0,1.0,-1.0}, {-1.0,-1.0,1.0},
   {1.0,-1.0,1.0}, {1.0,1.0,1.0}, {-1.0,1.0,1.0}};
```

如果先用下面的语句定义一个三维点类型，那么代码会更具有面向对象的风格：

```
typedef GLfloat point3[3];
```

立方体的顶点可以用下面的代码来指定：

```
point3 vertices[8] ={{-1.0,-1.0,-1.0},{1.0,-1.0,-1.0},
  {1.0,1.0,-1.0}, {-1.0,1.0,-1.0}, {-1.0,-1.0,1.0},
  {1.0,-1.0,1.0}, {1.0,1.0,1.0}, {-1.0,1.0,1.0}};
```

OpenGL在内部用四维齐次坐标来表示所有的顶点。对于使用三维表示的函数，例如glVertex3fv，图形系统在内部把这些表示转换成四维形式。

接下来可以用一组顶点来定义立方体的面。例如，下面的代码定义了立方体的一个面：

```
glBegin(GL_POLYGON);
  glVertex3fv(vertices[0]);
  glVertex3fv(vertices[3]);
  glVertex3fv(vertices[2]);
  glVertex3fv(vertices[1]);
glEnd();
```

与此类似，我们可以定义其余的面。注意我们已经按照和定义二维多边形完全一样的机制定义了三维多边形。

4.5.2　向内和向外的面

在定义一个三维多边形时，必须重视指定顶点的顺序。对第一个面，我们指定顶点的顺序是0、3、2、1。这个顺序和1、0、3、2是一样的，因为可以把多边形定义中的顶点序列看做是首尾相连的。然而，如果顺序是0、1、2、3就不同了。虽然它所描述的边界仍然相同，但是遍历多边形各边的顺序和0、3、2、1相反，如图4.29所示。这个顺序是重要的，因为每个多边形都有两个面。图形系统可以显示其中一个面，也可以两个面都显示。从照相机的角度来看，我们需要一种一致的方法来区别多边形的两个面。指定顶点的顺序提供了区别这两个面所需要的信息。

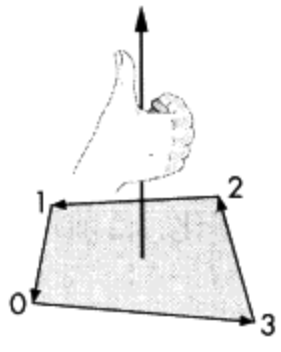


图4.29　遍历一个多边形的各条边

如果从外部观看一个对象的表面时其顶点是按照逆时针遍历的，则这个表面称做是向外的（outward facing）。这个方法也被称为右手规则（right-hand rule），因为如果让右手四指指向遍历顶点的方向，那么拇指指向对象表面的外部。

在例子中，按照0、3、2、1的顺序指定的是立方体向外的一个面，而0、1、2、3指定的是同一个多边形的背面。注意，对于像立方体这样的封闭对象，它的每一个面或者是向外的或者是向内的，而且不管在哪里观察，只要在这个对象的外面，那么对向内和向外的判断就是一致的。通过仔细指定正面和背面，我们可以去掉或者剔除（cull）不可见的面，或者使用不同的属性来显示正面和背面。第7章将进一步讨论隐藏面消除。

4.5.3　对象表示的数据结构

我们可以通过一组对顶点的指定来描述立方体。例如，可以调用 6 次

```
glBegin(GL_POLYGON)
```

每次调用之后都有 4 个顶点（通过 glVertex）和一个 glEnd，或者也可以用

```
glBegin(GL_QUADS)
```

后面跟 24 个顶点和一个 glEnd。这些方法都行得通，但它们都没有表示出立方体的拓扑（topology）性质，这和立方体的几何（geometry）性质是不同的。如果把立方体看做是多面体，那么它由 6 个面组成。这些面都是在顶点处相交的四边形，并且每个顶点由三个面共享。此外，顶点对定义了这些四边形的边，而且每条边由两个面共享。以上陈述描述了一个六面体的拓扑结构。这些陈述总是正确的，不管顶点的位置在哪里，即不管这个对象的几何性质如何[①]。

在本书的其余部分，我们会看到建立一种可以把对象的几何信息和拓扑信息分离开的数据结构有许多优点。本例使用一种称为顶点列表的数据结构，它既简单又有用，而且还可以扩展。

顶点的位置包含了对象的几何信息，这些数据可以存储在一个简单的列表或者数组里，例如 vertices[8]，这就是顶点列表（vertex list）。最顶层的实体是立方体，我们把它看做是由 6 个面组成。每个面包括 4 个有序顶点。每个顶点可以由其索引间接地指定。这个数据结构如图 4.30 所示，它的一个优点是：每个几何位置只出现一次，而不是在定义每个面片时重复出现。在一个交互式应用程序中，如果某个顶点的位置被用户改变，那么应用程序只需要把那个位置修改一次即可，无须搜索这个顶点在数据结构中的每一次出现。

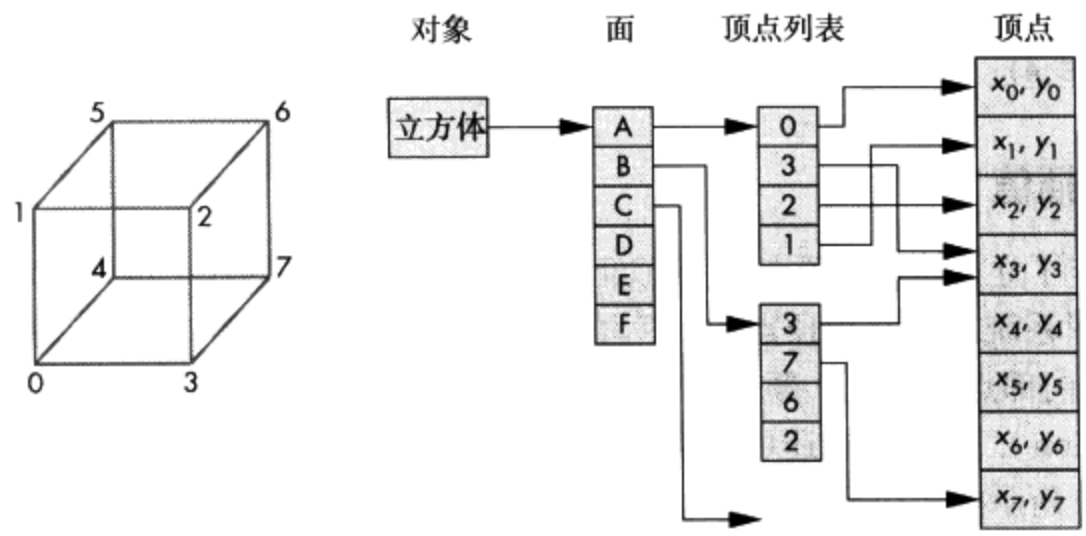


图 4.30　用顶点列表来表示立方体

4.5.4　彩色立方体

我们可以用顶点列表定义一个彩色立方体。第2章介绍了颜色立方体，其各个顶点的颜色分别是：黑、白、红、绿、蓝、青、品红、黄，我们就把这 8 种颜色指定为本例中彩色立方体各个顶点的颜色。我们定义一个函数 quad 来绘制四边形，这个函数的参数是指向顶点列表的指针。每个面的颜色由该面第一个顶点的索引来指定。最后，我们在函数 colorcube 中指定 6 个向外的面。建模彩色立方体的代码如下：

① 我们忽略了可能出现的特殊情形（奇性），例如，三个或者更多个顶点位于同一条直线或者顶点的位置使得所有的面都相交。

```
GLfloat vertices[8][3] = {{-1.0,-1.0,1.0},{-1.0,1.0,1.0},
   {1.0,1.0,1.0}, {1.0,-1.0,1.0}, {-1.0,-1.0,-1.0},
   {-1.0,1.0,-1.0}, {1.0,1.0,-1.0}, {1.0,-1.0,-1.0}};

GLfloat colors[8][3] = {{0.0,0.0,0.0},{1.0,0.0,0.0},
   {1.0,1.0,0.0}, {0.0,1.0,0.0}, {0.0,0.0,1.0},
   {1.0,0.0,1.0}, {1.0,1.0,1.0}, {0.0,1.0,1.0}};

void quad(int a, int b, int c, int d)
{
  glBegin(GL_QUADS);
     glColor3fv(colors[a]);
     glVertex3fv(vertices[a]);
     glColor3fv(colors[b]);
     glVertex3fv(vertices[b]);
     glColor3fv(colors[c]);
     glVertex3fv(vertices[c]);
     glColor3fv(colors[d]);
     glVertex3fv(vertices[d]);
  glEnd();
}

void colorcube()
{
   quad(0,3,2,1);
   quad(2,3,7,6);
   quad(0,4,7,3);
   quad(1,2,6,5);
   quad(4,5,6,7);
   quad(0,1,5,4);
}
```

4.5.5　双线性插值

虽然已经指定了立方体每个顶点的颜色，但图形系统还必须利用这个信息来确定多边形内部每个点的颜色。图形系统需要利用顶点的颜色填充多边形，或者说是利用顶点的颜色插值（interpolate）出多边形内部点的颜色。颜色插值有许多种方法，也许其中最重要的方法是双线性插值（bilinear interpolation），这种方法还可应用到其他场合。

考虑如图4.31所示的多边形。颜色C_0、C_1、C_2、C_3分别是应用程序指派给各个顶点的颜色。假定使用RGB颜色模式并且对每种原色分别进行插值。先在01和23这两条边上进行线性插值，于是这两条边上的颜色可以通过参数方程表示为：

$$C_{01}(\alpha) = (1-\alpha)C_0 + \alpha C_1$$

$$C_{23}(\alpha) = (1-\alpha)C_2 + \alpha C_3$$

当α从0变到1时，就得到了这两条边上所有点的颜色$C_{01}(\alpha)$和$C_{23}(\alpha)$。对一个给定的α，可以得到两条边上的两个颜色C_4和C_5。现在45这条线段两个端点的颜色都已求出，所以可以利用线性插值得到线段45上的颜色：

$$C_{45}(\beta) = (1-\beta)C_4 + \beta C_5$$

对于一个平面四边形，按照这种方法得到的每个颜色都对应于这个四边形中的一个点。如果4个顶点不在同一个平面上，虽然仍可以插值出一个颜色，但这个颜色在表面上的位置没有明确的定义。

为了避免出现这种情况，可以把四边形分成两个三角形，如图4.32所示，然后在两个三角形中插值颜色。这样，对于顶点颜色为 C_0、C_1 和 C_2 的三角形，我们仍然像前面那样先插值出 01 和 02 这两条边上的颜色：

$$C_{01}(\alpha) = (1-\alpha)C_0 + \alpha C_1$$

$$C_{02}(\alpha) = (1-\alpha)C_0 + \alpha C_2$$

由此计算出这两条边上任意两点的颜色 C_4 和 C_5，然后在这两个颜色之间进行线性插值就得出了三角形内部点的颜色。对另一个三角形也按照同样的方法插值颜色。

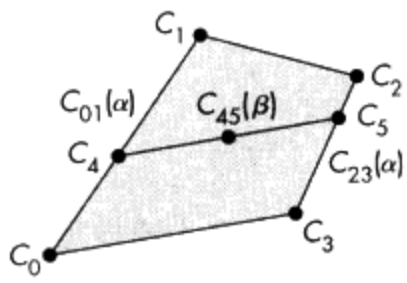


图 4.31　双线性插值

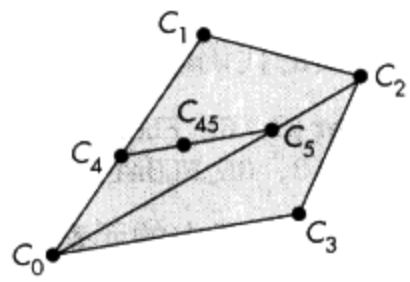


图 4.32　三角形内的双线性插值

在第 6 章和第 8 章将会看到，OpenGL 不只在插值颜色时使用了双线性插值，而且对其他可以逐个顶点指派的值也使用双线性插值[①]，不过插值公式的形式与上面讲的有些不同。

现在可以像在 2.9 节中对三维 Sierpinski 镂垫所做的那样，通过 glOrtho 提供的正交投影把彩色立方体显示出来。4.6 节介绍的变换可以使立方体动起来，还可以构造更复杂的对象。不过，现在先来介绍OpenGL的一个特性，它不仅减少了生成立方体的开销，还提供了一种处理立方体和其他多边体对象的高级方法。

4.5.6　顶点数组

虽然我们使用了顶点列表来建模立方体，但是立方体的绘制仍然需要调用很多次 OpenGL 函数。如果对每个顶点指派一种颜色，那么就需要 60 次函数调用：每个面需要调用 1 次 glBegin、1 次 glEnd、4 次 glColor 和 4 次 glVertex。每次函数调用都会增加开销，有的还需要传送数据。如果像第6章那样，对每个顶点指定一个不同的法向量，那么绘制立方体就需要更多次的函数调用。因此，尽管数据结构可以把立方体表示为一个三维几何对象，但绘制代码的执行速度可能不会很快。

顶点数组（vertex array）提供了一种方法来封装数据结构中的信息，使得我们只需要少数几次函数调用就可以绘制多面体对象。通过使用顶点数组，我们可以利用顶点定义一种数据结构并把该结构的信息传递给 OpenGL。当由这些数组定义的对象需要绘制时，可以通过少数几次函数调用让 OpenGL 遍历这个数据结构。

使用顶点数组包括三个步骤。第一步：开启顶点列表功能；第二步：把数组的位置和形式告诉 OpenGL。前两步可以作为初始化的一部分，而第三步通常是显示回调函数的一部分。下面通过立方体这个例子演示如何使用顶点数组。

OpenGL 支持许多不同类型的数组，包括顶点、颜色、颜色索引、法向量和纹理坐标，这些类型对应于在 glBegin 和 glEnd 之间可以定义的数据项。任何给定的应用程序通常只需要这些类型中的一部分。对于本例中的立方体，只需要颜色和顶点这两种类型。下面的代码开启对应的数组：

```
glEnableClientState(GL_COLOR_ARRAY);
glEnableClientState(GL_VERTEX_ARRAY);
```

① 现代图形卡可以支持在透视投影情形下给出正确结果的插值方法。

需要注意的是：与大多数OpenGL的状态信息不同，顶点数组的信息存放在客户端，而不是服务器端，所以在函数glEnableClientState的名称里包含客户（Client）这个单词。顶点数组和颜色数组同前面的一样并且可以设置成全局变量：

```
GLfloat vertices[8][3] = {{-1.0,-1.0,-1.0},{1.0,-1.0,-1.0},
{1.0,1.0,-1.0}, {-1.0,1.0,-1.0}, {-1.0,-1.0,1.0},
{1.0,-1.0,1.0}, {1.0,1.0,1.0}, {-1.0,1.0,1.0}};

GLfloat colors[8][3] = {{0.0,0.0,0.0},{1.0,0.0,0.0},
{1.0,1.0,0.0}, {0.0,1.0,0.0}, {0.0,0.0,1.0},
{1.0,0.0,1.0}, {1.0,1.0,1.0}, {0.0,1.0,1.0}};
```

接下来，我们用下面的代码确定数组的位置：

```
glVertexPointer(3, GL_FLOAT, 0, vertices);
glColorPointer(3, GL_FLOAT, 0, colors);
```

前三个参数的意义是：数组中的元素是三维的颜色和顶点，其数据类型为浮点型，并且它们在数组中是连续存储的。第4个参数是一个指针，它指向存储数据的数组。

下一步要做的是在数据结构中提供顶点和面之间的联系。为此，我们定义一个数组，其中包含了6个面的24个有序顶点索引：

```
GLubyte cubeIndices[24] = {0,3,2,1,     /* 面0 */
                           2,3,7,6,     /* 面1 */
                           0,4,7,3,     /* 面2 */
                           1,2,6,5,     /* 面3 */
                           4,5,6,7,     /* 面4 */
                           0,1,5,4};    /* 面5 */
```

这样，索引(0, 3, 2, 1)定义了第一个面，索引(2, 3, 7, 6)定义了第二个面，等等。注意，定义每个面的索引的顺序对应于向外的面。索引数组也可以定义成全局变量。

现在可以通过使用数组来绘制立方体了。当绘制数组中的元素时，所有开启的数组（在这里是颜色和顶点数组）都要被绘制。OpenGL提供了一些选项来指定绘制数组的方式。我们可以使用下面的函数：

```
glDrawElements(GLenum type, GLsizei n, GLenum format, void *pointer)
```

其中，type是数组定义的图元类型，比如线段或者多边形；n是想要绘制的元素个数；format指定了索引数组中数据的类型；pointer指向第一个索引。

为了绘制彩色立方体，应在显示回调函数里调用6次glDrawElements，每次绘制一个面：

```
for(i=0;i<6;i++)
   glDrawElements(GL_POLYGON, 4, GL_UNSIGNED_BYTE, &cubeIndices[4*i]);
```

这样，只要建立并初始化相应的数组，在每次绘制立方体时就只需要6次函数调用。后面我们会让立方体旋转起来，那时并不需要对顶点数组做改动，因为glDrawElements在绘制立方体时使用了当前的状态。

我们还可以做得更好。立方体的每个面都是四边形。因此，如果使用GL_QUADS而不是GL_POLYGON，就可以用单独一个函数调用绘制立方体：

```
glDrawElements(GL_QUADS, 24, GL_UNSIGNED_BYTE, cubeIndices);
```

因为GL_QUADS在每4个顶点之后开始下一个新的四边形。

顶点数组有两个重要的扩展，最新的软/硬件技术可以对它们提供支持。虽然像颜色这样的属性是 OpenGL 状态的一部分，但我们通常是逐个顶点地指定这些属性，所以在逻辑上它们似乎是顶点属性（vertex attribute）。顶点数组所支持的顶点属性（颜色、纹理坐标和法向量等）是图形系统提供的属性。另一方面，应用程序可能还有其他属性也是逐个顶点来指定的。例如，在与流体力学有关的应用程序中，我们可能希望每个顶点都有一个流速。第 9 章将介绍的可编程图形系统允许应用程序员定义自己的顶点属性，OpenGL 处理这些自定义属性的方式和处理标准属性是一样的。

顶点数组存储在客户端，从而可以最小化图形服务器所需的存储空间。因此，尽管顶点数组通过减少函数调用的次数提高了效率，每次绘制时它们仍旧需要在客户端和服务器端之间传递数据。既然图形硬件的存储量很大，为了尽可能提高效率，可以把顶点数据存储在顶点缓存对象（vertex buffer object）中。

4.6　仿射变换

变换是一个函数，它把一个点（或向量）映射成另一个点（或向量）。我们可以像图 4.33 那样直观地描绘出一个变换或者像下面这样把变换表示成函数的形式：

$$Q = T(P)$$

这是点的变换。而

$$v = R(u)$$

是向量的变换。如果使用齐次坐标，就能把向量和点都表示成四维列矩阵并且用一个函数来定义变换：

$$\mathbf{q} = f(\mathbf{p})$$
$$\mathbf{v} = f(\mathbf{u})$$

这是变换在一个给定标架下的表示。

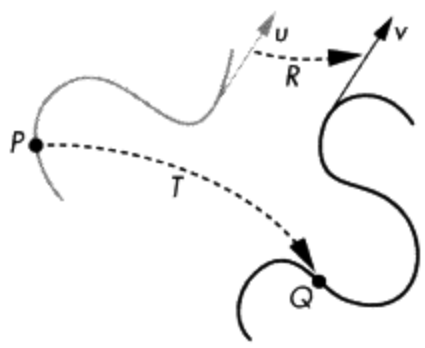


图 4.33　变换

上面对于变换的表述过于一般以至于不是很有用，因为它把所有的点和向量的单值映射都包括进去了。在实际应用中，即使函数 f 可以用一种方便的方式来描述，我们也得对一条曲线上的每一个点都执行变换。例如，如果要对一条线段执行一个一般的变换，那么在两个端点之间的所有点都要进行变换。

现在考虑一类受限制的变换。假定我们使用的是四维齐次坐标，那么点和向量都被表示成四元组[①]。如果对 f 加以限制，则可以得到一类有用的变换。最重要的限制是线性性。一个函数 f 是线性函数，当且仅当对于任何标量 α 和 β 以及任何两个顶点（或者向量）p 和 q，都有

① 我们只考虑这样的函数：它们把顶点映射成顶点，并且服从这一章和附录 B 讨论的向量和点的运算法则。

$$f(\alpha p + \beta q) = \alpha f(p) + \beta f(q)$$

线性函数的重要性在于：因为变换的线性组合等于线性组合的变换，所以只需要知道p和q的变换就可以求出它们的线性组合的变换，这样就无须直接计算每个线性组合的变换了。

如果使用点和向量的齐次坐标表示，那么可以把线性变换看成是在点（或者向量）的表示之间的映射，并且总可以用矩阵乘法来表示线性变换：

$$\mathbf{v} = \mathbf{C}\mathbf{u}$$

其中，$\mathbf{u}$和$\mathbf{v}$分别是变换前和变换后的表示，$\mathbf{C}$是一个方阵。把这个公式和4.3节讨论标架变换时得出的公式相比较，就会发现只要$\mathbf{C}$是非奇异的，那么这个线性变换就对应于一个标架变换。于是，我们可以用两种等价的观点看待线性变换：1）由于标架的改变而引起的顶点表示的改变；2）标架不变，对顶点进行变换。

当使用齐次坐标时，$\mathbf{C}$是一个4 × 4矩阵，它不会改变表示的第4个分量（w）。矩阵$\mathbf{C}$是4.3.4节中导出的矩阵$\mathbf{M}$的转置：

$$\mathbf{C} = \begin{bmatrix} \alpha_{11} & \alpha_{12} & \alpha_{13} & \alpha_{14} \\ \alpha_{21} & \alpha_{22} & \alpha_{23} & \alpha_{24} \\ \alpha_{31} & \alpha_{32} & \alpha_{33} & \alpha_{34} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

其中有12个元素可以任意取值，所以说这个变换有12个自由度（degree of freedom）。不过在仿射空间中，点和向量的齐次坐标表示稍有不同。任何向量都可以表示成：

$$\mathbf{u} = \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \alpha_3 \\ 0 \end{bmatrix}$$

任何点都可以表示为：

$$\mathbf{p} = \begin{bmatrix} \beta_1 \\ \beta_2 \\ \beta_3 \\ 1 \end{bmatrix}$$

如果把任意一个变换应用到一个向量：

$$\mathbf{v} = \mathbf{C}\mathbf{u}$$

那么可以看到$\mathbf{C}$中只有9个元素起作用，因此向量的变换只有9个自由度。点的仿射变换具有全部12个自由度。

还可以证明仿射变换把直线映射成直线。假定直线的方程为：

$$P(\alpha) = P_0 + \alpha d$$

其中，P_0是一个点，d是一个向量。在某个标架下，这条直线可以表示为：

$$\mathbf{p}(\alpha) = \mathbf{p}_0 + \alpha \mathbf{d}$$

其中，$\mathbf{p}_0$和$\mathbf{d}$分别是P_0和d在这个标架下的表示。对任意仿射变换矩阵$\mathbf{C}$，有

$$\mathbf{C}\mathbf{p}(\alpha) = \mathbf{C}\mathbf{p}_0 + \alpha \mathbf{C}\mathbf{d}$$

因此，我们可以先对 $\mathbf{p}_0$ 和 $\mathbf{d}$ 进行变换，然后使用任何一种绘制线段的方法就可以生成变换后的线段。如果使用直线方程的两点形式：

$$\mathbf{p}(\alpha) = \alpha\mathbf{p}_0 + (1-\alpha)\mathbf{p}_1$$

同样可以得出类似的结果。我们先对 $\mathbf{p}_0$ 和 $\mathbf{p}_1$ 进行变换，然后就可以绘制出变换后的线段。因为矩阵 $\mathbf{C}$ 中只有 12 个元素可以随意指定，所以对直线或者线段进行仿射变换也只有 12 个自由度。

虽然上面得到的结果是在抽象的数学空间中表述的，但这些结果在计算机图形学中的重要性体现在实际应用中。为了确定变换后的线段，只需要对两个端点的齐次坐标表示进行变换即可。因此，可以用流水线模型来实现图形系统，流水线中的仿射变换单元只需对端点进行变换，在光栅化阶段根据端点的处理结果就可以生成内部的点。

幸运的是，计算机图形学中用到大多数变换是仿射变换。这些变换包括旋转、平移和缩放。读者在第 5 章将会看到，稍做修改之后，就可以用这些结果来描述标准的平行和透视投影了。

4.7　平移、旋转和缩放

在前面的讨论中，我们有时把几何对象看成是抽象实体，有时又使用它们在某个给定标架下的表示。当编写应用程序时，必须使用表示。在这一节里，我们先给出几种最重要的仿射变换的表述，这些表述不依赖于任何特定的表示。然后，我们把这些变换应用到点和向量的表示上，由此求出这些变换的矩阵表述。4.8 节将介绍 OpenGL 如何实现这些变换。

变换可以看做是把点移动到新位置的某种方法，这些点定义了一个或者多个几何对象。尽管有许多变换可以把点移动到新的位置，但如果要求把一组点移动到新的位置并且保持它们之间的空间关系，那么绝大多数情况下只能找出一个变换。因此，尽管可以找出许多变换矩阵把彩色立方体的一个顶点从 P_0 移动到 Q_0，但是在它们中间只有一个能够保持立方体的大小和方向。

4.7.1　平移

平移（translation）变换把点沿着给定的方向移动固定的距离，如图 4.34 所示。我们只需要指定一个位移向量 d 就可以确定一个平移变换，因为变换后的点由下式给出：

$$P' = P + d$$

上式适用于对象上所有的点。注意，这个定义不依赖于任何标架或者表示。平移有三个自由度，因为它的位移向量有三个可以任意指定的分量。

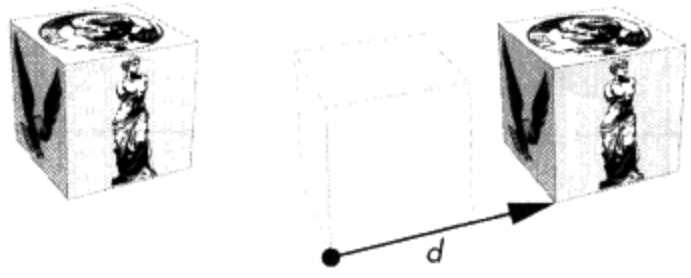


图 4.34　平移：(a) 在初始位置的对象；(b) 平移后的对象

4.7.2　旋转

确定旋转（rotation）比确定平移更困难一些，因为需要指定的参数更多。先来考虑一个简单的例子，如图 4.35 所示，我们在二维平面内围绕原点旋转一个点。因为已经指定了一个特定的点

（原点），所以有一个特定的标架。在这个标架下表示为(x, y)的点围绕原点旋转θ后位于表示为(x', y')的位置。通过把(x, y)和(x', y')用极坐标表示，可以得到描述这个旋转的标准方程：

$$x = \rho \cos \phi$$

$$y = \rho \sin \phi$$

$$x' = \rho \cos(\theta + \phi)$$

$$y' = \rho \sin(\theta + \phi)$$

利用两角和的正弦和余弦公式，得到：

$$x' = \rho \cos \phi \cos \theta - \rho \sin \phi \sin \theta = x \cos \theta - y \sin \theta$$

$$y' = \rho \cos \phi \sin \theta + \rho \sin \phi \cos \theta = x \sin \theta + y \cos \theta$$

上式可以写成矩阵的形式：

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

4.8节会把上式推广到三维空间。

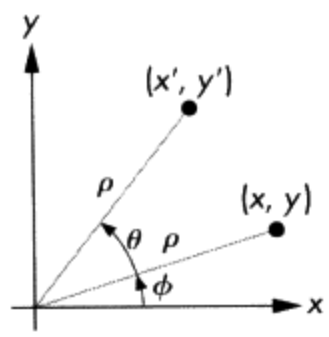


图4.35　二维旋转

需要注意的是：这个旋转有三个特征可以推广到其他旋转。

1. 存在一个点（本例中是原点）在旋转变换下保持不变。我们把这个点称为变换的不动点（fixed point）。图4.36所示是一个旋转变换，它的不动点不在原点，而在对象的中心。

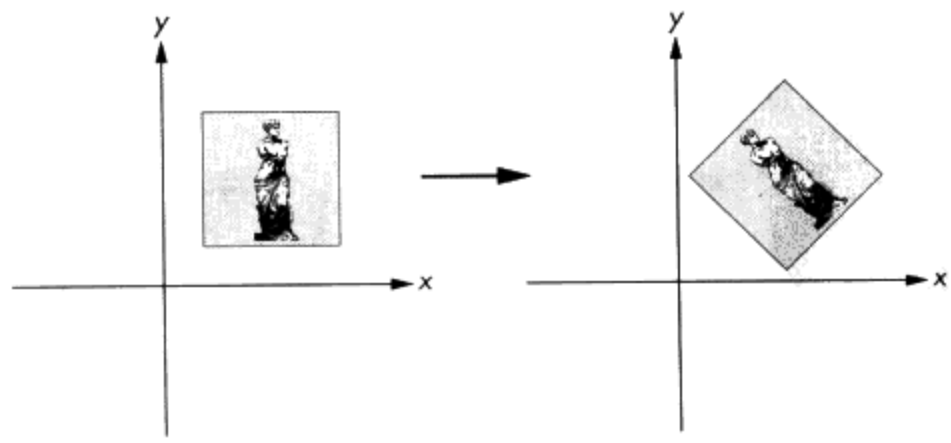


图4.36　旋转的不动点不在原点

2. 考虑到二维平面是三维空间的一部分，可以对三维的情况重新解释这个旋转。对于右手坐标系，如果把x轴和y轴按照标准方式画在纸面上，那么z轴的正方向指向纸面外。当逆着z轴的正方向朝原点看时，旋转的正方向应为逆时针方向。绕其他轴旋转的正方向也是这样定义的。

3. 在二维平面 $z=0$ 内的旋转等价于围绕 z 轴的三维旋转。z 坐标等于常数的平面上的点都以类似方式旋转，它们的 z 坐标不变。

利用上述分析可以定义一般的三维旋转，并且这个定义是不依赖于标架的。我们必须指定如图 4.37 所示的三个实体：不动点（P_f）、旋转角度（θ）和代表旋转轴的直线或者向量。如果不动点是给定的，那么旋转变换有三个自由度：两个角度用来确定一个向量的方向，第三个角度是绕这个向量旋转的角度。

旋转和平移都是刚体变换（rigid-body transformation）。旋转和平移的任何组合都不能改变对象的形状和体积，它们只能改变对象的位置和方向。因此，单靠旋转和平移不能给出所有可能的仿射变换。图 4.38 所示的变换是仿射变换，但不是刚体变换。

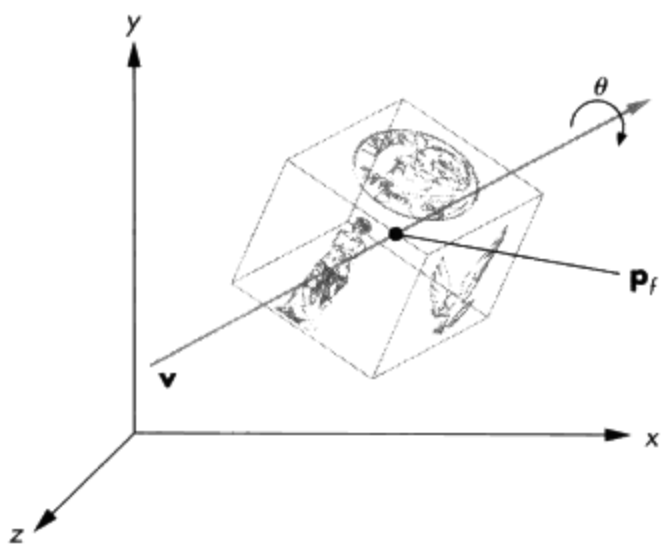


图 4.37　三维旋转

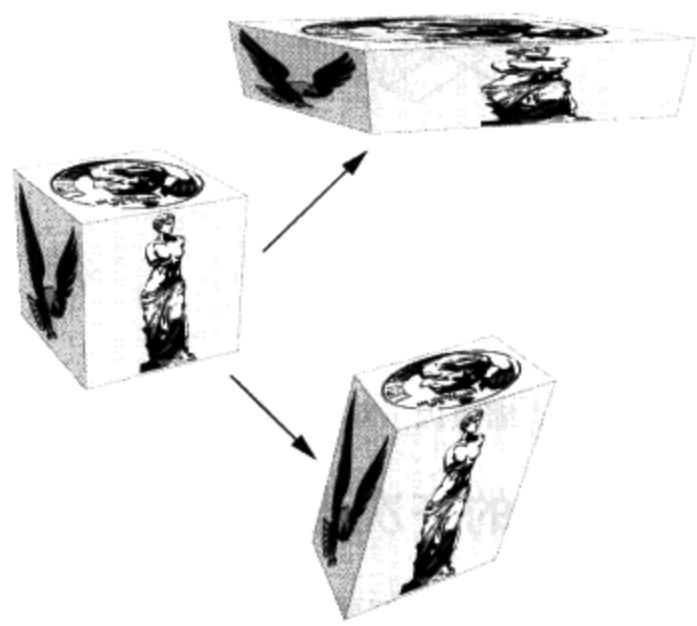


图 4.38　非刚体变换

4.7.3　缩放

缩放是一种仿射变换，但不是刚体变换，通过缩放变换可以放大或者缩小对象。图 4.39 所示是均匀缩放和非均匀缩放。为了构造出在建模和观察中使用的全部仿射变换，需要把一系列合适的缩放、平移和旋转变换组合起来，其中的缩放不一定是均匀的，所以有必要考虑非均匀缩放。

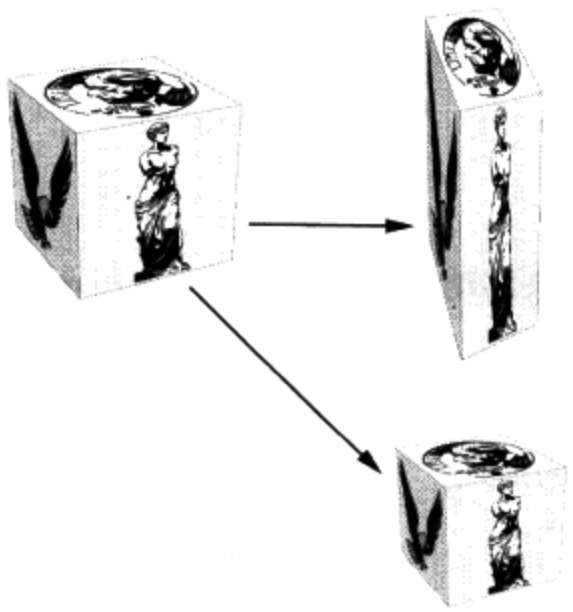


图 4.39　均匀缩放和非均匀缩放

从图4.40可以看到，缩放变换有一个不动点。因此，为了确定一个缩放变换，应该指定不动点、缩放的方向和缩放因子（α）。当$\alpha>1$时，对象沿着指定的方向伸长；当$0 \leqslant \alpha<1$时，对象沿着指定的方向缩短。如果α是负的（见图4.41），就得到了以不动点为中心沿缩放方向的反射（reflection）变换。缩放有6个自由度，因为可以任意指定不动点，此外还有三个独立的缩放因子。

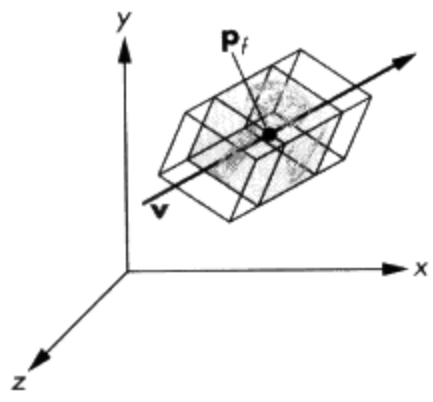


图4.40　缩放因子对缩放的影响

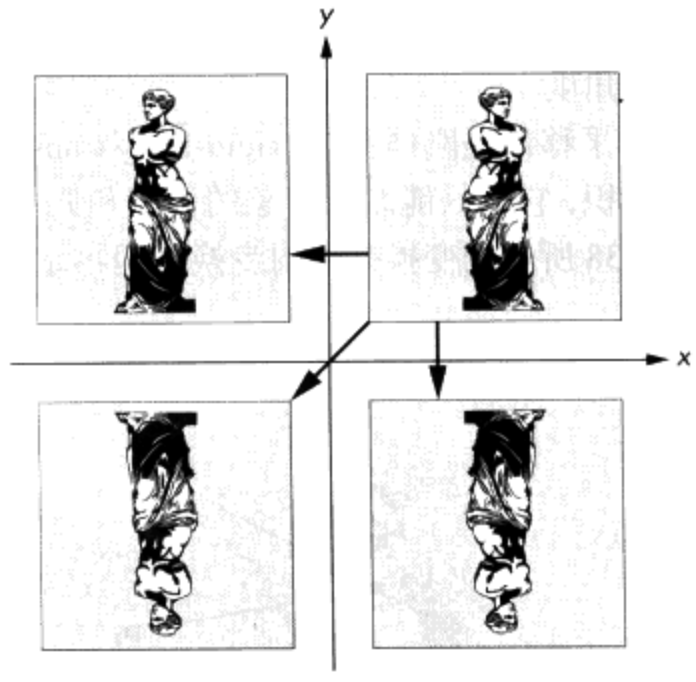


图4.41　反射

4.8　变换的齐次坐标表示

所有的图形API都要求使用参考系。因此，不能直接应用类似下面这样的高级表示：

$$Q = P + \alpha v$$

我们应该使用齐次坐标表示和类似下面这样的式子：

$$\mathbf{q} = \mathbf{p} + \alpha \mathbf{v}$$

在某个标架下，每个仿射变换都可以由如下形式的4×4矩阵来表示：

$$\mathbf{A} = \begin{bmatrix} \alpha_{11} & \alpha_{12} & \alpha_{13} & \alpha_{14} \\ \alpha_{21} & \alpha_{22} & \alpha_{23} & \alpha_{24} \\ \alpha_{31} & \alpha_{32} & \alpha_{33} & \alpha_{34} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4.8.1　平移

平移变换按照一个位移向量来移动点的位置。如果按照$\mathbf{d}$所指示的方向和距离把点$\mathbf{p}$移动到$\mathbf{p}'$，那么

$$\mathbf{p}' = \mathbf{p} + \mathbf{d}$$

把$\mathbf{p}$、$\mathbf{p}'$和$\mathbf{d}$写成齐次坐标的形式：

$$\mathbf{p} = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}, \qquad \mathbf{p}' = \begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix}, \qquad \mathbf{d} = \begin{bmatrix} \alpha_x \\ \alpha_y \\ \alpha_z \\ 0 \end{bmatrix}$$

于是可以采用分量形式表示平移变换：

$$x' = x + \alpha_x$$

$$y' = y + \alpha_y$$

$$z' = z + \alpha_z$$

它们可以写成列矩阵相加的形式，但这种表示不能很好地和其他仿射变换的表示相结合。不过用矩阵乘法也可以表示同样的结果：

$$\mathbf{p}' = \mathbf{T}\mathbf{p}$$

其中，

$$\mathbf{T} = \begin{bmatrix} 1 & 0 & 0 & \alpha_x \\ 0 & 1 & 0 & \alpha_y \\ 0 & 0 & 1 & \alpha_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

T 称为平移矩阵（translation matrix），有时把它写成 $\mathbf{T}(\alpha_x, \alpha_y, \alpha_z)$来强调三个独立的参数。

在点的齐次坐标表示中，第4个元素总是1，读者也许会觉得没有必要使用这个总是取固定值的元素。然而，如果使用三维表示：

$$\mathbf{q} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}, \qquad \mathbf{q}' = \begin{bmatrix} x' \\ y' \\ z' \end{bmatrix}$$

那么就不可能找出一个 3 × 3 矩阵 **D** 使得 $\mathbf{q}' = \mathbf{D}\mathbf{q}$ 能够表示平移向量为 **d** 的平移变换。由于这个原因，齐次坐标通常被看做是一种技巧，利用它可以把三维空间中的列矩阵相加转换成四维空间中的矩阵相乘。

平移矩阵的逆可以通过某种矩阵求逆的算法得出。因为把一个点平移 d 之后再平移 $-d$ 就回到了原来的位置，所以还可以用另一种方法求出平移矩阵的逆。这两种方法都得出了下面的逆矩阵：

$$\mathbf{T}^{-1}(\alpha_x, \alpha_y, \alpha_z) = \mathbf{T}(-\alpha_x, -\alpha_y, -\alpha_z) = \begin{bmatrix} 1 & 0 & 0 & -\alpha_x \\ 0 & 1 & 0 & -\alpha_y \\ 0 & 0 & 1 & -\alpha_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4.8.2　缩放

缩放和旋转都有一个不动点，这个点变换以后保持不变。现在设不动点为原点，以后可以通过变换的复合得到不动点在任意位置的变换。

不动点位于原点的缩放变换可以沿着三个坐标轴的方向进行彼此独立的缩放。这三个方向的缩放可以表示为：

$$x' = \beta_x x$$

$$y' = \beta_y y$$

$$z' = \beta_z z$$

利用齐次坐标，这三个方程可以组合在一起：

$$\mathbf{p}' = \mathbf{S}\mathbf{p}$$

其中，

$$\mathbf{S} = \mathbf{S}(\beta_x, \beta_y, \beta_z) = \begin{bmatrix} \beta_x & 0 & 0 & 0 \\ 0 & \beta_y & 0 & 0 \\ 0 & 0 & \beta_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

和平移矩阵一样，上面这个矩阵的最后一行与具体变换无关，只是使变换后表示的第4个分量仍然是1。

缩放矩阵的逆仍然是一个缩放矩阵，并且这两个矩阵的对应缩放系数互为倒数：

$$\mathbf{S}^{-1}(\beta_x, \beta_y, \beta_z) = \mathbf{S}\left(\frac{1}{\beta_x}, \frac{1}{\beta_y}, \frac{1}{\beta_z}\right)$$

4.8.3　旋转

我们先讨论不动点为原点的旋转，这样的旋转有三个自由度，分别对应于相互独立的绕着三个坐标轴的旋转。必须注意：矩阵乘法不满足交换律（见附录C）。先绕x轴旋转角度θ然后再绕y轴旋转角度ϕ，和先绕y轴旋转角度ϕ然后再绕x轴旋转角度θ是不同的变换，尽管只是交换了旋转的顺序。

利用4.7.2节讨论二维旋转得出的结果，可以直接得出绕坐标轴旋转的变换矩阵。在4.7.2节我们看到二维旋转实际上是在三维空间中绕z轴的旋转，而且绕z轴旋转之后z为常数的平面保持不变。因此，在三维空间中，绕z轴旋转角度θ可以用下列方程来表示：

$$\begin{aligned} x' &= x\cos\theta - y\sin\theta \\ y' &= x\sin\theta + y\cos\theta \\ z' &= z \end{aligned}$$

或者写成矩阵形式：

$$\mathbf{p}' = \mathbf{R}_z\mathbf{p}$$

其中，

$$\mathbf{R}_z = \mathbf{R}_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

利用同样的推理可以导出绕x轴和绕y轴旋转的矩阵。如果绕x轴旋转，那么x坐标不会改变，并且在每个x等于常数的平面内进行二维旋转。如果绕y轴旋转，那么y坐标不变。这两个变换矩阵为：

$$\mathbf{R}_x = \mathbf{R}_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\mathbf{R}_y = \mathbf{R}_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

其中正弦项的符号与在右手系下对旋转正方向的定义一致。

令 $\mathbf{R}$ 表示上面这三个旋转矩阵中的任何一个。先旋转角度 θ 然后再旋转角度 $-\theta$ 等于没有进行变换，于是，

$$\mathbf{R}^{-1}(\theta) = \mathbf{R}(-\theta)$$

此外，我们注意到所有的余弦项都出现在对角线上，而正弦项不在对角线上出现，于是利用下面的三角恒等式

$$\cos(-\theta) = \cos\theta$$
$$\sin(-\theta) = -\sin\theta$$

可得：

$$\mathbf{R}^{-1}(\theta) = \mathbf{R}^T(\theta)$$

任何不动点在原点的旋转矩阵都可以表示成绕三个坐标轴的旋转矩阵的乘积：

$$\mathbf{R} = \mathbf{R}_z\mathbf{R}_y\mathbf{R}_x$$

4.9.2节给出了这个结果的一个简单证明。因为矩阵乘积的转置等于转置矩阵的反序乘积，所以对任何旋转矩阵都有：

$$\mathbf{R}^{-1} = \mathbf{R}^T$$

如果一个矩阵的逆等于它的转置，则称它为正交矩阵（orthogonal matrix）。归一化的正交矩阵对应于不动点在原点的旋转。

4.8.4　剪切

虽然利用一系列的旋转、平移和缩放变换就可以构造出任何仿射变换，但是还有一类仿射变换如此重要以至于人们把它看做是一种基本的变换，而不是从其他变换推导出来的，这就是剪切（shear）变换。如图 4.42 所示，从 z 轴正向一侧观察一个中心位于原点的立方体，这个立方体的各条棱分别平行于三个坐标轴。如果向右拉立方体的顶面并且向左拉底面，则把立方体沿 x 方向进行了剪切。注意，y 坐标和 z 坐标都保持不变，因此为了和其他方向的剪切相区别，我们把这个变换称为 x 剪切。由图 4.43 可知，用一个角度 θ 就可以表征一个 x 剪切。这个剪切可用下面的方程来描述：

$$x' = x + y\cot\theta$$
$$y' = y$$
$$z' = z$$

由此可得剪切矩阵：

$$\mathbf{H}_x(\theta) = \begin{bmatrix} 1 & \cot\theta & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

为了抵消掉$\mathbf{H}_x(\theta)$的效果，只需以相同的角度向相反的方向剪切，因此，

$$\mathbf{H}_x^{-1}(\theta) = \mathbf{H}_x(-\theta)$$

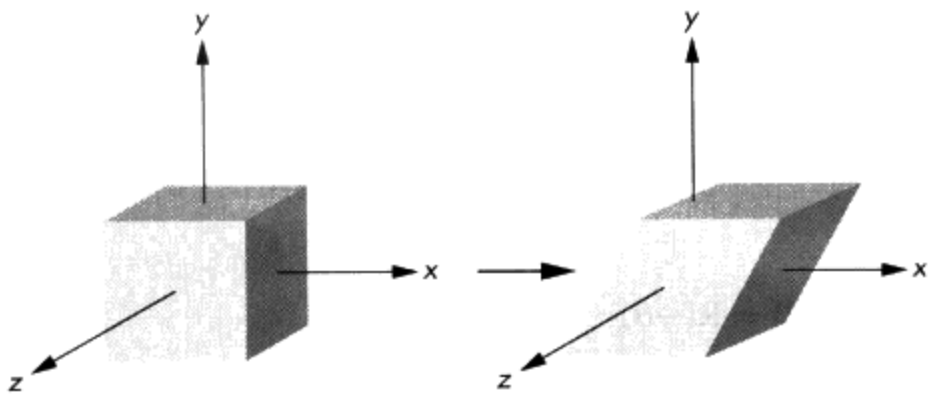


图 4.42　剪切

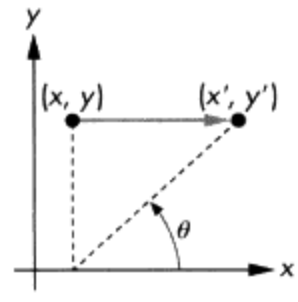


图 4.43　剪切矩阵的推导

4.9　变换的级联

前面介绍了几种基本的变换，这一节将通过相乘，或者称为级联（concatenate）一系列基本变换来构造几个仿射变换的例子。这种方法比直接定义一个任意变换更受欢迎，而且与图形系统的流水线结构非常吻合。

假定对点**p**相继执行三个变换，得到了点**q**。因为矩阵乘法满足结合律，所以可以把这个变换序列表示为：

$$\mathbf{q} = \mathbf{CBAp}$$

注意，上式中没有括号。这里的矩阵**A**、**B**和**C**可以是任意的4 × 4矩阵，尽管在实际应用中它们往往是仿射变换的矩阵。变换的顺序会影响计算的效率。如图4.44所示，一种办法是先执行变换**A**，然后执行**B**，最后执行**C**，这样的运算顺序对应于下式中加括号的方式：

$$\mathbf{q} = (\mathbf{C}(\mathbf{B}(\mathbf{Ap})))$$

如果只变换一个点，那么这个运算顺序最有效，因为每次矩阵相乘只需用列矩阵乘以方阵。如果要对许多点进行变换，那么可以分两步来进行。首先计算：

$$\mathbf{M} = \mathbf{CBA}$$

然后再用这个矩阵计算每个点的变换：

$$\mathbf{q} = \mathbf{Mp}$$

这样的运算顺序对应于图4.45所示的流水线。我们先计算出矩阵**M**，然后把它加载到一个流水线变换单元。简单地考虑一下运算的次数就会发现，尽管初始时要计算**M**，但在此之后对每个点只需进行一次矩阵乘法，所以和由此而节省的运算次数相比，计算**M**的额外开销可以忽略，因为可能要处理成千上万个点。下面举几个计算**M**的例子。

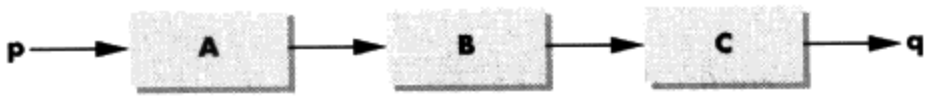


图 4.44　每次执行一个变换

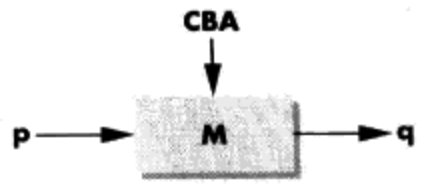


图 4.45　流水线变换

4.9.1　不动点在任意位置的旋转

我们已经知道了不动点在原点的变换矩阵（包括旋转、缩放和剪切），现在要对它们进行修改以得到不动点在任意位置的变换。我们以绕平行于z轴的直线旋转为例，所用的方法也适用于其他情况。

考虑一个中心在$\mathbf{p}_f$的立方体，它的各个面分别和三个坐标轴平行。我们要把这个立方体绕平行于z轴的一条直线旋转，并且立方体的中心是旋转的不动点，如图4.46所示。如果$\mathbf{p}_f$是原点，我们已经知道变换的矩阵是$\mathbf{R}_z(\theta)$，由此想到首先应该把立方体的中心移到原点，然后就可以应用$\mathbf{R}_z(\theta)$，最后再把对象移回原处，使得它的中心仍旧位于$\mathbf{p}_f$。这个变换序列如图4.47所示。如果用前面提到的基本仿射变换的符号来表示，那么第一个变换是$\mathbf{T}(-\mathbf{p}_f)$，第二个变换是$\mathbf{R}_z(\theta)$，最后一个变换是$\mathbf{T}(\mathbf{p}_f)$。把它们级联到一起就得到单独一个矩阵：

$$\mathbf{M} = \mathbf{T}(\mathbf{p}_f)\mathbf{R}_z(\theta)\mathbf{T}(-\mathbf{p}_f)$$

乘完之后的结果是：

$$\mathbf{M} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & x_f - x_f\cos\theta + y_f\sin\theta \\ \sin\theta & \cos\theta & 0 & y_f - x_f\sin\theta - y_f\cos\theta \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

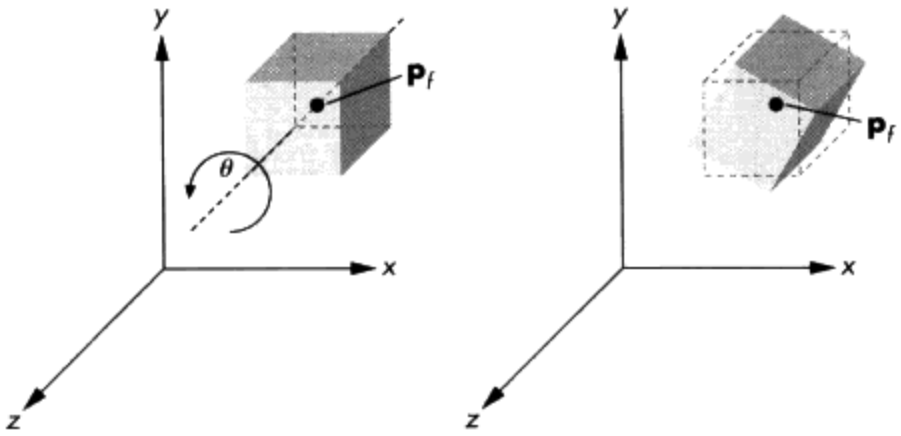


图 4.46　立方体绕通过其中心的直线旋转

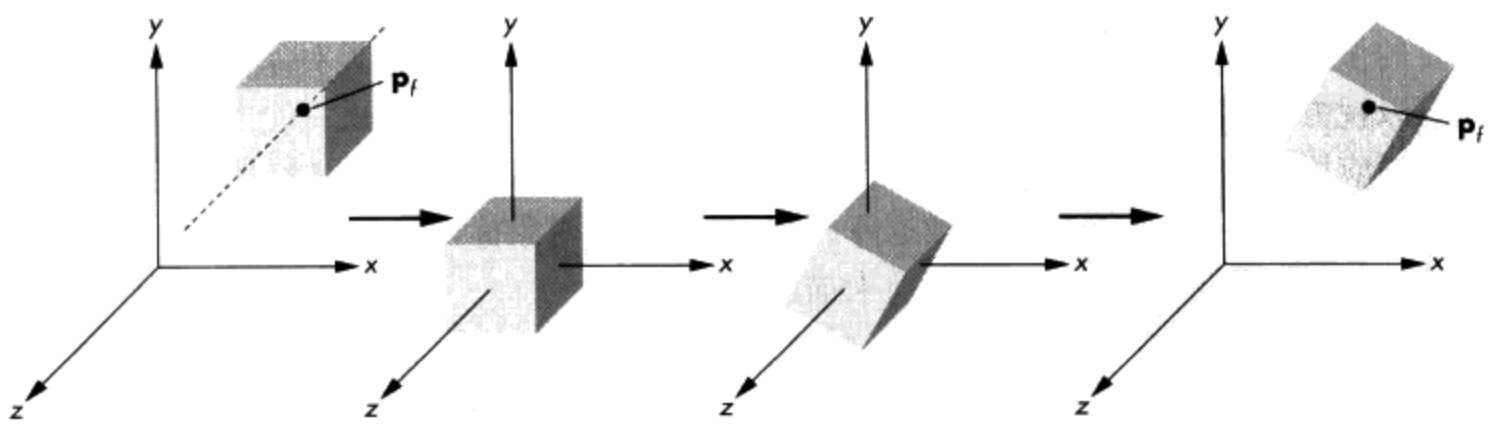


图 4.47　变换序列

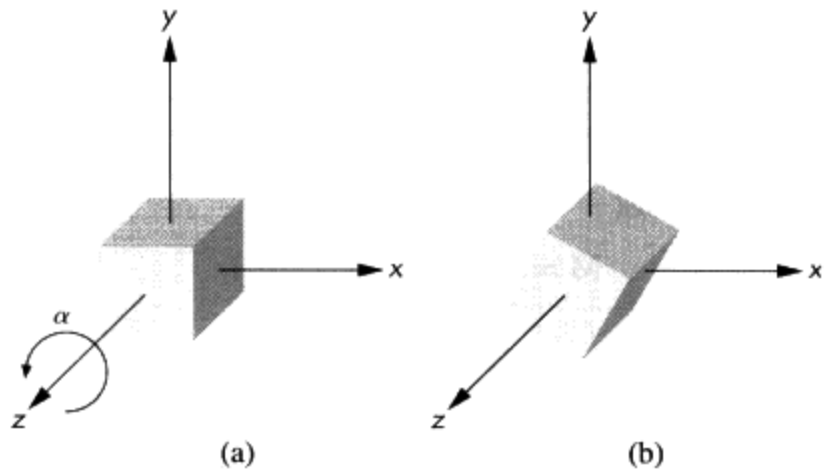
4.9.2　一般的旋转

现在证明任何一个以原点为不动点的旋转都可以分解成三个绕坐标轴的旋转。虽然最后得到的旋转矩阵是相同的，但绕三个坐标轴旋转的顺序不是唯一的（见习题4.10）。这里先绕z轴旋转，然后绕y轴旋转，最后绕x轴旋转。

考虑如图4.48(a)所示的立方体，它的中心位于原点并且各个面分别平行于三个坐标轴。我们把这个立方体绕 z 轴旋转角度 α，如图 4.48(b)所示。然后再把立方体绕 y 轴旋转角度 β，图 4.49 所示为俯视图。最后再把立方体绕 x 轴旋转角度 γ，图 4.50 所示为侧视图。总的旋转矩阵为：

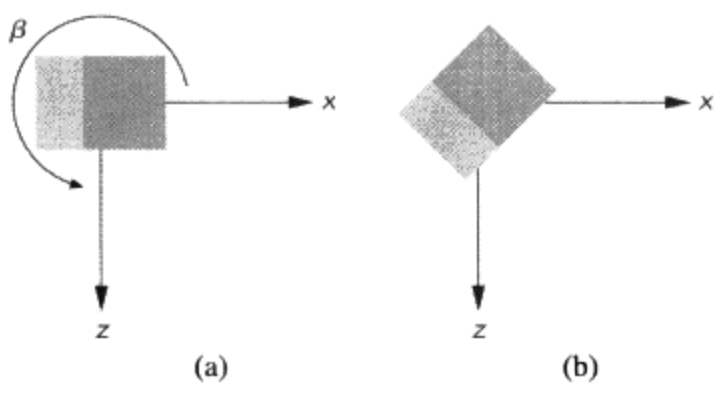
$$\mathbf{R} = \mathbf{R}_x\mathbf{R}_y\mathbf{R}_z$$

读者自己试验一下就会确信，总可以找到合适的 α、β 和 γ 使得立方体在经过三次旋转之后具有任何想要的方向，尽管在 4.9.4 节的例子中会看到求出这三个角度需要一定的技巧。



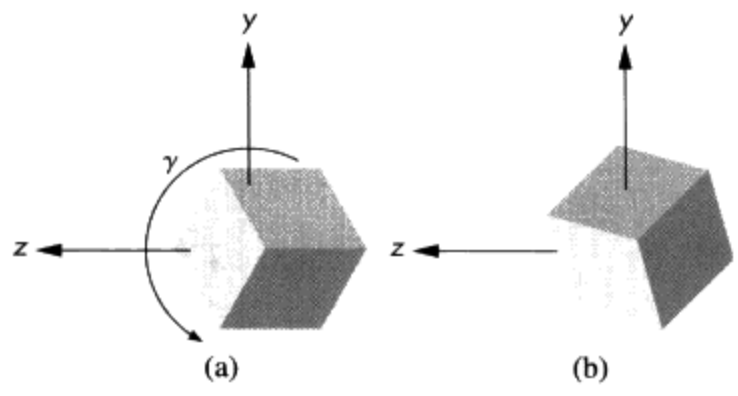
(a)　　(b)

图 4.48　绕 z 轴旋转立方体：(a) 旋转之前；(b) 旋转之后



(a)　　(b)

图 4.49　绕 y 轴旋转立方体：(a) 旋转之前；(b) 旋转之后



(a)　　(b)

图 4.50　绕 x 轴旋转立方体：(a) 旋转之前；(b) 旋转之后

4.9.3　实例变换

从前面把立方体旋转到任意方向的例子可以引申出一种建模方法。考虑一个由许多简单对象构成的场景，例如图4.51所示的场景。该场景建模的一种方法是通过指定对象的顶点使得每个对象出现在想要的位置并且具有期望的方向和大小。另一种方法是一次定义一类对象，对象的大小、位置和方向可以视方便而定。对象在场景中的每一次出现都是这种对象原型的一个实例（instance）。我们可以通过应用一个仿射变换，即实例变换（instance transformation）使得对象具有期望的大小、方向和位置。可以构建一个简单的对象数据库，然后通过一个对象标识符的列表（例如1代表立方体，2 代表球）和应用到每个对象的实例变换来描述场景。

应用实例变换的顺序如图 4.52 所示。对象通常在它们自己的标架下定义，这个标架的原点位于对象的质心并且对象的各面分别和对应的坐标轴平行。首先，缩放对象使其具有期望的大小；然后，应用一个旋转矩阵使对象具有正确的方向；最后，把对象平移到想要的位置。因此，实例变换的形式为：

$$\mathbf{M} = \mathbf{TRS}$$

通过实例变换来建模对象不但与流水线体系结构非常吻合，而且还可以和第3章介绍的显示列表很好地结合起来。一个需要使用多次的复杂对象可以作为显示列表一次加载到服务器中。为了显示它的每一个实例，只需要在执行显示列表之前把合适的实例变换发送到服务器即可。

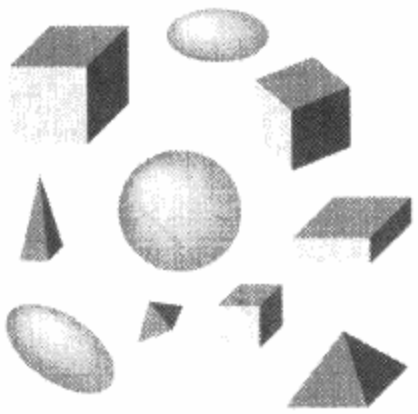


图 4.51　包含许多简单对象的场景

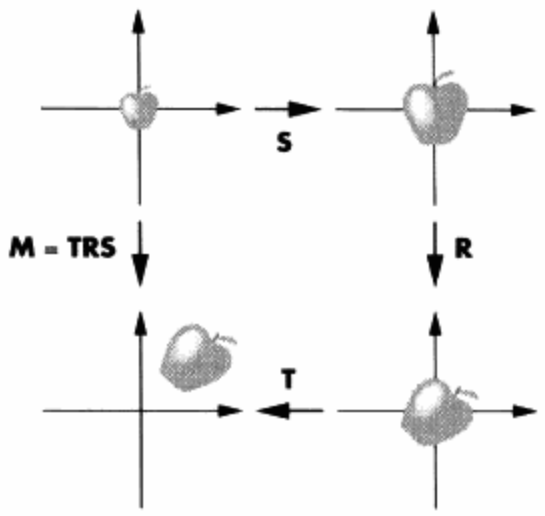


图 4.52　实例变换

4.9.4　绕任意轴的旋转

现在来看最后一个旋转的例子，我们将导出绕空间中任意一条直线旋转的变换矩阵，而且还要说明如何利用方向角确定方向。考虑旋转如图4.53所示的立方体，我们需要三个实体来指定这个旋转，这三个实体是：不动点 $\mathbf{p}_0$（假定不动点是立方体的中心），用来指定旋转轴方向的一个向量，还有旋转的角度。注意，这些实体都不依赖于标架，所以指定旋转的方式与坐标无关。然而，为了求出表示这个变换的仿射矩阵，必须使用某个标架。

有许多方法可以指定旋转轴的方向向量。一种方法是用两个点 $\mathbf{p}_1$ 和 $\mathbf{p}_2$ 来定义这个向量：

$$\mathbf{u} = \mathbf{p}_2 - \mathbf{p}_1$$

注意，这两个点的顺序决定了旋转角度 θ 的正方向，而且即使用一条通过点 $\mathbf{p}_0$ 的有向线段表示 $\mathbf{u}$，所起作用的也只是 $\mathbf{u}$ 的方向。用一个归一化的向量

$$\mathbf{v} = \frac{\mathbf{u}}{|\mathbf{u}|} = \begin{bmatrix} \alpha_x \\ \alpha_y \\ \alpha_z \end{bmatrix}$$

来代替 $\mathbf{u}$ 可以简化后面的步骤。我们已经看到了把不动点移到原点有助于求解问题。因此，第一个变换是平移 $\mathbf{T}(-\mathbf{p}_0)$，最后一个变换是 $\mathbf{T}(\mathbf{p}_0)$。在经过平移 $\mathbf{T}(-\mathbf{p}_0)$之后，旋转轴将通过原点，如图 4.54 所示。我们在上一个例子（见 4.9.2 节）中证明了任意一个旋转都可以通过三次绕坐标轴的旋转来合成。现在的问题更困难，因为我们不知道每次旋转的角度。这里采用的办法是通过两次旋转使得旋转轴 $\mathbf{v}$ 和 z 轴重合。然后就可以绕 z 轴旋转角度 θ，此后再把那两次使得旋转轴和 z 轴重合的旋转抵消掉。因此，我们要求的旋转矩阵可以写成下面的形式：

$$\mathbf{R} = \mathbf{R}_x(-\theta_x)\mathbf{R}_y(-\theta_y)\mathbf{R}_z(\theta)\mathbf{R}_y(\theta_y)\mathbf{R}_x(\theta_x)$$

这个旋转序列如图 4.55 所示。在求解 $\mathbf{R}$ 的过程中最困难的部分是如何确定 θ_x 和 θ_y。

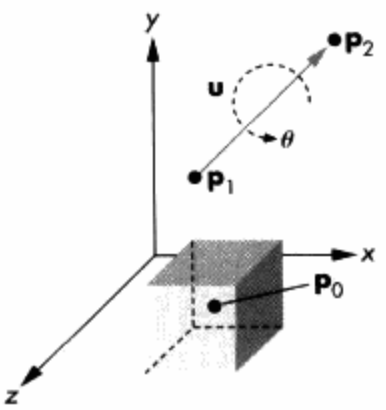


图 4.53　绕任意轴旋转立方体

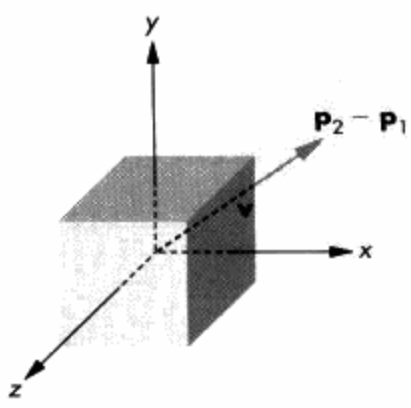


图 4.54　不动点移到原点

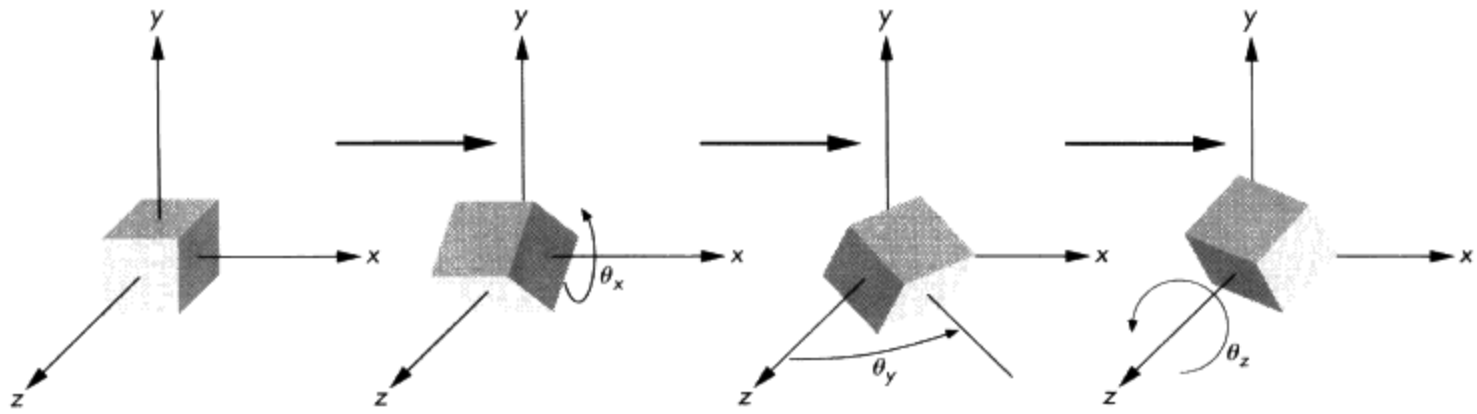


图 4.55　旋转序列

我们先从 **v** 的分量入手。因为 **v** 是一个单位向量，所以，

$$\alpha_x^2 + \alpha_y^2 + \alpha_z^2 = 1$$

从原点到点$(\alpha_x, \alpha_y, \alpha_z)$画一条有向线段，这条线段的长度为 1 并且方向和 **v** 相同。然后过点$(\alpha_x, \alpha_y, \alpha_z)$作三个坐标轴的垂线，如图 4.56 所示。这条有向线段（或者 **v**）和三个坐标轴之间的夹角 ϕ_x、ϕ_y、ϕ_z 就是方向角（direction angle）。下面的式子定义了方向余弦（direction cosine）：

$$\cos\phi_x = \alpha_x$$
$$\cos\phi_y = \alpha_y$$
$$\cos\phi_z = \alpha_z$$

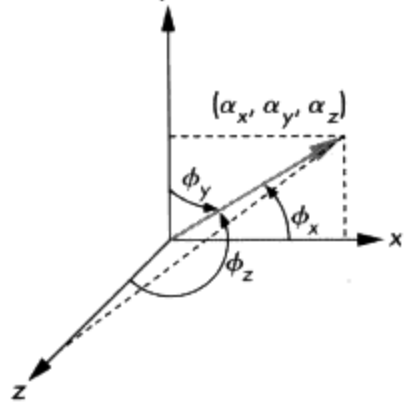


图 4.56　方向角

这三个方向角中只有两个是独立的，因为

$$\cos^2\phi_x + \cos^2\phi_y + \cos^2\phi_z = 1$$

利用这些角度就可以计算出 θ_x 和 θ_y。如图 4.57 所示，绕 x 轴旋转角度 θ_x 的结果是使连接原点和点$(\alpha_x, \alpha_y, \alpha_z)$的线段位于平面 $y = 0$ 内。旋转之前这条线段在平面 $x = 0$ 上的投影是该平面内长度为 d 的那条线段。我们还可以把平面$x = 0$想象成一面墙，并假定有一个点光源在x轴正向的极远处，那么在墙上所看到的线段是连接原点和点$(\alpha_x, \alpha_y, \alpha_z)$的线段的影子。注意，线段投影的长度小于线段本身的长度。我们可以说这条线段被投影缩短（foreshorten）为 $d = \sqrt{\alpha_y^2 + \alpha_z^2}$。要求的 θ_x 就是这个投影和 z 轴的夹角。不过由 θ_x 的正弦和余弦就可以确定旋转矩阵，所以不需要直接计算出 θ_x。求出的 $\mathbf{R}_x$ 为：

$$\mathbf{R}_x(\theta_x) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \alpha_z/d & -\alpha_y/d & 0 \\ 0 & \alpha_y/d & \alpha_z/d & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

用类似的方法可以求出 $\mathbf{R}_y$，这个旋转如图 4.58 所示。注意旋转的方向是顺时针，所以必须小心矩阵中正弦项的符号。求出的 $\mathbf{R}_y$ 为：

$$\mathbf{R}_y(\theta_y) = \begin{bmatrix} d & 0 & -\alpha_x & 0 \\ 0 & 1 & 0 & 0 \\ \alpha_x & 0 & d & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

最后把所有矩阵级联在一起得到：

$$\mathbf{M} = \mathbf{T}(\mathbf{p}_0)\mathbf{R}_x(-\theta_x)\mathbf{R}_y(-\theta_y)\mathbf{R}_z(\theta)\mathbf{R}_y(\theta_y)\mathbf{R}_x(\theta_x)\mathbf{T}(-\mathbf{p}_0)$$

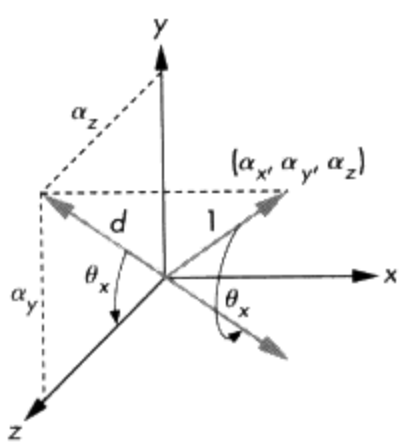


图 4.57　计算绕 x 轴旋转的角度

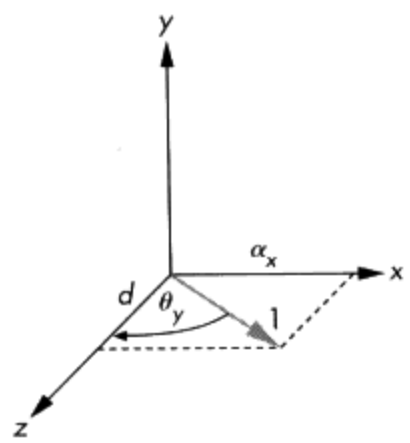


图 4.58　计算绕 y 轴旋转的角度

下面来看一个具体的例子。假定我们希望把一个对象绕经过原点和点(1, 2, 3)的直线旋转 45°。显然原点是不动点。第一步是求出旋转轴上到原点距离为 1 的点。为此，可把(1, 2, 3)归一化，如果用齐次坐标表示是$(1/\sqrt{14}, 2/\sqrt{14}, 3/\sqrt{14})$, $(1/\sqrt{14}, 2/\sqrt{14}, 3/\sqrt{14}, 1)$。先绕 x 轴旋转角度 $\arccos\frac{3}{\sqrt{13}}$，这使得变换为平面 $y = 0$ 内的$(1/\sqrt{14}, 2/\sqrt{14}, 3/\sqrt{14}, 1)$, $(1/\sqrt{14}, 0, \sqrt{13/14}, 1)$。绕 y 轴旋转的角度必须为 $-\arccos(\sqrt{13/14})$，以使得旋转轴和 z 轴重合。然后就可以绕 z 轴旋转所要求的 45°。最后再把前两次旋转抵消掉。把这 5 个矩阵级联，就得到了单独一个矩阵 $\mathbf{R}$：

$$\mathbf{R} = \mathbf{R}_x\left(-\arccos\frac{3}{\sqrt{13}}\right)\mathbf{R}_y\left(\arccos\sqrt{\frac{13}{14}}\right)\mathbf{R}_z(45)\mathbf{R}_y\left(-\arccos\sqrt{\frac{13}{14}}\right)$$

$$\mathbf{R}_x\left(\arccos\frac{3}{\sqrt{13}}\right)$$

$$= \begin{bmatrix} \frac{2+13\sqrt{2}}{28} & \frac{2-\sqrt{2}-3\sqrt{7}}{14} & \frac{6-3\sqrt{2}+4\sqrt{7}}{28} & 0 \\ \frac{2-\sqrt{2}+3\sqrt{7}}{14} & \frac{4+5\sqrt{2}}{14} & \frac{6-3\sqrt{2}-\sqrt{7}}{14} & 0 \\ \frac{6-3\sqrt{2}-4\sqrt{7}}{28} & \frac{6-3\sqrt{2}+\sqrt{7}}{14} & \frac{18+5\sqrt{2}}{28} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

这个矩阵不改变从原点到点(1, 2, 3)这条直线上任何点的位置。如果旋转的不动点是 $\mathbf{p}_f$，那么总的变换矩阵为：

$$\mathbf{M} = \mathbf{T}(\mathbf{p}_f)\mathbf{R}\mathbf{T}(-\mathbf{p}_f)$$

这个例子并不简单。它演示了一种强有力的方法，即把许多简单变换组合成一个复杂变换。在许多实际应用中都会遇到绕任意轴旋转的问题。这些问题之间的主要差别是指定旋转轴的方式不同。不过，我们通常可以用和这个例子相似的方法来确定方向角或者方向余弦。

4.10　OpenGL变换矩阵

现在讨论如何实现一个齐次坐标变换软件包以及这样的软件包应为用户提供怎样的接口。在OpenGL中有若干个矩阵是系统状态的一部分，这一章只用到了其中的模-视变换矩阵。这些状态矩阵由一组共同的函数来设置或修改，而具体是哪一个矩阵被设置或修改则由函数glMatrixMode来选择。在OpenGL中，模-视变换矩阵通常是一个仿射变换矩阵，因此像4.5节讨论的那样，它只有12个自由度。第5章将讨论的投影矩阵也是4 × 4矩阵，但不是仿射矩阵。OpenGL允许我们使用任意的4 × 4矩阵，这样就可以利用全部16个自由度了。

4.10.1　当前变换矩阵

大多数图形系统都使用当前变换矩阵（current transformation matrix，CTM）。系统对设置CTM之后定义的每个顶点都要应用CTM。如果改变了CTM，那么系统的状态也就变了。CTM是绘制流水线的一部分（见图4.59），因此，如果**p**是应用程序中定义的一个顶点，那么绘制流水线就会生成**Cp**。CTM是一个4 × 4矩阵，图形软件包提供了一组函数来对其进行修改。

图4.59　当前变换矩阵（CTM）

下面用**C**来代表CTM。初始时，它被设置成4 × 4单位阵；如果需要，也可以重新初始化。如果用符号 ← 来表示替换，那么可以把这个初始化操作表示为：

$$\mathbf{C} \leftarrow \mathbf{I}$$

对**C**进行操作的函数有两种形式：把CTM直接设置为某个矩阵或者用一个矩阵左乘或右乘CTM。OpenGL没有提供左乘。多数系统都提供的三种变换是：平移、以原点为不动点的缩放和以原点为不动点的旋转。可以用这些变换来修改CTM，用右乘的形式表为：

$$\mathbf{C} \leftarrow \mathbf{CT}$$
$$\mathbf{C} \leftarrow \mathbf{CS}$$
$$\mathbf{C} \leftarrow \mathbf{CR}$$

或者用加载的形式表示为：

$$\mathbf{C} \leftarrow \mathbf{T}$$
$$\mathbf{C} \leftarrow \mathbf{S}$$
$$\mathbf{C} \leftarrow \mathbf{R}$$

多数图形系统允许把CTM直接设置成任意矩阵**M**：

$$\mathbf{C} \leftarrow \mathbf{M}$$

或者用任意矩阵**M**右乘CTM：

$$\mathbf{C} \leftarrow \mathbf{CM}$$

4.10.2　旋转、平移和缩放

在OpenGL中，应用到所有图元的变换矩阵是模-视变换矩阵（GL_MODELVIEW）和投影变换矩阵（GL_PROJECTION）的乘积，所以可以把这两个矩阵的乘积看做CTM（见图4.60）。利用glMatrixMode可以选择想要的矩阵，然后单独对其进行设置或修改。下面的函数可以把选定的矩阵设置成任意矩阵。

```
glLoadMatrixf(pointer_to_matrix);
```

下面的函数把选定的矩阵设置为单位阵。

```
glLoadIdentity();
```

我们可以通过指向一维数组的指针来指定任意一个4 × 4矩阵，这个数组由矩阵的各列连在一起组成[①]，总共有16个元素。我们还可以用glMultMatrixf(pointer_to_matrix)来修改选定的矩阵。旋转、平移和缩放矩阵分别由下面三个函数指定：

```
glRotatef(angle, vx, vy, vz);
glTranslatef(dx, dy, dz);
glScalef(sx, sy, sz);
```

所有这三个函数通过右乘来改变选定的矩阵。对于旋转函数，旋转角度的单位是度，变量vx、vy、vz是旋转轴的方向向量的三个分量。平移函数的参数是位移向量的三个分量。缩放函数的参数是沿着每个坐标轴的缩放因子。

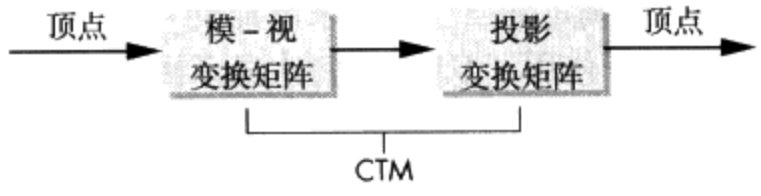


图4.60　模－视变换矩阵和投影变换矩阵

4.10.3　在OpenGL中实现具有任意不动点的旋转

在4.8节中已经讲过，对于旋转轴不经过原点的旋转，可以先把不动点移到原点，然后绕经过原点的旋转轴旋转，最后再把不动点移回初始的位置。下面的代码先设置矩阵模式，然后实现一个45°旋转，其旋转轴的方向向量从原点指向点(1, 2, 3)，并且(4, 5, 6)是不动点：

```
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
glTranslatef(4.0, 5.0, 6.0);
glRotatef(45.0, 1.0, 2.0, 3.0);
glTranslatef(-4.0, -5.0, -6.0);
```

注意，我们不一定要用4.9节中的方法来生成绕任意轴旋转的矩阵，尽管读者为了检验自己对旋转的理解可能会这样做，即通过级联绕三个坐标轴的旋转得到同样的矩阵。

4.10.4　变换的顺序

在上面的代码段中，所要求的函数调用是以相反的顺序出现的，读者可能会对此感到困惑。OpenGL中的规则是：最后指定的变换最先被应用。稍做检查就会明白，这样的顺序是右乘CTM的结果，因而既是正确的也是合理的。我们所指定的变换序列是：

$$\mathbf{C} \leftarrow \mathbf{I}$$
$$\mathbf{C} \leftarrow \mathbf{CT}(4.0, 5.0, 6.0)$$
$$\mathbf{C} \leftarrow \mathbf{CR}(45.0, 1.0, 2.0, 3.0)$$
$$\mathbf{C} \leftarrow \mathbf{CT}(-4.0, -5.0, -6.0)$$

每一步都对CTM右乘一个变换矩阵，最后生成了下面的矩阵：

$$\mathbf{C} = \mathbf{T}(4.0, 5.0, 6.0)\mathbf{R}(45.0, 1.0, 2.0, 3.0)\mathbf{T}(-4.0, -5.0, -6.0)$$

① 注意，这和C语言不同，C语言是按行来组织二维数组的。

这与在4.9节中得到的矩阵相一致，在这之后指定的每个顶点**p**都要用**C**来乘，因此新的顶点为：

$$\mathbf{q} = \mathbf{Cp}$$

还可以用其他方式来理解变换的顺序。一种方式是用栈来思考。改变CTM和把矩阵压入栈类似，当应用最后一个变换后，矩阵从栈中弹出的顺序和把它们压入栈的顺序相反。这个类比只是概念上的，并不是确切的，因为在OpenGL中调用变换函数后，矩阵立即就被修改了。

4.10.5　使立方体旋转起来

4.5节绘制了一个彩色立方体，现在我们编写一个程序让这个立方体旋转起来，旋转的方式由鼠标的三个按键来控制。在这个程序中要定义下面三个回调函数：

```
glutDisplayFunc(display);
glutIdleFunc(spincube);
glutMouseFunc(mouse);
```

函数display首先利用鼠标回调函数确定的三个角度来设置模－视变换矩阵，然后调用4.5.4节中的colorcube函数绘制立方体。这个例子使用了双缓存。每次显示回调函数执行时都要清空帧缓存和深度缓存，清空深度缓存是为了消除隐藏面。函数的最后要切换缓存。该函数的代码如下：

```
void display()
{
   glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
   glLoadIdentity();
   glRotatef(theta[0], 1.0, 0.0, 0.0);
   glRotatef(theta[1], 0.0, 1.0, 0.0);
   glRotatef(theta[2], 0.0, 0.0, 1.0);
   colorcube();
   glutSwapBuffers();
}
```

鼠标回调函数负责选择旋转轴：

```
void mouse(int btn, int state, int x, int y)
{
    if(btn==GLUT_LEFT_BUTTON && state == GLUT_DOWN) axis = 0;
    if(btn==GLUT_MIDDLE_BUTTON && state == GLUT_DOWN) axis = 1;
    if(btn==GLUT_RIGHT_BUTTON && state == GLUT_DOWN) axis = 2;
}
```

空闲回调函数在每次执行时把绕选定轴的旋转角度增加2°。

```
void spinCube()
{
   theta[axis] += 2.0;
   if( theta[axis] > 360.0 ) theta[axis] -= 360.0;
   glutPostRedisplay();
}
```

这里定义的键盘回调函数很简单，它可以使我们通过键盘终止程序：

```
void mykey(char key, int mousex, int mousey)
{
   if(key=='q'||key=='Q') exit();
}
```

直到第 5 章才会讨论隐藏面消除，不过就像在第 2 章所做的那样，利用 OpenGL 提供的函数进行隐藏面消除是很容易的，我们只需要清空深度缓存并通过glEnable(GL_DEPTH_TEST)开启这个选项，读者可以在附录 A 中找到完整的程序。

4.10.6　变换矩阵的加载、进栈和出栈

在多数情况下，可以通过旋转、平移和缩放组合出想要的变换矩阵。不过对一些变换，例如剪切，更容易的办法是直接设置变换矩阵。和加载一个单位阵相类似，我们可以用下面的函数调用把当前矩阵设置成一个 4 × 4 的齐次坐标矩阵：

```
glLoadMatrixf(myarray)
```

我们也可以通过下面的函数对当前矩阵右乘一个用户定义的矩阵：

```
glMultMatrixf(myarray)
```

myarray 是把矩阵的各列连起来组成的一维数组。因此，如果想要右乘 4 × 4 矩阵 m，可以像下面这样生成 myarray：

```
GLfloat m[4][4];
GLfloat myarray[16];

for(i=0;i<3;i++) for(j=0;j<3;j++)
   myarray[4*j+i]= m[i][j];
```

有时我们希望执行某个变换之后再把矩阵恢复到执行这个变换之前的状态。当执行的某个实例变换只适用于一个特定的对象，而对其余对象不能使用这个实例变换时，就会出现这种情况。此时不需要重新计算这个实例变换之前的矩阵状态，我们可以在应用实例变换之前，利用函数glPushMatrix把当前的变换矩阵压入栈中，以后再用函数glPopMatrix恢复。因此，经常可以看到下面的代码：

```
glPushMatrix();
glTranslatef(...);
glRotatef(...);
glScalef(...);

/* 此处是绘制对象的代码 */

glPopMatrix();
```

矩阵和属性的入栈和出栈对于显示列表极其重要。我们在显示列表开始时把状态变量压入各自的栈中，在显示列表结束时把它们从栈中弹出，这样就避免了由于在显示列表中改变状态而引起的任何副作用。

4.11　三维应用程序的接口

在4.10.5节，我们用三键鼠标来控制立方体绕哪个坐标轴旋转。用户通过这个接口只能和应用程序进行有限的交互。读者可能不希望把鼠标的三个键都用来控制旋转，这样就可以用鼠标控制其他功能，比如退出程序，而在前面的例子里是用键盘上的某个键来退出程序的。

在 4.9 节，我们注意到有许多方法可以使对象具有给定的方向。除了按照 x、y、z 的顺序绕平行于三个坐标轴的直线旋转以外，还可以先绕平行于 x 轴的直线旋转，然后绕平行于 y 轴的直线旋转，最后再绕平行于 x 轴的直线旋转。如果以这样的方式来旋转，只需两个鼠标按键就可以使对象

具有想要的方向。不过还有一个问题：这里的旋转只是绕着轴的一个方向。如果可以绕着一个轴正向或者反向旋转，当对象具有想要的方向时就停止旋转，那么控制对象的方向就更容易了。

GLUT允许用户把鼠标和键盘结合起来使用。例如，可以用鼠标左键使对象绕x轴正向旋转，用控制键结合鼠标左键使对象绕x轴反向旋转。

然而，上面这些方法都没能提供一个良好的用户接口。真正良好的接口应该更直观和更易于使用。下面讨论其他的方法，它们可以提供更有趣和更平滑的用户接口。

4.11.1　使用屏幕区域

假定我们想用鼠标的一个按键控制对象的方向，用另一个按键靠近或者远离对象，用第三个按键向左或者向右平移对象。所有这些功能都可以通过鼠标移动回调来实现。哪个按键被激活以及鼠标的位置作为参数返回给回调函数。我们可以用鼠标的位置来控制旋转轴的方向和旋转的速度、平移的方向和速度、还有对象靠近或者远离的速度。

正如刚才已经强调的那样，我们希望只绕两个坐标轴旋转就可以使对象具有任何方向。对象的方向可以用鼠标的左键和鼠标的位置来控制。我们可以用鼠标相对于屏幕中心的位置来控制绕x轴和y轴的旋转。如果左键被按下，但鼠标显示在屏幕的中心，那么不会引起对象的旋转。如果向上移动鼠标，对象将绕y轴顺时针旋转；如果向下移动鼠标，那么对象将绕y轴逆时针旋转。同样，向左或者向右移动鼠标将使对象绕x轴旋转。鼠标在屏幕上显示的位置到屏幕中心的距离可以用来控制旋转的速度。向斜上方或者斜下方移动鼠标可以使对象既绕x轴又绕y轴旋转。

用类似的方式来使用鼠标右键，我们可以左右和上下平移对象。我们还可以用鼠标中键来使对象远离或者靠近观察者，此时鼠标的位置控制对象沿z方向的平移。利用GLUT提供的函数实现这样的接口并不困难，我们把它作为一个习题留给读者来完成（见习题4.19）。

4.11.2　虚拟跟踪球

上面用鼠标的位置来控制绕两个轴的旋转，这为我们提供了跟踪球（见3.2节）的大部分功能。我们可以进一步利用鼠标和显示器生成一个图形的或者说虚拟的跟踪球。使用这种虚拟设备的一个优点是可以构建一个无摩擦的跟踪球，一旦开始旋转，它就不会停止，直到让它停下来为止。因此，这样的设备可以支持对象的连续旋转，而且可以改变旋转的速度和旋转轴的方向。对平移和其他能够通过鼠标控制的参数，也可以构建类似的虚拟设备。

我们先来把跟踪球的位置映射成鼠标的位置。考虑如图4.61所示的跟踪球。假定这个球的直径为1个长度单位。我们可以把球面上的一个点通过正交投影映射到平面$y=0$上，如图4.62所示。球面上的一点(x, y, z)被投影到平面上的$(x, 0, z)$。这个投影映射是可逆的，因为被投影到平面上的点是在球面上，它满足球面的方程：

$$x^2 + y^2 + z^2 = 1$$

因此，给定平面上的点$(x, 0, z)$，在半球面上和它对应的点必然是(x, y, z)，其中，

$$y = \sqrt{1 - x^2 - z^2}$$

利用从跟踪球位置到平面投影的逆映射就可以计算出虚拟跟踪球的三维信息，并且在鼠标运动时追踪虚拟跟踪球的位置。假定在半球面上有两个位置$\mathbf{p}_1$和$\mathbf{p}_2$，则从原点到这两个位置的两个向量确定了一个平面，如图4.63所示。该平面的法向量由这两个向量的叉积给出：

$$\mathbf{n} = \mathbf{p}_1 \times \mathbf{p}_2$$

跟踪球从位置 $\mathbf{p}_1$ 到位置 $\mathbf{p}_2$ 的运动可以通过绕 $\mathbf{n}$ 的一个旋转来实现。旋转的角度等于向量 $\mathbf{p}_1$ 和 $\mathbf{p}_2$ 之间的夹角，这个角度可以通过两个向量叉积的模得到。因为 $\mathbf{p}_1$ 和 $\mathbf{p}_2$ 的长度都是 1，所以

$$|\sin\theta| = |\mathbf{n}|$$

如果以较高的采样率跟踪鼠标的运动，那么检测到的位置改变会很小。所以我们不使用反三角函数计算 θ，而是利用下面的近似：

$$\sin\theta \approx \theta$$

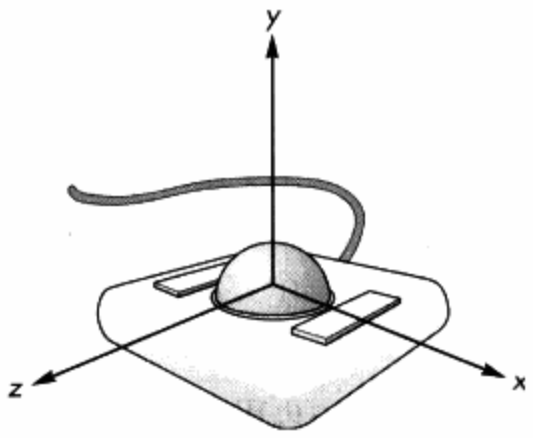


图 4.61　跟踪球标架

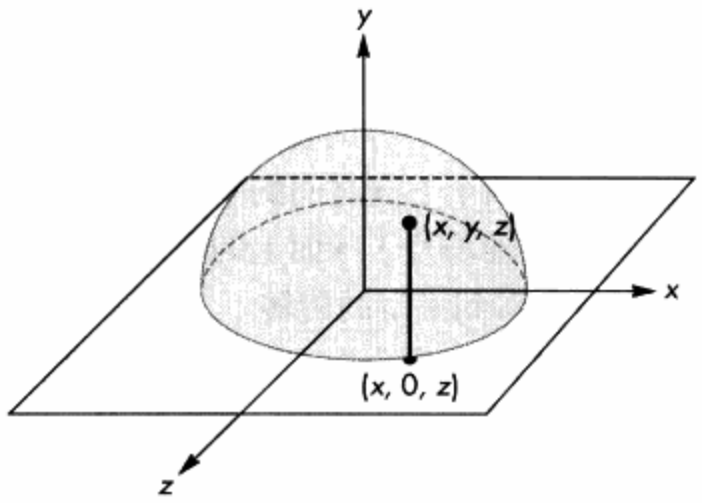


图 4.62　跟踪球位置在平面上的投影

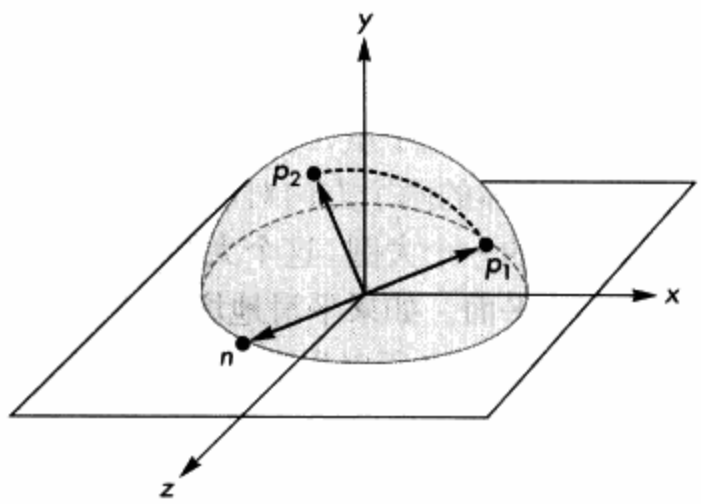


图 4.63　旋转平面的计算

通过使用GLUT中的空闲回调、鼠标移动回调和鼠标回调就可以实现虚拟跟踪球。为了控制是否跟踪鼠标和是否刷新显示，可以使用三个逻辑变量或者标志。这些变量的初始值为[①]：

```
bool trackingMouse = false;
bool trackballMove = false;
bool redrawContinue = false;
```

如果 redrawContinue 为真，那么空闲回调函数就迫使显示窗口被重新绘制；如果 trackingMouse 为真，鼠标移动回调函数就更新跟踪球的位置；如果trackballMove为真，就更新在绘图函数中使用的旋转矩阵。

鼠标回调函数可以修改这三个逻辑变量的值。当按下鼠标的一个按键时（可能是特定的一个按键也可能是任意一个按键，这取决于我们具体想要做什么），程序首先初始化跟踪球的位置，此后鼠标的位置一旦改变，鼠标移动回调函数就更新跟踪球的位置并使显示回调函数重新绘制立方体。当释放鼠标的这个按键后，程序就不再跟踪鼠标的位置了。我们可以用两个最近的鼠标位置定义一

① 如果编译器不支持 bool 类型，则可以使用宏定义 #define bool int，然后把 true 和 false 分别定义成 1 和 0。

个速度向量，就可以连续地更新旋转矩阵。于是，鼠标按键被释放以后，对象将以某个恒定速度持续旋转，这样的效果可以用一个理想的无摩擦的跟踪球实现，但是直接用实际的鼠标或者跟踪球无法实现。

读者在附录A里可以找到用虚拟跟踪球旋转立方体的程序。为了使程序运行得更快，我们在代码中做了一些简单的近似。这个程序还考虑到了可能出现的几个问题，例如鼠标的位置不是在跟踪球的半球面正下方该怎么办（当鼠标的位置在显示窗口的四角时就会发生这种情况）。

4.11.3　平滑的旋转

从这个跟踪球的例子可以看出，我们计算旋转矩阵的方法存在一个问题[①]。利用相对于三个坐标轴的角度（Euler角）来调整对象的方向，这使得我们通过级联绕x轴、y轴和z轴的简单旋转矩阵来获得绕任意轴旋转的矩阵。尽管OpenGL允许绕任意轴旋转，但我们通常采用级联的办法来确定旋转轴和相应的旋转角度。

考虑如何生成对象从一个方向运动到另一个方向的动画。原则上可以按下面的方式来确定一个合适的旋转矩阵：

$$\mathbf{R}(\theta) = \mathbf{R}_x(\theta_x)\mathbf{R}_y(\theta_y)\mathbf{R}_z(\theta_z)$$

上式把旋转矩阵表示为绕三个坐标轴旋转的乘积。如果想要生成对象在两个方向之间运动的一个图像序列，可以用小的步长改变绕三个坐标轴旋转的角度，这三个角度或者逐个改变或者同时改变。这样生成的图像序列在观察者看来不会是平滑的，观察者会觉察到绕每个坐标轴的旋转。

在跟踪球的例子里，我们看到可以用一种平滑的方式把立方体从一个方向直接旋转到另一个方向。因为立方体的两个方向对应于单位球面上的两个位置，所以在两个方向之间的平滑旋转对应于单位球面上的一个大圆。这个大圆对应于单独一次旋转，该旋转的轴垂直于球面上的那两个点和球心所确定的平面。如果平滑地增加这个旋转的角度，观察者就会看到平滑的旋转。

先前的方法之所以失败，在某种意义上是由于采用了基于坐标轴的数学表述。一种更深刻的方法包含在变换的矩阵表示中，这种方法不那么依赖坐标轴。考虑一个任意的旋转矩阵$\mathbf{R}$。设方向向量为$\mathbf{d}$的一条直线上所有的点在经过这个旋转后都保持不变，于是对这条直线上的任意一点$\mathbf{p}$都有

$$\mathbf{Rp} = \mathbf{p}$$

从矩阵$\mathbf{R}$的角度来说，列矩阵$\mathbf{p}$是$\mathbf{R}$的对应于特征值（eigenvalue）1的特征向量（eigenvector）（见附录C）。此外，对于方向$\mathbf{d}$，有

$$\mathbf{Rd} = \mathbf{d}$$

于是这个方向经过旋转后保持不变。因此，$\mathbf{d}$也是$\mathbf{R}$的对应于特征值1的特征向量。点$\mathbf{p}$必定是这个旋转的不动点，而向量$\mathbf{d}$必定是垂直于旋转轴的平面的法向量。从跟踪球的角度来看，计算出旋转轴等价于找出旋转矩阵的一个特定的特征向量。我们还可以从另一个角度看待这个问题：给定任意一个旋转矩阵，可以通过找出它的特征值和特征向量来确定旋转轴和不动点。

4.11.4　增量式旋转

设想给定了两个方向，我们想使一个对象，比如照相机，从一个方向平滑地运动到另一个方向。一种方法是像在虚拟跟踪球的例子中所做的那样，找出球面上的大圆路径并把所需的旋转角度

① 读者在初次阅读时可以跳过这一节。

分成多次来完成。这样，我们需要知道旋转轴、初始角度、结束角度以及角度增加的步长，这个步长由希望用多少步完成旋转来决定。代码的主循环具有下面的形式：

```
for(i=0, i<imax; i++)
{
    glRotatef(delta_theta, dx, dy, dz);
    draw_object()
}
```

这种方法的一个问题是需要对旋转矩阵中出现的三个角度计算正弦和余弦。如果只计算旋转矩阵一次，然后利用下面的代码重复使用这个矩阵，则可以做得更好：

```
GLfloat m[16];
glRotatef(dx, dy, dz, delta_theta);
glGetFloatv(GL_MODELVIEW_MATRIX, m);
for(i = 0, i<imax; i++)
{
    glMultMatrixf(m);
    draw_object()
}
```

我们也可以使用下面的小角度近似公式：

$$\sin\theta \approx \theta$$
$$\cos\theta \approx 1$$

如果通过 Euler 角来生成一个任意的旋转矩阵：

$$\mathbf{R} = \mathbf{R}_z(\psi)\mathbf{R}_y(\phi)\mathbf{R}_x(\theta)$$

则可以用两个近似公式把 **R** 写成：

$$\mathbf{R} = \begin{bmatrix} \cos\psi & -\sin\psi & 0 & 0 \\ \sin\psi & \cos\psi & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\phi & 0 & \sin\phi & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\phi & 0 & \cos\phi & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\approx \begin{bmatrix} 1 & -\psi & \phi & 0 \\ \psi & 1 & -\theta & 0 \\ -\phi & \theta & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4.12　四元数

四元数是复数的扩展，它提供了描述和处理旋转的另一种方法。尽管这种方法不像前面使用的方法那么直观，但它在生成动画和旋转的硬件实现方面有其优越之处。

4.12.1　复数和四元数

对二维的情形，用复数来表示像旋转这样的变换已经为大多数科学和工程技术专业的学生所熟知。令 **i** 表示虚数单位，即 $\mathbf{i}^2 = -1$。由 Euler 公式

$$e^{i\theta} = \cos\theta + \mathbf{i}\sin\theta$$

我们可以把一个复数 $\mathbf{c}$ 的极坐标表示写成：

$$\mathbf{c} = a + \mathbf{i}b = re^{i\theta}$$

其中，$r = \sqrt{a^2 + b^2}$，$\theta = \arctan b/a$。

设 $\mathbf{c}$ 绕原点旋转角度 ϕ 后变为 $\mathbf{c}'$，我们可以利用旋转矩阵求出 $\mathbf{c}'$，还可以利用复数的极坐标表示：

$$\mathbf{c}' = re^{i(\theta+\phi)} = re^{i\theta}e^{i\phi}$$

于是，$e^{i\phi}$ 是在复平面中的一个旋转算子，这为我们提供了一种在实际应用中更有效的变换方法。

然而，我们真正感兴趣的是三维空间中的旋转。三维旋转的问题更复杂，因为确定一个以原点为不动点的三维旋转需要指定一个方向（一个向量）和旋转的角度（一个标量）。一种办法是使用一种既包含向量又包含标量的表示。通常，这种表示被写成四元数（quaternion）的形式：

$$a = (q_0, q_1, q_2, q_3) = (q_0, \mathbf{q})$$

其中，$\mathbf{q} = (q_1, q_2, q_3)$。利用三个“复数” $\mathbf{i}$、$\mathbf{j}$ 和 $\mathbf{k}$ 的下列性质，可以定义四元数的运算：

$$\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = \mathbf{ijk} = -1$$

这三个数类似于三维空间中的单位向量，于是可以把 $\mathbf{q}$ 写成：

$$\mathbf{q} = q_1\mathbf{i} + q_2\mathbf{j} + q_3\mathbf{k}$$

现在可以利用 $\mathbf{i}$、$\mathbf{j}$ 和 $\mathbf{k}$ 之间的关系导出四元数的加法和乘法。设四元数 b 为：

$$b = (p_0, \mathbf{p})$$

利用向量的点积和叉积可得：

$$a + b = (p_0 + q_0, \mathbf{p} + \mathbf{q})$$
$$ab = (p_0q_0 - \mathbf{q}\cdot\mathbf{p}, q_0\mathbf{p} + p_0\mathbf{q} + \mathbf{q}\times\mathbf{p})$$

我们还可以按通常的方式定义四元数的模：

$$|a|^2 = q_0^2 + q_1^2 + q_2^2 + q_3^2 = q_0^2 + \mathbf{q}\cdot\mathbf{q}$$

四元数乘法有一个单位元，即(1, 0)。容易证明四元数的逆由下式给出：

$$a^{-1} = \frac{1}{|a|^2}(q_0, -\mathbf{q})$$

4.12.2　四元数和旋转

到现在为止，我们定义了一种新的数学对象。为了使它对我们有用，必须把它和几何实体联系起来并借助其运算来表示像旋转这样的变换。假定用四元数的向量部分代表空间中的一点：

$$p = (0, \mathbf{p})$$

这样，$\mathbf{p} = (x, y, z)$的分量给出了该点的位置。考虑四元数

$$r = \left(\cos\frac{\theta}{2}, \sin\frac{\theta}{2}\mathbf{v}\right)$$

其中，$\mathbf{v}$ 的长度为 1。可以证明四元数 r 的模为 1（$|r| = 1$），由此可得：

$$r^{-1} = \left(\cos\frac{\theta}{2}, -\sin\frac{\theta}{2}\mathbf{v}\right)$$

利用四元数的乘法可以得到一个新的四元数：

$$p' = rpr^{-1}$$

这个四元数形如$(0, \mathbf{p}')$，其中，

$$\mathbf{p}' = \cos^2\frac{\theta}{2}\mathbf{p} + \sin^2\frac{\theta}{2}(\mathbf{p}\cdot\mathbf{v})\mathbf{v} + 2\sin\frac{\theta}{2}\cos\frac{\theta}{2}(\mathbf{v}\times\mathbf{p}) - \sin\frac{\theta}{2}(\mathbf{v}\times\mathbf{p})\times\mathbf{v}$$

因此 p' 是一个点的表示。这里需要指出：p' 是 $\mathbf{p}$ 绕向量 $\mathbf{v}$ 旋转角度 θ 后的位置。这个陈述的正确性不太明显，但可以通过比较 p' 和一般旋转矩阵中的各个分量来证明。在给出证明之前，先来考虑这个结果的意义。以原点为不动点绕任意轴的旋转除了可以用矩阵表示以外，还可以用四元数乘法表示，因为它们都给出了相同的结果。四元数表示所需的运算次数更少，因而速度更快，并且现在的硬件和软件都支持四元数运算。

下面考虑几个例子。假定以原点为不动点绕 z 轴旋转角度 θ，单位向量 v 为$(0, 0, 1)$，四元数 r 为：

$$r = \cos\frac{\theta}{2} + \sin\frac{\theta}{2}(0, 0, 1)$$

任一点 $\mathbf{p} = (x, y, z)$经过旋转后对应的四元数为：

$$p' = rpr^{-1} = r(0, \mathbf{p})r^{-1} = (0, \mathbf{p}')$$

其中，

$$\mathbf{p}' = (x\cos\theta - y\sin\theta, x\sin\theta + y\cos\theta, z)$$

于是应该到了和矩阵表示一样的结果，但所需的运算量更少。如果依次绕三个坐标轴旋转的矩阵表示为 $\mathbf{R} = \mathbf{R}_x(\theta_x)\mathbf{R}_y(\theta_y)\mathbf{R}_z(\theta_z)$，那么还可以用对应四元数的乘积把这个旋转序列表示为 $r_x r_y r_z$。

现在回到绕任意轴的旋转，在 4.9.4 节，我们导出了如下形式的矩阵：

$$\mathbf{M} = \mathbf{T}(\mathbf{p}_0)\mathbf{R}_x(-\theta_x)\mathbf{R}_y(-\theta_y)\mathbf{R}_z(\theta_z)\mathbf{R}_y(\theta_y)\mathbf{R}_x(\theta_x)\mathbf{T}(-\mathbf{p}_0)$$

因为开始和结束时需要进行平移，所以不能用四元数来表示整个变换。不过可以根据 $p' = rpr^{-1}$ 中的元素确定出 $\mathbf{M}$ 中的旋转部分。因此，如果仍然设 $r = (\cos\frac{\theta}{2}, \sin\frac{\theta}{2}\mathbf{v})$，那么

$$\mathbf{R} = \begin{bmatrix} 1-2\sin^2\frac{\theta}{2}(v_y^2+v_z^2) & 2\sin^2\frac{\theta}{2}v_xv_y - 2\cos\frac{\theta}{2}\sin\frac{\theta}{2}v_z \\ 2\sin^2\frac{\theta}{2}v_xv_y + 2\cos\frac{\theta}{2}\sin\frac{\theta}{2}v_z & 1-2\sin^2\frac{\theta}{2}(v_x^2+v_z^2) \\ 2\sin^2\frac{\theta}{2}v_xv_z - 2\cos\frac{\theta}{2}\sin\frac{\theta}{2}v_y & 2\sin^2\frac{\theta}{2}v_yv_z + 2\cos\frac{\theta}{2}\sin\frac{\theta}{2}v_x \\ 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 2\sin^2\frac{\theta}{2}v_xv_z + 2\cos\frac{\theta}{2}\sin\frac{\theta}{2}v_y & 0 \\ 2\sin^2\frac{\theta}{2}v_yv_z - 2\cos\frac{\theta}{2}\sin\frac{\theta}{2}v_a & 0 \\ 1-2\sin^2\frac{\theta}{2}(v_x^2+v_y^2) & 0 \\ 0 & 1 \end{bmatrix}$$

如果应用下面的三角恒等式：

$$\cos\theta = \cos^2\frac{\theta}{2} - \sin^2\frac{\theta}{2} = 1 - 2\sin^2\frac{\theta}{2}$$

$$\sin\theta = 2\cos\frac{\theta}{2}\sin\frac{\theta}{2}$$

并且注意到 **v** 是一个单位向量：

$$v_x^2 + v_y^2 + v_z^2 = 1$$

那么可以把矩阵 **R** 写成人们更加熟悉的形式。因此，通过匹配 **R** 和 r 的分量，可以用四元数乘积得到 **M** 中的旋转部分。至于开始和结束时的两个平移，可以用通常的向量运算来表示。

四元数不但运算效率比旋转矩阵更高，而且在生成动画时还可以通过对四元数进行插值来获得旋转的平滑序列。第 10 章会讲到需要平滑旋转的更多例子。

小结和注释

本章从两种不同的但最终却是互补的观点来讨论计算机图形学中的数学。一种观点是：如果要理解在程序中执行的运算，就必须对图形学中遇到的几何对象进行数学抽象。另一种观点是：变换以及基于齐次坐标表示的变换技术是实现图形系统的基础。

数学工具来自向量分析和线性代数。不过，为了计算机图形学的目的，我们介绍这些工具的顺序和大多数学生学习它们的顺序相反。尤其是通常要先学习线性代数，然后再把向量空间的概念和 $\mathbf{R}^n$ 中的 n 元组联系起来。与此相反，我们先学习数学空间中的表示，然后把线性代数作为实现抽象类型的工具。

有两个原因促使我们寻求一种与坐标无关的方法。第一，我们想要表明，几何对象和变换的所有基本概念与后者的表示方式无关；第二，随着面向对象语言变得越来越流行，应用程序员将会直接处理对象，而不是处理对象的表示。在这一章的建议阅读资料里提到了一些几何编程系统的例子，从中可以看到这种方法的潜力。

齐次坐标是数学抽象的威力的一个精彩例证。通过在抽象的数学空间（仿射空间）中考虑问题，我们能够找到一种工具，从该工具可以直接导出利用软/硬件来实现变换的有效方法。

最后，我们介绍了 OpenGL 支持的仿射变换集，还讨论了把这些变换复合起来构成全部仿射变换的方法。通过组合几个简单类型的变换矩阵来构建想要的变换是一种强有力的方法。这一章末尾的几道习题会要求读者使用这种方法。第5章将在这些技术的基础上构建三维图形的投影图，第 10 章将使用变换来构造层级模型。

建议阅读资料

有许多关于向量分析和线性代数的教科书，但它们当中的大多数把这两者视为不同的议题分别加以讨论。在几何设计领域，人们普遍采用与坐标无关的向量空间方法来描述曲线和曲面，有关这方面的内容请参阅 Faux 和 Pratt[Fau80]的书。想要了解几何编程，可以参阅 DeRose 的书[DeR88, DeR89]。齐次坐标最早出现在几何学的文献中[Max51]，后来计算机图形学的研究人员发现了四元数在图形学中的应用[Rie81]。SGI公司的Geometry Engine[Cla82]最早用硬件实现了齐次坐标变换。现代硬件体系结构使用了能够执行齐次坐标变换的专用集成电路（ASIC）。

Shoemaker[Sho85]把四元数引入计算机图形学，用于生成动画。Kuipers 的书[Kui99]中有许多使用旋转矩阵和四元数的例子。

像Mathematics[Wol91]和MATLAB[Mat95]这样的软件工具为人们学习变换矩阵提供了非常棒的助手。

习题

4.1　证明下列每组变换中的两个变换都是可交换的：
a. 旋转和均匀缩放
b. 绕同一个轴的两个旋转
c. 两个平移

4.2　除了旋转的角度是到原点距离的 f 倍，扭转变换与以原点为不动点的旋转类似。编写一个程序按照用户提供的 f 值对 Sierpinski 镂垫进行扭转，这里的 Sierpinski 镂垫是通过细分三角形生成的。观察当细分的次数改变时镂垫的形状如何变化。

4.3　编写一个支持几何编程的函数库。这个函数库应该包含处理基本类型（点、线、向量）的函数，还要支持这些类型之间的运算，包括点乘和叉乘。它还应该允许改变标架。还可以创建和 OpenGL 的接口函数，这样就可以显示几何运算的结果。

4.4　如果只对二维图形感兴趣，那么可以用三维齐次坐标把点表示成 $\mathbf{p} = [x\ y\ 1]^T$，把向量表示成 $\mathbf{v} = [a\ b\ 0]^T$。求出 3 × 3 旋转矩阵、平移矩阵、缩放矩阵和剪切矩阵。二维空间中的仿射变换有多少个自由度？

4.5　我们可以通过少数几个点在变换前后的位置来确定一个仿射变换。对三维空间中的仿射变换，为了确定唯一的仿射变换，必须指定几个点在变换前后的位置？如果是二维空间中的仿射变换呢？

4.6　如果使用左手系但旋转正方向的定义保持不变，应该如何修改旋转矩阵？

4.7　证明由旋转和平移组成的任何变换序列都等价于先进行一个以原点为不动点的旋转然后再进行一个平移。

4.8　从旋转、平移和缩放变换推导出剪切变换。

4.9　在二维空间，可以用方程 $y = mx + h$ 确定一条直线。求关于这条直线反射的仿射变换。把这个结果扩展到在三维空间中关于一个平面反射。

4.10　4.8节指出任意一个旋转矩阵可以由绕三个坐标轴的旋转组合而成。如果只允许进行三次绕坐标轴的旋转，有多少种方法可以合成一个给定的旋转？绕三个坐标轴的旋转矩阵都是必需的吗？

4.11　在实例变换中增加剪切变换。说明如何利用扩充后的实例变换把单位立方体变成平行六面体。

4.12　求平面的齐次坐标表示。

4.13　求由 glRotate 确定的旋转矩阵，即假定原点是不动点并且旋转轴的方向和旋转角度由该函数的参数给定。

4.14　考虑常系数线性微分方程或者常系数线性差分方程（递归关系）的解。证明齐次方程的解构成了一个向量空间。一个特定的非齐次方程的解和仿射空间有什么联系？

4.15　编写一个程序按照如下方法生成Sierpinski镂垫。从一个白色的三角形开始迭代。在每次迭代时，利用几何变换在原来的三角形内绘制三个相似三角形，使得原来三角形的中心部分仍为白色，而中心部分以外的三个顶点附近的区域变为黑色。

4.16　假设一个立方体的中心位于原点，并且各棱分别平行于三个坐标轴。找出一个旋转矩阵，使得这个立方体经过旋转之后的方向关于三个坐标轴对称，如图 4.64 所示。

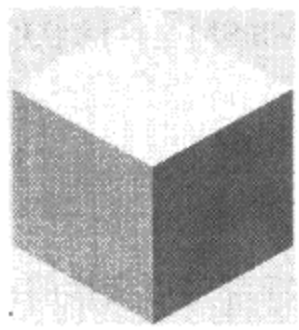


图4.64　立方体的方向关于三个坐标轴对称

4.17 我们已经利用三维空间中的顶点定义了像三维多边形这样的对象。设计一个算法来检验一组顶点是否确定了一个平面多边形。

4.18 不在一条直线上的三个顶点确定了一个平面。设计一个算法来判断三个顶点是否共线。

4.19 我们把一个实例变换定义为一个平移、一个旋转和一个缩放的乘积。如果改变应用这三类变换的顺序，还能获得相同的效果吗？

4.20 编写一个程序，该程序运行时可以用鼠标的一个按键调整立方体的方向，用另一个按键平移立方体，用第三个按键缩放立方体。

4.21 给定两个不平行的三维向量u和v，如何生成一个正交坐标系，使得u是基向量之一并且还有一个基向量也在u和v所确定的平面内。

4.22 绕z轴的增量式旋转可以由下面的矩阵来近似：

$$\begin{bmatrix} 1 & -\theta & 0 & 0 \\ \theta & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

用这个矩阵进行许多次旋转以后，会有什么不良后果？你能想出补救的办法吗？提示：考虑到原点距离为1的点。

4.23 绕x轴旋转90°和绕y轴旋转90°分别对应哪两个四元数？它们的乘积是多少？

4.24 确定旋转矩阵$\mathbf{R} = \mathbf{R}(\theta_x)\mathbf{R}(\theta_y)\mathbf{R}(\theta_z)$。找出与之对应的四元数。

4.25 利用四元数而不是旋转矩阵重新编写跟踪球程序。

4.26 用C++实现一个简单的函数库。这个函数库可以实现变换所需要的数学运算，它应该包含矩阵、向量和标架。

4.27 原则上，一个面向对象的系统可以把标量、向量和点作为基本类型提供给程序员。为什么许多流行的API没有这么做？

4.28 证明求和式

$$P = \alpha_1 P_1 + \alpha_2 P_2 + \cdots + \alpha_n P_n$$

有定义，当且仅当

$$\alpha_1 + \alpha_2 + \cdots + \alpha_n = 1$$

提示：先把前两项写成：

$$\begin{aligned} P &= \alpha_1 P_1 + \alpha_2 P_2 + \cdots = \alpha_1 P_1 + (\alpha_2 + \alpha_1 - \alpha_2) P_1 + \cdots \\ &= \alpha_1 (P_1 - P_2) + (\alpha_1 + \alpha_2) P_2 + \cdots \end{aligned}$$

然后照此递推下去。

第5章 观　　察

我们已经完成了对虚拟照相机模型前半部分的讨论，即在三维空间中指定对象。现在来考察描述虚拟照相机的多种方式。在这个过程中，我们还将讨论相关的一些议题，例如经典的观察技术与计算机观察之间的联系，以及如何用投影变换来实现投影。

本章内容包括三部分。首先讨论可以创建哪些类型的投影图以及需要多类投影图的原因。然后考察在应用程序中如何利用OpenGL函数指定一种特定的观察。我们将会看到观察过程包括两个步骤。第一个步骤使用模-视变换矩阵把对象的顶点表示从对象标架变换到眼标架。对象是在对象标架中定义的，而照相机位于眼标架的原点。获得了几何对象在眼标架下的表示之后，就可以使用规范的观察过程。第二个步骤是指定想要的投影类型（平行投影或透视投影）和视见体（或称为裁剪体，是场景中我们希望成像的那一部分）。投影类型和视见体确定了一个投影变换矩阵，该矩阵可以与模-视变换矩阵级联。此外，我们用一个简单的示例程序来演示如何利用OpenGL函数生成投影图。本章最后一部分将推导平行投影和透视投影的投影变换矩阵，这是两类最重要的投影。

5.1 经典观察和计算机观察

在讨论计算机图形系统和应用程序之间为三维观察而设置的接口之前，我们稍微偏离主题考虑一下经典的观察。这样做是出于两个原因。第一，许多以前由人手工完成的绘制任务，例如电影中的动画制作、建筑绘图、草图绘制和机械部件设计，现在通常由计算机辅助完成。这些领域的工作人员需要生成经典的投影图，例如等轴测投影、正视图以及各种透视投影，因此要求计算机也能绘制这样的投影图。第二，大多数API所使用的观察方法有许多优点，但也会引起一些困难，通过考察经典观察和计算机观察之间的联系，可以更好地理解这些优点和困难。

在第1章介绍虚拟照相机模型时，我们指出了经典观察和计算机观察之间的相似性。这两种观察的基本元素是相同的，都包括对象、观察者、投影线和投影平面（见图5.1）。投影线相交于投影中心（center of projection，COP）。COP对应于照相机中的透镜或者人眼中的晶状体，而在计算机图形系统中，对于透视投影而言，COP位于照相机标架（camera frame）的原点。所有图形系统都遵循第1章介绍的基于几何光学的模型。投影面是平面，并且投影线是直线。我们通常遇到的就是这种类型的投影，这类投影实现起来比较容易，如果利用绘制流水线模型来实现更是如此。

经典观察和计算机观察都允许观察者相距对象无穷远。注意，如果COP移动到无穷远，则投影线会变成平行线，此时可以用投影方向（direction of projection，DOP）代替COP，如图5.2所示。还要注意当COP向无穷远处移动时，我们可以把投影平面固定，这样所成的投影图大小基本不变，即使COP距离对象无穷远也是如此。如果COP在有限远处，那么生成的是透视投影图（perspective view）；如果COP在无限远处，那么生成的是平行投影图（parallel view）。对于平行投影，照相机标架的原点通常位于投影平面。

彩图13和彩图14分别是平行投影图和透视投影图。从这些彩图可以看出，能够既生成平行投影图又生成透视投影图对于像建筑设计这样的应用领域是很重要的。在支持两种投影的API中，用户很容易就可以在各种观察模式之间切换。大多数现代API既支持平行投影也支持透视投影。这些

系统生成的投影称为平面几何投影（planar geometric projection），因为投影面是平面并且投影线是直线。直线经透视投影和平行投影后仍是直线，但透视投影和平行投影一般不会保持角度。尽管平行投影是透视投影的极限情形，经典观察和计算机观察通常都把它们当做不同的类型。在经典观察中，人们手工绘制这两种投影图所用的方法是不同的，这一点任何上过制图课的人都知道；在计算机观察中，这两类投影的指定方式也是不同的。我们不是把平行投影看做透视投影的特殊情形，而是推导出极限情形的方程，然后直接从这些方程得到对应的投影变换矩阵。在现代流水线结构中，对应于两类投影的投影变换矩阵都可以加载到绘制流水线中。

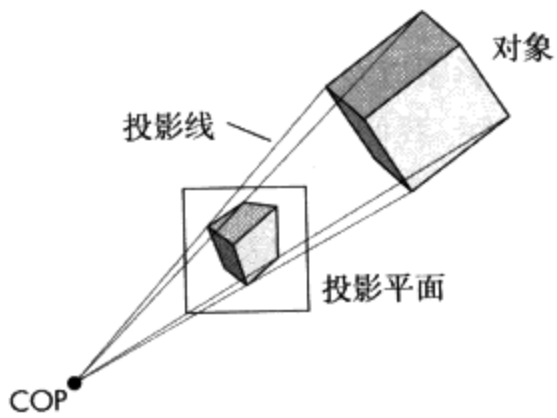


图 5.1　观察

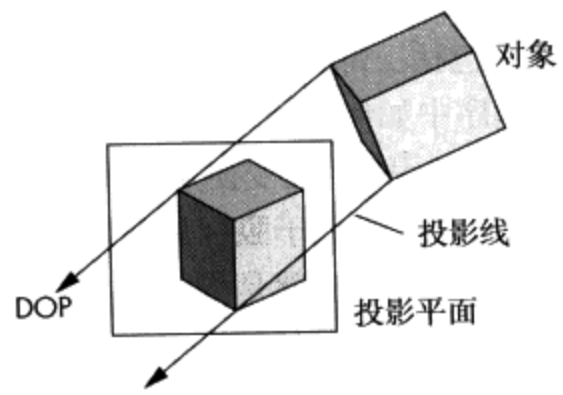


图 5.2　把投影中心移到无穷远

尽管计算机图形系统只有两类基本的观察（平行投影和透视投影），但经典观察却可以生成许多种不同的投影图，从多视图正交投影到一点、两点和三点透视。这个表面上的不一致是由于经典观察想要表明在对象、观察者和投影平面之间的特定位置关系，而在计算机图形学中，对这些元素的指定是完全独立的。

5.1.1　经典观察

当建筑师绘制建筑物的投影图时，他（她）知道想要显示建筑物的哪一面，因此也就知道观察者相对于建筑物应位于何处。每一种经典投影图都是由对象和观察者之间的特定位置关系确定的。

在经典观察中有一个基本的概念称为主面（principal face）。在像建筑设计这样的实际应用领域中遇到的对象，其表面往往由一些平面组成，每个平面都可以看成是一个主面。对于像建筑物这样的长方体对象，有前面、后面、顶面、底面、右面和左面这些自然的概念。此外，许多现实世界中的对象具有互相垂直的面，因此经常可以在这些对象的表面找到三个彼此正交的方向。

图5.3列举出了一些主要的投影图类型。我们先介绍平行投影和透视投影中受限制最多的投影，然后再介绍限制条件较少的投影。

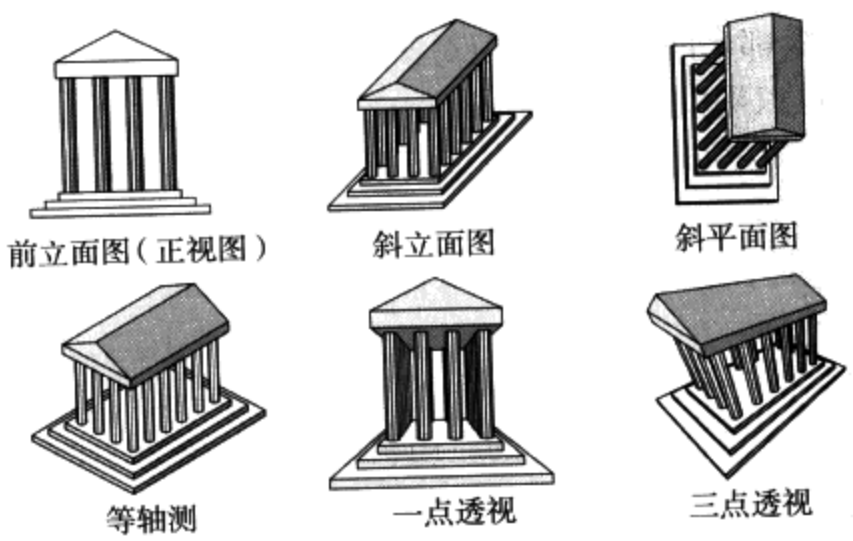


图 5.3　经典投影图

5.1.2 正投影

我们要讨论的第一个经典投影图是正投影（orthographic projection），如图5.4所示。在所有正投影（也称为正交投影）中，投影线与投影平面垂直。在多视图正投影（multiview orthographic projection）中，我们绘制多个投影图，每个投影图的投影平面平行于对象的某个主面。通常用三个视图（例如正视图、俯视图和右视图）来显示对象。从图5.5可以看出为什么需要多个视图。对于一个盒子形状的对象，在投影图中只能看到它的一个面，即平行于投影平面的那个面。观察者通常需要两个以上的视图才能从多视图正投影中体会出它是什么样子。从这些图像想象出对象的形状可能还要求观察者有一定的技能。这类投影图的重要性在于它不改变距离和角度。因为距离和形状都没有失真，所以多视图正投影很适于在施工图纸中使用。

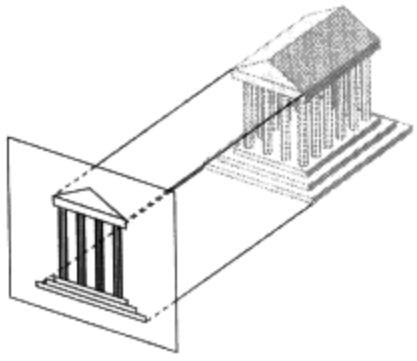


图5.4 正投影

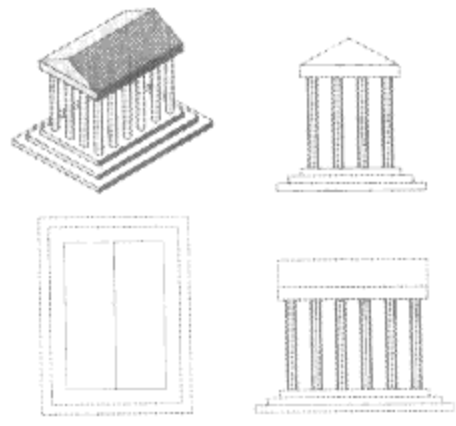


图5.5 神殿和它的多视图正投影（包括三个视图）

5.1.3 轴测投影

对于一个盒子形状的对象，如果想在一个投影图中看到它的多个主面，就必须去掉前面对投影图的诸多限制中的一个。在轴测（axonometric）投影中，投影线仍旧垂直于投影平面，如图5.6所示，但是投影平面相对于对象的方向可以是任意的。长方体形状的对象的一个顶点会有三个主面相交，如果投影平面的位置相对于这三个主面对称，则称为等轴测（isometric）投影。如果投影平面的位置相对于两个主面对称，则为正二测（dimetric）投影。一般的情形是正三测（trimetric）投影。这些投影图如图5.7所示。注意，对于等轴测投影，一条线段在图像空间里的长度比它在对象空间中的长度要短。这种距离的投影缩短（foreshortening）在三个主方向上都是相同的，因此仍然可以借助等轴测投影图测量距离。然而，对正二测投影，存在两个不同的投影缩短比率；对正三测投影，存在三个不同的投影缩短比率。此外，在轴测投影中，尽管平行线仍然是平行的，但角度可能会改变。圆投影后变成了椭圆。这种失真是我们为了在一个投影图中看到多个主面而付出的代价。轴测投影图广泛应用于建筑设计和机械设计等领域。

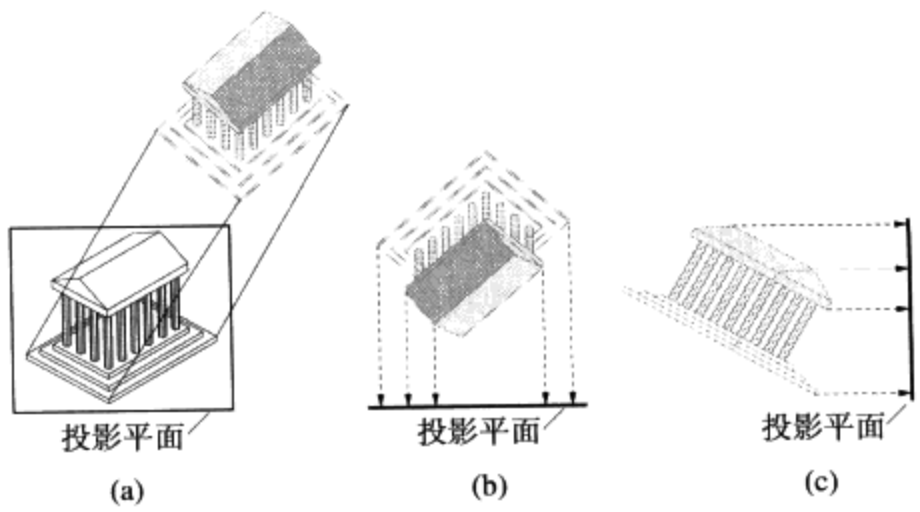


图5.6 轴测投影：(a) 正三测投影图的生成；(b) 俯视图；(c) 侧视图

图 5.7　轴测投影图

5.1.4　斜投影

斜（oblique）投影是最一般的平行投影。如图5.8所示，如果允许投影线与投影平面成任意角度，就得到了斜投影。因此，平行于投影平面的平面内的角度保持不变，平行于投影平面的平面内的圆在投影后仍为圆，不过可以看到对象的多个主面。斜投影图是最难用手工绘制的，它们看起来有些不自然。大多数物理观察设备，包括人类视觉系统，都包含一个透镜，并且该透镜与图像平面的位置关系是固定的（通常透镜平行于图像平面）。虽然这些设备生成的是透视投影图，但如果观察者远离对象，则可以近似实现平行投影，不过是正交投影，因为投影平面平行于透镜。我们在1.6节讨论虚拟照相机模型时介绍过折叠暗箱照相机，它可以生成近似的斜平行投影图。

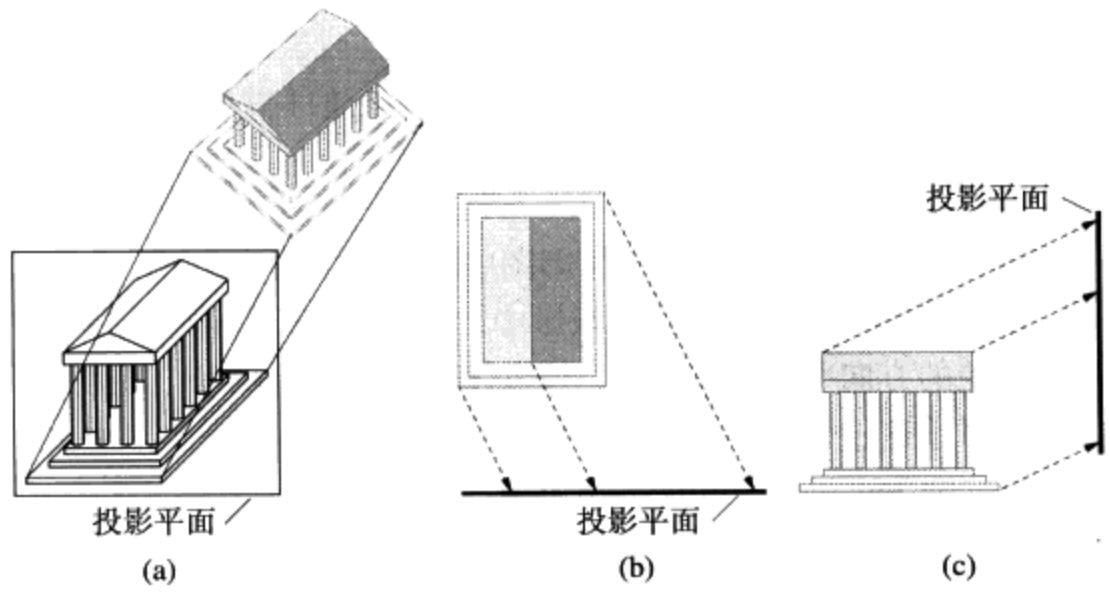


图 5.8　斜投影：(a) 斜投影图的生成；(b) 俯视图；(c) 侧视图

从应用程序编程人员的角度来看，不同的平行投影之间没有显著的差异。应用程序编程人员只需指定一种投影类型（平行投影或者透视投影）并对描述照相机的一组参数进行设置。应用程序编程人员所面对的问题是如何在观察过程中指定这些参数，以获得最佳的观察效果或者生成特定的一种经典投影图。

5.1.5　透视观察

所有的透视投影图都有一个特征：投影后尺寸会缩短（diminution）。对象距离观察者越远，它们所成的图像就越小。这种尺寸的缩短使得透视投影图看起来比较自然，但是由于线段缩短的程度依赖于线段到观察者的距离，所以不能借助透视投影图测量尺寸。因此，透视投影图主要用在建筑设计和动画制作等领域，这些应用领域要求生成看上去自然的图像。

在经典的透视投影中，观察者的位置相对于投影平面是对称的，如图5.9所示。因此，由投影平面内的窗口和投影中心确定的棱锥是一个对称的棱锥，或者说是一个正棱锥。在人类视觉系统中，这种对称性是由于晶状体与视网膜之间的固定位置关系造成的。对于标准照相机，这种对称性是由

于透镜和胶片平面之间的固定位置关系造成的。由于相似的原因，其他大多数物理成像设备中的透视投影也具有这种对称性。一些照相机，例如折叠暗箱照相机，具有可移动的胶片平面，因而能够生成一般的透视投影图。在计算机图形学中使用的模型包括这样的一般情形。

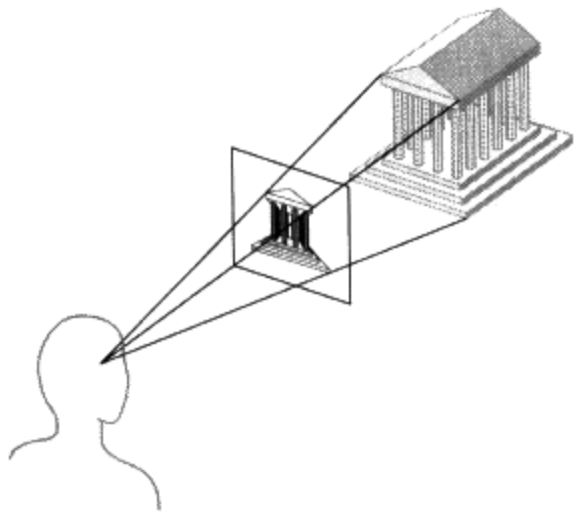


图 5.9 透视观察

人们通常把经典透视投影图分为一点、两点和三点透视（one-, two-, and three-point perspectives），这三种情形的差别在于对象的三个主方向中有几个平行于投影平面。考虑图 5.10 所示的一个建筑物的三个透视投影图。这个建筑物的整体形状和长方体类似，它的任何一角都包含三个主方向。对最一般的情形（三点透视），沿着每个主方向的平行线相交于有限远处的灭点（vanishing point）（见图 5.10(a)）。如果允许一个主方向平行于投影平面，则为两点投影（见图 5.10(b)），其中沿着两个主方向的平行线相交，而沿着第三个主方向的平行线投影后仍是平行的。在一点投影中（见图 5.10(c)），两个主方向平行于投影平面，因而只有一个灭点。显然，从程序员的角度来看，这三种透视投影只不过是一般透视投影的特殊情况，这与程序员看待平行观察的观点是类似的。我们将在 5.4 节实现一般的透视投影。

(a) (b) (c)

图 5.10 经典透视投影图：(a) 三点透视；(b) 两点透视；(c) 一点透视

5.2 计算机观察

现在从计算机的角度来讨论三维图形。因为计算机图形学中的观察建立在虚拟照相机模型的基础上，所以应该能够生成任何一种经典投影图。不过，这两者有本质的区别：所有的经典投影图都依赖于对象、观察者和投影线之间的特定位置关系，而在计算机图形学中，我们强调对象定义和照相机参数设置这两者之间的独立性。在OpenGL中，可以在透视投影照相机和正交投影照相机之间选择。OpenGL不能判断一个透视投影图是一点、两点还是三点透视，因为这需要知道对象和照相机之间的位置关系。权衡之下，我们更愿意拥有这种独立性，但如果应用程序需要一种特定类型的投影图，程序员将不得不指定照相机与对象之间的位置关系。

从流水线结构的角度来看，观察包括两个基本步骤。第一步，我们必须把照相机放到指定的位置并使其具有指定的方向。这一步由模－视变换来完成。顶点经过模－视变换之后，就得到了它们

在照相机标架下的表示。第二步是应用投影变换。这一步要对顶点应用指定的投影（正投影或者透视投影），并把指定的裁剪体变换成规范化的裁剪体。后几节将详细讨论这些步骤，不过在此处回忆一下OpenGL中默认的照相机是有好处的。如果不调用任何观察函数，OpenGL使用的照相机就是默认照相机。

OpenGL的默认照相机位于对象标架的原点，方向指向z轴的负方向。该照相机生成的是正交投影图。默认的视见体是中心位于原点并且边长为2的立方体。默认的投影平面为$z=0$，投影方向沿着z轴。因此，在这个视见体内的对象是可见的，于是可以在投影平面上成像，如图5.11所示。到目前为止，基于对这个默认照相机的了解，我们回避了观察过程中所有复杂的步骤。这样，在应用程序中，可以把对象定义在这个视见体内，并且我们知道这些对象会出现在生成的图像中。这种方法没有修改默认的模-视变换矩阵和投影变换矩阵，所以这两个矩阵仍然是初始时的单位矩阵。

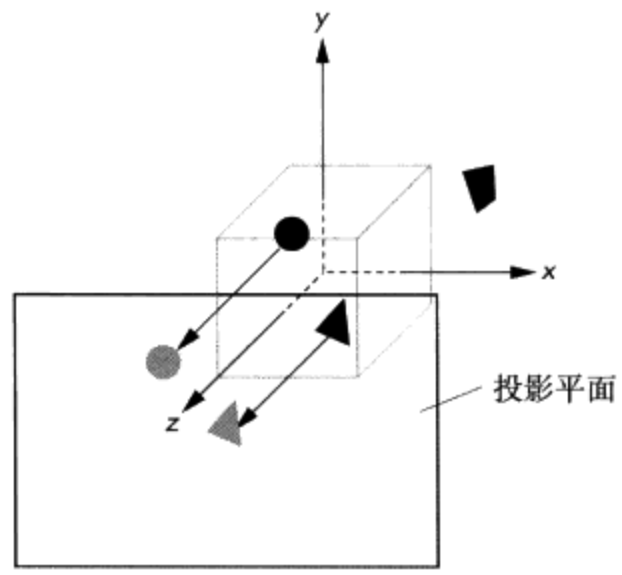


图5.11　使用默认照相机成像

接下来，我们通过旋转和平移改变初始时为单位阵的模-视变换矩阵，这样可以把照相机移动到想要的位置。我们在glOrtho中设置的参数可以改变初始时同为单位阵的投影变换矩阵，使得任意长方体内的对象可以被成像。在这一章，通过用模-视变换矩阵设置照相机的位置和方向并用投影变换矩阵生成正投影图和透视投影图，可以得到更多类型的投影图。

5.3　定位照相机

这一节讨论如何使照相机具有指定的位置和方向，而下一节将讨论如何指定想要的投影。虽然这一节把注意力放在OpenGL为三维图形提供的API上，我们也将简要介绍用于指定照相机的其他API。

在OpenGL中，模-视变换矩阵和投影变换矩阵级联起来后被应用到像顶点这样的几何实体。我们已经看到了模-视变换矩阵的一个用处，即把对象放到场景中合适的位置。模-视变换矩阵的另一个用处是从对象标架变换到照相机标架。

5.3.1　照相机标架的定位

第4章已经讲到，指定顶点时可以选择任何单位，并且可以通过一系列重新定位这些顶点的仿射变换定义一个模-视变换矩阵。模-视变换矩阵是建模变换与观察变换的级联。建模变换把对象从对象标架变换到世界标架，而观察变换把世界坐标变换成照相机坐标。因为我们通常不需要处理世界坐标，所以可以使用模-视变换矩阵，而不是分别使用建模矩阵和观察矩阵。

初始的模 – 视变换矩阵是单位阵，因此照相机标架和对象标架是重合的。在 OpenGL 中，初始照相机的方向是 z 轴的负方向（见图 5.12）。在大多数应用程序中，我们在原点附近建模对象，因此默认方位的照相机无法生成包含场景中全部对象的图像。这样，要么把照相机移走，使之远离我们希望成像的对象；要么把对象移到照相机的前面。这两个操作是等价的，因为它们都可以看做是在对象标架下指定照相机标架。

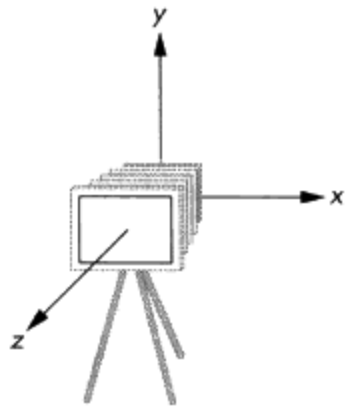


图 5.12　照相机的初始方位

假定初始时使用默认的模 – 视变换矩阵，然后通过函数 glVertex 来指定顶点，这样定义了一些对象。接下来对模 – 视变换矩阵的修改将使对象标架相对于照相机移动，这会影响以后定义的所有对象在照相机中成的像，因为它们的顶点是相对于重定位后的对象标架定义的。与此等价的是：从应用程序控制流程的角度来看，投影变换矩阵和模 – 视变换矩阵都是状态的一部分，因此在定义一个图元的时候，系统当前的状态决定了要应用到这个图元上的矩阵。

考虑如图 5.13 所示的照相机标架和对象标架之间的相对移动。图 5.13(a)所示为初始设置。定义在 **p** 处的顶点在这两个标架下的表示相同。当我们通过一系列变换把模 – 视变换矩阵变成 **C** 以后，对象标架和照相机标架之间的位置关系如图 5.13(b)所示。这两个标架不再相同，但是 **C** 包含了从照相机标架移动到对象标架的信息，或者说，它包含了把照相机从其初始位置（对象标架的原点）移开的信息。这时，如果调用函数 glVertex 定义一个位于 **q** 处的顶点，则该顶点在对象标架下的表示为 **q**，但它在照相机标架下的表示为 **Cq**。OpenGL 通过观察流水线把顶点变换到照相机标架中，并在内部保存顶点在照相机标架下的坐标。与之等价的看法是：照相机仍旧位于自己标架的原点，但对在这个标架下定义的图元要应用模 – 视变换矩阵。在实际编程时，可以采取这两种看法中的任何一种，但是要特别小心对图元的定义是在模 – 视变换矩阵改变之前还是之后。

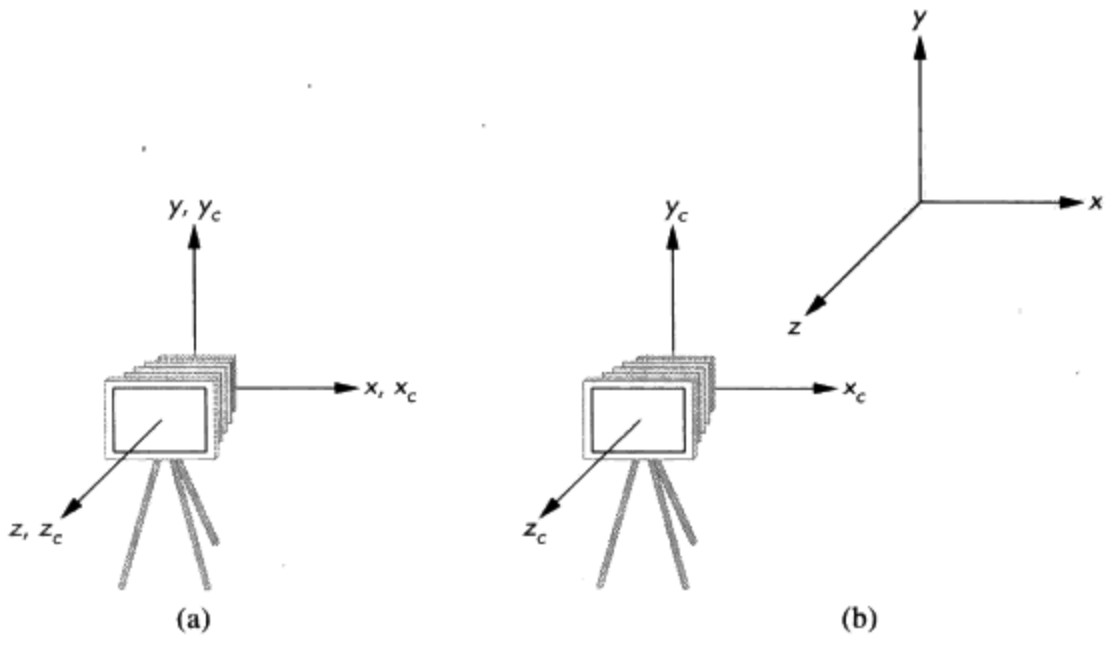


图 5.13　照相机标架和对象标架的相对移动：(a) 初始情况；
(b) 模 – 视变换矩阵改变之后的照相机标架和对象标架

在任何时候，照相机标架和对象标架之间的关系都由模-视变换矩阵的状态决定。虽然把建模变换和观察变换组合成单独一个矩阵开始时可能会导致混淆，但仔细分析之后就会发现这是个好办法。如果把照相机看做是一个几何对象，那么因为变换会改变对象的位置和方向，所以变换也会改变照相机相对于这些对象的位置和方向。

下一个问题是如何通过API指定想要的照相机的方位，以及如何在OpenGL中定位照相机。我们将简单介绍三种方法，本节介绍其中的一种，而5.3.2节和5.3.3节分别介绍另外两种。此外还有两种方法在习题中给出（见习题5.2和习题5.3）。

第一种方法通过应用一系列旋转和平移来改变模-视变换矩阵，从而间接指定照相机的方位。该方法是第4章讨论的实例变换的直接应用，但由于以下两个原因，我们在使用这种方法时必须小心。第一，我们通常希望在定义场景中的任何对象之前定位照相机[①]；第二，应用到照相机的变换所生成的图像看起来和我们期望看到的正好相反。

考虑一个中心在原点的对象。照相机位于初始位置（也在原点），方向指向z轴的负方向。假定想对这个对象朝着z轴正方向的那一面成像，那么必须把照相机从原点移走。如果照相机仍旧指向z轴的负方向，那么应该让照相机沿着z轴正方向后退，因此正确的变换是：

```
glTranslatef(0.0, 0.0, -d);
```

其中d是一个正数。

许多人愿意把这个变换解释成相对于对象标架移动照相机。在经典观察中可以找到这种观点的基础。在计算机图形学中，对象通常被看做是处于一个固定的标架中，为了得到想要的投影图，观察者必须移动到合适的位置。在经典观察中，观察者处于支配地位：从概念上来说，我们通过挑出想要成像的对象并使它处于想要的方位来进行观察。采用经典方法的一个后果是：距离是从观察者到对象来测量的，而不是像大多数物理成像系统那样，从对象到观察者来测量距离。因此，经典观察常常使用左手照相机标架。早期的图形系统遵从经典观察的方法，建模时使用右手坐标，而观察时使用左手坐标，这么做虽然在技术上是正确的，但会使用户感到困惑。在OpenGL中，对于像glOrtho这样的函数，距离是从照相机出发来测量的，虽然这与经典视图一致，但OpenGL保持了右手照相机标架。幸运的是，因为应用程序中涉及最多的是对象标架，应用程序员通常看不到任何内部表示，因而不需要关心这些看待观察的其他观点。

假定我们想从x轴的正半轴观察同一个对象。现在不但要把照相机从对象所在的位置移开，还要绕y轴旋转照相机，如图5.14所示。在绕y轴旋转90°之后必须进行平移。像4.10节讨论的那样，在程序中必须以相反的顺序调用变换函数，所以编写出的代码应该像下面这样：

```
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
glTranslatef(0.0, 0.0, -d);
glRotatef(-90.0, 0.0, 1.0, 0.0);
```

从两个标架的角度来看，我们先相对于照相机标架旋转对象标架，然后通过平移使这两个标架分离。

第2章和第4章曾经通过使用默认的投影变换矩阵来演示简单的三维例子。我们将在5.4节深入研究投影变换矩阵，该矩阵的初始值为单位阵。默认的设置是正投影，照相机位于原点，方向指向z轴的负方向。在第4章的立方体例子中，通过旋转立方体，我们可以看到想要看的面。正如刚才讨论的那样，旋转立方体等价于相对于照相机标架旋转立方体的标架。如果相对于立方体旋转照

① 在动画制作中，把对照相机方位的定义放在程序中的什么地方取决于是想把照相机固定在某个特定对象上，还是想把照相机放在场景中的某个固定位置（见习题5.3）。

相机，我们也可以得到相同的投影图。这种通过对照相机进行平移和旋转来进行观察的方法还可用于生成其他正投影视图，但要生成透视投影图则需要改变投影变换矩阵。

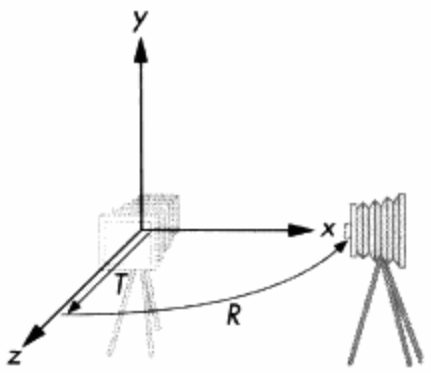


图 5.14 定位照相机

现在考虑如何生成立方体的等轴测投影图。我们还是假定立方体的中心位于原点，并且各棱分别和对应的坐标轴平行。因为默认的照相机位于立方体的中心，所以希望通过平移使得照相机离开立方体。当照相机的位置关于立方体的三个相邻面对称时，例如照相机位于从原点到点(1, 1, 1)的直线上，就可以得到等轴测投影图。我们可以把立方体从照相机的位置移开，然后旋转立方体，由此获得想要的投影图，或者等价地说，把照相机从立方体的位置移开，然后旋转照相机使它对准立方体。

假定初始时使用的是默认照相机。立方体总共有8个等轴测投影图（对每个顶点都有一个），我们可以生成其中的一个。首先绕 x 轴旋转立方体，直到看到两个对称的面，如图5.15(a)所示。很明显，这是旋转立方体 45° 的结果。接下来要绕 y 轴进行第二次旋转。我们旋转立方体直到获得想要的等轴测投影，这需要绕 y 轴旋转 −35.26°。如果读者不知道如何得出这个角度，可以先考虑立方体在经过第一次旋转之后所处的方位。从 z 轴正半轴观察，立方体如图5.15(a)所示，原来位于(−1, 1, 1)的顶点已经旋转到了$(-1, 0, \sqrt{2})$。如果从 x 轴正半轴观察，立方体如图5.15(b)所示，可以看出我们想要做的是把左边的顶点旋转到平面 $x = 0$ 内。所求的旋转角度位于一个两条直角边长度分别为1和$\sqrt{2}$的直角三角形中，所以该角度为 35.26°。不过我们要求顺时针旋转，所以角度应取负值。最后一步是把照相机从原点移开。这样，我们采取的步骤是首先相对于对象标架旋转照相机标架，然后使两个标架分离。经过这些变换后的模 – 视变换矩阵为：

$$\mathbf{M} = \mathbf{T}\mathbf{R}_x\mathbf{R}_y$$

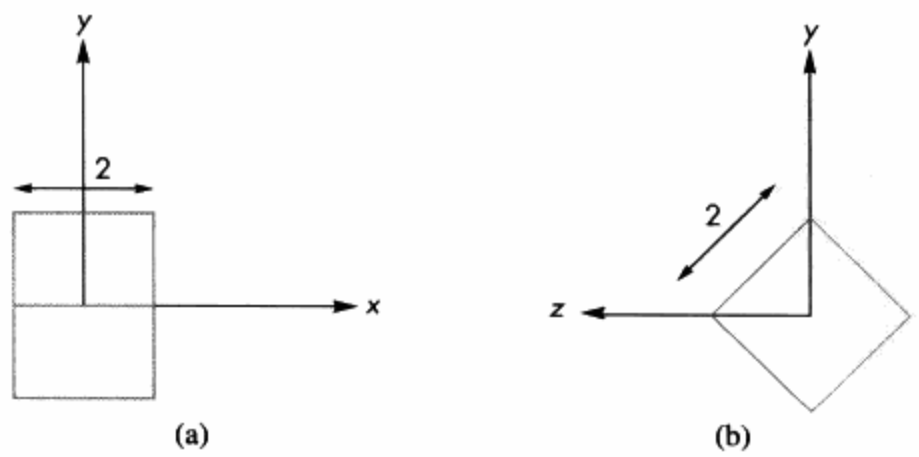


图 5.15 绕 x 轴旋转之后的立方体：(a) 从 z 轴的正半轴观察；(b) 从 x 轴的正半轴观察

把用齐次坐标表示的变换矩阵相乘就可以得出等轴测投影的模–视变换矩阵。两个旋转矩阵的乘积为：

$$\mathbf{R}=\mathbf{R}_x\mathbf{R}_y=\begin{bmatrix}1&0&0&0\\0&\sqrt{6}/3&-\sqrt{3}/3&0\\0&\sqrt{3}/3&\sqrt{6}/3&0\\0&0&0&1\end{bmatrix}\begin{bmatrix}\sqrt{2}/2&0&\sqrt{2}/2&0\\0&1&0&0\\-\sqrt{2}/2&0&\sqrt{2}/2&0\\0&0&0&1\end{bmatrix}$$

$$=\begin{bmatrix}\sqrt{2}/2&0&\sqrt{2}/2&0\\\sqrt{6}/6&\sqrt{6}/3&-\sqrt{6}/6&0\\-\sqrt{3}/3&\sqrt{3}/3&\sqrt{3}/3&0\\0&0&0&1\end{bmatrix}$$

容易验证原来位于(−1, 1, 1)的顶点被这个矩阵正确地变换到$(0, 0, \sqrt{3})$。再把平移$(0, 0, -d)$所对应的矩阵和这个旋转矩阵级联，可得：

$$\mathbf{TR}=\begin{bmatrix}\sqrt{2}/2&0&\sqrt{2}/2&0\\\sqrt{6}/6&\sqrt{6}/3&-\sqrt{6}/6&0\\-\sqrt{3}/3&\sqrt{3}/3&\sqrt{3}/3&-d\\0&0&0&1\end{bmatrix}$$

在OpenGL中，设置模－视变换矩阵的代码如下：

```
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
glTranslatef(0.0, 0.0, -d);
glRotatef(35.26, 1.0, 0.0, 0.0);
glRotatef(45.0, 0.0, 1.0, 0.0);
```

注意，由glOrtho设置的裁剪体是相对于照相机标架的。因此，对于正投影而言，照相机的平移不影响任何对象成像的大小，但会影响对象是否被裁剪，因为裁剪体是相对于照相机指定的。

5.3.2　一种观察API

上面为等轴测投影构造模－视变换矩阵的方法有些不太令人满意。虽然这种方法比较直观，但是它要求在指定变换之前计算各种角度，这对于应用程序来说不是一个好的接口。我们可以采取另一种方法来定位照相机，该方法类似于PHIGS和GKS-3D所使用的方法，PHIGS和GKS-3D是早期的两个标准三维图形API。我们仍然在对象标架下描述照相机的方位，这是观察过程的第一个步骤。在第二个步骤里我们指定想要获得的投影类型（透视或者平行），这等价于在OpenGL中设置投影变换矩阵。该步骤经常被称为规范化变换（normalization transformation），我们把它当做标架的变换来处理。照相机初始时仍然位于原点，方向指向z轴的负方向。我们想把照相机放到一个称为观察参考点（view-reference point，VRP）（见图5.16）的位置。VRP是在建模标架下给定的，用户通过调用像下面这样的函数来指定这个位置：

```
set_view_reference_point(x, y, z);
```

接下来指定照相机的方向，可以把这个指定分成两部分：指定观察平面法向量（view-plane normal，VPN）和指定观察正向向量（view-up vector，VUP）。VPN（图5.16中的$\mathbf{n}$）给出了投影平面的方向，或者说给出了照相机胶片平面的方向。一个平面的方向是由其法向量决定的，因此API包含一个像下面这样的函数来指定观察平面：

```
set_view_plane_normal(nx, ny, nz);
```

投影平面的方向并没有确定在照相机看来哪个方向是向上的方向。如果只指定VPN，我们还可以旋转照相机，同时保持它的胶片平面不变。如果还指定了VUP，照相机就完全确定下来了。用户通过调用像下面这样的函数来指定VUP：

```
set_view_up(vup_x, vup_y, vup_z);
```

为了获得方向向上的向量**v**，我们把VUP向量投影到观察平面（见图5.17）。借助投影，用户不需要计算出一个位于投影平面内的向量，任何不平行于**v**的向量也可以被指定为VUP。向量**v**与**n**正交。我们可以用叉积获得与这两个向量都正交的第三个向量**u**。这个新的正交坐标系通常被称为观察坐标系（viewing-coordinate system）或者**uvn**坐标系（u-v-n system）。观察坐标系再加上VRP就是我们想要的观察标架。从世界标架到观察标架的变换矩阵是观察方位矩阵（view orientation matrix），它和模－视变换矩阵中观察变换对应的部分等价。

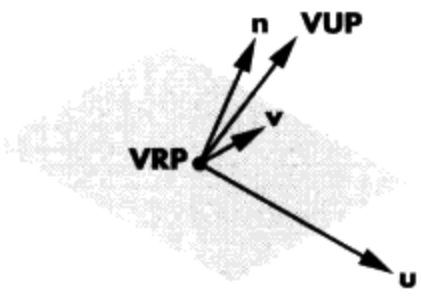


图5.16 观察标架

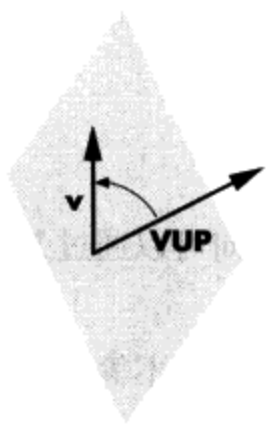


图5.17 观察正向向量的确定

我们可以利用旋转和平移的齐次坐标变换矩阵推导出观察方位矩阵。先指定观察参考点、观察平面法线和观察正向向量。设观察参考点为：

$$\mathbf{p} = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

观察平面法向量为：

$$\mathbf{n} = \begin{bmatrix} n_x \\ n_y \\ n_z \\ 0 \end{bmatrix}$$

观察正向向量为：

$$\mathbf{v_{up}} = \begin{bmatrix} v_{up_x} \\ v_{up_y} \\ v_{up_z} \\ 0 \end{bmatrix}$$

我们构造一个新的标架，它的原点是观察参考点，一个坐标轴的方向为观察平面法向量，另两个坐标轴的方向分别为**u**和**v**。在默认情况下，原先的x轴、y轴和z轴分别就是u轴、v轴和n轴，这对应于OpenGL中默认的模－视变换矩阵。观察参考点可以通过一个简单的平移$\mathbf{T}(-x, -y, -z)$来

处理，这个平移从观察标架到原先的原点。观察方位矩阵的剩余部分是由一个旋转确定的，所以观察方位矩阵 **V** 可以写成下面的形式：

$$\mathbf{V} = \mathbf{TR}$$

方向 **v** 必须和 **n** 正交，因此，

$$\mathbf{n} \cdot \mathbf{v} = 0$$

从图 5.16 可以看出，**v** 位于由 **n** 和 $\mathbf{v}_{up}$ 所确定的平面内，因而必定是这两个向量的线性组合：

$$\mathbf{v} = \alpha\mathbf{n} + \beta\mathbf{v}_{up}$$

如果暂时忽略向量的长度，那么可以令 $\beta = 1$，由此求得：

$$\alpha = -\frac{\mathbf{v}_{up} \cdot \mathbf{n}}{\mathbf{n} \cdot \mathbf{n}}$$

和

$$\mathbf{v} = \mathbf{v}_{up} - \frac{\mathbf{v}_{up} \cdot \mathbf{n}}{\mathbf{n} \cdot \mathbf{n}} \mathbf{n}$$

第三个正交的方向可以通过叉积得到：

$$\mathbf{u} = \mathbf{v} \times \mathbf{n}$$

这些向量的长度通常不等于1。可以分别对它们进行归一化，得到三个单位长度的向量 **u′**、**v′** 和 **n′**。矩阵

$$\mathbf{A} = \begin{bmatrix} u'_x & v'_x & n'_x & 0 \\ u'_y & v'_y & n'_y & 0 \\ u'_z & v'_z & n'_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

是一个旋转矩阵，利用它可以求出 **u′v′n′** 坐标系中的一个向量在原来坐标系下的表示。然而，我们真正要求的与此相反：我们想得到原来坐标系中的向量在 **u′v′n′** 坐标系下的表示。这样，待求的矩阵 **R** 等于 $\mathbf{A}^{-1}$，但因为 **A** 是旋转矩阵，所以，

$$\mathbf{R} = \mathbf{A}^{-1} = \mathbf{A}^{T}$$

最后再把 **R** 和平移矩阵 **T** 相乘，得到：

$$\mathbf{V} = \mathbf{RT} = \begin{bmatrix} u'_x & u'_y & u'_z & -xu'_x - yu'_y - zu'_z \\ v'_x & v'_y & v'_z & -xv'_x - yv'_y - zv'_z \\ n'_x & n'_y & n'_z & -xn'_x - yn'_y - zn'_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

注意，这里是用平移矩阵右乘 **R**，而上一节推导的结果是用平移矩阵左乘 **R**。对这个差异的一种解释是：在上一节的推导中，首先旋转其中一个标架，然后沿着一个方向使这两个标架分离，这个方向是在观察标架下表示的；而在这里的推导中，照相机的位置是在建模标架下表示的。对这个差异的另一种解释是：矩阵 **RT** 和 **TR** 的形式是相似的。矩阵乘积的旋转部分（左上角 3 × 3 子矩阵）相同，而且最下面一行也相同。最右边一列的前三个元素不同，这是因为旋转矩阵改变 **RT** 中的平移系数但不改变 **TR** 中的平移系数。对上一节中的等轴测投影的例子

$$\mathbf{n} = \begin{bmatrix} -1 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

$$\mathbf{v_{up}} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

照相机位于原来标架中的对角线上。如果令：

$$\mathbf{p} = \frac{\sqrt{3}}{3} \begin{bmatrix} -d \\ d \\ d \\ 1 \end{bmatrix}$$

我们就得到了和5.3.1节相同的模－视变换矩阵。

5.3.3 LookAt函数

通过指定VRP、VPN和VUP可以定位照相机，但这只是为定位照相机提供API的方法之一。在许多情况下我们愿意采用一种更直接的方法。考虑如图5.18所示的情景，其中照相机位于点**e**，该点称为视点（eye point），是在建模标架下指定的，照相机的方向指向另一个点**a**，该点称为参考点（at point）。这两个点确定了VPN和VRP。对视点和参考点这两个点做减法，所得的向量就给出了VPN：

$$\mathbf{vpn} = \mathbf{a} - \mathbf{e}$$

因为观察参考点就是视点，所以只需要再对照相机指定想要的观察正向。OpenGL的glu工具库提供了下面的函数：

```
void gluLookAt(
          GLdouble eyex, GLdouble eyey, GLdouble eyez, /* 视点 */
          GLdouble atx, GLdouble aty, GLdouble atx, /* 参考点 */
          GLdouble upx, GLdouble upy, GLdouble upz) /* 观察正向 */
```

利用这个函数就可以按照本节所讲的方式来定位照相机，因此通常使用下面的代码：

```
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
gluLookAt(eyex,eyey,eyez,
          atx, aty, atx,
          upx, upy, upz);

/* 这里是定义对象的代码 */
```

注意，我们可以把标准的旋转、平移和缩放作为定义对象的一部分。尽管这些变换也会改变模－视变换矩阵，但可以认为：gluLookAt是在定位照相机，而接下来影响模－视变换矩阵的那些操作是在定位对象，采取这种看法常常会在概念上带来方便。

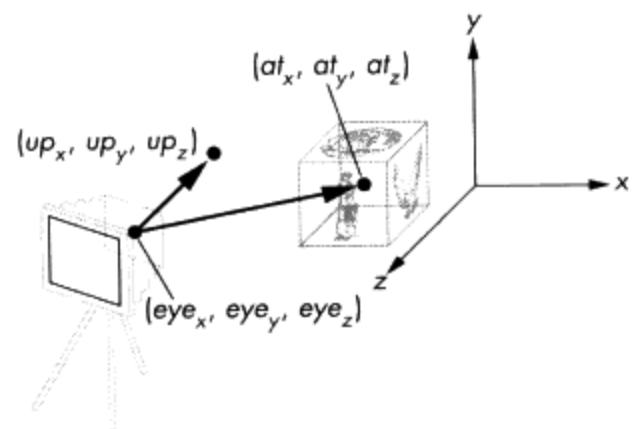


图 5.18　用 LookAt 函数定位照相机

5.3.4　其他观察 API

在许多应用程序中，前面已经介绍过的观察接口都不合适。考虑一个飞行模拟器，飞行员通常用三个角度，即滚转角（roll）、俯仰角（pitch）和偏航角（yaw）来指定方向。在定义这些角度时，基准点是飞行器的质心，基准方向是飞行器机身的三个轴向，如图5.19所示。这样，飞行员用这三个角度和从对象到飞行器质心的距离来指定对象的投影图。根据这三个角度和对象到飞行器质心的距离，可以用一个平移和三个简单的旋转构造出观察变换（见习题5.2）。

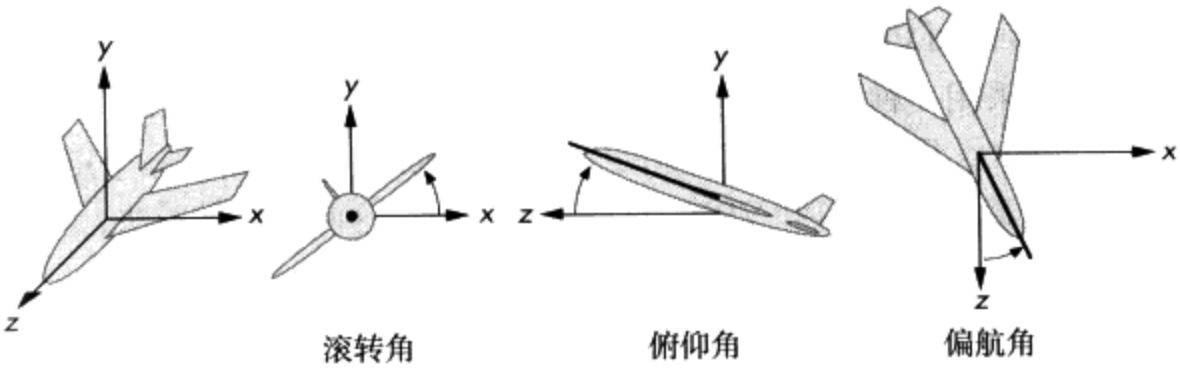


图 5.19　滚转角、俯仰角和偏航角

在许多应用程序中，指定观察最自然的方式是通过极坐标，而不是直角坐标。有的应用程序涉及一些对象绕另一些对象的旋转，这样的应用程序就属于此类。例如，考虑在天空中指定一颗恒星，从观察者看来，这颗恒星的方向由仰角和方位角给出（见图5.20）。假定观察者站在一个平面上，仰角（elevation）是在该平面之上恒星出现的角度。在观察者的位置定义一个法向量，并利用该法向量定义一个平面，这样不论观察者是否站在一个平面上，我们都可以定义仰角。在这个平面内可以定义另两个坐标轴，这样就构造出了一个观察坐标系。方位角（azimuth）是从该平面内的一个坐标轴旋转到观察者和恒星连线在该平面内投影的角度。照相机还可以绕它所指向的方向旋转，这个旋转的角度称为扭转角（twist angle）。

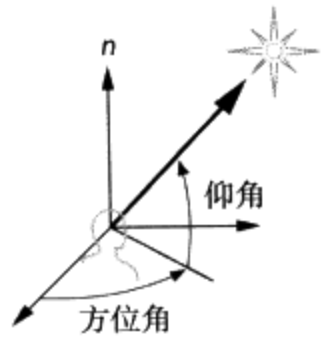


图 5.20　仰角和方位角

5.4　简单的投影

对于实际的照相机，在确定它的方位之后还必须选择透镜。在第 1 章已经看到，是透镜与胶片的尺寸（或者照相机后面的大小）共同决定了在照相机面前的场景中有多大部分被成像。与此等价，在计算机图形学中可以选择投影类型和观察参数。

对实际的照相机，若选择广角透镜则可获得最显著的透视投影效果：离照相机越近的对象所成的像越大。用长焦透镜获得的图像接近于平行投影图，而且照片中的对象看上去似乎是平的。大多数 API 为透视投影和平行投影提供不同的函数。OpenGL 也是如此，不过这两种投影可以使用相同的绘制流水线来实现，这一点将在 5.8 节和 5.9 节讲到。

正如对模 – 视变换矩阵所做的那样，可以用函数 glLoadMatrix 设置投影变换矩阵。此外，还可以用 OpenGL 函数设置最常用的观察条件。下面先考虑和投影有关的数学。借助齐次坐标，我们可以用一个 4 × 4 矩阵描述一个特定的投影。

5.4.1　透视投影

假定照相机位于观察标架的原点，方向指向 z 轴的负方向。图 5.21 给出了两种可能的情况。在图 5.21(a)中，照相机的后面垂直于 z 轴，并且平行于透镜。大多数物理成像设备都属于这种情况，包括人类视觉系统和简单的照相机。更一般的情况如图5.21(b)所示，此时照相机后面和前面之间的夹角可以是任意的。下面将详细讨论第一种情况，因为它更简单。不过，对简单情况的推导步骤也适用于一般情况，所以在讨论了简单的情况之后，对一般情况的推导就是一个直接的练习了（见习题 5.6）。

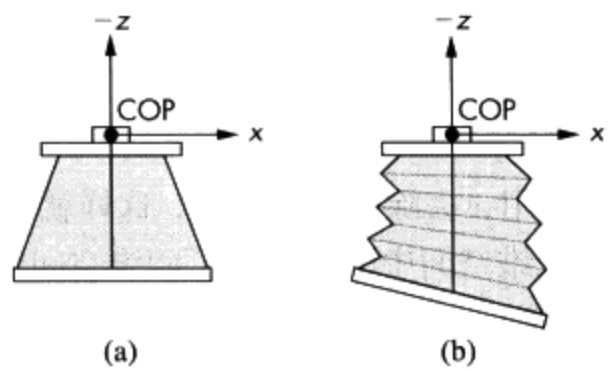


图 5.21　两个照相机：(a) 后面和前面平行；(b) 后面和前面不平行

第 1 章已经讲到，可以把投影平面放到投影中心的前面。如果对图 5.21(a)中的照相机也这么做，那么此时的透视投影如图 5.22 所示。空间中的一点(x, y, z)沿着一条投影线被投影到(x_p, y_p, z_p)。所有投影线都通过原点。因为投影平面和 z 轴垂直，所以

$$z_p = d$$

因为照相机指向 z 轴的负方向，所以投影平面位于 z 轴负方向一侧，并且 d 的值是负的。

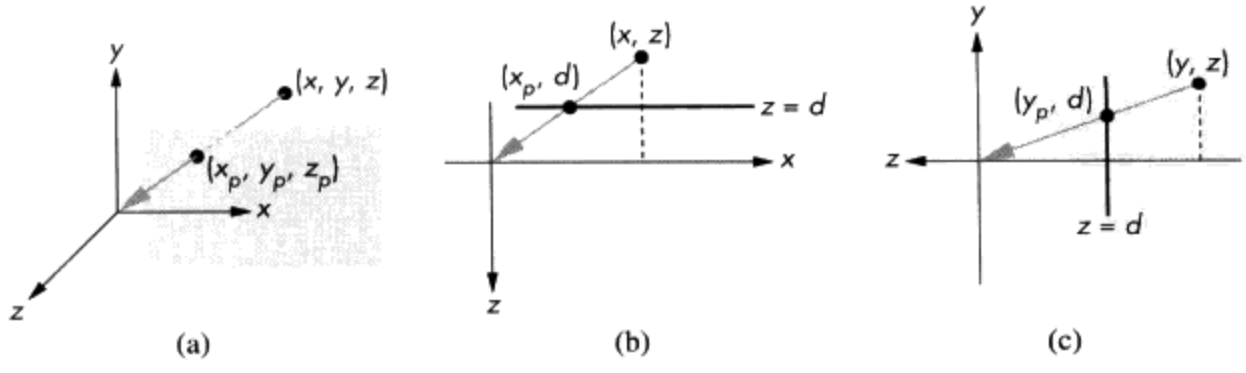


图 5.22　透视投影的三个视图：(a) 三维视图；(b) 俯视图；(c) 侧视图

从图5.22(b)所示的俯视图中可以找出两个相似三角形，它们的公共角的正切值相等，因此

$$\frac{x}{z} = \frac{x_p}{d}$$

于是

$$x_p = \frac{x}{z/d}$$

利用图5.22(c)所示的侧视图可以得到关于y_p的类似结果：

$$y_p = \frac{y}{z/d}$$

这些方程都是非线性的。用z去除导致了非均匀的透视缩短（nonuniform foreshortening）：对象离投影中心越远，所成的像就越小。

我们可以把投影过程看做是一个变换，它把点(x, y, z)映射成了另一个点(x_p, y_p, z_p)。尽管这种透视变换（perspective transformation）把直线仍变换成直线，但它不是仿射变换。透视变换也不是可逆的：因为一条投影线上所有的点都会投影到同一个点，我们不能从一个点的投影恢复出这个点。5.8节和5.9节将讨论在OpenGL中使用的透视变换的一种可逆形式，这种形式的透视变换保留了隐藏面消除中需要用到的距离。

齐次坐标也可以用于投影变换。在前面介绍齐次坐标时，我们把三维空间中的点(x, y, z)表示成四维空间中的点$(x, y, z, 1)$。现在对其进行推广，用更一般的四维点

$$\mathbf{p} = \begin{bmatrix} wx \\ wy \\ wz \\ w \end{bmatrix}$$

来表示(x, y, z)。只要$w \neq 0$，就可以用w去除前三个分量，这样就能从四维表示恢复出原来的三维点。在这种齐次坐标的新形式中，三维空间中的点变成了四维空间中经过原点的直线。我们仍然用4×4矩阵来表示变换，但是矩阵的最后一行可能和以前不同，于是这样的变换可能会改变w的值。

很明显，我们更愿意保持$w = 1$，否则为了恢复三维点就要做除法。然而，通过允许w改变，可以表示包括透视投影在内的更多的变换。考虑下面的矩阵：

$$\mathbf{M} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1/d & 0 \end{bmatrix}$$

矩阵**M**把点

$$\mathbf{p} = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

变换成点

$$\mathbf{q} = \begin{bmatrix} x \\ y \\ z \\ z/d \end{bmatrix}$$

初看起来**q**似乎没有意义。然而，不要忘记为了回到原来的三维空间，必须用第4个分量去除前三个分量，于是可得：

$$x_p = \frac{x}{z/d}$$

$$y_p = \frac{y}{z/d}$$

$$z_p = \frac{z}{z/d} = d$$

这些等式表示了一个简单的透视投影。在齐次坐标表示中，用w分量去除**q**得到的点

$$\mathbf{q}' = \begin{bmatrix} \frac{x}{z/d} \\ \frac{y}{z/d} \\ d \\ 1 \end{bmatrix} = \begin{bmatrix} x_p \\ y_p \\ z_p \\ 1 \end{bmatrix}$$

与**q**等价。

我们已经看到，通过定义一个在模-视变换矩阵之后应用的4 × 4投影变换矩阵，至少可以实现一个简单的透视投影。然而，我们必须在结束时执行透视除法（perspective division）。透视除法可以作为绘制流水线的一部分，如图5.23所示。

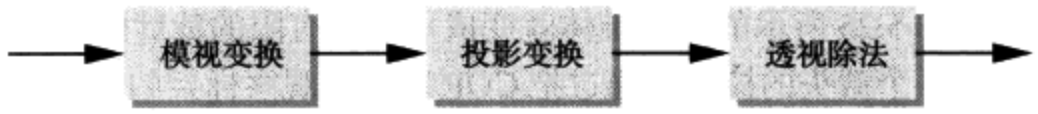


图5.23 投影流水线

5.4.2 正交投影

正交投影或者说正投影（orthographic projection）是平行投影的一种特殊情形：正交投影的投影线垂直于观察平面。如果用照相机实现正交投影，那么这个照相机的胶片平面应该平行于透镜，并且透镜的焦距无限大。无须利用当COP移动到无穷远时的透视投影和平行投影的关系，我们可以直接推导出平行投影的方程。在图5.24所示的正交投影中，投影平面为$z = 0$。当点被投影到这个平面时，它的x和y坐标保持不变，因此平行投影的方程为：

$$x_p = x$$
$$y_p = y$$
$$z_p = 0$$

利用齐次坐标，这些等式可以表示为：

$$\begin{bmatrix} x_p \\ y_p \\ z_p \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

这里不需要进行除法运算，但是在用硬件实现时，对透视投影和正交投影可以使用相同的绘制流水线。

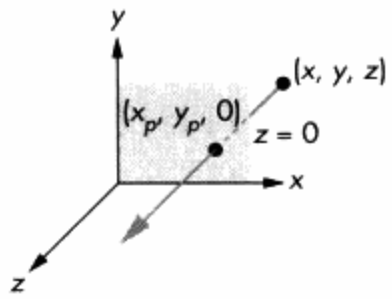


图 5.24　正交投影

上面已经推导出了两个简单的投影变换矩阵。对于一般的透视投影和平行投影，可以在应用这两个简单的投影变换矩阵之前，通过一系列变换把一般的情况转换为这两种简单情况之一。在推导一般的投影变换矩阵之前，我们先来看看 OpenGL 为应用程序编程人员指定投影所提供的 API。

5.5　OpenGL 中的投影

5.4 节推导投影变换矩阵时没有考虑照相机的性质（透镜的焦距或者胶片平面的尺寸）。图 5.25 所示是一个简单针孔照相机的视角（angle of view），这个针孔照相机和第 1 章讨论的类似。只有那些位于照相机视角范围内的对象才会被成像。如果照相机的后面是矩形，那么只有处于一个半无穷锥体内的对象才会被成像，这个半无穷锥体称为视见体（view volume），它的顶点位于 COP。这个视见体之外的对象从场景中裁剪（clip）掉了，因此前面对简单投影的描述并不完备，我们还没有考虑裁剪。

在大多数图形 API 中，指定投影函数的同时也指定裁剪参数。如果除了视角以外，再增加前裁剪面和后裁剪面，那么图 5.25 所示的无穷锥体就变成了一个有限的裁剪体，如图 5.26 所示。这个视见体是一个棱台，即一个截头的棱锥。通过把 COP 定义在观察标架的原点，我们只确定了一个参数。原则上，应该可以任意指定棱台的 6 个面。然而，如果以这种方式定义视见体，在应用程序中指定观察会比较困难，而且会使实现复杂化。在实际应用中，我们很少需要这种灵活性。OpenGL API 只提供了两个透视观察函数，其他 API 提供的函数和 OpenGL 有所不同，但也有类似的限制。

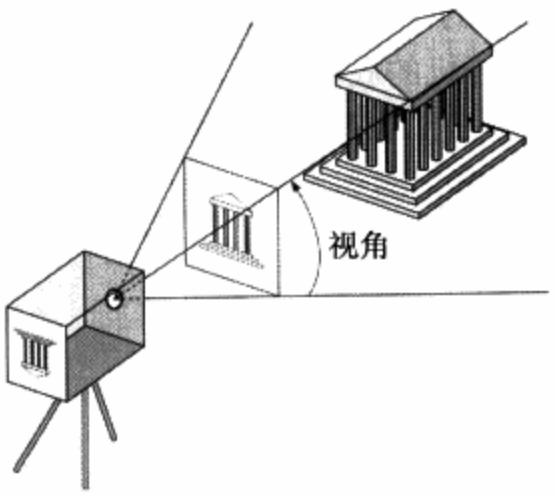


图 5.25　视见体

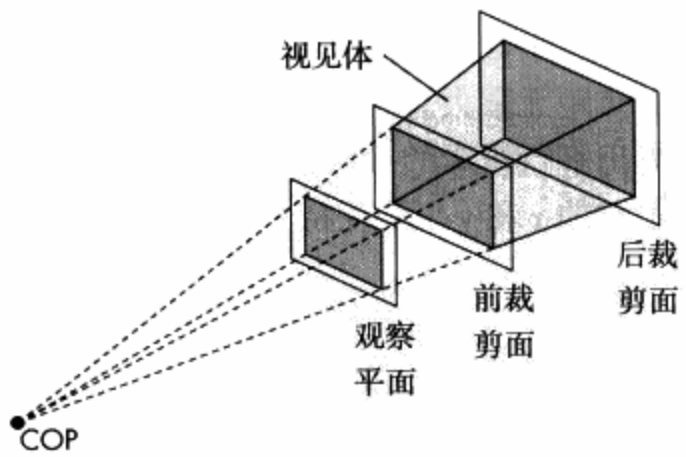


图 5.26　前裁剪面和后裁剪面

请读者注意，在 OpenGL 中，定位照相机的函数（如 gluLookAt 函数）会改变模 – 视变换矩阵并且其参数是用建模坐标给出的，而现在要介绍的函数将改变投影变换矩阵，这些函数的参数要用观察坐标给出。

5.5.1 OpenGL中的透视投影

OpenGL为指定透视投影提供了两个函数，为指定平行投影提供了一个函数。我们还可以通过加载，或者通过对初始的单位阵应用旋转、平移和缩放变换来直接生成投影变换矩阵。下面的函数可以用于指定透视投影：

```
glFrustum(GLdouble left, GLdouble right, GLdouble bottom, GLdouble top,
          GLdouble near, GLdouble far)
```

在观察标架下，这些参数的意义如图5.27所示。near和far分别是从COP（观察标架的原点）到前裁剪面和后裁剪面的距离，这两个裁剪面都平行于平面$z=0$。因为照相机的方向指向z轴的负方向，所以前（近）裁剪面的方程为$z=-\text{near}$，后（远）裁剪面的方程为$z=-\text{far}$。left、right、top和bottom这4个参数是在近（前）裁剪面内指定的。从COP顺着照相机指向的方向看去，平面$x=\text{left}$在照相机的左边。参数right、bottom和top的意义与left相似。尽管在几乎所有的应用程序中都有far > near > 0，但只要near ≠ far，由这些参数确定的投影变换矩阵就是有效的，不过在投影中心（原点）后面的对象所成的像将是倒立的，如果该对象位于近裁剪面和远裁剪面之间的话。

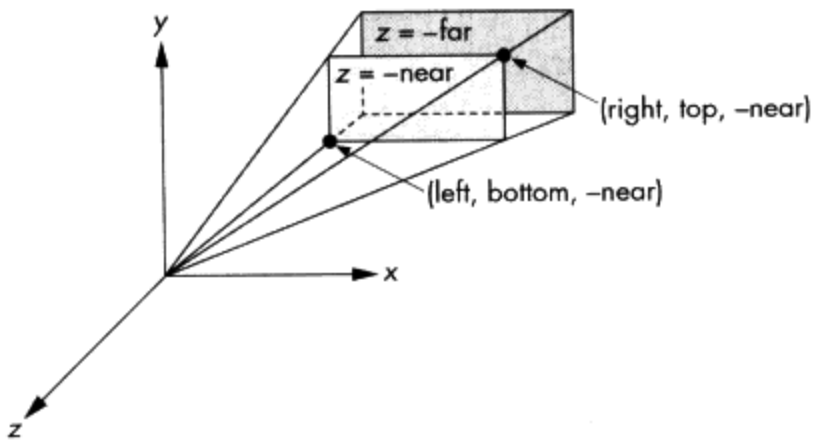


图5.27 指定一个棱台视见体

由函数glFrustum确定的投影变换矩阵将和当前的矩阵相乘，因此必须先选择矩阵模式。典型的代码如下：

```
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
glFrustum(left, right, bottom, top, near, far);
```

注意，这些参数不一定非要关于z轴对称，于是所确定的视见体也不一定关于z轴对称（正棱台）。在5.9节，读者会看到如何利用5.4节得出的简单投影变换矩阵推导出这个函数所确定的投影变换矩阵。

在许多应用程序中，通过视角或者视域来指定视见体是非常自然的。然而，如果投影平面是矩形而不是正方形，那么在俯视图和侧视图中所看到的视角是不同的（见图5.28）。角度fov是裁剪体的底面和顶面之间的夹角。OpenGL工具库中的函数

```
gluPerspective(GLdouble fovy, GLdouble aspect,
GLdouble near, GLdouble far)
```

允许用户指定沿y轴方向的视角和投影平面的宽高比。对近裁剪面和远裁剪面的指定和glFrustum一样。由这个函数确定的投影变换矩阵也会改变当前矩阵，所以在调用这个函数之前也得选择矩阵模式，并且通常要加载单位矩阵。

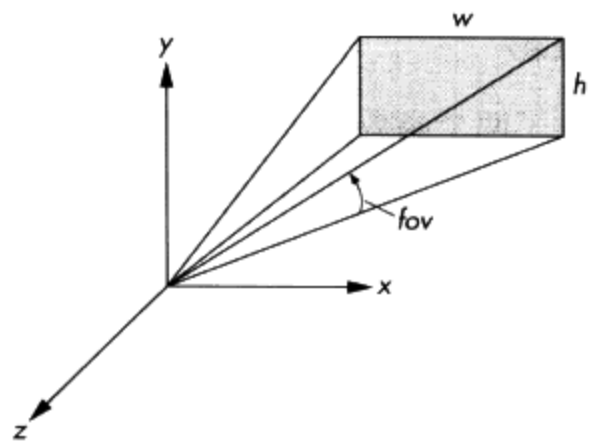


图 5.28　利用视域指定视见体

5.5.2　OpenGL 中的平行投影

OpenGL 提供的唯一一个平行投影观察函数是正交投影（正投影）观察函数：

```
glOrtho(GLdouble left, GLdouble right, GLdouble bottom, GLdouble top,
        GLdouble near, GLdouble far)
```

它的参数和 glFrustum 相同。这个函数所指定的视见体是一个长方体，如图 5.29 所示。近裁剪面和远裁剪面仍旧分别是 $z = -\text{near}$ 和 $z = -\text{far}$。

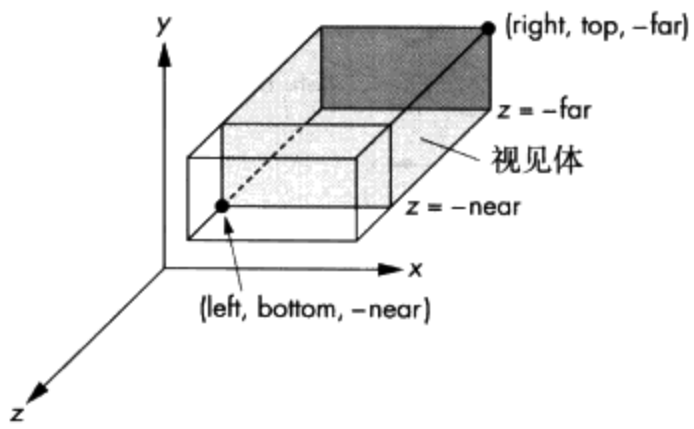


图 5.29　正投影

5.6　隐藏面消除

回过头看 4.10 节中的旋转立方体程序，现在可以在其中增加透视观察并允许照相机移动。我们先通过前面对观察的讨论来理解隐藏面消除，该技术在程序的第一个版本中用到过。如果一个立方体的各面不透明，那么只能看到它的三个面向前方的面。从基本的观察模型的角度来看，只有这些面被看到是因为它们挡住了投影线，使投影线不能到达任何其他的面。

不过，从计算机图形学的角度来看，立方体的所有 6 个面都已经被定义，并且这 6 个面都经过了绘制流水线的处理。因此，图形系统必须关心它所显示的是哪个面。从概念上来说，我们要寻求这样的算法，它们或者去掉观察者看不到的表面，此时称为隐藏面消除算法（hidden-surface-removal algorithm）；或者找出可以看到的表面，此时称为可见表面算法（visible-surface algorithm）。人们已经提出了许多隐藏面消除的方法，第7章将讨论其中的几种方法。OpenGL实现了一种特定的算法，即 z 缓存算法（z-buffer algorithm），并为用户使用该算法提供了三个函数。这里将简单介绍一下 z 缓存算法，到第 7 章还要给出进一步的讨论。

隐藏面消除算法可以分为两大类。对象空间算法（object-space algorithm）试图对场景中对象的表面排序，使得按照所排列的顺序来绘制表面可以得到正确的图像。例如，对于立方体，如果先绘制面向后方的面，那么后绘制的面向前方的面就会把它们覆盖，从而得到正确的图像。这类算法

不能很好地与绘制流水线相结合，因为在流水线结构中，对象通过绘制流水线的顺序是任意的。为了确定绘制对象的正确顺序，图形系统必须能够获取所有对象的信息，这样才能把它们按照从后到前的顺序排列。

图像空间算法（image-space algorithm）可以作为投影过程的一部分，这类算法试图确定每一条投影线与对象表面形成的所有交点之间的关系。z缓存算法是一种图像空间算法，它很适合在大多数图形系统的绘制流水线中实现，因为在绘制每个对象时可以保存部分信息。

z缓存算法的基本思想如图5.30所示。从COP出发的一条投影线和两个表面相交。因为圆比三角形离观察者更近，所以如果这条投影线与投影平面的交点对应于颜色缓存中的像素P，那么P的颜色应该由圆的颜色来决定。困难在于，当三角形和圆以任意的先后顺序通过绘制流水线时，如何按这个想法进行隐藏面消除。

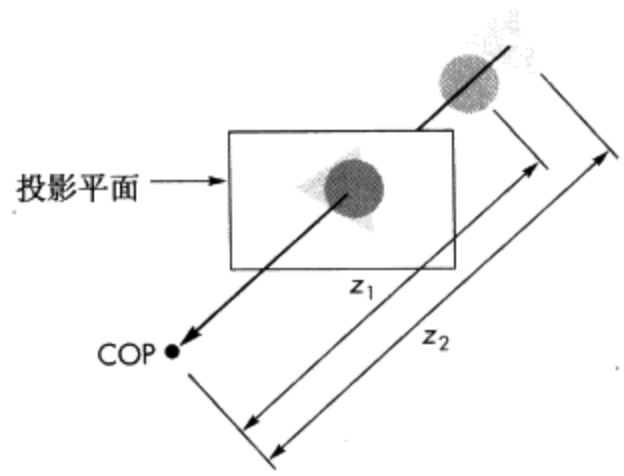


图5.30 z缓存算法

假定所有的对象都是多边形。如果在光栅化多边形时，我们能够保留在每条投影线上从COP或者投影平面到已经绘制的最近点的距离，就能在投影和填充随后绘制的多边形时更新这个信息。最终的结果是：我们只显示每条投影线上最近的点。为了在光栅化多边形时存储必要的深度信息，该算法需要一个深度缓存（depth buffer）或者称为z缓存（z buffer）。因为必须对颜色缓存中的每个像素都保存深度信息，所以z缓存的空间分辨率和颜色缓存相同。z缓存的深度对应于所支持的深度分辨率，通常是32位，现在的图形卡以浮点数的形式存储深度信息。z缓存是构成帧缓存的众多缓存之一，通常是图形卡存储器的一部分。

深度缓存的初始深度值对应于到观察者最远的距离。当光栅化裁剪体内的每个多边形时，我们计算出每个片元的深度（在多边形上对应的点到观察者的距离）。如果这个深度大于深度缓存中对应于该片元的深度值，那么在对应于这个片元的投影线上，一个已经光栅化的多边形距离观察者更近。因此，对于这个片元，我们忽略当前多边形的颜色，继续考虑这个多边形的下一个片元，对其进行同样的测试。但是如果这个深度小于z缓存中已有的深度值，那么在对应于这个片元的投影线上，正在绘制的多边形要比我们到目前为止已经看到的多边形都更近。因此，我们用这个多边形的颜色来替换颜色缓存中的像素颜色，并且更新z缓存中的深度值[①]。

对于图5.30所示的例子，我们看到如果三角形先通过绘制流水线，那么它的颜色和深度将被保存在颜色缓存和深度缓存中。当圆通过绘制流水线时，在圆与三角形重叠的区域，圆的颜色和深度将替换三角形的颜色和深度。如果先绘制圆，那么它的颜色和深度将保存在颜色缓存和深度缓存中。当绘制三角形时，在它们重叠的区域三角形的深度大于圆的深度，所以颜色缓存和深度缓存中的对应像素不会发生变化。

① 多边形的颜色由明暗处理（见第6章）和纹理映射（如果有关的功能被开启的话，见第8章）确定。

这个算法的主要优点是：它的复杂度正比于光栅化生成的片元数目，并且和不进行隐藏面消除的多边形投影和显示过程相比，实现这个算法只需要增加很少的运算量。第7章还要继续讨论这个问题。

从应用程序编程人员的角度来看，他们必须通过调用下面的函数来初始化深度缓存并开启隐藏面消除功能：

```
glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
glEnable(GL_DEPTH_TEST);
```

这里调用GLUT库中的函数进行初始化，并且在通常的RGB颜色模式和双缓存之外还定义了深度缓存。通过调用下面的函数，程序员可以在必要时为新的绘制清空颜色缓存和深度缓存：

```
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
```

5.6.1　背面剔除

对于像立方体这样的凸对象，法线方向背离观察者的面总是不可见的，因而可以在光栅化之前将其剔除掉。我们可以在OpenGL中调用下面的函数来开启背面剔除功能：

```
glEnable(GL_CULL);
```

然而，背面剔除只有对凸对象才能得到正确的图像。我们经常在使用z缓存算法（对任何多边形的组合都有效）的同时还使用背面剔除。例如，假定场景包含n个立方体，如果我们只使用z缓存算法，那么绘制流水线要处理$6n$个多边形。如果开启背面剔除功能，那么有一半的多边形可以在流水线处理的初期被剔除，因此只有$3n$个多边形需要经过绘制流水线全部阶段的处理。我们将在第7章进一步讨论背面剔除。

5.7　交互式网格显示

现在可以把所学的投影和三维对象建模知识结合起来编写一个交互式应用程序。我们将使用一个简单的网格模型，这个模型具有许多复杂CAD模型的特征。

5.7.1　网格

如果通过改变照相机的参数来响应用户的输入，我们就可以在场景中交互式地漫步。在介绍这个交互式接口之前，先来考虑数据显示的另一个例子：网格图。网格（mesh）是一组具有公共顶点和公共边的多边形。对一般的网格，其中的多边形可以含有任何数目的顶点，如图5.31所示。为了有效地存储和显示这样的网格，可能需要使用中等复杂程度的数据结构。在第2章建模球面时所用的是四边形和三角形网格，这类网格要比一般的多边形网格更容易处理，而且应用也很广泛。这里引入矩形网格来显示高程数据。高程数据确定了一个表面，比如地形。高程数据可能来自一个函数，该函数给出了一个参考值之上的高度，比如海拔高度；还可能来自一个表面上许多点的采样。

假定高度y由下面的函数给出：

$$y = f(x, z)$$

其中，(x, z)是二维表面，例如矩形区域内的点。因此，每一个点(x, z)都对应于一个y，如图5.32所示。有时我们把这样的表面称为2-1/2维表面（2-1/2-dimensional surface）或者高度场（height field）。2-1/2维表面有许多应用，但并不是所有的表面都能以这种形式来表示。例如，如果用一个xz坐标系来确定地球表面的位置，那么就可以用这样一个函数来表示每个位置处的高度。对于许多情形，

例如第2章里讨论的等高线，我们只知道函数f在一些离散点处的取值，于是有如下形式的一组离散采样值或者实验数据的测量值：

$$y_{ij} = f(x_i, z_j)$$

假定这些数据点之间是等间隔的：

$$x_i = x_0 + i\Delta x, \quad i = 0, \cdots, N$$
$$z_j = z_0 + j\Delta z, \quad j = 0, \cdots, M$$

其中，Δx和Δz分别是在x和z方向的采样间隔。如果知道f的解析式，那么可以对其采样以得到一组离散数据。

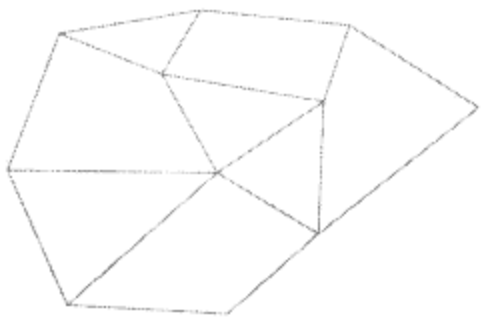


图5.31 网格

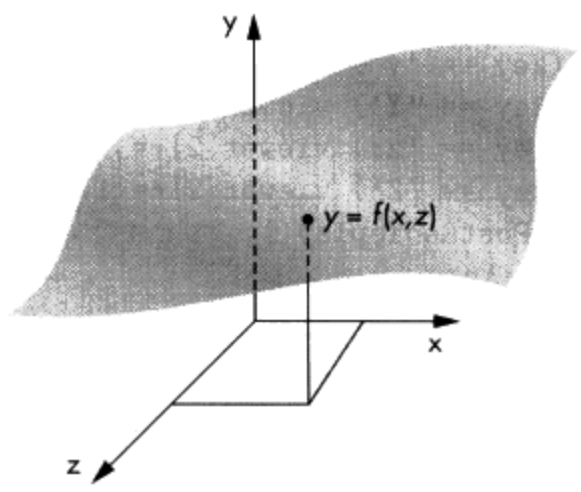


图5.32 高度场

生成曲面的一种简单方法是借助三角形网格或者四边形网格。我们可以用4个点y_{ij}、$y_{i+1,j}$、$y_{i+1,j+1}$和$y_{i,j+1}$生成一个四边形或者两个三角形。因此，上面那些数据定义了NM个四边形或者$2NM$个三角形。绘制这些三角形网格或者四边形网格的OpenGL程序很简单。显示回调函数只需要在遍历高程数据时生成四边形、四边形带或三角形、三角形带。如果生成四边形网格，则显示回调函数的部分代码如下：

```
for(i=0;i<N-1;i++) for(j=1;j<M-1;j++)

    glBegin(GL_QUADS)
      glVertex3i(i,y[i][j],j);
      glVertex3i(i+1,y[i+1][j],j);
      glVertex3i(i+1,y[i+1][j+1],j+1);
      glVertex3i(i,y[i][j+1],j+1);
    glEnd();
```

图5.33给出了一个矩形网格绘制的例子，该图所用的高程数据对应于夏威夷Honolulu的一部分地区。这些高程数据可以从本书的网站上获得。

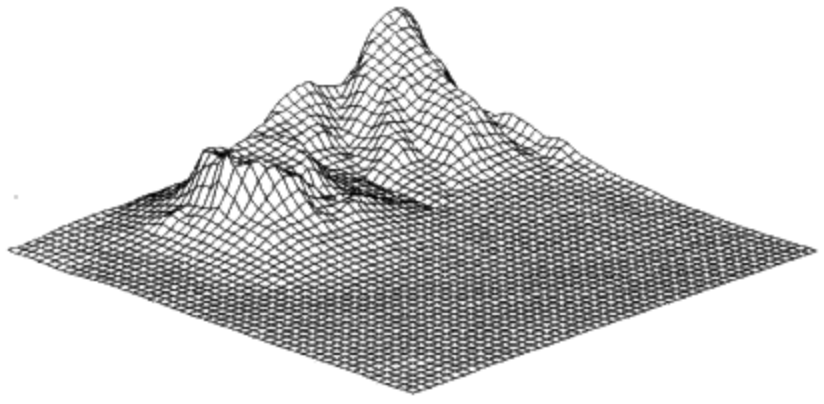


图5.33 Honolulu高程数据的网格图

5.7.2　在场景中漫步

下一步要指定照相机的参数并为程序增加交互功能。这里我们将使用透视观察，并且允许用户通过键盘上的x、X、y、Y、z、Z键来移动照相机，但照相机的方向总是指向立方体的中心。函数gluLookAt提供了一种重新定位照相机的简单方法。

我们需要做的只是对前面4.10节中的程序进行少量修改。定义一个数组viewer[3]来保存照相机的位置，该数组由下面这个简单的键盘回调函数keys来修改：

```
void keys(unsigned char key, int x, int y)
{
   if(key == 'x') viewer[0]-= 1.0;
   if(key == 'X') viewer[0]+= 1.0;
   if(key == 'y') viewer[1]-= 1.0;
   if(key == 'Y') viewer[1]+= 1.0;
   if(key == 'z') viewer[2]-= 1.0;
   if(key == 'Z') viewer[2]+= 1.0;
   glutPostRedisplay();
}
```

在显示回调函数中，把viewer作为照相机的位置并把原点作为参考点来调用gluLookAt。和原先一样，通过鼠标输入来旋转立方体。请读者注意在函数display中那些改变模－视变换矩阵的函数被调用的先后顺序：

```
void display()
{
   glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
   glLoadIdentity();
   gluLookAt(viewer[0],viewer[1],viewer[2],
             0.0, 0.0, 0.0,
             0.0, 1.0, 0.0);
   glRotatef(theta[0], 1.0, 0.0, 0.0);
   glRotatef(theta[1], 0.0, 1.0, 0.0);
   glRotatef(theta[2], 0.0, 0.0, 1.0);

/* 这里是绘制网格或者其他对象的代码 */
   mesh();

   glutSwapBuffers();
}
```

在reshape回调函数中可以调用glFrustum来指定照相机的透镜：

```
void myReshape(int w, int h)
{
   glViewport(0, 0, w, h);
   glMatrixMode(GL_PROJECTION);
   glLoadIdentity();
   if(w<=h) glFrustum(-2.0, 2.0, -2.0 * (GLfloat) h/ (GLfloat)
        w,2.0* (GLfloat) h / (GLfloat) w, 2.0, 20.0);
   else glFrustum(-2.0, 2.0, -2.0 * (GLfloat) w/ (GLfloat) h,
        2.0* (GLfloat) w / (GLfloat) h, 2.0, 20.0);
   glMatrixMode(GL_MODELVIEW);
}
```

注意，这里根据窗口的宽高比来设置glFrustum的参数。除了还要在main函数中注册键盘回调函数以外，程序的其余部分和4.10节的程序相同。完整的程序代码可以在附录A中找到。读者运行这个

程序就会观察到移动照相机以及改变透镜和视见体的各个面所产生的效果。注意，观察当移近网格时投影图的变化。观察者在改变位置时视线总是指向网格的中心，请读者注意观察由此产生的效果。

我们也可以用鼠标按键来移动观察者。可以用鼠标使观察者向前移动，也可以向左转或者向右转（见习题5.14）。不过，通过利用键盘来移动观察者，可以用鼠标来移动对象，就像在第4章的旋转立方体程序中所做的那样。

在这个例子中，我们通过gluLookAt来直接定位照相机。除此之外，还可以用其他方法定位照相机。一种方法是使用平移和旋转矩阵增量式地改变模–视变换矩阵。如果想在场景中移动观察者，但他的观察方向并不总是指向一个固定点，那么这种方法可能更有吸引力。我们还可以在程序中设置一个位置变量，在观察者移动时改变这个变量。此时模–视变换矩阵可以从头计算而不是增量式地改变。选择哪种方法取决于特定的应用和其他一些因素，例如，以增量方式改变模视变换矩阵很多次后可能会引起数值误差积累。

5.7.3 多边形偏移

上面的OpenGL程序有一些有趣的方面，我们可以在这些方面做各种修改。第一，如果使用所有的数据，那么绘制的网格会包含许多非常小的多边形。显示的线条密度太大会使人看起来不舒服，而且可能会表现出moiré条纹。因此，我们可能更愿意对数据进行子采样以获得一组新的采样，使其N和M的值更小。对数据的子采样可以通过每k个点中取一个，或者通过对一组数据点取平均值来实现。

第二，为了生成图5.33，我们不但要把数据点作为黑色的线段绘制一次，还要把数据点作为白色的填充多边形再绘制一次。绘制线段是为了显示网格，绘制多边形是为了让前面的数据遮挡住后面的数据。虽然利用OpenGL的隐藏面消除算法就可以正确地显示多边形，但是因为这些数据可以按一种结构化的顺序来组织，所以不需要执行标准的隐藏面消除过程。如果先绘制后面的数据再绘制前面的数据，那么前面的多边形就自动把后面的多边形隐藏起来了。在利用软件实现隐藏面消除的系统中，按这种方式来显示数据可能会比利用软件实现z缓存算法更快。按这种方式结构化的数据是2-1/2维的，这一点在5.7.1节提到过。虽然对于这种表示数据的特殊结构存在有效的绘制算法，但并不是所有的三维表面都能用这种结构来表示。

在生成图5.33时还使用了一个技巧。如果对同样的数据既绘制多边形又绘制闭合折线，例如执行像下面这样的代码，那么多边形和闭合折线都位于同一个平面：

```
glColor3f(1.0, 1.0, 1.0);
glBegin(GL_QUADS)
  glVertex3i(i,z[i][j],j);
  glVertex3i(i+1,z[i+1][j],j);
  glVertex3i(i+1,z[i+1][j+1],j+1);
  glVertex3i(i,z[i][j+1],j+1);
glEnd();
glColor3f(0.0, 0.0, 0.0);
glBegin(GL_LINE_LOOP)
  glVertex3i(i,z[i][j],j);
  glVertex3i(i+1,z[i+1][j],j);
  glVertex3i(i+1,z[i+1][j+1],j+1);
  glVertex3i(i,z[i][j+1],j+1);
glEnd();
```

由于绘制过程存在数值误差，即使先绘制多边形再绘制闭合折线，由相同顶点定义的多边形也经常会遮挡住闭合折线的一部分。虽然OpenGL没有提供这样一种模式，该模式可以把多边形的内部和

边用不同的颜色来绘制，但是我们可以开启多边形偏移模式（polygon offset mode）并用glPolygonOffset设置偏移参数。该模式会使多边形稍微远离观察者，这样观察者就能看到想要显示的线段了。

上面绘制的网格图是比较基本的，我们可以用许多方式对其进行扩展。第6章将讨论如何考虑光照和表面的材质属性以生成更逼真的图像。第8章将讨论如何将纹理映射到对象的表面。纹理贴图可以是来自照片的地貌图像，也可以是通过地图的数字化得到其他的数据。如果把这些技术结合起来，就能通过改变光源的位置使生成的图像依赖于一天中时间的变化。我们还可以利用第12章介绍的各种曲面表示类型来生成更平滑的表面。

5.8　平行投影变换矩阵

在5.4节推导投影变换矩阵的时候，对照相机的指定非常简单，所以导出的矩阵只对应于特定的观察视见体。OpenGL的投影变换矩阵不这么简单，因为它们支持更一般的观察视见体。在这一节和5.9节中，我们要推导出OpenGL的投影变换矩阵。因为透视投影是三维计算机图形学中一个非常重要的部分，所以理解透视投影对于编写用户应用程序和实现图形系统都是非常关键的。此外，虽然前面介绍的OpenGL投影函数对于大多数观察是足够了，但是OpenGL API没有直接提供像斜平行投影这样的投影。要想获得这种投影图，可以从头起设置一个投影变换矩阵或者在一种标准投影的基础上进行修改。我们可以把下面推导出的投影变换矩阵应用于这两种方法中的任何一种。

5.8.1　投影的规范化

在第1章介绍投影和在这一章的前面讨论经典投影时，我们把投影看做是这样一种技术，该技术把在三维空间中指定的点映射成二维投影平面上的点。这样的变换是不可逆的，因为一条投影线上的所有点都被映射成投影平面上的同一个点。

在计算机图形系统中，我们看待投影的方式与上面稍微有些不同。第一，我们在四维空间里使用齐次坐标来计算投影。第二，我们尽量保留深度信息（沿投影线的距离），这样可以在绘制流水线后面的阶段里进行隐藏面消除。第三，我们使用了一种称为投影规范化（projection normalization）的技术，如图5.34所示，该技术先把对象变形，使得变形后的对象的正交投影图与我们原来想要得到的对象的投影图相同，这样就把所有的投影都转化为正交投影。这里的扭曲变形由规范化矩阵（normalization matrix）来执行。如图5.35所示，把规范化矩阵和5.4.2节介绍的简单正交投影变换矩阵级联在一起，就可以得到一个齐次坐标矩阵，该矩阵可以用来生成想要的投影。

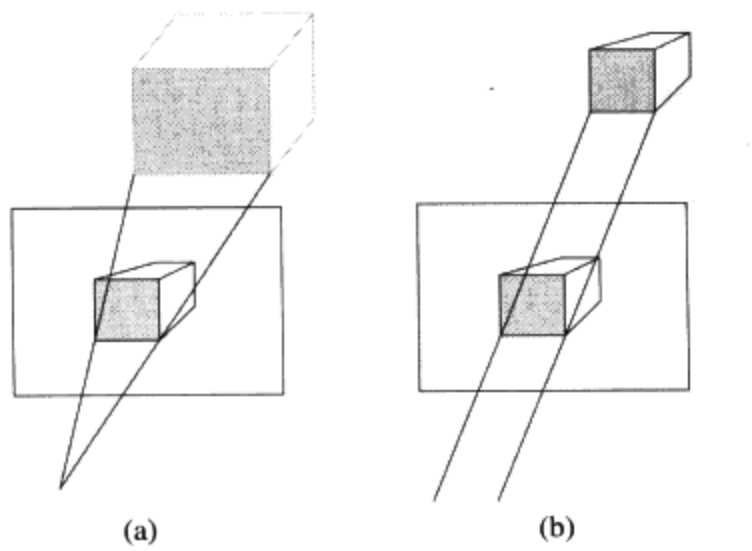


图5.34　预先把对象扭曲变形：(a) 透视投影；(b) 扭曲变形后的对象的正交投影

图 5.35 规范化变换

这种方法的一个优点是可以设计规范化矩阵，使得视见体变形为规范视见体（canonical view volume），该视见体是由下面的平面所定义的立方体：

$$x \pm 1$$
$$y \pm 1$$
$$z \pm 1$$

采用规范视见体的优点是：相同的绘制流水线可以既支持透视投影又支持平行投影，为此只需加载合适的规范化矩阵。此外，规范视见体还简化了裁剪过程，因为它的各面和相应的坐标轴分别平行。

在包括OpenGL在内的大多数系统中，投影规范化由投影变换矩阵（projection matrix）来实现。在OpenGL中，投影变换矩阵把对象变换到四维裁剪坐标系，接下来的透视除法把顶点变换到三维规范化设备坐标系，随后的视口变换把规范化设备坐标系下的点变换到窗口坐标系。这里要讨论的是其中的第一步，即推导出投影变换矩阵。

5.8.2 正投影变换矩阵

尽管平行观察是透视观察的特殊情形，我们还是先来讨论正交平行观察，然后再把规范化技术扩展到透视观察。

在OpenGL中，默认的投影变换矩阵是单位阵，这等价于执行下面的代码：

```
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
glOrtho(-1.0,1.0,-1.0, 1.0, -1.0, 1.0);
```

这个视见体实际上是规范视见体。如果点位于由平面 $x = \pm 1$、$y = \pm 1$ 和 $z = \pm 1$ 定义的立方体内，那么映射后仍在这个立方体内。如果点位于这个立方体之外，那么映射后仍位于这个立方体之外。显然，为了得到一般正交投影的投影变换矩阵，我们只需找到把glOrtho指定的长方体映射成这个立方体的变换矩阵即可。

在找出这个变换矩阵之前，我们先回忆一下glOrtho的最后两个参数的意义：它们分别是从照相机到近裁剪面和远裁剪面的距离。照相机位于原点，方向指向 z 轴的负方向。因此，近裁剪面位于 $z = 1.0$，在照相机的后方；远裁剪面位于 $z = -1$，在照相机的前方。正投影中的投影线是平行的，因而正投影在概念上类似于远离对象的长焦镜头照相机，但glOrtho的最后两个参数不可缺少，它们的重要性在于确定了哪些对象要被裁剪掉。

现在假定不使用OpenGL的默认投影变换矩阵，而是调用下面的glOrtho函数来设置投影变换矩阵：

```
glOrtho(left, right, bottom, top, near, far);
```

由此指定的视见体是一个长方体。相对于照相机而言，该视见体的右面是 $x = \text{left}$，左面是 $x = \text{right}$，顶面是 $y = \text{top}$，底面是 $y = \text{bottom}$，前面是近裁剪面 $z = -\text{near}$，后面是远裁剪面 $z = -\text{far}$。经过这个函数调用后，OpenGL的投影变换矩阵被设置成这样的变换矩阵，该矩阵把上面指定的视见体变换为中心位于原点、边长为2的立方体，如图5.36所示。这个矩阵通过缩放和平移把定义对象的那些顶点（由像glVertex这样的函数所指定）变换成规范视见体内的顶点。因此，在把这个变换应用到顶点以后，位于指定视见体内部的顶点将位于规范视见体内部，而位于指定视见体外的顶点将位

于规范视见体外。把上面这些结合起来考虑可以看到：投影变换矩阵由投影的类型和视见体决定，而视见体是通过调用 glOrtho 来指定的，其中的参数是相对于照相机给出的。照相机的方向和位置由模 - 视变换矩阵决定。OpenGL 把这两个矩阵级联到一起，用所得的乘积来变换对象的顶点。

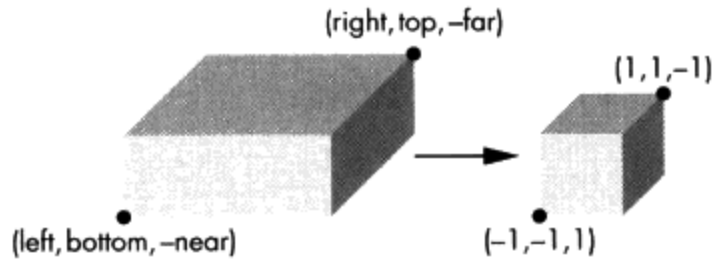


图 5.36　把视见体映射成规范视见体

利用仿射变换的知识就可以求出这个透视投影变换矩阵，我们需要完成两个变换：第一，要通过平移把指定视见体的中心移动到规范视见体的中心（原点）。第二，要缩放指定的视见体使得其每条边的长度都为 2（见图 5.36）。因此，这两个变换是 **T**(–(right + left)/2, –(top + bottom)/2, +(far + near)/2)和 S(2/(right – left), 2/(top – bottom), 2/(near – far))。如图 5.37 所示，把它们级联在一起，所得的投影变换矩阵如下：

$$\mathbf{P} = \mathbf{ST} = \begin{bmatrix} \frac{2}{\text{right}-\text{left}} & 0 & 0 & -\frac{\text{left}+\text{right}}{\text{right}-\text{left}} \\ 0 & \frac{2}{\text{top}-\text{bottom}} & 0 & -\frac{\text{top}+\text{bottom}}{\text{top}-\text{bottom}} \\ 0 & 0 & -\frac{2}{\text{far}-\text{near}} & -\frac{\text{far}+\text{near}}{\text{far}-\text{near}} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

这个矩阵把近裁剪面 $z = -\text{near}$ 映射成 $z = -1$，把远裁剪面 $z = -\text{far}$ 映射成 $z = 1$。因为照相机的方向指向 z 轴的负方向，所以投影线的方向是从 z 轴的负无穷远处指向原点。

图 5.37　用仿射变换来进行投影规范化

5.8.3　斜投影

OpenGL 的 glOrtho 函数只能提供一类有限的平行投影，即投影线垂直于投影平面的平行投影。正如读者在这一章前面所看到的，许多领域都会用到斜平行投影[①]。斜投影变换矩阵可以直接推导出来，不过，这里仍然采用前面推导一般正交投影时的方法。我们先把对象变形，使要求的投影转化为变形后对象的规范正交投影。

如图 5.38 所示，斜投影可以由投影线和投影平面之间的夹角来表征。在支持一般平行观察的 API 中，斜投影的视见体由 6 个面确定，其中近裁剪面和远裁剪面都平行于投影平面，右面、左面、顶面和底面都平行于投影方向，如图 5.39 所示。我们可以根据图 5.40 推导出斜投影的方程，该图所示的是斜投影的俯视图和侧视图，其中画出了投影线和投影平面 $z = 0$。角度 θ 和 ϕ 刻画了投影线的倾斜程度。在制图学中，像斜等测和斜二测这样的斜投影由这些角度的特定值来定义。除了使用这些角度以外，还可以通过其他方式来指定斜投影（见习题 5.9 和习题 5.10）。

① 注意，本书一直在使用斜投影绘制插图中的坐标轴（见习题 5.15）。

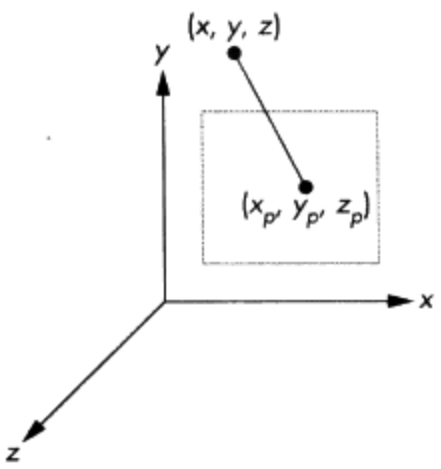


图 5.38　斜投影

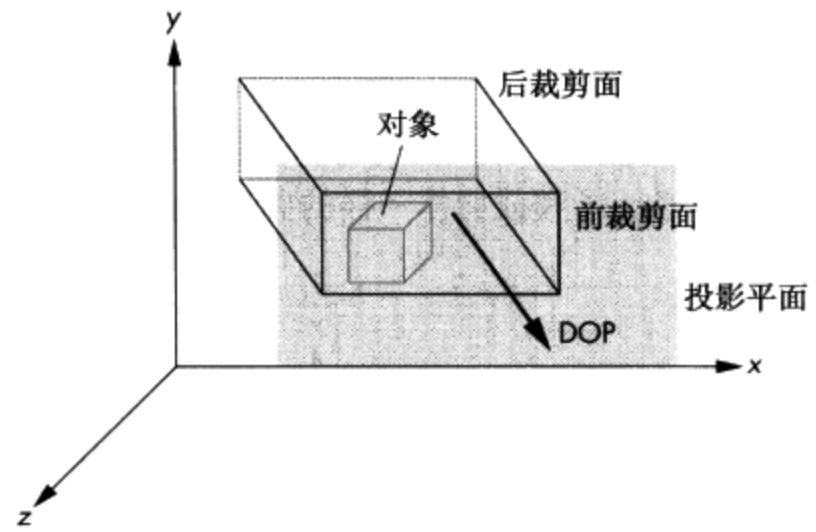


图 5.39　斜投影裁剪体

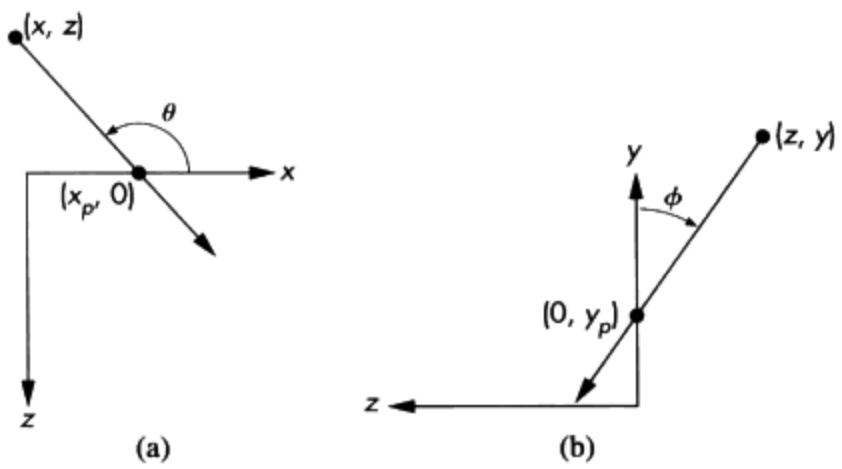


图 5.40　斜投影：(a) 俯视图；(b) 侧视图

考虑图 5.40 中的俯视图，我们注意到

$$\tan\theta = \frac{z}{x_p - x}$$

由此可求出 x_p：

$$x_p = x + z\cot\theta$$

同理

$$y_p = y + z\cot\phi$$

由投影平面的方程可得：

$$z_p = 0$$

这些结果可以表示为下面的齐次坐标矩阵：

$$\mathbf{P} = \begin{bmatrix} 1 & 0 & \cot\theta & 0 \\ 0 & 1 & \cot\phi & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

按照前一个例子中所采用的方法，可以把 $\mathbf{P}$ 分解为两个矩阵的乘积：

$$\mathbf{P} = \mathbf{M}_{\text{orth}}\mathbf{H}(\theta, \phi) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & \cot\theta & 0 \\ 0 & 1 & \cot\phi & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

其中，$\mathbf{H}(\theta, \phi)$是一个剪切矩阵。于是，我们可以这样来实现一个斜投影：首先对对象进行剪切变换 $\mathbf{H}(\theta, \phi)$，然后再进行正投影。图 5.41 给出了对位于斜投影视见体内的一个立方体进行剪切变换 $\mathbf{H}(\theta, \phi)$的效果。经过剪切变换后，裁剪体的侧面垂直于投影平面，但是立方体的侧面不再和底面垂直，因为立方体经历了和裁剪体一样的剪切变换。然而，变形后立方体的正投影和原来立方体的斜投影是相同的。

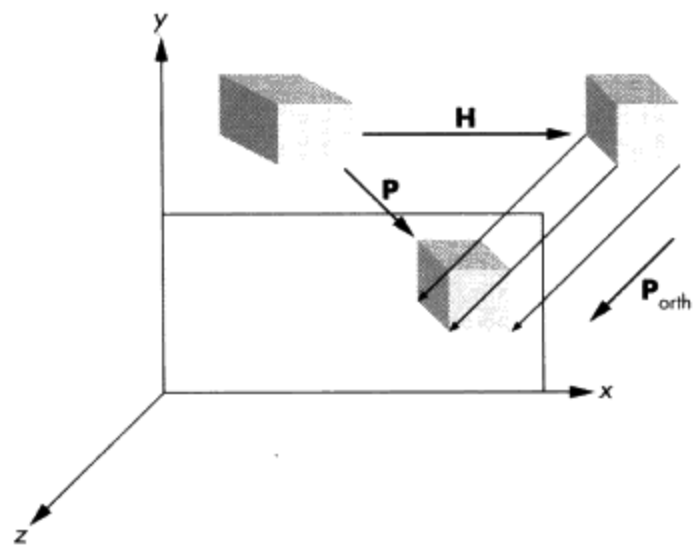


图 5.41　剪切变换的效果

现在还没有完成投影规范化，因为经过剪切变换后的视见体还不是规范视见体。我们必须应用和 5.8.2 节中相同的缩放和平移矩阵。因此，下面的变换矩阵

$$\mathbf{ST} = \begin{bmatrix} \frac{2}{\text{right}-\text{left}} & 0 & 0 & -\frac{\text{right}+\text{left}}{\text{right}-\text{left}} \\ 0 & \frac{2}{\text{top}-\text{bottom}} & 0 & -\frac{\text{top}+\text{bottom}}{\text{top}-\text{bottom}} \\ 0 & 0 & -\frac{2}{\text{far}-\text{near}} & -\frac{\text{far}+\text{near}}{\text{near}-\text{far}} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

必须在剪切变换之后和最后的正投影之前应用，于是最后的结果为：

$$\mathbf{P} = \mathbf{M}_{\text{orth}}\mathbf{STH}$$

left、right、bottom 和 top 的值对应于斜投影视见体经过剪切变换之后得到的长方体的顶点。这些值取决于原来视见体各个面的位置，为了确定它们，可能需要对原来视见体的顶点进行剪切变换。

5.9　透视投影变换矩阵

对于透视投影，我们采用和平行投影相似的方法：找出一个变换，使得对象的顶点在经过这个变换之后，只需对其进行一个简单的规范投影就可以获得想要的投影图。首先要确定规范视见体是什么；然后引入一个新的变换：投影规范化变换（perspective-normalization transformation），它把透视投影转化为正交投影；最后，推导出 OpenGL 中使用的透视投影变换矩阵。

5.9.1　透视投影的规范化

5.4 节中导出了一个简单的透视投影变换矩阵。对于投影平面为 $z = -1$ 并且投影中心位于原点的透视投影，其投影变换矩阵为：

$$\mathbf{M} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

为了生成图像，我们还需要指定裁剪体。假定视见体的侧面和投影平面之间的夹角为45°，那么视角固定为90°。换句话说，视见体是由下面这两个平面所构成的半无穷棱锥，如图5.42所示。

$$x = \pm z$$
$$y = \pm z$$

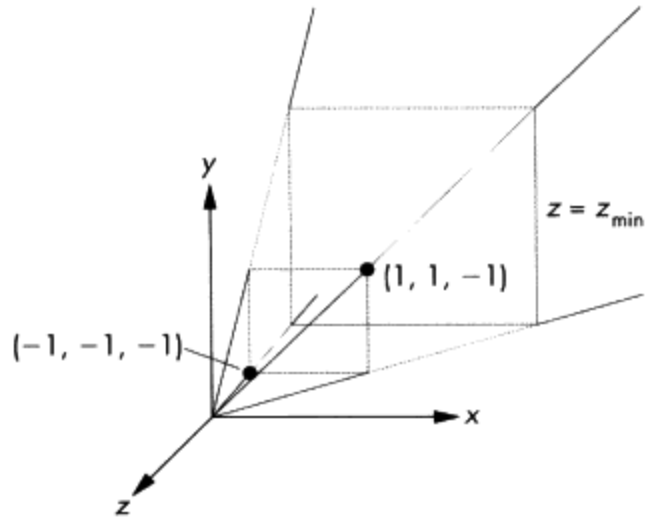


图 5.42　简单的透视投影

为了得到有限的视见体，我们还需指定近裁剪面 $z = -\text{near}$ 和远裁剪面 $z = -\text{far}$，其中 near 和 far 分别是从投影中心到近裁剪面和远裁剪面的距离，它们满足下面的不等式：

$$0 < \text{near} < \text{far}$$

考虑下面的矩阵：

$$\mathbf{N} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \alpha & \beta \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

它的形式和矩阵 **M** 有些类似，但非奇异。目前我们暂不确定 α 和 β 的值（但它们都不等于零）。对用齐次坐标表示的点 $\mathbf{p} = [x \ y \ z \ 1]^T$ 应用变换 **N**，所得的新点为 $\mathbf{q} = [x' \ y' \ z' \ w']^T$，其中，

$$x' = x$$
$$y' = y$$
$$z' = \alpha z + \beta$$
$$w' = -z$$

用 w' 去除前三个分量，就得到了一个三维空间中的点：

$$x'' = -\frac{x}{z}$$

$$y'' = -\frac{y}{z}$$

$$z'' = -\left(\alpha + \frac{\beta}{z}\right)$$

如果在变换 **N** 之后再进行沿 z 轴的正投影，那么总的变换矩阵为：

$$\mathbf{M}_{\text{orth}}\mathbf{N} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

这是一个简单的透视投影变换矩阵，对任意一点 **p** 应用该变换矩阵可得：

$$\mathbf{p}' = \mathbf{M}_{\text{orth}}\mathbf{N}\mathbf{p} = \begin{bmatrix} x \\ y \\ 0 \\ -z \end{bmatrix}$$

再对 $\mathbf{p}'$ 进行透视除法，就得到了想要的 x_p 和 y_p：

$$x_p = -\frac{x}{z}$$

$$y_p = -\frac{y}{z}$$

我们看到：先应用变换 **N**，然后再进行正交投影，所得的结果和透视投影相同。这个过程类似于通过剪切变换把斜投影转化为正交投影。

非奇异矩阵 **N** 把原来的视见体变换成一个新的视见体。现在我们来选择合适的 α 和 β，使得这个新的视见体是规范裁剪体。原来视见体的侧面

$$x = \pm z$$

被变换成平面

$$x'' = \pm 1$$

同样，原来视见体的侧面 $y = \pm z$ 被变换成

$$y'' = \pm 1$$

前裁剪面 $z = -\text{near}$ 被变换成平面

$$z'' = -\left(\alpha - \frac{\beta}{\text{near}}\right)$$

最后，后裁剪面 $z = -\text{far}$ 被变换成平面

$$z'' = -\left(\alpha - \frac{\beta}{\text{far}}\right)$$

如果令

$$\alpha = \frac{\text{near} + \text{far}}{\text{near} - \text{far}}$$

$$\beta = -\frac{2 * \text{near} * \text{far}}{\text{near} - \text{far}}$$

那么平面 $z = -\text{near}$ 被映射成平面，平面 $z = -\text{far}$ 被映射成平面，于是得到了规范裁剪体。这个变换如图5.43所示，该图画出了在视见体内的一个立方体经过该变换后变形的情形。**N**把棱台观察体变换成长方体，之后再进行正投影所得到的图像和透视投影相同。矩阵 **N** 称为透视规范化矩阵（perspective-normalization matrix）。映射

$$z'' = -\left(\alpha + \frac{\beta}{z}\right)$$

是非线性的，但它保持了深度的大小关系。因此，如果z_1和z_2是原来视见体内的两个点的深度并且有

$$z_1 > z_2$$

那么变换后的深度满足

$$z_1'' < z_2''$$

这样，在规范化视见体中仍然可以进行隐藏面消除，不过由于深度缓存的深度分辨度有限，透视规范化变换的非线性可能会导致数值精度问题。请读者注意，虽然原来位于$z = -1$的投影平面被**N**变换成了平面$z'' = \beta - \alpha$，但这不会产生什么不良后果，因为在**N**之后还要进行一个正投影。

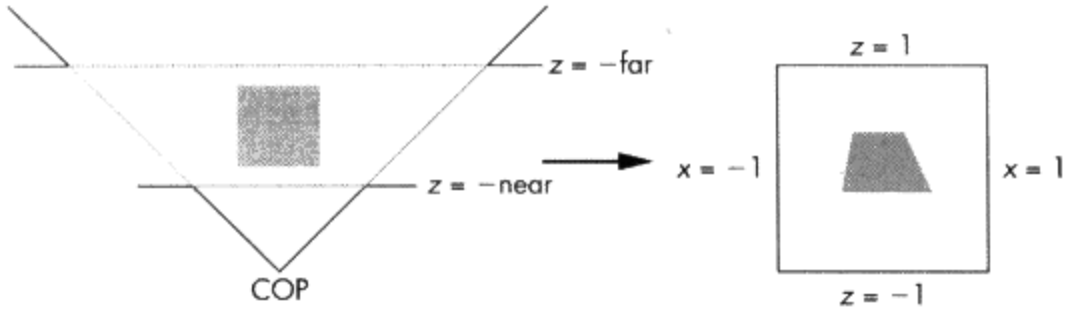


图5.43 视见体的透视规范化

我们已经看到，透视投影和平行投影都可以转化为正投影，不过只有在考虑绘制流水线的实现时，这样转化的优点才会最大化地体现出来。在定义顶点之前，只要把一个小心选择的投影变换矩阵加载到绘制流水线中，那么只需要一个观察流水线就可以实现所有可能的投影。第7章会详细讨论绘制流水线的实现，在那里会看到，投影规范化通过把所有的视见体转化为长方体可以简化裁剪和隐藏面消除。

5.9.2 OpenGL中的投影变换

OpenGL中的glFrustum函数没有限制视见体一定是对称的棱台。这个函数的参数如图5.44所示。为了得到OpenGL透视投影变换矩阵，需要先把该图所示的棱台视见体变换为如图5.42所示的棱台视见体，这和把斜平行投影转化为正交投影类似。我们首先通过剪切变换把不对称的棱台转化为对称棱台。从图5.44可以看到，经过剪切变换后，点((left + right)/2, (top + bottom)/2, −near)应位于(0, 0, −near)，由此可以确定剪切角。所需的剪切变换矩阵为：

$$\mathbf{H}(\theta, \phi) = \mathbf{H}\left(\cot^{-1}\left(\frac{\text{left} + \text{right}}{-2\text{near}}\right), \cot^{-1}\left(\frac{\text{top} + \text{bottom}}{-2\text{near}}\right)\right)$$

剪切变换后的棱台视见体由下面几个平面围成：

$$x = \pm\frac{\text{right} - \text{left}}{-2 * \text{near}}$$

$$y = \pm\frac{\text{top} - \text{bottom}}{-2 * \text{near}}$$

$$z = -\text{near}$$

$$z = -\text{far}$$

接下来应该通过缩放变换把这个棱台的侧面变为：

$$x = \pm z$$
$$y = \pm z$$

但不改变前裁剪面和后裁剪面。所需的缩放矩阵为S(–2*near/(right – left), –2*near/(top – bottom), 1)。注意，不需要远裁剪面 z = –far 的位置就可以唯一确定这个变换，这是因为在三维空间里一个仿射变换可以由其对4个点的变换结果来确定。在这里，确定仿射变换的4个点是棱台视见体的4个侧面和近裁剪面相交的4个顶点。

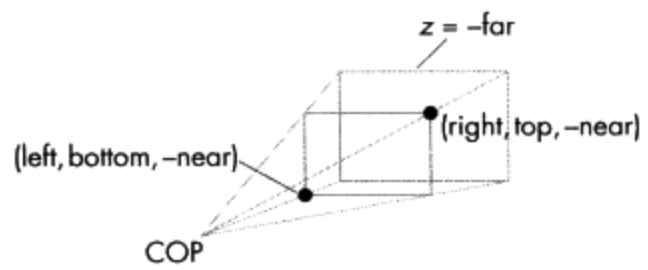


图 5.44　OpenGL 透视投影

为了使远裁剪面和近裁剪面在应用投影规范化变换之后分别位于 $z = -1$ 和 $z = 1$，我们应用下面的投影规范化矩阵 **N**：

$$\mathbf{N} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \alpha & \beta \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

其中，α 和 β 的取值和5.9.1节中的一样。最后得到的投影变换矩阵如下：

$$\mathbf{P} = \mathbf{NSH} = \begin{bmatrix} \frac{-2*\text{near}}{\text{right}-\text{left}} & 0 & \frac{\text{right}+\text{left}}{\text{right}-\text{left}} & 0 \\ 0 & \frac{-2*\text{near}}{\text{top}-\text{bottom}} & \frac{\text{top}+\text{bottom}}{\text{top}-\text{bottom}} & 0 \\ 0 & 0 & -\frac{\text{far}+\text{near}}{\text{far}-\text{near}} & \frac{2\text{far}*\text{near}}{\text{far}-\text{near}} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

5.10　投影和阴影

投影变换矩阵的一个有趣应用是生成简单的阴影。虽然阴影不是几何对象，但它是逼真图像的重要组成部分，因为阴影为场景中各对象之间的空间位置关系提供了许多视觉线索。从物理的角度来看，有光源才会产生阴影。如果任何光源都不能照射到某一个点，或者等价地讲，位于那个点的观察者不能看到任何光源，那么这个点就位于阴影之中。然而，如果唯一的光源位于投影中心，那么生成的图像中不会包含阴影，因为任何阴影都位于可见对象的后面。这种光照模式被称为“眼睛里的手电筒”模型，它对应于我们到目前为止所使用的简单光照模型。

为了在图像中添加符合物理规律的阴影效果，我们必须对光线和材质之间的相互作用有一定的理解。第6章将讨论这些内容，在那里会看到全局光照的计算是困难的，而且一般情况下全局计算不能实时进行。

然而，鉴于阴影效果在飞行模拟器等应用中的重要性，人们针对各种不同的情况想出了一些特殊的方法。考虑如图5.45所示的由点光源所生成的阴影。为了简单，假定阴影落在了地面上，或者说是落在了下面的平面上：

$$y = 0$$

这个阴影不但是一个平面多边形，称为阴影多边形（shadow polygon），而且还是原来的多边形在地面上的投影。说得再具体一些，阴影多边形是原来的多边形在地面上的投影，投影中心位于光源所在的位置。因此，如果在一个以光源为原点的标架内把多边形投影到地面所在的平面，那么就可以得到阴影多边形的顶点。之后，我们还要把这些顶点的表示转换为对象标架下的表示。不过在应用程序中无须具体实现上面的步骤，我们可以求出一个合适的投影变换矩阵，然后利用OpenGL来计算阴影多边形的顶点。

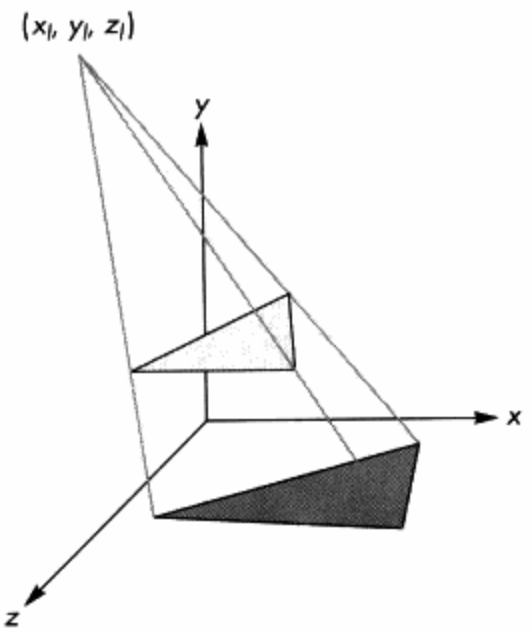


图 5.45 单个多边形的阴影

假定从位于(x_l, y_l, z_l)的光源处观察，那么看到的多边形如图 5.46(a)所示。如果通过平移 $\mathbf{T}(-x_l, -y_l, -z_l)$将光源移动到原点，如图 5.46(b)所示，那么通过一个简单的透视投影就可以求出阴影多边形。这个投影变换矩阵为：

$$\mathbf{M} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & \frac{1}{-y_l} & 0 & 0 \end{bmatrix}$$

最后再通过 $\mathbf{T}(x_l, y_l, z_l)$把所有的对象平移回原来的位置。这三个变换矩阵的级联把顶点(x, y, z)变换成：

$$x_p = x_l - \frac{x - x_l}{(y - y_l)/y_l}$$

$$y_p = 0$$

$$z_p = z_l - \frac{z - z_l}{(y - y_l)/y_l}$$

在 OpenGL 程序中，可以通过改变模 – 视变换矩阵来生成想要的阴影多边形，代码如下：

```
GLfloat m[16];                    /* 生成阴影的投影变换矩阵 */
for(i=0;i<15;i++) m[i]=0.0;
m[0]=m[5]=m[10]=1.0;
m[7]= -1.0/yl;

glColor3fv(polygon_color)
```

```
glBegin(GL_POLYGON)
.
.                                  /* 按通常的方式绘制多边形 */
glEnd();
glMatrixMode(GL_MODELVIEW);
glPushMatrix();                    /* 保存状态*/
glTranslatef(xl,yl,zl);            /* 平移回原来的位置 */
glMultMatrixf(m);                  /* 投影 */
glTranslate(-xl,-yl,-zl);          /* 把光源移动到原点*/
glColor3fv(shadow_color);
glBegin(GL_POLYGON);
.
.                                  /* 再次绘制多边形*/
glEnd();
glPopMatrix();                     /* 恢复状态 */
```

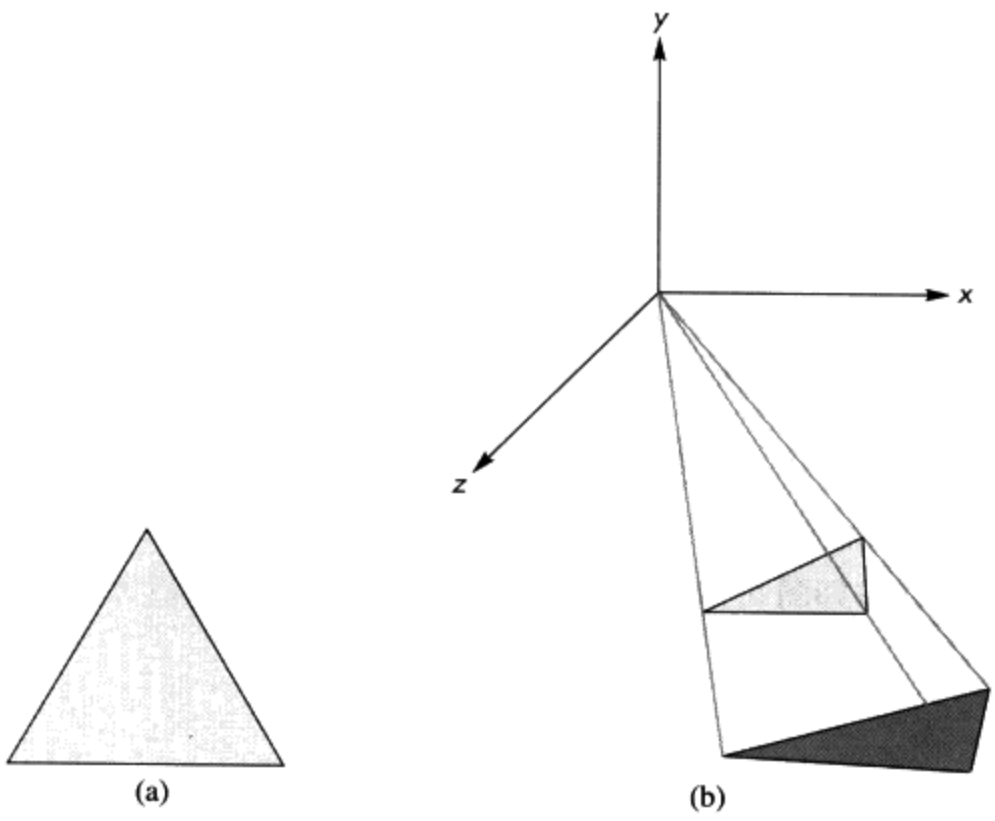


图 5.46　通过投影得到阴影多边形：(a) 从光源的位置看到的多边形；(b) 把光源移动到原点

请读者注意，虽然我们以光源为投影中心进行投影，但所使用的矩阵是模–视变换矩阵。我们把同一个多边形绘制了两次：第一次是按通常方式绘制，第二次绘制使用了修改后的模–视变换矩阵，它对多边形的顶点进行了变换。绘制原多边形和阴影多边形所用的观察条件是相同的。对立方体计算阴影的结果见彩图 38，相应的代码可以从本书的网页上下载，源程序文件为 cubevs.c。

对于简单的情况，比如飞机在平坦地面上投射单独一个阴影，这种方法的效果很好。此外，我们很容易就可以把这种方法从点光源推广到无穷远的光源（平行光源）（见习题5.17）。但是，如果对象在其他对象上投射阴影，这种方法就行不通了。第13章将讨论更一般但速度较慢的绘制方法，那些绘制方法把生成阴影作为绘制过程的一部分。

小结和注释

至此已经学了不少东西，现在读者已经能够编写出完整的、有意义的三维图形应用程序了。对读者来说，也许最有意义的练习就是编写这样一个程序。要想熟练地使用模–视变换函数和投影函数，就得多实践。

这一章给出了标准投影的数学推导。虽然大多数API不需要应用程序编程人员编写投影函数，但是理解了相关的数学就可以理解基于4 × 4矩阵级联的流水线实现方式。直到不久以前，应用程序编程人员还得在应用程序中实现投影，而且大多数硬件系统也不支持透视投影。

本书后面的部分有三个主题。第一个主题是通过扩展基本图元来进一步讨论建模。第10章介绍通过层级模型来描述简单对象之间更复杂的关系。第11章介绍基于过程而不是几何对象来建模。利用这种方法，我们在建模对象时可以考虑对象所需要的更多细节；可以在模型中引入物理定律；还可以建模不能由多边形描述的自然现象。第12章将摆脱平面对象，讨论曲线和曲面。这些对象由顶点定义，我们可以把它们分割成小的平面图元，从而可以使用同样的观察流水线。

第二个主题是图像的逼真性。虽然利用更复杂的对象可以构造更逼真的模型，但我们也可以探索更复杂的绘制方法。第6章讨论光线和材质的相互作用，这些相互作用体现了对象的特性。第7章将进一步考察隐藏面消除。第8章和第9章将讨论像纹理映射这样的技术，借助这些高级绘制技术，我们可以从简单的对象生成复杂的图像。

第三个主题是实现。第7章将更深入地讨论绘制流水线的实现。我们已经介绍了图形绘制流水线的主要功能单元。第7章将讨论每个单元所使用算法的细节。我们还将看到通过直接操作帧缓存可以为生成图像提供其他的方法。

建议阅读资料

Carlbom和Paciorek的文章[Car78]讨论了经典观察和计算机观察之间的联系。Rogers和Adams的书[Rog90]给出了许多投影变换矩阵的例子，这些矩阵对应于制图学中所使用的标准投影图。Foley等的书[Fol90]、Watt的书[Wat00]以及Hearn和Baker的书[Hea04]都对规范投影变换进行了推导。所有这些文献都遵从PHIGS，所以它们采用的API和本书中的稍有不同，尽管Foley对最一般的情形进行了推导。在这些参考文献中，有的使用列矩阵，有的使用行矩阵；有的向z轴正方向投影，有的向z轴负方向投影；COP的位置也各有不同。要想进一步了解OpenGL中的模－视变换矩阵和投影变换矩阵的用法，可以阅读 *OpenGL Programming Guide* [Ope05]。

习题

5.1 并不是所有的投影都是平面几何投影。举出两个投影的例子，分别说明投影面可以不是平面，投影线也可以不是直线。

5.2 一架飞机的方位由滚转角、俯仰角、偏航角和到某个对象的距离确定，根据这些参数求出模－视变换矩阵。

5.3 考虑一个绕着地球运行的卫星，它在地球上空的位置由极坐标指定。求使观察者的视线一直指向地球的模－视变换矩阵。在卫星运行时可以用该变换矩阵来显示地球的图像。

5.4 如何只利用叉积从VPN、VRP和VUP求出u和v方向？

5.5 能否通过只绕一个轴旋转立方体就得到它的等轴测投影图？请给出理由。

5.6 假定COP可以位于任意点并且投影平面的方向也是任意的，推导透视投影变换矩阵。

5.7 证明直线经透视投影后仍为直线。

5.8 过COP作平行于投影平面的平面，对该平面内的点计算投影将导致零作为除数。如果一条线段的两个端点分别位于该平面的两侧，那么这条线段的投影是什么？

5.9　定义一个或者多个API函数来指定斜投影。不用编写函数的代码，只需确定用户必须指定哪些参数。

5.10　已知斜投影裁剪体的前裁剪面和后裁剪面，以及前裁剪面与各侧面相交的右上角和左下角，求斜投影变换矩阵。

5.11　我们对所有的投影都进行规范化，这似乎意味着可以预先把所有的对象变形，于是只需支持正投影即可。请指出用这种方法构建图形系统时会遇到的问题。

5.12　如果COP不在原点，应该如何修改OpenGL投影变换矩阵？假定COP位于$(0, 0, d)$，投影平面为$z = 0$。

5.13　通过把二维对象沿着第三维拉伸，可以创建一类有趣的三维对象。例如，圆变成了圆柱面，直线变成了四边形，平面四边形变成了平行六面体。利用这个技术把习题2.8中的二维迷宫转换成三维迷宫。

5.14　扩展习题5.13中的迷宫程序，允许用户在迷宫中漫步。单击鼠标中键使用户向前移动，单击鼠标右键和左键分别使用户向右和向左旋转90°。

5.15　如果用正交投影来绘制坐标轴，那么x轴和y轴将位于纸面，但z轴将指出纸面。我们还可以把x轴和y轴之间的夹角画成90°，z轴与x轴之间的夹角画成−135°。求这个投影图所对应的投影变换矩阵。

5.16　编写一个程序，显示在一个盒子内旋转的立方体。这个盒子里有三个光源，每个光源把立方体的阴影投射到盒子的三个可见面之一。

5.17　设光源位于无穷远，其方向为(d_x, d_y, d_z)。求该光源把一个点投射到平面$ax + by + cz + d = 0$上的阴影。

5.18　利用第4章讨论的三维接口之一，编写一个程序，在包含一些简单对象的场景中移动照相机。

5.19　编写一个程序，在由细分四面体得到的三维Sierpinski镂垫中漫游。能否避免用户穿越三维Sierpinski镂垫的表面？

5.20　在动画制作中，我们经常可以借助二维模式来简化绘制过程，这些二维模式被映射成始终和照相机保持平行的平面多边形。编写一个程序，使得一个简单多边形在照相机移动时始终保持面向照相机。

5.21　为了获得立体图像需要生成两幅图像，这两幅图像所对应的观察位置稍有差异。假定观察者位于原点，两眼之间的距离为Δx。为了生成这样的两幅图像，应如何指定观察条件？

第6章　光照和明暗绘制

我们已经学习了如何构建三维图形模型以及如何显示它们。然而，如果绘制这些模型所得的图像会使我们失望：它们看起来像是平的，没有深度感，所以没能显示出模型的三维本性。这是因为我们做了一个不自然的假定：每个表面只有单独一种颜色。在这个假定下，球面的正投影是均匀着色的圆，立方体的投影图看起来像是平面六边形。如果观察在光线照射下的球面的照片，那么我们看到的不是均匀着色的圆，而是包含许多不同色调（shade）的圆。正是这种颜色的渐变使得二维图像看起来像是三维的。

在模型中没有考虑光线和对象表面之间的相互作用，从这一章开始将填补这个空缺。我们分别讨论光源的模型和最常见的几种描述光线-材质之间相互作用的模型。目的是在图形绘制的快速流水线结构中增加明暗处理功能，因此这里只讨论局部光照模型。与全局光照模型不同，在局部光照模型中，表面上一点的颜色值只取决于表面的材质属性、表面的局部几何性质以及光源的位置和性质，而与场景中其他的表面无关。这一章介绍OpenGL中的固定功能绘制流水线所使用的光照模型，这些模型要应用到每个顶点上。基于可编程GPU能够将顶点着色程序作用于每个顶点，并且能够把光照计算（即片元着色程序）应用到每个片元，而不是每个顶点，第9章将会考虑如何通过可编程着色器应用其他的光照模型。

与以前的讨论相一致，我们考察如何把明暗效果应用到多边形模型。我们通过递归的方法逼近一个球面，用它来测试各种光照[①]算法，然后讨论在OpenGL中如何指定光源和材质属性，并在近似球面的程序中增加光照效果。

在本章的最后，我们简单讨论生成全局光照效果的两种最重要的方法：光线跟踪和辐射度方法。

6.1　光线和材质

在第1章和第2章，我们介绍了有关人类颜色视觉的初步知识，但没有讨论光线和表面之间的相互作用。也许最一般的绘制方法是基于物理的绘制，在这种方法中，我们利用像能量守恒这样的物理原理推导出描述光线与材质相互作用的方程。

从物理的角度来看，表面可以像灯泡那样发射光线，也可以反射来自其他表面的光线。有些表面不但能反射光线，而且由于其内部发生的物理过程还能发射光线。当观察对象上的一点时，我们所看到的颜色取决于多个光源和多个反射表面之间的多次相互作用。我们可以把这些相互作用看做是一个递归过程。考虑图6.1所示的简单场景。从光源发出的到达表面A的那一部分光线被表面A向各个方向反射。一部分反射的光线到达表面B，这部分光线中又有一部分被表面B反射回表面A，其中又有一部分被表面A反射回表面B，如此不断反射。光线在表面之间的多次反射解释了一些细微的明暗效果，例如在相邻表面之间的颜色渗透。从数学上看，这个递归过程可以用一个积分方程来描述，即绘制方程（rendering equation），在原则上可以用它来求出场景中所有表面的明暗值。遗

① 在计算机图形学中，光照（lighting）指的是对光源、表面以及这两者之间的相互作用进行建模，而明暗（shading）通常指的是对多边形对象的光照处理。

憾的是，这个方程一般不能求出解析解，而数值方法对于实时渲染又不够快。我们还可以利用像辐射度方法和光线跟踪这样的近似解法。对于特殊类型的表面，这两种方法都是对绘制方程的良好近似。然而，不论是光线跟踪还是辐射度方法，用它们来绘制表面的速度还都赶不上利用建模-投影流水线来绘制多边形的速度。因此，这里主要讨论相对简单的Phong光照模型，该模型是符合物理规律和提高运算效率这两者之间的折中。第12章将讨论绘制方程以及辐射度方法和光线跟踪的更多细节。

这里不去考察全局的能量平衡，而是考察从光源（light source）发出的光线，光源是指自身发光的表面。我们对这些光线与场景中反射光线的表面之间的相互作用进行建模。这个方法和光线跟踪类似，但我们只考虑光源和表面之间的单独一次相互作用，而且不考虑从光源发出的光线在照射到某个表面之前被其他表面遮挡的可能性。这个问题包括两个独立的部分：首先，必须建模场景中的光源；其次，必须构建一个描述材质和光线之间相互作用的反射模型。

为了对该过程有一个总体认识，如图6.2所示，我们考察从一个点光源发出的光线。在第1章已经提到，观察者只看到了从光源出发后最终到达眼睛的那些光线，也许这些光线经过了一条很复杂的路径并且还与场景中的对象发生了多次相互作用。如果一条光线从光源出发后直接进入观察者的眼睛，那么观察者就看到了光源的颜色。如果这条光线照射到对观察者来说可见的表面，那么观察者所看到的颜色取决于光源和表面材质之间的相互作用，因为这个颜色是从表面反射到观察者眼中的光线的颜色。

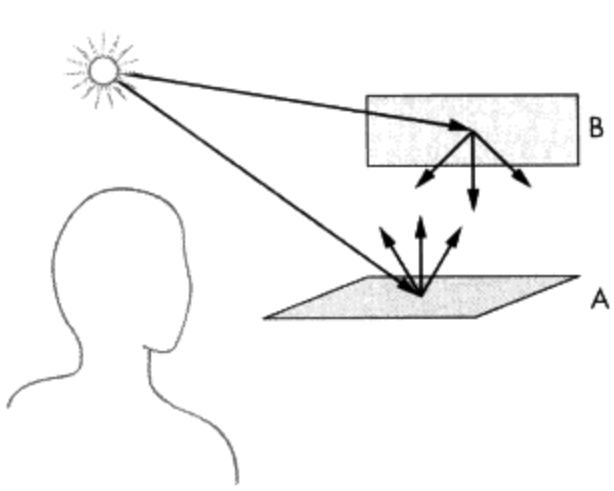


图6.1　光线被表面反射

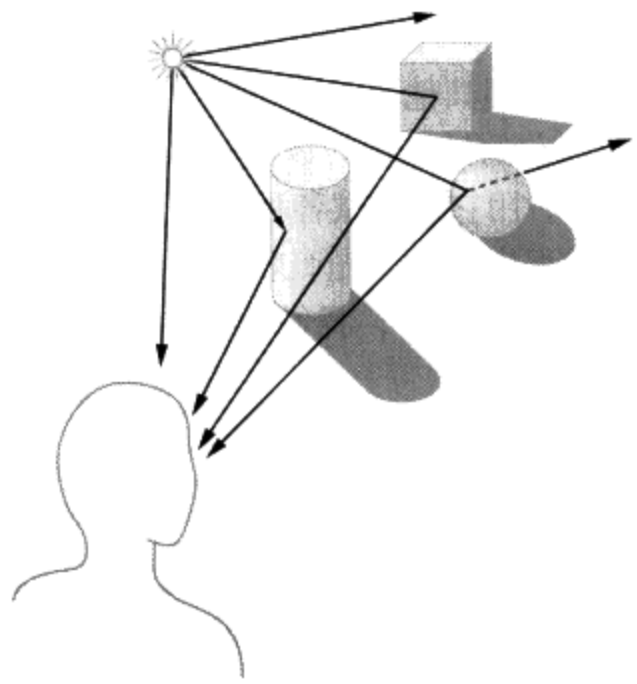


图6.2　光线和表面

在计算机图形学中，可以把投影平面放置在投影中心和观察者之间，如图6.3所示。从概念上讲，投影平面上裁剪窗口内的部分被映射到显示器，因此可以把投影平面用直线分割成许多小的矩形，每个小矩形都对应于显示器上的一个像素。因为只需考虑进入照相机的光线，而这些光线都要通过投影中心，所以可以从投影中心出发，考察通过裁剪窗口内的每个像素的投影线。如果假定所有表面都是不透明的，那么每条投影线遇到的第一个表面的颜色就决定了帧缓存中对应像素的颜色。

请读者注意，光源发出的大多数光线对图像是没有贡献的，因而我们对它们不感兴趣。因此，从投影中心出发向场景中投射（cast）光线更为有效，这是光线跟踪算法的第一步，我们将在6.10节和第13章进一步讨论光线跟踪。

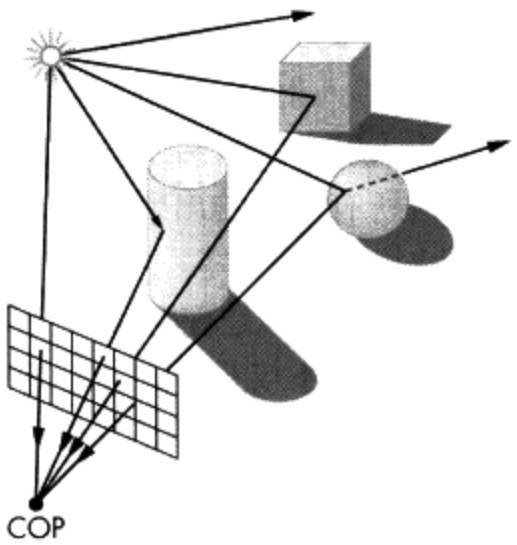


图 6.3　光线、表面和计算机成像

从图 6.2 可以看到，光线和对象之间的相互作用可以发生一次，也可以发生多次。正是这些相互作用的本性决定了一个对象是红色的还是棕色的，是明亮的还是黑暗的，是暗淡的还是有光泽的。当光线照射到表面时，一部分能量被吸收，一部分能量被反射。如果表面是不透明的，那么入射光线不是被反射就是被吸收；如果表面是透明的，那么还有一部分入射光线穿过表面，然后又照射到其他对象。这些相互作用和光线的波长有关。一个对象在白光照射下看起来是红色的，这是因为它把入射光线中除红色以外的频率成分都吸收掉了。一个对象看起来有光泽，是因为它的表面是光滑的。与此相反，一个暗淡的表面是粗糙的。对象的明暗值还依赖于其表面的方向，我们将会看到这个因素是由每一点处的法向量表征的。如图 6.4 所示，光线和材质之间的这些相互作用可以分成以下三类：

1. 镜面反射表面（specular surface）看起来是有光泽的，这是因为被反射或者散射（scatter）出去的大多数光线的方向都和反射角的方向很接近。镜子是理想的镜面反射表面（perfectly specular surface），可能有一部分入射光线被吸收，但所有反射出去的光线都沿着同一个方向，反射光线的方向服从入射角等于反射角这一规律。
2. 漫反射表面（diffuse surface）的特点是把入射光线向各个方向散射。粗糙的或用无光涂料粉刷的墙面是漫反射表面，许多自然材质也都是漫反射表面。例如，从飞机或者卫星上看去，地形表面就是一个漫反射表面。理想漫反射表面（perfectly diffuse surface）向各个方向散射的光线强度都相等，因此不同位置的观察者看到的反射光线都是一样的。
3. 半透明表面（translucent surface）允许入射光线的一部分进入表面并从对象的另一个位置再发射出去。这种折射（refraction）现象是玻璃和水的特征。半透明表面也会反射一部分入射光线。

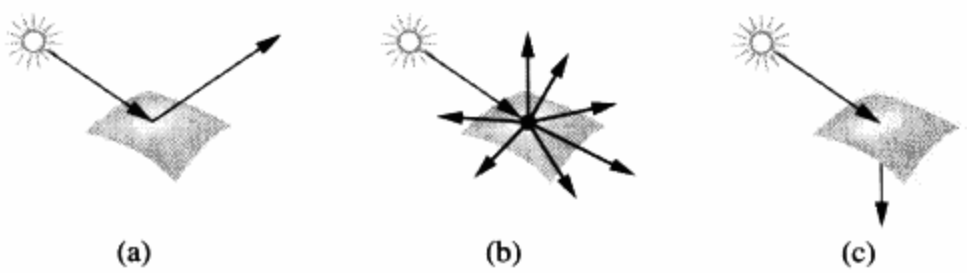


图 6.4　光线和材质之间的相互作用：(a) 镜面反射表面；(b) 漫反射表面；(c) 半透明表面

从物理角度来看，光线的反射、吸收和传输可以由单独一个函数来描述，这个函数称为双向反射分布函数（bidirectional reflection distribution function，BRDF）。考虑表面上的一点，光线可能会

从任意方向照射到这一点，而且被反射出去的方向也是任意的。因此，对于每一对入射方向和出射方向都有一个分数，这个分数的值描述了入射光线中有多大一部分被沿着出射方向反射出去。BDRF有5个自变量：光线的频率、描述入射方向的两个角度和描述出射方向的两个角度。对于理想的漫反射表面，BDRF很简单，因为对所有可能的出射方向它的值都相同。对于理想的镜面反射表面，BRDF只有在入射角等于反射角时才不等于零。

实际的物理表面既不是理想漫反射表面也不是理想镜面反射表面。虽然可以通过建模材质的BDRF来描述光线和材质的相互作用，但这样不仅很复杂而且不能导出有效的计算模型。因此我们转而采用能够快速计算的模型，特别是在绘制流水线结构中使用光照模型时。这类模型的基本想法是把一些简单模型的贡献加在一起，这些简单模型分别描述了漫反射、镜面反射和环境光反射，如果必要的话还有透射。6.3节和第9章将讨论这些理想化的光线与材质相互作用的模型。

首先，让我们来考虑光源的建模。

6.2　光源

离开表面的光线可能是表面自身发射出来的，也可能是被表面反射出来的。我们通常把光源看做是仅通过其内部能量来源发射光线的对象。然而，一个光源，比如灯泡，也可以反射来自周围环境的光线。在简单的光源模型里我们不考虑光源反射的光线。6.8节将介绍OpenGL设置材质属性的函数，届时会看到OpenGL中的表面可以反射光线，也可以发射光线。

考虑如图6.5所示的光源。光源表面上的每个点(x, y, z)都能发射光线，这些光线由光源投射光线的方向(θ, ϕ)和光源在每个波长λ产生的能量强度来表征。因此，一般的光源可以由6个自变量的照明函数（illumination function）$I(x, y, z, \theta, \phi, \lambda)$来表征。注意，指定一个方向需要两个角度，还要注意这里假定每个频率可以单独考虑。对于被光源照射的一个对象，可以通过在光源的表面上进行积分得到光源对这个对象的总贡献（见图6.6），在进行积分时应考虑到从光源到对象表面的出射角，还必须考虑光源和对象表面之间的距离。对于像灯泡这样的扩展光源，不管用解析方法还是数值方法，求这个积分都是困难的。通常情况下更容易的方法是用多个多边形建模扩展光源，其中每个多边形都是一个简单光源，或者用一组点光源来建模扩展光源。

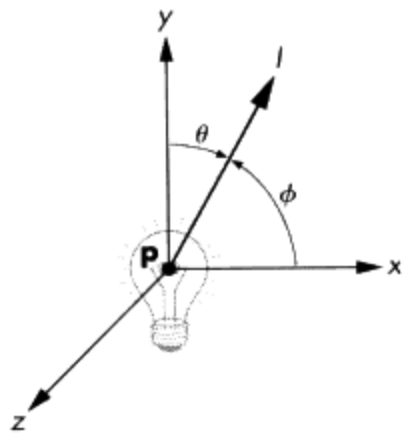


图6.5　光源

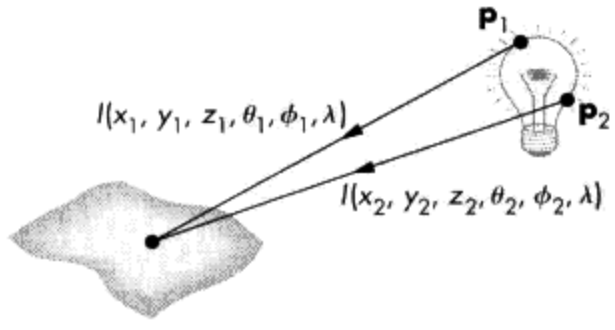


图6.6　累加来自光源的贡献

下面介绍4类基本的光源：环境光、点光源、聚光灯和远距离光源。这4类光源对于绘制大多数简单场景来说足够了。

6.2.1　彩色光源

光源所发出光线的能量不但随方向的不同而变化，还依频率的不同而变化。因此，一种符合物理规律的模型可能会比较复杂。不过，我们对人类视觉系统所建立的模型是以三色理论为基础的，

该理论认为人们感知的只是三刺激值，而不是光线能量按波长的完整分布。因此，对于大多数应用，可以把光源建模成具有红、绿、蓝三个分量，这三个分量和人类观察者看到的颜色分量相对应。于是，彩色光源可以用下面这个三元组来描述：

$$\mathbf{L} = (L_r, L_g, L_b)$$

其中的三个分量分别是独立的红、绿、蓝分量的强度。这样，我们用光源的红色（绿色或蓝色）分量来计算图像的红色（绿色或蓝色）分量。因为对光线和材质之间相互作用的计算包含三个相似但彼此独立的计算，所以往往只列出一个标量方程，该方程可以表示三个颜色分量中的任何一个。为此，用标量 $\mathbf{L}$ 来表示 L 的任何一个分量，即

$$L \in \{L_r, L_g, L_b\}$$

6.2.2　环境光

在许多场合里，例如教室或者厨房，我们对光源进行设计和布置，使得整个房间都得到均匀的照明。这样的照明效果往往通过大尺寸的光源来实现，这些光源内部装有散光器，目的是把光线向各个方向散射。至少在原则上，我们可以构建这种照明的准确模型，方法是建模所有的扩展光源，然后在对象表面上的每一点处对来自这些光源的照明进行积分。但构建一个这样的模型并用它来绘制场景对图形系统来说是个令人生畏的任务，尤其是对要求实时处理的系统[①]。我们可以转而考察这种光源想要达到的效果：使房间获得均匀的照明。这种均匀的照明称为环境光（ambient light）。如果采用第二种方法，我们可以假定在环境中的每一点处都有一个环境光强度。因此，环境光照明可以由一个强度三元组 $\mathbf{L}_a$[②]来表征，场景中每一点处的 $\mathbf{L}_a$ 都是相同的。

环境光有红、绿、蓝三个颜色分量：L_{ar}、L_{ag} 和 L_{ab}，我们将用标量 L_a 来表示这三个分量中的任何一个。虽然场景中每个位置获得的环境光照明都相同，但不同的表面可以反射出不同强度的光线，因为它们的材质属性可能不同。

环境光的颜色和场景中光源的颜色有关。例如，白色房间里红色的灯泡会产生红色的环境光。如果把灯关上，来自环境光的贡献就没有了。OpenGL 允许我们增加一个全局环境光，它不依赖于任何光源。增加这样一个全局环境光的优点是环境中将总有一定强度的照明，于是在观察体内的对象总会出现在图像中，只要它没有被其他对象遮挡。

6.2.3　点光源

理想的点光源（point source）向所有方向发射的光线的强度都相等。一个位于 $\mathbf{p}_0$ 的点光源可以用下面这个三元组来描述：

$$\mathbf{L}(\mathbf{p}_0) = (L_r(\mathbf{p}_0), L_g(\mathbf{p}_0), L_b(\mathbf{p}_0))$$

我们用 $L(\mathbf{p}_0)$ 来表示它的任何一个分量。点 $\mathbf{p}$ 从位于 $\mathbf{p}_0$ 的点光源所接收到的照明强度反比于这两点之间距离的平方。因此，点 $\mathbf{p}$（见图 6.7）从点光源接收到的光线强度可用下式来计算：

$$\mathbf{L}(\mathbf{p}, \mathbf{p}_0) = \frac{1}{|\mathbf{p} - \mathbf{p}_0|^2} L(\mathbf{p}_0)$$

该式给出了任一颜色分量的强度。

---

① 场景中每一点处光线的所有信息确定了光场（light field），光场可以用来构建新的图像。获取光场是一个复杂的过程，代价很高，生成的数据量也非常大，这是当前研究的一个热点。读者可以参考 Lev[96]。

② 此处原文为 $\mathbf{I}_a$，可能有误。——译者注

大多数应用都使用点光源，主要是因为点光源比较简单，而不是因为它们更接近物理实际。仅利用点光源来照明往往会使场景的亮暗反差太大，对象要么很亮、要么很暗。在真实环境中，大多数光源的尺寸不像点光源那样可以忽略，正是这一点使得场景中的亮度变化比较柔和。图6.8给出了一个例子，该图演示了一个有限大小的光源所生成的阴影，可以看到，一些区域完全处于阴影中，或者说在本影（umbra）中，而另一些区域处于部分阴影中，或者说在半影（penumbra）中。通过为场景设置环境光，我们可以减弱由点光源照明引起的过高的对比度。

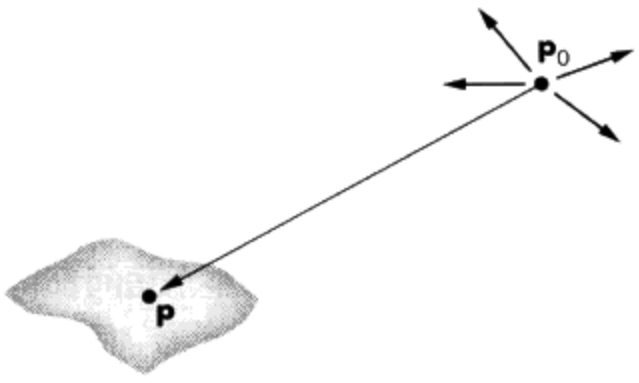


图6.7　点光源照明表面

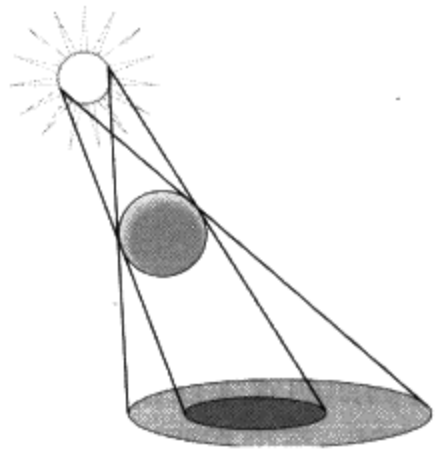


图6.8　由有限大小的光源所生成的阴影

把到光源的距离考虑进去也有助于解决只用点光源照明带来的亮暗相差太大的问题，我们用一个距离衰减因子来表示光线强度随距离增大的衰减。虽然平方反比距离衰减因子对于点光源是正确的，但在实际应用中通常把平方反比距离衰减因子用形如$(a + bd + cd^2)^{-1}$的距离衰减因子代替，其中d是$\mathbf{p}$和$\mathbf{p}_0$之间的距离。可以通过选择常数a、b和c的值来获得柔和的照明效果。此外，一定强度的环境光也能够平滑点光源的照明效果。注意，如果光源和场景中的表面相距很远，那么来自光源的光线就会变得足够均匀以至于不同表面处的距离衰减因子几乎相同。

6.2.4　聚光灯

聚光灯的特点是只在一个小的角度范围内发射光线。我们可以通过限制点光源发射光线的角度来构造一个简单的聚光灯模型。如图6.9所示，我们用一个顶点位于$\mathbf{p}_s$的圆锥来限制发光的角度，这个圆锥的对称轴方向为$\mathbf{l}_s$，对称轴和母线的夹角为θ。如果$\theta = 180°$，那么这个聚光灯就成为点光源。

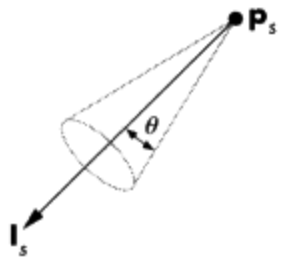


图6.9　聚光灯

更逼真的聚光灯模型可以通过其发光强度在圆锥内的分布来描述，通常越接近圆锥的中心，所发射光线的强度越大。因此，聚光灯的发光强度是光源方向$\mathbf{l}_s$和指向表面的向量$\mathbf{s}$之间夹角θ的函数（这个角度要小于ϕ），如图6.10所示。虽然可以按多种方式来定义这个函数，但通常都把它定义为$\cos^e \phi$，其中的指数e决定了发光强度随着ϕ的增大而衰减的速度（见图6.11）。

这一章很多地方都会讲到，进行光照计算时使用余弦函数是很方便的。如果$\mathbf{u}$和$\mathbf{v}$是任意两个单位长度的向量，那么就可以通过点乘来计算它们之间夹角θ的余弦。

$$\cos\theta = \mathbf{u} \cdot \mathbf{v}$$

这个计算只需要做三次乘法和两次加法。

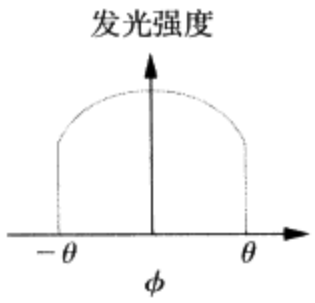


图 6.10　聚光灯的强度衰减依方向的变化

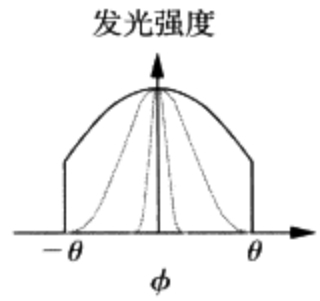


图 6.11　聚光灯模型中的指数对发光强度的影响

6.2.5　远距离光源

大多数明暗计算都需要知道从表面上一点指向光源的方向。当计算表面上每一点处的亮度时，需要反复计算这个方向向量，这是光照计算中的一个重要部分。然而，如果光源和表面之间的距离非常大，那么从表面的这一点移动到那一点时，方向向量几乎没有变化，这就好像在紧挨着的所有对象看来，太阳的光线都是来自同一个方向的。在图6.12中，我们把远距离光源替换成发射平行光线的光源，即平行光源。实际上，远距离光源的计算和平行投影的计算类似，不考虑光源的位置，取而代之的是光源的方向。因此，位于 $\mathbf{p}_0$ 的点光源可以用齐次坐标表示为一个四维列矩阵：

$$\mathbf{p}_0 = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

而远距离光源要用一个方向向量来描述，可以用齐次坐标表示为下面的列矩阵：

$$\mathbf{p}_0 = \begin{bmatrix} x \\ y \\ z \\ 0 \end{bmatrix}$$

使用远距离光源进行绘制要比使用近距离光源更有效。当然，使用远距离光源绘制的场景和使用近距离光源绘制的场景不同。幸运的是，OpenGL 对这两类光源都提供了支持。

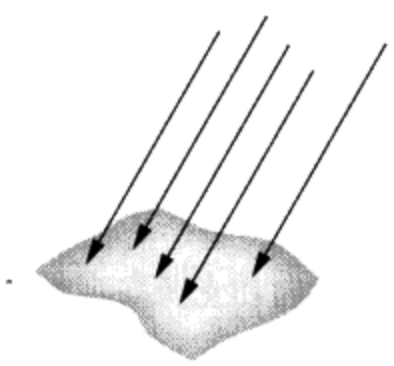


图 6.12　平行光源

6.3　Phong 光照模型

这里要介绍的光照模型首先是由 Phong 提出的，后来 Blinn 又对其进行了改进。实践表明，该模型不但计算效率高，而且和物理事实足够接近，以至于对各种各样的光照条件和材质属性获得的

绘制效果都很好。Phong-Blinn模型（即改进的Phong模型）是利用图形API进行光照和明暗绘制的基础，几乎所有的图形卡都实现了该光照模型。

Phong光照模型使用图6.13所示的4个向量来计算表面上任一点**p**的颜色值。如果表面是曲面，那么所有这4个向量都随着表面上位置的不同而改变。向量**n**是表面在**p**点处的法向量，6.4节将讨论如何计算法向量。向量**v**是从点**p**指向观察者或者COP的向量。对于扩展光源，向量**l**是从点**p**指向扩展光源上任意一点的向量；对于到目前为止一直假定的点光源，向量**l**是从点**p**指向点光源的向量。向量**r**的方向是从沿着向量**l**的方向入射的光线按照反射定律的出射方向。注意**r**由**n**和**l**确定，6.4节将讨论如何计算**r**。

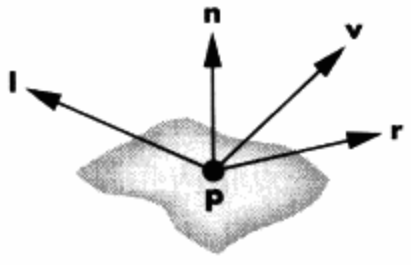


图6.13　Phong模型中用到的向量

Phong模型考虑了光线和材质之间的三种相互作用：环境光反射、漫反射和镜面反射，这些曾在6.1节介绍过。假定有一组点光源。对每个颜色分量，每个点光源都有独立的环境光分量、漫反射光分量和镜面反射光分量。

一个光源对每种光照类型都有三个独立的颜色分量，从物理角度来看这一点有些奇怪。不过，目的是尽可能接近实时生成逼真的明暗效果，而不是试图尽可能准确地建模BRDF。因此，我们通过对局部模型使用一些技巧来模拟那些本质上可能是全局的效果。

环境光可以看做是环境中的背景光，它可以用来模拟光源发出的光线和环境中的表面经多次相互作用之后造成的照明效果，而镜面反射光可以用来生成想要的镜面高光。

因此，如果光源模型包含独立的环境光分量、漫反射光分量和镜面反射光分量，那么就需要9个系数来表征表面上任一点**p**从光源所接收到的光线强度。我们可以把第i个光源的这9个系数排列成一个3 × 3的照明矩阵：

$$\mathbf{L}_i = \begin{bmatrix} L_{ira} & L_{iga} & L_{iba} \\ L_{ird} & L_{igd} & L_{ibd} \\ L_{irs} & L_{igs} & L_{ibs} \end{bmatrix}$$

这个矩阵的第一行是第i个光源的环境光分量的红、绿、蓝颜色分量，第二行是漫反射光分量，第三行是镜面反射光分量。这里假定没有考虑距离衰减因子。

为了建立光照模型，我们应在想要计算颜色值的每一点处对来自所有光源的贡献求和。对每个光源，都要计算照明矩阵中的9个元素分别对应的光线分量强度。例如，对于第i个光源的漫反射光的红色分量L_{ird}，可以计算出一个反射率R_{ird}，于是前者对**p**点处光线强度的贡献为$R_{ird}L_{ird}$。R_{ird}的值取决于材质属性、表面的方向、光源的方向和光源与观察者之间的距离。因此，每个点有9个反射率，它们可以排列成一个矩阵：

$$\mathbf{R}_i = \begin{bmatrix} R_{ira} & R_{iga} & R_{iba} \\ R_{ird} & R_{igd} & R_{ibd} \\ R_{irs} & R_{igs} & R_{ibs} \end{bmatrix}$$

把环境光反射分量、漫反射分量和镜面反射分量加在一起，就可以计算出来自每个彩色光源的贡献。例如，第i个光源对**p**点处光线强度的红色分量的贡献是来自这个光源的环境光反射的红色分量、漫反射的红色分量和镜面反射的红色分量三者之和：

$$I_{ir} = R_{ira}L_{ira} + R_{ird}L_{ird} + R_{irs}L_{irs}$$
$$= I_{ira} + I_{ird} + I_{irs}$$

把来自所有光源的贡献加在一起（可能还要加上全局环境光反射分量）就得到了总的光线强度，因此，**p** 点处光线强度的红色分量为：

$$I_r = \sum_i (I_{ira} + I_{ird} + I_{irs}) + I_{ar}$$

其中，I_{ar} 是全局环境光反射的红色分量。

我们注意到对于每个光源和每个颜色分量的计算都是相似的，差别只在于我们考虑的是环境光反射分量、漫反射分量还是镜面反射分量，因此可以省略脚标 i、r、g 和 b，用下面的公式来表示某点处的光线强度：

$$I = I_a + I_d + I_s = L_aR_a + L_dR_d + L_sR_s$$

上式的意思是对每个光源和每个颜色分量进行的计算都一样，最后可以再加上一个全局的环境光反射分量。

6.3.1　环境光反射

在表面上所有点处的环境光强度 L_a 都是相同的。环境光的一部分被表面吸收，还有一部分被表面反射，被反射部分的强度由环境光反射率 $R_a = k_a$ 确定。因为反射光线的能量是入射光线能量的一部分，所以必然有

$$0 \leqslant k_a \leqslant 1$$

于是

$$I_a = k_aL_a$$

这里 L_a 既可以代表任何单独的光源，也可以代表全局环境光。

一个表面当然有三个环境光反射系数：k_{ar}、k_{ag} 和 k_{ab}，并且这三个反射系数可以不相同。因此，如果一个球面的环境光反射系数的蓝色分量比较小并且红色和绿色分量比较大，那么在白色的环境光照射下，这个球面是黄色的。

6.3.2　漫反射

理想的漫反射表面把光线向所有方向均匀地散射，因此，这样的表面在所有观察者看来亮度都一样。不过，反射出去的光线强度既依赖于材质（因为有一部分入射光线被表面吸收了），也依赖于光源相对于表面的位置。漫反射表面的特点是比较粗糙。一个漫反射表面的横截面放大后如图6.14所示。入射角相差不大的入射光线被反射后的出射方向相差很大。理想漫反射表面是如此粗糙，以至于向各个方向反射的光线强度都相等。这样的表面有时被称为 Lambert 表面（Lambertian surface），可以用 Lambert 定律来建模。

考虑如图 6.15 所示的在太阳光照射下的漫反射平面。该表面在正午时最亮，在黎明和黄昏时最暗，这是因为根据 Lambert 定律，只有入射光线的垂直分量才对照明起作用。为了理解该定律，考虑一个小尺寸的平行光源，如图6.16所示，该光源发出的光线照射到一个平面上。当平行光源在这个（假想的）天空中降低高度时，同样的光线分布在更大的面积上，于是表面显得更暗淡一些。再来考虑图 6.15 中的点光源，现在可以用数学公式来描述漫反射。Lambert 定律表明：

$$R_d \propto \cos\theta$$

其中，θ是法向量**n**和光源的方向**l**之间的夹角。如果**l**和**n**都是单位长度的向量[①]，那么

$$\cos\theta = \mathbf{l}\cdot\mathbf{n}$$

漫反射系数k_d表示在入射的漫反射光中有多大一部分被反射出去，把k_d考虑进来就得到了下面表示漫反射光强度的公式：

$$I_d = k_d(\mathbf{l}\cdot\mathbf{n})L_d$$

如果希望考虑光线在传播过程中的衰减，我们仍旧可以使用二次距离衰减因子：

$$I_d = \frac{k_d}{a+bd+cd^2}(\mathbf{l}\cdot\mathbf{n})L_d$$

上面的公式有一个潜在的问题：如果光源在水平面以下的话$(\mathbf{l}\cdot\mathbf{n})L_d$会取负值。此时我们希望$(\mathbf{l}\cdot\mathbf{n})L_d$取零而不是负数，因此在实际应用时使用的是$\max((\mathbf{l}\cdot\mathbf{n})L_d, 0)$。

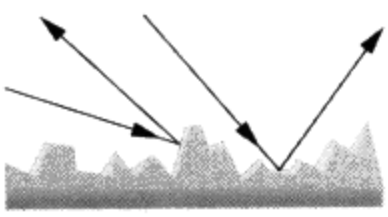


图6.14　粗糙表面

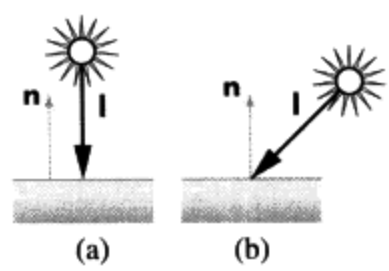


图6.15　太阳光照射在漫反射表面上：(a) 正午；(b) 下午

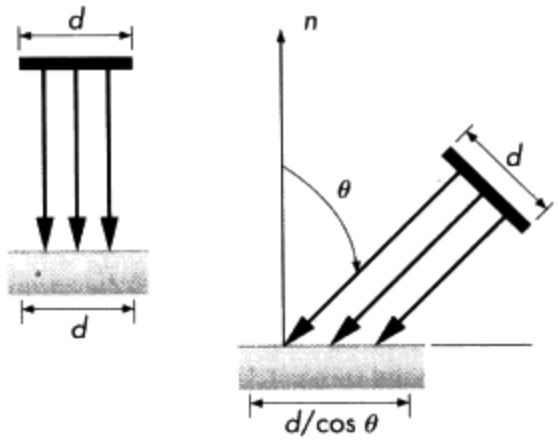


图6.16　根据Lambert定律，对照明有贡献的是入射光线中垂直于表面的分量：(a) 正午；(b) 下午

6.3.3　镜面反射

如果只利用环境光反射和漫反射，尽管生成的图像会有明暗渐变并且看起来有立体感，但所有的表面都没有光泽，有些像粉笔。图像中缺少的是高光，我们观察有光泽的对象时就会看到高光。这些高光通常表现出与环境光反射和漫反射不同的颜色。例如，红色的塑料球在白光照射下会有白色的高光，这是由在观察者方向的某些光源在球体表面形成的反射造成的（见图6.17）。

和漫反射表面是粗糙的相反，镜面反射表面是光滑的。表面越光滑，就越接近于镜子。从图6.18可以看到，表面越光滑，反射出去的光线就越集中在一个角度的附近，这个角度是一个理想的反射器（镜子或者理想镜面反射表面）的反射角。要建立镜面反射表面的逼真模型可能比较复杂，因为光线被散射的模式不是对称的。该模式和入射光线的波长有关，并且随反射角的变化而变化。

① 在明暗计算中，我们会反复用到像**l**和**n**这样的方向向量之间的点积。实际上，程序员和图形软件都应该一遇到这样的向量就把它们归一化。

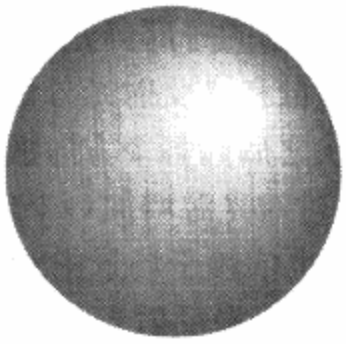


图 6.17　镜面反射高光

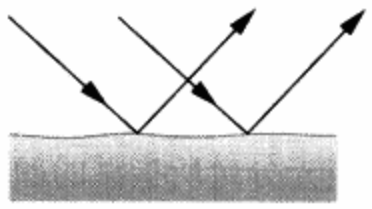


图 6.18　镜面反射表面

Phong 提出了一个近似模型，所需的计算量只比漫反射增加了一点。在该模型的公式中，除了漫反射分量以外，又增加了一项表示镜面反射。这样，在考虑漫反射分量时，我们把表面看做是粗糙的，而在考虑镜面反射分量时，我们把表面看做是光滑的。观察者所看到的光线强度取决于理想反射器反射光线的方向 **r** 和观察者的方向 **v** 两者之间的夹角 ϕ。Phong 模型使用了下面的公式：

$$I_s = k_s L_s \cos^{\alpha} \phi$$

其中，系数k_s（$0 \leqslant k_s \leqslant 1$）表示在入射的镜面反射光中有多大一部分被反射。指数α是高光（shininess）系数。从图 6.19 可以看到，当α增加时，反射的光线越来越集中在理想反射器的反射角附近。无限大的α对应于镜子，在 100 到 500 之间的α对应于大多数金属的表面，更小的α对应于高光区域比较大的材质。

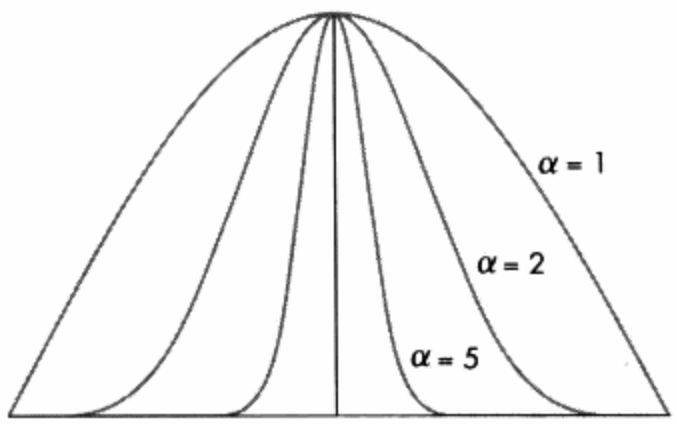


图 6.19　不同的高光系数对反射光线强度的效果

Phong 模型在计算方面的优势在于：如果已经把 **r** 和 **v** 归一化为单位向量，那么可以像计算漫反射分量那样利用点积来计算镜面反射分量：

$$I_s = k_s L_s \max((\mathbf{r} \cdot \mathbf{v})^{\alpha}, 0)$$

我们还可以像计算漫反射分量那样增加一个距离衰减因子。包含距离衰减因子的Phong模型（Phong model）可以用下式表示：

$$I = \frac{1}{a + bd + cd^2}(k_d L_d \max(\mathbf{l} \cdot \mathbf{n}, 0) + k_s L_s \max((\mathbf{r} \cdot \mathbf{v})^{\alpha}, 0)) + k_a L_a$$

上式适用于每个光源和每个颜色分量。

在Phong光照模型中，单独一个光源的环境光分量、漫反射光分量和镜面反射光分量可以具有不同的红、绿和蓝色分量，这一点可能不符合直觉。但是，我们在一个红色球面上看到的白色高光是光源在表面上所成的变形的像，或者是环境中其他亮度较强的白色对象的像，要正确地计算出这种高光需要进行全局而不是局部光照计算。因为我们无法求解绘制方程，所以只能使用各种技巧来试图获得逼真的绘制效果，其中一个技巧就是允许同一个光源的环境光分量、镜面反射光分量和漫反射光分量具有不同的颜色分量。

例如，考虑一个包含许多对象的场景。当打开灯时，光源发出的一部分光线直接照射到一个表面上，这部分光线对图像的贡献可以通过光源的镜面反射光分量和漫反射光分量来建模。然而，光源发出的其他大部分光线在被场景中的其他对象反射了许多次之后向各个方向散射，这些散射光线中又有一部分照射到我们正在考虑的这个表面上。我们可以通过对光源增加一个环境光分量来近似这部分贡献。对环境光分量所指派的颜色既依赖于光源的颜色也依赖于场景中对象的颜色，这是这种近似方法的一个不良后果。在某种程度上，同样的分析也适用于漫反射光。漫反射光在表面之间来回反射，于是我们在一个特定表面上看到的颜色依赖于场景中其他表面的颜色，通过精心选择光源的漫反射光分量和镜面反射光分量，我们仍然可以利用局部计算来近似全局效果。

我们已经在对象空间里详细讨论了Phong光照模型，但是在绘制流水线中进行光照计算的方式可以多种多样。在OpenGL中，默认情况下是在观察坐标系下对每个顶点进行光照计算，然后在绘制流水线后面的处理中通过插值计算出每个片元的颜色值。我们必须注意模–视变换和投影变换对光照模型中要用到的向量会有什么影响，因为这些变换会改变模型中的余弦项（见习题6.20）。因此，为了得到正确的明暗计算结果，要么在绘制流水线对顶点和向量进行处理时保持它们的空间位置关系，为此可能需要在绘制流水线中传递来自对象空间的附加信息；要么返回到绘制流水线之前的处理阶段以获得明暗计算所需的信息。

6.3.4　改进的 Phong 模型

如果利用Phong模型进行绘制，那么在计算镜面反射分量时对表面上的每个点都要计算点积 $\mathbf{r} \cdot \mathbf{v}$。单位向量

$$\mathbf{h} = \frac{\mathbf{l} + \mathbf{v}}{|\mathbf{l} + \mathbf{v}|}$$

位于观察向量和光源向量这两者的正中间，该向量称为半角向量（halfway vector）。利用半角向量，我们可以得到镜面反射分量的另一种近似。注意，如果法向量和半角向量重合，那么从表面反射出去的光线在观察者的方向上具有最大的强度。图6.20中包含了所涉及的全部5个向量，这里我们把 $\mathbf{n}$ 和 $\mathbf{h}$ 之间的夹角 ψ 称为半角（halfway angle）。当 $\mathbf{v}$ 位于 $\mathbf{l}$、$\mathbf{n}$ 和 $\mathbf{r}$ 所在的平面时，可以证明（见习题6.7）：

$$2\psi = \phi$$

如果把 $\mathbf{r} \cdot \mathbf{v}$ 替换成 $\mathbf{n} \cdot \mathbf{h}$，就无须计算 $\mathbf{r}$。不过，由于半角 ψ 比 ϕ 要小，因此如果把 $(\mathbf{r} \cdot \mathbf{v})^e$ 中的指数 e 用于计算 $(\mathbf{n} \cdot \mathbf{h})^e$，那么镜面反射高光的区域会变小。为了减小把 $(\mathbf{r} \cdot \mathbf{v})$ 替换成 $(\mathbf{n} \cdot \mathbf{h})$ 后对明暗计算带来的影响，可以把指数 e 换成另一个指数 e'，使得 $(\mathbf{n} \cdot \mathbf{h})^{e'}$ 比 $(\mathbf{n} \cdot \mathbf{h})^e$ 更接近 $(\mathbf{r} \cdot \mathbf{v})^e$。我们希望避免对每个点都计算 $\mathbf{r}$，这一点是很清楚的，不过为了充分理解把 $\mathbf{r} \cdot \mathbf{v}$ 替换成 $\mathbf{n} \cdot \mathbf{h}$ 后，究竟在什么情况下计算量会减少，读者应该考察所有的情况：平面和曲面、远距离光源和近距离光源以及观察者在近处和远处（见习题6.8）。

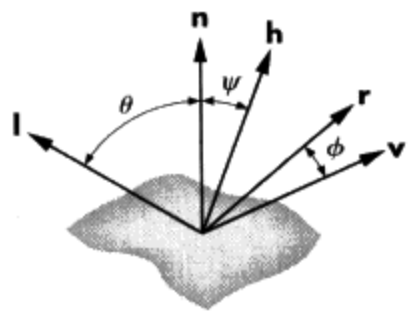


图6.20　半角向量的确定

上面对Phong模型进行了修改，利用半角向量来计算镜面反射分量，这种光照模型称为Blinn-Phong明暗模型或者改进的Phong明暗模型。这是OpenGL在默认情况下使用的模型，每个顶点在通过绘制流水线时都要按照这个模型进行明暗处理。读者需要注意，提出Phong模型和Blinn-Phong模型是为了获得BRDF的一种计算上可行的近似，而不是为了获得最佳的物理模型。利用第9章讨论的可编程流水线可以更好地近似BRDF，这些近似可以用硬件来实现。

彩图29展示的是在OpenGL中利用改进的Phong模型绘制的一组Utah茶壶（见12.10节）。注意，这些茶壶之所以看起来彼此不同是因为我们能够控制材质属性。这些茶壶演示了利用改进的Phong模型可以生成各种各样的明暗效果：从无光泽的表面到高度反光以至于看起来类似金属的表面。

6.4　向量的计算

我们已经导出的照明和反射模型具有足够广泛的适用范围：不管是曲面还是平面，也不管是平行投影还是透视投影，也不论是远距离的表面还是近距离的表面，都可以应用上节的光照模型。绘制场景的大部分计算涉及确定光照模型中所需要的向量和点积。在特殊情况下，这些计算有可能简化。例如，如果表面是平面多边形，那么表面上每一点处的法向量都相同；如果光源远离表面，那么光源方向对于表面上所有的点都是相同的。

这一节考察在一般情况下如何计算光照模型中所涉及的向量。在6.5节会看到当对象由平面多边形组成时，还可以利用其他技术。这种情况特别重要，因为包括OpenGL在内的大多数绘制器利用许多小的平面多边形来近似曲面。

6.4.1　法向量

在讨论法向量之前，我们先来回顾一下平面的方程。第4章中用下面的方程来表示平面：

$$f(\mathbf{p}) = ax + by + cz + d = 0$$

或者利用法向量和平面上的一点 $\mathbf{p}_0$ 把平面方程写成：

$$\mathbf{n} \cdot (\mathbf{p} - \mathbf{p}_0) = 0$$

比较平面方程的这两种形式，可以看到法向量 $\mathbf{n}$ 为：

$$\mathbf{n} = \begin{bmatrix} a \\ b \\ c \end{bmatrix}$$

或者利用齐次坐标表示为：

$$\mathbf{n} = \begin{bmatrix} a \\ b \\ c \\ 0 \end{bmatrix}$$

还可以用另一种方式确定平面的法向量：给定平面上的三个点 $\mathbf{p}_0$、$\mathbf{p}_1$、$\mathbf{p}_2$（这三个点不共线因而能够确定这个平面），那么向量 $\mathbf{p}_2 - \mathbf{p}_0$ 和 $\mathbf{p}_1 - \mathbf{p}_0$ 平行于该平面，于是可以利用它们的叉积得到法向量：

$$\mathbf{n} = (\mathbf{p}_2 - \mathbf{p}_0) \times (\mathbf{p}_1 - \mathbf{p}_0)$$

必须注意顶点在叉乘中出现的顺序，如果顺序颠倒了，会把表面的朝向从向外变成向内，这会影响到光照计算。一些图形系统利用在定义多边形时指定的前三个顶点自动计算出法向量。OpenGL没有这样做，但是我们将在6.5节看到，把计算法向量的任务留给应用程序可以为应用光照模型提供更多的灵活性。

对一个光滑的表面，可以在局部用平面来近似，这个近似平面的法向量给出了表面的局部方向。表面局部方向的计算依赖于表面表示的数学方式。假定一个表面由下面的隐式方程描述：

$$f(\mathbf{p}) = f(x, y, z) = 0$$

向量微积分的基本方法之一是考察多元Taylor级数的前几项，按照这个方法，如果$\mathbf{p}$和$\mathbf{p}_0$是光滑表面上相距很近的两点，那么

$$f(\mathbf{p}) = f(x, y, z) = (\mathbf{p_0}) = f(x_0, y_0, z_0) + \frac{\partial f}{\partial x}(x - x_0) + \frac{\partial f}{\partial y}(y - y_0)$$

$$+ \frac{\partial f}{\partial z}(z - z_0) + \cdots$$

如果定义梯度向量（gradient vector）为：

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \\ \frac{\partial f}{\partial z} \end{bmatrix}$$

那么可以写出下面的近似方程：

$$f(\mathbf{p}) = f(\mathbf{p_0}) + \nabla f^T(\mathbf{p} - \mathbf{p_0}) = 0$$

这是一个平面的方程，该平面的法向量是上面定义的梯度向量。

容易验证，平面的梯度就是它的法向量。对于由隐式方程表示的曲面，它的梯度给出的是局部法向量。现在考虑一个中心位于原点的单位球面，它可以由下面的隐式方程（implicit equation）[①]来表示：

$$f(x, y, z) = x^2 + y^2 + z^2 - 1 = 0$$

这个方程也可以写成下面的向量形式：

$$f(\mathbf{p}) = \mathbf{p} \cdot \mathbf{p} - 1 = 0$$

利用梯度求出的法向量为：

$$\mathbf{n} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \\ \frac{\partial f}{\partial z} \end{bmatrix} = \begin{bmatrix} 2x \\ 2y \\ 2z \end{bmatrix} = 2\mathbf{p}$$

因此，球面上每点处的法向量的方向是从原点指向这个点。对单位球面，规一化后的法向量为：

$$\mathbf{n} = \mathbf{p}$$

球面还可以表示成参数形式（parametric form），在这种表示中，球面上一点的x、y、z值都由两个参数u和v来表示：

① 第12章将详细讨论表示曲线和曲面的三种方式。

$$x = x(u, v)$$
$$y = y(u, v)$$
$$z = z(u, v)$$

这三个等式是彼此独立的。在第 12 章会看到，对于在计算机图形学中表示曲线和曲面来说，最有用的表示方式是参数形式，尽管对一个特定的表面可能会有多种参数表示。球心在原点的单位球面的一种参数表示为：

$$x(u, v) = \cos u \sin v$$
$$y(u, v) = \cos u \cos v$$
$$z(u, v) = \sin u$$

当 u 和 v 在 $-\pi/2 < u < \pi/2$，$-\pi < v < \pi$ 的范围内变化时，按上面的参数方程可以得到球面上所有的点。在使用参数形式时，可以从点 $\mathbf{p}(u, v) = [x(u, v)\ y(u, v)\ z(u, v)]^T$ 处的切平面（tangent plane）得到法向量，如图 6.21 所示。切平面给出了表面在一点处的局部方向。从表面在点 $\mathbf{p}$ 处的多元 Taylor 级数展开式中的线性项可以得到切平面的方程。由此可得：经过点 $\mathbf{p}$ 并且方向沿着下面两个向量的两条直线都位于切平面内：

$$\frac{\partial \mathbf{p}}{\partial u} = \begin{bmatrix} \frac{\partial x}{\partial u} \\ \frac{\partial y}{\partial u} \\ \frac{\partial z}{\partial u} \end{bmatrix}, \qquad \frac{\partial \mathbf{p}}{\partial v} = \begin{bmatrix} \frac{\partial x}{\partial v} \\ \frac{\partial y}{\partial v} \\ \frac{\partial z}{\partial v} \end{bmatrix}$$

可以通过它们的叉积得到法向量：

$$\mathbf{n} = \frac{\partial \mathbf{p}}{\partial u} \times \frac{\partial \mathbf{p}}{\partial v}$$

对单位球面可得：

$$\mathbf{n} = \cos u \begin{bmatrix} \cos u \sin v \\ \cos u \cos v \\ \sin u \end{bmatrix} = (\cos u)\mathbf{p}$$

我们只对 $\mathbf{n}$ 的方向感兴趣，因此可以除以 cosu，得到球面的单位法向量：

$$\mathbf{n} = \mathbf{p}$$

6.9 节将利用这个结果对球面的多边形逼近进行明暗绘制。

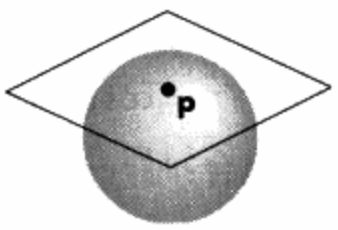


图 6.21　球面的切平面

从上面这些例子可以看到，对于多边形，可以直接从顶点数据计算出所需的法向量；对于曲面，如果知道其解析表达式（无论是隐式方程还是参数方程都可以），那么可以自己计算出所需的导数，然后把结果编写到程序里，但通常不能指望图形系统自动得出法向量的解析表达式。稍微从不同的角度来考察，我们看到在图形系统内部表面是由一组顶点来表示的，要想求出某一点处的法向量，必须通过和这个点距离相近的一些点来近似。实时图形系统的流水线结构使这个计算变得困难，因为我们一次处理一个顶点，于是图形系统可能没有可用的信息来计算给定点处法向量的近似值。因此，图形系统通常把计算法向量的任务留给应用程序来完成。

在OpenGL中，当前的法向量是状态的一部分，下面这两个函数都可以修改当前的法向量：

```
glNormal3f(nx, ny, nz);
glNormal3fv(pointer_to_normal);
```

通过在每次执行glVertex之前执行glNormal，可以把一个顶点和一个法向量联系起来。对每个顶点进行光照计算时使用的是当前的法向量。必须记住：这些法向量要由用户来确定。

6.4.2　反射角

一旦计算出了法向量，就可以用法向量和光源的方向来计算理想反射的方向。理想镜面的特征是：入射角等于反射角。这两个角度如图6.22所示。入射角（angle of incidence）是法线和光源（假定是点光源）方向之间的夹角，反射角（angle of reflection）是法线和反射光线之间的夹角。在平面内，只有一个反射方向能够满足入射角等于反射角这个条件。但是在空间中，这个条件不能确定欲求的反射方向，因为有无穷多个方向都满足入射角等于反射角的条件，因此必须再加上一个条件：在表面上的一点**p**处，入射光线、反射光线和法线必须位于同一平面内。由这两个条件可以由**n**和**l**确定**r**。我们感兴趣的是**r**的方向，而不是**r**的长度。不过，单位长度的向量会使绘制计算变得更容易，因此假定**l**和**n**都已经被归一化，即

$$|\mathbf{l}| = |\mathbf{n}| = 1$$

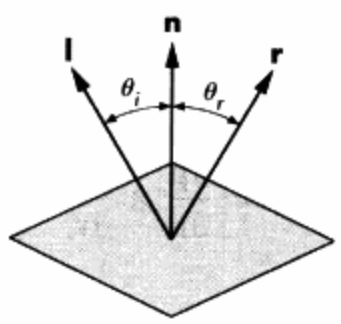


图6.22　理想的镜面反射

我们也希望

$$|\mathbf{r}| = 1$$

如果$\theta_i = \theta_r$，那么

$$\cos\theta_i = \cos\theta_r$$

我们可以利用点积把角度条件写成：

$$\cos\theta_i = \mathbf{l}\cdot\mathbf{n} = \cos\theta_r = \mathbf{n}\cdot\mathbf{r}$$

共面条件意味着可以把**r**写成**l**和**n**的线性组合：

$$\mathbf{r} = \alpha\mathbf{l} + \beta\mathbf{n}$$

等号两边都和**n**做点积，得：

$$\mathbf{n}\cdot\mathbf{r} = \alpha\mathbf{l}\cdot\mathbf{n} + \beta = \mathbf{l}\cdot\mathbf{n}$$

前面我们要求**r**的长度也是1，由此可以得到关于α和β的第二个方程：

$$1 = \mathbf{r}\cdot\mathbf{r} = \alpha^2 + 2\alpha\beta\mathbf{l}\cdot\mathbf{n} + \beta^2$$

求解这两个方程，可得：

$$\mathbf{r} = 2(\mathbf{l}\cdot\mathbf{n})\mathbf{n} - \mathbf{l}$$

虽然OpenGL中功能固定的绘制流水线使用改进的Phong模型，因而不需要计算反射向量，但是第9章介绍的可编程着色器可以实现需要用到反射向量的光照模型。像环境贴图这样的方法利用反射后的观察向量（见习题6.24）来确定在高度反光的表面（例如高度抛光的球面）上形成的周围环境的图像。

6.5　多边形明暗绘制

在大多数情况下，交互式的计算机图形系统绘制的是多边形。从硬件的角度来看，图形绘制系统为了利用绘制流水线处理多边形而进行了优化。图形绘制系统的性能是以每秒钟能够绘制的多边形数目来度量的，这里对多边形的绘制总是包括光照和明暗处理。从应用的角度来看，在许多CAD软件中，用户所设计的是多边形网格。即使这些软件支持曲面的绘制，那些曲面也是通过多边形网格来近似的。

因为多边形上所有的点都具有相同的法向量，所以法向量只需要计算一次。法向量往往存储在应用程序的数据结构中。在许多特殊情况下，比如远距离光源，多边形明暗绘制的效率还能进一步提高，同时我们也必须小心避免视觉假象。这一节讨论对多边形网格（如图6.23所示）进行明暗绘制的三种方法：均匀着色、平滑或者插值（Gouraud）着色和Phong着色。

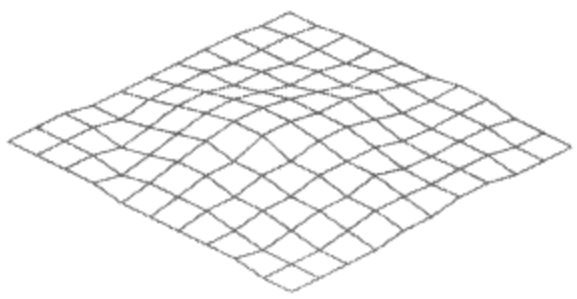


图 6.23　多边形网格

前两种方法基于每个顶点进行光照计算，即在绘制流水线的几何处理阶段对一个或者多个顶点进行明暗计算，然后在绘制流水线的片元处理阶段通过对这些顶点的颜色进行插值得到片元的颜色。可编程处理器的出现使得可以基于每个片元进行明暗处理。每个像素的颜色可以直接计算出来而不是通过插值实现，因此可以实时实现Phong着色这样的明暗绘制算法，而在这之前还只能通过离线绘制软件来实现。

6.5.1　均匀着色

明暗绘制所需的三个向量**l**、**n**和**v**都随表面上位置的不同而改变。不过，平面多边形上所有点处的**n**都相同；如果假定观察者在远处[①]，那么多边形上所有点处的**v**都相同；如果光源在远处，那么所有点处的**l**都相同。这里的“在远处”可以解释成无穷远处。按照这种解释，可以对明暗方程及其实现进行必要的调整，例如把光源的位置用光源的方向来代替。“在远处”还可以解释成多边形的尺寸相对于多边形到光源或者观察者的距离而言很小，如图6.24所示。图形系统或者用户程序经常使用第二个解释。

如果上面提到的这三个向量都是常量，那么对每个多边形只需要进行一次明暗计算，并且多边形上所有点的颜色值都相同。这种明暗绘制技术称为均匀着色（flat shading或constant shading）。在OpenGL中，下面的语句可以把进行明暗绘制的方法指定为均匀着色：

```
glShadeModel(GL_FLAT);
```

① 在OpenGL中，如果把局部观察者标记设置成false，那么在进行明暗绘制时就假定观察者在远处。

如果当前应用的是均匀着色，那么OpenGL在对单个多边形进行明暗绘制时使用和多边形第一个顶点相联系的法向量。对于像三角形带这样的图元，OpenGL对第一个三角形使用第三个顶点的法线，对第二个三角形使用第四个顶点的法线，以此类推。类似的规则适用于其他图元，例如四边形带。

图 6.24　和场景相距很远的光源和观察者

均匀着色会使多边形网格中不同的多边形具有不同的颜色。如果光源和观察者距离多边形比较近，那么每个多边形的**l**和**v**都可能不同。不过，如果用多边形网格来建模光滑表面，那么均匀着色产生的效果几乎总是会使人失望，因为相邻多边形之间的亮度差异即使非常小，在人们看来也很明显，如图6.25所示。人们对亮度的微小差异非常敏感，这是因为人类视觉系统具有一种称为侧抑制（lateral inhibition）的性质。当观察如图6.26所示的亮度递增的一组条带时，会觉得在亮度发生跳变的一侧有亮度过冲，而另一侧有亮度下冲，如图6.27所示。在亮度变化的边界，我们在较亮一侧看到一条更亮的线，在较暗一侧看到一条更暗的线，这称为Mach带（Mach bands）效应。Mach带效应可以从眼睛里的视锥细胞和视神经相连接的方式中找到解释，除了寻找更平滑的明暗绘制技术，使得相邻多边形的明暗值相差不太大以外，没有什么办法能够避免这种现象。

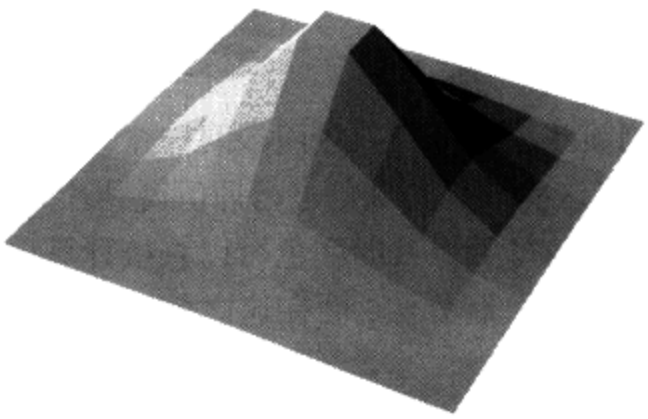


图 6.25　利用均匀着色绘制的多边形网格

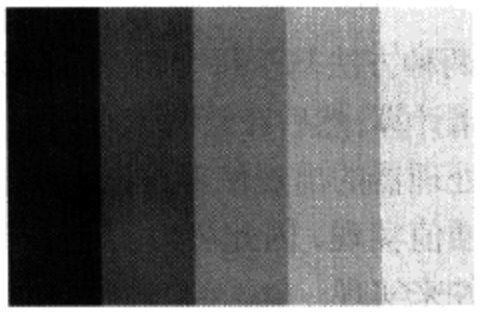


图 6.26　亮度阶梯

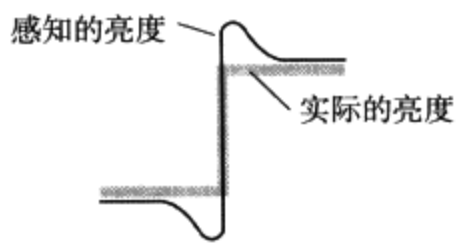


图 6.27　边缘处的实际亮度和感知亮度

6.5.2　平滑着色

在4.9节旋转立方体的例子中，我们看到OpenGL通过对指派给每个顶点的颜色进行插值，可以得出多边形中所有位置处的颜色值。平滑着色是在默认情况下OpenGL使用的明暗绘制方法，也可以通过下面的语句把当前的多边形明暗绘制方法显式地指定为平滑着色：

```
glShadeModel(GL_SMOOTH);
```

假定我们已经在OpenGL中开启了光照并把多边形明暗绘制方法指定为平滑着色，而且对多边形的每个顶点所指派的法向量都是这个多边形的法向量。OpenGL对每个顶点都根据材质属性以及计算出的 **v** 和 **l** 进行光照计算。注意，如果光源在远处，并且观察者在远处或者表面材质没有镜面反射分量，那么按照平滑（插值）着色绘制的结果是多边形内所有点的颜色值都相同。

对于多边形网格，如果每个顶点处都存在法向量，这一点会使任何关心数学正确性的人感到不安。因为多边形相交于网格内部的顶点，而每个多边形又都有自己的法向量，所以法向量在顶点处不连续。虽然这种情形使得数学处理变得复杂，但是Gouraud意识到，可以对每个顶点的法向量给出一种定义，利用该定义进行明暗绘制可以获得更平滑的插值效果。考虑一个位于网格内部的顶点，如图 6.28 所示，有 4 个多边形相交于该顶点，每个多边形都有自己的法向量。在 Gouraud 着色（Gouraud shading）这种明暗绘制方法中，我们对共用一个顶点的多边形的法向量取平均值，把归一化后的平均值定义为该顶点的法向量，对图 6.28 所示的这个顶点，其法向量定义为：

$$\mathbf{n} = \frac{\mathbf{n}_1 + \mathbf{n}_2 + \mathbf{n}_3 + \mathbf{n}_4}{|\mathbf{n}_1 + \mathbf{n}_2 + \mathbf{n}_3 + \mathbf{n}_4|}$$

从 OpenGL 的角度来看，Gouraud 着色似乎很简单，只需要正确地设置顶点的法向量并应用光照模型即可。因此，文献中经常对平滑着色和Gouraud着色不做区分。然而，这两个术语的混用掩盖了一个问题：每个顶点的法向量被定义成几个多边形的法向量的归一化平均值，可我们如何找出这几个多边形的法向量呢？如果程序按照线性结构来组织顶点，即指定一个顶点列表（列表中也包含其他属性），那么在计算法向量时缺少必要的信息：不知道哪些多边形共用一个顶点。如果利用流水线绘制器，那么一次绘制一个多边形，并且每个多边形的绘制不依赖于其他多边形的绘制。

我们所需要的当然是一种表示多边形网格的数据结构，其中包含了关于在每个顶点处有哪些多边形相交的信息，遍历这个数据结构就可以求出法向量的平均值。这样的数据结构至少应该包含多边形、顶点、法向量和材质属性。一种可能的数据结构如图 6.29 所示。这是一个修改后的顶点列表，其中左边的每个节点指向相交于这个顶点的多边形列表。这个数据结构可以通过每个多边形来访问，也可以通过一个表示多边形网格的数据结构（例如第4章用来表示立方体的顶点列表）来访问。

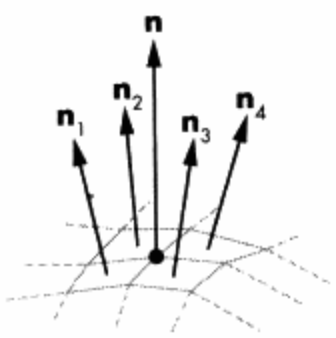


图 6.28　内部顶点附近的法向量

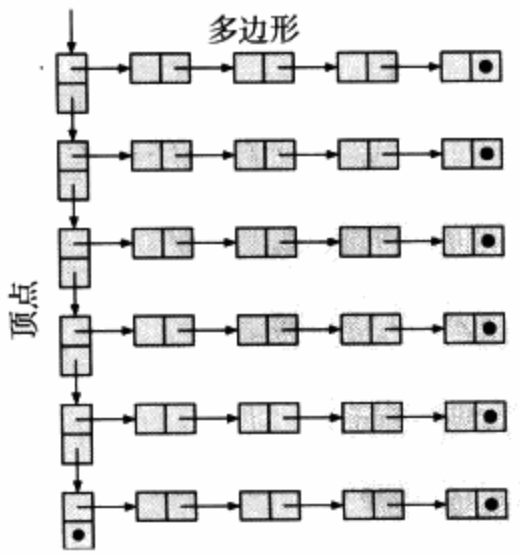


图 6.29　网格数据结构

彩图 8 和彩图 9 演示了利用 OpenGL 可以获得的明暗绘制效果。在彩图 8 中只有一个光源，所用的光照模型是改进的 Phong 模型，对每个多边形计算出的明暗值都是常数（均匀着色）。在生成彩图 9 时，每个顶点都指派了一个法向量，然后 OpenGL 计算出每个顶点的明暗值，并通过插值计算出多边形内部的明暗值。

彩图25演示了OpenGL生成的平滑明暗绘制效果。第2章和第3章曾把这个彩图作为例子，生成该图的源程序见附录A。图中立方体的8个顶点的颜色分别为黑、白、红、绿、蓝、青、品红和黄。如果当前多边形明暗绘制模式为平滑着色，那么OpenGL就自动在多边形内部进行颜色插值。

6.5.3 Phong着色

即便是对应用Gouraud着色获得的平滑图像，我们也还是能察觉出Mach带效应。Phong提出的方法是在多边形内对法向量进行插值，而不是像Gouraud着色那样对顶点的明暗值进行插值。插值出法向量之后就可以对多边形内的每一点都应用光照模型。注意，因为法向量给出了表面的局部方向，通过在多边形内对法向量进行插值，至少是对明暗绘制而言，可以使得绘制的表面看起来像是光滑的曲面，观察者不会察觉出是由平面多边形近似的。这解释了为什么利用Phong着色绘制的图像非常平滑。

图6.30所示是多边形网格中的一个多边形，该多边形和其他多边形共用一些顶点和边。为了求出某个顶点的法向量，可以对共用该顶点的那些多边形的法向量进行插值。接下来，可以像第4章所讲的那样应用双线性插值计算出多边形内任一点的法向量。考虑图6.31，我们对经过插值得出的顶点A和B处的法向量再进行插值，可以求出AB边上任一点的法向量：

$$\mathbf{n}(\alpha) = (1-\alpha)\mathbf{n}_A + \alpha\mathbf{n}_B$$

对多边形的每条边都可以进行类似的插值。多边形内任一点的法向量可以从两条边上的点的法向量得到：

$$\mathbf{n}(\alpha, \beta) = (1-\beta)\mathbf{n}_C + \beta\mathbf{n}_D$$

一旦知道了各点的法向量，就可以对各点进行彼此独立的明暗计算。通常这个计算过程可以和多边形的光栅化结合起来进行。直到最近，Phong着色仍然只能以离线方式实现，因为Phong着色需要在每个多边形内对法向量进行插值。从绘制流水线的角度来看，Phong着色要求把光照模型应用到每个片元上，因而也被称为基于片元的着色（per-fragment shading）。在最新的图形卡中，光栅化模块是可编程的，从而允许程序员直接对片元进行操作。因此，现在我们可以实时实现Phong着色，第9章将详细讨论这方面的内容。

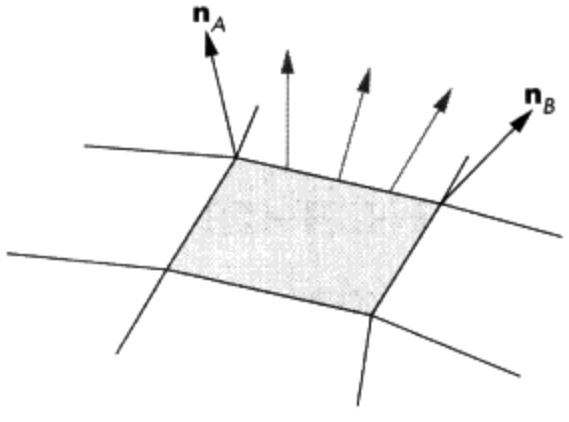


图6.30　多边形边上的法向量

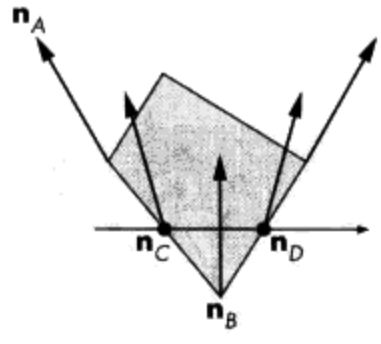


图6.31　Phong着色中对法向量的插值

6.6　通过递归细分逼近球面

前面我们把球面作为曲面的例子来说明了如何进行明暗计算。不过，球面不是OpenGL中的图元。GL工具库（GLU）和GL实用工具库（GLUT）都支持球面，前者通过二次曲面生成球面（我们将在第12章讨论二次曲面），后者通过多边形来近似球面。

这一节讨论如何利用多边形来近似球面，在此基础上编写一个简单的程序，该程序演示了明暗绘制参数和曲面的多边形逼近之间的相互影响。我们介绍递归细分（recursive subdivision）方法，这是一种强有力的技术，利用它可以以任意精度逼近曲线和曲面。

出发点是一个四面体，不过还可以从各个面被分割成三角形的正多面体开始近似[①]。正四面体是由 4 个等边三角形围成的，由 4 个顶点确定。我们从$(0, 0, 1)$、$(0, 2\sqrt{2}/3, -1/3)$、$(-\sqrt{6}/3, -\sqrt{2}/3, -1/3)$和$(\sqrt{6}/3, -\sqrt{2}/3, -1/3)$这4个顶点开始近似球面。这4个顶点都位于以原点为中心的单位球面上，彼此之间的距离都相等，因而它们确定了一个正四面体的 4 个面。习题6.6指出了一种计算这 4 个点坐标的方法。

通过绘制正四面体的线框图，可以得到球体的第一个近似。下面的语句指定正四面体的 4 个顶点：

```
v[4][3]={{0.0, 0.0, 1.0}, {0.0, 0.942809, -0.333333},
                {-0.816497, -0.471405, -0.333333},
                {0.816497, -0.471405, -0.333333}};
```

下面的函数用来绘制三角形：

```
void triangle( GLfloat *a, GLfloat *b, GLfloat *c)
{
  glBegin(GL_LINE_LOOP);
    glVertex3fv(a);
    glVertex3fv(b);
    glVertex3fv(c);
  glEnd();
}
```

可以用下面的函数来绘制四面体：

```
void tetrahedron()
{
   triangle(v[0], v[1], v[2] );
   triangle(v[3], v[2], v[1] );
   triangle(v[0], v[3], v[1] );
   triangle(v[0], v[2], v[3] );
}
```

其中顶点的顺序服从右手规则，因此修改上面的代码以便对多边形进行明暗绘制并不困难。如果再增加通常的初始化代码，程序就能生成如图6.32所示的图像，这是一个简单的正多面体，但对于近似球体来讲不算好。

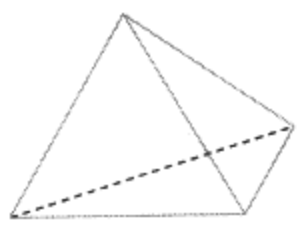


图 6.32　四面体

通过把四面体的每个面细分成更小的三角形，可以得到更接近球体的多面体。细分成三角形可以确保所有新增加的多边形都是平面图形。如图6.33所示，对三角形进行细分至少有三种方法。我们可以画出三角形的三条角平分线，它们相交于一点，于是就得到了三个新的三角形。我们还可以通过求三个顶点的平均值来计算出这三个顶点的质心，然后从质心到三个顶点画三条线，这样也可

① 正二十面体的 20 个面都是等边三角形，对于近似球体而言是一个很好的出发点，见[Ope05]。

以得到三个三角形。然而，这两种办法得到的新三角形和正四面体的4个面不同：它们不再是等边三角形了。回忆第2章绘制Sierpinski镂垫时采用的办法，我们可以连接三角形三条边的中点，组成4个等边三角形，如图6.33(c)所示。下面我们将使用这种办法细分三角形。

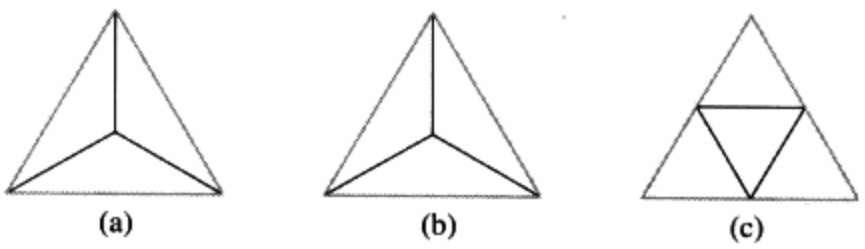


图6.33　通过(a)平分各角、(b)计算出中心、(c)等分各条边来细分三角形

按照上面的办法对四面体的一个面进行细分之后，得到的4个新三角形仍旧位于原来三角形所在的平面。我们可以通过归一化把新顶点移动到单位球面上，这些新顶点是原来三角形各边的中点。归一化可以通过像下面这样的简单函数来实现：

```
void normalize(GLfloat *p)
{
  double d=0.0;
  int i;
  for(i=0; i<3; i++) d+=p[i]*p[i];
  d=sqrt(d);
  if(d > 0.0) for(i=0; i<3; i++) p[i]/=d;
}
```

然后可以用下面的代码把三角形abc细分：

```
GLfloat v1[3], v2[3], v3[3];
int j;
for(j=0; j<3; j++) v1[j]=v[a][j]+v[b][j];
normalize(v1);
for(j=0; j<3; j++) v2[j]=v[a][j]+v[c][j];
normalize(v2);
for(j=0; j<3; j++) v3[j]=v[c][j]+v[b][j];
normalize(v3);
triangle(v[a], v2, v1);
triangle(v[c], v3, v2);
triangle(v[b], v1, v3);
triangle(v1, v2, v3);
```

如果在函数tetrahedron中使用类似上面的代码，就可以生成16个而不是4个三角形，不过我们更希望把细分过程重复进行n次以生成越来越接近球体的多面体。通过递归调用细分子程序，可以控制细分的次数。

首先，我们通过增加一个参数n使函数tetrahedron依赖于递归深度。

```
void tetrahedron(int n)
{
   divide_triangle(v[0], v[1], v[2] , n);
   divide_triangle(v[3], v[2], v[1], n );
   divide_triangle(v[0], v[3], v[1], n );
   divide_triangle(v[0], v[2], v[3], n );
}
```

如果n大于0，那么函数divide_triangle调用自身，但如果n已经减少到0就生成三角形。下面是这个函数的代码：

```
divide_triangle(GLfloat *a, GLfloat *b, GLfloat *c, int n)
{
 GLfloat v1[3], v2[3], v3[3];
 int j;
 if(n>0)
 {
   for(j=0; j<3; j++) v1[j]=a[j]+b[j];
   normalize(v1);
   for(j=0; j<3; j++) v2[j]=a[j]+c[j];
   normalize(v2);
   for(j=0; j<3; j++) v3[j]=c[j]+b[j];
   normalize(v3);
   divide_triangle(a ,v2, v1, n-1);
   divide_triangle(c ,v3, v2, n-1);
   divide_triangle(b ,v1, v3, n-1);
   divide_triangle(v1 ,v2, v3, n-1);
 }
 else triangle(a, b, c);
}
```

图6.34所示为利用这些代码绘制的近似球面。下面要对近似球面增加光照和明暗效果，为此我们先来考察 OpenGL API 如何处理光照和明暗。

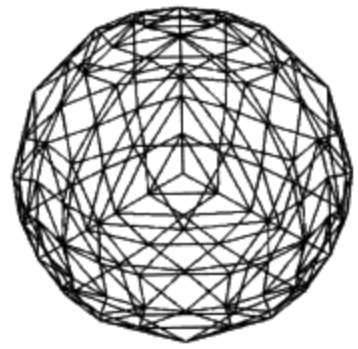


图 6.34　通过细分得到的近似球面

6.7　OpenGL 中的光源

OpenGL 支持 6.2 节讨论过的 4 类光源：点光源、聚光灯、环境光和远距离光源。在一个程序中允许出现的最大光源数目和 OpenGL 的具体实现有关，但至少是 8 个。我们必须单独指定和开启每个光源。虽然要指定的参数很多，但它们其实就是改进的 Phong 光照模型中的那些参数。下面这两个 OpenGL 函数分别指定所需要的向量和标量参数：

```
glLightfv(GLenum source, GLenum parameter, GLfloat *pointer_to_array)
glLightf(GLenum source, GLenum parameter, GLfloat value)
```

我们可以设置 4 个向量参数：光源的位置（或者方向）和这个光源的环境光强度、漫反射光强度和镜面反射光强度。

例如，假定我们希望对第一个光源 GL_LIGHT0 进行设置。下面的语句用齐次坐标表示光源的位置：

```
GLfloat light0_pos[]={1.0, 2.0, 3.0, 1.0};
```

如果像下面这样把第 4 个分量设置成 0，那么点光源就变成了远距离光源，前三个分量表示光源的方向：

```
GLfloat light0_dir[]={1.0, 2.0, 3.0, 0.0};
```

如果想把这个光源设置成具有白色的镜面反射光分量、红色的环境光分量和红色的漫反射光分量，可以使用下面的代码：

```
GLfloat diffuse0[]={1.0, 0.0, 0.0, 1.0};
GLfloat ambient0[]={1.0, 0.0, 0.0, 1.0};
GLfloat specular0[]={1.0, 1.0, 1.0, 1.0};

glEnable(GL_LIGHTING);
glEnable(GL_LIGHT0);

glLightfv(GL_LIGHT0, GL_POSITION, light0_pos);
glLightfv(GL_LIGHT0, GL_AMBIENT, ambient0);
glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuse0);
glLightfv(GL_LIGHT0, GL_SPECULAR, specular0);
```

注意，必须开启光照绘制并开启光源GL_LIGHT0。

还可以增加一个独立于任何光源的全局环境光。例如，如果想在所有光源都没有开启时场景中也有少量的白光照明，可以利用下面的代码：

```
GLfloat global_ambient[]={0.1, 0.1, 0.1, 1.0};
glLightModelfv(GL_LIGHT_MODEL_AMBIENT, global_ambient);
```

距离衰减因子的形式为：

$$f(d)=\frac{1}{a+bd+cd^2}$$

其中包含了常数项、线性项和二次项，通过函数glLightf可以分别对这些项进行设置。利用下面代码设置的结果和默认值相同：

```
glLightf(GL_LIGHT0, GL_CONSTANT_ATTENUATION, 1.0);
glLightf(GL_LIGHT0, GL_LINEAR_ATTENUATION, 0.0);
glLightf(GL_LIGHT0, GL_QUADRATIC_ATTENUATION, 0.0);
```

这些代码把距离衰减因子设置成常数。

通过设置聚光灯的方向（GL_SPOT_DIRECTION）、指数（GL_SPOT_EXPONENT）和角度（GL_SPOT_CUTOFF），我们可以把位置光源转换成聚光灯。所有这三个参数都由函数glLightf或者glLightfv来指定。

OpenGL还提供了另外两个光照参数：GL_LIGHT_MODEL_LOCAL_VIEWER和GL_LIGHT_MODEL_TWO_SIDE。光照计算往往很费时间，如果假定观察者距离场景无穷远，那么计算反射光线会变得容易一些，因为从场景中任一点到观察者的方向是常量。默认情况下OpenGL认为此假定成立，因为这对许多场景的影响很小。如果希望针对观察者的实际位置进行完全的光照计算，可以通过下面的代码来改变默认设置：

```
glLightModeli(GL_LIGHT_MODEL_LOCAL_VIEWER, GL_TRUE);
```

第4章曾讲到表面既有正面又有背面。对于多边形，我们按照右手规则通过顶点被指定的顺序来确定正面和背面。对大多数对象，我们只看到正面，因此不关心OpenGL如何计算背面的明暗值。例如，对于凸对象，如球面或者平行六面体（见图6.35），观察者不管位于何处都看不到背面。然而，如果去掉立方体的一个面或者切掉球面的一部分，如图6.36所示，那么处于合适位置的观察者就会看到背面，因此，我们必须对表面的正面和背面都进行正确的明暗绘制。下面的函数可以确保OpenGL正确绘制正面和背面：

```
glLightModeli(GL_LIGHT_MODEL_TWO_SIDED, GL_TRUE);
```

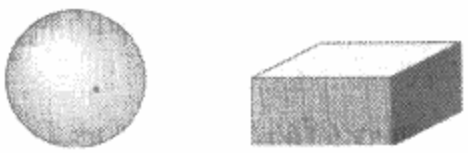


图 6.35　凸对象的明暗绘制

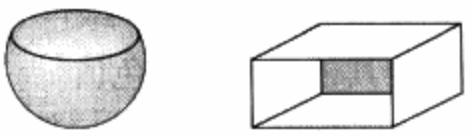


图 6.36　可见的背面

光源是特殊类型的几何对象，也像多边形和点那样具有几何属性，例如位置。因此，OpenGL 中的模–视变换也会影响光源。可以把光源指定在想要的位置，也可以先把它们定义在一个方便的位置然后再通过模–视变换将其移动到想要的位置。对象经过模–视变换后被转换到观察坐标系中，这里的模–视变换是在定义对象时生效的模–视变换。因此，通过小心调整程序中设置光源的代码相对于定义其他几何对象的代码的位置，可以实现这样的效果：光源在对象移动时保持静止；或者光源在对象保持静止时移动；或者光源和对象一起移动。注意，因为光源的位置或方向要和模–视变换矩阵相乘，所以被存储在观察坐标系中。在第 9 章讨论编程实现我们自己的着色器时，因为这个事实我们将在观察坐标系中计算明暗值。

6.8　OpenGL 中的材质属性

OpenGL 中的材质属性直接对应于所支持的光源和改进的 Phong 光照模型。我们可以为表面的正面和背面设置不同的材质属性。利用下面两个函数可以指定材质属性的所有参数：

```
glMaterialfv(GLenum face, GLenum type, GLfloat * pointer_to_array)
glMaterialf(GLenum face, GLenum type, GLfloat value)
```

例如，可以像下面这样通过三个数组来指定环境光反射系数 k_a、漫反射系数 k_d 和镜面反射系数 k_s 的每个颜色分量：

```
GLfloat ambient[] = {0.2, 0.2, 0.2, 1.0};
GLfloat diffuse[] = {1.0, 0.8, 0.0, 1.0};
GLfloat specular[]={1.0, 1.0, 1.0, 1.0};
```

按照上面的参数所指定的表面在光线照射下会反射少量的白色环境光，并反射黄色的漫反射光和白色的镜面反射光。调用下面的函数可以同时设置正面和背面的材质属性：

```
glMaterialfv(GL_FRONT_AND_BACK, GL_AMBIENT, ambient);
glMaterialfv(GL_FRONT_AND_BACK, GL_DIFFUSE, diffuse);
glMaterialfv(GL_FRONT_AND_BACK, GL_SPECULAR, specular);
```

如果镜面反射系数和漫反射系数相同（情况经常如此），那么把 GL_DIFFUSE_AND_SPECULAR 作为 type 参数就可以同时设置这两个反射系数。为了使正面和背面具有不同的材质属性，可以分别把 GL_FRONT 和 GL_BACK 作为 face 参数。表面的高光系数（镜面反射分量计算公式中的指数）由函数 glMaterialf 来指定，例如，

```
glMaterialf(GL_FRONT_AND_BACK, GL_SHININESS, 100.0);
```

在 OpenGL 中还可以指定表面的自发光分量，这会使表面本身可以发光。如果希望图像中出现光源，就可以使用这种方法。任何光源都不会影响表面的自发光分量，表面的自发光分量也不会影响任何其他表面，它只为表面的颜色值增加了一个固定的贡献。指定自发光强度的方式和其他材质属性类似，例如，

```
GLfloat emission[]={0.0, 0.3, 0.3, 1.0};

glMaterialfv(GL_FRONT_AND_BACK, GL_EMISSION, emission);
```

把表面的自发光设置成较弱的蓝-绿色（青色）光。然而，这个自发光对环境中的光照没有任何贡献，因此它不影响其他对象的颜色值。

材质属性也是OpenGL状态的一部分，除非被修改，否则它们的值不会改变。如果修改了材质属性，那么只会影响在修改之后定义的表面。如果用到了许多不同种类的材质，并且每当开始绘制一个新的多边形时都要改变材质属性，就会导致很大的开销。OpenGL提供了一种方法：利用glColorMaterial来改变单独一个材质属性。如果只修改单独一种材质属性的颜色分量，这种方法更有效，但利用glMaterial来设置材质属性的方法更一般。

从应用程序编程人员的角度来看，我们希望能通过单独一次函数调用就设置好材质属性，而不是对每个多边形都调用好多次glMaterial。为了按这种方式来设置材质属性，可以在应用程序中利用结构体或者类来定义材质对象。例如，考虑下面的typedef语句：

```
typedef struct materialStruct
   GLfloat ambient[4];
   GLfloat diffuse[4];
   GLfloat specular[4];
   GLfloat shininess;
 materialStruct;
```

有了上面的定义，就可以利用下面的代码来指定材质属性了：

```
materialStruct brassMaterials =
{
    0.33, 0.22, 0.03, 1.0,
    0.78, 0.57, 0.11, 1.0,
    0.99, 0.91, 0.81, 1.0,
    27.8
};
```

然后通过指针来访问这些属性：

```
currentMaterial=&brassMaterials;
```

我们可以利用下面这个函数来设置材质属性：

```
void materials( materialStruct *materials)
{
    glMaterialfv(GL_FRONT, GL_AMBIENT, currentMaterial->ambient);
    glMaterialfv(GL_FRONT, GL_DIFFUSE, currentMaterial->diffuse);
    glMaterialfv(GL_FRONT, GL_SPECULAR, currentMaterial->specular);
    glMaterialf(GL_FRONT, GL_SHININESS, currentMaterial->shininess);
}
```

6.9　球面模型的明暗绘制

现在可以对球面进行明暗绘制了。这里省略了OpenGL的标准初始化代码，读者在附录A中可以找到完整的程序。

要利用OpenGL中的光照模型对近似球面进行明暗绘制，必须指定法向量。一种简单的方法是对每个三角形进行均匀着色，为此我们利用三角形的三个顶点确定一个法向量，然后把这个法向量指派给三角形的第一个顶点。按照6.4节所讲的方法，我们先进行叉积再归一化。下面的函数可以实现叉积：

```
cross(GLfloat *a, GLfloat *b, GLfloat *c, GLfloat *d);
{
     d[0]=(b[1]-a[1])*(c[2]-a[2])-(b[2]-a[2])*(c[1]-a[1]);
```

```
    d[1]=(b[2]-a[2])*(c[0]-a[0])-(b[0]-a[0])*(c[2]-a[2]);
    d[2]=(b[0]-a[0])*(c[1]-a[1])-(b[1]-a[1])*(c[0]-a[0]);
    normalize(d);
}
```

假定已经设置并开启了光源，我们可以修改 triangle 函数，使其生成三角形的明暗图：

```
void triangle(GLfloat *a, GLfloat *b, GLfloat *c)
{
    GLfloat n[3];
    cross(a, b, c, n);
    glBegin(GL_POLYGON);
       glNormal3fv(n);
       glVertex3fv(a);
       glVertex3fv(b);
       glVertex3fv(c);
    glEnd();
}
```

对近似球面进行均匀着色的结果如图6.37所示。请读者注意，如果增加细分的次数，那么虽然球面的内部看起来会比较平滑，但仍然能分辨出球面图像的外侧边缘是由多边形的边组成的。这种外侧的边缘称为轮廓边缘（ silhouette edge ）。

对球面应用插值着色非常容易，因为我们知道每点**p**处的法向量的方向是从球心指向**p**。可以把真实的球面法向量指派给每个顶点，然后OpenGL通过对三角形三个顶点的明暗值进行插值就可以计算出三角形内部各点处的明暗值。我们把函数 triangle 修改为：

```
void triangle(GLfloat *a, GLfloat *b, GLfloat *c)
{
    GLfloat n[3];
    int i;
    glBegin(GL_POLYGON);
       for(i=0;i<3;i++) n[i]=a[i];
       normalize(n);
       glNormal3fv(n);
       glVertex3fv(a);
       for(i=0;i<3;i++) n[i]=b[i];
       normalize(n);
       glNormal3fv(n);
       glVertex3fv(b);
       for(i=0;i<3;i++) n[i]=c[i];
       normalize(n);
       glNormal3fv(n);
       glVertex3fv(c);
    glEnd();
}
```

按照这样定义的法向量绘制的结果如图 6.38 和彩图 37 所示。

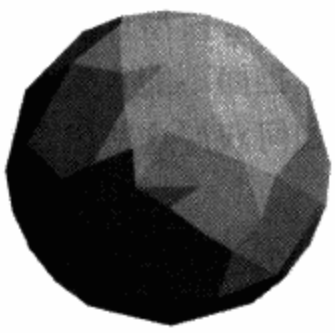


图 6.37　利用均匀着色绘制的球面

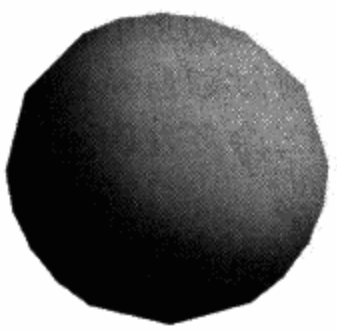


图 6.38　利用真实的球面法向量绘制的明暗图

虽然利用真实的球面法向量生成的明暗图比利用均匀着色生成的明暗图更逼真，但是这个例子不代表一般的情况，因为我们使用了已知解析表达式的法向量。我们也没有完全按照Gouraud着色算法来进行明暗绘制，因为在生成图6.38时使用了每个顶点处真实的球面法向量，所以该图比利用Gouraud着色生成的明暗图稍好一些，而且从该图可以看到，即使使用数目相对较少的多边形来近似球面也可以获得不错的效果。

假定我们希望利用Gouraud着色对近似球面进行明暗绘制。对于每一个顶点，需要知道共用这个顶点的所有多边形的法向量。程序没有使用可以包含这种信息的数据结构，也不包含能够把这些信息计算出来的代码。习题6.9和习题6.10要求构造这样一个数据结构。注意，有6个多边形相交于通过细分生成的一个顶点，而只有3个多边形相交于原来四面体的一个顶点。

6.10　全局照明

前面讨论的局部光照模型有局限性。例如，考虑一组球面，它们由一个远距离光源照明，如图6.39(a)所示。离光源更近的球面遮挡了一部分从光源射向其他球面的光线。然而，如果使用局部光照模型，每个球面的明暗计算彼此独立，那么所有球面的明暗图都一样（如图6.39(b)所示）。此外，如果这些球面像镜子一样反射光线，那么光线会在球面之间来回反射。因此，如果球面是磨光的，就会在一些球面上看到反射出的其他球面的像，甚至有可能看到经过多次反射而成的自身的像。前面讨论的光照模型对这些情况无能为力。局部光照模型也不能生成阴影，除非像第5章讲到的那样，对一些特殊情况使用一些技巧。

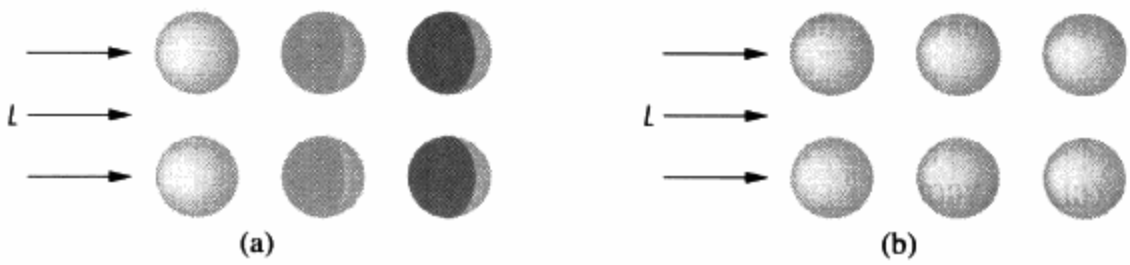


图6.39　一组球面的明暗图：(a) 全局光照模型；(b) 局部光照模型

阴影、反射成像以及对光线的遮挡，所有这些现象都是全局效应，它们需要用全局光照模型来模拟。虽然这样的模型存在，而且可能还非常简洁，但是它们在实际应用中与绘制流水线模型不相容。在绘制流水线中，每个多边形的绘制都是独立的，而且我们希望不管应用程序以怎样的顺序生成多边形，所得到的图像都相同。尽管这样的限制使得我们只能模拟有限的光照效果，但绘制场景的速度可以非常快。

要模拟全局效果可以使用其他的绘制方法。光线跟踪和辐射度方法是这些方法中最重要的两种。在特定的光照条件下，使用这两种方法可以获得非常好的效果。光线跟踪以虚拟照相机模型为基础，但是对于每条与某个多边形相交的投影线，在计算光源对交点处明暗值的直接贡献之前，先要确定是否有一个或者多个光源能够照射到这个交点。在图6.40中有三个多边形和一个光源。图中所示的投影线与三个多边形中的一个相交。局部绘制器利用改进的Phong模型计算交点处的明暗值。而光线跟踪器则会发现，从光源发出的光线由于被另一个多边形遮挡而不能直接照射到这个交点，但可以经过第三个多边形的反射到达这个交点。第13章将介绍如何获得这些信息以及如何进行所需的计算。

辐射度方法基于对能量的考虑。从物理的观点来看，一个场景中所有光线的能量是守恒的。因此，对于场景中所有的光线，包括从光源辐射出去的光线和被场景中各种表面反射的光线，存在一

个能量平衡。计算辐射度需要求解一个涉及所有表面的大规模方程组。第 13 章会讲到，光线跟踪最适于由高度反光的表面构成的场景，而辐射度方法最适于由理想漫反射表面构成的场景。

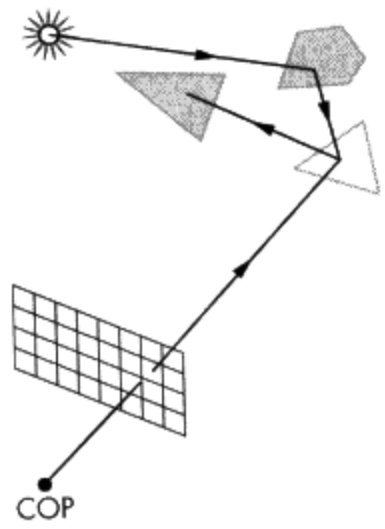


图 6.40　光源照射不到的多边形

虽然利用绘制流水线不能准确地模拟许多全局光照现象，但是这并不意味着利用OpenGL或者其他基于绘制流水线结构的API不能生成逼真的图像。我们可以借助对OpenGL和全局光照效果的了解来近似全局光照的效果。例如，第5章利用透视投影生成了简单的阴影。在过去几年里，计算机图形学中许多激动人心的进展就是利用可编程绘制流水线来实现一些全局光照效果。后面几章里会介绍许多这样的技术，包括纹理映射、多遍绘制和透明效果绘制。

直到不久以前，离线绘制器除了可以实现全局光照效果以外的另一个优点是能够以至少32位的精度进行光照计算。当进行实时明暗绘制时，精度往往只有8位，所以绘制的图像会失去一些细节。除了最低端的商用图形卡之外，现在所有的图形卡都支持高动态范围绘制（high dynamic range rendering）。然而，在电影业中使用的离线绘制器可以花费数小时来绘制动画中的单独一帧，所以我们不能期望实时绘制的图像也能达到与之相似的品质。

小结和注释

本章介绍了一种光照模型，它可以很好地与绘制流水线相结合。利用这种光照模型，我们可以生成各种各样的光照效果，还可以应用不同种类的光源。虽然这样的局部光照模型不能像光线跟踪那样生成全局光照效果，但是一个典型的图形工作站可以按照改进的Phong光照模型对由多边形组成的场景进行平滑着色，所需的时间与不进行明暗绘制差不多。从应用程序的角度来看，增加明暗绘制只需要设置描述光源和材质属性的参数。虽然我们介绍的局部光照模型有局限性，但是它很简单，效果也相当好，并且是包括 OpenGL 在内的大多数 API 所支持的反射模型的基础。

第9章将要介绍的可编程着色器极大地改变了光照和明暗绘制的面貌。我们不但可以应用新的方法对每个顶点进行明暗绘制，还可以利用片元着色器对每个片元进行光照计算，这样就无须在每个多边形内部插值颜色。像Phong着色这样以前在标准绘制流水线上无法实现的技术，现在已经可以由应用程序编程人员编程实现，并且所需的执行时间和应用改进的Phong模型进行绘制所需的时间差不多。此外，利用可编程着色器还可以生成许多新的明暗效果。

我们在近似球面时所使用的递归细分技术是一种强有力的方法，在第12章讨论曲线和曲面的绘制时还会用到这种方法的其他形式。有些建模技术利用了许多自然对象所具有的自相似性，这些建模技术也用到了递归细分方法。

这一章结束了对基于多边形的图形绘制系统的讨论。读者现在已经知道如何生成具有光照和明暗效果的图像。第8章将介绍生成更复杂图像的技术，比如纹理映射和融合，这些技术需要用到图形系统的像素级处理功能。

对于读者来说现在是一个练习编写应用程序的好时机。读者可以尝试修改各种各样的光照和明暗参数会对生成的图像有什么影响，还可以试着生成能够改变位置的光源（光源的移动可以独立于场景中的对象，也可以和对象一起移动）。读者可能会遇到这样的问题：生成的明暗图有一些小的缺陷，例如多边形之间存在缝隙，光线可以从中透过。许多这样的问题是由于绘制时的数值计算误差引起的。有许多技巧可以减少这些误差带来的影响，可能读者自己会发现一些技巧，其他技巧可以在本章的建议阅读资料中找到。

第7章将讨论光栅化。虽然我们已经看到了绘制流水线中不同模块实现各自功能的一些方法，但是还没有看到细节。随着对这些细节的深入讨论，读者会看到各个部分如何相互配合以构成一个整体，使得绘制流水线中的每个步骤只需要在前一个步骤执行的基础上进行少量的处理。

建议阅读资料

计算机图形学中的光照和明暗处理遵循两个平行的方法：物理的和计算的。从物理的角度来看，Kajiya的绘制方程[Kaj86]描述了环境中总的能量平衡，不过要列出这个方程需要知道每个表面的反射函数。一些反射模型用小的平面片来建模表面，Torrance-Sparrow模型[Tor67]和Cook-Torrance模型[Coo82]就是这样推导出来的。Hall[Hal89]和Foley[Fol90]的书对这些模型进行了讨论。

把环境光反射分量、漫反射分量和镜面反射分量结合在一起的计算模型属于Phong[Pho75]。半角向量首先是由Blinn[Bli77]提出的。Whitted[Whi80]提出了透射光线的基本模型，后来Heckbert和Hanrahan[Hec84]又对其进行了修改。Gouraud[Gou71]引入了插值着色。

如果想更有效地利用OpenGL提供的绘制功能，可以参考*OpenGL Programming Guide*[Ope05]，其中给出了许多有价值的建议。

习题

6.1　本章介绍了用于绘制多边形的一些简单的光照和反射模型，大多数图形系统和API也使用这些模型，请指出每个这样的模型在什么情形下会失效。对每种失效的情形，请给出一个场景的例子，使得模型的失效可以从该场景的图像中清楚地体现出来。

6.2　当利用OpenGL绘制一个明暗值应该相对均匀的多边形时，如果这个多边形比较大，那么绘制的结果往往是该多边形的一个区域比较亮而其他区域较暗。请解释出现这种现象的原因并给出解决办法。

6.3　在讨论Phong反射模型时，为什么没有考虑光源被其他表面遮挡的情形？

6.4　如何在进行绘制计算时把观察者到表面之间的距离考虑进去？

6.5　我们在描述表面的材质属性时使用了RGB加色模型，其实还可以使用减色模型，请问这样做有什么好处？

6.6　在单位球面上找出彼此之间距离相等的4个点，这些点确定了一个四面体。提示：可以把其中任意一个点放在(0, 1, 0)，让其他三个点位于平面 $y = -d$ 内，其中 d 是正数。

6.7　证明如果 **v** 和 **l**、**n**、**r** 位于同一平面内，那么半角满足：

$$2\psi = \phi$$

如果 **v** 和其他向量不共面，那么这些角度之间有什么关系？

6.8　考虑下面几种情形的所有组合：观察者在近处或远处，光源在近处或远处，表面是平直的或者弯曲的，漫反射或者镜面反射。对哪种组合明暗值的计算能够简化？对哪种组合使用半角向量会有帮助？请给出理由。

6.9　构造一个表示细分四面体的数据结构。遍历这个数据结构，对基于细分四面体获得的近似球面进行 Gouraud 明暗绘制。

6.10　重做习题 6.9，但从二十面体而不是从四面体开始细分。

6.11　构造一个表示四边形网格的数据结构。编写一个程序，对该数据结构所表示的网格进行明暗绘制。

6.12　编写一个程序，对四边形和四边形网格进行递归细分。

6.13　考虑相交于平面边界的两种材质。假定光线在这两种材质中的速度分别是 v_1 和 v_2。证明 Snell定律是下述事实的推论：光线从一种材质中的一点到第二种材质中的一点要经历这样的路径，这条路径在所有可能的路径中耗时最短。

6.14　证明为了使观察者接收到的反射光线强度最大，表面法向量的方向应与半角向量 **h** 的方向相同。

6.15　虽然我们还没有讨论帧缓存的操作，但是可以利用一个简单的函数构造一个光线跟踪器，这个函数形如 write_pixel(x, y, color)，它把帧缓存中(x, y)处的颜色值设置成 color（或者是RGB颜色或者是一个亮度）。请用伪码编写一个函数 ray，该函数递归跟踪一条投射光线。假定有一个现成的函数可供调用，该函数可以计算出光线与对象的交点。注意，考虑如何限制初始光线被递归跟踪的深度。

6.16　假定所用的图形系统提供了一个写像素的函数，请编写一个光线跟踪器。假设场景中只包含球面，并且使用球面的数学方程而不是用球面的多边形近似。

6.17　为习题 5.13 中的迷宫程序增加光源和明暗效果。

6.18　以附录 A 中生成球面的程序为基础，编写一个交互式程序，该程序在执行时允许改变一个或者多个光源的位置，还允许改变材质属性。尝试利用这个程序生成表面的图像，其中的表面具有熟悉的材质属性，例如各种金属、塑料和碳。

6.19　当几何数据通过观察流水线的时候，一些向量要经历一系列的旋转、平移、缩放以及投影变换，这些向量确定了 Phong 反射模型中的余弦项。在向量所经历的这些变换中，有哪些保持向量之间的夹角（如果有的话）？这个问题的回答对于明暗绘制的实现有什么意义？

6.20　估计 Phong 着色和 Gouraud 着色相比较所需的额外计算量。考虑习题 6.19 的结果。

6.21　推广 5.10 节中的阴影生成算法，使其可以处理任意方向的平面片。

6.22　修改 5.10 节中的阴影生成算法，使其可以用于光源位于远处的场景。提示：透视投影应变为平行投影。

6.23　把5.10节中的阴影生成算法和利用全局绘制算法生成阴影进行比较，请问哪类阴影可以被一种算法生成但不能被另一种算法生成？

6.24　考虑一个高度反光的球面，该球面的球心位于原点，半径为 1。假定观察者位于 **p**，他在球面上会看到哪些点的反射像？

第7章　从顶点到片元

现在介绍图形绘制流水线的后面几个处理过程：裁剪、光栅化和隐藏面消除。尽管我们还没有介绍OpenGL中可供应用程序编程人员使用的一些主要功能，如离散图元的处理、纹理映射和曲线曲面的绘制，但是有几个理由要在此首先考虑裁剪、光栅化和隐藏面消除等问题。首先，读者可能想知道自己使用的图形系统是如何处理应用程序的，如线段是如何被绘制到屏幕上的；多边形是如何被填充的；位于程序定义的视见体之外的图元又是如何被处理的。其次，我们观点是如果希望有效地使用图形绘制系统，那么有必要深入理解其实现过程：哪些步骤的实现比较容易，以及哪些步骤的实现对硬件和软件的要求较高。最后，对绘制流程的实现过程进行深入讨论有助于更好地使用最新图形硬件支持的新功能。

要学习绘制流程的实现过程必然要涉及相关算法的研究。当研究其他算法时，我们一定会仔细地考虑这样一些问题：算法的理论与实际性能，算法的硬件与软件实现，以及算法对某个具体应用的特性。尽管可以根据屏幕上生成的像素的正确性来测试某个OpenGL实现是否正常工作，但是对于我们使用的这个算法可能有多个参数选项可供选择。下面重点介绍一些基本的算法，这些算法是实现一个标准图形API所必需的，也是流水线绘制结构或光线跟踪等其他绘制方法所必需的。因此，我们对绘制流水线中的每一个主要任务都介绍一些不同的基本实现算法。

本章关心的是一些用于实现OpenGL绘制流水线的基本算法。我们将重点讨论三个问题：裁剪、光栅化和隐藏面消除。裁剪就是把位于视见体之外的对象删掉，从而在最终显示的图像上这些对象是不可见的。光栅化就是把经过裁剪后的对象处理成片元的过程。这些片元的属性会影响最终显示的图像的效果。隐藏面消除就是确定可见对象的片元，所谓可见对象就是位于视见体之内并且在视线方向没有被其他靠近视点的对象遮挡的对象。

7.1　图形绘制流水线的基本实现策略

我们首先从高层的角度来介绍图形绘制流水线的实现过程。在计算机图形学中，图形的生成过程以应用程序为起点，而以图像的生成为终点。我们可以把这个过程视为一个黑盒子（如图7.1所示），它的输入是程序定义的顶点和状态变量，即几何对象、属性和虚拟照相机的参数设置；它的输出是位于帧缓存中的彩色像素阵列。

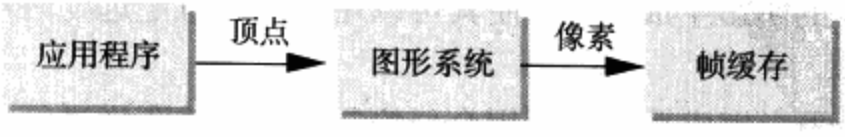


图7.1　图形生成过程的高层视图

在这个黑盒子内部，我们需要完成很多任务，包括几何变换、裁剪、明暗处理、隐藏面消除和图元（可在屏幕上显示的图元）的光栅化。虽然可以使用各种不同的策略将这些任务组织起来，但是不管采用哪种策略，都必须做两件事情：第一，必须使每个几何对象经过图形系统；第二，必须给颜色缓存中要显示的每个像素赋一个颜色值。

假定把这个黑盒子视为能执行所有绘制过程的单个程序，该程序的输入是用于定义几何对象的一组顶点，输出的是位于帧缓存中的像素。因为该程序必须给每个像素赋一个颜色值，并且必须处理每个几何图元（以及每个光源），所以这个程序至少有两个循环，它们分别用来对像素和几何图元进行反复处理。

如果希望编写这样的程序，那么必须首先解决下面的问题：应该使用哪个变量控制外层的循环？对该问题的不同回答决定了整个绘制过程的实现流程。该问题有两个基本的答案，在实现过程中采用的这两种策略通常称为基于图像空间的方法和基于对象空间的方法。

在基于对象空间的方法中，程序的外层循环控制几何对象的遍历。我们认为基于这种方法的程序的循环控制结构为：

```
for(each_object) render(object);
```

基于流水线结构的图形绘制系统适合于采用这种循环控制结构。几何对象的顶点由程序定义后流经一系列的模块，这些模块依次对这些顶点执行几何变换、明暗着色以及可见性判定。一个多边形可能要经过如图7.2所示的几何步骤的处理。我们注意到：一个多边形经过几何处理步骤后，对该多边形的光栅化处理有可能会影响帧缓存中相应像素的颜色。大部分基于对象空间方法的图形绘制系统采用绘制流水线的结构，其处理步骤对应的每一个任务都包含相应的硬件或软件模块，数据（顶点）向前流经整个图形绘制系统。

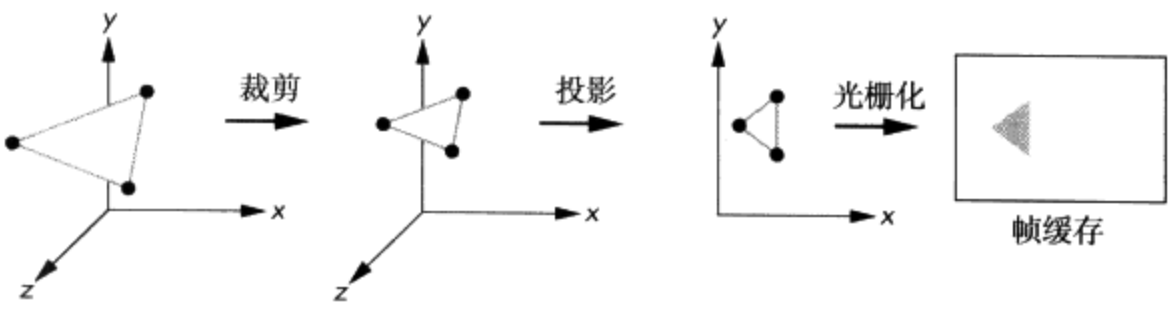


图7.2　基于对象空间的方法

过去，这种基于对象空间方法的主要限制在于需要大容量的内存以及单独处理每个对象所需的大量时间开销。任何经过几何处理步骤处理后的几何图元都可能会对帧缓存中的像素产生影响，因此整个颜色缓存以及其他各种缓存（如用于隐藏面消除的深度缓存）都必须具有显示屏幕大小的空间分辨率，并且在任何时候都可以访问其存储的数据。在高密度的低价内存出现之前，前面提到的需求过于苛刻。目前已有的各种基于流水线绘制结构的几何处理器每秒能够处理数以千万的多边形。实际上，正是因为对每个图元执行相同的操作，所以构建基于对象空间的图形系统所用的硬件处理速度很快，而其成本相对低廉，许多功能可由专用的芯片来实现。

如今，基于对象空间方法的图形绘制系统的主要局限性是它们无法处理大部分的全局计算。因为图形绘制系统按任意的顺序独立地处理每一个几何图元，所以像反射这种复杂的光照效果，由于涉及多个几何对象的相互作用，除了使用近似的方法，我们无法进行精确的处理。一个主要的例外就是隐藏面消除，它使用了*z*-buffer来存储全局信息。

对于基于图像空间方法的程序，它的最外层循环控制像素的遍历，或者控制每一行像素，即扫描线，这些像素构成了帧缓存。基于这种方法的程序的循环控制结构的伪代码为：

```
for(each_pixel) assign_a_color(pixel);
```

对于每一个像素，可以按相反的方向来确定哪些几何对象对该像素的颜色有贡献。这种方法的优点是：在任何时候只需要使用有限的显示内存，而且可以按屏幕刷新的速度和顺序来生成像素。因为对相邻像素（或相邻扫描线）的大部分计算操作，其处理结果不会有太大的差异，所以

可以在算法中充分利用这种连贯性，具体做法是开发增量式计算公式来处理算法的实现过程。该方法的主要缺陷是：如果不事先根据几何数据建立相应的数据结构，则无法确定哪些图元影响哪些像素。这样的数据结构可能非常复杂，而且意味着在绘制过程中可以随时访问所有的几何数据。对于大规模场景数据库所带来的问题是：即使采用很好的数据表示方法，也无法避免出现内存不足等问题。然而，因为基于图像空间的方法需要为每个像素访问所有的几何对象，所以这种方法非常适合于处理全局效果，如阴影和反射等效果。光线跟踪（见第13章）是基于图像空间方法的一个示例。

因为我们主要对交互式图形应用感兴趣，所以倾向于使用基于对象空间的方法，虽然我们也可能讨论一些适合于这两种方法的例子。此外，随着图形硬件成本的降低及速度的日益提高，目前没有得到广泛使用的方法在不久的将来可能是非常实用的方法。

7.2　图形绘制系统的4个主要任务

我们首先回顾绘制流水线的功能模块，重点是那些还没有详细讨论的模块。要绘制三维多边形这样的几何实体，即从用户程序中该几何实体的定义到最终在输出设备上的显示输出，任何一个图形绘制系统都必须执行4个主要任务：

- 建模
- 几何处理
- 光栅化
- 片元处理

图7.3显示了这些任务在一个流水线结构的图形绘制系统中的组织方式。无论采用前面介绍的哪种绘制方法，都必须执行这4个任务。

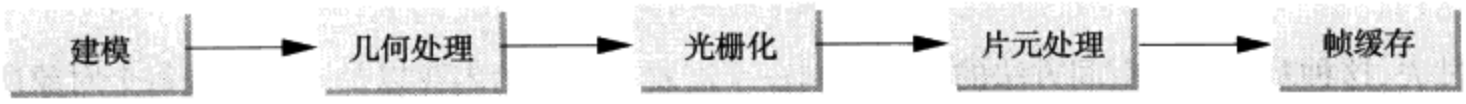


图7.3　图形绘制系统的4个主要任务

7.2.1　建模

建模的结果通常表现为一组表示几何对象的顶点数据，图形绘制系统的其余部分要用到这些几何数据。前面的章节中已经讲到过一些要求用户建模的例子，如第6章对球体的近似建模。在第10章和第11章，我们还将研究其他的建模技术。

我们可以把建模器视为一个用来生成几何对象的黑盒子，它通常是一个交互式的应用程序。建模器除了生成几何对象外，还能完成一些其他的任务，例如裁剪任务，即删除不会显示在屏幕上的部分对象，因为这部分对象位于视见体之外。用户也可以在自己的程序中生成几何对象，并希望图形绘制系统的其余部分能以这些几何对象生成的速度实时地处理这些对象；或者利用建模器来减少送入绘制流水线中的几何对象的数量，从而减轻图形绘制系统其余部分的负荷。后一种方法通常意味着建模器本身执行了一些本应在图形绘制系统其余部分执行的任务，尽管这两者使用的算法可能不相同。对于裁剪的情形，由于建模器从应用程序中获得了更多的特定信息（启发信息），所以它把图元送入标准的观察处理过程之前，通常要选择使用一个好的启发信息来删掉许多图元（如果不是大部分图元的话）。

7.2.2　几何处理

几何处理作用于顶点数据，其目标是确定哪些几何对象可以在屏幕上显示，并把灰度值或颜色值赋给这些对象的顶点。几何处理包括4个处理过程：投影、图元装配、裁剪和明暗处理。

通常情况下，几何处理的第一步是把几何对象的表示从对象坐标系（即建模坐标系）变换到照相机坐标系或观察坐标系。正如第5章讲到的，变换到观察坐标系只是观察处理过程的第一步，几何处理的第二步是利用投影变换矩阵把对象的顶点变换到规范化的视见体内，规范化的视见体是一个中心位于坐标原点的立方体，位于它里面的所有对象都是潜在的可见对象。上面的第一步使用了一个组合变换矩阵，我们称之为模－视变换矩阵。经过这两步处理之后，顶点位于裁剪坐标系中。上面第二步的规范化处理不仅把透视投影和正投影都变换成一个简单的正投影，而且还简化了其后面要进行的裁剪处理，我们将在7.7节介绍这个问题。

几何对象要经过一系列变换矩阵的变换处理，这些变换矩阵要么改变对象的几何形状和位置（建模变换），要么改变对象的坐标表示（观察变换）。最终，只有那些位于某个特定的视见体（view volume）内的图元经光栅化处理后才可以显示在屏幕上。然而，我们不能简单地对所有的对象都进行光栅化处理，而是希望图形硬件只处理那些完全或部分位于视见体内的图元。图形绘制系统必须在光栅化之前完成这个任务。这样做的一个理由是：对位于视见体之外的对象进行光栅化处理会影响系统的绘制效率，因为这些对象根本就不可见。另一个理由是：当顶点到达光栅器的时候，光栅器是不能对它们进行单独处理的，顶点必须首先装配成图元。部分位于视见体内的图元经裁剪处理后会生成新的图元，这些新图元包含新的顶点，我们必须对这些新的顶点执行光照计算。在进行裁剪处理之前，必须把顶点组合成对象，这个过程称为图元组装（primitive assembly）。

我们注意到：即使一个对象位于视见体的内部，如果它被其他离视点更近的对象遮挡了，那么该对象也是不可见的。隐藏面消除（hidden-surface removal）或可见面确定（visible-surface determination）算法就是用于确定对象之间的三维空间关系，它通常作为片元处理的一部分而执行。

必须把颜色赋给每个顶点。顶点的颜色要么由图形绘制系统的当前颜色决定，要么当开启光照时，由改进的Phong光照模型通过计算得到。每个顶点的光照计算都是几何处理的一部分。

裁剪处理完成后，保留的顶点（未裁剪的顶点）仍然使用四维的齐次坐标表示。图形绘制系统通过透视除法把顶点的这种表示形式转换成规范化设备坐标系中的三维坐标表示形式。

以上所有这些处理统称为前端处理（front-end processing）。所有这些处理都涉及三维计算，并且都需要用到浮点运算，它们对硬件和软件都有相同的要求。所有的前端处理都是基于顶点（vertex-by-vertex）的处理模式。我们将在7.3节讨论裁剪，它是仅有的一个目前还没有讨论的几何处理过程。

7.2.3　光栅化

即使在几何处理完成之后，我们仍然需要保留用于隐藏面消除的顶点深度信息。然而，只需要使用顶点的x和y坐标值来确定帧缓存中哪些像素的颜色值会受到某个图元的影响。例如，执行透视除法之后，最初在三维空间中由两个顶点定义的线段变成了在规范化的设备坐标系中由一对三维顶点定义的线段。为了生成一组片元，用来说明这些顶点所对应的像素在帧缓存中的位置，我们只需要顶点的x和y坐标分量，或者是这些顶点的正投影结果。我们通过一个称为光栅化（rasterization）或扫描转换（scan conversion）的处理过程来确定这些片元。对于线段，我们使用光栅化来确定哪些片元可用来近似表示投影后两个顶点之间的线段；对于多边形，则使用光栅化来确定哪些片元位于投影后的二维多边形内部。

片元的颜色要么由图形系统的当前颜色属性决定，要么使用第6章介绍的方法，通过顶点颜色的插值计算得到。对于比线段和多边形更复杂的对象，通常使用多条线段和多个多边形来近似表示它们，因此大部分图形绘制系统没有为这类复杂的对象开发专门的光栅化算法。第12章将讲到的一些特殊的曲线和曲面则属于特例。

光栅处理器处理的数据是规范化设备坐标系中的顶点数据，而输出的片元位于显示设备，即窗口坐标系（window coordinates）中。正如第2章和第5章讲到的，裁剪体的投影必须出现在指定的视口内。在OpenGL中，由于最后这个变换（即从规范化设备坐标系到窗口坐标系的变换）是在投影变换之后进行的，所以最后这个变换是在二维空间进行的。由于前面的变换已经规范化了视见体，所以视见体是一个边长为2的正方形，且它的4条边与视口的4条边对齐（如图7.4所示），因此该变换（视口变换）可用下面简单的公式表示：

$$x_v = x_{v\min} + \frac{x + 1.0}{2.0}(x_{v\max} - x_{v\min})$$

$$y_v = y_{v\min} + \frac{y + 1.0}{2.0}(y_{v\max} - y_{v\min})$$

$$z_v = z_{v\min} + \frac{z + 1.0}{2.0}(z_{v\max} - z_{v\min})$$

我们回顾一下前面介绍的透视变换，顶点的z值因透视变换的规范化处理而被非线性缩放，然而对象之间最初的深度顺序仍保持不变，因此可用于隐藏面消除。我们将使用屏幕坐标（screen coordinates）这个术语来表示一个二维坐标系，该坐标系与窗口坐标系相同，但缺少深度坐标。

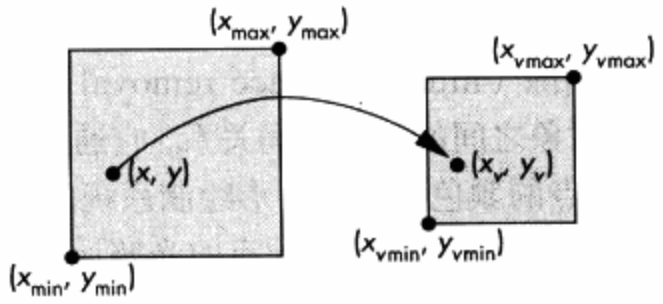


图7.4　视口变换

7.2.4　片元处理

在最简单的情形下，光栅处理器负责把颜色值赋给片元，并且这些颜色就是帧缓存中与这些片元对应的像素的颜色。然而，对于片元颜色的赋值还可能有许多其他方法。

OpenGL等图形绘制结构支持独立的像素流水线结构（见第8章），对于这样的绘制结构，像素流水线的输出与几何绘制流水线的输出在光栅化处理阶段之后的片元处理阶段会合[①]。下面我们分析当处理一个具有明暗效果和纹理贴图的多边形时发生的情形。顶点的光照计算是几何处理的一部分。只有光栅化之后的片元处理阶段才需要用到纹理值，此时图形绘制系统已经生成了多边形内部像素对应的片元。在光栅化之后的片元处理阶段，需要对顶点的颜色值和以及顶点的纹理坐标进行插值计算，而纹理参数决定了纹理颜色与片元颜色的组合方式，从而确定了颜色缓存中与片元相对应的像素的最终颜色。

① 此处原文为“像素流水线的输出与几何绘制流水线的输出在光栅化处理阶段会合”，根据第8章的图8.23，可能有误。——译者注

正如我们已经注意到的，位于视见体内部的对象如果被其他离视点更近的不透明对象遮挡，那么这样的对象是不可见的。这时所需的隐藏面消除处理是一种典型的基于片元（fragment-by-fragment）的处理模式。

到目前为止，我们一直都假定所有对象都是不透明的，因此位于其他对象后面的对象是不可见的。其实我们也可以假定那些对象是半透明的，并且允许部分光线穿过去。在这种情况下，片元的颜色必须与颜色缓存中已有像素的颜色进行融合。第 8 章将分析这种情况。

在大多数图形显示系统中，把帧缓存中的图像数据取出来并在输出设备上进行显示的过程是自动完成的，这并不是应用程序开发人员所关心的问题。然而，这个过程存在许多与显示质量有关的问题，例如用光栅显示器显示的图像所具有的锯齿走样。第 8 章将介绍用来减少锯齿或走样（aliasing）的算法，并讨论采样图像的颜色重构问题。

7.3　裁剪

现在回过头来讨论几何处理阶段的裁剪处理，它是一个用来确定哪些图元或图元的哪些部分位于裁剪体内部或应用程序定义的视见体内部的过程。在透视除法之前完成裁剪处理，如果裁剪后顶点的 w 分量不等于 1，那么有必要利用透视除法使该分量等于 1。所有图元潜在的可见部分（还没有进行隐藏面消除处理）都位于一个立方体的内部，立方体可以表示为：

$$-w \leqslant x \leqslant w$$
$$-w \leqslant y \leqslant w$$
$$-w \leqslant z \leqslant w$$

我们把这个坐标系称为裁剪坐标系，虽然用于生成正确图像的信息保留在这个坐标系中，但是该坐标系既不依赖应用程序定义的原始坐标单位，也不依赖某个特定的显示设备。我们还需注意：投影过程只完成了一部分，还必须执行透视除法以及最后的正投影。执行透视除法之后，应该得到对象在规范化的设备坐标系中的三维表示形式。由于在裁剪坐标系中进行裁剪处理，这样就可以避免对处于裁剪体之外的图元执行透视除法运算。

我们将重点讨论线段和多边形的裁剪，因为它们是绘制流水线中最常见的图元。尽管 OpenGL 绘制流水线可以裁剪三维对象，但是也有其他的图像绘制系统，它们把对象先投影到 x，y 平面后再进行裁剪处理。幸运的是，大部分高效的裁剪算法在二维和三维的情况下几乎是相同的，我们将重点讨论这些算法。

7.4　线段的裁剪

裁剪器（clipper）用来确定哪些图元或者图元的哪些部分可能会显示在屏幕上，并且把没有被裁剪掉的图元送入光栅处理器中。位于定义好的视见体内部的图元可以通过裁剪器，或者说被裁剪器接受，而不会在屏幕上显示的图元则被裁剪器删除（丢弃或剔除）。对于只有一部分位于视见体内部的图元，必须通过裁剪处理将位于视见体之外的部分裁剪掉。

裁剪处理可以发生在观察流水线的一个或多个地方。建模器可能进行裁剪处理，从而对图形硬件必须处理的图元进行限制。当把图元从三维对象投影为二维对象后也可以对它进行裁剪处理。在 OpenGL 中，在光栅化之前使用一个三维视见体对图元进行裁剪。我们将讨论一系列的裁剪器（裁剪算法）。无论是出于教学还是历史的原因，我们首先介绍两个二维线段裁剪算法。这两个算法都可以直接扩展到三维裁剪和多边形裁剪中。

7.4.1　Cohen-Sutherland裁剪算法

线段的二维裁剪问题如图7.5所示。我们现在暂时对二维裁剪问题进行如下假定：二维裁剪是在三维线段投影到投影平面之后才进行的，并且裁剪窗口是投影平面的一部分，该投影片面投影到了屏幕的视口中，所有的值都用实数表示。可以从图中看到，*AB*整条线段都显示在屏幕上，而*CD*整条线段都不显示在屏幕上。线段*EF*和*GH*在显示之前必须裁剪掉位于裁剪窗口之外的部分。尽管一条线段完全由它的两个端点确定，但是从线段*GH*可以看到，即使线段的两个端点都位于裁剪窗口之外，它的一部分线段仍有可能显示在屏幕上。

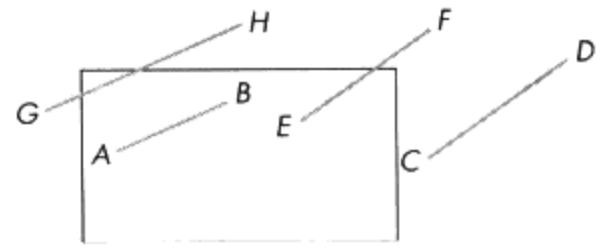


图7.5　二维裁剪

我们可以计算线段所在的直线与裁剪窗口各条边的交点，从而确定必要的裁剪信息。然而，如果可能，我们希望避免这种求交计算，因为每个求交过程都需要进行浮点除法运算。Cohen-Sutherland算法是首个使用浮点减法和位操作相结合的方法代替大量的高开销浮点乘法和除法的裁剪算法。

该算法首先把裁剪窗口的4条边延长到无限远，因此把整个二维空间分割为9个区域，如图7.6所示。我们按下面的方法给每个区域赋一个唯一的4位二进制数$b_0b_1b_2b_3$，或称之为线段的端点编码（即线段端点所在区域的编码）。假定(x, y)是该区域中的一个点，那么

$$b_0 = \begin{cases} 1, & y > y_{\max} \\ 0, & \text{其他} \end{cases}$$

同理，如果$y < y_{\min}$，那么$b_1 = 1$，b_2和b_3的值由线段端点的x坐标值分别与裁剪窗口左边界和右边界的位置关系决定。区域的最终编码结果如图7.6所示。对于线段的每个端点，首先需要计算它的编码，确定每条线段两个端点的编码需要8次浮点减法运算。

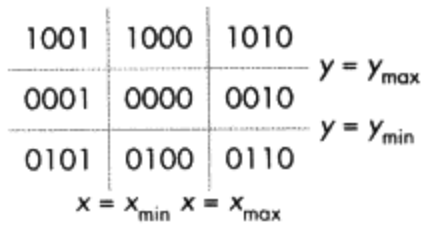


图7.6　空间分割与区域编码

考虑一条线段，它的两个端点编码分别为$o_1 = outcode(x_1, y_1)$和$o_2 = outcode(x_2, y_2)$。现在可以根据这两个端点的编码情况进行讨论，共有4种情况：

1. $(o_1 = o_2 = 0)$线段的两个端点都位于裁剪窗口内部，如图7.7所示的线段*AB*说明了这种情况。由于整条线段都位于裁剪窗口之内，所以需要把该线段送入光栅处理器进行光栅化处理。
2. （$o_1 \neq 0, o_2 = 0$或相反）线段的一个端点位于裁剪窗口之内，另一个端点位于裁剪窗口之外（如图7.7所示的线段*CD*），需要对这样的线段进行裁剪处理。非零的端点编码说明了线段与裁剪窗口的哪条边或哪两条边相交。对于这种情况，必须计算一个或两个交点。需要注意的是：当计算完一个交点后要计算该交点的编码，从而确定是否需要进行另一个求交运算。

3. (o_1 & $o_2 \neq 0$)通过对线段的两个端点编码进行按位与运算，我们可以确定这两个端点是否位于裁剪窗口某条裁剪边的同一侧。如果满足这个条件，那么可以丢弃该线段（如图7.7所示的线段 *EF*）。
4. (o_1 & $o_2 = 0$)线段的两个端点都位于裁剪窗口之外，但是它们分别位于裁剪窗口的两条不同裁剪边的外侧。从如图7.7所示的线段 *GH* 和 *IJ* 可以看出，仅仅从端点的编码无法区分是应该丢弃该线段还是把该线段的一部分裁剪掉。我们最多只能计算该线段与裁剪窗口某边的交点，并根据新的交点编码重新进行上述判断。

上面所有的端点编码判断只需要用到布尔运算。该算法只是在必要的时候才执行求交运算，如第 2 种情况或第 4 种情况，对于这种情况，端点的编码没有提供足够的信息。

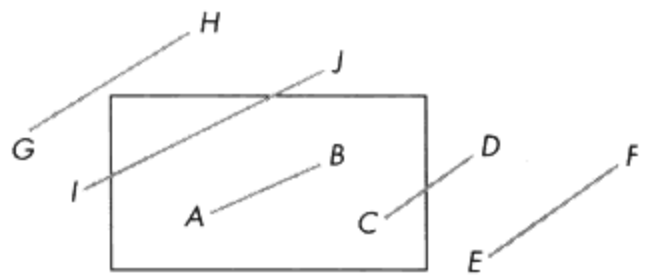


图 7.7　Cohen-Sutherland 算法中线段端点的各种编码情况

当要处理的线段非常多而实际显示的线段又很少时，Cohen-Sutherland 算法非常有效。在这种情况下，大部分线段完全位于裁剪矩形窗口一条边或两条边所在直线的外侧，因此可以根据它们的端点编码裁剪掉这些线段。该算法的另一个优点是可以扩展为三维图形的裁剪。该算法的主要缺点就是对某些线段必须相对于裁剪窗口 4 条边所在的直线依次进行裁剪处理。考虑图 7.7 所示的线段 *GH*，必须用裁剪窗口的左边和顶边所在的直线对它进行裁剪。一般来说，最简单的做法是利用端点的初始编码来确定裁剪窗口的第一条裁剪边，然后利用它对该线段进行裁剪。初始的线段经过第一次裁剪后，对新生成的端点进行编码，然后重复执行该过程。

我们还没有讨论如何计算所需的交点。求交运算取决于线段的表示形式，但是无论采用什么表示形式，只需要执行一次除法运算。如果使用标准的显式形式来表示直线，即

$$y = mx + h$$

其中，m 表示线段的斜率，h 是线段在 y 轴的截距，那么可以根据线段的两个端点来计算 m 和 h。然而，我们无法用这种形式表示垂直的线段，这是该表示方法的最大缺陷。如果只是对 Cohen-Sutherland 算法感兴趣，那么编写程序实现上述所有情况将非常简单，因为裁剪矩形的 4 条边与坐标轴平行。然而，我们感兴趣的不仅仅是裁剪，因为直线和线段的其他表示形式也非常重要。尤其是在计算机图形学中几乎总要使用直线的参数表示形式。第 4 章已经讲过直线的参数表示形式，其他类型的曲线的参数表示形式将在第 12 章介绍。

7.4.2　Liang-Barsky 裁剪算法

如果使用直线的参数表示形式，那么可以使用一种完全不同但更有效的方法对线段进行裁剪。假定有一条线段，用来定义该线段的两个端点分别为 $\mathbf{p}_1 = [x_1, y_1]^T$ 和 $\mathbf{p}_2 = [x_2, y_2]^T$。我们可以使用这两个端点来定义一条直线的参数方程，该参数方程要么用矩阵的形式表示：

$$\mathbf{p}(\alpha) = (1 - \alpha)\mathbf{p}_1 + \alpha\mathbf{p}_2$$

要么表示为两个标量方程：

$$x(\alpha) = (1-\alpha)x_1 + \alpha x_2$$
$$y(\alpha) = (1-\alpha)y_1 + \alpha y_2$$

注意，直线的这种表示方法具有很好的鲁棒性，对于水平直线或垂直直线的表示都不用修改方程。当参数 α 从0变化到1时，线段从 $\mathbf{p}_1$ 移动到 $\mathbf{p}_2$。如果 α 的值为负数，那么生成的点位于从 $\mathbf{p}_2$ 到 $\mathbf{p}_1$ 方向的延长线上。同理，如果 α 的值大于1，那么生成的点位于从 $\mathbf{p}_1$ 到 $\mathbf{p}_2$ 方向的延长线上。

下面分析如图7.8(a)所示的线段及其两端延长后对应的直线。只要该直线与裁剪窗口的边不平行（如果平行，那么这种情况更容易处理），那么它与裁剪窗口4条边的延长线有4个交点。这4个交点对应于4个参数值：α_1，α_2，α_3，α_4。其中的一个参数值对应于直线进入裁剪窗口时的交点，另一个参数值对应于直线离开裁剪窗口时的交点。现在暂时不用考虑如何计算这些交点，而是对这些参数值进行排序，从而确定哪些参数值对应的交点需要用于裁剪处理。对于这个例子，假定：

$$1 > \alpha_4 > \alpha_3 > \alpha_2 > \alpha_1 > 0$$

因此这4个交点都位于原线段的内部，其中在中间的两个参数值（α_2 和 α_3）决定了裁剪后需要保留的线段。这种情况与图7.8(b)所示的情况不一样，在这种情况下，线段的两个端点之间也有4个交点，但需要注意的是其对应的参数值的排序为：

$$1 > \alpha_4 > \alpha_2 > \alpha_3 > \alpha_1 > 0$$

直线与裁剪窗口的左边或右边相交之前，该直线已经与裁剪窗口的顶边和底边相交，因此必须丢弃整条线段。交点的其他排序情况可以采用类似的方法进行分析。

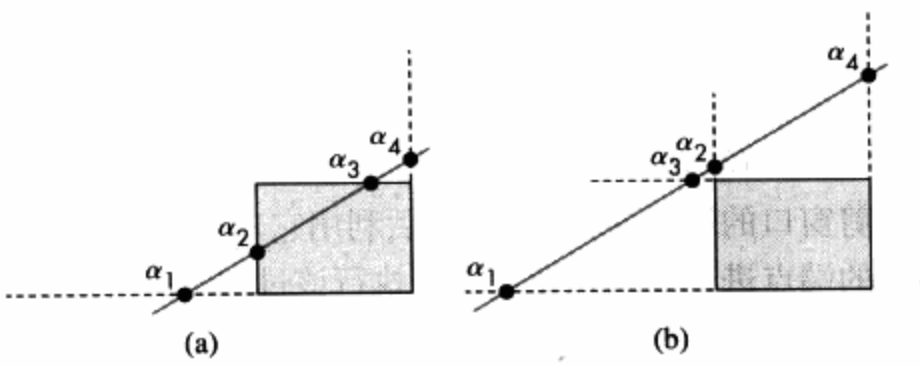


图7.8　参数直线与裁剪窗口的两种位置关系

为了有效地实现这个算法，必须在必要的时候才进行求交运算，因为许多直线在所有的4个交点计算出来之前就可以被丢弃。我们还希望尽可能避免浮点除法运算。如果使用直线的参数表示形式来计算它与裁剪窗口顶边的交点，那么交点的参数值为：

$$\alpha = \frac{y_{\max} - y_1}{y_2 - y_1}$$

对于裁剪窗口的其他3条边，也有类似的等式。考虑到每次求交点都要用到除法运算，我们并没有按这种方法计算交点，而是把上面的等式变换为：

$$\alpha(y_2 - y_1) = \alpha\Delta y = y_{\max} - y_1 = \Delta y_{\max}$$

该算法所需的所有测试都可以重新表示为关于 $\Delta y_{\max}$ 和 Δy 的关系，对于裁剪窗口的其他边也有类似的计算公式。因此，对裁剪线段的所有判断都不需要用到浮点除法运算。只有需要进行求交时（因为该线段必须要进行裁剪处理）才要用到浮点除法运算。相对于Cohen-Sutherland算法，该算法之所以效率很高，主要是因为它不用对线段进行多次裁剪处理，从而避免了裁剪算法的多次重复执行。我们并没有讨论其他一些高效的二维线段裁剪算法，这是因为这些算法与Cohen-Sutherland裁剪算法和Liang-Barsky裁剪算法不同，它们不能扩展到三维的情况。

7.5　多边形的裁剪

在绘制流水线的许多地方都会用到多边形的裁剪。我们当然可以使用屏幕的矩形窗口来裁剪多边形。然而，有时我们希望使用的裁剪窗口并非矩形。图形绘制系统的其他实现部分，如阴影生成和隐藏面消除，需要使用某些多边形裁剪其他的多边形。例如，图 7.9 显示了一个多边形的阴影，它是通过使用一个离光源较远的多边形裁剪一个离光源较近的多边形而得到的。许多反走样方法也会遇到使用多边形裁剪其他多边形的情形。

我们可以直接根据线段的裁剪算法，通过对多边形的每条边依次进行裁剪处理，从而得到多边形的裁剪算法。然而读者一定记得，多边形是一个具有内部区域的二维对象，根据多边形的不同形状，裁剪后的多边形可能包括多个多边形。下面分析如图 7.10(a)所示的非凸多边形（凹多边形）。如果用一个矩形来裁剪该凹多边形，那么得到如图7.10(b)所示的裁剪结果。看到这个图的许多读者都会得到这样的结论，即该多边形被裁剪后生成了3个多边形。遗憾的是，实现这样一个经裁剪后能增加多边形数量的裁剪器是一个比较困难的问题。如图7.11所示，可以把上面的裁剪结果视为单个多边形，该多边形的一些边与裁剪窗口的某些边重合，但是如果选择这种方案实现裁剪器，则会给图形绘制系统其他部分的实现带来麻烦。

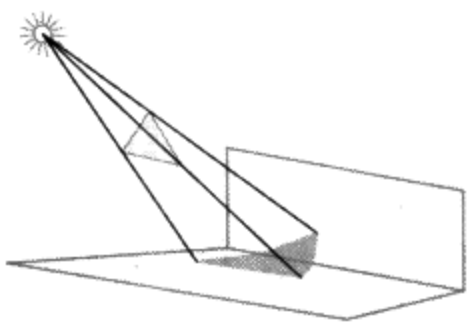


图 7.9　阴影生成中多边形裁剪的应用

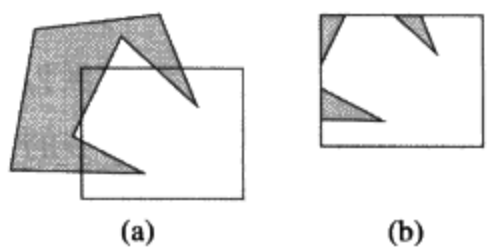


图 7.10　裁剪一个凹多边形：(a) 裁剪之前；(b) 裁剪之后

凸多边形不存在上面这样的问题。用矩形窗口裁剪一个凸多边形最多只能生成一个凸多边形（见习题7.3）。因此，图形绘制系统要么禁止使用凹多边形，要么把一个凹多边形剖分（细化）成一组凸多边形，如图 7.12 所示[①]。

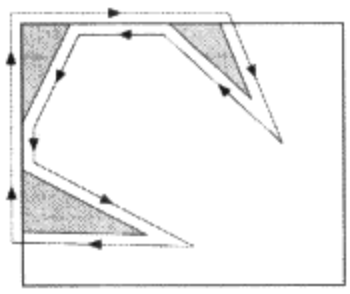


图 7.11　裁剪后生成单个多边形

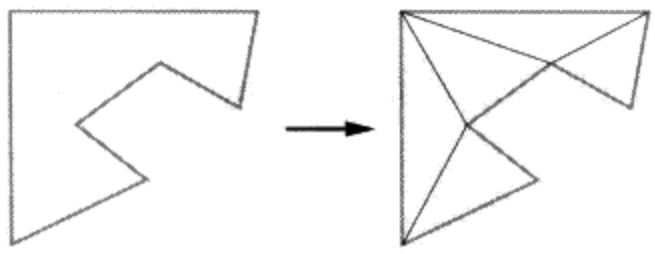


图 7.12　对一个凹多边形的细化处理

对于矩形裁剪区域，既可以使用Cohen-Sutherland算法也可以使用Liang-Barsky算法按基于逐边裁剪的方法对多边形进行裁剪。还有一种由 Sutherland 和 Hodgeman 提出的多边形裁剪算法，它非常适合于前面讨论过的流水线绘制结构。

可以把线段裁剪器视为一个黑盒子（如图7.13所示），它的输入是待测试或待裁剪线段的一对端点坐标，而它的输出或者是裁剪后线段的一对顶点坐标，或者是当输入线段完全位于裁剪窗口之外时，不输出任何东西。

① 在 OpenGL 的 GLU 库中包含细化函数。

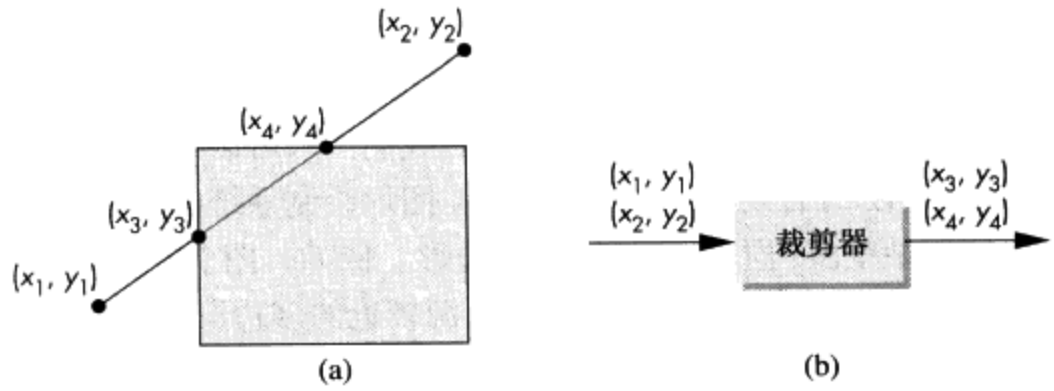


图 7.13　裁剪的两种不同观点：(a) 使用矩形窗口进行裁剪；(b) 把裁剪器视为一个黑盒子

与其把裁剪窗口看成4条线段，还不如把它视为由4条无限长的直线相交而成的矩形对象，这4条无限长的直线确定了裁剪窗口的顶边、底边、右边和左边。然后，可以把这样的裁剪器分解成由简单裁剪器组成的流水线结构，其中的每个裁剪器都使用裁剪窗口某条边延长后的直线对多边形进行裁剪处理。可以把其中的每个裁剪器视为一个黑盒子。

假定考虑只使用裁剪窗口的顶边来进行裁剪处理。我们可以把这个裁剪过程视为一个黑盒子（如图7.14所示），它的输入和输出都是一对顶点坐标，并把y_{max}作为裁剪器已知的输入参数。利用图7.15所示的相似三角形，如果存在交点，那么该交点的坐标值为：

$$x_3 = x_1 + (y_{max} - y_1)\frac{x_2 - x_1}{y_2 - y_1}$$
$$y_3 = y_{max}$$

因此，裁剪器返回下面3对顶点坐标中的某一对：$\{(x_1, y_1), (x_2, y_2)\}$、$\{(x_1, y_1), (x_i, y_{max})\}$或$\{(x_i, y_{max}), (x_2, y_2)\}$。可以分别使用裁剪窗口的底边、右边和左边独立地进行裁剪处理，裁剪时使用与上面相同的公式计算交点，但必要时需要交换公式中x和y的位置，并把当前裁剪边对应的坐标值代入公式。这4个裁剪器可以构成如图7.16所示的流水线结构。如果用硬件实现这种结构，那么对应的裁剪器可以同时作用在4个顶点上。图7.17所示的简单例子显示了对一个多边形进行连续裁剪的情况。

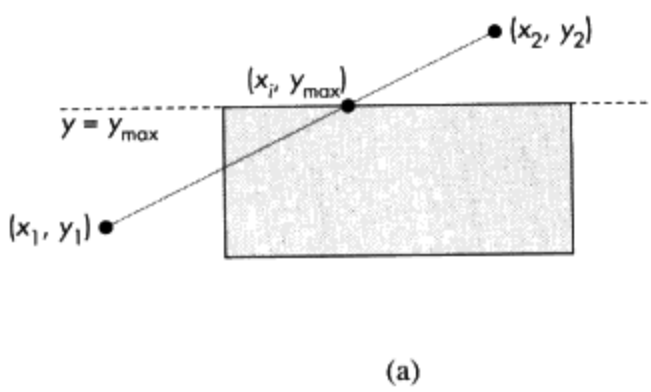


图 7.14　用裁剪窗口的顶边进行裁剪处理：(a) 裁剪的图示表示；(b) 裁剪的黑盒子表示

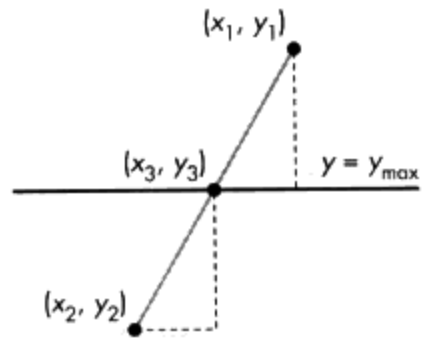


图 7.15　线段与裁剪窗口的顶边相交

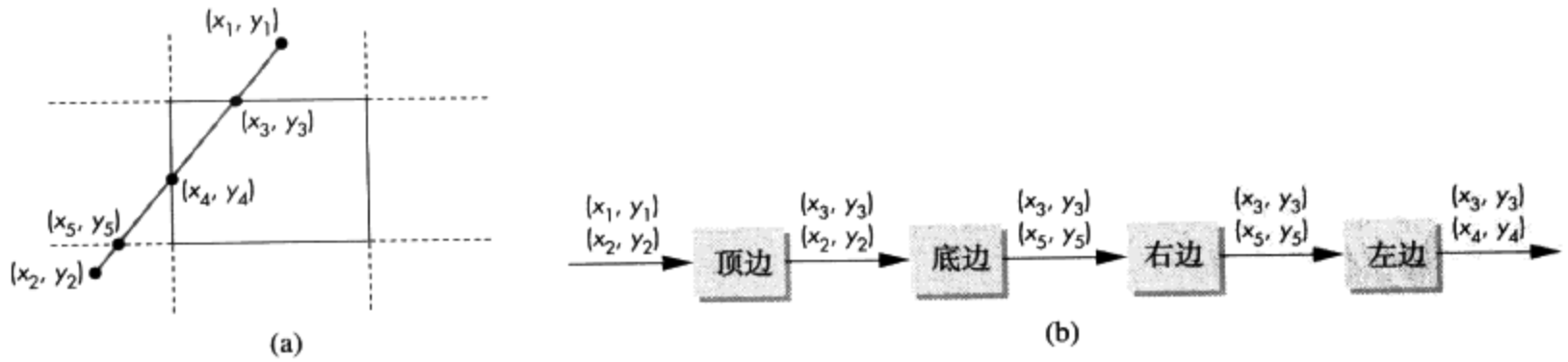


图 7.16　裁剪的流水线结构：(a) 裁剪问题；(b) 由裁剪器构成的流水线结构

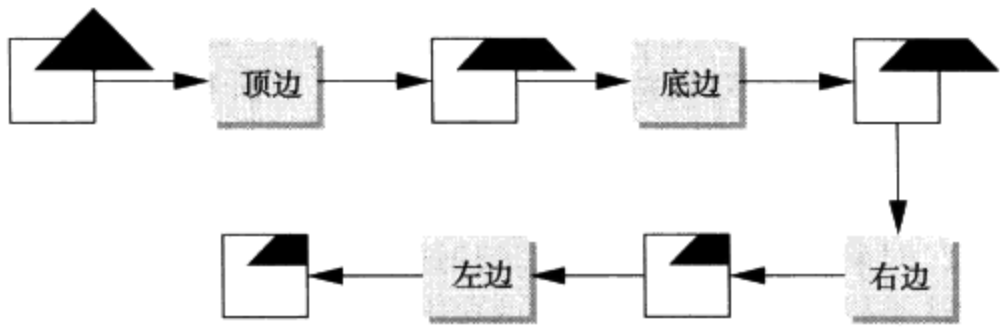


图 7.17　基于流水线结构的裁剪示例

7.6　其他图元的裁剪

从第 1 章到第 6 章，我们把重点放在编写这样的程序，程序中的几何对象是通过线段和平面多边形建立的。要绘制第 12 章将讨论的曲面对象，采用的方法是把它们细分成许多近似的平面小多边形。在图形的流水线绘制结构中，经常会发现一些刚介绍的裁剪器的变异形式。然而在很多情况下，我们希望对象到达图形硬件之前对它们进行裁剪处理，或者希望采用对其他图元做了专门优化的算法。

7.6.1　包围盒与包围体

假定有一个具有很多边的多边形，如图7.18(a)所示，我们可以采用前面介绍的某个裁剪算法对该多边形进行逐边裁剪。然而，我们可以看到该多边形完全位于裁剪窗口之外。通过观察发现可以使用与坐标轴对齐的包围盒或多边形区域（见图7.18(b)）来进行裁剪处理，多边形的包围盒是指与裁剪窗口的边对齐且包含该多边形的最小矩形。要计算多边形的包围盒，只需要比较多边形所有顶点的坐标值并求出最大和最小的 x 值和 y 值。

有了包围盒，通常可以避免对多边形进行复杂的逐边裁剪。下面分析如图 7.19 所示的 3 种情况。对于位于裁剪窗口上方的多边形，因为其顶点坐标最小的 y 值大于裁剪窗口顶边的 y 值，所以不需要进行任何裁剪。对于位于裁剪窗口内部的多边形，通过比较包围盒与裁剪窗口的各条边，可以确定该多边形完全位于裁剪窗口内。只有发现包围盒横跨裁剪窗口时，才需要对该多边形进行复杂的逐边裁剪处理。无论在二维情况下还是在三维情况下，包围盒都是一种非常强大的技术，因此建模系统通常都会自动地为每个对象建立相应的包围盒，并将包围盒随对象一起存储。

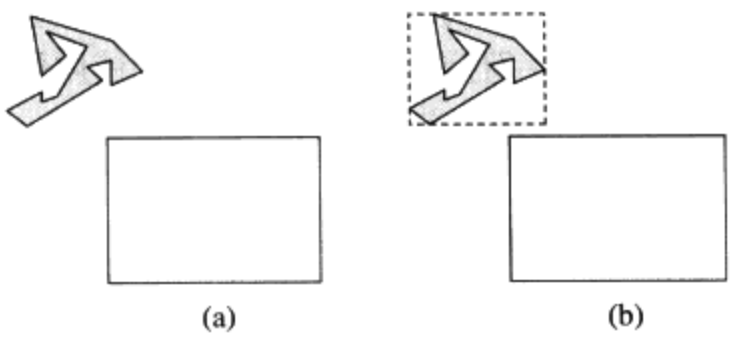


图 7.18　使用包围盒：(a) 多边形和裁剪窗口；(b) 多边形、包围盒和裁剪窗口

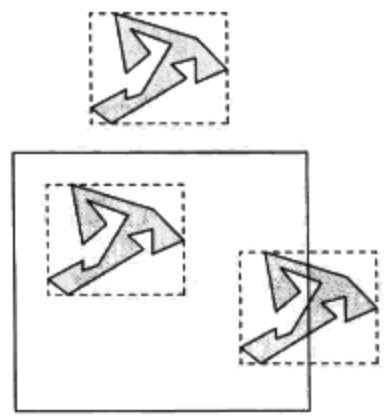


图 7.19　基于包围盒的多边形裁剪

各边与坐标轴对齐的包围盒在二维和三维情形下都适用。在三维情形下，可以在应用程序中采用三维包围盒来执行裁剪操作，从而减轻图形绘制流水线的负荷。其他形式的包围体，如球体，也

可以很好地用于裁剪操作。包围体的其他应用之一是碰撞检测（见第12章）。计算机游戏中的一个基本操作就是判定两个移动的实体对象是否发生碰撞。例如，下面考虑在交互式动画中处于运动状态的两个动画角色，我们需要知道二者何时会发生碰撞，从而及时改变它们的运动路线。碰撞检测问题与裁剪问题具有很多共性，这是因为在执行碰撞检测时，需要确定一个对象的包围体何时与另一个对象的包围体相交。对象可能很复杂并且要求快速地进行碰撞检测计算，这使得该问题处理起来非常困难。通常的做法是把每个对象放在相应的包围体内部，这个包围体可以是与坐标轴对齐的包围盒，也可以是一个球体，这样只要判断它们所对应的包围体是否相交即可，如果包围体相交，那么需要执行复杂的计算。

7.6.2　曲线、曲面和字符的裁剪

用数学方法可以定义各种各样的曲线和曲面，这使得设计一种能处理这些对象的通用算法变得非常困难。从图7.20所示的二维曲线可以看到潜在的裁剪困难。对于一条简单的曲面，如二维曲线，可以计算它与裁剪窗口的交点，虽然付出的计算开销要远高于线段的求交运算。对于更复杂的曲线，如螺旋线，不仅要用数值方法计算交点，甚至还要确定交点的数量，这是非常困难的。要避免出现这样的问题，可以使用线段来近似表示曲线，而使用平面多边形来近似表示曲面。对于曲线和曲面裁剪，包围盒也被证明是一种非常有效的方法，尤其是对二维曲线的裁剪，对于这种情形，我们虽然可以精确地计算曲线与裁剪窗口的交点，但是在执行求交运算之前最好确定这种计算的必要性。

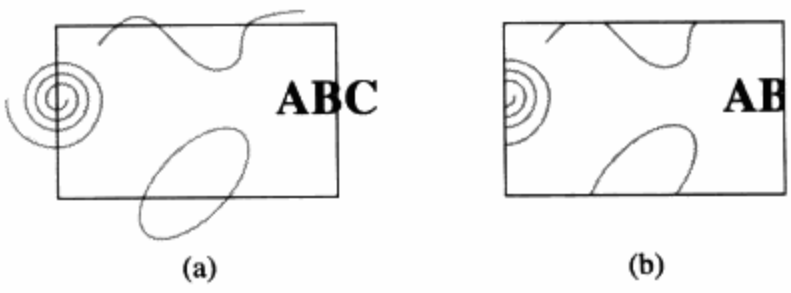


图7.20　曲线裁剪

对字符的裁剪处理因采用的图形API的不同而不同，许多图形API允许应用程序编程人员指定字符的绘制细节。有两种极端的情况：一种是把字符以位图的形式进行存储，并使用图形硬件直接绘制字符，而不对字符执行任何的几何处理操作。字符所需的任何裁剪操作都在帧缓存中进行；另一种是字符采用与其他任何的几何对象相同的定义方式，然后利用标准的观察流水线结构对它们进行处理。由于OpenGL不支持专门的字符图元，所以上面这两种字符处理方法都可以使用。应用程序编程人员可以根据应用需求选择其中的一种方法，即要么把字符定义成位图的形式并使用像素操作进行处理，要么把它定义成笔划字符并使用标准图元的几何操作进行处理。

7.6.3　帧缓存中的裁剪

我们也许会考虑把裁剪操作推迟到几何对象经投影并变换到屏幕坐标系之后进行。我们可以使用一种称为剪切（scissoring）的技术在帧缓存中执行裁剪处理。然而，在顶点到达帧缓存之前裁剪几何实体通常更好一些，因此，在帧缓存中的裁剪操作一般只适合于光栅对象，如像素块。

7.7　三维裁剪

对于三维的情形，我们使用包围体而不是平面上的包围区域来裁剪对象。把二维裁剪扩展到三维裁剪最简单的方法就是使用长方体作为裁剪体（见图7.21）：

$$x_{\min} \leqslant x \leqslant x_{\max}$$
$$y_{\min} \leqslant y \leqslant y_{\max}$$
$$z_{\min} \leqslant z \leqslant z_{\max}$$

或者在如下定义的裁剪空间中进行裁剪：

$$-w \leqslant x \leqslant w$$
$$-w \leqslant y \leqslant w$$
$$-w \leqslant z \leqslant w$$

前面介绍的3种裁剪算法（Cohen-Sutherland直线段裁剪算法、Liang-Barsky直线段裁剪算法和Sutherland-Hodgeman多边形裁剪算法）以及二维包围盒都可以扩展到三维的情形。对于Cohen-Sutherland直线段裁剪算法，可以使用6位二进制端点编码代替4位二进制端点编码，增加的2位表示线段的端点位于裁剪体的前面和后面的情形（如图7.22所示）。二维和三维裁剪使用的判断方法[①]实际上是完全相同的。

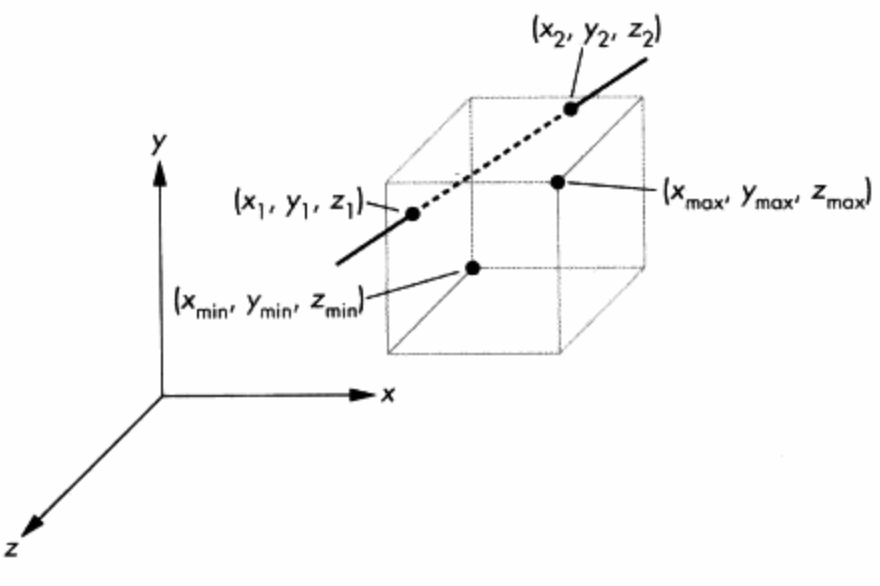


图7.21　使用长方体进行三维裁剪

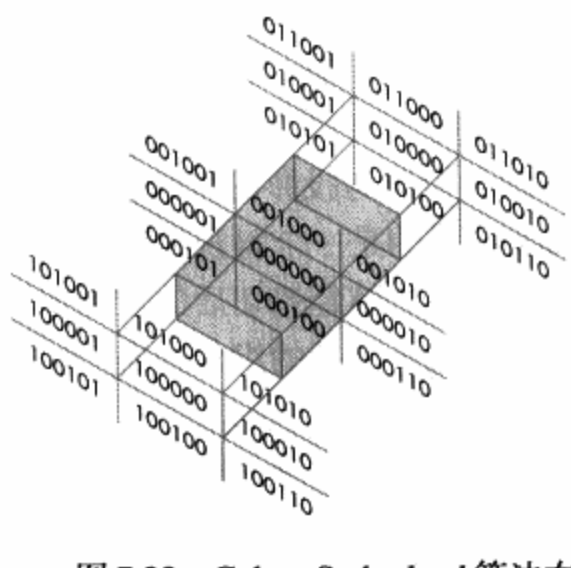


图7.22　Cohen-Sutherland算法在三维空间中的区域划分

对于Liang-Barsky算法，可以在二维的情形下增加下面的等式，从而得到线段的三维参数表示：

$$z(\alpha) = (1-\alpha)z_1 + \alpha z_2$$

我们必须考虑线段与裁剪体表面的6种相交情况，但是我们同样可以采用二维裁剪使用的方法。裁剪器的流水线结构增加了两个黑盒子，它们分别使用裁剪体的前面和后面来进行裁剪处理。

二维裁剪器和三维裁剪器的主要区别就是二维裁剪使用直线来裁剪其他的直线，而在三维裁剪中，要么使用平面来裁剪直线，要么使用平面来裁剪多边形。因此，我们必须修改二维裁剪中的求交计算公式。一个典型的求交计算是三维参数直线与某个平面相交的情形（如图7.23所示）。如果用矩阵的形式表示直线和平面的方程（其中，$\mathbf{n}$是平面的法向量，$\mathbf{p}_0$是平面上的某一点），那么需要通过求解下面的方程组来计算交点对应的α参数值：

$$\mathbf{p}(\alpha) = (1-\alpha)\mathbf{p}_1 + \alpha\mathbf{p}_2$$
$$\mathbf{n} \cdot (\mathbf{p}(\alpha) - \mathbf{p}_0) = 0$$

求解方程组得到的α参数值为：

$$\alpha = \frac{\mathbf{n} \cdot (\mathbf{p}_0 - \mathbf{p}_1)}{\mathbf{n} \cdot (\mathbf{p}_2 - \mathbf{p}_1)}$$

① 即通过编码判断线段与裁剪窗口或裁剪体的位置关系。——译者注

计算一个交点需要用到6次乘法运算和1次除法运算。然而，如果仔细研究一下视见体，就会发现上面的求交运算存在简化的可能。对于正投影图（如图7.24所示），视见体的是一个长方体，就像二维裁剪的处理情形，每次求交运算可以简化为单个除法运算。

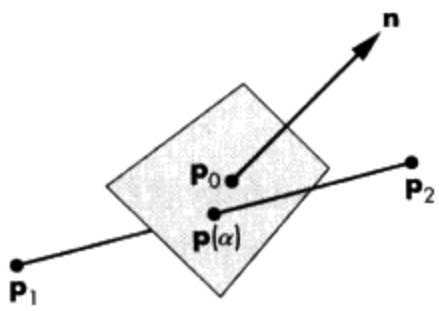


图7.23　直线与平面相交

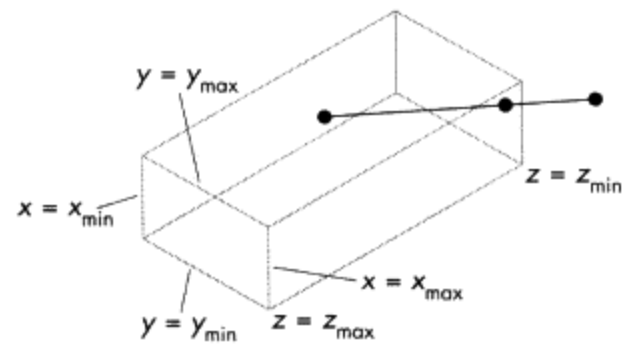


图7.24　使用正投影图进行裁剪

当考虑一个斜投影图（见图7.25）时，我们发现视见体不再是一个长方体。读者可能会想到，为了使用视见体的表面裁剪对象，必须执行点积计算，但使用第5章介绍的规范化处理会带来更多的优点。前面曾证明，一个斜投影等价于一个错切变换和一个投影变换的组合变换。尽管错切变换使对象发生了变形，但正是这个变形的对象使得它经过正投影后得到正确的投影图。如果平行六面体的各个面与投影的方向平行，那么错切变换也使裁剪体发生变形，使它变成了一个长方体。图7.26(a)显示的是一个斜视见体的俯视图，斜视见体内部包含一个立方体。图7.26(b)显示的是这个斜视见体和立方体经错切后的俯视图。就投影而言，直接进行斜投影变换所需的计算量与使用错切变换和一个投影变换的组合变换所需的计算量是相同的。当加入裁剪操作时，我们发现采用第二种方法具有明显的优点，这是因为该方法使用长方体来裁剪对象。这个例子说明了在算法的实现过程中考虑使用增量式处理步骤的重要性。如果单独分析投影或裁剪操作，则无法说明规范化处理的重要性。

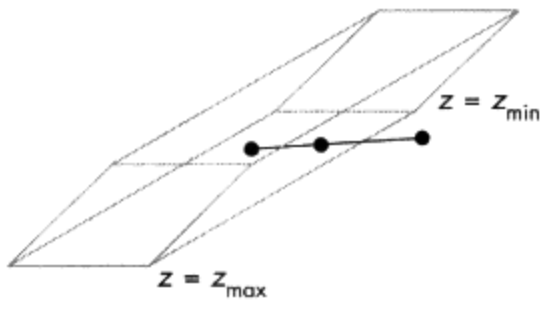


图7.25　使用斜投影图进行裁剪

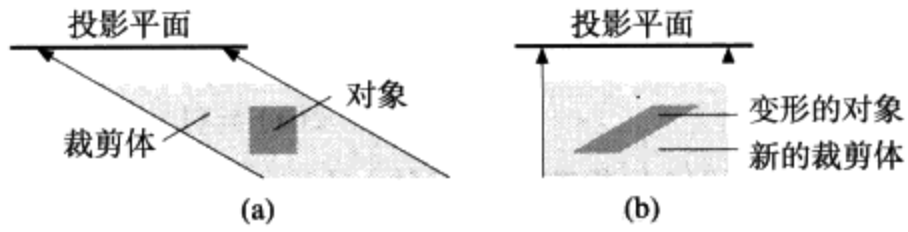


图7.26　错切引起的视见体变形：(a) 错切之前的俯视图；(b) 错切之后的俯视图

上面有关规范化处理的观点同样适用于透视投影。通过执行第5章介绍的透视投影的规范化变换而不是正投影变换，同样可以得到一个矩形裁剪体，从而简化了其后面的求交运算。

OpenGL还支持其他的裁剪平面，这种裁剪平面具有任意的朝向。因此，如果要在用户的程序中使用这种特性，那么只能使用软件来实现这种具有任意朝向裁剪面的裁剪操作，但需要以性能为代价。

7.8　光栅化

从在应用程序中定义几何实体到最终生成片元是一个漫长的绘制流程，下面介绍这个流程的最后一步，即图元的光栅化。本章只关心线段和多边形的光栅化，这两种图元都是由顶点定义的。假定已经对图元进行了裁剪处理，因此剩下的图元都位于视见体内部。

片元是潜在的像素。每个片元都具有一个在屏幕坐标系中的位置属性，它对应于颜色缓存中某个像素的位置。片元还具有其他属性，如颜色，它是光栅处理器根据图元顶点的颜色生成的。因此，如果开启了光照计算，那么光栅处理器以图元顶点的明暗值作为输入，并为它所输出的每个片元通过插值计算其明暗值。片元还包含用于隐藏面消除的深度信息。为了使后面的讨论更清晰易懂，在7.12节之前暂时不考虑隐藏面消除，因此我们直接在屏幕坐标系中处理问题。因为不考虑隐藏面消除、半透明片元和反走样等问题，所以我们可以在讨论光栅化算法时使用像素这个术语，即在这种特定的情形下，通过该算法生成的片元颜色就是像素的颜色。

我们进一步假定颜色缓存是一个 $n \times m$ 的像素阵列，（0，0）坐标对应于其左下角，并且可以使用图形绘制系统内部具有如下形式的单个函数来设置某个位置像素的颜色值：

```
write_pixel(int ix,int iy, int value);
```

其中的value参数要么是基于颜色索引模式的一个索引值，要么是一个指向RGBA颜色数组的指针。一方面，由于颜色缓存在本质上是离散的，所以我们讨论的像素位于整数值ix和iy对应的位置，其他位置的像素则没有意义。另一方面，屏幕坐标的取值范围与ix和iy的取值范围相同，但屏幕坐标的坐标值为实数。例如，我们通过计算得到某个片元对应的屏幕坐标位置为（63.4，157.9），但是我们必须知道对于最佳逼近该坐标的像素，其中心位置位于（63，158）或（63.5，157.5），这取决于像素中心的位置是整数值还是半整数值。

像素具有颜色属性，颜色值存储在颜色缓存中。根据显示器的不同特性，像素显示的形状和大小不一定相同，7.13节将详细说明这个问题。现在暂时假定像素显示为一个正方形，它的中心位置是像素的位置，而它的边长等于相邻两个像素之间的距离。在OpenGL中，像素的中心位于半整数值的位置，选择使用这种方法定义像素的中心位置具有某些优点（见习题7.19）。我们还假定从颜色缓存中读取像素的颜色值与显示像素这两个过程是按特定的速率同时进行的。这个假定对于许多支持双缓存的图形绘制系统来说都是成立的，这使我们可以把光栅化处理过程与帧缓存的显示过程看做是两个独立的过程。

最简单的线段扫描转换算法就是通常所说的DDA算法，它是以数字微分分析器（Digital Differential Analyzer）命名的，这种数字微分分析器是早期用于微分方程数字仿真的机电设备。因为直线满足下面的微分方程：

$$\frac{\mathrm{d}y}{\mathrm{d}x} = m$$

其中，m 是直线的斜率，生成一条线段相当于求解一个简单的数值微分方程。

假定有一条线段，它的两个端点分别定义为(x_1, y_1)和(x_2, y_2)。因为我们在颜色缓存中对线段进行光栅化处理，所以假定线段两个端点的坐标值是经过四舍五入处理后的整数值，因此线段的起始端点分别对应某个已知的像素[①]。线段的斜率为：

$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{\Delta y}{\Delta x}$$

假定：

$$0 \leqslant m \leqslant 1$$

① 这个假定对于算法的推导过程并不是必需的。如果对线段的端点采用定点数表示，并且使用定点数进行计算，那么在保留算法计算优势的同时会生成更加精确的光栅化结果。

我们可以利用对称性对斜率m为其他值的线段进行光栅化处理。该算法基于这样的思想，即当x坐标值从x_1变化到x_2时，我们把函数write_pixel中每个ix值对应的像素写入帧缓存中。如果要对如图7.27所示的线段进行光栅化处理，那么当x的变化量为x时，y的变化量必定为：

$$\Delta y = m\Delta x$$

当从x_1变化到x_2时，假定x的增量为1，那么y的增量为：

$$\Delta y = m$$

虽然每个x值都是整数，但是因为斜率m是浮点数，所以通过计算得到的每个y值并不是整数，为了找到合适的像素，必须对y值进行四舍五入处理，如图7.28所示。DDA算法的伪代码为：

```
for(ix=x1; ix <= x2; ix++)
{
     y+=m;
     write_pixel(x, round(y) , line_color);
}
```

算法中的round函数用于对一个实数进行四舍五入取整处理。我们把线段的最大斜率限制为1的原因可以从图7.29看到。DDA算法的一个基本思想就是对于每个x值，计算一个最佳的y值。对于斜率大于1的线段，由于两个着色像素之间的间隔很大，所以生成的近似线段无法令人接受。然而对于斜率大于1的线段，如果交换x和y，那么该算法就变为：对于每个y值，计算一个最佳的x值。对于同样的线段（如图7.29所示的线段），应该到如图7.30所示的近似线段。注意，利用这种对称性使我们不用考虑垂直线或水平线引起的任何潜在问题。读者也许希望推导负斜率线段的光栅化算法。

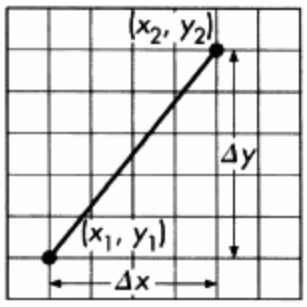


图7.27　屏幕窗口坐标系中的线段

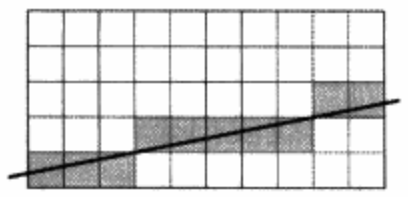


图7.28　利用DDA算法生成像素

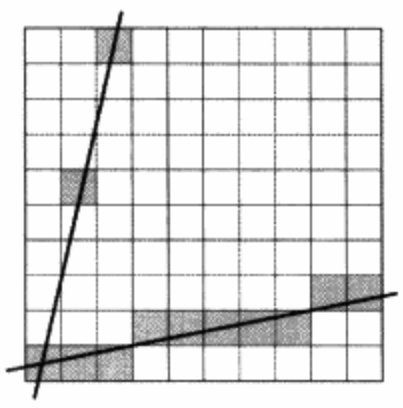


图7.29　光栅化斜率小于1和大于1的线段所生成的像素

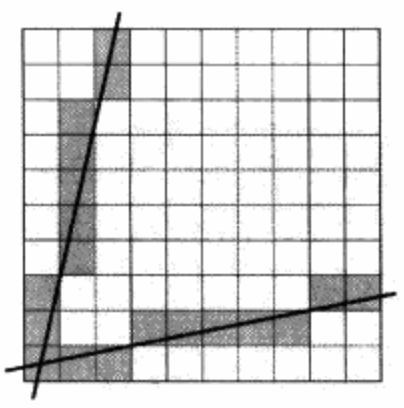


图7.30　用改进后的DDA算法生成的像素

因为线段是由它的两个端点确定的，所以我们可以使用插值方法给生成的每个像素赋予不同的颜色。通过改变所生成像素的颜色，还可以生成各种点画线图案，线段的这些显示效果与基本的光栅化算法没有什么关系，因为该算法的任务是确定需要给哪些像素着色，而不是确定使用哪种颜色给像素着色。

7.9　Bresenham 算法

DDA 算法看起来好像很有效，且该算法的编码实现确实非常容易，但是每生成一个像素都要用到一次浮点加法运算。Bresenham 提出的线段光栅化算法可以有效地避免使用浮点运算，而且它已经成为硬件和软件光栅处理器的标准算法。

类似于 DDA 算法，我们假定线段的起点(x_1, y_1)和终点(x_2, y_2)的坐标值都是整数，且斜率满足下面的条件：

$$0 \leqslant m \leqslant 1$$

借助于图 7.31 可以看到，该斜率条件对于 Bresenham 算法非常关键。在线段扫描转换过程的某一步，假定我们通过计算已经得到了一个$(i+\frac{1}{2}, j+\frac{1}{2})$位于位置的像素。该线段所在直线的方程为：

$$y = mx + h$$

当$x = i+\frac{1}{2}$时，该直线与$(i+\frac{1}{2}, j+\frac{1}{2})$处像素的垂直扫描线的实际交点一定是在垂直扫描线上位于该像素的半个像素内部的某一点①，否则，对该实际交点进行四舍五入处理后不可能得到当前这个位置的像素。如果计算下一个横坐标为$x = i+\frac{3}{2}$的像素，那么根据斜率满足的条件可知，此时对应的着色像素的位置坐标要么是$(i+\frac{3}{2}, j+\frac{1}{2})$，要么是$(i+\frac{3}{2}, j+\frac{3}{2})$。因此我们把问题归结为对这两个候选像素的选择，我们可以引入判定变量（decision variable）$d = b - a$来解决该问题，其中，a和b表示在$x = i+\frac{3}{2}$处，该线段分别离上方候选像素和下方候选像素的距离，如图 7.32 所示。如果$d<0$，那么线段靠近下方的像素，所以选择位于$(i+\frac{3}{2}, j+\frac{1}{2})$处的下方像素；否则，应选择位于$(i+\frac{3}{2}, j+\frac{3}{2})$处的上方像素。尽管可以利用线段的方程$y = mx + b$来计算$d$的值，但是由于$m$是浮点数，所以我们并不打算这样做。

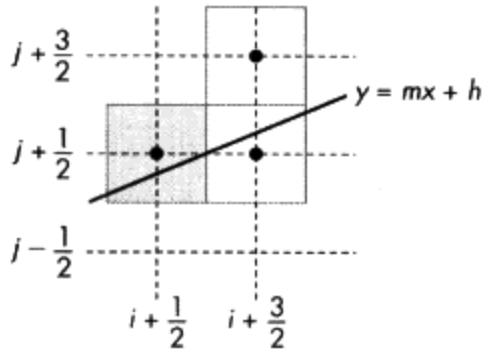


图 7.31　Bresenham 算法要满足的条件

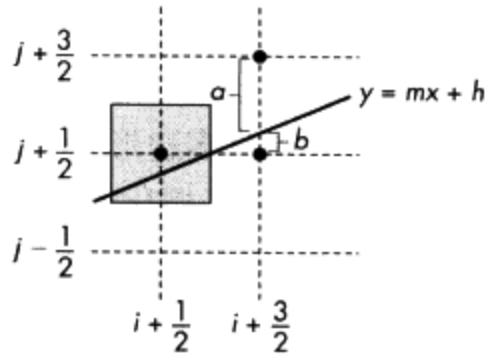


图 7.32　Bresenham 算法的判定变量

我们还要再增加两个处理步骤来优化 Bresenham 算法。首先，我们使用定点运算代替浮点运算。其次，要使该算法成为增量算法。首先使用一个新的判定变量代替d，即：

$$d = (x_2 - x_1)(b - a) = \Delta x(b - a)$$

由于我们根据d的符号来选择候选像素，所以新的判定变量并不影响候选像素的选择。如果利用线段的方程将a和b的值代入上式，并注意使用下面的两个等式，那么我们发现d是一个整数：

$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{\Delta y}{\Delta x}$$

$$h = y_2 - mx_2$$

虽然这样可以避免浮点运算，但是对d的直接计算需要多次用到定点运算。

① 假定像素中心位置的坐标值位于两个整数中间。

下面采用一种稍微不同的方法。假定 d_k 是 $x = k + \frac{1}{2}$ 时对应的 d 值。我们希望计算 d_k 的增量形式 d_{k+1}，它是 $x = k + \frac{3}{2}$ 时对应的 d 值。根据处像素的 y 坐标相对于前一个像素是否增加 1，我们分两种情况讨论，如图 7.33 所示。通过观察上方候选像素与直线之间的距离 a 的值，我们发现只有当 $x = k + \frac{1}{2}$ 处像素的 y 坐标相对于前一个像素增加 1（如图 7.33(b)所示）时，a 值增加 $1 - m$，或者 a 值减少 m（如图 7.33(a)所示）。同理，当增加 x 的值时，b 值要么增加 m（如图 7.33(a)所示），要么减少 $1 - m$（如图 7.33(b)所示）。如果计算 d 时乘以，那么 d 的增量要么等于 $2\Delta y$，要么等于 $2(\Delta y - \Delta x)$。我们可以使用下面这种迭代形式表示这个结果：

$$d_{k+1} = d_k + \begin{cases} 2\Delta y & \text{如果 } d_k < 0 \\ 2(\Delta y - \Delta x) & \text{其他} \end{cases}$$

因此颜色缓存中每个连续像素的计算只需用到一个加法运算和一个符号判断，该算法的效率非常高，以至于它已经被固化为图形芯片中的一条指令。对于判定变量初始值 d_0 的计算，请参见习题 7.14。

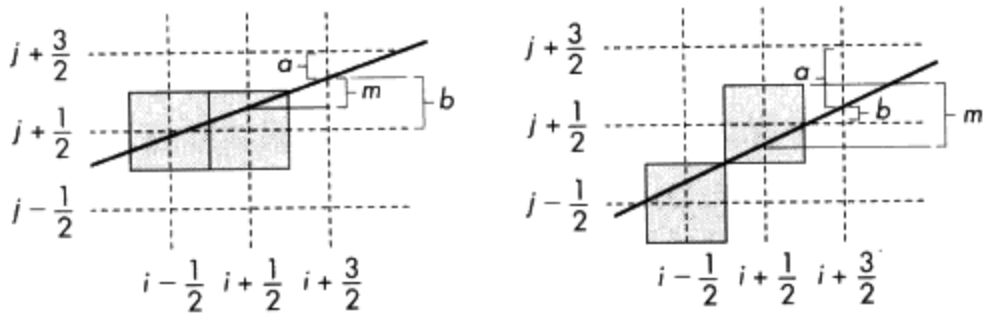


图 7.33　计算 a 和 b 的增量

7.10　多边形光栅化算法

最早出现的光栅图形系统给用户带来的主要优势之一就是能够显示填充的多边形。那时还无法做到用不同的颜色给多边形内部的每个点进行实时着色，光栅化多边形和多边形的扫描转换这两个术语在当时是指用单色填充多边形。不同于直线的光栅化算法，目前有许多可行的多边形光栅化算法可供用户选择，而直线的光栅化算法只有一种占统治地位。选择何种多边形光栅化算法主要取决于图形绘制系统的结构。我们将重点介绍那些既适合流水线绘制结构，又支持明暗绘制的多边形光栅化算法。7.10.4 ~ 7.10.6 节将简单介绍其他一些方法。

7.10.1　内外测试法

平面简单多边形（flat simple polygon）对内部区域具有清晰的定义。如果它们还满足凸多边形（convex polygon）的条件，那么使用OpenGL或其他图形绘制系统可以确保对它们进行正确的绘制。然而，我们在实际应用中会遇到更一般的多边形，可以使用多种方法对它们进行绘制。对于非平面多边形[①]，可以对它的投影多边形进行光栅化处理（见 7.10.2 节），或者使用它的前 3 个顶点确定的平面作为内部区域来使用。对于非简单平面多边形，必须确定一个给定的点是位于多边形的内部还是多边形的外部。从理论上来说，对多边形内部区域填充某种颜色的过程相当于判断多边形所在的平面上哪些点是多边形内部区域中的点。

相交测试（crossing test）或奇偶测试（odd-even test）是判断多边形内部 – 外部区域的最广泛使用的方法。假定 **p** 是多边形内部的某个点，从 **p** 点发出的射线一定与多边形的奇数条边相交。从

① 严格地说，并不存在这样的非平面多边形，因为我们无法定义它的内部区域，除非它是平面多边形。然而，从编程人员的观点来看，我们只需简单地给定一组顶点就可以定义一个多边形，而不用考虑这些顶点是否位于同一个平面内。

多边形外部某点发出并进入多边形的任意射线与多边形的偶数条边相交。因此，如果从多边形外部发出一条直线，该直线在到达某个待判定点之前与多边形的奇数条边相交，那么这个待判定点位于多边形的内部。对于如图7.34所示的星形多边形，其内部填充效果如图所示。奇偶测试法很容易实现，并且可以很好地与标准绘制算法结合在一起。我们通常使用扫描线代替射线，并根据扫描线与多边形交点的数量来确定多边形的内部区域和外部区域。

然而，我们希望使用填充算法填充星形多边形的效果应该如图7.35所示，而不是如图7.34所示的效果。我们使用环绕测试法（winding test）可以解决这个问题。环绕测试法认为多边形是一个绕某个点或某条线形成的一个结。为了实现环绕测试，我们把多边形的边视为有向边，即从任意一个顶点出发，按某个特定的方向（究竟是哪个方向并不重要）环绕多边形直到回到出发点。我们在多边形的边上使用箭头表示环绕的路径，如图7.35(b)所示。下面考虑一个任意的点，该点的环绕次数（winding number）定义为多边形的边环绕该点的次数。我们规定顺时针环绕为正，而逆时针环绕为负（或者相反）。因此，对于位于图7.35所示的星形多边形之外的点，因为没有边环绕它，所以它的环绕次数等于0；对于图7.34所示的已被填充的区域，位于该区域内的点的环绕次数等于1；对于图7.34所示的星形多边形中央未被填充的区域，位于该区域内的点的环绕次数等于2。如果改变填充规则，即规定环绕次数为非0的点位于多边形内部区域，那么星形多边形内部的填充效果如图7.35(a)所示。

图7.34　使用奇偶测试法的多边形填充效果

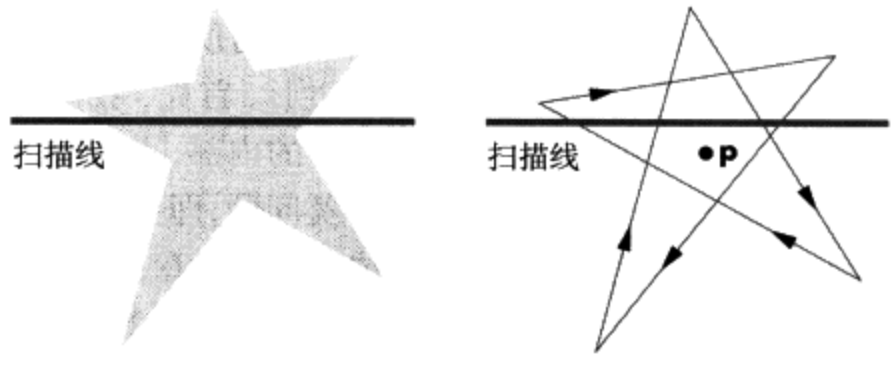


图7.35　基于环绕测试法的多边形填充效果

7.10.2　OpenGL与凹多边形

因为只有当绘制的多边形是凸多边形时，OpenGL才能确保正确地绘制该多边形，所以还有一个问题需要解决，即如何处理更一般的多边形。一种方法是要求应用程序确保所有的多边形遵从这个限制条件。另一种方法是通过软件的方法把给定的一般多边形细分成多个平面凸多边形（一般是三角形）。把给定的一般多边形细分成多个三角形的方法很多。一种好的细分算法不会产生细长的三角形，如果可能，细分算法生成的一组三角形应该满足良好的几何特性，如三角形条带或三角形扇形。OpenGL的GLU库提供了细分函数。对于一个没有孔的简单多边形，要细分这样的多边形，首先应定义一个细分对象（tessellator object），然后给它提供多边形的顶点，代码如下：

```
GLUtesslator *mytess;
mytess = gluNewTess();

gluTessBeginPolygon(mytess, NULL);
gluTessBeginContour(mytess);
for(i=0;i<nvertices;i++)
   glTessVertex(mytess,vertex[i],vertex[i]);
gluTessEndContour();
gluTessEndPolygon(mytess);
```

虽然有许多参数要设置，但细分的基本思想是：先定义细分对象的轮廓，然后通过细分算法生成所需的三角形并把它们送入图形绘制系统，图形绘制系统会根据当前的细分参数绘制这些三角形。

前面介绍的环绕次数在OpenGL的细分算法和NURBS裁剪曲面的细分（见第12章）中有着非常重要的作用。多边形的内部轮廓线相对于某一点的绕向（取决于细分参数）可以导致各种不同的绘制结果，如图7.36所示。本章后面推荐的参考文献对各种细分算法进行了详细的介绍。从OpenGL的观点来看，如果使用了细分算法，那么所有的绘制对象都是凸多边形。

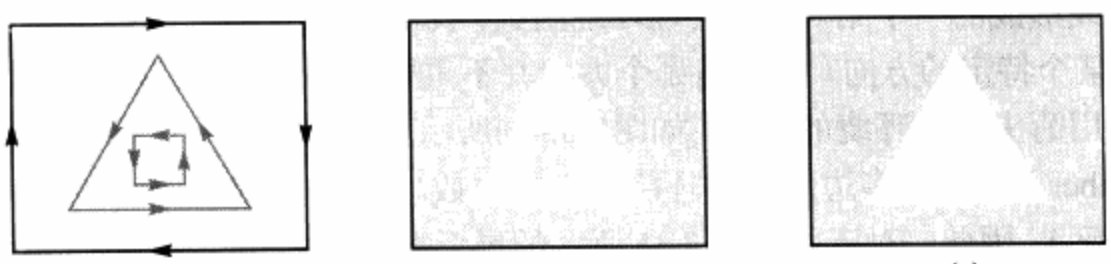


图 7.36　多边形的细分：(a) 多边形的边和轮廓线方向；(b) 细分 1；(c) 细分 2

7.10.3　填充与分类

另一种完全不同的多边形光栅化方法来自多边形处理器的思想，可以把多边形处理器看成是一个黑盒子，它的输入是一组二维多边形的顶点，输出是存储了正确像素阵列的帧缓存。为了讨论的方便性，假定使用单色填充每个多边形。首先，我们考虑单个多边形的情况。填充一个多边形的基本规则如下：如果某个点位于多边形的内部，那么使用多边形内部色（填充色）给它着色。这个概念上的算法说明多边形填充就是分类问题，即要求我们把帧缓存中所有的像素分成在多边形内部的一类和在多边形外部的一类。从这个观点来看，基于不同的顶点分类方法，可以得到各种不同的多边形填充算法，这里只介绍其中的3种多边形填充算法：

- 漫水填充（flood fill）算法[①]
- 扫描线填充算法
- 奇偶填充算法

7.10.4　漫水填充算法

为了显示一个未填充的多边形，可以使用Bresenham算法将该多边形的边光栅化为帧缓存中像素。假定只有两种颜色：背景色（白色）、前景色或绘制色（黑色）。对多边形的边进行光栅化处理后，我们把它的颜色置成前景色，一个简单多边形光栅化后在帧缓存中的着色效果如图7.37所示。如果能够在多边形的内部区域找到一个初始点(x, y)，即种子点（seed point），那么就可以递归地进行如下的测试：如果它的相邻像素不在多边形的边上，那么就用前景色对它着色。假定函数read_pixel用来返回某个像素的颜色，则漫水填充算法（flood-fill algorithm）可以用下面的伪代码表示：

```
flood_fill(int x, int y)
{
     if(read_pixel(x,y)==WHITE)
     {
          write_pixel(x,y,BLACK);
          flood_fill(x-1,y);
```

① 也称为种子填充算法。——译者注

```
            flood_fill(x+1,y);
            flood_fill(x,y-1);
            flood_fill(x,y+1);
        }
    }
```

如果用迭代的形式改写上面的递归代码，那么可以得到水漫填充算法的许多变异形式，其中的一种方法是每次处理一条扫描线。

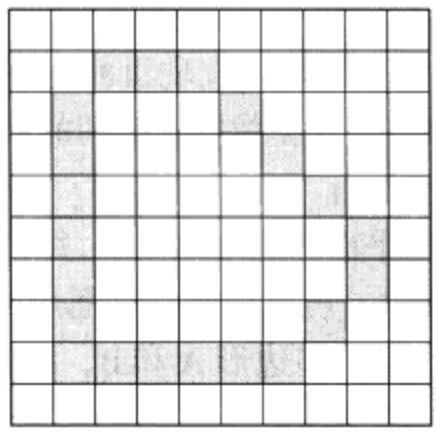


图 7.37　用多边形的边显示多边形

7.10.5　奇点处理

对于大多数的多边形填充算法，如果在使用时稍加注意，就可以把它们扩展到对其他形状的填充（见习题7.17）。多边形具有一个显著的优点，就是可以得到它的边的精确位置。然而，当多边形的顶点正好位于扫描线上时，多边形的填充也会出现问题。下面考虑如图7.38所示的两种情形。如果使用奇偶填充法的定义，那么必须对这两种情形区别对待。对于图7.38(a)所示情形，可以把扫描线与多边形的边在顶点处的交点数量计为0个或2个（扫描线与交点所在的两条边都相交）；对于图7.38(b)所示情形，扫描线与多边形的边在顶点处的交点数量计为1个（扫描线只与交点所在的两条边中的一条边相交）。

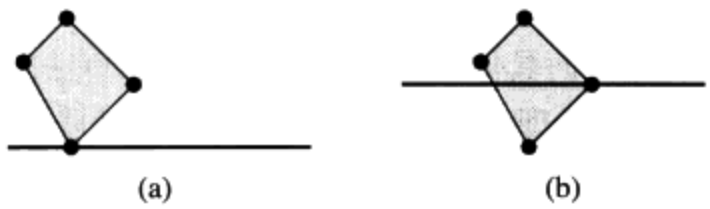


图 7.38　奇点处理：(a) 扫描线与多边形的 0 条边或两条边相交；(b) 扫描线与多边形的 1 条边相交

可以使用下面介绍的两种方法中的任一种来修改填充算法。我们可以先判断当前处理的奇点属于上面的哪种情形，然后就可以对交点的数量进行正确计数了。或者我们从一开始就阻止这种位于多边形边上的奇点的出现。如果我们能确保多边形所有顶点的y坐标值都不是整数，那么就不会出现奇点。如果发现某个奇点的y坐标值是整数，只需稍微扰动它的位置即可消除该奇点。还有另一种方法，如果在帧缓存中处理问题，该方法非常有价值，它利用了一个两倍于实际帧缓存分辨率的虚拟帧缓存。在虚拟帧缓存中，像素只位于y为偶数的位置，而所有顶点只位于y为奇数的位置。OpenGL采用的就是这种方法，也就是把像素中心放在两个整数的中间值所在的位置。

7.11　隐藏面消除

虽然通过光栅化处理生成的每个片元都对应于颜色缓存中的某个位置，但是如果某个片元所在的对象位于另一个不透明对象的后面（即被不透明对象所遮挡），那么我们并不希望通过给该片元

对应的像素着色而显示该片元。对于位于视见体内的每个对象，隐藏面消除（或可见面确定）用来确定这些对象的哪些部分是可见的，或者确定哪些对象在视线上被其他对象所遮挡。我们将介绍一些隐藏面消除算法用来处理仅由平面多边形组成的场景。因为大部分图形绘制系统在这一步已经将曲面细分成多个多边形，所以上面的这种选择是非常合适的。对这些算法稍加修改就可以处理线段的消隐问题（见习题7.7）。

7.11.1　基于对象空间和基于图像空间的方法

通过对隐藏面消除算法的研究，可以清楚地发现目前有许多这样的消隐算法，我们还可以发现这些算法在处理消隐问题时存在对象空间和图像空间之间的区别，以及评估这些算法在实现过程中对后继出现的算法产生持续影响的重要性。

下面考虑一个包含k个三维不透明平面多边形的场景，每个多边形都被认为是一个独立的对象。当从投影中心观察场景中所有的对象时，可以得到一种通用的基于对象空间的方法（object-space approach）。例如，考虑场景中两个这样的多边形A和B，它们之间的位置关系为下面的4种情形（如图7.39所示）：

1. 在视线方向上A完全遮挡了B，只显示A。
2. 在视线方向上B完全遮挡了A，只显示B。
3. A和B都完全可见，既显示A也显示B。
4. A和B有一部分相互遮挡，必须计算每个多边形的可见部分。

为了考虑更复杂的场景，可以把上述4种情形的判断和对多边形可见部分所需的计算看做是单个操作。下面采用迭代的方法处理该问题。首先从场景中的k个多边形中任意选择一个多边形，并把它与其余的$k-1$个多边形两两比较其位置关系，然后就可以知道该多边形的哪一部分可见（如果存在可见部分），并绘制其可见部分。处理完这个多边形之后，我们从其余的$k-1$个多边形中再任意选择一个多边形并重复上面的过程。每一步都要把一个多边形与其余的每个多边形进行两两比较，直到只剩下两个多边形，并对它们进行相互比较。我们可以很容易地确定这种方法的时间复杂度为$O(k^2)$。因此，根本不用研究任何具体的基于对象空间消隐算法的实现细节，我们就足以怀疑这种基于对象空间的方法能很好地处理包含少量多边形的场景消隐问题。

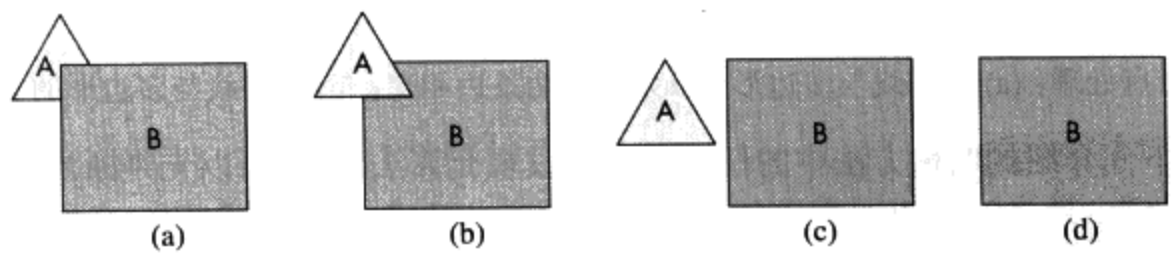


图7.39　两个多边形：(a) B部分遮挡了A；(b) A部分遮挡了B；(c) A和B都可见；(d) B完全遮挡了A

基于图像空间的方法（image-space approach）遵循观察和光线投射模型，如图7.40所示。下面考虑从投影中心发出并通过某个像素的一条光线。我们可以考虑这条光线与场景中k个多边形所在的每个平面相交的情形，确定该光线穿过哪些多边形所在的平面，并最终确定离投影中心最近的交点，然后使用多边形在交点处的颜色给对应的像素着色。这里用到的最基本运算就是光线与多边形的求交计算。如果显示分辨率大小为$n \times m$的图像，那么必须执行nmk次求交运算，其时间复杂度为$O(k)$[①]。在这里不用研究算法的细节，就可以得到算法复杂度的一个上限。通常情况下，时间

① 为了提高绘制的精确度，可以对每个像素使用多条光线。

复杂度上限$O(k)$正说明了基于图像空间的方法占优势地位的原因。上限$O(k)$是最坏情况下的时间复杂度，在实际中，对于这种基于图像空间方法的消隐问题，其执行效率会更高（见习题7.9）。然而，因为基于图像空间方法的消隐问题工作在片元或像素一级，所以其精确度受到帧缓存分辨率的限制。

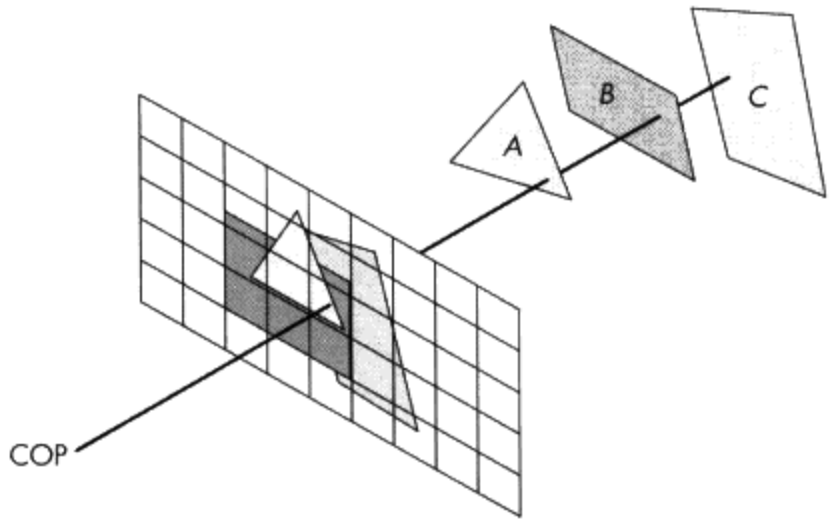


图7.40　基于图像空间的隐藏面消除

7.11.2　排序与隐藏面消除

对于基于对象空间的隐藏面消除算法，$O(k^2)$的时间复杂度可能使读者想起某些最坏情况下的排序算法，如冒泡排序。任何基于这种蛮力方法两两比较对象的排序方法都具有$O(k^2)$的时间复杂度，但是7.11.5节将[①]介绍的基于对象空间的排序算法与隐藏面消除本身有着更为直接的联系。如果我们能够根据对象离视点的距离来组织这些对象，就可以提出一种更为直接的方法来绘制这些对象。

基于这种比较，我们知道一个好的排序算法的时间复杂度为$O(k \log k)$。我们应该希望基于对象空间的隐藏面消除算法也能具有这样的时间复杂度，而事实就是如此。对于排序问题，有多个满足这个时间复杂度上限要求的排序算法。此外，还有一些问题涉及对象之间的比较（如碰撞检测问题），乍看起来，这类问题的时间复杂度似乎为$O(k^2)$，而事实上可以降到$O(k \log k)$。

7.11.3　扫描线填充算法

扫描线填充算法（scanline algorithm）之所以吸引人们的目光，是因为该方法具有按扫描显示像素的方式来生成这些像素的潜力。下面考虑如图7.41所示的多边形和它上面的一条扫描线。如果使用奇偶判断规则来定义多边形的内部区域，不难发现在扫描线上3组像素或3个填充区间（span）位于多边形的内部。我们注意到每个填充区间可以独立地进行光照或深度计算，某些具有并行区间处理器的图形硬件采用了这种方法。对于使用单色填充多边形的简单例子，当确定了各个填充区间之后，就可以使用填充色给每个区间内的像素着色。

填充区间是由扫描线与多边形的边形成的交点序列确定的。多边形的顶点包含用来确定这些交点的所有信息，但是我们用来表示多边形的方法确定了这些交点生成的顺序。例如，考虑如图7.42所示的多边形，它由一个有序的顶点序列表示。用来生成扫描线与多边形边线交点的最常见方法是逐一处理相邻两个顶点的边线。图7.42显示了这些交点，并按上面这种方法生成交点的顺序来编号。注意，可以通过增量计算的方式来计算边线上的这些交点的坐标值（见习题7.18）。然而，就多边形填充而言，这样的交点序列并不是我们所需要的。如果每次填充一条扫描线，那么我们希望对内部交点序列进行排序，即首先按扫描线、然后按每条扫描线上交点的x坐标值进行排序，如

① 此处原文为7.8.5，可能有误。——译者注

图7.43所示。使用蛮力方法可以把所有的交点排成所需的顺序。然而，对于一个非常大的或锯齿状的多边形，扫描线可能与它的很多边线相交，因此交点的数量 n 会变得非常之大（例如，考虑一个横跨半数扫描线的多边形），以至于时间复杂度为 $O(n \log n)$ 的排序对实时绘制系统而言，其计算速度太慢了。

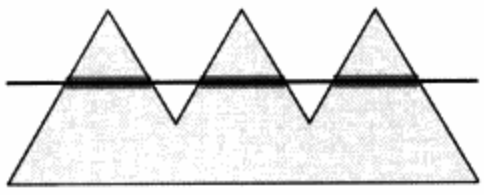


图 7.41　具有内部区间的多边形

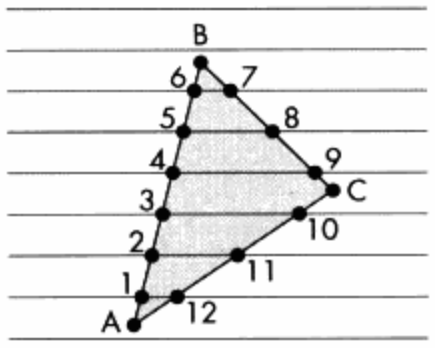


图 7.42　用顶点序列表示的多边形

许多填充算法并不采用这种常规的蛮力搜索方法。有一种算法起初被称为 y-x 算法，该算法为每条扫描线创建一个吊桶。随着算法对多边形边线的处理，边线与扫描线的交点放置在相应的吊桶中。在每个吊桶中，使用插入排序方法对每条扫描线上的交点序列的 x 坐标值进行排序。算法使用的数据结构如图7.44所示。我们再一次看到，一个合理选择的数据结构可以有效地提高算法的运行速度。我们甚至可以重新考虑多边形的表示方法，从而进一步提高算法的运行速度。如果这样做，那么可以得到 7.8 节介绍的扫描线算法[①]。

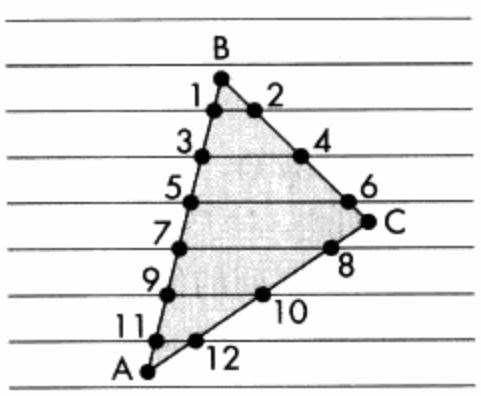


图 7.43　顶点序列的理想顺序

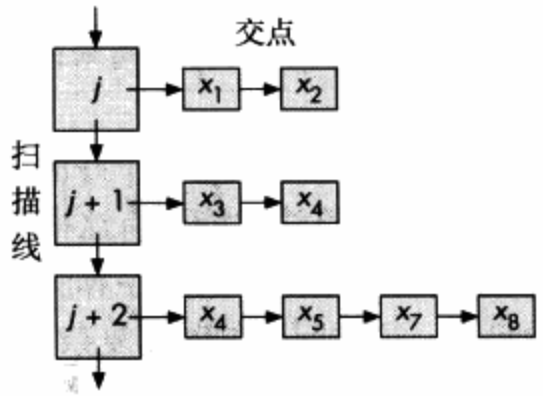


图 7.44　y-x 算法的数据结构

7.11.4　背面剔除

在第 6 章中，我们注意到在 OpenGL 中可以选择只绘制对象的正面（front-facing）多边形。对于无法看到背面多边形的情形，如由凸面体组成的场景，我们可以在执行隐藏面消除之前把所有的背面（back-facing）多边形剔除掉，从而减轻隐藏面消除的计算量。我们可以从图 7.45 推导出可被剔除（culling）的背面多边形的判断条件。从图 7.45 可以看到，如果多边形某个面朝外的法向量指向观察者，那么观察者看到的是多边形的正面。如果法向量与视线方向的夹角为 θ，那么多边形朝向观察者当且仅当满足条件：

$$-90 \leqslant \theta \leqslant 90$$

或者满足与之等价的条件：

$$\cos\theta \geqslant 0$$

第二个条件更容易判断，这是因为我们不用计算余弦，而只需利用下面的点积运算：

① 根据DDA算法，多边形边线上两个相邻交点的 x 坐标值之间存在增量关系，即边的连贯性，利用边的连贯性可以进一步提高算法的实现效率。——译者注

$$\mathbf{n} \cdot \mathbf{v} \geqslant 0$$

如果我们注意到背面剔除通常是在多边形变换到规范化的设备坐标系之后才进行的，那么可以进一步简化上面的判断条件。在规范化的设备坐标系中，所有的投影都是正投影，投影方向沿z轴方向。因此，在规范化的设备坐标系中，

$$\mathbf{v} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

因此，如果多边形所在平面的平面方程在规范化的设备坐标系中表示为：

$$ax + by + cz + d = 0$$

那么只需测试c的符号就可以确定所判断的多边形是正面还是背面。我们可以很容易地使用硬件或软件来实现这个测试，只需注意在应用程序中确保背面多边形剔除的正确性即可。

在OpenGL中，可以使用函数glCullFace来开启背面剔除功能。OpenGL采用了一种不同的方法来判断多边形的背面。该算法基于对多边形在屏幕坐标系中的面积的计算。考虑如图7.46所示的具有n个顶点的多边形，它的面积可以表示为：

$$\frac{1}{2}\sum_i (y_{i+1} + y_i)(x_{i+1} - x_i)$$

其中，顶点坐标的标号i以n为模数（见习题7.28）。如果面积为负值，那么该多边形表示背面多边形。

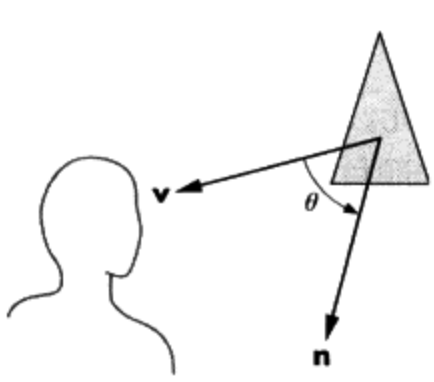


图7.45　背面判断

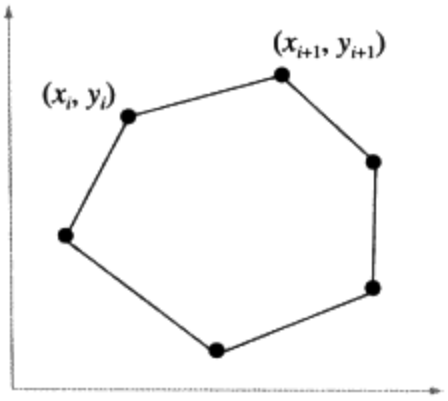


图7.46　计算多边形的面积

7.11.5　z-buffer算法

z-buffer算法是使用最广泛的隐藏面消除算法，它具有众多的优点，比如易于通过硬件或软件来实现，并且与图形的绘制流水线结构相兼容，在这种流水线绘制结构中，该算法能够以片元通过流水线的速度快速地执行。尽管该算法工作在图像空间，但是它的外层循环遍历每个多边形而不是每个像素，所以可以把它看成是7.10节讨论的多边形扫描转换过程的一部分。

假定正在对如图7.47所示的两个多边形中的一个多边形进行光栅化处理。从投影中心向某个像素发出一条射线，我们可以使用第6章介绍的顶点颜色插值公式来计算该射线与每个多边形交点的颜色。此外，我们还必须检测该交点的可见性。如果它是射线上离视点最近的交点，那么该交点是可见的。因此，如果正在光栅化多边形B，并且像素点到B的距离z_2比到A的距离z_1要小，那么屏幕上显示的是多边形B的颜色。相反，如果正在光栅化多边形A，那么射线与多边形A的交点对应的像素颜色不会显示在屏幕上。然而，由于对多边形是逐个进行处理的，所以当光栅化某个多边形的时候，并不知道所有其他多边形的信息。然而，如果保存每个片元的深度信息，那么随着对每个片元的处理，我们可以存储或更新它在帧缓存对应位置的深度信息。

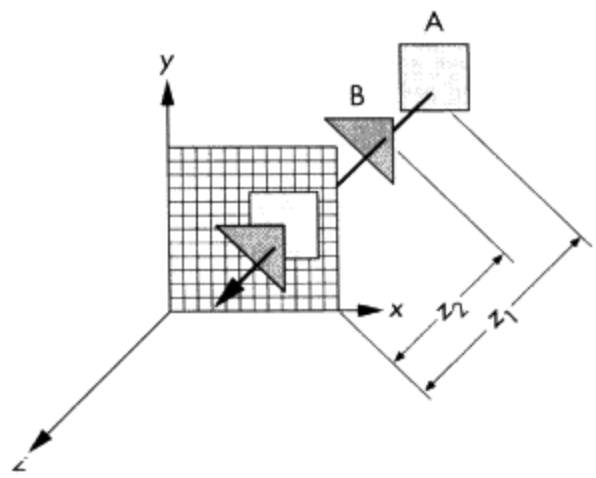


图 7.47　z-buffer 算法

假定有一个称为 z-buffer 的缓存，它与帧缓存具有相同大小的空间分辨率，其深度分辨率就是在存储距离（深度）时我们希望使用的二进制位数。例如，如果我们使用的显示器的分辨率为 1024 × 1280，并使用标准的整数表示深度值，那么可以使用空间分辨率为 1024 × 1280 且深度分辨率为 32 位的 z-buffer。z-buffer 算法执行时，深度缓存中存储的每个元素都初始化为离投影中心[①]的最大深度值，颜色缓存初始化为背景颜色。在光栅化和片元处理的任何时刻，z-buffer 总是在相应位置存储到目前为止通过该位置的射线与多边形交点的最近距离值。

下面介绍 z-buffer 算法的执行过程。我们首先采用 7.10 节介绍的某种方法对多边形逐个进行光栅化处理。对于某个多边形上的每个片元，它对应于通过某个像素位置的射线与该多边形的交点，我们都要计算片元到投影中心的距离（即片元的深度值），然后把片元的深度值与 z-buffer 相应位置的深度值进行比较。如果片元的深度值大于 z-buffer 相应位置的深度值，那么在它之前已处理的那个多边形上对应的片元离视点更近，所以当前处理的这个片元不可见。如果片元的深度值小于 z-buffer 相应位置的深度值[②]，那么找到了一个离视点更近的片元。此时我们使用该片元的深度值来更新 z-buffer 相应位置的深度值，并把颜色缓存中相应位置的颜色值更新为该片元的颜色。注意，从透视投影的角度来看，我们在 z-buffer 算法中使用的深度距离实际上是经过了第 5 章介绍的规范化变换之后的距离。虽然这个规范化变换是非线性的，但是并没有改变距离的相对大小。然而，这种非线性变换可能导致数值上的不精确性，尤其是离近裁剪面的距离很小的时候。

对于图形绘制系统的其他处理过程，用户并不知道 OpenGL 采用了哪种特定的实现算法，而对于隐藏面消除，OpenGL 采用了 z-buffer 算法。之所以会有这个例外，是因为每当生成新的图像时，要求用户必须在应用程序中初始化 z-buffer。

z-buffer 算法非常适合于使用基于图像空间的方法来实现，因为在其实现过程中可以采用计算开销非常小的增量计算方法。假定我们基于扫描线光栅化方法正在逐条扫描线地光栅化一个多边形（7.9 节讨论过这个方法）。如图 7.48 所示，多边形所在的平面方程可以表示为：

$$ax + by + cz + d = 0$$

假定(x_1, y_1, z_1)和(x_2, y_2, z_2)是多边形（平面）上的两个点。如果

$$\Delta x = x_2 - x_1$$
$$\Delta y = y_2 - y_1$$
$$\Delta z = z_2 - z_1$$

① 如果已经执行了透视投影的规范化变换，那么就应该用投影方向代替投影中心，这是因为变换后所有的投影线都是平行线。然而，这个变换并不影响 z-buffer 算法，因为我们可以从任意平面（如平面 $z = 0$）开始测量深度距离，而不一定要从投影中心测量深度距离。

② 在 OpenGL 中，可以使用函数 glDepthFunc 来确定当距离相等时的处理方法。

那么可以用差分的形式来写该平面的方程：

$$a\Delta x + b\Delta y + c\Delta z = 0$$

该方程位于窗口坐标系中，因此每条扫描线都对应于一条y为常量的直线，并且当沿着扫描线扫描时，$\Delta y = 0$。在某条扫描线上，我们按单位步长增加x的值，相当于在帧缓存中移动一个像素，所以Δx是常量。因此，当我们在扫描线上从一个点移动到另一个相邻点时，

$$\Delta z = -\frac{a}{c}\Delta x$$

这个值也是一个常量，所以对每个多边形只需要计算一次。

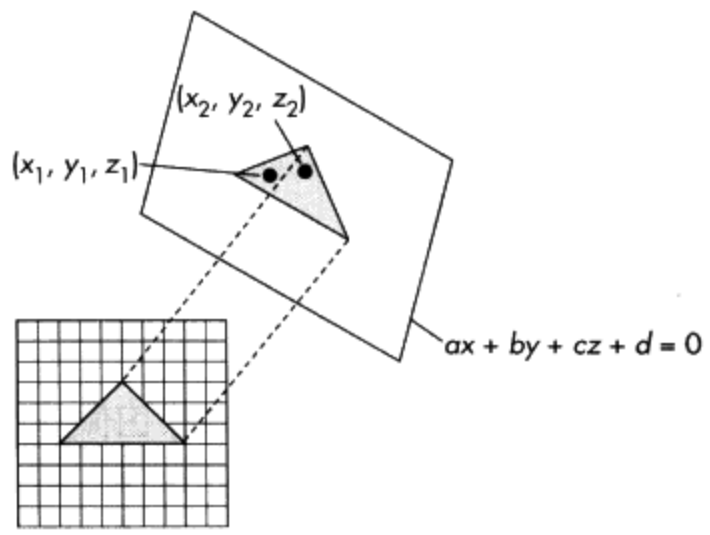


图 7.48　增量式 z-buffer 算法

尽管基于图像空间的算法在最坏情况下的性能与图元的数量成正比，但是z-buffer算法的性能与光栅化过程生成的片元数量成正比，而这取决于光栅化后的多边形的面积。

7.11.6　基于 z–buffer 的扫描转换算法

前面已经介绍了大部分有关多边形光栅化的基本内容。7.10.1节讨论了用于判断某个点是否位于多边形内部的奇偶测试法和环绕测试法。第6章介绍了基于插值计算的多边形着色方法。本节只需将这些零散的方法组合起来并考虑如何进一步提高光栅化算法的效率。

假定再一次考察绘制流水线的整个流程，重点考虑它对单个多边形的影响。顶点和法向量数据以一次一个的方式通过几何变换，在到达绘制流水线的裁剪阶段之前，必须把顶点组装成一个多边形。如果该多边形没有被裁剪掉，那么它的顶点和法向量被传送到绘制流水线的明暗绘制和隐藏面消除处理阶段。此时，虽然已经进行了投影的规范化处理，但是仍然保留了顶点的深度信息。如果我们希望采用基于插值计算的明暗处理方法，那么此时可以计算每个顶点的光强度。

还有3项任务需要完成：最终正投影变换的计算、隐藏面消除和明暗处理。如果灵活谨慎地使用z-buffer算法，那么可以同时完成这3项任务。考虑如图7.49所示的一个多边形的两种表示，左图表示的多边形位于三维规范化的设备坐标系中，而右图的多边形位于投影后的屏幕坐标系中。

我们采用的方法是对每个多边形进行逐个处理，且每次只处理一条扫描线。如图7.50所示，如果采用上述两种方式来表示同一个多边形，那么我们发现从屏幕坐标系逆向投影到规范化的设备坐标系中的一条扫描线对应于一条y为常量的直线。假定同时在该扫描线及其逆向投影直线上进行光栅化处理，对于位于屏幕坐标系中的扫描线，我们每次移动一个像素。我们可以按增量计算的方式来确定位于规范化设备坐标系中的这条直线上相应点的深度值，从而判断屏幕坐标系中的对应像素

是否对应于多边形上的可见点。如果通过光照计算已经得到了原多边形每个顶点的明暗值，那么就可以利用双线性插值来计算可见像素的正确颜色值[①]。相比于前面讨论过的其他每个处理过程，这个处理过程并不会增加额外的计算开销，因此系统的绘制性能主要受到多边形通过绘制流水线速度的限制。如果希望对上面处理的多边形做一些修改，如在多边形上应用位图案（即点画线图案）或纹理，那么只需稍加修改上述的处理过程即可。

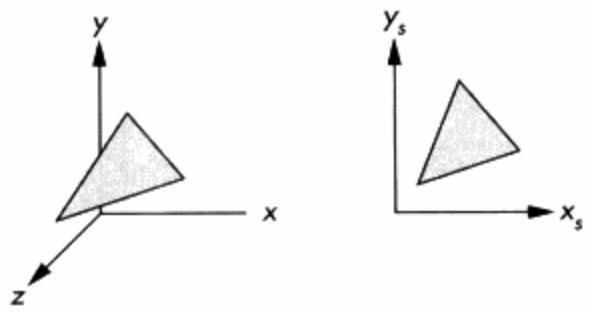


图 7.49　一个多边形的两种表示：(a) 在规范化的设备坐标系中；(b) 在屏幕坐标系中

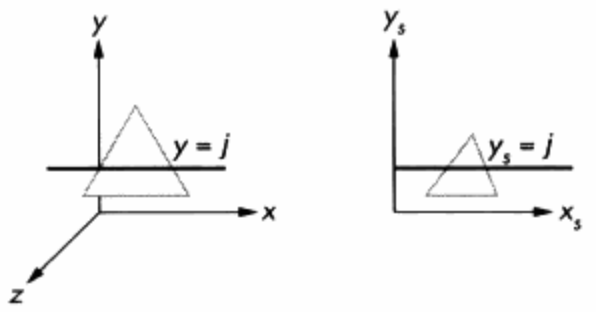


图 7.50　扫描线：(a) 在规范化的设备坐标系中；(b) 在屏幕坐标系中

7.11.7　深度排序和画家算法

虽然由于z-buffer算法的高效性和易于实现的优点使基于图像空间的消隐算法在硬件实现中占支配地位，但是在应用程序中也经常使用基于对象空间的消隐算法来减少多边形的数量。深度排序（depth sort）是实现基于对象空间消隐算法的一种最直接的方式。这里介绍的算法只处理由平面多边形构成的场景，可以把算法扩展到处理其他类型的对象。深度排序是一种称为画家算法（painter's algorithm）（一种更简单的消隐算法）的变异形式。

假定我们有一组多边形，这些多边形根据其离视点的远近进行排序。对于如图7.51(a)所示的例子，场景中有两个多边形，图7.51(b)所示的效果是观察者观察到的效果，即位于前面的多边形部分遮挡了另一多边形。为了正确地绘制场景，由于后面这个多边形有一部分区域是可见的，所以要把这部分可见区域绘制到帧缓存中（这个计算要求用一个多边形裁剪另一个多边形）。我们也可以使用另一种方法绘制这个场景，该方法类似于画家绘制这个场景时使用的方法。画家一般先画远处的整个多边形，然后再画前面的多边形，这样就可以在绘制过程中覆盖位于后面的且对观察者来说不可见的那部分多边形。场景中的这两个多边形都需要完全地绘制出来，最终通过这种从后向前绘制（back-to-front rendering）多边形的方法实现了隐藏面消除[②]。与该算法有关的两个问题是：如何对多边形进行排序，以及如何对相互重叠的多边形进行消隐处理。深度排序能够解决这两个问题，然而可以在许多应用中找到更有效的解决方法（见习题 7.10）。

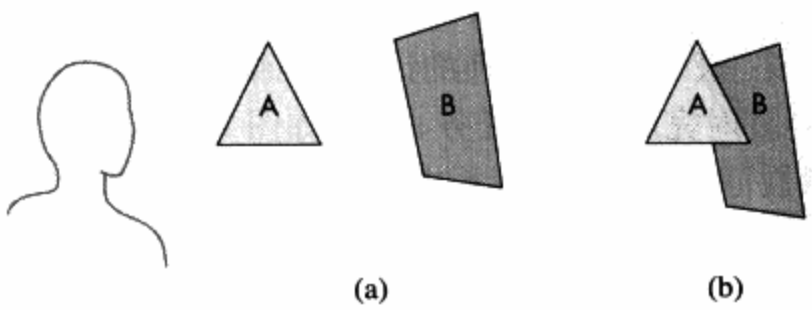


图 7.51　画家算法：(a) 两个多边形和观察者；(b) A 部分遮挡 B 的观察效果

① 最新的图形处理器以正确的透视方式执行插值计算。

② 使用从前向后绘制多边形的方法能更有效地解决某些应用问题。

假定我们已经通过计算得到了每个多边形的范围。深度排序的下一步处理就是根据多边形离视点的最大z值对所有的多边形进行排序，正因为这个处理过程，所以我们把这个算法称为深度排序算法。假定多边形的顺序如图7.52所示，该图显示了排序后每个多边形z值的取值范围。如果某个多边形的最小深度值（即z值）大于它后面某个多边形的最大深度值，那么我们可以从后向前绘制这些多边形，从而可以得到正确的绘制效果。例如，在图7.52中，多边形A位于所有其他多边形的后面，因此可以先绘制该多边形。然而，其他多边形不能仅仅根据z的取值范围来决定是否绘制相应的多边形。

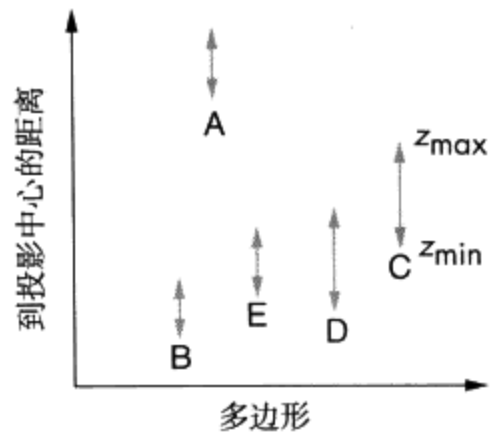


图 7.52　排序后每个多边形z值的取值范围

如果两个多边形的z值范围有重叠，那么仍然可以找到一个顺序来绘制它们，并得到正确的图像。为了找到这样一个正确的绘制顺序，深度排序算法需要按测试的难度依次执行一系列的测试。下面考虑z值范围重叠的两个多边形。最简单的测试方法就是检测它们的x取值范围或y取值范围是否有重叠（如图7.53所示）。如果它们的x取值范围或y取值范围不重叠[①]，那么这两个多边形互不遮挡，可以按任意的顺序绘制这两个多边形。即使这两个测试都失败了，我们仍可能找到一个绘制它们的正确顺序，图7.54说明了这种情况，其中一个多边形的所有顶点都位于另一个多边形所在平面的同一侧。为了确定两个多边形的位置关系是否属于这种情形，可以对这两个多边形的顶点进行处理（见习题7.12）。

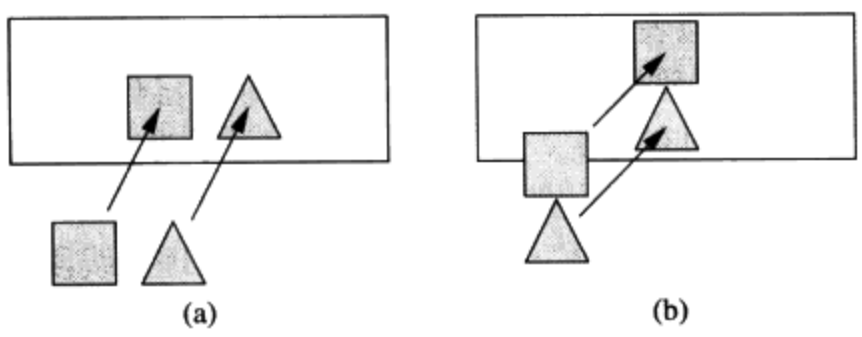


图 7.53　测试多边形的x取值范围和y取值范围是否重叠：(a) 两个多边形的x取值范围不重叠；(b) 两个多边形的y取值范围不重叠

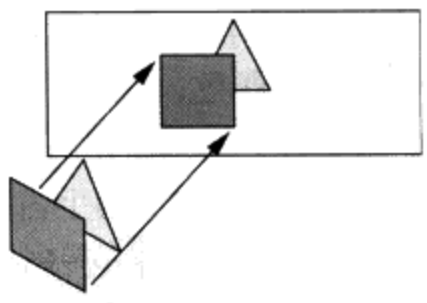


图 7.54　相互重叠的两个多边形

还有两种难以处理的情况。如果有3个或更多的多边形循环重叠，如图7.55所示，那么我们无法找到正确的绘制顺序。最好的办法就是至少把其中的一个多边形分割成两个多边形，从而得到一个新的多边形集，然后再试图找到一个正确的绘制顺序。第二种难以处理的情况如图7.56所示，一个多边形可以穿透另一个多边形。如果希望继续使用深度排序，那么必须执行复杂的求交计算（相当于用一个多边形裁剪另一个多边形）。如果相交的多边形有大量的顶点，那么不如试试使用其他

① 基于x取值范围或y取值范围的测试方法只能应用于多边形在xy平面上的平行投影视图。这是说明工作在透视投影经规范化变换后的规范化设备坐标系中好处的另一个例子。

计算量较少的算法。对深度排序的性能分析是非常困难的，这是因为对不同的特定应用来说，该算法使用难度较大的测试方法的次数是不同的。例如，我们处理的是用于描述实体对象表面的多边形，那么任意两个多边形都不会相交，然而显而易见的是，因为深度排序算法需要进行初始排序，所以它的时间复杂度至少是 $O(k \log k)$，其中 k 表示对象的数量。

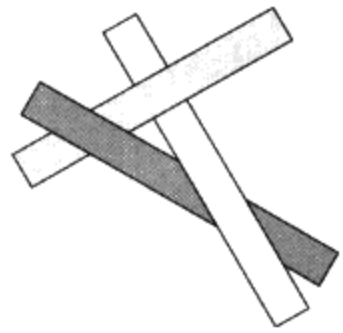


图 7.55　多边形循环重叠

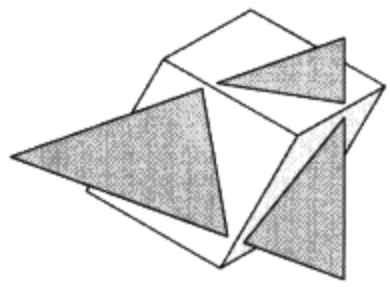


图 7.56　多边形相互穿透

7.12　反走样

线段和多边形的边线经光栅化处理后看起来呈锯齿状，即使在分辨率高达1024 × 1280的显示设备上，我们也能注意到屏幕显示的图像中存在这样的瑕疵。每当把具有无限分辨率的连续对象表示成具有有限分辨率的近似采样对象时，就会出现这种类型的误差。我们使用走样（aliasing）这个术语来描述这种效果，因为这与数字信号处理中的混叠现象有关系。

由于帧缓存的离散特性，走样误差是由下面3个相关的问题引起的。第一，如果帧缓存的大小为 $n \times m$，那么它存储的像素数量是固定的，并且我们只能生成某种像素图案来近似表示一条线段，许多不同的连续线段可能使用相同的像素图案来近似表示。我们说所有这些线段走样为同一个像素序列。给定某个像素序列，我们无法确定该像素序列是由哪条线段生成的。第二，像素固定放置在均匀的栅格上，无论把像素放在何处，都不能把它们放在等间隔分布的其他栅格位置上。第三，像素具有固定的大小和形状。

初看起来，我们似乎无法解决上面这些问题。像 Bresenham 这一类光栅化算法是一种最佳算法，因为这些算法总是选择最佳逼近的像素集来近似表示线段和多边形。然而，如果显示器能够显示两种以上的颜色，那么还存在其他的解决方法。尽管数学上的直线是一维实体，它有长度而没有宽度，但是为了使光栅化后的直线是可见的，它必须有宽度。假定每个像素都显示为1个单位宽度的正方形，因此它在屏幕上占据了一个单位高度和一个单位宽度的方形区域。这里使用基本的帧缓存，它只能以一个像素或一个像素的倍数为处理单位[①]，我们还认为帧缓存中一条理想线段的宽度为一个像素，如图7.57所示。当然，我们无法绘制这样的直线，因为它并不是由方形像素序列组成的线段。我们可以把Bresenham算法看做是一种使用真实像素来近似表示一个像素宽的理想直线的光栅化方法。如果观察一个像素宽的理想直线，我们发现它有一部分覆盖了许多单个像素大小的方形区域。通过扫描转换算法，对于斜率小于1的直线上的每个 x 值，我们都要精确地选择一个相应的像素值。然而，如果根据像素在理想直线上所占面积的比例来设置像素的亮度，那么就会得到如图 7.58(b)所示的看起来更平滑的图像。这就是所谓的区域平均反走样技术（antialiasing by area averaging），其计算过程类似于多边形的裁剪。还有其他的反走样方法，以及专门用于多边形等其他图元的反走样算法。彩图 12 显示了彩图 5 中对象的一小块区域的走样和反走样的效果。

① 有些帧缓存通过使用多重采样技术允许对小于一个像素的单位进行处理。

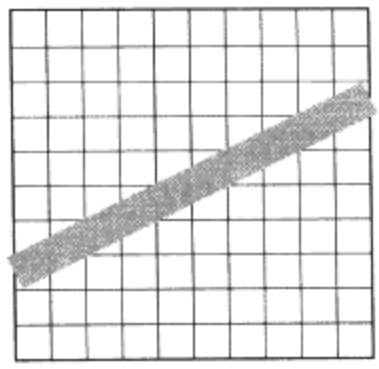


图 7.57　理想的光栅化直线

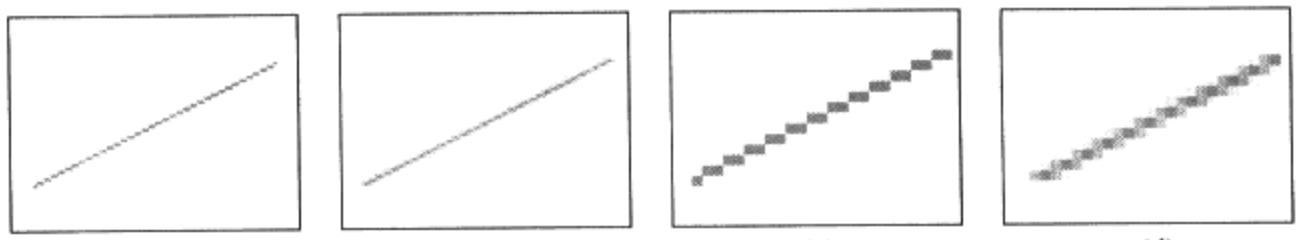


图 7.58　走样与反走样的线段：(a) 走样的线段；(b) 反走样的线段；(c) 放大的走样线段；(d) 放大的反走样线段

由于我们使用 *z*-buffer 这种简单的算法处理消隐问题，这会导致相关的走样问题。正如在 *z*-buffer算法中所描述的那样，某个像素的颜色由它所在的单个图元的明暗值决定。考虑如图7.59所示的像素，该像素由3个多边形所共享。如果这3个多边形的颜色互不相同，那么我们把离视点最近的那个多边形的颜色赋给该像素。如果把这3个三角形的颜色值基于它们在该像素中所占的面积进行加权平均后赋给该像素，那么可以得到更精确的图像。我们利用在第8章将要介绍的累积缓存就可以实现这种技术。

我们目前只讨论了一种类型的走样：空间域走样（spatial-domain aliasing）。当生成一系列图像（如动画）时，还必须考虑时间域走样（time-domain aliasing）。下面考虑在投影平面前方一个正在移动的小对象，该投影平面被分割成像素大小的栅格状，如图7.60所示。如果绘制过程从投影中心发射的光线通过每个像素的中心，并确定与光线相交的对象，那么有时光线与对象相交，而有时如果对象的投影非常小，则光线不会与对象相交。因此，当这个小对象移动时，观察者会看到该对象在屏幕上闪烁，这会使观察者产生不愉快的感觉。有很多种处理该问题的方法。例如，可以使用多条光线同时通过每个像素，这是光线跟踪算法常用的一种技术。所有反走样技术的一个共性是：使用反走样绘制所需的计算量远远大于关闭反走样绘制所需的计算量。在实际应用中，对于高分辨率图像，我们一般脱机进行反走样处理，并且只是当最终生成的图像需要时才进行反走样处理。

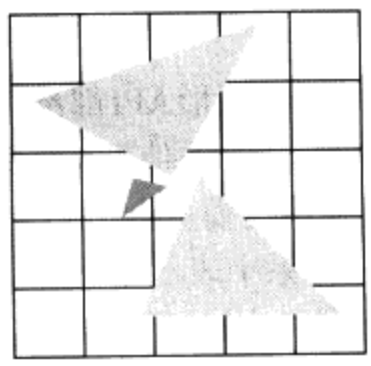


图 7.59　3个多边形共享一个像素

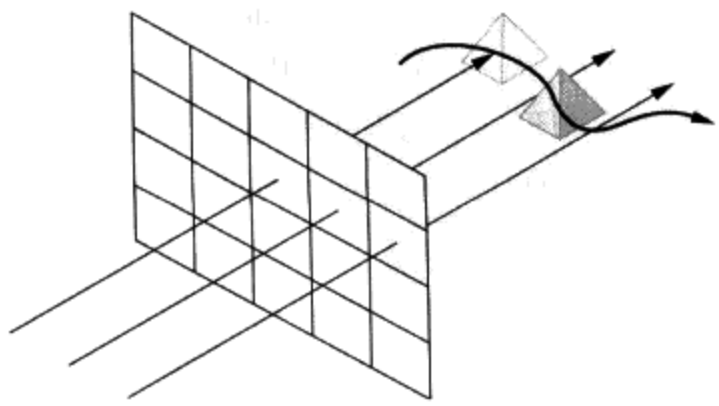


图 7.60　时间域走样

7.13　显示方面的问题

在大部分的交互式应用中，应用程序编程人员都不必关心帧缓存中的内容是如何显示出来的。从应用程序编程人员的观点来看，只要利用了双缓存，那么写缓存的过程与读缓存并显示的过程是相互独立的。为了使显示的图像不发生闪烁现象，图形显示硬件通常以60 ~ 85 Hz的刷新频率重新显示帧缓存中的当前内容，而应用程序编程人员只需关心程序的执行速度以及帧缓存的写入速度是否足够快。正如第3章讲到的，即使要绘制的图元没有以我们所期望的速度通过图形绘制系统的处理，我们仍可以利用双缓存技术来得到平滑显示的图像。

许多其他问题也会影响图像的显示质量，这些问题通常会使用户对程序的输出感到不满意。例如，虽然两个显示器标称具有相同的分辨率，但是它们可能具有大小不同的像素（见习题7.22和习题7.23）。

也许导致上述显示问题的最主要根源就是显示器的物理特性：如显示器可显示的颜色范围，以及如何把软件定义的颜色映射为显示系统所支持的基色。不同显示器的色域具有很大的区别。此外，因为不同显示系统的基色存在区别，所以即使两个不同显示器显示的可见颜色相同，但是从图形绘制系统送入这两个显示器的基色值却可能不同。另外，程序中定义的亮度值与显示亮度之间的映射关系是非线性的。

OpenGL没有直接解决这些问题，这是因为OpenGL使用的RGB颜色值独立于任何显示特性。此外，因为RGB基色的取值范围限制在0.0 ~ 1.0，所以很难说明人的视觉系统可察觉的整个颜色和亮度范围。然而，如果把第2章讨论的有关颜色和人的视觉系统方面的内容加以扩展，则可以获得一些对OpenGL中的颜色的额外控制。

7.13.1　颜色系统

基于人眼视觉的三色理论，我们对颜色的一个基本假设是：用于确定每个像素颜色的三个基色值对应于第2章介绍的三刺激值。因此，某一特定的颜色对应于如图7.61所示的颜色立方体中的一个点，该颜色可以形式化地表示为：

$$C = T_1\mathbf{R} + T_2\mathbf{G} + T_3\mathbf{B}$$

然而，不同的RGB颜色系统之间存在明显的区别。例如，假定在OpenGL中使用三元组（0.8，0.6，0.0）表示某种黄色。如果我们使用这些颜色值分别在CRT显示器和喷墨打印机上显示同一个多边形，那么尽管在这两种情况下，红色分量占最大值的80%，绿色分量占最大值的60%，且都没有蓝色成分，但是我们看到的多边形颜色可能不尽相同。之所以出现这种情况，是因为CRT显示器屏幕上的磷光粉和打印机的墨粉具有不同的颜色分布模型。因此，在这两种不同的显示设备中，不但它们可显示的颜色范围（即色域，color gamuts）不同，而且一个特定的RGB颜色值在这两种设备上显示出来的颜色也不相同。

由于在图形学领域我们强调设备无关的图形绘制系统，因此，大部分的图形API都没有考虑显示特性之间的真正差别。幸运的是，从有关的色度学文献中可以找到我们所需的信息。许多常见的颜色系统都存在相应的标准。例如，CRT显示器使用的是美国国家电视系统委员会（NTSC）制定的RGB颜色系统。我们可以把不同颜色系统的区别看成是在不同的坐标系中表示三刺激值。如果$\mathbf{C}_1 = [R_1, G_1, B_1]^T$和$\mathbf{C}_2 = [R_2, G_2, B_2]^T$是同一种颜色在两种不同的颜色系统中的表示，那么它们之间存在一个3 × 3的颜色转换矩阵$\mathbf{M}$：

$$\mathbf{C}_2 = \mathbf{M}\mathbf{C}_1$$

无论是根据色度学的有关文献还是通过实验来确定上式的颜色转换矩阵，我们都可以利用该矩阵使得在不同的输出设备上得到相似的显示效果。

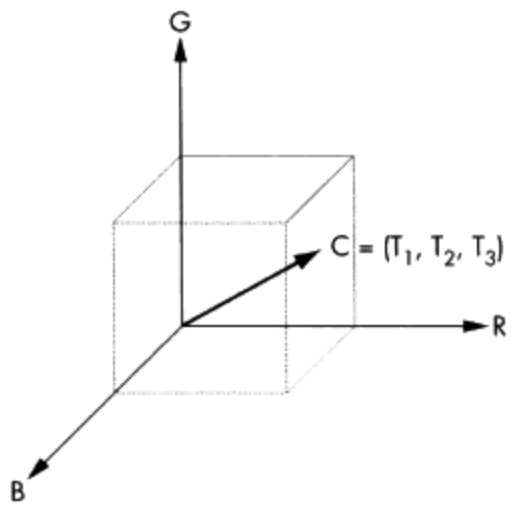


图 7.61　颜色立方体

即使使用上述方法，仍存在许多潜在的问题。第一，两种颜色系统的色域可能并不相同。因此，即使对三刺激值进行转换后，其中一个颜色系统的色域可能并不包括某种颜色。第二，印刷和图形艺术业一般采用四色减色系统（CMYK），它增加了黑色（K）作为第四个基色。RGB 和 CMYK 这两种颜色系统之间的转换通常需要大量的上下文信息，如我们正在显示的内容是文本还是几何实体。第三，我们使用的线性颜色理论本身具有一定的局限性。在颜色立方体内部任意两种颜色之间的距离并不是我们在视觉上感知到的这两种颜色的距离。例如，人的视觉对蓝色的变化特别敏感。像 YUV 和 CIE（国际照明委员会）的 Lab 颜色系统就可以用来解决这类问题。

大部分的颜色系统都是基于实际系统的基色，如 RGB 颜色系统和 CMYK 颜色系统分别基于 CRT显示器使用的磷光粉的颜色和打印机使用的墨粉的颜色，但是没有一种颜色系统能够产生人眼可见的所有颜色。大部分的图形标准都是基于理论上的三基色系统，即 XYZ 颜色系统，这里的 Y 基色代表颜色的亮度。在 XYZ 颜色系统中，所有的颜色都可以使用正的三刺激值来表示。我们使用 3 × 3 的矩阵把在 XYZ 颜色系统中表示的颜色转换成在标准的颜色系统中表示的颜色。从事颜色理论研究的专家通常更喜欢使用色度坐标（chromaticity coordinates）而不是三刺激值。某种颜色的色度是指该颜色的颜色分量分别在三基色中所占的比例成分。因此，如果三刺激值分别是 T_1、T_2 和 T_3，那么对于某个特定的 RGB 颜色，它的色度坐标是：

$$t_1 = \frac{T_1}{T_1 + T_2 + T_3}$$

$$t_2 = \frac{T_2}{T_1 + T_2 + T_3}$$

$$t_3 = \frac{T_3}{T_1 + T_2 + T_3}$$

把上面三个等式相加，得到：

$$t_1 + t_2 + t_3 = 1$$

因此我们可以工作在 t_1 和 t_2 所在的二维空间，只有需要的时候才通过上式计算 t_3 的值。色度坐标丢失了一个与颜色强度有关的值，即 $T_1 + T_2 + T_3$，而原三刺激值包含了这个信息。当处理颜色系统中的有关问题时，$T_1 + T_2 + T_3$ 这个值通常对颜色的生成或不同颜色系统之间颜色的匹配等问题并不是十分重要。

因为每个颜色的分量必须是非负数，所以色度值的范围限制在：

$$1 \geqslant t_i \geqslant 0$$

所有生成的颜色必须位于如图7.62所示的三角形内部。图7.63显示了XYZ颜色系统的色度三角形和可见光光谱颜色的曲线表示。对于XYZ颜色系统，该曲线必须位于色度三角形的内部。图7.63还显示了典型的彩色打印机或CRT显示器可显示的颜色范围（位于x，y色度坐标系中）。如果比较图中的色域三角形和可见光的光谱颜色曲线，就会发现位于该曲线内部且位于物理显示器的色域三角形之外的颜色是不能在物理设备上显示出来的。

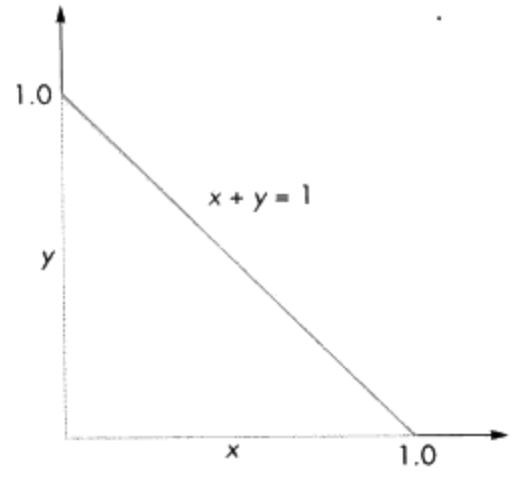


图7.62　在色度坐标系中可产生的颜色构成的颜色三角形

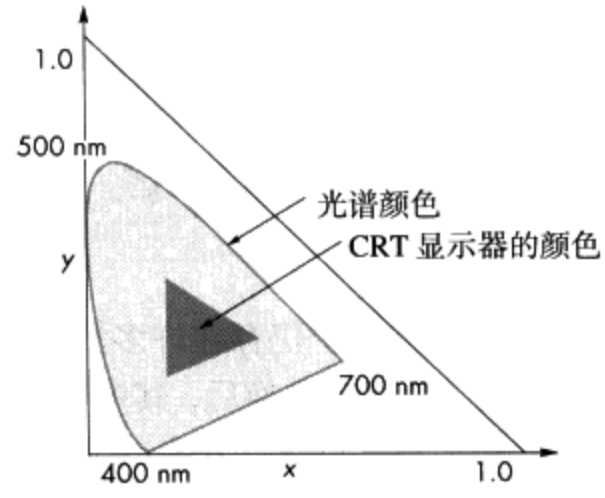


图7.63　可见光的光谱颜色和某种显示设备的色域

前面介绍的颜色系统存在一个缺陷，即对于RGB颜色，它是根据颜色本身产生和度量的方式来确定的，而不是根据人的眼睛对颜色的感知方式。当看到某种特定的颜色时，我们并不是使用三基色来描述这种颜色，而是基于颜色的其他属性，如使用颜色的名字和颜色的亮度等来描述颜色。艺术家和一些显示设备制造商通常使用色度－亮度－饱和度颜色系统（即HLS颜色系统）。色度（hue）是指颜色的名字，如红色、黄色、金色等。亮度（lightness）是指颜色看起来的明亮程度。我们可以使用饱和度（saturation）这种颜色属性把某种纯色和与其具有相同色调的颜色混入白色后得到的柔和色区分开。我们可以把这三个颜色属性与典型的RGB颜色联系起来看，如图7.64(a)所示。对于颜色立方体内部的某个特定点的颜色，它的亮度值表示该点离原点（黑色）的距离。如果注意到位于颜色立方体主对角线上的所有颜色（从黑色变化到白色）都是灰色且完全不饱和，那么就知道饱和度表示某个特定颜色离主对角线的距离。最后，色调通常表示为颜色矢量在颜色立方体内的指向。我们通常使用一个颜色圆锥体来描述HLS颜色，如图7.64(b)所示，或者使用一个底部重合的双圆锥体来描述HLS颜色。从我们自己的观点来看，可以把HLS颜色系统看做是RGB颜色系统的极坐标表示形式。

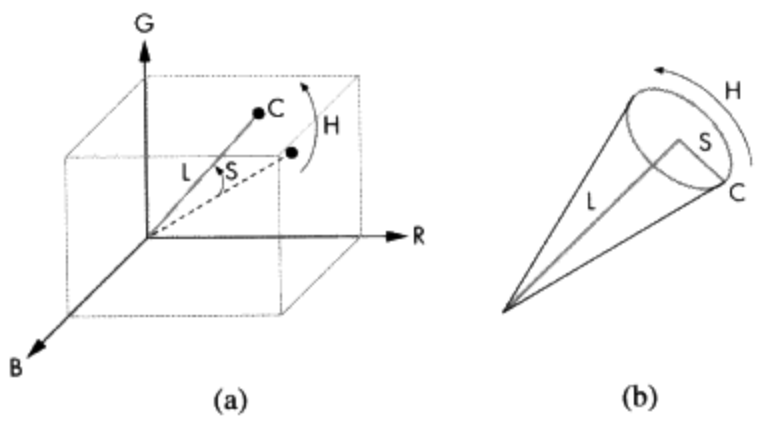


图7.64　色度－亮度－饱和度（HLS）颜色系统：(a) 使用RGB颜色立方体描述HLS颜色；(b) 使用单个圆锥体描述HLS颜色

7.13.2　颜色矩阵

OpenGL提供了一个颜色矩阵，它是最新图形硬件所支持的图像处理功能子集的一部分。该矩阵简化了不同颜色系统之间的转换。颜色矩阵是一个4 × 4的矩阵，按列存储的顺序存放在一个具有16个元素的数组中。颜色矩阵作用在RGBA颜色值上，可以看成是绘制流水线的一部分，利用下面的矩阵乘法可以把某个$r'g'b'a'$颜色值变换为另一种新的颜色值：

$$\begin{bmatrix} r' \\ g' \\ b' \\ a' \end{bmatrix} = \mathbf{C} \begin{bmatrix} r \\ g \\ b \\ a \end{bmatrix}$$

因此，如果我们处理一个$A = 1$的不透明的表面，那么可以使用下面的矩阵把颜色从加色表示形式转换为减色表示形式：

$$C = \begin{bmatrix} -1 & 0 & 0 & 1 \\ 0 & -1 & 0 & 1 \\ 0 & 0 & -1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

与其他OpenGL矩阵的处理方式类似，可以按下面的方式把当前矩阵模式设置为颜色矩阵，然后在应用程序中生成该矩阵：

```
glMatrixMode(GL_COLOR);
```

7.13.3　γ校正

第2章曾把亮度定义为人眼感知到的光强度，并发现人的视觉系统按对数关系感知光强度，如图7.65所示。根据人的视觉系统所具有的这种特性，我们可以得到这样一个结论：如果希望亮度看起来按等间隔的步长递增，那么赋给像素的光强值应该按指数形式递增。我们可以根据显示设备所能产生的最小和最大光强值通过计算得到亮度变化的步长。

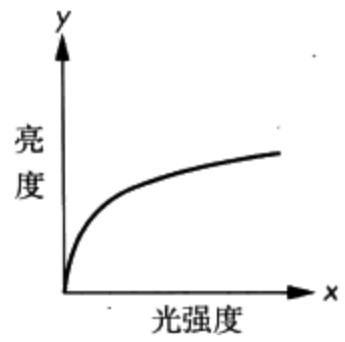


图7.65　按对数关系变化的亮度曲线

此外，CRT显示器的光强度I和电压V之间的关系为：

$$I \propto V^{\gamma}$$

或者

$$\log I = c_0 + \gamma \log V$$

其中，常量γ和c_0是某种特定的CRT显示器所固有的特性。上面的两个公式表明：对于帧缓存中的同一个颜色值，两个不同的显示器显示的亮度可能完全不同。解决该问题的一个办法是利用显示器的查找表，该查找表中的值可以根据显示器的特性而调整，即γ校正（gamma correction）。

包括CRT显示器、平板显示器和数字投影仪在内的许多显示设备还存在另外一个问题。对于这些显示设备来说，即使没有任何信号输入，它们也无法显示真正的黑色（true black）。这个最小的光强显示值称为暗场（dark field）值，这个暗场问题是一个非常棘手的问题，尤其是使用CRT显示器来显示图像时。显示器的对比度是指最大亮度与最小亮度的比值。采用最新的显示技术能使显示器达到数千的对比度。

7.13.4　抖动输出技术和半色调输出技术

我们根据两个属性指定一个颜色缓存，即空间分辨率（像素的数量）和精度（显示的颜色数量）。如果把这两个独立的属性看成是固定不变的，那么可以认为高分辨率的黑白激光打印机只能显示1比特的像素。这个结论似乎也适用于任何黑白媒介，如书籍等，即它们都无法显示多级灰度图像。我们根据实际使用经验知道情况并不是这样的，解决的方法是以空间分辨率为代价换取灰度级或彩色的精度。印刷业中使用的半色调（halftoning）输出技术是指利用摄影术中的方法来模拟灰度级，采用的方法是创建大小变化的黑色圆点图案。人的视觉系统具有把小圆点聚集起来的集成特性，因此人眼看到的并不是孤立的小圆点，而是一个小区域内的像素的平均灰度，它正比于其中的黑色像素与小区域内所有像素个数的比值。

因为已显示像素的大小和位置是固定的，所以数字化的半色调图案不尽相同。下面我们分析一个4 × 4的1比特像素数组，如图7.66所示。如果从远处看这个图案，那么看到的不是孤立的像素，而是一个由黑色像素的数量所确定的灰度级。对于这个4 × 4的像素组，尽管有2^{16}种不同的黑白像素组合模式，但是只有17种可能的灰度，对应于该像素数组中黑色像素的个数（0 ~ 16个）。目前有许多用于生成半色调图案或抖动（dither）图案的算法。最简单的算法选择17种半色调图案（如本例），并利用它们显示具有17个灰度级而不是2个灰度级的图像，尽管要以降低4倍的空间分辨率作为代价。

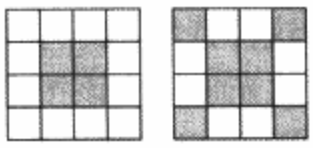


图7.66　数字化的半色调图案

由于简单的算法总是使用相同的像素数组来模拟灰度级，所以当显示任何规则的图像时，就会产生拍频波形图案或波纹状图案。每当我们对这种规则的现象进行成像时，就会产生这样的图案，这是因为我们看到的是频谱值的和值与差值。这种效果与第8章将要讨论的走样问题有着密切的关系。

很多抖动输出技术只是简单地对像素的亮度值或每个颜色分量的最低位进行随机化处理。而更高级的抖动输出算法采用了随机化过程来确保生成的图案具有正确的平均特性，同时又避免了重复性，从而不会产生波纹状效果（见习题7.26）。

半色调（抖动）技术也通常用来处理彩色图像的输出，尤其是硬拷贝输出设备，如喷墨打印机，它要么产生所有的颜色，要么只产生黑白两种颜色。对每个颜色分量进行抖动处理可以产生更多视觉上可见的色彩。OpenGL支持以这样的方式显示图像，并允许用户开启抖动输出功能（glEnable(GL_DITHER)）。利用颜色的抖动输出技术可以使彩色显示器输出的图像更平滑，在默认状态下，OpenGL的抖动功能处于开启状态。因为抖动输出是一种非常有效的技术，只需给每个颜色分量分配有限的位数就可以输出效果不错的图像，所以可以允许帧缓存具有有限大小的内存容量。在许多应用中，人们需要利用OpenGL的查询函数glGetIntegerv来获取每个颜色分量占用的位数，当使用第8章将要介绍的某些技术从帧缓存中读取像素时，这些获取的信息就非常重要了。如果开

启了抖动功能并从帧缓存中读取像素，那么对于写入的具有相同RGB颜色值的像素，当读取这些像素时，返回的RGB颜色值却不同。如果这些差异会严重影响图像的输出效果，那么在从帧缓存中读取像素之前应该关闭抖动功能。

小结和注释

本章对图形绘制系统的整个实现过程进行了概述，并重点讨论了其中的一些主要算法。不管一个图形绘制系统的实现细节如何，无论它的任务主要用硬件实现还是软件实现，无论采用的是专用的图形工作站还是简单的图形终端，也无论采用何种图形API，我们都必须实现相同的任务。这些任务包括几何变换、裁剪和光栅化的实现。图形硬件、软件和图形API之间的关系是一个令人感兴趣的话题。

几何处理引擎是许多Silicon图形工作站的主要组成部分，它是一个超大规模集成电路芯片，通过硬件绘制流水线执行几何变换和裁剪等任务。GL（Graphic Library）是OpenGL的前身，是为用户使用Silicon图形工作站而开发的API。许多OpenGL的文献也是基于绘制流水线的方法。然而读者要牢记，OpenGL只是一个图形API，它并不提供底层的实现细节。原则上，通过OpenGL程序生成的图像也可以使用光线跟踪法得到。我们之所以强调图形的绘制流水线结构主要是基于两个原因。第一，绘制流水线结构有助于应用程序编程人员理解图像生成的过程。第二，在目前，基于绘制流水线的观点可以得到有效的硬件实现和软件实现。

z-buffer算法是说明图形硬件和软件之间关系的一个很好的例子。人们在15年前就使用了许多隐藏面消除算法，*z*-buffer算法只是其中的一个。随着高密度廉价存储器的出现，*z*-buffer算法已成为隐藏面消除算法中占支配地位的算法。

还有一个相关的例子就是图形工作站的绘制结构，在过去几年，其使用的专用图形芯片取得了巨大的发展，不仅图形绘制系统性能的增长速度远远超过了摩尔定律，而且图形处理器还增加了许多新的特性。正如第9章将要讲到的，新的可编程结构为人们提供了很多新的机会。

那么，图形绘制系统将来会朝什么方向发展呢？当然，图形绘制系统的运行速度会变得越来越快，其价格会变得越来越便宜。相比于其他的因素，图形硬件的发展更有可能决定未来图形绘制系统的结构。目前，游戏业驱动了图形硬件的发展。现在只要花不到200美元就可以买到一个图形卡，其性能远超过几年前价值超过10万美元的图形工作站。为了满足计算机游戏业的需求，这些图形卡的特性和性能都做了优化处理。因此，前面介绍的各种图形功能并没有得到同样的性能加速。此外，新的硬件特性出现的速度远快于它们成为标准图形API的一部分的速度。然而，图形处理器运行速度的快速增长迫切要求图形学和科学界的研究人员提出新的算法，以解决那些到目前为止一直使用传统的绘制结构解决的问题。

在软件方面，最新图形硬件的低成本和高速度使得软件开发人员开发的绘制软件允许用户在绘制时间和绘制质量之间取得平衡。因此，用户可以把一些光线跟踪的对象增加到场景中，增加的数量取决于用户愿意等待的绘制时间。标准图形API的未来并不明朗。一方面，科学界的用户更喜欢稳定的图形API（如OpenGL），因此基于OpenGL图形API开发的应用程序具有更长的生命周期；另一方面，用户希望使用一些新的硬件特性，而所有图形绘制系统都不支持这些新特性。

目前许多处在研究阶段的高级绘制结构都使用了大规模并行机制。如何有效地使用并行机制解决计算机图形学中的问题，仍然是一个值得讨论的问题。前面介绍的两种绘制方法，即基于对象空间的方法和基于图像空间的方法，导致了用来开发并行绘制器的两种完全不同的方法，第13章将进一步讨论这两种方法。

本章只是初步探讨了图形绘制流水线的实现过程。有关图形绘制流水线每一阶段的算法实现都有大量的文献可查阅，下面的建议阅读资料有助于读者进一步深入研究这方面的问题。

建议阅读资料

相比于本教材，Rogers[Rog98]以及Foley及其同事[Fol90]的著作介绍了更多的有关图形绘制流水线的算法。读者也可以阅读Graphic Gems系列教材[Gra90, Gra91, Gra92, Gra94, Gra95]。像Möller和Haines[Mol02]以及Eberly[Ebe01]等最近几年的著作介绍了图形硬件的最新进展情况。

Cohen-Sutherland裁剪算法[Sut63]是计算机图形学的早期算法，Bresenham直线光栅化算法[Bre63，Bre87]最初用于控制笔式绘图仪的操作，也是一个早期的图形学算法。有关Liang-Barsky直线段裁剪算法和Sutherland-Hogman多边形裁剪算法可以参阅文献[Lia84]和[Sut74a]。

z-buffer算法最初由Catmull[Cat75]提出。Sutherland的文献[Sut74b]对隐藏面消除的各种方法进行了综述。

本章并没有介绍有关图形硬件的细节问题，这并不意味着图形硬件方面的内容很简单或让人感到乏味。现代图形处理器绘制图形实体的速度取决于精巧的硬件设计[Cla82，Ake88，Ake93]。Molnar和Fuchs在文献[Fol90]中讨论了图形硬件的各种设计方法。

Pratt在文献[Pra78]中给出了各种颜色系统之间的转换矩阵。Jarvis的文献[Jar76]和Knuth的文献[Knu87]讨论了半色调和抖动输出技术。

习题

7.1　考虑下面两条用参数方程表示的线段：

$$\mathbf{p}(\alpha) = (1-\alpha)\mathbf{p}_1 + \alpha\mathbf{p}_2$$
$$\mathbf{q}(\beta) = (1-\beta)\mathbf{q}_1 + \beta\mathbf{q}_2$$

请设计一个用于判断这两条线段是否相交的算法，如果相交，需要计算交点。

7.2　对习题7.1的算法进行扩展，设计一个用于判断两个多边形是否相交的算法。

7.3　证明用一个凸对象裁剪另一个凸对象的结果最多也是一个凸对象。

7.4　如何使基于图像空间的绘制方法和基于对象空间的绘制方法能够并行实现？

7.5　因为可以使用齐次坐标表示多边形的法向量和顶点坐标，所以可以对这两者进行模－视变换操作。证明法向量经过模－视变换后不会保持不变。

7.6　推导视口变换矩阵。要求使用用来表示二维仿射变换的三维缩放变换矩阵和三维平移变换矩阵来表示这个视口变换矩阵。

7.7　在光栅图形显示系统出现之前，图形显示系统只能显示直线。编程人员利用隐藏线消除技术来产生三维图像。目前有许多图形API可用来绘制仅仅由直线构成的线框图，且消除了线框图中由隐藏线构成的隐藏面。这种隐藏线消除问题与我们介绍的多边形隐藏面消除有何不同？设计隐藏线消除算法，对由平面多边形的边线构成的对象进行消隐。

7.8　我们经常要显示函数$y = f(x, z)$对应的曲面，采用的方法是等间隔地取x和z，然后通过计算一组$\{f(x_i, z_j)\}$值来显示一个矩形网格。因为曲面的一部分在视点方向可能被其他部分遮挡，所以需要进行隐藏面消除处理。设计两个算法，分别利用隐藏面消除和隐藏线消除方法显示这样的网格。

7.9　尽管我们认为基于图像空间的隐藏面消除算法的时间复杂度与多边形的数量成正比，但是对算法的性能研究表明其时间复杂度几乎为常量级。请解释这个结论。

7.10　考虑一个仅仅由三维实心多面体构成的场景，能否设计一个基于对象空间的隐藏面消除算法？如果已知所有的多面体都是凸多面体，这对设计算法有多大帮助？

7.11　我们可以把基于对象空间的隐藏面消除算法类比成排序算法，然而我们知道前者的时间复杂度为$O(k^2)$。我们还知道只有在最坏情况下，排序算法才有这么差的性能，而大部分情况下其时间复杂度为$O(k \log k)$。能否因此认为基于对象空间的隐藏面消除算法也有同样的时间复杂度？请对回答给出解释。

7.12　设计一个算法，用来判断一个平面多边形是否完全位于另一个平面多边形的一侧。

7.13　基于图像空间的隐藏面消除算法与光线跟踪有什么区别？可否把光线跟踪方法用于解决隐藏面消除问题？这种方法的优缺点是什么？

7.14　编写一个程序，用来确定基于Bresenham算法的光栅化线段上某个像素的位置。注意，程序要考虑线段斜率的所有可能情况以及线段两个端点所有可能的位置情况。判定变量的初始值是多少？

7.15　可以把 Bresenham 直线光栅化算法扩展为对圆的光栅化。为了证实 Bresenham 直线光栅化算法的这种可扩展性，我们考虑一个圆点位于坐标原点的圆。必须用算法生成圆周的哪部分？圆周的哪些部分可以通过对称性得到？能否找到圆周的某一部分，使得当通过扫描线转换算法得到该部分上的一个点时，可以减少用于确定下一个像素的候选像素的数目？

7.16　如何使用种子填充算法生成一个迷宫的图形，就像在习题 2.7 中生成的图形一样。

7.17　假定想通过下面的方法把种子填充算法扩展到任意封闭的曲线，即先对该曲线进行扫描转换，然后使用与多边形填充相同的算法填充该封闭的曲线。如果使用这种方法，会出现什么问题？

7.18　考虑多边形上位于顶点(x_1, y_1)和(x_2, y_2)之间的一条边线。设计一个有效的算法用来计算所有扫描线与该边线的交点。假定工作在屏幕窗口坐标系。

7.19　对于多边形填充算法来说，多边形的垂直边和水平边可能会引起问题。对于本章已介绍的多边形填充算法，如何处理这些情形？

7.20　在二维图形绘制系统中，如果两个多边形重叠，那么通过给每个多边形赋一个优先级属性，就可以确保所有的二维图形绘制系统以相同的顺序绘制它们。系统按逆优先级的顺序绘制多边形，也就是说，最后绘制最高优先级的多边形。为了考虑多边形的优先级，如何修改多边形填充算法？

7.21　光线跟踪方法中使用的标准反走样技术不仅向每个像素的中心投射光线，而且还向像素的 4 个角投射光线。相对于只向像素的中心投射光线，这种基于反走样技术的光线跟踪方法增加了哪些计算量？

7.22　尽管一个理想的像素是边长为 1 个单位的正方形，但是大部分CRT显示系统产生圆形像素，可以近似地表示为具有均匀亮度的圆。如果一个完全点亮的单位正方形的光强度为 1，而一个完全没有点亮的单位正方形的光强度为 0，那么圆形像素的光强度如何随着圆的半径变化而变化？

7.23　考虑一个显示圆形像素的二值显示器（只显示两种颜色的显示器）。是用大圆作为前景色像素好，还是用小圆作为前景色像素好？请对回答给出解释。

7.24 为什么有时把CRT显示器中电子束的散焦现象称为“穷人的反走样（the poor person's antialiasing）”？

7.25 假定某个单色显示器的最小光强度输出为I_{min}（因为CRT显示器不可能产生全黑的输出），它的最大光强度输出为I_{max}。假定按对数关系感知光强度，应该如何选择光强度的k个级别，从而使得人眼感知的亮度是等间隔地变亮？

7.26 根据下面的思想设计一个半色调算法。假定灰度级在0.0到1.0之间变化，并且有一个随机数生成器，它能产生均匀分布在该区间的随机数。如果选择某个灰度级g，那么在生成的随机数中，有g%的随机数小于g。

7.27 因为观察者能够察觉图像中相邻明暗值之间的差异，所以对于只能显示少量颜色或灰度级的显示器，它生成的图像可能具有轮廓效应。避免出现这种视觉效应的一种方法是给像素值增加少量的噪声（抖动）。为什么这是一种行之有效的方法？应该给像素值增加多大的噪声？能否因此认为：加入噪声后得到的退化图像的质量要高于原始图像？

7.28 已知一个二维多边形的顶点$\{x_i, y_i\}$，说明为什么可以使用公式$\frac{1}{2}\sum_i(y_{i+1}+y_i)(x_{i+1}-x_i)$计算该多边形的面积。如果多边形的面积为负数，这表示什么意思？提示：已知x轴上对应的x值，考虑由两个连续顶点序列形成的梯形面积。

第8章　离散技术

多年来，计算机图形学只研究如何处理几何对象，如线段、多边形和多面体。光栅显示系统已经出现了30多年，但直到最近，应用程序编程人员才能间接地访问帧缓存中的像素内容。虽然线段和多边形被光栅化为存储在帧缓存中的像素，但是还没有对每个独立像素进行读/写操作的API函数可供应用程序编程人员使用。过去十几年提出的许多令人激动的方法都依赖于帧缓存中各种缓存的相互作用。当可以使用API函数读/写这些离散缓存时，许多相关的技术才成为可能，纹理映射、反走样、图像合成和alpha融合只是其中的几个技术。本章将介绍这些技术，重点是那些被OpenGL或类似的API所支持的技术。

我们首先详细介绍帧缓存的相关概念，并介绍如何对帧缓存进行读/写操作，还将学习如何处理数字图像的像素阵列。然后介绍纹理映射技术，在绘制场景的过程中可以使用这些纹理映射技术，它使绘制的对象表面看起来非常复杂，哪怕这个表面只有一个多边形。所有这些技术都使用了像素阵列，我们可以使用这些技术来增强第7章介绍的绘制过程，从而使绘制的图像具有虚幻的视觉效果。之后将介绍OpenGL API所支持的其他类型的缓存，以及如何把这些缓存应用到新的应用中，尤其是将会详细介绍图像的组合或合成技术。本章要用到RBGA颜色模式的第4个“颜色”通道，读者将看到，使用这个颜色通道对图像进行融合可以生成透明的效果。最后介绍走样问题，只要我们处理的是离散对象，就会产生走样。

8.1　缓存

前面已经使用了两类缓存：颜色缓存和深度缓存。本节将介绍其他类型的缓存。所有这些缓存的共性是：它们本质上都是离散的，即它们的空间分辨率和深度分辨率都是有限的。如图8.1所示，我们把一个（二维）[①]缓存定义为一个大小为 $n \times m$ 且深度为 k 个二进制位的内存区域。

正如所看到的那样，OpenGL把帧缓存定义为由各种不同类型的缓存组成，包括前端和后端颜色缓存、深度缓存以及本章将要介绍的其他缓存。因此，帧缓存空间的某个位置对应的 k 比特位可能为下面的任意一种情况：调色板的索引值（例如，可能是占用一个字节的指针）、32位的像素颜色值、表示像素深度的整数值，甚至是浮点数。图8.2显示了OpenGL帧缓存及其组成部分。如果考虑整个帧缓存，那么帧缓存的分辨率 $n \times m$ 正好对应显示器的空间分辨率，而帧缓存的深度（即 k 的值）可以超过几百位，虽然对于我们目前见到的最简单的情况，前端和后端颜色缓存共需要使用64位，深度缓存需要32位。对于某个具体的缓存，其数值精确性或精度（precision）由它的深度决定。因此，如果某个帧缓存的前端颜色缓存和后端颜色缓存分别使用了32位，那么RGBA的每个颜色分量的精度为8个二进制位。

当处理帧缓存时，一般每次只处理其中的一个缓存。因此，后面用缓存这个术语来表示帧缓存中的某个特定缓存。帧缓存中每个缓存的空间分辨率为 $n \times m$，深度为 k 位。然而，不同类型的缓存，k 的取值不同。对于一个颜色缓存，它的 k 值由系统能显示的颜色数量决定，对于RGB显示模式，通常是24位，而对于RGBA显示模式则是32位。对于深度缓存，它的 k 值由系统所能支持的

① 我们还有一维、三维和四维缓存。

深度精度决定，通常是32位浮点数或整数。我们使用位平面（bitplane）表示缓存中k个大小为$n \times m$的平面中的任意一个，使用像素（pixel）表示空间某个特定位置对应的所有k个二进制位。根据这个定义，一个像素可能是一个字节，一个整数，甚至是一个浮点数，这取决于我们使用了帧缓存中的哪个缓存，以及数据在缓存中的存储方式。

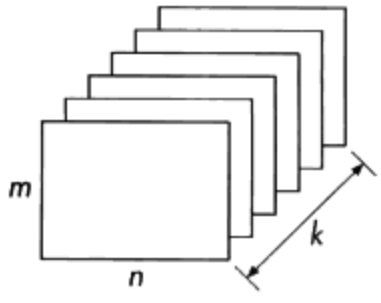


图8.1　缓存

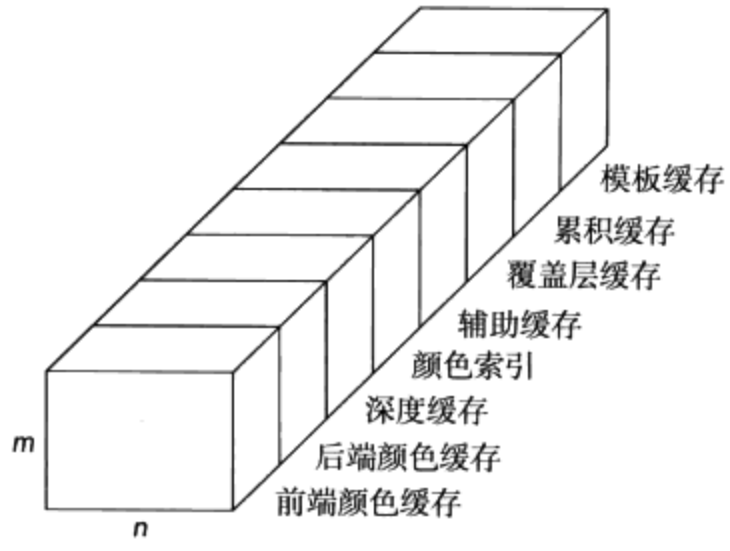


图8.2　OpenGL的帧缓存

因为帧缓存的实现细节对于程序员来说就是一个黑盒子，所以应用程序编程人员一般并不知道帧缓存中的信息是如何存储的。因此，应用程序通过OpenGL函数把信息传送（写入或绘制）到帧缓存，或者从帧缓存中获取（读取）信息。当应用程序读/写像素时，数据不仅要在普通的处理器内存与图形卡里的图形内存（帧缓存）之间传输，而且还要重新调整这些数据的格式，使之与帧缓存匹配。因此，我们通常认为的数字图像，例如JPEG-、GIF-或TIFF-格式的图像，它们只是位于处理过程中的应用层。应用程序编程人员不仅要苦于对这些特定的图像格式进行翻译，这样才能通过OpenGL函数把图像数据传送到帧缓存中，而且还必须考虑图像数据在处理器内存与帧缓存之间的传输时间。如果应用程序编程人员还知道数据在任意类型的缓存中的内部存储格式，则通常能编写出执行效率更高的程序。

8.2　数字图像

在介绍图形系统如何通过像素或位操作处理数字图像之前，我们首先讨论数字图像的含义[①]。我们通常要在程序中处理以像素阵列表示的图像。这些图像的大小和格式不尽相同，具体取决于要处理的图像类型。例如，如果要处理RGB图像，通常用一个字节表示颜色的每个分量，其取值范围为0～255。因此，可以在应用程序中按下面的形式定义一幅512 × 512的图像：

```
GLubyte myimage[512][512][3];
```

如果处理的是单色图像或灰度图像，那么每个像素的值表示一个从0（黑）到255（白）的灰度级，因此使用下面的形式定义灰度图像：

```
GLubyte myimage[512][512];
```

如果处理的是颜色索引模式的图像，那么每个像素是一个指向颜色表（调色板）的索引，因此我们使用unsigned char类型存储图像阵列，并使用3个调色板分别存储颜色分量：

① 许多参考书经常使用术语图像（image）而不是术语数字图像（digital image）。图像这个术语可能会与计算机图形学中的图像（image）混淆，后者是指计算机成像的结果，即给定几何对象和虚拟照相机，并通过投影处理来生成我们称为图像的过程。在本章中，根据清晰的上下文，应该不会引起混淆。

```
GLubyte myimage[512][512], red[256], green[256], blue[256];
```

因此在这个例子中，如果给定某个索引值，那么红色的颜色值为red[myimage[i][j]]。生成数字图像的一种方法是在应用程序中编写代码。例如，假定要生成一幅512 × 512的图像，该图像由一个8 × 8的红黑交错排列的正方形棋盘构成，就像我们在游戏中见到的那种棋盘。可以使用下面的程序代码生成这样的棋盘图像：

```
GLubyte check[512][512][3];
int i,j;
for (i=0; i<512; i++)
   for(j=0; j<512; j++)
   {
   for(k=0;k<3;k++)
      if((8*i+j/64)%64) check[i][j][0]=255;
         else check[i][j][k]=0;
   }
```

通常情况下，这种通过编写代码来生成图像的方法只局限于那些包含规则图案的图像。更常用的方法是：可以直接从数据中获取图像。例如，如果从实验或仿真中获取了一组实数，那么可以把这组数据缩放到0 ~ 255这个范围，然后再把这些数据转换成无符号字符类型，从而得到一幅灰度图像。

还有一种获取图像的方法，这种方法由于Internet的影响而变得非常盛行。这种方法通过扫描像照片这样的连续图像，或直接使用数码照相机获取数字图像。每种图像都使用众多标准图像格式的某一种来进行存储。GIF、TIFF、PS、EPS和JPEG是目前最流行的图像格式，在这些图像格式中，有的按某种顺序对图像数据直接编码，有的采用无损压缩编码，有的采用有损压缩编码。每种图像格式都是基于某个特定的应用需求而产生的。例如，PostScript（PS格式）图像由PostScript语言定义，主要用于控制打印机。PostScript图像能精确地把图像数据（RGB颜色值或亮度值）编码成由7个二进制位形成的ASCII字符集，因此大多数类型的打印机及其他一些大型设备都识别PostScript图像。封装的PostScript（EPS）图像与PostScript图像有类似的格式，但是它还包含额外的信息，这些信息有助于预览图像。GIF图像是颜色索引图像，因此它存储了一个调色板和图像的索引阵列。

TIFF图像有两种形式。在第一种形式中，所有的图像数据被直接编码，这类TIFF图像的头部信息描述了图像数据的组织形式。在第二种形式中，图像数据通过编码而被压缩存储。由于大部分图像包含许多冗余的数据，所以图像压缩是可行的。例如，很多图像大片区域上的颜色或亮度只发生很少的变化。通过算法可以消除这些冗余数据，因此可以得到原始图像的压缩版本，它只需要少量的存储空间。压缩形式的TIFF图像采用了Lempel-Ziv算法，该算法是一种性能优良的无损压缩算法，它能压缩原始图像并且能够精确地对它进行还原。JPEG图像采用的压缩算法允许还原图像时存在细小的误差，因此JPEG图像具有非常高的压缩比，也就是在没有失真或很少失真的情况下，原始图像文件所占的比特数与压缩图像文件所占的比特数之比。图8.3显示了分辨率为1200 × 1200的灰度图像的3种不同版本：图8.3(a)是没有压缩的原始TIFF图像；图8.3(b)和图8.3(c)分别是具有不同压缩比的JPEG图像。它们对应的图像文件大小分别为1 440 198字节、80 109字节和38 962字节。因此，对于TIFF图像，它的每个像素占用1字节，加上198字节的头部信息和尾部信息。对于JPEG图像，它们的压缩比大约是18和37。即使对于上面的高压缩比图像，几乎在图像中没有出现可察觉的失真现象。如果把原始的图像保存为PostScript图像，那么图像文件的大小大约为TIFF图像文件的两倍，这是因为每个字节都要转换成两个由7个二进制位形成的ASCII字符，每两个这

样的ASCII字符需要两个字节的存储空间。如果把原始图像保存为压缩形式的TIFF文件，那么只需要大约一半的存储空间。使用ZIP格式的文件也会有同样的结果，ZIP格式是一种非常流行的可用于压缩任意文件的格式。图像的压缩比与图像有关。ZIP格式文件采用的是无损压缩方法，其压缩比远小于JPEG有损压缩图像，而这种JEPG压缩图像和原始图像在视觉上几乎无法区分开。正因为JEPG压缩图像的这种优良特性，使它成为Internet上最流行的传送图像的格式。

(a)　(b)　(c)

图8.3　(a) 原始的TIFF灰度图像；(b) 压缩比为18的JPEG图像；(c) 压缩比为37的JPEG图像

大量的图像格式给图形API带来了很多问题。一些图像格式非常简单，而另一些图像格式则相当复杂。OpenGL API为了避免这些格式问题，它没有包含任何图像格式。正如8.3节将讲到的，虽然OpenGL能够处理内存中用标准数组类型存储的图像，但是必须由应用程序编程人员来负责从处理器内存中读/写某种特定格式的图像。我们并不打算在本章介绍如何处理这些问题，因为对任意一种特定图像格式的处理都需要讨论它的细节。读者可以在本章后面的建议阅读资料中找到需要的信息。

通过使用几何绘制流水线生成三维场景的图像，然后可以从帧缓存读取这些图像，因此可以直接从图形系统中获取数字图像。我们将在本章的后面介绍如何处理所需的操作。

8.3　缓存的写操作

在现代图形显示系统中，应用程序既可以写缓存，也可以读缓存。有两个因素使得这两个操作区别于通常情况下计算机内存的读/写操作。第一个因素是：我们只是偶尔需要对缓存读/写单个的像素或者一个二进制位，而往往是读/写像素（或位）构成的矩形块，即所谓的位块（bit block）。例如，当填充一个多边形的时候，我们每次光栅化一整条扫描线；当显示一个光栅字符的时候，我们把一个小的像素块写入帧缓存；当对缓存执行清空操作的时候，我们一次性改变缓存中的所有值。因此，使硬件和软件都支持对像素矩形块的高效操作是非常有用的，我们把这种操作称为位块传送（bitblt）操作，这些操作也称为光栅操作（raster-ops）。

假定需要从某个缓存（即源缓存）把大小为$n \times m$的像素块复制到同一缓存或另一缓存（即目标缓存）。像素块的传送操作如图8.4所示。一个典型的位块传送写操作可写成如下的形式：

```
write_block(source, n, m, x, y, destination, u, v);
```

其中，source和destination是缓存。该操作把源缓存中左下角位于(x, y)的大小为n × m的像素块写入起始位置在(u, v)的目标缓存中。虽然有许多细节必须考虑，如源块超出了目标块的边界时会发生什么情况，但是位块传送操作的本质在于通过单个的函数调用来实现对整个目标块的修改。

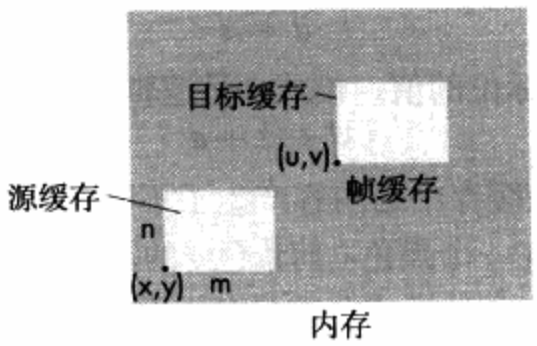


图 8.4　位块的写操作

注意，从硬件的角度来看，这种像素块的处理过程不具备几何对象处理过程的任何特征，用来优化位块传送操作的硬件与几何绘制流水线具有完全不同的结构，所以OpenGL的体系结构既包含几何绘制流水线，又包含像素操作流水线，它们的实现通常是互为独立的。

8.3.1　写入模式

常规的内存写操作与位块传送操作的第二个区别是后者对缓存有各种不同的写入模式。例如第 3 章介绍的逻辑操作，这样利用 XOR 写模式就可以绘制橡皮条拖引线。OpenGL 支持 16 种不同的逻辑操作。为了充分理解所有这些操作的功能，下面分析缓存写操作的实现过程。

写内存的概念通常是指将内存中的内容替换掉。例如，下面是C语言程序中的一条赋值语句：

```
y=x;
```

这条语句的执行会使得 y 存储单元中的值被 x 存储单元中的值替换掉。

除了这种写模式之外，还有其他的方式。假定一次只能处理缓存中的一位。分析如图 8.5 所示的写模式，图中的 *s* 称为源位（source bit），它表示要写入内存的位，也许需要把它转换为另一种形式；*d* 称为目标位（destination bit），它表示内存中被写入的位置。与第 3 章一样，如果允许在写操作执行之前执行读操作，那么可以使用下面的替换函数 *f* 表示写操作：

$$d \leftarrow f(d, s)$$

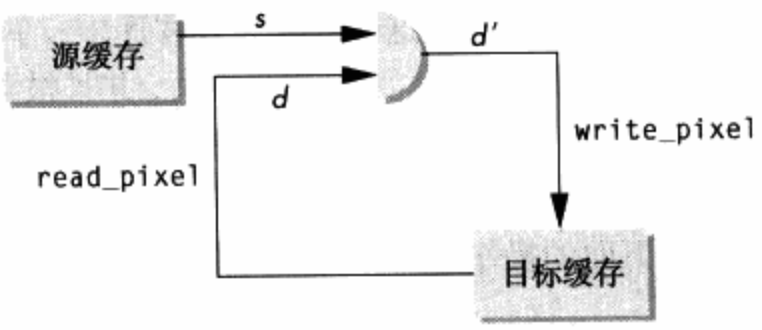


图 8.5　写入模式

对于源位和目标位都只有一位的情况，函数 *f* 只有 16 种可能的方式，即源和目标对应的两个位之间的 16 种逻辑操作。这 16 种逻辑操作如图 8.6 所示，图右侧共 16 列，其中每列对应于函数 *f* 的一种方式。我们可以使用每一列中的二进制数值表示一种写入模式（writing mode），也就是说，可以使用每一列定义的逻辑操作表示写入模式。假定逻辑值“1”表示一种背景颜色（如白色），而“0”表示一种前景色（如黑色），我们分析函数 *f* 在各种方式下的返回值。模式 0 和模式 15 是清空操作，它们把目标位修改为前景色或背景色。目标位的新值与源位和目标位的初始值没有关系。模式 3 和模式 7 属于常规的写入模式，模式 3 是：

$$d \leftarrow s$$

它只是简单地用源位的值替换目标位的值。模式7是逻辑或操作：

$$d \leftarrow s + d$$

图8.7说明了这两种写入模式对帧缓存中的内容产生了不同的影响。在这个例子中，我们把虚线写入帧缓存之前，帧缓存中已经存在一个黑色（前景色）的矩形。虽然这两种模式都在背景色上写入前景色，但是如果我们尝试使用这两种模式在前景色上写入背景色，得到的结果将不一样。究竟采用哪种模式取决于应用程序编程人员希望得到的效果。

s	d	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

图8.6　写入模式

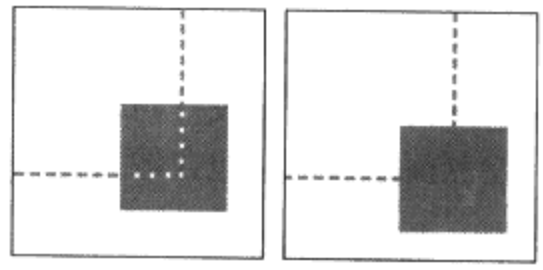


图8.7　基于模式3和模式7的写入操作

8.3.2　异或写入模式

模式6是异或操作XOR，用⊕表示，它是最有趣的一种写入模式。与模式3和模式7不同，模式6的实现要求必须读取目标位的信息。异或写入模式之所以具有强大的功能，是因为如果s和d是二进制变量，那么异或逻辑满足下面的性质：

$$d = (d \oplus s) \oplus s$$

因此，如果对某一个源位执行两次异或操作，那么返回目标位的初始状态。

异或写入模式最重要的应用与交互有关。第3章使用XOR模式实现了橡皮条操作。我们还可以使用该模式实现那些需要对部分显示内容重新绘制的操作。

下面分析在交互式应用程序中使用菜单时会发生什么情况，如第3章介绍的多边形建模应用程序就是这样一个使用了菜单的交互式程序。在这样的交互式程序中，单击鼠标后会出现一个菜单，它覆盖了部分屏幕。当用户选择一个菜单项后，菜单就会消失，并且被菜单覆盖的屏幕区域又恢复到原来的状态。上面这个过程的实现要用到离屏存储器（off-screen memory）[①]，也称为后端存储器（backing store）。假定菜单通过一个位阵列M存储在离屏存储器中，而该菜单将出现的屏幕区域存储在另一个位阵列S中。分析下面的操作序列：

$$S \leftarrow S \oplus M$$

$$M \leftarrow S \oplus M$$

$$S \leftarrow S \oplus M$$

假定XOR操作作用于S和M这两个位阵列中相应的二进制位。如果把第一个等式的结果代入第二个等式，并把第二个等式的结果代入第三个等式，那么当3个操作结束后，发现菜单会显示在屏幕上。现在被菜单覆盖的屏幕区域上的原始内容存储在离屏存储器中，而离屏存储器原来存储的是菜单的位阵列。我们通过3个位块传送（bitblt）操作交换了菜单位阵列图像与屏幕区域位阵列图像。这种交换方法与常规交换模式有显著的区别，后者使用替换写入模式，且需要使用临时存储空间来实现交换。

① 离屏存储器中的内容不会显示在屏幕上，可用它实现双缓存技术。——译者注

这种技术有许多变异形式。一种是在屏幕上移动光标而不影响其下面的区域；另一种是作为多边形扫描转换过程的一部分，使用单一的颜色填充多边形。注意，因为许多图形API是在光栅显示系统成为标准之前出现的，所以这些API不一定支持XOR写入模式。

8.4　OpenGL中的位操作与像素操作

数字图像的每个像素包含k个二进制位，对于黑白二值图像，k等于1；对于RGBA图像，k等于32；而对于高分辨率彩色应用，k甚至可以等于更大的数值，因此k可以取其间的任意一个数值。尽管在理论上，1位图像（或位图）与8位（1字节）或24位（3字节）图像在本质上属于同一类型，但是硬件和软件处理这些图像的方式却不尽相同。从硬件的角度来看，位图的实现只涉及逻辑操作，而且每次只工作在缓存的一个位平面上。从软件的角度来看，即使位图与多位图像可能使用相同的物理内存，但是它们的使用方式却不同。一般来说，位图可用来表示字体、掩码和图案。OpenGL允许用户通过两组不同的操作来处理位图或多位图像。

当处理像字体或光标这样的位图时，位块传送操作（bitblt）才有意义。但是如何把这些操作用于多位像素和多位颜色值还不是很清楚，因为这16种位操作的结果一般无法转换成视觉上看起来合理的图像，尽管OpenGL允许人们对基本图元使用逻辑操作。OpenGL通过第3章介绍的glLogicOp函数实现逻辑操作。

OpenGL不仅支持独立的像素流水线和各种缓存，而且还允许数据在这些缓存之间以及在缓存和处理器内存之间进行传送。根据使用的缓存类型，我们可以在缓存中存储索引、颜色分量和像素深度等信息。此外，我们还可以使用标准的数据类型（字节、整数和浮点数）把数据存储在处理器内存中。如此之多的数据格式和类型使我们难以编写高效的位图和像素处理程序。这里并不打算讨论所有相关的内容，而是讨论那些被OpenGL支持的功能。

8.4.1　OpenGL缓存和像素流水线

OpenGL支持多种类型的缓存，其中包括：

- 颜色缓存
- 深度缓存
- 累积缓存
- 模板缓存

颜色缓存包括那些其内容可以在输出设备上显示的缓存，但是，正如第3章讨论双缓存结构时讲到的，颜色缓存可以包括两个独立的读缓存和写缓存。在双缓存结构中，这两个颜色缓存分别称为前端缓存和后端缓存。我们也许希望使用右缓存和左缓存来生成立体图像对，右眼和左眼看到的图像分别存储在这两个缓存中。对于立体动画，可能需要使用4个颜色缓存。一般来说，我们把几何对象绘制到颜色缓存中，但是我们也可以直接对缓存进行读/写操作。只有颜色缓存中的内容可以被显示出来。

深度缓存一般用于隐藏面消除，但是我们同样可以对它进行读/写操作。8.12节将说明累积缓存用于组合多个图像的各种操作。模板缓存通常用于屏蔽操作，例如，我们可以把这个缓存中的数值作为掩码，并由此来决定是否将颜色缓存中的某个像素绘制到屏幕上。

现在假定只处理颜色缓存（通常是后端缓存），尽管许多这样的操作可以应用于其他的缓存。注意，某个特定的OpenGL实现可能只支持这些缓存中的某些缓存，而且一个缓存的深度也与具体的OpenGL实现有关。

OpenGL系统维护一个光栅位置（raster position）变量。我们可以认为这个光栅位置是一个位于屏幕坐标系中的内部光标，根据当前的光栅位置可以把光栅化图元写入缓存中。我们可以使用函数glRasterPos的各种形式来设置光栅位置。例如，下面的函数

```
glRasterPos3f(x, y, z);
```

使用了3个浮点数来指定当前的光栅位置，这3个浮点数在变换到屏幕坐标系之前要经过模-视变换和投影变换。与任何其他的几何对象一样，光栅位置的存储形式也采用了四维齐次坐标。然而，当光栅位置经绘制流水线处理而变换成屏幕坐标系中的位置后，像素或位的写入操作必须使用光栅单位（整数），像素的中心位置位于两个整数之间的中间处。注意，如果光栅位置不在视见体的内部，那么就不绘制任何像素，整个位图或像素矩形块都会被裁剪掉。

在OpenGL中有一个当前绘制缓存（current drawing buffer），绘制操作把用户程序中的像素数据绘制到这个缓存中；另外还有一个当前读缓存（current read buffer），用户程序中任何读像素的操作都可以从这个缓存读取数据。这两个缓存可以是OpenGL支持的任意一个缓存。在默认情况下，它们都是双缓存模式的后端缓存，但是可以使用glDrawBuffer和glReadBuffer函数改变当前的缓存。

由于应用程序生成的像素从处理器内存流经到帧缓存的某个缓存中，其间要进行一系列操作。这一系列操作构成了如图8.8所示的像素流水线结构。注意，虽然表面看起来应用程序中定义的像素可以直接写到OpenGL缓存中，但实际上它们与几何图元一样，首先流经光栅处理器并生成片元。这种结构可以统一处理几何图元和离散图元，这为同时使用这两类图元的应用程序带来了便利。

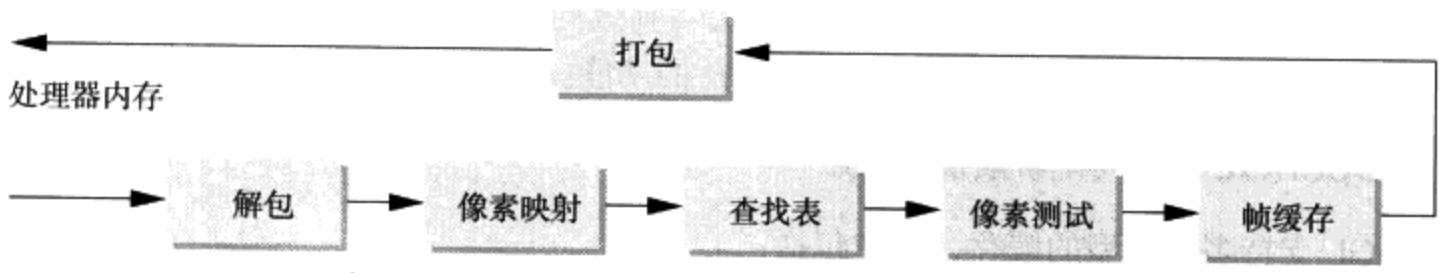


图8.8　OpenGL像素流水线结构

第一个操作是解包（unpacking），即把用户程序中的像素格式转换成OpenGL内部使用的像素格式；之后通过查找表把这些像素映射为新的值；紧接着再对这些像素进行一系列测试，通过测试确定像素是否要写入帧缓存，如果需要写入，还要确定像素写入帧缓存的方式。例如，我们可以屏蔽某些不需要绘制的区域，并使用逻辑操作确定像素与帧缓存中已有像素的组合方式。如果要把从缓存中读取的信息放回处理器内存，那么必须把OpenGL内部格式的数据打包成应用程序的数据格式。

8.4.2　位图

OpenGL对1个二进制位的像素阵列（即bitmaps，位图）的处理方法不同于多个二进制位的像素阵列（即pixelmaps，像素图），并且它们对此提供了不同的编程接口。位图通常利用XOR写入模式，主要用于字体和光标的绘制。绘制字体和光标都需要使用当前的光栅位置来设置位图的位置。因此，一旦位图位于当前缓存，那么位图的每位对应于该缓存中的一个像素。我们把位图的位当做掩码来使用，它决定了与该位置相对应的像素是否需要改变颜色。如果位图的某个位为0，那么对应的像素颜色不变；如果位图的某个位为1，那么对应的像素需要根据当前光栅颜色和绘制模式来决定如何修改它的颜色。我们使用函数glColor设置光栅颜色，除非最后一次调用glRasterPos函数

设置了光栅颜色，否则它与绘制几何图元的颜色相同。假定位图的某位与颜色缓存中的一个红色像素相对应，并且当前的光栅颜色为绿色。如果位图的这个位为0，那么对应的像素仍为红色。如果位图的这个位为1，并且当前使用默认的替换写入模式，那么对应像素的颜色由红色变为绿色。然而，如果使用XOR写入模式，那么对应像素的颜色将变为黄色，这是红色和绿色的24位RGB二进制表示形式按异或运算的结果。

我们使用下面的函数把位图阵列放在当前光栅位置（假定它是缓存中的一个有效位置）：

```
glBitmap(width, height, xo, yo, xi,yi, bitmap);
```

其中，width和height的单位是位。浮点数xo和yo表示相对于当前光栅位置的偏移值，它们给出了位图左下角的起始绘制位置（如图8.9所示）。浮点数xi和yi表示位图绘制后当前光栅位置的移动值。

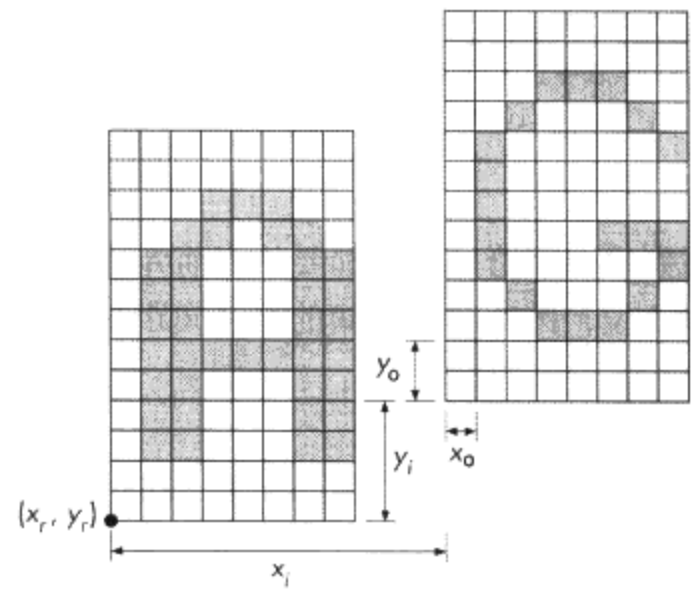


图 8.9　一个位图的输出

8.4.3　光栅字体

我们使用光栅字体来说明光栅位置的用法。在第3章，我们发现光栅字符的绘制速度要远远快于笔画字符。通常情况下，字体定义了某种特定字型的字符集，如Times、Courier或Computer Modern等字体。每种字体都有各种不同的大小[①]，如10点或24点等，还有各种样式，如粗体或斜体等。然而，我们使用光栅字体这个术语表示一定大小和样式的字符集。因此，GLUT库提供的8 × 13字体可以表示为GLUT_BITMAP_8_BY_13；大小为10点的均衡字体[②]可以表示为GLUT_BITMAP_TIMES_ROMAN_10。一种典型的字体包含128个或256个字符。光栅字体中的每个字符都定义为一个位图案，如图8.10所示。注意，当把字体定义为位图案时，字符在屏幕上显示的大小取决于屏幕的分辨率，这种关系并不适合几何对象的绘制。

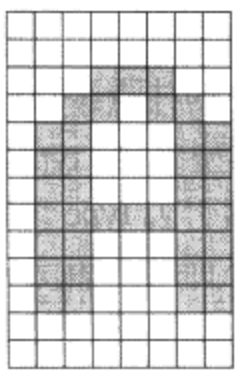


图 8.10　字符的定义

① 1英寸包含72点。

② 均衡字体是指并非所有字符都有相同宽度的字体，而对于单型字体，所有字符的宽度都相同。

我们把位图字体设计成既可以在一定大小的空间显示，又有很好的可读性。因此，正如从图8.10看到的那样，除了最小的字体，在其他所有的字体中，字符之间必须有水平间距和垂直间距；必须为部分笔划伸过基线的字母留有一定的空间，如小写字母g和y；还必须考虑大小写字母的区别。一旦设计好一种字体，就可以把它存储为任意的位图。例如，如果希望定义一种包含128个ASCII字符的8 × 13字体，那么可以使用下面的数组：

```
GLubyte my_font[128][13];
```

与第3章的处理方法相同，我们也可以使用显示列表为每个字符定义一个字体。假定已经定义了my_font数组，那么可以使用下面的代码生成这些显示列表（见3.4节）[①]：

```
base=glGenLists(128);
for(i=0; i<128; i++)
{
     glNewList(base+i, GL_COMPILE);
     glBitMap(8, 13, 0.0, 2.0, 10.0, 0.0, my_font[i]);
     glEndList();
}
```

在这个例子中，我们注意到每个字符显示时都要从当前光栅位置上移2位，并且在绘制每个字符之后，光栅位置再向右移动2位，因此字符之间的水平间距和垂直间距都为2位。

8.4.4　像素与图像

在OpenGL中，为了使用户能够处理不同类型的图像，在对图像的像素进行处理时，OpenGL使用各种格式把每个像素表示为一个多元组，多元组的每个分量占用一个字节。例如，处理RGB图像时，图像中每个像素的颜色分量值使用一个字节存储其颜色值，或者在处理单色（灰度或亮度）图像时，图像的每个像素用单个浮点数表示。

在OpenGL中有3个函数用来移动图像，这3个函数是：

```
glReadPixels(GLint x, GLint y, GLsizei width, GLsizei height,
          GLenum format, GLenum type, GLvoid *image)
glDrawPixels(GLsizei width, GLsizei height, GLenum format,
          GLenum type, GLvoid *image)
glCopyPixels(GLint x,GLint y, GLsizei width, GLsizei height,
          GLenum buffer)
```

函数glReadPixels和glDrawPixels用来在OpenGL缓存与位于处理器内存的阵列图像之间进行像素矩形块的读/写操作；而函数glCopyPixels的功能是：在帧缓存中从左下角(x, y)开始读取一个一定大小的像素块，然后把这个像素块从当前光栅位置开始复制到帧缓存的另一个区域。在函数glReadPixels中，我们可以指定帧缓存的偏移位置(x, y)，并从该位置读取大小为width × height的像素块。虽然在大多数应用程序中，我们使用默认的后端颜色缓存，但是我们可以使用函数glReadBuffer和glDrawBuffer选择OpenGL的任意缓存用于读/写操作。

上述函数中的格式参数（format）和类型参数（type）用来描述处理器内存中的图像。format参数描述图像的类型（RGB、RGBA或颜色索引），type参数是应用程序使用的数据类型（int，float）。因此，对于8.2节定义的图像，我们使用下面的数组定义一个RGB图像：

```
GLubyte myimage[512][512][3]
```

在把这个图像写入当前绘制缓存之前，首先需要设置光栅位置：

```
glRasterPos2i(startx, starty);
```

① 也许我们浪费了一部分空间用来存储前32个ASCII字符。因为这些字符通常是非打印字符，仅仅起控制作用，它们不会产生任何可见的输出。

然后使用函数 glDrawPixels 把图像写入当前绘制缓存：

```
glDrawPixels(512, 512, GL_RGB, GL_UNSIGNED_BYTE, myimage);
```

我们可能还会遇到一个令人困惑的问题，即不同的处理器通常使用不同的方法组织内存中的数据。例如，在 int（整型）这样的多字节数据类型中，不同的处理器对最高有效字节的规定不尽相同。有些处理器允许数据只能对齐于4字节边界，因此像myimage这样的数组在不同计算机中的存储形式可能不同。这种与系统相关的特性给 OpenGL 带来了负面影响，尤其是在读/写像素的时候。为此，OpenGL 提供了一个函数 glPixelStore，应用程序可使用该函数来描述数据打包和解包的形式。一般来说，系统的默认设置使我们不必使用该函数，但是情况不会总是如此。

8.4.5　查找表

在 OpenGL 中，当颜色存储到缓存中时，可以使用查找表来修改它们的颜色。假定系统的缓存能够存储 8 位颜色分量，那么每个分量具有 256 种可能的值，可以把其中的每个值看成是查找表的索引值，而查找表包含 256 项，如图 8.11 所示。如果查找表中的每个分量值也占用 8 位，那么就可以通过索引值从查找表中选择一个新值作为该分量的值。默认情况下，查找表提供了一个从输入值到输出值的一一映射，可以利用函数 glPixelMap 载入用户定义的映射表。

这些映射与第 2 章介绍的颜色查找表（查色表）在本质上是相同的。如果在颜色索引值与灰度图像的灰度值之间建立一个对应关系，那么我们可以通过这些映射把灰度值转换为彩色值，从而达到图像增强的目的。通过这种方式生成的彩色图像称为伪彩色（pseudo color）图像。

用户在生成伪彩色图像时遇到的主要问题是如何从 RGB 颜色模型可能包含的颜色中（一般包含 2^{24} 种颜色）选择 256 种颜色作为调色板。一种常用的选择方法是基于热效应测量表，即使用暖色（如红色、白色和黄色）表示重要的值，而使用冷色（如蓝色和绿色）表示不是很重要的值。例如，假定对于我们要处理的灰度图像，最小的灰度值表示最不重要的东西，而最大的灰度值表示最重要的东西。图 8.12 显示了一种可能的映射，它把灰度值映射为 RGB 颜色值，从而得到了一个热效应测量表。彩图 23 说明了热效应测量表在科学计算可视化中的应用，彩图中的色彩表示由计算机模型生成的地幔的温度。

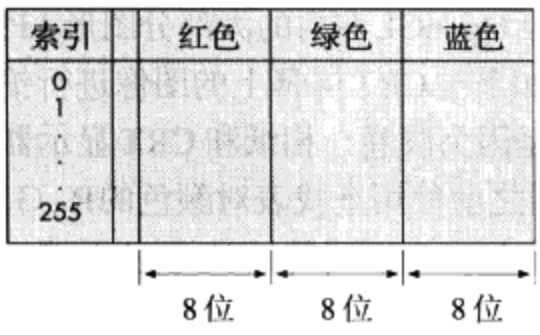


图 8.11　RGB 颜色查找表

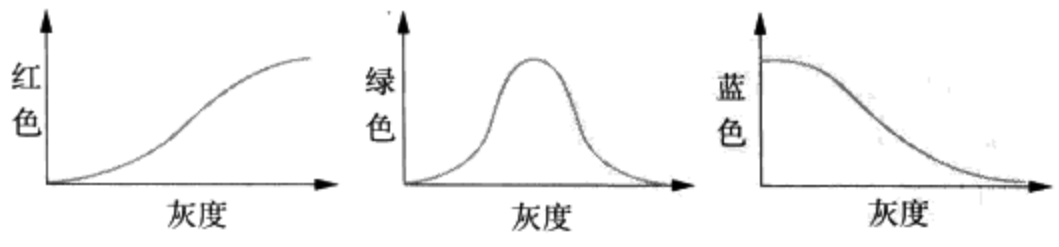


图 8.12　热效应颜色映射曲线

假设我们定义了一个大小为 $n \times m$ 的灰度图像，并希望把这个图像转换成伪彩色图像：

```
GLubyte image[n][m];
```

我们为每个颜色分量按如下方式定义相应的颜色查找表：

```
GLfloat redmap[256], greenmap[256], bluemap[256];
```

这是第 11 章将要使用的一个简单映射表。我们可以通过下面的代码实现上述查找表，即当灰度增加时，红色分量的值增加，而蓝色分量的值减少，绿色分量的值是一个随机数：

```
for(i=0;i<256;i++)
{
    redmap[i]=i/255.;
```

```
    greenmap[i]=(rand()%256)/255.0; /* 0.0到1.0之间的一个随机数 */
    bluemap[i]=1.0-i/255.;
}

glPixelMapfv(GL_PIXEL_MAP_I_TO_R, 256, redmap);
glPixelMapfv(GL_PIXEL_MAP_I_TO_G, 256, greenmap);
glPixelMapfv(GL_PIXEL_MAP_I_TO_B, 256, bluemap);
```

读者此时应该能够很容易地实现其他的颜色查找表。显示回调函数的响应函数display只需执行清屏操作并将像素写入帧缓存中，代码如下：

```
void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glDrawPixels(n,m,GL_COLOR_INDEX, GL_UNSIGNED_BYTE, image);
    glFlush();
}
```

我们还可以把查找表用于颜色转换和亮度调整。例如，我们使用计算机图形学中的着色模型生成的图像通常比较暗且缺乏对比度。对于生成的灰度图像，如果把它的亮度值（或灰度值）的分布绘制成曲线，就可以得到如图8.13所示的直方图，该直方图说明了所生成的灰度图像的大部分灰度值处在比较小的范围。如果把如图8.14所示的查找表作用于灰度图像上的每个像素，就可以把低灰度值映射为高灰度值，从而使图像显得更亮。变换后得到的灰度图像的灰度分布会更加均匀，而且仍然分布在整个灰度范围内，因此改善了原始图像的对比度。

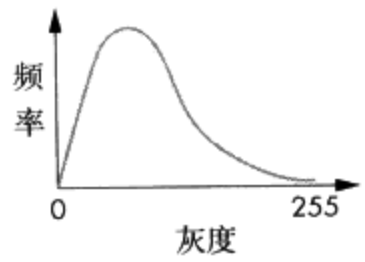


图8.13　暗图像的直方图

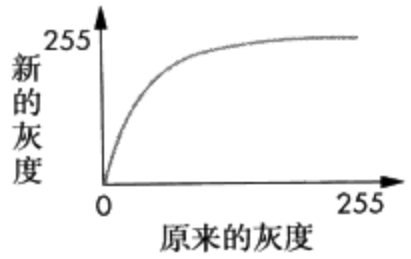


图8.14　使图像变亮的查找表

我们还可以把查找表用于颜色转换。正如第7章讨论的，在包括OpenGL在内的大部分图形API中，RGB颜色值并不是针对某个特定的图形系统。通常情况下，如果对CRT屏幕上的图像进行拍照，则发现冲洗出来的照片与屏幕上的图像存在明显的区别，这是因为胶卷、相纸和CRT显示器的颜色属性以及观察者的感觉特性之间存在差异。我们可以在拍照之前使用查找表对颜色的R、G、B分量进行颜色校正，也称为颜色平衡（color balancing）。因此，查找表为不同的RGB颜色系统之间的颜色转换提供了一种方法。

直方图的计算和基于查找表的颜色转换都是图像处理领域的核心内容。8.12节将进一步研究这些内容以及其他的图像操作，并且讨论在OpenGL绘制结构内部是如何支持这些图像操作的。

8.5　举例

现在分析3个例子，用来说明像素的写入操作。第一个例子把某种显示设备的麦克斯韦颜色三角形（Maxwell triangle）显示出来。第二个例子通过建立一个伪颜色缓存，介绍一种用于测试算法的简单方法，例如测试第7章介绍的算法。我们在第3章基于选择方法实现了拾取操作，最后一个例子说明了另一种基于缓存实现拾取操作的简单方法。

8.5.1 显示某种显示设备的色域

7.13.1 节讲到，某种显示设备可以产生的颜色在色度坐标系中位于一个称为麦克斯韦三角形（Maxwell triangle）的区域内。我们可以很容易地把自己使用的显示器的色域显示出来。我们希望生成如下定义的 RGB 图像：

```
#define N 1024

GLfloat image[N][N][3];
```

考虑下面的循环语句：

```
int i, j, k;
for(i=0; i<N; i++) for(j=0;j<N; j++)
{
   image[i][j][0] = (float) i/N:
   image[i][j][1] = (float) j/N;
   image[i][j][2] = 1.0 - image[i][j][0] - image[i][j][1];
   if(image[i][j][2] < 0.0) for(k=0;k<3'k++) image[i][j][k] = 0.0;
}
```

我们在色度空间中显示图像 image 的前两个颜色分量（红色和绿色）。第三个颜色分量（蓝色）必须满足 $b = 1 - r - g$ 且不能为负数。我们用黑色给麦克斯韦三角形之外的像素着色。显示回调函数很简单，如下所示：

```
void display()
{
   glClear(GL_COLOR_BUFFER_BIT);
   glDrawPixels(N, N, GL_RGB, GL_FLOAT, image);
   glFlush();
}
```

这个例子虽然很小，但是本章后面的习题对该例子做了一些扩充。

8.5.2 测试算法

像素写操作的另一个简单例子是测试第7章介绍的一些算法。我们在初始化时建立一个 500 × 500 的绘图窗口，并按下面的方式设置相应的观察变换：

```
glMatrixMode(GL_PROJECTION);
glLoadIdentity;
gluOrtho2D(0.0, 500.0, 0.0, 500.0);
glViewport(0, 0, 500, 500);
```

每次执行下面的程序都会在指定的位置绘制单个的黑色像素：

```
GLubyte black[1][1][3]={0, 0, 0};
glRasterPos2i(ix, iy);
glDrawPixels(1, 1, GL_RGB, GL_UNSIGNED_BYTE, black);
```

draw_pixel 函数很简单，如下所示：

```
void draw_pixel(int ix, int iy, int value)
{
    glRasterPos2i(ix, iy);
    glDrawPixels(1, 1, GL_RGB, GL_UNSIGNED_BYTE, value);
}
```

其中的参数value是一个指向RGB颜色数组的指针。附录A中有一个程序利用了Bresenham算法对直线进行光栅化并将该直线的像素写入一个500 × 500的伪帧缓存中。

8.5.3　用于拾取操作的缓存

缓存为建立和操纵图像提供了无限的机会。我们可以把缓存作为实现交互式操作的一种机制，这初看起来似乎是不可能的。下面讨论拾取操作的问题。第3章介绍了用于拾取操作的OpenGL机制，它使用选择和命中列表来提供用于识别屏幕上对象所需的信息。这种机制虽然具有通用性，但需要编写大量的实现代码。如果屏幕上需要识别的对象不是很多，那么可以利用缓存作为实现拾取操作的另一种机制，这种机制实现起来更容易。

假定只有256个（假定使用8位深入缓存）需要识别的对象。当绘制图元时，我们需要把正在绘制的对象的标识符放在一个额外的缓存中，标识符存放的位置对应于帧缓存中该对象片元颜色的存储位置。因此，在这个额外缓存中存储的“颜色值”是帧缓存中位于相应位置的对象所对应的标识符。我们可以使用OpenGL的像素读取函数来获取这些颜色，因此可以把拾取操作看做是从这个额外缓存中读取光标所在位置处像素对应的标识符的值。如果建立一个把颜色值映射为对象标识符的查找表，那么这种方法也适用于24位的颜色缓存。如果把这个过程与z-buffer算法结合起来，那么在这个额外缓存中对应每个像素都保存了一个穿过该像素的光线上离视点最近的对象的标识符。

8.6　映射方法

离散数据最重要的用途之一就是表面绘制。前面介绍的建模过程是：先用一组几何图元表示对象，然后绘制这些图元，这种建模方法有一定的局限性。例如，我们分析用计算机绘制一个虚拟橘子的过程。我们可能首先想到的就是绘制一个球体。从第6章的讨论可知，可以使用一些三角形生成一个近似的球体，然后使用与真实橘子表面相似的材质绘制这些三角形。遗憾的是，这种绘制方法太正规，以至于绘制的橘子看起来不像真实的橘子。我们还可以使用第12章将要讨论的方法，即使用某种类型的曲面表示橘子模型，然后绘制这些曲面，这种方法可以使我们能更好地控制虚拟橘子的形状，但是生成的橘子图像看起来仍然不是很逼真。尽管采用这种方法绘制的橘子在总体属性方面还能令人满意，如形状和颜色，但是缺少真实橘子表面复杂的细节特征。如果试图采用给对象模型增加更多多边形的方法来增加对象表面的细节，那么即使使用每秒能够绘制上千万个多边形的图形硬件，仍可能使绘制流水线的负荷过重。

还有一种方法，它并不是试图建立一个越来越复杂的对象模型，而是建立一个简单的模型，并把增加对象表面的细节作为绘制过程的一部分。正如第7章讲到的，当使用OpenGL绘制对象表面时，不管是多边形表面还是曲面表面，经过绘制流水线的处理最终都要变换成片元（fragment），每个片元对应于帧缓存中的一个像素。片元具有颜色、深度和其他信息，系统根据这些信息来决定这些片元是如何影响它们所对应的像素的。作为光栅处理过程的一部分，我们必须通过计算得到每个片元的亮度值或颜色值。从第6章开始使用改进的Phong模型来确定多边形顶点的颜色，通过顶点颜色在多边形表面上的插值计算得到对象表面其他所有像素的颜色。然而在光栅化之后的片元处理过程中，我们可以修改这些颜色值。可以这样理解映射算法，它要么根据二维阵列（图像）修改明暗绘制算法，要么使用映射关系改变对象的表面参数（如材质属性和法向量等），从而修改明暗绘制算法。有3种主要的映射方法：

- 纹理映射

- 凹凸映射
- 环境映射

纹理映射（texture mapping）使用一个图案（或纹理）确定片元的颜色。这些图案要么是一些具有固定样式的图案，如经常用于填充多边形的规则图案；要么是一些通过过程纹理生成方法得到的图案；还可以是一些通过数字化处理后得到的数字图像。如图 8.15 所示，对于上面所有这些情形，我们都可以把纹理映射到对象的表面，从而刻画对象的表面细节特征，这个映射过程作为对象表面绘制过程的一部分。

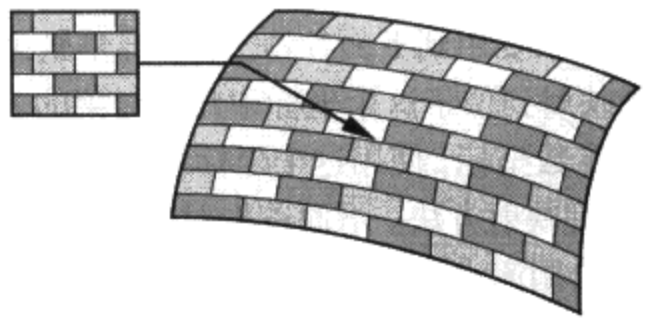


图 8.15　把一个图案纹理映射到一个表面上

纹理映射把图案映射到光滑的表面，从而给表面增加细节特征，然而凹凸映射（bump map）在明暗处理过程中对法向量进行扰动，从而使表面形状看起来有微小的起伏变化，就像真实的橘子表面上的凹凸不平。我们还可以使用反射映射（Reflection map）或环境映射(environment map）生成具有光线跟踪效果的图像，而不必跟踪反射光线，在这种情况下，当绘制对象表面时，把环境图像映射到这个表面上。

上面 3 种方法具有很多共同点，即作为片元处理过程的一部分，所有这些方法都改变了单个片元的明暗值；所有这些方法都依赖于一维、二维或三维数字图像；所有这些方法都降低了对象表示的几何复杂度，而生成的图像看起来像是复杂的几何对象。然而，所有这些方法都受走样误差的影响。

本书的彩图包含了各种二维映射的例子。彩图 11 是使用 OpenGL 的环境映射绘制的效果，它说明了如何使用单个纹理贴图生成具有高反射表面效果的图像，而不用进行全局光照计算。彩图27通过纹理映射把纹理映射到桌子的表面，彩图 14 使用纹理映射生成砖块图案，彩图 19 中的许多细节来自纹理贴图。在虚拟现实和可视化仿真等应用领域，需要考虑系统的实时性能。当在具有硬件纹理映射功能的工作站上运行可视化仿真等应用程序时，可以采用纹理映射技术增加对象表面的细节，而又不会明显降低系统的绘制性能。

然而，对于标准的固定功能 OpenGL 绘制流水线，这 3 种映射方法存在明显的区别。标准纹理映射得到了 OpenGL 基本的绘制流水线的支持，并且它充分利用了几何绘制流水线和像素流水线。环境映射是标准纹理映射的一种特殊情况，但是，如果我们能够修改片元处理程序，那么对它做些修改就能生成各种新的效果，第9章将讨论这个问题。由于凹凸映射要求对每个片元单独进行处理，因此我们把这种方法推迟到第 9 章讨论。

8.7　纹理映射

纹理就是图案。纹理的形式很多，它可以是规则的图案，如条纹图案和棋盘图案，也可以是自然界的物质具有的复杂图案。在真实的环境中，我们可以通过纹理来区别具有相同大小和形状的对象。因此，如果希望生成具有逼真细节的虚拟对象，那么可以把纹理映射到我们建立的对象表面上，从而扩展系统目前已有的功能。

纹理的维度可以是一维、二维、三维或四维。例如，可以使用一维纹理图案得到曲线的颜色。可以使用三维纹理在一个实心块材质上雕刻出某个对象。因为二维表面在计算机图形学中占有非常重要的地位，所以把二维纹理映射到表面上是目前最常用的纹理映射方法，这也是我们唯一要详细介绍的纹理映射方法。然而，对于不同维度的纹理，其纹理映射过程大致相同，因此，我们虽然只介绍二维纹理映射方法，但这并不影响对其他纹理映射方法的理解。

8.7.1　二维纹理映射

虽然纹理映射的实现方法很多，但是它们都需要在3个或4个不同的坐标中经过一系列的映射变换步骤。在纹理映射的各个步骤中要用到下面各种坐标：屏幕坐标，最终生成的图像显示在这个坐标中；世界坐标，需要把纹理映射到位于这个坐标中几何对象的表面上；纹理坐标，使用这个坐标表示纹理空间中的一个位置；参数坐标，使用这个坐标定义曲面。我们根据使用的表面类型以及使用的绘制结构而选用不同的纹理映射方法。下面首先对纹理进行一般性的讨论，介绍各种映射，然后说明像OpenGL这样的实时绘制流水线结构是如何处理纹理映射的。

在大部分应用程序中，纹理最初的形式就是8.2节介绍的二维图像。因此，这样的纹理可以使用程序来生成，或者通过扫描相片得到，但是不管它们的最初形式如何，最终都要以数组形式存储在处理器内存中。我们把这些数组中的元素称为纹素（texel）或纹理元素而不是像素，主要是为了强调它们的应用方式。然而，我们目前暂时把这个数组视为一个连续的二维矩形纹理图案$T(s, t)$。其中两个独立的变量s和t称为纹理坐标[①]。不失一般性，我们可以把纹理坐标变换到(0, 1)这个区间。

纹理映射把一个纹素映射到几何对象的每个点，几何对象上的每个点又映射到屏幕坐标进行显示。如果几何对象采用齐次坐标(x, y, z, w)的表示形式，那么有下面的函数：

$$x = x(s, t)$$
$$y = y(s, t)$$
$$z = z(s, t)$$
$$w = w(s, t)$$

我们遇到的困难之一是：虽然这些函数在理论上都存在，但实际上可能无法找到这些函数。此外，我们还要担心逆运算问题，即如果已知几何对象上某个点(x, y, z)或(x, y, z, w)，如何才能找到对应的纹理坐标呢？或者，如何用下面的逆函数计算纹素$T(s, t)$呢？

$$s = s(x, y, z, w)$$
$$t = t(x, y, z, w)$$

如果使用参数(u, v)表示几何对象的表面，就像在6.4节使用参数表示球面，那么还存在其他的映射函数，即从(u, v)到(x, y, z)或(x, y, z, w)的映射，(x, y, z)或(x, y, z, w)表示几何对象在世界坐标系中的位置。尽管人们都知道这种映射可用于一些简单的表面，如球面、三角形和第12章将介绍的曲面，但是我们还是需要从参数坐标(u, v)到纹理坐标的映射，有时还需要从纹理坐标到参数坐标的逆映射。

我们还必须考虑从世界坐标到屏幕坐标的投影变换过程，这个变换过程沿绘制流水线经过了观察坐标系、裁剪坐标系和规范化的设备坐标系。我们可以把这个过程抽象为一个关于纹理坐标s和t的映射函数，某个纹素$T(s, t)$与颜色缓存中某个位置的颜色值相对应，通过这个函数可以知道颜

① 在四维坐标系中，纹理坐标是(s, t, r, q)空间中的一个值。

色缓存中的这个颜色值对最终显示图像的贡献。因此，有下面形式的从纹理坐标到屏幕坐标的映射函数：

$$x_s = x_s(s, t)$$
$$y_s = y_s(s, t)$$

其中，(x_s, y_s)表示颜色缓存中的某个位置。

根据采用的算法和图形绘制结构，我们可能还希望建立从颜色缓存中像素的位置到对像素颜色有贡献的纹素位置（纹理坐标）的映射函数。

对于表面使用参数表示的几何对象的纹理映射，如图8.16所示，它使用了两个同时进行的映射：第一个是从纹理坐标到世界坐标的映射，第二个是从参数坐标到世界坐标的映射。然后通过第三个映射把对象从世界坐标变换到屏幕坐标。

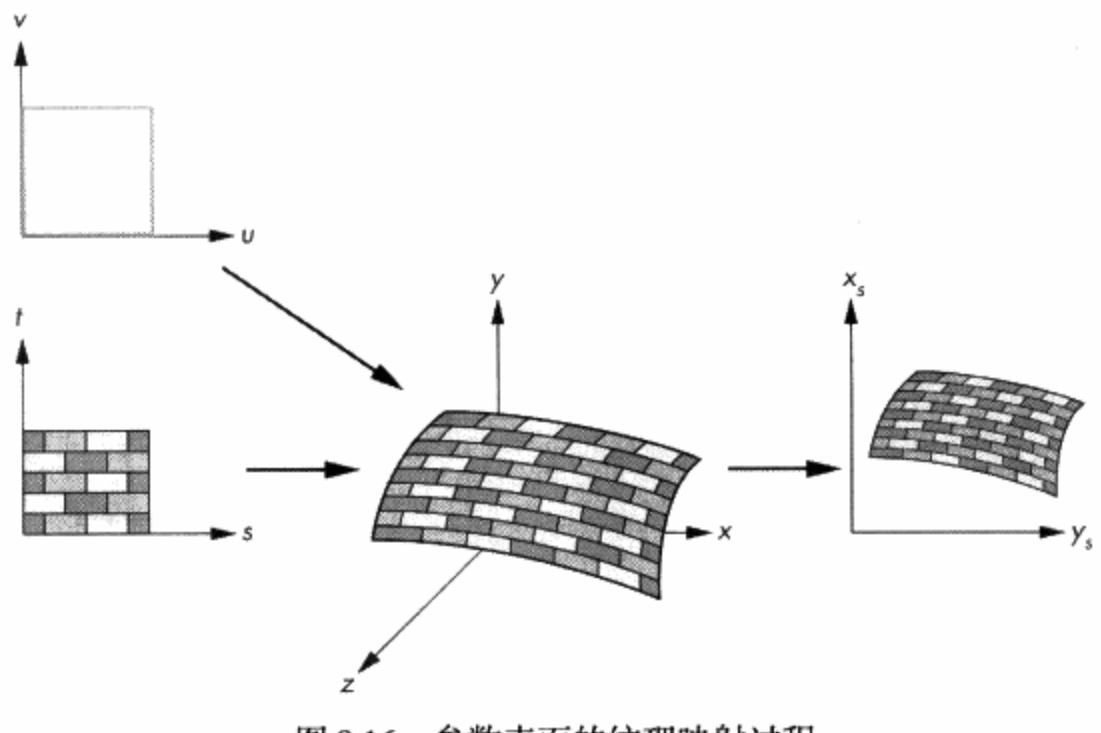


图8.16　参数表面的纹理映射过程

从理论上看，纹理映射过程非常简单。把纹理图案中的一块小区域映射到几何对象表面上的某个区域，因此，纹理图案中的这块小区域对应于最终显示的图像上的某个像素。如果假定纹素T的值是RGB颜色值，那么可以使用这些值来修改几何对象的表面颜色，几何对象表面的颜色值可能是通过光照模型计算得到的，或者只根据纹理值对几何对象表面的颜色进行赋值。这里的颜色赋值操作是作为片元颜色赋值的一部分执行的。

如果进一步深入研究，我们会面临很多困难。首先，我们必须确定从纹理坐标到世界坐标的映射函数。我们通常把一个二维纹理定义在纹理空间的一个矩形区域内。从这个矩形区域到三维空间中某个任意区域的映射或许是一个非常复杂的函数，或许具有一些不合人意的性质。例如，如果要把一个矩形映射到球面上，那么一定会产生距离和形状上的变形。其次，由于绘制过程在本质上是基于逐个像素绘制的，所以我们对屏幕坐标到纹理坐标的逆映射更感兴趣。仅当需要确定某个像素的颜色时，我们才需要确定纹理图像上对应位置的纹理坐标，这是一个从屏幕坐标到纹理坐标的计算过程。第三个问题是：因为每个像素对应于显示屏幕上一个很小的矩形区域，所以我们对点到点的映射并不感兴趣，而对区域到区域的映射感兴趣。这里同样会遇到潜在的走样问题，如果要避免走样引起的瑕疵，如波状图案或网纹状图案，就必须谨慎地处理这些问题。

图8.17说明了纹理映射过程可能会遇到的一些困难。假定要计算中心位置位于屏幕坐标(x_s, y_s)处的正方形像素的颜色，正方形像素的中心位置(x_s, y_s)对应于世界坐标系中的某个点(x, y, z)。但是

如果对象的表面是曲面，那么把像素的4个角点逆向投影到世界坐标系会得到一个弯曲的像素原像（preimage）。就纹理图像$T(s, t)$而言，把像素逆向投影到纹理坐标空间与会得到一个原像，该原像所在的纹理区域会对图中像素的颜色产生影响。

我们暂不讨论如何求解逆向映射函数的问题，而是先讨论像素颜色的确定问题。一种方法是把像素中心逆向投影到纹理坐标空间，从而得到某个纹理坐标对应的纹理值。这种方法虽然很简单，但是会受到走样问题的影响，对于周期性纹理，这种走样尤为严重。图8.18说明了这个走样问题。该图有一个周期性纹理和一个平整的表面。每个像素的中心点在纹理坐标空间的逆向投影正好位于两条暗线之间，因此对应的纹理值总是浅色值。更一般的情况是：如果不考虑像素的有限大小，那么会使得到的图像出现网纹状走样。一种更好的但也是更难实现的方法是计算原像周围对应的纹理值的平均值，并把它作为像素的颜色值。注意，这种方法也不是很完美的方法。对于图8.18所示的例子，我们可以把平均纹理值作为像素的颜色值，但是仍得不到原始条带状的纹理图案。由于帧缓存和纹理都只有有限大小的分辨率，所以最终得到的图像仍然存在走样瑕疵。当纹理包含规则的高频成分时，这些问题将更加突出。

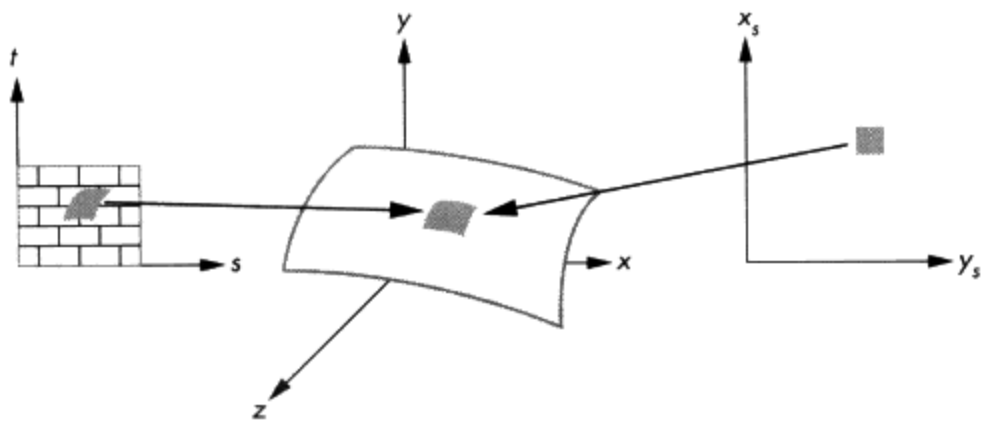


图8.17　弯曲的像素原像

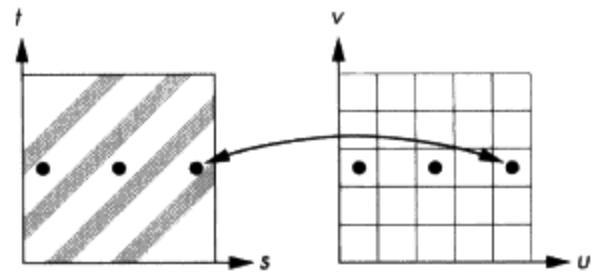


图8.18　纹理生成过程的走样现象

现在回过头来讨论映射问题。在计算机图形学中，大部分曲面都是用参数表示的。曲面上任意一个点$\mathbf{p}$是参数u和v的函数。对于任意一对参数值，可以得到曲面上的一个点：

$$\mathbf{p}(u, v) = \begin{bmatrix} x(u, v) \\ y(u, v) \\ z(u, v) \end{bmatrix}$$

第12章将详细介绍这种曲面的推导过程。如果已知参数曲面，那么可以根据如下形式的线性映射关系把纹理空间中的任意一个纹素$T(s, t)$映射到曲面$\mathbf{p}(u, v)$上的某个点：

$$u = as + bt + c$$
$$v = ds + et + f$$

只要$ae \neq bd$，该映射就是可逆的。线性映射使我们可以很容易地把纹理映射到参数曲面的一组片面上。例如，如图8.19所示，如果由左下角$(s_{\min}, t_{\min})$和右上角$(s_{\max}, t_{\max})$确定的纹理面片对应于左下角为$(u_{\min}, v_{\min})$、右上角为$(u_{\max}, v_{\max})$的曲面面片，那么从纹理面片到曲面片面的映射关系为：

$$u = u_{\min} + \frac{s - s_{\min}}{s_{\max} - s_{\min}}(u_{\max} - u_{\min})$$

$$v = v_{\min} + \frac{t - t_{\min}}{t_{\max} - t_{\min}}(v_{\max} - v_{\min})$$

这个映射公式虽然很容易使用，但是它并没有考虑表面的弯曲情形。对于弯曲的曲面，大小相等的纹理面片必须拉伸才能覆盖对应的曲面面片。

解决映射问题的另一种方法是使用两步映射。第一步映射是把纹理映射到一个简单的三维中间表面上，如球面、圆柱面或立方体表面。第二步映射再把带有映射纹理的中间表面映射到需要绘制的对象表面上。这种两步映射过程既适用于几何对象表面，又适用于参数对象表面。无论是几何对象表面，还是参数对象表面，下面的例子在本质上是相同的。

假定纹理坐标的取值范围在单位正方形内，并且使用高为 h、半径为 r 的圆柱面作为纹理映射的中间表面，如图 8.20 所示。圆柱面上的任意一点可以用下面的参数方程表示：

$$x = r\cos(2\pi u)$$
$$y = r\sin(2\pi u)$$
$$z = v/h$$

其中，u 和 v 的取值范围都是(0, 1)。因此，可以使用下面的映射公式：

$$s = u$$
$$t = v$$

如果纹理映射只考虑圆柱体的侧面，而不考虑上、下两个底面，那么我们能够无失真地映射纹理。然而，如果把纹理映射到像球面这样的封闭对象表面上，那么一定会引入形状失真。这个问题类似于使用地球的二维图像表示地图存在的问题。如果仔细观察地图集中的各种地图，读者会发现其形状和距离都存在失真。无论是纹理映射还是其他映射设计方法都必须根据我们希望失真出现的部位来选择某种表示方法。例如，众所周知的墨卡托投影（Mercator projection）将大部分失真集中在两极位置。如果使用半径为 r 的球面作为纹理映射的中间表面，那么可能的映射关系为：

$$x = r\cos(2\pi u)$$
$$y = r\sin(2\pi u)\cos(2\pi v)$$
$$z = r\sin(2\pi u)\sin(2\pi v)$$

我们还可以使用矩形盒子作为纹理映射的中间表面，如图8.21所示。在该图中，我们把纹理映射到一个没有被打开的盒子上，就像用一张纸板包装一个盒子。我们通常把这种映射方法用于环境映射（见 8.10 节）。

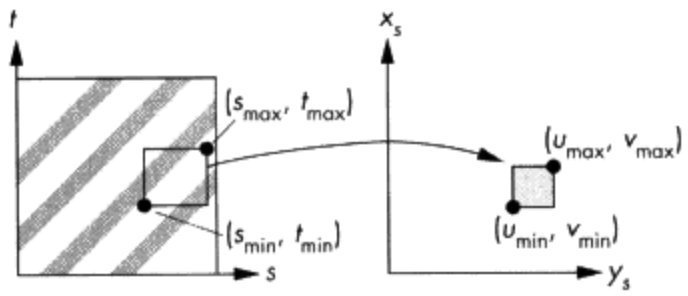


图 8.19　线性纹理映射

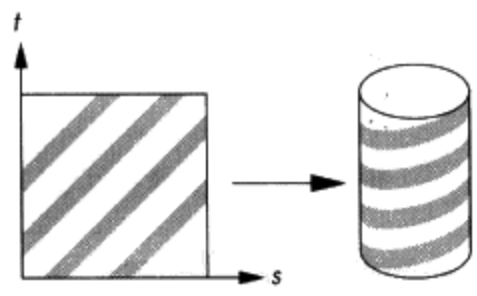


图 8.20　把纹理映射到圆柱面

第二步映射是把中间对象表面上的纹理值映射到要绘制的对象表面上。图8.22说明了3种可能的方法。第一种方法如图8.22(a)所示，我们取中间对象表面上某点的纹理值，沿着该点的法向量方向直到与我们要绘制的对象相交，然后把中间对象表面上的纹理值作为该交点的纹理值。我们还可以采用与第一种方法相反的方向计算对象表面的纹理值，即从要绘制的对象表面上的某个点出发，并沿该点的法向量方向直到与中间对象相交，从这个交点可以得到对象表面的纹理值，如图8.22(b)所示。第三种可选的方法是：如果知道绘制对象的中心，那么可以从该中心位置到对象表面上某一点画一条直线，然后计算该直线与中间对象表面的交点，如图8.22(c)所示。把直线与中间对象表面交点处的纹理值赋给绘制对象表面相应的点。

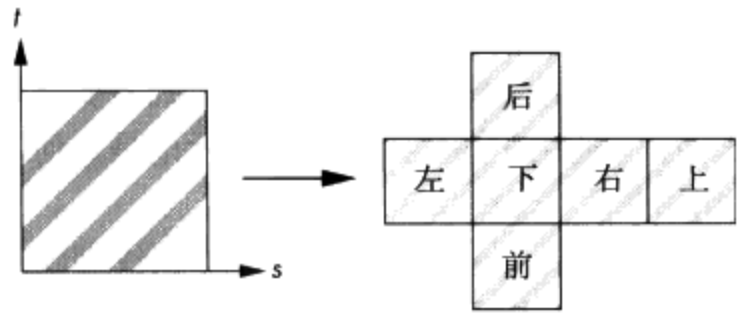


图 8.21　把纹理映射到矩形盒子上

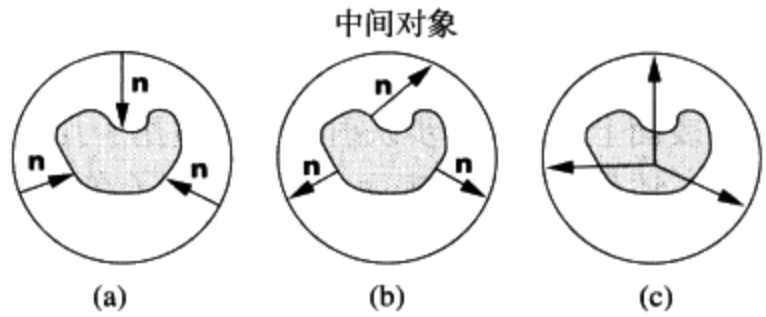


图 8.22　第二步映射：(a) 利用中间对象表面的法向量；(b) 利用对象表面的法向量；(c) 利用对象的中心

8.8　OpenGL 中的纹理映射

OpenGL支持各种纹理映射选项。OpenGL 1.0的纹理映射功能只能将一维和二维纹理映射到一维到四维图形对象表面上。目前三维纹理映射已成为OpenGL功能的一部分，且得到了大部分图形硬件的支持。由于二维纹理映射需要设置许多选项和参数，所以我们重点介绍OpenGL的二维纹理映射功能。

OpenGL的纹理映射与它的绘制流水线结构有关。目前已经知道，OpenGL实际上有两条平行的流水线，即几何绘制流水线和像素流水线。对于位图和像素矩形块的显示，这两条流水线在光栅化阶段合二为一。对于纹理映射，像素流水线与几何绘制流水线中位于光栅化后的片元处理单元合二为一，如图8.23所示，这种结构决定了OpenGL支持的纹理映射类型。尤其要注意的是：纹理映射作为片元处理的一部分执行。之后通过z-buffer算法对片元处理单元生成的每个片元进行可见性测试。我们可以把纹理映射视为明暗着色处理的一部分，但是该部分的处理过程是基于片元一级进行的。纹理坐标与法向量和颜色的处理方式非常类似。它们都是通过OpenGL的状态变量与顶点相关联，然后在多边形表面上对顶点的纹理坐标进行插值计算，从而得到多边形表面上其他位置的纹理坐标。

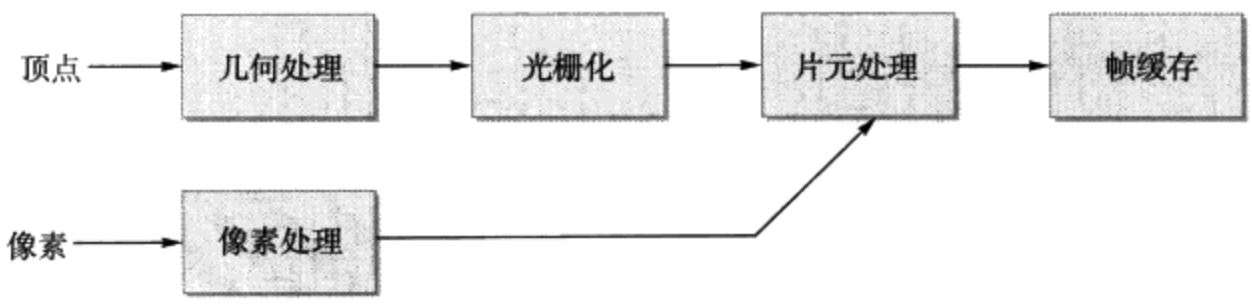


图 8.23　像素流水线和几何绘制流水线

8.8.1　二维纹理映射

二维纹理映射首先要定义一个纹素（texel）数组，它与二维像素矩形块的定义相同。假定我们定义了一个512 × 512的图像my_texels，该图像的数据可以由程序生成，也可能是从图像文件中读取：

```
GLubyte my_texels[512][512][3];
```

利用下面的函数把该数组指定为一个二维纹理（通常作为初始化的一部分）：

```
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, 512, 512, 0,
             GL_RGB, GL_UNSIGNED_BYTE, my_texels);
```

更一般的情况是使用下面的函数指定二维纹理：

```
glTexImage2D*(GLenum target, GLint level, GLint iformat,
    GLsizei width, GLsizei height, GLint border, GLenum format
    GLenum type, GLvoid *tarray)
```

我们可以使用target参数选择单个图像，如上面的例子所示，也可以用来设置立方体贴图（见8.10节），或者用来测试是否有足够的纹理内存用来存储纹理图像。level参数用于mipmapping映射，level等于0表示使用mipmapping多分辨率纹理图像的最高级，或者表示不使用mipmapping映射，即使用原始纹理图像。参数format与参数type[①]用来说明像素阵列在处理器内存中的存储方式，像素阵列的读/写操作函数glReadPixels和glWritePixels也用到了这两个参数，这些函数参数的相似性也是预料之中的，因为纹素阵列最初以像素阵列的形式存储在处理器内存中，并流经像素流水线。然而，通过函数glTexImage2D处理之后的像素流经像素流水线，最终存储在纹理内存中，纹理内存并不是帧缓存的一部分。在目前最新的图形显示系统中，这三种内存，即处理器内存、帧缓存和纹理内存在物理上通常是独立的，每种内存都有自己的特性和局限性。

我们可以使用参数level和border精确地控制纹理的处理方式。后面会讨论这些参数。与其他选项的设置相同，我们还必须启动纹理映射：

```
glEnable(GL_TEXTURE_2D);
```

设置纹理映射的第二个步骤是确定将纹理映射到几何对象表面上的方式。OpenGL使用s和t表示纹理坐标，它们在纹理图像上的取值范围都是(0.0, 1.0)。对于本例，（0.0, 0.0）位置的纹理值对应于纹素my_texels[0][0]，而(1.0, 1.0)位置的纹理值对应于纹素my_texels[511][511]，如图8.24所示。单位区间内的任意一个s和t的纹理值对应于一个唯一的纹素。因为应用程序可以指定单位区间之外的纹理值，所以OpenGL提供了两个选项用来处理这样的纹理值，我们马上就会讨论这个问题。

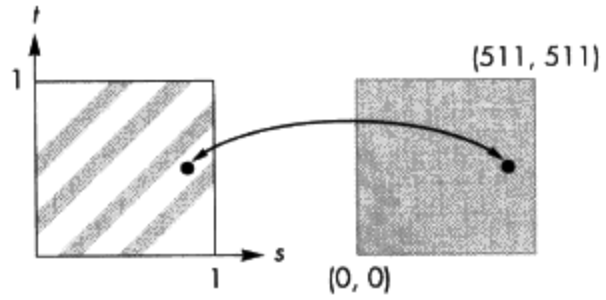


图8.24　纹理坐标与纹素数组之间的映射

OpenGL把s和t的值作为系统状态的一部分，这样可以利用应用程序来确定顶点的纹理坐标。我们使用下面的函数设置顶点的纹理坐标：

```
glTexCoord2f(s,t);
```

因此，我们可以使用类似于处理颜色和法向量的方法来处理纹理坐标。当处理一个顶点的时候，OpenGL使用当前的纹理坐标作为该顶点的纹理坐标。如果需要改变顶点的纹理坐标，那么必须在定义顶点之前重新设置纹理坐标。例如，如果要把定义好的纹理映射到一个四边形上，可以使用下面的代码：

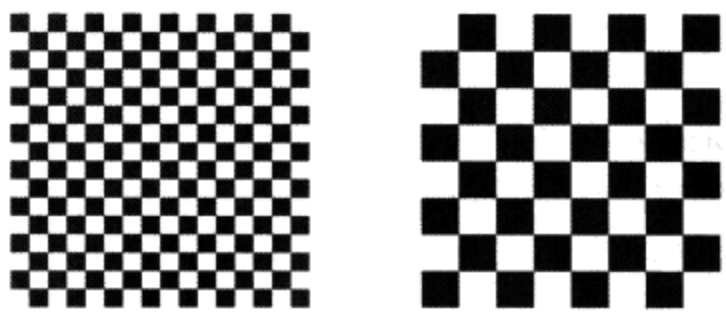
```
glBegin(GL_QUAD);
 glTexCoord2f(0.0,0.0);
 glVertex3f(x1, y1, z1);
 glTexCoord2f(1.0,0.0);
 glVertex3f(x2, y2, z2);
 glTexCoord2f(1.0,1.0);
 glVertex3f(x3, y3, z3);
```

① 原文为第三个参数和第四个参数，可能有误。——译者注

```
    glTexCoord2f(0.0,1.0);
    glVertex3f(x4, y4, z4);
  glEnd();
```

在定义每个顶点之前，还可以给顶点设置新的法向量和颜色。就像OpenGL对多边形顶点颜色进行双线性插值从而确定多边形内部颜色一样，OpenGL也使用同样的方法通过插值计算确定多边形内部点的纹理坐标。

在如图8.25(a)所示的例子中，我们把整个纹理映射到一个矩形上。如果只使用s和t的一部分取值范围，如(0.0, 0.5)，那么只使用my_texels数组中的部分纹素用于纹理映射，得到如图8.25(b)所示的图像。OpenGL在整个四边形上对s和t进行插值计算，然后把这些计算结果逆映射到my_ texels数组中对应的纹素上。这个四边形纹理映射的例子很简单，因为纹理坐标和顶点之间的映射关系很明显。对于一般的多边形，应用程序编程人员必须确定几何对象顶点的纹理坐标。图8.26显示了使用同一个纹理进行纹理映射的可能情形。图8.26(a)和图8.26(b)使用同一个三角形，但使用了不同的纹理坐标。注意图8.26(c)中由于插值计算导致的失真，并注意梯形被当做两个三角形进行绘制的情形。



(a)　　(b)

图8.25　把棋盘纹理映射到一个四边形上：(a) 使用整个纹素数组；(b) 使用部分纹素数组

(a)　　(b)　　(c)

图8.26　把纹理映射到多边形上：(a)和(b) 把棋盘纹理映射到一个三角形上；(c) 把棋盘纹理映射到一个梯形上

OpenGL纹理映射的基本过程很简单：定义一个颜色数组作为纹理值，然后设置顶点的纹理坐标。遗憾的是，在我们能够有效地使用纹理之前，必须讨论几个令人头疼的问题。要解决这些问题，需要权衡图像的质量与效率之间的关系。

其中的一个问题是：当纹理坐标s或t的取值位于范围(0.0, 1.0)之外时，如何解释s或t的取值呢？一般来说，如果s或t的取值在范围(0.0, 1.0)之外，我们要么重复（repeat）映射该纹理，要么把(0.0, 1.0)范围之外的纹理坐标值截取（clamp）为0.0或1.0，也就是说，把小于(0.0, 1.0)的纹理坐标值或大于(0.0, 1.0)的纹理坐标值分别表示为0.0和1.0。对于重复纹理映射的方式，使用如下的方式设置相应的参数：

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
```

对于纹理坐标t，使用参数GL_TEXTURE_WRAP_T；对于截取映射的方式，使用参数GL_CLAMP_T。

8.8.2　纹理采样

纹理走样是我们要讨论的一个主要问题。当把纹理坐标映射到纹素数组时，我们正好得到对应纹素的中心位置，这种情况很少出现。解决这个问题的一种方法是：从纹素数组中取这样一个纹素值，该纹素的位置最佳逼近通过双线性插值计算得到的纹素位置。我们把这种方法称为点采样（point sampling），这是一种最容易产生走样误差的方法。解决该问题的另一种更好的但计算量也更大的方法是对于使用上述点采样方法得到的纹素，我们对最靠近该纹素的周围4个纹素计算加权平均值，并把该值作为纹理坐标映射到纹素数组的纹素值。这种方法称为线性滤波（linear filtering）。在图8.27中可以看到一个位于纹素内部的采样点及其相邻的4个纹素，通过对顶点纹理坐标进行双线性插值来确定采样点的位置，而根据采样点相邻的纹素值来计算该点平滑的纹素值。如果使用线性滤波，那么在处理纹素阵列的边缘时会出现问题，因为我们需要在纹素阵列之外增加额外的纹素值。我们可以这样解决该问难，即将函数 glTexImage2D 中的 border 参数设置为 1，从而在纹素阵列的周围增加 1 个纹素宽度的边缘。因此，如果纹理存在边缘，那么纹理阵列包含的纹素个数为 $(2^m + 2) \times (2^n + 2)$。

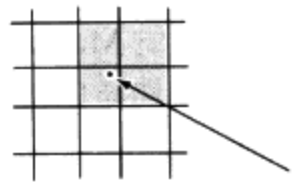


图 8.27　使用线性滤波方法处理的纹素

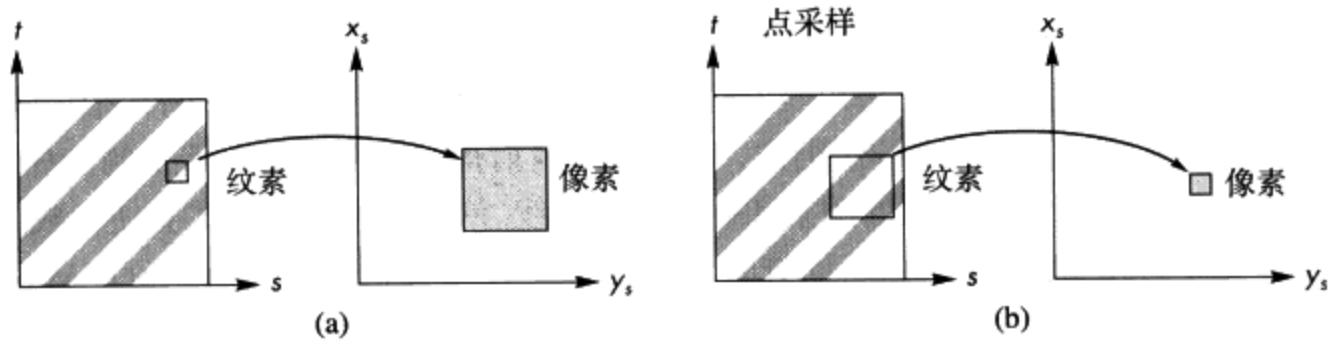


图 8.28　纹素到像素的映射关系：(a) 放大；(b) 缩小

然而更麻烦的是：我们还需要确定纹素值到纹理值的映射关系。如图 8.28 所示，屏幕上需要着色的像素可能比一个纹素大，也可能比一个纹素小。

对于第一种情况，屏幕上多个像素对应于单个纹素（即放大）；对于第二种情况，屏幕上单个像素对应于多个纹素（即缩小）。对于这两种滤波方法（放大滤波和缩小滤波），计算速度最快的滤波方式就是使用最佳逼近点采用的值[①]。我们可以使用下面的函数为纹理的放大和缩小滤波设置相关的纹理控制选项：

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
```

此外，还可以使用滤波方式参数 GL_LINEAR 代替 GL_NEAREST，从而可以通过线性滤波的方式得到颜色更平滑、走样误差更小的图像。

此外还可以使用 OpenGL 的另一个纹理滤波方式来处理纹理缩小问题，即 Mipmapping 技术。如果对象投影到屏幕某个区域的大小与纹素阵列的大小相比很小，则不必使用原纹素阵列的分辨

① 即把下面的纹理控制函数 glTexParameteri 中的滤波方式参数设置为 GL_NEAREST。——译者注

率显示该对象。我们可以使用OpenGL生成一系列大小递减的纹素阵列（即Mipmap层级纹理），然后OpenGL会自动选择一个大小合适的纹素阵列。对于一个大小为64 × 64的原始纹素阵列，可以使用OpenGL中的GLU函数设置大小为32 × 32、16 × 16、8 × 8、4 × 4、2 × 2和1 × 1的纹素阵列：

```
gluBuild2DMipmaps(GL_TEXTURE_2D, GL_RGB, 64, 64, GL_RGB,
   GL_UNSIGNED_BYTE, my_texels);
```

除此之外，我们还可以直接使用函数glTexImage2D中的level参数设置某个相应大小的纹素阵列。参数level表示某个纹理阵列的Mipmap层级纹理中的某一级。因此，第0级对应原始纹理图像；第1级对应的纹理图像，其分辨率为上一级纹理图像的一半，以此类推。然而，我们可以在每次调用函数glTexImage2D时用一个指针参数指向任意的纹素图像，因此在Mipmap层级纹理的不同层级上可以是完全不同的图像。如果调用下面的函数，OpenGL会自动控制Mipmap纹理的滤波方式：

```
glTexParameteri(GL_TEXTURE_2D,
    GL_TEXTURE_MIN_FILTER,GL_NEAREST_MIPMAP_NEAREST);
```

该函数把Mipmap纹理的滤波方式设置为GL_NEAREST_MIPMAP_NEAREST，即要求OpenGL使用点采样方式得到相邻的Mipmap纹理，并在得到的Mipmap纹理内部也使用点采样方式。我们还可以在通过点采样方式得到的相邻Mipmap纹理内部使用线性滤波方式(GL_NEAREST_MIPMAP_LINEAR)，或者使用线性滤波方式得到相邻的Mipmap纹理，并在得到的Mipmap纹理内部使用点采样方式(GL_LINEAR_MIPMAP_NEAREST)，或者都使用线性滤波的方式(GL_LINEAR_MIPMAP_LINEAR)。图8.29显示了分别使用点采样、线性滤波、基于点采样的Mipmapping以及基于线性滤波的Mipmapping这4种方式进行纹理映射的区别。绘制的对象是一个四边形，其对应的透视投影图看起来几乎就是一个三角形。纹理是一组黑白相间的平行条纹，似乎在四边形的远端集中于一点。我们注意到，由于纹理的规则性，通过纹理映射得到的图像具有严重的走样现象。使用点采样方式映射纹理会产生波纹状的图案和锯齿状的直线。使用线性滤波方式会使生成的直线更光滑，但是仍有明显的波纹状图案。由于滤波的作用，黑白条纹之间的纹素是灰色的。Mipmapping映射方式也会使用灰色的颜色代替这种黑白两色图案上的黑白颜色，该灰色的颜色值是黑白两种颜色值的平均值。对于离观察者最远的那部分对象，它的纹素颜色是灰色的，并且和背景融合在一起。在特定的Mipmap纹理中使用点采样的Mipmapping映射方式仍出现了锯齿现象，但是当我们使用线性滤波Mipmapping映射方式时，这种锯齿现象就被消除了。目前图形处理单元（GPU）速度的日益提升以及GPU内置的大容量纹理内存使得应用程序可以使用纹理滤波及Mipmapping技术而不会影响系统的性能。

在OpenGL中使用纹理的最后一个问题是纹理与绘制对象表面颜色的相互作用。对于RGB颜色模型，存在多种纹理映射方式。其中的一种纹理映射方式是：在进行纹理映射之前，先计算对象表面的明暗色调值，然后把纹理的颜色分量分别与明暗色调值的颜色分量相乘，从而通过纹理调制对象表面原有的颜色。这种纹理调制的方法是OpenGL默认的纹理映射方式，可以通过下面的函数设置这种映射方式：

```
glTexEnvi(GL_TEX_ENV, GL_TEX_ENV_MCDE, GL_MODULATE);
```

如果用GL_DECAL替换GL_MODULATE，那么对象的颜色完全由纹理的颜色决定。我们把这种方法称为贴花方法（decaling）。

真正的纹理映射还与投影类型有关。一般来说，OpenGL在屏幕空间使用双线性插值计算某点的纹理值。对正投影来说，这种双线性映射可以得到正确的纹理值，但是对透视投影来说，由于深

度缩放是非线性的，所以无法得到正确的结果。我们可以让OpenGL在以时间开销为代价的前提下使用一种更好的插值方法（如果OpenGL的实现支持该方法），调用的函数如下所示：

```
glHint(GL_PERSPECTIVE_CORRECTION, GL_NICEST);
```

对于目前的图形卡来说，这种正确的透视插值方法受GPU支持并且是默认的设置。但是这种正确的透视插值方法应该避免出现图8.29中的问题。

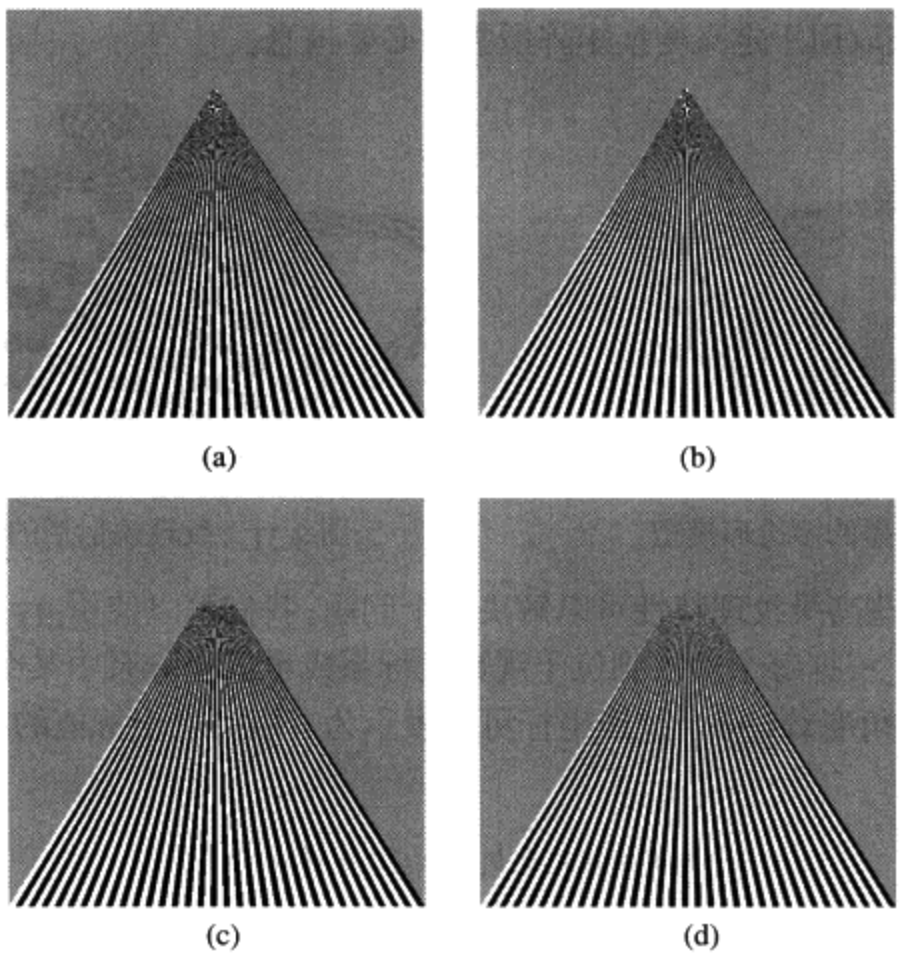


图8.29 把纹理映射到四边形上：(a) 点采样；(b) 线性滤波；(c) Mipmapping 点采样；(d) Mipmapping 线性滤波

8.8.3 使用纹理坐标

目前所举的例子都隐含这样一个假设，即已经知道了如何给对象的顶点赋纹理坐标。如果我们要处理一组大小相同的矩形多边形，那么给它们的顶点赋纹理坐标是非常简单的。我们还可以充分利用这样一个事实，即就像顶点坐标一样，纹理坐标也可以使用四维数组来存储。我们可以使用4×4的纹理变换矩阵对纹理坐标进行变换操作。纹理矩阵初始化为单位矩阵，并且对它的处理方式与模-视变换矩阵和投影变换矩阵的处理方式相同。如果要使用纹理变换矩阵，那么首先需要按下面的方式设置当前变换矩阵：

```
glMatrixMode(GL_TEXTURE);
```

之后，我们可以使用纹理变换矩阵对纹理进行缩放和定位等操作，从而使纹理能够与几何对象、照相机或光源一起运动。

然而，如果用一组多边形近似表示一个曲面，那么对这些多边形顶点进行纹理坐标的赋值并非易事。下面分析如图8.30所示的犹他大学茶壶[①]，它是用多边形建立的近似模型。虽然该模型只使用了四边形来建模，但是这些四边形的大小不尽相同，在曲率较大的表面区域上的四边形比较小，而在比较平坦表面区域上的四边形比较大。图8.31显示的是在没有对多边形的不同大小进行调整的

① 第12章将详细讨论犹他大学茶壶的绘制。

情况下把棋盘纹理映射到茶壶表面的效果。正如从图中看到的，把一组相同的纹理坐标赋给每个多边形的顶点时，为了适应大小不同的多边形，纹理映射过程会根据需要对纹理的大小进行缩放调整。因此，像在茶壶柄这样的区域，需要使用较多的小多边形来更好地近似表示这部分曲面，与茶壶的体部相比，这部分区域上的黑白矩形比较小。在某些应用中，这样的映射结果是可以接受的。然而，如果茶壶的表面都使用同一种材质，那么我们希望看到茶壶表面的所有部分显示相同的图案。在理论上，可以使用纹理变换矩阵对纹理坐标进行缩放操作，从而得到需要的效果，然而在实际中，几乎不可能根据模型来获取用于定义变换矩阵所需的必要信息。

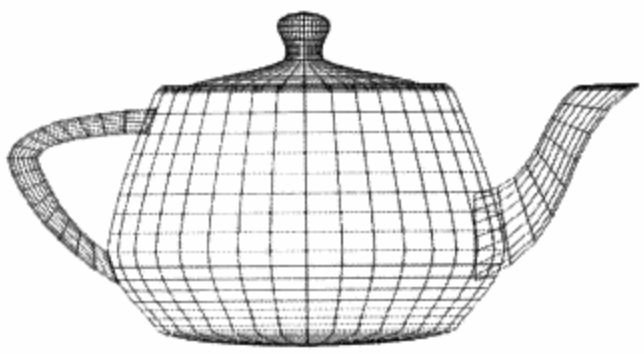


图 8.30　犹他大学茶壶的多边形模型

图 8.31　纹理映射后的犹他大学茶壶

OpenGL提供的一些高级纹理特性可以解决这个问题。我们可以使用glTexGen函数来自动生成纹理坐标，即让OpenGL根据每个顶点到位于观察坐标系或世界坐标系中某个平面的距离来计算它们的纹理坐标。从数学角度看，每个纹理坐标可以表示为一组齐次坐标值的线性组合。因此，对于纹理坐标 s 和 t：

$$s = a_s x + b_s y + c_s z + d_s w$$
$$t = a_t x + b_t y + c_t z + d_t w$$

我们把上式中的系数表示为数组，并通过下面的 glTexGen 函数把它们传送给 OpenGL：

```
GLfloat planes[ ] = {0.5, 0.0, 0.0, 0.5}; /* s =x/2+1/2 */
GLfloat planet[ ] = {0.0, 0.5, 0.0, 0.5}; /* t =y/2+1/2 */
glTexGeni(GL_S, GL_TEXTURE_GEN_MODE, GL_OBJECT_LINEAR);
glTexGeni(GL_T, GL_TEXTURE_GEN_MODE, GL_OBJECT_LINEAR);
glTexGenfv(GL_S, GL_OBJECT_LINEAR, planes);
glTexGenfv(GL_T, GL_OBJECT_LINEAR, planet);
```

前两个 TexGen 函数调用语句指定了纹理坐标和平面所在的空间坐标系，OpenGL 据此可以计算多边形的每个顶点到该平面的距离。后两个函数调用语句提供了纹理坐标的线性组合系数。我们还必须为每个纹理坐标开启自动纹理生成功能：

```
glEnable(GL_TEXTURE_GEN_S);
glEnable(GL_TEXTURE_GEN_T);
```

图8.32(a)显示的是使用GL_OBJECT_LINEAR选项，即在世界坐标系中使用纹理坐标自动生成功能绘制的茶壶。图 8.32(b)使用相同的平面方程，但在观察坐标系（GL_EYE_LINEAR）中使用纹理坐标自动生成功能绘制茶壶。通过在世界坐标系中的计算，纹理和对象关联在一起，因此纹理会随着对象一起旋转。如果使用观察坐标系，那么当我们对对象进行几何变换时，纹理图案也会随之变化，对象看起来好像穿过了纹理场。这种技术的一个重要应用就是地形的生成和地形纹理的生成。我们还可以把表面的特征作为纹理直接映射到三维网格上。

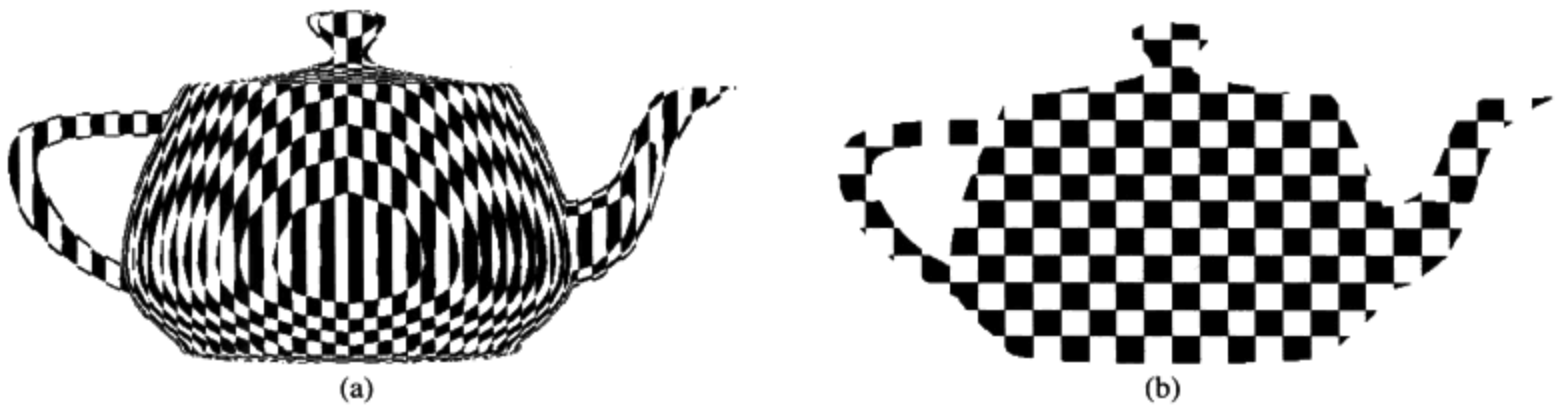


图 8.32 使用纹理坐标自动生成功能绘制的茶壶：(a) 在世界坐标系；(b) 在观察坐标系

8.8.4 纹理对象

在 OpenGL 中，每一类纹理（一维、二维或三维纹理）都存在一个唯一的当前纹理，它作为 OpenGL 状态的一部分。因此，如果需要用到多个纹理（例如，可能需要将不同的纹理映射到同一场景中的不同对象表面上），那么每当要使用另一个纹理时，就要调用函数 glTexImage2D，这种处理方法的效率很低。每当执行函数 glTexImage2D 时，另一个纹理图像被载入内存并把原来的纹理替换掉。即使有足够大的纹理内存（纹理映射需要使用大量的内存），如果使用函数 glTexImage2D，这个问题仍然存在。

OpenGL 1.1 引入了纹理对象（texture object）的概念，我们可以在应用程序中定义这样的纹理对象，它包括纹理阵列和各种纹理参数，通过这些参数可以控制纹理映射到对象表面的方式。只要有足够的内存来存储纹理阵列，那么即使当前纹理改变了，这些纹理对象仍然驻留在纹理内存中。如果没有足够大的纹理内存，那么应用程序可以定义纹理对象的优先级，这样就可以控制从纹理内存删除纹理对象的顺序。

如果要建立一个二维纹理对象，首先要调用函数 glBindTexture，代码如下：

```
GLuint my_texture = 1;
glBindTexture(GL_TEXTURE_2D, my_texture);
```

位于glBindTexture后面的其他纹理函数用来定义纹理图像及纹理参数。当使用新的纹理对象名再次调用glBindTexture函数时，可以定义一个新的纹理对象。当在后面使用一个已定义好的纹理对象名调用函数 glBindTexture 时，表示把该纹理对象设置为当前纹理对象。与显示列表一样，OpenGL 也提供了用来查找未使用纹理对象名的函数（glGenTexture）和删除纹理对象的函数（glDeleteTexture）。

8.8.5 多通道纹理映射

到目前为止，我们只讨论了把单个纹理映射到对象表面上的情形。然而，如果使用多个纹理进行绘制，那么许多表面的绘制效果可能会实现得更好些。例如，假定我们希望给对象增加阴影效果，而该对象表面的颜色本身又是由纹理映射决定的，则可以使用纹理映射实现阴影效果。但是如果绘制系统每次只能应用单个纹理，则无法使用这种方法。

但是，如果我们使用的图形系统支持如图 8.33 所示的多纹理单元，那么就可以完成这个任务。纹理单元序列是一个流水线结构，其中的每个纹理单元作为一个独立的纹理映射阶段，其输入是前一个纹理单元的输出。最近版本的 OpenGL 支持这个功能。

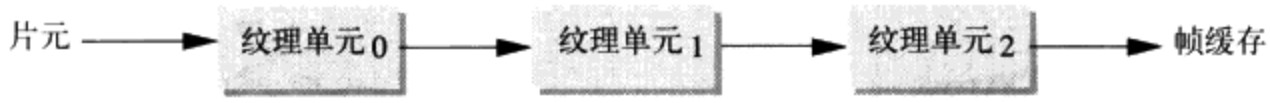


图 8.33　纹理单元序列

假定希望使用两个纹理单元。我们可以在程序的初始化阶段定义两个纹理对象，然后依次激活每个纹理单元并确定每个纹理对象的映射方式。通常采用如下形式的代码：

```
glActiveTexture(GL_TEXTURE0); /* 纹理单元0 */
glEnable(GL_TEXTURE_2D);
glBindTexture(GL_TEXTURE_2D, object0);
glTexEnvironment(...); /* 纹理对象0的映射方式 */
glActiveTexture(GL_TEXTURE1); /* 纹理单元1*/
glEnable(GL_TEXTURE_2D);
glBindTexture(GL_TEXTURE_2D, object1);
glTexEnvironment(...); /* 纹理对象1的映射方式 */
```

每个纹理单元可以使用不同的纹理坐标。我们可以在应用程序中使用glMultiTexCoord*函数为每个纹理单元设置纹理坐标。对于上面的例子，可以使用以下形式的代码为每个纹理单元定义纹理坐标：

```
GLfloat point[3], s, t, ss, tt;

glMultiTexCoord2f(GL_TEXTURE0, s, t);
glMultiTexCoord2f(GL_TEXTURE1, ss, tt);
glVertex3fv(point);
```

8.9　纹理生成

纹理映射最大的用处之一是不用生成大量的几何对象就可以为对象表面提供复杂的特征信息。高端图形系统可以实时地处理二维纹理映射，即对于生成的每一帧画面，纹理作为绘制过程的一部分映射到对象表面上，其绘制速度几乎与没有纹理映射的绘制过程相同。目前个人计算机的图形显卡（即GPU）配有大量的纹理内存，这使得游戏开发人员能够利用纹理映射创建复杂的动态环境效果。

例如，如果我们希望模拟场景中的草地，那么可以把一幅草地的图像映射到地面上，通过扫描照片就可以获取草地的图像，这比通过几何建模的方式建立看起来像草地一样的二维或三维几何对象要快得多。在纹理映射应用程序中，我们并不生成地形表面的真实细节，而是通过纹理映射把真实的数字化纹理图像映射到三维地形模型的表面上。

我们还可以使用过程建模方法来生成纹理图案。其中尤其令我们感兴趣的是自然界中的一些图案，如沙漠、草地或矿石的纹理。这些纹理既表现出一定的结构性（规则图案），又表现出大量的随机性。生成这种纹理的大多数方法都是从一个随机数生成器开始，并使用算法对它的输出（噪声）进行处理，如图 8.34 所示。第 11 章将详细介绍过程噪声。

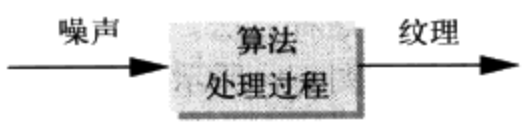


图 8.34　纹理生成

三维纹理场$T(s, t, r)$的生成是二维纹理生成技术的直接扩展。在实际应用中使用三维纹理有某些优点。最重要的是：可以把纹理坐标(s, t, r)对应的纹理值直接映射到三维对象上的(x, y, z)坐标点，因此可以完全避免二维纹理映射带来的问题，应用程序编程人员只需使用所需的纹理属性定义

函数 $T(s, t, r)$。从理论上来看，三维纹理映射的过程类似于在一个实心木块上雕刻一个三维对象，这个实心木块空间的颜色由所映射的三维纹理确定。因此这种三维纹理映射技术可用来生成这样一类对象，它们看起来好像是从岩石中雕刻出来的。纹理生成过程就是定义函数 $T(s, t, r)$，可以通过该函数显示像大理石和花岗岩等材质的纹路。

使用三维纹理还具有某些其他优点。假定我们有一个二维纹理，它是通过拍照或从自然界材质（如石头）模型得到的。现在假定我们希望建立一个立方体，这个立方体看起来像是用石头材质制作的。如果使用二维纹理映射，那么必须把同一个纹理图案映射到立方体的6个面上。为了使这个立方体看起来更真实，还必须使纹理图像在立方体的边缘（即两个纹理相交的地方）上看起来是连续变化的。当使用自然材质的二维纹理图案时，实际上不可能在立方体的边缘上得到连续匹配的图案。注意，在立方体的顶点处，因为这是3个纹理相交的地方，所以问题更严重（见习题8.24）。我们通常采用滤波技术及纹理边缘的特殊处理来得到视觉上可接受的效果。然而，如果使用三维纹理，那么就不会产生这些问题。

8.10 环境贴图

高反射表面具有镜面反射的特性，它可以反射周围的环境。例如，考虑位于房间中央的一个有光泽的金属球，我们可以在金属球表面上看到房间里东西的变形图像。很明显，这种效果需要用到全局光照信息，因为如果不考虑其他的场景，我们就无法正确地绘制这个金属球。基于物理的绘制方法（如光线跟踪方法）可以生成具有这种效果的图像，然而在实际中，光线跟踪所需的计算非常耗时，以至于无法用于实时应用中。然而，我们可以使用环境或反射贴图并使用各种纹理映射技术来产生视觉上可以接受的近似效果。

下面分两部分来讨论这个问题。在介绍基本概念之后，我们将分析立方体贴图和球体贴图，它们都得到了OpenGL标准纹理映射的支持。我们通过多边形每个顶点的纹理坐标来映射这些贴图。在第9章，当介绍了如何编写片元程序之后，将介绍如何使用片元生成更精确的贴图以及如何生成凹凸贴图等更复杂的效果，这种效果在固定功能的绘制流水线上是无法得到的。

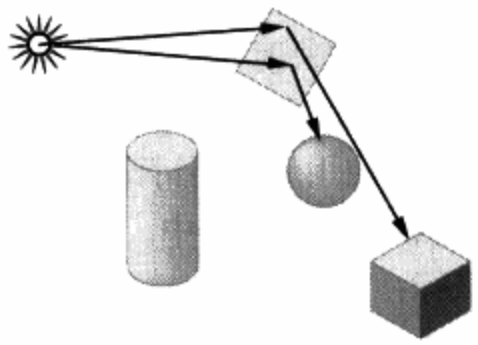


图 8.35 一个包含镜面的场景

环境贴图的基本思想很简单。下面分析如图8.35所示的镜面，可以把它视为具有高反射材质特性的多边形表面。从图形绘制系统的观点来看，如果观察者的位置和多边形的法向量都已知，那么根据第6章的公式可以计算光线的反射角。如果我们跟踪反射角的反射光线直到与场景中的对象相交为止，就可以得到镜面反射光线的强度（即镜面的颜色值）。当然，这个颜色值是多边形的明暗绘制结果，它表示场景中光源和多边形表面材质的相互作用。因此可以得到镜面颜色的近似值，它作为两步绘制过程的一部分，这里的两步绘制过程在某些方面类似于8.7节介绍的两步纹理映射过程。在第一遍绘制过程中，我们绘制的场景不包含镜面多边形，绘制的时候把照相机放在镜面中心并指向镜面的法向量方向。因此，我们绘制的场景对象图像就好像从镜子中“看到”的一样。这

个图像虽然不完全正确（见习题8.3），但足以令人满意。之后执行第二遍绘制过程，作为一个常规的绘制过程，它把照相机放回到场景中原来的位置，并使用前面得到的图像作为颜色值（纹理值）映射到镜面多边形上。

这种方法存在两个问题。第一，在第一遍绘制过程中得到的图像并不完全正确，因为生成这个图像时并没有考虑场景中的镜面对象。第二，必须解决映射问题。在第一遍绘制过程中，我们应该把场景投影到哪个表面上？又应该把照相机放在什么地方？我们很可能需要场景中的所有信息，因为在实际操作时可能要移动场景中的镜面对象，在这种情况下，我们会从镜面中看到场景不同部分的连续帧画面，因此，一个简单的投影无法满足这种情形。

有许多方法可以用来解决这个投影问题。典型的方法是把环境投影到位于投影中心的一个球面上。在图8.36中可以看到一些位于球面之外且要投影到该球面的多边形。注意，位于球心位置的观察者不能区分他们所看到的多边形是位于原来的位置还是位于球面的投影上，这种幻觉类似于在天文馆中看星星，看起来仿佛位于无限远处的“星星”实际上是光线在位于观众上方的半球面上的投影。

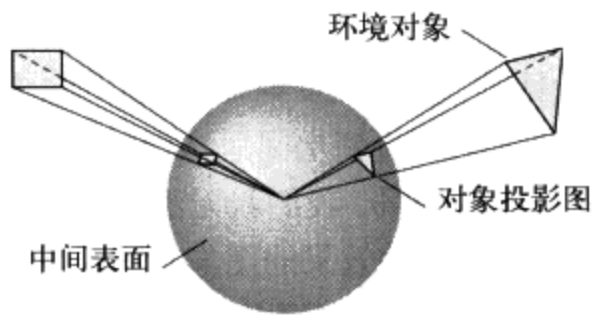


图 8.36　环境映射

在最初的环境映射中，球面被许多经度线和纬度线分割成矩形区域，纹理就映射到这些矩形块上。尽管这种方法理论上很简单，但是在球体的两极存在形状无限失真的问题。从计算的角度来看，这种映射方法并不能很好地避免形状上的失真，而且需要进行大量的三角函数计算。

OpenGL 支持一种称为球面映射（sphere mapping）的方法，它是上述方法的一种改进形式。应用程序提供一个圆环状图像，它是环境映射到球体表面后该球体的正投影图。这种方法的优点是从反射矢量到位于该圆环上的二维纹理坐标的映射非常简单，而且可以用硬件或软件实现。这种方法的难点在于获取所需的圆环状图像。我们可以用一些特殊的投影方法来得到这个圆环状图像的近似图像，如使用具有广角镜头的透视投影或使用后面要讨论的立方体投影等其他类型的投影方法。我们通过函数 glTexImage2D 把纹理图像载入纹理内存中。我们可以自己来生成纹理坐标，但更简单的做法是让 OpenGL 自动生成纹理坐标：

```
glTexGeni(GL_S, GL_TEXTURE_GEN_MODE, GL_SPHERE_MAP);
glTexGeni(GL_T, GL_TEXTURE_GEN_MODE, GL_SPHERE_MAP);

glEnable(GL_TEXTURE_GEN_S);
glEnable(GL_TEXTURE_GEN_T);
```

借助于图8.37，我们可以理解用来生成纹理坐标的方程。如果采用从观察者到纹理贴图的逆向处理方式，那么这可能是理解该方程推导过程最容易的方法。假定纹理贴图位于 $z = -d$ 平面上，其中 d 是一个正数，并且通过正投影把它投影到球心位于坐标原点的单位球面上。因此，如果该平面上的纹理坐标为 (s, t)，那么投影线与球面的交点坐标为 $(s, t, \sqrt{1.0 - s^2 - t^2})$。对于球心位于坐标原点的单位球体来说，球面上任意一点的坐标也正好是该点单位法向量的分量。然后我们就可以根据第 6 章介绍的方法计算该点的反射方向：

$$\mathbf{r} = 2(\mathbf{n} \cdot \mathbf{v})\mathbf{n} - \mathbf{v}$$

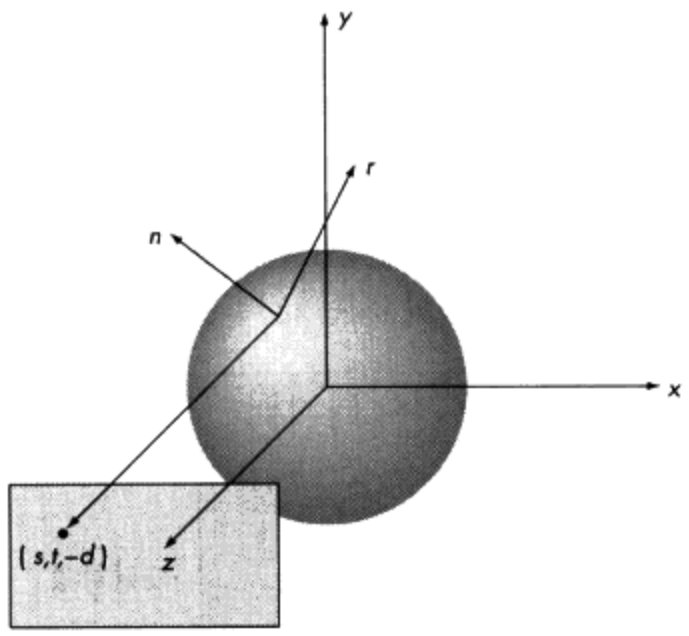


图 8.37　反射贴图

其中，

$$\mathbf{v} = \begin{bmatrix} s \\ t \\ 0 \end{bmatrix}$$

$$\mathbf{n} = \begin{bmatrix} s \\ t \\ \sqrt{1.0 - s^2 - t^2} \end{bmatrix}$$

矢量 $\mathbf{r}$ 指向环境，因此任何与 $\mathbf{r}$ 相交的对象都有纹理坐标(s, t)。然而，这个推导过程是逆向的，因为在实际计算纹理坐标时是从由顶点定义的对象出发的。已知 $\mathbf{r}$，可以计算 s 和 t，我们发现，如果

$$\mathbf{r} = \begin{bmatrix} r_x \\ r_y \\ r_z \end{bmatrix}$$

那么

$$s = \frac{r_x}{f} + \frac{1}{2}$$

$$t = \frac{r_y}{f} + \frac{1}{2}$$

其中，

$$f = 2\sqrt{r_x^2 + r_y^2 + (r_z + 1)^2}$$

如果把所有的对象都变换到观察坐标系，那么利用从原点到顶点的单位矢量 $\mathbf{v}$ 以及顶点的法向量 $\mathbf{n}$ 即可计算 $\mathbf{r}$。

上述处理过程所揭示的一些问题说明该方法只是一种近似方法。只有对于位于原点处的顶点，反射贴图才是正确的。在原理上，每个顶点都有自己的反射贴图。实际上，对象上的每个点都有自己的贴图，而不是通过插值计算得到的近似值。对象离原点越远，误差也就越大。尽管如此，在大多数情况下，反射映射可以得到视觉上可接受的结果，尤其是在电影和游戏的动画中。

如果希望利用图形系统来计算环境贴图，那么我们更愿意使用图形系统所支持的标准投影。对于房间这样的环境，用来生成环境贴图最自然的中间对象就是盒子。我们计算6个投影图，分别对应于四面墙、地面和天花板，计算投影图所用的6个虚拟照相机都位于盒子的中心，并且分别指向不同的方向。目前暂时把得到的这6幅投影图视为一个完整的环境贴图，并用如图8.38所示的方法从环境贴图中获取对象的纹理值。彩图33显示了Pixar动画工作室制作的一个动画短片“Geri's Game”中的一帧画面，其中Geri老头戴的眼镜镜片上的反射图案就是通过反射贴图生成的。反射贴图通过在盒子的6个面上投影计算得到（见彩图34），然后把它映射到Geri老头的镜片上。

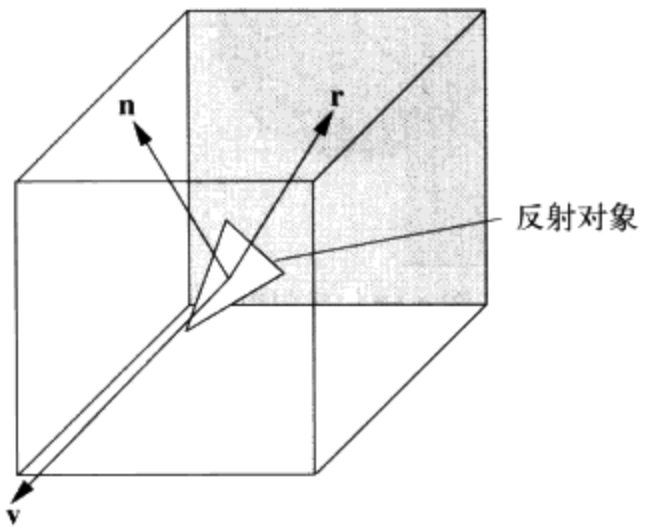


图8.38　反射立方体贴图

我们还可以在程序中计算这6幅图像，然后使用它们来计算OpenGL球面映射所需的圆环状图像。我们注意到所有这些方法都存在几何变形和走样的问题。此外，除非应用程序重新计算环境贴图，否则当观察者移动位置时就无法得到正确的结果。

不管图像是如何计算得到的，一旦得到了它们，就可以在OpenGL中通过6次函数调用来生成立方体贴图，立方体的中心位于坐标原点处，每次函数调用为立方体的一个面计算环境贴图。因此，如果我们有一幅512 × 512的RGBA图像imagexp，它作为立方体位于x轴正方向的那个面的贴图，我们可以调用下面的函数语句来实现：

```
glTexImage2D(GL_TEXTURE_CUBE_MAP_POSIITVE_X, 0, GL_RGBA, 512, 512,
             0, GL_RGBA, GL_UNSIGNED_BYTE, imagexp)'
```

对于反射贴图，我们通过OpenGL函数自动计算其纹理坐标。然而，立方体贴图在本质上与球面贴图不同，球面贴图除了要执行特殊的坐标计算之外，它更像标准的二维纹理贴图。这里必须使用三维纹理坐标，并开启立方体映射功能：

```
glTexGeni(GL_S, GL_TEXTURE_GEN_MODE, GL_REFLECTION_MAP);
glTexGeni(GL_T, GL_TEXTURE_GEN_MODE, GL_REFLECTION_MAP);
glTexGeni(GL_R, GL_TEXTURE_GEN_MODE, GL_REFLECTION_MAP);
glEnable(GL_TEXTURE_GEN_S);
glEnable(GL_TEXTURE_GEN_T);
glEnable(GL_TEXTURE_GEN_R);
glEnable(GL_TEXTURE_CUBE_MAP);
```

立方体贴图还有许多其他用途，第9章将详细介绍有关内容。我们可以使用立方体贴图方法快速地计算法向量；可以用它来模拟光与材质相互作用生成的更为复杂的现象（如折射）；还可以用它来生成凹凸贴图和位移贴图。

以上这些技术都是多通道绘制（multipass rendering或multirendering）的例子，在这种技术的使用过程中，为了生成单幅图像，必须计算多幅图像，每一幅图像都要使用绘制流水线。随着图形

卡的计算性能发展到一定程度，即从不同的视角多次绘制同一个场景所花的时间比合理刷新屏幕所需的时间还要少时，多通道绘制方法也就变得越来越重要了。8.12节将介绍累积缓存，它支持多通道绘制以及一些需要使用多通道绘制的技术。

8.11　合成技术

到目前为止，我们一直假定只需要生成单幅图像，并且成像对象的表面是不透明的。OpenGL支持这样一种机制，即通过alpha融合（alpha blending），它除了生成其他效果之外，还可以生成半透明的图像。alpha通道（alpha channel）是RGBA（或RGBα）颜色模型的第4个颜色分量，与其他的颜色一样，应用程序也可以控制每个像素的A值（α或值）。然而，在RGBA颜色模式中，如果开启了alpha融合功能，那么α值用来控制RGB值写入帧缓存的方式。因为来自多个对象的不同片元可以对同一个像素的颜色有贡献，所以我们说这些对象的颜色融合或组合在一起。我们可以使用同样的机制对图像进行融合。

8.11.1　不透明度与融合

一个表面的不透明度表示穿透该表面光线的数量。一个完全不透明表面的不透明度为1（$\alpha=1$），它会阻止所有到达其表面的光线穿过该表面。不透明度为0的表面是完全透明的，所有的光线都能穿过它。如果一个表面的不透明度为α，那么它的透明度（transparency）或半透明度（translucency）为$1-\alpha$。

考虑如图8.39所示的3个受到均匀光照的多边形。假定中间的多边形是不透明的，而前面那个离观察者最近的多边形是半透明的。如果前面那个多边形是完全透明的，那么观察者只能看到中间的那个多边形。然而，如果前面那个多边形只是部分不透明（或部分透明），类似于有色玻璃，那么观察者看到的颜色是前面和中间这两个多边形的颜色融合后的颜色。因为中间那个多边形是不透明的，所以观察者无法看到后面的那个多边形。如果前面那个多边形的颜色是红色，而中间的多边形是蓝色，那么由于红色和蓝色相融合，观察者看到的颜色是品红色。如果中间那个多边形只是部分不透明，那么观察者看到的是所有这3个多边形颜色相融合的颜色。

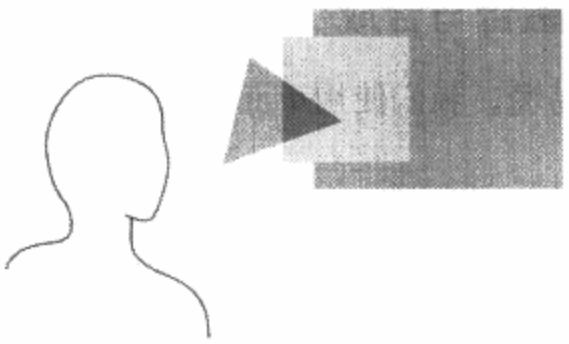


图8.39　半透明和不透明多边形

在计算机图形学中，通常一次只把一个多边形绘制到帧缓存中。因此，如果希望使用融合或组合效果，那么需要使用某种方法把不透明度作为片元处理过程的一部分。这里要用到源像素和目标像素的概念，就像8.3节用到的源位和目标位。当处理一个多边形的时候，我们要对像素大小的片元进行处理，如果片元可见，那么根据所使用的明暗处理模型来计算它的颜色值。到目前为止，我们使用片元的颜色（它是通过明暗处理模型和相关的纹理映射技术计算得到的）来确定帧缓存中某个像素的颜色，该像素与屏幕坐标系中的这个片元相对应。如果把片元当做源像素，并且把帧缓存中的像素当做目标像素，那么有多种方式组合这两者的值。使用α值是控制片元与片元之间融合的

一种方法。融合两个多边形的颜色类似于把两片有色玻璃组合成一片玻璃，该玻璃具有更大的不透明度，并且它的颜色与任意一块原始玻璃的颜色都不一样。

如果使用包含4个元素（RGBα）的数组分别表示源像素和目标像素：

$$\mathbf{s} = [\, s_r \quad s_g \quad s_b \quad s_a \,]$$
$$\mathbf{d} = [\, d_r \quad d_g \quad d_b \quad d_a \,]$$

那么使用下面的组合运算 d' 代替 $\mathbf{d}$：

$$\mathbf{d}' = [\, b_r s_r + c_r d_r \quad b_g s_g + c_g d_g \quad b_b s_b + c_b d_b \quad b_a s_a + c_a d_a \,]$$

常量数组 $\mathbf{b} = [b_r \;\; b_g \;\; b_b \;\; b_a]$ 和 $\mathbf{c} = [c_r \;\; c_g \;\; c_b \;\; c_a]$ 分别是源融合因子和目标融合因子。与处理RGB颜色值的方法相同，我们把大于1.0的 α 值限制为或截取为最大值1.0，而小于0的 α 值截取为0.0。可以通过选择 α 值及源与目标值的组合方法来获取各种不同的融合效果。

8.11.2　图像合成

α 融合最直截了当的应用就是对以像素阵列形式存储的一组图像（或者是通过独立绘制生成的一组数据）进行合成并把它们显示出来。在这种情况下，我们可以把每幅图像都视为对最终图像有相等贡献的辐射对象。通常情况下，我们希望最终图像的RGB颜色值控制在0～1的范围，而不必对大于1的值执行截取（clamp）操作。因此，要么对每幅图像的颜色值进行缩放，要么利用源融合因子和目标融合因子来控制最终图像RGB颜色值的取值范围。

假定有 n 幅图像，它们对最终显示的图像有等同的贡献。对于某个给定的像素，第 i 幅图像的颜色贡献值为 $\mathbf{C}_i\alpha_i$，其中 $\mathbf{C}_i$ 表示颜色三元组 (R_i, G_i, B_i)。如果用 $\frac{1}{n}\mathbf{C}_i$ 代替 $\mathbf{C}_i$，并用 $\frac{1}{n}$ 代替 α_i，那么只要把每幅图像的贡献值增加到帧缓存中（假定帧缓存的颜色值被初始化为黑色，且 $\alpha = 0$）。还有另一种方法，我们使用值为1的目标融合因子，并使用值为 α 的源融合因子，可以把每幅图像中的每个像素的 α 值设置为 $\frac{1}{n}$，这样就使得源融合因子为 $\frac{1}{n}$。尽管这两种方法生成同样的图像，但是如果硬件支持图像的合成，那么第二种方法可能更有效。注意，如果 n 的取值很大，那么融合因子为 $\frac{1}{n}$ 的这种形式可能会导致颜色分辨率的严重丢失。8.12节将解决这个问题。

8.11.3　OpenGL中图像的融合与合成

OpenGL中的融合机制简明易懂。我们使用下面的函数开启融合功能：

```
glEnable(GL_BLEND);
```

然后使用下面的函数设置所需的源融合因子和目标融合因子：

```
glBlendFunc(source_factor, destination factor);
```

OpenGL预先定义好了很多融合因子，包括：值为1的融合因子（GL_ONE）和值为0的融合因子（GL_ZERO），值为 α 和 $1-\alpha$ 的源融合因子（GL_SRC_ALPHA和GL_ONE_MINUS_SRC_ALPHA），值为 α 和 $1-\alpha$ 的目标融合因子（GL_DST_ALPHA和GL_ONE_MINUS_DST_ALPHA）。应用程序只需指定所需的选项，然后就可以使用RGBA颜色值。

对于大部分可供选择的融合因子，图像合成的主要问题是绘制多边形的顺序会影响最终显示的图像。例如，许多应用程序使用源 α 值作为源融合因子，而使用 $1-\alpha$ 作为目标融合因子，最终显示图像的颜色和不透明度为：

$$(R_{d'}, G_{d'}, B_{d'}, \alpha_{d'}) = (\alpha_s R_s + (1-\alpha_s)R_d, \alpha_s G + (1-\alpha_s)G_d, \alpha_s B_s + (1-\alpha_s)B_d, \alpha_s\alpha_d + (1-\alpha_s)\alpha_d)$$

上述公式能确保最终图像的颜色和不透明度都不会达到饱和。然而，最终图像的颜色值和α值与多边形的绘制顺序有关。因此，与大部分OpenGL程序不同，在这些OpenGL程序中，用户不必考虑多边形光栅化的顺序，而要得到图形融合的正确效果，现在必须在应用程序中控制多边形光栅化的顺序。

对场景中的一个不透明对象和半透明对象进行合成时，会出现一个微妙的但视觉上看来又很明显的错误。一般来说，当使用融合时，我们不会开启隐藏面消除功能，因为如果不这样的话，那么那些被已绘制多边形遮挡的多边形就不会被光栅化，从而也就不会对最终的图像有贡献。在一个既有不透明多边形又有半透明多边形的场景中，任何位于不透明多边形后面的多边形都不需要绘制，但是位于不透明多边形前面的半透明多边形则需要进行合成。有一个解决该问题的简单方法，它不需要应用程序确定多边形的绘制顺序。我们可以像往常一样开启隐藏面消除功能，当要绘制任何半透明多边形时，把z-buffer置成只读状态。我们可以通过下面的函数做到这一点：

```
glDepthMask(GL_FALSE);
```

当深度缓存被置为只读状态时，如果一个要绘制的半透明多边形被某个已绘制的不透明多边形所遮挡，那么可以把它丢弃；而如果一个要绘制的半透明多边形位于任意一个已绘制的多边形的前面，那么需要把这个半透明多边形的颜色与位于其后面的多边形融合。然而，因为绘制半透明多边形时，z-buffer的状态是只读的，所以z-buffer中的深度值不会被改变。如果绘制的是不透明多边形，那么深度屏蔽重新置为真，并按常规的z-buffer算法绘制多边形。注意，因为合成的结果取决于合成的顺序，所以我们可能会注意到按上述方法合成的图像具有瑕疵，这是由于我们是按任意的顺序绘制半透明多边形。如果我们愿意对所有的半透明多边形先进行排序，那么就可以首先绘制所有的不透明多边形，然后把z-buffer置成只读状态，并按照从后向前的顺序绘制半透明多边形，这样可以确保得到正确的融合效果。

8.11.4　再探反走样

α通道的主要应用之一就是反走样。因为一条直线要可见就必须有一定的宽度，直线的默认宽度是一个像素宽。我们不可能绘制比一个像素还要细的直线了。如图8.40所示，如果不是水平线或垂直线，那么该直线有一部分覆盖了帧缓存中的多个像素。假定作为绘制过程几何处理过程的一部分，当处理某个片元时，我们把它对应像素的α值设置为0到1之间的一个数值，这个数值表示该被片元覆盖像素的面积。然后使用这个α值来调制该片元绘制到帧缓存中时对应像素的颜色。我们使用值为$1-\alpha$的目标融合因子，以及值为α的源融合因子。然而，如果在某个像素上有多个片元互相重叠，那么存在许多可能的情况，如图8.41所示。图8.41(a)是片元之间没有互相重叠的情况，而在图8.41(b)中，片元之间确实存在重叠现象。我们从某个具体的绘制系统来分析这个问题，这里假定该绘制系统一次只能绘制一个多边形。对于这个简单的例子，假定在一个不透明的背景上开始绘制对象，并且初始化时帧缓存中的背景颜色为$\mathbf{C}_0$。因为此时像素的任何部分都没有被多边形的片元覆盖，所以设置$\alpha_0=0$。当绘制第一个多边形的时候，目标像素的颜色为：

$$\mathbf{C}_d=(1-\alpha_1)\mathbf{C}_0+\alpha_1\mathbf{C}_1$$

目标像素的α值设置为：

$$\alpha_d=\alpha_1$$

因此，如果一个片元覆盖了整个像素（$\alpha_1=1$），那么把它的颜色赋给目标像素，且目标像素不透明。如果背景是黑色，那么目标像素的颜色为$\alpha_1\mathbf{C}_1$。现在分析第二个多边形的片元，该多边形与第一个多边形在转角处覆盖同一个像素。在绘制了第一个多边形之后，如何再增加第二个多边形片元的颜

色和α值，这取决于如何解释这两个多边形的重叠情况。如果它们之间没有重叠现象，那么目标像素的新颜色值是通过把目标像素颜色与片元颜色进行融合而得到的，新的颜色值和α值为：

$$\mathbf{C}_d = (1-\alpha_2)((1-\alpha_1)\mathbf{C}_0 + \alpha_1\mathbf{C}_1) + \alpha_2\mathbf{C}_2$$
$$\alpha_d = \alpha_1 + \alpha_2$$

得到的新颜色是两种颜色的融合，并且不需要进行截取（clamp）处理。得到的α值表示被片元覆盖的像素面积的新比例。然而，最终的像素颜色取决于多边形的绘制顺序。更难处理的问题是如何处理两个重叠的片元，以及如何判断它们是否重叠。有一种方法采用的是概率的观点，如果片元1占用像素面积的比例为α_1，片元2占用同一个像素面积的比例为α_2，并且我们没有任何其他的有关信息来确定这两个片元在像素中的位置，那么这两个片元重叠的平均面积为$\alpha_1\alpha_2$，图8.42表示了这种平均的情况。因此，目标像素新的α值为：

$$\alpha_d = \alpha_1 + \alpha_2 - \alpha_1\alpha_2$$

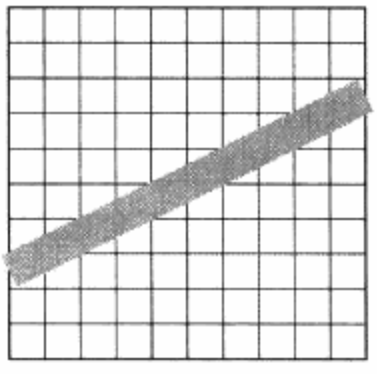


图 8.40　光栅直线

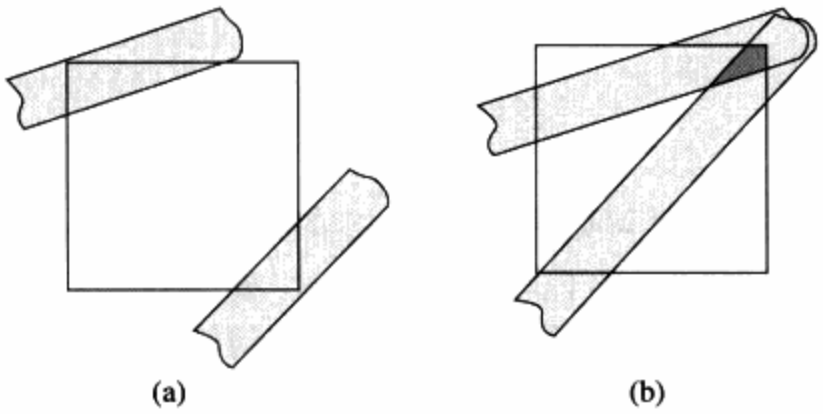


图 8.41　片元：(a) 没有重叠；(b) 有重叠

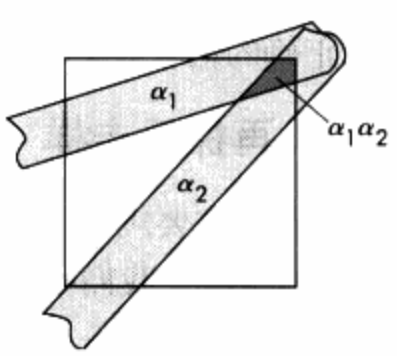


图 8.42　片元重叠的平均面积

最终像素颜色的计算是一个更复杂的问题，因为必须确定两个片元的前后位置关系，或者，我们甚至需要确定这两个片元是否需要融合。无论做什么样的假设，我们都可以定义一个合适的融合方式。我们注意到，在流水线结构的绘制系统中，生成多边形的顺序与它们到观察者的距离没有任何关系。然而，如果把α融合与隐藏面消除结合起来考虑，就可以利用深度信息来确定多边形的前后位置关系。

在OpenGL中，如果为点、直线和多边形开启了融合和平滑功能，那么在程序中不用显式地对α值进行组合运算就可以开启反走样功能。例如，我们可以使用下面的代码开启这三个图元的反走样功能。

```
glEnable(GL_POINT_SMOOTH);
glEnable(GL_LINE_SMOOTH);
glEnable(GL_POLYGON_SMOOTH);
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
```

启动反走样功能可能会极大地降低系统的性能。彩图12显示了OpenGL中对多边形的反走样处理效果。

8.11.5　从后向前或从前往后绘制多边形

虽然可以利用α通道生成半透明效果，但是如果不考虑光照与对象表面的作用方式以及光线和投影线穿过半透明对象时发生的情况，就不可能生成符合物理规律的透明效果。对于图8.43所示的场景，我们可以看到一些可能会遇到的问题。我们暂时不考虑光线通过透明对象表面时发生的折射现象，因为使用流水线结构的多边形绘制系统很难处理这种折射效果。假定位于后面的多边形[①]是不透明的，但具有反射特性，并且离视点比较近的两个多边形是半透明的。通过跟踪从光源发出的各种光线，我们发现光线与场景对象相互作用的情形有多种可能。一些光线射到后面的多边形上，对应像素的颜色值就是投影线与多边形交点处的明暗值。对于这些光线，还需要区分两种情形，一种情况是光源发出的光线直接射到后面多边形上的某些点上，另一种情况是光源发出的光线穿过前面一个或两个半透明多边形之后射到后面多边形的某些点上。对于那些只穿过一个半透明多边形的光线，必须根据该多边形的颜色和不透明度来调整对应像素的颜色。对于这种情形，我们还需要增加一项用来表示光线射到前面多边形并发射到视点的光强度。对于那些穿过前面两个半透明多边形的光线，必须考虑它们的组合效果。还必须考虑光线穿过某种材质时的融合或折射（refraction）效果。

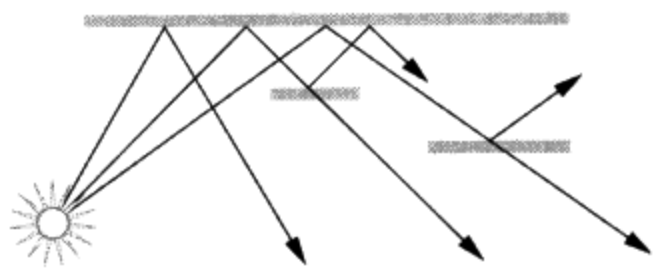


图 8.43　包含半透明对象的场景

对于流水线结构的绘制系统，虽然可以完成这个任务，但是非常困难，这是因为当每个多边形经过流水线时，我们必须确定每个多边形对最终像素的颜色值的贡献，而不是同时考虑所有多边形对某个像素颜色值的贡献。在某些需要使用一致的且逼真的方式处理半透明效果的应用中，通常需要在应用程序中对多边形按从前向后的顺序进行排序，然后根据应用的需求，使用OpenGL的融合功能按从前向后或从后往前的顺序来绘制这些多边形（见习题8.27）。

8.11.6　景深和烟雾效果

景深效果处理是三维图形中最古老的技术之一。在光栅显示系统出现之前，我们使用的图形系统只能绘制线条。相比于离视点较近的直线，我们把离视点较远的那些直线绘制得更暗些，从而使人看起来有深度的感觉，我们把这种技术称为景深效果处理（depth cueing）。我们可以对这种思想进行扩展，即在处理每个片元的时候融合与距离有关的颜色，这样就可以在对象和视点之间创建一种半透明空间的幻觉。

假定用f表示雾化因子（fog factor），而用z表示要绘制的片元与视点之间的距离。如果片元的颜色为$\mathbf{C}_s$，并且雾的颜色为$\mathbf{C}_f$，那么绘制时使用的颜色为：

$$\mathbf{C}_{s'} = f\mathbf{C}_s + (1-f)\mathbf{C}_f$$

如果f值在某个最小值和最大值之间线性变化，那么就可以得到景深效果。如果雾化因子按指数的形式变化（如图8.44所示），那么得到的效果更像是雾的效果。OpenGL支持雾密度的线性分布、

① 离视点比较远的多边形。——译者注

指数分布和高斯分布。例如，在RGBA颜色模式下，可以通过下面的函数调用来设置雾密度函数 $f = e^{-0.5z^2}$：

```
GLfloat fcolor[4] = {...};
glEnable(GL_FOG);
glFogf(GL_FOG_MODE, GL_EXP);
glFogf(GL_FOG_DENSITY, 0.5);
glFogfv(GL_FOG_COLOR, fcolor);
```

注意，我们是在观察坐标系中定义 z 值。

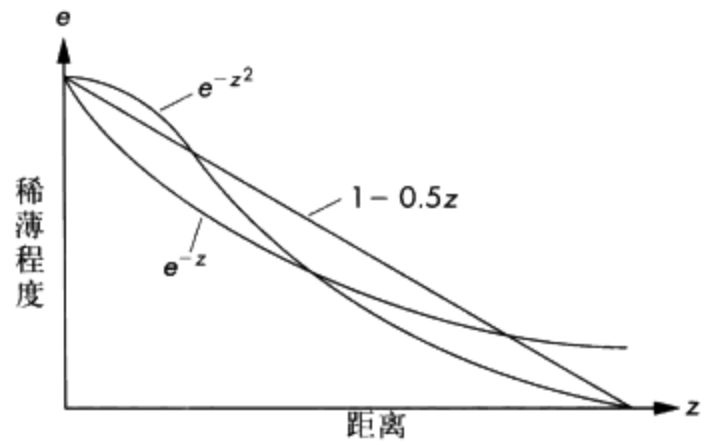


图 8.44　雾密度

8.12　多通道绘制与累积缓存

还有另一个缓存，如果存在的话，它对实现许多其他的功能非常有用。下面考虑对一组图像进行合成会遇到的问题，其中每个图像的颜色分量都占用一个字节。如果只是简单地把这些图像的RGB颜色值累加到典型的32位颜色缓存中，那么许多颜色的分量值会溢出，即大于一个字节所能表示的最大值。如果把颜色缓存中的值截取（clamp）到一个字节所能表示的最大值，那么最终生成的图像的颜色看起来好像被冲淡了，这是因为合成图像中的大部分颜色会接近白色。如果我们设法采用这样的方法来解决这个问题，即在对这些图像进行合成之前对每幅图像的颜色值进行压缩，则会降低颜色的分辨率。例如，我们可以分析这样的情况，如果希望合成64幅图像，其中每幅图像的每个颜色分量都具有8位的分辨率，那么为了避免出现溢出的情况，我们必须把图像的每个颜色分量压缩到原来的1/64，或者说，需要把图像的每个颜色分量都减少6位的精度。

解决该问题的一个方法是建立一个缓存，即累积缓存（accumulation buffer），它与帧缓存具有相同的空间分辨率，但是具有更高的深度分辨率。我们可以把累积缓存视为一种特殊的颜色缓存，其各个分量用浮点数进行存储。因此，我们可以使用额外的深度分辨率把连续的图像绘制到同一个位置而又能保持其数值精度。我们可以把累积缓存应用到许多场合。

在OpenGL中，首先可以采用与其他缓存相同的方法对累积缓存进行清零操作，然后就可以通过函数glAccum将颜色缓存中的值加到或乘到累积缓存中，或者把累积缓存中的内容复制到颜色缓存。例如，对于下面这段代码：

```
glClear(GL_ACCUM_BUFFER_BIT);
for(i=0; i< num_mages; i++)
{
      glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
      display_image(i);
      glAccum(GL_ACCUM, 1.0/ (float) num_images);
}
glAccum(GL_RETURN, 1.0);
```

这段代码使用用户自定义的函数display_image把图像序列写入到写缓存中。其中的每幅图像按图像数目分之一的缩放因子加到或者累积到累积缓存中。最后，再把累积的图像复制回写缓存中。注意，我们可能会使用融合的方法实现这个任务，但是为了避免执行截取操作，我们不得不对颜色值进行缩放并存储在低分辨率的颜色缓存中。

起初，累积缓存是通过软件实现的，它使用了处理器内存作为缓存。因此，以这种方式实现的累积缓存其速度往往是非常慢的。最新的GPU具有浮点数据类型的帧缓存，允许用户使用标准的颜色缓存实时地实现大部分累积缓存多通道绘制技术。

8.12.1　场景反走样

累积缓存的重要应用之一就是反走样。与8.10.4节讨论的对直线和多边形的反走样不同，我们可以使用累积缓存对整个场景进行反走样。这种方法的基本思想是：如果使场景中的所有对象或者观察者做微小的移动（小于一个像素），然后再重新绘制这个场景，那么会得到一个不同于原始图像的走样图像。如果对得到的多个走样图像进行平均处理，就可以平滑掉走样图像中的走样效果。通常情况下，通过移动或抖动观察者的位置来得到走样图像要简单一些，这是因为只需要改变函数glPerspective或glOrtho中的参数即可。然而，我们必须在程序中通过计算来确定屏幕坐标系中的微小变化与观察坐标系中对应变化的关系。从采样理论（见8.13节）的观点来看，我们实际上使用了更高的分辨率来对整个场景世界进行采样，或进行超采样（supersample），然后使用平均滤波的思想从这些采样中重构场景世界。

8.12.2　凹凸贴图和浮雕效果

第9章将介绍另一种称为凹凸映射的纹理映射技术，通过这种技术绘制的对象表面看起来似乎具有丰富的凹凸细节，对象表面这种看似粗糙的效果受到光源位置的影响。典型的凹凸映射是基于片元到片元的模式实现的，然而我们可以使用累积缓存来得到凹凸映射非常好的近似效果。

一个具有凹凸细节特征的表面具有浮雕的视觉效果，这是因为由于表面上的凹凸细节相对于光源的朝向不同，所以我们看到表面不同位置的凹凸细节与光源相互作用的明暗效果有区别。相对于背向光源的表面，朝向光源的表面看起来会更亮些。表面明暗效果的变化信息由表面的法向量和光源矢量之间的关系确定。虽然可以使用这两个矢量的点积来表示它们之间的关系，但是我们还可以使用位移矩阵**D**并结合累积缓存的方法来实现表面的凹凸效果。

下面分析这样一幅图像，它表示位移矩阵**D**及其绕光源方向旋转后得到的位移矩阵之间的差异。对于表面上朝向光源的点，该图像对应的像素值为正值，而对于表面上背离光源的点，该图像对应的像素值为负值。我们可以把这种思想作为凹凸映射过程的一部分。

假定首先把场景绘制到累积缓存中，使用**D**作为纹理贴图。注意，由于**D**中的每个值只有一个分量，所以其对应的纹理图像是单色的。然后我们沿光源方向对**D**进行移位，并使用这个旋转后的图像作为纹理再次绘制场景，这次我们把得到的结果与累积缓存中的值做减法运算。最后，我们不使用**D**，而是按正常方式绘制图像并把结果加到累积缓存中。

我们注意到，为了实现第二步对**D**的移位操作，需要知道光源相对于表面的方向。然而，我们具备创建作用于每个顶点的齐次坐标矩阵所需的信息。我们可以利用每个顶点的法向量（它是系统状态的一部分）以及用来定义位移函数$d(u, v)$朝向的两个矢量中的某一个来实现第二步对**D**的移位操作。这些纹理坐标位于对象表面的切平面（相切于表面的法向量）上。因此，我们可以使用

上述两个矢量中的任意一个矢量与表面的法向量进行差积运算，从而得到第三个矢量，我们还可以利用这三个矢量确定所需的旋转矩阵。

8.12.3　图像处理

我们可以把累积缓存与像素映射结合起来使用，这样可以处理各种图像处理操作。假定要处理的对象是一个离散的图像，该图像可能是通过绘制得到的，也可能是通过扫描仪对一幅连续的图像进行数字化处理得到的。我们可以用一个 $N \times M$ 的标量矩阵表示该图像：

$$\mathbf{A} = [\, a_{ij} \,]$$

如果要单独处理彩色图像的每个颜色分量，那么可以把矩阵**A**中的每个元素视为单个颜色分量或灰度（亮度）级。矩阵 **A** 经一个线性滤波器处理后得到矩阵 **B**，矩阵 **B** 中的每个元素表示为：

$$b_{ij} = \sum_{k=-m}^{m} \sum_{l=-n}^{n} h_{kl}\, a_{i+k,j+l}$$

我们说矩阵 **B** 是矩阵 **A** 与滤波矩阵 **H** 做卷积运算的结果。通常情况下，m 和 n 的值都很小，我们可以把 **H** 表示为一个很小的（$2m+1 \times 2n+1$）卷积矩阵（convolution matrix）。

我们观察如图 8.45 所示的 $m = n = 1$ 的卷积矩阵。对于矩阵 **A** 中的每一个像素，我们把卷积矩阵 **H** 作用于 a_{ij}，并对它周围的像素计算加权平均值。卷积矩阵中的值表示权重。例如，对于 $n = m = 1$ 的情形，可以使用下面的 3 × 3 矩阵计算每个像素与它相邻的 4 个像素的平均值：

$$\mathbf{H} = \frac{1}{5}\begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

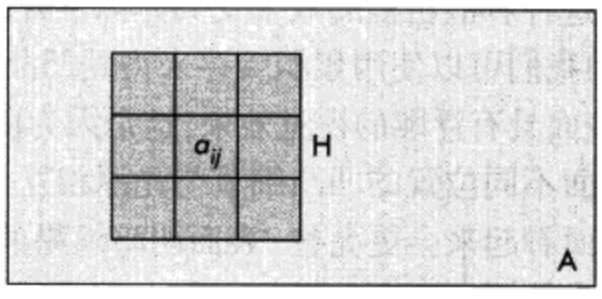


图 8.45　滤波与卷积

该滤波矩阵可用于反走样。我们还可以使用更多的相邻像素参与加权平均计算，并增加中间像素点的权重，例如下面的滤波矩阵：

$$\mathbf{H} = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

我们注意到，如果希望经过滤波处理后得到的矩阵 **B** 与 **A** 有相同的维数，那么必须定义矩阵 **A** 的边界。这种小矩阵还可以用于其他操作。例如，可以使用下面的矩阵检测图像中值的变化或图像的边界。

$$\mathbf{H} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

如果 **H** 是 $k \times k$ 的卷积矩阵，那么实现滤波功能的方法是把 k^2 个图像累加到帧缓存中，每次都使用函数 glAccum 中不同的滤波系数把处理后的矩阵 **A** 加到累积缓存中。

8.12.4　用于图像处理的 OpenGL 扩展

虽然可以使用累积缓存实现许多图像处理操作，但是如果认真分析其处理过程，就会发现使用累积缓存的方法在实现效率上存在严重的问题。如果需要把图像传送到累积缓存中或从累积缓存中取图像数据，那么就要在处理器内存、帧缓存和纹理内存之间传送大量的数据。这些数据在传送过程中还需要对数据阵列进行打包和解包处理，这会进一步降低处理速度。

我们开发累积缓存技术需要考虑两件事情。第一，在现有的图形 API 中增加支持图像处理的 API 应该非常简单。第二，增加图像处理操作不应该对图形硬件做大的改动。实际上，我们已经解决了这两个问题。从 OpenGL 的角度来看，OpenGL 中有一个图像处理子集，它定义了专门用于处理最常用的图像处理操作的 API。图像处理子集中定义的这些操作是 OpenGL 的一个扩展，因此不需要所有的 OpenGL 实现都支持这些操作。然而，在所有的 OpenGL 扩展中，图像处理子集是得到最普遍支持的一个扩展。OpenGL 支持的图像处理功能包括：直方图、卷积（滤波）、颜色查找表和颜色变换处理。

在硬件方面，我们看到图形硬件已经支持图像处理流水线结构。典型的图像处理流水线位于图形硬件的内部，并且它处理的对象是解包后的像素。因此，我们需要在应用程序中对图像数据进行解包并只传送一次即可。像查找表、滤波和颜色变换等图像处理操作，对于其中的任意一个图像处理操作，都可以在该操作结束后再把图像数据传送回处理器内存。

8.12.5　其他多通道绘制方法

我们还可以把累积缓存用于时间滤波和深度滤波。例如，如果抖动某个对象并且多次绘制该对象而保持其他对象的位置不变，那么在最终生成的图像中，这个抖动的对象会变得模糊。如果该对象沿着某条路径移动，而不是随机地抖动，那么可以看到该对象留下的轨迹，这种运动模糊（motion-blur）的效果类似于对运动的对象拍照时曝光时间过长产生的效果。为了用更大的不透明度绘制最终位置的对象，或者生成具有速度差的效果，我们可以调节函数 glAccum 中的常量参数。

此外还可以使用深度滤波来生成聚焦的效果。使用真实的照相机不可能拍摄到所有对象都位于聚焦范围内的照片。离照相机一定距离（即照相机的景深）范围的对象位于聚焦范围内；而景深范围之外的对象不在聚焦范围内，这些对象看起来比较模糊。在计算机图形学，可以生成无限景深的图像，这是因为不必担心真实照相机镜头的限制。然而，人们有时也希望生成看起来像真实照相机拍摄的图像，或希望部分场景对象散焦，这样可以突显位于特定景深内的对象。我们可以再一次使用累积缓存。这一次，我们需要想办法以某种方式移动视点，使得某个特定的平面固定不变，如图 8.46 所示。假定希望使平面 $z = z_f$ 作为聚焦平面，并且使视见体的近裁剪距离（$z = z_{\min}$）和远裁剪距离（$z = z_{\max}$）保持不变。如果使用函数 glFrustum 进行透视投影，那么需要指定近裁剪矩形（$x_{\min}$, $x_{\max}$, $y_{\min}$, $y_{\max}$）。如果由原点沿 x 轴方向移动视点 Δx 距离，那么必须把 $x_{\min}$ 修改为：

$$x'_{\min} = x_{\min} + \frac{\Delta x}{z_f}(z_f - z_{\text{near}})$$

上面的方程同样适用于 $x_{\max}$、$y_{\min}$ 和 $y_{\max}$。当增加 Δx 和 Δy 的值时，就会得到一个比较窄的景深。

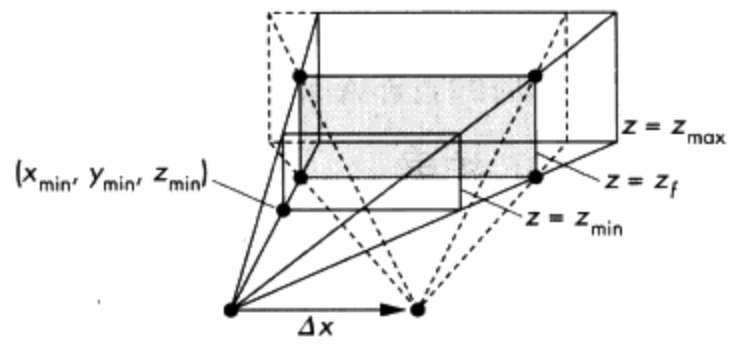


图 8.46　景深的调节

8.13　采样与走样

我们目前已经见到了各种将连续表示的实体转换为该实体离散近似表示的应用，这种转换会导致屏幕显示的图像出现视觉上可见的误差。我们使用走样这个术语来刻画这种误差的特性[①]。当处理缓存内容时，我们总是把它作为数字图像来处理，因此，如果使用不慎，那么这些误差会造成极端的情形。这一节将讨论数字图像的性质，介绍有助于理解引起走样误差的相关因素以及如何减轻因误差引起的走样效果。

我们从一个连续的二维图像$f(x, y)$开始讨论。图像的f值要么表示单色图像的灰度级，要么表示彩色图像的某个颜色分量值。在计算机中，只能处理数字图像，它是一个n行、m列共nm个像素组成的阵列，每个像素占用k位。将连续图像转换为离散图像需要经过两个处理步骤。第一步，必须在nm个网格点上对连续图像进行采样，从而获得一组采样值$\{f_{ij}\}$，其中的每一个采样值f都是在连续图像上一个区域内测得的。然后通过量化过程把每一个采样值转换成一个k位的像素值。

8.13.1　采样理论

如图8.47所示，假定有一个矩形网格，我们要在其网格点上获取采样值f。如果假定网格在空间上是等分的，那么一个理想的采样器生成的采样值可能为：

$$f_{ij} = f(x_0 + ih_x, y_0 + jh_y)$$

其中，h_x和h_y分别是在x方向和y方向网格点之间的距离。现在不考虑这样一个事实，即实际使用的采样器不可能有如此的精度，而是考虑另外两个重要的问题。第一个问题是：在这种理想化的采样过程中会产生什么样的误差？也就是说，原始图像经采样后有多少信息包含在采样图像中？第二个问题是：我们能否把数字图像还原为连续图像而不产生额外的误差？后一个步骤称为重构（reconstruction），它用来描述显示过程，如将帧缓存中的内容显示到显示器上就需要用到重构。

要从数学上分析这些问题，需要用到傅里叶变换理论，它是应用数学的一个分支，非常适合于解释信号处理中的问题。傅里叶变换理论的本质是：无论是时间函数还是空间函数都可以分解为一系列的正弦函数，也可能分解为包含无数个频率的正弦波。这是一个在声学中最为人们熟悉的概念，在声学中，通常将某个特定的声音与它的频率或频谱联系起来。对于一个二维图像，我们认为它是由两个空间频率的正弦曲线构成的，把它们组合到一起就生成了该图像。图8.48(a)显示了一个一维函数；图8.48(b)显示的是两个二维函数，可以用这两个函数合成图8.48(a)所示的一维函数；图8.49显示的是二维周期函数。因此，每个二维空间函数$f(x, y)$都有两种等价的表示方式：一种是

① 这里使用走样（aliasing）这个术语来描述这种效果，因为这与数字信号处理中的混叠（aliasing）现象有关系，我们在以下的译文中把“aliasing”统一翻译成“走样”。——译者注

它的空间表示形式 $f(x, y)$，一种是它的频谱表示形式（即频率域表示）$g(\xi, \eta)$，g 的值表示在二维空间频率域(ξ, η)上对f的贡献。通过使用这种频率域函数的表示方法，我们发现在频率域解释包括采样在内的许多现象都会变得更容易。

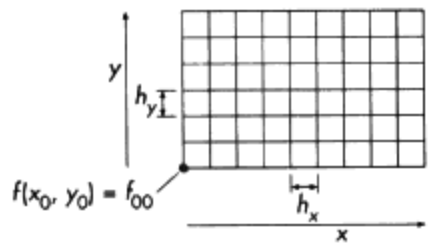


图 8.47　采样网格

图 8.48　一维函数的分解：(a) 函数；(b) 分量函数

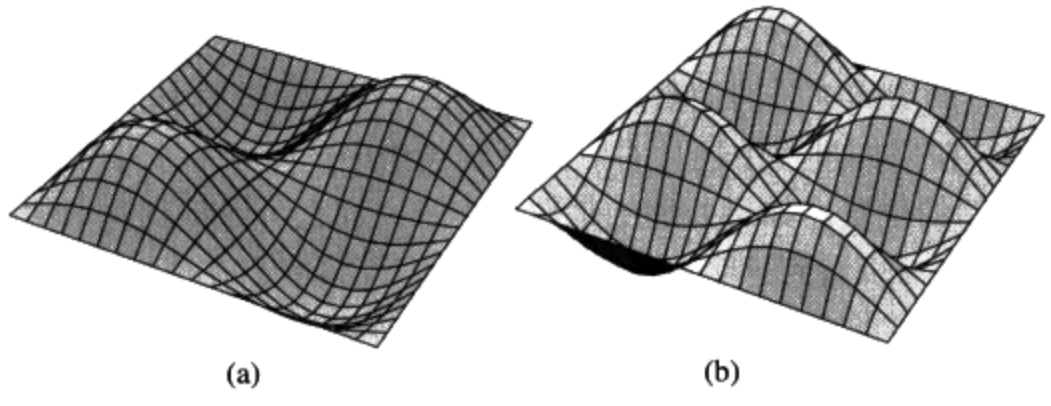


图 8.49　二维周期函数

为了不被数学问题所困扰，如果我们在不加证明的情况下能够接受一个基本定理，即奈奎斯特采样定理（Nyquist sampling theorem），那么就可以解释采样的因果关系。该理论包括两部分：我们可以利用它的第一部分来讨论采样误差，而利用第二部分讨论重构问题。8.13.2 节将讨论第二部分。

奈奎斯特采样定理（第一部分）：当且仅当对连续函数的采样频率大于该函数最高频率的两倍时，该连续函数的理想采样值才包含原函数的所有信息。

因此，如果不想丢失信息，那么必须在频率域把采样函数的频率限制在一个窗口内，该窗口之外的函数的频率值为零，并且该窗口以原点为中心，其宽度小于采样频率。避免出现采样失真现象的最低频率，即采样频率的一半，我们称之为奈奎斯特频率（Nyquist frequency）。如果在某个频域窗口之外函数的频谱值为零，那么该函数称为带宽受限（band-limited）函数。对于一个二维图像，它的采样频率分别取决于二维采样网格在 x 方向和 y 方向上网格点之间的间隔，即 $1/h_x$ 和 $1/h_y$。奈奎斯特采样定理认为一个理想的采样过程要采集无限个采样点，其中的每个采样点都对应采样网格点上的精确值。实际上，我们只能采集有限数量的采样点，这是因为采样点的数量受到帧缓存分辨率的限制。因此，我们无法得到一个真正的带宽受限函数。尽管该结论是傅里叶理论的一个数学推论，但是可以观察到，在一组有限的采样点中存在固有的不确定性，这只是因为我们不知道位于采样区域之外的函数[①]。

如果违背奈奎斯特采样定理，那么其结果必然会导致走样误差。只要分析一下理想的采样过程就会明白“走样”这个术语的由来。无论是原函数还是它的一组采样点都有频率域的表示形式。采样函数的频谱成分是对原函数频谱的多次复制得到的，采样点之间的距离为采样频率。考虑如

① 这句话的意思是：除了一组采样点之外，我们对其所对应的基本函数毫无所知。如果掌握了函数的其他信息，如函数的周期性、函数在某个有限区间的表示形式，那么就可以确定整个函数。

图8.50(a)所示的一维函数，图中标示了采样点。图8.50(b)显示的是原始一维函数的频谱；图8.50(c)显示的是采样函数的频谱，它是对图8.50(b)中频谱的多次复制得到的[①]。因为我们使用的采样频率大于奈奎斯特频率，所以在各个重复的频谱之间有一定的间隔。

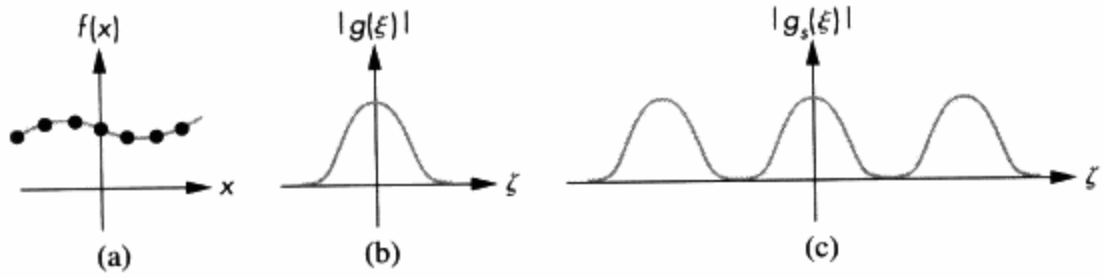


图 8.50　带宽受限函数：(a) 函数及其在空间域中的采样点；(b) 原函数的频谱；(c)采样函数的频谱

现在考虑图 8.51 所示的情形。在这种情形中，我们违背了奈奎斯特采样定理，因此在各个重复频谱之间出现了相互重叠的现象。考虑该图的中间部分，其对应的放大形式如图8.52所示，该图只显示了中心位于原点的中间部分的频谱及其右侧频谱，右侧频谱的中心位于ξ_s。频率ξ_0比奈奎斯特频率$\xi_s/2$大。然而，在采样过程中还从右侧的复制频谱中产生了一个小于奈奎斯特频率的复制频率ξ_0，它位于$(\xi_s - \xi_0)$的位置。如果处理的是数字声音，那么可以听到这个频率上的声音；如果处理的是二维图像，那么可以看到这个频率上图像。我们说频率ξ_0在位置$(\xi_s - \xi_0)$发生走样。注意，一旦发生走样，那么就无法将原函数中位于该频率的数据与通过采样过程在该频率放置的数据区分开。

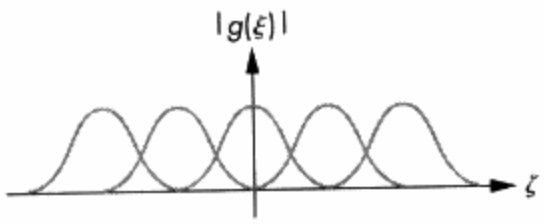


图 8.51　重复频谱之间的重叠

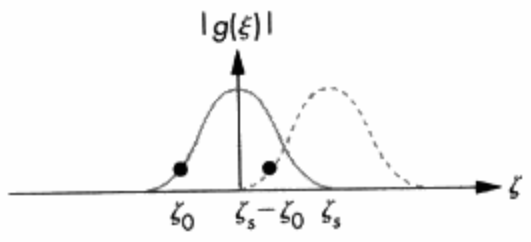


图 8.52　走样

我们不使用傅里叶分析方法，通过仔细分析如图 8.53 所示的正弦函数也可以说明走样和频率数据的不确定性。如果以两倍于正弦函数的频率对它进行采样，那么从两个采样点就可以把它还原为初始的正弦函数。然而，同样的这两个采样点既可以来自两倍于该频率的正弦函数，也可以来自其他频率为基频率多倍的正弦函数。所有这些频率都是同一原始频率的走样。然而，如果我们知道数据是带宽受限的，那么采样结果只能描述原始正弦函数。

如果对真实世界中的图像进行频谱分析，我们将发现大部分图像的频谱成分集中在低频部分。因此，虽然不可能构造大小有限的带宽受限图像，但是由于图像中只有很少的频率成分大于奈奎斯特频率，而小于奈奎斯特频率的频率成分一般不发生走样，所以图像的走样误差很小。这个结论的一个特例情况就是连续图像中包含规则性（周期性）的数据。在频率域表示中，这种规则性使得图像的大部分数据集中在几个频率上。如果其中的某个频率大于奈奎斯特频率，那么会出现非常明显的走样现象，如拍频波形图案（beat pattern）或网状波纹图案。读者可能已经注意到的这类走样的例子还包括：当电视里的人物穿着条纹衬衫或格子领带时的画面效果；打印计算机屏幕上的图像（半调图像）时出现波形图案的效果；已耕作过的农田的数字图像也会出现波纹图案的效果。

通常在扫描图像之前要进行滤波预处理，或者控制扫描仪要采样的数据区域，这样可以减少走样现象。图8.54显示了对同一幅图像进行扫描处理的两种不同方式。图8.54(a)使用的是一个理想的扫描仪，它在连续图像的某个点上测量采样值，所以可以使用下面的公式计算采样值：

① 因为傅里叶变换产生的频率域成分都是复数形式，所以我们显示的是频谱的幅度。

$$f_{ij} = f(x_i, y_i)$$

图8.54(b)使用的是一个更为实际的扫描仪，它通过对一个小区域内采样点的加权平均来计算某点的采样值，计算公式如下：

$$f_{ij} = \int_{x_i - s/2}^{x_i + s/2} \int_{y_i - s/2}^{y_i + s/2} f(x, y) w(x, y) \mathrm{d}y \mathrm{d}x$$

通过选取窗口 s 的大小以及权重函数 w，可以削弱图像中的高频成分，从而减少走样现象。幸运的是，实际扫描仪必须在一个大小有限的区域进行扫描，我们把这个区域称为采样孔径（sampling aperture），因此，即使用户对走样问题一无所知，他们在使用扫描仪时也使用了反走样技术。

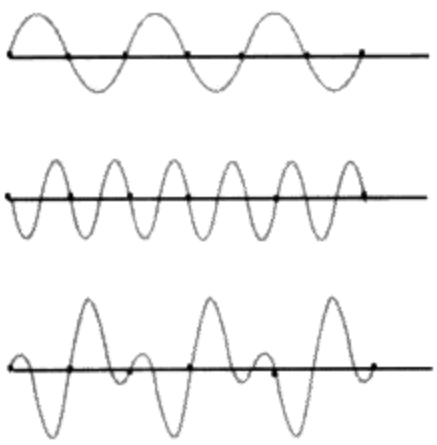
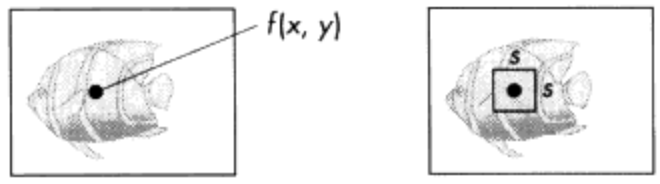


图 8.53　正弦函数的走样



(a)　　(b)

图 8.54　扫描图像：(a) 点采样法；(b) 区域平均法

8.13.2　重构

假定我们有一组采样点（无限个采样点），并且使用的采样频率远大于奈奎斯特频率。从采样点到连续函数的重构问题基于奈奎斯特采样定理的第二部分。

奈奎斯特采样定理（第二部分）：可以从连续函数 $f(x)$ 的采样点 $\{f_i\}$ 重构该连续函数，计算公式为：

$$f(x) = \sum_{i=-\infty}^{\infty} f_i \operatorname{sinc}(x - x_i)$$

其中的函数 $\operatorname{sinc}(x)$ 如图 8.55 所示，它定义为：

$$\operatorname{sinc}(x) = \frac{\sin \pi x}{\pi x}$$

从二维连续函数 $f(x, y)$ 的理想采样点 $\{f_{ij}\}$ 重构该二维函数的计算公式为：

$$f(x, y) = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} f_{ij} \operatorname{sinc}(x - x_i) \operatorname{sinc}(y - y_j)$$

上面这些公式都基于这样的事实：可以利用低通滤波器从图 8.50 所示的采样过程生成的无数个重复频谱中获取一个频谱，从而可以恢复一个在频率域中没有走样的函数，这里的低通滤波器是指除区间 $(-\xi_s/2, \xi_s/2)$ 之外值为零的滤波函数。一维函数的重构过程如图 8.56 所示。二维函数的重构需要使用如图 8.57 所示的二维 sinc 函数。遗憾的是，我们无法利用物理显示器将 sinc 函数显示出来，这是因为该函数存在小于零的一侧。下面分析该函数在 CRT 显示器上的显示问题。我们从一个用来表示一组采样点的数字图像开始讨论。对于每个采样点，我们在光栅显示器上某个栅格点的

中心位置显示一个光点，如图8.58所示。采样值的大小用来决定光点的光强，或者用来调制轰击屏幕的电子束所包含的电子数量。通过使用电子束的聚焦等技术，我们可以控制光点的形状。根据重构公式可知，电子束应该显示二维函数sinc的形状，但是因为电子束轰击荧光物质后释放能量，屏幕上所有光点的亮度肯定是非负值。因此，显示过程必定会产生误差。通过分析显示的光点与理想sinc函数的近似程度，可以评价某个实际显示器的显示性能。图8.59显示了sinc函数和它的几个近似表示。大部分CRT显示器上的光点形状呈高斯形状，而LCD显示器矩形像素的亮度形状呈矩形形状。注意，我们可以使近似光点变得更宽或更窄。如果在频率域分析光点的轮廓，将会发现比较宽的光点在低频更加准确，而在高频却不太准确。在实际中，我们采用折中的方案来选取光点的大小。不同显示器在视觉上可见的差异表现在其光点轮廓的不同。

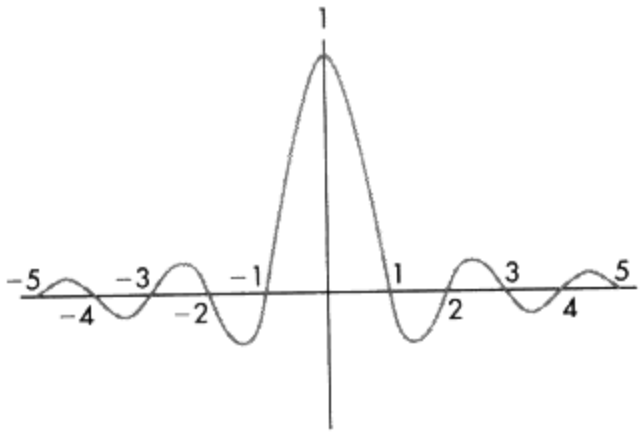


图8.55　sinc函数曲线

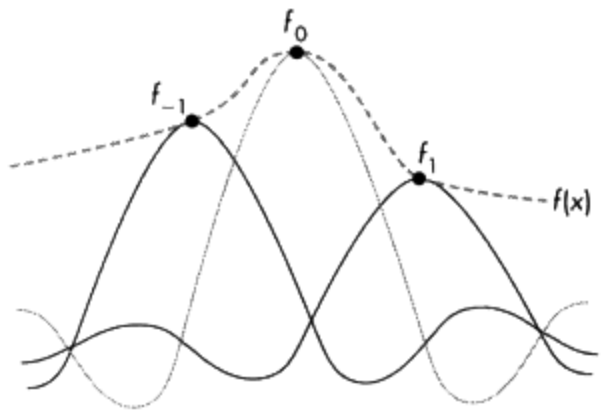


图8.56　一维重构过程

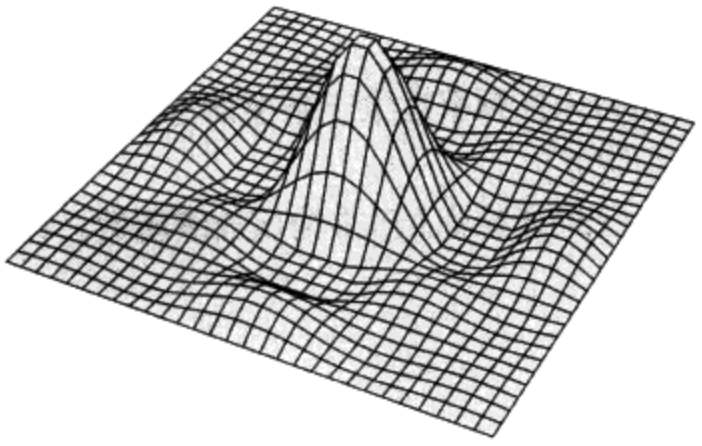


图8.57　二维sinc函数

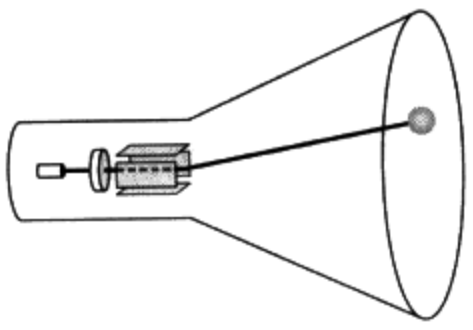


图8.58　在CRT屏幕上显示一个采样点

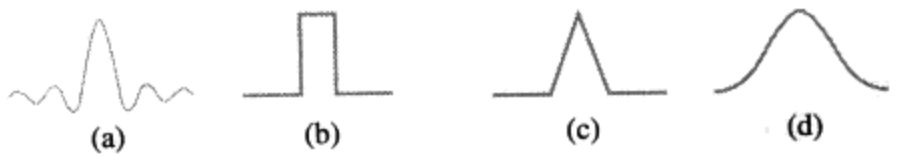


图8.59　显示光点：(a) 理想的光点；(b) 矩形近似表示；(c) 逐段线近似表示；(d) 高斯近似表示

8.13.3　量化

对采样的数学分析可以解释很多重要的现象。然而，我们还没有讨论如何将每个采样点量化成k个离散的量化级。已知标量函数g的取值范围为：

$$g_{\min} \leqslant g \leqslant g_{\max}$$

量化器（quantizer）是一个函数q，如果$g_i \leqslant g \leqslant g_{i+1}$，则

$$q(g) = q_i$$

因此，对每一个g值，我们都给它赋一个值，该值是k个量化级中的某个值，如图8.60所示。通常情况下，设计一个量化器要考虑量化级$\{q_i\}$的选择以及阈值$\{g_i\}$的选择。如果我们知道g和$p(g)$的概率分布，那么通过下面的公式可以求得使均方差最小的值：

$$e = \int (g - q(g))^2 p(g) \mathrm{d}g$$

然而，我们通常根据第1章讨论的人的感知来设计量化器。一个简单而实用的方法是：对于所设计的量化器，我们不应该能够检测到单级变化产生的效果，但是能够检测到两极变化产生的效果。对于视觉系统给定的阈值，为了检测亮度上的变化，我们通常需要为每个采样点至少分配7个或8个比特位（128级或256级）。我们还需要考虑人眼对光强的响应呈对数分布这个特性。为此，通常要求量化级按指数分布，从而使得从一级变化到下一级时，会得到几乎相等的视觉误差。

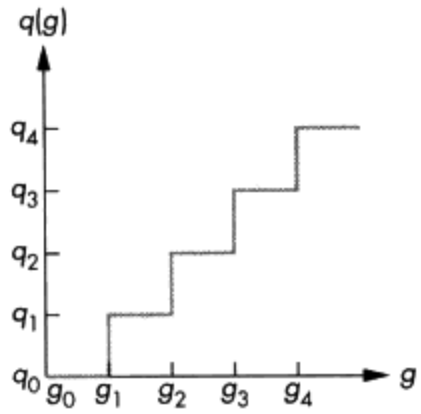


图8.60　量化

小结和注释

在计算机图形学的早期，研究人员只从事二维和三维几何对象的研究，而把二维图像的研究归为图像处理这一类。随着硬件的发展，图形与图像处理系统实际上是密不可分的。对于那些从事图像合成（当然也是计算机图形学的一个重要组成部分）的研究人员来说，这两个研究领域的结合导致了许多新技术的出现。把二维图像或纹理映射到三维对象表面的想法在10年前是不可想象的，对于几何对象表面的绘制，基于这种方法的时间开销并不比使用固定的明暗绘制方法的时间开销大。现在，这些都已成为司空见惯的技术了。

像纹理映射这样的技术对实时图形学产生了巨大影响。在动画、虚拟现实和科学计算可视化等领域，人们利用硬件的纹理映射功能就能够给绘制的图像增加细节特征，而且不会增加几何绘制流水线的负荷。应用程序开发人员通过使用基于累积缓存和α通道的图像合成技术可以实现反走样处理以及生成雾和景深等效果，直到最近，这些任务才能在图形创建后，在不同类型的图形硬件结构上实现。

映射方法是说明图形硬件、软件和应用程序之间关系的一个最好的例子。对于纹理映射，尽管在早期完全是用软件实现的（用算法来描述和实现），但是当人们发现它具有建立视觉上非常复杂的场景的能力时，硬件开发商就开始在图形硬件系统中内置了大量的纹理内存。一旦能够在硬件上实现纹理映射，就可以实时地处理纹理映射，从而导致了许多应用程序的重新设计，尤其是计算机游戏。

目前GPU在功能和性能上的快速发展为研究提供了许多新的途径[①]。第一，图形绘制流水线现在是可编程的，第9章将讨论这个主题。我们将看到由于片元处理器的可编程性，使得许多新的实

① 除了以下介绍的几点变化之外，最新GPU使用的统一渲染架构已经代替了传统的绘制流水线结构。——译者注

时纹理处理技术成为可能。第二，由于目前的GPU内置了大量的内存，所以可以消除离散处理方法的主要瓶颈之一，即在处理器内存与GPU之间需要频繁地传送图像数据。第三，通过为片元处理提供高度并行的机制，新设计的GPU结构能快速地处理离散数据。第四，目前GPU使用浮点类型的帧缓存，从而消除了图像数据处理中的许多精度问题。

本章集中介绍了各种被目前已有硬件和图形API支持的离散技术。在这些技术中，有许多是很新的技术；还有许多是在最近的文献中出现的新技术，这些新技术只有通过OpenGL扩展才能使用，尤其是基于可编程处理器开发的新技术。第9章将介绍可编程处理器。

建议阅读资料

环境映射首次由Blinn和Newell[Bli76]提出。Catmull第一次使用了纹理映射，读者可以阅读Heckbert[Hec86]有关纹理映射方面的综述。SGI公司的Reality引擎首次在硬件上支持纹理映射，读者可以阅读Akeley的文献[Ake93]。Perlin和Hoffert[Per89]设计了噪声函数，可用来生成二维和三维纹理贴图。在Ebert等的文献[Ebe98]中讨论了许多纹理合成技术。

自从光栅图形显示系统出现以来，计算机图形学中的走样问题就成为了一个非常重要的问题，有关这方面的内容可参见Crow[Cro81]的文献。研究人员首先关注的是线段光栅化中的走样问题，但后来也研究动画[Mag85]和光线跟踪[Gla89]中存在的其他形式的走样问题。有些图像处理方面的书籍[Pra78, Gon08, Cas96]介绍了二维信号处理和走样问题。Glassner的著作[Gla95]以及Watt和Policarpo的合著[Wat98]主要面向计算机图形学的实践人员。

包扩α通道的使用在内的许多合成技术都是由Porter和Duff [Por84]提出的。在OpenGL Programmer's Guide [Ope05]这本书中有许多介绍如何使用缓存的例子。最近的文献包括许多如何使用缓存的新例子。读者可以阅读最近的*Computer Graphics*期刊和*IEEE Computer Graphics and Application*期刊。

大部分标准图像格式的技术细节可以在[Mia99，Mur94]文献中找到。

习题

8.1　说明如何使用XOR写入模式实现奇偶填充算法。

8.2　使用XOR模式实现在屏幕上来回移动光标会出现什么样的视觉效果？

8.3　使用环境贴图生成的图像与对同一场景使用光线跟踪法生成的图像之间有什么区别？

8.4　在电影和电视中，汽车和四轮马车的轮子通常看起来沿错误的方向旋转。是什么原因导致了这种效果？能否修正这个问题？解释理由。

8.5　我们可能希望这样显示采样数据，即简单地绘制每个采样点，然后由人的视觉系统把这些点合成所需的形状。为什么当采样数据接近奈奎斯特极限时使用这种方法会出现问题？

8.6　为什么当演员在电视屏幕上经过时，他们穿戴的条纹衬衫和领带的图案会产生变化？

8.7　执行反走样时为什么要对数据进行预处理，而不是后处理？

8.8　假定有两个半透明的表面，它们的不透明度分别是α_1和α_2。把这两个半透明表面拼合成一个半透明材质，它的不透明度是多少？请给出计算该组合材质透明度的公式。

8.9　假定把半透明表面视为通过该表面的光的滤波器。根据补色CMY颜色模型设计一个融合模型。

8.10　在8.9节，我们使用$1-\alpha$和α分别表示目标融合因子和源融合因子。如果目标融合因子为1，而源融合因子仍为α，那么产生的视觉效果会有什么不同？

8.11 编写一个利用位图创建画刷的程序，该画刷可以对图像进行渐变着色。同时利用融合技术在程序中增加橡皮擦功能，它能渐变地擦除屏幕上的图像。

8.12 设计一个使用纹理映射技术显示三维像素阵列（即体素）的算法。

8.13 说明如何使用图像的灰度直方图得到一个颜色查找表，通过这个颜色查找表使得修正后的图像具有均匀的直方图。

8.14 当使用抖动的方法对场景进行超采样时，为什么要使用随机的抖动图案？

8.15 假定使用条纹和棋盘这样的规则图案对一组对象进行纹理映射。如果把投影方式从平行投影切换到透视投影，我们看到的走样图案会发生什么变化？

8.16 考虑一个由许多简单对象组成的场景，如许多大小不等的平行六面体实例。假定只有一个纹理贴图并要求把这个纹理映射到场景中所有对象的表面上。如何进行纹理映射才会使得每个对象的每个表面上的纹理图案具有相同的大小？

8.17 编写一个使用Mipmap纹理贴图的程序，要求其中的每个Mipmap贴图来自不同的图像。能否找到该程序的一个实际应用？

8.18 使用自己编写的图像处理程序实现卷积运算功能，或者使用OpenGL在图像处理方面的扩展为灰度图像实现一个通用的 3 × 3 的滤波程序。

8.19 通过数码相机或其他来源获取一幅数字图像，然后将 3 × 3 的平滑滤波器和锐化滤波器对该图像分别做平滑处理和锐化处理。对于处理后的图像，请仔细观察其边缘发生的变化。

8.20 重做习题8.19，但是首先要在原始图像上增加少量的随机噪声。请说明这两道习题的结果有什么区别。

8.21 如果你的图形系统支持图像处理方面的OpenGL扩展，请比较使用这些扩展实现滤波与使用处理器内存自己编写代码实现滤波在性能上的区别。

8.22 改变图像对比度最有效的方法之一就是允许用户以交互的方式设计一个查找表。考虑图8.14所示的曲线，该曲线是由3条连接的线段近似表示的。编写一个程序显示一幅图像，并允许用户以交互的方式修改线段，紧接着显示经修改后的变换曲线变换后得到的图像。

8.23 采用类似于习题8.22的方法编写一个交互式程序，它允许用户以交互的方式设计伪彩色图像变换曲线。

8.24 设计一个算法把立方体贴图的纹理值转换为球体贴图的纹理值。

8.25 编写一个交互式程序，它返回屏幕上像素的颜色。

8.26 假定希望建立一个立方体，它的每个表面都映射黑白棋盘图案的纹理。能否对立方体进行纹理映射，使得当旋转立方体的表面时表面的颜色交替变化？

8.27 试说明在什么类型的应用中，读者更愿意使用从前往后的绘制，而不使用从后向前的绘制。

8.28 色度坐标中的色域相当于用原色定义的RGB颜色空间中的颜色三角形。编写一个程序显示该颜色三角形以及位于它外面的立方体的边。该颜色三角形上每个点的颜色由它在RGB颜色空间的坐标决定。

8.29 推导一个用于NTSC制式颜色模型与RGB颜色模型之间的颜色转换矩阵，并利用该矩阵在 *xy* 色域坐标中重新显示读者所使用的显示器的色域。

第9章　可编程着色器

在设计图形绘制流水线时，假定绘制流水线中每个绘制模块的功能都是固定的。因此，当希望通过光线与材质的相互作用来计算对象表面的颜色时，就只能使用Phong光照模型，这是由于它是OpenGL规范中固定功能的绘制流水线所唯一支持的光照模型，而且直到最近，它还是大多数图形硬件唯一支持的光照模型。在这种情况下，只有一个光照模型可供使用，而且除此之外，光照计算也只能作用于每个顶点。固定功能的片元处理器在图元上对每个顶点的最终颜色通过插值计算图元上所有像素的颜色。如果希望使用其他光照或着色模型，那么必须借助于脱机绘制器。

在过去的几年中，图形处理器发生了巨大的变化。现在无论是顶点处理器还是片元处理器都是用户可编程的[①]。我们可以在顶点处理器和片元处理器上分别编写顶点着色器（Vertex Shader）和片元着色器（Fragment Shader）来生成复杂的视觉效果，其绘制速度与标准的固定功能绘制流水线一样快。

本章引入了可编程着色器的概念。首先，我们回顾一下研究人员开发用于编写着色器的程序设计语言所付出的努力，这些努力的最终成果是OpenGL着色语言（OpenGL Shading Language，GLSL），它现在是OpenGL的一个标准部分，可以用来开发各种各样的用于计算顶点位置和颜色等属性的顶点着色器。最后，我们还设计了片元着色器，可以用它来编写程序对每个片元进行计算，并最终计算出每个像素的颜色。通过对片元着色器的讨论，还将介绍许多纹理映射的新方法。

9.1　可编程流水线

第6章提出Phong模型和改进的Phong模型时，强调的主要是其计算效率，而对物理上光线与材质是如何相互作用的却考虑得很少。改进的Phong模型在这方面尤为突出，它尽管与物理上的真实感没有太大关联，但是使用该模型绘制的图像却能适用于很多场合。因此，随着图形硬件，尤其是图形绘制流水线结构的发展，就很自然地把改进的Phong模型加入OpenGL规范，并在商用图形硬件中加以实现。

改进的Phong模型适合于模拟光滑的平面，例如塑胶制品或金属，也就是具有各向同性（isotropic）的BDRF（Bidirectional Reflectance Distribution Function，双向反射率分布函数）属性的材质。这种材质的属性在各个方向上都是相同的。然而，一般情况下，我们需要一种在物理上更加真实的模型，它能够模拟大多数真实世界中具有非对称或各向异性（anisotropic）属性的材质，这类材质包括皮肤、织物、液体等。当处理半透明材质时，还需要考虑折射（refraction），即光线穿过具有不同属性的两种材质时发生的偏转现象。我们还希望能够解释这样一个事实，即折射光线的偏转角度是光的波长的函数。

在其他情况下，我们需要一些非真实感图形的绘制效果。例如，我们可能需要模拟一个画家的笔画风格，或创建类似卡通的着色效果，而这样的效果大都只能在片元级别上经处理才能获得，而

① 目前最新的图形处理器中增加了一个可编程单元，即几何处理器，Geometry Processor。——译者注

使用固定功能的绘制流水线是无法做到的。例如，对于9.12节将要讨论的凹凸映射，通过改变每个片元法向量的方法，使得当光源或对象表面移动时，对象表面呈现出逼真的复杂效果。这些算法或效果大多数是在20多年前设计的，但只能使用像RenderMan这样的非实时绘制器才能产生所需的绘制结果。最近在图形绘制结构上所取得的进展已经极大地改变了这种状况。

考虑图9.1所示的绘制流水线结构，该图与第1章讨论的绘制流水线结构相同。首先，我们处理顶点，这时需要注意，由于顶点处理过程要进行模–视变换和投影变换，所以顶点在观察坐标系里的表示只能发生在顶点处理器中。顶点处理结束后，每个顶点都经过了模–视变换和投影变换，从而得到一个新的位置坐标，将颜色值赋给每个顶点，并且根据程序中状态的设置情况，可能还要将纹理坐标等其他属性赋给每个顶点。然后，把顶点组装成图元并进行裁剪处理。没有被裁剪掉的潜在的可见图元经过光栅化后产生片元。最后，片元被处理后生成最终的显示图像。与固定功能的绘制流水线不同，现在顶点处理器和片元处理器都是可编程的，即用户可以编写称为着色器（Shaders）的应用程序，这些程序经编译后装载到相应的图形处理器中。

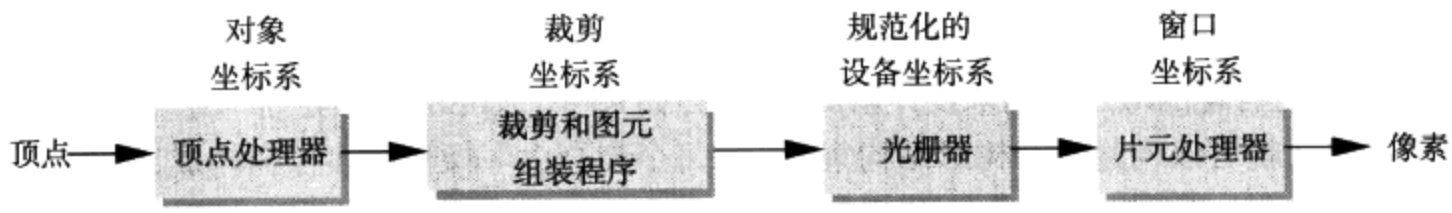


图 9.1　流水线结构

9.2　着色语言

在为着色器设计编程模型之前，首先需要一种描述光照模型和着色器的方法。受RenderMan着色语言的启示，我们将把着色器看做是某种程序设计语言中的数学表达式，包括变量、常量以及它们之间的运算操作；也可以用树结构来描述这种表达式，这样就可以利用树的各种遍历算法来求解该表达式的值。因此，在描述表达式的语言和计算这些表达式的算法之间就产生了直接关联。

9.2.1　着色树

考虑第6章介绍的Phong明暗处理模型，在不考虑衰减距离项时对应的公式为：

$$I = k_d L_d \mathbf{l} \cdot \mathbf{n} + k_s L_s (\mathbf{r} \cdot \mathbf{v})^{\alpha} + k_a L_a I_a$$

当图形系统计算该表达式时[①]，**n**和**l**的值可以从OpenGL的状态得到，而**r**可以按照下面的公式计算得到：

$$\mathbf{r} = 2(\mathbf{l} \cdot \mathbf{n})\mathbf{n} - \mathbf{l}$$

这两个公式都要用到包括指数运算在内的算术运算和包括点积运算在内的矢量运算。可以使用如图9.2所示的树结构[②]来表示这些方程，我们把这种类型的树称为表达式树（expression trees）。变量和常量出现在终止节点上，而所有内部节点都用来表示运算。因为所有的运算都是二元运算，所以最后得到的树是一棵所有内部节点都仅有两个孩子节点的二叉树。

① 为了防止点积出现负值，在实际计算时使用的是$f * \max(0, \mathbf{l} \cdot \mathbf{n})$和$\max(0, \mathbf{r} \cdot \mathbf{v})$，其中，如果$\mathbf{l} \cdot \mathbf{n}$的结果为正数，则因子$f$等于1，否则等于0。

② 第10章将深入讨论树结构的使用。

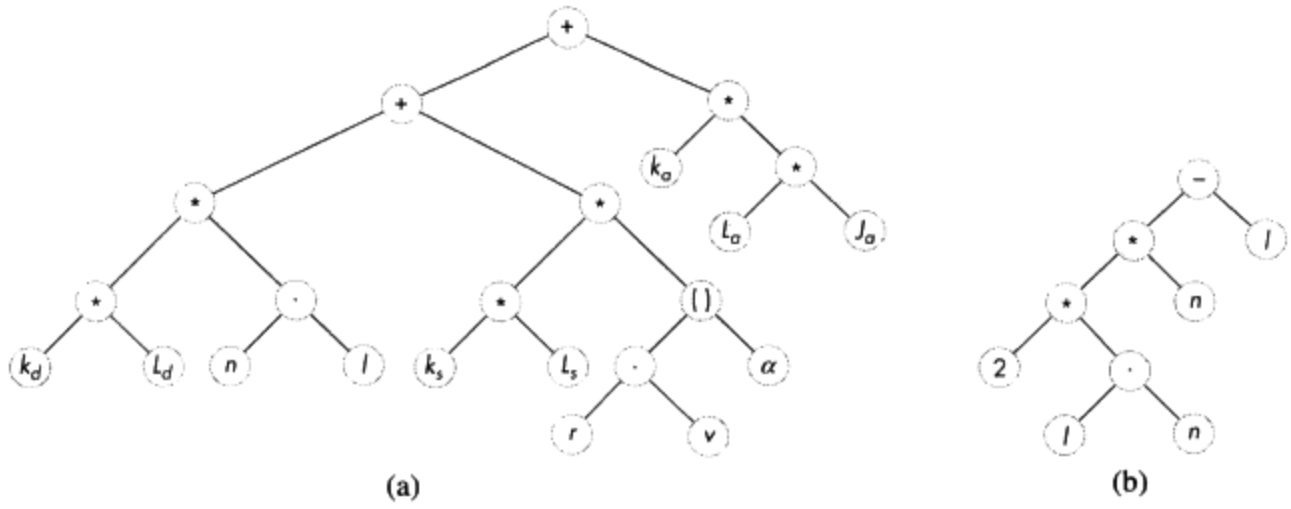


图 9.2　表达式树：(a) 用于 Phong 明暗处理的表达式树；(b) 用于计算反射矢量的表达式树

计算一个算术表达式相当于遍历其对应的树结构，也就是访问每个节点，并执行内部节点所需的数学运算。从这个意义上讲，树形结构及其遍历算法为数学表达式（例如Phong着色器的计算公式对应的数学表达式）的表示和计算提供了一种方法。

表达式树更有趣的应用是用来设计新的着色器。如果一个图形系统给定了一组变量及它所支持的运算符，那么就可以构建一个树形结构集，其中每个集合由一棵树或多棵树构成，定义了一种不同的表面着色方法。RenderMan着色语言采用的就是这种方法。假定法向量和光源参数等变量来自环境，并且可以使用一组标量和矢量运算组合起来构成表达式树。根据数据结构的基本理论，我们知道算术表达式与其对应的二叉树是等价的。因此，我们可以使用标准的遍历树的算法来计算用来定义着色器的表达式的值。从另外一个稍微不同的角度看，着色树可以被一个由图形处理器运行的程序所替代。这种开发着色器的编程方法得到了高级语言的支持，可以使用这种高级语言在最新的图形卡上对绘制流水线的可编程器件进行编程。

9.3　OpenGL 扩展

在详细介绍可编程着色器的细节之前（这些细节进一步增加了 OpenGL 的复杂性），我们暂时讨论一下 OpenGL 的扩展机制，这种机制能够把一些最新的技术纳入图形硬件和软件系统中，从而使 OpenGL 能紧跟图形学的发展步伐。诸如 OpenGL 等图形 API 的发展为应用程序编程人员访问最新图形硬件提供的硬件特性提供了一个界面。当可供使用的图形硬件特性越来越多，处理器的处理速度越来越快，提供的内存越来越大时，更复杂的图形处理技术便成为可能，甚至成为常规技术。应用程序编程人员自然希望这些图形处理技术能得到图形 API 的支持。

API开发人员必须面对的一个问题是：如何通过已有的API开发新的API来支持这些最新特性，而又不必让程序员修改已有的程序代码。一种方法是发行新版本的API，新版本增加了一些新特性，但向后兼容以保证旧版本下的程序代码使用新版本API仍能正常运行；另一种办法是在已有API的基础上增加一些可选的扩展项，这使得应用程序可以访问特定的硬件特性。这两种机制在OpenGL里都被采用了。然而，为了访问可编程图形处理器的新特性，还需要一套新的编程工具。

9.3.1　OpenGL 版本和扩展

OpenGL的一个主要特性就是它的API非常稳定。在过去十几年里，OpenGL发布了多个版本，从 OpenGL 1.0 升级到 OpenGL 1.5，每个版本都兼容其前面的版本。2004 年发布的 OpenGL 2.0 版本虽然在功能上做了重大更新，但是仍然保持了对以前版本代码的兼容性。当前的最新版本是 OpenGL 2.1。因此，基于OpenGL旧版本编写的任何程序在其后继版本上都能够如常运行。OpenGL

版本的变化反映了硬件技术的最新发展，这些硬件技术在许多图形处理器上得到了广泛应用。例如，随着硬件对纹理映射支持力度的增大，新版本的OpenGL包含了许多用于纹理映射的新函数，如纹理对象、Mipmap映射和三维纹理等特性都增加到了OpenGL的新版本[①]中。

鉴于OpenGL版本代表的是大多数用户和硬件供应商的共识，特定的图形卡或高端工作站可以用来支持某些在其他图形硬件上不具备的特性，尤其是用于开发计算机游戏的商用图形卡就属于这种情况。虽然如此，拥有这类特定图形硬件的编程人员还是希望能够通过OpenGL函数访问这些硬件的特性，尽管这些特性不是很通用，甚至没有得到最近版本API的支持。解决这个问题的办法之一就是使用OpenGL的扩展机制。个别的硬件供应商通过提供新的OpenGL函数供用户访问硬件特性，但这些函数可能只在某些特定制造商提供的图形硬件产品中才能正常工作，供应商是通过OpenGL的扩展函数名来识别的。例如，一个具有类似glCommandNV的扩展名表示该函数是由NVIDIA公司提供的。其他硬件制造商在他们的硬件上也能实现同样的扩展。那些被广泛使用的扩展在得到OpenGL结构评审委员会（Architectural Review Board，ARB）认可后被命名为ARB扩展[②]。

当可编程的顶点着色器首次实现时，OpenGL结构评审委员会就批准其作为低级顶点程序的扩展，之后又能支持低级片元程序。然而，最初的可编程硬件接口并没有得到广泛的使用，最初的支持可编程硬件的软件接口类似于通用计算机的汇编语言接口，应用编程人员不得不编写相当冗长的程序用来跟踪底层的细节，例如图形处理器的寄存器赋值和寄存器与内存之间数据的移动等。OpenGL扩展提供了一种机制用来将这种类汇编语言的代码装载到可编程绘制流水线中，尽管这种机制允许用户使用可编程着色器，但它具有汇编语言的所有缺点。随着GPU结构变得越来越复杂，使用汇编语言编写着色器也变得越来越困难。目前对大多数用户来说，用于编写着色器代码的高级编程接口和编译器已经取代了汇编语言编程接口，而这正与标准的程序设计语言的发展过程相类似。

9.3.2　GLSL和Cg

两种最令人感兴趣的高级着色语言是OpenGL着色语言（OpenGL Shading Language，GLSL）和Cg（它是C for Graphics首字母的缩写），这两种语言非常相似，但具有不同的目标用户。两种语言都是基于C语言风格并且包含了C语言的大部分编程结构，但它们都增加了便于用户编写着色器的语言特性和数据类型。

两种语言的主要区别在于，Cg被设计用来支持具有可移植性的跨平台（包括OpenGL和微软的DirectX）着色器。Cg实际上与微软的高级着色语言（High Level Shading Language，HLSL）完全相同，因此对于Windows开发人员来说具有优势，它允许开发人员同时开发DirectX和OpenGL着色器。然而，OpenGL与Cg着色器之间的接口要比OpenGL与GLSL之间的接口复杂。GLSL是OpenGL 2.0的一部分，因此得到了许多硬件供应商和许多平台的支持，从而使用GLSL开发OpenGL着色器要比使用Cg更简单。因此，本书将重点介绍GLSL，但同时要明白，这两种语言的相似之处远多于它们的不同之处。

9.4　OpenGL着色语言

OpenGL着色语言是一个类似于C语言语法的高级语言，程序员可以用它编写顶点着色器和片元着色器。OpenGL 2.0规范包含OpenGL着色语言。尽管使用GLSL语言编写的顶点程序和片元程

① 原文为早期版本，可能有误。——译者注

② 在2006年，ARB被Kronos协会下的OpenGL工作组取代。参见www.opengl.org。

序是用于不同上下文中的，但在语法上却只有非常小的差别。在讨论GLSL语言之前，首先分析这两种着色器必须执行的不同任务。

9.4.1　顶点程序

顶点程序（vertex program）或顶点着色器（vertex shader）将使用其自定义的运算操作代替由顶点处理器执行的固定功能的运算操作。如果应用程序没有提供顶点程序，那么可编程的顶点处理器就会执行OpenGL固定功能顶点处理器中的标准运算操作。对于通过绘制流水线顶点处理器的每一个顶点都要执行一次顶点程序。每个顶点程序都必须输出一些供光栅器做进一步处理所需的信息，如至少应输出顶点位置信息供光栅器使用。对于每个顶点，都可以使用应用程序定义的顶点位置信息和大部分OpenGL状态信息作为顶点程序的输入，这些状态信息包括当前的颜色、纹理坐标、材质属性和变换矩阵等。此外，应用程序还可以针对每个顶点将其他与应用程序相关的信息传递给顶点程序。

有一些运算操作实际上是需要在每个顶点程序中都要执行的。回顾前面介绍的内容，我们知道大多数应用程序是在对象空间中定义顶点的位置，顶点处理器首先使用模-视变换矩阵将这些顶点的位置变换到观察坐标系，然后再使用投影变换矩阵将这些顶点变换到裁剪坐标系。因为固定功能的顶点运算操作被应用程序提供的顶点程序所替代，所以几乎所有顶点程序都必须执行的一项任务就是把输入顶点程序的顶点位置从对象坐标系变换到裁剪坐标系。因为顶点程序能够访问OpenGL的状态，所以它可以访问标准的变换矩阵，或者计算顶点程序内部的变换矩阵。

下面是一个简单且完整的顶点程序：

```
/* 直通顶点程序 */

void main(void)
{
    gl_Position = gl_ProjectionMatrix*(gl_ModelViewMatrix*gl_Vertex);
}
```

这个顶点程序只是把对象坐标系中的每个顶点位置（gl_Vertex）首先与模-视变换矩阵相乘，然后与投影变换矩阵相乘，最后得到位于裁剪坐标系中的顶点位置（gl_Position）。顶点程序中的4个变量都是OpenGL的状态变量，因此不必在顶点程序中声明。应用程序中每次调用glVertex函数都会触发顶点程序的执行。这个顶点程序太简单，没有涉及设置顶点颜色或其他顶点属性等操作，这些工作留到绘制流水线后面的片元处理器中来处理。因为这个顶点程序除了传送顶点的位置信息外不做任何其他操作，所以有时把它称为“直通着色器”（pass through shader）。

下面是一个稍微复杂的顶点程序，该程序把每个顶点的颜色设置为红色：

```
/* 简单的顶点程序 */
const vec4 red = vec4(1.0, 0.0, 0.0, 1.0); /* c++风格的构造函数 */

void main(void)
{
   gl_Position =  gl_ModelViewProjectionMatrx*gl_Vertex;
   gl_FrontColor = red;
}
```

该程序完成两个任务。第一个任务是把输入的顶点位置（gl_Vertex）与投影变换矩阵和模-视变换矩阵连续相乘，将其变换到裁剪坐标系中的新位置（gl_Position）。另一个任务是将每个顶点的颜色

设置成红色[①]。因为把顶点的位置从对象坐标系变换到裁剪坐标系是一项很常见的任务，所以GLSL语言通过内置变量 gl_ModelViewProjectionMatrix 提供了预先计算好的投影变换矩阵和模－视变换矩阵的乘积。注意，以 gl_ 为前缀的变量名称说明该变量属于 OpenGL 的状态变量。假定使用这个顶点程序的某个应用程序包含下面的代码：

```
glBegin(GL_POLYGON);
  glVertex3fv(v0);
  glVertex3fv(v1);
  glVertex3fv(v2);
glEnd();
```

其中，顶点 v0、v1 和 v2 在应用程序的前面已经定义好了。每当应用程序调用 glVertex 函数时，该函数就会触发顶点程序并把一个新的顶点位置（gl_Vertex）参数传送给它，顶点位置参数采用顶点的四维齐次坐标表示形式。顶点程序每次运行都将输出一个颜色值（gl_FrontColor）和一个新的顶点位置（gl_Position），这些输出数据将沿绘制流水线传送给图元组装程序。图9.3说明了这个过程。在这个顶点程序中，由于计算了顶点颜色，所以每个顶点输出时可能被设置了不同的颜色。我们也可以在应用程序中直接使用内置变量gl_Color为每个顶点设置颜色，顶点程序可以使用下面的代码将颜色值传送给光栅器：

```
gl_FrontColor = gl_Color;
```

注意，顶点程序有一个以 main 命名的函数，它可以调用其他使用 GLSL 语言编写的函数。这种调用关系同样适用于片元程序。

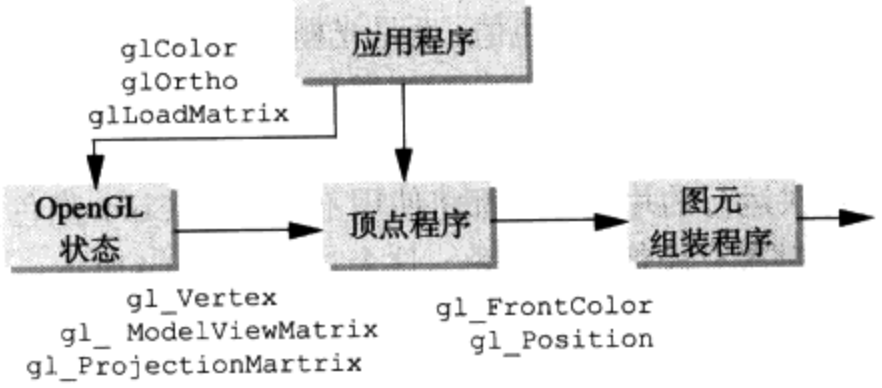


图 9.3　顶点程序的结构

GLSL语言包含了一些新的数据类型和它们之间相关的运算符。其中，模－视变换矩阵和投影变换矩阵的数据类型是mat4，而顶点位置的数据类型是vec4，乘法运算符*已被重载，这样矩阵和矢量就可以如愿地直接相乘。因此，在前面那个简单的顶点程序中有如下代码，最后生成的位置数据类型是 vec4：

```
gl_Position = gl_ModelViewProjectionMatrix*gl_Vertex;
```

到目前为止，我们一直使用列矩阵和行矩阵而不使用矢量这样的术语，主要是为了避免混淆作为几何类型的矢量与使用行或列矩阵表示的矢量。GLSL 语言定义了 vector 矢量数据类型，相当于 C 语言的一维数组。GLSL 语言还保留了用于表示矩阵的 matrix 数据类型，相当于 C 语言的二维数组。本章采用 GLSL 语言的术语。

在前面的例子中，我们使用GLSL语言的内置变量gl_FrontColor和gl_Position作为顶点程序的输出。正面顶点颜色值（front color）用于光栅器进行颜色插值计算，插值得到的颜色传送到绘制

① 这里假定没有开启对象的两面（正面和背面）光照效果计算模式。如果需要计算对象的两面光照效果，由于现在背面可见，且正面与背面的材质不相同，那么我们编写的顶点程序还需要计算 gl_BackColor。

流水线后面的片元程序或固定功能的片元处理器。光栅器使用每个顶点的位置来指定它产生的每个片元的位置。注意，定义的red颜色矢量前面有一个修饰符const，这个修饰符确保每次触发的顶点程序使用的颜色不变，而通常情况是：每次执行顶点程序后输出的顶点位置和颜色值都会改变。同时还要注意使用C++风格的构造函数来初始化red颜色矢量。

9.4.2　片元程序

使用GLSL语言编写的片元程序与顶点程序具有相同的语法。然而，片元程序在光栅器之后执行，因此它们处理的对象是每个片元而不是每个顶点。

再次分析前面的小示例，该顶点程序把对象正面的颜色传送给图元组装程序和裁剪处理过程。如果某个顶点没有被裁剪掉，则首先会被传送到图元组装程序，然后传送到光栅器，由光栅器生成片元，然后这些片元要么被固定功能的片元处理器处理，要么被应用程序定义好的片元程序（fragment program）或片元着色器（fragment shader）处理。顶点颜色和位置等属性通过光栅器的插值计算，得到图元其他所有点的颜色和位置，从而生成对应的每个片元的属性。对于最简单的情况，片元处理器仅仅使用这些片元属性而不对它们做任何修改。一个最简单的片元程序如下：

```
/* 直通片元程序 */
void main()
{
   gl_FragColor = gl_Color;
}
```

这个程序虽然看起来比较简单，但它的运行方式与顶点程序有很大的区别。片元程序中gl_Color的值并不是在顶点程序中生成的颜色值，而是光栅器在图元的表面根据顶点处理器生成的gl_FrontColor顶点颜色值（如果对象的正面和背面光照计算都开启，还包括gl_BackColor）进行插值计算得到的片元颜色值，片元的颜色值再传送给片元程序做进一步处理。因此，除非所有的顶点具有相同的颜色值，否则每次运行的片元程序都将使用不同的gl_Color颜色值。最后，片元程序生成的片元颜色可以用来修改帧缓存中像素的颜色。这个过程如图9.4所示。下面介绍GLSL语言的主要特性，然后设计一些更复杂的片元程序。

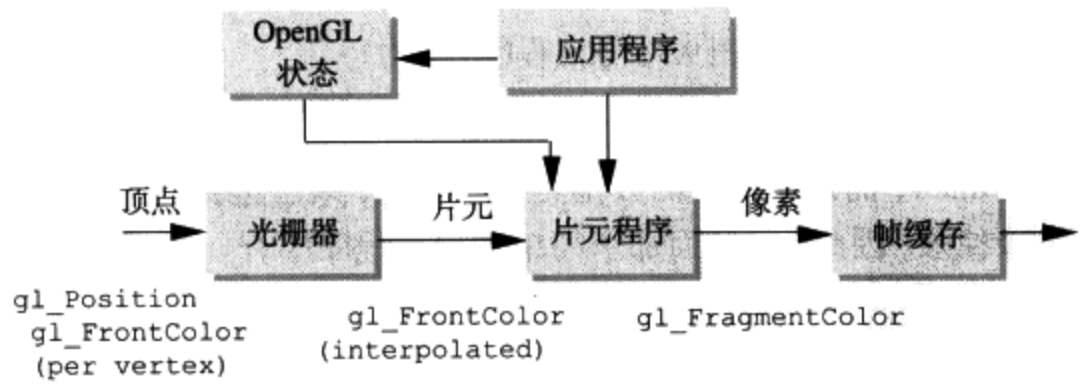


图9.4　片元程序的结构

9.5　OpenGL着色语言

OpenGL着色语言基于C程序设计语言，并且与C语言具有类似的命名规则、数据类型和控制结构。然而，由于顶点程序和片元程序的执行环境与普通程序（包括调用它们的OpenGL应用程序）的执行环境大不相同，所以GLSL语言与C语言之间还是存在显著区别的。

9.5.1　GLSL执行过程

现在分析一个典型的OpenGL应用程序的运行过程。大多数OpenGL函数只能改变OpenGL的状态，而不会使对象的任何顶点数据沿绘制流水线向后传送而改变对象的显示效果。然而，一旦调用glVertex函数或任何能产生顶点的OpenGL函数，绘制流水线就开始工作。对于固定功能的绘制流水线，为了确定顶点所属的图元是否可见，需要进行各种各样的计算。如果图元可见，就给帧缓存中相应的像素着色，这些计算并不改变OpenGL的状态。其后运行的glVertex函数会产生同样的计算过程，但如果在两次调用glVertex函数之间OpenGL的状态值改变了，那么后一个glVertex函数将使用更新后的状态值。

如果把这些计算看做是实现顶点处理器的软件和硬件所做的工作，发现该程序运行过程中需要多种不同类型的变量。其中一些变量会随着输出的顶点属性（如顶点颜色属性）的计算而改变，而另一些变量的值取决于OpenGL状态的变量，并不会在计算过程中改变。在每个顶点被处理之后，为了使下一个顶点的计算过程保持不变，大多数内置变量必须初始化为最初的值。其他变量必须经过计算并将其结果传送到绘制流水线的下一个处理过程。因此，当用用户编写的顶点程序取代固定功能的顶点处理器时，必须能够区分这些不同类型的变量。

一个片元程序同样具有输入变量、内置变量和输出变量。片元程序和顶点程序的主要区别在于片元程序处理的对象是每一个片元而不是顶点。片元程序中一些变量的值由OpenGL的状态提供，并不会随计算过程而改变。另一些基于片元到片元模式的变量在片元程序中可能会改变。大多数片元程序的变量必须在对片元处理之前进行初始化。

一个典型的应用程序不仅可以改变状态变量和纹理贴图等实体对象的值，而且还可能会使用多个顶点程序和片元程序。因此，还需要讨论如何加载多个着色程序，如何将它们链接成一个GLSL程序，以及如何将这个GLSL程序设置成能在OpenGL中使用的程序。这个过程并不是GLSL语言的一部分，而是要通过OpenGL内核中的一组函数来完成。

因为每个顶点都要在独立于任何其他顶点的情形下触发当前顶点程序的运行，同理，每个片元触发当前片元程序的运行，所以存在利用大规模并行机制加速处理顶点和片元的潜力。目前高端图形处理器包含了能并行执行顶点和片元程序的多核可编程着色器。

9.5.2　数据类型与修饰符

GLSL语言的基本数据类型与C和C++语言类似，但它的标量数据类型目前还只限于单精度浮点类型float、单一的整数类型int和布尔类型bool。数组和结构体采用标准C的方式定义。

GLSL语言引入了矢量和矩阵这两种新的数据类型，用于表示在计算机图形学中所使用的2 × 2、3 × 3和4 × 4的矩阵。其中，矢量是一种特殊的一维数组，有多种类型的矢量可供使用，包括浮点矢量（vec2, vec3, vec4）、整型矢量（ivec2, ivec3, ivec4）和布尔矢量（bvec2, bvec3, bvec4）。矢量类型可用来存储顶点位置、颜色或纹理坐标等，其中的元素可通过位置（x, y, z, w）、颜色（r, g, b, a）或纹理坐标（s, t, p, q）的索引下标来获取。因此，如果c是一个矢量，那么c[1]、c.y、c.g和c.t都表示c的第二个成员。注意，这种表示方法的灵活性只是为了使编写的代码可读性更强，而不含任何语义信息。因此，没有理由说c.y不包含颜色或纹理信息。

目前，GLSL语言中的矩阵都是行数和列数相等的浮点类型矩阵（mat2, mat3, mat4），与OpenGL中的矩阵一样都是采用按列存储的方式。不过，矩阵元素的引用方式与C/C++相同，如果m是一个矩阵，那么m[1]表示第2行，而m[1][2]表示第2行、第3列的元素。GLSL语言支持标准C语言的一维数组，并采用类似C语言的结构体类型来实现多维数组的功能。

GLSL语言采用C++风格的构造函数对矢量和矩阵进行初始化，例如，下面的代码就完成了对一个矢量的初始化：

```
vec3 a = vec3(1.0, -2.0, 5.0);
```

构造函数还可以用于数据类型之间的转换，例如：

```
vec2 b = vec2(a);
```

它使用vec3类型的变量a的前两个分量形成一个新的vec2类型的变量b。

变量的前面可以加上各种不同的修饰符。对于那些不是函数参数或函数临时变量的一般变量（非局部变量）来说，可以在它们前面加attribute、uniform、varying或const等修饰符。const修饰符与C语言中的const类似但更加严格，它确保对应的变量值不会被着色器改变。因此，可以按下面的方式用const来定义值恒定不变的标量或矢量：

```
const float one = 1.0;
const vec3 origin = vec2(1.0, 2.0, 3.0);
```

attribute修饰符用来修饰顶点程序中每个顶点至多变化一次的变量（属性变量）。有两种不同类型的attribute变量：第一类是与顶点关联的OpenGL状态变量，如颜色、位置、纹理坐标和法向量等，这些就是所谓的内置变量（built-in variables），它不需要在着色器中声明就可直接使用，譬如在前面那个简单的例子里，gl_Vertex和gl_Color都是内置的由attribute修饰的属性变量；第二类是GLSL语言所允许的程序员在应用程序中定义的其他顶点属性，这样便于把其他基于顶点的信息传送给着色器。例如，在科学计算可视化应用中，我们可能希望把一个用标量表示的温度或一个用矢量表示的流速作为每个顶点的额外属性。只有浮点类型变量前可以加attribute修饰符。用户可以用下面的代码自定义带attribute修饰符的属性变量：

```
attribute float temperature;
attribute vec3 velocity;
```

因为代码中顶点属性变量值的改变是基于每个顶点的，所以这些变量就不能在片元程序中声明。9.6节将会介绍如何使用OpenGL函数把属性变量与应用程序中的变量绑定在一起。当顶点程序运行时，程序中的属性变量值就是在应用程序中被绑定的变量值，而不会在顶点程序中发生改变。

uniform修饰符主要用在应用程序中针对一组图元而定义的变量上，也就是在glBegin与glEnd范围之外赋值的变量。uniform变量提供了一种在应用程序、顶点程序和片元程序之间共享数据的机制。和attribute变量一样，uniform变量的值在着色器中也不会发生改变，因此可以利用uniform变量把对于一组图元都不变的数据传送到着色器中。例如，如果想要对应用程序中定义的一组顶点的包围盒进行计算，那么为了简化计算过程，只需要把这个包围盒数据传送给着色器即可。

虽然将变量称为“变化的变量”（即在变量前面加varying修饰符）乍听起来有些奇怪，但是用varying修饰的变量提供了一种把数据从顶点程序传送到片元程序的机制。这些变量虽然是基于每个顶点定义的，但是由光栅器根据它们的值在图元的表面进行插值计算就能得到每个片元的数据。与attribute修饰的变量一样，varying修饰的变量既可以是内置的，也可以是用户自定义变量。

现在分析一下颜色属性是如何在固定功能绘制流水线上确定的。顶点处理器为每个顶点计算颜色或明暗值，片元的颜色是由光栅器通过对顶点的颜色值进行插值计算得到的。同理，片元的纹理坐标也是通过对每个顶点纹理坐标进行插值计算得到的。顶点颜色和纹理坐标都是varying变量。在前面那个很简单的顶点程序里，gl_FrontColor是一个内置的varying变量。如果编写一个更复杂的顶点程序，它可以为每个顶点计算不同的颜色，那么使用varying修饰的变量gl_FrontColor就能确保每个片元都有相应的插值颜色，无论是否编写自己的片元程序。

光栅器能够自动对顶点程序中用户自定义的varying变量进行插值计算。如果仅仅是在顶点程序中定义了varying变量，而在片元程序中却并不使用它，这将是毫无意义的。因此，用户自定义的varying变量必须在顶点程序和片元程序中都要出现。所以，如果在顶点程序中定义了一个varying变量，那么就应该再编写一个相应的片元程序对它做进一步处理。虽然之前那个简单的顶点程序并不需要这样一个片元程序，但在功能上与其等价的顶点程序代码如下：

```
const vec4 red = vec4(1.0, 0.0, 0.0, 1.0);

varying vec4 color_out; /* varying变量 */
void main(void)
{

   gl_Position =  gl_ModelViewProjectionMatrix*gl_Vertex;
   color_out = red;
}
```

这个顶点程序所需要的一个最小的直通片元程序代码如下：

```
varying vec4 color_out;

void main()
{
   gl_FragColor = color_out;
}
```

9.5.3　运算符与函数

GLSL语言大部分运算符的优先级规则与C语言相同。然而，由于GLSL语言没有指定浮点类型和整数类型的内部格式，所以该语言不支持位操作运算[①]。

C语言常用的类型运算符以及GLSL特有的矢量和矩阵类型运算符都已经被重载定义，所以矩阵与矢量这两种运算符可以如我们希望的那样使用，例如下面的代码都是有意义的：

```
mat4 a;
vec4 b, c, d;

c = b*a;
d = a*b;
```

在上面的代码中，GLSL语言把矢量b当做一个行矩阵来计算c，而把矢量b当做一个列矩阵来计算d。因此，尽管c和d都是相同的矢量类型，但是它们的值却不一样。

GLSL语言还有一个运算符称为混合运算符（swizzling operator），它是C语言选择运算符（.）的一个变体。混合运算符允许程序员从矢量类型中选择多个元素，我们可以通过使用混合运算符编写掩码来选择或重新排列矢量的元素。例如，下面的代码就可以改变从矢量中选择的元素的值：

```
vec4 a =  vec4(1.0, 2.0, 3.0, 1.0);

a.x = 2.0;
a.yz = vec2(-1.0, 4.0);
```

也可以按下面代码所示的方式交换矢量中的元素：

```
a.xy =  a.yx;
```

注意，在使用混合运算符时，其后面列出的分量名称可以是（x, y, z, w）、（r, g, b, a）或（s, t, p, q）的任何一种形式，只要不在同一个混合运算符后面对它们进行混合使用即可。

① 许多GPU通过扩展支持位操作。

GLSL语言有许多内置的函数，包括三角函数（sin, cos, tan）、反三角形函数（asin, acos, atan）和数学函数（pow, log2, sqrt, abs, max, min）等。其中比较重要的是用于几何计算的函数，这些函数与计算机图形学中用到的矢量有关。因此，GLSL语言中内置了这样一些几何计算函数，例如长度函数、距离函数、点积函数、矢量归一化函数和反射函数。这些函数大多数都已被重载定义，既可以处理浮点数据，也可以处理矢量数据。我们在后面将举例说明这些函数及其他一些内置函数的用法。纹理函数将在9.10节中讨论。

作为函数参数的变量需要在其前面附加一些其他的修饰符。因为顶点程序和片元程序特有的执行方式以及该语言缺乏指针类型，所以GLSL语言使用了一种称为按值－返回调用（call by value-return）的函数调用机制。函数的参数使用in（默认修饰符）、out或inout修饰符。GLSL函数具有返回类型，被返回的变量是通过将其从被调用函数中复制到调用函数中来实现的。输入的参数也是通过复制的方式从调用函数传递到被调用函数。如果函数的一个参数前面有in修饰符，那么它只能通过复制的方式接受调用函数传递过来的参数，而不能在被调用函数返回时再将其值复制给调用函数的参数，即使这个参数的值在被调用函数中发生了改变；被out修饰的参数在被调用函数中是不能定义的，但是在被调用函数中可以改变它的值，在被调用函数返回时再将其值以复制的方式返回给调用函数；被inout修饰的参数不仅有in修饰符的参数传入过程，而且还有out修饰符的参数返回过程。

注意，目前GLSL语言还不支持指针类型。因为矢量和矩阵属于基本的数据类型，所以它们和其他基本数据类型一样可以以复制的方式传入函数或从函数中返回。

9.6　着色器与OpenGL程序的连接

使用GLSL语言编写的着色器是独立于任何OpenGL应用程序的。虽然目前有各种各样的开发环境可供用户编写和测试着色器，但在某些时候，必须把着色器与OpenGL应用程序连接起来才能运行。

有一组OpenGL函数专门用来处理这样的连接工作，包括如何创建顶点对象和着色器对象，如何将这些对象与OpenGL应用程序进行连接，以及利用变量在OpenGL应用程序与着色器之间传送数据。

通常使用以下8个步骤来对应用程序中的一个或多个着色器进行初始化：

1. 将着色器读入内存。
2. 创建一个程序对象。
3. 创建着色器对象。
4. 把着色器对象绑定到程序对象。
5. 编译着色器。
6. 将所有的程序连接起来。
7. 选择当前的程序对象。
8. 把应用程序和着色器之间的uniform变量及attribute变量关联起来。

通常，程序员编写的着色器是以源码形式存储在文件中的，并由应用程序将其读入内存，之后OpenGL把着色器看做非终结字符串，这是OpenGL使用着色器最常见的一种方式。例如，下面的代码读入一个文件并创建所需的字符串：

```
#include <stdio.h>

static char* readShaderSource(const char* shaderFile)
{
   /* 着色器读入程序 */
   /* 从文件创建非终结字符串 */

    FILE* fp = fopen(shaderFile, "r");
    char* buf;
    long size;

    if(fp==NULL) return(NULL);
    fseek(fp, 0L, SEEK_END); /* 搜寻到着色器文件的结束位置*/
    size = ftell(fp);        /* 获取文件的大小 */
    fseek(fp, 0L, SEEK_SET); /* 搜寻到着色器文件的起始位置*/

    buf = (char*) malloc((size + 1) * sizeof(char));
    fread(buf, 1, size, fp);
    buf[size] = ' ';
    fclose(fp);
    return buf;
}
```

上面的代码使用C语言的文件函数获取它所打开的着色器文件的大小，分配相应大小的内存空间并把着色器文件读入字符串里，最后用一个空字符作为该字符串的结束符。

假设给定下面两个着色器文件：

```
GLchar vShaderFile[] = "myVertexShader.glsl";
GLchar fShaderFile[] = "myFragmentShader.glsl";
```

可以使用下面的代码把着色器读入内存并检测这些文件是否存在：

```
GLchar *vSource, *fSource;

vSource = readShaderSource(vShaderFile);
if(vSource == NULL)
{
   printf("failed to read vertex shader");
   exit(EXIT_FAILURE);
}
fSource = readShaderSource(fShaderFile);
if(fsource == NULL)
{
   printf("failed to read fragment shader");
   exit(EXIT_FAILURE);
}
```

现在就可以创建一个程序对象（program object）并给定它的名字（或标识符）：

```
GLunit myProgObj;
myProgObj = glCreateProgram();
```

程序对象是一个容器，可以存放多个着色器和其他GLSL函数。如下面的代码所示，可以按类似的方式创建着色器对象：

```
GLuint vShader, fShader;
vShader = glCreateShader(GL_VERTEX_SHADER);
fShader = glCreateShader(GL_FRAGMENT_SHADER);
```

之后，就可以通过下面的代码把这些着色器对象添加到程序对象中：

```
glAttachShader(program, vShader);
glAttachShader(program, fShader);
```

注意，此时着色器代码还没有和程序对象产生真正的关联，下面的代码才真正创建了这两者之间的关联：

```
glShaderSource(vShader, 1, (const GLchar**) &vSource, NULL);
glShaderSource(fShader, 1, (const GLchar**) &fSource, NULL);
```

glShaderSource函数的第一个参数是着色器对象的名字，用来装载着色器的源码。第二个参数是字符串缓冲区的编号，函数从该缓冲区中取出着色器源码。在上面的这个例子里，我们使用函数readShaderSource把着色器源码读入一个单一的缓冲区里。第三个参数是一个指向指针数组的指针，此数组的每个指针都指向对应的字符串缓冲区，就上面介绍的这个例子而言，这个指针指向一个字符串数组，该数组存储readShaderSource函数中用到的着色器源码。第四个参数NULL表明字符串以null作为结束标志，这个参数也可以是所指向的缓冲区数组的长度值。现在可以使用下面的代码编译着色器：

```
glCompileShader(vShader);
glCompileShader(fShader);
```

这时，我们希望确定着色器代码及其编译过程都没有错误。下面的函数在状态变量不等于GL_TRUE时输出一条错误信息并退出程序[①]：

```
static void checkError(GLint status, const char *msg)
{
if (status != GL_TRUE)
   {
    printf("%s", msg);
    exit(EXIT_FAILURE);
   }
}
```

着色器的状态信息通过查询函数glGetShaderiv获取，该函数负责返回着色器的参数。在这种情况下，可以使用下面的代码检测编译状态，并在状态值不等于GL_TRUE时退出程序：

```
glGetShaderiv(vShader, GL_COMPILE_STATUS, &status);
checkError(status, "Failed to compile the vertex shader.");

glGetShaderiv(fShader, GL_COMPILE_STATUS, &status);
checkError(status, "Failed to compile the fragment shader.");
```

现在就可以使用下面的代码将所有的程序连接在一起了：

```
glLinkProgram(myProgObj);
```

因为可以创建多个程序对象，所以必须确定当前所使用的是哪一个对象。对于刚创建的程序对象来说，由于只是创建了一个程序对象，所以下面的步骤是选择并运行这个程序对象：

```
glUseProgram(myProgObj);
```

然后使用函数glGetProgramiv检测它的状态，代码如下：

```
glGetProgramiv(program, GL_LINK_STATUS, &status);
checkError(status, "Failed to link the shader program object.");
```

① 对于普通的OpenGL函数，可以使用函数glGetError查询错误并检测其返回状态GL_NO_ERROR的值。

当检测到错误时，用于编译和连接的错误检测并不能给出我们所需要的信息。我们可以使用函数 glGetShaderiv 和 glGetShaderInfoLog 获取着色器对象更具体的信息，而使用 glGetProgramiv 和 glGetProgramInfoLog 获取程序对象的具体信息。着色器对象与程序对象具有类似的信息获取机制。我们使用函数 glGetShaderiv 或 glGetProgramiv 来确定程序是否有错误及错误信息字符串的长度，使用 glGetShaderInfoLog 或 glGetProgramInfoLog 函数用来获取错误信息字符串。附录 A 的示例程序给出了这些函数的用法。

接下来需要把着色器中的变量与 OpenGL 程序中的变量关联起来。main 程序通过连接时建立的表结构为着色器中的顶点属性变量建立索引值，并通过函数 glGetAttribLocation 获取所需的索引值。然后，OpenGL 应用程序就可以根据这些索引值并使用不同形式的 glVertexAttrib 函数来获取着色器对属性变量的计算结果。例如，假定在着色器里把一个 RGBA 颜色值赋给一个名为 myColor 的颜色变量，就可以使用下面的代码获取 myColor 变量的索引：

```
GLuint colorAttr;
colorAttr = glGetAttribLocation(myProgObj, "myColor");
```

之后就可以在 OpenGL 程序中使用下面的代码引用这个变量的值：

```
GLfloat color[4];
glVertexAttrib4fv(colorAttr, color);
```

这段代码把顶点着色器中变量 myColor 的值赋给 OpenGL 程序的 color 变量。

同样的方法适用于 uniform 变量。假定需要在应用程序中计算一个名为 angle 的 uniform 变量，并将计算结果传给着色器。首先获取该变量的索引：

```
GLint angleParam;
angleParam = glGetUniformLocation(myProgObj, "angle");
```

之后在应用程序中计算它的值：

```
GLfloat my_angle;

my_angle = 5.0; /* 或其他值 */
```

然后使用下面的代码把它的值传给着色器：

```
glUniform1f(angleParam, my_angle);
```

函数 glUniform 具有从一维到四维的形式以及指针形式（函数名的后面有字符 v），它不但可以传送标量值，也可以传送矢量值。例如，下面的代码把应用程序中的变量值传送给着色器内的 unifrom 变量 redColor：

```
GLfloat red[4] = {1.0, 0.0, 0.0, 1.0};
GLint ColorParam;

colorParam = glGetUniformLocation(myProgObj, "redColor")
glUniform4fv(colorParam, red)
```

可以使用不同形式的 glUniformMatrix 函数给着色器中 uniform 类型的矩阵赋值。

glGetUniform1fv 和 glUniform1iv 函数可以用来查询着色器中 uniform 变量的值。然而，因为 uniform 变量的值在着色器中是不能改变的，所以这些函数很少会用到，这里不再对这些函数进行深入讨论。

尽管还有相当多的 OpenGL 函数没有介绍，但其中大多数只是用于初始化设置，它们的用法在不同的应用程序之间并没有太大的区别。如果分析在一个典型的应用程序中需要处理的任务，就可

以知道程序中包含两个基本的部分。第一部分，首先需要编写各种着色器，并把它们连接到程序对象中。这部分任务可以放到OpenGL应用程序的初始化函数中实现。在程序的第二部分，即在OpenGL程序的执行过程中，我们要么从着色器获取数据，要么把应用程序中的数据传给着色器。通常，在使用固定功能绘制流水线的应用程序中，我们一般在设置颜色、纹理坐标、法向量和其他程序变量时执行这些操作。

附录A提供了一个完整程序，其中包含一个基本的顶点程序和片元程序。下面重点讨论顶点程序的典型应用。

9.7　移动顶点

现在介绍顶点着色器的几个例子。第一组例子涉及顶点的移动，可以看做是实现许多动画策略的一部分。在这些例子中，必须在OpenGL程序中生成顶点的初始位置，这是因为glVertex函数的调用会触发顶点程序的运行。在前面这些例子中，顶点着色器完成的任务本可以在应用程序中完成，但通常我们希望在顶点程序中完成这项任务，因为这样可以让CPU完成其他的任务。第二组例子涉及顶点颜色的确定。我们发现，通过自己编写顶点程序可以更多地增加对顶点处理的控制，可以把更多使用CPU的计算转移到GPU中来。

9.7.1　缩放顶点位置

最简单的一个顶点着色器是这样一个程序，它用来改变每个顶点的位置，每个顶点都会触发顶点程序的执行。然而需要注意的是：一旦使用了自己编写的顶点程序，那么就必须自己处理固定功能顶点处理器的所有功能，尤其是要自己负责把顶点位置从对象坐标系变换到裁剪坐标系。在之前的例子中，可以使用模-视变换矩阵和投影变换矩阵来实现这个变换，这些变换矩阵是OpenGL状态的一部分。我们也可以不使用内置的变换矩阵，而在顶点程序中计算每个顶点在裁剪空间的位置。

在上面的第一个例子中，顶点程序对每个顶点进行缩放，使得对象看起来是放大或缩小了。缩放因子以时间为参数并按正弦规律变化，该时间参数是一个uniform变量，其值来自OpenGL程序：

```
/* 按正弦规律移动顶点的顶点程序 */

uniform float time; /* 其值来自OpenGL程序 */

void main()
{
  float s;
  s = 1.0 + 0.5*sin(time);
  gl_Position = gl_ModelViewProjectionMatrix*(vec4(s, s, s, 1.0)
                *gl_Vertex);
}
```

注意，我们使用了OpenGL系统状态中的模-视变换矩阵和投影变换矩阵的乘积，并且把缩放因子只作用于矢量的前三个分量（见习题9.19）。由于在使用投影变换矩阵和模-视变换矩阵对每个顶点进行变换之前先把缩放因子作用于gl_Vertex，所以对象的缩放操作是在对象坐标系而不是在裁剪坐标系中完成的。在OpenGL应用程序中，我们使用下面的代码设置时间变量：

```
GLuint timeParameter;

timeParam = glGetUniformLocation(myProgObj, "time");
```

要么利用GLUT库中的空闲回调函数来增加时间，要么通过glUniform函数把当前程序流逝（或运行）的时间作为顶点程序中时间参数的值，代码如下：

```
void idle()
{
   glUniform1f(timeParam, glutGet(GLUT_ELAPSED_TIME));
   glutPostRedisplay();
}
```

注意，代码中表示程序流逝时间的单位是毫秒，因此我们可能希望调整顶点程序中的时间单位，这可以通过修改顶点程序中的缩放因子来实现。例如，如果使用下面的代码计算缩放因子，那么可以将时间单位转换为秒：

```
s = 1.0 + 0.5*sin(0.001*time);
```

如果让对象每个顶点随它们的位置而变化，那么这个例子就会看起来更加有趣。假定我们考虑一个高度场，该高度场的高度表示为每个顶点的y值，我们只改变高度场的高度值。在5.7节，我们按下面的代码形式使用显示回调函数和空闲回调函数显示高度场数据：

```
int i,j;
glBegin(GL_QUADS)
for(i=0; i<N; i++) for(j=0; j<N;j++)
{
       glVertex3f((float)i/N, data[i][j], (float)j/N);
       glVertex3f((float)i/N, data[i][j], (float)(j+1)/N);
       glVertex3f((float)(i+1)/N, data[i][j], (float)(j+1)/N);
       glVertex3f((float)(i+1)/N, data[i][j], (float)(j)/N);
}
glEnd();
```

上面的代码把每个顶点的高度数据存储在data数组中，并把每个顶点的x坐标和z坐标定义在(0, 1)范围内。可以使用下面的顶点程序改变顶点的高度值：

```
/* 改变高度场的顶点程序 */

uniform float time;

void main()
{
    float s = 0.001; // 时间缩放因子
    float xf = 5.0;  // x方向的频率
    float zf = 5.0;  // z方向的频率
    float h  = 0.1   // 高度缩放因子
    vec4 t = gl_Vertex;
    t.y = gl_Vertex.y + h*sin(s * time + xf*gl_Vertex.x) *
          sin(s*time + zf*gl_Vertex.z);
    gl_Position = gl_ModelViewProjectionMatrix * t;
    gl_FrontColor = gl_Color;
}
```

注意，由于gl_Vertex是一个内置的uniform变量，所以不能在顶点程序中改变它的y值。我们在顶点程序中定义了一个临时着色器变量t，并将其初始化为gl_Vertex变量的一个副本，这样其值就可以被修改了。

9.7.2　渐变动画

有一种实现动画的标准技术称为关键帧（key framing）。使用这种技术可以在某些时刻或者在某些位置定义场景中对象的位置，即定义关键帧。之后，就可以在连续的关键帧或关键位置之间进行插值计算，从而得到中间帧（in-between frame）或中间位置。

这种想法的一个变异形式就是渐变（morphing），使用这种技术能够将一个对象平滑地渐变为另一个对象。实现渐变的一种方法是：把所定义对象的一组顶点的位置（或其他顶点属性）修改为其他对象对应的属性值。下面分析一个简单的例子，它使用了两个数组分别用来存放数目相等的顶点。

假定有两组顶点。第一组顶点用来指定对象A，第二组顶点指定对象B，A和B都是多边形。如果希望把对象A渐变成对象B，那么这两个对象的顶点数目必须相等，并且两个对象对应的顶点需要互相匹配。这样，第一个顶点数组中的第k个顶点应该渐变为第二个数组的第k个顶点。一般来说，对象的顶点集是动画师借助软件工具生成的，动画师可以使用这种软件工具给对象增加额外的顶点，这样可以确保两个对象的顶点集数目相等且相互匹配。图9.5给出了一个二维多边形渐变成另一个二维多边形的过程。

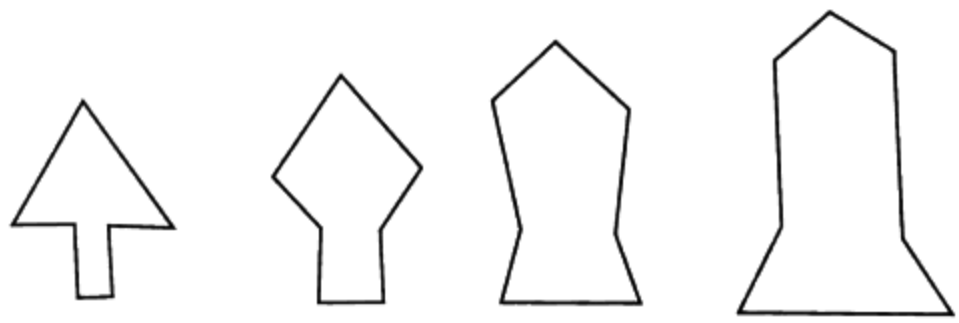


图9.5　渐变过程：(a) 对象A；(b) 1/3的A渐变为B；(c) 2/3的A渐变为B；(d) 对象B

顶点程序需要输出一个顶点，它是对OpenGL应用程序提供的两个顶点进行插值计算得到的。这其中存在的问题是：通过内置的uniform变量gl_Vertex每次只能把一个顶点传送给顶点程序。不过，可以使用应用程序定义的顶点属性把另一个对应的顶点传送给顶点程序。我们还利用应用程序把另一个变量传送给顶点程序中的uniform变量，该变量决定插值计算时每个顶点的位置权重。下面是一个顶点着色器：

```
attribute vec4 vertices2;
uniform float blend;

void main()
{
    vec4 t = vec4(mix(gl_Vertex.xyz, vertices2.xyz, blend), 1.0);
    gl_Position = gl_ModelViewProjectionMatrix*t;
    gl_FrontColor =  gl_Color;
}
```

代码中的GLSL函数mix计算两个顶点的仿射组合。例如，代码中函数mix（gl_Vertex, vertices2, blend)通过对顶点gl_Vertex和vertices2的组合计算得到顶点位置的x分量是(1-blend)*gl_Vertex.x+blend*vertices2.x。在OpenGL应用程序中，我们需要使用下面的代码定义所需的顶点属性vertices2和uniform参数blend：

```
GLuint blendParam, vertices2Param;

blendParam = glGetUniformLocation(program, "blend");
vertices2Param = glGetAttribLocation(program, "vertices2");
```

在 GLUT 库的显示回调函数里，我们会看到类似下面的代码：

```
#define N 50 /* 顶点的个数*/
GLfloat vertices_two[N][3], vertices[N][3];

void mydisplay()
{
  blend = .....; /* 设置blend值 */

  glBegin(GL_TRIANGLES)
     for(i = 0; i< N; i++)
     {
        glUniform(blendParam, blend);
        /* 设置颜色和其他顶点属性 */
        glVertexAttrib3fv(vertices2Param_location, vertices_two[i]);
        glVertex3fv(vertices[i]);
     }
  glEnd();

  glutSwapBuffers();
}
```

我们可能希望对该程序稍做修改，使之能对颜色、纹理坐标和其他顶点属性进行组合计算（见习题 9.1）。

9.7.3　粒子系统

粒子系统是模拟云、树、水和头发等复杂的真实对象最重要的方法之一，是计算机图形学众多过程建模方法中的一种，第 11 章[①]将介绍该主题。所有粒子系统的一个基本思想是：要么根据实际的物理规律，要么根据自己拟定的物理规律来使用粒子模拟真实世界中点的运动。在粒子运动的每一步，我们都要为它们确定新的位置，并在每个粒子所在的位置绘制任何对象。例如，如果要绘制云彩，那么我们可能使用一组半透明的多边形，其中每个多边形都以粒子所在的位置为中心；如果要绘制一块布料，那么可以使用一个用来表示网格的位置数组，网格的每个顶点就是一个位置随时间变化而不断变化的粒子。

顶点程序可以很好地用来创建粒子系统，其中一个原因就是顶点程序可以为每个粒子做大量的定位工作，并且其处理速度远远超过了 CPU。因为顶点程序不能生成新的顶点，所以 OpenGL 应用程序至少必须把创建的每个粒子作为一个顶点，这样便能通过顶点触发顶点程序。一个特别简单但很容易扩充（见习题 9.2）的例子是只受重力作用的粒子系统的生成。

下面考虑一个遵从牛顿定律的理想质点，假定该质点的初始位置为(x_0, y_0, z_0)，初始速度为(v_x, v_y, v_z)。如果忽略摩擦力等因素，那么在 t 时刻，质点的位置可以根据下列公式得到（重力加速度为 g）：

$$x(t) = x_0 + v_x t$$
$$y(t) = y_0 + v_y t + \frac{g}{2} t^2$$
$$z(t) = z_0 + v_z t$$

因此，可以把位于顶点程序中的时间 time 和重力加速度 g[②]定义为 uniform 变量，并通过 OpenGL 应用程序把时间和重力加速度传送给顶点程序中对应的 uniform 变量，还可以通过 gl_Vertex 把每个粒

① 此处原文为“第 12 章”，可能有误。——译者注

② 此处原文为“时间 time 和每个粒子的初始速度”，可能有误。——译者注

子的初始位置传送给顶点程序。每当顶点程序运行时，它就会计算顶点的当前位置。因为时间变量由应用程序负责更新的，所以每次调用GLUT库中的显示回调函数时，顶点程序就会计算顶点新的位置。下面是为这样的粒子系统而设计的一个简单的顶点程序：

```
attribute vec3 vel;
uniform float time, g;
void main()
{
    vec4 temp_position = gl_Vertex;
    temp_position.x = temp_position.x + vel.x*time;
    temp_position.y = temp_position.y + vel.y*time + g/(2.0)*time*time;
    temp_position.z = temp_position.z + vel.z*time;
    gl_Position =  gl_ModelViewProjectionMatrix * temp_position);
}
```

在这个程序中，我们通过使用attribute修饰的变量把每个粒子特有的信息传送给顶点程序，这是由于我们可以使用显示回调函数在函数glBegin(GL_POINTS)和glEnd()之间一次生成所有的粒子，每个粒子的速度可能各不相同。我们认为所有粒子的时间和重力加速度都是相同的。

我们可以在许多方面扩充这个例子。这个顶点程序的一个简单应用是用它来模拟烟火，方法是用顶点程序处理表示粒子的每个顶点，并把每个顶点以相同的初始位置送入顶点程序，但每个顶点的初始速度稍有不同。此外，随着时间变化，我们还可以改变粒子的颜色，甚至让粒子消失（见习题9.3）。

如果顶点是一个比粒子更复杂的对象的一部分，那么整个对象都要受到重力的影响。对上面这个例子的一个有趣的修改是：当粒子撞到地面上时，可以利用reflect函数改变它的速度，这样可以产生弹跳效果（见习题9.5）。

9.8　基于顶点程序的光照计算

顶点程序最重要的一个应用也许是使用它来代替固定功能绘制流水线中改进的Phong光照模型。我们首先编写顶点程序来实现Phong光照模型和改进的Phong光照模型，这样做的原因主要是为了介绍在顶点程序中用来计算光照效果的结构和函数，然后再介绍其他的光照计算。

9.8.1　Phong光照计算

第6章介绍的改进的Phong光照模型用于计算正向表面光照强度的公式（未考虑距离因素）如下：

$$I = k_d L_d \mathbf{l} \cdot \mathbf{n} + k_s L_s (\mathbf{n} \cdot \mathbf{h})^{\alpha} + k_\alpha L_\alpha I_\alpha$$

那时我们利用这个公式计算RGB颜色的每个分量值。因此，所有的漫反射系数、镜面反射系数和环境光系数常量都表示为三元数组或四元数组，它们对应于光源的颜色分量。

当编写用于计算顶点颜色的顶点程序时，必须注意对象当前所处的坐标系。我们知道，在顶点程序中必须把顶点位置从对象坐标系变换到裁剪坐标系。然而，我们还要处理可能出现在不同坐标系中的法向量、灯光和纹理坐标等。例如，在顶点程序中通过OpenGL内部状态访问光源时，光源的位置经过模–视变换矩阵自动变换到了观察坐标系。因此，通常我们面临选择使用哪个坐标系进行计算的问题。一旦选定了工作坐标系，就必须把所有的数据都变换到这个坐标系。例如，如果我们决定使用观察坐标系，那么除了使用模–视变换矩阵和投影变换矩阵（gl_ModelViewProjection-Matrix）把顶点（gl_Vertex）变换到裁剪坐标系，从而得到供片元程序处理的新的顶点位置

（gl_Position）之外，还必须仅使用模－视变换矩阵（gl_ModelViewMatrix）把顶点（gl_Vertex）变换到位于观察坐标系中的顶点。

为了简化第一个例子，假定在OpenGL程序中仅定义了一个光源（GL_LIGHT0）和材质属性。下面逐句分析Phong光照顶点程序。首先，必须把顶点位置变换到裁剪坐标系，以供片元程序使用：

```
gl_Position = gl_ModelViewProjectionMatrix*gl_Vertex;
```

然后，在观察坐标系中计算所需的所有矢量。对象空间的法向量gl_Normal也是一个内置的uniform变量，但我们必须确保它已经变换到观察坐标系里，并且是一个单位矢量。然而第6章曾经讨论过，为了使法向量和光源方向之间的夹角保持不变，必须使用模－视变换矩阵左上角3 × 3的转置矩阵来对对象空间的法向量进行坐标变换，这个3 × 3的转置矩阵称为法向量矩阵（normal matrix），我们可以使用OpenGL的一个内置变量gl_NormalMatrix来表示它。此外还必须确保经过法向量矩阵变换后的法向量是单位矢量。下面这行代码就可以完成上述两个任务：

```
vec3 N = normalize(gl_NormalMatrix*gl_Normal);
```

我们还需要把顶点变换到观察坐标系，代码如下：

```
vec4 eyePosition = gl_ModelViewMatrix * gl_Vertex;
```

第i个光源的所有参数都定义为内置的uniform变量，可以使用内置的结构体gl_ LightSource[i]获取所需的参数。下面的代码使用光源的位置参数计算光源矢量：

```
vec4 eyeLightPos = gl_LightSource[0].position;
vec3 L = normalize(eyeLightPos.xyz - eyePosition.xyz);
```

在观察坐标系中，视点位于坐标系原点，视点的矢量可以使用下面的代码来计算：

```
vec3 E = -normalize(eyePosition.xyz);
```

现在可以使用下面的代码计算反射矢量：

```
vec3 R = reflect(L, N);
```

下面是计算半角矢量的代码：

```
vec3 H = normalize(L + E);
```

至此，利用Phong模型或改进的Phong模型计算顶点颜色所需的所有信息都已经具备。对象表面正面的材质数据存放在内置的结构体gl_FrontMaterial中。如果不考虑距离项，颜色的漫反射分量可由下面的代码实现：

```
vec4 diffuse = max(dot(L, N),0.0)*gl_FrontMaterial.diffuse
               *gl_LightSource[0].diffuse;
```

GLSL语言有一个内置的变量gl_FrontLightProduct，它可用来表示正面材质属性与每个光源的光源属性的点积运算，因此可以简化用于计算漫反射分量的代码：

```
vec4 diffuse = max(dot(L, N),0.0)*gl_FrontLightProduct[0].diffuse;
```

使用同样的结构，可以计算顶点颜色的环境光分量：

```
vec4 ambient = gl_FrontLightProduct[0].ambient;
```

漫反射光和环境光对每个顶点颜色的贡献在Phong模型和改进的Phong模型中都是一样的，改进的Phong模型使用下面的代码计算镜面反射分量：

```
specular = f*pow(max(dot(N, H), 0.0), gl_FrontMaterial.shininess)
        *gl_FrontLightProduct[0].specular;
```

其中，因子f的值由光源矢量L与对象表面顶点法向量N点积（即dot(L, N)）的符号来确定，负数取0，其他情况取1。因此，如果光源位于对象表面的下面，那么镜面反射光对顶点颜色的贡献为0。对于Phong模型和改进的Phong模型，虽然已经有了所有所需的信息，但是，如果希望使用这两个光照模型生成相同的图像，还必须在OpenGL应用程序中改变镜面反射系数（shininess）的值。一种可取的办法是通过应用程序把一个校正因子传送给顶点程序，代码如下：

```
uniform float factor;

specular = pow(max(dot(E, R),0.0), factor*gl_FrontMaterial.shininess
           *gl_FrontLightProduct[0].specular;
```

习题9.9给出了对校正因子进行计算的方法，现在就可以将环境光、漫反射光和镜面反射光的贡献加起来作为对象正面顶点的颜色值：

```
gl_FrontColor = ambient + diffuse + specular;
```

在前面的讨论中，我们使用内置的uniform变量从OpenGL状态中获取大多数所需的变量值。在其他情形下，还可以通过函数的参数或非内置的uniform变量把这些数据传送给着色器。如果使用一个片元程序，那么经常采用varying变量把顶点程序的结果向前传递。当我们在9.10节中讨论纹理映射时，将会看到很多这方面的例子。

把前面的讨论归纳起来，就得到了一个使用改进的Phong模型计算每个顶点颜色值的顶点程序：

```
void main()
{
    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;

    vec4 ambient;
    vec4 diffuse;
    vec4 specular;
    float f = 1.0;
    vec4 eyePosition = gl_ModelViewMatrix * gl_Vertex;
    vec4 eyeLightPos = gl_LightSource[0].position;
    vec3 N = normalize(gl_NormalMatrix * gl_Normal);
    vec3 L = normalize(eyeLightPos.xyz - eyePosition.xyz);
    vec3 E = -normalize(eyePosition.xyz);
    vec3 H = normalize(L + E);
    float Kd = max(dot(L, N), 0.0);
    float Ks = pow(max(dot(N, H), 0.0), gl_FrontMaterial.shininess);
    if(dot(L,N)<0.0) f = 0.0;
    ambient = gl_FrontLightProduct[0].ambient;
    diffuse = Kd*gl_FrontLightProduct[0].diffuse;
    specular = f*Ks*gl_FrontLightProduct[0].specular;
    gl_FrontColor = ambient+diffuse+specular;
}
```

9.8.2　非真实感着色

顶点程序还能把更逼真的光照模型融入到实时场景绘制中，9.11节将引入折射和色散等效果。然而，许多基于可编程着色器的应用涉及使用着色器生成非真实感效果的问题。下面介绍两个有趣的例子，一个是只使用少数几种颜色表示表面的漫反射效果，另一个是突出对象的轮廓，我们希望使用这两种技术绘制具有卡通效果的图像。

假定在顶点程序中定义了两种颜色，代码如下：

```
const vec4 yellow = {1.0, 1.0, 0.0, 1.0};
const vec4 red = {1.0, 0.0, 0.0, 1.0};
```

之后我们就可以在这两种颜色之间进行切换（比如根据漫反射颜色的强度）。下面的代码就是根据颜色和法向量来指定对象正面的颜色：

```
if(dot(L, N) > 0.5) gl_FrontColor = yellow;
   else gl_FrontColor = red;
```

其实还可以使用更简单的方法切换这两种颜色，即根据漫反射的颜色确定一个阈值，对象的颜色随着其形状和光源位置的变化而变化。

我们还可以绘制对象的轮廓。确定轮廓线的一种方法就是根据dot(E, N)计算结果的符号，如果点积的符号为正，则表示顶点面向视点；而如果点积的符号为负，则表示顶点背向视点。因此，可以把这个值与一个较小的值进行比较，根据比较结果，给顶点赋一个颜色（如黑色）：

```
vec3 black = {0.0, 0.0, 0.0, 1.0};
float t = 0.1 // 或其他较小的值

if(abs(dot(E, N)) < t) glFrontColor = black;
```

彩图 15 是使用 3 种颜色绘制的具有卡通效果的茶壶图像，图像的轮廓使用黑色。

9.9　片元着色器

顶点程序作为图形绘制流水线前端几何处理的一部分，然而片元程序处理的是单个片元，每个片元对像素的颜色有直接贡献。因此，片元程序就为生成基于像素的图像提供了可能。

片元程序一个最有趣的特性就是提供了一种新的方式处理纹理映射。我们将特别介绍使用立方体贴图（cube map）生成类似于环境映射（environment map）和位移映射（displacement map）中的折射效果。相对于顶点程序来说，片元程序能提供更多的处理方法，并且其中一些方法能够提高程序的效率。

在语法上，使用GLSL语言编写的片元程序和顶点程序完全相同，它们具有相同的数据类型、大部分相同的修饰符及相同的函数。然而，片元程序的运行方式和顶点程序却是完全不同的。下面首先回顾一下片元程序的功能。

光栅器把所有没有被裁剪掉的图元都生成片元，每个片元对应于帧缓存里面的一个像素，并且具有颜色及帧缓存中的位置等属性，其他属性则由光栅器对顶点属性进行插值计算而得到。因此，即使对于下面的“直通片元程序”，

```
/* 直通片元程序 */
void main(void)
{
   gl_FragColor = gl_Color;
}
```

每个片元的颜色都是通过对顶点的颜色进行插值计算得到的，这些顶点的颜色值要么由应用程序的OpenGL状态值确定并经固定功能绘制流水线计算得到，要么由顶点程序计算得到。同样，每个片元的纹理坐标也可以由顶点的纹理坐标通过插值计算得到，而不管纹理坐标值是如何得到的。另外，顶点程序输出的 varying 变量都会自动被光栅器在每个图元上执行插值计算。

9.10　基于顶点与基于片元的光照计算

几乎每个基于顶点属性定义的变量都可以在片元程序中进行图元的插值计算，所以我们可以基于片元而非顶点进行光照计算。第6章讨论了3种多边形着色方法。其中，最简单的方法是均匀着色，即用单一的颜色对整个多边形着色。这个颜色值既可以在应用程序中设置，也可以在绘制流水线中使用改进的Phong光照模型在多边形的第一个顶点上计算得到。使用均匀着色方法需要在OpenGL程序中把着色模式设置为GL_FLAT。第二种方法是平滑着色，它是固定功能绘制流水线的默认着色方法。使用这种着色方法时，首先在顶点程序中计算每个顶点的颜色，然后在片元处理器中对顶点的颜色在多边形表面上进行插值运算。这个处理过程与9.8.1节中讨论的Phong顶点程序一样。片元程序并不用关心顶点颜色是如何计算得到的，而只是在多边形上对顶点颜色进行插值计算即可。

第三种方法是Phong着色，该方法是在多边形上对顶点位置和法向量（而不是顶点颜色）进行插值计算，然后根据插值后的法向量使用光照模型计算每个片元的颜色。在可编程的片元程序出现之前，这种基于片元的着色方法只能采用脱机处理方法（非实时处理方法）实现，与使用固定功能绘制流水线提供的改进Phong模型相比，这种方法的时间开销非常大。而使用片元程序进行基于片元的光照计算，因为有了最新GPU的高速计算能力，即使要对每个片元进行光照计算，我们仍可以交互地处理复杂几何模型。

现在我们准备编写一个片元程序，假定其只有一个光源（GL_LIGHT0），并使用改进的Phong光照模型计算每个片元的颜色。因为数据需要从顶点程序传送到片元程序，所以要编写两个着色器程序，并使用了varying变量，这是一种与9.8.1节不同的方法。

在顶点程序中，需要计算对象在观察坐标系中的位置和法向量，并将这些数据传送给片元程序。在这个例子中，假定顶点程序中的顶点法向量来自OpenGL的状态变量gl_Normal。注意，每个顶点的法向量无论是要根据周围的多边形重新计算，还是对整个图元保持不变，都与着色器程序无关。下面是一个基本的顶点程序，该程序和前面讨论的基于顶点的改进Phong着色器一样计算相同的矢量，但不对任何光照分量进行计算，而是把这些光照计算留给片元程序：

```
// 顶点程序，实现基于片元的 Phong 光照计算

varying vec3 N;
varying vec3 L;
varying vec3 E;

void main()
{
    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;

    vec4 eyePosition = gl_ModelViewMatrix * gl_Vertex;
    vec4 eyeLightPos = gl_LightSource[0].position;

    N = normalize(gl_NormalMatrix * gl_Normal);
    L = eyeLightPos.xyz - eyePosition.xyz;
    E = -eyePosition.xyz;
}
```

值得注意的是：为了编写正确的顶点程序，必须在顶点程序中计算gl_Position，而半角矢量就无须在顶点程序中计算了。

片元程序所需的光照和材质属性等数据可以通过OpenGL状态变量来提供。因为顶点法向量、光源向量和视点向量都可以从顶点程序传送过来，共享同一个命名空间，并将自动在图元表面上进行插值计算，因此，下面这个片元程序看起来非常像9.8.1节介绍的改进的Phong光照模型顶点程序：

```
// 基于片元的 Phong 着色模型片元程序

 varying vec3 N;
 varying vec3 L;
 varying vec3 E;

void main()
{
     vec3 Normal = normalize(N);
     vec3 Light  = normalize(L);
     vec3 Eye    = normalize(E);
     vec3 Half   = normalize(Eye+Light);
     float f = 1.0;

    float Kd = max(dot(Normal, Light), 0.0);
    float Ks = pow(max(dot(Half, Normal), 0.0),
               gl_FrontMaterial.shininess);

    vec4 diffuse  = Kd * gl_FrontLightProduct[0].diffuse;
    if(dot(Normal, Light)<0.0) f = 0.0;
    vec4 specular = f*Ks * gl_FrontLightProduct[0].specular;
    vec4 ambient  = gl_FrontLightProduct[0].ambient;

    gl_FragColor = ambient + diffuse + specular;
}
```

图9.6、图9.7和彩图30显示了分别基于顶点着色和基于片元着色绘制的茶壶的差异。它们都是使用相同的材质和光照属性生成的，但基于片元着色绘制的图像能更好地表现镜面反射的高光效果。

图9.6　基于顶点的Phong着色效果

图9.7　基于片元的Phong着色效果

9.11　sampler变量类型

一般来说，顶点程序可以计算纹理坐标或者直接输出纹理坐标，然后光栅器对输出的纹理坐标在图元表面上进行插值计算，从而得到每个片元的纹理坐标，纹理映射要在片元处理时才会进行。这样，在可编程绘制流水线中有许多令人感兴趣的应用都与片元程序中的纹理采样有关，这也就不足为奇了。

处理纹理时的最大困难可能就是需要设置大量的纹理选项，包括纹理包装模式（wraping mode）、纹理滤波（filter）、mipmapping、纹理矩阵和打包参数等。此外，因为可以创建多个纹理对象，所以着色器能够存取多个纹理。如果GPU支持多纹理单元，那么着色器就可以对其中的任何一个纹理对象进行采样。GLSL语言允许程序通过一个新的变量类型sampler访问纹理对象，这样就把人们从烦琐的选项设置中解脱出来。

sampler变量用来访问特定的纹理对象，包括它的所有参数。OpenGL支持的纹理类型在GLSL语言中都有相应的sampler类型，GLSL语言特别提供了一维sampler（sampler1D）、二维sampler（sampler2D）、三维sampler（sampler3D）和立方体贴图（samplerCube）。

一般来说，顶点程序只需考虑纹理坐标即可，而由片元程序通过sampler变量来处理纹理映射。OpenGL应用程序通过glUniform1i函数把sampler变量传送给着色器，这样就将sampler变量与纹理对象和纹理单元关联起来。假定我们按第8章的方法在应用程序中定义了一个纹理对象作为默认纹理，该纹理对应于纹理单元0。当程序对象myProgObj所装载的着色器经过编译、连接后，片元程序就可以通过一个uniform变量把程序中的sampler变量与应用程序中定义的纹理对象关联起来。下面的代码用来设置这个uniform变量：

```
GLuint texMapLocation;

texMapLocation = glGetUniformLocation(myProgObj, "texMap");
```

其中，texMap是片元程序中sampler变量的名字。使用下面的代码即可实现片元程序对纹理对象的访问：

```
glUniform1i(texMapLocation, 0);
```

这句代码可以放在应用程序的OpenGL变量初始化部分或某个回调函数中。

因为sampler变量属于uniform变量，所以它的值不能被顶点程序或片元程序改变。一旦定义了一个sampler变量，就可以用它来提供着色器所需的纹理值，其中需要用到texture1D、texture2D、texture3D及textureCube等纹理函数中的某个函数。sampler变量所需的纹理坐标由应用程序通过内置的顶点属性变量gl_MultiTexCoord传送给顶点程序，然后光栅器对输出的纹理坐标进行插值计算得到每个片元的纹理坐标，并把它存放在矢量类型变量gl_TexCoord中。除此之外，也可以把顶点属性变量作为纹理坐标传送给着色器，或者直接在着色器中通过计算得到纹理坐标。下面是一个简单的顶点程序：

```
void main()
{
    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;
    gl_TexCoord[0] = gl_MultiTexCoord0;
}
```

该顶点程序计算在裁剪坐标系中的顶点位置，并从纹理单元0获取纹理坐标。

下面这个片元程序实现了完全通过二维纹理映射生成每个片元的颜色：

```
uniform sampler2D texMap;

void main()
{
    gl_FragColor = texture2D(texMap, gl_TexCoord[0].st);
}
```

如果通过一个顶点属性变量把纹理坐标传给顶点程序，那么就可以使用一个varying变量把纹理坐标传送给片元程序。顶点程序的代码如下：

```
attribute vec2 vertexTexST;
varying vec2 texcoord;

texcoord = vertexTexST;
```

之后，片元程序就可以使用一个与之对应的 varying 变量 texcoord 来访问 sampler 变量。

其中一个简单但重要的应用就是把纹理映射与基于片元的漫反射着色结合起来。下面这个顶点程序除了提供纹理坐标之外，还将计算对象的空间向量和光源位置：

```
varying vec3 N;
varying vec3 L;

void main()
{
    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;
    gl_TexCoord[0] = gl_MultiTexCoord0;

    vec4 eyePosition = gl_ModelViewMatrix * gl_Vertex;
    vec4 eyeLightPos = gl_LightSource[0].position;

    N = normalize(gl_NormalMatrix * gl_Normal);
    L = normalize(eyeLightPos.xyz - eyePosition.xyz);
}
```

与之对应的片元程序如下：

```
varying vec3 N;
varying vec3 L;
uniform sampler2D texMap;

void main()
{
    vec3 Normal = normalize(N);
    vec3 Light  = normalize(L);
    float Kd = max(dot(Normal, Light), 0.0);
    vec4 diffuse  = Kd * gl_FrontLightProduct[0].diffuse;
    vec4 texColor = texture2D(texMap, gl_TexCoord[0].st);
    gl_FragColor = diffuse*texColor;
}
```

这个例子和固定功能绘制流水线唯一的区别就在于它是基于每个片元计算漫反射分量。在下面两节中，我们将讨论片元程序的其他应用，是一些用固定功能绘制流水线所无法近似实现的应用。

9.12　立方体贴图

第8章介绍了如何使用6幅二维图像定义一个立方体贴图。利用片元程序就可以在各种应用中使用立方体贴图。在应用程序中，首先要设置一个立方体纹理对象，创建一个 uniform 变量，然后通过该变量把纹理单元设置为默认的单元（或者应用程序使用的任意其他单元），代码如下：

```
texMapLocation = glGetUniformLocation(myProgObj, "myCube");
GLuint texunit = 0;
glUniform1i(texMapLocation, texunit);
```

在片元程序中可以利用由光栅器插值计算得到的三维纹理坐标从samplerCube变量中获取纹理值，代码如下：

```
uniform samplerCube myCube;
varying vec3 texcoord;

vec4 texColor = textureCube(myCube, texcoord);
```

代码中的纹理坐标可以通过顶点程序中的一个varying变量由应用程序提供，或者由OpenGL的状态变量提供，还可以完全在片元程序中确定。下面将介绍一些选择使用这些纹理坐标的有趣方法。

9.12.1　反射贴图

图9.8给出了反射贴图或环境贴图所需的计算，假定场景已经映射到了立方体上。反射贴图和简单立方体贴图的主要区别在于：访问反射贴图所需的纹理应该使用反射矢量而不是观察矢量。在顶点程序中需要计算每个顶点的反射矢量，然后片元程序在图元上对这些顶点的反射矢量进行插值计算。

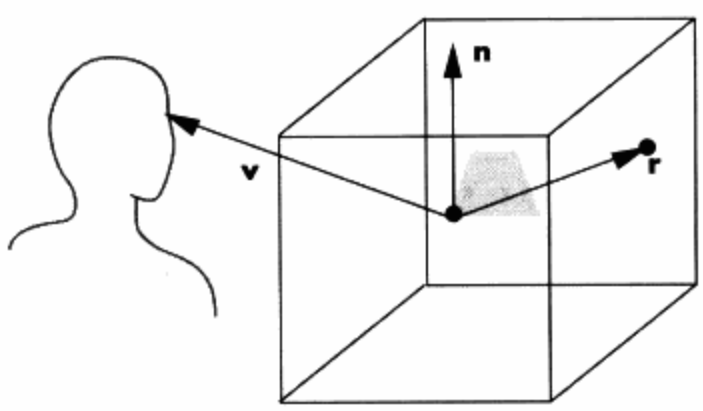


图9.8　反射立方体贴图

下面的代码给出了一个基于片元的光照计算顶点程序的简化版本：

```
varying vec3 R;

void main()
{
    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;
    vec3 N = normalize(gl_NormalMatrix*gl_Normal);
    vec4 eyePos = gl_ModelViewMatrix*gl_Vertex;
    R = reflect(-eyePos.xyz, N);
}
```

该程序对观察坐标系中每个顶点的反射矢量（该矢量是一个varying变量）进行了计算。如果希望立方体表面的颜色完全由纹理决定，那么片元程序如下所示：

```
varying vec3 R;
uniform samplerCube texMap;

void main()
{
    gl_FragColor = textureCube(texMap, R);
}
```

为了生成更复杂的光照效果，可以采用类似于前面介绍的改进的Phong光照顶点程序，即片元的颜色由镜面反射光、漫反射光和环境光等确定。

然而，我们必须要确定在着色器中需要使用的是哪个坐标系。本例与前面例子的主要区别是：反射贴图是在世界坐标系中计算的。由于对象位置和法向量是在对象坐标系中定义的，在应用程序中就需要通过建模变换把它们变换到世界坐标系中。因为模-视变换直接把对象从对象坐标变换为

观察坐标，所以通常看不到对象在世界坐标系中的表示。在许多应用中，我们直接定义对象而不使用建模变换，因而对象坐标系与世界坐标系重合①。然而，当计算反射贴图时，我们希望编写的程序能够进行建模变换，完成该任务的一种方法就是在应用程序中计算建模变换矩阵，然后把它作为一个 uniform 变量传送给片元程序使用。还需要注意的是：我们需要使用建模变换矩阵的转置矩阵变换对象的法向量。然而，如果把转置矩阵作为另一个 uniform 变量传送给片元程序，那么为了得到正确的法向量结果，必须右乘法向量。彩图 16 绘制的茶壶颜色是由反射贴图确定的，茶壶嵌入在立方体内部，立方体 6 个面的颜色分别是红、绿、蓝、青色、品红和黄色。

9.12.2　折射

目前讨论的环境贴图或反射贴图都是基于反射对象的表面不透明这样一个假设。此外，还可以假定对象是半透明的，这样我们所看到的对象表面的光线一部分是由着色器确定的，另一部分光线是被周围的环境反射到对象表面，还有一部分光线穿过对象，这样就可以看到对象后面的环境所反射的光线对表面的颜色贡献。

在编写所需的程序之前，我们需要一个反映光线如何穿过半透明材质的模型。如图 9.9 所示，我们考虑一种可以折射所有入射光线的表面。如果光线在两种不同材质中的传播速度不同，那么光线的方向在材质表面会发生改变。假设 η_l 和 η_t 表示材质的折射率（indices of refraction），即用于度量光线在对象表面两侧的两种材质里的相对传播速度。根据斯涅耳定律（见习题 9.13）有下面的公式：

$$\frac{\sin\theta_l}{\sin\theta_t} = \frac{\eta_t}{\eta_l}$$

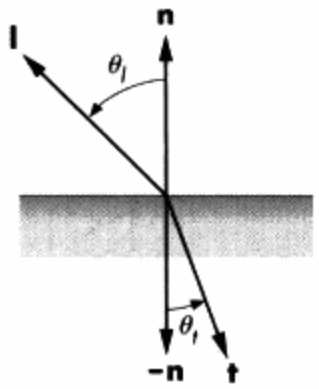


图 9.9　理想的光折射

我们可以按下面的方法求出折射光线的方向 **t**。通过 **n** 和 **l** 先计算出 $\cos\theta_l$；如果 **n** 和 **l** 都是单位矢量，那么 $\cos\theta_l$ 就是这两个矢量的点积；从而根据 $\cos\theta_l$ 可以计算 $\sin\theta_l$，然后根据斯涅耳定律计算 $\sin\theta_t$；最后就可以计算出 $\cos\theta_t$ 的值。假设 $\eta = \eta_t/\eta_l$，则有：

$$\cos\theta_t = \left(1 - \frac{1}{\eta^2}(1 - \cos^2\theta_l)\right)^{\frac{1}{2}}$$

与计算反射光线一样，**n**、**l** 和 **t** 这三个矢量必须位于同一平面。因此有：

$$\mathbf{t} = \alpha\mathbf{n} + \beta\mathbf{l}$$

如果进一步假定 **t** 为单位矢量，那么采用类似于推导反射光线的方法就可以得到下面的公式：

$$\mathbf{t} = -\frac{1}{\eta}\mathbf{l} - \left(\cos\theta_t - \frac{1}{\eta}\cos\theta_l\right)\mathbf{n}$$

① 原文为建模坐标系与对象坐标系重合，可能有误。——译者注

公式中的前两个负号是由于t的方向背向表面的背面。当上述cos θ_t计算公式中的开根号项里的值为零（即sin $\theta_t = \eta$）时，称所对应的θ_t为临界角（critical angle）。当光线以这个临界角射到表面时，折射光线会沿着表面方向发射；当θ_t大于临界角时，入射光线全部被反射，而没有折射光线。

假定有一个由半透明材质组成的对象，光线在该材质中的传播速度小于在空气中的传播速度。因此，直接通过该对象表面的光线就会发生折射，根据斯涅耳定律就可以确定折射光线与对象内部的交点位置。如果使用立方体贴图来绘制场景，那么除了使用的是折射矢量而不是反射矢量之外，程序看起来与前面的例子相类似。根据法向量和位置，可以使用内置函数refract来计算光线的折射矢量。顶点程序的代码如下：

```
varying vec3 RF; // 折射矢量
uniform float eta; // 折射率比值

RF = refract(E, N, eta);
```

代码中E的计算过程在前面已经介绍过，在片元程序中的代码如下：

```
varying vec3 RF;
uniform samplerCube texMap;

vec4 texColor = textureCube(texMap, RF);
```

图9.10给出了光线与折射表面相互作用的更一般情形。一部分光线被折射，一部分光线被反射，其余的光线被表面吸收。对于折射光线，其中一部分以类似于镜面反射的方式发生散射，只不过这里的散射光线集中在t方向周围。因此，对于位于折射光一侧的观察者来说，折射模型中还需要一个与$\mathbf{t}\cdot\mathbf{v}$成正比的项。我们也可以使用类似前面的半角矢量来简化该项的计算（见习题9.14）。

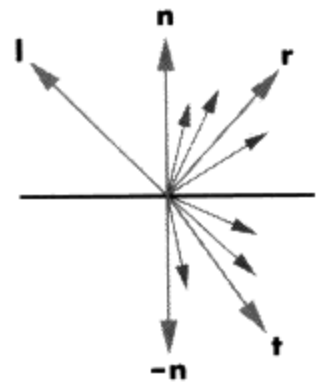


图9.10　光线与表面的相互作用

我们还可以利用一个简单的片元程序、反射矢量和折射矢量来绘制更加真实的效果。当光线照射到一个真实的半透明材质上，其中一部分光线被吸收，一部分光线被反射，一部分光线穿过材质发生折射。与我们之前使用过的简单光照模型不同，发生折射和反射的光的数量比例取决于光线与法向量之间的角度。另外，这种比例关系还取决于光的波长和光的偏振现象，其中的物理关系在菲涅耳方程（Fresnel equation）中进行了描述，但菲涅耳方程已经超出了本书的讨论范畴（请参阅本章后面的建议阅读资料），而且它的复杂性与出于程序效率考虑所需的简单性相矛盾。因此，我们在片元程序或顶点程序中将使用近似的计算方程。

假定使用环境贴图根据反射角和折射角已经分别计算并求得了反射光的颜色C_r和折射光的颜色C_t。我们使用的就是对这两个颜色进行仿射组合后得到的颜色，仿射组合系数是r。因此，计算片元颜色C_f的公式如下：

$$C_f = rC_r + (1-r)C_t$$

如果能比较容易地计算并得到公式中的r系数，那么可以用这个值去近似表示菲涅耳项的值。如果使菲涅耳项的值取决于光线和法向量之间角度的余弦值，那么可以使用$\mathbf{l}\cdot\mathbf{n}$表示菲涅耳项，这个值是计算漫反射光时一个很容易计算的项。如果光线和法向量之间角度在$-90^\circ \sim 90^\circ$之间变化，那么菲涅耳项的值在$0 \sim 1$之间变化。如果像处理镜面反射中的$\mathbf{r}\cdot\mathbf{v}$项一样，对菲涅耳项的值进行指数运算，那么可以使它的取值位于一个大于0且尽量小的范围内。因此可以使用下面的近似形式：

$$r = \max(0, (\mathbf{l}\cdot\mathbf{n})^p)$$

我们还可以使用其他一些更灵活的形式：

$$r = \max(0, \min(1, a + b(1 + \mathbf{l}\cdot\mathbf{n})^p)$$

上式中，不仅p是可变的，而且a和b也都是可变的。

我们还可以模拟折射与波长的关系。当光线进入一种光速较慢的材质中时，波长较短的光比波长较长的光折射得多，这就可以解释光线穿过棱镜时生成彩虹的现象。因为我们使用RGB颜色模型，并且只处理3个颜色分量而不是全部波长的光，所以只能近似表示这种效果，即色散现象（chromatic dispersion）。如果对每个颜色分量都采用不同的折射率（见习题9.16），那么就会在图像中看到在对象边缘出现了彩色条纹。

注意，我们讨论的只是光线进入光速比空气慢的材质中时所发生的现象。如果材质的厚度有限，那么当光线离开材质时也会发生折射现象（见习题9.20）。

9.12.3　法向量贴图

除了颜色之外，立方体贴图还可以用来查找其他的值。因为立方体贴图存储了RGBA颜色，所以可以在立方体贴图的每个位置最多存储4个值。对于立方体贴图的扩展应用，其中一个巧妙的例子就是法向量贴图，我们可以使用函数normalize对三维矢量进行归一化，代码如下：

```
vec3 a = {1.0, 2.0, 3.0};
vec3 b = normalize (a);
```

而典型的归一化操作需要执行3次乘法运算计算每个分量的平方、2次加法运算和1次开平方根运算。如果在片元程序中执行归一化操作，那么GPU的计算开销会相当大，因为它需要对每个片元都执行这些操作。

假定存在一条直线，这条直线的一个端点为原点，则直线上任意两点间的矢量，尤其是任意一点到原点间的矢量，都被归一化为同一个矢量。假定要计算该矢量与立方体的交点，可以把归一化矢量的分量放在这些纹理坐标系中。当然，为了生成立方体贴图，必须预先计算这些值，这个过程还是相当简单的，下面将介绍该计算。

考虑如图9.11所示的立方体贴图，它由6个平面确定，即$x = \pm 1$、$y = \pm 1$和$z = \pm 1$。如果给定一个矢量$\mathbf{v} = (a, b, c)$，它的一个端点在原点，那么与该矢量相交的立方体表面是由分量大小最大的那个分量及其符号决定。因此，矢量（1.0, 2.0, 3.0）与立方体表面$z = 1$相交，而矢量（1.0, –3.0, 2.0）与立方体表面$y = -1$相交。对于第一种情况，交点是（0.33, 0.67, 1.0），而对于第二种情况，交点是（0.33, –1.0, 0.67）。

归一化后的矢量分量取值范围在$-1.0 \sim 1.0$之间，而纹理分量是颜色，其取值范围在$0.0 \sim 1.0$之间。如果先把归一化矢量的分量压缩到$0.0 \sim 1.0$的范围，那么就可以把归一化矢量存储为颜色值。一个简单的转换函数如下：

```
GLfloat compress(GLfloat color_component)
{
    return 0.5*color_component+0.5;
}
```

该函数把（-1.0, 1.0）范围内的值压缩到（0.0, 1.0）范围。下面的函数执行相反的操作：

```
GLfloat uncompress(GLfloat texture_component)
{
    return 2.0*texture_component-1.0;
}
```

因此，RGB颜色值（0.33, 0.67, 1.0）可以压缩为（0.66, 0.83, 1.0），而（0.33, -1.0, 0.67）会被压缩为（0.66, 0.0, 0.83）。

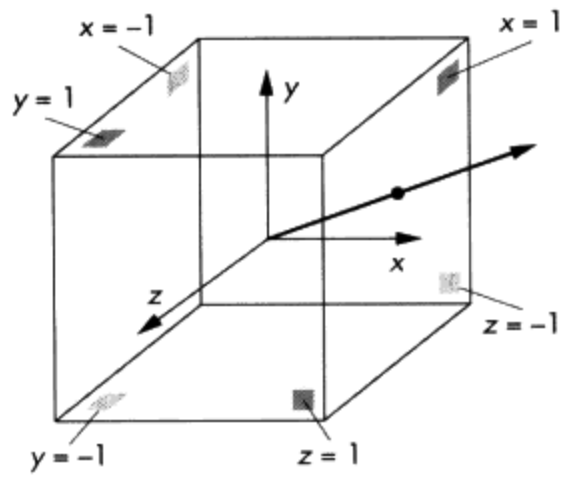


图9.11　立方体贴图

因此，可以使用立方体贴图生成一个法向量贴图。对于立方体6个面上的任意一个纹素，它的RGB颜色值就是从原点到该点归一化矢量的压缩值。因此，纹理坐标s、t、r对应（0.33, 0.67, 1.0），在立方体贴图中纹素的RGB值为（0.63, 0.765, 0.90），它是归一化矢量（0.26, 0.53, 0.80）的压缩值。在片元程序中，我们经常会看到下面形式的代码：

```
vec3 normal;
vec3 texCoord;
samplerCube NormalMap;

normal = 2.0*textureCube(NormalMap, texCoord)-1.0;
```

生成法向量贴图需要做大量的计算，并且需要定义存储空间。但在通常情况下，如果使用64 × 64或128 × 128的纹理贴图表示立方体的表面，那么这样的精度就足以满足大多数应用程序的需求了，而且所需要的内存开销也是最小的。目前，使用最新的图形卡处理归一化函数的速度已经足够快，甚至不需要使用法向量贴图，而且这个思想还可以应用于计算其他函数。对于立方体贴图更一般的应用，可以基于纹理采样使用片元程序来实现GPU内部的数学操作。

9.13　凹凸映射

在本章结束之前，我们讨论几个凹凸映射的例子。第8章介绍的凹凸映射作为一种纹理映射技术，它不用增加对象的几何复杂度就能够生成具有复杂表面效果的图像。与简单的纹理映射不同，当光源或对象移动时，凹凸映射会显示对象表面明暗效果的变化，从而使得一个光滑的对象表面看起来发生变化。

首先回顾第8章生成橘子图像的例子。如果有一幅橘子的照片，那么就可以把它作为纹理映射到球体的表面。然而，当移动光源或旋转对象时，我们会立即注意到生成的并不是一个真实橘子的

图像，而只是一个橘子模型的图像。出现这个问题的原因在于，真实的橘子外形不仅仅是颜色的变化，而主要特征在于它表面有许多微小的变化，而这种效果是不能通过纹理映射生成的。当绘制对象的表面时，凹凸映射技术通过扰动表面的法向量来改变它的形状，其表面经着色处理后得到的颜色反映了表面几何特性的变化。环境贴图等技术无须可编程着色器就可以实现，而凹凸映射在没有可编程着色器的情况下就不可能实时运行。

9.13.1　计算凹凸贴图

通过观察可知，对象表面任一点的法向量确定了表面在该点处的朝向。如果对表面上的每个点都施加一个微小的扰动，那么就可以生成一个外形起伏变化的表面。如果这种法向量的扰动只发生在表面着色期间，那么就可以使用光滑的表面模型，它具有平滑的法向量，使用某种方式可以把它绘制成看起来具有复杂效果的表面。因为只是对法向量进行扰动，所以对扰动后的表面进行计算之后才能产生正确的绘制效果，但在实际处理时我们并不用创建这个法向量被扰动后生成的复杂表面模型。

我们可以采取许多不同的方式扰动表面的法向量。下面是一个对参数曲面非常有效的方法。假设 $\mathbf{p}(u, v)$是参数表面上的一个点，该点的偏微分位于曲面上该点处的切平面上：

$$\mathbf{p}_u = \begin{bmatrix} \frac{\partial x}{\partial u} \\ \frac{\partial y}{\partial u} \\ \frac{\partial z}{\partial u} \end{bmatrix}, \qquad \mathbf{p}_v = \begin{bmatrix} \frac{\partial x}{\partial v} \\ \frac{\partial y}{\partial v} \\ \frac{\partial z}{\partial v} \end{bmatrix}$$

对这两个偏微分的叉积进行归一化处理就可以得到该点的单位法向量，即：

$$\mathbf{n} = \frac{\mathbf{p}_u \times \mathbf{p}_v}{|\mathbf{p}_u \times \mathbf{p}_v|}$$

假定使用凹凸函数（bump function）或移位函数（displacement function），即 $d(u, v)$，将表面沿着法向量方向移动，再假定已知 $d(u, v)$且它的值很小（$|d(u, v)| << 1$），则移位后的表面为：

$$\mathbf{p}' = \mathbf{p} + d(u, v)\mathbf{n}$$

由于这个移位后的表面比移位前具有更高的几何复杂度，并且会降低系统的绘制速度，所以我们并不想创建这个移位后的表面，而只是希望使该表面看上去已经进行了移位处理。我们可以通过改变法向量 $\mathbf{n}$ 而不是位置矢量 $\mathbf{p}$ 获得所需的效果，并且在对表面进行着色计算时，需要使用扰动后的法向量。

扰动点 $\mathbf{p}'$ 处的法向量用下面的叉积来计算：

$$\mathbf{n}' = \mathbf{p}'_u \times \mathbf{p}'_v$$

其中，$\mathbf{p}'_u$ 和 $\mathbf{p}'_v$ 这两个偏微分值是通过对方程 $\mathbf{p}'$ 两端求微分计算得到的，即：

$$\mathbf{p}'_u = \mathbf{p}_u + \frac{\partial d}{\partial u}\mathbf{n} + d(u, v)\mathbf{n}_u$$

$$\mathbf{p}'_v = \mathbf{p}_v + \frac{\partial d}{\partial v}\mathbf{n} + d(u, v)\mathbf{n}_v$$

如果 d 的值很小，那么可以忽略上面两个方程最右边的那个项，通过计算它们的叉积得到近似的扰动向量 $\mathbf{n}'$（注意，$\mathbf{n} \times \mathbf{n} = 0$）：

$$\mathbf{n}' \approx \mathbf{n} + \frac{\partial d}{\partial u}\mathbf{n} \times \mathbf{p}_v + \frac{\partial d}{\partial v}\mathbf{n} \times \mathbf{p}_u$$

上式右边的两个项都是移位量，即原法向量与扰动后的法向量之差。由于两个向量的叉积与这两个向量都正交，所以上式右边的两个叉积产生的矢量都位于**p**点的切平面上，同时，这两个叉积矢量之和也位于这个切平面上。

虽然$\mathbf{p}'_u$和$\mathbf{p}'_v$都位于与$\mathbf{n}'$正交的切平面上，但是它们并不一定相互正交。我们使用叉积运算可以得到一个正交基和一个相应的旋转矩阵。首先，我们归一化$\mathbf{n}'$和$\mathbf{p}'_u$，分别得到下面两个矢量：

$$\mathbf{m} = \frac{\mathbf{n}'}{|\mathbf{n}'|}$$

$$\mathbf{t} = \frac{\mathbf{p}'_u}{|\mathbf{p}'_u|}$$

然后通过使用**m**和**t**的叉积可以得到与之正交的第三个矢量**b**：

$$\mathbf{b} = \mathbf{m} \times \mathbf{t}$$

其中，矢量**t**称为**p**点的切线向量（tangent vector），矢量**b**称为**p**点的副法线向量（binormal vector）。矩阵

$$\mathbf{M} = [\,\mathbf{t} \quad \mathbf{b} \quad \mathbf{m}\,]^T$$

是旋转矩阵，它把对象从原空间中的表示变换到由**t**、**b**和**m**组成的新空间中的表示。新空间有时也称为切向量空间（tangent space）。因为切矢量和副法向矢量会随表面上的每个点变化，所以切平面空间是一个局部坐标系统。

为了更好地理解之前介绍的另一个标架（这个标架对于表面上的点而言是局部标架）的含义，下面分析平面$z=0$的凹凸贴图。其表面可以表示为下面的隐式形式：

$$f(x, y) = ax + by + c = 0$$

如果假设$u=x$且$v=y$，那么，如果$a \neq 0$，则：

$$\mathbf{p}(u, v) = \begin{bmatrix} u \\ -\frac{b}{a}v - \frac{c}{a} \\ 0 \end{bmatrix}$$

矢量$\partial\mathbf{p}/\partial u$及$\partial\mathbf{p}/\partial v$经归一化处理得到下面的正交向量：

$$\frac{\mathbf{p}'_u}{|\mathbf{p}'_u|} = [\,1 \quad 0 \quad 0\,]^T$$

$$\frac{\mathbf{p}'_v}{|\mathbf{p}'_v|} = [\,0 \quad 1 \quad 0\,]^T$$

因为可以证明这两个矢量是正交的，所以可以把它们当做切线向量和副法线向量。向量**n**的单位矢量为：

$$\mathbf{n} = [\,0 \quad 0 \quad 1\,]^T$$

对于这种情形，假设移位函数为$d(x, y)$。为了确定凹凸贴图，需要使用两个函数来计算$\partial d/\partial x$和$\partial d/\partial y$的值。如果通过分析可以得到这两个函数，那么就可以在应用程序或着色器中对其进行计算。然而我们通常是把移位函数$d(x, y)$看做是对像素数组$\mathbf{D} = [d_{ij}]$的采样值。计算凹凸贴图所需的偏微分值可以近似等于下面数组中相邻两个元素的差分：

$$\frac{\partial d}{\partial x} \propto d_{ij} - d_{i-1,j}$$

$$\frac{\partial d}{\partial y} \propto d_{ij} - d_{i,j-1}$$

这些数组可以在应用程序中预先计算出来，并以纹理（称为法向量贴图）的形式存储。片元程序可以通过一个 sampler 变量来获取相应的纹理值。

在编写所需的着色器之前，先来分析一下一般的情形，也就是当多边形表面不在 $z=0$ 所在的平面上时，处理凹凸贴图有什么不同。在前面那个简单的情形中，切向量空间坐标轴与对象空间或世界坐标轴是重合的。通常情况下，一个表面的法向量不一定指向 z 方向或其他特定的坐标轴方向。另外，虽然切线向量和副法线向量相互正交，但是法向量在世界坐标系或对象坐标系中没有固定的方向。由于移位函数沿着法向量度量移位的大小，所以其偏微分位于任意的平面上。然而，这个移位函数在切向量空间沿 z 坐标方向。因此，由法向量、切线向量和副法线向量组成的 **M** 矩阵之所以很重要，主要是因为可以利用该矩阵变换到局部坐标系中，而在该坐标系中，可以按前面介绍的方式计算凹凸贴图。在实现凹凸贴图时，一般是先计算出这个矩阵，然后利用该矩阵把对象空间的向量变换到切向量空间所在的局部坐标系中。因为切向量空间是局部坐标系，所以每个片元法向量的变化不相同。对于多边形格网，如果每个多边形使用相同的向量，那么我们可以简化计算，应用程序只需把多边形的切线向量和副法线向量传给顶点程序，对每个多边形只需要传送一次。

9.13.2　示例

现在可以编写凹凸贴图的顶点程序和片元程序了。当对表面某点进行光照计算时，需要知道表面的法向量、光源向量、半角向量和顶点位置等，它们通常是位于观察坐标系或对象坐标系中，而不管是使用法向量贴图还是在片元程序中通过编程来计算扰动向量，都要在纹理空间坐标中进行移位处理。所以为了得到正确的绘制结果，必须把法向量贴图变换到对象空间坐标系，或者把对象空间坐标系变换为纹理空间坐标系。一般来说，后者需要的计算量更少，这是因为它是在顶点程序中基于每个顶点执行的，而不是基于每个片元执行的。我们已经知道，用来把对象空间变换为纹理空间的变换矩阵正好是一个由法向量、切线和副法向量组成的矩阵。对象空间的法向量一般由内置的 gl_Normal 提供，切向量作为一个顶点属性由应用程序提供，这两个向量需要逐个多边形或逐个顶点计算；副法向量可以在着色器中使用叉积函数来计算。这些计算都是在顶点程序中执行的，顶点程序输出的光源矢量和视点矢量位于切向量坐标系中，它们被传送给片元程序使用。因为位于切向量坐标系中的法向量总是指向 z 轴的正方向，所以使用视点矢量和光源矢量就足以在切向量空间执行光照计算了。

在这个简单的例子中，我们是在纹理空间中进行计算，因此需要把光源矢量和视点矢量都变换到纹理空间中。所需的变换矩阵由法向量、切向量和副法向量组成。法向量可以由 OpenGL 的系统状态变量（gl_Normal）提供；切线可以通过在应用程序中定义一个顶点属性（tangento）来提供，然后就可以使用叉积来计算副法向量。由于法向量和切线都是在对象空间中定义的，所以需要首先与法向量矩阵相乘。

与通常情形一样，应用程序通过 gl_Vertex 提供的视点矢量必须先变换到裁剪坐标系，然后变换到纹理坐标系。为了得到视点在纹理坐标系中的表示，必须首先把它们变换到观察坐标系，然后把作用于光源矢量的矩阵应用到视点上。下面是顶点程序：

```
/* 凹凸贴图顶点程序 */

varying vec3 L; /* 在纹理坐标系中的光源矢量 */
varying vec3 V; /* 在对象坐标系中的切向量 */
attribute vec3 objTangent; /* 在纹理坐标系中的视点矢量 */

void main()
{
    gl_Position = gl_ModelViewProjectionMatrix*gl_Vertex;

    gl_TexCoord[0] = gl_MultTexCoord0;
    vec3 eyePosition = vec3(gl_ModelViewMatrix*gl_Vertex);
    vec3 eyeLightPos = vec3 (gl_LightSource[0].position);

   /* 观察坐标系中的法向量、切线向量和副法线向量 */

    vec3 N = normalize(gl_NormalMatrix*gl_Normal);
    vec3 T  = normalize(gl_NormalMatrix*objTangent);
    vec3 B = cross(N, T);

    /* 纹理空间的光源矢量 */

    L.x = dot(T, eyeLightPos-eyePosition);
    L.y = dot(B, eyeLightPos-eyePosition);
    L.z = dot(N, eyeLightPos-eyePosition);

    L = normalize(L);

    /* 纹理空间的视点矢量 */

    V.x = dot(T, -eyeLightPos);
    V.y = dot(B, -eyeLightPos);
    V.z = dot(N, -eyeLightPos);

    V = normalize(V);
}
```

第一个片元程序使用正弦曲线的形式来模拟移位函数：

$$d(x, y) = a \sin(bx) \sin(by)$$

我们可以在着色器中使用这个移位函数来计算所需的偏微分。对于微量的移位，在纹理坐标系中使用的扰动向量为：

$$\mathbf{n} = \begin{bmatrix} \frac{\partial d}{\partial x} \\ \frac{\partial d}{\partial y} \\ 1 \end{bmatrix} = \begin{bmatrix} ab \cos(bx) \sin(by) \\ ab \sin(bx) \cos(by) \\ 1 \end{bmatrix}$$

在着色器中，使用纹理坐标系的s和t代替上式的x和y。由于扰动向量不是单位矢量，所以在着色器中计算漫反射项之前需要把扰动向量和纹理坐标系中的光源矢量归一化为单位矢量。下面是片元程序：

```
varying vec3 L;

void main()
{
     float a = 0.5 // 正弦波的幅度
```

```
    float b =10.0 // 正弦波的频率
    vec3 N = vec3(0.0, 0.0, 1.0);
    float s = gl_TexCoord[0].s;
    float t = gl_TexCoord[0].t;
    N.x = a*b*sin(b*s)*cos(b*t);
    N.y = a*b*sin(b*t)*cos(b*s);
    N = normalize(N);
    vec3 LL = normalize(L);
    float Kd = max(dot(N, LL), 0.0);

   gl_FragColor = Kd * gl_FrontLightProduct[0].diffuse;

}
```

另一种做法是在应用程序中提供归一化的扰动向量，它被存储为一个纹理贴图，称为法向量贴图（normal map）。因为纹理贴图存储的是颜色值，颜色分量的取值范围是（0.0, 1.0），而归一化的法向量分量的取值范围是（-1.0, 1.0），所以必须在应用程序中压缩法向量的每个分量c的范围，压缩公式为：

$$c' = 0.5 * c + 0.5$$

而在着色器中以相反的方式还原为原来的范围：

$$c = 2.0 * c' - 1.0$$

如果使用这种方法，片元程序对应的代码如下：

```
varying vec3 L;
varying vec3 V;
uniform sampler2D texMap;

void main()
{
     vec4 N = texture2D(texMap, gl_TexCoord[0].st);
     vec3 NN = normalize(2.0*N.xyz-1.0);
     vec3 LL = normalize(L);
     float Kd = max(dot(NN, LL), 0.0);

    vec4 diffuse  = Kd * gl_FrontLightProduct[0].diffuse;

    gl_FragColor = diffuse;
}
```

注意，这个例子并没有使用在顶点程序中计算得到的位于纹理坐标系中的视点矢量。如果希望增加镜面反射项，则需要计算这个矢量（见习题9.22）。目前讨论的仅仅是表面的凹凸贴图。在凹凸贴图最强有力的应用中，有许多是与第11章中所讨论的过程纹理生成结合起来使用的。

图9.12显示了单个多边形上的凹凸贴图。其实现方法是在一个矩形区域的y方向进行移位处理。对于左边的图像，光源位于多边形的正上方；而对于右边的图像，光源位于同一高度，但在xz平面旋转了45°。

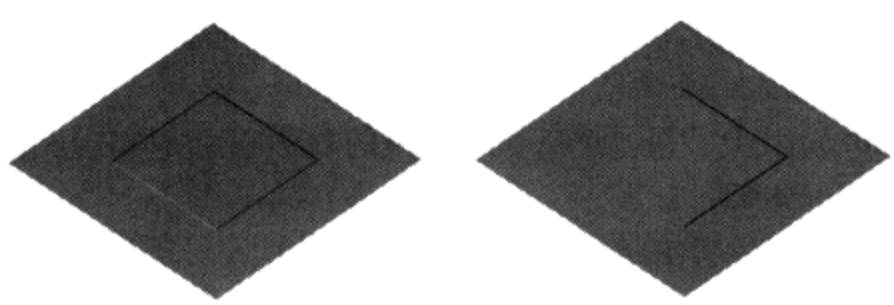


图9.12　一个矩形表面的凹凸贴图

小结和注释

NVIDIA GeForce 3是第一个可编程GPU，虽然最初的可编程GPU只支持顶点程序，但不久，许多硬件制造商就制造出了可同时支持顶点程序和片元程序的GPU。

尽管顶点程序在某些应用中受到关注，但是在功能上有其自身的限制。尤其是顶点程序仅可以处理纹理坐标而不能处理纹理本身。现在有了对片元程序的支持，从而打开了通向新技术的大门，这样，我们就既可以使用传统方法也可以使用新的方法来对纹理进行处理。在可编程绘制流水线不断发展的同时，GPU的纹理内存也越来越大，而且也增加了对多纹理单元以及并行片元处理器的支持。许多技术都采用了基于片元的光照计算，不仅可以使用新的模型和基于片元-片元的模式来处理纹理，而且哪怕是基于每个片元简单地使用标准的改进型Phong光照模型也能改善图像的视觉质量。对于计算机图形学来说，实时处理凹凸贴图等技术的能力对图形应用开发人员产生了巨大的影响。例如，许多具有三维真实感环境的视频游戏都大量使用了凹凸贴图。

最近GPU方面的发展又为人们提供了更多的功能。尽管顶点程序和片元程序都支持浮点类型的数据，但是直到最近，帧缓存仍然是使用8位的精度来存储每个颜色分量。最新的GPU具有浮点型帧缓存，从而解决了在图像合成或图像融合等技术中所遇到的精度问题。将时间倒退几年，从通用计算的角度来看现在的GPU，我们发现，价格只有几百美元的GPU，其执行操作速度却达到了以往的超级计算机的水平。现在的问题是：GPU的体系结构远不同于标准的CPU，GPU的高运行速度来自于其各个部件的组织方式以及对矢量的4个分量的并行处理机制。目前，如何使用GPU进行通用计算（简称GP-GPU）引起了人们的极大兴趣。对于特定的应用，我们首先思考原来的标准算法，再提出能够充分利用GPU性能的新算法。一个简单的例子就是把纹理贴图用于功能查找表，前面使用的法向量贴图采用的就是这种方法。片元程序在GP-GPU方面的其他应用包括信号处理和线性代数。

几乎每隔一个月都会有新的GPU特性出现，从而极大地影响到了人们对GPU的使用。例如，不久GPU内部将会增加一个称为几何着色器（geometry shader）的可编程处理单元[①]，它的出现将会消除一个顶点不能生成其他顶点的功能限制。GPU特性的发展使得GPU能够胜任曲面细分等任务，这些任务在以前只能使用CPU完成。

建议阅读资料

要了解RenderMan着色语言，可以参阅The RenderMan Companion[Ups89]。

有关GLSL语言的标准参考资料是The OpenGL Shading Language[Ros04]，也可以参考The Cg Tutorial[Fer03]。虽然GLSL语言与Cg语言之间存在区别，但是使用其中一种语言编写的着色器可以很容易地转换为另一种语言写的着色器。这两本书都提供了许多顶点程序和片元程序示例。NVIDIA网站(www.nvidia.com/developer)和ATI网站(www.ati.com/developer)还提供了许多其他的示例和文档。OpenGL网站(www.opengl.org)提供了各个版本的OpenGL规范，尤其是对扩展规范的说明，扩展规范定义了OpenGL程序与可编程着色器之间的接口。

对菲涅耳项的讨论可以参考一些计算图形学教材，如[Fol90]和[Hil01]等。有关菲涅耳方程近似求解的讨论可以参考[Fer03]。

凹凸映射首先由Blinn提出[Bli77]。要在不同类型的表面正确地实现凹凸贴图，其处理细节不尽相同，详细内容可以参考[Fer03]。

有关GPU的许多算法可以参考[Fer04]。

① 目前最新的GPU已经有了几何着色器。——译者注

习题

9.1　编写一个应用程序，它使用顶点程序把一个对象渐变成另一个对象。除了顶点之外，还需要对顶点颜色等其他变量进行渐变。

9.2　实现一个模拟瀑布的粒子系统，由应用程序生成粒子，新粒子的数量随着时间的增加而增加。粒子的初始速度和位置都在一个较小的范围内。

9.3　编写一个应用程序，它使用粒子系统和顶点程序绘制烟火的效果。粒子的颜色随时间变化而变化，每个粒子看上去有不同的生命周期。

9.4　在许多粒子系统中，粒子之间通过吸引力和排斥力相互作用。能否只使用一个顶点程序实现这样的粒子系统？解释理由。

9.5　编写一个顶点程序绘制一个球的弹跳效果。球的初始速度和位置由应用程序提供。提示：读者可以使用 reflect 函数计算球撞击地面反弹后的新方向。球每次撞击地面后在垂直方向的速度都会减去一个常量，即恢复系数（coefficient of restitution）。

9.6　创建一个太阳系的模型，使用顶点程序移动行星和任意的卫星。

9.7　在 Phong 顶点程序和改进的 Phong 顶点程序中增加距离项。

9.8　试对改进的 Phong 顶点程序进行修改，使它可以使用多个光源。

9.9　在第 6 章，我们注意到，如果光源矢量、法向量和反射矢量位于同一个平面内，那么半角矢量等于视点矢量和光源矢量的一半。请使用这个结论并根据改进的 Phong 模型中的高光系数，推导用来计算 Phong 模型高光系数的公式。

9.10　由于可编程着色器把 CPU 的一些负荷转移给了 GPU，所以能够使代码的效率更高。然而，因为把太多的功能都固化到了GPU中，所以许多有关效率的传统假定就不再成立了。分别使用下面 3 种方法编写 OpenGL 程序：(a) 固定功能绘制流水线；(b) 基于顶点程序的改进的 Phong 着色器；(c) 基于顶点程序的 Phong 着色器。比较这 3 种方法的效率。

9.11　重新编写 Phong 着色器，使其计算过程在对象坐标系而不是观察坐标系中执行。

9.12　编写一个实现 Phong 模型着色的片元程序。

9.13　编写顶点程序和片元程序，实现基于球形贴图的环境映射。

9.14　光在两点之间沿最短的路径传播，请根据这个假定推导出折射定律。

9.15　试推导类似于计算镜面反射光的半角矢量，用它来计算折射光对表面颜色的贡献。

9.16　重新编写计算折射光的顶点和片元程序。要求考虑色散现象，让应用程序为红、绿、蓝颜色分量分别提供折射率。

9.17　假定通过凹凸贴图把地球的高度场映射到一个球体上。如何使用凹凸贴图而不是实际改变球体表面的高度场来绘制反映高度场的图像。

9.18　编写一个程序，把凹凸贴图 $d(x, y)$应用到一个参数表示的球面。

9.19　在 9.7.1 节，如果使用一个标量 s 而不是矢量$[s, s, s, 1]$来对每个顶点进行缩放，分析程序的运行结果。

9.20　分析光进入和离开较薄介质时的效果。光在这种介质中传播的速度小于在空气中传播的速度。如何修改立方体贴图的折射光计算公式？

9.21　如何修改凹凸映射着色器才能使它适合于处理多边形网格？

9.22　在 9.13 节中介绍的凹凸映射示例中增加镜面反射项。

9.23　我们讨论的凹凸贴图与表面法向量方向微小的移位有关，考虑有没有对该假设不成立的例子。

9.24　修改凹凸贴图着色器，使它能够处理远处的光源以及镜面反射项。

第10章　层级建模方法

模型是对世界的抽象，这个世界既可以是我们生活的真实世界，也可以是我们利用计算机生成的虚拟世界。我们都熟悉在科学和工程领域中使用的数学模型。这些模型使用数学方程来描述我们希望研究的物理现象。在计算机科学领域，人们使用抽象数据类型表示对象之间的结构关系；在计算机图形学领域，人们使用几何对象构建这个世界的模型。当建立一个数学模型时，需要仔细地选择数学方程，该方程应适合于描述人们要仿真的物理现象。虽然常微分方程可能适合于描述弹簧-质点系统的动态行为，但是我们可能使用偏微分方程描述湍流流动现象。在计算机图形学领域，我们也经历了类似的过程，例如需要选择模型使用的图元，并表示这些图元之间的关系。就像选择数学模型时有多种方法一样，人们选择的几何模型应该能够充分利用图形系统的性能。

本章将使用多种不同的方法构造并使用几何对象模型。此外还将介绍一些用于构造复杂模型的简单几何对象，这些简单的几何对象或者是图形系统所支持的图元，或者是由这些图元构成的用户自定义对象。我们把第4章介绍的几何变换推广到对象之间的层级关系。我们介绍的这些建模技术适合于机器人和角色动画等应用，在这些应用中，对象的动态行为表现为对象模型各部分之间的层级关系。

层级结构是一个表示能力很强的概念，它是面向对象方法的一个重要组成部分。我们把对象的层级模型扩展到整个场景，包括虚拟照相机、灯光和材质属性。基于这样的层级模型，可以把图形API扩展为面向对象的系统，并且还能使读者深入理解如何在网络和分布式环境中使用图形系统。

10.1　图符和实例

我们首先关心的是如何存储一个由许多复杂对象组成的模型。有两个直接相关的问题需要解决：第一个问题是如何定义一个复杂的对象，它比我们目前处理过的对象更复杂；第二个问题是如何表示这些复杂对象的组合。大多数图形API对图元采取最小完备的观点，即它们只包含少数的基本图元，而让用户通过这些基本图元来构建更复杂的对象。其他一些与OpenGL一起使用的库，如GLU库和GLUT库，这些库函数为方便程序员的使用，提供了一些建立在基本图元之上的复杂对象。假定本章可以使用这些库函数提供的一些基本的三维对象。

我们可以采取非层级结构的建模方法，该方法将这些对象视为一些图符（symbol），并把所要建模的世界看做是由这些离散的图符组合而成的。图符包括几何对象、字体和一些应用程序专用的图形对象。图符通常根据自身的几何特征表示为合适的大小和方位。例如，如图10.1所示，在表示一个圆柱体时，通常使它的中心轴与坐标系的某个主轴平行，其高度一般为1个单位，半径为1个单位，并且它的底面圆心与坐标系的原点重合。

包括OpenGL在内的大多数图形API都把图符所在的坐标系［有时称为建模坐标系（model frame）］和世界坐标系（world frame）区分开来。当图符只是一种形状符号时，比如在CAD应用中可能使用的电路元件符号，它们没有物理单位，那么区分这两种坐标系对人们构建复杂的对象会

有帮助。在OpenGL应用程序中，必须通过几何变换把图符从建模坐标系变换到世界坐标系。因此，一个特定图符的模-视变换矩阵是由两个矩阵相乘得到的，一个是把图符从建模坐标系变换到世界坐标系的实例变换矩阵，另一个是把图符从世界坐标系变换到观察坐标系的观察变换矩阵。

我们可以使用第4章介绍的实例变换把每个图符实例按所需的大小、方向和位置放入场景中。因此，下面的实例变换矩阵是平移变换矩阵、旋转变换矩阵和缩放变换矩阵（也可能是错切变换矩阵）相乘的结果，如图 10.2 所示。

$$\mathbf{M} = \mathbf{TRS}$$

OpenGL 程序经常包含下面形式的代码：

```
glTranslatef(dx, dy, dz);
glRotatef(angle, rx, ry, rz);
glScalef(sx, sy, sz);
glutSolidCube(side); /* 或其他的图符 */
```

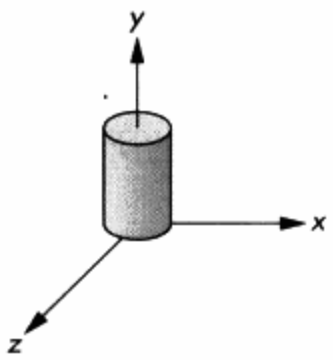


图 10.1　圆柱体图符

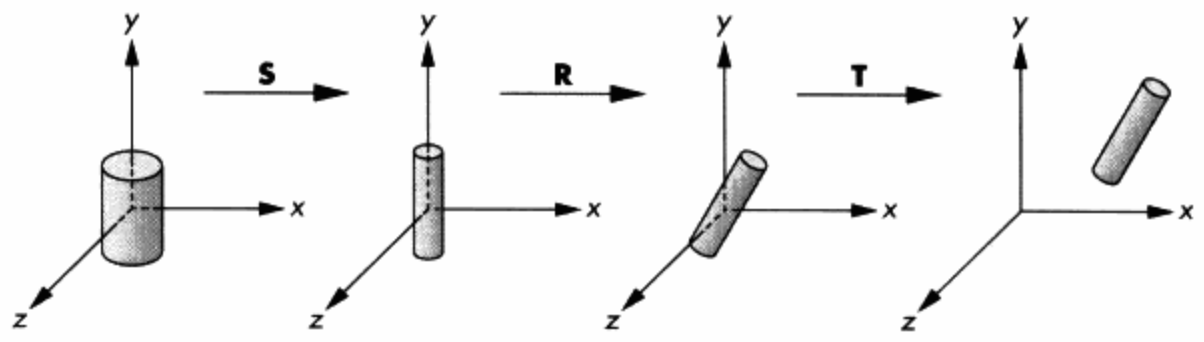


图 10.2　实例变换

我们也可以使用如图 10.3 所示的表格形式表示这个模型。这里假定每个图符都有一个唯一的数值标识符。该表格存储了这个标识符以及构建实例变换矩阵所需的一些必要参数。该表格表明这种建模技术并不包含反映对象之间关系的信息。然而，该表格包含用来绘制对象所需的所有信息，因此它是一个简单的数据结构，或者是一个用来描述一组几何对象的模型。我们可以从表格中搜索某个对象，改变某个对象的实例变换，添加或删除对象。然而，这种表格单一线性的表示限制了其应用。

图符	缩放	旋转	平移
1	s_x, s_y, s_z	$\theta_x, \theta_y, \theta_z$	d_x, d_y, d_z
2			
3			
1			
1			
.			
.			

图 10.3　图符 – 实例变换表

10.2　层级模型

假定需要建立一个汽车动画模型。我们可以使用 5 个部件组成这个模型—— 一个车身底盘和 4 个轮子（如图 10.4 所示）——其中每个部件都可以使用标准的图元来表示。图 10.5 表示的是这个简单动画模型的两帧画面。我们发现如果轮子的半径是 r，那么当轮子转动 360° 时，汽车向前或向后移动的距离是 $2\pi r$，据此可以编写一个程序来生成这个动画。该程序中有 4 个函数分别用来生成

汽车的4个轮子，另一个函数用来生成车身底盘。所有这些函数使用相同的输入参数，例如汽车所需的速度和方向。使用伪代码编写程序大致如下：

```
{
   float s; /* 速度 */
   float d[3] /* 方向 */
   float t /* 时间 */

   /* 确定汽车在时刻t时的速度和方向 */

   draw_right_front_wheel(s,d);
   draw_left_front_wheel(s,d);
   draw_right_rear_wheel(s,d);
   draw_left_rear_wheel(s,d);
   draw_chassis(s,d);
}
```

我们并不希望编写的程序是这样的一种形式。因为它是线性的，无法描述汽车各个部件之间的关系。我们可以使用汽车各部件之间存在的两种关系。首先，不能把汽车的运行与轮子的运行分隔开。如果汽车向前运动，那么轮子必定也跟着转动[①]。其次，可以使用这样一个事实，即汽车的所有轮子都是相同的，只是它们位置和朝向不同而已。

图 10.4　汽车模型

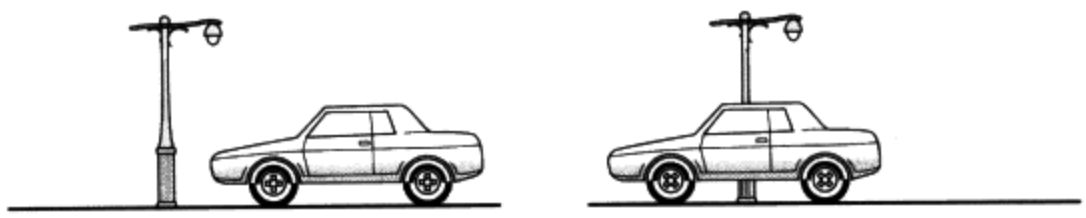


图 10.5　汽车动画模型的两帧画面

我们既可以使用抽象的方法，也可以使用图这种直观的方法表示复杂模型各个部分之间的关系。在数学上，一个图（graph）由一些节点（或顶点）及一些边组成。一条边连接一对节点，也可能连接这个节点本身。图上的边可以具有方向性，本章使用的图都是有向图（directed graph），这种图上的边从一个节点离开，进入另一个节点。

我们使用的最重要的一种图是树。树是一种没有闭路或环路的有向图。另外，除了根节点，每个节点只有一条边进入。因此，除了根节点，每个节点都有一个双亲节点，有一条边从双亲节点出发进入该节点，并且它有一个或多个孩子节点，它们与双亲节点之间通过边连接。没有孩子的节点称为终端节点或叶子。图10.6所示的就是树结构，它表示汽车模型各个部件之间的关系。车身底盘是根节点，4个轮子是它的孩子节点。虽然数学上的图由一组集合元素组成，但是在实际应用中，边和节点可以包含其他的额外信息。在这个例子中，每个节点包含定义几何对象所需的信息。轮子的位置和朝向信息可以存储在轮子的节点里，也可以存储在它们与双亲节点之间相连的边上。

由于大多数汽车的4个轮子都是相同的，所以4个轮子节点存储相同的信息并绘制每个轮子是一种低效的方法。我们使用实例变换背后的思想，只需在汽车模型中使用一个轮子原型。如果采用这种方法，那么可以使用如图10.7所示的有向无环图（direct acyclic graph，DAG）代替上面的树结构。在有向无环图中，尽管存在环路，但是不能沿着有向边环绕。因此，如果从某个节点沿着任意一条有向边路径前进，那么该路径终止于另一个节点，在实际应用中，使用有向无环图并不比使

① 我们无法说清楚是轮子带动车身底盘，还是车身底盘带动轮子。从图形学的观点看，后者可能更有用。

用树更难处理。对于这个汽车模型，我们可以把这个轮子原型每个实例的位置信息存储于车身底盘节点中，或轮子节点中，或每条边上。

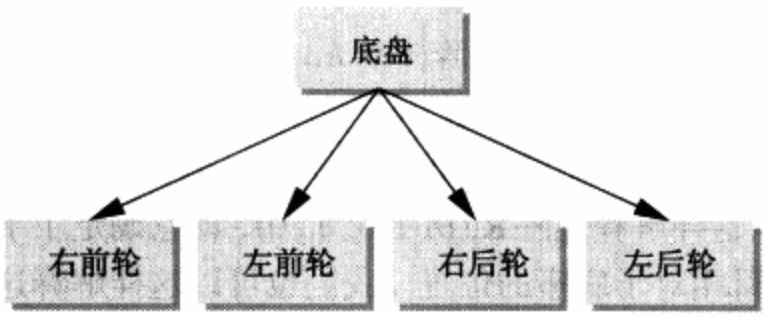


图 10.6　汽车的树形结构

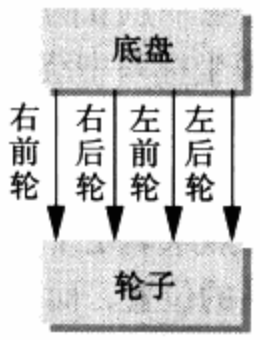


图 10.7　汽车的有向无环图（DAG）模型

树和有向无环图都是用来表示物理模型各部分之间关系的层级（hierarchical）表示方法。在这两种表示方法中，模型的每个部分可以和其他部分（它们的双亲和孩子）通过边关联。后面将讨论如何在图形程序中表示这种层级结构。

10.3　机器人手臂

机器人学科为人们研究层级建模提供了很多便利条件。考虑图10.8(a)所示的一个简单机器人手臂。我们使用3个简单的对象或图符建立这个模型，比如使用两个平行六面体和一个圆柱体。其中的每个图符都可以使用基本图元来建立。

机器人手臂包含如图10.8(b)所示的3个部分。这个机械装置具有3个自由度，其中的两个自由度是机械部件之间的关节连接角度（joint angle），第三个自由度是底座相对于地面某个固定点的旋转角度。在这个模型中，每个关节连接角度确定了某个部件相对于与它相连的另一部件的位置；对于底座来说，关节连接角度确定了底座相对于周围环境的位置。每个关节连接角度都在其对应部件所在的坐标系中度量其大小。我们可以把底座绕垂直轴旋转 θ 角度。该旋转角度是底座平面上从 x 轴到某个固定点的角度。机器手的下臂通过一个关节与底座相连，该关节允许下臂在其所在坐标系的 $z=0$ 平面上旋转。下臂的旋转角度用 ϕ 表示，该旋转角度是从所在坐标系的 x 轴到下臂的角度。上臂通过一个类似的关节与下臂相连，上臂的旋转角度为 ψ，与度量下臂旋转角度的方法类似，上臂旋转角度是从所在坐标系的 x 轴到上臂的角度。当这些角度改变时，可以把上臂和下臂的坐标系看成是相对于底座做旋转运动。通对这三个角度的控制，可以在三维空间中对上臂的顶端位置进行定位。

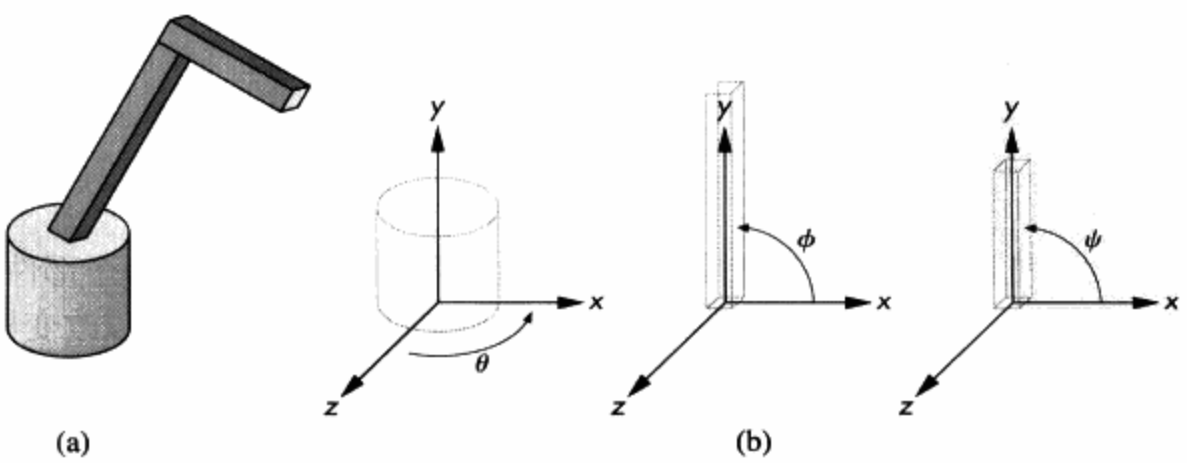


图 10.8　机器人手臂：(a)整体模型；(b)各个部件

假定我们想编写一个程序来绘制这个简单的机器人模型。我们与其单独定义机器人的每个部件及其运动，不如采用增量的方法。机器人的底座可以在其坐标系中绕 y 轴旋转 θ 角度。因此，我们可以表示底座上任意一点 $\mathbf{p}$ 的运动，即用旋转矩阵 $\mathbf{R}_y(\theta)$ 作用于该点。

下臂在其建模坐标系中绕 z 轴旋转，但是必须通过平移变换矩阵 $\mathbf{T}(0, h_1, 0)$ 把该坐标系移动到底座的上方，其中 h_1 是底座的高度，即要求移动到底座与下臂相连的关节处。然而，如果旋转底座，那么必须通过旋转矩阵 $\mathbf{R}_y(\theta)$ 旋转与它相连的下臂。因此，可以把组合矩阵 $\mathbf{R}_y(\theta)\mathbf{T}(0, h_1, 0)\ \mathbf{R}_z(\phi)$ 作用于下臂任意顶点来确定下臂的位置。我们可以这样解释矩阵 $\mathbf{R}_y(\theta)\ \mathbf{T}(0, h_1, 0)$，即它确定了下臂相对于世界坐标系的位置，而矩阵 $\mathbf{R}_z(\phi)$ 确定了下臂相对于底座的位置。我们也可以这样解释这些矩阵，即它们表示下臂和底座的建模坐标系相对于某个世界坐标系的位置，如图 10.9 所示。

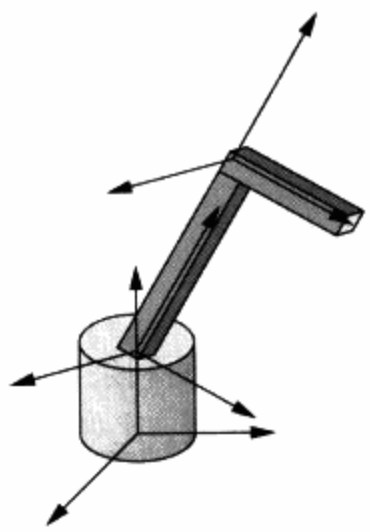


图 10.9　机器人各个部件的运动与各个部件的坐标系

当采用类似的方法推导上臂的位置时，我们发现相对于下臂，必须先把平移变换矩阵 $\mathbf{T}(0, h_2, 0)$ 作用于上臂，然后再把旋转变换矩阵 $\mathbf{R}_z(\psi)$ 作用于上臂。因此控制上臂的组合变换矩阵是 $\mathbf{R}_y(\theta)\ \mathbf{T}(0, h_1, 0)\ \mathbf{R}_z(\phi)\ \mathbf{T}(0, h_2, 0)\mathbf{R}_z(\psi)$。下面是 OpenGL 程序中一个用于绘制机器人的 display 函数，它用到了各个部件的关节连接角度，该函数说明了如何以增量的方式改变模–视变换矩阵，并有效地绘制模型的各个组成部分：

```
display()
{
    glRotatef(theta, 0.0, 1.0, 0.0);
    base();
    glTranslatef(0.0, h1, 0.0);
    glRotatef(phi, 0.0, 0.0, 1.0);
    lower_arm();
    glTranslatef(0.0, h2, 0.0);
    glRotatef(psi, 0.0, 0.0, 1.0);
    upper_arm();
}
```

我们注意到虽然描述了机器人手臂各个部件之间的位置关系，但没有考虑到每个部件的具体细节。只要关节的位置不变，那么只需通过改变上面代码中绘制手臂的3个函数就可以改变机器人手臂的外形。机器人手臂各部件的相互独立性使我们可以编写用来描述每个部件的独立函数，从而实现机器人的动画绘制程序。图 10.10 以树的形式说明了机器人手臂各部件之间的关系。附录 A 中的完整程序实现了该树形结构，它允许用户通过菜单并使用鼠标控制机器人手臂动画。该程序使用一个圆柱体构建机器人的底座模型，而使用两个平行六面体构建机器人的手臂模型。

回到图 10.10 所示的树形结构，我们可以把这这种结构看成是由节点和边组成的树或图。如果把所有必要的信息都存储在节点里面而不是边上，那么每个节点（如图 10.11 所示）至少需要下面 3 项内容：

1. 函数指针，该指针指向一个用于绘制该节点所表示的对象的函数。
2. 齐次坐标矩阵，表示该节点（及其孩子节点）相对于其双亲节点的位置、缩放和旋转变换。
3. 指向孩子节点的指针。

当然，我们可以在节点里存储其他的信息，例如作用于节点的一些属性信息（颜色、纹理、材质属性）。绘制这种树形结构表示的对象需要执行树的遍历（traversal）操作。所谓树的遍历就是要求访问树的每个节点；在访问每个节点时，必须计算相应的变换矩阵，该矩阵作用于该节点指向的图元，并且要求显示这些图元。OpenGL 程序采用增量的方法实现这种遍历操作。

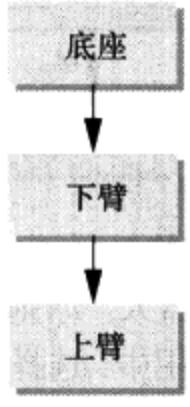


图 10.10　图 10.8 所示的机器人树形结构

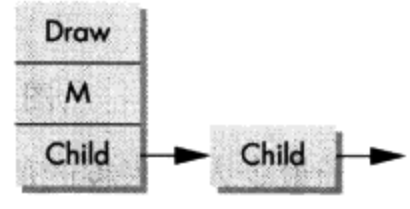


图 10.11　节点表示

这个例子很简单：树中的每个双亲只有一个孩子节点。下一个例子将说明如何处理更复杂的模型。

10.4　树与树的遍历

图 10.12 所示的是一个类人机器人的盒状表示，它可用于机器人建模或虚拟现实应用中。如果把机器人的躯干作为根节点，那么可以使用如图 10.13 所示的树形结构表示该机器人。一旦确定了机器人躯干的位置，就可以通过与之相连的关节角度确定该模型其他部件的位置和方向。我们可以通过定义这些关节的运动来生成机器人的动画。在一个简单的机器人模型中，与前面的机器人手臂一样，其膝关节和肘关节都只有一个自由度，而颈关节可能有两个或三个自由度。

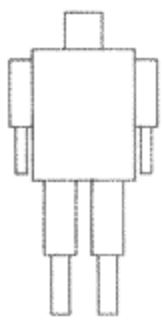


图 10.12　类人机器人

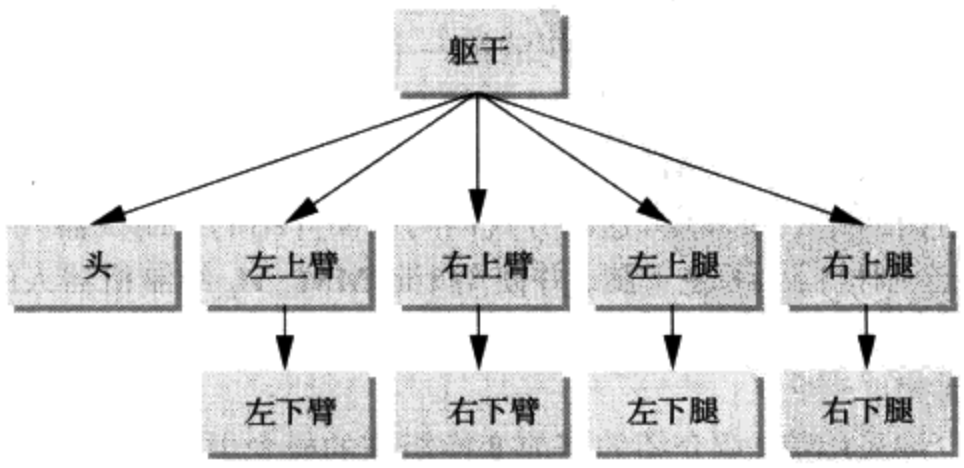


图 10.13　图 10.12 所示机器人的树形表示

假定我们定义了head和left_upper_arm之类的函数，这些函数用来在建模坐标系中绘制机器人的各个部件（图符）。与前面生成机器人手臂的动画类似，现在要建立这棵树的一组节点，这里采用的方法是为每个节点定义一个相对于双亲节点的变换矩阵，通过这个变换矩阵，可以确定对应节点的位置。如果假定定义好了机器人各个部件的大小，那么每个节点的变换矩阵是一个平移变换矩阵与一个旋转变换矩阵的乘积形式。如图10.14所示，我们可以使用这些矩阵标示树的边。记住，每个变换矩阵表示从双亲节点到孩子节点的递增变换。

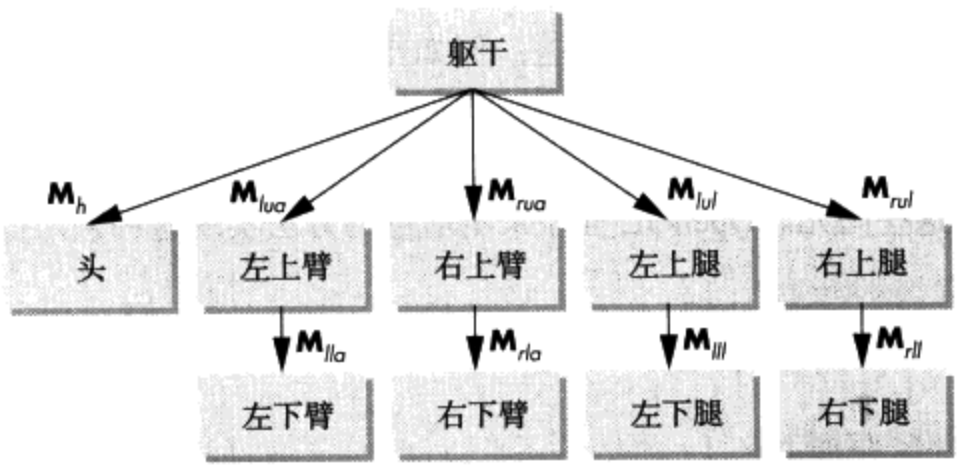


图10.14　边上标示了变换矩阵的树形表示

这个例子最令人感兴趣的部分是如何通过遍历这棵树绘制机器人。原则上，我们可以使用任何树的遍历算法，如深度优先或宽度优先搜索算法。虽然在许多应用中，选择哪种遍历算法无关紧要，但是我们有充分的理由认为应该始终使用同一种遍历算法。这里使用先左后右的深度优先遍历算法。也就是说，我们沿着树左子树开始访问节点，一直遍历到树叶子节点，然后回溯到树上层访问第一个右子树，照此一直递归下去。树的这种遍历方法称为先序遍历（pre-order traversal）。

可以采取两种方式编写树的遍历函数。我们可以编写代码来显式地遍历树，当遍历树时，可以使用栈结构存储所需的矩阵和属性信息。我们也可以使用递归函数实现树的遍历。对于第二种方法，由于系统隐式地存储了矩阵和属性信息，所以编写的程序比较简单。因为这两种方法都有用，并且了解其开发过程有助于深入理解图形系统的构建过程，所以下面介绍这两种方法。

10.4.1　基于栈的遍历方法

下面分析如何使用函数figure绘制机器人。我们可以使用GLUT库中的显示回调函数来调用figure函数，或者使用鼠标回调函数来调用该函数，这样可以使用鼠标控制机器人的关节角度，从而实现机器人动画。当调用figure函数时，模-视变换矩阵$\mathbf{M}$决定了机器人相对于场景中其他对象（包括照相机）的位置。遍历树的第一个节点时绘制机器人的躯干，该节点对应的变换矩阵$\mathbf{M}$作用于后面遍历的所有图元。然后，我们遍历最左边的子树，该节点表示机器人的头部。我们使用更新后的模-视变换矩阵$\mathbf{MM}_h$作用于绘制头部的函数head。接着，我们回溯到躯干节点，然后沿着子树往下访问左上腿节点。这部分代码与前面介绍的绘制机器人手臂动画的代码类似。我们使用矩阵$\mathbf{MM}_{lul}$绘制机器人的左上腿，并使用矩阵$\mathbf{MM}_{lul}\mathbf{M}_{lll}$绘制机器人的左下腿。接着依次访问机器人的右腿、左臂和右臂。每当切换访问到机器人的另一肢时，都要先回溯到根节点，并把当前的变换矩阵恢复到$\mathbf{M}$。

也许采用第4章介绍的当前变换矩阵的概念更容易解释机器人绘制函数，假设模-视变换矩阵$\mathbf{C}$表示作用于某个节点的当前变换矩阵[①]。矩阵$\mathbf{C}$的初始值为$\mathbf{M}$，绘制机器人的头部节点时，矩阵

① 目前可以忽略投影变换矩阵。

C的值更新为$\mathbf{MM}_h$，后来更新为$\mathbf{MM}_{lul}\mathbf{M}_{ll}$，以此类推。每当调用机器人某个部件的绘制函数时，用户程序必须正确设置当前的变换矩阵**C**。例如，当回溯并访问机器人的右上臂时，需要把当前变换矩阵**C**重新置为**M**。此时不用重新构建矩阵**M**（或其他在更复杂的模型中需要重新使用的矩阵），而是使用函数 glPushMatrix 把它存储在堆栈里，使用函数 glPopMatrix 恢复该矩阵。因此，树的遍历代码中除了平移和旋转等变换矩阵外，还有对模 – 视变换矩阵的压栈和出栈操作。

下面分析 figure 函数（没有考虑参数）开头部分的代码：

```
figure()
{
    glPushMatrix();
       torso();
       glTranslate();
       glRotate3():
       head();
    glPopMatrix();
    glPushMatrix();
       glTranslate();
       glRotate3();
       left_upper_leg();
       glTranslate();
       glRotate3();
       left_lower_leg();
    glPopMatrix();
    glPushMatrix();
       glTranslate();
       glRotate3();
       right_upper_leg();
    glPopMatrix();
    glPushMatrix();
    .
    .
    .
```

第一个 glPushMatrix 把当前的模 – 视变换矩阵的一个备份压入堆栈的顶部（假定在此之前调用了 glMatrixMode (GL_MODELVIEW)函数）。由于在堆栈中保留了当前变换矩阵的一个备份，所以可以立即进行其他的几何变换从而改变当前变换矩阵。后面调用的两个函数 glTranslate 和 glRotate 确定了$\mathbf{M}_h$矩阵，并把它与最初的模 – 视变换矩阵相乘。之后，我们可以绘制组成头部的图元。head 函数后面的 glPopMatrix 函数用于恢复最初的模 – 视变换矩阵。注意，我们必须再次使用 glPushMatrix 函数在堆栈中保存最初的模 – 视变换矩阵，这样就可以在绘制机器人右臂时恢复这个最初的模 – 视变换矩阵。按同样的方法可以完成该函数的其余部分。

附录 A 包含绘制这个机器人的完整代码，程序提供的菜单可以让用户改变机器人各个关节的旋转角度。机器人的各个部件使用 OpenGL 提供的二次曲面绘制函数（椭圆和圆柱体绘制函数）绘制，整个模型的绘制采用了第 6 章介绍的明暗处理方法。

我们还没有考虑在层级模型的遍历中如何处理颜色和材质等属性。属性是状态变量：属性值一旦设置好了就不会改变，直到重新设置一个新的状态值。因此，遍历树的时候必须小心谨慎。例如，假定在绘制机器人躯干的代码中把当前颜色设置为红色，然后在绘制机器人头部的代码中把颜色设置为蓝色。如果后面的代码不改变颜色值，那么遍历这棵树的其余部分时，绘制颜色仍是蓝色，并且在退出 figure 函数后绘制的颜色还是蓝色。这个例子说明了一个特定的遍历算法可能会得到不同的绘制结果，因为遍历树中节点的次序会影响当前的状态。

这种情形也许有点不尽人意，但还是有解决办法的。OpenGL提供了glPushAttrib和glPopAttrib两个函数[①]，这两个函数可以让我们像处理模－视变换矩阵一样处理属性值。如果在函数figure的入口处把属性值压入属性堆栈中，而在函数figure退出时从堆栈中弹出属性值，那么就能把属性值恢复到初始状态。此外，我们还可以在figure函数内部使用附加的压栈和出栈操作以便更彻底地控制属性的处理方式。OpenGL把状态分成不同的组，允许用户把任意一组属性值压入堆栈中。用户只需对函数glPushAttrib参数的掩码位设置相应的值。属性组包括光照，因此可以把材质属性和光照压入栈中，另外还有多边形属性和线段属性。

在更复杂的模型中，我们使用递归函数实现这些想法。例如，如果希望使用更细致的头部模型——它有眼睛、耳朵、鼻子和嘴——那么可以分别对这些部件建模。头部本身也是一种层级模型，它的代码也包括对矩阵和属性的压栈、出栈操作。

虽然我们只讨论了树形结构，但是如果两个或多个节点调用相同的绘制函数，那么我们处理的是一个有向无环图（DAG），而对DAG的处理并不会带来额外的困难。在OpenGL中，显示列表可以调用其他的显示列表，因此我们还有另一种建立树和DAG层级模型的方法。

彩图36[②]显示了仿真中使用的机器人和人体的层级模型。需要使用具有高级交互功能的软件建立这些对象，而绘制这些模型有赖于对层级结构的遍历。

我们用来描述层级对象的方法是切实可行的，但是也有局限性。我们编写的代码很清晰，而且需要程序员对属性和矩阵进行压栈和出栈操作。实际上，我们实现了一种基于栈的树形表示。上面的代码只是针对某个特定的例子，是不能改变或调整的，因此很难进行扩展或动态地使用。代码还有一个局限性，即没有区分建模过程和绘制过程。虽然许多程序开发人员都采用这种方式编写代码，但是我们之所以使用它主要是为了说明OpenGL程序实现树形层级结构的流程。下面介绍一种更一般、更强大的树形层级结构实现方法。

10.5　使用树形数据结构

第二种方法使用标准的树形数据结构表示层级模型，然后使用与模型无关的遍历算法绘制它。我们使用左孩子、右兄弟结构。

考虑树的另一种表示方法，如图10.15所示。使用这种结构组织树中的节点时，把位于同级的节点从左到右链接起来，即将某个节点的所有孩子节点从最左边的孩子到最右边的孩子表示成第二个链表。第二个链表按向下的方向排列，如图10.15所示。这种表示结构可以用于描述机器人层级模型，但是该结构还缺少图形信息。

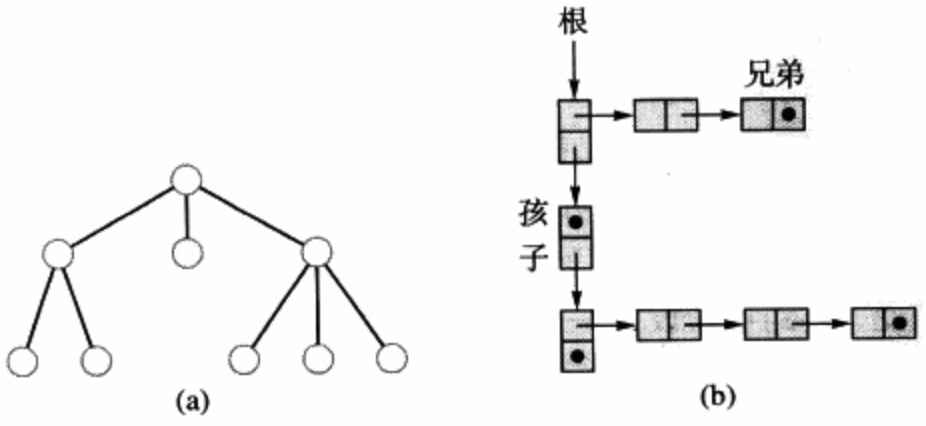


图10.15　(a) 树形表示；(b) 左孩子、右兄弟表示

① 矩阵和属性的压栈、出栈是一种费时的操作。可采用的另一个办法是在应用程序中由程序员来谨慎地管理状态变量。

② 此处原文为彩图16、20和26，可能有误。——译者注

每个节点必须存储绘制对象所需的信息：定义对象的函数，以及用来确定该对象相对于它的双亲节点位置的齐次坐标矩阵。分析下面用来存储节点信息的结构体类型：

```
typedef struct treenode
{
   GLfloat m[16];
   void (*f)();
   struct treenode *sibling;
   struct treenode *child;
} treenode;
```

其中，数组 m 按列方式存储了一个 4 × 4 的齐次坐标矩阵，这是 OpenGL 的一种标准存储方式。当绘制节点对象时，必须首先把该矩阵与模 – 视变换矩阵相乘，然后调用对象的绘制函数 f，该函数里面包含了对象的所有图元。这个结构体中还有两个指针，一个指向它右边的兄弟节点，另一个指向它最左边的孩子节点。如果某个节点没有兄弟节点或最左边的孩子节点，那么只需存储空指针（NULL）。对于前面的机器人模型，我们定义了 10 个节点，它们对应于机器人的 10 个部件：

```
treenode torso_node, head_node, lua_node, rua_node, lll_node,
        rll_node, lla_node, rla_node, rul_node, lul_node;
```

我们可以在 main 函数（程序的主函数）或 myinit 函数（自定义的初始化函数）中定义这些节点。例如，考虑机器人层级树结构的根节点——躯干（torso）节点。躯干可以绕 y 轴旋转。我们可以在代码中通过计算旋转变换矩阵的每个元素来生成该矩阵，也可以使用 OpenGL 生成这个旋转变换矩阵（即使用系统查询函数 glGetFloatv 获取模 – 视变换矩阵），然后通过使用下面的代码把旋转变换矩阵复制给数组 m：

```
glLoadIdentity();
glRotatef(theta[0], 0.0, 1.0, 0.0);
glGetFloatv(GL_MODELVIEW_MATRIX,torso_node.m);
```

我们使用函数 torso 绘制一个圆柱体，它表示机器人的躯干。躯干节点没有兄弟节点，其最左边的孩子节点是 head 节点，所以躯干节点的结构成员可用下面的代码表示：

```
torso_node.f = torso;
torso_node.sibling = NULL;
torso_node.child = &head_node;
```

左上手臂节点的代码如下：

```
glLoadIdentity();
glTranslatef(-(TORSO_RADIUS+UPPER_ARM_RADIUS),
   0.9*TORSO_HEIGHT, 0.0);
glRotatef(theta[3], 1.0, 0.0, 0.0);
glGetFloatv(GL_MODELVIEW_MATRIX,lua_node.m);
lua_node.f = left_upper_arm;
lua_node.sibling = &rua_node;
lua_node.child = &lla_node;
```

左上手臂相对于躯干的移动位置以及它绕关节旋转的角度确定了左上手臂的变换矩阵。代码中平移函数的参数确定了左上手臂的支点位置。该节点的孩子节点是左下手臂，它右边的兄弟节点是右上手臂。其他节点可以采用类似的方式定义。

这里使用与 10.4 节相同的顺序（先序遍历）遍历这棵树，下面是遍历这棵树的递归代码：

```
void traverse(treenode* root)
{
```

```
  if(root==NULL) return;
  glPushMatrix();
     glMultMatrixf(root->m);
     root->f();
     if(root->child!=NULL) traverse(root->child);
  glPopMatrix();
  if(root->sibling!=NULL) traverse(root->sibling);
}
```

为了绘制一个非空节点，我们首先保存图形系统的状态(glPushMatrix)，然后使用节点的变换矩阵修改模-视变换矩阵（假定已经使用了正确的矩阵模式）。而后就可以使用该节点成员f所指向的函数来绘制该节点对象。最后，我们递归地遍历它的所有孩子节点。注意，因为已经使用了局部变换矩阵与模-视变换矩阵相乘，所以把这个修改后的乘积矩阵传给了它的孩子节点。然而，对于它的兄弟节点，由于它们都有自己的局部矩阵，所以不再使用这个修改后的变换矩阵。因此，访问它的兄弟节点之前，必须使用glPopMatrix函数把模-视变换矩阵恢复到初始状态。如果要在节点内部改变它的属性值，那么可以在绘制函数中执行属性的压栈和出栈操作，也可以在把模-视变换矩阵压栈的同时对属性也执行压栈操作。

这个遍历方法的优点之一就是它与特定的树完全无关。因此，可以使用下面常见的显示回调函数：

```
void display(void)
{
  glClear(GL_COLOR_BUFFER_BIT|GL_DEPTH_BUFFER_BIT);
  glLoadIdentity();
  traverse(&torso_node);
  glutSwapBuffers();
}
```

完整的程序代码可参见附录A。通过这个程序，同样可以绘制机器人的动画效果。动画执行过程中，我们可以使用菜单选择机器人的各个关节，并通过鼠标按键增加或减少关节的旋转角度。因此，程序的动态特性可使用鼠标的回调函数实现，它可以改变关节旋转的角度，重新计算相应的变换矩阵，然后发出重新绘制的命令：

```
void mouse(int btn, int state, int x, int y)
{
  if(btn==GLUT_LEFT_BUTTON && state == GLUT_DOWN)
  {
    theta[angle] += 5.0;
    if( theta[angle] > 360.0 ) theta[angle] -= 360.0;
  }
  if(btn==GLUT_RIGHT_BUTTON && state == GLUT_DOWN)
  {
    theta[angle] -= 5.0;
    if( theta[angle] < 360.0 ) theta[angle] += 360.0;
  }
  glPushMatrix();
  switch(angle)
  {
    case 0 :
      glLoadIdentity();
      glRotatef(theta[0], 0.0, 1.0, 0.0);
      glGetFloatv(GL_MODELVIEW_MATRIX,torso_node.m);
      break;
```

```
/* 其他情况*/
  }
  glPopMatrix();
  glutPostRedisplay();
}
```

还有一个特性可用来说明这种方法的灵活性。当程序执行时，可以增加或删除节点，因此节点是动态的而不是静态的。我们可以使用下面用C语言编写的代码创建动态节点[①]：

```
typedef treenode* tree_ptr;
tree_ptr torso_ptr;
torso_ptr = (treenode*) malloc(sizeof(treenode));
```

节点的定义方法与前面一样，例如：

```
glLoadIdentity();
glTranslatef(-(TORSO_RADIUS+UPPER_ARM_RADIUS),
   0.9*TORSO_HEIGHT, 0.0);
glRotatef(theta[3], 1.0, 0.0, 0.0);
glGetFloatv(GL_MODELVIEW_MATRIX,lua_ptr->m);
lua_ptr->f = left_upper_arm;
lua_ptr->sibling =  rua_ptr;
lua_ptr->child = lla_ptr;
```

遍历函数为：

```
traverse(torso_ptr);
```

对于这个绘制机器人的例子，使用动态方法并没有带来特别的好处。然而，在一个更通用的环境中，可以使用这种方法创建以交互式方式动态变化的结构。例如，可以利用这种方式编写一个应用程序，它允许我们编辑机器人，即根据需要增加或删除某些部件。这样的实现方法正是10.8节将讨论的场景树的基础。彩图 36 显示的是机器人动画的某一帧画面。

注意，当编写好了机器人的绘制代码时，我们发现对层级图形的遍历顺序是固定的。如果使用某种其他的遍历算法，并在层级图形内部改变变换矩阵或属性的值，那么可能会生成效果不同的图像。如果根据代码的层次性，认真考虑每一个节点的属性和变换矩阵的压栈、出栈操作，那么就可以避免这些潜在的问题（虽然频繁使用压栈、出栈操作会影响绘制性能）。

10.6　动画

前面举的两个例子——机器手臂和机器人——都是关节式连接模型：这些模型都包含一些由关节连接起来的刚体部件。只要改变很少的一些参数值就可以使这些模型随时间变化而改变位置，即绘制这些模型的动画效果。层级模型综合考虑了模型各个部件的物理关系，因此可以使用这种层级结构模拟它们的合成运动。我们还没有讨论如何让参数随时间变化以取得所需的动画效果。

在众多的动画绘制方法中，有几种基本的技术对实现关节形体动画特别重要。这些技术既来自传统的手工动画，又来自机器人的研究成果。

对于前面的机器手臂模型，我们考虑的问题是：如何把它上臂的顶端从一个位置移动到另一个位置。这个模型具有3个自由度，即我们能够指定的3个关节旋转角度。虽然每组关节角度可以唯一确定关节顶端的位置，但是反之则不然。假定要把手臂的顶端放在某个特定的位置，也许无法

① 从 10.7 节开始讨论图形对象时，我们将使用 C++ 语法定义这些数据类型。

找到能实现这个目标的一组关节角度或多个关节角度，而单个关节角度可以把它放在某个特定的位置。

如何仅根据关节角度确定模型各部件的位置是动力学（kinematics）的研究范畴。我们可以使用层级建模方法，要么利用数值方法确定模型各部件的位置，要么求出显式方程，通过该方程就可以根据关节角度计算模型上任意点的位置。因此，如果θ是关节角度数组，$\mathbf{p}$是模型的顶点数组，那么动力学模型表示为：

$$\mathbf{p} = f(\theta)$$

同理，如果指定了关节角度的变化率，即关节速度，那么就可以得到模型上任意一点的速度。

动力学模型忽略了惯性和摩擦力等因素。我们可以推导出更复杂的微分方程，它可以描述模型在受到外力作用时的动态行为——这是机器人这个学科研究的课题。

尽管运动学和动力学都是描述模型前向运动行为的方法，然而在动画领域，我们更关心反向运动学（inverse kinematics）和反向动力学（inverse dynamics）：给定模型的某个特定状态，如何调整关节角度才能达到这个位置状态？这里需要解决两个主要问题。第一，假定某个场景包括机器人和其他一些对象，我们必须确定是否存在一组关节角度满足这个状态。或许下面这个反函数的返回值不是唯一的：

$$\theta = g(\mathbf{p})$$

对于某个给定的$\mathbf{p}$，一般来说，我们无法确定是否存在某个θ使$\mathbf{p}$满足这个位置状态，我们也无法确定是否存在多个θ值满足这个方程。即使能够计算出一组关节角度，也必须确保当执行关节动画序列时，这个模型不会撞倒任何障碍物上或违背任何物理约束条件。虽然对于机器人这样的简单模型，我们也许可以得到根据位置计算关节角度的方程，但是通常情况下，我们无法做到这一点，因为前向方程的逆方程一般不是唯一的。机器人模型具有11个自由度，我们能够想象求解这样的问题是多么困难。

克服这些困难的一个基本方法来自于传统的手工动画绘制技术。在关键帧（key-frame animation）动画中，动画师在某些关键的时刻（即关键帧）设置对象的位置。在手工动画绘制过程中，动画师绘制其余的帧，这是一个中间帧插值（in-between）的过程。在计算机图形学领域，可以通过对关键帧之间的关节角度的插值自动实现中间帧插值，或者采用简单的近似方法求出关键帧之间的动态方程。正如第9章讲到的，利用最新GPU的可编程特性，中间帧插值所需的大量运算可以作为绘制流水线的一部分而自动处理。我们也可以利用第12章将介绍的样条曲线绘制平滑的关键帧动画。虽然可以编写代码实现插值，但是如果要自己选择关键帧以及对象在关键帧中的位置，那么手艺精湛的动画师和友好的交互方法都是必不可少的。

10.7　图形对象

虽然已经介绍了多种图形模式，但是我们目前主要基于虚拟照相机模型的流水线绘制图形。之所以选择使用这种图形模型是因为我们希望利用目前已有的硬件和软件支持交互式三维图形应用。因此，我们强调了立即模式的图形绘制系统，即应用程序中定义图元的语句被执行时，这些图元立即被绘制到屏幕上。一旦图元被绘制到屏幕上，除了屏幕上的图像以及保留在帧缓存中的图像内容，这些图元不会留下任何记录。如果要改变图元的属性或位置信息，则必须在应用程序中重新绘制图元。

另外，我们希望介绍实现图形绘制的基本原理，这种想法导致了我们在开发图形程序时总要考虑底层的实现细节。尽管这种做法有许多优点，但这使我们未能使用其他高级的图形应用开发技术。

现在我们转向更抽象的方法并介绍两个重要的概念。首先，对多边形和矢量等几何对象概念进行扩展，使它包含图形程序中的大多数对象，如观察者、灯光和材质属性。其次，我们把注意力集中于那些图像绘制出来之后仍然存在的对象，以及那些我们甚至从来不去绘制的对象。我们已经知道，可以使用第 3 章介绍的显示列表创建这些对象。这里将讨论其他的方法，如使用 C++ 的类或 C 语言中的结构体。尽管 OpenGL API 并不直接支持这种方法，但是不必因此放弃使用 OpenGL。我们仍然使用 OpenGL 绘制图形，而将所开发的程序作为 OpenGL 之上更抽象的软件层。

10.7.1　方法、属性和消息

我们编写的程序可以处理数据。数据的形式是多样的，从数字、字符串到应用程序中建立的几何实体。在传统的命令式程序设计范型中，程序员编写代码来处理数据，这里的代码通常指函数。数据通过函数的参数传递给函数，处理完后再以类似的方式返回。为了处理传递给它的数据，函数必须知道这些数据的组织形式。例如，考虑前面的许多示例使用过的立方体模型。我们知道立方体可以使用多种不同的建模方法，包括顶点指针、边表和多边形顶点表。应用程序开发人员可能并不关心立方体的建模方法，而更愿意把立方体看做是一个原子实体或对象。此外，他也不关心立方体是如何绘制到屏幕上的细节内容：比如它使用了哪个明暗处理模型或使用了何种多边形区域填充算法。他可以做这样的假设，即立方体“知道如何绘制自己”，从概念上说，就是把绘制算法绑定到对象上。从某种意义来说，OpenGL 是支持这种观点的，因为它使用图形系统的状态控制绘制过程。例如，立方体表面的颜色、方位和照射到其表面的灯光都可以称为图形系统状态的一部分，它们与立方体使用的建模方法没有关系。

然而，如果需要处理一个物理立方体，那么会发现这种观点有点让人感到奇怪。物理立方体的位置以及它的颜色、大小和方位都绑定在物理对象上。尽管可以利用 OpenGL 把某些属性绑定到一个虚拟对象上，采用的方法是使用属性值和变换矩阵的压栈、出栈操作，但是底层的编程模型不能很好地支持这些设想。例如，立方体的几何变换函数必须事先知道该立方体的表示方式，图 10.16 说明了命令式程序设计范型。

应用程序开发人员需要编写一个函数，它的输入参数分别指向立方体数据的指针和变换矩阵。函数处理完数据后将控制权移交给应用程序（函数也可能返回一些值）。

面向对象的设计方法和面向对象的编程思想使用一种完全不同的方式处理对象。即使在面向对象的早期，如 Smalltalk 语言，也认为计算机图形学为面向对象这种功能强大的方法提供了很好的例证。软件领域最近的发展趋势也表明可以把面向对象的设计方法与图形的绘制流水线结合起来构建更具表达力、功能更强的图形系统。

面向对象的编程语言把对象定义为模块，并使用这种模块建立程序。这些模块包含用来定义模块所需的数据，如立方体的顶点、模块的特性（属性）以及处理模块数据及其属性的函数（方法）。我们通过给对象发送消息的方式调用对象的方法。这种面向对象的程序设计范型如图 10.17 所示。

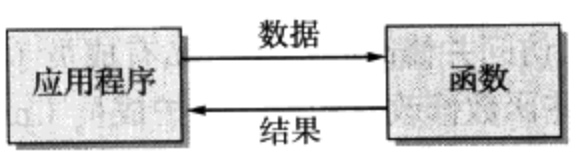


图 10.16　命令式程序设计范型

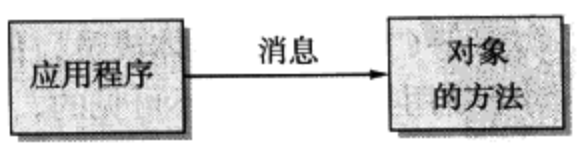


图 10.17　面向对象程序设计范型

采用面向对象的编程范型给应用程序开发人员带来的好处是：他现在不必知道立方体的表示方法，而只需知道立方体支持哪些功能——他可以给立方体发送什么消息。

虽然C语言的结构体类型具有对象的某些性质，但是C语言不完全支持面向对象的方法。在C++语言中，类取代了结构体。C++的类具有两种重要的属性，可以通过对这两个属性的使用认识到面向对象方法的优越性。C语言程序员理解这些面向对象的概念应该不会有困难。

10.7.2　一个立方体对象

假定希望使用C语言创建一个立方体对象，该对象附有一个颜色属性和一个齐次坐标变换矩阵。在C语言中，可以使用以下形式的结构体：

```
struct  cube
{
   float color[3];
   float matrix[4][4];

/* 从这里开始编写实现过程的代码 */

}
```

结构体中的实现部分包含如何实际表示立方体的信息。一般来说，应用程序开发人员并不需要这些信息，而只需改变立方体的颜色和变换矩阵。

一旦定义好了结构体，就可以创建立方体对象的一个实例，它和其他基本数据类型一样：

```
cube a, b;
```

立方体对象的每个实例可以改变其对应的结构体属性成员值。因此，可以通过下面的代码把立方体的颜色值设置为红色：

```
a.color[0]=1.0;
a.color[1]=a.color[2]=0.0;
```

现在读者应该知道这种结构体是如何在OpenGL系统中实现的。例如，在立方体的实现代码中可能看到下面一行语句：

```
glColor3fv(color);
```

虽然我们创建了一个可重用的立方体对象，但在如何处理它或绘制它等方面却受到限制。我们可以编写一个绘制立方体的函数，代码如下：

```
render_cube(a);
```

或者调用下面的函数旋转立方体：

```
rotate_cube(a, theta, d);
```

其中，d是旋转轴方向的矢量。

虽然这种方法是可行的，但是有很多的限制。首先，对于每类对象，需要单独定义它的绘制和旋转函数。其次，应用程序可以访问代码的实现部分。C++中的类解决了这两个问题。C++中的类可以包含公有（public）成员、私有（private）成员或受保护的（protected）成员。公有成员（public member）类似于C语言的结构体成员，任何函数都可以访问并修改它的值。私有成员（private member）对于使用类的函数是不可见的，并且不允许这些函数修改它的值。保护成员（protected member）对于位于同一层次的类是可见的。为了访问某些特定的类，程序员也可以把某些类声明为友元（friend）。因此，使用C++语言可以这样定义立方体：

```
class  cube
{
 public:
   float color[3];
   float matrix[4][4];
 private:

/* 从这里开始编写实现过程的代码 */

}
```

因此，这样定义立方体类可以保护和隐藏实现细节。此外，我们可以在C++类中定义成员函数或方法。如果在类中增加了成员函数，那么面向对象的思想就更加清晰了。假设在立方体类的公有成员部分增加下面的成员函数：

```
void render();
void translate(float x, float y, float z);
void rotate(float theta, float axis_x, float axis_y,
            float axis_z);
```

现在一个应用程序就可以创建、平移、旋转或绘制一个立方体，代码如下：

```
cube a;
a.rotate(45.0, 1.0, 0.0, 0.0);
a.translate(1.0, 2.0, 3.0);
a.render();
```

从概念上讲，这段代码假定立方体实例“知道”如何旋转自己，通过执行代码 a. rotate 给立方体对象发送消息，要求立方体执行旋转操作。通过正确调用OpenGL里的几何变换函数，就能很容易地编写立方体类中旋转和平移方法的实现代码。

我们并不清楚像a. render这样的函数到底做了什么，或者旋转函数与图形系统的状态之间的关系。有一种方法可以理解使用面向对象的方法所带来的变化，也就是注意到，我们不能再使用立即模式的图形系统。每个对象实例在程序运行期间将一直存在，并且可以在程序中添加代码修改它的属性值。由于对象实例被绘制到屏幕上后OpenGL的显示列表可以继续存在，所以使用显示列表来实现对象实例也许是一种非常有用的方法，但可能还有其他的实现方法。尤为重要的是：我们创建的对象以某种对应用程序不可见的形式一直存在于系统的某个地方。对象的属性仍然在系统的内部存在，并且可以使用 rotate 这样的函数修改这些属性。render 函数根据对象的状态而不是系统的当前状态使对象得到重新绘制。在 OpenGL 设置中，绘制过程可能包含类似下面的执行代码：

```
glBegin(GL_POLYGON);
    glVertex3f(...);
    .
    .
    .
    glVertex3f(...);
glEnd();
```

其中，代码所需的顶点信息来自结构体中的实现部分，或者来自实例对应的显示列表的重新绘制。

10.7.3　实现立方体对象

下面以立方体对象的实现为例详细介绍如何开发类的私有成员部分。这里只介绍 OpenGL 中多边形（也许是四边形）的简单实现过程。因此，我们以下面的顶点实现为基础来实现多边形的绘制：

```
GLfloat vertices[6][3]

glBeginGL_POLYGON; /* 开始绘制第一个面 */
   glVertex3fv(vertices[0]);
   glVertex3fv(vertices[1]);
   glVertex3fv(vertices[2]);
   glVertex3fv(vertices[3]);
glEnd(); /* 结束绘制第一个面 */
   /* 绘制其余的5个面 */
```

但是正如第4章所讲到的，使用这种方法实现多边形的绘制需要更多不必要的函数调用。我们可以使用自定义的顶点数组或OpenGL内置的顶点数组，也可以根据对象的使用方式定义其他更有效的数据结构。

如果在类的实现部分包含更多的信息，那么可能会获得更好的效果。其中我们感兴趣的是那些对绘制过程有帮助的信息。因此，虽然OpenGL可以通过z-buffer算法执行正确的隐藏面消除，但是如果在绘制流水线的早期阶段利用独立的可见性裁剪算法将那些不可见的对象裁剪掉，那么系统的绘制效率会更高，10.10节将讨论可见性裁剪问题。为了支持这个功能，需要在类中包含对象的包围盒信息。例如，可以在类的私有成员部分增加对象的包围盒信息，它与坐标轴对齐。对于多边形对象，只需要保存顶点x，y，z的最小值和最大值（这里的顶点坐标是指多边形经对象中存储的变换矩阵变换后的顶点坐标）[①]。

10.7.4　对象与层级结构

面向对象设计方法的一个主要优点就是代码重用，以及从少量简单对象构建更复杂的对象的能力。如10.4节所介绍的，我们使用一些圆柱体就可以构建一个机器人，并可以构建机器人对象的多个实例，每个实例都有自己的颜色、大小、位置和方位。拟人机器人类可以引用手臂类和腿类，汽车类可以引用轮子类和车身底盘类，因此，我们又得到了这种类似于10.5节介绍的树形表示结构。

在面向对象的设计方法中，通常需要表示那些比树的双亲-孩子表示关系更复杂的关系。正如我们使用过的树形结构，最复杂的对象位于树的根部，双亲与孩子之间的关系是一种“has-a”（拥有）关系。因此，杆形机器人拥有两只手臂和两条腿，而汽车拥有4个轮子和一个车身底盘。

我们也可以用一种不同的方式来理解这种层级结构，即简单的对象位于层级结构的顶端，双亲与孩子之间的关系是一种“is-a”（属于）关系。这种层级结构是一种典型的分类法。例如，哺乳动物属于动物，而人属于哺乳动物。我们曾使用这种关系对投影方法进行分类。例如，平行投影属于平面几何投影，斜投影属于平行投影。使用“is-a”关系，可以从简单对象定义多个复杂对象，并允许复杂对象继承简单对象的属性。因此，如果编写好了实现平行投影的代码，那么实现斜投影的代码可以使用平行投影的代码，并只需重新定义从一般的平行投影转化为斜投影所需的数据。对于几何对象，可以定义基对象，它具有一组默认的属性，如颜色和材质等。然后应用程序开发人员就可以在子对象中使用这些属性，或者在子对象中修改这些属性。C++等面向对象的编程语言支持这些概念，它们提供了子类和继承等语言特性。

10.7.5　几何对象

假定现在希望建立一个面向对象的图形系统。该系统应该包括哪些对象呢？很明显，虽然我们需要使用点、矢量、多边形、矩形和三角形（可能需要使用子类）等对象，但是我们并不清楚如何

① 正如第9章介绍的，也可以把包围盒信息传给可编程着色器。

处理属性、光源和观察者。例如，材质属性作为立方体等对象的一部分，还是作为单独的对象？答案是其中之一或两者都是。我们可以建立一个立方体类，它包含第6章介绍的Phong模型中的环境光、漫反射光和镜面反射光等成员属性。我们还可以单独定义材质类，其代码如下：

```
class material
{
 public:
   float specular[3];
   float shininess;
   float diffuse[3];
   float ambient[3];
}
```

然后就可以通过立方体类的成员函数把材质赋给几何对象：

```
cube a;
material b;
a.setMaterial(b);
```

光源也是几何对象——光源的属性包括位置和方位——我们可以很方便地增加一个光源对象：

```
class light
{
  public:
     boolean type;
     boolean near;
     float position[3];
     float orientation[3];
     float specular[3];
     float diffuse[3];
     float ambient[3];
}
```

一旦建立了一组几何对象，就可以使用它们来描述场景。为了充分利用前面介绍的层级关系的优越性，下面介绍一种称为场景图（scene graph）的新树形结构。

10.8　场景图

如果考虑使用哪些对象描述场景图，那么我们发现，除了图元以及由图元构成的几何对象之外，还需要其他对象，如灯光和虚拟照相机等。我们也可以使用顶点和矢量来定义这些对象，它们与几何图元类似，也具有颜色等属性。正是所有这些对象描述了一个场景，而且这些对象之间存在层级关系。例如，在程序中定义一个图元时，就可以利用当前虚拟照相机的参数来绘制对应的图像。如果在两个几何对象的定义之间改变了虚拟照相机的镜头，那么在生成的图像中，这两个几何对象看起来是从不同的视点观察到的。然而我们使用真实的照相机却无法得到这样的图像，这个例子说明了图形系统的强大功能。我们可以把前面介绍的树结构扩展为场景图，并用它来描述几何对象、虚拟照相机、光照和属性之间的关系。

因为我们可以编写一个图形应用程序遍历一个图，所以可以扩展图的节点内容，使它能描述整个场景的对象。一种可能的方法是使用树形数据结构，树上的每个节点除了具有实例矩阵和一个指向绘制函数的指针之外，还包含各种属性信息。另一种可能的方法是定义新的节点类型，如属性定义节点和矩阵变换节点。分析图 10.18 所示的树形结构。我们为颜色和模－视变换矩阵定义了专门的节点。位于上方有分支的地方是一个特殊节点，称为组节点（group node），它的作用是把两个孩子节点隔离开。我们可以使用 group 节点保存进入节点时的状态，因此可以把位于 group 节点下面

的子树与其他的树隔离开来。如果使用前面介绍的先序遍历算法，那么对应的OpenGL代码的形式如下：

```
glPushAttrib
glPushMatrix
   glColor
   glTranslate
   glRotate
   object1
   glTranslate
   object2
glPopMatrix
glPushMatrix
   glTranslate
   glRotate
   object3
glPopMatrix
glPopAttrib
```

group节点相当于OpenGL的压栈和出栈函数。上面的代码在结束之前保存并恢复了属性和模－视变换矩阵。代码设置了图形的绘制颜色并将颜色应用于树的其余节点，遍历树的方式类似于前面介绍的机器人示例。

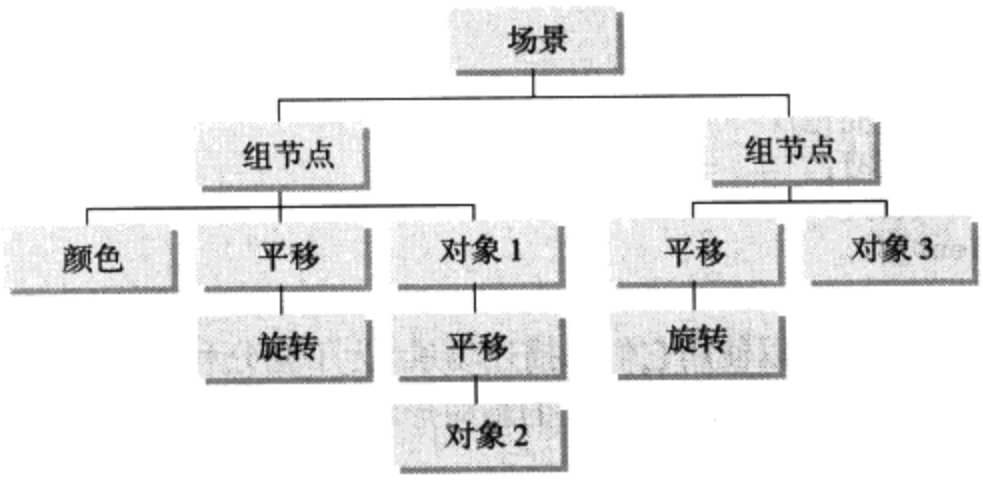


图10.18　场景树

我们发现可以进一步使用属性堆栈和矩阵堆栈来存储观察参数。因此，我们可以在场景树上建立虚拟照相机的节点。虽然可能不希望场景里的每个对象都使用不同的照相机观察，但是我们也许需要使用多个照相机观察同一组对象构成的场景。例如，建筑师和工程师经常使用不同视点下的正投影视图和正等测视图。这样的图像可以使用具有多个照相机的场景图来生成。

我们可以使用树形结构以完全机械的方式生成一个程序，从这种意义上说，刚刚介绍的场景图等价于一个OpenGL程序。Open Inventor及其后来出现的开放场景图（Open Scene Graph，OSG）都采用这种方法来建立图形应用程序，这里的Open Inventor和OSG都是建立在OpenGL之上的面向对象的图形API。Open Inventor和OSG程序用来建立、处理和绘制场景图，程序的执行导致对场景图的遍历，场景图的遍历又会调用OpenGL实现的图形函数。

场景图的概念与10.7节介绍的面向对象的编程范型能够很好地对应起来。我们把所有的图元、属性、变换视为软件对象，并可以定义用来处理这些实体的类。从这个观点来看，我们可以充分利用面向对象技术中的数据封装等概念用简单的程序构建很复杂的场景，这些程序可以使用预定义好的软件对象。我们甚至可以通过场景图上以节点形式存在的软件对象来支持动画的绘制，这些软件

对象可以通过改变节点的参数来重新绘制场景。虽然在Open Inventor中，我们仍然使用OpenGL绘制软件对象，但是场景图本身是一个包括场景中所有元素的数据库。OpenGL在这里只是作为绘制引擎，通过对场景图的遍历，它可以把数据库绘制成所需的图像，而我们并没有把OpenGL用于描述场景。计算机游戏使用了与之类似的方法，这样游戏玩家可以修改场景图，并实时地遍历和绘制场景图。

图 10.19 说明了一种现代的图形系统结构，它是图形软件系统经演化而来的。其中，OpenGL是绘制引擎，它通常位于另一层称为硬件抽象层（hardware abstraction layer，HAL）的上面，是用于与底层的物理硬件通信的虚拟机。位于OpenGL上面的是面向对象层，它支持场景图和存储机制。根据应用程序的功能需求，用户可以对任意层编写应用程序。

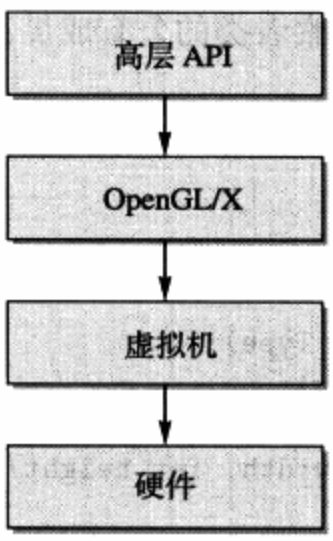


图 10.19　现代的图形系统结构

10.9　一个简单的场景图 API

虽然目前已经存在一些标准的场景图 API，包括虚拟现实建模语言（VRML）、Java3D 和开放场景图（OSG），但是没有一个占主导地位[①]。这里并不讨论这些 API，而是自己构建一个简单的、却可以扩展的场景图 API。我们所设计的这个 API 包含许多复杂场景图 API 共有的基本要素，而且具有易于扩展的特性。我们从公共接口开始设计，用户可以使用这些接口来定义并显示场景图。然后使用 10.5 节介绍的左孩子、右兄弟数据结构介绍一种可能的实现方法。

设计使用了两个基类[②]：一个节点类和一个观察者类（视点类）。我们使用节点类构建场景中的所有节点，包括几何对象、虚拟照相机对象、材质对象和光源对象。然后可以使用这些节点构建一棵树，即场景图。观察者对象为我们利用 OpenGL 程序观察场景图提供了一种简单的机制。

10.9.1　节点类

节点基类的公共接口定义为：

```
class Node
{
 public:
  Node();
  virtual ~Node();
  virtual void Render();
  void AddChild(Node *);
```

① 如果读者熟悉面向对象的编程方法而又不想亲自动手实现自己的场景图，那么可以跳过这节。

② 这个场景图 API 及其实现过程来自新墨西哥州大学 Ye Cong 的课程设计。

```
    friend class GLViewer;

    // 在这里定义类的私有成员
};
```

为了创建动态场景图，该节点类包括构造函数和析构函数，用于创建和删除节点。可以使用节点类的AddChild方法给节点添加孩子节点，从而构建层级树形结构。然而，当后面讨论实现时，我们会发现添加孩子节点的顺序会影响场景图的绘制结果。

由于我们可能会用到许多类型的节点（几何、照相机、材质、光源），所以每个节点必须拥有自己的绘制方法，当遍历场景图访问某个节点时，需要执行该节点对应的绘制方法。

我们可以让观察者调用绘制过程，还可以在观察者类中利用GLUT库处理OpenGL与特定的窗口操作系统之间的交互功能。下面是观察者类的公有成员：

```
class GLViewer
{
public:
  GLViewer();
  ~GLViewer();

  void SetValue(Enum PName, Enum Type);
  void Init(int argc, char **argv);
  void Show(Node *N);
  int CreateWin(char *name, int width, int height);

  // 在这里定义类的私有成员
};
```

我们可以有多个观察者，因此能够在两个OpenGL窗口分别使用不同的照相机绘制两个不同的视图。我们还可以把不同的节点传送给Show方法，这样可以使用不同的OpenGL窗口观察同一场景的不同部分。Show方法初始化为对整个场景图的遍历。

观察者类和节点类都有许多参数，这些参数用于定义对象的属性（如颜色、线型、材质等）和绘制选项（如双缓存和隐藏面消除等）。我们采用类似于OpenGL和GLUT的方法，即把符号常量定义为枚举类型：

```
enum Enum
{
  PERSPECTIVE, ORTHO, POSITION, AIMAT, UPDIRECTION, ASPECT,
  NEAR, FAR, YANGLE, BLACK, WHITE, RED, GREEN, YELLOW, BLUE,
  MAGENTA, CYAN, GRAY, WIDTH, HEIGHT,DEPTH,
  AMBIENT, DIFFUSE, SPECULAR, SPOT_DIRECTION, DROPOFFRATE,
  CUTOFFANGLE, EMISSION, SHININESS, TRANSLATION, ROTATION, SCALE,
  BUFFER, SINGLE, DOUBLE, RADIUS, STYLE, POINTSIZE, LINEWIDTH,
  FILLED, LINE, POINT, BACKCOLOR
};
```

下面是一个典型的应用程序中的代码片断，它在屏幕上建立一个窗口并绘制场景图：

```
GLViewer *MyViewer=new GLViewer;

MyViewer->Init(argc, argv);
MyViewer->SetValue(BACKCOLOR, GRAY);
MyViewer->SetValue(BUFFER, DOUBLE);
MyViewer->CreateWin("Working Hard", 500, 500);

MyViewer->Show(Root);
```

节点基类有 5 个主要的子类：几何节点、光源节点、照相机节点、属性节点和变换节点。从 OpenGL 的角度来看，有两类操作：生成图元的操作和改变状态的操作。然而，从场景图的角度来看，这两类操作都要在节点内部初始化。

10.9.2　几何节点

我们处理的场景图既要考虑基本的对象，如线段和多边形，又要考虑更复杂的对象，如立方体、圆柱体、三角形和球体等。几何节点有许多公共属性，包括颜色属性、材质属性以及作用于节点的实例矩阵。我们可以在应用程序中设置这些属性，可以在几何节点类中定义用来设置这些属性的方法：

```
class Geometry: public Node
{
 public:
  Geometry();
  ~Geometry();

  void SetColor(Enum icolor);
  void SetColor(float red, float green, float blue);
  void SetColorv(float *color);
  void SetColor(Color *colorp);
  void SetMaterial(Enum type, float red, float green, float blue,
                   float alpha);
  void SetMaterialv(Enum type, float *materialv);
  void SetMaterial(Enum type, float materialv);
  void SetMaterial(Material *material);
  void SetTransform(Enum, float *transform, int order);
  void SetTransform(Enum type, float xparam, float yparam, float zparam,
                    int order);
  void SetTransform(Enum type, float angle, float xparam, float yparam,
                    float zparam, int order);
  void SetTransform(Transformation *transform);
  void SetStyle(Enum pname, Enum eparam);
  void SetStyle(Enum pname, float fparam);
  void SetStyle(DrawStyle *stylev);
  virtual void Render();

 private:
   // 实现部分
}
```

注意，用户可以直接使用几何节点中的属性设置方法来设置几何对象的属性，如颜色属性。不同的属性设置方法有不同的参数，下面是具有 3 个参数的颜色属性设置方法：

```
myObject->SetColor(red, green , blue);
```

或者用户也可以先定义一个属性对象，然后把这个属性对象连接到一个或多个几何节点中，如下面所示的代码：

```
Color *myColor = new Color;
myColor->SetValue(red, green, blue);
myObject->SetColor(myColor);
```

现在可以讨论一些具体的几何节点。最简单的几何节点是线段，我们可以使用线段节点直接设置线段的两个端点，或者利用一个指针来设置线段，该指针指向包含两个顶点的顶点数组。注意，

在这个简单的系统中只包含三维顶点。作为一个简单的练习，读者可以增加二维顶点。另外还要注意，由于C++语言允许函数的重载，所以对于各种情形我们只使用一个函数，即SetVertices函数：

```
class Line: public Geometry
{
 public:
  Line(){};
  void SetVertices(float *vert0, float *vert1);
  void SetVerticesv(float v[][3]);
  void Render();
 private:
   // 实现部分
};
```

另外两个简单的几何对象分别是中心位于原点的球体以及以y轴为旋转轴、底面在xoz平面上的圆柱体：

```
class Sphere: public Geometry
{
 public:
  Sphere(){};
  Sphere(float R);
  void SetValue(Enum Pname, float param);
  void Render();
 private:
   // 实现部分
};
```

```
class Cylinder: public Geometry
{
public:
 Cylinder(){};
 void SetValue(Enum Pname, float param);
 void Render();
private:
  // 实现部分
};
```

还可以考虑一般形式的多边形，类的定义如下：

```
class Polygon: public Geometry
{
 public:
  Polygon(){};
  void SetVerticesv(float v[][3], int);
  void Render();
 private:
   // 实现部分
};
```

10.9.3　照相机类

因为照相机的大部分参数可以通过相应的设置函数来指定，所以照相机类的接口非常简单。照相机类构造函数的参数决定了其绘制的是透视投影图形还是正交投影图形。可以使用设置函数来设置照相机的方位，设置函数的参数与OpenGL的gluLookAt函数[①]的参数完全一样：

① 原文为Lookat函数，可能有误。——译者注

```
class Camera: public Node
{
 public:
  Camera(Enum CType);

  void Render();
  void SetValuev(Enum PName, float *v);
  void SetValue(Enum PName, float v1, float v2, float v3);
  void SetValue(Enum PName, float v);
 private:
   // 实现部分
};
```

下面是一个定义透视投影照相机的例子：

```
Camera *Camera1=new Camera(PERSPECTIVE);

Camera1->SetValue(POSITION, 2.2, 0.9, 3);
Camera1->SetValue(AIMAT, 0, 0, 0);
Camera1->SetValue(UPDIRECTION, 0, 1, 0);
Camera1->SetValue(ASPECT, 1);
Camera1->SetValue(NEAR, 1);
Camera1->SetValue(FAR, 20);
Camera1->SetValue(YANGLE, 50);
```

从OpenGL的观点来看，上面有些参数会影响模-视变换矩阵，而其他一些参数会影响投影变换矩阵。从面向对象观点来看，这些参数只不过定义了一个虚拟照相机。

10.9.4　光源和材质

光源和材质具有简单的公有接口，可以用来设置改进后Phong模型的标准参数。因为在遍历场景图时，场景图可能会改变状态变量，所以我们希望在场景图中能够打开和关闭光源。可以通过定义一个开关对象来解决这个问题：

```
class Light: public Node
{
 public:
  Light();
  void Render();

  void SetValue(Enum PName, Enum color);
  void SetValue(Enum PName, float v1, float v2, float v3, float v4 );
  void SetValue(Enum PName, float v1, float v2, float v3);
  void SetValue(Enum PName, float f);
  void SetValuev(Enum PName, float *);
  void TurnOn();
  void TurnOff();

  friend class TurnOff;

 private:
   // 实现部分
};
```

```
class TurnOff: public Node
{
 public:
  TurnOff(Light *L);
  void Render();
 private:
   // 实现部分
};
```

因此，如果希望设置一个光源并能够控制其开关，那么必须同时创建光源和开关对象，代码如下：

```
Light *Light1=new Light;
TurnOff *Off1=new TurnOff(Light1);

Light1->SetValue(POSITION, -2, -3, 1.5, 1);
Light1->SetValue(SPOT_DIRECTION, 2, 3, -1.5);
Light1->SetValue(CUTOFFANGLE, 40.0);
Light1->TurnOn();
```

材质节点类似于光源节点，包含一个绘制方法和一些用来设置标准材质参数的参数设置方法：

```
class Material: public Node
{
 public:
  Material();
  void SetValuev(Enum Pname, float *);
  void SetValue(Enum Pname, float v1, float v2, float v3, float v4);
  void SetValue(Enum Pname, float Value);
  void Render();
 private:
   // 实现部分
};
```

因此，可以这样定义材质节点：

```
Material *myMat = new Material;

myMat->SetValue(DIFFUSE, 0.0, 0.0, 1.0, 1.0);
myMat->SetValue(AMBIENT, 0.0, 0.0, 1.0, 1.0);
myMat->SetValue(SPECULAR, 1.0, 1.0, 1.0, 1.0);
myMat->SetValue(SHININESS, 100.0);
```

然后把它赋给几何对象：

```
myObject->SetMaterial(myMat);
```

10.9.5　几何变换类

几何变换类包括标准的旋转、平移和缩放变换矩阵，它们的参数可以使用SetValue方法设置。可以使用SetValue方法中的Order参数定义变换矩阵序列及其相乘的顺序：

```
class Transformation: public Node
{
 public:
  Transformation();
  void SetValuev(Enum Pname, float *, int Order);
  void SetValue(Enum Pname, float x, float y, float z, int Order);
```

```
    void SetValue(Enum Pname, float a, float x, float y, float z,
                  int Order);
    void Render();
  private:
    // 实现部分
};
```

对于任意一个几何节点，可以使用它的成员方法SetTransform把一个几何变换实例绑定到这个对象上，代码如下：

```
Cylinder myCylinder = new Cylinder;
myCylinder->SetTransformation(SCALE, sx, sy, sz, 0);
myCylinder->SetTransformation(TRANSLATION, tx, ty, tz, 2);
myCylinder->SetTransformation(ROTATION, theta, rx, ry, rz, 1);
```

因此在上面的这个例子中，由 Order 参数决定了计算的顺序是先缩放，然后旋转，最后再平移。

10.9.6　机器人的绘制程序

在讨论具体实现之前，我们再次分析机器人这个简单的例子。附录 A 给出了完整的代码：

```
#include "Scene.h"

int main(int argc, char **argv)

{

  // 定义一个光源

Light *Light1=new Light;
TurnOff *Off1=new TurnOff(Light1);

// 在此设置光源的参数
// 定义一个照相机对象

Camera *Camera1=new Camera(PERSPECTIVE);

// 设置照相机参数

// 定义材质对象

Material *RobotMat=new Material;

// 设置材质参数

RobotMat->SetValue(DIFFUSE, 0.0, 0.0, 1.0, 1.0);
RobotMat->SetValue(AMBIENT, 0.0, 0.0, 1.0, 1.0);
RobotMat->SetValue(SPECULAR, 1.0, 1.0, 1.0, 1.0);
RobotMat->SetValue(SHININESS, 100.0);

// 定义其他的材质

// 定义机器人几何节点

Sphere *Head=new Sphere;

Cylinder *Torso = new Cylinder;
```

```
Cylinder *UpperArmL=new Cylinder;
Cylinder *UpperArmR=new Cylinder;
Cylinder *LowerArmL=new Cylinder;
Cylinder *LowerArmR=new Cylinder;
Cylinder *UpperLegL=new Cylinder;
Cylinder *UpperLegR=new Cylinder;
Cylinder *LowerLegL=new Cylinder;
Cylinder *LowerLegR=new Cylinder;

Transformation *HeadTrans=new Transformation;
Transformation *UpArmLTrans=new Transformation;

// 左上臂的几何变换

HeadTrans->SetValue(TRANSLATION, 0, 0,
   BaseLen+BaseRadius+BaseRadius/3, 0);
UpArmLTrans->SetValue(TRANSLATION, 0, 0, -UpLen, 0);
UpArmLTrans->SetValue(ROTATION, -45, 0, 1, 0, 1);
UpArmLTrans->SetValue(TRANSLATION, 0, BaseRadius+Radius, BaseLen, 2);

// 设置其他的变换矩阵

UpperArmL->SetTransform(UpArmLTrans);
LowerArmL->SetTransform(LowArmTrans);
// 把几何变换实例赋给相应的节点

// 设置机器人各部件之间的关系
RobotMat->AddChild(Light1);
Light1->AddChild(Base);

Torso->AddChild(Off1);

Torso->AddChild(UpperArmL);
UpperArmL->AddChild(LowerArmL);
Torso->AddChild(UpperArmR);
UpperArmR->AddChild(LowerArmR);
Torso->AddChild(UpperLegL);
UpperLegL->AddChild(LowerLegL);
Torso->AddChild(UpperLegR);
UpperLegR->AddChild(LowerLegR);

// 定义机器人的几何变换节点

Transformation *Trans1=new Transformation;

Trans1->SetValue(TRANSLATION, -0.5, 0, 0, 2);

Trans1->AddChild(RobotMat);

// 根节点
Node *Root=new Node;

Root->AddChild(Trans1);
Root->AddChild(Camera1);

// 定义观察者对象
```

```
    GLViewer *MyViewer=new GLViewer;

    // 设置观察者参数

    // 绘制过程

    MyViewer->Show(Root);

    return 0;
}
```

10.9.7　实现观察者类

首先讨论观察者类的实现。它首先初始化遍历过程，而且必须与窗口操作系统进行交互。这里使用GLUT库编写程序界面，这使得观察者类与已经实现的类相兼容。下面是观察者类的定义，该定义包括类的私有成员和实现方法：

```
class GLViewer
{
 public:
  GLViewer();
  ~GLViewer();

  void SetValue(Enum PName, Enum Type);
  void Init(int argc, char **argv);
  void Show(Node *N);
  void CreateWin(char* name, int width, int height);

 private:
  void GLInit();
  int ViewerIndex;
  char *WinName;
  float BackColor[3];

  static Node *Root[3];
  static int ViewerNum;
  static int BufType[3];
  static int WinWidth[3];
  static int WinHeight[3];
  static void Reshape0(int w, int h);
  static void Display0();
  static void Reshape1(int w, int h);
  static void Display1();
  static void Reshape2(int w, int h);
  static void Display2();

};
```

我们实现的这个观察者类能够建立多达3个独立的观察者。这些观察者的信息以静态成员的形式存储。每个观察者可以有自己的显示回调函数（glutDisplayFunc）和窗口调整回调函数（glutReshapeFunc）。因为使用了GLUT库，所以必须使用一些技巧才能实现多观察者功能。例如，必须确保只有一个观察者调用glutMainLoop函数。Root数组存储了每个观察者的根节点。观察者类的构造函数建立一个默认的窗口，代码如下：

```
GLViewer::GLViewer()
{
  int i;

  ViewerIndex=ViewerNum;
  ViewerNum++;

  WinName="";
  BufType[ViewerIndex]=GLUT_DOUBLE;
  Root[ViewerIndex]=NULL;
  for(i=0; i<3; i++)
    BackColor[i]=0.0;
}
```

可以用CreateWin方法创建一个非默认窗口，并且用SetValue方法设置其参数，代码如下：

```
GLViewer::~GLViewer()
{
  if(Root[ViewerIndex])
    {
      delete Root[ViewerIndex];
      Root[ViewerIndex]=NULL;
    }
}

void GLViewer::Init(int argc, char **argv)
{
  if(ViewerIndex==0)
    glutInit(&argc, argv);
}

void GLViewer::CreateWin(char *Name, int Width, int Height)
{
  WinName=Name;
  WinWidth[ViewerIndex]=Width;
  WinHeight[ViewerIndex]=Height;

}

void GLViewer::SetValue(Enum PName, Enum Type)
{
  switch(PName)
    {
    case BUFFER:
      if(Type==DOUBLE)
    BufType[ViewerIndex]=GLUT_DOUBLE;
      else if(Type==SINGLE)
    BufType[ViewerIndex]=GLUT_SINGLE;
      break;
    case BACKCOLOR:
      Root[ViewerIndex]->GetColor(Type, BackColor);
      break;
    default:
      break;
    }
}
```

因为使用了OpenGL和GLUT库来实现绘制过程，所以遍历过程必须作为显示回调函数的一部分。因此，每个观察者都有一个显示回调函数，它从观察者的根节点开始遍历。这里所用的遍历方法就是节点类中的遍历方法：

```
void GLViewer::Show(Node *N)
{
  GLInit();
  Root[ViewerIndex]=N;
  if(ViewerIndex==(ViewerNum-1))
    glutMainLoop();
}

void GLViewer::GLInit()
{
  glutInitDisplayMode(BufType[ViewerIndex] | GLUT_RGB | GLUT_DEPTH);
  glutInitWindowSize(WinWidth[ViewerIndex], WinHeight[ViewerIndex]);
  glutCreateWindow(WinName);
  switch(ViewerIndex)
    {
    case 0:
      glutReshapeFunc(Reshape0);
      glutDisplayFunc(Display0);
      break;
    case 1:
      glutReshapeFunc(Reshape1);
      glutDisplayFunc(Display1);
      break;
    case 2:
      glutReshapeFunc(Reshape2);
      glutDisplayFunc(Display2);
      break;
    default:
      break;
    }

  glEnable(GL_DEPTH_TEST);
  glClearColor(BackColor[0], BackColor[1], BackColor[2], 1.0);
}

void GLViewer::Display0()
{
  glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
  Root[0]->Traverse();
  if(BufType[0]==GLUT_DOUBLE)
    glutSwapBuffers();
  glFlush();
}
void GLViewer::Reshape0(int w, int h)
{
  glViewport(0, 0, (GLsizei)w, (GLsizei)h);
  WinWidth[0]=w;
  WinHeight[0]=h;
}

// 此处是其他一些类似的显示函数和窗口大小调整函数
```

10.9.8　实现节点类

节点类的私有部分包含建立基于左兄弟、右孩子表示方法的树形结构。我们还在节点类的私有部分包含了树的遍历方法，该方法类似于前面的机器人遍历方法，但这里可以使用任何标准的遍历算法。

节点类还有一个保护成员KeepMatrix，它用于把当前的模-视变换矩阵和投影变换矩阵压入堆栈中。KeepMatrix是一个布尔变量，可以使用这个变量来确定是否把当前的矩阵传给孩子节点，这样就不需要再额外使用group节点了。下面是基本的遍历代码：

```
void Node::Traverse()
{
  if(!KeepMatrix)
    glPushMatrix();

  Render();
  if(LeftChild!=NULL)
    LeftChild->Traverse();
  if(!KeepMatrix)
    glPopMatrix();
  if(RightSibling!=NULL)
    RightSibling->Traverse();
}
```

注意，由于使用左兄弟、右孩子方法表示树形结构，这意味着当遍历一棵树时，会按照预定的顺序访问孩子、兄弟节点。

显示器屏幕上绘制的图像结果取决于每个节点的绘制方法。不同类型的节点（几何、光源、材质），其绘制方法不尽相同，但是所有的绘制方法都需要调用OpenGL函数。下面分析几个例子。首先分析几何节点，通过观察它的保护成员，我们发现需要使用公有方法对它们中的一些成员存储的参数进行设置：

```
class Geometry: public Node
{
 public:
  // 公有成员

 protected:
  Color *ColorNode;
  Material *MatNode;
  Transformation *TransNode;
  DrawStyle *StyleNode;
};
```

几何节点类中的许多方法对所有几何节点来说都是相同的，并且使用的基本参数也是类似的。可以使用下面的构造函数创建一个几何节点：

```
Geometry::Geometry()
{
  ColorNode=NULL;
  MatNode=NULL;
  TransNode=NULL;
  StyleNode=NULL;
}
```

可以使用下面的析构函数删除几何对象：

```
Geometry::~Geometry()
{
  if(ColorNode)
    {
      delete ColorNode;
      ColorNode=NULL;
    }
// 其他类型的对象也可以这样处理
```

所有设置参数的方法都非常相似。例如，可以使用下面的代码设置颜色参数：

```
void Geometry::SetColor(Enum C)
{
  if(ColorNode==NULL)
    ColorNode=new Color;

  ColorNode->SetValue(C);
}

void Geometry::SetColor(float v1, float v2, float v3)
{
  if(ColorNode==NULL)
    ColorNode=new Color;

  ColorNode->SetValue(v1, v2, v3);
}
```

然而，为了绘制每个几何节点，需要为每个特定类型的几何节点定义相应的绘制方法。例如多边形，有下面的成员和方法：

```
class Polygon: public Geometry
{
 public:
  Polygon();
  void SetVerticesv(float v[][3], int);
  void Render();

 private:
  float Vertices[1000][3];
  int Size;
};
```

下面是它的一个简单实现：

```
void Polygon::SetVerticesv(float v[][3], int S)
{
  int i, j;
  Size=S;

  for(i=0; i<S; i++)
    for(j=0; j<3; j++)
      Vertices[i][j]=v[i][j];
}

void Polygon::Render()
{
  int i;
```

```
    glPushAttrib(GL_ALL_ATTRIB_BITS);

    if(ColorNode)
      ColorNode->Render();
    if(MatNode)
      MatNode->Render();
    if(TransNode)
      TransNode->Render();
    if(StyleNode)
      StyleNode->Render();

    glBegin(GL_POLYGON);
    for(i=0; i<Size; i++)
      glVertex3fv(Vertices[i]);
    glEnd();

    glPopAttrib();

  }
```

注意，为了不受由绘制内部节点引起的状态变化的影响，代码使用了压栈、出栈操作来保存和还原属性值。内部节点使用它们自己的类中的绘制方法进行绘制。例如，颜色节点具有两个私有成员：

```
class Color: public Node
{
 public:
  Color();
  void SetValuev(float *v);
  void SetValue(float v1, float v2, float v3);
  void SetValue(Enum Type);
  void Render();

 private:
  bool Changed;
  float Color3f[3];
};
```

每当要设置颜色值时，标志变量Changed的值被置为逻辑真。因此，颜色类的绘制方法很简单，代码如下：

```
void Color::Render()
{
  if(Changed) glColor3fv(Color3f);
}
```

虽然实现所有的节点还需要编写很长的代码，但是其实现过程却很直观。

10.10　开放场景图（OSG）

前面实现的这个简单的场景图虽然列举了场景图的许多关键要素，但是其功能受到很多限制。开放场景图也许是最流行的一个功能齐全的场景图API，它提供了许多前面那个示例中不具备的功能。除了支持更广泛的节点类型之外，还有两个对OSG尤为关键的概念。

首先，像OSG这种位于OpenGL之上的更抽象的高层软件所带来的优点之一就是：它可以实现CPU与图形处理单元（GPU）之间的负载平衡。下面分析在OpenGL中是如何处理几何数据的。应用程序通过给定的一组顶点来生成图元。正如我们所看到的，OpenGL主要关心的是绘制功能。

所有的几何图元都会至少流经绘制流水线的某些阶段。只有在顶点处理单元末端时，位于视见体之外的图元才被裁剪掉。如果从视点方向看某个图元，如果它被另外一个不透明的几何对象遮挡，则这个图元不会被绘制出来，但是这个图元还是流经了后面的大部分绘制流水线，而仅仅在隐藏面消除阶段才将它剔除。尽管目前的GPU可以每秒处理数百万的顶点，但是许多应用程序所处理的几何对象非常复杂，以至于GPU无法以足够高的帧率绘制这样的复杂场景。OSG使用了遮挡剔除（occlusion culling）和层次细节（level of detail，LOD）这两种实时绘制策略来减少复杂场景的绘制负荷。

遮挡剔除用来裁剪那些被其他对象遮挡的不可见对象，裁剪的时机是它们进入绘制流水线之前。如图10.20所示，位于视见体内的一个正方形在视线方向遮挡了一个三角形。尽管使用z-buffer算法能够得到反映遮挡关系的正确绘制结果，但是由于OpenGL独立地处理每个对象，所以它无法发现这种遮挡关系。然而，所有的几何对象作为节点存储在OSG场景图上，这对观察者来说是有用的信息。因此，OSG可以使用众多算法中的任一种遍历场景图并裁剪被遮挡的对象。10.11节将讨论一种基于二叉空间区分（binary spatial partitioning，BSP）的遮挡剔除方法。

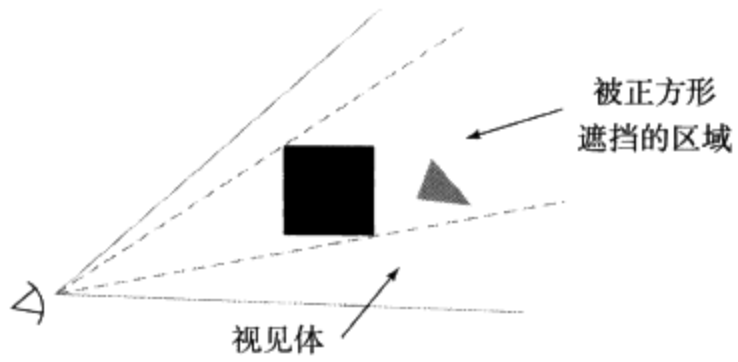


图 10.20　遮挡

第二个实时绘制策略基于这样一个想法，它类似于前面介绍的Mipmap纹理贴图。如果能够确定一个几何对象会被绘制在屏幕上一个很小的区域内，那么就不必把这个几何对象的所有细节都绘制出来。我们所需的这类信息可以再次放到场景图中。OSG场景图有一个LOD节点，该节点的孩子节点是一些反映某个对象不同几何复杂度的模型。可以通过应用程序设置这些LOD节点。在遍历场景图期间，由OSG确定使用对象的哪个LOD模型。

LOD绘制技术不仅对OSG很重要，而且对实时图形应用开发也非常重要，比如使用专有游戏引擎开发的交互式游戏等。游戏引擎是一个非常庞大而又复杂的软件对象，它可能包含数百万行的代码。尽管某个游戏引擎可能使用OpenGL或DirectX绘制图形，并充分利用可编程着色器，但是它还必须管理游戏玩家，甚至是多玩家的复杂交互操作。因此，游戏引擎使用场景图来维护包括几何对象和纹理贴图在内的所有必要信息，并在处理场景图的过程中广泛地使用LOD实时绘制技术。10.11节将讨论因特网上的图形涉及的一些相关问题。

前面介绍的简单场景图与OSG的第二个区别在于绘制每帧图像时场景图的遍历过程不同。OSG使用三次遍历过程，而我们实现的简单场景图只使用了一次遍历过程。遍历场景图是为了创建可用于绘制场景的几何对象列表。这个列表包含的几何对象具有最好的几何层次细节，并且只包含那些经遮挡剔除处理后保留的几何对象。此外，存储几何对象的方式能确保正确地绘制半透明的几何对象。

第一次遍历过程处理场景图的更新，所采用的方法是在应用程序中使用响应交互操作或几何对象修改的回调函数。第二次遍历过程建立用于绘制场景的几何对象列表，这次遍历过程使用了遮挡剔除、半透明物绘制、LOD和包围盒绘制技术。最后一次遍历过程访问几何对象列表并调用相应的OpenGL函数绘制几何对象。

10.11　图形和因特网

在结束讨论场景图之前，我们分析一下有关图形在因特网上如何传送的一些问题。尤其令我们感兴趣的是多玩家游戏，这类游戏涉及成千上万同时参与的游戏玩家，其中的每个玩家都可能影响其他玩家的场景图；另外我们对那些具有大量模型的应用程序也感兴趣，这些模型可能分布于多个站点。

因特网几乎对所有的通信和计算机应用都产生了巨大的影响。它可以使人们以多种不同的形式传播信息，并使许多新的交互方式成为可能。为了充分挖掘因特网的潜能，需要有效地传送图形信息，建立可供多个站点同时处理的应用程序，并且访问分布于多个站点的资源。OpenGL及其扩展对开发基于网络的三维应用和标准产生了巨大的影响。下面将采用面向图形方法并分析开发基于因特网的图形应用需要哪些扩展。其中有些概念读者可能比较熟悉。为了有效地绘制图形，我们使用客户/服务器模型。此外还将讨论如何实现独立于API的图形应用程序。

10.11.1　网络和协议

第3章基于客户/服务器模型讨论网络时，并没有讨论网络上的信息的传送细节。大部分的网络体系结构都是基于如图10.21所示的多层模型。该模型是国际标准组织（ISO）标准模型的简化版本。最底层是物理层，是传送比特流的物理介质。这一层包括连接计算机的线缆、电话线、光纤以及任何能够在网络实体之间传送比特流信息的硬件设备。物理层的上一层是数据链路层，它确保以可靠的方式在两个网络实体之间传送比特流。通常，这一层的寻址方案确保按正确的地址发送和接收信息，它还负责错误检测以及错误校验等功能。

数据链路层的上一层是网络层，它规定了不同的计算机上两个进程之间传送信息的协议。鉴于底层协议主要针对点对点的通信，我们更关心高层协议上进程之间的通信，基于这种方式的通信，信息包在到达目的地之前可能流经许多网络实体。对于组成因特网的数百万台计算机来说，最基本的协议就是传输控制协议和互联网协议（简称TCP/IP），这两个协议都位于这一层。Internet的一个定义是指使用互联网协议的松散连接的计算机网络。在网络上相互通信的应用程序位于最高层。当使用像OpenGL这样的图形API时，我们就工作在这一层。

下面分析一个使用了第3章讨论的显示列表的应用程序。在某台计算机中有一个应用程序（客户端），它需要使用远程图形服务器绘制客户端应用程序定义的图形对象，而这些图形对象存放在显示列表或场景图中。在协议层的最底层，它可能使用以太网把计算机连接起来，并使用标准的以太网协议进行通信。在中间层，两台计算机使用TCP/IP进行通信。在中间层的上一层，它们使用OpenGL扩展中的X协议交换图形信息。从应用程序的角度来看，我们只需在应用程序中使用显示列表或建立场景图。图10.22说明了OpenGL的通信协议层次结构。

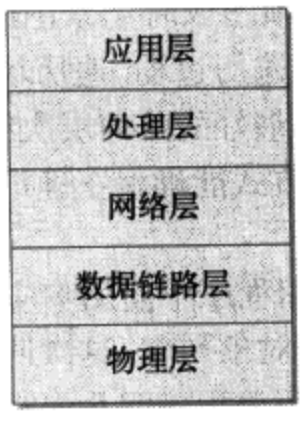


图10.21　协议层

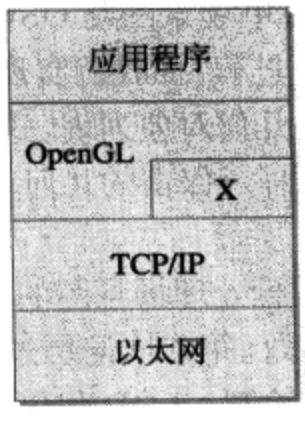


图10.22　OpenGL的通信协议

协议栈的出现为高层应用程序在各种各样的标准网络之间通信提供了可行的方法。如果要求所有的图形应用编程人员使用OpenGL开发图形应用程序，那么可以使用像显示列表这样的客户/服务器方法在网络上开发相当多的图形应用程序。但并不是所有的图形应用编程人员使用相同的API，然而，即便所有的图形应用编程人员都使用同一个API，也无法确保所有支持这个API的机器具有相同的结构。还有其他可能存在的问题。让我们暂时把图形放一边，而是从另一个不同的角度来分析网络，即把网络看做是从不同的信息源收集信息的系统。

10.11.2　超媒体和 HTML

随着 Internet 的发展，一些标准的高层协议为人们普遍接受，这些协议可用来传送电子邮件、文件和其他类型的信息。比如X Window视窗系统允许用户在远程计算机上打开窗口并传送基本的图形信息。然而，随着 Internet 规模的日渐扩大，人们可获取的信息越来越多，用户需要使用更先进的方法共享以分布式方式存储且具有不同格式的信息。

共享这样的信息需要 3 个基本要素：1）寻址方案，它可以使用户确定网络上资源的位置；2）信息编码的方法，除了简单的文本和其他资源的链接之外，还要对其他信息进行编码；3）一种以交互式方式搜索信息的方法。

前两个要素已经被欧洲粒子物理研究所（CERN）的研究人员解决，他们建立了万维网（World Wide Web），这个网络实际上是一个网络超文本系统。文件资源由唯一的统一资源定位器（Uniform Resource Locator，URL）识别，URL包括三部分内容：传送文件的协议，文件所在的服务器地址，文件在服务器中的存储位置。例如，本教材的支持材料对应的URL是http://www.cs.unm.edu/~angel/BOOK。其中第一部分（http）表示使用超文本传输协议http传送信息，第二部分（www.cs.unm.edu）表示新墨西哥州大学计算机科学系的万维网服务器地址，最后一部分表示信息存储在目录public_html/BOOK下的用户账号angel中。因为这个URL没有指定具体的文件，所以返回默认的文件，即主页。

CERN的另一贡献就是设计了超文本标记语言（Hypertext Markup Language，HTML），该语言提供了一种描述网页文档的简单方法，网页文档包含文本、对其他文档的超链接以及图像。HTML文档一般是用 ASCII 码或某个标准的扩展字符集表示的文本文件。

URL寻址方案和HTML文档相结合为我们在互联网上访问资源提供了可行的方法。但是伊利诺斯大学的国家超级计算机应用中心（NCSA）发布其开发的第一个 Mosaic 浏览器之前，用户在 Web 上获取或搜索资源并不是一件容易的事情。浏览器是一个交互式程序，它允许用户在 Web 上搜索并下载资料。Mosaic 和后来的 Netscape Navigator 为我们在 Web 上冲浪打开了便捷之门。

10.11.3　数据库和 VRML

HTML为在Web上支持交互式三维图形只提供有限的资源。主要原因是HTML只能识别两种常见格式（GIF和JPEG）的光栅图像，其存储形式是像素阵列。HTML没有任何几何对象的概念，甚至连二维几何对象的概念也没有。使用者很快认识到了这个局限性，并提出了多个有关设计一种新的网页语言的建议，最终设计并实现了虚拟现实建模语言（Virtual Reality Modeling Language，VRML）。

在网络上传送图形的方法有很多。我们可以使用HTML简单地传送图像，但是这种方法有明显的局限性。第二种方法是传送使用OpenGL等标准API编写的源代码。这种方法具有显著的优点，并为制定图形标准提供了重要依据。然而，这种传送源代码的方法要求用户在网络的客户端对程序进行编译，例如，用户可以在浏览器中编译程序。第三种方法是使用客户/服务器模型，就像OpenGL

中使用的客户/服务器模型。然而要使用这种方法，必须使用统一的API并使用统一的格式传送显示列表。Java语言采用的就是这种思想。

还有第四种方法，这种方法在图形如何创建、存储和传输上提供了极大的灵活性。我们可以创建一个描述场景的数据库，它包括几何对象、光源、材质和其他属性。该数据库可以以标准的文本格式进行存储，然后可以很方便地在网络上传送。数据库的接收端可以使用本地的任何硬件和软件绘制场景。VRML采用的就是这种方法。

与第2章讨论的设计图形API应该支持哪些功能一样，数据库的设计同样需要考虑这些问题。由于许多要考虑的问题是相同的（如支持哪些图元），因此设计数据库元素时，选择一个被人们普遍接受且支持多个体系结构的API是一种明智的做法。OpenGL显然是可选的API，但是它缺乏用来描述各个部件分布于Web不同位置的复杂场景所必需的面向对象的功能。VRML选择使用了Open Inventor数据库。正如10.8节指出的，Open Inventor建立在OpenGL之上，并且增加了包括场景图在内的许多其他功能。Open Inventor允许用户使用文本文件存储场景。一旦有了到数据库的链接，就有了用于描述各个组件分布于网络上的三维场景的大量信息。

VRML与OpenGL紧密相关。VRML提供了用来描述分布式三维场景的方法，并且VRML浏览器可以从Web上获取这样的场景。因此，OpenGL与VRML之间存在非常密切的关系。虽然在原则上VRML浏览器可以使用基于流水线结构、光线跟踪法或辐射度方法的任何一种绘制引擎，但事实上，由于底层的数据库基于Open Inventor，所以人们往往选择基于OpenGL的VRML浏览器。

10.11.4　Java与Java小应用程序

当使用前面讨论过的模型和工具时，我们在Web上进行交互操作时会出现问题。由于VRML数据库是文本格式，所以它要求使用客户端程序来绘制数据库中的内容。因此，在某台机器上开发程序的编程人员不可能知道他编写的程序能否在另一台机器上执行，因为这两台机器可能完全不一样。

Java解决了这个问题的一部分，方法是使用软件的方法创建一个可以在任何机器上实现的虚拟机。Java程序被编译成字节码（byte code），这种字节码程序可以在任何Java虚拟机上运行，不管机器的底层硬件如何。因此，客户和服务器在Web上交换字节码程序。我们把这种字节码小程序称为Java小程序（applet），它能被标准的Web浏览器理解，并且为Web增加了大量的动态行为。

Java最初只包括二维图形。有一个称为Java3D的Java扩展工具包，我们可以用它创建和绘制场景图，尽管能否实现其功能还有待观察。目前，在Web上传送图形的方法很多，希望今后几年内能有一种统一的方法，这种方法能充分利用OpenGL的交互性能和绘制性能，此外，它还能充分利用场景图的面向对象特性。

10.12　其他树形结构

树形结构和DAG（有向无环图）为描述场景提供了强有力的工具，树形结构在计算机图形学中还有许多其他的使用方式，这里介绍其中的三种。第一种使用表达式树来描述实体对象的层级结构；另外两种用来描述空间层级结构，主要用来提高诸多绘制算法的效率。

10.12.1　CSG树

前面使用对象的多边形表示方法有许多优点，但也有一些缺点。其中最主要的缺点是多边形只能描述像多面体这样实心三维对象的表面。在CAD应用中，每当必须使用图形对象的质量和转

动惯量等体积属性时，这个局限性就会带来许多麻烦。此外，因为显示对象时只是绘制它的边缘或表面，所以显示的图形可能存在歧义。例如，图 10.23 所示的线框图既可以解释为一个中间带孔的立方体，即从立方体的中间挖去一个圆柱体，也可以解释为一个由两种不同材质组成的实心立方体。

物体的体素构造表示法（Constructive Solid Geometry，CSG）可以解决这些问题。假定考虑一组原子几何实体，如平行六面体、圆柱体和球体。这些对象不仅包括颜色和反射率等表面属性，还包括大小和密度等体积属性。为了描述由这类对象组成的场景，需要考虑用来构造每个对象的空间点集。也就是说，因为每个对象对应一组点集，所以可以使用集合代数由这些实体原子对象构造新的对象。

基于 CSG 的建模方法使用了三种集合运算：并、交和差。集合 A 和集合 B 的并（union），记为 $A \cup B$，它包含集合 A 或 B 中的所有元素。集合 A 和 B 的交集，记为 $A \cap B$，它包含同时属于集合 A 和集合 B 中的元素。集合 A 和 B 的差集，记为 $A - B$，它包含属于集合 A 但不属于集合 B 的元素。图 10.24 显示了两个对象以及这两个对象经过三种集合运算处理后可能的结果。

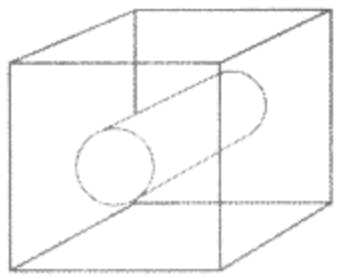


图 10.23　可能具有两种不同解释的线框

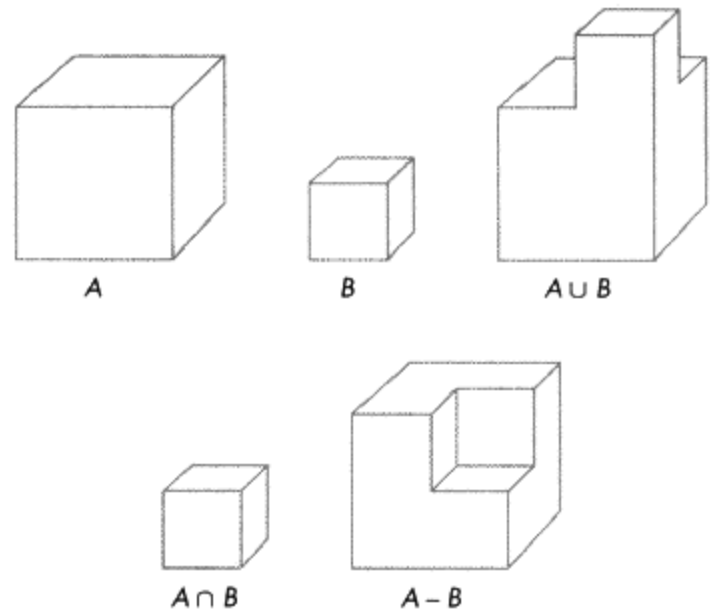


图 10.24　集合运算

我们使用代数表达式表示对象。例如，代数表达式$(A - B) \cap (C \cup D)$可能表示的是如图 10.25 所示的对象。

我们一般使用表达式树来存储、解析代数表达式，在表达式树中，内部节点存储运算符，终止节点存储操作数。例如，图 10.26 所示的 CSG 树表示图 10.25 所示的对象（$A - B$）∩（$C \cup D$）。可以使用后序遍历法来计算或绘制这棵CSG树，这里的后序遍历法是指递归地计算节点的左子树值，然后递归地计算节点的右子树值，最后使用这两个值计算该节点本身的值。可以使用光线跟踪的变体方法绘制用 CSG 树表示的对象，见习题 10.10 和第 13 章。

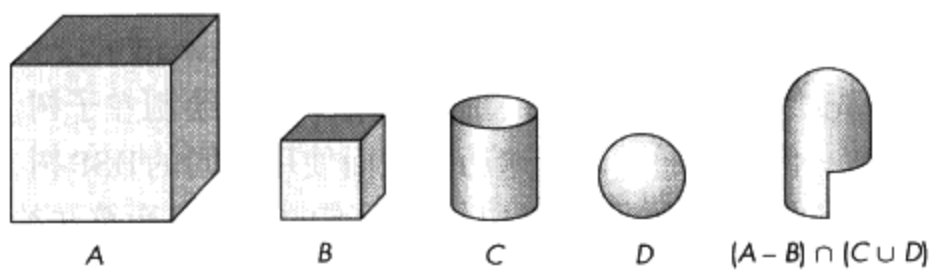


图 10.25　CSG 对象

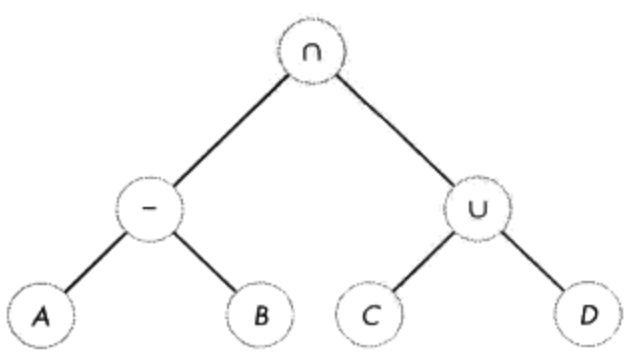


图 10.26　CSG 树

10.12.2　BSP树

场景图和CSG树描述了对象各部分之间的层级关系。我们还可以使用树形结构描述场景对象空间，并把对象组之间的空间关系封装起来。我们可以利用对象之间的空间关系设计快速的可见性测试方法，它可以确定哪些对象是可见的，因此避免了使用z-buffer算法对所有对象进行可见性测试（指隐藏面消除）。这些技术在计算机游戏的实时动画绘制中非常重要。

通过观察我们发现一个平面可以把一个三维空间划分或分割成两部分（两个半空间），据此我们提出建立空间层级结构的一种方法。连续地使用平面不断划分空间，可以得到越来越小的空间区分。在二维空间中，可以使用直线划分空间。

考虑图10.27所示的多边形，观察者位于如图所示的位置。正如第7章讨论的，为了绘制正确的图像，需要按一定的顺序绘制这些多边形。我们不想在每次绘制这些多边形时都使用像深度排序这样的算法，而是事先把多边形之间的相对位置信息存储在一棵树中。在构建这棵树的过程中，每次使用一个多边形作为划分平面把多边形分成两组，一组位于该平面之前，另一组位于该平之后。例如，分析一个简单的空间，在这个空间中，所有的多边形互相平行，它们的法向量与z轴平行。这种假设可使我们更容易地理解这个算法，而且只要任何多边形所在的平面把其他的多边形分成前后两组，那么这个假设不会影响算法的正确性。如图10.28所示，假设从z轴方向观察这个空间。

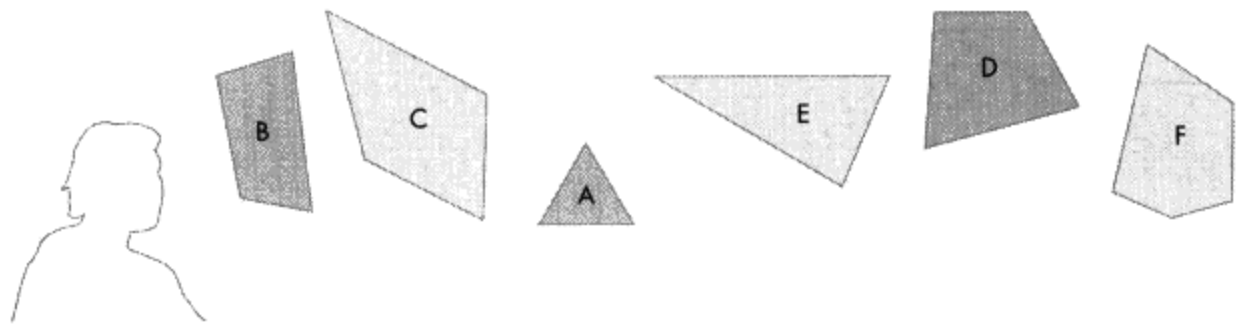


图10.27　一组多边形与一个观察者

平面A把多边形分成两组，一组包含B和C，它们位于A的前面，另一组包含D、E和F，它们位于A的后面。我们使用多边形A所在的这个平面开始构建二叉空间区分树（Binary Spatial-Partition，简称BSP树），它保存了划分平面和用于划分的顺序。因此，在图10.29所示的BSP树中，A是根节点，B和C位于左子树中，D、E和F位于右子树中。对左子树进行递归，用多边形C所在的平面作为划分面，而B位于C的前面，据此可以建立左子树。对右子树进行递归，多边形D所在的平面把E和F区分开，E位于D前面，F位于D的后面，据此可以建立右子树。注意，由于选择多边形作为划分平面的顺序可能不相同，所以对于一组给定的多边形有可能存在多棵BSP树。在一般的情况下，如果一个划分平面与一个多边形相交，那么就把该多边形分割成两个多边形，一个位于划分平面的前面，另一个位于划分平面的后面，这有点类似于第7章介绍的深度排序算法处理重叠多边形使用的方法。

我们可以使用这棵树来绘制这些多边形，方法是对这棵树进行后向中序遍历（backward in-order traversal）。也就是说，我们递归地遍历这棵树，先绘制右子树，然后绘制根节点，最后绘制左子树。BSP树的一个优点是：即使改变了观察者的位置，仍可以通过改变遍历算法而使用相同的BSP树。例如，如图10.30所示，如果观察者移动到多边形的另一侧，那么可以使用标准的中序遍历算法绘制由这些多边形组成的场景，即先绘制左子树，再绘制根节点，最后绘制右子树。还需注意的是：只要平面能把多组多边形或对象划分成不同的组，称之为簇（cluster），那么就可以使用这种递归算

法。因此，可以把多边形按多面体对象进行分组，然后又把这些多面体对象按簇进行分组。这样就可以在每个簇内部使用上面的递归算法。在飞行模拟器这样的应用中，场景模型不发生变化，但观察者的位置发生变化，在绘制场景的过程中使用BSP树可以有效地实现可见面确定算法。BSP树包含绘制多边形所需的所有信息，观察者的当前位置决定了使用的遍历算法。

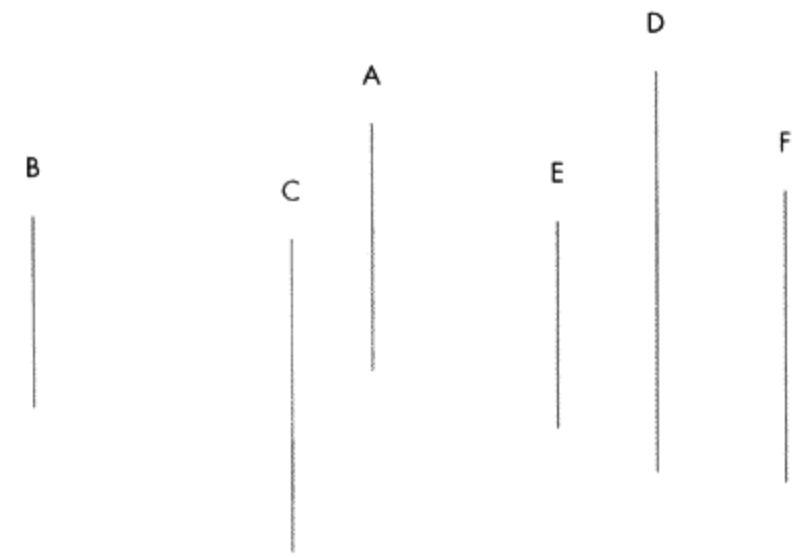


图 10.28　多边形的俯视图

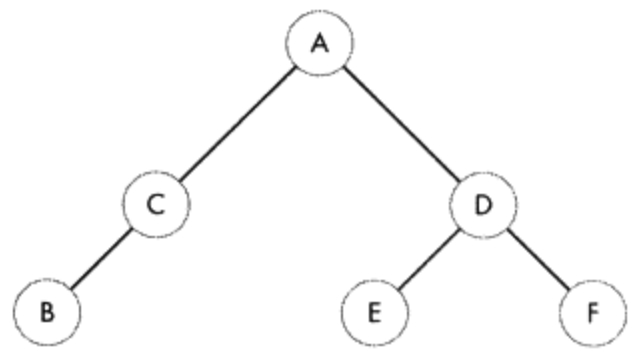


图 10.29　二叉空间区分树（BSP 树）

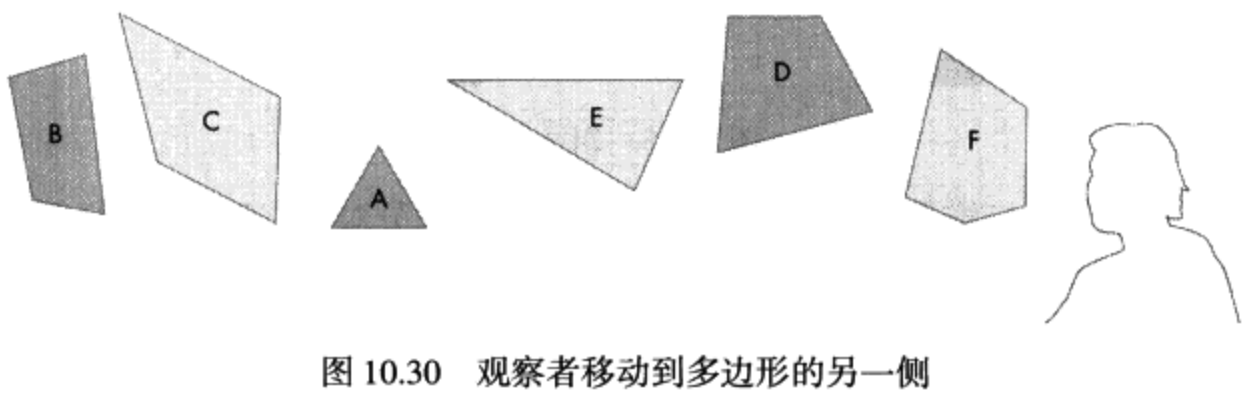


图 10.30　观察者移动到多边形的另一侧

10.12.3　四叉树和八叉树

BSP 树的一个缺点是用于划分多边形的平面可以具有任意的方位，因此构建 BSP 树的计算开销很大，如多边形排序与时常出现的多边形分割[①]都需要大量的计算开销。四叉树和八叉树由于使用与坐标轴平行的划分平面或划分直线，所以可以避免出现这个问题。

考虑图10.31所示的二维图像。假定该图像由黑白像素组成，它可能是一个三维场景的绘制结果。如果希望存储这个场景，那么可以把该图像存储为一个二进制数组。但是我们注意到图像的像素之间存在大量的连贯性，同一颜色的像素会聚集在一起。如图10.32所示，我们通过画两条直线把整个区域分割成 4 个象限。我们注意到其中一个象限的像素颜色都是白色，只需给该象限赋一个颜色值即可。对于其他 3 个象限，我们再次分割，只要某个象限包含多余一种颜色的像素就一直分割下去。我们把分割的信息存储在一棵称为四叉树（quadtree）的层级结构中，在这棵树中，每一层对应于一次细分，并且每个节点都有 4 个孩子节点。因此，最初的原始图像对应的四叉树如图 10.33 所示。

由于建立四叉树所采用的方法是使用与坐标轴平行的直线来划分平面空间，所以四叉树的建立与遍历比前面介绍的BSP树对应的操作更简单。四叉树一个尤为重要的优点就是可以使用它来减少存储图像所需的内存空间。

① 当划分多边形的平面与某个多边形相交时，需要分割该多边形。——译者注

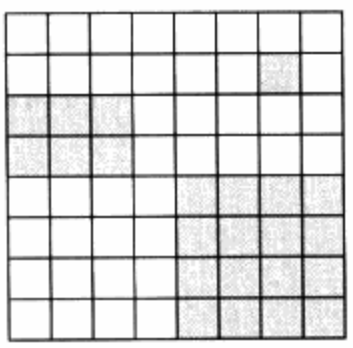


图 10.31　二维像素空间

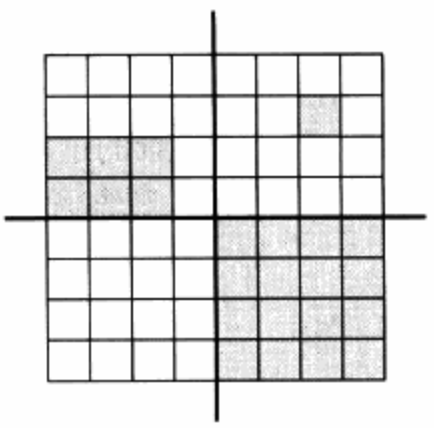


图 10.32　空间的第一次细分

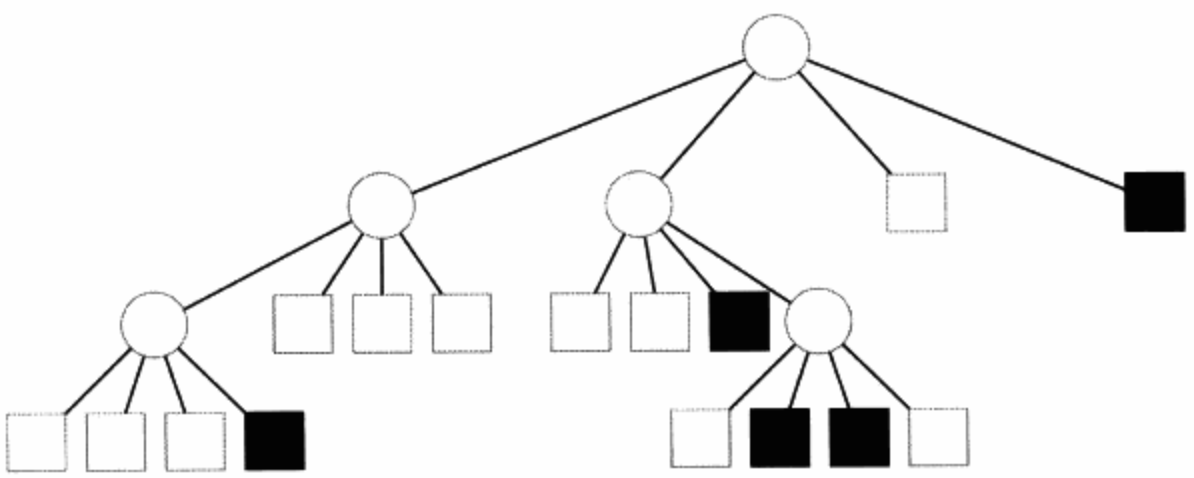


图 10.33　四叉树

四叉树主要用来划分二维平面空间。也可以使用四叉树划分对象空间，划分的方式类似于BSP树，因此可以根据观察者的位置按某个顺序遍历预先建立的四叉树，这样可以正确地绘制每个区域内的对象。在三维空间，四叉树扩展为八叉树。我们使用与坐标轴平行的平面划分空间，每次把空间划分成 8 个子空间，如图 10.34 所示。

八叉树可用来表示体数据集，它由基本的体素（voxel）构成，如图 10.35 所示。前面讨论的四叉树和八叉树不仅可以按像素或体素划分空间，还可以用来把空间按对象进行划分。例如，为了裁剪对象，可以递归地划分二维或三维空间。每次划分后，都要把每个对象的包围盒与划分后的矩形或立方体进行比较，从而确定该对象是否位于某个划分后的子空间区域内。

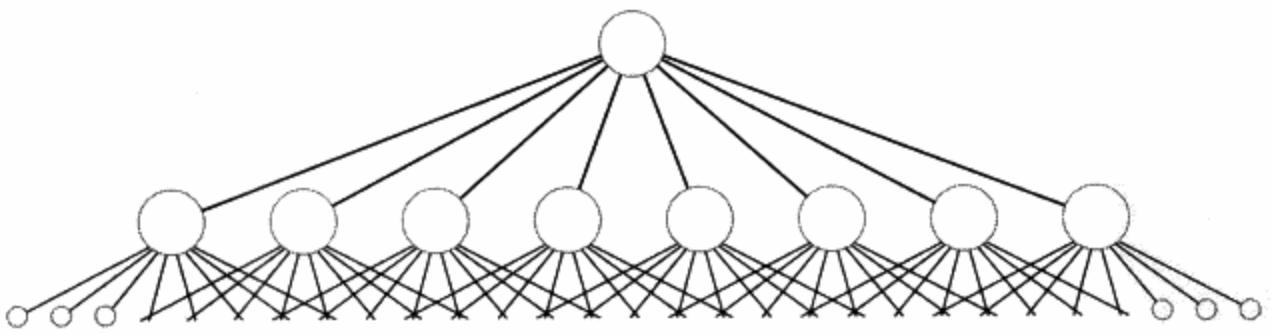


图 10.34　八叉树

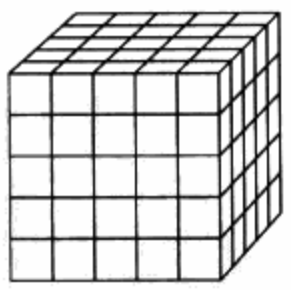


图 10.35　体数据集

小结和注释

可以使用最新的硬件设备快速绘制几何对象，这为人们开发各种建模系统提供了新的便利条件。作为计算机图形学的用户，如果希望充分利用已有的图形系统，那么必须掌握大量的建模技术。我们不仅可以深入研究本章以及下一章讨论的各种建模技术，而且还可以将这些技术组合起来形成新的建模技术。读者可以阅读后面指定的建议阅读资料以进一步研究各种建模方法。

本章介绍的基本主题适用于大部分建模方法。一种方法是使用层级结构来表示场景中对象之间的关系。我们可以使用树和DAG这样的基本数据结构来表示对象之间的这种层级关系，遍历这些数据结构已成为绘制过程的一部分。应用程序开发人员在OSG（Open Scene Graph）、VRML和Java3D中使用场景图的时候，只要对预定义的对象和用户自定义的软件模块进行组合就可以建立复杂的动画场景。正如我们在第9章看到的，树形结构模型还可用于描述复杂的着色器，这些着色器可以处理光源、材质属性、大气效应以及各种局部反射模型之间的相互作用。我们可以使用RenderMan、Cg或GLSL实现这些方法。

面向对象的方法是处理复杂图形应用和基于网络的分布式应用的标准方法。遗憾的是，目前还没有一个统一的面向对象的图形API。尽管如此，目前大多数高端系统在实际绘制图形时还处在OpenGL这一层，原因是OpenGL API更接近硬件层，这样可以更有效地利用硬件的性能。因此，无论是应用程序开发人员还是系统开发人员都需要熟悉各个层次的API。

第11章将介绍一种完全不同的基于过程的建模方法，它可以作为本章介绍的层级建模方法的一个补充。

建议阅读资料

在30多年前的图形学文献[New73]中就有如何使用矩阵栈进行层级变换的描述。PHIGS[ANSI88]是第一个把层级变换作为标准软件包一部分的图形API。文献Watt[Wat92]介绍了人物关节动画的有关内容。Lassiter的论文[Las87]说明了在电影行业中使用的传统动画技术与计算机图形学中动画技术之间的关系。

BSP树首次由Fuchs、Kedem和Naylor提出[Fuc80]，主要用于可见性测试，后来用于其他许多应用领域，如CSG等。阅读[Mol02]可以了解BSP的其他应用。

场景图是Open Inventor的核心[Wer94]。Open Inventor的数据库格式是VRML语言的基础[Har96]。许多最近的API都使用了面向对象的方法，如Java3D[Swo00]和DirectX[Kov97]。要想研究Java和Java小程序，可以阅读[Cha98]和[Arn96]。树是RenderMan着色语言中不可缺少的一部分[Ups89]，在RenderMan着色语言中，可以使用树来创建着色器。Maya等建模系统允许用户为不同的对象定义各种着色器和绘制算法。文献[Mar07]介绍了开放场景图（OSG）。

可见性测试的许多应用可以在文献[Mol02]中找到。

文献[Ebe01]讨论了在游戏引擎的设计中如何使用场景图。文献[Mau06]以Torque游戏引擎为例为游戏编程人员描述了游戏引擎的运行方式。

习题

10.1　对于本章的简单机器人手臂模型，计算上臂顶端能够到达的顶点集。

10.2　根据关节角度求简单机器人手臂上任意点的位置方程。能否从上臂顶端的位置确定关节角度？解释你的理由。

10.3　在空间给定机器人手臂可到达的两个点，根据关节角度计算这两个点之间的路径。

10.4　根据图符-实例变换表，编写一个简单的电路布局程序。图符包括电路元件的形状符号，如电阻器、电容器和电感，或者逻辑电路的各种门电路符号（AND, OR, NOT）。

10.5　我们把用于搜索的二叉树描述为由节点构成的链表，每个节点包含一个指向孩子节点的指针。编写一个OpenGL程序，它可以根据你对二叉树的描述以图形化的方式显示这棵二叉树。

10.6 机器人学只是说明场景各个部分合成运动的一个例子，其中某些对象的运动依赖于其他对象的运动。其他的例子包括自行车（带有轮子）、飞机（带有推进器）、旋转木马（带有马）。举一个合成运动的例子，并编写一个图形程序模拟它的合成运动。

10.7 给定两个顶点数量相等的多边形，编写一个程序生成一系列的图像，使得将其中的一个多边形变换为另一个多边形。

10.8 给9.5节的树的节点增加一个属性并对树的遍历算法进行必要的修改。

10.9 建立一个简单的场景图系统，它包括多边形、材质、一个观察者和光源。

10.10 为什么光线跟踪或光线投影算法适合于绘制用CSG树表示的场景。

10.11 说明如何使用四叉树生成不同分辨率的图像。

10.12 编写一个程序，它允许用户从一些基本的形状建立简单的关节人体模型。程序应该允许用户放置关节，并绘制最终的关节人体模型的动画效果。

10.13 能否设计一个独立于遍历算法的场景图？

10.14 在本章实现的场景图中增加一个功能，使我们能够以文本格式存储场景图，并且能够从文件中读取场景。

10.15 给场景图的对象增加动画绘制功能。

10.16 在10.3节的机器人手臂的末端增加一只手或“握爪”。

10.17 给10.3节的机器人添加轮子，从而使它具有在平面上移动的功能。

10.18 如果把对象划分成簇，那么BSP树的效率可能会更高。可以使用簇的包围盒进行可见性测试。编写程序实现这个算法并把它用于场景图绘制器。

第11章　过程建模方法

到目前为止，我们都是假定使用表面来表示要绘制的几何对象，并且用凸多边形表示（或近似）几何对象的表面。之所以使用多边形表示几何对象，主要是因为我们可以很容易地使用多边形表示这些几何对象，并且能够在现有的图形系统上绘制这些对象。人们在计算机图形学领域取得的成就证实了多边形模型的重要性。

然而，就在人们在CAD应用、飞行模拟器、计算机动画、交互式视频游戏以及电影特效制作中使用多边形模型的时候，用户和应用开发人员都认识到了这种建模方法的局限性。像云彩、烟雾以及水等物理对象并不适合使用这种多边形建模方法。多边形建模方法无法给这些物理对象增加物理约束条件，也无法表示物理对象的复杂行为。为了解决这些问题，研究人员提出了过程建模方法，它使用算法表示对象底层的物理现象，因此在绘制过程中只有需要的时候才绘制多边形。

11.1　基于算法的模型

当回顾计算机图形学的发展历史时，我们发现人们希望创建更具真实感图形的愿望总是超过计算机硬件的发展。虽然在现有的商用图形硬件上能够每秒绘制5000多万个多边形，但是像飞行模拟、虚拟现实和计算机游戏等应用可能需要超过每秒5亿个多边形的绘制速度。而且，当绘制速度提高以后，生成的多边形数量也急剧增加。单个数据集包含的多边形可能超过10亿个。

然而，许多应用程序之所以仍有需求使用多边形建模方法，是因为它们使用了现有的软件和建模方法。敏锐的研究人员和应用程序开发人员建议，如果能够绘制这样一个模型，它只生成那些既是可见的，而且投影后至少具有一个像素大小的多边形，那么就不需要生成如此之多的多边形。我们在前面的章节中已经看到了体现这种想法的一些示例，例如，在多边形到达绘制流水线之前，我们考虑尽可能通过可见性裁剪把不可见的多边形裁剪掉①。然而，一种更有效的方法是重新考虑建模方法并寻求一种新的建模技术，即过程建模方法（procedural method），这种建模方法采用与目前介绍的建模方法完全不同的方式生成几何对象。过程建模方法包括很多具体的技术方法，它们具有的共性是都使用算法的方式描述对象，并且作为绘制过程的一部分，即只有在需要的时候才生成多边形。

在许多方面，可以把过程建模与在计算机中表示无理数（如平方根、正弦和余弦函数等）的方法进行类比，这有助于对过程建模的理解。例如，考虑$\sqrt{2}$的3种表示方法，其数值表示法为：

$$\sqrt{2} = 1.414\cdots$$

我们可以在后面随意添加任意多的数字。或者使用更抽象的方法，我们把$\sqrt{2}$定义为满足下面方程的一个正数解：

$$x^2 = 2$$

但是在计算机中，$\sqrt{2}$可能是运行某个算法的结果。例如，下面分析牛顿迭代法。假设初始$\sqrt{2}$的近似结果为$x_0 = 1$，并通过下面的公式进行迭代计算：

---

① 可见性裁剪包括：视见体裁剪、背面裁剪和遮挡剔除。——译者注

$$x_{k+1}=\frac{x_k}{2}+\frac{1}{x_k}$$

每个x_k的后继值是$\sqrt{2}$的一个更好的近似值。从这个角度看，$\sqrt{2}$是通过一个算法定义的，或者说是由一个程序定义的。对于计算机图形学中要处理的对象，可以采取类似的方法。例如，可以把一个球心在原点的球体定义为一个满足下面方程的数学对象：

$$x^2+y^2+z^2=r^2$$

我们使用第6章介绍的四面体细分处理方法及实现这种细分的代码也能生成这样的一个球体。使用第二种绘制球体的方法带来的好处是：绘制较小的球体（在屏幕空间）比绘制大球体需要更少的三角形。

多边形建模存在的另一个问题是很难把计算机图形学与物理规律结合起来。虽然可以基于多边形建模方法建立真实世界的图形对象并绘制相应的动画效果，但是要模拟使这些图形对象像真实实体一样运动且不会相互穿透这样的物理效果是非常困难的。

我们只介绍众多的过程建模方法中的4种。在第一种方法中，首先介绍遵守牛顿定律的粒子，然后设计一个粒子系统，该系统通过求解一组微分方程（完成数千个粒子的常规数值计算任务）能够模拟一些复杂的行为。大量粒子的方位决定了标准几何对象在世界坐标系中的位置。

第二种方法是基于语言的建模，可以使用这种建模方法控制对象的复杂度，方法是使用类似于表示自然语言和计算机程序设计语言的文法模型代替多边形模型。利用这些模型只需使用少量的规则（这些规则可用于生成所需的图形实体）就可以近似地表示自然界的许多对象。如果把这种建模方法与分形几何结合起来，那么只需使用绘制对象所需的一定数量的多边形就能生成最终的图像。

第三种方法是分形几何，这种建模方法基于在许多自然现象中看到的自相似性。分形几何提供了一种生成任意层次细节模型的方法。最后，本章将介绍基于过程噪声（procedural noise）的建模方法，通过这种方法可以把数量可控的随机性引入模型中。我们可以使用过程噪声创建纹理贴图、模拟流体模型的不稳定行为、实现动画中的逼真运动行为以及绘制像云彩这样外形模糊的对象。

11.2　基于物理的模型和粒子系统

计算机图形学中建模方法的一个最大优点，也是它的一个最大缺点就是可以选择任意的方法建立模型。我们建立的图形对象与它具有的物理逼真性没有任何关联。从历史的观点来看，人们对建模持有的态度是：只要建立的模型看起来像真实的对象，这样的模型就足以满足大多数需求。由于摆脱了物理模型所带来的限制，这种限制可能并不为人们所知，或者这种限制太复杂以至于无法进行实时模拟，所以可以建立计算机游戏和电影中的种种特效。在科学计算可视化等领域，这种不拘于几何对象物理属性的灵活的建模方式使得数学家可以“看到”在三维空间根本不存在的形状并以全新的方法显示信息。此外，研究人员和工程师们可以构造对象原型，所用的构造方法却不受现有材质和设备的限制。

然而，当人们要模拟真实世界中的对象并希望在显示器上观察到模拟的结果时，就陷入了困境。一般来说，为一组在空间中运动的对象建模并不困难，但是要跟踪两个对象何时碰撞，而且要让图形系统按物理规律来响应这种碰撞却是非常棘手的问题。事实上，在计算机图形学中，让一个球穿墙而入，这比让这个球碰到墙面后按正确的物理规律从墙面反弹回来要容易得多。

最近，研究人员对基于物理的建模方法（physically-based modeling）越来越感兴趣，这是一种让图形对象遵守物理规律的建模方法。我们可以使用两种相关的方法建立这种模型。一种方法是模

拟基本过程的物理规律，并利用这种物理规律来控制图形对象的行为。例如，如果想让一个实体对象看起来像在空中翻筋斗，并在各个表面之间来回弹跳，那么至少在理论上可以使用动力学和连续介质力学（continuum mechanics）方面的知识推导所需的方程。这种方法涉及的内容远远超出了计算机图形学导论课的范畴，因此这里不再深入介绍这种方法。另一种方法是把基本的物理原理和数学约束条件结合起来控制对象的行为。粒子系统使用的就是这种方法。

粒子系统（particle system）是许多粒子的集合，这里的粒子通常是指质点，可以通过求解微分方程组来确定粒子的动态行为。粒子系统已经在许多领域用来模拟对象的各种行为。例如在流体力学领域，研究人员使用粒子系统模拟湍流的行为。一般人们在实际中并不求解偏微分方程，而是生成一群受各种力作用以及受各种约束条件限制的粒子来模拟系统的动态行为。我们还可以使用粒子建立实体对象模型。例如，可以使用一个三维粒子阵列来模拟可变形的实体对象，且其中的粒子由弹簧连接起来。当对象受到外力的作用时，粒子就会移动并使它们的位置近似表示实体对象的形状。

计算机图形学领域的研究人员已经使用粒子系统建立了许多对象的模型，如烟火、鸟的群聚行为以及波浪起伏的行为等。在这些应用中，粒子系统的动力学规律确定了粒子的位置，而在每个位置上，我们可以放置一个图形对象，而不是一个点。

在所有这些示例中，我们处理的都是一群粒子，我们把其中的每个粒子视为一个质点。根据物理规律可得到粒子的运动方程，并用数值方法求解这些方法，从而得到这些粒子在每个时刻的状态。在最后一步，我们把每个粒子作为一个图形对象来绘制，对于烟火粒子系统，粒子可能是一个有颜色的点，或者对于粒子动画系统，粒子可能是一个卡通人物。

11.3　牛顿粒子

下面分析一组遵守牛顿定律的粒子。虽然没有理由说不能使用其他的物理定律或自己虚构的一组物理定律，但是首先介绍牛顿粒子的好处是：利用这种简单的、为人们所理解的物理定律，我们可以模拟范围更广的物理行为。一个牛顿粒子必须遵从牛顿第二定律，它表明粒子的质量（m）乘以粒子的加速度（$\mathbf{a}$）等于作用于粒子上的所有外力之和（$\mathbf{f}$），或者表示为下面的公式：

$$m\mathbf{a} = \mathbf{f}$$

注意，加速度和力通常都是三维矢量。牛顿定律的一个结论是：对于一个理想的质点粒子，即全部质量集中在一个点上的粒子，它的状态完全由其位置和速度决定。因此在三维空间中，一个理想的粒子具有6个自由度，一个具有n个粒子的系统则有$6n$个状态变量，它们表示系统中所有粒子的位置和速度。在某个参考坐标系中，第i个粒子的状态由两个三维列矩阵决定[①]，一个是位置矩阵：

$$\mathbf{p}_i = \begin{bmatrix} x_i \\ y_i \\ z_i \end{bmatrix}$$

另一个是矢量矩阵：

$$\mathbf{v}_i = \begin{bmatrix} \dot{x}_i \\ \dot{y}_i \\ \dot{z}_i \end{bmatrix} = \begin{bmatrix} \frac{dx}{dt} \\ \frac{dy}{dt} \\ \frac{dz}{dt} \end{bmatrix}$$

① 这里之所以选用三维列矩阵而不是奇次坐标的表示形式，是为了使这种表示形式和物理文献中这些公式的表示形式保持一致，同时也为了简化最终得到的微分方程。

因为加速度是速度的导数，而速度是位置的导数，所以可以把粒子的牛顿第二定律写成下面形式的一阶微分方程（由于每个粒子有6个自由度，所以一共有6组方程）：

$$\dot{\mathbf{p}}_i = \mathbf{v}_i$$

$$\dot{\mathbf{v}}_i = \frac{1}{m_i}\mathbf{f}_i(t)$$

因此，对于一个具有n个粒子的粒子系统，它的动力学规律由$6n$组上述形式的常微分方程决定。

除了状态之外，每个粒子还有许多其他的属性，这些属性包括它的质量（m_i）以及改变粒子行为和反映粒子显示方式的一组属性。例如，有些属性决定了绘制粒子的方式，并决定了粒子的颜色、形状和表面的性质。注意，尽管一个简单粒子系统的动力学行为取决于每个被视为质点的粒子的行为，但是应用程序编程人员可以指定每个粒子的绘制方式。例如，在一个表示人群的场景中，每个粒子可能表示一个人；或者在化学合成应用中，每个粒子可能表示一个分子；或者在模拟风中飘扬的旗帜的应用中，每个粒子可能表示一小块布。对于上面的任何一种情形，底层的粒子系统决定了粒子质量中心的位置和速度。一旦确定了粒子的位置，就可以把所需的图形对象放在这个位置上。

作用于粒子上的一组力$\{\mathbf{f}_i\}$决定了系统的行为。这组力取决于粒子系统的状态，并且随时间而改变。这些力可以建立在简单物理学原理的基础上（如弹簧力）；或者建立在物理约束条件的基础上，这些物理约束条件是我们希望强加到系统上的条件；或者建立在外力的基础上，如重力，这是我们希望施加到系统上的力。只要精心设计作用于系统上的力，就能得到所需要的系统行为。

我们可以使用数值方法求解系统的动力学状态，这里的数值方法通过逐步逼近的方法求解微分方程组的近似解。一个典型的粒子系统在一个仿真周期内需要完成3个功能，即通过用户自定义函数计算作用于n个粒子上的力；通过微分方程求解系统受这些力作用后的状态；根据这些粒子的新位置及其属性绘制粒子所处位置的任何图形对象。因此，粒子系统仿真循环的伪代码实现形式如下：

```
float time, delta; float state[6n], force[3n];
state=get_initial_state();
for(time=t0; time<time_final; time+=delta)
 {
 /* 计算力 */
    force=force_function(state, time);

 /* 求解标准的微分方程 */
    state=ode(force, state, time, delta);

 /* 显示结果 */
    render(state, time);
}
```

在一个给定的具体应用中，我们设计的主要部分是计算作用在每个粒子上的力的函数。

11.3.1　独立的粒子

有许多简单的方法可以模拟粒子相互作用并确定作用在每个粒子上的力。如果作用在某个粒子上的力独立于其他粒子，那么作用在第i个粒子上的力可以用下面的方程描述：

$$\mathbf{f}_i = \mathbf{f}_i(\mathbf{p}_i, \mathbf{v}_i)$$

在最简单的情况下，每个粒子只受重力常量的作用：

$$\mathbf{f}_i/m_i = \mathbf{g}$$

如果这个力的方向朝下，那么

$$\mathbf{g} = \begin{bmatrix} 0 \\ -g \\ 0 \end{bmatrix}$$

其中，g 是正数，并且每个粒子的运动轨迹是弧状抛物线。如果增加一个与粒子速度成正比的项，那么可以模拟粒子受反向摩擦力作用的情形。如果粒子的颜色等属性会随时间变换，并且每个粒子都有一个随机的生命周期，那么可以使用粒子系统模拟烟火等现象。更一般的情况是：外力可以独立地作用在每个粒子上。如果允许粒子在空中随意飘移，并且把每个粒子都绘制成一个很大的图形对象而不是一个点，那么可以使用独立的粒子模拟云彩或水流。

彩图 17 显示了使用独立的粒子模拟焊接时产生的火花。每个粒子都独立于其他的粒子，并且受到重力的作用，而且每个粒子都具有一个随机的生命周期。彩图 21 和彩图 22 显示了动画片《企鹅冲浪》中使用粒子系统绘制的波浪，这里使用粒子来模拟浪花。

11.3.2　弹簧力

如果在一个具有 n 个粒子的系统中，所有的粒子相互独立，那么计算某个粒子上受力的时间复杂度为 $O(n)$。而在最一般的情形下，对某个粒子的受力计算可能需要考虑它与其他粒子之间的相互作用力，因此粒子受力计算的时间复杂度为 $O(n^2)$。对于一个含有大量粒子的粒子系统，$O(n^2)$的时间复杂度会极大地降低系统的计算速度。通常情况下，我们可以这样降低粒子受力计算的复杂度，方法是只考虑该粒子与其他靠近它的粒子的相互作用力。

下面讨论一个例子，我们使用粒子建立一个随时间变化的表面，如在风中飘动的窗帘或旗帜。如图 11.1 所示的粒子网格，我们把每个粒子的位置视为矩形网格上的一个顶点。粒子网格的形状之所以会随时间而变换，是由于每个粒子既受到外部的作用力，如重力或风力等，又受到粒子之间的相互作用力，它把粒子聚集在一起，使粒子网格看起来像是连续的曲面。通过计算某个粒子与离它最近的粒子之间的作用力，可以近似模拟粒子的第二种受力，即粒子之间的相互作用力。因此，如果 $\mathbf{p}_{ij}$ 是粒子网格第 i 行、第 j 列的某个粒子，那么要计算 $\mathbf{p}_{ij}$ 与其他粒子之间的相互作用力，只需考虑它与 $\mathbf{p}_{i+1,j}$, $\mathbf{p}_{i-1,j}$, $\mathbf{p}_{i,j+1}$, $\mathbf{p}_{i,j-1}$ 4 个粒子之间的相互作用力，其时间复杂度为 $O(n)$。

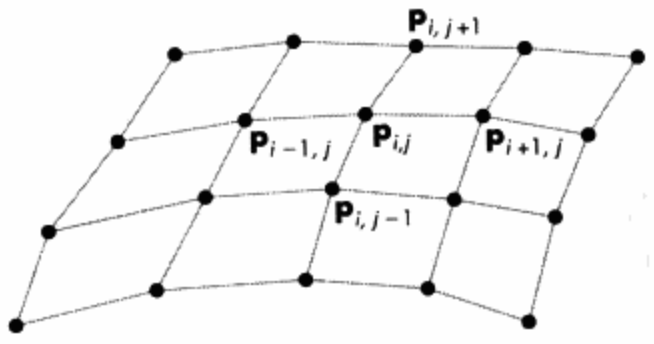


图 11.1　粒子网格

有一种方法用来模拟粒子之间的相互作用力，即认为两个相邻的粒子是用弹簧连接的。如图 11.2 所示，我们分析分别位于 $\mathbf{p}$ 和 $\mathbf{q}$ 的两个相邻的粒子，它们用一根弹簧连接起来。假设 $\mathbf{f}$ 表示 $\mathbf{q}$ 作用于 $\mathbf{p}$ 的力，那么 $-\mathbf{f}$ 表示 $\mathbf{p}$ 作用于 $\mathbf{q}$ 的力。弹簧的静止长度为 s，它表示弹簧在不受外力并且处于静止状态时两个粒子之间的距离。当弹簧被拉伸时，作用在弹簧上的弹力方向为 $\mathbf{d} = \mathbf{p} - \mathbf{q}$，也就是沿着两个粒子之间的直线方向。这个弹力遵循胡克定律（Hooke's law）：

$$\mathbf{f} = -k_s(|\mathbf{d}| - s)\frac{\mathbf{d}}{|\mathbf{d}|}$$

其中，k_s是弹簧系数，它是一个常量，s是弹簧处于静止状态时的长度。胡克定律表明，两个粒子被拉伸开的距离越大，那么把粒子拉回平衡点的力也就越大。相反，当弹簧的两个端点向中间挤压时，那么这两个端点在弹力的作用下要分开，并且其距离要恢复到静止长度。但是，这里介绍的胡克定律没有考虑系统中的阻尼力（或摩擦力）。这种没有阻尼力的弹簧－质点系统在受到扰动时就会一致振动下去。我们可以在胡克定律中增加一个阻力或阻尼项。阻尼力和弹力作用在同一方向上，但是其大小依赖于粒子之间的振动速度。如图11.3所示，粒子之间对阻尼力有影响的一小部分振荡速度所占的比例正比于速度矢量在两个粒子之间连线上的投影分量。可以使用下面的数学公式表示带阻尼项的胡克定律：

$$\mathbf{f} = -\left(k_s(|\mathbf{d}| - s) + k_d\frac{\dot{\mathbf{d}}\cdot\mathbf{d}}{|\mathbf{d}|}\right)\frac{\mathbf{d}}{|\mathbf{d}|}$$

其中，k_d是阻尼系数，它是一个常量，粒子之间的振动速度为：

$$\dot{\mathbf{d}} = \dot{\mathbf{p}} - \dot{\mathbf{q}}$$

一个具有阻尼项的弹簧－质点系统在没有外力作用的情况下最终会恢复到静止状态。

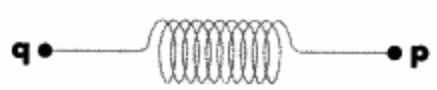


图11.2　由弹簧连接的两个粒子

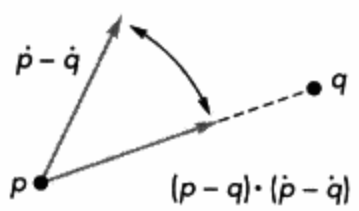


图11.3　计算弹簧阻尼力

彩图35中的4幅图像显示了由大量粒子生成的网格。网格内部的每个粒子通过弹簧与其周围的4个粒子相连。这些粒子还受到外部风力的作用。在粒子系统的每个生命周期中，一旦确定了粒子的位置，就可以使用纹理映射（见7.5节）等技术绘制对象的网格，从而显示对象表面的外观细节。

11.3.3　吸引力与排斥力

使用弹簧的弹力可以将一群粒子聚集在一起，而排斥力可以使粒子相互分开，吸引力可以使粒子相互靠拢。我们可以使用排斥力使粒子分散在物体的表面上，或者，如果粒子表示对象的位置，那么可以使用排斥力来使对象避免相互碰撞。我们可以使用吸引力来建立太阳系模型，或建立用来模拟卫星绕地球旋转的应用程序。除了符号区别之外，吸引力和排斥力的数学方程在本质上是相同的。粒子行为的物理模型可能同时包含吸引力和排斥力（见习题11.14）。

假设两个粒子分别位于位置$\mathbf{p}$和$\mathbf{q}$，排斥力作用在$\mathbf{d} = \mathbf{p} - \mathbf{q}$的方向上，并且其大小与粒子之间的距离成反比。例如，可以使用下面的表达式计算满足距离平方反比定律（inverse-square-law）的排斥力：

$$\mathbf{f} = -k_r\frac{\mathbf{d}}{|\mathbf{d}|^3}$$

只要把前面的负号变成正号，并且用$(m_a m_b)/g$代替k_r，就可以得到质量分别为m_a和m_b的两个粒子间的吸引力，其中g为重力系数，它是一个常量。

在更一般的情况下，系统中的每个粒子都受到所有其他粒子的作用力，计算吸引力和排斥力的时间复杂度为 $O(n^2)$。在使用弹簧连接的粒子网格中，粒子的拓扑结构一般保持不变，与这种粒子网格不同，受到吸引力和排斥力作用的粒子通常会改变它们的相对位置。因此，要使吸引力和排斥力的计算复杂度不为 $O(n^2)$，所使用的方法将会更复杂。一种方法是把空间分割为许多小的三维单元格，每个单元格包含多个粒子，也可能不包含任何粒子，如图 11.4 所示。

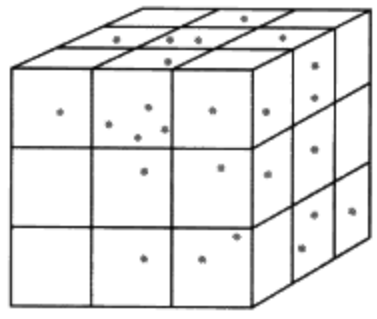


图 11.4　把空间分割成小的单元格

对于那些与距离平方成反比的作用力，可以选择合适大小的单元格，使得其他单元格或其周围单元格的粒子对该单元格中粒子的作用力可以忽略。如果这种分割可行，那么 $O(n^2)$的计算复杂度可以降低到 $O(n)$。然而，这种分割本身需要时间开销，并且粒子会从一个单元格移动到另一个单元格。第一个问题的难度取决于特定的应用，一般不费多大开销就可以得到粒子的初始分布，但是有时必须进行排序处理。解决第二个问题采用的方法是：在粒子系统的每个时间步之后[①]，检查粒子的新位置并重新分布粒子，也许需要重新分割空间，从而改变单元格的大小。我们可以使用各种数据结构来存储粒子和单元格的信息。

另一种常用的方法是使用两个粒子之间的相互作用和力场来代替系统中粒子的相互作用。例如，当计算地球表面上某个粒子的质点所受的重力时，我们使用了重力场的值，而并没有计算粒子的质点与地球中心的某个巨大质点之间的吸引力。如果只考虑地球的质量以及地球表面的质点，那么这两种方法需要相同的计算开销。然而，如果还要考虑月球对质点的吸引力，那么情况将会变得更复杂。如果使用质点来计算，那么对于地球表面的某个质点，需要两次计算质点到质点的作用力。但是如果知道重力场，那么质点的受力计算与前面一样。当然，如果考虑月球的作用力，重力场的计算也会变得更复杂。然而对于粒子系统，通常可以忽略较远的粒子，因此这种粒子－力场方法的效率可能会更高。我们经常要计算一个网格上的近似力场，然后使用离每个粒子最近的网格点上的力场值作为这些粒子的受力。当计算每个粒子的新状态后，我们可以更新力场。通常这两种方法都能够将粒子受力计算的复杂度从 $O(n^2)$降低到 $O(n \log n)$。

11.4　求解粒子系统方程

下面考虑一个具有 n 个粒子的粒子系统。如果把粒子的受力情况限于前面介绍的简单情况，那么可以使用下面形式的 $6n$ 个常微分方程来描述整个粒子系统：

$$\dot{\mathbf{u}} = \mathbf{g}(\mathbf{u}, t)$$

其中，$\mathbf{u}$ 是存储 n 个粒子的位置或速度的数组，它包含 $6n$ 个元素；$\mathbf{g}$ 包含所有作用在粒子上的外力。因此，如果有粒子 $\mathbf{a}$ 和 $\mathbf{b}$，它们通过一根无阻尼弹簧相连，那么有：

$$\begin{aligned}\mathbf{u}^T &= [\,u_0 \quad u_1 \quad u_2 \quad u_3 \quad u_4 \quad u_5 \quad u_6 \quad u_7 \quad u_8 \quad u_9 \quad u_{10} \quad u_{11}\,] \\ &= [\,a_x \quad a_y \quad a_z \quad \dot{a}_x \quad \dot{a}_y \quad \dot{a}_z \quad b_x \quad b_y \quad b_z \quad \dot{b}_x \quad \dot{b}_y \quad \dot{b}_z\,]\end{aligned}$$

① 即一次仿真循环结束后。——译者注

假设已知外力和粒子系统在任何时刻 t 时的状态 $\mathbf{u}$，那么可以计算：

$$\mathbf{g}^T = \begin{bmatrix} u_3 & u_4 & u_5 & -kd_x & -kd_y & -kd_z & u_9 & u_{10} & u_{11} & kd_x & kd_y & kd_z \end{bmatrix}$$

其中，k 是弹簧系数常量，d_x、d_y 和 d_z 是 $\mathbf{a}$ 和 $\mathbf{b}$ 之间距离 d 的单位矢量的3个分量。因此，必须先计算 d：

$$\mathbf{d} = \frac{1}{\sqrt{(u_0 - u_5)^2 + (u_1 - u_6)^2 + (u_2 - u_7)^2}} \begin{bmatrix} u_0 - u_6 \\ u_1 - u_7 \\ u_2 - u_8 \end{bmatrix}$$

数值常微分方程的求解依赖于通过对 $\mathbf{g}$ 的计算来逼近下一时刻的 $\mathbf{u}$。可以根据泰勒定理把它展开成对一系列微分方程的求解，其中最简单的称为欧拉方法。假定对下面的式子

$$\dot{\mathbf{u}} = \mathbf{g}(\mathbf{u}, t)$$

在一个很小的时间间隔 h 上求积分：

$$\int_t^{t+h} \dot{\mathbf{u}} \mathrm{d}\tau = \mathbf{u}(t+h) - \mathbf{u}(t) = \int_t^{t+h} \mathbf{g}(\mathbf{u}, \tau) \mathrm{d}\tau$$

如果 h 是一个很小的值，那么可以使用 $\mathbf{g}$ 在时刻 t 的值代替它在区间 $[t, t+h]$ 上的积分值，因此，

$$\mathbf{u}(t+h) \approx \mathbf{u}(t) + h\mathbf{g}(\mathbf{u}(t), t)$$

这个公式说明，可以利用在 t 时刻的导数值来计算 $\mathbf{u}(t+h)$ 的近似值，如图11.5所示。

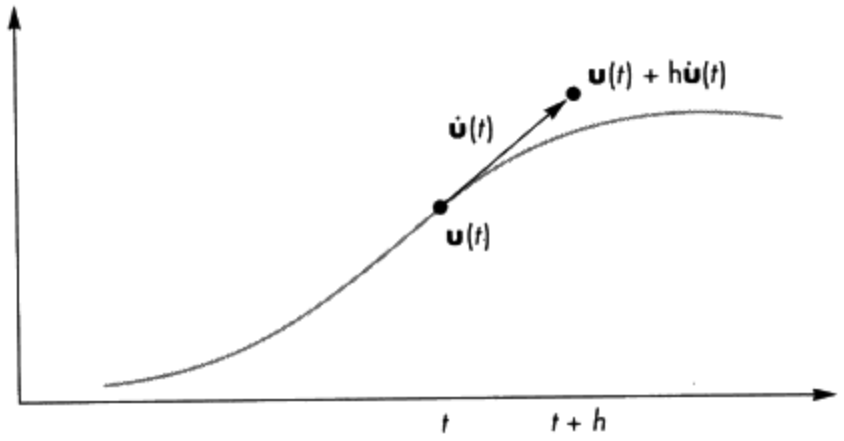


图11.5　微分方程的近似求解

上述公式与泰勒展开式的前两项正好匹配，可以把该式写为：

$$\mathbf{u}(t+h) = \mathbf{u}(t) + h\dot{\mathbf{u}}(t) + O(h^2) = \mathbf{u}(t) + h\mathbf{g}(\mathbf{u}(t), t) + O(h^2)$$

这个公式说明了近似计算带来的误差与步长的平方成正比。

这个方法很容易实现。我们计算 t 时刻粒子的受力（外力和粒子之间的相互作用力），计算 $\mathbf{g}$ 的值，然后把它与 h 相乘，再把结果与当前的状态值相加。我们可以使用该方法不断迭代计算在 $t+2h$，$t+3h$，…时刻的值。上述迭代过程对每个时间步长只要为粒子计算一次受力即可。

欧拉法存在两个潜在的问题：精确性和稳定性，它们都受步长的影响。欧拉法的精确性与步长的平方成正比。因此，要提高精确性就必须减小步长，而这又会增加求解系统方程的时间开销。另一个潜在的更为严重的问题是稳定性。当一步一步迭代计算时，每一步的误差来自两个方面：一个是使用泰勒级数产生的近似误差，另一个是在计算函数时产生的数值误差。当计算后继的状态时，这些误差要么互相抵消，要么累积起来使误差大到无法接受的程度，以至于最终无法得到正确的解。

我们把这种行为称为数值不稳定性（numerical instability）。幸运的是，对于前面使用的这种标准类型的作用力，如果把步长取得足够小，那么就可以确保稳定性。但遗憾的是，用于确保稳定性所取的步长也许足够小以至于无法在合理的时间内求解数值方程。对于弹簧-质点系统来说，这种不稳定性行为尤为显著，这是因为在弹簧-质点系统中，弹簧系数常量决定了系统的刚度，从而导致了所谓的病态微分方程[①]。

有两种通用的方法可以解决该问题。一种方法是寻求另一种常微分方程求解方法，我们称之为病态方程（stiff equation）求解方法，这个话题已经超出了本书的范畴。另一种方法是寻找其他的微分方程求解方法，使用这种方法求解微分方程的思想与欧拉法相似，但是每一步计算都将采用更高的精确度。下面推导其中的一种方法，因为这样可以使读者深入理解这一类微分方程的求解方法。我们将在本章后面的建议阅读资料中给出这两种方法的参考文献。

和前面一样，假定先在一个很小的时间区间内求微分方程的积分，即

$$\mathbf{u}(t+h) = \mathbf{u}(t) + \int_t^{t+h} \mathbf{g}(\mathbf{u}, \tau)\mathrm{d}\tau$$

我们这次使用区间$[t, t+h]$上的平均值来近似表示上式的积分值，即

$$\int_t^{t+h} \mathbf{g}(\mathbf{u}, \tau)\mathrm{d}\tau \approx \frac{h}{2}(\mathbf{g}(\mathbf{u}(t), t) + \mathbf{g}(\mathbf{u}(t+h), t+h))$$

现在的问题是：我们并不知道$\mathbf{g}(\mathbf{u}(t+h), t+h)$的值，而只知道$\mathbf{g}(\mathbf{u}(t), t)$的值。我们可以使用欧拉法求解$\mathbf{g}(\mathbf{u}(t+h), t+h)$的近似值，也就是利用下面的公式：

$$\mathbf{g}(\mathbf{u}(t+h), t+h) \approx \mathbf{g}(\mathbf{u}(t) + h\mathbf{g}(\mathbf{u}(t), t), t+h)$$

该方法称为改进的欧拉法（improved Euler method）或二阶龙格-库塔法（Runge-Kutta method of order 2）。注意，从t到$t+h$，我们需要对$\mathbf{g}$的值计算两遍。然而，如果使用泰勒公式估计每一步的误差，那么发现误差现在为$O(h^3)$。因此，尽管使用这种方法时每一步的计算量增加了，但是却可以使用更长的步长，当它使用的步长比稳定的欧拉法使用的步长还要大时，该方法仍是稳定的。一般来说，可以按渐增的方式提高每一步的精确度，从而推导出一套方法，称为龙格-库塔方程式。最常用的是四阶龙格-库塔方法，它的每一步的误差为$O(h^4)$，并且需要4次函数计算。在实际中，我们根据这4次函数计算可以得到精确度更高的结果，其误差是$O(h^5)$。更重要的是：对于一个好的微分方程数值求解方法，为了确保数值稳定性，它应该能够动态地调节步长的大小。

11.5　约束条件

前面仅仅根据一组微分方程来改变一群粒子的状态，这对于模拟真实世界中的碰撞等行为还是不够的。虽然一个对象碰撞墙面时的撞击力遵循牛顿定律，但是如果用一群粒子表示该对象，那么最终实现的系统对大多数应用来说太复杂了。在实际中，我们可以把一些条件（比如两个实体不能互相穿透）视为约束条件，这些约束条件独立于那些决定每个粒子行为的物理定律。

我们可以把两类约束强加于粒子。一类是硬约束（hard constraint），是指那些必须严格满足的条件。例如，一个球碰撞墙面后一定会弹回来，它不可能穿墙而过。我们也不允许球仅仅接近墙面，然后就从另一个方向离开。另一类是软约束（soft constraint），是指那些只需近似满足即可的条件。例如，我们希望粒子网格上两个粒子之间的距离近似为某个指定的距离即可。

① 即前面介绍的数值不稳定性。——译者注

11.5.1　碰撞

一般来说，虽然把硬约束强加于粒子比较困难，但是在某些情况下可以对理想的质点粒子直接进行处理。下面分析一个碰撞问题。我们可以把这个问题分为两部分：碰撞检测和碰撞响应。假定有一群粒子和一些其他的几何对象，并且粒子之间相互排斥，因此只需考虑粒子与其他几何对象之间的碰撞。如果有n个粒子和m个用于定义几何对象的多边形，那么在仿真循环的每一时间步都要检查是否有粒子穿过多边形。

如图11.6所示，假设有一个粒子穿过了某个多边形。我们可以检测到粒子与多边形的碰撞，方法是把粒子的坐标值代入多边形所在的平面方程。如果求解微分方程所用的时间步长很小，那么可以认为在一个时间步长内，粒子的速度是一个常量。因此，可以使用线性插值的方法来计算粒子与多边形实际发生碰撞的时间。

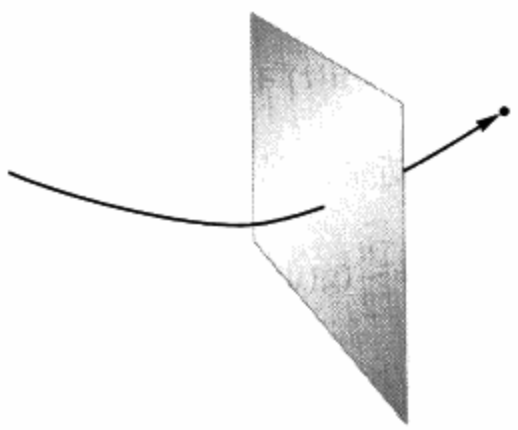


图11.6　穿过多边形的粒子

粒子与多边形发生碰撞后的情形类似于光线在对象表面的反射。如果发生弹性碰撞（elastic collision），那么粒子的能量没有任何损失，因此它的速度也不会改变。然而，碰撞后粒子的方向与光线的理想镜面反射方向相同。因此，如图11.7所示，如果已知碰撞点$\mathbf{P}_c$的法向量和粒子碰撞之前的位置$\mathbf{P}_0$，那么类似于在第6章使用的方法，可以使用粒子到表面的矢量以及表面的法向量来计算理想镜面反射的方向，即粒子碰撞后的方向$\mathbf{r}$：

$$\mathbf{r} = 2(\mathbf{p}_0 - \mathbf{p}_c) \cdot \mathbf{n}\ \mathbf{n} - (\mathbf{P}_0 - \mathbf{P}_c)$$

如图11.8所示，粒子沿反射方向运动的距离与粒子在没有碰撞检测的情况下直接穿过多边形后运动的距离相等。

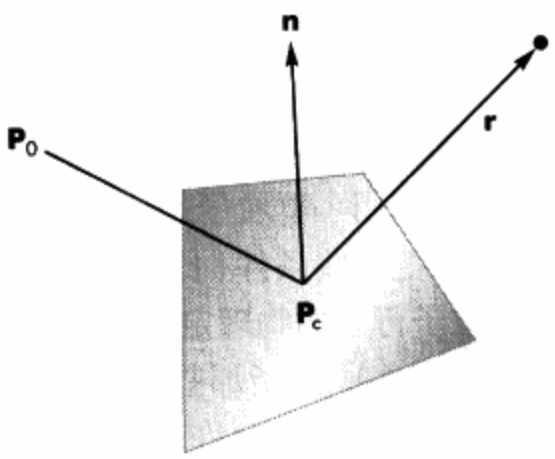


图11.7　粒子反射

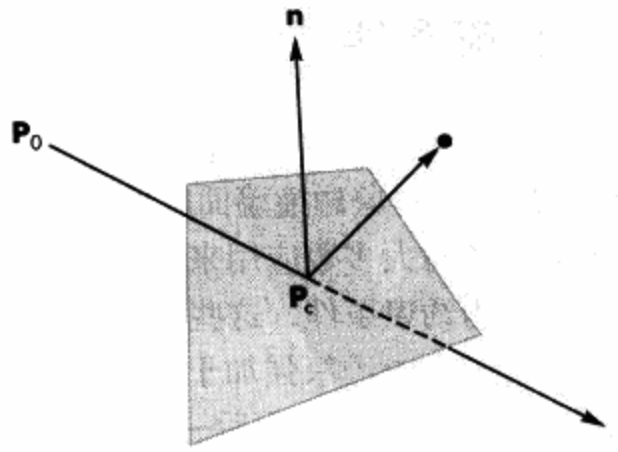


图11.8　粒子碰撞后的位置

碰撞后粒子速度的大小不变，但是其方向变为反射方向。或者换句话说，粒子切向速度分量的方向（即沿多边形表面方向的分量）不变，而其法向速度分量的方向正好与原来的相反。

除了弹性碰撞之外，还有一种计算稍复杂的非弹性碰撞（inelastic collision），粒子与其他的几何对象发生这种碰撞后会损失一部分能量。一个粒子的恢复因子（coefficient of restitution）是指粒子碰撞后的法向速度分量与碰撞之前的法向速度分量的比值，因此需要计算非弹性碰撞的反射角，而且粒子碰撞后，其法向速度分量的减小量由恢复因子决定。请参见 11.6.4 节的非弹性碰撞示例。

处理碰撞的最大开销就是碰撞检测。在一些游戏应用中，使用近似的碰撞检测一般就足够了。在这种情况下，可以使用包围盒这样的简单对象代替由大量多边形构成的复杂对象，这样可以简化碰撞检测。

我们注意到使用粒子系统可以使大小有限的对象避免所需的复杂计算。在许多方面，解决碰撞问题有点类似于两个任意对象之间的相互裁剪问题，从概念上讲，其计算过程非常简单，但实际上非常耗时且麻烦。此外，如果要处理的对象大小有限，那么还必须考虑惯性力，因而增加了系统方程组的维数，因此还必须克服高维方程组引起的计算困难。因此，在计算机图形学中，通常使用理想的质点粒子，从而获得一个近似的解；另外，我们通常把对象放在粒子所在的位置进行绘制，从而获得比较令人满意的结果。

还有一个经常出现并且能够被正确处理的硬约束就是接触力。如图 11.9 所示，假定有这样一个粒子，它受到一个使它沿表面前进的推力的作用。这个粒子既不能穿过表面，而且由于这个作用在它上面的推力，它也不能从表面弹回来。可以证明，这个粒子受到切向推力分量的作用，即沿表面方向的推力分量。我们还可以使粒子在这个方向受到摩擦力的作用。

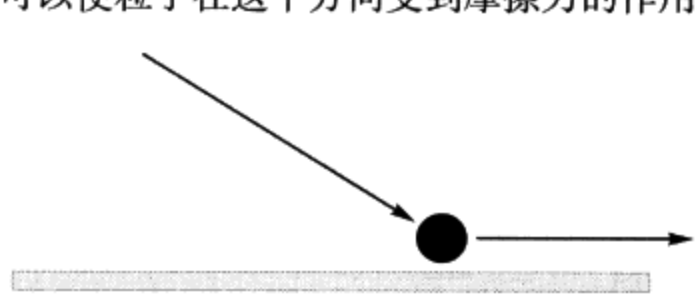


图 11.9　接触力

我们注意到碰撞检测的时间复杂度通常是 $O(n^2)$，而一旦检测到碰撞，碰撞响应程序一般并没有这么复杂。例如，考虑撞球游戏，球可以在桌子上四处运动；或者再考虑一个模拟分子运动的系统，分子在一个有界容器中运动。由于任何两个粒子都可能发生碰撞，所以可以在仿真循环的每个时间步内采用蛮力方法检查所有粒子之间是否会碰撞。可以利用包围盒测试法以及硬件对碰撞检测的支持来设计快速的碰撞检测算法。

11.5.2　软约束

大多数硬约束条件都难以实施。例如，如果希望一个粒子的速度不超过某个最大值，或者要求所有粒子的能量都保持不变，那么实施这些约束条件的数学方程远比前面介绍的方程要复杂得多，并且这样的约束条件并总能得到一组简单的常微分方程。

在许多情况下可以使用软约束，也就是那些只需近似满足的条件。例如，如果要判断位置 $\mathbf{p}$ 处的一个粒子是否处在位置 $\mathbf{p}_0$ 的附近，那么可以使用补偿函数（penalty function）$|\mathbf{p}-\mathbf{p}_0|^2$。这个函数的值越小，就越接近约束条件。这个函数是能量函数（energy function）的一个例子，能量函数的值表示存储在系统中某种形式的能量值。在物理学中，能量函数可以表示系统中势能或动能的大小。物理定律可以写成微分方程的形式，比如前面用于描述粒子行为的微分方程，也可写成最小化能量函数的形式。采用后一种形式的一个优点是：可以直接使用势能或动能函数的形式表示约束条

件或系统的预期行为。把这些能量表达式转换成力学公式是一个机械的过程，但是它涉及的数学细节超出了本书讨论的范畴。

彩图31说明了粒子系统在科学计算可视化领域中的应用。在这幅彩图示例中，粒子被添入到一个三维数据集中，每个粒子都试图寻找其所在的等势面。粒子根据约束条件寻找相应的等势面，然而排斥力使位于同一等势面上的粒子分开。

11.6　一个简单的子系统

下面构建一个可扩展为更复杂行为的简单粒子系统，并以此结束我们开发的子系统。我们使用的都是牛顿粒子，因此可以利用位置和速度来表示粒子的状态。此外，每个粒子都有自己的颜色和质量。首先给出粒子的结构：

```
typedef struct particle
{
     int color;
     float position[3];
     float velocity[3];
     float mass;
} particle;
```

粒子系统是一个由粒子组成的数组：

```
particle particles[MAX_NUM_PARTICLES];
```

11.6.1　绘制粒子

如果知道了粒子的位置，那么就可以使用你喜欢的任何图元（几何图元或光栅图元）绘制它。最简单的情况是把每个粒子绘制成一个点。下面是显示回调函数，该函数通过循环遍历粒子数组的每个元素来绘制所有的粒子：

```
void myDisplay()
{
    int i;

    glClear(GL_COLOR_BUFFER_BIT);
    glBegin(GL_POINTS);
    for(i=0; i<num_particles; i++)

       glColor3fv(colors[particles[i].color]);
       glVertex3fv(particles[i].position);

    glEnd();
}
```

粒子的颜色存储在如下形式的数组colors中：

```
GLfloat colors[8][3]={{0.0, 0.0, 0.0}, {1.0, 0.0, 0.0}, {0.0, 1.0, 0.0},
    {0.0, 0.0, 1.0}, {0.0, 1.0, 1.0}, {1.0, 0.0, 1.0}, {1.0, 1.0, 0.0},
    {1.0, 1.0, 1.0}};
```

11.6.2　更新粒子的位置

我们使用GLUT库的空闲回调函数更新粒子的位置，其具体实现用到了程序的执行时间和欧拉积分：

```
float last_time, present_time;
float num_particles;

void myIdle()
{
    int i, j;
    float dt;
    present_time = glutGet(GLUT_ELAPSED_TIME); /* 单位为毫秒 */
    dt = 0.001*(present_time -  last_time); /* 单位为秒 */
    for(i=0; i<num_particles; i++)
    {
       for(j=0; j<3; j++)
       {
           particles[i].position[j]+=dt*particles[i].velocity[j];
           particles[i].velocity[j]+=dt*forces(i,j)/particles[i].mass;
        }
        collision(i);
    }
    last_time = present_time;
    glutPostRedisplay();
}
```

我们使用粒子的速度来更新它的位置，并且通过计算作用在粒子上的力来更新它的速度。假定时间间隔足够短，以至于更新粒子状态时能够计算作用在每个粒子上的力。一种更稳健的方法是通过预处理计算所有粒子的受力，并把结果存放在一个数组中，我们可以直接使用这个数组来更新粒子的状态。

我们使用collision函数来使粒子始终位于盒子里面，也可以使用这个函数来处理粒子之间的碰撞。

11.6.3　初始化

首先创建默认数量的粒子，这些粒子随机地位于一个以坐标原点为中心、边长等于2的立方体内部，并且给每个粒子赋予一个随机的速度。我们使用define语句定义粒子的初始数量和默认速度，不过可以通过菜单修改粒子的数量和速度：

```
float num_particles = INITIAL_NUM_PARTICLES;
float speed = INITIAL_SPEED;

void myinit()
{
     int  i, j;
     for(i=0; i<num_particles; i++)
     {
         particles[i].mass = 1.0; /* 默认的粒子质量 */
         particles[i].color = i%8; /* 颜色数组中存放了 8 种颜色 */
         for(j=0; j<3; j++)
         {
             particles[i].position[j] = 2.0*((float) random()/
                RAND_MAX)-1.0;
             particles[i].velocity[j] = speed*2.0*((float) random()/
                RAND_MAX)-1.0;
         }
     }
}
```

11.6.4　碰撞

我们使用collision函数使粒子始终位于最初与坐标轴对齐的盒子里面。采用的方法是：先增加每个粒子的位置，然后检查粒子是否与盒子的边界碰撞。如果粒子与盒子的边界碰撞，那么把粒子的反弹视为理想的反射。因此，只需在法向量方向改变速度的正负号，并且使粒子位于盒子的另一端。如果粒子的恢复因子小于1.0，那么当粒子与盒子的边界碰撞时，它们的速度会慢下来：

```
float coef; /* 粒子的恢复因子 */

void collision(int n)
{
   int i;
   for (i=0; i<3; i++)
   {
      if(particles[n].position[i]>=1.0)
      {
         particles[n].velocity[i] = -coef*particles[n].velocity[i];
         particles[n].position[i] =
                                 1.0-coef*(particles[n].position[i]-1.0);
      }
        if(particles[n].position[i]<=-1.0)
      {
        particles[n].velocity[i] = -coef*particles[n].velocity[i];
        particles[n].position[i] = -1.0-
                                   coef*(particles[n].position[i]+1.0);
      }
   }
}
```

11.6.5　作用力

如果作用在粒子上的力为零，那么粒子将在盒子里面不断地按直线路径来回反弹。如果粒子的恢复因子小于1.0，那么粒子最终会停下来。最容易想到的就是作用在粒子上的重力。例如，如果所有的粒子都具有相同的质量，那么可以在y方向上增加一个重力项，代码如下：

```
bool gravity =  TRUE;

float forces(int i, int j)
{
   if(!gravity) return (0.0);
     else if(j==1) return(-1.0);
      else return(0.0);
}
```

我们还可以增加作用在粒子上的排斥力或吸引力（见习题11.14），方法是在每次迭代的开始计算所有粒子之间的距离，然后使用距离平方反比定律计算排斥力或吸引力。本章后面的习题对该系统的扩展提出了一些建议。

11.6.6　群聚行为

粒子系统中最有趣的一类应用就是模拟粒子群的复杂行为。也许更准确的说法是：可以使用粒子相互作用的简单规则来生成看起来非常复杂的行为。典型的实例就是模拟鸟的群聚行为。在每

只鸟不知道所有其他鸟的位置的情况下，一大群鸟是如何维持这个群体的呢？我们对这个简单的粒子系统做一些小的改动，并由此来研究实现这种群聚行为的可能性。

一种方法是改变每个粒子的方向，使粒子趋向群体的中心位置。因此，每当更新系统时，都要计算粒子的平均位置，代码如下：

```
float cm[3];
for (k=0; k<3; k++)
{
   cm[k]=0;
   for(i=0;i<num_particles;i++) cm[k]+=particles[i].postition[k];
   cm[k]/=num_particles;
}
```

如图 11.10 所示，现在可以计算速度矢量的新方向，它位于更新后的速度矢量 particles[i].velocity 和粒子位置 particles[i].position 到平均位置形成的矢量之间（见习题 11.19）。

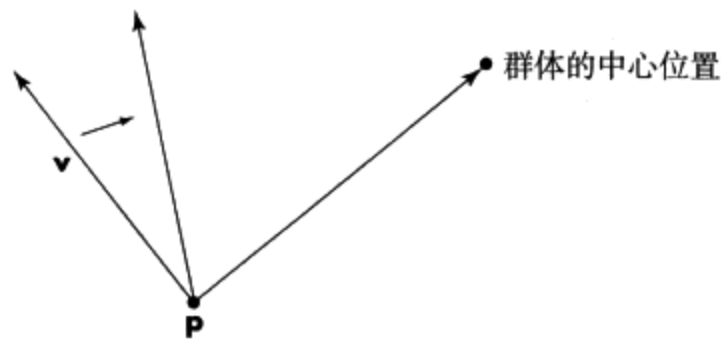


图 11.10　改变粒子的方向

11.7　基于语言的模型

类似第10章中介绍的树和有向无环图（DAG），这类图形只能提供一种表示对象之间层级关系的方式。本节要寻找基于语言的模型来表示关系。这些方法不仅能提供表示关系的替换方式，而且能引出描述像树木和地形这些对象的过程方法。

如果考察自然界的对象，例如树木，我们会发现虽然没有两棵树是相同的，但是我们可以毫不费力地说出两种树之间的差异。虽然学者们已经提出程序在每次运行时给出不同实现的各种方法，但是这些方法有清楚的规则来定义结构。我们考虑使用树形数据结构来生成类似树木的对象。

在计算机科学中，树形数据结构用于描述语句中的语法成分，计算机和自然语言都是如此。对于计算机程序，进行这种语法分析是编译计算机程序语句的一部分。对于自然语言，我们分析语句来确定它们在语法上是否正确。从正确语句语法分析得出的树给出那个语句的结构或语法，树中各个元素（即单词）的解释给出那个语句的意义或语义。

如果仅考察一个语言的语法，就会发现在语言规则和用来表示这种语言语句的树的形状之间存在直接的关联。我们可以把这个想法扩充到绘制具有层级结构的图形对象，把一组规则和树结构模型关联起来，这种系统被称为树文法（tree grammar）。一个文法可以用一组符号和一组符号替换规则或产生式（production）来定义，产生式确定如何用一个或多个符号来替换一个符号，典型规则写成如下形式：

$$A \rightarrow BC$$
$$B \rightarrow ABA$$

一旦给定一组产生式，就能生成无限长度的字符串。一般来说，任何时候都有一个以上的规则可以应用于一个给定的符号，而且，如果随机选取应用的规则，那么每次执行程序时都能生成不同的字

符串。我们不仅可以编写产生这种字符串的程序，而且可以把这些字符串作为输入并测试这些字符串是否是由一组给定规则所生成的字符串集合中的有效成员。这样就可以得到用于生成某个特定类型的对象的一组规则，例如一棵树或一丛灌木，而且，我们也可以根据生成对象外部形状的文法来编写识别场景对象的独立程序。

对字符串中符号的解释把字符串转换成一个图形对象。有许多方法可以用来生成规则并把得到的字符串解释成图形对象。一种方法来自海龟图形系统（turtle graphics system）（见习题2.4）。在海龟图形系统中，可以使用3种基本方式来处理图形光标或海龟（turtle）。海龟可以向前移动1个单元，向右或向左转。假定海龟的转向角度是固定的，则可以将这3种操作表示为F（前）、R（右）和L（左）。由这些操作形成的任何字符串都有一个简单的图形解释。例如，如果转向角度是120°，那么字符串FRFRFR产生一个等边三角形。我们使用两个特殊的符号[和]来表示海龟状态（海龟的位置和走向）的压栈和出栈操作（等价于使用括号的操作）。考虑一条产生式规则

$$F \rightarrow FLFRRFLF$$

且转向角度为60°。这条规则的图形解释如图11.11所示。如果同时将此规则应用到F的所有情况，就可以得到如图11.12(a)所示的曲线；如果把它应用到一个三角形，就能得出如图11.12(b)所示的封闭曲线。这两种曲线分别被称为Koch曲线（Koch curve）和Koch雪花（Koch snowflake）。如果每次执行算法时都按比例改变曲线的几何外形，使原始顶点的位置不变，则会发现在每次迭代中我们正在生成一条更长的曲线，但是这条曲线总是位于同一个盒子的内部。在极限情况下，我们有了一条具有无限长度的曲线，它本身永远不会相交，但位于一个有限大小的盒子中。这条曲线也是连续的，但它的导数处处不连续。

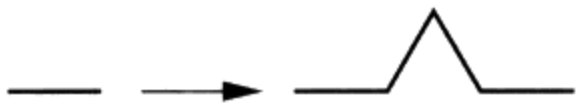


图11.11　Koch曲线规则

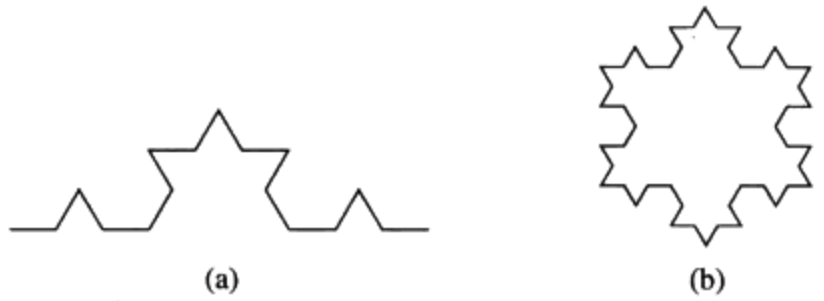


图11.12　空间充填曲线：(a) Koch曲线；(b) Koch雪花

另一个经典例子是Hilbert曲线（Hilbert curve），该曲线由4个简单的原语形成，这4个原语可分别称为A_0、B_0、C_0和D_0，如图11.13所示，每个都是一阶Hilbert曲线，图中的箭头指明了在一个角落画每个箭头的开始地方。每个N阶Hilbert曲线都有4种类型，可称为A_N、B_N、C_N和D_N。按照下面的规则，通过结合这4类Hilbert曲线，可以从$N-1$阶曲线形成N阶曲线：

$$\begin{aligned}
A_N &= B_{N-1} \uparrow A_{N-1} \rightarrow A_{N-1} \downarrow C_{N-1} \\
B_N &= A_{N-1} \rightarrow B_{N-1} \uparrow B_{N-1} \leftarrow D_{N-1} \\
C_N &= D_{N-1} \leftarrow C_{N-1} \downarrow C_{N-1} \rightarrow A_{N-1} \\
D_N &= C_{N-1} \downarrow D_{N-1} \leftarrow D_{N-1} \uparrow B_{N-1}
\end{aligned}$$

这些规则可解释为第N阶模型是由特定方向的$N-1$阶的4类模型的组合形成的。从图11.14所示的一阶曲线A_1可以看出，必须使用对应于公式中箭头的链接来连接模型。还要注意每个模型从不同的角落开始，当画曲线的时候，若不考虑箭头和链接，可得出如图11.15所示的曲线。

当画更高阶曲线时，如果按比例缩小链路的长度，那么就像Koch曲线一样，可以证明，当Hilbert曲线变得愈来愈长时，本身永远不相交，而且总是位于同一个盒子的内部。在极限情况下，Hilbert曲线填充盒子内部的每个点，我们把这样的Hilbert曲线称为空间填充曲线（space-filling curve）。

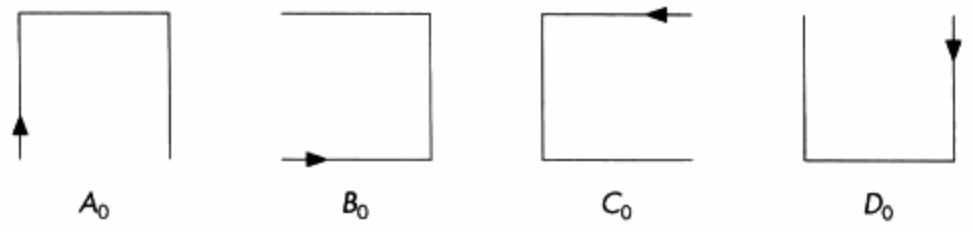


图 11.13　0 阶 Hilbert 模型

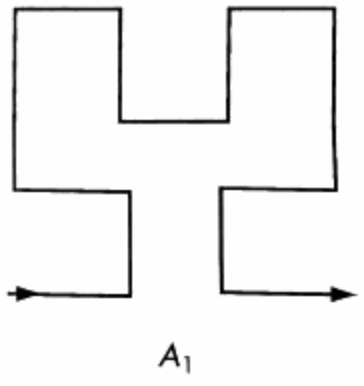


图 11.14　类型 A 的 Hilbert 规则

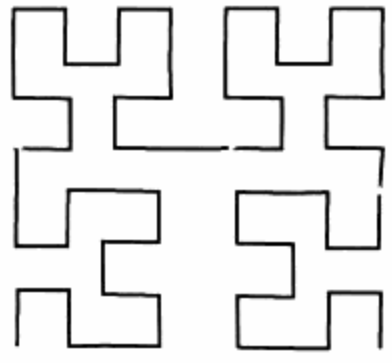


图 11.15　二阶 Hilbert 曲线

我们可以使用压栈和出栈操作符开发侧面分支，考虑规则

$$F \rightarrow F[RF]F[LF]F$$

且转向角度为 27°（如图 11.16 所示）。注意，我们开始于底部，转向角度表示相对于向前方向的左偏角度或右偏角度。

如果从单条线段开始应用规则，则结果对象如图 11.17 所示。我们可用若干方式继续进行，一种方法是再将规则应用到序列中的每个 F 中，得出如图 11.18 所示的对象。我们也可以调整对应于乌龟向前移动的长度使得在后续迭代中分支变得较小。该对象类似一丛灌木，如果迭代更多次数时就更像一丛灌木了。然而，如果只有一条规则并且以并行的方式应用该规则，那么这将导致每丛灌木看上去都差不多。

图 11.16　规则 $F \rightarrow F[RF]F[LF]F$

图 11.17　图 11.16 中规则的第二次迭代

一个更有意义的策略是将规则随机应用于 F 的出现，如果这样做，单个规则就可以生成如图 11.18[①]所示的两个对象。如果增加更多的产生式并对用于确定下一步哪条规则将被应用的概率函数进行控制，那么就容许用户生成各种类型的树。只要稍微修改，也可以在分支末端画出树叶。

这个策略的一个诱人之处是仅基于少量的规则和若干参数就定义了一类对象。假定我们想创建一组树。一种直接方法就是生成所需的大量对象，把每棵树表示成一组几何对象（如直线、多边形、曲线等图元）。在一个复杂场景中，由于所生成图元的数量巨大而使图形绘制系统不堪重负。可

① 原文为如图 11.19，可能有误。——译者注

是由于可视条件的限制，其中大部分图元并不会出现在最终绘制的图像中，这是因为它们被裁剪掉了或离视点太远而无法以可见的大小绘制出来。相反，如果使用过程建模的方法，就可以用简单的算法描述对象，并且只有在需要时才把几何对象绘制出来，而且可以把几何对象绘制到所需要的任意细节层次。

我们也可以根据形状和仿射变换来直接描述文法，即创建形状文法（shape grammar）。下面考虑老朋友 Sierpinski 镂垫。我们可以根据 3 个仿射变换来定义细分步骤，每一步都把初始三角形按比例缩小到原来的一半，并在不同位置放置这个小拷贝，如图 11.19 所示。我们可以随机应用这些规则，或者可以并行应用所有这 3 个规则。无论是哪种情况，在极限情况下都可以得到 Sierpinski 镂垫。

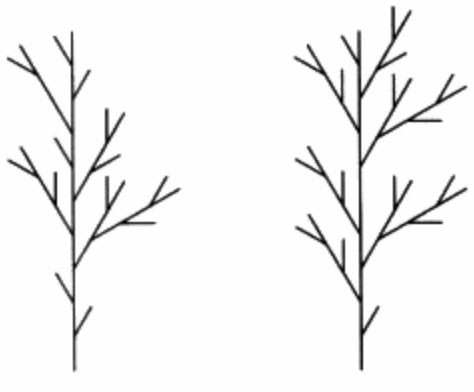


图 11.18　随机应用图 11.16 的规则得到的结果

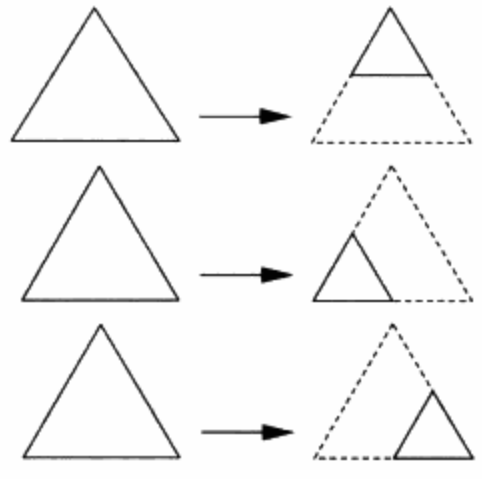


图 11.19　Sierpinski 镂垫的 3 个规则

现在有了 3 个相关的过程建模方法，它们可用来生成自然对象模型或感兴趣的数学对象模型。Koch 曲线和 Sierpinski 镂垫的例子引入了一种产生过程的新方法，我们可以递归地应用这种方法，而且每次执行时产生与原对象在形状上类似的细节，这种现象可以通过分形几何来研究。

11.8　递归方法和分形

基于语言的过程建模方法仅提供了一种用简单程序生成复杂对象的方法，另一种是基于分形几何（fractal geometry）的方法，它利用许多现实世界对象的自相似性。分形几何方法是由曼德尔布罗特（Mandelbrot）开发的，他创建了一个数学分支，使人们能研究使用普通几何工具无法描述的感兴趣的现象。计算机图形学工作者利用分形几何的思想不仅创建了精美且复杂的对象，而且还模型化了由其他方法难以模型化的很多现实世界实体。由分形几何方法生成的图形对象称为分形物（graftal）。

11.8.1　标尺和长度

分形几何有两大支柱：与几何尺寸和自相似性有关。我们可以从引出分形几何的下列问题之一来研究几何尺寸和自相似性：一条海岸线的长度是多少？假定我们有一张海岸线的地图。由于海岸线是弯曲起伏的而且不规则，所以可以拿一根线，把它沿海岸线放置，然后测量绳子的长度，并利用地图的比例尺把测量的长度转换为实际距离。然而，如果我们拿另一份地图，可以近距离地显示海岸线，那么就能看到更多的细节。增加的细节看上去很像第一张地图，但是可以看到更多的小弯和突起。如果拿绳子测量第二张地图的长度，在考虑了两张地图比例误差的情况下，我们将测量出

更精确的距离。我们可以不断地接近海岸线并试着用更高的精度一直把这个实验做下去。我们会发现新的细节，也许甚至达到测量沿岸的各个鹅卵石的层次。原则上可以一直进行这个过程直至达到分子的层次，每次都会看到具有更多细节的类似图像，并测量到更精确的长度。

如果想要得到任何有用的信息，或者至少两个人都认可的测量，那么就必须限制地图的分辨率，或者换句话说，必须选择我们能测量的最小单位。在计算机图形学中，如果使用透视图，则会面临相似的问题，这是因为看到的细节取决于我们距离对象有多远。

我们可以通过考虑在 11.7 节中介绍的 Koch 雪花的递归从数学上处理这些问题，此处，长度为 1 的每条线段都由长度为 1/3（如图 11.20 所示）的 4 条线段来替换。因此，每一次替换一条线段都是用最初曲线长度的 4/3 来扩展相同的两个端点的距离。当迭代无穷次时，如果考虑极限，就会出现维数问题。这条曲线不能是通常的一维曲线，因为在极限情况下，它有无限的长度并且它的一阶导数处处不连续。它也不是二维对象，因为它不填充一个平面二维区域。可以通过定义分数维来解决这个问题。

图 11.20　延长 Koch 曲线

11.8.2　分形维数

考虑一条长度为 1 的线段、一个单位正方形和一个单位立方体，如图 11.21 所示。在对维数（dimension）的任意合理的定义下，线段、正方形和立方体分别是一维、二维和三维对象。假定有一把分辨率为 h 的尺子，$h=\frac{1}{n}$ 是我们能测量的最小单位，假定 n 是整数，我们可依据 h 把上述每个对象划分成相同的单位，如图 11.22 所示。我们把线段划分成 $k=n$ 个相同的线段，把正方形划分成 $k=n^2$ 个小正方形，把立方体划分成 $k=n^3$ 个小立方体。每种情况下，都可以说通过把初始对象按比例因子 h 缩小并且把它重复 k 次，就创建了新对象。假定 d 是这些对象中任意一个对象的维数，则整体是部分之和，这是细分中恒久不变的事实。对任意对象，在数学上都具有等式：

$$\frac{k}{n^d}=kn^{-d}=1$$

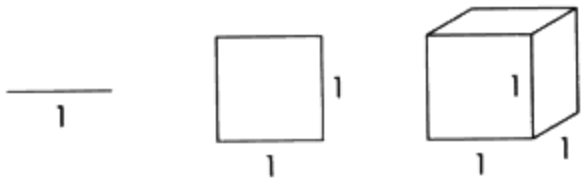


图 11.21　线段、正方形和立方体

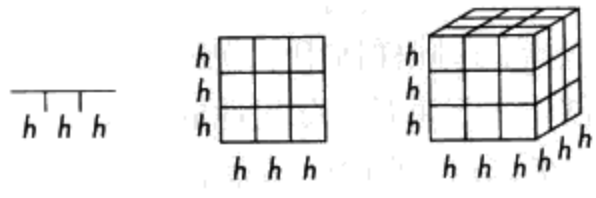


图 11.22　以 $h=\frac{1}{3}$ 细分对象

为了求解 d，可以把分形维数（fractal dimension）定义为：

$$d=\frac{\ln k}{\ln n}$$

换句话说，一个对象的分形维数是由细分创建相似对象的数量来决定的。考虑 Koch 曲线，通过用因子 3 对初始线段细分（按比例缩小）来创建 4 个相似对象，则相应的分形维数是：

$$d=\frac{\ln 4}{\ln 3}=1.261\ 86$$

现在考虑Sierpinski镂垫，按比例缩小步骤如图11.23所示，每次用因子2来分解一条边，我们保留创建的4个三角形中的3个，而且

$$d = \frac{\ln 3}{\ln 2} = 1.584\ 96$$

在这两个例子中，可以把由细分创建的对象看做比曲线占据更多的空间，但比填充的区域占据更小的空间。我们从四面体开始并对它的每一个面进行细分，就可以创建一个三维空间中的镂垫实体，如图11.24所示。我们使4个四面体在初始的顶点上，丢弃中间的区域，这样创建的对象具有的分形维数是：

$$d = \frac{\ln 4}{\ln 2} = 2$$

即使它不是在平面上。另外，虽然每次再分减少了体积，但却增加了表面积。假定我们从一个立方体开始并把它划分成三分之一的组合，如图11.25所示。接着移去立方体中心，采用的方法是推出立方体每个面的中心块以及立方体的中心块，这样留下原始子立方体27块中的20块，此对象具有的分形维数是：

$$d = \frac{\ln 20}{\ln 3} = 2.726\ 83$$

虽然这些构造是有意义的而且迭代的任何层次都可以很容易地用图形来生成，但是它们本身对场景的建模是没有用处的。然而，如果增加随机性，就得到强有力的建模技术。

图11.23　Sierpinski镂垫的细分

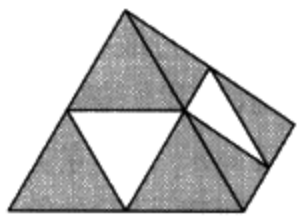


图11.24　实体镂垫

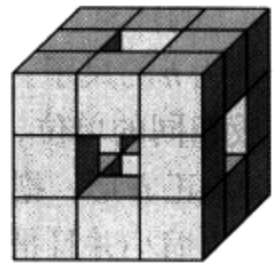


图11.25　立方体的细分

11.8.3　中点划分及布朗运动

一条分形曲线具有的维数为$1 \leqslant d < 2$，具有较低分形维数的曲线比具有较高分形维数的曲线显得更光滑，这同样适用于分形维数$2 \leqslant d < 3$的表面。在计算机图形学中，有很多这样的情形，即我们想创建看起来具有随机性的曲线或表面，而这些曲线或表面上具有的大量凹凸属性却是可测量的。举例来说，一座山脉的轮廓形成一条比沙漠地平线更粗糙（具有更高的分形维数）的曲线。同样，山脉地形的表面模型比农田表面具有更高的分形维数。我们也常常想以依赖分辨率的方式来生成这些对象。例如，如果要为实时性要求很高的应用领域（比如在飞行模拟器中）生成地形的细节，那么应该只为飞机附近的那些区域以高分辨率来生成地形的细节。

液体中粒子的随机运动称为布朗运动（Brownian motion）。模仿这种运动可提供一种能生成自然曲线和表面的有趣方法。物理学家通过构造折线来模拟布朗运动，折线上的每个后继点用一个任意距离和任意方向的前驱点来替换。真实的布朗运动以一个产生匹配物理粒子路径轨迹的特定随机数分布器为基础。在计算机图形学中，人们更关心快速计算以及产生粗糙程度可控的曲线的能力，因此，我们在这个更宽广的意义上使用术语布朗运动。

虽然可以试图通过直接生成折线的方法来产生布朗运动，但更有效的方法是使用简单递归过程。考虑如图11.26(a)所示的线段，找它的中点，然后用一个随机距离在法向量方向替换中点，如

图 11.26(b)所示。我们可以重复这个过程任意次数以便生成像图 11.27 那样的曲线。随机数生成器的变元或平均位移应该由一个因子来按比例缩小，该因子通常是 1/2，这是因为在每个阶段线段每次都要被缩短。也可以以随机的方向替换中点，而不是仅仅沿着法向量方向。如果随机数总是正数，我们就能创建地平线。如果使用零均值高斯随机数生成器，变量与 $l^{2(2-d)}$ 成比例，l 是将被细分的线段的长度，则 d 是最终曲线的分形维数，$d = 1.5$ 对应于真实的布朗运动。

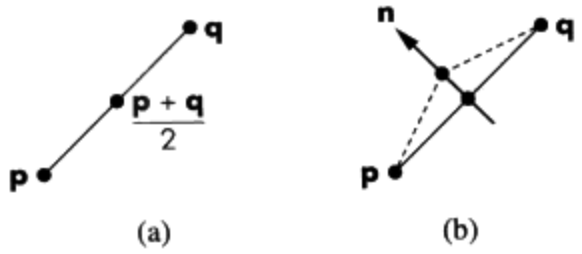


图 11.26　中点替换：(a) 初始线段；(b) 再分后的线段

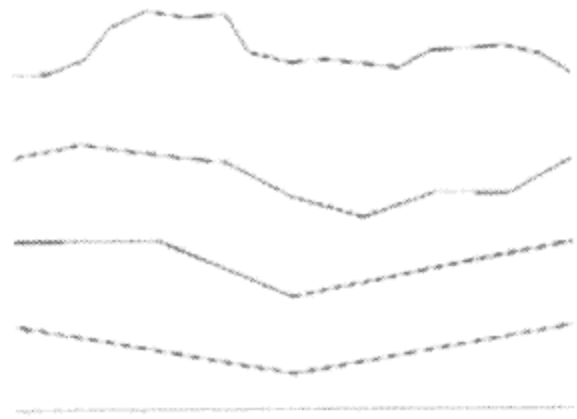
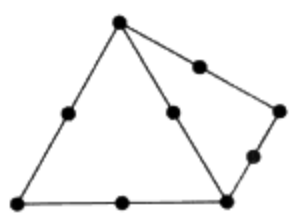


图 11.27　用 1、2、4、8 和 16 分割的分形曲线

11.8.4　分形山脉

计算机图形学中最著名的分形应用就是生成山脉和地形，通过增加在中点的替换，可以通过四面体的细分过程来产生山脉。考虑四面体的一个面，如图 11.28 所示。首先，找出各面的中点，然后替换每个中点[①]，创建 4 个新三角形，接着再来一次，通过控制随机数生成器的变元，可以控制产生对象的粗糙度。注意，如果要创建拓扑上正确的对象并且不将它们自己折叠，那么对于如何生成随机数要格外小心，更多的信息可参看本章末的建议阅读资料。

这个算法同样可很好地应用到任何网格。我们可以从 xz 平面上的矩形平面网格开始，把每个矩形再分成 4 个小矩形，在 y 轴向上替换所有顶点。图 11.29 所示为这个过程的一个例子。11.9 节将介绍可用于生成地形的另一种方法。



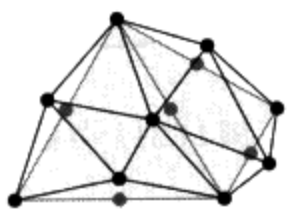
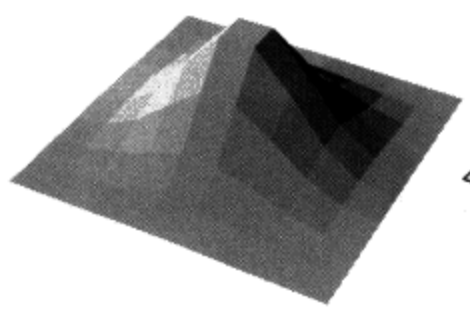
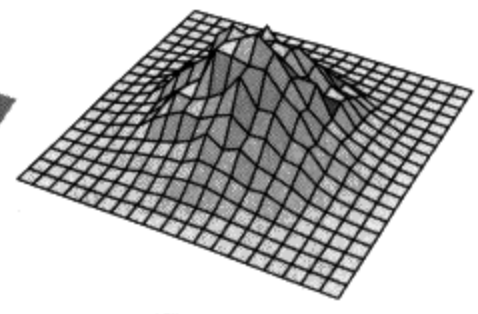


图 11.28　四面体小平面的中点细分



(a)



(b)

图 11.29　分形地形：(a) 网格；(b) 用替换顶点细分网格

11.8.5　Mandelbrot 集

著名的Mandelbrot集是分形几何中一个有趣的例子，利用OpenGL的像素绘制功能可以很容易地生成Mandelbrot集。虽然Mandelbrot集很容易生成，但它在生成模式上显示出无限的复杂性。它也提供了生成图像并使用查色表的良好例子。在下面的讨论中，假定读者具有复数运算的基础。

我们把复平面一个点表示为

$$\mathbf{z} = x + \mathbf{i}y$$

① 也可以替换初始顶点。

此处 x 是 $\mathbf{z}$ 的实部，y 是虚部（如图 11.30 所示）。如果 $\mathbf{z}_1 = x_1 + \mathbf{i}y_1$，$\mathbf{z}_2 = x_2 + \mathbf{i}y_2$ 是 2 个复数，复数加法和乘法定义为：

$$\mathbf{z}_1 + \mathbf{z}_2 = x_1 + x_2 + \mathbf{i}(y_1 + y_2)$$ ①

$$\mathbf{z}_1\mathbf{z}_2 = x_1x_2 - y_1y_2 + \mathbf{i}(x_1y_2 + x_2y_1)$$

纯虚数 $\mathbf{i}$ 具有特性 $\mathbf{i}^2 = -1$。复数 $\mathbf{z}$ 的模用下式给出：

$$|\mathbf{z}|^2 = x^2 + y^2$$

在复平面中，函数

$$\mathbf{w} = F(\mathbf{z})$$

实现复数点到复数点的映射。可以使用该函数来定义复数的递归形式：

$$\mathbf{z}_{k+1} = F(\mathbf{z}_k)$$

此处 $\mathbf{z}_0 = \mathbf{c}$ 是给定的初始值。如果绘制特定开始点 $\mathbf{z}_k$ 的位置，从图 11.31 中可看出有若干可能性。对一个特定函数 F，某些初始值产生直到无限大的序列，另一些可能周期性地重复，还有些序列聚合到称为吸引子（attractor）的点。例如，考虑函数：

$$\mathbf{z}_{k+1} = \mathbf{z}_k^2$$

此处 $\mathbf{z}_0 = \mathbf{c}$。如果 $\mathbf{c}$ 位于单位圆之外，则序列 $\{\mathbf{z}_k\}$ 发散；如果 $\mathbf{c}$ 位于单位圆之内，则 $\{\mathbf{z}_k\}$ 收敛于一个位于圆心的吸引子；如果 $|\mathbf{c}| = 1$，则每个 $\mathbf{z}_k$ 都在单位圆上。如果把 $|\mathbf{c}| = 1$ 的点考虑进去，则可以看出，根据 $\mathbf{c}$ 的值就能生成有限数量的点或者是单位圆上的所有点。

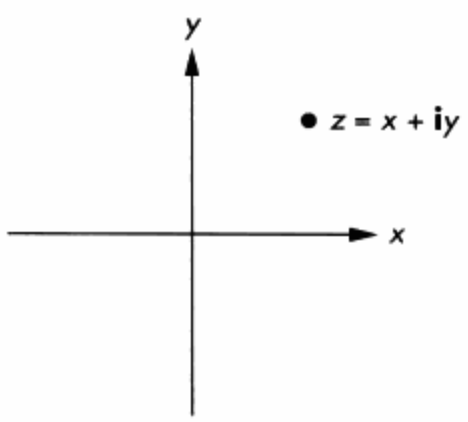


图 11.30　复平面

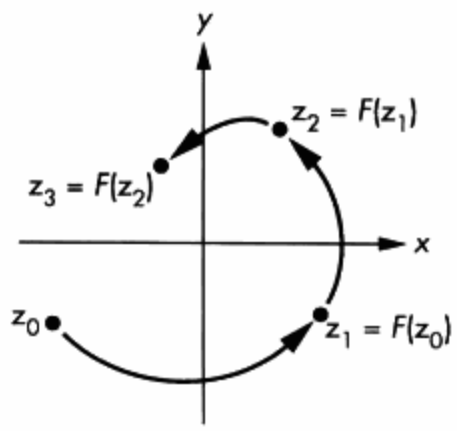


图 11.31　复数递归路径

一个更有趣的例子是函数

$$\mathbf{z}_{k+1} = \mathbf{z}_k^2 + \mathbf{c}$$

其 $\mathbf{z}_0 = 0 + \mathbf{i}0$。点 $\mathbf{c}$ 在 Mandelbrot 集（Mandelbrot set）中当且仅当由这个递归式产生的点是有限的。这样，可以把复平面分成两组点：一组属于 Mandelbrot 集，一组不属于这个集合。从图形上看，如果平面矩形区域以及彩色点在 Mandelbrot 集合中，则把它们处理成黑色；如果不在集合中，则处理为白色（如图 11.32(a)所示）。然而，正是集合边缘上的点显示出最大的复杂性，因而经常要放大这些区域。

Mandelbrot集的计算可能是费时的，有一些技巧可用来加快其处理速度。也许人们希望既能改变窗口的大小也能改变它的中心，但是以 $\mathbf{c} = -0.5 + \mathbf{i}0.0$ 为中心的区域是最令人感兴趣的。

通常在若干次迭代之后就能说出是否一个点会趋于无限。例如，如果 $|\mathbf{z}_k| > 4$，后继值就会更大，因此可以停止迭代。更困难的是说出在边界附近的一个点是否将收敛。因此，随后产生的通常

① 此处原文为 $\mathbf{z}_1 + \mathbf{z}_2 = x_1 + x_1 + \mathbf{i}(y_1 + y_2)$，可能有误。——译者注

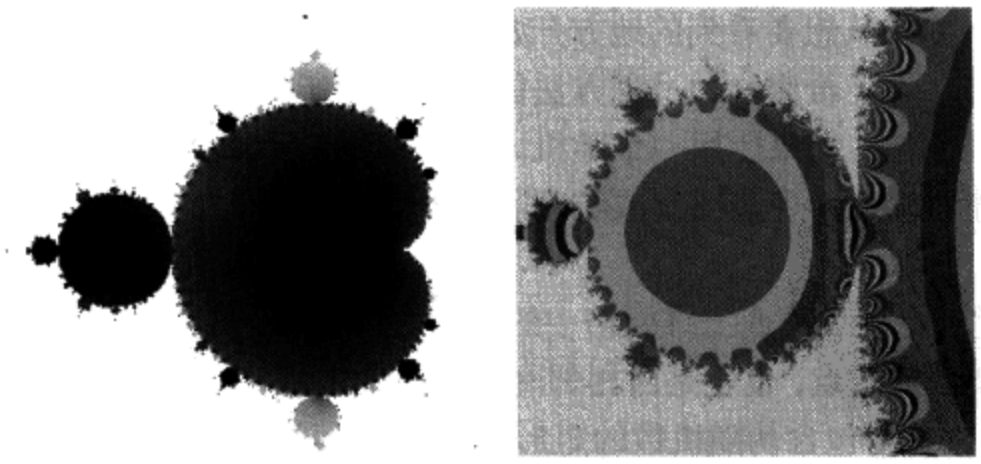
是集合的近似值。我们固定一个迭代的最大数。给定一个 **c**，如果能确定这个点发散，则将图像中对应 **c** 的点着白色。如果最大迭代数之后 $|\mathbf{z}_k|$ 小于某个阈值，则确定它在集合中并把它着为黑色。对于 $|\mathbf{z}_k|$ 的其他值，我们把一个纯色赋给对应于 **c** 的点。这些颜色通常是基于最后迭代之后 $|\mathbf{z}_k|$ 的值，也许这些点如何快速收敛或发散是交替的。

附录 A 包含一个生成集合近似值的程序。用户可以设置矩形的大小和中心，以及要执行的迭代次数。数 $\mathbf{z}_k$ 的大小可按比例在 0 ~ 255 范围内取值。通过遍历所有的像素直至达到迭代的最大次数就可以生成一个大小为 $n \times m$ 的 1 字节（每个像素占 1 字节）数组图像。画面由两部分组成。首先，在初始化时定义颜色映射模式。所显示的模式使用亮度值的浮点表示，其取值范围为 (0.0, 1.0)。我们定义一个红色亮度值的映射模式，把非红色赋给黑色 (0.0)，把全红色赋给白色 (1.0)，在这两个值之间线性插入其他亮度的值。对于蓝色，定义 0 为非蓝，1 为全蓝[①]。我们随机分配绿色的亮度值。这种绿色分配增强了显示亮度变化区域的细节（见图 11.32(b)）。代码如下：

```
for(i=0;i<256;i++)
{
    redmap[i]=i/255.;
    greenmap[i]=(rand()%256)/255.0; /* 在0.0至1.0之间的随机数 */
    bluemap[i]=1.0-i/255.;
}
glPixelMapfv(GL_PIXEL_MAP_I_TO_R, 256, redmap);
glPixelMapfv(GL_PIXEL_MAP_I_TO_G, 256, greenmap);
glPixelMapfv(GL_PIXEL_MAP_I_TO_B, 256, bluemap);
```

读者应该能毫不费力地实现其他颜色的映射模式。显示函数仅要求清屏并将像素发送到帧缓存中：

```
void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glDrawPixels(n,m,GL_COLOR_INDEX, GL_UNSIGNED_BYTE, image);
    glFlush();
}
```



(a)　(b)

图 11.32　Mandelbrot 集：(a) 黑白着色；(b) 边缘细节

11.9　过程噪声

我们使用了伪随机数生成器生成 Sierpinski 镂垫并用于分形细分。其实，在计算机图形学中伪随机数生成器在许多其他方面也有应用，从纹理的生成到自然对象的模拟，譬如云彩和液体。然而，既有实际的也有理论上的原因，我们使用的简单随机数生成器并不是应用中的一个良好选择。

① 此处原文为定义从 0 亮度的全蓝到全亮度的非蓝，可能有误。——译者注

让我们从白噪声（white noise）的概念谈起。白噪声是从电路热中子激发的放射性中接收到的或者是当调节到没有信号的频道时从电视屏幕上看到的。理想的白噪声具有的特性是：如果检查样本的一个序列就能发现在样本之间没有相关性，因此不能从先前的样本预测下一个样本。从数学上看，白噪声具有能谱平坦的特性（能谱是在频率域能看到的平均谱），所有频率都以相同强度出现。

在计算机中我们能生成伪随机数序列，这些随机数生成器（例如我们使用过的函数rand）产生无关联的序列，虽然它们能足够好地满足大多数应用，但是长时间执行这种随机数生成器之后该序列会重复出现，它们并不是真正的随机。然而，对很多其他应用来说白噪声不是人们需要的。人们往往需要随机性但并不想后继样本全部无关联。例如，假定想生成一个交互游戏的某个地形，我们可以使用一个高度随机变化的多边形网格。真正的白噪声会由于噪声中的高频成分给出一个非常粗糙的表面。如果想要绘制一个具有相当光滑地形的更逼真的表面，应该让相邻顶点彼此靠近。同样，我们想从噪声生成器中去除高频成分、至少能改变光谱或修改光谱的颜色。

在白噪声的高频成分中还有一个问题：频谱混叠。就像第8章讲到的，采样会引起Nyquist率的频谱混叠，这会引起图像中出现令人讨厌的视觉瑕疵。

对于这些问题有各种各样可能的解决方法。假定要生成一个带宽受限的随机样本序列并且知道我们想让哪些频率出现。我们可以从具有低频、随机振幅和相位的正弦光谱项之和中采样。这种傅里叶合成（Fourier synthesis）方法原则上可行，但是对每个样本要求进行大量的三角函数求值。另一类方法是基于图11.33。如果处理过程是白噪声的数字过滤，那么就可以在要求的大小和相位上设计过滤器来包含我们要的频率。因为这个噪声具有不均匀的频谱，它常常被称为有色噪声（colored noise），这个方法原则上也可行，但是计算量可能是很可观的。

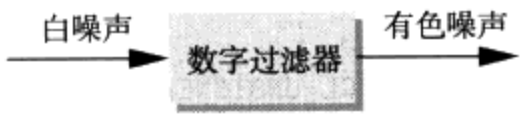


图11.33　生成相关的随机数

在一个方法中，如果从想要看到的地方开始，则可以设计一个以图11.33为基础的过程方法，这在计算上更加可行。除了希望这种过程方法能够最小化所要求的计算之外，还希望这种过程方法具有可重复性和局部性。如果使用一个随机方法来形成一种图案或纹理，那么当重新生成一个对象时，我们希望能精确地再现该图案或纹理。我们也希望仅仅使用局部数据就能够生成图案或纹理。

假定要生成一个一维、二维或三维网格中的伪随机序列，网格中的点是整数。我们可以使用网格点上的值生成非整数值的点，也就是由相邻网格值所确定的单元之间的点。例如，假定要生成一个二维纹理，则从形成一个包含伪随机数的矩形数组开始，可以使用这个数组通过插值计算得到任何纹理坐标为(s, t)的值。一个简单的插值方法是寻找对应于特定纹理坐标(s, t)的网格单元，并在网格单元的角点值上进行双线性插值计算，从而得到一个内部值。

这种方法产生出的噪声称为值噪声（value noise）。可以通过选择使用数组的多大部分来控制光滑度。例如，假定生成一个256 × 256的伪随机数数组并使用双线性插值来形成128 × 128的纹理，可以使用单一的单元并仅使用单元角点的4个值插入要求的128 × 128个值。在这种情况下，会得到非常光滑的纹理图像。另外，也可以选择使用数组的大部分数据。如果使用数组的4 × 4部分，在每16个单元上通过插值计算得到64个纹理值。因为将使用16个伪随机数，因而该纹理比第

一个例子会显示出更多的细节变化。我们也可以使用伪随机数组的128 × 128部分，在这种情况下，不需要任何插值计算就能得到完全无关联的纹理。

这种过程的问题是：由于在每个单元上进行双线性插值，所以当逐个单元形成纹理时，就会导致视觉上可见的瑕疵现象。通过使用一个插值公式可以解决这个问题，以便给出更光滑的结果，该插值公式使用邻接单元的数据。最普通的方法是使用三次多项式类型，我们将在第 12 章讨论。如果不涉及任何特定类型的细节，那么我们注意到一个三次多项式有 4 个系数，因此需要 4 个数据点来定义它。对于一个二维过程，需要 8 个邻接单元的数据或 16（4 × 4）个数据点来确定单元内的值。在三维情况下，需要26个邻接单元的数据或64个数据点。虽然我们将得到一个更光滑的结果，但是在二维或三维中计算量和要求的数据处理使该方法困难重重。

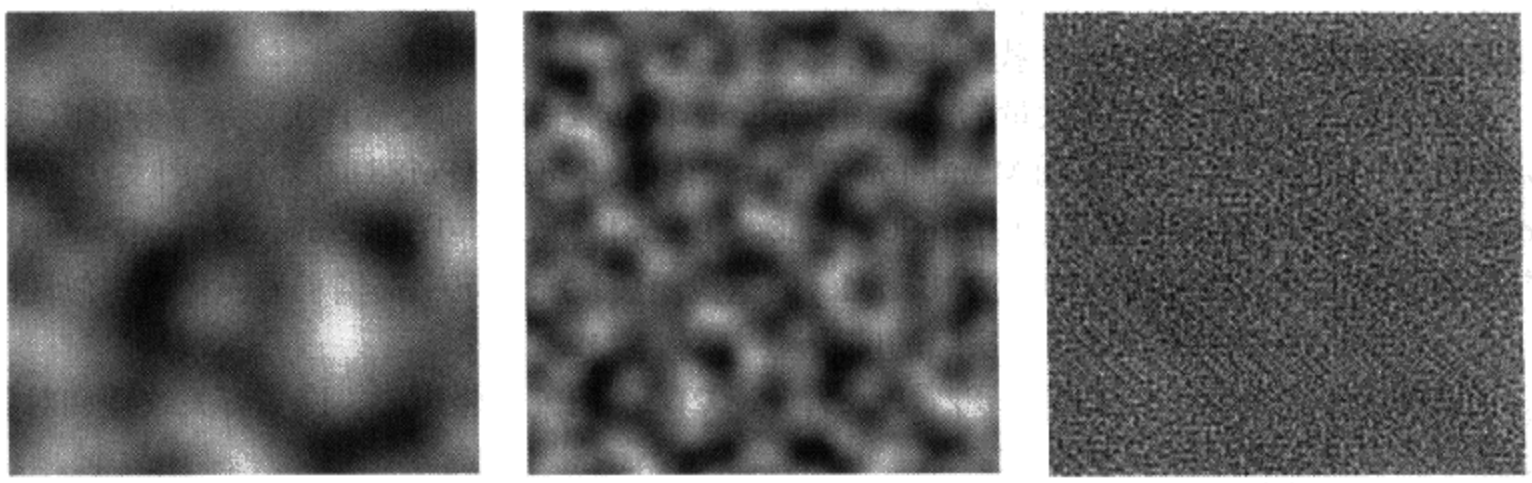
解决这个问题的一个办法是使用梯度噪声（gradient noise）。假定把三维噪声模型化为连续函数 $n(x, y, z)$[①]。在网格点(i, j, k)附近（此处 i、j 和 k 都是整数），可以用泰勒级数的前几项来逼近 $n(x, y, z)$：

$$n(x, y, z) \approx n(i, j, k) + (x - i)\frac{\partial n}{\partial x} + (y - j)\frac{\partial n}{\partial y} + (z - k)\frac{\partial n}{\partial z}$$

向量

$$\mathbf{g} = \begin{bmatrix} g_x \\ g_y \\ g_z \end{bmatrix} = \begin{bmatrix} \frac{\partial n}{\partial x} \\ \frac{\partial n}{\partial x} \\ \frac{\partial n}{\partial x} \end{bmatrix}$$

是在(i, j, k)的梯度。注意，$x - i$、$y - j$ 和 $z - k$ 是一个单元内部位置的小数部分。为了生成梯度噪声，首先在每个网格点计算规范化的伪随机梯度向量。我们可以通过在单位球体上生成一组均匀分布的随机点来得到这些向量。我们把网格点上的值固定为零（$n(i, j, k) = 0$）。在三维中，每个单元都有 8 个梯度向量（单元的每个角点有一个梯度向量），我们可以利用这些梯度向量来逼近 $n(x, y, z)$。常规的技术是使用这些梯度的过滤（或光滑）插值来生成单元内点的噪声。在噪声的原始生成者之后，以这种方式产生的噪声常常被称为柏林噪声（Perlin noise），或只称为噪声。这个噪声函数是 RenderMan 和 GLSL 着色语言的内置函数。由于在具体实现噪声时使用了哈希表（hash table），因此不必寻找全部网格的随机梯度而只需要所需要的 256 个或 512 个伪随机数。图 11.34 显示的是在 3 种不同频率下的二维梯度噪声，每种都是来自同一伪随机数数组的大小为256 × 256的灰度图像。



(a)　(b)　(c)

图 11.34　梯度噪声图像：(a) 使用噪声；(b) 使用 50 倍噪声；(c) 使用 100 倍噪声

① 此处原文为 $n(x, y, x)$，可能有误。——译者注

过程噪声已经在很多方面得到应用。例如，把噪声添加到机器人模型的关节连接位置可以增强模型的逼真感。增加非线性（例如取噪声生成器的绝对值）就能生成用于模拟湍流流动现象以及产生纹理的序列。过程噪声也用于定义模糊对象，如云彩的模拟等。

小结和注释

过程建模方法具有很多优点，因为人们可以控制所生成的图元的数量，并控制过程中产生这些图元的时机。同样重要的是过程图形提供了一种面向对象的建模方法，这是一种日益重要的方法。

物理学与计算机图形学的结合提供了一系列技术，这些技术有希望产生物理上正确的动画并提供新的建模技术。粒子系统只不过是一个基于物理建模的例子，但是它们提供了一种具有广泛应用性的技术。最有趣的也是读者在这方面能做到的有益的练习之一就是动手建立一个粒子系统。

粒子方法通常用在商业性的动画片中，两种方法都用于模拟物理现象，比如模拟烟火、云彩、流水以及确定动画角色的位置。粒子方法也已成为一种模拟物理现象的标准方法，常常用来替换局部复杂的微分方程模型，即使图形结果没必要时也会使用。

分形方法提供了用简单算法和程序生成对象的又一种方法，它产生看上去具有很高复杂性的图像。过程噪声几乎是所有过程建模方法的核心，而且当把它与一种或多种我们介绍过的其他方法结合时常常能最好地说明其真正功效。

展望未来，我们看到图形学方法与来自物理的、数学的和其他学科的方法进一步交织在一起。历史上，一旦给定了现有的计算能力，人们愿意接受“看上去不错”的视觉效果，然而却无法做到真实地模拟像仿真和交互式游戏等应用中正确的物理现象。即使在人们可以花好几天绘制一帧画面的应用中，真正的物理现象仍然太复杂以至于难以精确模拟。然而，随着可计算能力的不断进步以及使用这种能力的成本的降低，我们期望在计算机图形学的所有应用中看到愈来愈多的基于物理的高逼真建模。

建议阅读资料

Reeves[Ree83]首次把粒子系统引入了计算机图形学中。从那时起，它们被用于模拟各种现象，包括鸟群的聚集[Rey87]、流体流动、火焰燃烧、青草的模拟以及曲面的绘制[Wit94a]。粒子系统也被广泛应用于物理学和数学中，并提供一种替换方法来解决描述流体流动和固体力学这类偏微分方程的复杂系统问题，例如，读者可参阅[Gre88]以了解这方面的内容。方法遵循Witkin[Wit94b]。过程建模的很多例子可在[Ebe02]中找到。

关于分形及其相关方法有大量的文献。由Fournier[Fou82]撰写的文章首次显示了分形山脉。关于分形数学的更深入研究可参阅Mandelbrot[Man82]以及Peitgen[Pei88]的著作。图文法（graph grammar）的使用出现在很多情形中[Pru90，Smi84，Lin68]。Hill[Hil07]和Prusinkiewicz[Pru90]提出了有趣的空间填充曲线和曲面的方法。Barnsley的迭代函数系统[Bar93]提供了另一种自相似性的使用方法，它们在图像压缩领域都有实际的应用。

梯度噪声源于Perlin的文章[Per82，Per85，Per02]。当研究值噪声时，许多有关纹理和对象生成的应用可以在文献[Ebe02]中找到。

习题

11.1　从单个等边三角形开始找出一组生成Sierpinski镂垫的产生式。

11.2　如何确定一个海岸线的分形维数？如何验证海岸线的形状确实是一个分形？

11.3　从第6章介绍的用于绘制近似球体的四面体细分程序开始，把这个程序转换为生成一个分形山脉的程序。

11.4　试把一个二叉树（例如可把它用于搜索）描述为具有指向孩子指针的节点链表。编写一个采用这种描述的 OpenGL 程序并用图示的方式显示这棵树。

11.5　编写一个简单的弹簧 - 质点粒子系统，把这个粒子系统绘制成四边形网格，该系统允许用户以交互的方式把粒子放入初始位置。

11.6　通过给粒子系统添加外部力来扩展习题 11.5，创建一个旗子在风中飘扬的图像。

11.7　编写一个分形网格的程序，试将实际的高程（elevation）数据用于网格的初始位置。

11.8　编写一个程序，给定两个具有相同顶点数目的多边形，产生一个把一个多边形转换成另一个多边形的图像序列。

11.9　如果使用用于 Mandelbrot 集的基本公式，但是这次固定复数 c 的值并为收敛处找出初始点的集合，我们就具有复数 c 的 Julia 集（Julia set）。写一个程序显示 Julia 集。提示：使用在 Mandelbrot 集边缘附近的 c 值。

11.10　写一个粒子系统模拟由焊接或由烟火生成的火花。

11.11　扩充习题 11.10，模拟一个多面体的爆炸。

11.12　将 α 混合（见第 8 章）、球体生成（见第 6 章）以及分形结合起来创建云彩。

11.13　使用分形法生成一个虚拟行星的表面，输出应该显示大陆和海洋。

11.14　在 Lennard-Jones 粒子系统中，一个力使粒子互相吸引，这个力与粒子之间距离的 12 次幂的倒数成正比；另一个力使它们互相排斥，这个力与同样距离的 24 次幂的倒数成正比。在一个盒子中模拟这个系统，为了便于仿真，可以假定一个离开盒子的粒子从盒子另一端重新进入这个盒子。

11.15　创建一个粒子系统，其中感兴趣的区域被划分成同样大小的立方体。一个粒子只能与它自己的立方体以及相邻于它的立方体中的粒子交互。

11.16　在动画中，粒子系统用于给出人物的位置。一旦人物的位置被确定后，就可把人物的二维图像作为纹理映射到这些位置的多边形上。构造这样一个系统来移动人物，人物既受到使它们朝希望的方向上移动的外力又受到用于避免碰撞的排斥力。如何能让多边形面向照相机？

11.17　将粒子的生命周期增加到粒子系统上。使用粒子的链表结构而不是数组重新实现粒子系统，使得粒子能被更容易地增加或删除。

11.18　用接近于球体的明暗色调表现例子中粒子系统内的粒子。

11.19　通过连接一个链中的 4 个点，用一个弹簧 - 质点系统来模拟一根头发或一根草。

11.20　用各种群聚算法进行实验。例如，粒子可以更新它们的速度并移动到所有粒子群的中心。另一个方法是让每个粒子移向一个“朋友”粒子。

11.21　把过程噪声增加到人物模型中使得在静止时每个关节有轻微的移动。

11.22　使用过程噪声实现一个分形地形，包括移动视点以及对观察到的场景进行放大和缩小的功能。

第12章　曲线和曲面

我们周围的世界由形状各异的物体组成。然而，在计算机图形学中，人们仍然使用平面图形来绘制虚拟的世界，对此人们总能提出充分和可靠的理由。例如，图形系统可以高效地绘制三维平面多边形，包括进行不可见面的消隐、明暗处理和纹理映射等操作。可以利用前面章节介绍的定义球体模型所采用的方法，用一组平面多边形定义虚拟环境或真实环境中的弯曲对象。或者，如同本章所介绍的方法，也可以给应用程序编程人员提供在程序中定义弯曲对象的方法，而将这些对象的最终绘制过程留给特定的图形绘制系统去处理。

本章将介绍曲线和曲面的3种建模方法，并将参数多项式的表示形式作为重点。此外还会讨论在当前的图形系统中如何绘制曲线和曲面，其处理过程通常需要将弯曲对象细分成一组平面图元。由于该绘制过程是特定图形绘制系统的一部分，所以这个绘制过程对应用程序编程人员来说是透明的。然而，理解其工作方式有助于读者更好地认识使用曲线和曲面所面临的现实局限性。

12.1　曲线和曲面的表示形式

在讨论参数多项式曲线和曲面之前，首先概括介绍曲线和曲面的3种主要表示形式：显式形式、隐式形式和参数形式，并且以直线、圆、平面和球面为例来阐述各种方法进行建模的优势和不足。

12.1.1　显式表示形式

二维曲线的显式表示形式是将一个变量（因变量）用另一个变量（自变量）表示。在x, y坐标系中可以写成：

$$y = f(x)$$

幸运的话，还能够将上述关系转换为x关于y的函数：

$$x = g(y)$$

但是不能保证对于任意一条曲线肯定存在上述某个形式。通常根据斜率m和y轴截距h关系的方程来表示直线：

$$y = mx + h$$

但是这个方程无法表示垂直于x轴的直线。对于图形系统，或者更宽泛地说，对于所有设计与使用曲线和曲面的领域来说，这只是众多坐标系依赖所引起的问题之一。直线和圆的存在独立于它们的表示形式，任何不能对特定方向（例如垂直方向）进行表示的形式都存在严重缺陷。

圆更能说明问题。考虑一个以原点为圆心、r为半径的圆。圆的曲率是一个常量，反映了圆弧在某点的弯曲程度。没有能够比圆更为对称的封闭的二维曲线了，然而，使用显式形式来表示圆，最多只能写出半圆的方程：

$$y = \sqrt{r^2 - x^2}$$

另一个半圆的显式方程为：

$$y = -\sqrt{r^2 - x^2}$$

此外，还必须限定这两个方程成立的条件：

$$0 \leqslant |x| \leqslant r$$

在三维空间中，曲线显式表示形式需要两个方程。例如，仍然使用x表示自变量，则对应存在两个因变量：

$$y = f(x)$$
$$z = g(x)$$

曲面表示需要两个自变量，可以采用如下形式：

$$z = f(x, y)$$

同样，三维空间中曲线或者曲面也许并不存在显式表示形式。例如，方程

$$y = ax + b$$
$$z = cx + d$$

描述了三维空间中的直线，但是却不能表示以x为常数的平面上的直线。同样，由方程$z = f(x, y)$表示的曲面不能表示球面，因为对于给定的x和y，可能对应于0个、1个或2个球面上的点。

12.1.2　隐式表示形式

我们所使用的曲线和曲面大多数具有隐式形式。在二维空间中，曲线隐式形式方程可以表示为：

$$f(x, y) = 0$$

直线和以原点为圆心的圆有其各自的隐式表示形式：

$$ax + by + c = 0$$
$$x^2 + y^2 - r^2 = 0$$

函数f实际上是一种度量或者隶属函数，将空间上所有的点分割为曲线上的点和曲线以外的点，这样就可以通过计算f来判断点(x, y)是否位于曲线上。然而在通常情况下，对于某个x值，我们并不具备求解曲线上对应的y值的方法，反之亦然。隐式形式比显式形式具有较小的坐标系依赖性，并且能够表示任意直线和圆。

三维空间中，隐式表达式

$$f(x, y, z) = 0$$

表示一个曲面。例如，平面可以通过常量a、b、c和d来表示：

$$ax + by + cz + d = 0$$

以原点为圆心、r为半径的球面可以表示为：

$$x^2 + y^2 + z^2 - r^2 = 0$$

三维曲线不易于隐式表示，可以将曲线表示为曲面的交线，前提是存在这样的曲面：

$$f(x, y, z) = 0$$
$$g(x, y, z) = 0$$

这样，如果某点(x, y, z)同时位于两个曲面上，则该点必定位于曲面的交线上。实际应用中的曲线和曲面通常具有隐式形式，但是由于难以求解曲线或曲面上的点，因此其应用受到限制。

代数曲面（algebraic surface）是指函数$f(x, y, z)$包含3个变量的多项式之和，其中二次曲面具有特殊的重要性，二次曲面是f中每一项最高次数为2的曲面[①]。之所以对二次曲面感兴趣，不仅因为一些实用的曲面（例如球面、圆柱面和圆锥面）都是二次的，而且因为与直线相交时最多产生两个交点。我们将利用这一特性在12.9节绘制二次曲面，在第13章用于光线跟踪。

12.1.3　参数形式

曲线的参数形式是以自变量u为参数来表示曲线上的点的每一个空间变量值。在三维空间中，3个显式函数分别为：

$$x = x(u)$$
$$y = y(u)$$
$$z = z(u)$$

参数形式的优点之一是在二维和三维空间中形式一致。在二维情况下，只需去掉z的方程。对参数形式形象的描述是：随着u的改变，点$\mathbf{p}(u) = [x(u)\ \ y(u)\ \ z(u)]^T$的运动轨迹如图12.1所示。可以将某点的导数

$$\frac{\mathrm{d}\mathbf{p}(u)}{\mathrm{d}u} = \begin{bmatrix} \frac{\mathrm{d}x(u)}{\mathrm{d}u} \\ \frac{\mathrm{d}y(u)}{\mathrm{d}u} \\ \frac{\mathrm{d}z(u)}{\mathrm{d}u} \end{bmatrix}$$

看做点沿轨迹运动的速度或者曲线的切线。

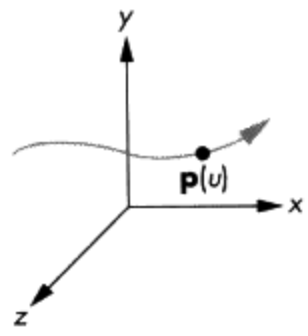


图12.1　参数曲线

曲面的参数形式需要两个参数，可以使用如下3个方程：

$$x = x(u, v)$$
$$y = y(u, v)$$
$$z = z(u, v)$$

或者，使用列矩阵来表示：

$$\mathbf{p}(u, v) = \begin{bmatrix} x(u, v) \\ y(u, v) \\ z(u, v) \end{bmatrix}$$

随着u和v在某区间变化，生成曲面上的所有的点$\mathbf{p}(u, v)$。与第6章讲述的球面一样，由列矩阵

① 次是指项的各变量指数之和，所以x、yz或z^2都是二次多项式，xy^2不是二次多项式。

$$\frac{\partial \mathbf{p}}{\partial u} = \begin{bmatrix} \frac{\partial x(u,v)}{\partial u} \\ \frac{\partial y(u,v)}{\partial u} \\ \frac{\partial z(u,v)}{\partial u} \end{bmatrix}$$

和

$$\frac{\partial \mathbf{p}}{\partial v} = \begin{bmatrix} \frac{\partial x(u,v)}{\partial v} \\ \frac{\partial y(u,v)}{\partial v} \\ \frac{\partial z(u,v)}{\partial v} \end{bmatrix}$$

表示的向量确定了曲面上任一点的切平面。而且，只要向量互不平行，则其叉积可以确定该点的法线（如图 12.2 所示）：

$$\mathbf{n} = \partial \mathbf{p}/\partial u \times \partial \mathbf{p}/\partial v$$

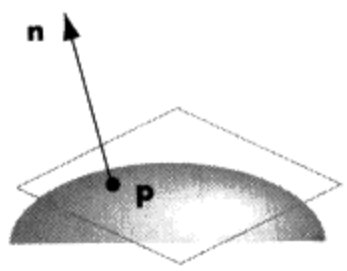


图 12.2　参数曲面上某点的切平面和法线

曲线和曲面的参数表示形式对于计算机图形学是最灵活和最健壮的表示形式。然而由于仍然使用 x、y 和 z 表示，因此参数表示并没有完全消除对于特定坐标系或者标架的依赖。可以创建一个仅仅基于 $\mathbf{p}(u)$ 的曲线和 $\mathbf{p}(u, v)$ 的曲面系统。例如，在三维空间中描述曲线经常用到 Frenet 标架，它的坐标轴被定义为曲线上每一点的切线和法线。与第 9 章讨论的凹凸映射一样，需要计算副法线方向作为坐标系的第三个方向。然而这个标架对于曲线上每一个点都是不一样的，不过就图形学应用而言，在特定标架中的 x、y、z 的参数表示形式已足够健壮了。

12.1.4　参数多项式曲线

曲线或曲面的参数表示形式并非是唯一的，可以有多种表示方式。在计算机图形学中使用最多的形式，对于曲线来说是参数 u 的多项式，而对于曲面来说则是参数 u、v 的多项式，其原因将在 12.2 节中讨论。

考虑下面的曲线表示形式[①]：

$$\mathbf{p}(u) = \begin{bmatrix} x(u) \\ y(u) \\ z(u) \end{bmatrix}$$

它可以表示为如下 n 阶[②]参数多项式：

$$\mathbf{p}(u) = \sum_{k=0}^{n} u^k \mathbf{c}_k$$

其中，每一个 $\mathbf{c}_k$ 都包含相互独立的 x、y 和 z 分量，即

① 在这里没有必要使用齐次坐标。在 12.8 节中，我们将在齐次坐标里推导 NURBS 曲线。

② OpenGL 大于 1 的次（degree）称为阶（order）。

$$\mathbf{c}_k = \begin{bmatrix} c_{xk} \\ c_{yk} \\ c_{zk} \end{bmatrix}$$

这 $n+1$ 个列矩阵 $\{\mathbf{c}_k\}$ 是 $\mathbf{p}$ 的系数，对于任意的 $\mathbf{p}$，系数选择有 $3(n+1)$ 个自由度。由于 x、y 和 z 分量相互独立，所以可以使用 3 个独立的方程，它们都具有如下的形式：

$$p(u) = \sum_{k=0}^{n} u^k c_k$$

这里 p 表示 x、y 或 z 之中任意一个多项式，因此 $p(u)$ 有 $n+1$ 个自由度。我们将曲线定义在 u 的任意区间内：

$$u_{\min} \leqslant u \leqslant u_{\max}$$

但是为了不失一般性（见习题 12.3），假设 $0 \leqslant u \leqslant 1$。随着 u 在区间内的变动，生成如图 12.3 所示的曲线段。

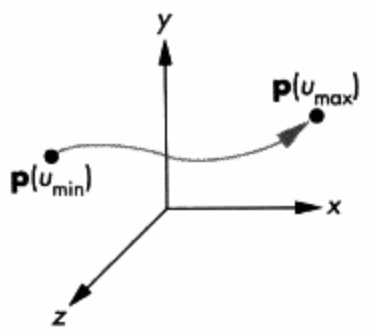


图 12.3　曲线段

12.1.5　参数多项式曲面

参数多项式曲面可以定义为：

$$\mathbf{p}(u, v) = \begin{bmatrix} x(u, v) \\ y(u, v) \\ z(u, v) \end{bmatrix} = \sum_{i=0}^{n} \sum_{j=0}^{m} \mathbf{c}_{ij} u^i v^j$$

因此必须确定 $3(n+1)(m+1)$ 个系数以确定曲面 $\mathbf{p}(u, v)$。在以后的讨论中，将总是取 $n = m$，并且令 u 和 v 在矩形 $0 \leqslant u, v \leqslant 1$ 范围内变动，得到如图 12.4 所示的曲面片。注意，如果保持 u 或者 v 不变，而变换另一个参数，则产生一簇曲线，因此可以将曲面片看做曲线簇的极限。根据这个思想，可以先定义参数多项式曲线，再生成具有相似特性的曲面。

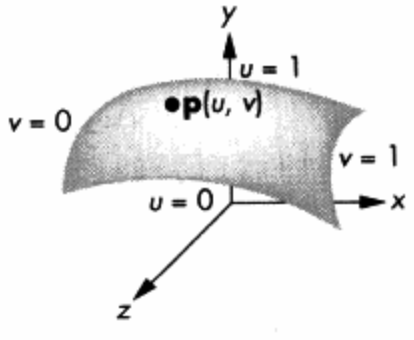


图 12.4　曲面片

12.2　设计准则

曲线和曲面在图形学和计算机辅助设计中的应用方式与在其他领域以及之前的使用方式不同。由于图形学中的诸多要求，我们往往使用低阶参数多项式，这些要求包括：

- 形状的局部控制
- 平滑度和连续性
- 可导性
- 稳定性
- 易于绘制

下面通过一个简单的例子深入理解这些准则。假设想要使用可弯曲的木条建造一个飞机模型的框架。我们可以先构建一组横截面，然后使用长木条将其连接以形成机身，如图 12.5 所示，我们可以由飞机照片或者飞机横截面的草图入手设计如图12.6所示的横截面，可以求取横截面的单一全局表示，但是这个表示可能并不满足需求。木条在断裂之前可以被平滑地弯曲到一定的程度，因此只能近似地拟合图 12.6 中的曲线，实际形成的截面曲线更像图 12.7 所示。现实中，人们可能以多根木条构造横截面，每根木条成为横截面上的一段曲线。因此，我们不仅仅要求每一条曲线段必须是平滑的，而且希望曲线段连接点也具有一定的平滑度。

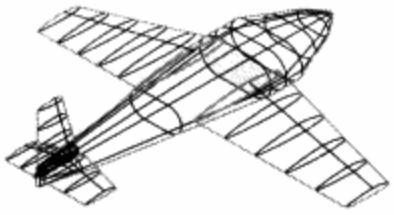


图 12.5　飞机模型

图 12.6　截面曲线

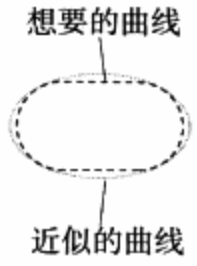


图 12.7　截面曲线的近似表示

注意，虽然能够确保每段曲线是平滑的，但是却不能保证连接点也是平滑的。如图 12.8 所示，尽管两条曲线段是平滑的，但连接点导数却是不连续的（平滑性一般由沿着曲线的导数表示），不平滑的曲线对我们来说没有任何意义。通常，一阶导数连续的曲线比一阶导数不连续的曲线更平滑（高阶的导数也有类似的结论）。这个概念将在 12.3 节详细讨论。现在，需要确保对于曲线多项式

$$\mathbf{p}(u) = \sum_{k=0}^{n} \mathbf{c}_k u^k$$

导数存在，并且可以求解计算。这样，我们仅仅会在连接点遇到连续性问题。

我们倾向于单独设计每条曲线段，而不是使用单一的公式设计所有的曲线段，原因之一是：希望以交互的方式控制图形形状，能够通过适当调整来达到设计要求，并且希望当改动某一段曲线时，将仅仅影响该曲线段的形状。这种局部控制只是稳定性原则的一个方面：输入参数值的微小变化将仅仅引起输出参数的微小改变。换言之，自变量的微小改变将引起因变量的微小改变。

通过与曲线比对，可以将木条弯曲以逼近所需要的形状。我们更希望能够使用少量控制点或者数据点来设计所需形状。图 12.9 显示了一条曲线及其控制点，注意，曲线经过（或者说插值）某些控制点，却只是靠近其他点。本章自始至终都将讲到，在计算机图形学和 CAD 中，如果曲线能够平滑地靠近控制点，那么人们通常是感到满意的。

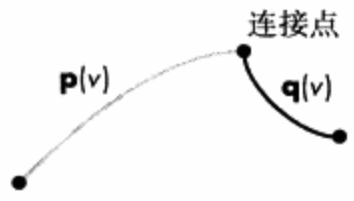


图 12.8　连接点导数的不连续性

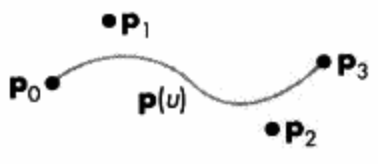


图 12.9　曲线段和控制点

上述实例说明了使用参数多项式曲线的理由。12.7节和12.8节将讨论的样条曲线得名于造船工程师设计船体形状的可弯曲的木条或者金属条。每一个样条用钉子固定在恰当的位置，样条材质的弯曲特性使其成为一条多项式曲线。

回到计算机图形学，切记我们最终要将曲线或曲面绘制到屏幕上，因此如果一个好的数学方法表示的曲线和曲面不易绘制，那么它的价值是非常有限的。不论曲线还是曲面，我们都希望采用类似于绘制平面对象的方法来绘制，包括颜色渲染、明暗处理和纹理映射等技术。

12.3　三次参数多项式曲线

采用参数多项式表示曲线，必须确定多项式的次数。一方面，如果曲线的次数比较高，则有多个可供设置的参数以形成所需要的形状，但是曲线上点的计算开销巨大。此外，随着曲线多项式阶的升高，曲线光滑性会变差。另一方面，如果次数太低，也许就没有足够的参数进行设置。但是，如果只是需要在一个较小的区间内设计一段曲线，那么低阶多项式就可以满足大多数要求，尽管自由度不多，但是也足够生成所需形状的曲线段了。因此，大多数设计者最初都由三次参数多项式曲线入手。

我们可以使用如下形式的行矩阵和列矩阵表示三次参数多项式：

$$\mathbf{p}(u) = \mathbf{c}_0 + \mathbf{c}_1 u + \mathbf{c}_2 u^2 + \mathbf{c}_3 u^3 = \sum_{k=0}^{3} \mathbf{c}_k u^k = \mathbf{u}^T \mathbf{c}$$

其中，

$$\mathbf{c} = \begin{bmatrix} \mathbf{c}_0 \\ \mathbf{c}_1 \\ \mathbf{c}_2 \\ \mathbf{c}_3 \end{bmatrix}, \qquad \mathbf{u} = \begin{bmatrix} 1 \\ u \\ u^2 \\ u^3 \end{bmatrix}, \qquad \mathbf{c}_k = \begin{bmatrix} c_{kx} \\ c_{ky} \\ c_{kz} \end{bmatrix}$$

c是多项式系数的列矩阵，通过控制点计算其值。使用控制点数据的不同方式将导出几种不同类型的三次曲线。每种类型的曲线多项式都包含12个方程，每个方程有12个未知数。但是因为x、y和z是独立的自变量，因此可以将这些方程分为3组，每组4个方程，每个方程有4个未知数。在12.8.4节讨论NURBS曲线时，我们将使用齐次坐标系，因此需要使用w坐标，这样则有4个方程组，每组有4个方程，每个方程有4个未知数。

特定类型的三次曲线的设计将基于参数u的一组值。这些值也许以插值条件的形式出现，要求曲线经过这些值对应的点；或者要求多项式插值这些值的导数。也可以给出两段曲线在连接点位置的一些平滑性条件，以确保连接点有各种连续性条件；最后，也可以是更宽松的条件，仅仅要求曲线逼近一些点。不同的条件将定义不同类型的曲线，这取决于如何使用一组已知数据，因此相同的数据不只能够定义一条曲线。

12.4　插值

我们首先讨论的三次参数多项式是三次插值多项式。尽管在计算机图形学中很少使用插值多项式曲线，但是人们比较熟悉这一类多项式，它的推导过程将展示获得其他类型三次多项式必需的步骤。而且对插值多项式的分析可以阐述三次多项式的许多重要特性，人们就是基于这些特性来评价曲线或曲面的。

假设三维空间中的 4 个控制点是 $\mathbf{p}_0$、$\mathbf{p}_1$、$\mathbf{p}_2$ 和 $\mathbf{p}_3$，表示为如下形式：

$$\mathbf{p}_k = \begin{bmatrix} x_k \\ y_k \\ z_k \end{bmatrix}$$

求解 $\mathbf{p}(u) = \mathbf{u}^T\mathbf{c}$ 的系数 $\mathbf{c}$，使得多项式插值这 4 个控制点。求解相对简单，对于这 4 个三维插值点，我们有 12 个条件和 12 个未知数。但是首先必须明确在何处进行插值，即参数 u 的值，由于缺乏信息，而且 u 始终在区间[0, 1]变化，我们可以等间距取值 $u = 0, \frac{1}{3}, \frac{2}{3}, 1$。因此，这 4 个条件如下：

$$\mathbf{p}_0 = \mathbf{p}(0) = \mathbf{c}_0$$

$$\mathbf{p}_1 = \mathbf{p}\left(\frac{1}{3}\right) = \mathbf{c}_0 + \frac{1}{3}\mathbf{c}_1 + \left(\frac{1}{3}\right)^2\mathbf{c}_2 + \left(\frac{1}{3}\right)^3\mathbf{c}_3$$

$$\mathbf{p}_2 = \mathbf{p}\left(\frac{2}{3}\right) = \mathbf{c}_0 + \frac{2}{3}\mathbf{c}_1 + \left(\frac{2}{3}\right)^2\mathbf{c}_2 + \left(\frac{2}{3}\right)^3\mathbf{c}_3$$

$$\mathbf{p}_3 = \mathbf{p}(1) = \mathbf{c}_0 + \mathbf{c}_1 + \mathbf{c}_2 + \mathbf{c}_3$$

可以将这些方程写成如下矩阵形式：

$$\mathbf{p} = \mathbf{A}\mathbf{c}$$

其中，

$$\mathbf{p} = \begin{bmatrix} \mathbf{p}_0 \\ \mathbf{p}_1 \\ \mathbf{p}_2 \\ \mathbf{p}_3 \end{bmatrix}$$

而

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & \frac{1}{3} & \left(\frac{1}{3}\right)^2 & \left(\frac{1}{3}\right)^3 \\ 1 & \frac{2}{3} & \left(\frac{2}{3}\right)^2 & \left(\frac{2}{3}\right)^3 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

通过分析会发现如果将 $\mathbf{p}$ 和 $\mathbf{c}$ 看做是 12 个元素的列向量，那么上述计算违反了矩阵相乘的规则。但是换种方式，将 $\mathbf{p}$ 和 $\mathbf{c}$ 看做 4 个元素的列向量，其中的元素又都是 3 个元素的列向量，这样，$\mathbf{c}$ 的一个元素（3 个元素的列向量）与 A 的一个元素（标量）的相乘，得到与 $\mathbf{p}$ 的元素一致的 3 个元素的列向量[①]。可以证明 A 是非奇异矩阵，而且可以通过计算其转置逆矩阵来获得插值几何矩阵（interpolating geometry matrix）：

$$\mathbf{M}_I = \mathbf{A}^{-1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -5.5 & 9 & -4.5 & 1 \\ 9 & -22.5 & 18 & -4.5 \\ -4.5 & 13.5 & -13.5 & 4.5 \end{bmatrix}$$

所求系数为：

① 我们使用行矩阵表示 $\mathbf{p}$ 和 $\mathbf{c}$ 的元素，这种情况下，矩阵满足乘法规则，因为 4 × 4 的矩阵可以乘以 4 × 3 的矩阵。然而这个方法不适用于曲面。还可以采用张量计算方法，但是这个方法超出了本书的讨论范围。

$$\mathbf{c} = \mathbf{M}_I \mathbf{p}$$

已知控制点序列 $\mathbf{p}_0, \mathbf{p}_1, \cdots, \mathbf{p}_m$，当然可以插值所有控制点来定义一条 m 次多项式曲线，推导过程与三次多项式推导过程相似。而我们更倾向于定义一组三次插值曲线，其中每条仅插值4个控制点。为了获得连接点的连续性，我们使用前段曲线的最后一个控制点作为下一线段的起点（见图12.10）。这样我们使用 $\mathbf{p}_0$、$\mathbf{p}_1$、$\mathbf{p}_2$、$\mathbf{p}_3$ 定义第一个线段，使用 $\mathbf{p}_3$、$\mathbf{p}_4$、$\mathbf{p}_5$、$\mathbf{p}_6$ 定义第二段，以此类推。注意，如果每条线段定义在参数 u 的(0, 1)区间上，那么在每段曲线的矩阵 $\mathbf{M}_I$ 都相同。尽管用这种方法生成的一组曲线是连续的，但是连接点处的导数并不连续。

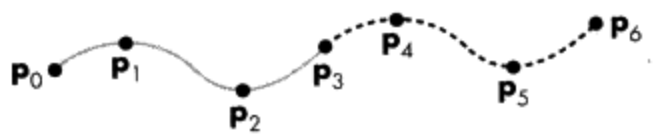


图12.10　插值曲线段的连接

12.4.1　调和函数

我们通过对插值多项式曲线方程稍做变换来深入探讨其平滑性。将插值系数代入多项式中，得到：

$$\mathbf{p}(u) = \mathbf{u}^T \mathbf{c} = \mathbf{u}^T \mathbf{M}_I \mathbf{p}$$

也可以将上式写为如下形式：

$$\mathbf{p}(u) = \mathbf{b}(u)^T \mathbf{p}$$

其中，

$$\mathbf{b}(u) = \mathbf{M}_I^T \mathbf{u}$$

是以如下4个调和函数（blending function）为元素的列矩阵：

$$\mathbf{b}(u) = \begin{bmatrix} b_0(u) \\ b_1(u) \\ b_2(u) \\ b_3(u) \end{bmatrix}$$

每一个调和多项式都是三次的。如果使用这些调和函数表述 $\mathbf{p}(u)$，则有：

$$\mathbf{p}(u) = b_0(u)\mathbf{p}_0 + b_1(u)\mathbf{p}_1 + b_2(u)\mathbf{p}_2 + b_3(u)\mathbf{p}_3 = \sum_{i=0}^{3} b_i(u)\mathbf{p}_i$$

从中可以看出多项式以控制点为权值进行调和，并且每个控制点对曲线的影响不同。三次插值多项式的调和函数如下，曲线如图12.11所示：

$$b_0(u) = -\frac{9}{2}\left(u - \frac{1}{3}\right)\left(u - \frac{2}{3}\right)(u - 1)$$

$$b_1(u) = \frac{27}{2}u\left(u - \frac{2}{3}\right)(u - 1)$$

$$b_2(u) = -\frac{27}{2}u\left(u - \frac{1}{3}\right)(u - 1)$$

$$b_3(u) = \frac{9}{2}u\left(u - \frac{1}{3}\right)\left(u - \frac{2}{3}\right)$$

因为所有调和函数的零值点都位于封闭区间[0, 1]内，因此调和函数曲线在区间内起伏很大，非常不平滑。这是由于插值要求曲线必须穿过控制点而不仅仅是靠近所致，这种现象在高阶插值多项式中更为明显。再加上连接点导数的不连续性导致了插值多项式在计算机图形学中的使用非常有限。然而其推导和分析过程将有助于我们获得更平滑的三次曲线类型。

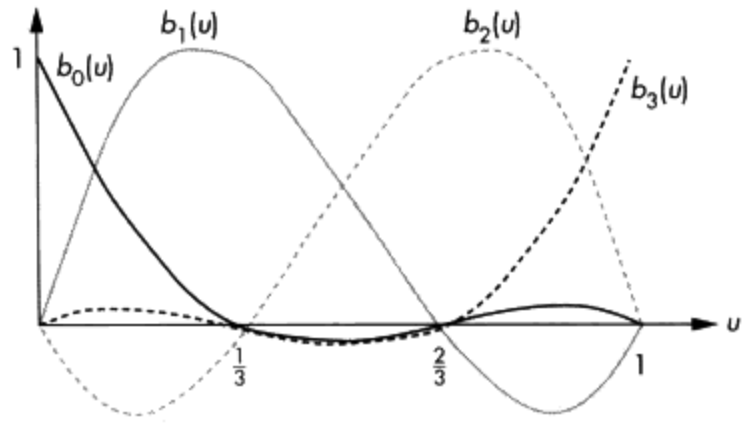


图 12.11　插值多项式的调和函数

12.4.2　三次插值曲面片

插值曲线可以自然地推广到插值曲面片。双三次曲面片（bicubic surface patch）可以表示为如下形式：

$$\mathbf{p}(u, v) = \sum_{i=0}^{3} \sum_{j=0}^{3} u^i v^j \mathbf{c}_{ij}$$

其中，$\mathbf{c}_{ij}$ 是包含 3 个元素的列矩阵，它表示多项式第 ij 项的 x、y 和 z 的系数。如果定义一个 4 × 4 的矩阵，每个元素是包含 3 个元素的列矩阵：

$$\mathbf{C} = [\, \mathbf{c}_{ij} \,]$$

则可以将曲面片表示为：

$$\mathbf{p}(u, v) = \mathbf{u}^T \mathbf{C} \mathbf{v}$$

其中，

$$\mathbf{v} = \begin{bmatrix} 1 \\ v \\ v^2 \\ v^3 \end{bmatrix}$$

这样通过 **C** 的 48 个元素即 16 个 3 元素向量，从而定义了一个双三次多项式曲面片。

利用已知的 16 个三维控制点 $\mathbf{p}_{ij}$, $i = 0, \cdots, 3, j = 0, \cdots, 3$，可以定义如图 12.12 所示插值曲面片。如果在 u 和 v 等间距处进行插值 $0, \frac{1}{3}, \frac{2}{3}, 1$，则可以得到 3 组方程，每组有 16 个方程，每个方程有 16 个未知数。例如，对于 $u = v = 0$，3 个独立的方程如下：

$$\mathbf{p}_{00} = [\, 1 \quad 0 \quad 0 \quad 0 \,] \mathbf{C} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \mathbf{c}_{00}$$

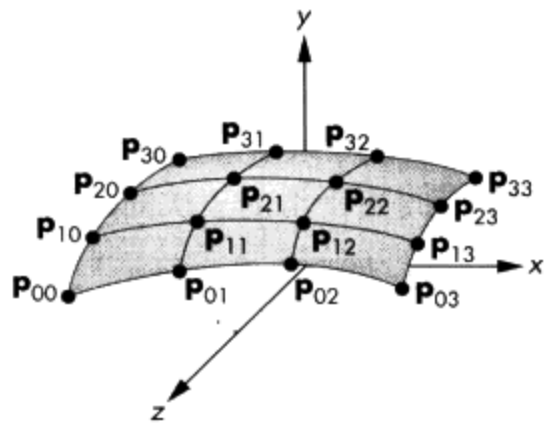


图 12.12　插值曲面片

我们并不是写出所有方程并求解，而是采用一个更直接的方式。当 $v=0$ 时，应得到一个以 u 为参数的曲线，并且插值 $\mathbf{p}_{00}$、$\mathbf{p}_{10}$、$\mathbf{p}_{20}$ 和 $\mathbf{p}_{30}$。利用插值曲线公式，可以将曲线写为如下形式：

$$\mathbf{p}(u,0)=\mathbf{u}^T\mathbf{M}_I\begin{bmatrix}\mathbf{p}_{00}\\ \mathbf{p}_{10}\\ \mathbf{p}_{20}\\ \mathbf{p}_{30}\end{bmatrix}=\mathbf{u}^T\mathbf{C}\begin{bmatrix}1\\0\\0\\0\end{bmatrix}$$

同样，$v=\frac{1}{3},\frac{2}{3},1$ 的其他值定义了另外 3 条插值曲线，曲线多项式都具有相似的形式。这些多项式组成了所有的 16 个方程：

$$\mathbf{u}^T\mathbf{M}_I\mathbf{P}=\mathbf{u}^T\mathbf{C}\mathbf{A}^T$$

其中，$\mathbf{A}$ 是 $\mathbf{M}_I$ 的逆矩阵。求解这些方程得到所求系数矩阵：

$$\mathbf{C}=\mathbf{M}_I\mathbf{P}\mathbf{M}_I^T$$

代入曲面方程，得到：

$$\mathbf{p}(u,v)=\mathbf{u}^T\mathbf{M}_I\mathbf{P}\mathbf{M}_I^T\mathbf{v}$$

可以从不同的角度分析上述结果。首先，插值曲面可以由插值曲线导出（其他类型曲线也可以采用该方法推广到曲面）。另外，可以将调和函数推广到曲面，通过使用 $\mathbf{M}_I^T\mathbf{u}$ 表示调和函数，可以将曲面片插值方程改写为如下的形式：

$$\mathbf{p}(u,v)=\sum_{i=0}^{3}\sum_{j=0}^{3}b_i(u)b_j(v)\mathbf{p}_{ij}$$

每一项 $b_i(u)b_j(v)$ 表示一个调和曲面片（blending patch），通过加权调和形成一个曲面，其权值为控制点的值。调和曲面片的基本特性由相应的插值曲线调和函数的性质决定，因此曲面的大多数特性和曲线相似。特别是因为函数 $b_i(u)b_j(v)$ 的零值点位于 u，v 的单位方格之内，所以调和曲面片不是非常平滑。利用该方法得到的曲面称为张量积曲面（tensor product surface），双三次张量积曲面是参数最高次数为 3 的曲面的子集，是一种可分离曲面（separable surface），可以写为如下形式：

$$\mathbf{p}(u,v)=\mathbf{f}(\mathbf{u})\mathbf{g}(v)$$

其中，$\mathbf{f}$ 和 $\mathbf{g}$ 分别是适当取值的行和列矩阵。这个曲面的优点在于可以独立地处理 u 或 v 的函数。

12.5　Hermite 曲线和曲面

生成插值曲线和曲面的方法也可以用来生成其他类型的曲线和曲面，其主要的区别在于控制点数据的使用。

12.5.1　Hermite 形式

假设开始时只有控制点 $\mathbf{p}_0$ 和 $\mathbf{p}_3$[①]，要求曲线在参数 $u=0$ 和 $u=1$ 时插值控制点。利用之前的推导过程，应该到两个方程：

$$\mathbf{p}(0)=\mathbf{p}_0=\mathbf{c}_0$$
$$\mathbf{p}(1)=\mathbf{p}_3=\mathbf{c}_0+\mathbf{c}_1+\mathbf{c}_2+\mathbf{c}_3$$

假设已知多项式在 $u=0$ 和 $u=1$ 处的导数，则可以推导出另外两个方程。三次参数多项式的导数是如下形式的二次参数多项式：

$$\mathbf{p}'(u)=\begin{bmatrix}\frac{dx}{du}\\ \frac{dy}{du}\\ \frac{dz}{du}\end{bmatrix}=\mathbf{c}_1+2u\mathbf{c}_2+3u^2\mathbf{c}_3$$

如果用 $\mathbf{p}'_0$ 和 $\mathbf{p}'_3$ 表示导数，那么这两个方程（见图 12.13）为：

$$\mathbf{p}'_0=\mathbf{p}'(0)=\mathbf{c}_1$$
$$\mathbf{p}'_3=\mathbf{p}'(1)=\mathbf{c}_1+2\mathbf{c}_2+3\mathbf{c}_3$$

可将其表示为矩阵形式：

$$\begin{bmatrix}\mathbf{p}_0\\ \mathbf{p}_3\\ \mathbf{p}'_0\\ \mathbf{p}'_3\end{bmatrix}=\begin{bmatrix}1&0&0&0\\ 1&1&1&1\\ 0&1&0&0\\ 0&1&2&3\end{bmatrix}\mathbf{c}$$

若令 $\mathbf{q}$ 表示数据矩阵：

$$\mathbf{q}=\begin{bmatrix}\mathbf{p}_0\\ \mathbf{p}_3\\ \mathbf{p}'_0\\ \mathbf{p}'_3\end{bmatrix}$$

求解上述方程，得到系数矩阵 $\mathbf{c}$：

$$\mathbf{c}=\mathbf{M}_H\mathbf{q}$$

其中，$\mathbf{M}_H$ 是 Hermite 几何矩阵：

$$\mathbf{M}_H=\begin{bmatrix}1&0&0&0\\ 0&0&1&0\\ -3&3&-2&-1\\ 2&-2&1&1\end{bmatrix}$$

所得多项式为：

$$\mathbf{p}(u)=\mathbf{u}^T\mathbf{M}_H\mathbf{q}$$

图 12.14 显示了该方法的应用，其中曲线段共享连接处插值点和导数，这样所得曲线多项式及其一阶导数在所有曲线段上都是连续的。

---

① 我们使用与插值一致的编码方式，12.6 节的 Bezier 曲线也是这样处理的。

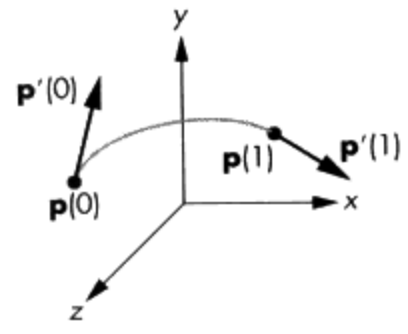


图 12.13　三次 Hermite 曲线的定义

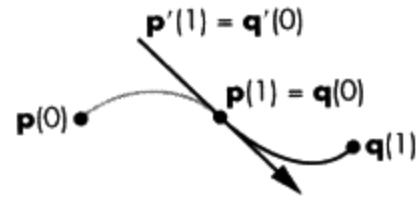


图 12.14　连接点的 Hermite 形式

通过变换上述多项式为如下形式来讨论 Hermite 形式平滑性的问题：

$$\mathbf{p}(u) = \mathbf{b}(u)^T\mathbf{q}$$

其中新的调和函数如下：

$$\mathbf{b}(u) = \mathbf{M}_H^T\mathbf{u} = \begin{bmatrix} 2u^3 - 3u^2 + 1 \\ -2u^3 + 3u^2 \\ u^3 - 2u^2 + u \\ u^3 - u^2 \end{bmatrix}$$

这 4 个函数在区间(0, 1)上没有零值点，而且比插值多项式的调和函数更加平滑（见习题 12.16）。

可以通过这些调和函数定义一个双三次 Hermite 曲面片：

$$\mathbf{p}(u, v) = \sum_{i=0}^{3}\sum_{j=0}^{3} b_i(u)b_j(v)\mathbf{q}_{ij}$$

其中，$\mathbf{Q} = [\mathbf{q}_{ij}]$是 $\mathbf{q}$ 到曲面的推广，表示曲面控制点数据。对于曲面片而言，这个方程恰恰是一个正规表达式。$\mathbf{Q}$ 的元素和 $\mathbf{p}(u, v)$的导数之间没有明确关系，通常选择 $\mathbf{Q}$ 的 4 个元素为曲面片 4 个角点的插值点，其他的元素为曲面片角点的导数。然而在大多数交互应用中，用户输入的是控制点数据，而并非其导数数据，因此，除非有求导公式，否则没有办法获得这些导数。然而利用 Hermite 曲线和曲面的推导方法可以推导出 12.6 节介绍的 Bezier 曲线和曲面。

12.5.2　几何与参数连续性

在讨论 Bezier 及样条曲线和曲面之前，我们先讨论曲线连续性和导数的一些相关问题。考虑图 12.15 中的连接点，假设连接点左侧曲线多项式是 $\mathbf{p}(u)$而右侧是 $\mathbf{q}(u)$。建立 $\mathbf{p}(u)$在 $u = 1$ 时与 $\mathbf{q}(u)$在 $u = 0$ 时，以及 $\mathbf{p}'(u)$在 $u = 1$ 时与 $\mathbf{q}'(u)$在 $u = 0$ 时的对应关系，以得到各种连续性条件。如果希望曲线是连续的，则需满足以下条件：

$$\mathbf{p}(1) = \begin{bmatrix} p_x(1) \\ p_y(1) \\ p_z(1) \end{bmatrix} = \mathbf{q}(0) = \begin{bmatrix} q_x(0) \\ q_y(0) \\ q_z(0) \end{bmatrix}$$

3 个分量在连接点必须对应相等，我们称这种特性为 C^0 参数连续性（parametric continuity）。

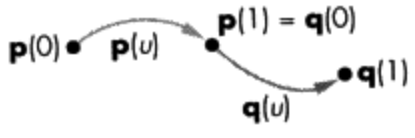


图 12.15　连接点的连续性

导数所要满足的条件与 Hermite 曲线中的要求一样，即

$$\mathbf{p}'(1)=\begin{bmatrix}p'_x(1)\\p'_y(1)\\p'_z(1)\end{bmatrix}=\mathbf{q}'(0)=\begin{bmatrix}q'_x(0)\\q'_y(0)\\q'_z(0)\end{bmatrix}$$

如果参数方程和一阶导数都对应相等，则曲线具有 C^1 参数连续性。

通过仔细观察几何图形，发现可以采取另一种方法来确保连续性。在三维空间中，曲线上某点的导数即是这一点的切线。假设只要求两个曲线段连接点的导数成比例，而不需要导数相等，即对于某个正数 α：

$$\mathbf{p}'(1)=\alpha\mathbf{q}'(0)$$

如果两个曲线的切线向量成比例，那么它们方向相同，但是模不同，则称该连续性为 G^1 几何连续性（geometric continuity）[①]。这种情况下只需要两个约束条件，而并非三个，可以利用剩下的一个自由度来满足其他条件。将这个思想推广到高阶导数，就可以同时得到 C^n 和 G^n 连续性。

尽管在连接点仅有 G^1 连续性的两条曲线在连接点具有连续的切线，连接点两侧切线的比例（也就是切线向量的模的比值）具有重要的作用。如图 12.16 所示，两条不同的曲线 $\mathbf{p}(u)$和 $\mathbf{q}(u)$共享端点，并且端点处切线方向相同、大小不同。这个结论常被用到许多画图程序，用户可以交互地改变切线大小，而保留其方向不变。然而，在其他的应用中，例如动画，若以一组曲线段描述对象的运动路径，G^1 连续性就不够了（见习题 12.11）。

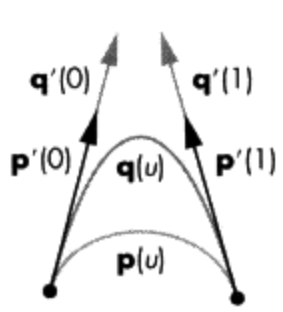


图 12.16　G^1 连续时改变切线矢量的大小

12.6　Bezier 曲线和曲面

将Hermite形式和插值形式进行对比是有争议的，这两类曲线形式具有某些相似性但却是明显不同的。尽管它们都具有三次多项式曲线形式，但是使用不相同的数据，因此它们不能在同等条件上进行比较。我们可以采用一组控制点推导出插值曲线，也可以使用相同控制点拟合曲线导数值推导出 Hermite 曲线。同样，利用这一组控制点还可以推导出非常逼近 Hermite 曲线的 Bezier 曲线，由于采用了与插值曲线相同的数据，所以与插值曲线具有可比较性。另外，由于 Bezier 曲线不需要导数数据，因此非常适用于图形学和 CAD。

12.6.1　Bezier 曲线

仍然使用 4 个控制点：$\mathbf{p}_0$、$\mathbf{p}_1$、$\mathbf{p}_2$ 和 $\mathbf{p}_3$，要求三次多项式 $\mathbf{p}(u)$插值端点：

$$\mathbf{p}_0=\mathbf{p}(0)$$

$$\mathbf{p}_3=\mathbf{p}(1)$$

① G^0 连续性等同于 C^0 连续性。

Bezier 提出并不使用控制点 $\mathbf{p}_2$ 和 $\mathbf{p}_3$ 进行插值，而是使用它们来拟合曲线在 $u=0$ 和 $u=1$ 的切线。可以使用如下线性拟合公式：

$$\mathbf{p}'(0) \approx \frac{\mathbf{p}_1 - \mathbf{p}_0}{\frac{1}{3}} = 3(\mathbf{p}_1 - \mathbf{p}_0)$$

$$\mathbf{p}'(1) \approx \frac{\mathbf{p}_3 - \mathbf{p}_2}{\frac{1}{3}} = 3(\mathbf{p}_3 - \mathbf{p}_2)$$

如图 12.17 所示。将上述公式代入参数多项式 $\mathbf{p}(u) = \mathbf{u}^T\mathbf{c}$ 在两个端点的导数公式，可以得到如下两个条件：

$$3\mathbf{p}_1 - 3\mathbf{p}_0 = \mathbf{c}_1$$
$$3\mathbf{p}_3 - 3\mathbf{p}_2 = \mathbf{c}_1 + 2\mathbf{c}_2 + 3\mathbf{c}_3$$

再加上插值方程

$$\mathbf{p}_0 = \mathbf{c}_0$$
$$\mathbf{p}_3 = \mathbf{c}_0 + \mathbf{c}_1 + \mathbf{c}_2 + \mathbf{c}_3$$

可以获得包含 4 个方程、每个方程有 4 个未知数的 3 组方程。采用之前的方法，求解得到：

$$\mathbf{c} = \mathbf{M}_B\mathbf{p}$$

其中，$\mathbf{M}_B$ 是 Bezier 几何矩阵：

$$\mathbf{M}_B = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -3 & 3 & 0 & 0 \\ 3 & -6 & 3 & 0 \\ -1 & 3 & -3 & 1 \end{bmatrix}$$

于是，三次 Bezier 多项式为：

$$\mathbf{p}(u) = \mathbf{u}^T\mathbf{M}_B\mathbf{p}$$

与插值多项式一样，对于一组控制点 $\mathbf{p}_0, \cdots, \mathbf{p}_n$，我们使用 $\mathbf{p}_0$、$\mathbf{p}_1$、$\mathbf{p}_2$ 和 $\mathbf{p}_3$ 生成第一条曲线，使用 $\mathbf{p}_3$、$\mathbf{p}_4$、$\mathbf{p}_5$ 和 $\mathbf{p}_6$ 生成第二条，以此类推。需要明确所得的曲线只具有 C^0 连续性，因为在连接点的左侧和右侧使用了不同的拟合公式，所以不具备 Hermite 多项式的 C^1 连续性。

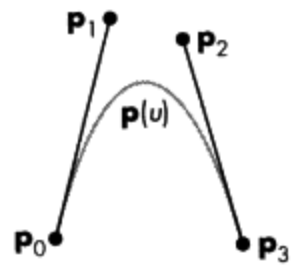


图 12.17　切线逼近

可以通过仔细分析图 12.18 所示的调和函数得出 Bezier 曲线的优势。将曲线写为：

$$\mathbf{p}(u) = \mathbf{b}(u)^T\mathbf{p}$$

其中，

$$\mathbf{b}(u) = \mathbf{M}_B^T\mathbf{u} = \begin{bmatrix} (1-u)^3 \\ 3u(1-u)^2 \\ 3u^2(1-u) \\ u^3 \end{bmatrix}$$

这 4 个函数是如下 Bernstein 多项式的特例：

$$b_{kd}(u) = \frac{d!}{k!(d-k)!}u^k(1-u)^{d-k}$$

该多项式具有显著的特性。首先，多项式所有的零值点位于 $u = 0$ 或者 $u = 1$ 的位置；而且，任意调和函数都有：

$$0 < b_{id}(u)$$

由于在 $0 < u < 1$ 区间内不存在零值点，因此每个调和函数曲线必定平滑，在如下区间也具有这一特性（见习题 12.4）：

$$b_{id}(u) \leqslant 1$$

并且

$$\sum_{i=0}^{d} b_{id}(u) = 1$$

在这些条件下，以调和函数表示的三次 Bezier 多项式如下：

$$\mathbf{p}(u) = \sum_{i=0}^{3} b_i(u)\mathbf{p}_i$$

$\mathbf{p}(u)$是一个凸性组合，因此，曲线必定位于 4 个控制点构成的凸包中，如图 12.19 所示。这样，尽管 Bezier 多项式曲线没有插值所有的控制点，也不会偏离它们。这两个特性，加之曲线采用的数据都是控制点，使用户易于以交互的方式使用 Bezier 曲线。用户可以输入 4 个控制点来定义一个初始曲线，然后通过操纵这些点来改变曲线的形状。

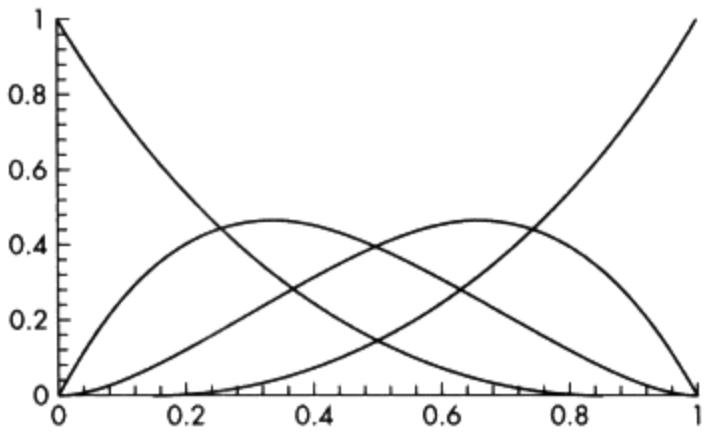


图 12.18　三次 Bezier 曲线的调和函数

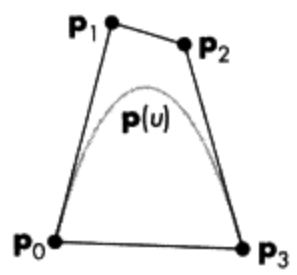


图 12.19　凸包与 Bezier 多项式曲线

12.6.2　Bezier 曲面片

我们可以通过修改调和函数生成 Bezier 曲面片（Bezier surface patch）。设 $\mathbf{P}$ 是控制点的 4 × 4 数组：

$$\mathbf{P} = \left[\mathbf{p}_{ij}\right]$$

那么相应的 Bezier 曲面片为：

$$\mathbf{p}(u, v) = \sum_{i=0}^{3}\sum_{j=0}^{3} b_i(u)b_j(v)\mathbf{p}_{ij} = \mathbf{u}^T\mathbf{M}_B\mathbf{P}\mathbf{M}_B^T\mathbf{v}$$

这个曲面片完全包含在由控制点（见图12.20）形成的凸包中，并且插值$\mathbf{p}_{00}$、$\mathbf{p}_{03}$、$\mathbf{p}_{30}$和$\mathbf{p}_{33}$。其他方程用于拟合曲面片角点位置的导数。

我们可以计算$u = v = 0$位置的角点$\mathbf{p}(u)$和一阶偏导数：

$$\mathbf{p}(0, 0) = \mathbf{p}_{00}$$

$$\frac{\partial \mathbf{p}}{\partial u}(0, 0) = 3(\mathbf{p}_{10} - \mathbf{p}_{00})$$

$$\frac{\partial \mathbf{p}}{\partial v}(0, 0) = 3(\mathbf{p}_{01} - \mathbf{p}_{00})$$

$$\frac{\partial^2 \mathbf{p}}{\partial u \partial v}(0, 0) = 9(\mathbf{p}_{00} - \mathbf{p}_{01} + \mathbf{p}_{10} - \mathbf{p}_{11})$$

前3个条件是Bezier曲线的推广，第4个条件可以看做曲面片在角点位置偏离平面的程度或者扭曲度的测量。这几个点定义的四边形如图12.21所示，只有扭曲度为零时，这些点才位于同一平面上。彩图7正是使用Bezier曲面参数多项式由高程数据生成的平滑曲面。

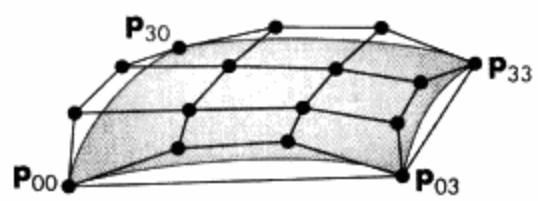


图12.20　Bezier曲面片

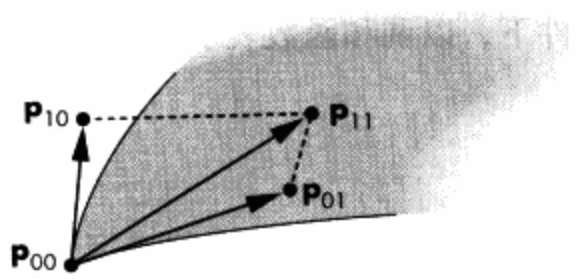


图12.21　Bezier曲面片角点位置的扭曲

12.7　三次B样条

三次Bezier曲线和曲面广泛在实际应用中采用，但其有一个根本的局限：在连接点（或者曲面片边缘）只有C^0连续性。例如，如果使用三次Bezier曲线来设计12.2节的模型飞机横截面，那么我们可能并不满意曲线段连接点的平滑性。

我们对三次多项式的利用似乎达到了极限，如果需要更加平滑的曲线，则不得不使用高阶多项式，或者缩短区间来使用更多的曲线段多项式。这两种策略都是可行的，但是还有另外一个策略：使用相同的控制点数据，但不要求多项式插值任何点。采用曲线逼近控制点的方式，可以在连接点获得更高的平滑性，在实际中会得到令人满意的结果。

12.7.1　三次B样条曲线

这一节将通过B样条曲线的特例阐述如何在连接点获得具有C^2连续性的三次曲线。我们将在12.8节对生成B样条的一般性方法进行简要介绍，该方法非常通用，Bezier曲线就是其特例。考虑一组控制点中的4个：$\{\mathbf{p}_{i-2}, \mathbf{p}_{i-1}, \mathbf{p}_i, \mathbf{p}_{i+1}\}$。我们之前介绍的方法是利用这4个点定义一条三次曲线，随着参数u从0到1变化，曲线的跨度从$\mathbf{p}_{i-2}$到$\mathbf{p}_{i+1}$，插值$\mathbf{p}_{i-2}$和$\mathbf{p}_{i+1}$。换一种方法，假设随着u从0到1变化，使得曲线仅仅跨越中间两个控制点，如图12.22所示。同样，我们使用$\{\mathbf{p}_{i-3}, \mathbf{p}_{i-2}, \mathbf{p}_{i-1}, \mathbf{p}_i\}$定义$\mathbf{p}_{i-2}$和$\mathbf{p}_{i-1}$，以及$\{\mathbf{p}_{i-1}, \mathbf{p}_i, \mathbf{p}_{i+1}, \mathbf{p}_{i+2}\}$定义$\mathbf{p}_i$和$\mathbf{p}_{i+1}$之间的一段曲线。假设曲线$\mathbf{p}(u)$是跨越$\mathbf{p}_{i-1}$和$\mathbf{p}_i$之间的曲线，其左侧的曲线$\mathbf{q}(u)$跨越$\mathbf{p}_{i-2}$和$\mathbf{p}_{i-1}$，令$\mathbf{p}(0)$和$\mathbf{p}(1)$位置的条件对应相等，类似之前的推导方法，可以使用矩阵$\mathbf{M}$表示三次多项式：

$$\mathbf{p}(u) = \mathbf{u}^T \mathbf{M} \mathbf{p}$$

其中，**p** 是控制点列向量：

$$\mathbf{p} = \begin{bmatrix} \mathbf{p}_{i-2} \\ \mathbf{p}_{i-1} \\ \mathbf{p}_i \\ \mathbf{p}_{i+1} \end{bmatrix}$$

可以使用相同的矩阵表示 $\mathbf{q}(u)$：

$$\mathbf{q}(u) = \mathbf{u}^T\mathbf{M}\mathbf{q}$$

其中，

$$\mathbf{q} = \begin{bmatrix} \mathbf{p}_{i-3} \\ \mathbf{p}_{i-2} \\ \mathbf{p}_{i-1} \\ \mathbf{p}_i \end{bmatrix}$$

原则上可以写出 $\mathbf{p}(0)$对应于 $\mathbf{q}(1)$的一组条件，而且可以写出曲线在 $\mathbf{p}(1)$位置的导数值对应于以 $\mathbf{q}(1)$为起点的右侧曲线多项式的导数值的条件。例如，条件

$$\mathbf{p}(0) = \mathbf{q}(1)$$

表示两条曲线连接点的连续性，而并没有要求插值任何一个控制点。根据这个条件可以得到求解 **M** 的一个方程。显而易见，有多组类似的条件可供使用，每一组条件定义一个不同的矩阵。

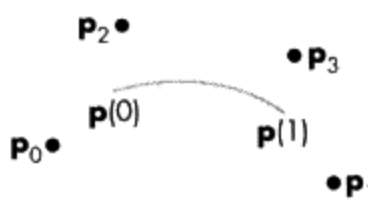


图 12.22　由 4 个控制点中间的两个点定义的曲线

实际上两段曲线是以对称方式逼近连接点的，据此我们采取一条捷径来推导出矩阵的一般形式。不可以使用 $\mathbf{p}_{i-3}$表示任何在 $\mathbf{q}(1)$位置的条件，因为这个控制点不会出现在 $\mathbf{p}(u)$的方程中。同样，也不能使用 $\mathbf{p}_{i+1}$ 计算 $\mathbf{p}(0)$处的任何条件。对称性的两个条件如下：

$$\mathbf{p}(0) = \mathbf{q}(1) = \frac{1}{6}(\mathbf{p}_{i-2} + 4\mathbf{p}_{i-1} + \mathbf{p}_i)$$

$$\mathbf{p}'(0) = \mathbf{q}'(1) = \frac{1}{2}(\mathbf{p}_i - \mathbf{p}_{i-2})$$

可将 $\mathbf{p}(u)$写做系数矩阵 **c** 的形式：

$$\mathbf{p}(u) = \mathbf{u}^T\mathbf{c}$$

上述条件可以表示为：

$$\mathbf{c}_0 = \frac{1}{6}(\mathbf{p}_{i-2} + 4\mathbf{p}_{i-1} + \mathbf{p}_i)$$

$$\mathbf{c}_1 = \frac{1}{2}(\mathbf{p}_i - \mathbf{p}_{i-2})$$

在 $\mathbf{p}(1)$位置应用对称性条件，则有：

$$\mathbf{p}(1) = \mathbf{c}_0 + \mathbf{c}_1 + \mathbf{c}_2 + \mathbf{c}_3 = \frac{1}{6}(\mathbf{p}_{i-1} + 4\mathbf{p}_i + \mathbf{p}_{i+1})$$

$$\mathbf{p}'(1) = \mathbf{c}_1 + 2\mathbf{c}_2 + 3\mathbf{c}_3 = \frac{1}{2}(\mathbf{p}_{i+1} - \mathbf{p}_{i-1})$$

利用这4个方程求解系数矩阵 $\mathbf{c}$，进而可以求解 **B** 样条几何矩阵 $\mathbf{M}_S$：

$$\mathbf{M}_S = \frac{1}{6}\begin{bmatrix} 1 & 4 & 1 & 0 \\ -3 & 0 & 3 & 0 \\ 3 & -6 & 3 & 0 \\ -1 & 3 & -3 & 1 \end{bmatrix}$$

我们通过分析以下调和函数来阐述利用上述矩阵生成的多项式的重要特性：

$$\mathbf{b}(u) = \mathbf{M}_S^T\mathbf{u} = \frac{1}{6}\begin{bmatrix} (1-u)^3 \\ 4 - 6u^2 + 3u^3 \\ 1 + 3u + 3u^2 - 3u^3 \\ u^3 \end{bmatrix}$$

函数曲线如图12.23所示。采用与Bezier多项式相同的方法可以证明：

$$\sum_{i=0}^{3} b_i(u) = 1$$

而且，在区间 $0 \leqslant u \leqslant 1$：

$$0 \leqslant b_i(u) \leqslant 1$$

因此，曲线必定位于控制点凸包内，如图12.24所示。注意，曲线仅仅存在于凸包的部分区域。这样定义的曲线具有 C^1 连续性，可以通过计算在 $u=0$ 和 $u=1$ 的 $\mathbf{p}''(u)$证明它也具有 C^2 连续性[①]，并且该值在曲线左侧和右侧保持一致。正因为如此，样条曲线非常重要。从物理学观点看，金属可以弯曲到二次导数连续的程度。从视觉效果看，由 C^2 连续的三次曲线段组成的曲线看上去是平滑的，即使是在连接点。

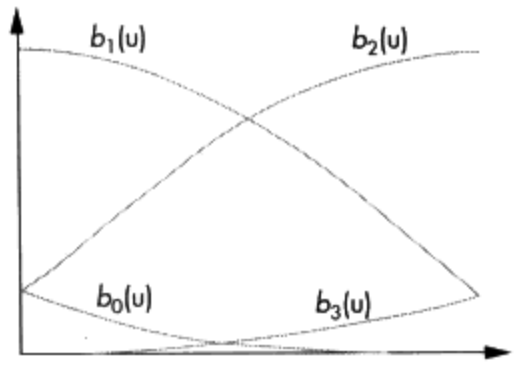


图12.23　样条曲线的调和函数

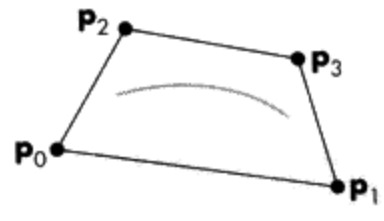


图12.24　样条曲线凸包

尽管使用与三次Bezier曲线相同的控制点数据推导出的三次样条曲线更平滑，但是我们必须意识到其工作量是Bezier曲线或插值曲线的3倍，原因是曲线只跨越控制点 $i-1$ 和 i 之间，而使用相同数据的Bezier曲线则跨越控制点 $i-2$ 和 $i+1$ 之间。因此，每增加一个控制点，必须重新计算样条曲线，然而对于Bezier曲线，增加3个控制点才需要重新计算一条曲线。

12.7.2　B样条和基函数

如果从单一控制点的角度而不是单一区间的角度来讨论样条曲线，则有助于更深入地分析样条曲线的特性。每一个控制点对处在其相邻4个区间的样条都有贡献，这一特性确保了样条曲线的局

① 如果仅仅关心 G^2 连续性，而不是 C^2 连续性，那么在设计曲线时可以使用多余的自由度增加灵活性，参阅Barsky [Bar83]。

部性。也就是说，如果改变一个控制点，仅仅影响 4 个相邻区间曲线。例如，在 $u = 0$ 和 $u = 1$ 之间，控制点 $\mathbf{p}_i$ 与调和函数 $b_2(u)$相乘，却以权因子 $b_1(u + 1)$影响左侧区间的曲线 $\mathbf{q}(u)$。因此为了得到左边的区间，必须将 u 值向左移 1。

某个控制点的总权值可以用 $B_i(u)\mathbf{p}_i$ 表示，其中 B_i 是如下形式的函数：

$$B_i(u) = \begin{cases} 0 & u < i-2 \\ b_0(u+2) & i-2 \leqslant u < i-1 \\ b_1(u+1) & i-1 \leqslant u < i \\ b_2(u) & i \leqslant u < i+1 \\ b_3(u-1) & i+1 \leqslant u < i+2 \\ 0 & u \geqslant i+2 \end{cases}$$

该函数曲线如图 12.25 所示。给定一组控制点 $\mathbf{p}_0, \cdots, \mathbf{p}_m$，可以用单个公式表示整个样条曲线[①]：

$$\mathbf{p}(u) = \sum_{i=1}^{m-1} B_i(u)\mathbf{p}_i$$

上述公式表明 $B(u - i)$函数集中的每个函数都是由另一函数移位得到的，这个函数集合形成了三次 B 样条曲线的基函数。给定一组控制点，在整个区间对基函数线性组合形成了一个分段多项式曲线 $\mathbf{p}(u)$。图 12.26 显示了 B 样条曲线以及每个基函数的贡献值。将上述观点加以推广，在区间内允许高阶多项式，并且允许不同区间使用不同的多项式，据此我们将在12.8节阐述关于样条函数的一般性原理。

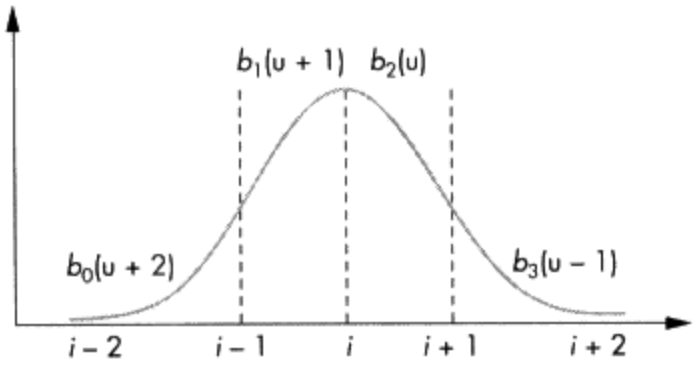


图 12.25　样条曲线基函数

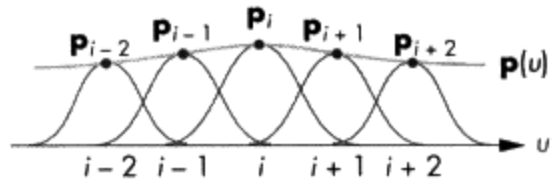


图 12.26　区间逼近函数

12.7.3　样条曲面

可以采用与之前相似的方法定义 B 样条曲面。从 B 样条调和函数入手，曲面片可以表示为：

$$\mathbf{p}(u, v) = \sum_{i=0}^{3}\sum_{j=0}^{3} b_i(u)b_j(v)\mathbf{p}_{ij}$$

这个公式和其他曲面片表示形式相同，但是从图 12.27 可以看出，这个曲面片仅仅位于中央区域，因此工作量是Bezier曲面的9倍。然而，因为凸包性质的继承性，以及在B样条曲线性质决定的曲面边缘的附加连续性条件，使用相同数据的 B 样条曲面片比 Bezier 曲面片显著平滑。

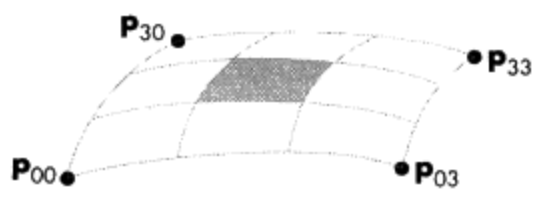


图 12.27　样条曲面片

① 12.8 节将确定样条曲线的起始和终止条件。

12.8　普通B样条

假设已知一组控制点 $\mathbf{p}_0, \cdots, \mathbf{p}_m$，一般拟合曲线的方法就是寻找一个定义在区间 $u_{\min} \leqslant u \leqslant u_{\max}$ 上的函数 $\mathbf{p}(u) = [x(u)\ \ y(u)\ \ z(u)]^T$，要求所表示的曲线是平滑的而且在某种程度上是逼近控制点的。假设已知一组称为节点（knot）的参数值$\{u_k\}$，满足如下条件：

$$u_{\min} = u_0 \leqslant u_1 \leqslant \cdots \leqslant u_n = u_{\max}$$

则称序列 $u_0, u_1, \cdots, u_n$ 为节点数组（knot array）[①]。样条曲线多项式 $\mathbf{p}(u)$是定义在节点上的 d 阶多项式：

$$\mathbf{p}(u) = \sum_{j=0}^{d} \mathbf{c}_{jk} u^j, \quad u_k < u < u_{k+1}$$

因此，定义一条 d 阶样条曲线，必须确定 $n(d+1)$个三维系数 $\mathbf{c}_{jk}$，可以通过在节点应用各种连续性条件并且在控制点应用插值条件来获得所需要的方程。

例如，如果 $d=3$，则每个节点区间上的曲线多项式都是三次的，那么，对于一个给定的 n，必须定义 $4n$ 个条件。$n+1$ 个节点的序列包含 $n-1$ 个内部节点，如果要求在内部节点有 C^2 连续性，则需要 $3n-3$ 个条件。此外，如果要求插值 $n+1$ 个控制点，则总共需要 $4n-2$ 个条件。当然，可以有多种方式确定另外两个条件，例如，拟合曲线端点的切线。然而，这个特殊的样条是全局范围求解的，我们必须求解包含$4n$个未知数的$4n$个方程，而且求解每一个系数都依赖于所有的控制点。因此，尽管形成一条插值所有控制点并且平滑的样条曲线，但其并不适用于计算机图形学和CAD。

12.8.1　B样条的递归定义

定义B样条曲线的方法基于一组基函数或调和函数，每个基函数仅在某些节点区间内定义为非零值。可以将 $\mathbf{p}(u)$表示为：

$$\mathbf{p}(u) = \sum_{i=0}^{m} B_{id}(u)\mathbf{p}_i$$

其中，函数 $B_{id}(u)$是区间$(u_{i_{\min}}, u_{i_{\max}})$上的 d 阶多项式（除节点处），而且在区间以外取值为零。B样条的名字来自于术语基样条（Basic Spline），由于一组函数$\{B_{id}(u)\}$形成给定节点序列和阶的基函数。尽管有很多定义样条基函数的方法，但是Cox-deBoor递归定义方法尤其重要[②]：

$$B_{k0} = \begin{cases} 1, & u_k \leqslant u \leqslant u_{k+1} \\ 0, & \text{其他} \end{cases}$$

$$B_{kd} = \frac{u - u_k}{u_{k+d} - u_k} B_{k,d-1}(u) + \frac{u_{k+d} - u}{u_{k+d+1} - u_{k+1}} B_{k+1,\,d-1}(u)$$

第一级基函数 B_{k0} 中的每个函数只在某一特定区间上是常数，其他地方取值为零；第二级基函数 B_{k1} 中的每个函数在两个区间上是一次函数，其他地方取值为零；第三级基函数 B_{k2} 中的每个函数在3个区间上是二次函数；以此类推（见图12.28）。总之，B_{kd} 中的每个函数在 u_k 与 u_{k+d+1} 之间的 $d+1$个区间上是 d 阶非零多项式，并且节点处具有 C^{d-1} 连续性，而且满足凸包性条件：

---

① 大多数研究人员称这个节点序列为向量，但是这与仅使用向量表示有向线段的思想相矛盾。

② 这个公式也被称为deCasteljau递归。

$$\sum_{i=0}^{m} B_{i,d}(u) = 1$$

和

$$0 \leqslant B_{id}(u) \leqslant 1$$

然而，由于每一个 B_{id} 在 $d+1$ 个区间都是非零的，所以每一个控制点只影响 $d+1$ 个区间，所得曲线将落在 $d+1$ 个控制点形成的凸包中。

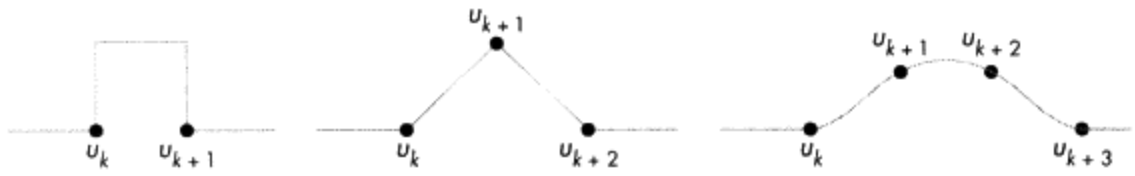


图 12.28　前 3 个基函数

仔细分析 Cox-deBoor 公式发现，每一步递归都是对前一步所得函数的线性插值，即 k 阶多项式的线性插值得到 $k+1$ 阶多项式。

样条基函数由给定阶和节点序列决定，注意，因为递归定义从 u_0 到 u_{n+1} 的样条需要 u_0 到 u_{n+d} 的节点序列，所以需要 $d-1$ 个额外的节点来定义样条，这些额外的节点值由样条曲线起点和终点的条件决定。

除了 $u_k \leqslant u_{k+1}$，我们并没有讨论关于节点取值的问题。如果递归等于 1 的时候出现了 0/0，那么可以取重复的（或者说多重的）节点；等间距的节点序列定义的样条称为均匀样条（uniform spline）；然而，还可以通过采用非均匀间隔的节点但是仍采用重复($u_k = u_{k+1}$)节点得到更加灵活的样条曲线。下面讨论其中几种情形。

12.8.2　均匀样条

将 Cox-deBoor 公式应用到等间距节点序列{0, 1, 2, ⋯, n}上，则可以推导出 12.7 节讨论的三次 B 样条曲线。使用与前面一样的编号方式（Cox-deBoor 公式下标的移位），在节点 k 和 $k+1$ 之间采用控制点 $\mathbf{p}_{k-1}$、$\mathbf{p}_k$、$\mathbf{p}_{k+1}$ 和 $\mathbf{p}_{k+2}$，仅仅得到一条定义在区间 $u=0$ 到 $u=n-1$ 之上的曲线，如图 12.29 所示的样条曲线并没有跨越 4 个节点。特定情况下，可以利用控制点的周期性定义跨越整个节点序列的样条曲线，如图 12.30 所示，这个均匀周期性 B 样条的特性在于其每个基函数都是通过对前一个基函数移位得到的。

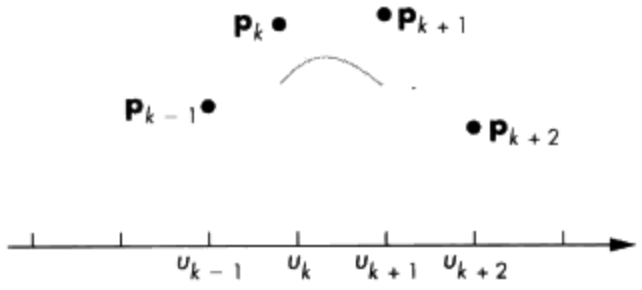


图 12.29　均匀 B 样条曲线

图 12.30　均匀周期 B 样条曲线

12.8.3　非均匀 B 样条

采用多重节点序列可以使得样条曲线更靠近重复节点，如果端节点重复 $d+1$ 次，d 阶 B 样条必定插值该点。因此，解决样条未能插值整个控制点序列的一个方法就是重复端节点，进行强制插值，并在其他地方采用均匀间隔节点，这样形成的样条称为开放样条（open spline）。

节点序列{0, 0, 0, 0, 1, 2, ⋯, $n-1$, n, n, n, n}经常用于生成三次B样条。而序列{0, 0, 0, 0, 1, 1, 1, 1}非常特殊，它所对应的三次B样条恰好是三次Bezier曲线。通常可以重复内部节点，从而得到任意间隔的节点序列。

12.8.4 NURBS

讨论B样条时，我们使用向量$[x(u)\ y(u)\ z(u)]^T$表示$\mathbf{p}(u)$，而在二维空间中，只需使用$[x(u)\ y(u)]^T$，其他方程保持不变。实际上，推广到四维空间，B样条方程仍保持不变。考虑如下三维控制点：

$$\mathbf{p}_i = [x_i \quad y_i \quad z_i]$$

在加权齐次坐标下，该点可表示为：

$$\mathbf{q}_i = w_i \begin{bmatrix} x_i \\ y_i \\ z_i \\ 1 \end{bmatrix}$$

其思想是使用权重w_i来调整控制点的重要性。我们可以使用这些加权的点来生成四维B样条，其中前3个分量是加权控制点的B样条：

$$\mathbf{q}(u) = \begin{bmatrix} x(u) \\ y(u) \\ z(u) \end{bmatrix} = \sum_{i=0}^{n} B_{i,d}(u) w_i \mathbf{p}_i$$

w分量是由权重集合导出的标量B样条多项式：

$$w(u) = \sum_{i=0}^{n} B_{i,d}(u) w_i$$

在奇次坐标系下，w分量可能不等于1，因此必须通过透视除法来导出三维点：

$$\mathbf{p}(u) = \frac{1}{w(u)} \mathbf{q}(u) = \frac{\sum_{i=0}^{n} B_{i,d}(u) w_i \mathbf{p}_i}{\sum_{i=0}^{n} B_{i,d}(u) w_i}$$

$\mathbf{p}(u)$的每个分量都是u的有理函数，而且由于对节点没有施加任何约束条件，应该到一个非均匀有理B样条曲线（NonUniform Rational B-Spline，NURBS）。

NURBS曲线保留了三维B样条曲线的所有特性，比如凸包性和连续性，加之其自身的两个特性，使得NURBS曲线在计算机图形学和CAD中具有重要地位。

如果对B样条曲线或曲面应用仿射变换，可以变换控制点得到与B样条曲线和曲面相同的曲线和曲面。但是由于透视变换不是仿射变换，因此绝大多数样条在透视图里不能得到正确的显示，但是NURBS曲线的透视除法确保了NURBS曲线可以在透视图里得到正确的显示。

二次曲面通常由代数隐式形式表示，如果使用非有理样条，仅仅能够近似表示这些曲面。然而，二次曲面可以表示为二次NURBS曲线的特例，因此，对于广泛应用的曲线和曲面可以采用单一的NURBS曲线进行建模（见习题12.14和习题12.15）。彩图9显示了由NURBS建模方法生成的网格，该网格形成了彩图5中的曲面。OpenGL最终使用多边形绘制NURBS网格。

12.8.5 Catmull-Rom样条

如果放宽曲线和曲面必须依赖控制点凸包的限制，那么就可以使用已知数据来形成其他类型的样条，其中最常用的是Catmull-Rom样条。

仍然使用推导Bezier曲线的4个控制点 $\mathbf{p}_0$、$\mathbf{p}_1$、$\mathbf{p}_2$ 和 $\mathbf{p}_3$。如果并不插值 $\mathbf{p}_0$ 和 $\mathbf{p}_1$，而是插值中间的 $\mathbf{p}_1$ 和 $\mathbf{p}_2$，则三次多项式为：

$$\mathbf{p}(0) = \mathbf{p}_1$$
$$\mathbf{p}(1) = \mathbf{p}_2$$

这样，与B样条一样，多项式将被定义在一个小区间内，而且每增加一个新的控制点就会生成一条新的曲线。

我们使用点 $\mathbf{p}_0$ 和 $\mathbf{p}_3$ 来拟合控制点 $\mathbf{p}_0$ 和 $\mathbf{p}_1$ 处的切线（如图 12.31 所示）：

$$\mathbf{p}'(0) \approx \frac{\mathbf{p}_2 - \mathbf{p}_0}{2}$$
$$\mathbf{p}'(1) \approx \frac{\mathbf{p}_3 - \mathbf{p}_1}{2}$$

这样，对以下曲线具有 4 个条件：

$$\mathbf{p}(u) = \mathbf{c}_0 + \mathbf{c}_1 u + \mathbf{c}_2 u^2 + \mathbf{c}_3 u^3$$

从而得出 4 个方程：

$$\mathbf{p}_1 = \mathbf{c}_0$$
$$\mathbf{p}_2 = \mathbf{c}_0 + \mathbf{c}_1 + \mathbf{c}_2 + \mathbf{c}_3$$
$$\frac{\mathbf{p}_2 - \mathbf{p}_0}{2} = \mathbf{c}_1$$
$$\frac{\mathbf{p}_3 - \mathbf{p}_1}{2} = \mathbf{c}_1 + 2\mathbf{c}_2 + 3\mathbf{c}_3$$

注意，由于 u 从0到1变化，并且仅仅从 $\mathbf{p}_1$ 到 $\mathbf{p}_2$，于是 $\mathbf{p}_0$ 和 $\mathbf{p}_2$ 被参数空间的两个单位 $\mathbf{p}_1$ 和 $\mathbf{p}_3$ 间隔。除此之外，这4个条件确保了曲线的连续性，而且在控制点有连续的一阶导数，但是曲线并不满足凸包性。

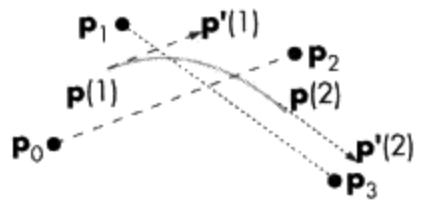


图 12.31　构建 Catmull-Rom 样条曲线

求解这 4 个方程得到：

$$\mathbf{p}(u) = \mathbf{u}^T \mathbf{M}_R \mathbf{p}$$

其中，$\mathbf{M}_R$ 是 Catmull-Rom 几何矩阵：

$$\mathbf{M}_R = \frac{1}{2}\begin{bmatrix} -1 & 3 & -3 & 0 \\ 2 & -5 & 4 & -1 \\ -1 & 0 & 1 & 0 \\ 0 & 2 & 0 & 0 \end{bmatrix}$$

12.9　曲线和曲面的绘制

利用曲线和曲面建立场景之后，需要寻找一种绘制的方法。有多种方法可供选择：其一，计算从投影中心射出的光线与物体的交点。然而，除了二次曲面（见 12.11 节），交点计算需要求解高阶

非线性方程，而且阶数一般都非常高，难以进行实时计算。第二个方法是使用大量的点，以构成标准平面物体，再通过标准平面物体对曲面进行逼近。我们将重点放在参数多项式曲线和曲面的绘制方法上。

12.9.1　多项式求值方法

假设有一个标准区间上的多项式：

$$\mathbf{p}(u) = \sum_{i=0}^{n} \mathbf{c}_i u^i, \quad 0 \leqslant u \leqslant 1$$

可以通过一组参数值$\{u_k\}$计算$\mathbf{p}(u)$的值，然后使用折线（或者GL_LINE_STRIP）来近似表示曲线。我们将各项进行组合，而不是独立地计算每一项u_k，得到下面的形式：

$$\mathbf{p}(u) = \mathbf{c}_0 + u(\mathbf{c}_1 + u(\mathbf{c}_2 + u(\cdots + \mathbf{c}_n u)))$$

这样，计算每一个$p(u_k)$仅仅需要n次乘法，这个算法被称为Horner算法。对于标准三次多项式$\mathbf{p}(u)$，各项组合为如下形式：

$$\mathbf{p}(u) = \mathbf{c}_0 + u(\mathbf{c}_1 + u(\mathbf{c}_2 + u\mathbf{c}_3))$$

如果$\{u_i\}$是非均匀间隔的，则可以使用前向差分法（forward difference）来计算$\mathbf{p}(u_k)$，那么只需复杂度为$O(n)$的加法运算，而不需要乘法。前向差分法具有如下的递归定义：

$$\Delta^{(0)}\mathbf{p}(u_k) = \mathbf{p}(u_k)$$

$$\Delta^{(1)}\mathbf{p}(u_k) = \mathbf{p}(u_{k+1}) - \mathbf{p}(u_k)$$

$$\Delta^{(m+1)}\mathbf{p}(u_k) = \Delta^{(m)}\mathbf{p}(u_{k+1}) - \Delta^{(m)}\mathbf{p}(u_k)$$

如果$u_{k+1} - u_k = h$是一个常数，n阶多项式是$\mathbf{p}(u)$，则可以证明：对于所有的k，$\Delta^{(n)}\mathbf{p}(u_k)$是一个常数。图12.32表示如何利用此结论求解三次多项式：

$$p(u) = 1 + 3u + 2u^2 + u^3$$

计算$\Delta^{(n)}p(u_0)$需要前$n+1$个$p(u_k)$的值，如果已经求得$\Delta^{(n)}p(u_0)$，则可以将其复制到如图12.33所示的表中，使用递推来计算后续的$p(u_k)$的值：

$$\Delta^{(m-1)}(p_{k+1}) = \Delta^{(m)}p(u_k) + \Delta^{(m-1)}p(u_k)$$

这个方法非常有效，但也是有缺陷的，因其仅仅适用于均匀网格，而且易于累积数值误差。

t	0	1	2	3	4	5
$\mathbf{p}$	1	7	23	55	109	191
$\Delta^{(1)}\mathbf{p}$	6	16	32	54	82	
$\Delta^{(2)}\mathbf{p}$	10	16	22	28		
$\Delta^{(3)}\mathbf{p}$	6	6	6			

图12.32　构造前向差分表

t	0	1	2	3	4	5
$\mathbf{p}$	1	7	23	55	109	191
$\Delta^{(1)}\mathbf{p}$	6	16	32	54	82	
$\Delta^{(2)}\mathbf{p}$	10	16	22	28		
$\Delta^{(3)}\mathbf{p}$	6	6	6			

图12.33　前向差分表的使用

12.9.2　递归细分Bezier多项式

Bezier曲线的递归细分方法可以实现精良的绘制，该方法基于Bezier曲线的凸包，而且不需要显式计算多项式。假设已知一个三次Bezier多项式（这个方法也适用于高次Bezier曲线）曲线位于

控制点凸包内，我们将曲线分为两个独立的多项式$\mathbf{l}(u)$和$\mathbf{r}(u)$，其参数区间是原区间的一半。由于原多项式是三次的，分割后的$\mathbf{l}(u)$和$\mathbf{r}(u)$也是三次的。注意，因为$\mathbf{l}(u)$和$\mathbf{r}(u)$参数区间是原区间的一半，所以必须对$\mathbf{l}$和$\mathbf{r}$的参数u进行参数变换。这样，随着u在区间(0,1)范围变化，$\mathbf{l}(u)$形成$\mathbf{p}(u)$的左半轨迹，$\mathbf{r}(u)$是右半轨迹，各由 4 个控制点定义并形成其凸包。使用$\{\mathbf{l}_0, \mathbf{l}_1, \mathbf{l}_2, \mathbf{l}_3\}$和$\{\mathbf{r}_0, \mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3\}$表示这两组控制点，$\mathbf{p}(u)$的原始控制点是$\{\mathbf{p}_0, \mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3\}$，控制点及其凸包如图 12.34 所示。注意，$\mathbf{l}$和$\mathbf{r}$的凸包都位于$\mathbf{p}$的凸包内，这一特性称为 Bezier 曲线的变差缩减性（variation-diminishing property）。

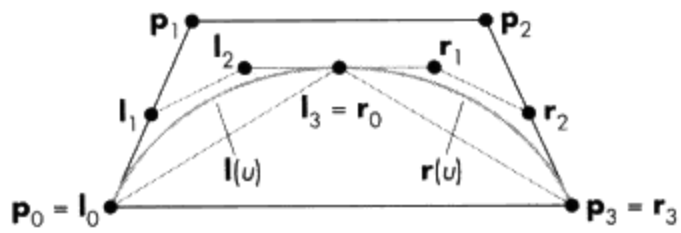


图 12.34　凸包和控制点

对于左边的多项式曲线，可以通过测量线段$\mathbf{l}_1\mathbf{l}_2$距离线段$\mathbf{l}_0\mathbf{l}_3$的距离判断该凸包的扁平性。如果距离足够近，则可以通过线段来代替曲线；反之，可将$\mathbf{l}$分为两部分并且继续判断两个新凸包的扁平性。由此定义了一个递归方法，而不需要计算多项式曲线的值，不过仍需要讨论如何确定$\{\mathbf{l}_0, \mathbf{l}_1, \mathbf{l}_2, \mathbf{l}_3\}$和$\{\mathbf{r}_0, \mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3\}$。下面介绍如何求得$\mathbf{l}(u)$的凸包，计算对于$\mathbf{r}(u)$的凸包是对称的。首先考虑如下的曲线多项式：

$$\mathbf{p}(u) = \mathbf{u}^T \mathbf{M}_B \begin{bmatrix} \mathbf{p}_0 \\ \mathbf{p}_1 \\ \mathbf{p}_2 \\ \mathbf{p}_3 \end{bmatrix}$$

其中，

$$\mathbf{M}_B = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -3 & 3 & 0 & 0 \\ 3 & -6 & 3 & 0 \\ -1 & 3 & -3 & -1 \end{bmatrix}$$

多项式$\mathbf{l}(u)$必须插值$\mathbf{p}(0)$和$\mathbf{p}\left(\frac{1}{2}\right)$，因此，

$$\mathbf{l}(0) = \mathbf{l}_0 = \mathbf{p}_0$$

$$\mathbf{l}(1) = \mathbf{l}_3 = \mathbf{p}\left(\frac{1}{2}\right) = \frac{1}{8}(\mathbf{p}_0 + 3\mathbf{p}_1 + 3\mathbf{p}_2 + \mathbf{p}_3)$$

当$u = 0$时，$\mathbf{l}$的斜率与$\mathbf{p}$的斜率相等，但是，因为$\mathbf{l}$的参数u的区间是$(0, \frac{1}{2})$，而$\mathbf{p}(u)$的参数u的区间是(0, 1)，则有参数变换$\bar{u} = 2u$。因此，$\mathbf{l}$和$\mathbf{p}$的导数的关系是$\mathrm{d}\bar{u} = 2\mathrm{d}u$，而且

$$\mathbf{l}'(0) = 3(\mathbf{l}_1 - \mathbf{l}_0) = \mathbf{p}'(0) = \frac{3}{2}(\mathbf{p}_1 - \mathbf{p}_0)$$

同样，中间位置的导数为：

$$\mathbf{l}'(1) = (\mathbf{l}_3 - \mathbf{l}_2) = \mathbf{p}'\left(\frac{1}{2}\right) = \frac{3}{8}(-\mathbf{p}_0 - \mathbf{p}_1 + \mathbf{p}_2 + \mathbf{p}_3)$$

可以对这4个方程进行代数求解，此外，还可以借助图12.35所示的几何作图法同时求得左侧和右侧的控制点集合。首先，插值条件要求：

$$\mathbf{l}_0 = \mathbf{p}_0$$

$$\mathbf{r}_3 = \mathbf{p}_3$$

可以通过在上述4个方程上进行代入求得曲线左、右两个端点的斜率而得到：

$$\mathbf{l}_1 = \frac{1}{2}(\mathbf{p}_0 + \mathbf{p}_1)$$

$$\mathbf{r}_2 = \frac{1}{2}(\mathbf{p}_2 + \mathbf{p}_3)$$

内部控制点由以下方程给定：

$$\mathbf{l}_2 = \frac{1}{2}\left(\mathbf{l}_1 + \frac{1}{2}(\mathbf{p}_1 + \mathbf{p}_2)\right)$$

$$\mathbf{r}_1 = \frac{1}{2}\left(\mathbf{r}_2 + \frac{1}{2}(\mathbf{p}_1 + \mathbf{p}_2)\right)$$

最后，共享的中间控制点由下面的方程给定：

这个公式的优点在于可以仅使用变换（一分为二）和加法就可以确定两组控制点集合。然而，分割方法的优点之一在于它的自适应性，而且在绘制过程中可能只对某些边进行细分。由于实际的曲线绘制只发生在绘制过程的光栅化阶段，而且绘制是在屏幕坐标或窗口坐标系里进行的，因此显示器的分辨率决定了凸包的细分次数（见习题12.18）。

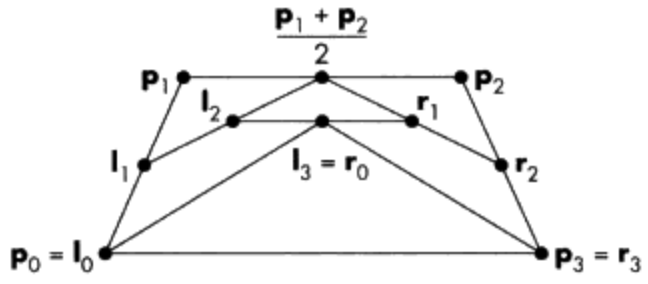


图12.35　曲线细分

12.9.3　其他类型多项式曲线的细分绘制

曲线多项式可以是Bezier多项式，也可以是插值多项式，还可以是B样条曲线多项式，或者是某控制点集的其他类型的多项式。由于Bezier细分算法的高效性，通常将其他类型的曲线转换为Bezier曲线，然后使用细分算法。通过Bezier曲线公式可以推导出与其他类型曲线多项式之间的转换算法。对于一个三次Bezier曲线，可以将其表示为如下形式的Bezier矩阵$\mathbf{M}_B$形式：

$$\mathbf{p}(u) = \mathbf{u}^T\mathbf{M}_B\mathbf{p}$$

其中，$\mathbf{p}$是控制点向量。该多项式还可以表示成另一种形式：

$$\mathbf{p}(u) = \mathbf{u}^T\mathbf{M}\mathbf{q}$$

其中，$\mathbf{M}$是某类型曲线多项式的几何矩阵，$\mathbf{q}$是控制点向量。假设这两个多项式定义在相同的区间上，如果令

$$\mathbf{q} = \mathbf{M}^{-1}\mathbf{M}_B\mathbf{p}$$

则上述两个多项式表示同一条曲线。对于从插值曲线多项式到 Bezier 曲线多项式，转换矩阵为：

$$\mathbf{M}_B^{-1}\mathbf{M}_I = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -\frac{5}{6} & 3 & -\frac{3}{2} & \frac{1}{3} \\ \frac{1}{3} & -\frac{3}{2} & 3 & -\frac{5}{6} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

三次 B 样条曲线多项式与三次 Bezier 曲线多项式之间的转换矩阵为：

$$\mathbf{M}_B^{-1}\mathbf{M}_S = \frac{1}{6}\begin{bmatrix} 1 & 4 & 1 & 0 \\ 0 & 4 & 2 & 0 \\ 0 & 2 & 4 & 0 \\ 0 & 1 & 4 & 1 \end{bmatrix}$$

图 12.36 显示了 4 个控制点及其定义的三次 Bezier 曲线、插值曲线和样条曲线均由 OpenGL 的 Bezier 求值器生成。插值和样条形式都是作为 Bezier 曲线生成的，但是它们的控制点是原控制点使用矩阵 $\mathbf{M}_B^{-1}\mathbf{M}_I$ 和 $\mathbf{M}_B^{-1}\mathbf{M}_S$ 导出的新控制点。注意，生成的样条曲线仅位于第二和第三原始控制点之间。

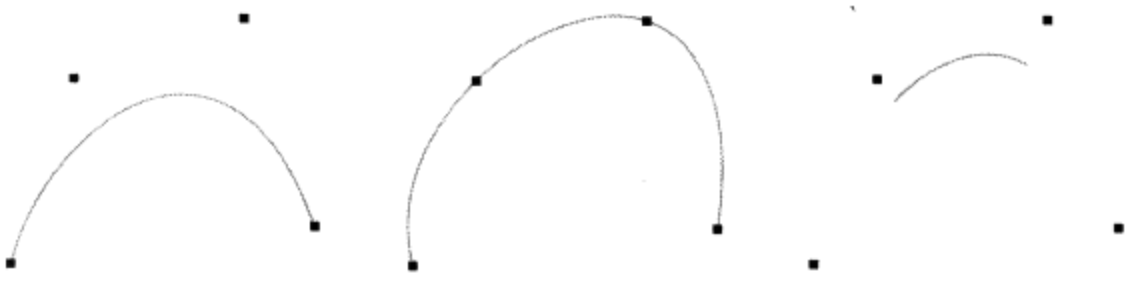


图 12.36　通过控制点转换以 Bezier 曲线形式表示的三次多项式曲线：
(a) Bezier 多项式曲线；(b) 插值多项式曲线；(c) B 样条多项式曲线

12.9.4　细分 Bezier 曲面

我们将细分算法推广到 Bezier 曲面。考虑如图 12.37 所示的三次曲面，它有 16 个控制点。每一行或每一列的 4 个控制点确定了一个可以被细分的 Bezier 曲线。细分算法需要确保将曲面片一分为四，然而曲面片中心没有控制点，为此需要分两个步骤进行。

首先，将曲线细分方法应用于 v 方向的 16 个控制点定义的 4 条曲线。因此，对于参数 u 的每个值 0、1/3、2/3、1，应得到两组点，每组 4 个，并且共享中间点。原曲线上此时有 7 个不同的控制点，如图 12.38 中圆点所示。在图中可看到 3 种圆点：细分以后仍然保留的原始的控制点（灰色），细分之后丢弃的控制点（白色），以及细分后生成的新的控制点（黑色）。接下来在 u 方向对这些控制点进行曲线细分，在 v 等于 0、1/3、2/3、1 其中之一的行，此时每个 v 值对应一组控制点，一共 7 组，每组有 4 个点，各定义一条 Bezier 曲线。若在 u 方向进行细分，同样得到两组点，每组有 4 个点，并且共享中间点，如图 12.39 所示。如果将这些点分成 4 组，每组有 16 个点，并且共享边缘上的点（如图 12.40 所示），则每个四边形包含 16 个点，它们是细分后 Bezier 曲面的控制点。

判断新凸包是否足够扁平以停止细分相比于 Bezier 曲线的扁平性计算更困难。很多绘制器使用一个固定的细分次数，通常由用户控制。如果需要绘制高质量的图形，可以细分直到投影后的凸包尺寸小于一个像素点。

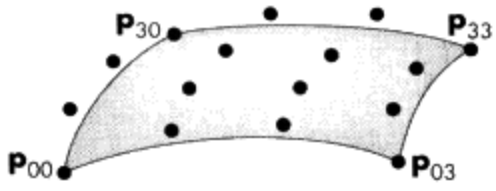


图 12.37　三次 Bezier 曲面

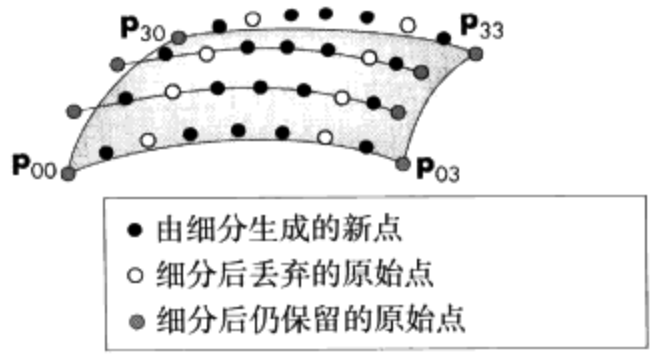


图 12.38　曲面的第一次细分

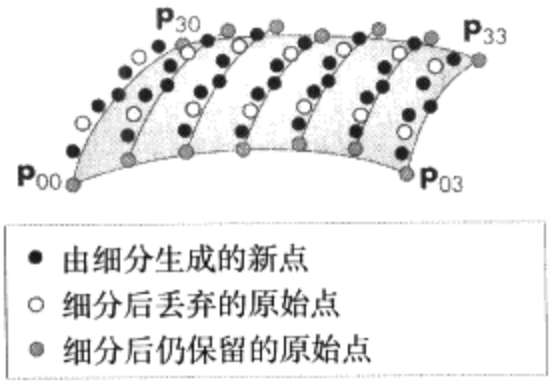


图 12.39　曲面第二次细分后点的排列情况

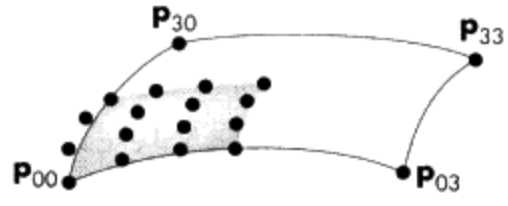


图 12.40　四分之一细分曲面

12.10　Utah 茶壶模型

我们以一个三次Bezier曲面片递归细分的实例结束对参数曲面的讨论，该实例生成的对象是著名的Utah茶壶。为了给测试各种绘制算法提供测试数据源，犹他大学的Mike Newell创建了茶壶数据，这些数据已经在图形学领域应用了30余年。茶壶数据包括由306个控制点（从1到306编号）定义的32个三次Bezier曲面片。前12个曲面片定义了茶壶躯干部分；接下来的4个曲面片定义了壶柄；后续4个曲面片定义了壶嘴，跟着的8个曲面片定义了壶盖；最后4个曲面片定义了壶底。这些数据很容易获取。

为了便于说明，我们设定每一个曲面片细分n次，而且在细分之后，以线框模式或者多边形（由曲面片的4个角点确定）模式绘制最终顶点。绘制可以通过以下函数执行（线框模式）：

```
typedef GLfloat point3[3];
void draw_patch(point3 p[4][4])
{
    glBegin(GL_LINE_LOOP);
       glVertex3fv(p[0][0]);
       glVertex3fv(p[3][0]);
       glVertex3fv(p[3][3]);
       glVertex3fv(p[0][3]);
    glEnd();
}
```

通过point3类型，使用三次曲线c的曲线分割器，建立曲面分割器：

```
void divide_curve(point3 c[4], point3 r[4], point3 l[4])
{
     int i;
     point3 t;
     for(i=0;i<3;i++)
    {
         l[0][i]=c[0][i];
         r[3][i]=c[3][i];
```

```
            l[1][i]=(c[1][i]+c[0][i])/2;
            r[2][i]=(c[2][i]+c[3][i])/2;
            t[i]=(l[1][i]+r[2][i])/2;
            l[2][i]=(t[i]+l[1][i])/2;
            r[1][i]=(t[i]+r[2][i])/2;
            l[3][i]=r[0][i]=(l[2][i]+r[1][i])/2;
        }
    }
```

假设已知矩阵转置函数transpose，则曲面片分割算法的编写相对容易，尽管其效率略低，代码如下：

```
void divide_patch(point3 p[4][4], int n)
{
     point3 q[4][4], r[4][4], s[4][4], t[4][4];
     point3 a[4][4], b[4][4];
     int i,j, k;
     if(n==0) draw_patch(p);
     else
     {
          for(k=0; k<4; k++) divide_curve(p[k], a[k], b[k]);
          transpose(a);
          transpose(b);
          for(k=0; k<4; k++)
          {
               divide_curve(a[k], q[k], r[k]);
               divide_curve(b[k], s[k], t[k]);
          }
          divide_patch(q, n-1);
          divide_patch(r, n-1);
          divide_patch(s, n-1);
          divide_patch(t, n-1);
     }
}
```

完整的茶壶绘制程序使用附录 A 中的投影多边形，这个程序中包含茶壶数据。

GLUT实用工具库提供了茶壶对象，它可以自动生成用于明暗处理的法向量。我们使用它来说明递归细分算法。图 12.41 显示了 3 次递归细分之后的线框模式的茶壶，并且用单色渲染。不同的曲面片有不同的曲率和大小，所以对它们采用一样深度的递归细分会生成很多不必要的小多边形。

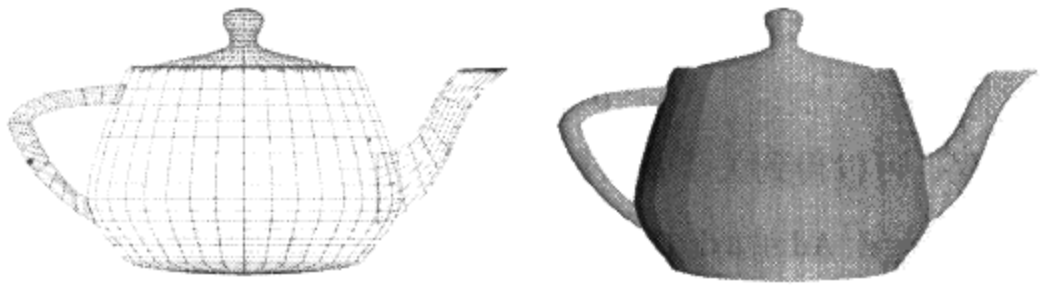


图 12.41　绘制茶壶

12.12 节介绍的 OpenGL Bezier 曲线函数允许通过 API 进行一些简单的函数调用来渲染。这些函数通常是递归的，但是运行效率应该比由用户程序实现要高。

12.11　代数曲面

尽管二次曲面是NURBS曲线的特例，但是这类代数曲面非常重要，值得专门进行讨论。二次曲面是 12.1 节介绍的代数曲面最重要的一类。

12.11.1　二次曲面

二次曲面由隐式的代数方程表示，其每一项都是$x^iy^jz^k$形式的多项式，其中$i+j+k \leqslant 2$。任何二次曲面都可以表示为：

$$q(x,y,z) = a_{11}x^2 + 2a_{12}xy + a_{22}y^2 + a_{33}z^2 + 2a_{23}yz + 2a_{13}xz + b_1x + b_2y + b_3z + c = 0$$

这类曲面对象包括椭圆面、抛物面和双曲面。我们以三维列向量$\mathbf{p} = [x\ y\ z]^T$将二次曲面一般形式表示为如下方程：

$$\mathbf{p}^T\mathbf{A}\mathbf{p} + \mathbf{b}^T\mathbf{p} + c = 0$$

其中，

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{12} & a_{22} & a_{23} \\ a_{13} & a_{23} & a_{33} \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

矩阵$\mathbf{A}$中这10个独立的系数，以及$\mathbf{b}$和c确定了一个二次曲面。为了分类，在不改变曲面类型的前提下，应用一系列旋转和平移变换将二次曲面方程转换为一个标准形式。在三维空间中，可以将这种变换写为：

$$\mathbf{p}' = \mathbf{M}\mathbf{p} + \mathbf{d}$$

这个变换通过矩阵$\mathbf{M}^T\mathbf{A}\mathbf{M}$替换$\mathbf{A}$生成了另外一个二次曲面。可以选取矩阵$\mathbf{M}$为旋转矩阵，使得$\mathbf{D} = \mathbf{M}^T\mathbf{A}\mathbf{M}$是一个对角矩阵，对角元素决定了二次曲面的类型。例如，椭圆曲面的二次方程可以写为如下形式：

$$a'_{11}x'^2 + a'_{22}y'^2 + a_{33}z'^2 - c' = 0$$

其中所有的系数都是正数。注意，因为可以通过一个仿射变换将二次方程转换为一个标准形式，并且二次曲面的类型不会发生改变，因此非常符合其他标准图元。

12.11.2　光线跟踪绘制曲面

可以通过求解标量二次方程确定其与光线的交集，因此二次曲面易于绘制。我们用参数方程的形式表示$\mathbf{d}$方向的由$\mathbf{p}_0$射出的光线：

$$\mathbf{p} = \mathbf{p}_0 + \alpha\mathbf{d}$$

将其代入二次曲面方程，可以得到关于α的标量方程：

$$\alpha^2\mathbf{d}^T\mathbf{A}\mathbf{d} + \alpha\mathbf{d}^T(\mathbf{b} + 2\mathbf{A}\mathbf{p}_0) + \mathbf{p}_0^T\mathbf{A}\mathbf{p}_0 + \mathbf{b}^T\mathbf{d} + c = 0$$

任意二次方程可能有0个、1个或者2个实数解。可以使用这些解将一个二次曲面绘制到帧缓冲区中，或者作为光线跟踪计算的一部分。此外，通过下面的导数公式可以计算任意点的法线，因此还可以将标准的明暗处理模型应用于二次曲面上的每一个点：

$$\mathbf{n} = \begin{bmatrix} \frac{\partial q}{\partial x} \\ \frac{\partial q}{\partial y} \\ \frac{\partial q}{\partial z} \end{bmatrix} = 2\mathbf{A}\mathbf{p} - \mathbf{b}$$

这个绘制方法可以推广到任意的代数曲面。假设已知一个代数曲面：

$$q(\mathbf{p}) = q(x, y, z) = 0$$

作为绘制过程的一个步骤，我们从投影中心向屏幕上的每个像素投射一束光线，每束光线可以表示为如下的参数形式：

$$\mathbf{p}(\alpha) = \mathbf{p}_0 + \alpha\mathbf{d}$$

将其代入到 q 的方程则得到一个关于 α 的隐式多项式方程：

$$q(\mathbf{p}(\alpha)) = 0$$

可以通过数值计算的方法求解交点。对于二次曲面，就是求解二次曲面公式（二次方程的求根公式）。如果最高次项为 $x^i y^j z^k$，则有 $i+j+k$ 个交点，那么绘制曲面需要消耗大量时间。

12.11.3　曲线和曲面的分割

让我们从另一个角度重新考察 12.9.2 节的细分公式。曲线由 4 个点 $\mathbf{p}_0$、$\mathbf{p}_1$、$\mathbf{p}_2$、$\mathbf{p}_3$ 开始，以 7 个点结束。使用 $\mathbf{s}_0, \cdots, \mathbf{s}_6$ 表示这些新的控制点，每一组控制点定义了一条分段线性曲线，如图 12.42 所示。

图 12.42　(a) 由 4 个点确定的分段线性曲线；(b) 一次分割后的分段线性曲线

可以使用细分公式确定这两组顶点之间的关系：

$$\mathbf{s}_0 = \mathbf{p}_0$$

$$\mathbf{s}_1 = \tfrac{1}{2}(\mathbf{p}_0 + \mathbf{p}_1)$$

$$\mathbf{s}_2 = \tfrac{1}{4}(\mathbf{p}_0 + 2\mathbf{p}_1 + \mathbf{p}_2)$$

$$\mathbf{s}_3 = \tfrac{1}{8}(\mathbf{p}_0 + 3\mathbf{p}_1 + 3\mathbf{p}_2 + \mathbf{p}_3)$$

$$\mathbf{s}_4 = \tfrac{1}{4}(\mathbf{p}_1 + 2\mathbf{p}_2 + \mathbf{p}_3)$$

$$\mathbf{s}_5 = \tfrac{1}{2}(\mathbf{p}_2 + \mathbf{p}_3)$$

$$\mathbf{s}_6 = \mathbf{p}_3$$

第二条曲线被看做是第一条的细化。正如在 12.9.2 节所看到的，我们可以继续这个迭代的过程，并且在极限下收敛于 B 样条曲线。然而，在实际中，只要新顶点定义的分段曲线看起来足够光滑，就认为迭代已经达到足够的次数。需要实施多少次迭代取决于凸包投影的大小，这可以由摄像机参数确定。因此，细分方法允许我们以不同的细节层次绘制曲线。

这些思想及其优点不仅局限于 B 样条。近几年，涌现出了各种曲线细分的方法。一些方法要求插值某些点，例如 $\mathbf{p}_0$ 和 $\mathbf{p}_3$，而另一些并不要求插值任何初始控制点。但是不管是哪种情形，曲线细分最终都收敛于一条平滑曲线。

12.11.4　网格细分

下面讨论如何将上述思想应用到曲面上。12.9.2节已经介绍了一种方法，但是我们更感兴趣的是如何将这些思想应用到多边形网格上。曲面细分理论的出现，既解决了这些思想的理论性问题，也提供了实际应用的方法。

在实际应用中，许多建模程序生成的网格都是三角形或四边形网格，因此我们将重点放在三角形网格和四边形网格的细分上，而并不是生成一个一般性的细分方法。如果需要细分的是一般网格，可以先将其转化为三角形或四边形网格。

使用Catmull-Clark方法可以将任意网格细分为四边形网格。首先，将每条边二等分，边的中间产生一个新的顶点。在每一个多边形的质心生成一个顶点，质心即是多边形顶点的平均值点。然后连接每个初始顶点及其两侧的新顶点，并且将这两个新的顶点连接到质心，这样就形成了一个新四边形网格。图12.43显示了简单多边形的细分过程，注意每一个实例的细分都生成了一个四边形网格。

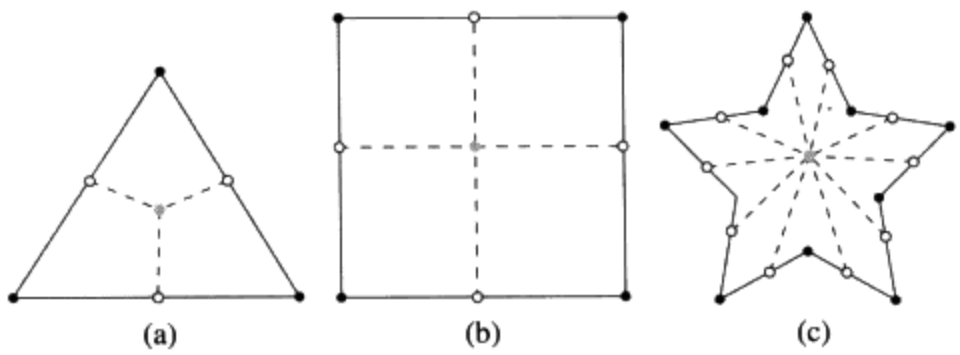


图12.43　多边形细分：(a) 三角形细分；(b) 矩形细分；(c) 星形多边形细分

显而易见，对生成的四边形网格进行细分会生成一个更加细化的四边形网格。然而如何能够生成一个更加平滑的曲面呢？特别是确保顶点尽可能的连续。

考虑以下过程。首先，计算每一个多边形顶点的平均位置（质心）；然后，用共享顶点的所有四边形的质心的平均值替代原顶点。经过这样的处理，就可以得到比较光滑的曲面。但是对于度不是4的顶点，平滑性急剧下降。因此Catmull-Clark方案增加了一个平均化步骤来生成平滑的曲面。平均化对于度不是4的顶点，使用下面的公式计算其新顶点：

$$\mathbf{p} = \mathbf{p}_0 + \frac{4}{k}\mathbf{p}_1$$

其中，$\mathbf{p}_0$是在平均化之前的顶点位置，$\mathbf{p}_1$是平均化之后的顶点位置，k是顶点的度。一个顶点的度是共享此顶点的多边形的数目。这种方法往往对尖角位置的顶点比外部顶点移动较大的距离。图12.44显示了单矩形的一组分割，在图12.44(a)中，原始的顶点用黑色表示，边线中点用白色表示；初始多边形的质心用蓝色表示，细分后多边形的质心用彩色表示。图12.44(b)显示了平均化后顶点位置的移动。图12.44(c)显示了应用调整因子之后的最终Catmull-Clark细分结果。

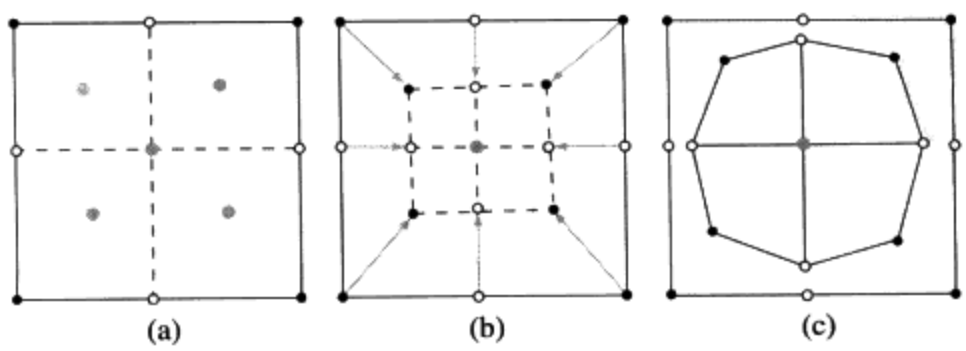


图12.44　Catmull-Clark分割

但是将 Catmull-Clark 细分方法应用于三角形网格并不能得到同样令人满意的结果，因为内部度较高的顶点在细分过程中没有发生改变。对于三角形网格，可以采用一个称为 Loop 的简单的细分方法，我们将其看做是普通细分方法的变体。该方法以标准的三角形细分方法开始，通过连接三角形每条边的中点，将一个三角形细分为 4 个三角形。细分步骤与前面相同，但是使用各顶点不同的权重来计算质心，受影响顶点的权重为 1/4，另外两个顶点的权重为 3/8。在 Catmull-Clark 方法的平均步骤之前和之后进行加权平均计算，就可以得到一个更加平滑的曲面，Loop 方法使用的权值是 $\frac{5}{3} - \frac{8}{3}(\frac{3}{8} + \frac{1}{4}\cos(\frac{2\pi}{k}))^2$。图 12.45 显示了一个简单的三角形网格以及经过细分过程之后的网格。

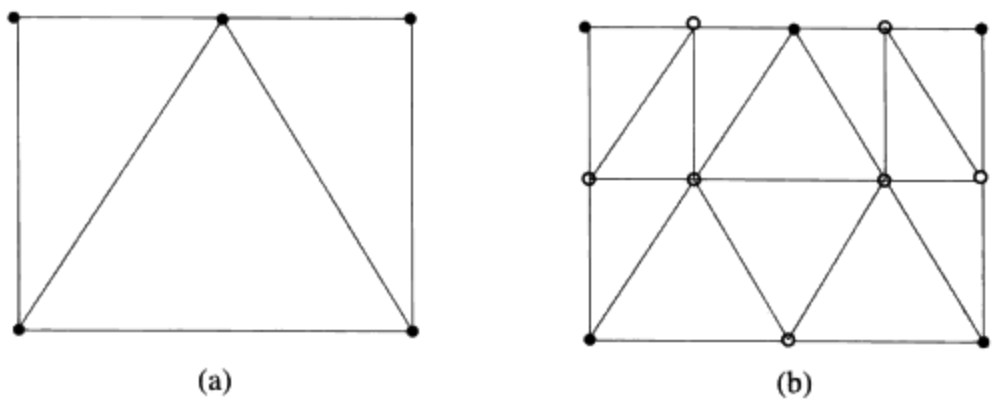


图 12.45　Loop 细分方法：(a) 三角网格；(b) 一次细分后的三角网格

彩图33所示图形就是使用曲面细分方法得到的。图12.46显示了通过曲面细分算法生成的一组网格，并且渲染了最高分辨率的网格。注意，初始的网格包含具有不同边数的多边形以及不同度的顶点，并且每一步细分都生成了一个比细分前更加平滑的曲面。

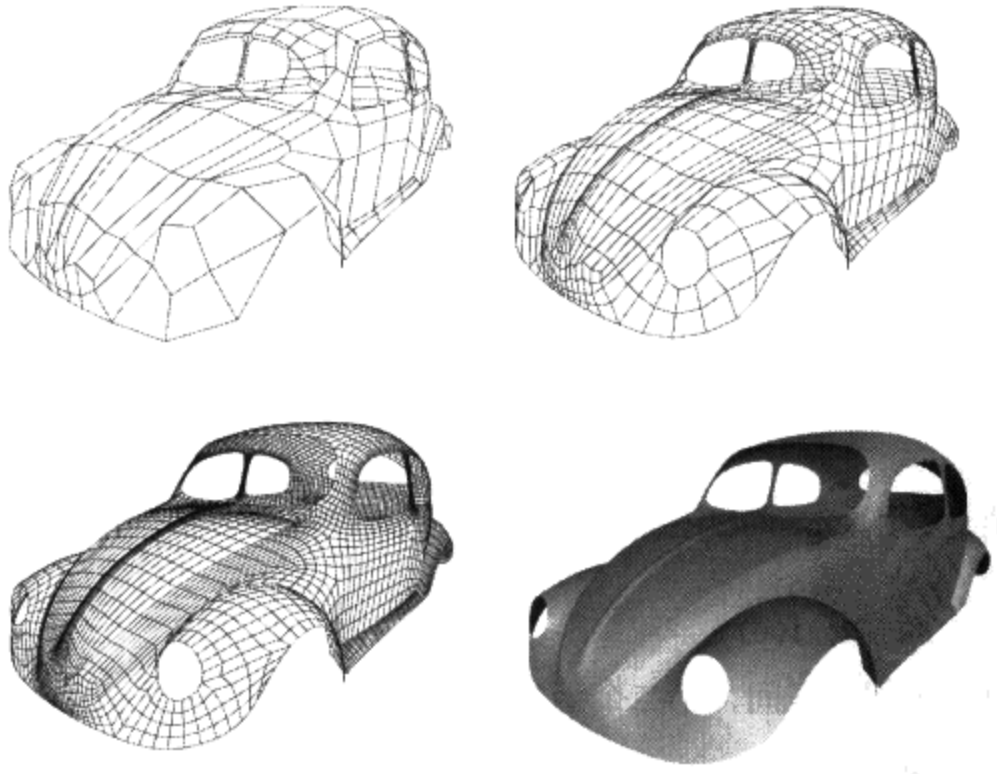


图 12.46　连续细分多边形网格及网格渲染（该图像由 Caltech Multi-Res 建模小组提供）

我们仍然没有解决细分过程中一些棘手的问题，例如，当需要在两个多边形的共享边上插入新顶点时，为了确保得到一致的结果所需要的数据结构。读者可以参考本章后面建议阅读资料中的文献。

12.12　OpenGL 中的曲线和曲面

OpenGL采用求值器机制生成Bezier曲线和曲面。可以使用求值器计算任意次Bernstein多项式的值，并且不要求控制点具有均匀间隔。由于可以用Bezier曲线和曲面通过生成新的控制点来生成

其他类型的多项式曲线和曲面，所以这个机制是灵活的。还可以用来生成一维、二维、三维和四维曲线和曲面。一维求值器通过计算单变量的Bernstein多项式生成曲线。我们还可以使用一维求值器来定义颜色映射或者动画实时路径。GLU库在四维空间中使用这个机制生成NURBS曲线。二维求值器计算两个变量的Bernstein多项式，主要用来生成曲面。除了定义曲线和曲面，OpenGL求值器函数还能够提供颜色、法线和纹理坐标。

12.12.1　Bezier曲线

以下函数为曲线定义了一个一维求值器并作为初始化的一部分：

```
glMap1f(type, u_min, u_max, stride, order, point_array)
```

参数type说明使用Bezier多项式求值器得到的对象类型，可以是三维点和四维点、RGBA颜色、法线、颜色索引和一维至四维纹理坐标。控制点数据位于point_arry指向的数组中。u_min和u_max的值说明了参数的取值范围。stride的值表示曲线段跨越的控制点的数目。例如，对于三次B样条曲线，每段曲线跨3个点。参数order是多项式次数加1。那么，定义一条(0, 1)区间上的三次B样条曲线，我们需要设置一个三维曲线求值器：

```
point data[] = {···}
glMap1f(GL_MAP2_VERTEX_3, 0.0, 1.0, 3, 4, data);
```

我们可以同时定义多个开启的求值器来进行求值，例如，同时定义曲线和法线的求值器。每一个求值器必须通过以下函数开启：

```
glEnable(type);
```

设置好求值器后，则使用以下函数利用当前开启的求值器进行计算：

```
glEvalCoord1f(u);
```

这里，glEvalCoord1f可以替换任何glVertex、glColor和glNormal的组合，这取决于开启了哪个求值器。假设已经为区间(0, 10)上的三次Bezier曲线定义了求值器和控制点数组，则可以通过如下函数得到曲线上100个等间距的点：

```
glBegin(GL_LINE_STRIP)
     for(i=0; i<100; i++) glEvalCoord1f( (float) i/100.0);
glEnd();
```

如果u是等间距取值的，则OpenGL还提供了另一种生成曲线的方法，使用如下函数：

```
glMapGrid1f(100,0.0, 10.0);
glEvalMesh1(GL_LINE, 0, 100);
```

第一个函数建立100个等间距区间，第二个函数生成所需曲线。

12.12.2　Bezier曲面

Bezier曲面的生成方式与曲线相似。可以使用函数glMap2和glEvalCoord2，但是首先需要确定u和v两个参数。例如，可以使用以下函数调用建立一个$(0, 1) \times (0, 1)$区域内的双三次Bezier曲面：

```
glMap2f(GL_MAP2_VERTEX_3, 0.0, 1.0, 3, 4, 0.0, 1.0, 12, 4, data);
```

为了灵活地设计曲面，必须设置所有函数的order和stride参数值。注意，因为数组data按行存储信息，所以第二个参数的stride值是12。因此，从当前行的某个元素要经过3个浮点数才能到达同一

行的下一个元素，而必须经过12个浮点数才能到达同一列的下一行元素。求值器调用取决于所需的绘制模式，可以是线框模式，也可以是多边形明暗绘制模式。对于线框模式网格，采用以下代码进行一组求值器的调用：

```
for(j=0; j<100; j++)
{
    glBegin(GL_LINE_STRIP);
       for(i=0; i<100; i++)
         glEvalCoord2f( (float) i/100.0, (float) j/100.0);
    glEnd();
    glBegin(GL_LINE_STRIP);
       for(i=0; i<100; i++)
         glEvalCoord2f( (float) j/100.0, (float) i/100.0);
    glEnd();
}
```

若绘制具有明暗效果的四边形网格，调用采用如下形式：

```
for(j=0; j<99; j++)
{
    glBegin(GL_QUAD_STRIP);
    for(i=0; i<=100; i++)
    {
      glEvalCoord2f( (float) i/100.0, (float) j/100.0);
      glEvalCoord2f( (float) (i+1)/100.0, (float) j/100.0);
    }
    glEnd();
}
```

对于均匀网格，可以使用 glMapGrid2 和 glEvalMesh2。因此，作为初始化的一部分需要进行如下说明：

```
glMapGrid2f(100, 0.0, 1.0, 100, 0.0, 1.0);
```

作为显示函数的一部分，需要调用：

```
glEvalMesh2(GL_FILL,0,100,0,100);
```

但是仍然存在一个问题，除非开启光照，否则绘制时将只能得到单一着色的多边形。此时可以依照第6章介绍的方法开启光照函数，但是阴影计算需要顶点的法线。虽然可以通过区分合适的多项式自己计算法线，我们也可以像使用求值器一样，要求 OpenGL 自动生成法线，仅仅需要使用下面的函数在开启光照选项的同时开启下面的选项：

```
glEnable(GL_AUTO_NORMAL);
```

12.12.3　绘制茶壶

绘制茶壶可以使用OpenGL的Bezier函数。假设已经读入顶点和曲面片信息，并且放置在如下数组中：

```
GLfloat data[32][4][4];
```

如果希望以线框模式绘制茶壶，可以通过以下函数进行简单的网格初始化：

```
void myinit()
{
  glEnable(GL_MAP2_VERTEX_3);
  glMapGrid2f(20, 0.0, 1.0, 20, 0.0, 1.0);
}
```

然后，在显示函数调用中绘制曲线段：

```
for(k=0;k<32;k++)
{
  glMap2f(GL_MAP2_VERTEX_3, 0, 1, 3, 4, 0, 1, 12, 4,
                  &data[k][0][0][0]);
  for (j = 0; j <= 8; j++)
  {
     glBegin(GL_LINE_STRIP);
     for (i = 0; i <= 30; i++)
         glEvalCoord2f((GLfloat)i/30.0, (GLfloat)j/8.0);
     glEnd();
     glBegin(GL_LINE_STRIP);
     for (i = 0; i <= 30; i++)
         glEvalCoord2f((GLfloat)j/8.0, (GLfloat)i/30.0);
     glEnd();
  }
}
```

注意，必须为每一个曲面片建立一个映射。为了绘制具有阴影效果的茶壶，我们需要一个更复杂的初始化过程以开启光照并设置材质属性：

```
void myinit()
{
  glEnable(GL_MAP2_VERTEX_3);
  glEnable(GL_AUTO_NORMAL);
  glMapGrid2f(8, 0.0, 1.0, 8, 0.0, 1.0);
  glEnable(GL_DEPTH_TEST);

  glEnable(GL_LIGHTING);
  glEnable(GL_LIGHT0);

  glLightfv(GL_LIGHT0, GL_AMBIENT, ambient);
  glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuse);
  glLightfv(GL_LIGHT0, GL_POSITION, position);

  glMaterialfv(GL_FRONT, GL_DIFFUSE, mat_diffuse);
  glMaterialfv(GL_FRONT, GL_AMBIENT, mat_ambient);
}
```

然后，在显示函数中调用以下函数绘制茶壶：

```
for(k=0;k<32;k++)
{
    glMap2f(GL_MAP2_VERTEX_3, 0, 1, 3, 4, 0, 1, 12, 4,
                    &data[k][0][0][0]);
    glEvalMesh2(GL_FILL,0,8,0,8);
}
```

12.12.4　NURBS 函数

可以使用求值器来生成非等间距的点以生成四维曲线和曲面。回忆之前的结论，可以通过生成一个合适的控制点集并将任意的多项式形式变换为一个Bezier形式，从而得到生成NURBS曲线和曲面所有必需的信息。与其让应用程序开发人员实现这些计算过程，不如使用OpenGL提供的GLU实用库。GLU提供了一组NURBS函数，这些函数允许用户设置各种附加选项以达到更好的绘制结果。GLU中共有5个函数用于处理NURBS曲面：

```
gluNewNurbsRenderer
gluNurbsProperty
gluBeginNurbsSurface
gluNurbsSurface
gluEndNurbsSurface
```

前两个函数定义了NURBS曲面对象及其绘制方式，后三个函数用来生成对象的表面。对于NURBS曲线，最后三个函数需要替换为 gluBeginNurbsCurve、gluNurbsCurve 和 gluEndNurbsCurve。

GLU 库还提供了另一个实用的功能——修剪曲线。修剪曲线是从一个表面上裁剪掉一个封闭曲线形成的区域，如图 12.47 所示。用户可以使用 NURBS 曲线函数 gluNurbsCurve，或者使用分段线性曲线函数gluPwlCurve定义一个修剪曲线。修剪曲线在glNurbsSurface函数调用之后被定义，在函数 glBeginTrim 和 glEndTrim 调用之间定义修剪曲线的控制点。

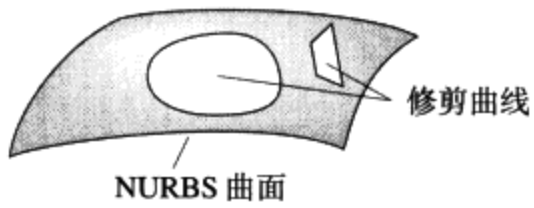


图 12.47　曲面修剪

12.12.5　二次曲面

OpenGL 通过 GLU 实用库支持多种类型的二次曲面对象（如圆盘、圆柱体和球体）。与其他图形一样，可以对这些二次曲面对象进行缩放、平移、旋转、明暗值的计算和纹理映射等操作。二次曲面绘制程序允许法线和纹理坐标的自动生成。OpenGL采用多边形来近似表示这些对象，多边形个数由用户确定。

假设需要使用一个圆柱体作为第10章中的机器人底座（见附录A），那么首先需要定义一个二次曲面对象：

```
GLUquadricObj *p;
p = gluNewQuadric();
```

然后定义二次曲面的绘制方式。如果希望使用当前的颜色以线框模式绘制机器人，则可以使用如下函数调用：

```
gluQuadricDrawStyle(p, GLU_LINE);
```

现在可以通过如下函数生成以原点为中心、中心轴沿 y 轴的圆柱体：

```
gluCylinder(p, BASE_RADIUS, BASE_RADIUS, BASE_HEIGHT, 5, 5);
```

前三个参数规定了顶面和底面半径以及圆柱体的高度，后两个参数规定了每一个以y为常量的圆弧的分段数目，以及沿 y 轴方向将圆柱体分割的段数。

小结和注释

之前仅仅对曲面这个深奥而又重要的课题进行了简单的讨论。同样，我们再次将视点集中于使用标准的 API（如 OpenGL）图形系统可以做些什么。从这一角度看，使用参数 Bezier 曲线和曲面具有非常大的优势。参数形式是健壮和易于交互使用的，因为定义曲线或曲面所需的顶点数据是可以被输入和交互控制的。Bezier 曲线和曲面的细分算法使得人们可以使用任意所需的精确度来绘制对象。

我们看到，尽管Bezier曲面易于渲染，但是样条曲线提供了更高的平滑度和可控制性。在本章的建议阅读资料中介绍了CAD领域所使用的样条的很多变体。

由于光线和二次曲面的交集计算只需要求解一个标量二次方程，因此二次曲面可以应用于光线跟踪中。但是判断光线与多边形定义的平面的交点是否位于多边形内部，比求解二次曲面交点更困难。因此，很多光线跟踪器仅仅应用于无限的平面、二次曲面或者凸多边形。

曲面细分由于以下两个原因已经变得日趋重要。第一，因为商业化的硬件设备可以以高速率绘制多边形，所以可以将表面细分成大量的多边形，以得到更光滑的表面，而且绘制速度快于将表面表示为少量曲面片的情形。然而，未来的硬件技术如果将曲面的绘制过程嵌入到光栅器中，那么这一优势也许将不复存在。第二，因为可以将曲面细分为任意的细节层次，那么当只是投影到屏幕的一个小区域时，就不需要一个高度细分的曲面，通过这样的方法可以有效地利用细分操作。

建议阅读资料

Farin的著作[Far88]对曲线和曲面做了精彩的介绍。有趣的是：在该书的序言里，Bezier介绍了以他名字命名的Bezier曲线和曲面，而Bezier和deCasteljau几乎同时发现了Bezier曲线和曲面。但deCasteljau的发现发表在一个未曾出版的技术报告里，所以直到近期，deCasteljau才获得本应属于他的荣誉。Rogers[Rog90]、Foley[Fo190]、Bartels[Bar87]和Watt[Wat01]等论著都介绍了各种形式的样条曲线和曲面。要想了解NURBS，请参阅Rogers的著作[Rog00]。[Cat74]提出了Catmull-Rom样条。

Faux撰写的书[Fau80]中讨论了关于曲线和曲面的坐标无关表示法以及Frenet框架。

虽然Glassner编辑的[Gla89]一书中主要讨论的是光线跟踪法，但其中由Haines撰写的部分有大量与二次曲面和其他代数曲面相关的内容。

近期，关于细分曲线和细分曲面的研究非常活跃。如果希望了解该领域里的精华成果，可以参阅[Che95]、[Deb96]、[Gor96]、[Lev96]、[Sei96]和[Tor96]。本章的讨论依据的是[War04]。[Cat78]中提出了Catmull-Clark细分方法。同时请参阅[War03]和[Sta03]。

习题

12.1　已知代数曲面$f(x, y, z) = 0$，其中f的每一项最高次数为m。请问f中最多可以有多少项？

12.2　显式方程$y = f(x)$和$z = g(x)$可以表示哪些类型的曲线？

12.3　假设已知一个多项式$p(u) = \sum_{k=0}^{n} c_k u^k$，求多项式$q(v) = \sum_{k=0}^{n} d_k v^k$，使得区间$(a, b)$上的每个点$p$在$0 \leqslant v \leqslant 1$范围内都存在一个点$v$，满足$p(u) = q(v)$。

12.4　证明：只要三次插值曲线的4个控制点分别定义在参数u的不同值上，那么插值几何矩阵总是存在。

12.5　证明：在区间(0, 1)上，Bernstein多项式必定小于1。

12.6　证明三次样条的C^2连续性。

12.7　10.9节中已经证明通过选择一个合适的控制点集合（或者说凸包），可以将三次多项式改写为Bezier多项式。利用这个结论，简述如何利用OpenGL的Bezier绘制器绘制插值曲线。

12.8　求二次曲面的齐次坐标表示。

12.9　使用自适应细分方法绘制Bezier曲面片，并且将每个曲面片细分为不同次数。请问在曲面片边缘是否保留了连续性，并说明理由。

12.10 编制 OpenGL 程序，输入一组控制点，输出相应的插值曲线、B 样条和 Bezier 曲线。

12.11 假设使用一组样条曲线表示物体在动画过程中的路径。请分析 G^1 和 C^1 连续性对于动画效果的区别。

12.12 扩展第 3 章的多边形绘制程序的功能，使之能够绘制 Bezier 曲线，并且允许用户交互定义控制点。

12.13 推导 Bezier 曲面片扁平性的简单测试方法。

12.14 基于节点序列{0, 0, 0, 0, 1, 1, 1, 1}和权值 $w_0 = w_2 = 1$，$w_1 = w$ 推导开放有理周期 B 样条。

12.15 利用习题 12.14 的结果证明：如果 $w = \frac{r}{1-r}$，当 $0 \leqslant r \leqslant 1$ 时，有理二次开放 B 样条曲线是圆锥曲线。提示：分 $r < 1/2$ 和 $r > 1/2$ 两种情况讨论。

12.16 求解 Hermite 调和函数的零值点。为什么零值点表明 Hermite 曲线在区间(0, 1)是平滑的。

12.17 分析 Hermite 曲面片控制点和曲面角点位置偏导数之间的关系。

12.18 对于 1024 × 1280 显示器，绘制三次多项式曲面需要的最大的细分次数是多少？

12.19 假设有 3 个点 P_0、P_1 和 P_2。首先通过参数线段连接相邻的点，其中每条线段的参数 u 在区间[0, 1]变化。然后对相邻两条线段进行插值，即在参数 u 具有相同值的点之间建立连线，然后在这条连线上求得参数 u 为同一个值的点。写出曲线表达式。

12.20 将习题 12.19 扩展为 4 个控制点。首先对生成的 3 条曲线进行线性插值，然后在得到的两条曲线上继续进行插值，写出由 4 个点确定的最终曲线的表达式。

12.21 如果控制点 P_0 和 P_1 的值相同，那么三次 Bezier 曲线会出现怎样的情况？

12.22 假设细分 Bezier 曲面片，首先沿 u 方向细分，然后在 v 方向只分割生成的两个曲面片的其中之一。证明这一过程产生的曲面上存在一个缝隙，并给出该问题的简单解决方法。

12.23 编写程序，进行三角形或者四边形网格细分。调试成功后，增加平均化步骤以形成更加平滑的曲面。

12.24 求解三次 Catmull-Rom 样条的调和函数及其零值点。

12.25 求解能够将 Catmull-Rom 样条数据转换为 Bezier 曲线控制点数据的矩阵，使之能够生成相同的曲线。

第13章　高级绘制

作为本书的最后一章，这里主要介绍图形绘制流程的其他方法，交互式应用软件中的许多硬件和软件都支持这些方法。有多种原因促使我们介绍这些高级绘制方法。首先，我们希望在场景中绘制全局光照之类的效果，但是这样的高逼真效果不大可能进行实时绘制。其次，我们希望绘制高质量的图像，而目前的标准显示器因为达不到这类图像的高分辨率而无法绘制这类图像。例如，对于计算机生成的视频，即使是单帧画面也可能包含超过500万的像素，这需要几个小时的绘制时间。

首先，本章讨论光线跟踪，然后研究绘制方程，该绘制方程为全局绘制提供了物理基础。尽管一般来说不能求解这个绘制方程，但是可以研究特定的明暗绘制模型的特殊解法，这些方法是后面提出的辐射度绘制方法的基础。

其次，本章将介绍计算机簇的并行绘制技术，利用该技术可以绘制大规模的几何数据。这些技术使我们能够绘制出远远高于标准显示设备分辨率的图像。

最后，本章将介绍基于图像的绘制技术。首先从二维图像出发，然后通过逆向获取的三维信息来生成新的图像。这些绘制方法结合了计算机图形学、计算机视觉和图像处理中的相关技术。

13.1　超越流水线绘制结构

我们目前做的所有事情都表明，如果给定一个包含几何对象、虚拟照相机、光源和各种属性的场景描述，就能够使用现有的硬件和软件，以接近实时的速度将这个场景绘制出来。这种观点表明，我们可以使用OpenGL体系结构和图形硬件支持的流水线绘制结构绘制场景。尽管人们提出了大量的技术方法解决许多图形应用中的问题，并克服了上述流水线绘制结构所支持的局部光照模型的不足，但是我们能做的事情仍然受到许多限制。例如，还不能使用目前的流水线绘制结构高逼真近似模拟许多全局光照效果。我们还希望生成比标准工作站更高分辨率的图像，这种图像具有更少的走样现象。很多情况下，我们希望以较低的绘制速度，或者使用多台计算机实现这些目标。本章将介绍各种技术，这些技术都是当前研究人员和专业人员感兴趣的。

首先，我们介绍基于物理成像原理的其他绘制技术。我们对图像成像最初的讨论是基于光线传播方向的。这个方法基于一个非常简单的物理模型，并导致了光线跟踪的绘制模式。我们将比前面的章节更详细地研究这个模型，并说明如何着手设计自己的光线跟踪器。

除了光线跟踪，我们还介绍了其他基于物理的绘制方法。我们将研究一种基于能量守恒的方法，并分析一个积分方程，即绘制方程，该方程描述了一个具有光源和反射表面的封闭环境。虽然一般情况下不能求解这个方程，但当环境中所有表面都是完全漫反射面时，我们提出了一个称为辐射度的绘制方法，该方法满足这个绘制方程。

此外还要介绍两种介于基于物理绘制器和实时绘制器之间的方法。一种是RenderMan采用的方法，另一种是基于图像的绘制方法。这两种方法虽然各不相同，但在动画产业中都占有很重要的地位。

最后，我们介绍如何解决大规模数据集和高分辨率显示的问题。这两个问题是相关的，因为大规模数据集包含的细节要求以高分辨率显示，而这个分辨率超过了日常使用的标准LCD显示器的分辨率。本章将考虑能充分利用处理器和图形卡并行性的解决方案。

13.2　光线跟踪

在许多方面，光线跟踪都是对基于局部光照模型绘制的一个逻辑扩展。该方法基于我们先前的观察结果，即对于所有光源发出的光线，对最终绘制的图像有贡献的只是那些进入虚拟照相机镜头且经过投影中心的光线。图13.1显示了一个点光源与一些理想镜面反射面之间可能的几种相互作用方式。进入虚拟照相机镜头的光线，有的直接来自于光源，有的来自于光源与虚拟照相机（即视点）可见的对象表面之间相互作用形成的光线，这些光线可能是经过其他对象表面多次反射形成的反射光线，或者是经过一个或多个对象表面的折射形成的折射光线。

离开光源的大部分光线并没有进入虚拟照相机的镜头，这些光线对最终绘制的图像没有任何贡献。因此，试图跟踪光源发出的所有光线来进行绘制是一个很耗时的方法。然而，如果按相反的方向来看光线，并且只考虑那些来自于投影中心的光线，那么可知，这些投射光线（cast ray）对绘制的图像一定有贡献。因此光线跟踪器的初始模型如图13.2所示。该图包括一个成像平面，并且成像平面被划分成像素大小的区域。由于必须给每个像素赋予一种颜色，所以必须至少投射一条光线通过每个像素。每条投射光线要么与对象表面相交，要么与光源相交，或者不与对象表面相交而射到无穷远处。对应于后面这种情况的像素点可赋予背景颜色。当投影光线与对象表面相交时（目前假定场景中所有对象表面都是不透明的），则需要计算交点处的明暗值或颜色值。如果只是简单地利用第6章介绍的改进的Phong模型来计算每个交点的明暗值或颜色值，那么绘制的图像与使用局部光照模型绘制的图像没有区别。然而，我们可以使用光线跟踪模型处理更多的事情。

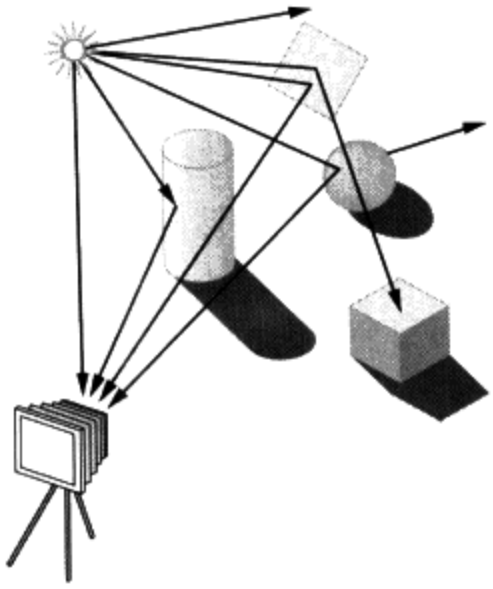


图 13.1　离开光源的光线

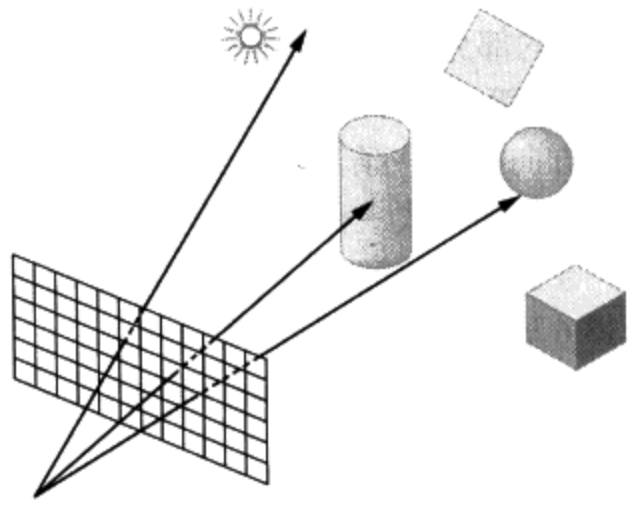


图 13.2　光线投射模型

我们注意到，到目前为止描述的光线跟踪过程与在流水线绘制过程中所使用的处理步骤是相同的，包括对象建模、投影和可见面判断。然而，正如第7章讲到的，这两个过程的计算顺序是不一样的。流水线绘制结构基于逐个顶点的绘制方式，而光线跟踪则基于逐个像素的绘制方式。

在光线跟踪中，我们并不是马上使用反射模型，而是先检测投射光线和对象表面之间的交点是否被光源照亮。从对象表面上的点到每个光源，都需要计算阴影光线（shadow ray）或试探光线（feeler ray）（如图 13.3 所示）。如果阴影光线在到达光源前与对象表面相交，那么光线被遮挡而不能到达考虑中的那个点，该点处于阴影中，至少是这个光源的阴影。如果对象表面上的某点到光源的光线被其他对象遮挡，则不需要计算光源在该点的光照强度。如果所有对象表面都是不透明的，而且不考虑对象表面之间的散射光，那么得到的图像是在没有用光线跟踪时绘制得到的图像上再增

加了阴影效果。这样做的代价是需要在投射光线和对象表面之间的每一个交点上做类似于隐藏面消除的运算。

假如对于一些具有高反射属性的表面，如图13.4所示的那些对象，可以跟踪阴影光线在对象表面之间的来回反射，直到反射到无穷远处，或者与光源相交为止。这个计算过程使用递归来实现，并且还要考虑表面的材质对光的吸收作用。

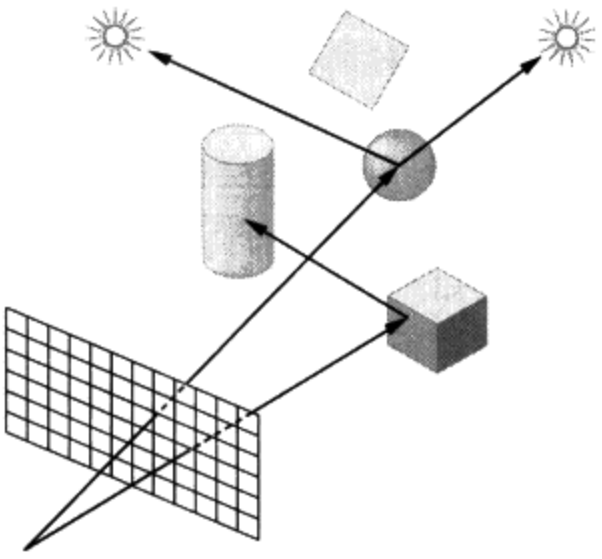


图13.3　阴影光线

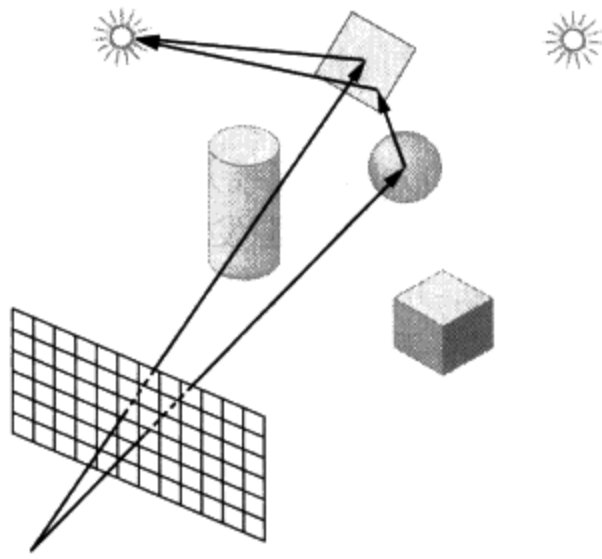


图13.4　镜面反射表面的光线跟踪

光线跟踪特别适合于处理同时具有反射和折射属性的表面。利用这种基本模式，我们沿着投射光线一直到具有以下特性的对象表面（如图13.5所示），即如果离开光源的光线射到该表面上的一点，那么有一部分光线被表面吸收，而另一部分光线以漫反射形式反射出去。入射光一部分被表面吸收，其余的部分则分为折射光和反射光。从投射光线的角度来看，如果光源在交点处可见，则需要做3件事。第一，必须利用标准的反射模型，计算光源在这个交点的贡献；第二，必须按照理想反射光的方向投射一条光线；第三，必须在折射光线的方向投射一条光线。这两条光线与原先的投射光线的处理方式一样，也就是说，它们可能和其他的对象表面相交；或终止于光源、或射到无穷远处。与上述的反射光线和折射光线相交的任何其他对象表面又会产生新的反射光线和折射光线。图13.6显示了一条投射光线和它经过一个简单场景的路径。图13.7描述了所生成的光线跟踪树。光线跟踪树是在光线跟踪过程中动态生成的，它说明了必须跟踪哪些光线。

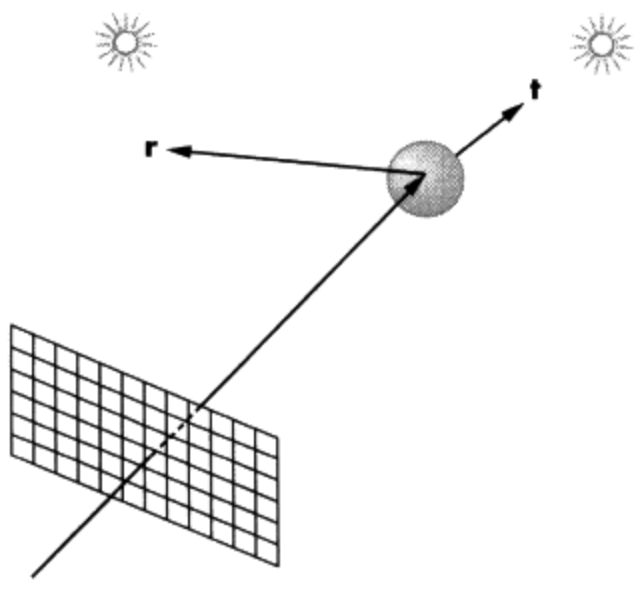


图13.5　反射和折射表面的光线跟踪

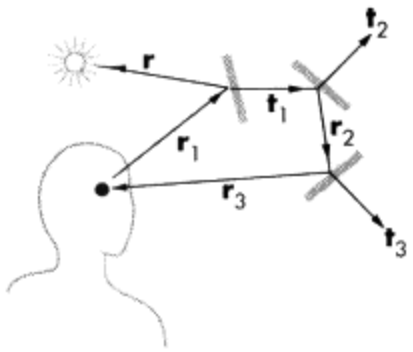


图 13.6　光线跟踪的简单场景

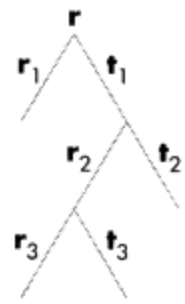


图 13.7　对应于图 13.6 的光线跟踪树

彩图 24 是对一个三维数据集进行光线投射绘制的图像，该图没有跟踪阴影光线。然而如果把皮肤看做是透明的，而肌肉是不透明的，那么这种光线跟踪模式只能够绘制肌肉的图像。这幅图像是在一个并行处理器上绘制的，它充分利用了这样一个事实，即可以独立跟踪不同的投射光线。

尽管光线跟踪器使用了 Blinn-Phong 光照模型来包含光线与对象表面交点处的漫反射项，但是我们忽略了在该交点处的漫反射光。如果试图跟踪这些光线，则要处理的光线太多，以至于光线跟踪器有可能永远不能结束计算。因此，光线跟踪器最适合于绘制具有高反射属性表面的场景。彩图 27 是用一个公用的光线跟踪器绘制的图像。尽管这个场景只包含几个对象，但是如果不使用光线跟踪，就无法真实地绘制出这些表面的反射和透明效果。另外，要注意场景阴影的复杂性，这是光线跟踪自动生成的另一个效果。该图像也说明了利用光线跟踪器实现纹理映射并不比使用流水线绘制结构实现纹理映射更困难。

13.3　构建一个简单的光线跟踪器

描述光线跟踪器最简单的方法是递归，即通过一个递归函数跟踪一条光线，它的反射光线和折射光线又调用这个函数本身。光线跟踪的大部分工作用来计算光线和对象表面的交点。实现一个可以处理各种对象的光线跟踪器之所以很困难的一个原因是：当加入比较复杂的对象时，大量的求交计算将成为一个问题。因此，大多数基本的光线跟踪器只支持平面和二次曲面。

我们已经分析了决定光线跟踪过程要考虑的一些基本要素。构造一个可以处理简单对象（如二次曲面和多面体）的简单的、递归光线跟踪器是相当容易的。这一节讨论构建光线跟踪器所需的基本结构及相关函数。读者可以参考本章后面的建议阅读资料来了解具体细节。

我们需要定义两个基本函数。递归函数 raytrace 用来跟踪一条光线，该光线由一个点和一个方向确定，函数返回与光线相交的第一个对象表面的明暗值。递归函数 raytrace 会调用函数 intersect 计算指定的光线与最近对象表面的交点位置。

13.3.1　光线跟踪递归算法

这里使用伪代码来描述 raytrace 递归函数。递归函数的参数（即跟踪的光线）通过起点 p 和方向 d 来确定，函数返回颜色值 c。为了防止光线跟踪递归算法陷入死循环，可以指定一个最大的递归步数 max。为了简单起见，假定只有一个光源，而且光源属性、场景对象的描述及其表面属性都定义为全局变量。如果还有其他的光源，可以使用与处理单光源类似的方法把它们对图像的贡献添加进来：

```
color c = trace(point p, vector d, int step)
{

   color local, reflected, transmitted;
```

```
    point q;
    normal n;
    if(step > max) return(background_color);

    q = intersect(p, d, status);

    if(status == light_source) return(light_source_color);
    if(status == no_intersection) return(background_color);

    n = normal(q);
    r = reflect(q, n);
    t = transmit(q, n);

    local = phong(q, n, r);
    reflected = trace(q, r, step+1);
    transmitted = trace(q, t, step+1);

    return(local + reflected + transmitted);
}
```

注意，当计算反射光颜色和折射光颜色时必须考虑到表面在反射和折射之前吸收了多少入射光的能量。如果递归次数超过了最大递归的步数，则返回背景颜色。否则，使用intersect函数计算给定的光线与最近对象表面的交点。intersect函数必须能访问场景数据库中的所有对象，而且能求出光线与光线跟踪器所支持的所有类型对象表面的交点。因此，光线跟踪器大部分的时间开销以及代码复杂度都隐藏在该函数里。13.3.2节将讨论一些求交计算的问题。

如果光线没有和任何对象相交，那么intersect函数返回一个状态变量，并从trace函数返回背景颜色。同样，如果光线和光源相交，那么返回光源的颜色。如果intersect函数返回一个交点，那么交点处的颜色值由3部分组成：利用改进的Phong模型（或其他光照模型）计算得到的局部光照颜色、反射光颜色和折射光颜色（如果对象表面是半透明的）。在计算这些颜色之前，必须计算交点的法向量以及反射光线和折射光线的方向，具体计算方法可以参见第6章和第9章。法向量计算的复杂度取决于光线跟踪器所支持的对象类型，这个计算过程是函数raytrace的一部分。

局部光照颜色的计算要求检测光源相对于最近交点的可见性，因此从最近的交点向光源投射一条试探光线或阴影光线，并检测阴影光线是否和其他对象相交。我们发现这个过程也是一个递归过程，因为阴影光线可能与一个反射表面相交，如一面镜子，或者半透明的对象表面，如一片玻璃。另外，如果阴影光线与一个自发光对象相交，则有一部分来自该光源的光线对q点的颜色有贡献。通常忽略这部分光的贡献，因为它们会非常明显地降低计算的速度。实际使用的光线跟踪算法一般采用折中的方案，这种方案实现的光照计算在物理上并不一定完全正确。

紧接着，程序中有两个递归调用，分别调用raytrace计算从q点发出的反射光线和折射光线对p点颜色值的贡献。正是因为这两个递归调用使得这段代码成为一个光线跟踪器，而不是简单的光线投射方法，在这种简单的光线投射方法中，我们只需发现第一个交点并把光照模型作用于这个交点。最后，把这3种颜色值相加获得p点的颜色。

13.3.2　求交计算

一个典型的光线跟踪器的大部分时间用于intersect函数中的求交计算。因此，必须将场景对象限制为容易求交的几何对象。如果使用场景对象表面的隐式表示形式，那么可以以一种简捷的方式表示一般的求交问题。因此，如果一个对象表面定义为：

$$f(x, y, z) = f(\mathbf{p}) = 0$$

并且来自 $\mathbf{p}_0$，方向为 $\mathbf{d}$ 的光线用参数的形式表示为：

$$\mathbf{p}(t) = \mathbf{p}_0 + t\mathbf{d}$$

那么交点位置处参数 t 的值应满足：

$$f(\mathbf{p}_0 + t\mathbf{d}) = 0$$

上式是 t 的一个标量方程。如果 f 是一个代数曲面，那么 f 是形式为 $x^i y^j z^k$ 的多项式之和，并且 $f(\mathbf{p}_0 + t\mathbf{d})$ 是一个关于 t 的多项式，因此求交就转化为寻找多项式所有根的问题。遗憾的是，只有很少的几种情况不需要使用数值方法。

一种情况是二次曲面。第 12 章曾讲过所有的二次曲面都可以写为二次形式：

$$\mathbf{p}^T\mathbf{A}\mathbf{p} + \mathbf{b}^T\mathbf{p} + c = 0$$

将光线方程代入上式，得到一个求解参数 t 值的标量二次方程，通过求解方程可得到 0 个、1 个或 2 个解。由于该二次方程的求解只需一次开平方根运算，所以使用光线跟踪处理二次曲面的求交运算并不是太难。另外，在求平方根之前，可以删除那些与二次曲面不相交的光线以及那些和二次曲面相切的光线，这样可以进一步简化计算。

例如，考虑一个球心在 $\mathbf{p}_c$、半径为 r 的球体，它可以表示为：

$$(\mathbf{p} - \mathbf{p}_c) \cdot (\mathbf{p} - \mathbf{p}_c) - r^2 = 0$$

把下面的光线方程代入上面的球体方程：

$$\mathbf{p}(t) = \mathbf{p}_0 + t\mathbf{d}$$

应该到一个二次方程：

$$\mathbf{d} \cdot \mathbf{d}t^2 + 2(\mathbf{p}_0 - \mathbf{p}_c) \cdot \mathbf{d}t + (\mathbf{p}_0 - \mathbf{p}_c) \cdot (\mathbf{p}_0 - \mathbf{p}_c) - r^2 = 0$$

对于平面求交问题也比较简单。可以把光线方程代入下面的平面方程：

$$\mathbf{p} \cdot \mathbf{n} + c = 0$$

这样可以得到一个只需要做一次除法运算的标量方程。因此，对于光线：

$$\mathbf{p} = \mathbf{p}_0 + t\mathbf{d}$$

可以通过计算得到交点的参数 t 的值：

$$t = -\frac{\mathbf{p}_0 \cdot \mathbf{n} + c}{\mathbf{n} \cdot \mathbf{d}}$$

然而，平面本身在场景建模中的应用是有限的。我们通常要么对构成凸面对象（多面体）的多个平面的求交感兴趣，要么对用于定义一个平面多边形的平面求交感兴趣。对于多边形，必须判定交点是在多边形内部还是外部。这种测试的难度取决于多边形是否是凸多边形，如果不是凸多边形，取决于是否是简单多边形。这些问题类似于第 7 章讨论的多边形绘制问题。凸多边形和光线的求交测试非常简单，这和下面要讨论的光线与多面体的求交测试非常类似。

尽管可以通过表面定义多面体，但是也可以把多面体定义为多个平面相交形成的凸面对象。因此，一个平行六面体使用 6 个平面定义，而一个四面体使用 4 个平面定义。对于光线跟踪，这种定义方式的优点在于：可以使用简单的光线与平面求交方程推导出光线与多面体的求交测试。

我们提出了如下的测试方法。假定定义多面体的所有平面都具有一个指向外部的法向量。考虑图13.8所示的与多面体相交的光线，它进入和离开多面体各仅一次。该光线一定是从面向光线的平面进入，并且从背向光线的平面离开。然而，这条光线一定也和构成多面体的其他平面相交（与光线平行的平面除外）。

下面考虑光线与所有正向平面（也就是那些法向量指向光线起点的平面）的交点，光线的入点一定是光线与所有正向平面交点中最远的那个交点。与此类似，光线的出点一定是光线与所有背向平面交点中最近的那个交点，而且入点一定比出点更接近光线起点。如果考虑一条与上面提到的同一多面体不相交的光线，如图13.9所示，那么会发现光线与正向平面最远的交点比光线与背向平面最近的交点离起点更近。因此，测试方法是按任意顺序通过光线-平面求交运算得到这些可能的入点和出点，并且每当求得一个交点，就要更新这些可能的入点和出点。如果发现一个可能的出点比当前入点更近，或者一个可能的入点比当前出点更远，那么终止测试。

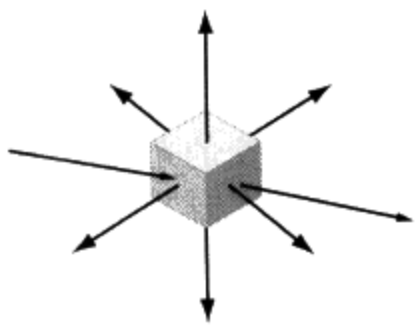


图13.8　光线与法向量指向外部的多面体相交

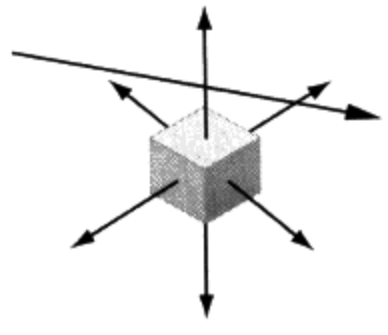


图13.9　光线与法向量指向外部的多面体不相交

分析如图13.10所示的用于测试光线与凸多边形交点的二维实例。这里用直线代替平面，但是道理是一样的。假定光线与直线求交的顺序是1，2，3，4。从直线1开始，通过计算直线的法向量和光线方向的点积，观察点积的符号就可以判断直线1是面向光线的初始点。光线和直线1的交点产生一个可能的入点。直线2背向光线的初始点，因此产生一个可能的出点，该出点比当前可能的入点更远。直线3产生一个更近的出点，但仍然比入点更远。直线4生成一个比当前出点更远的出点，因此不需更新出点。至此我们测试了所有直线，并得出光线穿过了该多边形的结论。

图13.11是同样的直线及其构成的同样的凸多边形，但说明了与多边形不相交的一条光线。光线与直线1的交点仍然产生了一个可能的入点。光线与直线2及直线3的交点仍然是比入点更远的可能的出点。但是光线和直线4的交点产生了一个比入点更近的出点，这种情况表明光线和多边形没有相交。

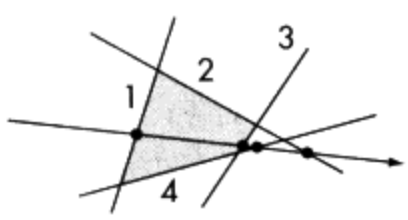


图13.10　光线与凸多边形相交

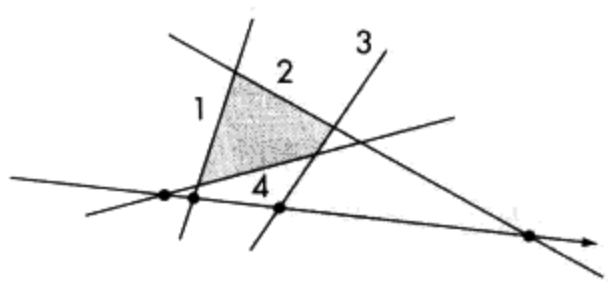


图13.11　光线与凸多边形不相交

13.3.3　光线跟踪的变异形式

大多数光线跟踪器都采用多种方法来判断何时结束递归过程。一种相当容易实现的方法是忽略跟踪距离超过某个阈值的所有光线，或假定这些光线都射到无穷远处。要实现这种测试方法，可以假定场景中的所有对象都位于一个以原点为中心的大球体里面。因此，如果认为该球体是用特定

的背景颜色着色的对象，那么每当通过求交计算确定这个球体是最近的对象时，就可以终止某条光线的递归过程并返回背景颜色。

另一个简单的终止策略是检查光线剩余能量所占的比重。当光线穿过一个半透明材质或者从一个光泽的对象表面反射时，可以估算出折射光线和反射光线的能量在入射光线能量中所占的比重，以及被对象表面所吸收能量的比重。如果在光线跟踪器中增加一个能量参数：

```
trace(point p, vector d, int steps, float energy)
```

那么只需在程序中增加一行代码用来检查是否还剩下足够的能量继续跟踪光线即可。

我们可以提出许多改进方法来提高光线跟踪器的运算速度或者精确度。例如，可以很容易地使用迭代的方法代替光线跟踪器里的递归过程。通过使用包围盒或包围球，可以避免大量的求交计算，因为光线与这类对象的求交运算速度非常快。包围体通常用来有效地组织对象，就像第10章介绍的BSP树。

由于光线跟踪是一种基于采样的方法，所以必然存在走样误差。正如第8章讲到的，如果采样点取得不够多，就会产生走样误差。然而在基本的光线跟踪器里，运算量和光线数量成正比。许多光线跟踪器使用了随机采样的方法，在这种方法中，在哪里选择下一条投射光线取决于已经投射的光线的处理结果。因此，如果光线与某个特定的区域里任何对象没有交点，那么几乎没有光线投射到这个区域；与此相反，如果光线与某个区域有很多交点，那么要增加指向这个区域的投射光线的数量。RenderMan采用的就是这个策略（见13.6节）。虽然我们认为随机采样只是在随机场景里才起作用（因为可能在某些区域里的微小对象出现采样精度不高的情况），但是随机采样的优点是：其绘制的图像一般不会出现均匀采样绘制的图像所特有的波纹状图案走样现象。

因为每条光线的处理独立于其他的光线，所以光线跟踪本质上是一个并行处理过程。然而，问题是每条光线都可能和任何对象相交。因此，光线的每次跟踪都需要访问所有的对象。另外，当跟踪反射光线和折射光线时，可能会丢失一些可以帮助我们避免大量数据传送的局部信息。因此，并行光线跟踪器最适合于内存共享的并行体系结构。随着具有64位寻址能力的多核处理单元的问世，商用计算机能够支持足够大的内存，这使得光线跟踪在许多应用领域变得切实可行。

13.4　绘制方程

大多数物理定律都可以表示为守恒定律，如动量守恒和能量守恒。由于光是一种能量，所以可以提出一种基于能量的方法来取代光线跟踪绘制方法。分析图13.12所示的一个封闭场景。这里使用表面图元来定义该封闭场景以及其中的对象、光源。从物理上看，包括光源表面在内，所有对象表面的建模方式都是一样的。尽管每个对象的表面可能有不同的参数，但是都遵循相同的物理规律。任何对象的表面都吸收并反射一部分光线。任何对象的表面都是一个发光器。从光线跟踪的角度看，我们之所以看到场景的明暗效果是由于无数的光线在场景中来回传播的结果。这些光线从光源出发，一直到所有能量被吸收完才结束。然而，我们观察到的场景处于稳定状态，也就是说，任何对象的表面都有自己的明暗效果。我们并没有看到光线是如何来回传播的，看到的也只是最后的结果。可以直接使用能量的方法实现这个稳定的状态，因此可以避免跟踪许多来回反射的光线。

现在只分析如图13.13所示的单个对象表面。我们看到来自许多不同方向进入该表面的光线以及其他一些从该表面射出的光线。离开该表面的光线由两部分组成，如果该表面也是光源，那么离开表面的一部分光线来自它发射的光，另一部分光线来自其他表面形成的入射光的折射光线。因此，进入表面的光线也包含该表面的发射光及来自其他对象表面的反射光。

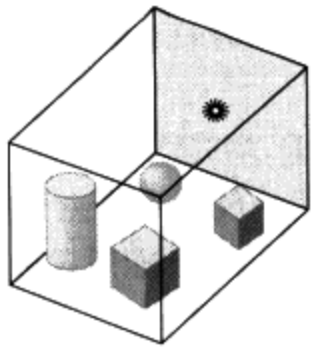


图 13.12　具有 4 个对象和一个光源的封闭场景

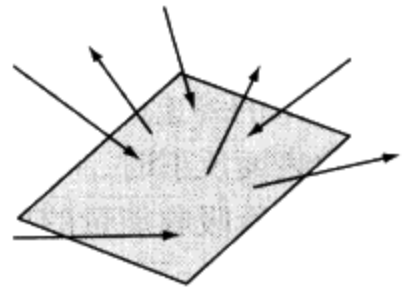


图 13.13　一个简单表面

如图 13.14 所示，考虑两个任意的点 **p** 和 **p′** 来简化分析过程。如果分析进入和离开 **p** 的光线，则其能量必须平衡。因此，如果在 **p** 处有一个点光源，那么该光源发射的能量及反射光能量之和必定等于所有来自 **p′** 的入射光能量。令 $i(\mathbf{p}, \mathbf{p}')$表示离开 **p′** 并到达 **p** 的光的强度[①]。绘制方程

$$i(\mathbf{p}, \mathbf{p}') = v(\mathbf{p}, \mathbf{p}')(\epsilon(\mathbf{p}, \mathbf{p}') + \int \rho(\mathbf{p}, \mathbf{p}', \mathbf{p}'')i(\mathbf{p}', \mathbf{p}'')\mathrm{d}\mathbf{p}'')$$

表示能量的平衡。离开 **p′** 的光的强度由两部分组成。如果 **p′** 是发光器（即光源），那么绘制方程中存在一项，表示 **p′** 到 **p** 方向的光的强度。绘制方程中的第二项来自于所有可能的点(**p″**)发出的在 **p′** 到 **p** 方向的反射光线的强度。反射函数表示 **p′** 处的表面材质属性。绘制方程中的项有两个可能的值。如果在 **p** 和 **p′** 之间有一个不透明的表面，那么这个表面遮挡了 **p′**，从 **p′** 发出光线无法到达 **p** 点。在这种情况下，$v(\mathbf{p}, \mathbf{p}') = 0$，否则必须考虑 **p** 和 **p′** 之间的距离，并且

$$v(\mathbf{p}, \mathbf{p}') = \frac{1}{r^2}$$

公式中 r 表示两个点之间的距离。

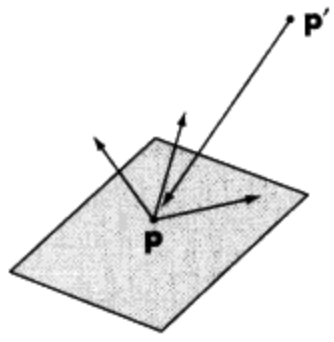


图 13.14　从 **p′** 到 **p** 的光线

尽管这个绘制方程的形式非常简单，但是求解该方程并非易事。主要的困难在于维度的数量。因为 **p** 和 **p′** 是三维空间中的点，$i(\mathbf{p}, \mathbf{p}')$有 6 个变量，而 ρ 有 9 个变量。另外，我们还没有包含光的波长这个在计算颜色时有必要考虑的变量。

有人努力尝试过使用数值方法求解这个绘制方程的一般形式。这些方法的大多数属于蒙特卡罗（Monte Carlo）方法，这是一种类似于随机采样的方法。目前，光子映射（photon mapping）成为一种切实可行的方法。光子映射从光源产生能携带能量的光子那一刻开始，一直跟踪每个光子直到其能量被场景中对象表面吸收为止。光子从创建到被吸收一般要经过多次反射和折射。这种方法潜在的优点在于它能模拟真实世界场景的复杂光照效果。

① 我们尽量避免介绍辐射度的单位和专用术语。我们介绍光的强度。能量是强度在时间上的积分，但是如果光源的强度不变，那么就处于稳定状态，并且这个时间上的区别就无关紧要了。大多数参考文献处理的都是单位面积上的能量或强度（即能量流），而不是能量或强度。

前面讨论光线跟踪时，我们认为：因为从光源发出的光子只有很少一部分到达观察者，所以从光源开始跟踪它发射的光线是一种很低效的方法。但是，光子映射使用了很多巧妙的策略，这使得光子的跟踪过程在计算上是切实可行的。尤其是光子映射结合了能量守恒和蒙特卡罗方法。例如，考虑光线与漫反射表面相互作用时的情况。正如我们所看到的，表面反射的光线向各个方向扩散。而在光子映射中，当一个光子与漫反射表面相互作用时，这个光子要么被反射，要么被表面吸收。光子无论是被吸收还是被反射，如果存在反射，那么光子的反射角是以随机方式确定的，这种随机方式使得绘制的场景效果在整体上是真实的。因此，两个光子即使以相同的入射角作用于同一对象表面的同一位置，两者产生的效果可能是截然不同的。光源产生的光子越多，其结果就越准确，但是这要以跟踪更多的光子作为代价。

可以根据一些特殊情况来简化上述绘制方程。例如，对于理想的镜面反射表面，反射函数只有当入射角和反射角相等并且入射光和反射光位于同一个平面时才为非零值。在这些特殊的情况下，可以把光线跟踪作为求解上述绘制方程的方法。

另一个特殊情况是：当所有的表面是理想的漫反射表面时，也可以得到一种切实可行的绘制方法。在这种情形下，光在各个方向均匀地反射，因此光的强度函数只与**p**点有关。下一节将讨论这种特殊情况。

13.5　辐射度方法

一种简化上述绘制方程的方法是：考虑的场景中所有对象表面都是理想的漫反射器。因为从不同视点观看一个理想的漫反射表面得到的结果是一样的，所以基于这种特殊情形的 BRDF（Bidirectional Reflectance Distribution Function，双向反射率分布函数）方程比上述通用绘制方程简单得多。

我们注意到，虽然一个理想的漫反射表面将光线均匀地反射到各个方向，即使这个表面是平的，其表面不同区域的明暗效果也是有变化的，而使用Phong光照模型则无法绘制这种明暗变化的效果。假定有一个如图 13.15 所示的简单场景，场景中所有对象都具有理想的漫反射表面。如果使用一个位于很远的光源（平行光源）绘制该场景，那么场景对象的每个表面绘制的颜色是不变的。然而，如果这是一个真实的场景，那么红墙产生的一部分漫反射光线会落在白墙上，这样会导致这些红光与附近白墙上反射的白光互相混合。白墙的漫反射光也会对附近的红墙产生同样的效果。简单绘制模型并没有考虑这些漫反射之间的相互作用（diffuse-diffuse interaction）。

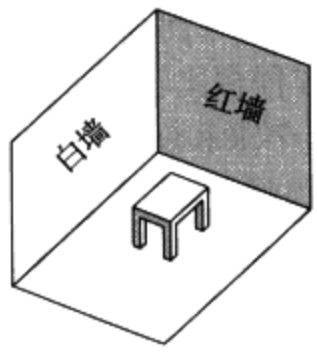


图 13.15　具有漫反射对象表面的简单场景

一个理想的全局绘制器需要考虑上述的相互作用。辐射度方法通过采用能量守恒的方法能够很好地模拟这些相互作用，这种方法最先应用于解决热量传播的问题。

基本的辐射度方法把场景分解成许多小平面多边形，或称为面片（patch），每个面片都假定是理想的漫反射面，并且每个面片的明暗值是一个常数。首先要计算这些面片的明暗值。一旦计算出

各面片的明暗值，就能够有效地将颜色赋给每个面片，每个面片的颜色值与视点无关。我们已经有效地将颜色赋给三维场景里的一组多边形面片。现在，可以随意将视点放置在所需的位置，并按常规的方式采用流水线绘制结构来绘制场景。

13.5.1　辐射度方程

假定场景包含 n 个面片（patch），并标号为 1 到 n。第 i 个面片的辐射度标记为 b_i，它表示该面片单位面积上发射的光的强度（能量/单位时间）。我们一般以 W/m²（瓦特/米²）为单位表示辐射度。由于以固定的波长测量光的强度，所以我们认为辐射度函数 $b_i(\lambda)$ 决定第 i 个面片的颜色。假定第 i 个面片的面积为 a_i。由于我们假定每个面片都是理想的漫反射表面，那么离开第 i 个面片总的光强度是 $b_i a_i$。我们使用前面绘制方程的推导方法，发光体发出的光的强度包括两部分：一部分是辐射光，并假定整个面片的辐射强度是一个常量；另一部分是反射光，它是由所有其他面片发出并到达第 i 个面片的光的强度。因此，有如下公式：

$$b_i a_i = e_i a_i + \rho_i \sum_{j=0}^{n} f_{ji} b_j a_j$$

其中，f_{ij} 项称为面片 i 和面片 j 之间的排列因子（form factor），它表示光线从面片 i 到面片 j 的能量比重。排列因子的大小取决于两个面片间的朝向、两个面片间的距离以及是否有其他面片遮挡了从面片 j 到面片 i 的光线。我们将在下一小节讨论排列因子的计算。令面片 i 的反射率为 ρ_i。

排列因子 f_{ij} 和 f_{ji} 之间存在一个简单的关系，称为对等等式（reciprocity equation）：

$$f_{ij} a_i = f_{ji} a_j$$

将上式代入前面的面片强度公式，可以得到：

$$b_i a_i = e_i a_i + \rho_i \sum_{j=0}^{n} f_{ij} b_j a_i$$

公式两边除以 a_i，可以得到面片的辐射度方程：

$$b_i = e_i + \rho_i \sum_{j=0}^{n} f_{ij} b_j$$

这个公式就是辐射度方程（radiosity equation）。

假定通过计算得到了排列因子，那么有一个包含 n 个线性方程的方程组，该方程组有 n 个未知的辐射度变量。我们把这个方程和绘制方程相比较就会发现，如果面片被划分得越来越小，当被划分到无穷小时，则会有无穷多个面片。这个求和就变成了积分，而最终的辐射度方程将成为绘制方程的一个特例，即所有对象表面都是理想的漫反射表面。

可以把辐射度定义成下面的列矩阵，这样可以将上述方程改写成矩阵的形式：

$$\mathbf{b} = [b_i]$$

如果面片是发光体，其发射光的强度矩阵为：

$$\mathbf{e} = [e_i]$$

反射系数构成的对角线矩阵写为：

$$\mathbf{R} = [\, r_{ij} \,], \qquad a_{ij} = \begin{cases} \rho_i & \text{如果 } i = j \\ 0 & \text{其他} \end{cases}$$

排列因子矩阵写为：

$$\mathbf{F} = [f_{ij}]$$

现在可以得到辐射度的一组方程：

$$\mathbf{b} = \mathbf{e} - \mathbf{RFb}$$

计算辐射度的正式求解公式可写为：

$$\mathbf{b} = [\mathbf{I} - \mathbf{RF}]^{-1}\mathbf{e}$$

13.5.2　求解辐射度方程

虽然可以证明辐射度方程一定有解，但是真正的问题在于这样做的可行性。一个典型的场景包含成千上万个面片，因此使用高斯消元法等直接的方法求解辐射度方程是不可行的。许多方法考虑矩阵**F**是一个稀疏矩阵。由于大多数面片之间实际上相距很远，某个面片发射或反射的光线几乎没法到达大多数其他的面片，因此矩阵 **F** 的大多数元素实际上等于零。

根据迭代法求解稀疏矩阵对应的方程组，使用该方法需要这些稀疏矩阵进行高效的乘法运算。假定使用上述的面片辐射度方程得到迭代公式：

$$\mathbf{b}^{k+1} = \mathbf{e} - \mathbf{RFb}^{k}$$

该方程的每次迭代都需要做矩阵乘法运算 $\mathbf{RFb}^k$，假定 **F** 是一个稀疏矩阵，那么其时间复杂度为 $O(n)$，而不是一般情况下的 $O(n^2)$。对于单个面片的辐射度，有下面的公式：

$$b_i^{k+1} = e_i - \sum_{j=1}^{n} \rho_i f_{ij} b_j^k$$

这种方法称为雅可比（Jacobi）方法，无论其初始值 $\mathbf{b}^0$ 如何，对应问题求解的过程总是收敛的。

一般来说，面片没有自反射特性，所以 $f_{ij} = 0$。如果使用该结论修改上述公式，则得到高斯－赛德尔（Gauss-Seidel）方法：

$$b_i^{k+1} = e_i - \sum_{j=1}^{i-1} \rho_i f_{ij} b_j^{k+1} - \sum_{j=i+1}^{n} \rho_i f_{ij} b_j^k$$

可能还有另一种更加有利于理解其物理规律的迭代方法。考虑下面的标量公式：

$$\frac{1}{1-x} = \sum_{i=0}^{\infty} x^i$$

该公式在 $|x| < 1$ 时总成立。对于 **RF**，该公式的矩阵形式为[①]：

$$[\mathbf{I} - \mathbf{RF}]^{-1} = \sum_{i=0}^{\infty} (\mathbf{RF})^i$$

因此，可以把 **b** 写为：

$$\begin{aligned}\mathbf{b} &= \sum_{i=0}^{\infty} (\mathbf{RF})^i \mathbf{e} \\ &= \mathbf{e} + (\mathbf{RF})\mathbf{e} + (\mathbf{RF})^2\mathbf{e} + (\mathbf{RF})^3\mathbf{e} + \cdots\end{aligned}$$

① 如果 RF 所有特征值的模小于 1（对于辐射度方程该条件一定成立），那么该公式收敛。

我们使用这个表达式并在某个地方终止计算，就可以得到**b**的一个近似值。该表达式的每一项都有一个有趣的物理含义。表达式的第一项**e**是每个面片直接发出的光线。因此，如果该表达式只含这一项，那么**b**的近似解只表示光源的照射效果。表达式的第二项**(RF)e**增加了光源到其他面片形成的一次反射光线。表达式的下一项增加了两次反射的贡献，其余的项以此类推。

13.5.3　排列因子的计算

现在已经推导出计算面片辐射度相当有效的方法，但是还没有讨论排列因子的计算。我们首先讨论计算两个面片之间排列因子的一般推导公式，然后研究如何利用已有的工具得到排列因子的近似值。

考虑两个具有理想漫反射性质的平面面片，如图13.16所示的P_i和P_j，该图没有考虑其他可能导致遮挡的面片。每个面片都有一个表示其朝向的法向量。尽管每个面片朝各个方向均匀地发射光线，但是由于面片P_j上两个不同的点到达面片P_i上某一点的距离不一样，所以从面片P_j上两个不同的点到达面片P_i上任意一个点的光线的能量并不相等。因此，为了计算所有从P_j到P_i的光线，必须对面片P_j上的所有点求积分。同理，面片P_i上两个不同的点从面片P_j上的任意一点接收的光线是不相等的，因此为了确定P_i接收的光线，也必须对面片P_i上的所有点求积分。

根据对第6章介绍的漫反射明暗效果计算的理解，可以推导出需要的积分公式。如图3.17所示，考虑每个面片上的一个小区域，$\mathrm{d}a_i$和$\mathrm{d}a_j$，每个小区域可以认为是一个理想的漫反射器。这两个小区域相距r，r表示分别位于这个小区域中心的两个点$\mathbf{p}_i$和$\mathbf{p}_j$的距离。离开$\mathrm{d}a_j$的光线沿着方向$\mathbf{d}=\mathbf{p}_i-\mathbf{p}_j$传播。然而，该光线的强度按$\cos\phi_j$衰减，其中$\phi_j$是面片$j$的法向量和向量**d**之间的夹角。同理，达到$\mathrm{d}a_j$的光线也按$\cos\phi_j$衰减，其中$\phi_j$是面片$i$的法向量和向量**d**之间的夹角。为了正确计算两个面片的形状因子f_{ij}，还必须考虑这两个面片之间的距离以及可能的遮挡。考虑这两种情况，首先定义项o_{ij}：

$$o_{ij}=\begin{cases}1 & \text{如果从 } \mathbf{p}_i \text{ 点看 } \mathbf{p}_j \text{ 可见}\\ 0 & \text{其他}\end{cases}$$

接着，对面片整个面积计算平均值，就可以得到排列因子的方程：

$$f_{ij}=\frac{1}{a_i}\int_{a_i}\int_{a_j}o_{ij}\frac{\cos\phi_i\cos\phi_j}{\pi r^2}\mathrm{d}a_i\mathrm{d}a_j$$

尽管上面这个积分形式很简单，但只有几种特殊的情况能够分析求解。对于真实的场景，需要使用数值求解的方法。因为有n^2个排列因子，所以许多方法只能在精确性和实时性之间进行折中。

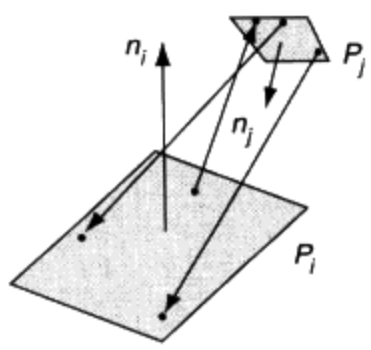


图13.16　两个面片

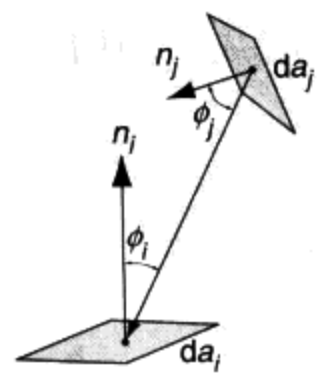


图3.17　两个小面片之间的透视关系

这里简单介绍两种方法。第一种方法采用了第8章讨论纹理映射时用到的两步映射的概念。再次考虑两个面片，但如图13.18所示，这一次有一个半球体把这两个面片隔开。假定需要计算从面

片 P_i 的某个点 $\mathbf{p}_i$ 到达面片 P_j 上的光，我们将半球体的中心放置在 $\mathbf{p}_i$ 点，并调整面片的朝向使得面片的法向量朝上。现在把面片 P_j 投影到半球体上，我们发现可以使用投影后的面片而不是原始的面片来计算排列因子。如果转换到极坐标系，那么投影后的面片排列因子计算方程就简单多了。然而，对于 P_i 的每一个小区域，都必须放置一个半球体，然后把各个区域的贡献累加起来。

如图 13.19 所示，对于大多数图形应用，一种更简单的方法是使用半立方体而不是半球体。半立方体的中心位置和半球体一样，但是半立方体的表面被分割为许多称为像素的小正方形。到达面片 P_i 的光线与使用的中间表面类型无关。使用半立方体的优点是：半立方体的表面要么平行于 P_i，要么垂直于 P_i。因此，可以把 P_j 直接投影到半立方体上，然后计算半立方体上每个像素对到达 P_i 点的光照贡献。因此，如果半立方体的表面被分割为 m 个像素，标号为 $1 \sim m$，那么我们能够计算出面片 P_j 投影的且从 P_i 点可见的那些像素的贡献值，把这些贡献值累加起来就可得到 Δ 排列因子（delta form factor）Δf_{ij}，它表示面片 P_j 对半立方体中心处的小区域 $\mathrm{d}a_i$ 的贡献。把所有 Δ 排列因子的贡献值加起来，就可以得到所需要的形状因子。一旦知道面片 P_j 是否投影到某个像素上，该像素的贡献值就可以通过分析计算出来，这种计算方法和在光线跟踪中用来确定光源方向上一个对象是否可见的方法类似。该方法的详细讨论可参考本章后面的建议阅读资料。

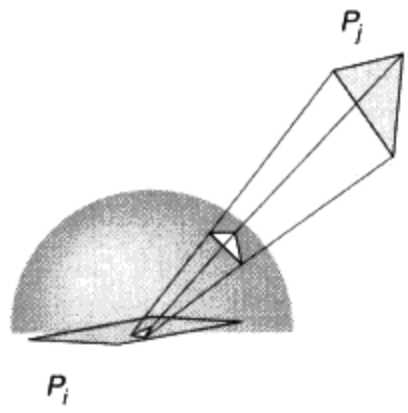


图 13.18　把面片投影到半球体上

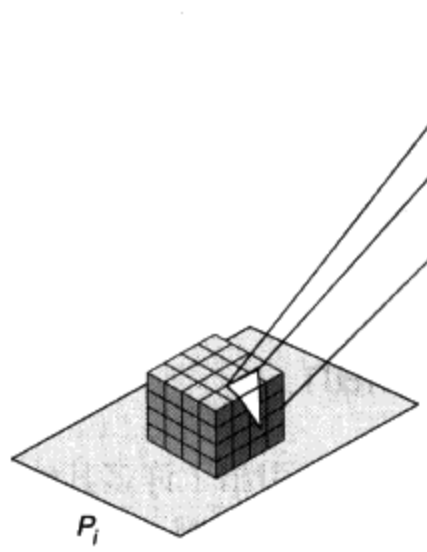


图 13.19　把面片投影到半立方体和另一个面片上

另一种计算排列因子的方法是充分利用图形系统快速计算简单场景的能力。假定要计算 f_{ij}。如果只用来自面片 P_j 的且大小已知的光能照亮 P_i，那么可以得到所需的排列因子的测量值。我们可以把点光源放在面片 P_j 上，然后使用任何一个已有的绘制器绘制场景，这样可以得到这个测量值的近似值。由于面片 P_i 和面片 P_j 之间可能有其他面片形成遮挡，因此必须使用能处理阴影的绘制器。

13.5.4　实现辐射度算法

在实际应用中，辐射度绘制方法有 3 个主要步骤。首先，如图 13.20 所示，将场景分割为一个由许多面片组成的网格。这个步骤需要一些技巧，因为分割后的面片越多，需要计算的排列因子也越多。然而，正是因为把对象表面分割为许多的面片，才能使用辐射度方法绘制出场景面片之间的漫反射能相互作用的精细图像。一般使用交互式方法创建初始网格，这样可以在对象表面间的转角等区域放置更多的面片，我们希望这些区域上面片之间漫反射的相互作用效果能够被绘制出来。另一种方法是基于这样一个观察：一个大的表面的辐射度等于其细分后子区域辐射度的面积加权和。因此，我们可以从一个很粗糙的网格开始，然后对它逐步细化（渐进辐射度方法）。一旦得到了一个网格，就可以计算其排列因子，这是整个过程计算最密集的一步。

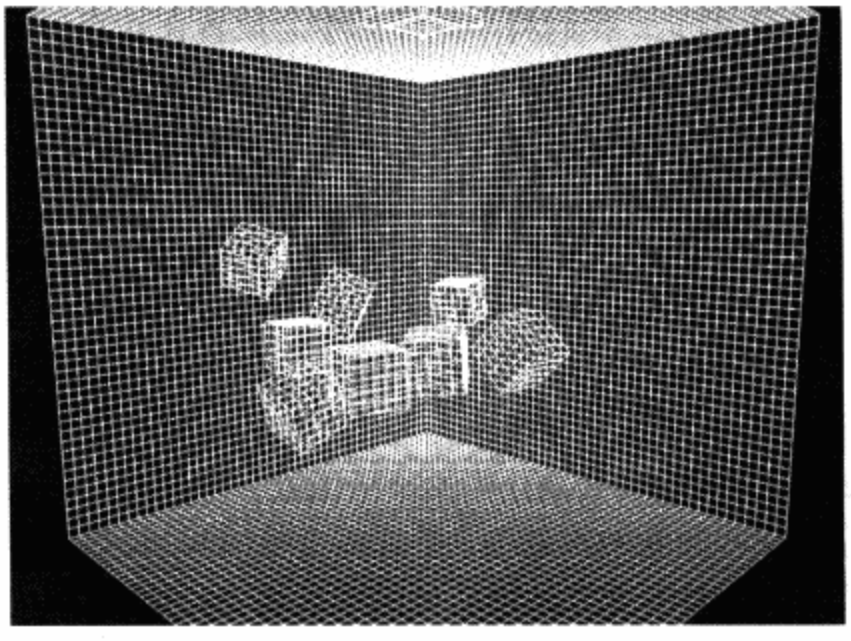


图 13.20　把表面分割为面片（由克莱姆森大学的 A. Van Pernis, K. Rasche 和 R. Geist 提供）

一旦得到网格及其排列因子，就可以求解辐射度方程。我们使用场景中的光源值可以得到辐射矩阵 **e**，而通过把颜色赋给表面得到矩阵 **R**。现在就可以通过求解辐射度方程确定矩阵 **b** 了。

矩阵 **b** 中的元素作为面片的新颜色值。现在可以将视点放置在场景的任意位置并使用传统的绘制器绘制该场景。

彩图28是用辐射度方法绘制的。这个彩图的初始网格如图13.20所示，然后使用一个粒子系统改变这个网格，以便得到一个更好的面片集。该图说明了使用辐射度方法绘制由漫反射对象构成的内部场景的能力。

13.6　RenderMan

由于动画产业的需求，引出了许多其他的绘制方法。尽管动画设计的过程中需要交互性，但是最终产生的图像却不需要实时绘制。更重要的是生成的图像没有瑕疵，比如由于走样引起的锯齿和波纹状的图案。然而，即使使用大量的计算机，如使用绘制场（render farmer）来绘制动画，我们也无法用光线跟踪或辐射度方法绘制一部正片长度的电影所需的大量的帧画面，这两种绘制方法的唯一任务是以所需的分辨率绘制高逼真的场景。此外，单独使用光线跟踪器或辐射度绘制器都无法生成艺术品质级别的图像。

RenderMan 接口规范以第 1 章介绍的建模－绘制模式为基础。我们以交互的方式设计场景，并使用第6章介绍的简单类型的绘制器绘制场景。但场景设计完成后，我们首先使用一个文件来描述场景中的对象、光源、材质属性、照相机、动作和纹理，然后将这个文件送入高质量的绘制器或绘制场。

原则上，这种脱机的绘制器可以是任何类型的绘制器。然而，由于动画产业的特殊需求，Pixar 不仅开发了接口规范（即 RenderMan），还开发了名为 Reyes 的绘制器，设计该绘制器是为了生成商业电影所需要的高品质图像。与光线跟踪器一样，Reyes 每次也只是处理一个像素。与光线跟踪器不同的是：Reyes 在设计时并没有考虑全局光照效果。由于 Reyes 每次只处理一个像素，所以它以某个分辨率收集所有来自场景对象发出的光线，这样可以避免产生走样问题。Reyes 把多边形表示的对象或曲面表示的对象都分割为微多边形，这种微多边形是非常小的四边形，它们投影后的大小大约是半个像素。因为每个微多边形都投影到一个很小的区域，因此可以使用均匀着色的方法来简化绘制过程。在对表面分割处理时，通过微多边形的仔细着色来实现对象表面的光滑着色。

Reyes融合了其他许多有趣的技术。Reyes使用了随机采样而不是点采样来减少视觉上的走样效果。一般来说，为了更有效地使用纹理，Reyes每次只处理一帧画面上的一个小区域。注意，虽然Reyes是被精心设计的，但是绘制一个包含大量对象和复杂光照效果的单一场景也需要几个小时。

现有的这类绘制器很多，其中有一些是公用或共享软件。有些绘制器在一个产品里能够包含不同的绘制类型。因此，可以使用光线跟踪法绘制一个具有光泽表面的对象子集。同理，也可以使用辐射度方法绘制对象表面的某个其他子集。总之，这些绘制器可支持各种效果的绘制，并允许用户在绘制时间、复杂度和图像质量之间达到平衡。

13.7　并行绘制

在许多应用中，尤其是包含大量几何数据的科学可视化应用，我们从一个可能包含5亿多个数据点的数据集中绘制图像，并产生1亿多个多边形。这种情况提出了两个挑战。首先，目前即使是最好的商业显示器也只包含大约200万个像素，那么如何才能显示这么多的多边形呢？其次，如果由于生成新的数据或由于变换操作改变了原始数据而需要显示多帧图像，那么甚至需要比高端系统更快的绘制速度来绘制这么大规模的几何数据。

解决显示分辨率问题的常用方法是建立一个幕墙（power wall），即由一组投影仪投射的大型投影平面（如图 13.21 所示），其中每个投影仪都具有高清投影设备的分辨率，即 1024 × 1280 或 1920 × 1200。一般来说，从投影仪发射的光线在边缘处逐渐减弱，并且所有显示的图像之间稍有重叠，从而形成无缝的图像。我们也可以使用一组标准尺寸的液晶显示器（LCD）来组建高分辨率显示器，虽然使用该方法的代价是能够看到窗格之间小的间隔，这会使整个窗口看起来是由许多窗格拼接而成的。

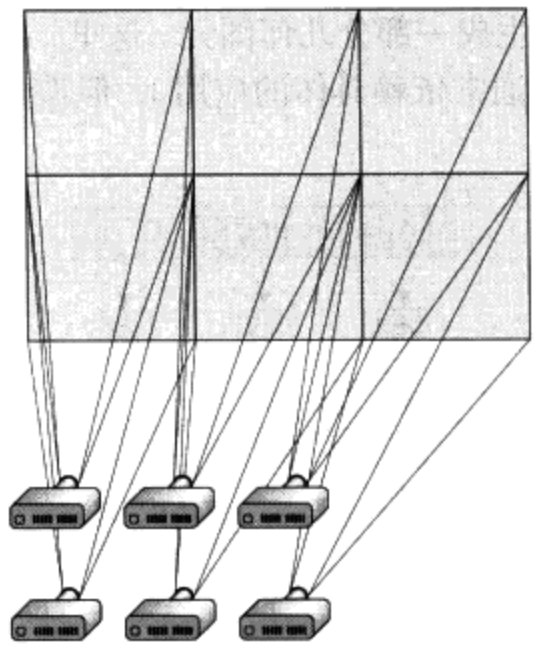


图 13.21　使用 6 个投影仪的幕墙

解决这两个问题的一个方法是使用通过高速网络连接起来的标准计算机簇。每台计算机都配备一个商用的图形显示卡。注意，这样的配置只是高性能计算领域主要变革的一个方面。以往，超级计算机都由昂贵的快速处理器组成，它在设计时通常考虑了其高度的并行性。这些处理器是根据用户需求量身设计的，一般需要特殊的接口、外设和环境，因此价格非常昂贵，一般只有一些政府实验室和大公司才能买得起这种超级计算机。最近几年，商用处理器的速度大大提高且价格也不是很贵。同样是这些技术的使用，使人们设计并开发了插件式图形显示卡，其绘制性能可达到每秒数百万个多边形和每秒上亿个像素。由这样的插件组装成的计算机可以通过标准的网络连

接并以每秒千兆的速率传送数据。因此，可以把这些计算机簇连接起来，其中每个节点的成本也就是1000～3000美元之间。

然而，我们可以使用多种分配方法将绘制场景的任务分配给多个处理器。最简单的方法是在每个处理器中执行同样的应用程序，但是每个处理器使用一个不同的窗口，该窗口与每个处理器在输出图像阵列中的相应显示位置是一一对应的。对于一些小的应用，这种方法可能奏效，但是对于复杂的应用，因为每个处理器需要处理所有的任务，并没有利用多处理器带来的优势，所以使用这种简单方法的应用程序运行会很慢。我们将讨论3种其他的方法。这3种方法的主要区别是在绘制过程的哪个步骤将图元指定或排序到显示器的正确区域。根据这种排序操作的处理时机，可以指定3种绘制方法，即先排序、中排序和后排序。

假定这里包含的大量处理器只有两种类型：几何处理器和光栅处理器。这种区分对应于第7章讨论的绘制流水线的两个阶段。几何处理器处理绘制流水线前端的浮点计算，包括变换、裁剪和明暗处理。光栅处理器操控二进制比特位，并处理扫描转换等操作。注意，现代的通用处理器和图形处理器都能够执行这些任务。因此，下面采用的策略既适用于CPU也适用于GPU。在多核处理器芯片里面或在GPU里面，我们能够在不同的节点中进行任务的并行处理。这种根据图元排序的分类模式有助于组织绘制系统的结构。

13.7.1　中排序绘制方法

考虑有一组几何处理器（每一个用G表示）和光栅处理器（每一个用R表示），它们按如图13.22所示的方式连接。假定有一个应用程序要生成大量的几何图元。这个程序能以两种显而易见的方式来使用多个几何处理器。一种方式是：让应用程序运行在一个单独的处理器上，然后把应用程序生成的几何图元的不同部分依次传送给不同的处理器。另一种方式是：可以在多个处理器上运行这个应用程序，每个处理器只生成一部分几何图元。这里，我们不必关心几何图元如何被传送给几何处理器（因为最好的方法通常依赖具体的应用），但需要关心如何充分利用现有的几何处理器。

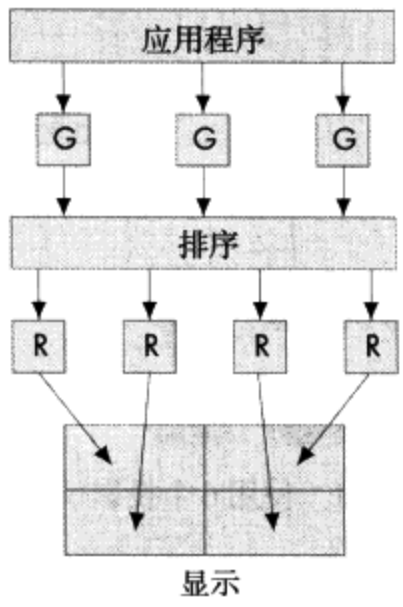


图13.22　中排序绘制方法

假定可以将任意类型的图元传送给任何一个独立工作的几何处理器。当并行使用多个处理器时，我们最关心是负载平衡（load balancing）问题，也就是说，让每个处理器的工作量大致相同，这样不会出现某个处理器长时间处于闲置的状态，这样会造成资源的浪费。一种显而易见的方法是根据处理器的个数均匀划分场景对象的坐标空间。但是这种方法经常导致很差的负载平衡，这是因

为在许多应用中，几何图元在对象空间里并不是均匀分布的。另一种方法是：当对象生成后，可以在处理器之间均匀地分配几何图元，而与几何对象的位置无关。因此，如果有 n 个处理器，我们可能把第一个几何实体传送给第一个处理器，把第二个几何实体传送给第二个处理器，把第 n 个几何实体传送给第 n 个处理器，第 $n+1$ 个几何实体传送给第一个处理器，以此类推。下面考虑光栅处理器。我们可以把每个光栅处理器对应于帧缓存的不同区域，或对应于显示器的不同区域。因此，每个光栅处理器只绘制显示器某个固定的区域。

现在的问题是如何把几何处理器的输出传送给光栅处理器。我们注意到每个几何处理器能够处理显示器上任意位置的对象。因此，必须对几何处理器的输出进行排序，并把几何处理器生成的图元传送给正确的光栅处理器，所以图元在光栅化处理之前必须先实施某种排序操作，这种结构称为中排序。

这种结构在几年前的高端图形工作站上很盛行，当时有专门的硬件来处理每项任务，并且还有快速的内部总线用于排序过程中数据的传递。目前的GPU包含多个几何处理器和多个片元处理器，因此可以把它看做中排序处理器。我们一般趋于把包含单个 GPU 的商用专业显卡视为一个几何处理器和一个光栅处理器的组合，因此在 GPU 内部集结了并行特性。现在的问题是如何使用一组常用显卡或GPU。如果把GPU或CPU作为几何处理器或光栅处理器，并且使用标准的网络将它们连接起来，那么中排序方法的排序过程可能成为该方法的瓶颈，而另外两种方法则被证明是更简单的方法。

13.7.2　后排序绘制方法

对于中排序绘制方法，几何处理器和光栅处理器的个数可能是不同的。现在假定每个几何处理器连接到它自己的光栅处理器，如图 13.23 所示。基于这种结构可以得到一组标准计算机的连接，每台计算机都有自己的图形卡，对于最新的图形卡，每个显卡具有多个内置的顶点和片元处理器。我们还是不需要关心每个处理器是如何获取应用程序的数据的，而只要关心这个结构如何处理应用程序生成的几何图元。

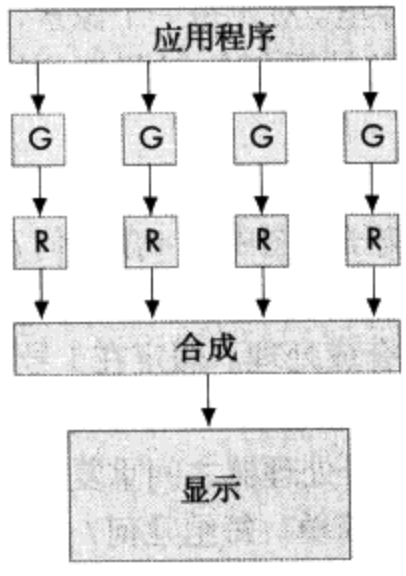


图 13.23　后排序绘制方法

就像中排序绘制方法那样，为了使几何处理器达到负载平衡，把几何图元传送给几何处理器的顺序并没有考虑几何图元被光栅化后在显示器上的位置。然而，正是因为这种几何分配及中间缺乏排序的方式，使得每个光栅处理器必定有一个与显示器大小一样的帧缓存。因为每对几何/光栅处理器都有一个完整的绘制流程，所以对于绘制的每一部分几何图元，每对处理器都可以生成一幅经

过面消隐的正确图像。图13.24显示了3幅正确的图像，而第4幅显示了如何把前3幅图像组合成一个包含所有几何图元的正确图像。

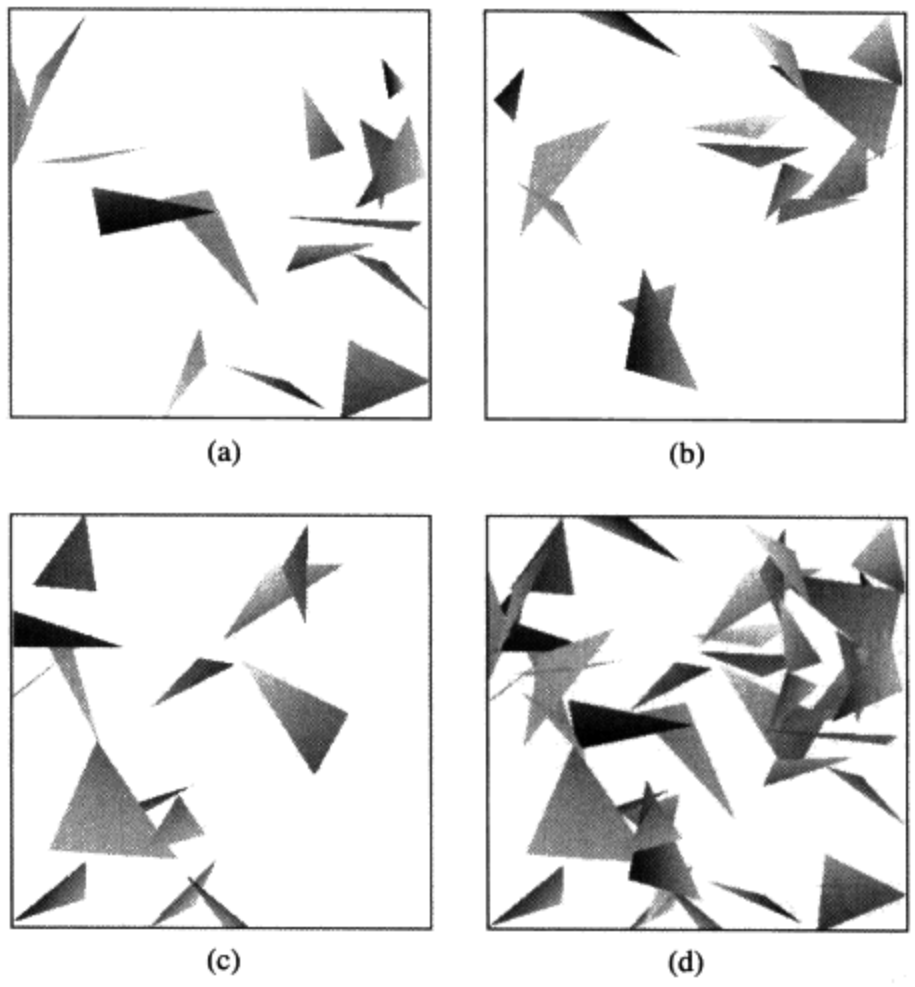


图13.24　后排序绘制方法的例子：(a)~(c) 部分绘制；(d) 合成图像（由新墨西哥州大学的Ge Li提供）

如图13.24所示，可以通过合成步骤对部分图像进行合成。为了进行合成计算，我们不仅需要几何处理器颜色缓存中的图像，而且还需要像素的深度信息，因为对每个像素，都需要知道哪一个光栅处理器中包含离视点最近的点对应的像素[①]。幸运的是，如果使用标准的OpenGL绘制流水线，那么需要的深度信息可以保存在z缓存里。对于每一个像素，只需要比较每一个z缓存里的深度值，然后把最近深度的颜色值写到处理器的帧缓存中。这里遇到的困难是如何才能高效地比较存储在多个处理器中的深度信息。

理论上，最简单的方法［也称为二叉树合成（binary-tree compositing）］是将两个处理器中的信息进行合成。分析图13.25所示的例子，图中有4条几何/光栅绘制流程，编号为0~3。对0号和1号处理器的信息进行合成可以生成它们所能见到的几何对象的正确图像，与此同时，2号和3号处理器对它们的信息也进行同样的合成处理。假定在1号和3号处理器上合成新的图像。因此，0号和2号处理器必须将它们各自的颜色缓存和z缓存中的信息都传送到相邻的处理器（分别是1号和3号处理器）。然后在1号处理器和3号处理器之间重复这个过程，最终生成的图像存放在3号处理器的帧缓存。注意，其实现代码非常简单。每组几何/光栅处理器都按常规的流程进行绘制。合成步骤只需要调用glReadPixels函数并进行一些简单的比较。但是，在每一个合成步骤的后一个步骤，仍然需要使用前一步骤使用的一半数量的处理器。在最后一步，最终合成的图像位于单个处理器中。

还有另一种称为二叉交换合成（binary-swap compositing）的方法可用于处理器的信息合成，该方法可以避免处理器闲置的问题。在这种方法中，每个处理器负责绘制最终图像的一部分。因此，

① 简单起见，假定所有的几何对象都是不透明的。

为了合成正确的图像，每个处理器都能看到所有的数据。如果有 n 个处理器参与图像合成的处理，那么可以按图 13.26 所示的环形方式安放处理器。合成过程需要 n 个步骤（而不是二叉树合成所需的 $\log n$ 个步骤）。第一步，0 号处理器将其帧缓存中的第 0 个区域传送到 1 号处理器，并且从 n 号处理器接收第 n 个区域。其他处理器也做类似的传送和接收相邻处理器的颜色缓存和深度缓存中相应区域信息的任务。这样，每个处理器都可以更新显示屏幕的一个相应区域，这个区域对应于来自两个相邻的处理器相应区域的数据。例如 0 号处理器会更新第 n 个区域。第二步，0 号处理器接收 n 号处理器的第 $n-1$ 个区域的数据，该区域恰好对应于来自 n 号和 $n-1$ 号处理器的数据。0 号处理器也要从第 n 个区域发送数据，其他处理器的一部分帧缓存也要做类似的数据发送任务。现在所有处理器都有了来自 3 个处理器的正确数据区域。可以归纳得到，经过 $n-1$ 个步骤之后，每个处理器仅包含最终图像的 $1/n$。虽然我们执行了更多的步骤，但是交换的数据要比二叉树合成的方法少得多，而且在每个步骤都使用了所有的处理器。

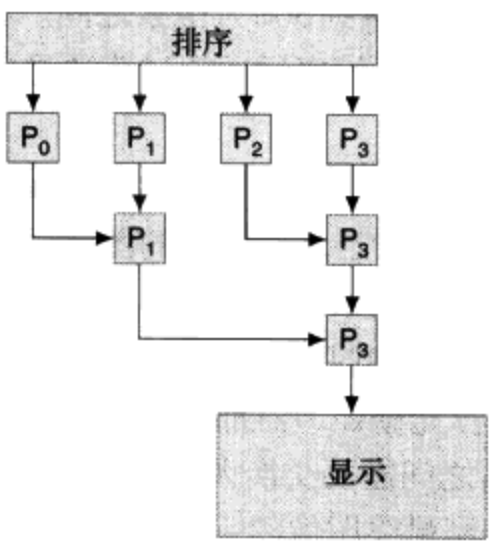


图 13.25　二叉树合成

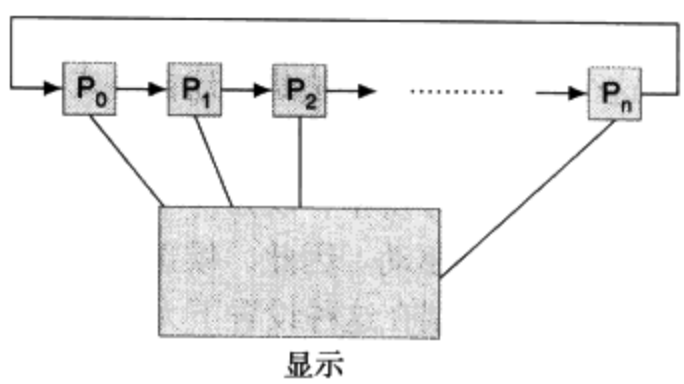


图 13.26　二叉交换合成

13.7.3　先排序绘制方法

后排序绘制方法最吸引人的特性之一是：可以把几何处理器和光栅处理器进行配组，并使用配有标准图形卡的普通计算机。假定在绘制流程的一开始就能确定每个图元的最终显示位置，那么就可以把要显示的每一个区域分配给每一对几何 / 光栅处理器，这样可以避免网络上的数据合成。这种结构如图 13.27 所示。在这种结构的前端放置了一个处理器，它负责把图元分配给相应的处理器进行处理。

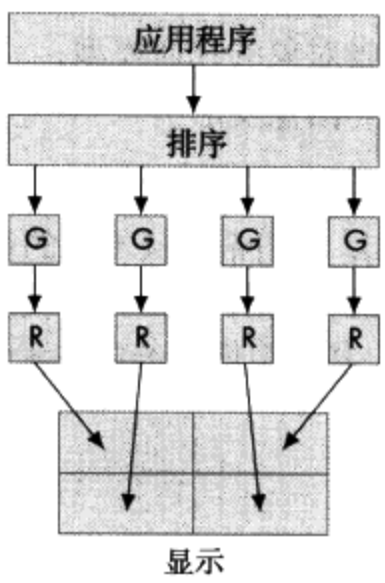


图 13.27　先排序绘制方法

这种先排序绘制方案可行的关键是结构前端的排序处理。从某种意义上说，这似乎是不可能的，因为这意味着在求解这种基于几何绘制流水线的问题之前就已经知道了问题的解，即图元在显示器的位置。但是事情并不是毫无希望的。许多问题具有结构化的组织形式，这使得我们可以提前获取相关的信息。我们还可以使用glGetFloatv函数从绘制流水线中获取信息，从而得到从建模坐标系到屏幕坐标系的映射。此外，我们不必确保获取的信息总是正确的。如果一个图元跨越显示器的若干区域，该图元可以被传送给多个几何处理器。即使没有正确地把某个图元传送给相应的处理器，这个处理器也能够把它传送给正确的处理器。因为每个几何处理器都要执行一个裁剪步骤，所以能确保最后得到的图像是正确的。

先排序绘制方法不需要解决负载平衡问题，因为如果屏幕的某些区域只有很少的图元，那么相应的处理器也许不会超负荷运行。然而，相对于后排序绘制方法，先排序绘制方法有一个主要的优点：它对于生成高分辨率图像是一种理想的方法。假定希望以大大超过普通CRT或LCD显示器的分辨率显示绘制结果，而这些普通显示器的分辨率一般在100万像素到300万像素之间。当我们要绘制一些可能包含超过1亿个几何图元的高分辨率数据时，就需要分辨率远远超过普通显示器的高分辨率显示器。

解决这个问题的一个办法是构建窗格式显示器或由一组标准显示器组成的幕墙。构成幕墙的窗格可以是CRT、LCD面板或投影仪的输出。从图形绘制的角度来看，希望绘制的图像分辨率可以达到所有显示器组的分辨率，该分辨率可能超过4000 × 4000。一般来说，这些显示器由一簇配备商用图形卡的计算机驱动。因此，候选的绘制策略包括先排序和后排序绘制。

然而，后排序绘制在这种设置下无法工作，因为每组几何/光栅处理器都必须有一个与最终绘制的图像大小一样的帧缓存，因此在合成这一步骤，在处理器之间需要交换大量的数据。先排序绘制方法不存在这个问题。每组几何/光栅处理器只需要负责绘制最终图像的一个很小的区域，这个区域一般是标准帧缓存大小的图像。

13.8　基于图像的绘制

近年来，人们对基于图像的绘制问题产生了极大的兴趣，即从一组二维图像开始，从中提取三维信息或者从中产生新的图像。经过多年的发展，这个问题已经出现了多种形式。在以往基于图像绘制的应用研究中出现的一些最重要的例子包括：

- 使用航拍照片获取地形信息；
- 使用一组二维X射线图像获取计算机轴向断层三维图像（即CT）；
- 从机器人拍摄的图像中获取场景对象的几何模型；
- 把一幅图像变形为另一幅图像（图像变形）。

最新的应用集中在从精心收集并保存的一组图像中创建新的图像。例如，假定获取了对象的一组照片（如人、建筑物或者CAD模型），我们希望从不同的视点观看这个对象。如果有了这个三维模型，则只需简单移动视点或对象，就能构建新的图像。但是如果只有二维信息，我们该怎么办呢？这些问题都属于基于图像的绘制问题。基于图像的绘制技术涉及的学科包括计算机图形学、图像处理和计算机视觉。

13.8.1　一个简单的例子

我们通过讨论如图13.28所示的问题来了解一些有关基于图像的绘制问题。图的左边是一个位于点$\mathbf{p}_1$的透视照相机，而右边是另一个位于点$\mathbf{p}_2$的照相机。这两个照相机都对点$\mathbf{q}$进行拍照。假

定知道这些照相机的所有信息（位置、朝向、视域），我们能够从照相机拍摄到的两张图像来决定点 **q** 的信息吗？图 13.29 是该问题简化后的俯视图，其中两个照相机都位于 x 轴上，它们的成像平面位于与 $z = d$ 平行的位置。利用标准的投影公式，可以得到以下两个关系：

$$\frac{x_1 - x_{p_1}}{d} = \frac{x - x_1}{z}$$

$$\frac{x_2 - x_{p_2}}{d} = \frac{x - x_2}{z}$$

求解这两个包含未知变量 x 和 z 的线性方程，可以得到：

$$z = \frac{d\Delta x}{\Delta x_p - \Delta x}$$

$$x = \frac{x_{p_1}\Delta x - x_1\Delta x_p}{\Delta x_p - \Delta x}$$

式中，$\Delta x = x_2 - x_1$，$\Delta x_p = x_{p2} - x_{p1}$。

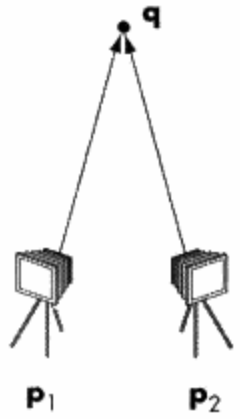


图 13.28　两个照相机对同一个点成像

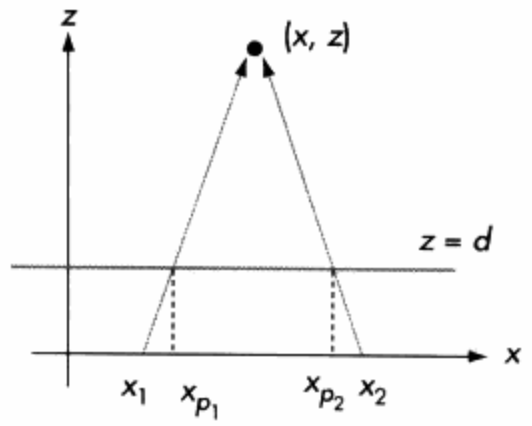


图 13.29　两个照相机的俯视图

因此，从这两幅图像可以确定点 **q** 的位置。这个结果与照相机的位置无关，移动照相机只会使得公式变得稍微复杂一些。一旦得到点 **q** 的位置，就可以从任意的视点得到它的图像。

进一步仔细分析，我们会发现一些实际问题。首先，存在一些数值问题。照相机的位置测量值哪怕出现十分小的误差也可能造成点 **q** 的近似位置值出现很大的误差。这种数值问题已经给地形测量等传统应用带来了很多麻烦。解决该问题的一种方法是使用两种以上的测量结果，然后确定一个最佳逼近理想位置的估计值。

此外还存在其他的严重问题。例如，如何获取点 $\mathbf{p}_1$ 和点 $\mathbf{p}_2$ 的位置信息，给定两个照相机拍摄的两幅图像，我们需要一种识别对应点的方法。这是计算机视觉中的一个基本问题，也是一个目前没有很好解决方案的问题。注意，如图 13.30 所示，如果存在遮挡关系，那么同一点就不可能同时出现在两幅图像上。

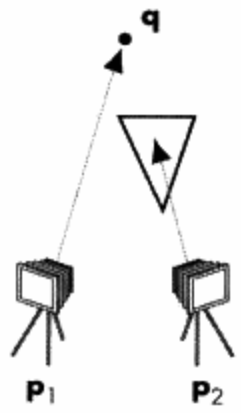


图 13.30　有遮挡的成像

许多早期的方法都是基于纯图像的，即使用统计方法寻找对应点。其他一些技术采用交互式方法，需要用户识别对应点。最近，在计算机图形学领域，出现了一些用于解决该问题的新颖方法。下面将介绍几种值得关注的方法。读者可以在本章后面的建议阅读资料中找到每种方法的描述细节。

要解决基于纯图像的方法中遇到的问题，一种方法是使用几何模型而不是点来记录位置信息。例如，在一个真实环境中，我们或许知道许多对象都是由直角平行六面体构成的，因此可以利用这些额外的信息推导出非常精确的位置信息。

另一种解决上述问题的办法是基于图像的技术，即从一个图像序列中产生某个视点的新图像。这个一般性问题的变异形式已经应用于电影行业、虚拟现实应用中的远距离观察对象以及用来产生新的图像，例如Apple公司的QuickTime VR。

其他一些研究人员已经对二维图像与其在三维环境里的光线分布之间的数学关系进行了研究。每个二维图像都是一个四维光场的采样结果。这种解决问题的方式类似于计算机轴向断层照相术（即CT）如何从二维投影构造三维图像，即从多个照相机得到的二维投影图像可用来重构三维世界。在这些方法中，其中有两种方法称为照明图（Lumigraph）和光场绘制（Light-Field Rendering）。因为光场包含了场景的所有信息，许多研究人员对如何测量光场的兴趣不断增加，但是由于该方法涉及的数据量极大，所以这是一种直到现在也无法实现的方法。光场测量的一个有趣应用是场景光照的再现。在这个应用中，可以使用其他位置的光源产生的光场来改变场景中的光照位置并在新位置重新照亮场景。

小结和注释

本章的讨论说明有许多不同的绘制方法可以使用。绘制方程包含了进行全局光照绘制所需的基本物理原理。遗憾的是，对于一般的情形，即使采用数值方法，绘制方程中所含的未知变量也太多而无法求解。虽然辐射度和光线跟踪这两种方法对场景中表面的类型做了完全相反的假设，但是两种方法都能够处理某些全局效果。随着GPU的性能变得越来越强大，它们可以处理其他高级绘制方法所需要的大量计算。因此，对于一些实时应用，采用流水线绘制方法与其他高级绘制方法之间的区别在目前不是很大。

尽管现在的计算机处理速度和价格使人们使用光线跟踪一个场景和使用辐射度方法进行计算成为可能，而这在几年前却是不可能做到的，但是在计算机图形学领域，并不是所有的问题都能使用这些技术得到求解。如果考察一下电影、电视和游戏产业的发展变化，这似乎表明，人们能够使用众多的建模方法和各种商用或共享的绘制器来绘制真实感图像。然而，人们越来越赞同这样一个观点：绘制真实感的图像并不是最终的目标。因此，我们发现研究人员对把真实感绘制、计算机建模与传统的手工动画结合起来研究的这样一些领域越来越感兴趣。很多基于图像的绘制方法非常适合许多这样的应用领域。

人们在图形客户端看到的大多数成果都是在计算机游戏驱动下取得的。看来，无论现在的处理器速度有多快、价格有多便宜，消费者对更优秀的计算机游戏的需求将一直激励开发者开发速度更快的高性能处理器。这就好像随着高清晰电视（HDTV）正成为业界的标准，人们可以看到各种各样的高分辨率显示器并能以合理的价格买到。

从科学计算领域来看，用商用计算机簇取代传统的超级计算机将继续对科学计算可视化发挥巨大的作用。运行在这些计算机簇上的应用程序生成的大规模数据集将驱动图形方面的应用开发。这

种科学计算可视化应用不但需要高分辨率显示器来显示所生成的图像，而且存储大规模数据集所带来的困难也促使人们希望以这些数据被生成的速度来对其进行可视化。

我们对将来的计算机体系结构以及它将如何影响计算机图形学还不是很清楚。像Apple MacPro这样的商用计算机具有多条总线，它们支持多个图形卡和多个处理器，每一个处理器都是多核的。Game box 正开始使用其他的一些组件，例如 IBM 的单元处理器，它负责驱动索尼的 PlayStation 3。如何才能更好地使用这些组件是一个公共的问题。我们在很大程度上确信，在计算机图形学领域仍然有许多问题有待人们去探索。

建议阅读资料

光线跟踪方法由 Appel[App68]首次提出，并经 Whitted[Whi80]推广应用。许多早期有关光线跟踪的算法都收录在 Joy 及其同事编著的论文集[Joy88]中。如果读者打算编写自己的光线跟踪器，那么Glassner[Gla89]编写的著作非常有帮助。光线跟踪方法中许多相交的测试方法在很多地方都有介绍，例如著作[Gla89]中由 Haines 编写的章节，以及图形学精粹系列[Gra90, Gra91, Gra92, Gra94, Gra95]。

绘制方程的提出要归功于 Kajiya[Kaj86]。辐射度是一种基于早期用于热传播[Sie81]的方法。Goral和他的同事[Gor84]首次把辐射度方法应用到了计算机图形学中。自从将辐射度方法引入计算机图形学以来，许多研究人员在如何提高其效率[Coh85，Coh88，Coh93]及增加镜面反射项[Sil89]方面做了大量的工作。使用点光源计算排列因子的方法在[Kel97]中有介绍。光子映射方法最早是由 Jensen[Jen01]推广的。

RenderMan 接口规范在[Ups89]中有介绍。而 Reyes 绘制结构最早在[Coo87]中有介绍。Maya [Wat02]允许使用多种类型的绘制器。

并行绘制的排序分类最早由Molnar和他的同事[Mol94]提出。Infinite Reality Graphics 等SGI高端图形工作站[Mon97]使用了中排序结构的优点。后排序结构是作为Pixel Flow结构[Mol92]的一部分提出的。二叉交换合成的方法由[Ma94]首次提出。使用商用计算机簇进行后排序绘制的软件在[Hum01]中有介绍。幕墙在[Her00, Che00]中有介绍。

通过帧图像的变形实现基于图像的绘制是微软 Talisman 硬件的一部分[Tor96]。苹果公司的Quicktime VR[Che95]是基于这样的方法来实现基于图像绘制的，即基于一个 360° 的全景图，从其中的一个视点创建新的视图。Debevec 和他的同事[Deb96]研究发现，通过一个基于模型的方法，可以从少量的图像构造出不同视点的新图像。其他的变形方法在[Sei96]中有介绍。对照明图(Lumigraph)[Gor96]和光场绘制[Lev96]的研究奠定了基于图像绘制技术的数学基础。基于图像的光照应用在[Rei05]有介绍。

习题

13.1　多边形可以使用单个平面里的一组相交的直线来描述，根据这个思想，请设计一个判断某个点是否在一个凸多边形内部的测试算法。

13.2　将习题 13.1 中的算法扩展到由相交平面构成的多面体。

13.3　试推导中心位于原点的圆环的隐式方程。推导该方程时需要注意：当一个平面横截圆环时会得到两个半径相等的圆。

13.4　利用习题13.3的结果，说明通过使用二次方程求光线与圆环的交点能够对一个圆环进行光线跟踪。

13.5 分析一条穿过球体的光线。求解这条光线上离球心最近的点。提示：考虑一条经过球心且垂直于该光线的直线。如何把这个结果用于求交测试？

13.6 在光线跟踪器中，可以通过增加光线的数量来获得更高的精度。假定对于每个像素，从像素的中心以及像素的4个拐角分别投射光线。与一个像素投射一条光线的光线跟踪器相比，这种方法需要增加多少额外的运算量？

13.7 在并行绘制的中排序绘制方法中，在几何处理器与光栅处理器之间必须传送哪些信息？

13.8 如果允许绘制透明的对象，那么对并行绘制策略必须做什么样的修改？

13.9 一种对并行计算机进行分类的方法是根据它们的内存是由各个处理器共享，还是每个处理器都有自己的内存，而其他的处理器是无法访问的。试说明这种分类方法对我们讨论过的绘制策略有什么影响？

13.10 将本章讨论的从两个视点对同一个点成像的简单例子扩展到更一般的情形，即这两个视点可位于三维空间的任意位置。

13.11 构造一个只能处理平面和球面的简单光线跟踪器。因特网上有许多有趣的数据集可用来测试所编写的代码的正确性。

13.12 假定有一个最高项是 $x^iy^jz^k$ 的代数多项式函数。当求解光线与该函数定义的曲面的交点时，这个多项式的次数为多少？

13.13 再次分析一个最高项为 $x^iy^jz^k$ 的代数多项式函数。如果 $i=j=k$，那么当使用一条参数光线与曲面做求交运算时，所生成的多项式中包含多少项？

13.14 对一个或多个OpenGL程序，试计算它们每秒绘制三角形面片的个数。相对于整个绘制流水线，试确定隐藏面消除、明暗绘制、纹理映射和光栅化操作的时间开销比例。如果读者使用商用图形卡，所测量的图形卡性能与其说明书上的性能相比较有何不同？

13.15 确定读者使用的图形卡的像素性能。判断它每秒能读/写像素的数量。读像素和写像素的速度不同吗？这与纹理映射的写操作有区别吗？

13.16 使用OpenGL为每个处理器构建一个后排序绘制器。读者可以编写应用程序生成三角形或三角网格来测试它的性能。

13.17 当增加更多的处理器时，请解释为什么先排序绘制方法的性能最终会变得更差。

附录A　示例程序

该附录包含了教材中许多示例程序的源代码。读者可以从网页（www.cs.unm.edu/~angel）下载这些程序以及教材中引用的其他程序的最新源代码。读者也可以从该网页下载一个可用于大部分Linux操作系统的makefile文件，以及其他的示例程序和使用说明。其他的示例程序（包括OpenGL Programmer's Guide和GLUT中的某些程序）可以从OpenGL的官方网站（http://www.opengl.com)下载。

OpenGL是几乎所有工作站事实上的一个图形开发标准。对于使用微软Windows操作系统的用户来说，OpenGL核心库（GL）的动态链接库（OpenGL32.dll）以及OpenGL实用程序库（GLU）的动态链接库（GLU32.dll）都存储在系统文件夹下，与之对应的静态链接库（.lib文件）和头文件则由程序设计语言的编译器（如Visual C++编译器）提供，而与之对应的GLUT文件（glut32.dll、glut32.lib和glut.h）则可从OpenGL的官方网站下载。特定显卡的驱动程序结合硬件和相关的软件技术即可实现OpenGL。因此，只要正确安装了这类特定显卡的驱动程序，那么在所有系统上使用OpenGL的方法都是相同的。不同的显卡在性能和以扩展形式提供的附加功能上有区别。

大部分Linux分发版都包含OpenGL的Mesa实现版本（www.mesa3D.org）和GLUT（实用程序工具库）。尽管Mesa是OpenGL的纯软件实现版本（并为那些对OpenGL实现问题感兴趣的人员提供了源代码），但是越来越多的商用显卡制造商提供了Linux操作系统下的显卡驱动程序，这可以使用户充分利用显卡的性能。

该附录中的示例程序使用了GLUT库来实现与Windows系统的交互。函数名遵循OpenGL Programmer's Guide和GLUT Users Guide中指定的命名规则。这些示例程序共享了许多相同的代码。读者可能会发现这些示例程序中有些函数的代码几乎是相同的，如reshape回调函数、初始化函数和main函数。因此，每个函数只有在第一次用到时，才给出详细的注释。

所有这些示例程序最重要的设计准则就是说明图形绘制的原理，而不考虑程序的运行效率。读者可以找到很多方法来扩展这些程序，并使这些程序的运行效率更高。在某些示例程序中，如果直接使用OpenGL的功能而不是示例程序使用的方法，那么可以使用完全不同的方法来获取相同的视觉效果。

该附录包含下面的示例程序：

1. 生成5000个点的Sierpinski镂垫程序（见第2章）
2. Sierpinski镂垫程序的递归实现（见第2章）
3. 三维Sierpinski镂垫的递归实现（见第2章）
4. Marching Square程序（见第2章）
5. 简单多边形建模程序（见第3章）
6. 使用单缓存和双缓存实现的正方形旋转程序（见第3章）
7. 使用选择模式实现的拾取程序（见第3章）
8. 立方体旋转程序（见第4章）
9. 使用顶点数组实现的立方体旋转程序（见第4章）

10. 使用虚拟跟踪球实现的立方体旋转程序（见第4章）
11. 场景漫游程序（见第5章）
12. 利用递归细分法生成近似球体的程序（见第6章）
13. Mandelbrot集绘制程序（见第8章）①
14. 使用Brensenham算法绘制线段的程序（见第8章）
15. 带纹理的立方体旋转程序（见第8章）
16. 一个完整的GLSL程序（见第9章）
17. 使用简单的场景图API定义并绘制一个场景图的程序（见第10章）
18. 粒子模拟程序（见第11章）
19. 绘制三次Bézier曲线的交互式程序（见第12章）

A.1　Sierpinski镂垫程序

```
/* 使用随机选择的顶点和中分点绘制二维 Sierpinski 镂垫 */

#include <GL/glut.h>

/* 读者必须根据头文件 glut.h 在系统中的实
   际存储位置来修改 include 头文件的位置*/

/* 头文件 glut.h 通常已经包含了 gl.h 和 glu.h */
void myinit()
{
/* 属性 */

      glClearColor(1.0, 1.0, 1.0, 1.0); /* 白色的背景 */
      glColor3f(1.0, 0.0, 0.0); /* 绘图颜色为红色 */

/* 设置观察条件 */
/* 在观察坐标系中定义一个大小为 50.0 × 50.0 的观察窗口，窗口的左下角为观察坐标系的坐标原点*/

      glMatrixMode(GL_PROJECTION);
      glLoadIdentity();
      gluOrtho2D(0.0, 50.0, 0.0, 50.0);
      glMatrixMode(GL_MODELVIEW);
}

void display()
{

    GLfloat vertices[3][2]={{0.0,0.0},{25.0,50.0},{50.0,0.0}};
         /* 一个三角形的顶点数据 */

    int i, j, k;
    int rand();        /* 标准的随机数生成器 */
    GLfloat p[2] ={7.5, 5.0};
         /* 在三角形内部任意位置的一个初始点 */
    glClear(GL_COLOR_BUFFER_BIT);  /* 清空绘图窗口，并把绘图窗口的颜色设置为背景颜色 */
```

① 该程序是本书第四版第8章的一个程序，第五版已将该程序放在第11章。——译者注

```
    glBegin(GL_POINTS);

/* 计算并绘制5000个新的点 */

    for( k=0; k<5000; k++)
    {
        j=rand( )%3; /* 随机选择一个三角形的顶点 */

     /* 计算随机选择的顶点和三角形内部的初始点之间的中点 */

        p[0] = (p[0]+vertices[j][0])/2.0;
        p[1] = (p[1]+vertices[j][1])/2.0;

     /* 绘制新生成的点 */

        glVertex2fv(p);

     }
     glEnd( );
     glFlush(); /* 清空缓存① */
}
int main(int argc, char** argv)
{

/* 初始化GLUT库函数的常用形式*/

    glutInit(&argc,argv);
    glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);
                                /* 这是默认的显示模式，可以省略该语句 */
    glutInitWindowSize(500,500); /* 大小为500 × 500像素的绘图窗口 */
    glutInitWindowPosition(0,0); /* 把绘图窗口的左上角放在屏幕左上角的位置 */
    glutCreateWindow("Sierpinski Gasket"); /* 绘图窗口的标题 */
    glutDisplayFunc(display);
              /* 注册显示回调函数，当窗口打开等事件发生时会调用该显示回调函数 */
    myinit(); /* 设置属性 */
    glutMainLoop(); /* 进入事件循环 */
}
```

A.2 递归实现 Sierpinski 镂垫程序

```
/* 通过递归细分三角形的方法生成Sierpinski镂垫 */
/* 通过命令行输入递归的次数 */

#include <GL/glut.h>

/* 初始的三角形 */

GLfloat v[3][2]={{-1.0, -0.58}, {1.0, -0.58}, {0.0, 1.15}};

int n;

void triangle( point2 a, point2 b, point2 c)

/* 定义某个三角形 */
```

① 原文注释可能有误，函数glFlush的正确功能是强制执行所有的绘制命令，完成图形的绘制。——译者注

```
{
        glVertex2fv(a);
        glVertex2fv(b);
        glVertex2fv(c);
}

void divide_triangle(GLfloat *a, GLfloat *b, GLfloat *c, int m)
{

/* 基于顶点的数量对三角形进行细分处理 */

    GLfloat v0[2], v1[2], v2[2];
    int j;
    if(m>0)
    {
        for(j=0; j<2; j++) v0[j]=(a[j]+b[j])/2;
        for(j=0; j<2; j++) v1[j]=(a[j]+c[j])/2;
        for(j=0; j<2; j++) v2[j]=(b[j]+c[j])/2;
        divide_triangle(a, v0, v1, m-1);
        divide_triangle(c, v1, v2, m-1);
        divide_triangle(b, v2, v0, m-1);
     }
     else triangle(a,b,c); /* 递归结束时绘制三角形 */
 }


 void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glBegin(GL_TRIANGLES);
    divide_triangle(v[0], v[1], v[2], n);
    glEnd();
    glFlush();
}

void myinit()
{
     glMatrixMode(GL_PROJECTION);
     glLoadIdentity();
     gluOrtho2D(-2.0, 2.0, -2.0, 2.0);
     glMatrixMode(GL_MODELVIEW);
     glClearColor (1.0, 1.0, 1.0, 1.0);
     glColor3f(0.0,0.0,0.0);
 }

 int main(int argc, char **argv)
 {
     n=atoi(argv[1]); /* 或者在此输入三角形细分的步数 */
     glutInit(&argc, argv);
     glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
     glutInitWindowSize(500, 500);
     glutCreateWindow("Sierpinski Gasket");
     glutDisplayFunc(display);
     myinit();
     glutMainLoop();
  }
```

A.3 三维 Sierpinski 镂垫的递归实现程序

```
/* 通过递归细分四面体的方法生成 Sierpinski 镂垫 */
/* 通过命令行输入递归的次数 */
#include <stdlib.h>
#include <GL/glut.h>

/* 初始的四面体 */

GLfloat v[4][3]={{0.0, 0.0, 1.0}, {0.0, 0.942809, -0.33333},
      {-0.816497, -0.471405, -0.333333}, {0.816497, -0.471405,
       -0.333333}};

GLfloat colors[4][3] = {{1.0, 0.0, 0.0}, {0.0, 1.0, 0.0},
      {0.0, 0.0, 1.0}, {0.0, 0.0, 0.0}};

int n;

void triangle(GLfloat *va, GLfloat *vb, GLfloat *vc)
{
       glVertex3fv(va);
       glVertex3fv(vb);
       glVertex3fv(vc);
}

void tetra(GLfloat *a, GLfloat *b, GLfloat *c, GLfloat *d)
{
    glColor3fv(colors[0]);
    triangle(a, b, c);
    glColor3fv(colors[1]);
    triangle(a, c, d);
    glColor3fv(colors[2]);
    triangle(a, d, b);
    glColor3fv(colors[3]);
    triangle(b, d, c);
}

void divide_tetra(GLfloat *a, GLfloat *b, GLfloat *c, GLfloat *d, int m)
{

    GLfloat mid[6][3];
    int j;
    if(m>0)
    {
        /* 计算6个中点 */

        for(j=0; j<3; j++) mid[0][j]=(a[j]+b[j])/2;
        for(j=0; j<3; j++) mid[1][j]=(a[j]+c[j])/2;
        for(j=0; j<3; j++) mid[2][j]=(a[j]+d[j])/2;
        for(j=0; j<3; j++) mid[3][j]=(b[j]+c[j])/2;
        for(j=0; j<3; j++) mid[4][j]=(c[j]+d[j])/2;
        for(j=0; j<3; j++) mid[5][j]=(b[j]+d[j])/2;

        /* 通过细分生成4个四面体 */
```

```
        divide_tetra(a, mid[0], mid[1], mid[2], m-1);
        divide_tetra(mid[0], b, mid[3], mid[5], m-1);
        divide_tetra(mid[1], mid[3], c, mid[4], m-1);
        divide_tetra(mid[2], mid[4], d, mid[5], m-1);

    }
    else(tetra(a,b,c,d)); /* 递归结束时绘制四面体 */
}


void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glBegin(GL_TRIANGLES);
    divide_tetra(v[0], v[1], v[2], v[3], n);
    glEnd();
    glFlush();
}


void myReshape(int w, int h)
{
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    if (w <= h)
        glOrtho(-2.0, 2.0, -2.0 * (GLfloat) h / (GLfloat) w,
            2.0 * (GLfloat) h / (GLfloat) w, -10.0, 10.0);
    else
        glOrtho(-2.0 * (GLfloat) w / (GLfloat) h,
            2.0 * (GLfloat) w / (GLfloat) h, -2.0, 2.0, -10.0, 10.0);
    glMatrixMode(GL_MODELVIEW);
    glutPostRedisplay();
}


int main(int argc, char **argv)
{
    n=atoi(argv[1]); /* 或者在此输入四面体细分的步数 */
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(500, 500);
    glutCreateWindow("3D Gasket");
    glutReshapeFunc(myReshape);
    glutDisplayFunc(display);
    glEnable(GL_DEPTH_TEST);
    glClearColor (1.0, 1.0, 1.0, 1.0);
    glutMainLoop();
}
```

A.4 MARCHING SQUARES 程序

```
/* 利用 marching squares 方法生成等高线 */

/* 定义区域大小 */
```

```
#define X_MAX 1.0
#define Y_MAX 1.0
#define X_MIN -1.0
#define Y_MIN -1.0

/* 定义单元的个数 */

#define N_X 50
#define N_Y 50

/* 定义等高线的值 */

#define THRESHOLD 0.0

#include <GL/glut.h>

void display()
{
   double f(double,double);
   int cell(double, double, double, double);
   void lines(int, int, int, double, double, double, double);

   double data[N_X][N_Y];
   int i,j;
   int c;

   glClear(GL_COLOR_BUFFER_BIT);

/* 把函数计算的结果存储在 data 数组中 */

   for(i=0;i<N_X;i++) for (j=0;j<N_Y;j++)
      data[i][j]=f(X_MIN+i*(X_MAX-(X_MIN))/(N_X-1.0),Y_MIN+j*
         (Y_MAX-(Y_MIN))/(N_Y-1.0));

   /* 处理每个单元 */

   for(i=0;i<N_X;i++) for (j=0;j<N_Y;j++)
   {
      c=cell(data[i][j], data[i+1][j], data[i+1][j+1], data[i][j+1]);
      lines(c,i,j,data[i][j], data[i+1][j], data[i+1][j+1],
         data[i][j+1]);
   }
glFlush();
}

/* 定义函数 f(x, y) */

double f(double x, double y)
{

   double a=0.49, b=0.5;

   /* 卡西尼卵形线 */
```

```
    return (x*x+y*y+a*a)*(x*x+y*y+a*a)-4*a*a*x*x-b*b*b*b;
}

/* 定义单元顶点 */

int cell(double a, double b, double c , double d)
{
    int n=0;
    if(a>THRESHOLD) n+=1;
    if(b>THRESHOLD) n+=8;
    if(c>THRESHOLD) n+=4;
    if(d>THRESHOLD) n+=2;
    return n;
}

/* 为每个单元绘制线段 */

void lines(int num, int i, int j, double a, double b, double c,
     double d)
{
void draw_one(int, int, int, double, double, double, double);
void draw_adjacent(int, int, int, double, double, double, double);
void draw_opposite(int, int, int, double, double, double, double);
     switch(num)
     {
     case 1: case 2: case 4: case 7: case 8: case 11: case 13: case 14:
        draw_one(num, i,j,a,b,c,d);
        break;
     case 3: case 6: case 9: case 12:
        draw_adjacent(num,i,j,a,b,c,d);
        break;
     case 5: case 10:
        draw_opposite(num, i,j,a,b,c,d);
        break;
     case 0: case 15: break;
     }
}

void draw_one(int num, int i, int j, double a, double b, double c,
  double d)
{
  double x1, y1, x2, y2;
  double ox, oy;
  double dx, dy;
  dx=(X_MAX-(X_MIN))/(N_X-1.0);
  dy=(Y_MAX-(Y_MIN))/(N_Y-1.0);
  ox=X_MIN+i*(X_MAX-(X_MIN))/(N_X-1.0);
  oy=Y_MIN+j*(Y_MAX-(Y_MIN))/(N_Y-1.0);
    switch(num)
    {
    case 1: case 14:
      x1=ox;
      y1=oy+dy*(THRESHOLD-a)/(d-a);
      x2=ox+dx*(THRESHOLD-a)/(b-a);
      y2=oy;
      break;
```

```
    case 2: case 13:
      x1=ox;
      y1=oy+dy*(THRESHOLD-a)/(d-a);
      x2=ox+dx*(THRESHOLD-d)/(c-d);
      y2=oy+dy;
      break;
    case 4: case 11:
      x1=ox+dx*(THRESHOLD-d)/(c-d);
      y1=oy+dy;
      x2=ox+dx;
      y2=oy+dy*(THRESHOLD-b)/(c-b);
      break;
    case 7: case 8:
      x1=ox+dx*(THRESHOLD-a)/(b-a);
      y1=oy;
      x2=ox+dx;
      y2=oy+dy*(THRESHOLD-b)/(c-b);
      break;
    }
  glBegin(GL_LINES);
    glVertex2d(x1, y1);
    glVertex2d(x2, y2);
  glEnd();
}


void draw_adjacent(int num, int i, int j, double a, double b, double c,
double d)
{
  double x1, y1, x2, y2;
  double ox, oy;
  double dx, dy;
  dx=(X_MAX-(X_MIN))/(N_X-1.0);
  dy=(Y_MAX-(Y_MIN))/(N_Y-1.0);
  ox=X_MIN+i*(X_MAX-(X_MIN))/(N_X-1.0);
  oy=Y_MIN+j*(Y_MAX-(Y_MIN))/(N_Y-1.0);
  switch(num)
  {
    case 3: case 12:
      x1=ox+dx*(THRESHOLD-a)/(b-a);
      y1=oy;
      x2=ox+dx*(THRESHOLD-d)/(c-d);
      y2=oy+dy;
      break;
    case 6: case 9:
      x1=ox;
      y1=oy+dy*(THRESHOLD-a)/(d-a);
      x2=ox+dx;
      y2=oy+dy*(THRESHOLD-b)/(c-b);
      break;
  }
  glBegin(GL_LINES);
    glVertex2d(x1, y1);
    glVertex2d(x2, y2);
```

```
  glEnd();

}

void draw_opposite(int num, int i, int j,
  double a, double b, double c, double d)
{
  double x1,y1,x2,y2,x3,y3,x4,y4;
  double ox, oy;
  double dx, dy;
  dx=(X_MAX-(X_MIN))/(N_X-1.0);
  dy=(Y_MAX-(Y_MIN))/(N_Y-1.0);
  ox=X_MIN+i*(X_MAX-(X_MIN))/(N_X-1.0);
  oy=Y_MIN+j*(Y_MAX-(Y_MIN))/(N_Y-1.0);
  switch(num)
  {
    case5:
      x1=ox;
      y1=oy+dy*(THRESHOLD-a)/(d-a);
      x2=ox+dx*(THRESHOLD-a)/(b-a);
      y2=oy;
      x3=ox+dx*(THRESHOLD-d)/(c-d);
      y3=oy+dy;
      x4=ox+dx;
      y4=oy+dy*(THRESHOLD-b)/(c-b);
      break;
    case 10:
      x1=ox;
      y1=oy+dy*(THRESHOLD-a)/(d-a);
      x2=ox+dx*(THRESHOLD-d)/(c-d);
      y2=oy+dy;
      x3=ox+dy*(THRESHOLD-a)/(b-a);
      y3=oy;
      x4=ox+dx;
      y4=oy+dy*(THRESHOLD-b)/(c-b);
      break;
  }
  glBegin(GL_LINES);
    glVertex2d(x1, y1);
    glVertex2d(x2, y2);
    glVertex2d(x3, y3);
    glVertex2d(x4, y4);
  glEnd();
}

void myReshape(int w, int h)
{
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    if (w <= h)
       gluOrtho2D(X_MIN, X_MAX, Y_MIN * (GLfloat) h / (GLfloat) w,
       Y_MAX * (GLfloat) h / (GLfloat) w);
    else
    gluOrtho2D(X_MIN * (GLfloat) w / (GLfloat) h,
```

```
    X_MAX * (GLfloat) w / (GLfloat) h, Y_MIN, Y_MAX);
    glMatrixMode(GL_MODELVIEW);
}

int main(int argc, char **argv)
{
   glutInit(&argc, argv);
   glutInitWindowSize(500, 500);
   glutCreateWindow("contour plot");
   glutReshapeFunc(myReshape);
   glutDisplayFunc(display);
   glClearColor(0.0,0.0,0.0,1.0);
   glColor3f(1.0,1.0,1.0);
   glutMainLoop();
}
```

A.5 多边形建模程序

```
/* 多边形建模器 */

#define MAX_POLYGONS 8
#define MAX_VERTICES 10

typedef int bool;
#define TRUE 1
#define FALSE 0

#include <stdlib.h>
#include <stdio.h>
#include <GLUT/glut.h>

void myMouse(int,int, int, int);
void myMotion(int, int);
void myDisplay();
void myReshape(int, int);
void color_menu(int);
void main_menu(int);
int pick_polygon(int x, int y);

void myinit();

/* 全局变量 */

/* 定义多边形的结构体类型 */

typedef struct polygon
{
     int color; /* 颜色索引 */
     bool used; /* 如果多边形已被定义，则为TRUE */
     int xmin, xmax, ymin, ymax; /* 包围盒 */
     float xc, yc; /* 多边形的中心 */
     int nvertices; /* 顶点的数目 */
     int x[MAX_VERTICES]; /* 顶点 */
     int y[MAX_VERTICES];
} polygon;
```

```
/* 定义标记变量 */

bool picking = FALSE; /* 如果处于拾取状态，则为TRUE */
bool moving = FALSE; /* 如果处于移动多边形的状态，则为TRUE */
int in_polygon = -1; /* 不在任何多边形的内部 */
int present_color = 0; /* 默认的颜色 */

GLsizei wh = 500, ww = 500; /* 初始窗口的大小 */
int draw_mode = 0; /* 绘制模式 */

GLfloat colors[8][3]={{0.0, 0.0, 0.0}, {1.0, 0.0, 0.0},{0.0, 1.0, 0.0},
    {0.0, 0.0, 1.0}, {0.0, 1.0, 1.0}, {1.0, 0.0, 1.0}, {1.0, 1.0, 0.0},
    {1.0, 1.0, 1.0}};

polygon polygons[MAX_POLYGONS];


void myReshape(int w, int h)
{

        glMatrixMode(GL_PROJECTION);
        glLoadIdentity();
        gluOrtho2D(0.0, (GLdouble)w, 0.0, (GLdouble)h);
        glMatrixMode(GL_MODELVIEW);
        glLoadIdentity();

        glViewport(0,0,w,h);
        ww = w;
        wh = h;
}

void myinit()
{
        int  i;

/* 把清屏颜色设置为灰色 */

        glClearColor(0.5, 0.5, 0.5, 1.0);

/* 把所有的多边形标记为未定义状态 */

        for(i = 0; i<MAX_POLYGONS; i++) polygons[i].used = FALSE;
}


void myMouse(int btn, int state, int x, int y)
{
    int i,j;
    y = wh-y;
    if(btn==GLUT_LEFT_BUTTON && state==GLUT_DOWN &&!picking&&!moving)

    /* 增加顶点 */

    {
```

```
    moving = FALSE;
    if(in_polygon>=0)
    {
         if(polygons[in_polygon].nvertices == MAX_VERTICES)
         {
             printf("exceeds maximum number vertices\n");
             exit(0);
         }
         i = polygons[in_polygon].nvertices;
         polygons[in_polygon].x[i] = x;
         polygons[in_polygon].y[i] = y;
         polygons[in_polygon].nvertices++;
    }
}
if(btn==GLUT_LEFT_BUTTON && state==GLUT_DOWN &&picking&&!moving)
{

/* 删除多边形 */

    picking = FALSE;
    moving = FALSE;
    j = pick_polygon(x,y);
    if(j >= 0)
    {
                polygons[j].used = FALSE;
                in_polygon = -1;
                glutPostRedisplay();
    }
}

int pick_polygon(int x, int y)
{

        /* 根据每个多边形的包围盒寻找第一个包含拾取位置的多边形 */

        int i;
        for(i=0; i<MAX_POLYGONS; i++)
        {
             if(polygons[i].used)
                if((x>=polygons[i].xmin)&&(x<=polygons[i].xmax)&&
                   (y>=polygons[i].ymin)&&(y<polygons[i].ymax))
                {
                    in_polygon = i;
                    moving = TRUE;
                    return(i);
                }
        printf("not in a polygon\n");
        return(-1);
        }
}
```

```
void myMotion(int x, int y)
{

   /* 判断是否位于多边形内部 */

   float dx, dy;
   int i,j;
   if(moving)
   {
       y = wh - y;
       j = pick_polygon(x, y);
       if(j<0)
          {
                printf("not in a polygon\n");
                return;
           }

       /* 如果位于多边形内部，则移动多边形 */

       dx = x - polygons[j].xc;
       dy = y - polygons[j].yc;
       for(i = 0; i< polygons[j].nvertices; i++)
       {
           polygons[j].x[i] += dx;
           polygons[j].y[i] += dy;
       }

       /* 更新包围盒 */

        polygons[j].xc += dx;
        polygons[j].yc += dy;
        if(dx>0) polygons[j].xmax += dx;
          else polygons[j].xmin += dx;
        if(dy>0) polygons[j].ymax += dy;
          else polygons[j].ymin += dy;
        glutPostRedisplay();
    }
}

void color_menu(int index)
{
    present_color =  index;
    if(in_polygon>=0) polygons[in_polygon].color = index;
}

void main_menu(int index)
{
   int i;
   switch(index)
{
   case(1):  /* 建立一个新的多边形 */
  {

      moving = FALSE;
      for(i=0; i<MAX_POLYGONS; i++) if(polygons[i].used == FALSE) break;
```

```
        if(i == MAX_POLYGONS)
        {
            printf("exceeeded maximum number of polygons\n");
            exit(0);
        }
            polygons[i].color = present_color;
            polygons[i].used = TRUE;
            polygons[i].nvertices = 0;
            in_polygon = i;
            picking = FALSE;
            break;
    }
case(2):    /* 结束多边形的定义并计算多边形的包围盒和它的中心位置 */
{
            moving = FALSE;
            if(in_polygon>=0)
            {
               polygons[in_polygon].xmax = polygons[in_polygon].xmin
                  = polygons[in_polygon].x[0];
               polygons[in_polygon].ymax = polygons[in_polygon].ymin
                  = polygons[in_polygon].y[0];
               polygons[in_polygon].xc = polygons[in_polygon].x[0];
               polygons[in_polygon].yc = polygons[in_polygon].y[0];
               for(i=1;i<polygons[in_polygon].nvertices;i++)
               {
                 if(polygons[in_polygon].x[i]<polygons
                    [in_polygon].xmin)polygons[in_polygon].xmin
                    = polygons[in_polygon].x[i];
                 else if(polygons[in_polygon].x[i]>polygons
                    [in_polygon].xmax)polygons[in_polygon].xmax
                    = polygons[in_polygon].x[i];
                 if(polygons[in_polygon].y[i]<polygons
                    [in_polygon].ymin)polygons[in_polygon].ymin
                    = polygons[in_polygon].y[i];
                 else if(polygons[in_polygon].y[i]>polygons
                    [in_polygon].ymax)
                    polygons[in_polygon].ymax = polygons
                    [in_polygon].y[i];
                 polygons[in_polygon].xc += polygons[in_polygon].x[i];
                 polygons[in_polygon].yc += polygons[in_polygon].y[i];
               }
               polygons[in_polygon].xc = polygons[in_polygon]
                   .xc/polygons [in_polygon].nvertices;
                   polygons[in_polygon].yc = polygons[in_polygon]
                       .yc/polygons[in_polygon].nvertices;
                }
    in_polygon = -1;
            glutPostRedisplay();
    break;
}
case(3):  /* 设置拾取模式 */
{
            picking = TRUE;
            moving = FALSE;
    break;
}
```

```
        case(4):  /* 设置移动模式 */
        {
                moving = TRUE;
                break;
        }
    case(5):  /* 退出 */
    {
        exit(0);
        break;
    }
   }
}

void myDisplay()
{

    /* 显示所有的活动多边形（即已被定义的多边形）*/

    int i, j;

    glClear(GL_COLOR_BUFFER_BIT);
    for(i=0; i<MAX_POLYGONS; i++)
    {
       if(polygons[i].used)
       {
           glColor3fv(colors[polygons[i].color]);
           glBegin(GL_POLYGON);
           for(j=0; j<polygons[i].nvertices; j++) glVertex2i
               (polygons[i].x[j], polygons[i].y[j]);
           glEnd();
       }
    }
    glFlush();
}

int main(int argc, char** argv)
{
    int c_menu;

    glutInit(&argc,argv);
    glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(500, 500);
    glutCreateWindow("polygon modeler");
    glutDisplayFunc(myDisplay);
    myinit ();
    c_menu = glutCreateMenu(color_menu);
    glutAddMenuEntry("Black",0);
    glutAddMenuEntry("Red",1);
    glutAddMenuEntry("Green",2);
    glutAddMenuEntry("Blue",3);
    glutAddMenuEntry("Cyan",4);
    glutAddMenuEntry("Magenta",5);
    glutAddMenuEntry("Yellow",6);
    glutAddMenuEntry("White",7);
```

```
    glutCreateMenu(main_menu);
    glutAddMenuEntry("new polygon", 1);
    glutAddMenuEntry("end polygon", 2);
    glutAddMenuEntry("delete polygon", 3);
    glutAddMenuEntry("move polygon", 4);
    glutAddMenuEntry("quit",5);
    glutAddSubMenu("Colors", c_menu);
    glutAttachMenu(GLUT_MIDDLE_BUTTON);

    glutReshapeFunc (myReshape);
    glutMouseFunc (myMouse);
    glutMotionFunc(myMotion);
    glutMainLoop();

}
```

A.6 利用双缓存的程序

```
/*
 *  double.c
 *  该程序说明如何利用双缓存来实现平滑的
 *  动画效果。单击鼠标左键使正方形旋转，
 *  单击鼠标中键使正方形停止旋转
 */
#include <GL/glut.h>
#include <stdlib.h>
#include <math.h>
#define DEGREES_TO_RADIANS 3.14159/180.0

static GLfloat spin = 0.0;
GLfloat x, y;
int singleb, doubleb;

void square()
{
    glBegin(GL_QUADS);
       glVertex2f(x,y);
       glVertex2f(-y,x);
       glVertex2f(-x,-y);
       glVertex2f(y,-x);
    glEnd();
}


void displayd()
{
    glClear (GL_COLOR_BUFFER_BIT);
    square();
    glutSwapBuffers ();
}

void displays()
{
    glClear (GL_COLOR_BUFFER_BIT);
    square();
```

```
    glFlush();
}

void spinDisplay ()
{
    spin = spin + 2.0;
    if (spin > 360.0) spin = spin - 360.0;
    x= 25.0*cos(DEGREES_TO_RADIANS * spin);
    y= 25.0*sin(DEGREES_TO_RADIANS * spin);
    glutSetWindow(singleb);
    glutPostRedisplay();
    glutSetWindow(doubleb);
    glutPostRedisplay();
}

void myinit ()
{
    glClearColor (0.0, 0.0, 0.0, 1.0);
    glColor3f (1.0, 1.0, 1.0);
    glShadeModel (GL_FLAT);
}

void mouse(int btn, int state, int x, int y)
{
     if(btn==GLUT_LEFT_BUTTON && state==GLUT_DOWN)
         glutIdleFunc(spinDisplay);
     if(btn==GLUT_MIDDLE_BUTTON && state==GLUT_DOWN)
         glutIdleFunc(NULL);
}

void myReshape(int w, int h)
{
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    if (w <= h)
    glOrtho (-50.0, 50.0, -50.0*(GLfloat)h/(GLfloat)w,
        50.0*(GLfloat)h/(GLfloat)w, -1.0, 1.0);
    else
    glOrtho (-50.0*(GLfloat)w/(GLfloat)h,
        50.0*(GLfloat)w/(GLfloat)h, -50.0, 50.0, -1.0, 1.0);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity ();
}

/*  主循环
 *  设置绘图窗口的初始大小、标题和
 *  RGBA 显示模式，并处理输入事件
 */
int main(int argc, char** argv)
{

    glutInit(&argc,argv);
    glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);
    singleb=glutCreateWindow("single buffered");
    myinit ();
```

```
    glutDisplayFunc(displays);
    glutReshapeFunc (myReshape);
    glutIdleFunc (spinDisplay);
    glutMouseFunc (mouse);
    glutInitDisplayMode (GLUT_DOUBLE | GLUT_RGB);
    doubleb=glutCreateWindow("double buffered");
    myinit ();
    glutDisplayFunc(displayd);
    glutReshapeFunc (myReshape);
    glutIdleFunc (spinDisplay);
    glutMouseFunc (mouse);

    glutMainLoop();

}
```

A.7 使用选择模式实现的拾取程序

```
#include <stdlib.h>
#include <stdio.h>
#include <GL/glut.h>


void init()
{
   glClearColor (0.0, 0.0, 0.0, 0.0);
}

void drawObjects(GLenum mode)
{
    if(mode == GL_SELECT) glLoadName(1);
    glColor3f(1.0, 0.0, 0.0);
    glRectf(-0.5, -0.5, 1.0, 1.0);
    if(mode == GL_SELECT) glLoadName(2);
    glColor3f(0.0, 0.0, 1.0);
    glRectf(-1.0, -1.0, 0.5, 0.5);
}

void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    drawObjects(GL_RENDER);
    glFlush();
}

/*  processHits 函数的功能是把选择数组里的
 *  内容打印出来
 */
void processHits (GLint hits, GLuint buffer[])
{
   unsigned int i, j;
   GLuint ii, jj, names, *ptr;

   printf ("hits = %d\n", hits);
   ptr = (GLuint *) buffer;
   for (i = 0; i < hits; i++)
```

```
   { /* 对于每个命中的对象 */
      names = *ptr;
      ptr+=3;
      for (j = 0; j < names; j++)
      { /* 对于每个对象名称 */
         if(*ptr==1) printf ("red rectangle\n");
         else printf ("blue rectangle\n");
         ptr++;
      }
      printf ("\n");
   }
}

#define SIZE 512

void mouse(int button, int state, int x, int y)
{
   GLuint selectBuf[SIZE];
   GLint hits;
   GLint viewport[4];

   if (button == GLUT_LEFT_BUTTON && state == GLUT_DOWN)
   {
   glGetIntegerv (GL_VIEWPORT, viewport);

   glSelectBuffer (SIZE, selectBuf);
   glRenderMode(GL_SELECT);
   glInitNames();
   glPushName(0);

   glMatrixMode (GL_PROJECTION);
   glPushMatrix ();
   glLoadIdentity ();
/* 在光标附近建立一个大小为5 × 5像素的拾取区域 */
   gluPickMatrix ((GLdouble) x, (GLdouble) (viewport[3] - y),
                  5.0, 5.0, viewport);
   gluOrtho2D (-2.0, 2.0, -2.0, 2.0);
   drawObjects(GL_SELECT);


   glMatrixMode (GL_PROJECTION);
   glPopMatrix ();
   glFlush ();

   hits = glRenderMode (GL_RENDER);
   processHits (hits, selectBuf);

   glutPostRedisplay();
   }
}


void reshape(int w, int h)
{
   glViewport(0, 0, w, h);
```

```
   glMatrixMode(GL_PROJECTION);
   glLoadIdentity();
   gluOrtho2D (-2.0, 2.0, -2.0, 2.0);
   glMatrixMode(GL_MODELVIEW);
   glLoadIdentity();
}

void keyboard(unsigned char key, int x, int y)
{
   switch (key)
   {
      case 27:
         exit(0);
         break;
   }
}

/* 主循环 */
int main(int argc, char** argv)
{
   glutInit(&argc, argv);
   glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);
   glutInitWindowSize (500, 500);
   glutInitWindowPosition (100, 100);
   glutCreateWindow (argv[0]);
   init ();
   glutReshapeFunc (reshape);
   glutDisplayFunc(display);
   glutMouseFunc (mouse);
   glutKeyboardFunc (keyboard);
   glutMainLoop();
   return 0;
}
```

A.8 立方体旋转程序

```
/* 旋转立方体，并对立方体表面进行颜色插值 */

/* 该程序说明如何利用奇次坐标几何变换
   和简单的数据结构来表示第4章的立方体 */

/* 把颜色赋给立方体的顶点*/
/* 立方体的中心位于坐标原点 */

#include <stdlib.h>
#include <GL/glut.h>
 GLfloat vertices[][3] = {{-1.0,-1.0,-1.0},{1.0,-1.0,-1.0},
 {1.0,1.0,-1.0}, {-1.0,1.0,-1.0}, {-1.0,-1.0,1.0},
 {1.0,-1.0,1.0}, {1.0,1.0,1.0}, {-1.0,1.0,1.0}};

 GLfloat colors[][3] = {{0.0,0.0,0.0},{1.0,0.0,0.0},
 {1.0,1.0,0.0}, {0.0,1.0,0.0}, {0.0,0.0,1.0},
 {1.0,0.0,1.0}, {1.0,1.0,1.0}, {0.0,1.0,1.0}};

void polygon(int a, int b, int c, int d)
{
```

```
/* 通过顶点数组绘制一个多边形 */

 glBegin(GL_POLYGON);
 glColor3fv(colors[a]);
 glVertex3fv(vertices[a]);
 glColor3fv(colors[b]);
 glVertex3fv(vertices[b]);
 glColor3fv(colors[c]);
 glVertex3fv(vertices[c]);
 glColor3fv(colors[d]);
 glVertex3fv(vertices[d]);
 glEnd();
}

void colorcube()
{

/* 把顶点与多边形的表面对应起来 */

polygon(0,3,2,1);
polygon(2,3,7,6);
polygon(0,4,7,3);
polygon(1,2,6,5);
polygon(4,5,6,7);
polygon(0,1,5,4);
}

static GLfloat theta[] = {0.0,0.0,0.0};
static GLint axis = 2;

void display()
{

/* 显示回调函数，清空帧缓存和z-buffer里的内容，旋转并绘
   制立方体，交换前端缓存与后端缓存里的内容 */

 glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
 glLoadIdentity();
 glRotatef(theta[0], 1.0, 0.0, 0.0);
 glRotatef(theta[1], 0.0, 1.0, 0.0);
 glRotatef(theta[2], 0.0, 0.0, 1.0);

 colorcube();

 glFlush();
 glutSwapBuffers();
}

void spinCube()
{

/* 空闲回调函数，将立方体绕选择的轴旋转2° */

 theta[axis] += 2.0;
 if(theta[axis] > 360.0 ) theta[axis] -= 360.0;
```

```
 /* display(); */
 glutPostRedisplay();
}

void mouse(int btn, int state, int x, int y)
{

/* 鼠标回调函数，选择立方体的旋转轴 */

  if(btn==GLUT_LEFT_BUTTON && state == GLUT_DOWN) axis = 0;
  if(btn==GLUT_MIDDLE_BUTTON && state == GLUT_DOWN) axis = 1;
  if(btn==GLUT_RIGHT_BUTTON && state == GLUT_DOWN) axis = 2;
}

void myReshape(int w, int h)
{
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    if (w <= h)
        glOrtho(-2.0, 2.0, -2.0 * (GLfloat) h / (GLfloat) w,
            2.0 * (GLfloat) h / (GLfloat) w, -10.0, 10.0);
    else
        glOrtho(-2.0 * (GLfloat) w / (GLfloat) h,
            2.0 * (GLfloat) w / (GLfloat) h, -2.0, 2.0, -10.0, 10.0);
    glMatrixMode(GL_MODELVIEW);
}

int main(int argc, char **argv)
{
    glutInit(&argc, argv);
/* 既需要双缓存又需要 z-buffer */

    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(500, 500);
    glutCreateWindow("colorcube");
    glutReshapeFunc(myReshape);
    glutDisplayFunc(display);
    glutIdleFunc(spinCube);
    glutMouseFunc(mouse);
    glEnable(GL_DEPTH_TEST); /* 开启隐藏面消除功能 */
    glutMainLoop();
}
```

A.9 利用顶点数组旋转立方体的程序

```
/* 旋转立方体，并对立方体表面进行颜色插值 */

/* 该程序说明如何利用奇次坐标几何变换和
   简单的数据结构来表示第 4 章的立方体 */
/* 把颜色赋给立方体的顶点 */
/* 立方体的中心位于坐标原点 */

#include <stdlib.h>
```

```
#include <GL/glut.h>


GLfloat vertices[] = {-1.0,-1.0,-1.0,1.0,-1.0,-1.0,
1.0,1.0,-1.0, -1.0,1.0,-1.0, -1.0,-1.0,1.0,
1.0,-1.0,1.0, 1.0,1.0,1.0, -1.0,1.0,1.0};

GLfloat colors[] = {0.0,0.0,0.0,1.0,0.0,0.0,
1.0,1.0,0.0, 0.0,1.0,0.0, 0.0,0.0,1.0,
1.0,0.0,1.0, 1.0,1.0,1.0, 0.0,1.0,1.0};

GLubyte cubeIndices[]={0,3,2,1,2,3,7,6,0,4,7,3,1,2,6,5,
4,5,6,7,0,1,5,4};


static GLfloat theta[] = {0.0,0.0,0.0};
static GLint axis = 2;

void display()
{
/* 显示回调函数，清空帧缓存和z-buffer里的内容，旋转并绘
   制立方体，交换前端缓存与后端缓存里的内容 */

 glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
 glLoadIdentity();
 glRotatef(theta[0], 1.0, 0.0, 0.0);
 glRotatef(theta[1], 0.0, 1.0, 0.0);
 glRotatef(theta[2], 0.0, 0.0, 1.0);

 glDrawElements(GL_QUADS, 24, GL_UNSIGNED_BYTE, cubeIndices);

 glBegin(GL_LINES);
   glVertex3f(0.0,0.0,0.0);
   glVertex3f(1.0,1.0,1.0);
 glEnd();
 glFlush();
 glutSwapBuffers();
}

void spinCube()
{

/* 空闲回调函数，将立方体绕选择的轴旋转2° */

 theta[axis] += 2.0;
 if( theta[axis] > 360.0 ) theta[axis] -= 360.0;
 glutPostRedisplay();
}

void mouse(int btn, int state, int x, int y)
{

/* 鼠标回调函数，选择立方体的旋转轴 */

 if(btn==GLUT_LEFT_BUTTON && state == GLUT_DOWN) axis = 0;
```

```
 if(btn==GLUT_MIDDLE_BUTTON && state == GLUT_DOWN) axis = 1;
 if(btn==GLUT_RIGHT_BUTTON && state == GLUT_DOWN) axis = 2;
}

void myReshape(int w, int h)
{
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    if (w <= h)
        glOrtho(-2.0, 2.0, -2.0 * (GLfloat) h / (GLfloat) w,
            2.0 * (GLfloat) h / (GLfloat) w, -10.0, 10.0);
    else
        glOrtho(-2.0 * (GLfloat) w / (GLfloat) h,
            2.0 * (GLfloat) w / (GLfloat) h, -2.0, 2.0, -10.0, 10.0);
    glMatrixMode(GL_MODELVIEW);
}

int main(int argc, char **argv)
{

/* 既需要双缓存又需要 z-buffer */

   glutInit(&argc, argv);
   glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
   glutInitWindowSize(500, 500);
   glutCreateWindow("colorcube");
   glutReshapeFunc(myReshape);
   glutDisplayFunc(display);
   glutIdleFunc(spinCube);
   glutMouseFunc(mouse);
   glEnable(GL_DEPTH_TEST); /* 开启隐藏面消除功能 */
   glEnableClientState(GL_COLOR_ARRAY);
   glEnableClientState(GL_NORMAL_ARRAY);
   glEnableClientState(GL_VERTEX_ARRAY);
   glVertexPointer(3, GL_FLOAT, 0, vertices);
   glColorPointer(3,GL_FLOAT, 0, colors);
   glColor3f(1.0,1.0,1.0);
   glutMainLoop();
}
```

A.10 利用虚拟跟踪球实现立方体旋转的程序

```
/* 利用虚拟跟踪球旋转立方体的演示程序 */

#include <math.h>
#include <GL/glut.h>

#define bool int
#define false 0
#define true 1

int  winWidth, winHeight;

float  angle = 0.0, axis[3], trans[3];
bool  trackingMouse = false;
```

```
bool  redrawContinue = false;
bool    trackballMove = false;

/*
** 绘制立方体
*/
GLfloat vertices[][3] = {
    {-1.0,-1.0,-1.0}, {1.0,-1.0,-1.0}, {1.0,1.0,-1.0}, {-1.0,1.0,-1.0},
    {-1.0,-1.0,1.0}, {1.0,-1.0,1.0}, {1.0,1.0,1.0}, {-1.0,1.0,1.0}
};
GLfloat colors[][3] = {
    {0.0,0.0,0.0},{1.0,0.0,0.0}, {1.0,1.0,0.0}, {0.0,1.0,0.0},
    {0.0,0.0,1.0}, {1.0,0.0,1.0}, {1.0,1.0,1.0}, {0.0,1.0,1.0}
};

void polygon(int a, int b, int c , int d, int face)
{

   /* 通过顶点数组绘制一个多边形 */

   glBegin(GL_POLYGON);
     glColor3fv(colors[a]);
     glVertex3fv(vertices[a]);
     glVertex3fv(vertices[b]);
     glVertex3fv(vertices[c]);
     glVertex3fv(vertices[d]);
   glEnd();
}

void colorcube()
{

    /* 把顶点与多边形的表面对应起来 */

    polygon(1,0,3,2,0);
    polygon(3,7,6,2,1);
    polygon(7,3,0,4,2);
    polygon(2,6,5,1,3);
    polygon(4,5,6,7,4);
    polygon(5,4,0,1,5);
}

/*
** 下面这些函数实现一个简单的类似于跟踪球的运动控制
*/

float lastPos[3] = {0.0F, 0.0F, 0.0F};
int curx, cury;
int startX, startY;

void trackball_ptov(int x, int y, int width, int height, float v[3])
{
    float d, a;

    /* 把x，y投影到一个球体的球面上，球体的中心位于width × height的矩形区域内 */
```

```
    v[0] = (2.0F*x - width) / width;
    v[1] = (height - 2.0F*y) / height;
    d = (float) sqrt(v[0]*v[0] + v[1]*v[1]);
    v[2] = (float) cos((M_PI/2.0F) * ((d < 1.0F) ? d : 1.0F));
    a = 1.0F / (float) sqrt(v[0]*v[0] + v[1]*v[1] + v[2]*v[2]);
    v[0] *= a;
    v[1] *= a;
    v[2] *= a;
}

void mouseMotion(int x, int y)
{
    float curPos[3], dx, dy, dz;

    trackball_ptov(x, y, winWidth, winHeight, curPos);
if(trackingMouse)
{
    dx = curPos[0] - lastPos[0];
    dy = curPos[1] - lastPos[1];
    dz = curPos[2] - lastPos[2];

    if (dx || dy || dz) {
    angle = 90.0F * sqrt(dx*dx + dy*dy + dz*dz);

    axis[0] = lastPos[1]*curPos[2] - lastPos[2]*curPos[1];
    axis[1] = lastPos[2]*curPos[0] - lastPos[0]*curPos[2];
    axis[2] = lastPos[0]*curPos[1] - lastPos[1]*curPos[0];

    lastPos[0] = curPos[0];
    lastPos[1] = curPos[1];
    lastPos[2] = curPos[2];
       }
}

    glutPostRedisplay();
}

void startMotion(int x, int y)
{

    trackingMouse = true;
    redrawContinue = false;
    startX = x; startY = y;
    curx = x; cury = y;
    trackball_ptov(x, y, winWidth, winHeight, lastPos);
    trackballMove=true;
}
void stopMotion(int x, int y)
{

    trackingMouse = false;

    if (startX != x || startY != y) {
```

```
    redrawContinue = true;
       }
    else {
      angle = 0.0F;
      redrawContinue = false;
      trackballMove = false;
       }
}

void display()
{
    glClear(GL_COLOR_BUFFER_BIT|GL_DEPTH_BUFFER_BIT);

    /* 观察变换 */

    if (trackballMove) {
     glRotatef(angle, axis[0], axis[1], axis[2]);

    }
 colorcube();
    glutSwapBuffers();
}

void mouseButton(int button, int state, int x, int y)
{
 if(button==GLUT_RIGHT_BUTTON) exit(0);
 if(button==GLUT_LEFT_BUTTON) switch(state)
 {
    case GLUT_DOWN:
    y=winHeight-y;
    startMotion( x,y);
    break;
    case GLUT_UP:
    stopMotion( x,y);
    break;
       }
}

void myReshape(int w, int h)
{
    glViewport(0, 0, w, h);
    winWidth = w;
    winHeight = h;
}

void spinCube()
{
    if (redrawContinue) glutPostRedisplay();
}
int main(int argc, char **argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(500, 500);
    glutCreateWindow("colorcube");
```

```
    glutReshapeFunc(myReshape);
    glutDisplayFunc(display);
    glutIdleFunc(spinCube);
    glutMouseFunc(mouseButton);
    glutMotionFunc(mouseMotion);
    glEnable(GL_DEPTH_TEST);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glOrtho(-2.0, 2.0, -2.0, 2.0, -2.0, 2.0);
    glMatrixMode(GL_MODELVIEW);
    glutMainLoop();
}
```

A.11 通过移动视点实现场景漫游的程序

```
/* 第5章通过移动视点旋转正方形的一个程序 */
/* 立方体的定义和显示类似于立方体旋转程序 */

/* 在显示回调函数中利用 gluLookAt 函数确定视点的位置,
   并可以通过 x, X, y, Y, z, Z 键改变视点的位置
   在 reshape 回调函数中设置透视观察参数 */

#include <stdlib.h>
#include <GL/glut.h>


 GLfloat vertices[][3] = {{-1.0,-1.0,-1.0},{1.0,-1.0,-1.0},
 {1.0,1.0,-1.0}, {-1.0,1.0,-1.0}, {-1.0,-1.0,1.0},
 {1.0,-1.0,1.0}, {1.0,1.0,1.0}, {-1.0,1.0,1.0}};

 GLfloat colors[][3] = {{0.0,0.0,0.0},{1.0,0.0,0.0},
 {1.0,1.0,0.0}, {0.0,1.0,0.0}, {0.0,0.0,1.0},
 {1.0,0.0,1.0}, {1.0,1.0,1.0}, {0.0,1.0,1.0}};

void polygon(int a, int b, int c , int d)
{
 glBegin(GL_POLYGON);
  glColor3fv(colors[a]);
  glVertex3fv(vertices[a]);
  glVertex3fv(vertices[b]);
  glVertex3fv(vertices[c]);
  glVertex3fv(vertices[d]);
 glEnd();
}

void colorcube()
{
 polygon(0,3,2,1);
 polygon(2,3,7,6);
 polygon(0,4,7,3);
 polygon(1,2,6,5);
 polygon(4,5,6,7);
 polygon(0,1,5,4);
}
```

```
static GLfloat theta[] = {0.0,0.0,0.0};
static GLint axis = 2;
static GLdouble viewer[]= {0.0, 0.0, 5.0}; /* 初始化视
                                              点位置*/

void display()
{

 glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

/* 通过模-视变换矩阵更新视点的位置 */

 glLoadIdentity();
 gluLookAt(viewer[0],viewer[1],viewer[2], 0.0, 0.0, 0.0,
                                          0.0, 1.0, 0.0);

/* 旋转立方体 */

 glRotatef(theta[0], 1.0, 0.0, 0.0);
 glRotatef(theta[1], 0.0, 1.0, 0.0);
 glRotatef(theta[2], 0.0, 0.0, 1.0);
 colorcube();

 glFlush();
 glutSwapBuffers();
}

void mouse(int btn, int state, int x, int y)
{
 if(btn==GLUT_LEFT_BUTTON && state == GLUT_DOWN) axis = 0;
 if(btn==GLUT_MIDDLE_BUTTON && state == GLUT_DOWN) axis = 1;
 if(btn==GLUT_RIGHT_BUTTON && state == GLUT_DOWN) axis = 2;
 theta[axis] += 2.0;
 if( theta[axis] > 360.0 ) theta[axis] -= 360.0;
 display();
}

void keys(unsigned char key, int x, int y)
{

/* 利用x, X, y, Y, z, Z键移动视点的位置 */

   if(key == 'x') viewer[0]-= 1.0;
   if(key == 'X') viewer[0]+= 1.0;
   if(key == 'y') viewer[1]-= 1.0;
   if(key == 'Y') viewer[1]+= 1.0;
   if(key == 'z') viewer[2]-= 1.0;
   if(key == 'Z') viewer[2]+= 1.0;
   display();
}

void myReshape(int w, int h)
{
 glViewport(0, 0, w, h);
```

```
/* 使用透视观察 */

 glMatrixMode(GL_PROJECTION);
 glLoadIdentity();
 if(w<=h) glFrustum(-2.0, 2.0, -2.0 * (GLfloat) h/ (GLfloat) w,
       2.0* (GLfloat) h / (GLfloat) w, 2.0, 20.0);
 else glFrustum(-2.0, 2.0, -2.0 * (GLfloat) w/ (GLfloat) h,
       2.0* (GLfloat) w / (GLfloat) h, 2.0, 20.0);

/* 或者使用 gluPerspective 函数 */
 /* gluPerspective(45.0, w/h, 1.0, 10.0); */

 glMatrixMode(GL_MODELVIEW);
}

int main(int argc, char **argv)
{
 glutInit(&argc, argv);
 glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
 glutInitWindowSize(500, 500);
 glutCreateWindow("colorcube");
 glutReshapeFunc(myReshape);
 glutDisplayFunc(display);
 glutMouseFunc(mouse);
 glutKeyboardFunc(keys);
 glEnable(GL_DEPTH_TEST);
 glutMainLoop();
}
```

A.12 球体绘制程序

```
/* 通过递归细分四面体生成球体（第6章）。三
   种显示模式：线框图、均匀着色和插值着色 */

/* 该程序在 init()函数中还定义了材质和光源 */

/* mode 0 为线框图, mode 1 为均匀
   着色，mode 3 为插值着色*/

#include <stdlib.h>
#include <GL/glut.h>


typedef float point[4];

/* 初始化四面体 */

point3 v[]={{0.0, 0.0, 1.0}, {0.0, 0.942809, -0.33333},
   {-0.816497, -0.471405, -0.333333}, {0.816497, -0.471405, -0.333333}};

static GLfloat theta[] = {0.0,0.0,0.0};

int n;
int mode;
```

```
void triangle( point3 a, point3 b, point3 c)

/* 利用GL_LINE_LOOP模式显示一个三角形的线框图，定义一个法
   向量用于均匀着色，定义3个法向量用于插值着色 */
{
    if (mode==0) glBegin(GL_LINE_LOOP);
    else glBegin(GL_POLYGON);
       if(mode==1) glNormal3fv(a);
       if(mode==2) glNormal3fv(a);
       glVertex3fv(a);
       if(mode==2) glNormal3fv(b);
       glVertex3fv(b);
       if(mode==2) glNormal3fv(c);
       glVertex3fv(c);
    glEnd();
}

void normal(point3 p)
{

/* 矢量的归一化 */

    double sqrt();
    float d =0.0;
    int i;
    for(i=0; i<3; i++) d+=p[i]*p[i];
    d=sqrt(d);
    if(d>0.0) for(i=0; i<3; i++) p[i]/=d;
}

void divide_triangle(point3 a, point3 b, point3 c, int m)
{

/* 基于顶点的数目细分三角形
应用右手规则生成对象的外向表面 */

    point3 v1, v2, v3;
    int j;
    if(m>0)
    {
        for(j=0; j<3; j++) v1[j]=a[j]+b[j];
        normal(v1);
        for(j=0; j<3; j++) v2[j]=a[j]+c[j];
        normal(v2);
        for(j=0; j<3; j++) v3[j]=b[j]+c[j];
         normal(v3);
        divide_triangle(a, v1, v2, m-1);
        divide_triangle(c, v2, v3, m-1);
        divide_triangle(b, v3, v1, m-1);
        divide_triangle(v1, v3, v2, m-1);
    }
    else triangle(a,b,c); /* 递归结束时绘制三角形 */
}
```

```
void tetrahedron( int m)
{

/* 对四面体的表面应用三角细分 */

    divide_triangle(v[0], v[1], v[2], m);
    divide_triangle(v[3], v[2], v[1], m);
    divide_triangle(v[0], v[3], v[1], m);
    divide_triangle(v[0], v[2], v[3], m);
}

void display()
{

/* 逐一显示这三种模式下的四面体 */
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();
    mode=0;
    tetrahedron(n);
    mode=1;
    glTranslatef(-2.0, 0.0,0.0);
    tetrahedron(n);
    mode=2;
    glTranslatef( 4.0, 0.0,0.0);
    tetrahedron(n);


    glFlush();
}

void myReshape(int w, int h)
{
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    if (w <= h)
        glOrtho(-4.0, 4.0, -4.0 * (GLfloat) h / (GLfloat) w,
            4.0 * (GLfloat) h / (GLfloat) w, -10.0, 10.0);
    else
        glOrtho(-4.0 * (GLfloat) w / (GLfloat) h,
            4.0 * (GLfloat) w / (GLfloat) h, -4.0, 4.0, -10.0, 10.0);
    glMatrixMode(GL_MODELVIEW);
    display();
}


void myinit()
{
    GLfloat mat_specular[]={1.0, 1.0, 1.0, 1.0};
    GLfloat mat_diffuse[]={1.0, 1.0, 1.0, 1.0};
    GLfloat mat_ambient[]={1.0, 1.0, 1.0, 1.0};
    GLfloat mat_shininess={100.0};
    GLfloat light_ambient[]={0.0, 0.0, 0.0, 1.0};
    GLfloat light_diffuse[]={1.0, 1.0, 1.0, 1.0};
    GLfloat light_specular[]={1.0, 1.0, 1.0, 1.0};
```

```
/* 为光源0分别设置环境光、漫反射光和镜面反射光的参数 */

    glLightfv(GL_LIGHT0, GL_AMBIENT, light_ambient);
    glLightfv(GL_LIGHT0, GL_DIFFUSE, light_diffuse);
    glLightfv(GL_LIGHT0, GL_SPECULAR, light_specular);

/* 为所有多边形的正面定义材质属性 */
    glMaterialfv(GL_FRONT, GL_SPECULAR, mat_specular);
    glMaterialfv(GL_FRONT, GL_AMBIENT, mat_ambient);
    glMaterialfv(GL_FRONT, GL_DIFFUSE, mat_diffuse);
    glMaterialf(GL_FRONT, GL_SHININESS, mat_shininess);

    glShadeModel(GL_SMOOTH); /* 开启平滑着色功能 */
    glEnable(GL_LIGHTING); /* 开启光照功能 */
    glEnable(GL_LIGHT0);  /* 开启光源0 */
    glEnable(GL_DEPTH_TEST); /* 开启z-buffer功能 */
    glClearColor (1.0, 1.0, 1.0, 1.0);
    glColor3f (0.0, 0.0, 0.0);
}

int main(int argc, char **argv)
{
    n=atoi(argv[1]);
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(500, 500);
    glutCreateWindow("sphere");
    myinit();
    glutReshapeFunc(myReshape);
    glutDisplayFunc(display);
    glutMainLoop();
}
```

A.13 Mandelbrot集绘制程序

```
#include <stdio.h>
#include <stdlib.h>
#include <GL/glut.h>

/* 默认的数据 */
/* 可以通过命令行输入其他的值 */

#define CENTERX -0.5
#define CENTERY 0.5
#define HEIGHT 0.5
#define WIDTH 0.5
#define MAX_ITER 100

/* 生成一个N × M的数组 */

#define N 500
#define M 500

float height = HEIGHT; /* 复平面上窗口的大小 */
float width = WIDTH;
```

```
float cx = CENTERX; /* 复平面上窗口的中心 */
float cy = CENTERY;
int max = MAX_ITER; /* 每个点迭代的次数 */

int n=N;
int m=M;

/* 用无符号字节类型（GLubyte）定义图像 */

GLubyte image[N][M];

/* 定义复数数据类型以及复数加法、减法和乘法函数 */
/* 也许其计算开销不值一提 */

typedef float complex[2];

void add(complex a, complex b, complex p)
{
    p[0]=a[0]+b[0];
    p[1]=a[1]+b[1];
}

void mult(complex a, complex b, complex p)
{
    p[0]=a[0]*b[0]-a[1]*b[1];
    p[1]=a[0]*b[1]+a[1]*b[0];
}

float mag2(complex a)
{
    return(a[0]*a[0]+a[1]*a[1]);
}

void form(float a, float b, complex p)
{
    p[0]=a;
    p[1]=b;
}

void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glDrawPixels(n,m,GL_COLOR_INDEX, GL_UNSIGNED_BYTE, image);
}


void myReshape(int w, int h)
{
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    if (w <= h)
    gluOrtho2D(0.0, 0.0, (GLfloat) n, (GLfloat) m* (GLfloat) h /
       (GLfloat) w);
    else
```

```
    gluOrtho2D(0.0, 0.0, (GLfloat) n * (GLfloat) w / (GLfloat)
       h,(GLfloat) m);
    glMatrixMode(GL_MODELVIEW);
    display();
}

void myinit()
{
    float redmap[256], greenmap[256],bluemap[256];
    int i;

    glClearColor (1.0, 1.0, 1.0, 1.0);
    gluOrtho2D(0.0, 0.0, (GLfloat) n, (GLfloat) m);

/* 定义伪彩色的映射关系，当灰度值增加时，红色分量的值增加，而蓝色分量的值减
   少，绿色分量是随机数*/


    for(i=0;i<256;i++)
    {
         redmap[i]=i/255.;
         greenmap[i]=rand()%255;
         bluemap[i]=1.0-i/255.;
    }

    glPixelMapfv(GL_PIXEL_MAP_I_TO_R, 256, redmap);
    glPixelMapfv(GL_PIXEL_MAP_I_TO_G, 256, greenmap);
    glPixelMapfv(GL_PIXEL_MAP_I_TO_B, 256, bluemap);
}


main(int argc, char *argv[])
{
    int i, j, k;
    float x, y, v;
    complex c0, c, d;

    scanf("%f", &cx); /* 中心位置的横坐标 */
    scanf("%f", &cy); /* 中心位置的纵坐标 */
    scanf("%f", &width); /* 矩形的宽度 */
    height=width; /* 矩形的高度 */
    scanf("%d",&max); /* 最大迭代次数 */
    for (i=0; i<n; i++) for(j=0; j<m; j++)
    {

/* 开始点 */

    x= i *(width/(n-1)) + cx -width/2;
    y= j *(height/(m-1)) + cy -height/2;

    form(0,0,c);
    form(x,y,c0);

/* 复数迭代 */
```

```
        for(k=0; k<max; k++)
            {
            mult(c,c,d);
            add(d,c0,c);
            v=mag2(c);
            if(v>4.0) break; /* 这里假定：如果 mag>4，那么不在集合内*/
            }

/* 根据点的亮度赋予灰度级 */
            if(v>1.0) v=1.0; /* 如果大于 1，则截取为 1 */
            image[i][j]=255*v;
        }


        glutInit(&argc, argv);
        glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB );
        glutInitWindowSize(N, M);
        glutCreateWindow("mandlebrot");
        myinit();
        glutReshapeFunc(myReshape);
        glutDisplayFunc(display);

        glutMainLoop();


}
```

A.14 Brensenham 画线算法

```
#define BLACK 0
#include <GL/glut.h>
#include <stdio.h>

void draw_pixel(int ix, int iy, int value)
{
  glBegin(GL_POINTS);
    glVertex2i( ix, iy);
  glEnd();
}

bres(int x1,int y1,int x2,int y2)
{
  int dx, dy, i, e;
  int incx, incy, inc1, inc2;
  int x,y;

  dx = x2 - x1;
  dy = y2 - y1;

  if(dx < 0) dx = -dx;
  if(dy < 0) dy = -dy;
  incx = 1;
  if(x2 < x1) incx = -1;
  incy = 1;
  if(y2 < y1) incy = -1;
```

```
x=x1;
y=y1;

if(dx > dy)
  {
    draw_pixel(x,y, BLACK);
    e = 2*dy - dx;
    inc1 = 2*( dy -dx);
    inc2 = 2*dy;
    for(i = 0; i < dx; i++)
    {
       if(e >= 0)
       {
          y += incy;
          e += inc1;
       }
       else e += inc2;
       x += incx;
       draw_pixel(x,y, BLACK);
    }
 }
 else
 {
    draw_pixel(x,y, BLACK);
    e = 2*dx - dy;
    inc1 = 2*( dx - dy);
    inc2 = 2*dx;
    for(i = 0; i < dy; i++)
    {
       if(e >= 0)
       {
          x += incx;
          e += inc1;
       }
       else e += inc2;
       y += incy;
       draw_pixel(x,y, BLACK);
    }
  }
 }

void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    bres(200, 200, 100, 50);
    glFlush();
}

void myinit()
{
    glClearColor(1.0, 1.0, 1.0, 1.0);
    glColor3f(1.0, 0.0, 0.0);
    glPointSize(1.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
```

```
    gluOrtho2D(0.0, 499.0, 0.0, 499.0);
}

void main(int argc, char** argv)
{

/* 标准的GLUT初始化*/

    glutInit(&argc,argv);
    glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);
        /* 这是默认的显示模式，可以省略该语句 */
    glutInitWindowSize(500,500); /* 大小为500 × 500像素的绘图窗口 */
    glutInitWindowPosition(0,0); /* 把绘图窗口的左上角放在屏幕左上角的位置 */
    glutCreateWindow("Bresenham's Algorithm"); /* 绘图窗口的标题 */
    glutDisplayFunc(display);
        /* 注册显示回调函数，当窗口打开等事件发生时会调用该显示回调函数 */

    myinit(); /* 设置属性 */

    glutMainLoop(); /* 进入事件循环 */
}
```

A.15 带纹理的立方体旋转程序

```
#include <stdlib.h>
#include <GL/glut.h>
GLfloat planes[]= {-1.0, 0.0, 1.0, 0.0};
GLfloat planet[]= {0.0, -1.0,  0.0, 1.0};
GLfloat vertices[][3] = {{-1.0,-1.0,-1.0},{1.0,-1.0,-1.0},
    {1.0,1.0,-1.0}, {-1.0,1.0,-1.0}, {-1.0,-1.0,1.0},
    {1.0,-1.0,1.0}, {1.0,1.0,1.0}, {-1.0,1.0,1.0}};
GLfloat colors[][4] = {{0.0,0.0,0.0,0.5},{1.0,0.0,0.0,0.5},
    {1.0,1.0,0.0,0.5}, {0.0,1.0,0.0,0.5}, {0.0,0.0,1.0,0.5},
    {1.0,0.0,1.0,0.5}, {1.0,1.0,1.0,0.5}, {0.0,1.0,1.0,0.5}};

void polygon(int a, int b, int c, int d)
{
    glBegin(GL_POLYGON);
    glColor4fv(colors[a]);
    glTexCoord2f(0.0,0.0);
    glVertex3fv(vertices[a]);
    glColor4fv(colors[b]);
    glTexCoord2f(0.0,1.0);
    glVertex3fv(vertices[b]);
    glColor4fv(colors[c]);
    glTexCoord2f(1.0,1.0);
    glVertex3fv(vertices[c]);
    glColor4fv(colors[d]);
    glTexCoord2f(1.0,0.0);
    glVertex3fv(vertices[d]);
    glEnd();
}

void colorcube()
{
```

```
/* 把顶点与多边形的表面对应起来 */

    polygon(0,3,2,1);
    polygon(2,3,7,6);
    polygon(0,4,7,3);
    polygon(1,2,6,5);
    polygon(4,5,6,7);
    polygon(0,1,5,4);
}

static GLfloat theta[] = {0.0,0.0,0.0};
static GLint axis = 2;

void display()

{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();
    glRotatef(theta[0], 1.0, 0.0, 0.0);
    glRotatef(theta[1], 0.0, 1.0, 0.0);
    glRotatef(theta[2], 0.0, 0.0, 1.0);
    colorcube();
    glutSwapBuffers();
}

void spinCube()
{
    theta[axis] += 2.0;
    if( theta[axis] > 360.0 ) theta[axis] -= 360.0;
    glutPostRedisplay();
}

void mouse(int btn, int state, int x, int y)
{
    if(btn==GLUT_LEFT_BUTTON && state == GLUT_DOWN) axis = 0;
    if(btn==GLUT_MIDDLE_BUTTON && state == GLUT_DOWN) axis = 1;
    if(btn==GLUT_RIGHT_BUTTON && state == GLUT_DOWN) axis = 2;
}

void myReshape(int w, int h)
{
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    if (w <= h)
        glOrtho(-2.0, 2.0, -2.0 * (GLfloat) h / (GLfloat) w,
            2.0 * (GLfloat) h / (GLfloat) w, -10.0, 10.0);
    else
        glOrtho(-2.0 * (GLfloat) w / (GLfloat) h,
            2.0 * (GLfloat) w / (GLfloat) h, -2.0, 2.0, -10.0, 10.0);
    glMatrixMode(GL_MODELVIEW);
}

void key(unsigned char k, int x, int y)
{
```

```
    if(k == '1') glutIdleFunc(spinCube);
    if(k == '2') glutIdleFunc(NULL);
    if(k == 'q') exit(0);
}

int main(int argc, char **argv)
{
   GLubyte image[64][64][3];
   int i, j, r, c;
   for(i=0;i<64;i++)
   {
     for(j=0;j<64;j++)
     {
       c = ((((i&0x8)==0)^((j&0x8))==0))*255;
       image[i][j][0]= (GLubyte) c;
       image[i][j][1]= (GLubyte) c;
       image[i][j][2]= (GLubyte) c;
     }
   }
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(500, 500);
    glutCreateWindow("colorcube");
    glutReshapeFunc(myReshape);
    glutDisplayFunc(display);
    glutIdleFunc(spinCube);
    glutMouseFunc(mouse);
    glEnable(GL_DEPTH_TEST);
    glEnable(GL_TEXTURE_2D);
    glTexImage2D(GL_TEXTURE_2D,0,3,64,64,0,GL_RGB,GL_UNSIGNED_BYTE,
       image);
    glTexParameterf(GL_TEXTURE_2D,GL_TEXTURE_WRAP_S,GL_REPEAT);
    glTexParameterf(GL_TEXTURE_2D,GL_TEXTURE_WRAP_T,GL_REPEAT);
    glTexParameterf(GL_TEXTURE_2D,GL_TEXTURE_MAG_FILTER,GL_NEAREST);
    glTexParameterf(GL_TEXTURE_2D,GL_TEXTURE_MIN_FILTER,GL_NEAREST);
    glutKeyboardFunc(key);
    glClearColor(1.0,1.0,1.0,1.0);
    glutMainLoop();
}
```

A.16 GLSL 示例程序

```
/* GLSL 演示程序 */

/* 设置着色器使用的平面网格和程序运行的时间参数 */
/* 顶点着色器按正弦方式改变网格的高度 */
/* 使用直通片元着色器（直接通过片元着色器而不改变片元的属性）*/

#include <stdio.h>
#include <stdlib.h>
#include <GLUT/glut.h>

#define N 64 /* 网格分辨率 */

const GLdouble nearVal     = 1.0; /* 视见体的近裁剪平面到视点的距离 */
```

```
const GLdouble farVal   = 20.0; /* 视见体的远裁剪平面到视点的距离 */
GLuint         program  = 0; /* 程序对象标识符 */
GLint          timeParam;

GLchar *ebuffer; /* 用来存储出错信息的缓存*/
GLsizei elength; /* 出错信息的长度 */

GLfloat data[N][N]; /* 用来存储高度数据（高度场）的数组 */

/* 着色器读入函数 */
/* 在文件中创建空终止字符串 */

static char* readShaderSource(const char* shaderFile)
{
    FILE* fp = fopen(shaderFile, "r");
    char* buf;
    long size;

    if(fp==NULL) return NULL;
    fseek(fp, 0L, SEEK_END);
    size = ftell(fp);
    fseek(fp, 0L, SEEK_SET);
    buf = (char*) malloc((size + 1) * sizeof(char));
    fread(buf, 1, size, fp);
    fclose(fp);
    return buf;
}

/* 标准的OpenGL初始化 */

static void init()
{
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
    glColor3f(0.0, 0.0, 0.0);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glOrtho(-0.75,0.75,-0.75,0.75,-5.5,5.5);
}

/* GLSL初始化 */

static void initShader(const GLchar* vShaderFile, const GLchar*
    fShaderFile)
{
    GLint status;
    GLchar *vSource, *fSource;
    GLuint vShader, fShader;

/* 读入着色器文件 */

vSource = readShaderSource(vShaderFile);
if(vSource==NULL)
{
    printf( "Failed to read vertex shaderi\n");
```

```
    exit(EXIT_FAILURE);
}

fSource = readShaderSource(fShaderFile);
if(fSource==NULL)
{
    printf("Failed to read fragment shader\n");
    exit(EXIT_FAILURE);
}

/* 创建程序和着色器对象 */

vShader = glCreateShader(GL_VERTEX_SHADER);
fShader = glCreateShader(GL_FRAGMENT_SHADER);
program = glCreateProgram();

/* 把着色器绑定到程序对象 */

glAttachShader(program, vShader);
glAttachShader(program, fShader);

/* 读入着色器 */

glShaderSource(vShader, 1, (const GLchar**) &vSource, NULL);
glShaderSource(fShader, 1, (const GLchar**) &fSource, NULL);

/* 编译顶点着色器 */

glCompileShader(vShader);

/* 错误检测 */

glGetShaderiv(vShader, GL_COMPILE_STATUS, &status);
if(status==GL_FALSE)
{
   printf("Failed to compile the vertex shader.\n");
   glGetShaderiv(vShader, GL_INFO_LOG_LENGTH, &elength);
   ebuffer = malloc(elength*sizeof(char));
   glGetShaderInfoLog(vShader, elength, NULL, ebuffer);
   printf("%s\n", ebuffer);
   exit(EXIT_FAILURE);
}

    /* 编译片元着色器 */

    glCompileShader(fShader);

    /* 错误检测 */

    glGetShaderiv(fShader, GL_COMPILE_STATUS, &status);
    if(status==GL_FALSE)
    {
       printf("Failed to compile the fragment shader.\n");
       glGetShaderiv(fShader, GL_INFO_LOG_LENGTH, &elength);
       ebuffer = malloc(elength*sizeof(char));
```

```
        glGetShaderInfoLog(fShader, elength, NULL, ebuffer);
        printf("%s\n", ebuffer);
        exit(EXIT_FAILURE);
    }

    /* 链接和错误检测 */

    glLinkProgram(program);
    glGetProgramiv(program, GL_LINK_STATUS, &status);
    if(status==GL_FALSE)
    {
        printf("Failed to link program object.\n");
        glGetProgramiv(program, GL_INFO_LOG_LENGTH, &elength);
        ebuffer = malloc(elength*sizeof(char));
        glGetProgramInfoLog(program, elength, &elength, ebuffer);
        printf("%s\n", ebuffer);
        exit(EXIT_FAILURE);
    }

    /* 使用程序对象 */

    glUseProgram(program);

    /* 设置 uniform 参数 */

    timeParam = glGetUniformLocation(program, "time");
}
```

```
void mesh()
{
    int i,j;
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    gluLookAt(2.0, 2.0, 2.0, 0.5, 0.0, 0.5, 0.0, 1.0, 0.0);
    for(i=0; i<N; i++) for(j=0; j<N;j++)
    {
        glBegin(GL_LINE_LOOP);
            glVertex3f((float)i/N, data[i][j], (float)j/N);
            glVertex3f((float)i/N, data[i][j], (float)(j+1)/N);
            glVertex3f((float)(i+1)/N, data[i][j], (float)(j+1)/N);
            glVertex3f((float)(i+1)/N, data[i][j], (float)(j)/N);
        glEnd();
    }
}

static void draw(void)
{
    /* 把程序运行的时间传送给着色器 */

    glUniform1f(timeParam, glutGet(GLUT_ELAPSED_TIME));
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    mesh();
    glutSwapBuffers();
}
```

```
static void reshape(int w, int h)
{
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glOrtho(-0.75,0.75,-0.75,0.75,-5.5,5.5);

    glViewport(0, 0, w, h);
    glutPostRedisplay();
}

static void keyboard(unsigned char key, int x, int y)
{
    switch (key) {
    case 27:
    case 'Q':
    case 'q':
        exit(EXIT_SUCCESS);
        break;
    default:
        break;
    }
}

void idle()
{
   glUniform1f(timeParam, (GLfloat) glutGet(GLUT_ELAPSED_TIME));
   glutPostRedisplay();
}

int main(int argc, char** argv)
{
    int i,j;

    /* 平面网格 */

    for(i=0;i<N;i++) for(j=0;j<N;j++) data[i][j]=0.0;

    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_RGBA | GLUT_DOUBLE);
    glutInitWindowSize(512, 512);
    glutCreateWindow("Simple GLSL example");
    glutDisplayFunc(draw);
    glutReshapeFunc(reshape);
    glutKeyboardFunc(keyboard);
    glutIdleFunc(idle);

    init();
    initShader("vmesh.glsl", "fPassthrough.glsl");

    glutMainLoop();
}


// vmesh.glsl

// 顶点着色器随着时间和距离按正弦方式改变网格的高度
```

```
uniform float time; /*以毫秒为单位*/

void main()
{
    float s;
    vec4 t = gl_Vertex;
    t.y = 0.1*sin(0.001*time+5.0*gl_Vertex.x)*sin(0.001*time+5.0*
       gl_Vertex.z);
    gl_Position = gl_ModelViewProjectionMatrix * t;
    gl_FrontColor = gl_Color;
}

//fPassThrough.glsl

// 直接通过片元着色器

void main()
{
     gl_FragColor = gl_Color;
}
```

A.17 场景图程序

```
//Scene.cc

#include "Scene.h"

#define BaseRadius 0.2
#define Radius 0.08
#define BaseLen 1
#define UpLen 0.6
#define LowLen 0.6
#define EyeRadius 0.04
#define ChairLegLen 0.55

int main(int argc, char **argv)
{

  float v[][3]={{-0.3,-0.2,0.0},{0.3, -0.2, 0.0},
        {0.3, 0.2, 0.0},{-0.3, 0.2, 0.0} };

  // 光源节点
  Light *Light1=new Light;
  Light *Light2=new Light;
  TurnOff *Off1=new TurnOff(Light1);
  TurnOff *Off2=new TurnOff(Light2);

  // 设置光源的属性值
  Light1->SetValue(POSITION, -2, -3, 1.5, 1);
  Light1->SetValue(SPOT_DIRECTION, 2, 3, -1.5);
  Light1->SetValue(CUTOFFANGLE, 40.0);
  Light1->TurnOn();

  Light2->SetValue(POSITION, 5, 5, 5, 0);
  Light2->SetValue(SPECULAR, 1.0, 1.0, 1.0, 1.0);
```

```
Light2->SetValue(DIFFUSE, 1.0, 1.0, 1.0, 1.0);
Light2->TurnOn();


// 虚拟照相机节点
Camera *Camera1=new Camera(PERSPECTIVE);

Camera1->SetValue(POSITION, 2.2, 0.9, 3);
Camera1->SetValue(AIMAT, 0, 0, 0);
Camera1->SetValue(UPDIRECTION, 0, 1, 0);
Camera1->SetValue(ASPECT, 1);
Camera1->SetValue(NEAR, 1);
Camera1->SetValue(FAR, 20);
Camera1->SetValue(YANGLE, 50);

// 机器人节点
Material *RobotMat=new Material;
Material *EyeMat=new Material;

Cylinder *Base=new Cylinder;
Sphere *Head=new Sphere;
Sphere *EyeL=new Sphere;
Sphere *EyeR=new Sphere;
Cylinder *UpperArmL=new Cylinder;
Cylinder *UpperArmR=new Cylinder;
Cylinder *LowerArmL=new Cylinder;
Cylinder *LowerArmR=new Cylinder;
Cylinder *UpperLegL=new Cylinder;
Cylinder *UpperLegR=new Cylinder;
Cylinder *LowerLegL=new Cylinder;
Cylinder *LowerLegR=new Cylinder;
Polygon *Paper=new Polygon;

Transformation *EyeLTrans=new Transformation;
Transformation *EyeRTrans=new Transformation;
Transformation *HeadTrans=new Transformation;
Transformation *UpArmLTrans=new Transformation;
Transformation *UpArmRTrans=new Transformation;
Transformation *LowArmTrans=new Transformation;
Transformation *UpLegLTrans=new Transformation;
Transformation *UpLegRTrans=new Transformation;
Transformation *LowLegTrans=new Transformation;
Transformation *BaseTrans=new Transformation;

// 设置机器人的属性值
RobotMat->SetValue(DIFFUSE, 0.0, 0.0, 1.0, 1.0);
RobotMat->SetValue(AMBIENT, 0.0, 0.0, 1.0, 1.0);
RobotMat->SetValue(SPECULAR, 1.0, 1.0, 1.0, 1.0);
RobotMat->SetValue(SHININESS, 100.0);

EyeMat->SetValue(DIFFUSE, 1.0, 1.0, 1.0, 1.0);
EyeMat->SetValue(AMBIENT, 1.0, 1.0, 1.0, 1.0);
EyeMat->SetValue(SPECULAR, 1.0, 1.0, 1.0, 1.0);
EyeMat->SetValue(SHININESS, 100.0);
```

```
Base->SetValue(HEIGHT, BaseLen);
Base->SetValue(RADIUS, BaseRadius);
Head->SetValue(RADIUS, BaseRadius);
EyeL->SetValue(RADIUS, EyeRadius);
EyeR->SetValue(RADIUS, EyeRadius);
UpperArmL->SetValue(HEIGHT, UpLen);
UpperArmL->SetValue(RADIUS, Radius);
LowerArmL->SetValue(HEIGHT, LowLen);
LowerArmL->SetValue(RADIUS, Radius);
UpperArmR->SetValue(HEIGHT, UpLen);
UpperArmR->SetValue(RADIUS, Radius);
LowerArmR->SetValue(HEIGHT, LowLen);
LowerArmR->SetValue(RADIUS, Radius);
UpperLegL->SetValue(HEIGHT, UpLen);
UpperLegL->SetValue(RADIUS, Radius);
LowerLegL->SetValue(HEIGHT, LowLen);
LowerLegL->SetValue(RADIUS, Radius);
UpperLegR->SetValue(HEIGHT, UpLen);
UpperLegR->SetValue(RADIUS, Radius);
LowerLegR->SetValue(HEIGHT, LowLen);
LowerLegR->SetValue(RADIUS, Radius);

Paper->SetVerticesv(v, 4);
Paper->SetMaterial(EyeMat);

EyeLTrans->SetValue(TRANSLATION, BaseRadius-EyeRadius/2, 0, 0, 0);
EyeLTrans->SetValue(ROTATION, 30, 0, 0, 1, 1);
EyeRTrans->SetValue(TRANSLATION, BaseRadius-EyeRadius/2, 0, 0, 0);
EyeRTrans->SetValue(ROTATION, -30, 0, 0, 1, 1);
HeadTrans->SetValue(TRANSLATION, 0, 0,
   BaseLen+BaseRadius+BaseRadius/3, 0);
UpArmLTrans->SetValue(TRANSLATION, 0, 0, -UpLen, 0);
UpArmLTrans->SetValue(ROTATION, -45, 0, 1, 0, 1);
UpArmLTrans->SetValue(TRANSLATION, 0, BaseRadius+Radius, BaseLen, 2);
UpArmRTrans->SetValue(TRANSLATION, 0, 0, -UpLen, 0);
UpArmRTrans->SetValue(ROTATION, -45, 0, 1, 0, 1);
UpArmRTrans->SetValue(TRANSLATION, 0, -(BaseRadius+Radius),
   BaseLen, 2);
LowArmTrans->SetValue(TRANSLATION, 0, 0, -UpLen, 0);
LowArmTrans->SetValue(ROTATION, -45, 0, 1, 0, 1);
UpLegLTrans->SetValue(TRANSLATION, 0, 0, -UpLen, 0);
UpLegLTrans->SetValue(ROTATION, -90, 0, 1, 0, 1);
UpLegLTrans->SetValue(TRANSLATION, 0, BaseRadius+Radius, 0, 2);
UpLegRTrans->SetValue(TRANSLATION, 0, 0, -UpLen, 0);
UpLegRTrans->SetValue(ROTATION, -100, 0, 1, 0, 1);
UpLegRTrans->SetValue(TRANSLATION, 0, -(BaseRadius+Radius), 0, 2);
LowLegTrans->SetValue(TRANSLATION, 0, 0, -UpLen, 0);
LowLegTrans->SetValue(ROTATION, 95, 0, 1, 0, 1);
BaseTrans->SetValue(ROTATION, -90, 1, 0, 0, 0);
BaseTrans->SetValue(ROTATION, -10, 0, 0, 1, 1);

Head->SetTransform(HeadTrans);
EyeL->SetTransform(EyeLTrans);
EyeR->SetTransform(EyeRTrans);
EyeR->SetMaterial(EyeMat);
```

```
EyeL->SetMaterial(EyeMat);
UpperArmL->SetTransform(UpArmLTrans);
LowerArmL->SetTransform(LowArmTrans);
UpperArmR->SetTransform(UpArmRTrans);
LowerArmR->SetTransform(LowArmTrans);
UpperLegL->SetTransform(UpLegLTrans);
LowerLegL->SetTransform(LowLegTrans);
UpperLegR->SetTransform(UpLegRTrans);
LowerLegR->SetTransform(LowLegTrans);
Base->SetTransform(BaseTrans);

Paper->SetTransform(ROTATION, 20, 0, 1, 0, 0);
Paper->SetTransform(TRANSLATION, 0.05, -0.15, -0.2, 1);

// 设置机器人各部位的关系
RobotMat->AddChild(Light1);
Light1->AddChild(Base);

Base->AddChild(Off1);
Off1->AddChild(Light2);
Light2->AddChild(Head);
Head->AddChild(Off2);
Head->AddChild(EyeL);
Head->AddChild(EyeR);

Base->AddChild(UpperArmL);
UpperArmL->AddChild(LowerArmL);
Base->AddChild(UpperArmR);
UpperArmR->AddChild(LowerArmR);
Base->AddChild(UpperLegL);
UpperLegL->AddChild(LowerLegL);
Base->AddChild(UpperLegR);
UpperLegR->AddChild(LowerLegR);
LowerArmR->AddChild(Paper);

// 椅子节点
Cube *Seat=new Cube(0.7, 0.06, 0.7);
Line *Leg1=new Line;
Line *Leg2=new Line;
Line *Leg3=new Line;
Line *Leg4=new Line;
DrawStyle *LegStyle=new DrawStyle;
Color *ChairColor=new Color;

// 设置椅子的属性值
float v1[][3]={{-0.3, 0, 0.3},{-0.35, -1*ChairLegLen, 0.3}};
float v2[][3]={{0.3, 0, 0.3},{0.35, -1*ChairLegLen, 0.3}};
float v3[][3]={{0.3, 0, -0.3},{0.35, -1*ChairLegLen, -0.3}};
float v4[][3]={{-0.3, 0, -0.3},{-0.35, -1*ChairLegLen, -0.3}};

ChairColor->SetValue(RED);
Seat->SetTransform(TRANSLATION, 0, -0.15, 0, 0);
```

```
LegStyle->SetValue(LINEWIDTH, 7);
Leg1->SetStyle(LegStyle);
Leg1->SetVerticesv(v1);
Leg2->SetStyle(LegStyle);
Leg2->SetVerticesv(v2);
Leg3->SetStyle(LegStyle);
Leg3->SetVerticesv(v3);
Leg4->SetStyle(LegStyle);
Leg4->SetVerticesv(v4);
Seat->AddChild(Leg1);
Seat->AddChild(Leg2);
Seat->AddChild(Leg3);
Seat->AddChild(Leg4);
ChairColor->AddChild(Seat);

// 机器人和椅子的几何变换节点
Transformation *Trans1=new Transformation;

Trans1->SetValue(TRANSLATION, -0.5, 0, 0, 2);

Trans1->AddChild(ChairColor);
Trans1->AddChild(RobotMat);

// 根节点
Node *Root=new Node;

Root->AddChild(Trans1);
Root->AddChild(Camera1);

// 视点
GLViewer *MyViewer=new GLViewer;

MyViewer->Init(argc, argv);
MyViewer->SetValue(BACKCOLOR, GRAY);
MyViewer->SetValue(BUFFER, DOUBLE);
MyViewer->CreateWin("Working Hard", 500, 500);

GLViewer *MyViewer2=new GLViewer;

MyViewer2->Init(argc, argv);
MyViewer2->SetValue(BACKCOLOR, MAGENTA);
MyViewer2->CreateWin("Working Hard2", 200, 200);
MyViewer2->SetValue(BUFFER, DOUBLE);

MyViewer->Show(Root);
MyViewer2->Show(Root);
return 0;
}
```

A.18 粒子系统程序

```
/* 在一个盒子里生成粒子 */

#define MAX_NUM_PARTICLES 1000
#define INITIAL_NUM_PARTICLES 25
```

```
#define INITIAL_POINT_SIZE 5.0
#define INITIAL_SPEED 1.0

typedef int bool;
#define TRUE 1
#define FALSE 0

#include <stdlib.h>
#include <stdio.h>
#include <GLUT/glut.h>

void myDisplay();
void myIdle();
void myReshape(int, int);
void main_menu(int);
void collision(int);
float forces(int, int);

void myinit();

/* 全局变量 */

int num_particles; /* 粒子的数目 */

/* 粒子结构体*/
typedef struct particle
{
     int color;
     float position[3];
     float velocity[3];
     float mass;
} particle;

particle particles[MAX_NUM_PARTICLES]; /* 粒子系统 */

/* 粒子系统的初始状态 */

int present_time;
int last_time;
int num_particles = INITIAL_NUM_PARTICLES;
float point_size = INITIAL_POINT_SIZE;
float speed = INITIAL_SPEED;
bool gravity = FALSE; /* 不考虑重力 */
bool elastic = FALSE; /* 不考虑弹性恢复 */
float coef = 1.0; /* 完全弹性碰撞 */

GLsizei wh = 500, ww = 500; /* 初始窗口的大小*/

GLfloat colors[8][3]={{0.0, 0.0, 0.0}, {1.0, 0.0, 0.0},{0.0, 1.0, 0.0},
    {0.0, 0.0, 1.0}, {0.0, 1.0, 1.0}, {1.0, 0.0, 1.0}, {1.0, 1.0, 0.0},
    {1.0, 1.0, 1.0}};

/* 每当改变窗口大小或移动窗口时就会调用 reshap 回调函数 */
```

```
void myReshape(int w, int h)
{

/* 调整裁剪视见体 */

        glMatrixMode(GL_PROJECTION);
        glLoadIdentity();
        glOrtho(-2.0, 2.0, -2.0, 2.0, -4.0, 4.0);
        glMatrixMode(GL_MODELVIEW);
        glLoadIdentity();
        gluLookAt(1.5,1.0,1.0,0.0,0.0,0.0,0.0,1.0,0.0);

/* 调整视口并重新绘制 */

        if(w<h) glViewport(0,0,w,w);
        else glViewport(0,0,h,h);


/* 为绘图程序使用的窗口设置大小（全局变量） */

        ww = w;
        wh = h;
}

void myinit()
{
        int  i, j;

/* 设置粒子的随机的位置和速度 */

       for(i=0; i<num_particles; i++)
       {
           particles[i].mass = 1.0;
           particles[i].color = i%8;
           for(j=0; j<3; j++)
           {
               particles[i].position[j] = 2.0*((float)
                  random()/RAND_MAX)-1.0;
               particles[i].velocity[j] = speed*2.0*((float)
                  random()/RAND_MAX)-1.0;
           }
       }
       glPointSize(point_size);

/* 把清屏颜色设置为灰色 */

        glClearColor(0.5, 0.5, 0.5, 1.0);
}

void myIdle()
{
    int i, j;
    float dt;
    present_time = glutGet(GLUT_ELAPSED_TIME);
    dt = 0.001*(present_time -  last_time);
```

```
    for(i=0; i<num_particles; i++)
    {
       for(j=0; j<3; j++)
       {
           particles[i].position[j]+=dt*particles[i].velocity[j];
           particles[i].velocity[j]+=dt*forces(i,j)/particles[i].mass;
       }
        collision(i);
    }
    last_time = present_time;
    glutPostRedisplay();
}

float forces(int i, int j)
{
   float force = 0.0;
   if(gravity&&j==1) force = -1.0; /* 简单的重力 */
   return(force);
}

void collision(int n)

/* 检测粒子与立方体的碰撞并在必要的时候使粒子发生反弹
*/

{
     int i;
     for (i=0; i<3; i++)
     {
           if(particles[n].position[i]>=1.0)
           {
                particles[n].velocity[i] =
                   -coef*particles[n].velocity[i];
                particles[n].position[i] = 1.0
                   -coef*(particles[n].position[i]-1.0);
           }
           if(particles[n].position[i]<=-1.0)
           {
                particles[n].velocity[i] =
                   -coef*particles[n].velocity[i];
                particles[n].position[i] =
                   -1.0-coef*(particles[n].position[i]+1.0);
           }
     }
}

void main_menu(int index)
{
   switch(index)
   {
      case(1):
    {
                num_particles = 2*num_particles;
                myinit();
                break;
        }
```

```
case(2):
{
            num_particles = num_particles/2;
            myinit();
    break;
}
case(3):
{
            speed = 2.0*speed;
            myinit();
    break;
}
    case(4):
    {
            speed = speed/2.0;
            myinit();
            break;
    }
    case(5):
    {
            point_size = 2.0*point_size;
            myinit();
            break;
    }
    case(6):
    {
            point_size = point_size/2.0;
            if(point_size<1.0) point_size = 1.0;
            myinit();
            break;
        }
        case(7):
        {
                gravity = !gravity;
                myinit();
                break;
        }
        case(8):
        {
                elastic = !elastic;
                if(elastic) coef = 0.9;
                  else coef = 1.0;
                myinit();
                break;
        }
    case(9):
    {
        exit(0);
        break;
    }
   }
}

void myDisplay()
{
    int i;
```

```
    glClear(GL_COLOR_BUFFER_BIT);
    glBegin(GL_POINTS); /* 绘制所有的粒子 */
    for(i=0; i<num_particles; i++)
    {
       glColor3fv(colors[particles[i].color]);
       glVertex3fv(particles[i].position);
    }
    glEnd();
    glColor3f(0.0,0.0,0.0);
    glutWireCube(2.2);  /* 绘制一个盒子的线框图 */
    glutSwapBuffers();
}


int main(int argc, char** argv)
{
    glutInit(&argc,argv);
    glutInitDisplayMode (GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(500, 500);
    glutCreateWindow("particle system");
    glutDisplayFunc(myDisplay);
    myinit ();

    glutCreateMenu(main_menu);
    glutAddMenuEntry("more particles", 1);
    glutAddMenuEntry("fewer particles", 2);
    glutAddMenuEntry("faster", 3);
    glutAddMenuEntry("slower", 4);
    glutAddMenuEntry("larger particles", 5);
    glutAddMenuEntry("smaller particles", 6);
    glutAddMenuEntry("toggle gravity",7);
    glutAddMenuEntry("toggle restitution",8);
    glutAddMenuEntry("quit",9);
    glutAttachMenu(GLUT_MIDDLE_BUTTON);

    glutIdleFunc(myIdle);
    glutReshapeFunc (myReshape);
    glutMainLoop();
}
```

A.19 Bezier 曲线绘制程序

```
/*
**  curves.c
**
**  Bezier 曲线绘制程序
**
**
**    q—退出程序
**    c—清屏
**    e—擦除曲线
**    b—绘制 Bezier 曲线
**
*/
```

```
#include <GL/glut.h>

void keyboard(unsigned char key, int x, int y);

#define MAX_CPTS  25   /* 控制点的最大固定数目 */

GLfloat cpts[MAX_CPTS][3];
int ncpts = 0;
static int width = 500, height = 500;   /* 绘图窗口的宽度和高度*/


void drawCurves()
{
    int i;
    for(i=0; i<ncpts-3; i +=3)
    {
        /* 利用OpenGL求值器绘制曲线 */

        glMap1f(GL_MAP1_VERTEX_3, 0.0, 1.0, 3, 4, cpts[i]);
        glMapGrid1f(30, 0.0, 1.0);
        glEvalMesh1(GL_LINE, 0, 30);
    }
    glFlush();
}


void display()
{
    int i;
    glClear(GL_COLOR_BUFFER_BIT);


    glBegin(GL_POINTS);
    for (i = 0; i < ncpts; i++)
        glVertex3fv(cpts[i]);
    glEnd();

    glFlush();
}


void mouse(int button, int state, int x, int y)
{
    float wx, wy;

    if (button == GLUT_LEFT_BUTTON && state == GLUT_DOWN)
    {

       /* 把鼠标的位置逆向变换到世界坐标系 */
       wx = (2.0 * x) / (float)(width - 1) - 1.0;
       wy = (2.0 * (height - 1 - y)) / (float)(height - 1) - 1.0;


       /* 判断是否还可以定义更多的控制点 */
       if (ncpts == MAX_CPTS) return;
```

```
        /* 存储控制点的位置 */
        cpts[ncpts][0] = wx;
        cpts[ncpts][1] = wy;
        cpts[ncpts][2] = 0.0;
        ncpts++;

        /* 绘制控制点 */
        glColor3f(0.0, 0.0, 0.0);
        glPointSize(5.0);
        glBegin(GL_POINTS);
           glVertex3f(wx, wy, 0.0);
        glEnd();
        glFlush();
    }
}


void keyboard(unsigned char key, int x, int y)
{
    switch (key)
    {
        case 'q': case 'Q':
            exit(0);
            break;
        case 'c': case 'C':
            ncpts = 0;
            glutPostRedisplay();
            break;
        case 'e': case 'E':
            glutPostRedisplay();
            break;
        case 'b': case 'B':
            drawCurves();
            break;
    }
}

void reshape(int w, int h)
{
    width = w;
    height = h;

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glOrtho(-1.0, 1.0, -1.0, 1.0, -1.0, 1.0);
    glMatrixMode(GL_MODELVIEW);
    glViewport(0, 0, w, h);
}

int main(int argc, char **argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_RGB);
    glutInitWindowSize(width, height);
```

```
    glutCreateWindow("curves");
    glutDisplayFunc(display);
    glutMouseFunc(mouse);
    glutKeyboardFunc(keyboard);
    glutReshapeFunc(reshape);
    glClearColor(1.0, 1.0, 1.0, 1.0);
    glColor3f(0.0, 0.0, 0.0);
    glPointSize(5.0);
    glEnable(GL_MAP1_VERTEX_3);
    glutMainLoop();
}
```

附录B　空　　间

在计算机图形学中，需要表示和处理像点和线段这样的几何元素，为此要用到的数学知识可以在对各种类型的抽象空间的研究中找到。在附录B中，我们复习三类这样的空间：线性（向量）空间、仿射空间和Euclid空间。向量空间（线性空间）只包含两类对象：标量（例如实数）和向量。仿射空间（affine space）比向量空间增加了一类元素：点。Euclid空间（Euclidean space）又增加了距离的概念。

在计算机图形学中，人们感兴趣的向量是有向线段和用来表示它们的n元数组。附录C讨论处理n元组的工具：矩阵代数。在这个附录中，我们考虑一些基本的概念和规则。也许下面的看法是有帮助的：可以将这些实体（标量、向量、点）看成是抽象数据类型，而公理则定义了这些实体的运算。

B.1　标量

实数及其运算是标量场（scalar field）的一个例子。令S为一个集合，其中的元素$\alpha, \beta, \cdots$称为标量（scalar）。在两个标量之间定义了两种基本的运算，这两种运算通常被称做加法和乘法，并且分别用算符+和·表示[①]。因此，$\forall \alpha, \beta \in S, \alpha + \beta \in S, \alpha \cdot \beta \in S$。这两种运算满足交换律、结合律和分配律，即$\forall \alpha, \beta, \gamma \in S$：

$$\alpha + \beta = \beta + \alpha$$
$$\alpha \cdot \beta = \beta \cdot \alpha$$
$$\alpha + (\beta + \gamma) = (\alpha + \beta) + \gamma$$
$$\alpha \cdot (\beta \cdot \gamma) = (\alpha \cdot \beta) \cdot \gamma$$
$$\alpha \cdot (\beta + \gamma) = (\alpha \cdot \beta) + (\alpha \cdot \gamma)$$

有两个特殊的标量：加法的单位元（0）和乘法的单位元（1）。$\forall \alpha \in S$，这两个单位元满足：

$$\alpha + 0 = 0 + \alpha = \alpha$$
$$\alpha \cdot 1 = 1 \cdot \alpha = \alpha$$

每个元素α都有一个加法逆元素，记为$-\alpha$。每个非0元素α都有一个乘法逆元素，记做$\alpha^{-1} \in S$。这两个逆元素满足：

$$\alpha + (-\alpha) = 0$$
$$\alpha \cdot \alpha^{-1} = 1$$

实数及通常的实数加法和乘法构成了一个标量场，复数（加法和乘法分别是复数的加法和乘法）和有理函数（两个多项式的比）也是如此。

B.2　向量空间

向量空间中除了有标量以外，还包含第二类实体：向量（vector）。关于向量定义了两种运算：向量－向量加法和标量－向量乘法。设u、v、w是向量空间V中的向量。向量－向量加法是封闭的

① 如果不会产生混淆，通常把$\alpha \cdot \beta$写成$\alpha\beta$。

$(u+v \in V, \forall u, v \in V)$，满足交换律$(u+v=v+u)$，也满足结合律$(u+(v+w)=(u+v)+w)$。有一个特殊的向量：零向量（zero vector）0。$\forall u \in V$，有：

$$u+\mathbf{0}=u$$

每个向量u都有一个加法的逆元，记为$-u$，它满足：

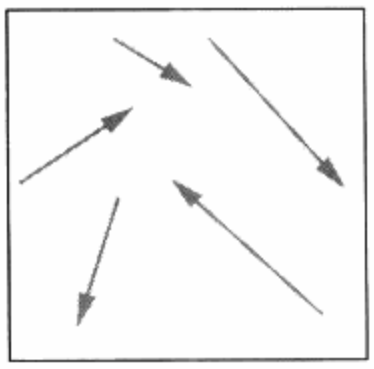
$$u+(-u)=\mathbf{0}$$

标量－向量乘法是这样定义的：对任意的标量α和任意的向量u，αu是V中的一个向量。标量－向量乘法满足分配律，因此

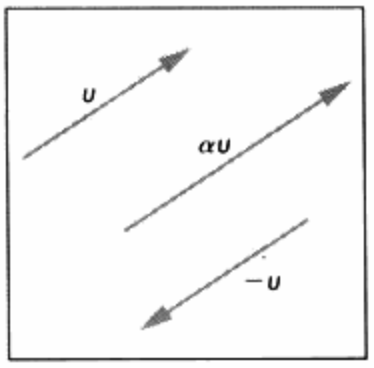
$$\alpha(u+v)=\alpha u+\alpha v$$
$$(\alpha+\beta)u=\alpha u+\beta u$$

我们要用到的两个向量空间的例子是几何向量（有向线段）和实数的n元组。考虑如图B.1所示的有向线段。如果标量是实数，那么标量－向量乘法改变向量的长度，但不改变向量的方向（见图B.2）。

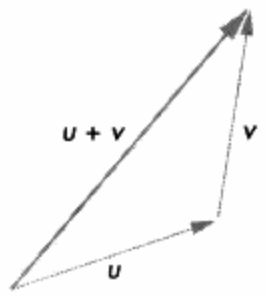
向量－向量加法可以由三角形法则（head-to-tail axiom）来定义，这个规则对于有向线段很容易用图形来表示。如图B.3所示，把u和v的首尾相连，就可以得到向量$u+v$。读者可以证明这样定义的向量－向量加法满足向量场的所有规则。



图B.1　有向线段



图B.2　标量－向量乘法



图B.3　向量加法的三角形法则

向量空间的第二个例子是标量的n元组，通常是实数或者复数的n元组。因此，一个向量可以写成下面的形式：

$$v=(v_1, v_2, \ldots, v_n)$$

标量－向量乘法和向量－向量加法由下面的公式给出：

$$\begin{aligned} u+v &= (u_1, u_2, \ldots, u_n)+(v_1, v_2, \ldots, v_n) \\ &= (u_1+v_1, u_2+v_2, \ldots, u_n+v_n) \\ \alpha v &= (\alpha v_1, \alpha v_2, \ldots, \alpha v_n) \end{aligned}$$

由n元实数组构成的空间记为$\mathbf{R}^n$，在这个向量空间里可以利用矩阵代数来处理向量的运算（见附录C）。

在向量空间中，线性无关和基的概念非常重要。n个向量$u_1, u_2, \cdots, u_n$的线性组合是形如下式的一个向量：

$$u=\alpha_1 u_1+\alpha_2 u_2+\ldots+\alpha_n u_n$$

如果使得

$$\alpha_1 u_1+\alpha_2 u_2 \ldots+\alpha_n u_n=0$$

成立的唯一一组标量是：

$$\alpha_1 = \alpha_2 = \ldots = \alpha_n = 0$$

则称这组向量是线性无关的（linearly independent）。在一个向量空间里，线性无关向量组中向量数目的最大值是这个空间的维数（dimension）。如果一个向量空间的维数是 n，那么任何 n 个线性无关的向量都构成了一个基（basis）。如果 $v_1, v_2, \cdots, v_n$ 是 V 的一个基，那么任何向量 v 都可以由这组基向量唯一地表示：

$$v = \beta_1 v_1 + \beta_2 v_2 + \ldots + \beta_n v_n$$

其中 $\{\beta_i\}$ 是 v 关于基 $v_1, v_2, \cdots, v_n$ 的表示（representation）。如果 $v'_1, v'_2, \ldots, v'_n$ 是另一个基（基中向量的数目是不变的），那么 v 关于这个基也有一个表示，即：

$$v = \beta'_1 v'_1 + \beta'_2 v'_2 + \ldots + \beta'_n v'_n$$

存在一个 $n \times n$ 矩阵 $\mathbf{M}$ 使得

$$\begin{bmatrix} \beta'_1 \\ \beta'_2 \\ \vdots \\ \beta'_N \end{bmatrix} = \mathbf{M} \begin{bmatrix} \beta_1 \\ \beta_2 \\ \vdots \\ \beta_N \end{bmatrix}$$

在附录C中有 $\mathbf{M}$ 的推导过程。这个矩阵给出了一种通过简单的线性变换来改变表示的方法，该方法只涉及矩阵乘法所需的标量运算。更一般地讲，一旦知道了向量空间的一个基，就可以只对表示进行处理。如果标量是实数，那么可以对实数 n 元组进行运算并使用矩阵代数，而无须在原来的抽象向量空间中进行运算。

B.3 仿射空间

在向量空间中，没有像位置和距离这样几何概念。如果把由有向线段构成的向量空间作为考虑几何问题的自然向量空间，那么会遇到困难，因为这些向量就像物理中的向量那样，具有大小和方向但没有位置。图 B.4 中所示的向量都是相同的。

如果用坐标系来考虑问题，就可以把一个向量用一组基向量来表示，这组基向量定义了一个坐标系（coordinate system）。图 B.5(a)中有一个特殊的参考点，即原点（origin），3 个基向量都以原点为起点。然而，因为向量没有位置，所以向量可以位于任意位置，如图 B.5(b)所示。此外，我们没有办法表示这个特殊的点，因为向量空间中只包含向量和标量。

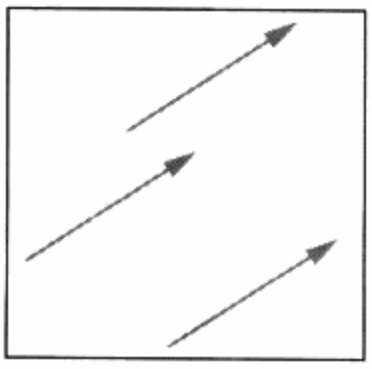


图 B.4 相同的向量

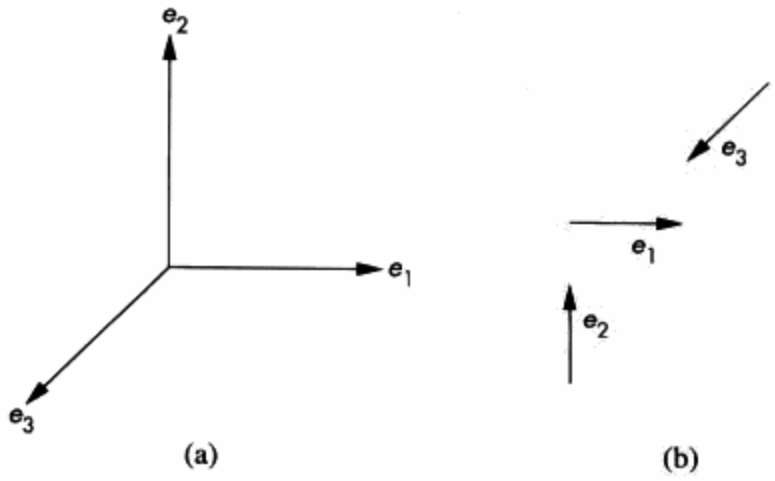


图 B.5 坐标系：(a) 起点位于原点的基向量；(b) 任意位置处的基向量

这个困难可以通过引入仿射空间来解决，仿射空间比向量空间多一类实体——点。设P、Q、R为仿射空间中的点。在仿射空间中定义了一种新的运算——点-点减法（point-point subtraction），这种运算的结果是一个向量。因此，如果P和Q是任意两个点，减法

$$v = P - Q$$

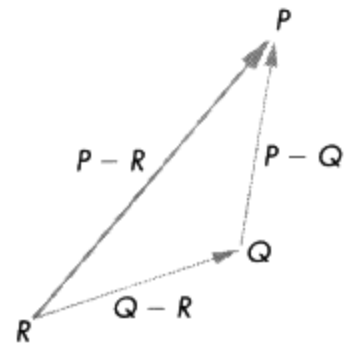
的结果总是V中的一个向量。反之，对任意v和P，总可以找到一个Q使得上面的关系成立。因此可以定义一种向量-点加法：

$$Q = v + P$$

由三角形法则可得，对任意3个点P、Q、R，都有

$$(P - Q) + (Q - R) = (P - R)$$

如果用从点Q到点P的线段来表示向量的大小，并用箭头表示向量的方向，那么三角形法则可以用图B.6来表示。



图B.6　点的三角形法则

仿射空间的许多性质来自仿射几何，也许其中最重要的性质是：如果使用标架而不是坐标系，那么就可以在仿射空间中既表示向量又表示点。一个标架（frame）包括一个点P_0和一组向量v_1，v_2, ⋯, v_n，这组向量定义了向量空间中的一个基。给定一个标架，任意一个向量可以唯一地表示为：

$$v = \alpha_1 v_1 + \alpha_2 v_2 + \cdots + \alpha_n v_n$$

任意一个点可以唯一地表示为：

$$P = P_0 + \beta_1 v_1 + \beta_2 v_2 + \cdots + \beta_n v_n$$

这里的两组标量$\{\alpha_1, \cdots, \alpha_n\}$和$\{\beta_1, \cdots, \beta_n\}$分别给出了向量和点的表示，每个表示包括$n$个标量。可以把点$P_0$看做标架的原点，所有点都是相对于这个参考点定义的。

如果不改变原点，就只需关心和坐标系变换相对应的标架变换。然而，在计算机图形学中，通常需要改变标架，并在不同的标架下表示对象。例如，通常在一个物理标架下定义对象，观察者或者照相机可以在这个标架下表示。但是，作为图像生成过程的一部分，在照相机标架下表示对象的位置是有好处的。照相机标架的原点通常位于投影中心。

B.4　Euclid空间

虽然仿射空间包含了构建几何模型的必要元素，但是在仿射空间中不能度量两个点相距多远，或者说没有向量长度的概念。Euclid空间中有这样的概念。严格地说，Euclid空间只包含向量和标量。

设E是一个Euclid空间，它是一个包含标量$(\alpha, \beta, \gamma, \cdots)$和向量$(u, v, w, \cdots)$的向量空间。假定这里的标量是通常的实数。我们定义一种新的运算——内积（点积），该运算由两个向量得到一个实数。内积必须满足这样的性质：对于任意3个向量u、v、w和标量α、β，都有：

$$u \cdot v = v \cdot u$$
$$(\alpha u + \beta v) \cdot w = \alpha u \cdot w + \beta v \cdot w$$

如果 $v \neq 0$，那么 $v \cdot v > 0$

$$\mathbf{0} \cdot \mathbf{0} = 0$$

如果

$$u \cdot v = 0$$

那么 u 和 v 是正交的（orthogonal）。一个向量的大小（长度）通常定义为：

$$|v| = \sqrt{v \cdot v}$$

一旦把仿射空间中的概念添加到Euclid空间中，例如点很自然就得到了两点之间距离的一种度量，因为对于任意两个点 P 和 Q，$P - Q$ 是一个向量，因此

$$|P - Q| = \sqrt{(P - Q) \cdot (P - Q)}$$

可以利用内积为两个向量之间的夹角定义一种度量：

$$u \cdot v = |u||v| \cos\theta$$

容易证明，按上式定义的 $\cos\theta$ 在 –1 和 1 之间，而且如果两个向量正交，那么 $\cos\theta$ 为 0；如果两个向量平行($u = \alpha v$)，那么 $\cos\theta$ 的绝对值为 1。

B.5　投影

由正交性可以导出一些重要的几何概念。投影（projection）的概念来自这样一个问题：求从点到直线或者平面的最短距离。该问题等价于下面的问题：给定两个向量，把其中一个向量分解成两个分量，使得一个分量平行于另一个向量，另一个分量正交于另一个向量。如果用有向线段来表示向量，那么这个分解如图 B.7 所示。假定 v 是第一个向量，w 是第二个向量，则 w 可以写成：

$$w = \alpha v + u$$

其中，αv 是平行分量，u 是正交分量。为了使 u 和 v 正交，必须有

$$u \cdot v = 0$$

于是

$$w \cdot v = \alpha v \cdot v + u \cdot v = \alpha v \cdot v$$

由此可得：

$$\alpha = \frac{w \cdot v}{v \cdot v}$$

向量 αv 是 w 在 v 上的投影。正交分量 u 为：

$$u = w - \frac{w \cdot v}{v \cdot v} v$$

可以把这个结果推广到从任意一组线性无关向量构造出一组正交向量。

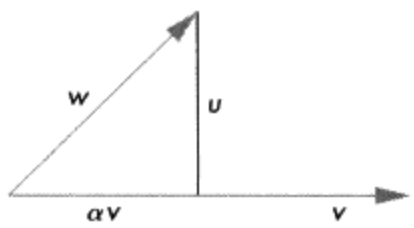


图 B.7　一个向量在另一个向量上的投影

B.6　GRAM-SCHMIDT正交化

给定n维空间中的一组基向量$a_1, a_2, \cdots, a_n$，可以直接构造出另一个基$b_1, b_2, \cdots, b_n$，这个新构造出的基是标准正交的（orthonormal），即其中每个向量的长度都为1，并且两两正交：

$$b_i \cdot b_j = \begin{cases} 1 & \text{如果 } i = j \\ 0 & \text{其他} \end{cases}$$

因此，使用正交坐标系（Cartesian坐标系）并不失一般性。

我们采用递推的方法来构造标准正交基。设

$$b_2 = a_2 + \alpha b_1$$

通过选择一个合适的α可以使得b_2和b_1正交。把上式两边和b_1进行内积，可得：

$$b_2 \cdot b_1 = 0 = a_2 \cdot b_1 + \alpha b_1 \cdot b_1$$

可以求得：

$$\alpha = -\frac{a_2 \cdot b_1}{b_1 \cdot b_1}$$

和

$$b_2 = a_2 - \frac{a_2 \cdot b_1}{b_1 \cdot b_1} b_1$$

这样，通过从a_2中减去平行于b_1的分量，即a_2在b_1上的投影，就得到了两个正交向量b_1和b_2。

一般的递推步骤为找到一个形如下式的向量：

$$b_k = a_k + \sum_{i=1}^{k-1} \alpha_i b_i$$

使得它和$b_1, \cdots, b_{k-1}$正交。由$k-1$个正交条件可以得出：

$$\alpha_i = -\frac{a_k \cdot b_i}{b_i \cdot b_i}$$

我们可以在正交化过程的最后归一化每个向量，即把b_i替换成$b_i/|b_i|$，还可以在刚计算出每个b_i时立刻就归一化，后一种方法的效率更高。

建议阅读资料

现在有许多关于线性代数和向量空间的书籍都非常值得一读。对于计算机图形学研究人员来说，更可取的方法是从向量空间的思想出发，并把线性代数看做是一种处理一般向量空间的工具。遗憾的是，大多数线性代数教科书只考虑由实数n元组构成的Euclid空间$\mathbf{R}^n$。读者可以参考Bowyer和Woodwark的书[Bow83]，以及Banchoff和Werner的书[Ban83]。

仿射空间可以从多个角度引入，Foley的书[Fol90]采用了一种更几何化的方法。

习题

B.1　证明全体复数构成一个标量场。在这个标量场中，加法单位元和乘法单位元分别是什么？

B.2　证明全体有理函数构成一个标量场。

B.3　证明全体实系数的有理函数构成一个向量空间。

B.4　证明基所包含的向量的个数是唯一的。

B.5　考虑 n 个实函数组成的集合 $\{f_i(x)\}$, $i = 1, \cdots, n$。说明如何从该集合构造一个函数的向量空间。求这个空间的维数并写出它的一个基。

B.6　证明次数不超过 n 的全体多项式构成一个 n 维向量空间。

B.7　最重要的Euclid空间是由 n 元组 $(a_1, \cdots, a_n)$ 构成的空间 $\mathbf{R}^n$。定义这个空间中的向量加法和标量 – 向量乘法。$\mathbf{R}^n$ 中的点积运算是什么?

B.8　给定 $\mathbf{R}^3$ 中的 3 个向量，如何判断它们是否构成一个基?

B.9　考虑 $\mathbf{R}^3$ 中的 3 个向量: (1, 0, 0)、(1, 1, 0)和(1, 1, 1)。证明它们是线性无关的。从这些向量导出一个标准正交基，设第一个基向量是(1, 0, 0)。

附录C　矩　　阵

在计算机图形学中，矩阵主要用来表示坐标系变换和标架变换。术语“向量”在向量分析和线性代数中的含义并不完全相同。遗憾的是，在计算机图形学中，这两个领域的知识都要用到，这样对术语“向量”的解释已引起了混淆。为了避免混淆，我们使用术语“行矩阵”和“列矩阵”，而不是线性代数中的术语“行向量”和“列向量”。我们用“向量”表示有向线段，偶尔也像在附录B中那样，用“向量”表示向量空间中的元素，这时把它看成一种抽象数据类型。

这个附录复习在计算机图形学中处理矩阵时会用到的一些主要结果。我们几乎总是使用4 × 4矩阵，因此，线性代数中有关一般矩阵的内容（例如任意方阵的逆）在图形学中用处不大。大多数图形系统直接用硬件或者软件实现4 × 4矩阵的求逆。

C.1　定义

一个矩阵（matrix）是由标量组成的$n \times m$阵列，这个阵列在概念上组织成n行m列。通常，n和m分别称为矩阵的行维数和列维数（dimension）。如果$n = m$，则称这个矩阵是n阶的方阵（square matrix）。我们几乎总是把标量当做实数，尽管这里的大多数结果对于复数也是成立的。矩阵**A**的元素是标量集合$\{a_{ij}\}$，$i = 1, \cdots, n, j = 1, \cdots, m$中的元素。可以利用**A**中的元素把**A**表示为：

$$\mathbf{A} = [\, a_{ij} \,]$$

把矩阵**A**的行和列互换，就得到了一个$m \times n$矩阵，该矩阵称为**A**的转置（transpose），记为$\mathbf{A}^T$，可表示为：

$$\mathbf{A}^T = [\, a_{ji} \,]$$

只有一列的矩阵（$n \times 1$矩阵）称为列矩阵（column matrix），只有一行的矩阵（$1 \times n$矩阵）称为行矩阵（row matrix）。我们用小写字母来表示列矩阵：

$$\mathbf{b} = [\, b_i \,]$$

行矩阵的转置是列矩阵，记为$\mathbf{b}^T$。

C.2　矩阵的运算

矩阵有3种基本的运算：标量–矩阵乘法、矩阵–矩阵加法和矩阵–矩阵乘法。读者可以假定标量是实数，尽管当矩阵的元素和标量属于同一类型时，所有这3种运算都可以按相同的方式来定义。

标量–矩阵乘法（scalar-matrix multiplication）对于任意大小的矩阵都有定义，这种运算简单地把矩阵的每个元素都乘以一个标量α，可以表示为：

$$\alpha\mathbf{A} = [\, \alpha a_{ij} \,]$$

矩阵–矩阵加法（matrix-matrix addition）定义为对两个矩阵的对应元素相加。只有当两个矩阵的尺寸相同时矩阵的和才有意义。两个相同尺寸的矩阵的和由下式给出：

$$\mathbf{C} = \mathbf{A} + \mathbf{B} = [\, a_{ij} + b_{ij} \,]$$

按照矩阵–矩阵乘法（matrix-matrix multiplication）的定义，一个 $n \times l$ 矩阵 **A** 与一个 $l \times m$ 矩阵 **B** 的乘积是下面的 $n \times m$ 矩阵：

$$\mathbf{C} = \mathbf{AB} = [\, c_{ij} \,]$$

其中，

$$c_{ij} = \sum_{k=1}^{l} a_{ik} b_{kj}$$

因此，矩阵–矩阵乘法只有在 **A** 的列数等于 **B** 的行数时才有定义。我们可以把 **AB** 说成 **A** 左乘 **B**，也可以说成 **B** 右乘 **A**。

标量–矩阵乘法具有一些简单的性质。例如，对于任意矩阵 **A** 和任意标量 α，β 都有

$$\alpha(\beta\mathbf{A}) = (\alpha\beta)\mathbf{A}$$
$$\alpha\beta\mathbf{A} = \beta\alpha\mathbf{A}$$

所有这些性质都来自一个事实：标量–矩阵乘法可以归结为对矩阵的标量元素进行的标量乘法。矩阵–矩阵加法满足交换律，对任意 $n \times m$ 矩阵 **A** 和 **B** 都有：

$$\mathbf{A} + \mathbf{B} = \mathbf{B} + \mathbf{A}$$

矩阵加法也满足结合律，对任意 3 个 $n \times m$ 矩阵 **A**、**B** 和 **C** 都有：

$$\mathbf{A} + (\mathbf{B} + \mathbf{C}) = (\mathbf{A} + \mathbf{B}) + \mathbf{C}$$

虽然矩阵–矩阵乘法也满足结合律：

$$\mathbf{A}(\mathbf{BC}) = (\mathbf{AB})\mathbf{C}$$

但几乎总是有 $\mathbf{AB} \neq \mathbf{BA}$，不仅如此，**AB** 和 **BA** 两者中还可能一个有定义而另一个根本没有定义。这些结果表明，当在图形学中用矩阵表示像平移和旋转这样的变换时，执行变换的顺序是重要的。先旋转再平移不等价于先平移再旋转。不过，先旋转然后平移最后缩放，与先旋转然后再执行先平移后缩放的组合变换，二者的效果相同。

单位矩阵 **I** 是这样一个方阵：它的对角线上的元素都是 1，其余的元素都是 0：

$$\mathbf{I} = [\, a_{ij} \,], \quad a_{ij} = \begin{cases} 1 & \text{如果 } i = j \\ 0 & \text{其他} \end{cases}$$

因此有：

$$\mathbf{AI} = \mathbf{A}$$
$$\mathbf{IB} = \mathbf{B}$$

如果其中的矩阵乘法有定义的话。

C.3　行矩阵和列矩阵

人们对 $1 \times n$ 行矩阵和 $n \times 1$ 列矩阵特别感兴趣。三维空间中的一个向量或者一个点[①]在某个标架的表示为一个列矩阵：

$$\mathbf{p} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

① 第 4 章介绍的齐次坐标表示把点的表示和向量的表示区别开来。

我们用小写字母来表示列矩阵。**p** 的转置是行矩阵：

$$\mathbf{p}^T = [x \quad y \quad z]$$

因为 $n \times l$ 矩阵和 $l \times m$ 矩阵的乘积是 $n \times m$ 矩阵，所以 n 阶方阵和 n 维列矩阵的乘积是 n 维列矩阵。我们用二维、三维或者四维列矩阵来表示点（或者向量），用方阵来表示点（或者向量）的变换。于是，下面的式子是变换后的点（或向量）的表示：

$$\mathbf{p}' = \mathbf{Ap}$$

而下面的式子表示多个变换的级联（concatenation）：

$$\mathbf{p}' = \mathbf{ABCp}$$

注意，因为矩阵 - 矩阵乘法满足结合律，所以上式中不需要括号。

许多图形学的书籍用行矩阵来表示点。如果使用行矩阵，那么因为矩阵乘积的转置满足下面的性质：

$$(\mathbf{AB})^T = \mathbf{B}^T\mathbf{A}^T$$

所以 3 个变换的级联可以表示成：

$$\mathbf{p}'^T = \mathbf{p}^T\mathbf{C}^T\mathbf{B}^T\mathbf{A}^T$$

这种形式的优点在于：书写变换的顺序和执行变换的顺序是一样的，都是先 **C**，接着 **B**，最后 **A**。然而，几乎所有的科技、数学和工程文献都使用列矩阵而不是行矩阵，因此本书也使用列矩阵。虽然选择使用列矩阵还是行矩阵在概念上是简单的，但是在实际应用时读者必须要小心API采用的是哪种表示，因为在这两种表示中不仅变换矩阵相乘的顺序是相反的，而且对应的变换矩阵也不相同，它们是互为转置的关系。

C.4　秩

在计算机图形学中，矩阵主要用来表示点和变换。如果一个方阵表示点或者向量的变换，那么人们通常会对这个变换是不是可逆的（invertible）感兴趣。假定

$$\mathbf{q} = \mathbf{Ap}$$

我们想知道能否找到一个方阵 **B**，使得

$$\mathbf{p} = \mathbf{Bq}$$

把其中的 **q** 替换掉，得出：

$$\mathbf{p} = \mathbf{Bq} = \mathbf{BAp} = \mathbf{Ip} = \mathbf{p}$$

于是

$$\mathbf{BA} = \mathbf{I}$$

如果存在这样一个矩阵 **B**，则称它是 **A** 的逆（inverse），并称 **A** 是非奇异的（nonsingular）。我们把不可逆的矩阵称为是奇异的（singular）。**A** 的逆记为 $\mathbf{A}^{-1}$。

关于逆矩阵的一个基本结果为：一个方阵的逆存在，当且仅当它的行列式不等于零。**A** 的行列式记为 $|\mathbf{A}|$，虽然这是一个标量，但除了对于阶数较低的方阵以外，计算行列式和求逆矩阵所需的计算量差不多。对于 n 阶矩阵，这两个计算的复杂度都为 $O(n^3)$。对于计算机图形学中感兴趣的二阶、三阶和四阶矩阵，可以直接计算出它们的行列式和逆矩阵。此外，还可以借助几何推理求逆

矩阵。例如，平移变换的逆是一个反方向的平移变换，因此一个平移矩阵的逆必然还是平移矩阵。第4章讨论了这些内容。

对一般的行维数和列维数不相等的方阵，秩的概念是重要的。我们可以把一个方阵看做是一个行矩阵，其中的每个元素是列矩阵，或者等价地将其看成一个列矩阵，其中的每个元素是行矩阵。按照附录B中向量空间的概念，一个 $n \times m$ 矩阵的行是Euclid空间 $\mathbf{R}^m$ 中的元素，而它的列是 $\mathbf{R}^n$ 中的元素。我们可以确定一个矩阵有多少行（或者列）是线性无关的（linearly independent）。行（列）秩（rank）是线性无关的最大行（列）数。一个 $n \times n$ 矩阵的行秩和列秩相等，并且方阵是非奇异的，当且仅当它的秩等于 n。因此，一个矩阵是可逆的，当且仅当它的行（或列）是线性无关的。

C.5 表示的变换

可以利用矩阵来表示基的变换。假定有一个 n 维向量空间，设 $\{u_1, u_2, \cdots, u_n\}$ 和 $\{v_1, v_2, \cdots, v_n\}$ 是这个向量空间的两个基。于是，一个给定的向量 v 可以表示成：

$$v = \alpha_1 u_1 + \alpha_2 u_2 + \cdots + \alpha_n u_n$$

也可以表示成：

$$v = \beta_1 v_1 + \beta_2 v_2 + \cdots + \beta_n v_n$$

因此，$(\alpha_1, \alpha_2, \cdots, \alpha_n)$ 和 $(\beta_1, \beta_2, \cdots, \beta_n)$ 是 v 的两个不同的表示，这两个表示都可以等价地写成Euclid空间 $\mathbf{R}^n$ 中的向量或者 n 维列矩阵的形式。当处理表示而不是向量时，必须确保所用的记号能够体现出这两者的差异。可以把向量 v 的表示写成：

$$\mathbf{v} = [\,\alpha_1 \quad \alpha_2 \quad \cdots \quad \alpha_n\,]^T$$

还可以写成：

$$\mathbf{v}' = [\,\beta_1 \quad \beta_2 \quad \cdots \quad \beta_n\,]^T$$

这要看采用的是哪个基。

现在考虑如何从表示 $\mathbf{v}$ 变换到表示 $\mathbf{v}'$。基 $\{u_1, u_2, \cdots, u_n\}$ 中的每个向量都可以用基 $\{v_1, v_2, \cdots, v_n\}$ 来表示，因此，存在一组标量 γ_{ij}，使得：

$$u_i = \gamma_{i1} v_1 + \gamma_{i2} v_2 + \cdots + \gamma_{in} v_n, \quad i = 1, \cdots, n$$

可以把所有表示 u_i 的式子写成矩阵形式：

$$\begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{bmatrix} = \mathbf{A} \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}$$

其中，$\mathbf{A}$ 是下面的 $n \times n$ 矩阵：

$$\mathbf{A} = [\,\gamma_{ij}\,]$$

可以利用列矩阵把 $\mathbf{v}$ 用基 $\{u_1, u_2, \cdots, u_n\}$ 表示的式子写成：

$$\mathbf{v} = \mathbf{a}^T \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{bmatrix}$$

其中，

$$\mathbf{a} = [\alpha_i]$$

定义 **b** 为：

$$\mathbf{b} = [\beta_i]$$

于是可以把 **v** 用基$\{v_1, v_2, \cdots, v_n\}$表示的式子写成：

$$\mathbf{v}' = \mathbf{b}^T \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}$$

矩阵 **A** 把这两个基联系起来，因此通过直接的替换可得：

$$\mathbf{b}^T = \mathbf{a}^T \mathbf{A}$$

矩阵 **A** 是这两个基之间的变换的矩阵表示（matrix representation），利用这个矩阵可以在两个基下的表示之间来回切换。换句话说，可以处理由标量组成的矩阵而不是抽象的向量。对于几何问题，虽然向量是有向线段，但可以通过标量的集合来表示它们，并且可以通过对这些标量的操作来表示基的变换或者向量的变换。

C.6　叉积

给定三维空间中两个不平行的向量 u 和 v，它们的叉积是这样一个向量 w，该向量与 u 和 v 都正交。不管在哪个坐标系下表示向量，都有：

$$w \cdot u = w \cdot v = 0$$

因为重要的是 w 的方向而不是它的长度，所以可以再任意指定一个条件，这样就有3个条件来确定 w 的表示中的3个分量。在一个特定的坐标系下，如果 u 的表示的分量是 α_1，α_2，α_3，v 的表示的分量是 β_1，β_2，β_3，那么 u 和 v 的叉积（cross product）在这个坐标系下的表示定义为：

$$\mathbf{w} = \mathbf{u} \times \mathbf{v} = \begin{bmatrix} \alpha_2\beta_3 - \alpha_3\beta_2 \\ \alpha_3\beta_1 - \alpha_1\beta_3 \\ \alpha_1\beta_2 - \alpha_2\beta_1 \end{bmatrix}$$

注意，向量 w 是由 u 和 v 定义的，只有需要计算 w 在一个特定坐标系下的表示时才使用上式。叉积 $u \times v$ 的方向和右手坐标系是一致的[①]。例如，如果把右手坐标系的 x 轴和 y 轴看做两个向量，那么叉积 $x \times y$ 指向 z 轴的正方向。

C.7　特征值和特征向量

方阵是一种算子，它把一个列矩阵变换成相同维数的另一个列矩阵。因为列矩阵可以表示点或者向量，所以人们对这样的问题感兴趣：什么时候点或者向量经过变换后保持不变？例如，对每个旋转矩阵都有一个特别的点：不动点，这个点在变换之后保持不变。考虑一个稍微一般些的问题，即矩阵方程

① 右手坐标系的3个坐标轴的正方向是这样定义的：右手拇指、食指和中指的方向分别对应于 x 轴、y 轴和 z 轴的正方向。与此等价的是：在一张纸上，如果 x 轴的正方向从左向右，y 轴的正方向从下向上，那么 z 轴的正方向垂直纸面向外。

$$\mathbf{Mu} = \lambda\mathbf{u}$$

何时会对某个λ有一个非平凡解（即$\mathbf{u}$不是零矩阵）？如果存在这样的解，那么$\mathbf{M}$把某些向量$\mathbf{u}$变换成自身与一个标量的乘积，这些向量称为这个矩阵的本征向量（eigenvector）。使这个关系成立的标量λ称为这个矩阵的本征值（eigenvalue）。本征值和本征向量也称为特征值（characteristic value）和特征向量（characteristic vector）。特征值刻画了矩阵的许多性质，这些性质在诸如表示变换这样的操作下保持不变。

可以通过求解与上面等价的矩阵方程

$$\mathbf{Mu} - \lambda\mathbf{u} = \mathbf{Mu} - \lambda\mathbf{Iu} = (\mathbf{M} - \lambda\mathbf{I})\mathbf{u} = \mathbf{0}$$

来找出特征值。这个方程有非平凡的解，当且仅当行列式[①]

$$|\mathbf{M} - \lambda\mathbf{I}| = 0$$

如果$\mathbf{M}$的阶数为n，那么这个行列式是λ的n次多项式。这样总共有n个根，其中有些可能是重根或者是复根。对每个不同的根，可以找到与之相对应的特征向量。注意，每个特征向量和一个标量数乘的结果仍然是特征向量，所以可以选择具有单位长度的特征向量。对应于不同特征值的特征向量是线性无关的，因此，如果$\mathbf{M}$有n个相异的特征值，那么对应于这些相异特征值的任何n个特征向量都构成n维空间的一个基。

如果有重根，那么情况更复杂。然而，对于要在图形学中用到的矩阵来说，不需要担心这种情况。如果$\mathbf{R}$是一个3×3旋转矩阵，$\mathbf{p} = [x \ y \ z]^T$是其不动点，那么

$$\mathbf{Rp} = \mathbf{p}$$

因此，每个旋转矩阵必定有一个特征值为1。无论是在三维空间中考虑问题还是使用第4章介绍的四维齐次坐标表示，这个结果都是成立的。

假定$\mathbf{T}$是一个非奇异阵。考虑下面的矩阵：

$$\mathbf{Q} = \mathbf{T}^{-1}\mathbf{MT}$$

它的特征值和特征向量是下面方程的解：

$$\mathbf{Qv} = \mathbf{T}^{-1}\mathbf{MTv} = \lambda\mathbf{v}$$

在上式两边左乘$\mathbf{T}$，得出：

$$\mathbf{MTv} = \lambda\mathbf{Tv}$$

因此，$\mathbf{Q}$的特征值和$\mathbf{M}$的特征值相同，并且$\mathbf{M}$的特征向量是$\mathbf{Q}$的特征向量用$\mathbf{T}$变换的结果。称矩阵$\mathbf{M}$和$\mathbf{Q}$是相似的（similar）。计算机图形学中用到的许多变换都涉及相似矩阵。对这个结果的一个解释是：矩阵的一些基本属性，例如特征值，在坐标系变换后保持不变。如果能够找到一个相似变换把$\mathbf{M}$变换为一个对角阵$\mathbf{Q}$，那么$\mathbf{Q}$的对角线上的元素是这两个矩阵的特征值。

可以从几何的角度来理解特征值和特征向量。考虑一个中心在原点的椭球面，它的对称轴分别平行于3个坐标轴。该椭球面的方程为：

$$\lambda_1 x^2 + \lambda_2 y^2 + \lambda_3 z^2 = 1$$

其中，λ_1、λ_2和λ_3都是正数，这个方程还可以写成矩阵形式：

$$[x \quad y \quad z] \begin{bmatrix} \lambda_1 & 0 & 0 \\ 0 & \lambda_2 & 0 \\ 0 & 0 & \lambda_3 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = 1$$

① 更一般的结果（称为Fredholm二择一定理）为：n个未知数、n个方程的线性方程组$\mathbf{Ax} = \mathbf{b}$有唯一解，当且仅当$|\mathbf{A}| \neq 0$，如果$|\mathbf{A}| = 0$，那么存在多个非平凡解。

因此，λ_1、λ_2和λ_3都是上式中对角阵的特征值，并且分别是椭球面的3个半轴的长度平方的倒数。如果通过旋转矩阵来进行坐标系变换，那么就得到了一个新的椭球面，这个新的椭球面的对称轴不再和3个坐标轴分别平行。然而，我们并没有改变椭球面的3个半轴的长度，这样的属性在坐标系变换后保持不变。

建议阅读资料

关于线性代数和矩阵的标准参考书包括Strang[Str93]、Banchoff和Werner[Ban83]。还可以参考Rogers和Adams的书[Rog90]，以及Graphics Gems系列[Gra90, Gra91, Gra92, Gra94, Gra95]。

在图形学的文献中关于使用行矩阵还是列矩阵的分歧很早就有。早期的图形学书籍[New73]使用了行矩阵。虽然一些书仍然使用行矩阵[Wat00]，但现在的趋势是使用列矩阵[Fol90]。在API内部，可能不清楚它采用的是哪一种表示，因为方阵的元素可以表示成包含n^2个元素的一个简单数组。一些API，例如OpenGL，只允许用户定义的矩阵右乘内部的状态矩阵；而其他API，例如PHIGS，既允许左乘也允许右乘。

习题

C.1 考虑R3中的两个基：{(1, 0, 0), (1, 1, 0), (1, 1, 1)}和{(1, 0, 0), (0, 1, 0), (0, 0, 1)}。求出在这两个基下的表示之间的变换矩阵，证明这两个矩阵互为逆矩阵。

C.2 考虑由次数不超过2的多项式构成的向量空间。证明$\{1, x, x^2\}$和$\{1, 1+x, 1+x+x^2\}$都是这个向量空间的基。请写出多项式$1+2x+3x^2$在这两个基下的表示。求出在这两个基下的表示之间的变换矩阵。

C.3 假定i、j和k分别是直角坐标系的x轴、y轴和z轴正方向的单位向量。证明叉积$u \times v$可由下式给出：

$$u \times v = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ u_1 & u_2 & u_3 \\ v_1 & v_2 & v_3 \end{vmatrix}$$

C.4 证明在$\mathbf{R}^3$中

$$|u \times v| = |u||v|| \sin \theta|$$

其中，θ是u和v的夹角。

C.5 求二维旋转矩阵

$$\mathbf{R} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

的特征向量和特征值。

C.6 求三维旋转矩阵

$$\mathbf{R} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

的特征值和特征向量。

附录 D　OpenGL 函数简介

D.1　定义简单几何对象

```
void glVertex[234][sifd](TYPE xcoordinate, TYPE ycoordinate,...)
void glVertex[234][sifd]v(TYPE *coordinates)
```

定义顶点的二维、三维或四维坐标位置。坐标值的类型可以指定为短整型 s、整型 i、浮点型 f 或双精度浮点型 d。如果使用 v，则参数是一个指向坐标数组的指针。

```
void glBegin(glEnum mode)
```

初始化一个新的图元类型并在其后定义顶点集。图元类型参数 mode 的取值包括 GL_POINTS、GL_LINES 和 GL_POLYGON。

```
void glEnd()
```

结束顶点序列。

```
void glutStrokeCharacter(void* font, int char)
```

在当前光栅位置使用font指定的笔划字体绘制ASCII字符char。字体包括GLUT_STROKE_MONO_ROMAN 和 GLUT_STROKE_ROMAN。

```
void glRectsifd(TYPE x1, TYPE y1, TYPE x2, TYPE y2)
void glRectsifdv(TYPE *v1, TYPE *v2)
```

使用斜对角上的两个顶点（或指向顶点的指针）定义一个与 x、y 坐标轴对齐的二维矩形，其中顶点的坐标值为标准的数据类型。

```
void glutWireTeapot(GLdouble size)
void glutSolidTeapot(GLdouble size)
```

生成一个大小为 size 的且具有法向量和纹理坐标的犹他大学茶壶。

D.2　属性

```
void glPolygonMode(glEnum faces, glEnum mode)
```

设置多边形表面的绘制模式（GL_FRONT, GL_BACK, GL_FRONT_AND_BACK）。参数mode的取值可以为 GL_POINTS、GL_LINES 或 GL_FILL。

```
void glColor3[b i f d ub us ui](TYPE r, TYPE g, TYPE b)
void glColor4[b i f d ub us ui](TYPE r, TYPE g, TYPE b, TYPE a)
void glColor[34][b i f d ub us ui]v(TYPE *color)
```

设置当前的RGB（或RGBA）颜色。有效的类型为byte(b)、int(i)、float(f)、double(d)、unsigned byte(ub)、unsigned short(us)和unsigned int(ui)。浮点类型的最大值和最小值分别为 1.0 和 0.0，而离散类型的最大值和最小值是相应的无符号类型的最大值和最小值，如unsigned byte的最大值和最小值分别为 255 和 0。

```
void glClearColor(GLclampf r, GLclampf g, GLclampf b, GLclampf a)
```

设置当前用来清除颜色缓存的RGBA清屏颜色。GLclampf类型的变量是位于0.0和1.0之间的浮点数。

```
void glIndexi[s i f d ub](TYPE index)
```

设置当前的颜色索引值。

```
void glutSetColor(int index, float r, float g, float b)
```

设置索引值index对应的RGB颜色。

```
void glPointSize(GLfloat size)
```

以像素为单位设置点的大小属性。

```
void glPushAttrib(GLbitfield mask)
void glPopAttrib()
```

把属性值压入属性栈或从属性栈中弹出属性值。属性组（如GL_LINE_BIT和GL_POLYGON_BIT）可以利用逻辑OR形成mask参数值。可以利用GL_ALL_ATTRIBUTE_BITS把所有的属性值压入属性栈。

```
void glPolygonOffset(GLfloat factor, GLfloat units)
```

通过参数factor和参数units的线性组合设置多边形的偏移深度。线性组合计算中的乘法因子（常量）的值取决于多边形的偏移程度和深度值的精度。

D.3　窗口系统函数

```
void glFlush()
```

强制执行缓存中所有的OpenGL命令。

```
void glutInit(int *argc, char **argv)
```

初始化GLUT。该函数的参数来自主函数main的参数，并可供应用程序使用。

```
int glutCreateWindow(char *title)
```

在屏幕上创建一个窗口。字符串title表示窗口的标题。当有多个窗口时，函数的返回值表示窗口的引用值，通过它可以访问对应的窗口。

```
void glutInitDisplayMode(unsigned int mode)
```

利用参数mode的值来初始化显示模式。参数mode的值由不同选项之间的逻辑OR运算得到，这些选项包括颜色模型（GLUT_RGB，GLUT_INDEX）和缓存类型（GLUT_SINGLE，GLUT_DOUBLE）。

```
void glutInitWindowSize(int width, int height)
```

以像素为单位定义绘图窗口的初始高度和宽度。

```
void glutInitWindowPosition(int x, int y)
```

以像素为单位定义绘图窗口左上角的初始位置。

```
void glViewport(int x, int y, GLsizei width, GLsizei height)
```

以像素为单位定义一个大小为width × height的视口，相对于绘图窗口的原点，它的左上角的位置为(x, y)。

```
void glutMainLoop()
```

使程序进入事件处理循环。该函数必须是 main 主函数的最后一条语句。

```
void glutDisplayFunc(void (*func)(void))
```

注册显示回调函数 func，只有当窗口需要重绘时才执行该函数。

```
void glutPostRedisplay()
```

要求当前的回调函数返回时执行显示回调函数。

```
void glutSwapBuffers()
```

交换前端缓存和后端缓存中的内容。

```
void glutSetWindow(int id)
```

把标识符为 id 的窗口设置为当前窗口。

D.4　交互函数

```
void glutMouseFunc(void *f(int button, int state, int x, int y)
```

注册鼠标回调函数 f。该回调函数返回鼠标的按键（GLUT_LEFT_BUTTON，GLUT_MIDDLE_BUTTON，GLUT_RIGHT_BUTTON）、鼠标按键的状态（GLUT_UP，GLUT_DOWN）以及相对于窗口右上角鼠标光标的位置。

```
void glutReshapeFunc(void *f(int width, int height)
```

注册窗口改变大小回调函数 f。该回调函数返回新窗口的高度和宽度。窗口改变大小回调函数调用显示回调函数。

```
void glutKeyboardFunc(void *f(char key, int width, int height)
```

注册键盘回调函数 f。该回调函数返回按键的 ASCII 码和鼠标的位置。

```
void glutIdleFunc(void (*f)(void))
```

注册空闲回调函数 f，当没有其他事件要处理时调用该回调函数。

```
int glutCreateMenu(void (*f)(int value))
```

该函数返回一个顶层菜单的标识符并注册菜单回调函数f,value为传递给该回调函数的数值,它由所选择的菜单项对应的整数值决定。

```
void glutSetMenu(int id)
```

把标识符为 id 的菜单设置为当前菜单。

```
void glutAddMenuEntry(char *name, int value)
```

把名字为 name 的菜单项添加到当前菜单中。当选择该菜单项时，把 value 值传递给菜单回调函数。

```
void glutAttachMenu(int button)
```

把当前窗口的一个鼠标按键与当前菜单关联起来。

```
void glutAddSubMenu(char *name, int menu)
```

把一个名字为 name 的子菜单添加到当前菜单中。menu 值是子菜单创建时返回的标识符。

```
void glutTimerFunc(int delay, void (*f)(int v), int value)
```

注册定时器回调函数f，并延迟事件循环delay毫秒。当指定的时间到达后，执行一个带有参数v的回调函数。其中value是传递给该回调函数f的一个数值。

```
void glutMotionFunc(void (*f)(int x, int y)
```

注册鼠标移动回调函数f。当至少一个鼠标按键按下并移动时，该回调函数返回鼠标光标的位置。

```
void glutPassiveMotionFunc(void (*f)(int x, int y)
```

注册鼠标移动回调函数f。当鼠标移动时，该回调函数返回鼠标光标的位置。

D.5　功能开启函数

```
void glEnable(GLenum feature)
```

开启OpenGL的某个功能。可开启的功能包括GL_DEPTH_TEST、GL_LIGHTING、GL_LIGHTi、GL_TEXTURE_1D、GL_TEXTURE_2D、GL_TEXTURE_3D、GL_LINE_SMOOTH、GL_POLYGON_SMOOTH、GL_POINT_SMOOTH、GL_BLEND、GL_LINE_STIPPLE、GL_POLYGON_STIPPLE、GL_FOG、GL_NORMALIZE。

```
void glDisable(GLenum feature)
```

关闭OpenGL的某个功能。

D.6　几何变换函数

```
void glMatrixMode(GLenum mode)
```

该函数指定哪个矩阵受后续几何变换的影响。参数mode可以为GL_MODELVIEW、GL_PROJECTION或GL_TEXTURE。

```
void glLoadIdentity()
```

把当前变换矩阵设置为单位矩阵。

```
void glPushMatrix()
void glPopMatrix()
```

把当前矩阵模式对应的矩阵压入矩阵栈或从矩阵栈弹出。

```
void glRotate[fd](TYPE angle, TYPE dx, TYPE dy, TYPE dz)
```

把当前矩阵修改为绕某个轴(dx, dy, dz)旋转angle角度的旋转变换矩阵。TYPE要么是GLfloat类型，要么是GLdouble类型。

```
void glTranslate[fd]( TYPE x, TYPE y, TYPE z)
```

把当前矩阵修改为沿x、y、z坐标轴平移量为(x, y, z)的平移变换矩阵。TYPE要么是GLfloat类型，要么是GLdouble类型。

```
void glScale[fd]( TYPE sx, TYPE sy, TYPE sz)
```

把当前矩阵修改为在x、y、z坐标轴方向缩放量为(sx, sy, sz)的缩放变换矩阵。TYPE要么是GLfloat类型，要么是GLdouble类型。

```
void glLoadMatrix[fd]( TYPE *m)
```

把包含16个元素且类型TYPE为GLfloat或GLdouble的数组m装载到当前矩阵中。假定数组m中的元素是按列存储的。

```
void glMultMatrix[fd]( TYPE *m)
```

把包含 16 个元素且类型 TYPE 为 GLfloat 或 GLdouble 的数组 m 装载到一个 4 × 4 的矩阵中，并把该矩阵右乘当前矩阵。假定数组 m 中的元素是按列存储的。

D.7　观察变换函数

```
void glOrtho(GLdouble left, GLdouble right, GLdouble bottom,
   GLdouble top, GLdouble near, GLdouble far)
```

定义一个正投影视见体，函数中所有的参数都相对于投影平面的中心位置。

```
void gluOrtho2D(GLdouble left, GLdouble right, GLdouble bottom,
   GLdouble top)
```

在平面 $z=0$ 上定义一个二维观察矩形。

```
void gluLookAt(GLdouble eyex, GLdouble eyey, GLdouble eyez,
     GLdouble atx, GLdouble aty, GLdouble atz
     GLdouble upx, GLdouble upy, GLdouble upz)
```

用一个视点位于 eye 位置、观察方向点位于 at 位置、向上的观察矢量为 up 的观察变换矩阵右乘当前矩阵。

```
void glFrustum(GLdouble left, GLdouble right, GLdouble bottom,
   GLdouble top, GLdouble near, GLdouble far)
```

定义一个透视投影视见体，函数中所有的参数都相对于观察坐标系的投影中心（原点）。

```
void gluPerspective(GLdouble fov, GLdouble aspect, GLdouble near,
  GLdouble far)
```

定义一个透视投影视见体，该视见体在 y 方向上的视角为 fov（以度为单位），前裁剪面的纵横比为 aspect，投影中心到前裁剪面和后裁剪面的距离分别为 near 和 far。

D.8　定义离散图元

```
void glRasterPos[234]sifd(TYPE xcoord, TYPE ycoord,.....)
void glRasterPos[234]sifdv(TYPE *coordinates)
```

定义当前光栅位置。该函数的参数与函数 glVertex 的参数相同。

```
void glutBitmapCharacter(void* font, int char)
```

在当前光栅位置利用 font 指定的光栅字体绘制 ASCII 字符 char。字体 font 可以为 GLUT_BITMAP_TIMES_ROMAN_10或GLUT_BITMAP_TIMES_ROMAN_8_BY_13。以字符的宽度作为光栅位置的增量。

```
void Bitmap(GLsizei width, GLsizei height, GLfloat x0, GLfloat yo
    GLfloat xi, GLfloat yi, GLubyte *bits)
```

绘制大小为 width × height 位图图像 bits。位图相对于当前光栅位置的偏移量为 xo 和 yo，当位图绘制完成后，光栅位置的增量为 xi 和 yi。

```
glReadPixels(GLint x, GLint y, GLsizei width, GLsizei height,
          GLenum format, GLenum type, GLvoid *image)
```

从当前读缓存的 x、y 位置开始把一个大小为 width × height 的矩形像素块读入到 image 数组中。像素在读缓存中的格式由参数 format 指定，并使用由参数 type 指定的数据类型写入 image 数组中。

```
glDrawPixels(GLsizei width, GLsizei height, GLenum format, GLenum type,
GLvoid *image)
```

根据image数组绘制一个大小为width × height的矩形像素区域。像素使用format格式写入，并使用type类型读取。

```
glCopyPixels(GLint x, GLint y, GLsizei width, GLsizei height,
          GLenum buffer)
```

在由参数buffer指定的缓存中从当前光栅位置把一个大小为width × height的像素块复制到同一缓存的x、y位置。

```
glPixelStore[if](GLenum param, TYPE value)
```

把像素存储参数param设置为value值。参数param包括GL_UNPACK_SWAP_BYTES、GL_PACK_SWAP_BYTES、GL_PACK_ALIGNMENT、GL_UNPACK_ALIGNMENT。

```
glPixelMap[ui us f]v(GLenum map, GLint size, TYPE *array)
```

为指定类型的像素设置一个具有size个元素的像素映射表。参数map的值包括GL_PIXEL_MAP_R_TO_R（或G_TO_G、B_TO_B）和GL_PIXEL_MAP_I_TO_R（或I_TO_G、I_TO_B）。

D.9　显示列表

```
void glNewList(GLunint name, GLenum mode)
```

该函数以参数name为标识符开始建立一个新的显示列表。参数mode的值为GL_COMPILE或GL_COMPILE_AND_EXECUTE，若要使显示列表中函数语句只存入而不执行，则用GL_COMPILE；若要使列表中的函数语句存入表中且按立即模式执行一次，则用GL_COMPILE_AND_EXECUTE。

```
void glEndList()
```

结束显示列表的定义。

```
void glCallList(GLunint name)
```

执行标识符为name的显示列表。

```
void glListBase(GLunint offset)
```

为函数glCallLists设置偏移量。

```
void glCallLists(GLsizei num, GLenum type, GLvoid* lists)
```

利用存储在数组lists中的且类型为type的整数标识符来执行num个显示列表。执行显示列表时，把glListBase设置的偏移量（默认为0）加到每个显示列表对应的标识符上。

```
GLuint glGenLists(GLsizei n)
```

该函数创建n个连续的显示列表标识符，并返回第一个显示列表的标识符。

D.10　拾取

```
GLint glRenderMode(GLenum mode)
```

设置绘制模式（GL_RENDER，GL_SELECTION，GL_FEEDBACK）。当由选择模式（GL_SELECTION）进入常规绘制模式（GL_RENDER）后，该函数的返回值为命中的数量；当由反馈模式（GL_FEEDBACK）进入常规绘制模式（GL_RENDER）后，该函数的返回值为图元的数量。

```
void glSelectBuffer(GLsizei n, GLunint *buff)
```

指定一个用于存储选择数据的大小为 n 的数组 buff。

```
void glInitNames()
```

初始化名称栈。

```
void glPushName(GLuint name)
```

把参数 name 指定的名称压入名称栈中。

```
void glPopName()
```

把栈顶的名称从名称栈中弹出。

```
void glLoadName(GLuint name)
```

用参数 name 指定的名称替换名称栈的栈顶元素。

```
void gluPickMatrix(GLdouble x, GLdouble y, GLdouble w,
   GLdouble h, GLint *vp)
```

该函数为拾取操作创建了一个投影矩阵，它把绘制限制在一个以(x, y)为中心、大小为 w × h 的区域内，而(x, y)是在视口 vp 内的窗口坐标系中的一个坐标点。

D.11　光照

```
void glNormalbsidf(TYPE dx, TYPE dy, TYPE dz)
void glNormalbsidfv(TYPE *v)
```

把当前的法向量设置为(dx, dy, dz)或指向数组 v 的指针。

```
void glLight[if](GLenum light, GLenum param, TYPE value)
void glLight[if]v(GLenum light, GLenum param, TYPE *v)
```

为光源 light 的参数 param 设置标量值或矢量值。

```
void glLightModel[if](GLenum param, TYPE value)
void glLightModel[if]v(GLenum param, TYPE *value)
```

为光照模型 param 设置标量值或矢量值，param 取值为 GL_LIGHT_MODEL_AMBIENT、GL_LIGHT_MODEL_LOCAL_VIEWER、GL_LIGHT_MODEL_TWO_SIDE。

```
void glMaterial[if](GLenum face, GLenum param, TYPE value)
void glMaterial[if]v(GLenum face, GLenum param, TYPE *value)
```

为对象表面 face（GL_FRONT，GL_BACK，GL_FRONT_AND_BACK）的材质参数 param 设置标量值或矢量值。

D.12　纹理映射

```
glTexImage2D[ui us f]v(GLenum target, GLint level, GLint iformat,
    GLsizei width, GLsizei height, GLint border, GLenum format
    GLenum type, GLvoid *texels)
```

设置大小为 width × height 且纹素类型为 type、存储格式为 format 的二维纹理，数组 texels 中每个像素的格式为 iformat，参数 boder 可以指定为 0 或 1。

```
glTexCoord[1234sifd](TYPE scoord, TYPE tcoord,...)
glTexCoord[1234sifd]v(TYPE *vcoord)
```

通过参数scoord、tcoord、…或数组vcoord定义纹理坐标。

```
glTexParameter[if](GLenum target, GLenum param, TYPE value)
glTexParameter[if]v(GLenum target, GLenum param, TYPE *value)
```

为target类型（GL_TEXTURE_1D，GL_TEXTURE_2D或GL_TEXTURE_3D）的纹理参数param设置标量值或矢量值value。

```
gluBuild2DMipmaps(GLenum target, GLint iformat, GLint width,
   GLint height, GLenum format, GLenum type, void *texels)
```

建立并载入一组Mipmap贴图。该函数的参数与函数glTexImage2D的参数相同。

```
glTexEnv[if](GLenum target, GLenum param, TYPE value)
glTexEnv[if]v(GLenum target, GLenum param, TYPE *value)
```

把纹理参数param的值设置为value，参数target必须为GL_TEXTURE_ENV。

```
glTexGen[ifd](GLenum texcoord, GLenum name, TYPE param)
glTexGen[ifd]v(GLenum texcoord, GLenum name, TYPE *param)
```

为纹理坐标参数texcoord（GL_S，GL_T，GL_R或GL_Q）设置自动生成纹理坐标。如果参数name的取值为GL_TEXTURE_GEN_MODE，那么参数param（GL_SPHERE_MAP、GL_REFLECTION_MAP或GL_NORMAL_MAP）用来指定生成纹理坐标值的方法。如果参数name的取值为GL_OBJECT_LINEAR或GL_EYE_LINEAR，那么参数param是一个数组，该数组中的值用来确定一个平面。

```
glGenTextures(GLsizei n, GLuint name)
```

产生n个纹理对象标识符并存储在数组name中，函数返回指向数组第一个元素的指针。

```
glBindTexture(GLenum target, GLuint name)
```

把参数name指定的纹理标识符绑定到参数target（GL_TEXTURE_1D，GL_TEXTURE_2D，GL_TEXTURE_3D，GL_TEXTURE_CUBE_MAP）指定的目标纹理上。

```
glDeleteTextures(GLsizei n, GLuint 8namearray)
```

从数组namearray（数组用来存储纹理对象的名称）中删除n个纹理对象。

D.13　状态和缓存操作函数

```
void glDrawBuffer(GLenum buffer)
```

选择用来绘制的颜色缓存buffer。

```
void glLogicOp(GLenum op)
```

如果启动了GL_COLOR_LOGIC_OP功能，那么该函数用来从16中逻辑写入模式中选择一种。写入模式包括默认的替换模式（GL_COPY）或异或模式（GL_XOR）。

```
glHint(GLenum option, GLenum hint)
```

用参数hint指定的行为控制特定的绘制性能选项option。

```
glDepthMask(GLboolean flag)
```

通过设置参数flag使深度缓存为只读状态（GL_FALSE）或可写状态（GL_TRUE）。

```
glAccum(GLenum operation, GLfloat value)
```

使用常量value定义累积缓存执行的操作（GL_ACCUM，GL_LOAD，GL_RETURN，GL_ADD，GL_MULT）。

D.14　顶点数组

```
void glEnableClientState(GLenum array)
```

启用由参数array指定的顶点数组，可启用的数组包括GL_VERTEX_ARRAY、GL_COLOR_ARRAY、GL_TEXTURE_COORDINATE_ARRAY和GL_NORMAL_ARRAY。

```
void glDisableClientState(GLenum array)
```

关闭由参数array指定的顶点数组。

```
void glVertexPointer(GLint dim, GLenum type. GLsizei stride,
   GLvoid *array)
void glColorPointer(GLint dim, GLenum type. GLsizei stride,
   GLvoid *array)
void glNormalPointer(GLint dim, GLenum type. GLsizei stride,
   GLvoid *array)
void glTexCoordPointer(GLint dim, GLenum type. GLsizei stride,
   GLvoid *array)
```

为顶点数组array定义数据。数据的类型由参数type确定，数组的维数由参数dim确定。参数strike表示连续存储的数据之间的字节数（0表示数据以紧缩的格式存储在数组中）。

```
void glDrawElements(GLenum mode, GLsizei n, GLenum type, void *indices)
```

利用顶点数组中的数据绘制由参数mode指定的OpenGL标准图元类型（如GL_POLYGON或GL_LINES）。参数n表示绘制图元的数量，参数type为索引值的类型（GL_UNSIGNED_BYTE，GL_UNSIGNED_SHORE，GL_UNSIGNED_INT），参数indices为指向索引存储位置的指针。

D.15　融合函数

```
void glBlendFunc(GLenum source, GLenum destination)
```

设置源融合因子和目标融合因子。可选参数包括GL_ONE、GL_ZERO、GL_SRC_COLOR、GL_SRC_ALPHA、GL_ONE_MINUS_SRC_COLOR、GL_ONE_MINUS_SRC_ALPHA、GL_DST_COLOR、GL_ONE_MINUS_DST_COLOR、GL_DST_ALPHA、GL_ONE_MINUS_DST_ALPHA。

```
void glFog[if](GLenum param, TYPE value)
void glFog[if]v(GLenum param, TYPE value)
```

设置雾参数param的值为value。参数param的取值包括GL_FOG_MODE（对应的value取值包括GL_LINEAR、GL_EXP、GL_EXP2）和GL_FOG_COLOR和GL_FOG_DENSITY。

D.16　查询函数

```
void glGetBooleanv(GLenum name, GLboolean *param)
void glGetIntegerv(GLenum name, GLinteger *param)
void glGetFloatv(GLenum name, GLfloat *param)
void glGetDoublev(GLenum name, GLdouble *param)
void glGetPointerv(GLenum name, GLvoid **param)
```

把参数name对应的当前值写入param数组中。

```
int glutGet(GLenum state)
```

返回某个GLUT状态变量的当前值，如GLUT_WINDOW_WIDTH、GLUT_WINDOW_HEIGHT、GLUT_ELAPSED_TIME。

D.17　曲线与曲面函数

```
void glMap1[fd](GLenum entity, TYPE u0, TYPE u1, GLint stride,
    GLint order, TYPE *data)
void glMap2[fd](GLenum entity, TYPE u0, TYPE u1, GLint ustride,
    GLint uorder, TYPE v0, TYPE v1, GLint vstride, GLint vorder,
        TYPE *data)
```

在(u0, u1)和(v0, v1)范围内利用order - 1次Berstein多项式为entity设置一维或二维求值器。参数entity的取值包括GL_MAP{12}_VERTEX_{234}、GL_MAP{12}_NORMAL、GL_MAP{12}_COLOR_{34}。参数TYPE表示数组data的数据类型，参数stride表示一段曲线跨度上连续控制点的数量。

```
void glEvalCoord1[fd](TYPE u)
void glEvalCoord1[fd]v(TYPE u)
void glEvalCoord2[fd](TYPE u, TYPE v)
void glEvalCoord2[fd]v(TYPE u, TYPE v)
```

计算参数u和v对应的求值器的值。

```
void glMapGrid1fd(GLint n, TYPE u0, TYPE u1)
void glMapGrid2fd(GLint n, TYPE u0, TYPE u1, GLint m, TYPE v0, TYPE v1)
```

在u0和u1之间分n步设置等间隔的空间网格（以及在v0和v1之间分m步设置等间隔的空间网格）。

```
void glEvalMesh1(GLenum mode, GLint first, GLint last)
void glEvalMesh2(GLenum mode, GLint ufirst, GLint ulast, GLint vfirst,
GLint vlast)
```

按参数mode指定的模式（GL_LINE，GL_POINT，GL_FILL）在first到last区间绘制所有已开启的求值器。

D.18　GLU二次曲线函数

```
GLUquadricObj* gluNewQuadric()
```

新建一个二次曲面对象并返回其指针。

```
void gluQuadricDrawStyle(GLUquadricObj *obj, GLenum style)
```

为二次曲线对象obj设置绘制类型（GLU_POINT，GLU_LINE，GLU_FILL，GLU_SILHOUETTE）。

```
void gluQuadricNormals(GLUquadricObj *obj, GLenum mode)
void gluQuadricTexture(GLUquadricObj *obj, GLboolean mode)
```

为二次曲线对象obj设置自动生成法向量（GLU_NONE，GLU_FLAT，GLU_SMOOTH）或自动生成纹理坐标（GLU_TRUE，GLU_FALSE）的模式。

```
void gluSphere(GLUquadricObj *obj, GLdouble radius, GLint slices, GLint
stacks)
```

利用slices条经线和stacks条纬线把一个二次曲面对象定义为半径为radius的球体。

```
void gluCylinder(GLUquadricObj *obj, GLdouble base, GLdouble top,
        GLdouble height, GLint slices, GLint stacks)
```

在 z 轴方向进行 slices 次细分，并绕圆柱体进行 stacks 次细分把一个二次曲面对象定义为一个中心位于原点的圆柱体，该圆柱体沿 z 方向的高度为 height，底面半径为 base，顶面半径为 top。

D.19　GLSL 函数

```
GLuint glCreateProgram()
```

创建一个空程序对象并返回它的标识符。

```
GLuint glCreateShader(GLenum type)
```

创建一个类型为GL_VERTEX_SHADER或GL_FRAGMENT_SHADER的空着色器对象并返回它的标识符。

```
void glShaderSource(GLuint shader, GLsizei nstrings, const GLchar
     **strings, const GLint *lengths)
```

使用 nstrings 个长度为 lengths 的字符数组 strings 为着色器 shader 指定源代码。如果着色器是单个以 null 结尾的字符串，那么 nstrings 为 1，lengths 为 NULL。

```
void glCompileShader(GLuint shader)
```

编译由参数 shader 指定的着色器对象。

```
void glAttachShader(GLunit program, GLuint shader)
```

把由参数 shader 指定的着色器对象绑定到程序对象 program 上。

```
void glLinkProgram(GLuint program)
```

把应用程序和程序对象 program 中的着色器对象连接成一个可执行文件。

```
GLint glGetAttribLocation(GLuint program, const GLchar *name)
```

从连接程序对象 program 中返回属性 name 的索引。

```
void glVertexAttrib[1234][sfd](GLunint index, TYPE value1,
    TYPE value2,...)
void glVertexAttrib[123][sfd]v(GLunint index, TYPE *value)
```

用参数 index 指定的索引设置顶点的属性值 value。

```
GLint glGetUniformLocation(GLuint program, const GLchar *name)
```

从连接程序对象 program 中返回 uniform 变量 name 的索引。

```
void glUniform1234[if](GLint index, TYPE value)
void glUniform1234[if]v(GLint index, GLsizei num, TYPE value)
void glUniformMatrix[234]f(GLint index, GLsizei num,
     GLboolean transpose, const GLfloat *value)
```

修改由参数 index 指定的 uniform 变量、数组或矩阵的值。对于数组和矩阵，num 表示要修改元素的个数。

```
void glGetProgram(GLunint program, GLenum pname, GLinit *param)
```

通过参数 params 返回程序对象 program 中对象参数 pname 的值。对象参数 pname 包括连接状态 GL_LINK_STATUS（返回值为 GL_TRUE 或 GL_FALSE）和 GL_INFO_LOG_LENGTH（返回值为用于存储信息日志的字符缓存的大小）。

```
void glGetShaderiv(GLuint shader, GLenum pname, GLint *param)
```

通过参数params返回着色器对象shader中对象参数pname的值。对象参数pname包括编译状态GL_COMPILE_STATUS（返回值为GL_TRUE或GL_FALSE）和GL_INFO_LOG_LENGTH（返回值为用于存储信息日志的字符缓存的大小）。

```
void getProgramInfoLog(GLunint program, GLsizei maxL, GLsizei *len,
    GLchar *infoLog)
```

通过大小为maxL的字符数组infoLog返回程序对象program的信息日志，并通过参数len返回字符数组infoLog中的字符串长度。

```
void getShaderInfoLog(GLunint program, GLsizei maxL, GLsizei *len,
    GLchar *infoLog)
```

通过大小为maxL的字符数组infoLog返回着色器对象shader的信息日志，并通过参数len返回字符数组infoLog中的字符串长度。

参考文献

Ado85 Adobe Systems Incorporated, *PostScript Language Reference Manual*, Addison-Wesley, Reading, MA, 1985.

Ake88 Akeley, K., and T. Jermoluk, "High Performance Polygon Rendering," *Computer Graphics*, 22(4), 239–246, 1988.

Ake93 Akeley, K., "Reality Engine Graphics," *Computer Graphics*, 109–116, 1993.

Ang90 Angel, E., *Computer Graphics*, Addison-Wesley, Reading, MA, 1990.

Ang08 Angel, E., *OpenGL, A Primer,* Third Edition, Addison-Wesley, Reading, MA, 2008.

ANSI85 American National Standards Institute (ANSI), *American National Standard for Information Processing Systems—Computer Graphics—Graphical Kernel System (GKS) Functional Description,* ANSI, X3.124-1985, ANSI, New York, 1985.

ANSI88 American National Standards Institute (ANSI), *American National Standard for Information Processing Systems—Programmer's Hierarchical Interactive Graphics System (PHIGS),* ANSI, X3.144-1988, ANSI, New York, 1988.

App68 Appel, A., "Some Techniques for Shading Machine Renderings of Solids," *Spring Joint Computer Conference,* 37–45, 1968.

Arn96 Arnold, K., and J. Gosling, *The Java Programming Language,* Addison-Wesley, Reading, MA, 1996.

Ban83 Banchoff, T., and J. Werner, *Linear Algebra Through Geometry,* Springer-Verlag, New York, 1983.

Bar93 Barnsley, M., *Fractals Everywhere,* Second Edition, Academic Press, San Diego, CA, 1993.

Bar83 Barsky, B.A., and C. Beatty, "Local Control of Bias and Tension in Beta-Splines," *ACM Transactions on Graphics,* 2(2), 109–134, 1983.

Bar87 Bartels, R.H., C. Beatty, and B.A. Barsky, *An Introduction to Splines for Use in Computer Graphics and Geometric Modeling*, Morgan Kaufmann, Los Altos, CA, 1987.

Bli76 Blinn, J.F., and M.E. Newell, "Texture and Reflection in Computer Generated Images," *CACM,* 19(10), 542–547, 1976.

Bli77 Blinn, J.F., "Models of Light Reflection for Computer-Synthesized Pictures," *Computer Graphics,* 11(2), 192–198, 1977.

Bow83 Bowyer, A., and J. Woodwark, *A Programmer's Geometry*, Butterworth, London, 1983.

Bre65 Bresenham, J.E., "Algorithm for Computer Control of a Digital Plotter," *IBM Systems Journal*, January, 25–30, 1965.

Bre87 Bresenham, J.E., "Ambiguities in Incremental Line Rastering," *IEEE Computer Graphics and Applications*, May, 31–43, 1987.

Car78 Carlbom, I., and J. Paciorek, "Planar Geometric Projection and Viewing Transformations," *Computing Surveys*, 10(4), 465–502, 1978.

Cas96 Castleman, K.C., *Digital Image Processing,* Prentice-Hall, Englewood Cliffs, NJ, 1996.

Cat75 Catmull, E., "A Hidden-Surface Algorithm with Antialiasing," *Computer Graphics*, 12(3), 6–11, 1975.

Cat78 Catmull, E., and J. Clark, "Recursively Generated B-Spline Surfaces on Arbitrary Topological Meshes," *Proceedings of Computer-Aided Design 10*, 350-355, 1978.

Cha98 Chan, P., and R. Lee, *The Java Class Libraries: Java.Applet, Java.Awt, Java.Beans* (Vol. 2), Addison-Wesley, Reading, MA, 1998.

Che95 Chen, S.E., "Quicktime VR: An Image-Based Approach to Virtual Environment Navigation," *Computer Graphics*, 29–38, 1995.

Che00 Chen, K.L. et al., "Building and Using a Scalable Display Wall System," *IEEE Computer Graphics and Applications*, 20(4), 29–37, 2000.

Cla82 Clark, J.E., "The Geometry Engine: A VLSI Geometry System for Graphics," *Computer Graphics*, 16, 127–133, 1982.

Coh85 Cohen, M.F., and D.P. Greenberg, "The Hemi-Cube: A Radiosity Solution for Complex Environments," *Computer Graphics,* 19(3), 31–40, 1985.

Coh88 Cohen, M.F., S.E. Chen, J.R.Wallace, and D.P. Greenberg, "A Progressive Refinement Approach to Fast Radiosity Image Generation," *Computer Graphics,* 22(4), 75–84, 1988.

Coh93 Cohen, M.F., and J.R. Wallace, *Radiosity and Realistic Image Synthesis*, Academic Press Professional, Boston, MA, 1993.

Coo82 Cook, R.L., and K.E. Torrance, "A Reflectance Model for Computer Graphics," *ACM Transactions on Graphics*, 1(1), 7–24, 1982.

Coo87 Cook, R.L., L. Carpenter, and E. Catmull, "The Reyes Image Rendering Architecture," *Computer Graphics*, 21(4), 95–102, July 1987.

Cro81 Crow, F.C., "A Comparison of Antialiasing Techniques," *IEEE Computer Graphics and Applications,* 1(1), 40–48, 1981.

Cro97 Crossno, P.J., and E. Angel, "Isosurface Extraction Using Particle Systems," *IEEE Visualization*, 1997.

Deb96 Debevec, P.E., C.J. Taylor, and J. Malik, "Modeling and Rendering Architecture from Photographs: A Hybrid Geometry- and Image-Based Approach," *Computer Graphics*, 11–20, 1996.

DeR88 DeRose, T.D., "A Coordinate-Free Approach to Geometric Programming," SIGGRAPH Course Notes, *SIGGRAPH*, 1988.

DeR89 DeRose, T.D., "A Coordinate-Free Approach to Geometric Programming," *Theory and Practice of Geometric Modeling*, W. Strasser and H.P. Seidel (Eds.), Springer-Verlag, Berlin, 1989.

Dre88 Drebin, R.A., L. Carpenter, and P. Hanrahan, "Volume Rendering," *Computer Graphics*, 22(4), 65–74, 1988.

Ebe01 Eberly, D.H., *3D Game Engine Design,* Morgan Kaufmann, San Francisco, CA, 2001.

Ebe02 Ebert, D., F.K. Musgrave, D. Peachey, K. Perlin, and S. Worley, *Texturing and Modeling, A Procedural Approach*, Third Edition, Morgan Kaufman, San Francisco, 2002.

End84 Enderle, G., K. Kansy, and G. Pfaff, *Computer Graphics Programming: GKS—The Graphics Standard*, Springer-Verlag, Berlin, 1984.

Eng68 Engelbart, D.C., and W.K. English, "A Research Center for Augmenting Human Intellect," *Fall Joint Computer Conference,* Thompson Books, Washington, DC, 1968.

Far88 Farin, G., *Curves and Surfaces for Computer Aided Geometric Design,* Academic Press, New York, 1988.

Fau80 Faux, I.D., and M.J. Pratt, *Computational Geometry for Design and Manufacturing*, Halsted, Chichester, England, 1980.

Fern03 Fernando, R., and M.J. Kilgard, *The Cg Tutorial: The Definitive Guide to Programmable Real-Time Graphics*, Addison-Wesley, Reading, MA, 2003.

Fern04 Fernando, R., *GPU Gems: Programming Techniques, Tips, and Tricks for Real-Time Graphics*, Addison-Wesley, Reading, MA, 2004.

Fol90 Foley, J.D., A. van Dam, S.K. Feiner, and J.F. Hughes, *Computer Graphics,* Second Edition, Addison-Wesley, Reading, MA, 1990 (C Version 1996).

Fol94 Foley, J.D., A. van Dam, S.K. Feiner, J. F. Hughes, and R. Phillips, *Introduction to Computer Graphics,* Addison-Wesley, Reading, MA, 1994.

Fos96 Fosner, R., *OpenGL Programming for Windows 95 and Windows NT,* Addison-Wesley, Reading, MA, 1996.

Fou82 Fournier, A., D. Fussell, and L. Carpenter, "Computer Rendering of Stochastic Models," *CACM*, 25(6), 371–384, 1982.

Fuc77 Fuchs, H., J. Duran, and B. Johnson, "A System for Automatic Acquisition of Three-Dimensional Data," *Proceedings of the 1977 NCC*, AFIPS Press, 49–53, Montvale, NJ, 1977.

Fuc80 Fuchs, H., Z.M. Kedem, and B.F. Naylor, "On Visible Surface Generation by A Priori Tree Structures," *SIGGRAPH 80*, 124–133, 1980.

Gal95 Gallagar, R.S., *Computer Visualization: Graphics Techniques for Scientific and Engineering Analysis*, CRC Press, Boca Raton, FL, 1995.

Gla89 Glassner, A.S. (Ed.), *An Introduction to Ray Tracing,* Academic Press, New York, 1989.

Gla95 Glassner, A.S., *Principles of Digital Image Synthesis,* Morgan Kaufmann, San Francisco, 1995.

Gol83 Goldberg, A., and D. Robson, *Smalltalk–80: The Language and Its Implementation*, Addison-Wesley, Reading, MA, 1983.

Gon08 Gonzalez, R., and R. E.Woods, *Digital Image Processing*, Third Edition, Addison-Wesley, Reading, MA, 2008.

Gor84 Goral, C.M., K.E. Torrance, D.P. Greenberg, and B. Battaile, "Modeling the Interaction of Light Between Diffuse Surfaces," *Computer Graphics (SIGGRAPH 84)*, 18(3), 213–222, 1984.

Gor96 Gortler, S.J., R. Grzeszczuk, R. Szeliski, and M.F. Cohen, "The Lumigraph," *Computer Graphics*, 43–54, 1996.

Gou71 Gouraud, H., "Computer Display of Curved Surfaces," *IEEE Trans. Computers,* C-20, 623–628, 1971.

Gra90 *Graphics Gems I*, Glassner, A.S. (Ed.), Academic Press, San Diego, CA, 1990.

Gra91 *Graphics Gems II*, Arvo, J. (Ed.), Academic Press, San Diego, CA, 1991.

Gra92 *Graphics Gems III*, Kirk, D. (Ed.), Academic Press, San Diego, CA, 1992.

Gra94 *Graphics Gems IV*, Heckbert, P. (Ed.), Academic Press, San Diego, CA, 1994.

Gra95 *Graphics Gems V*, Paeth, A. (Ed.), Academic Press, San Diego, CA, 1995.

Gra03 Gray, K., *The Microsoft DirectX 9 Programmable Graphics Pipeline*, Microsoft Press, 2003.

Gre88 Greengard, L.F., *The Rapid Evolution of Potential Fields in Particle Systems,* MIT Press, Cambridge, MA, 1988.

Hal89 Hall, R., *Illumination and Color in Computer Generated Imagery,* Springer-Verlag, New York, 1989.

Har96 Hartman, J., and J. Wernecke, *The VRML 2.0 Handbook*, Addison-Wesley, Reading, MA, 1996.

Hea94 Hearn, D., and M.P. Baker, *Computer Graphics*, Second Edition, Prentice-Hall, Englewood Cliffs, NJ, 1994.

Hec84 Heckbert, P.S., and P. Hanrahan, "Beam Tracing Polygonal Objects," *Computer Graphics,* 18(3), 119–127, 1984.

Hec86 Heckbert, P.S., "Survey of Texture Mapping," *IEEE Computer Graphics and Applications,* 6(11), 56–67, 1986.

Her79 Herman, G.T., and H.K. Liu, "Three-Dimensional Display of Human Organs from Computed Tomograms," *Computer Graphics and Image Processing,* 9, 1–21, 1979.

Her00 Hereld, M., I.R. Judson, and R.L. Stevens, "Tutorial: Introduction to Building Projection-Based Tiled Display Systems," *IEEE Computer Graphics and Applications,* 20(4), 22–26, 2000.

Hil07 Hill, F.S., Jr. and S.M. Kelley,, *Computer Graphics*, Third Edition, Prentice Hall, Upper Saddle River, NJ, 2007.

Hop83 Hopgood, F.R.A., D.A. Duce, J.A. Gallop, and D.C. Sutcliffe, *Introduction to the Graphical Kernel System: GKS*, Academic Press, London, 1983.

Hop91 Hopgood, F.R.A., and D.A. Duce, *A Primer for PHIGS,* John Wiley & Sons, Chichester, England, 1991.

Hum01 Humphreys, G., M. Elridge, I. Buck, G. Stoll, M. Everett, and P. Hanrahan, "WireGL: A Scalable Graphics System for Clusters," *SIGGRAPH 2001.*

ISO88 International Standards Organization, *International Standard Information Processing Systems—Computer Graphics—Graphical Kernel System for Three Dimensions (GKS-3D),* ISO Document Number 8805:1988(E), American National Standards Institute, New York, 1988.

Jar76 Jarvis, J.F., C.N. Judice, and W.H. Ninke, "A Survey of Techniques for the Image Display of Continuous Tone Pictures on Bilevel Displays," *Computer Graphics and Image Processing,* 5(1), 13–40, 1976.

Jen01 Jensen, H.W., "Realistic Image Synthesis Using Photon Mapping," AK Peters, 2001.

Joy88 Joy, K.I., C.W. Grant, N.L. Max, and L. Hatfield, *Computer Graphics: Image Synthesis*, Computer Society Press,Washington, DC, 1988.

Kaj86 Kajiya, J.T., "The Rendering Equation," *Computer Graphics,* 20(4), 143–150, 1986.

Kel97 Keller, H., "Instant Randiosity," *Computer Graphics*, 49–56, 1997.

Kil94a Kilgard, M.J., "OpenGL and X, Part 3: Integrated OpenGL with Motif," *The X Journal,* SIGS Publications, July/August 1994.

Kil94b Kilgard, M.J., "An OpenGL Toolkit," *The X Journal,* SIGS Publications, November/ December 1994.

Kil96 Kilgard, M.J., *OpenGL Programming for the X Windows System,* Addison-Wesley, Reading, MA, 1996.

Knu87 Knuth, D.E., "Digital Halftones by Dot Diffusion," *ACM Transactions on Graphics,* 6(40), 245–273, 1987.

Kov97 Kovatch, P.J., *The Awesome Power of Direct3D/DirectX,* Manning Publications Company, Greenwich, CT, 1997.

Kue08 Kuehhne, R. P., and J. D. Sullivan, *OpenGL Programming on Mac OS X,* Addison-Wesley, Boston, MA, 2008.

Kui99 Kuipers, J.B., *Quaternions and Rotation Sequences,* Princeton University Press, Princeton, NJ, 1999.

Las87 Lasseter, J., "Principles of Traditional Animation Applied to 3D Computer Animation," *Computer Graphics*, 21(4), 33–44, 1987.

Lev88 Levoy, M., "Display of Surface from Volume Data," *IEEE Computer Graphics and Applications,* 8(3), 29–37, 1988.

Lev96 Levoy, M., and P. Hanrahan, "Light Field Rendering," *Computer Graphics*, 31–42, 1996.

Lia84 Liang, Y., and B. Barsky, "A New Concept and Method for Line Clipping," *ACM Transactions on Graphics*, 3(1), 1–22, 1984.

Lin68 Lindenmayer, A., "Mathematical Models for Cellular Interactions in Biology" *Journal of Theoretical Biology,* 18, 280–315, 1968.

Lin01 Linholm, E., M.J. Kilgard, and H. Morelton, "A User-Programmable Vertex Engine," *SIGGRAPH,* 2001.

Lor87 Lorensen,W.E., and H.E. Cline, "Marching Cubes: AHigh Resolution 3D Surface Construction Algorithm," *Computer Graphics,* 21(4), 163–169, 1987.

Ma94 Ma, K.L., J. Painter, C. Hansen, and M. Krogh, "Parallel Volume Rendering Using Binary-Swap Compositing," *IEEE Computer Graphics and Applications,* 14(4), 59–68, 1994.

Mag85 Magnenat-Thalmann, N., and D. Thalmann, *Computer Animation: Theory and Practice,* Springer-Verlag, Tokyo, 1985.

Man82 Mandelbrot, B., *The Fractal Geometry of Nature,* Freeman Press, New York, 1982.

Mat95 The MathWorks, *Student Edition of MatLab Version 4 Users Guide,* Prentice-Hall, Englewood Cliffs, NJ, 1995.

Mau06 Maurina, E., *The Game Programmer's Guide to Torque,* A. K. Peters, 2006.

Max51 Maxwell, E.A., *General Homogeneous Coordinates in Space of Three Dimensions*, Cambridge University Press, Cambridge, England, 1951.

Mia99 Miamo, J., *Compressed Image File Formats*, ACM Press, New York, 1999.

Mol92 Molnar, S., J. Eyles, and J. Poulton, "PixelFlow: High-Speed Rendering Using Image Composition," *Computer Graphics*, 26(2), 231–240, 1992.

Mol94 Molnar, S., M. Cox, D. Ellsworth, and H. Fuchs, "A Sorting Classification of Parallel Rendering," *IEEE Computer Graphics and Applications*, 26(2), 231–240, 1994.

Mol02 M¨oller, T., and E. Haines, *Real-Time Rendering*, Second, Edition, AK Peters, Natick, MA, 2002.

Mon97 Montrym, J., D. Baum, D. Dignam, and C.Migdal, "InfiniteReality: A Real-Time Graphics System," *Computer Graphics*, 293–392, 1997.

Mur94 Murray, J.D., and W. Van Ryper, *Encyclopedia of Graphics File Formats,* O'Reilly and Associates, Sebastopol, CA, 1994.

New73 Newman, W.M., and R.F. Sproull, *Principles of Interactive Computer Graphics*, McGraw-Hill, New York, 1973.

Ngu07 Nguyen, H. (Ed), *GPU Gems 3,* Addison-Wesley Professional, Boston, MA, 2007.

Nie94 Nielson, J., *Usability Engineering*, Academic Press, New York, 1994.

Nie97 Nielson, G.M., H. Hagen, and H. Muller, *Scientific Visualization: Overviews, Methodologies, and Techniques*, IEEE Computer Society, Piscataway, NJ, 1997.

Ope04 OpenGL Architecture Review Board, *OpenGL Programming Guide*, Fifth Edition, Addison-Wesley, Reading, MA, 2004.

Ope05 OpenGL Architecture Review Board, *OpenGL ReferenceManual*, Fourth Edition, Addison-Wesley, Reading, MA, 2005.

OSF89 Open Software Foundation, *OSF/Motif Style Guide,* Prentice-Hall, Englewood Cliffs, NJ, 1989.

Ost94 Osterhaut, J., *Tcl and the Tk Toolkit,* Addison-Wesley, Reading, MA, 1994.

Pap81 Papert, S., *LOGO: A Language for Learning*, Creative Computer Press, Middletown, NJ, 1981.

Pav95 Pavlidis, T., *Interactive Computer Graphics in X,* PWS Publishing, Boston, MA, 1995.

Per89 Perlin, K., and E. Hoffert, "Hypertexture," *Computer Graphics,* 23(3), 253–262, 1989.

Pha05 Pharr, M., and R. Fernando (Eds), *GPU Gems 2: Programming Techniques for High-Performance Graphics and General-Purpose Computation,* Addison-Wesley Professional, Boston, MA, 2005.

PHI89 PHIGS+ Committee, "PHIGS+ Functional Description, Revision 3.0," *Computer Graphics,* 22(3), 125–218, July 1989.

Pho75 Phong, B.T., "Illumination for Computer Generated Scenes," *Communications of the ACM*, 18(6), 311–317, 1975.

Pei88 Peitgen, H.O., and S. Saupe (Ed.), *The Science of Fractal Images,* Springer-Verlag, New York, 1988.

Pik84 Pike, R., L. Guibas, and D. Ingalls, "Bitmap Graphics," *Computer Graphics*, 18(3), 135–160, 1984.

Por84 Porter, T., and T. Duff, "Compositing Digital Images," *Computer Graphics,* 18(3), 253–259, 1984.

Pra78 Pratt,W.K., *Digital Image Processing*,Wiley, New York, 1978.

Pru90 Prusinkiewicz, P., and A. Lindenmayer, *The Algorithmic Beauty of Plants,* Springer-Verlag, Berlin, 1990.

Ree83 Reeves, W.T., "Particle Systems—A Technique for Modeling a Class of Fuzzy Objects," *Computer Graphics*, 17(3), 359–376, 1983.

Rei05 Reinhard, Erik, G. Ward, S. Pattanaik, and P. Debevec, *High Dynamic Range Imaging: Acquisition, Display, and Image-Based Lighting,*Morgan Kaufmann, San Francisco, 2005.

Rey87 Reynolds, C.W., "Flocks, Herds, and Schools: A Distributed Behavioral Model," *Computer Graphics*, 21(4), 25–34, 1987.

Rie8 Riesenfeld, R.F., "Homogeneous Coordinates and Projective Planes in Computer Graphics," *IEEE Computer Graphics and Applications*, 1(1), 50–56, 1981.

Rog98 Rogers, D.F., *Procedural Elements for Computer Graphics,* Second Edition, McGraw-Hill, New York, 1998.

Rog90 Rogers, D.F., and J.A. Adams, *Mathematical Elements for Computer Graphics,* McGraw-Hill, New York, 1990.

Rog00 Rogers, D.F., *An Introduction to NURBS: With Historical Perspective,* Morgan Kaufmann, San Francisco, CA, 2000.

Ros06 Rost, R.J., *OpenGL Shading Language*, Second Edition, Addison-Wesley, Reading, MA, 2006.

Sch88 Schiefler, R.W., J. Gettys, and R. Newman, *X Window System,* Digital Press, Woburn, MA, 1988.

Sch97 Schneiderman, B., *Designing the User Interface: Strategies for Effective Human–Computer Interaction,* Third Edition, Addison-Wesley, Reading, MA, 1997.

Schr98 Schroeder, W., K. Martin, and B. Lorensen, *The Visualization Toolkit*, Prentice-Hall, Upper Saddle River, NJ, 1998.

Sei96 Seitz, S.M., and C.R. Dyer, "View Morphing," *Computer Graphics*, 21–30, 1996.

Seg92 Segal, M., and K. Akeley, *The OpenGL Graphics System: A Specification,* Version 1.0, Silicon Graphics, Mountain View, CA, 1992.

Shi03 Shirley, P., R. K.Morley, and K.Morley, *Realistic Ray Tracing*, Second Edition, A. K. Peters, 2003.

Shi05 Shirley, P., M. Ashikhmin, M. Gleicher, S. Marschner, E. Reinhard, K. Sung, W. Thompson, and P. Willemsen, *Fundamentals of Computer Graphics*, Second Edition, A. K. Peters, 2005.

Sho85 Shoemaker, K., "Animating Rotation with Quaternion Curves," *Computer Graphics*, 19(3), 245–254, 1985.

Sie81 Siegel, R., and J. Howell, *Thermal Radiation Heat Transfer,* Hemisphere, Washington, DC, 1981.

Sil89 Sillion, F.X., and C. Puech, "A General Two-Pass Method Integrating Specular and Diffuse Reflection," *Computer Graphics,* 22(3), 335–344, 1989.

Smi84 Smith, A.R., "Plants, Fractals and Formal Languages," *Computer Graphics,* 18(3), 1–10, 1984.

Sta03 Stam, J., and C. Loop, "Quad/Triangle Subdivision," *Computer Graphics Forum* 22, 1–7, 2003.

Str93 Strang, G., *Introduction to Linear Alegbra,* Wellesley-Cambridge Press, Wellesley, MA, 1993.

Suf07 Suffern, K., *Ray Tracing from the Ground Up,* A. K. Peters, 2007.

Sut63 Sutherland, I.E., *Sketchpad, A Man–Machine Graphical Communication System, SJCC*, 329, Spartan Books, Baltimore, MD, 1963.

Sut74a Sutherland, I.E., and G.W. Hodgeman, "Reentrant Polygon Clipping," *Communications of the ACM*, 17, 32–42, 1974.

Sut74b Sutherland, I.E., R.F. Sproull, and R.A. Schumacker, "A Characterization of Ten Hidden-Surface Algorithms," *Computer Surveys*, 6(1), 1–55, 1974.

Swo00 Swoizral, H., K. Rushforth, and M. Deering, *The Java 3D API Specification,* Second Edition, Addison-Wesley, Reading, MA, 2000.

Tor67 Torrance, K.E., and E.M. Sparrow, "Theory for Off–Specular Reflection from Roughened Surfaces," *Journal of the Optical Society of America,* 57(9), 1105–1114, 1967.

Tor96 Torborg, J., and J.T. Kajiya, "Talisman: Commodity Realtime 3D Graphics for the PC," *Computer Graphics*, 353–363, 1996.

Tuf83 Tufte, E.R., *The Visual Display of Quantitative Information*, Graphics Press, Cheshire, CT, 1983.

Tuf90 Tufte, E.R., *Envisioning Information*, Graphics Press, Cheshire, CT, 1990.

Tuf97 Tufte, E.R., *Visual Explanations*, Graphics Press, Cheshire, CT, 1997.

Ups89 Upstill, S., *The RenderMan Companion: A Programmer's Guide to Realistic Computer Graphics,* Addison-Wesley, Reading, MA, 1989.

Van94 Van Gelder, A., and J. Wilhelms, "Topological Considerations in Isosurface Generation," *ACM Transactions on Graphics,* 13(4), 337–375, 1994.

War94 Ward, G., "The RADIANCE Lighting Simulation and Rendering System," *Computer Graphics*, July 1994.

War03 Warren, J., and H. Weimer, *Subdivision Methods for Geometric Design,* Morgan Kaufmann, San Francisco, 2003.

War04 Warren, J., and S. Schaefer, "A Factored Approach to Subdivision Surfaces," *IEEE Computer Graphics and Applications*, 24(3), 74-81, 2004.

Wat92 Watt, A., and M.Watt, *Advanced Animation and Rendering Techniques,* Addison-Wesley, Wokingham, England, 1992.

Wat98 Watt, A., and F. Policarpo, *The Computer Image*, Addison-Wesley, Wokingham, England, 1998.

Wat00 Watt, A., *3D Computer Graphics*, Third Edition, Addison-Wesley, Wokingham, England, 2000.

Wat02 Watkins, A., *The Maya 4 Handbook*, Charles River Media, Hingham, MA, 2002.

Wer94 Wernecke, J., *The Inventor Mentor*, Addison-Wesley, Reading, MA, 1994.

Wes90 Westover, L., "Footprint Evaluation for Volume Rendering," *Computer Graphics*, 24(4), 367–376, 1990.

Whi80 Whitted, T., "An Improved Illumination Model for Shaded Display," *Communications of ACM*, 23(6), 343–348, 1980.

Wit94a Witkin, A.P., and P.S. Heckbert, "Using Particles to Sample and Control Implicit Surfaces," *Computer Graphics,* 28(3), 269–277, 1994.

Wit94b Witkin, A. (Editor), *An Introduction to Physically Based Modeling,* Course Notes, *SIGGRAPH 94,* 1994.

Wol91 Wolfram, S., *Mathematica*, Addison-Wesley, Reading, MA, 1991.

Wri08 Wright, R.S., Jr., B. Lipchak and H. Haemel, *OpenGL Superbible*, Fourth Edition, Addison-Wesley, Boston, MA, 2008.

Wys82 Wyszecki, G., andW.S. Stiles, *Color Science*,Wiley, New York, 1982.